CUBRID

Version 11.4

Table of Contents

Introduction to Manual 9

- Manual Contents
- Glossary
- Manual Conventions
- Version Name and Version String Conventions

Introduction to CUBRID 13

- System Architecture
- CUBRID Characteristics

Installing and Upgrading 25

- Installing and Running CUBRID 25
- Configuring Environment Variables 39
- Port Setting
- Upgrade 56
- Database Migration under HA Environment
- Uninstalling CUBRID 68

Getting Started 70

- Starting the CUBRID Service 70
- Query Tools 73
- Management Tools
- Drivers

CSQL Interpreter 79

- Introduction to the CSQL Interpreter
- Executing CSQL
- Session Commands

CUBRID SQL 118

- Writing Rules 119
- Data Types 135
- Data Definition Statements 215
- Operators and Functions 331
- Data Manipulation Statements 744

- Query Optimization 834
- Partitioning 975
- Globalization 995
- Transaction and Lock 1061
- Trigger 1121
- CUBRID DBLink 1140
- Class Inheritance 1164
- Database Administration 1178
- User Schema 1264
- System Catalog 1266

CUBRID Procedural Language 1343

- Create a Stored Procedure 1344
- Call a Stored Procedure 1357
- Stored Procedure Authorization 1359
- Transaction Commit and Rollback Support 1362
- Performance Tuning 1364
- System Package 1370
- PL/CSQL 1374
- Java Stored Procedure 1424
- Method 1467

CUBRID Management 1469

- Controlling CUBRID Processes 1472
- CUBRID Services
- Database Server
- Broker
- Gateway
- CUBRID Manager Server
- CUBRID Procedural Langauge Server
- Database Management 1560
- cubrid Utilities 1568
- System Parameters 1725
- SystemTap 1843
- cubrid scripts 1856
- Troubleshooting 1862
- DDL Audit Log 1870

CUBRID HA 1872

- CUBRID HA Concept
- CUBRID HA Features
- Quick Start
- Environment Configuration
- Connecting a Broker to DB
- Running and Monitoring
- Structures of HA
- HA Constraints
- Operational Scenarios
- Building Replication
- Detection of Replication Mismatch
- Determine the Cause of Failover
- Rebuilding Replication Script

CUBRID Security 2012

- Packet Encryption
- ACL (Access Control List)
- Authorization
- TDE (Transparent Data Encryption)

CUBRID SHARD 2023

- Database Sharding
- CUBRID SHARD Terminologies
- CUBRID SHARD Main Features
- Quick Start
- Architecture and Configuration
- Running and Monitoring
- Configuration Test
- CUBRID SHARD Log
- Constraints

API Reference 2052

- JDBC Driver 2052
- CCI Driver 2095
- PHP Driver 2263
- PDO Driver 2294
- ODBC Driver 2308

- ADO.NET Driver 2325
- Perl Driver 2338
- Python Driver 2340
- Ruby Driver 2346
- Node.js Driver 2351

Release Notes 2352

- 11.4 Release Notes 2352
- General Information

CUBRID 11.4 User Manual

Topic Quick Reference Table

General	Administrators	Developers	Authority & Security	Connectors & API	Error Messages & Logs
Introduction to CUBRID	Configuring Environment Variables	Operators and Functions	User Management	JDBC Driver	Error Message-Related Parameters
Getting Started	Port Setting	Data Manipulation Statements	Limiting Database Server Access	CCI Overview	CCI Error Codes and Error Messages
Installing and Upgrading	System Parameters	An Overview of Globalization	Limiting Broker Access	PHP Driver	JDBC Error Codes and Error Messages
CUBRID SQL	Controlling CUBRID Processes	Join Methods	blocking DDL	PDO Driver	Database Server Errors
CUBRID Procedural Language	cubrid Utilities	Partitioning	blocking DML w/o WHERE	ODBC Driver	CAS Error
	Database Volume	Cursor Holdability	TDE (Transparent Data Encryption)	ADO.NET Driver	HA Error Messages

General	Administrators	Developers	Authority & Security	Connectors & API	Error Messages & Logs
	backupdb	PL/CSQL		Perl Driver	Database Server Log
	restoredb			Python Driver	Broker Logs
	Troubleshooting			Ruby Driver	
	CUBRID HA			Node.js Driver	

Table of Contents

Introduction to Manual

Manual Contents

The contents of the CUBRID Database Management System (CUBRID DBMS) product manual are as follows:

- Introduction to CUBRID: This chapter provides a description of the structure and characteristics of the CUBRID DBMS.
- Installing and Upgrading: This chapter explains how to install and run CUBRID, configure environment variables and ports, perform upgrades, migrate databases in an HA environment, and uninstall CUBRID.
- Getting Started: The "Getting Started with CUBRID" provides users with a brief explanation on what to do when first starting CUBRID. The chapter contains information on how to install and execute the system, used ports on accessing to CUBRID and provides simple explanations on the CUBRID query tools.
- CSQL Interpreter: CSQL is an application that allows you to use SQL statements through a command-driven interface. This chapter explains how to use the CSQL Interpreter and associated commands.
- CUBRID SQL: This chapter describes SQL syntaxes such as data types, functions and operators, data retrieval or table manipulation. The chapter also provides SQL syntaxes used for indexes, triggers, partitioning, serial and user information changes, etc.
- CUBRID Procedural Language: This chapter describes the procedural language extension provided by CUBRID for using stored procedures and stored functions. Stored procedures and functions help handle complex business logic that is difficult to implement with SQL alone and simplify data manipulation tasks.
- CUBRID Management: This chapter provides instructions on how to create, drop, back up, restore and migrate a database, configuring globalization, and executing CUBRID HA. Also it includes instructions on how to use the **cubrid** utility, which starts and stops the server, broker, and CUBRID Manager server, etc. Also, this chapter provides instructions on setting system

parameters that may influence the performance. It provides information on how to use the configuration file for the server and broker, and describes the meaning of each parameter.

- CUBRID HA: This chapter explains how the High Availability (HA) feature is applied in CUBRID. It keeps databases synchronized at all times across multiple servers, and if a failure occurs on the active server, another server automatically takes over the service.
- CUBRID SHARD: This chapter explains the sharding feature in CUBRID, which horizontally partitions and distributes data across physically separate databases. CUBRID SHARD is used to distribute and process large volumes of data and high throughput workloads that exceed the capacity of a single database instance.
- CUBRID Security: This chapter describes the CUBRID security features such as packet encryption, ACL(Access Control List), authorization, and TDE(Transparent Data Encryption).
- API Reference: The “Performance Tuning” chapter provides instructions on setting system parameters that may influence the performance. This chapter provides information on how to use the configuration file for the server and broker, and describes the meaning of each parameter.
- Release Notes: This chapter provides a description of additions, changes, improvements and bug fixes.

Glossary

CUBRID is an object-relational database management system (ORDBMS), which supports object-oriented concepts such as inheritance. In this manual, relational database terminologies are also used along with object-oriented terminologies for better understanding. Object-oriented terminologies such as class, instance and attribute is used to describe concepts including inheritance, and relational database terminologies are mainly used to describe common SQL syntax.

Relational Database	CUBRID
table	class, table
column	attribute, column
record	instance, record

Relational Database	CUBRID
data type	domain, data type

Manual Conventions

The following table provides conventions on definitions used in the CUBRID Database Management System product manual to identify “statements,” “commands” and “reference within texts.”

Convention	Description	Example
Italics	*Italics* type represents variable names and user-defined values (system, database, table, column and file) in examples.	*persistent stringVariableName* :
Boldface	**Boldface** type represents names such as the member function name, class name, constants, CUBRID keyword or names such as other required characters.	**fetch** ( ) member function
Constant Width	Constant Width type represents segments of code example or describes a command’s execution and results.	csql database_name
UPPER-CASE	UPPER-CASE represents the CUBRID keyword (see **Boldface**).	**SELECT**

Convention	Description	Example
Single Quotes (’ ‘)	Single quotes (’ ‘) are used with braces and brackets and represent the necessary sections of a syntax. Single quotes are also used to enclose strings.	{‘{’ *const_list* ‘}’}
Brackets ([ ])	Brackets ([ ]) represents optional parameters or keywords.	[**ONLY**]
Vertical bar (|)	Vertical bar (|) represents that one or another option can be specified.	[**COLUMN** | **ATTRIBUTE**]
A parameter enclosed by braces ({ })	A parameter enclosed by braces represents that one of those parameters must be specified in a statement syntax.	**CREATE** { **TABLE** | **CLASS** }
A value enclosed by braces ({ })	A value enclosed by braces an element consisting of collection.	{2, 4, 6}
Braces with ellipsis ({ }...)	Braces before an ellipsis represents that a parameter can be repeated.	{, *class_name* }...
Angle brackets (< >)	Angle brackets represent a single key or a series of key strokes.	<Ctrl+n> }...

Version Name and Version String Conventions

Rules for version naming and string since CUBRID 10.0 are as follows:

- Version name: CUBRID M.m Patch p (Major version, Minor version, Patch version if necessary) CUBRID 10.1 Patch 1 (CUBRID 10.1 P1 in short)
- Version string: M.m.p.build_number (Major version, Minor version, Patch version, Build number) 10.2.0.8787-a31ea42

  Build number consists of two parts which are separated by a hyphen. The former is the number of changes from the base revision, which monotonically increases. The later is the SHA-1 hash of the build built.

Rules for version naming and string since CUBRID 9.0 are as follows:

- Version name: CUBRID M.m Patch p (Major version, Minor version, Patch version if necessary) CUBRID 9.2 Patch 1 (CUBRID 9.2 P1 in short)
- Version string: M.m.p.build_number (Major version, Minor version, Patch version, Build number) 9.2.1.0012

Rules for version naming and string before CUBRID 9.0 are as follows:

- Version name: CUBRID 2008 RM.m Patch p (2008 for Major version, Minor version, Patch version, Build number) CUBRID 2008 R4.1 Patch 1
- Version string: 8.m.p.build_number (Major version, Minor version, Patch version, Build number) 8.4.1.1001

Introduction to CUBRID

This chapter explains the architecture and features of CUBRID. CUBRID is an object-relational database management system (DBMS) consisting of the database server, the broker, and the CUBRID Manager. It is optimized for Internet data services, and provides various user-friendly features.

This chapter covers the following topics:

- System Architecture: This page contains information about database volume structure, server process, broker process, interface modules, etc.

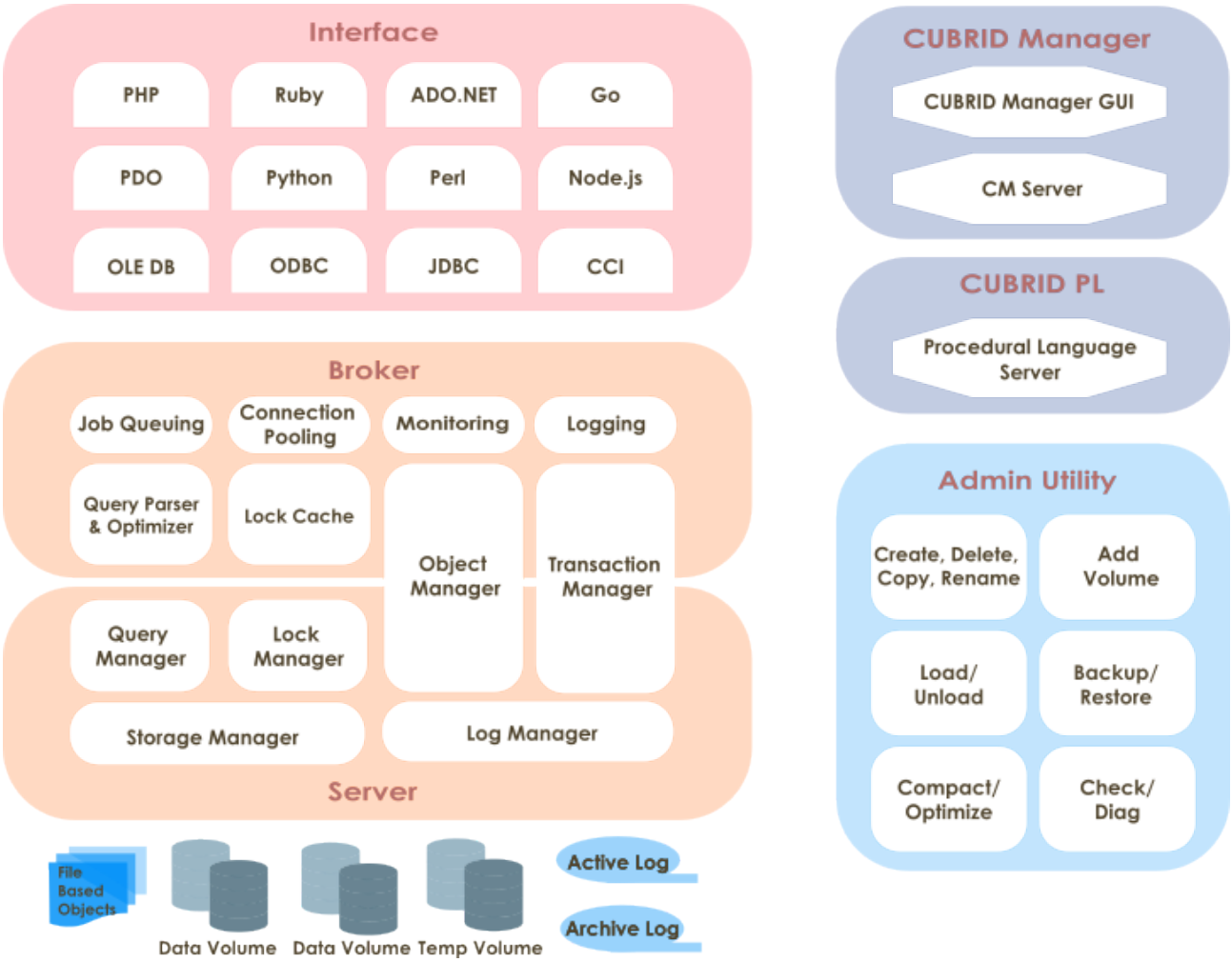
- Features of CUBRID: This page contains information about transaction, backup and recovery, partitioning, index, HA, Java stored procedure, click counter, relational data model extension, etc.

System Architecture

Process Structure

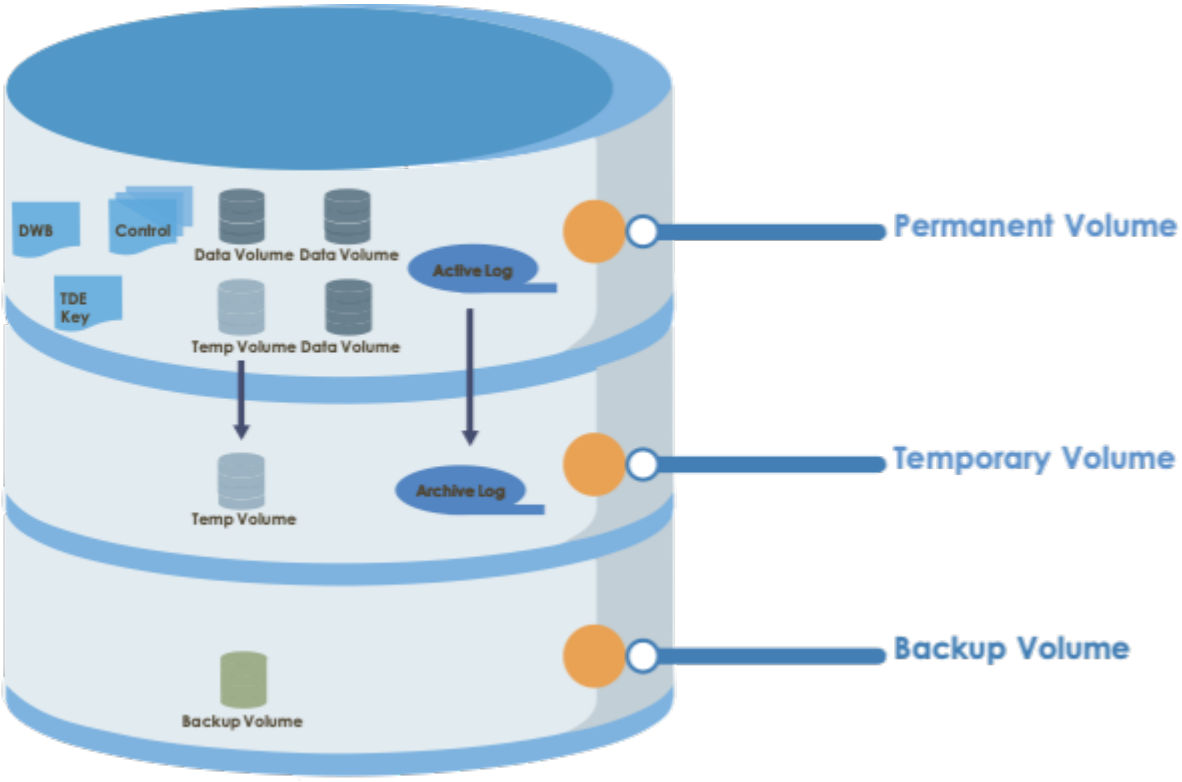
CUBRID is an object-relational database management system (DBMS) consisting of the database server, the broker, and the CUBRID Manager.

- As the core component of the CUBRID database management system, the database server stores and manages data in multi-threaded client/server architecture. The database server processes the queries requested by users and manages objects in the database. The CUBRID database server provides seamless transactions using locking and logging methods even when multiple users use the database at once. It also supports database backup and restore for the operation.
- The broker is a CUBRID-specific middleware that relays the communication between the database server and external applications. It provides functions including connection pooling, monitoring, and log tracing and analysis.
- The CUBRID Manager is a GUI tool that allows users to remotely manage the database and the broker. It also provides the Query Editor, a convenient tool that allows users to execute SQL queries on the database server.
- The CUBRID procedural langauge (PL) server is a server that compiles and executes stored procedures and functions requested from the database server.



Database Volume Structure

The following diagram illustrates the CUBRID database volume structure. As you can see, the database is divided into three volumes: permanent, temporary and backup. This chapter will examine each volume and its characteristics.



For commands to create, add or delete the database volume, see createdb, addvoldb and deletedb.

Permanent Volume

Data Volumes

Permanent data volumes are database volumes that exists permanently once they are created.

It usually stores data that needs to be persistent after database restart or crash. The possible types of permanent data are:

- Tables (rows and multimedia data) are internally stored into heap files and heap overflow files, one file for each table.
- Indexes (keys and multimedia data) are internally stored into b-tree files and b-tree overflow files, one file for each index.
- System data is internally stored into several types of files: file tracker, vacuum data, dropped files tracker, and class names hash.

User can specifically assign some permanent data volumes to store temporary data. These volumes are permanent in the sense that they are never destroyed, but behave similarly to Temporary Volume.

Note

If you have used older CUBRID version, you may know that we used to have several types of permanent data volumes: **generic**, **data** and **index**. This classification is deprecated, and although these options are still allowed for **cubrid createdb** and **cubrid addvoldb** commands, the created volumes will be no different and they will store any type of permanent data. The volume purpose is only classified into permanent data and temporary data.

Control File

The control file contains the volume, backup and log information in the database.

- **Volume Information**: The information that includes names, locations and internal volume identifiers of all the volumes in the database. When the database restarts, the CUBRID reads the volume information control file. It records a new entry to that file when a new database volume is added.
- **Backup Information**: Locations of all the backups for data, index, and generic volumes are recorded to a backup information control file. This control file is maintained where the log files are managed.
- **Log Information**: This information contains names of all active and archive logs. With the log information control file, you can verify the archive log information. The log information control file is created and managed at the same location as the log files.

Control files include the information about locations of database volumes, backups and logs. Since these files will be read when the database restarts, users must not modify them arbitrarily.

Active Log

Active log is a log that contains recent changes to the database. If a problem occurs, you can use active and archive logs to restore the database completely up to the point of the last commit before the occurrence of the fault.

Double Write Buffer File

Double write buffer file stores copies of data pages being flushed to disk as a protection against I/O errors. A detailed description of this file can be found in Database Volume section.

TDE Key File

TDE (Transparent Data Encryption) key file contains keys for database encryption. The keys in the file are managed using the TDE utility. For more information on this, see File-based Master Key Management and TDE Utility.

Temporary Volume

Temporary data volume has the opposite meaning to the permanent volume. That is, the temporary volume is a storage file created temporarily which gets destroyed when the server process terminates. These volumes are used to store intermediate and final results of query processing and sorting.

These files provide space to store intermediary and final results of queries. Based on the size of required temporary data, it will be first stored in memory (the space size is determined by the system parameter **temp_file_memory_size_in_pages** specified in **cubrid.conf**). Exceeding data has to be stored on disk.

Database will usually create and use temporary volumes to allocate disk space for temporary data. They user may however assign permanent database volumes with the purpose of storing temporary data using by running **cubrid addvoldb -p temp** command. If such volumes exist, they will have priority over temporary volumes when disk space is allocated for temporary data.

The examples of queries that can use temporary data are as follows:

- Queries creating the resultset like **SELECT**
- Queries including **GROUP BY** or **ORDER BY**
- Queries including a subquery
- Queries executing sort-merge join
- Queries including the **CREATE INDEX** statement

To have complete control on the disk space used for temporary data and to prevent it from consuming all system disk space, our recommendation is to:

- create permanent database volumes in advance to secure the required space for temporary data
- limit the size of the space used in the temporary volumes when a queries are executed by setting **temp_file_max_size_in_pages** parameter in **cubrid.conf** (there is no limit by default).

Once temporary temp volume is created, it is maintained until a database restarts and its size cannot be reduced. It is recommended to make temporary temp volume automatically delete by restarting a database if its size is too big.

- **Filename of the temporary volumes**: The filename format of a temporary volume is *db_name*_**t***num*, where *db_name* is the database name and *num* is the volume identifier. The volume identifier is decremented by 1 from 32766.
- **Configuring the temporary volume size**: The number of temporary volumes to be created is determined by the system depending on the space size needed for processing transactions. However, users can limit the total temporary volume size by configuring the

temp_file_max_size_in_pages parameter value in the system parameter configuration file (**cubrid.conf**). The default value is -1, which means it can be created as long as free space is available. If the **temp_file_max_size_in_pages** parameter value is configured to 0, no temporary volumes will be created, and the system will have to rely exclusively on permanent volumes assigned for temporary data.

- **Configuring storing location of temporary volumes**: By default, temporary volumes are created where the first database volume was created. However, you can specify a different directory to store temporary volumes by configuring the **temp_volume_path** parameter value.
- **Deleting temporary volumes**: Temporary volumes exist only while the database is running. Therefore, you must not delete the temporary volumes when running servers. They are deleted when database servers are normally terminated. When database servers are abnormally terminated, temporary volumes are deleted on servers restart.

Note

Normally, permanent volumes are used to store permanent data, and temporary volumes are used to store temporary data. You can assign permanent volumes to store temporary data, but temporary volumes will never store permanent data!

Archive Log

Archive log is a volume to store logs continuously created after exhausting available active log space that contains recent changes. If the value of system parameter **log_max_archives** is larger than 0, the archive log volume will be generated only after exhausting available active log volume space. The initial value is set to 0 when installing CUBRID. The number of archive log files is kept on the storage by setting the value of **log_max_archives**. The unnecessary archive log files should be deleted for getting the free space by the configuration of log_max_archives, but this value should be set properly to use for restoring the database.

To get more information on the above, see Managing Archive Logs.

Background Archive Log

Background archive log is a volume used in the background with log archiving temporarily before creating archive logs. It is created as the same volume size as active log and stored.

Backup Volume

Backup volume is a database snapshot; based on such backup and log volumes, you can restore transactions to a certain point of time.

You can use the **cubrid backupdb** utility to copy all the data needed for database restore, or configure the **backup_volume_max_size_bytes** parameter value in the database configuration file (**cubrid.conf**) to adjust the backup volume partitioning size.

Database Server

Database Server Process

Each database has a single server process. The server process is the core component of the CUBRID database server, and handles a user's requests by directly accessing database and log files. The client process connects to the server process via TCP/IP communication. Each server process creates threads to handle requests by multiple client processes. System parameters can be configured for each database, that is, for each server process. The server process can connect to as many client processes as specified by the **max_clients** parameter value.

Master Process

The master process is a broker process that allows the client process to connect to and communicate with the server process. One master process runs for each host. (To be exact, one master process exists for each connection port number specified in the **cubrid.conf** system parameter file.) While the master process listens on the TCP/IP port specified, the client process connects to the master process through that port. The master process changes a socket to server port so that the server process can handle connection.

Execution Mode

All CUBRID programs except the server process have two modes: client/server mode and standalone mode.

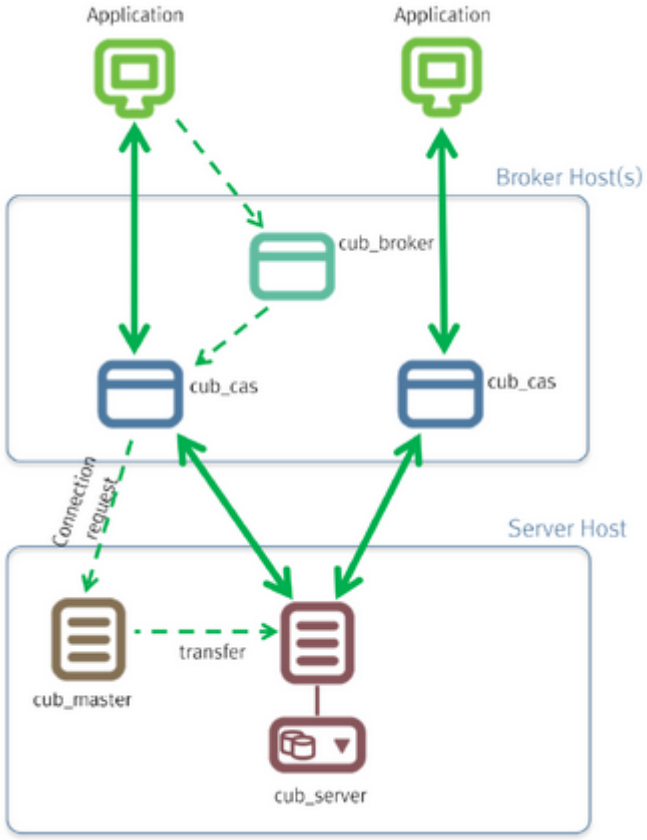
- In client/server mode, applications access server processes by operating themselves as client processes.
- In standalone mode, applications include functionalities of server processes so that the applications can access database files by themselves.

For example, database creation and restore utilities run in standalone mode so they can use the database exclusively by denying the access by multiple users. Another example is that the CSQL Interpreter can either connect to the server process in client/server mode or execute SQL statements by accessing the database in standalone mode. Note that one database cannot be accessed simultaneously by server processes and standalone programs.

Broker

The broker is a middleware that allows various application clients to connect to the database server. As shown below, the CUBRID system, which includes the broker, has multi-layered

architecture consisting of application clients, cub_broker, cub_cas, and cub_server (database server).



Application Client

The interfaces that can be used in application clients include C-API (CCI, CUBRID Call Interface), ODBC, JDBC, PHP, Python, Ruby, OLE DB, ADO.NET, Node.js, etc.

cub_cas

cub_cas (CUBRID Common Application Server and broker application server (CAS in short)) acts as a common application server used by all the application clients that request connections. cub_cas also acts as the database server's client and provides the connection to the database server upon the client's request. The number of cub_cas(s) running in the service pool can be specified in the **cubrid_broker.conf** file, and this number is dynamically adjusted by cub_broker.

cub_cas is a program linked to the CUBRID database server's client library and functions as a client module in the database server process (cub_server). In the client module, tasks such as query parsing, optimization, execution plan creation are performed.

cub_broker

cub_broker relays the connection between the application client and the cub_cas. That is, when an application client requests access, the **cub_broker** checks the status of the **cub_cas** through the shared memory, and then delivers the request to an accessible **cub_cas** . It then returns the processing results of the request from the **cub_cas** to the application client.

The **cub_broker** also manages the server load by adjusting the number of **cub_cas** (s) in the service pool and monitors and manages the status of the **cub_cas**. If the **cub_broker** delivers the request to **cub_cas** but the connection to **cub_cas** 1 fails because of an abnormal termination, it sends an error message about the connection failure to the application client and restarts **cub_cas** 1. Restarted **cub_cas** 1 is now in a normal stand-by mode, and will be reconnected by a new request from a new application client.

Shared Memory

The status information of the **cub_cas** is stored in the shared memory, and the **cub_broker** refers to this information to relay the connection to the application client. With the status information stored in the shared memory, the system manager can identify which task the **cub_cas** is currently performing or which application client's request is currently being processed.

Interface Module

CUBRID provides various Application Programming Interfaces (APIs). The following APIs are supported by CUBRID as follows:

- JDBC: A standard API used to create database applications in Java.
- ODBC: A standard API used to create database applications on Windows. ODBC driver is written based on CCI library.
- OLE DB: An API used to create COM-based database applications on Windows. OLE DB provider is written based on CCI library.
- PHP: CUBRID provides a PHP interface module to create database applications in the PHP environment. PHP driver is written based on CCI library.
- CCI: CCI is a C language interface provided by CUBRID. The interface module is provided as a C library.

All interface modules access the database server through the broker. The broker is a middleware that allows various application clients to connect to the database server. When it receives a request from an interface module, it calls a native C API provided by the database server's client library.

CUBRID Characteristics

Transaction Support

CUBRID supports the following features to completely ensure the atomicity, consistency, isolation and durability in transactions.

- Supporting commit, rollback, savepoint per transaction
- Ensuring transaction consistency in the event of system or database failure
- Ensuring transaction consistency between replications
- Supporting multiple granularity locking of databases, tables and records
- Resolving deadlocks automatically

Database Backup and Restore

A database backup is the process of copying CUBRID database volumes, control files and log files; a database restore is the process of restoring the database to a certain point in time using backup files, active logs and archive logs copied by the backup process. For a restore, there must be the same operating system and the same version of CUBRID installed as in the backup environment. The backup methods which CUBRID supports include online, offline and incremental backups; the restore methods include restore using incremental backups as well as partial and full restore.

Table Partitioning

Partitioning is a method by which a table is divided into multiple independent logical units. Each logical unit is called a partition, and each partition is divided into a different physical space. This will lead performance improvement by only allowing access to the partition when retrieving records. CUBRID provides three partitioning methods:

- Range partitioning: Divides a table based on the range of a column value
- Hash partitioning: Divides a table based on the hash value of a column
- List partitioning: Divides a table based on the column value list

Supports a Variety of Index Functions

CUBRID supports the following index functions to utilize indexes while executing a variety of conditional queries.

- Descending Index Scan: Descending Index Scan is available only with Ascending Index Scan, without creating separate descending indexes.
- Covering Index: When the column of a **SELECT** list is included in the index, the requested data can be obtained with an index scan.
- **ORDER BY** clause optimization: If the required record sorting order is identical to the order of indexes, no additional sorting is required (Skip ORDER BY).

- **GROUP BY** clause optimization: If all columns in the **GROUP BY** clause are included in the indexes, they are available to use while executing queries. Therefore, no additional sorting is required (Skip GROUP BY).

HA feature

CUBRID provides High Availability(HA) feature to minimize system down time while continuing normal operation of server in the event of hardware, software, or network failure. The structure of CUBRID HA is shared-nothing. CUBRID monitors its system and status on a real time basis with the CUBRID Heartbeat and performs failover when failure occurs. It follows the two steps below to synchronize data from the master database server to slave database server.

- A transaction log multiplication step where the transaction log created in the database server is replicated in real time to another node
- A transaction log reflection step where data is applied to the slave database server through the analysis of the transaction log being replicated in real time

Stored Procedure

A stored procedure is a method to decrease the complexity of applications and to improve the reusability, security and performance through the separation of database logic and middleware logic. CUBRID supports the SQL extension language PL/CSQL and can also be written in the general-purpose language Java. To use the stored procedure feature, the procedural langauge server must be running. (See CUBRID Procedural Langauge Server) For more details on stored procedures, refer to CUBRID Procedural Language.

Click Counter

In the Internet environment, it is common to store and keep counting information like page view in the database to track search history.

The above scenario is generally implemented by using the **SELECT** and **UPDATE** statements; SELECT retrieves the data and UPDATE increases the number of clicks for the retrieved queries.

This approach can cause significant performance degradation due to increased lock contention for **UPDATE** when a number of **SELECT** statements are executed against the same data.

To address this issue, CUBRID introduces the new concept of the Click Counter that will support optimized features in the Web in terms of usability and performance, and provides the `INCR()` function and the **WITH INCREMENT FOR** statement.

Extending the Relational Data Model

- **Collection**

  For the relational data model, it is not allowed that a single column has multiple values. In CUBRID, however, you can create a column with several values. For this purpose, collection data

types are provided in CUBRID. The collection data type is mainly divided into **SET**, **MULTISET** and **LIST**; the types are distinguished by duplicated availability and order.

  - **SET**: A collection type that does not allow the duplication of elements. Elements are stored without duplication after being sorted regardless of their order of entry.
  - **MULTISET**: A collection type that allows the duplication of elements. The order of entry is not considered.
  - **LIST**: A collection type that allows the duplication of elements. Unlike with **SET** and **MULTISET**, the order of entry is maintained.

- **JSON**

  JavaScript Object Notation (JSON) has become the de facto standard for data-interchanging. JSON, one of the semi-structured data, is not allowed to reside in the relational data model. In CUBRID, however, you can create and query JSON documents using SQL functions for JSON. you can define a JSON data type column and stores a JSON document into the column.

- **Inheritance**

  Inheritance is a concept to reuse columns and methods of a super class (table) in those of a sub class. CUBRID supports reusability through inheritance. By using inheritance provided by CUBRID, you can create a super class with some common columns and then create a sub class inherited from the super class with some unique columns added. In this way, you can create a database model which can minimize the number of columns.

Installing and Upgrading

For information about ports used by CUBRID, please refer to Port Setting section; in Linux, every configuration except for **APPL_SERVER_PORT** is same as Windows.

You can download various CUBRID tools and drivers in https://www.cubrid.org/downloads.

Installing and Running CUBRID

Supported Platforms and System Requirements

The platforms supported by CUBRID and hardware/software requirements for the installation are as follows:

Supported Platforms	Required Memory	Required Disk Space
• Windows 64 Bit Windows 7 or higher • Linux family 64 Bit(Linux kernel 2.4, glibc 2.3.4 or higher)	1GB or more	2GB or more(*)

(*): Requires a 500MB of free disk space on the initial installation; requires approximately 1.5GB of free disk space with a database creating with default options.

CUBRID Admin Client is not automatically installed when installing the CUBRID package. For this reason, if you require CUBRID Admin you must install it separately. The CUBRID can be downloaded from http://ftp.cubrid.org.

A variety of drivers such as JDBC, ODBC and PHP can also be downloaded from http://ftp.cubrid.org.

For more information on the CUBRID engine, tools, and drivers, see https://www.cubrid.org.

Compatibility

Application Compatibility

- Applications that use JDBC, PHP or CCI APIs from 2008 R4.1 or higher version of CUBRID can access the CUBRID 11.4 database. However, you must link the CUBRID 11.4 library or use the driver to use the added/improved features of JDBC, PHP or CCI interfaces. In order to use Date/Time Types with Timezone which are introduced as 10.0, users should upgrade drivers.
- Drivers lower version than 10.2 interpret JSON type columns as Varchar.
- Note that query results may differ from those given in the earlier version because new reserved words have been added, and the specifications for some queries have been changed.
- An application that is developed by using the GLO class can be used after it is converted to an application or schema suitable to the BLOB or CLOB type.

CUBRID Admin Compatibility

- CUBRID Admin guarantees backward compatibility with the servers using CUBRID 10.0 or higher and uses the CUBRID JDBC driver that matches each server version. However, you must use a CUBRID Admin that is higher than CUBRID servers in version in order to utilize all the features

of CUBRID Admin. The CUBRID JDBC driver is included in the $CUBRID/jdbc directory when CUBRID is installed($CUBRID on Linux, %CUBRID% on Windows).

Note

Old version users should upgrade all of driver, broker, DB server; Data migration should be done because 11.3 or early version's DB volume is not compatible with 11.4 version. For upgrade and data migration, see Upgrade.

Interoperability between CUBRID DB server and broker

- If the CUBRID DB server and its broker server are operated separately, CUBRID version between them should be the same, but if just the patch version is different, their interoperability is guaranteed.

  For example, 2008 R4.1 Patch1 broker is compatible with 2008 R4.1 Patch 10 DB server, but not compatible with 2008 R4.3 DB server. 9.1 Patch 1 broker is compatible with 9.1 Patch 10 DB server, but not compatible with 9.2 DB server.

- Even if the operating systems are different, their interoperability is guaranteed if the bit version of a DB server is identical to the bit version of a broker server.

  For example, the 64-bit DB server for Linux is interoperable with the 64-bit broker server for Windows.

  For the relation between DB server and broker, see Introduction to CUBRID.

Installing and Running CUBRID on Linux

Checklist before Installing

Check the following before installing CUBRID for Linux.

- glibc version

  Only supports glibc 2.3.4 or later. The glibc version can be checked as follows:

```
%rpm -q glibc
```

- 64-bit

  As 10.0, CUBRID supports only 64-bit Linux. You can check the version as follows:

```
% uname -a
Linux host_name 2.6.32-696.20.1.el6.x86_64 #1 SMP Fri Jan 26
17:51:45 UTC 2018 x86_64 x86_64 x86_64 GNU/Linux
```

Make sure to install the CUBRID 64-bit version on 64-bit Linux.

- The libraries that should be added.
  - Curses Library (rpm -q ncurses)

    CUBRID is packaged with version 5 of Curses library. If you have the latest version and you are not able to downgrade your system, you may need to install the ncurses-compat-libs package.
  - gcrypt Library (rpm -q libgcrypt)
  - stdc++ Library (rpm -q libstdc++)
- Check if the mapping between hostnames and IP addresses are correct in the /etc/hosts file.

  If hostnames and IP addresses are matched incorrectly, DB server cannot be started normally. Therefore, check if they are correctly mapped.

Installing CUBRID

The installation program consists of shell scripts that contain binary; thus it can be installed automatically. The following example shows how to install CUBRID with the "CUBRID-11.4-latest-Linux.x86_64.sh" file on the Linux.

```
$ sh CUBRID-11.4-latest-Linux.x86_64.sh
Do you agree to the above license terms? (yes or no) : yes
Do you want to install this software(CUBRID) to the default(/home1/
cub_user/CUBRID) directory? (yes or no) [Default: yes] : yes
Install CUBRID to '/home1/cub_user/CUBRID' ...
In case a different version of the CUBRID product is being used in
other machines,
please note that the CUBRID 11.4 servers are only compatible with
the CUBRID 11.4 clients and vice versa.
Do you want to continue? (yes or no) [Default: yes] : yes
Copying old .cubrid.sh to .cubrid.sh.bak ...

CUBRID has been successfully installed.

demodb has been successfully created.

If you want to use CUBRID, run the following commands
$ . /home1/cub_user/.cubrid.sh
$ cubrid service start
```

As shown in the example above, after installing the downloaded file (CUBRID-11.4-latest-Linux.x86_64.sh), you must run .cubrid.sh (or .cubrid.csh) according to your shell environment to set the environment variables required to use the CUBRID database. Additionally, to automatically set these variables upon login, you need to manually register .cubrid.sh (or .cubrid.csh) in a shell startup file such as .bashrc (or .cshrc), depending on your shell environment.

```
$ . /home1/cub_user/.cubrid.sh
```

After CUBRID is installed, you can start CUBRID Manager server and CUBRID broker as follows.

```
$ cubrid service start
```

When you want to check whether CUBRID Manager server and CUBRID broker works well, you can use **grep** command in Linux as follows.

```
$ ps -ef | grep cub_
cub_user 15200 1 0 18:57   00:00:00 cub_master
cub_user 15205 1 0 18:57 pts/17 00:00:00 cub_broker
cub_user 15210 1 0 18:57 pts/17 00:00:00 query_editor_cub_cas_1
cub_user 15211 1 0 18:57 pts/17 00:00:00 query_editor_cub_cas_2
cub_user 15212 1 0 18:57 pts/17 00:00:00 query_editor_cub_cas_3
cub_user 15213 1 0 18:57 pts/17 00:00:00 query_editor_cub_cas_4
cub_user 15214 1 0 18:57 pts/17 00:00:00 query_editor_cub_cas_5
cub_user 15217 1 0 18:57 pts/17 00:00:00 cub_broker
cub_user 15222 1 0 18:57 pts/17 00:00:00 broker1_cub_cas_1
cub_user 15223 1 0 18:57 pts/17 00:00:00 broker1_cub_cas_2
cub_user 15224 1 0 18:57 pts/17 00:00:00 broker1_cub_cas_3
cub_user 15225 1 0 18:57 pts/17 00:00:00 broker1_cub_cas_4
cub_user 15226 1 0 18:57 pts/17 00:00:00 broker1_cub_cas_5
cub_user 15229 1 0 18:57   00:00:00 cub_auto start
cub_user 15232 1 0 18:57   00:00:00 cub_js start
```

Installing CUBRID (rpm File)

You can install CUBRID by using rpm file that is created on CentOS 6. The way of installing and uninstalling CUBRID is the same as that of using general rpm utility. While CUBRID is being installed, a new system group (cubrid) and a user account (cubrid) are created. After installation is complete, you should log in with a cubrid user account to start a CUBRID service.:

```
$ rpm -Uvh CUBRID-11.4-latest-Linux.x86_64.rpm
```

When rpm is executed, CUBRID is installed in the “cubrid” home directory (/opt/cubrid) and related configuration file (cubrid.[c]sh) is installed in the /etc/profile.d directory. Note that *demodb* is not automatically installed. Therefore, you must executed /opt/cubrid/demo/

make_cubrid_demo.sh with “cubrid” Linux ID. When installation is complete, enter the code below to start CUBRID with “cubrid” Linux ID.

```
$ cubrid service start
```

Note

· **RPM and dependency**

You must check RPM dependency when installing with RPM. If you ignore (--nodeps) dependency, it may not be executed.

· **cubrid account and DB exists even if you remove RPM package**

Even if you remove RPM, user accounts and databases that are created after installing, you must remove it manually, if needed.

· **Running CUBRID automatically in Linux when the system is started**

When you use SH package to install CUBRID, the cubrid script will be included in the $CUBRID/share/init.d directory. In this file, you can find the environment variable, **CUBRID_USER**. You should change this variable to the Linux account with which CUBRID has been installed and register it in /etc/init.d, then you can use service or chkconfig command to run CUBRID automatically when the Linux system is started.

When you use RPM package to install CUBRID, the cubrid script will be included in /etc/init.d. But you still need to change the environment variable, $CUBRID_USER from “cubrid” script file.

· **In /etc/hosts file, check if a hostname and an IP address mapping is normal**

If a hostname and an IP address is abnormally mapped, you cannot start DB server. Therefore, you should check if they are normally mapped.

Upgrading CUBRID

When you specify an installation directory where the previous version of CUBRID is already installed, a message which asks to overwrite files in the directory will appear. Entering **no** will stop the installation.

```
Directory '/home1/cub_user/CUBRID' exist!
If a CUBRID service is running on this directory, it may be
```

```
terminated abnormally.
And if you don't have right access permission on this
directory(subdirectories or files), install operation will be failed.
Overwrite anyway? (yes or no) [Default: no] : yes
```

Choose whether to overwrite the existing configuration files during the CUBRID installation. Entering **yes** will overwrite and back up them as extension .bak files.

```
The configuration file (.conf or .pass) already exists. Do you want
to overwrite it? (yes or no) : yes
```

For more information on upgrading a database from a previous version to a new version, see Upgrade.

Configuring Environment

You can modify the environment such as service ports etc. edit the parameters of a configuration file located in the **$CUBRID/conf** directory. See Installing and Running CUBRID on Windows for more information.

Installing CUBRID Interfaces

You can download interface modules such as CCI, JDBC, PHP, ODBC, OLE DB, ADO.NET, Ruby, Python and Node.js from https://www.cubrid.org/downloads.

A simple description on each driver can be found on API Reference.

Installing CUBRID Tools

You can download various tools including CUBRID Admin and CUBRID Migration Toolkit from https://www.cubrid.org/downloads.

Installing and Running CUBRID on Windows

Checklist before Installing

You should check the below before installing CUBRID for Windows.

- 64-bit

  CUBRID supports only 64-bit Windows. You can check the version by selecting [My Computer] > [System Properties]. Make sure to install a CUBRID 64-bit version on 64-bit Windows.

> **⚠ Warning**

10.1 would be the last release of 32-bit Windows.

Installation Process

Step 1: Specifying the directory to install

Step 2: Creating a sample database

To create a sample database, it requires about 1.5GB disk space.

Step 3: Completing the installation

CUBRID Service Tray appears on the right bottom.

Note

CUBRID Service is automatically started when the system is rebooted. If you want to stop the when the system is rebooted, change the "Start parameters" of "CUBRIDService" as "Stop"; "Control Panel > Administrative Tools > Services" and double-clicking "CUBRIDService", then pop-up window will be shown.

Checklist After Installation

- Whether the start of CUBRID Service Tray or not

  If CUBRID Service Tray is not automatically started when starting a system, confirm the following.

  - Check if Task Scheduler is started in [Start button] > [Control panel] > [Administrative Tools] > [Services]; if not, start Task Scheduler.
  - Check if CUBRID Service Tray is registered in [Start button] > [All Programs] > [Startup]; if not, register CUBRID Service Tray.

Upgrading CUBRID

To install a new version of CUBRID in an environment in which a previous version has already been installed, select [CUBRID Service Tray] > [Exit] from the menu to stop currently running services, and then remove the previous version of CUBRID. Note that when you are prompted with "Do you want to delete all the existing version of databases and the configuration files?" you must select "No" to protect the existing databases.

For more information on upgrading a database from a previous version to a new version, see Upgrade.

Configuring Environment

You can change configuration such as service ports to meet the user environment by changing the parameter values of following files which are located in the **%CUBRID%\conf** directory. If a firewall has been configured, the ports used in CUBRID need to be opened.

- **cm.conf**

  A configuration file for CUBRID Manager. The port that the Manager server process uses is called **cm_port** and its default value is **8001**.

- **cubrid.conf**

  A configuration file for server. You can use it to configure the following values: database memory, the number threads based on the number of concurrent users, communication port between broker and server, etc. The port that a master process uses is called cubrid_port_id and its default value is 1523. For details, see cubrid.conf Configuration File and Default Parameters.

- **cubrid_broker.conf**

  A configuration file for broker. You can use it to configure the following values: broker port, the number of application servers (CAS), SQL LOG, etc. The port that a broker uses is called **BROKER_PORT**. A port you see in the drivers such as JDBC is its corresponding broker's port. **APPL_SERVER_PORT** is a port that a broker application server (CAS) uses and it is added only in Windows. The default value is **BROKER_PORT** +1. The number of ports used is the same as the number of CAS, starting from the specified port's number plus 1. For details, see Parameter by Broker. For example, if the value of **APPL_SERVER_PORT** is 35000 and the maximum number of CASes by **MAX_NUM_APPL_SERVER** is 50, then listening ports on CASes are 35000, 35001, ..., 35049. For more details, see Parameter by Broker.

  The **CCI_DEFAULT_AUTOCOMMIT** broker parameter is supported since 2008 R4.0. The default value in the version is **OFF** and it is later changed to **ON**. Therefore, users who have upgraded from 2008 R4.0 to 2008 R4.1 or later versions should change this value to **OFF** or configure the autocommit mode to **OFF**.

Installing CUBRID Interfaces

You can download interface modules such as CCI, JDBC, PHP, ODBC, OLE DB, ADO.NET, Ruby, Python and Node.js from https://www.cubrid.org/downloads.

A simple description on each driver can be found on API Reference.

Installing CUBRID Tools

You can download various tools including CUBRID Admin and CUBRID Migration Toolkit from https://www.cubrid.org/downloads.

Installing with a Compressed Package

Installing CUBRID with tar.gz on Linux

Checklist before Installing

Check the following before installing CUBRID for Linux.

- glibc version

  Only supports glibc 2.3.4 or later. The glibc version can be checked as follows:

```
%rpm -q glibc
```

- 64-bit

  As 10.0, CUBRID supports only 64-bit Linux. You can check the version as follows:

```
% uname -a
Linux host_name 2.6.18-53.1.14.el5xen #1 SMP Wed Mar 5 12:08:17 EST
2008 x86_64 x86_64 x86_64 GNU/Linux
```

  Make sure to install the CUBRID 64-bit version on 64-bit Linux.

- The libraries that should be added.
  - Curses Library (rpm -q ncurses)

    CUBRID is packaged with version 5 of Curses library. You may need to install ncurses-compat-libs package if your system has newer version and downgrade is not possible.
  - gcrypt Library (rpm -q libgcrypt)
  - stdc++ Library (rpm -q libstdc++)
- Check if the mapping between hostnames and IP addresses are correct in the /etc/hosts file.

  If hostnames and IP addresses are matched incorrectly, DB server cannot be started normally. Therefore, check if they are correctly mapped.

Installation Process

Specifying the Directory to Install

- Decompress the compressed file to the directory to install.

```
tar xvfz CUBRID-11.4-latest-Linux.x86_64.tar.gz /home1/
cub_user/
```

CUBRID directory is created under /home1/cub_user/ and files are created under CUBRID directory.

Specifying Environment Variables

1. Add below environment variables to a shell script which is run automatically and located under the home directory of a user.

   You may have to create a directory for **$CUBRID_DATABASES**. You can designate any directory you have enough permission.

   The below is an example to add environment variables to .bash_profile when you run on the bash shell.

```
export CUBRID=/home1/cub_user/CUBRID
export CUBRID_DATABASES=$CUBRID/databases
```

2. Add CUBRID JDBC library filename to the **CLASSPATH** environment variable.

```
export CLASSPATH=$CUBRID/jdbc/cubrid_jdbc.jar:$CLASSPATH
```

3. Add CUBRID bin directory to **PATH** environment variables.

```
export PATH=$CUBRID/bin:$PATH
```

Creating DB

- Move to the directory to create DB on the console and create DB.

```
cd $CUBRID_DATABASES
mkdir testdb
cd testdb
cubrid createdb --db-volume-size=128M --log-volume-
size=128M testdb en_US
```

Auto-starting when Booting

- "cubrid" script is included in the **$CUBRID/share/init.d** directory. Change the value of **$CUBRID_USER** environment variable into the Linux account which installed

CUBRID and register this script to **/etc/init.d**; then you can start automatically by using "service" or "chkconfig" command.

Auto-starting DB

- To start DB automatically when you booting a system, change the below in **$CUBRID/conf/cubrid.conf**.

```
[service]
service=server, broker, manager
server=testdb
```

- In the "service" parameter, processes to be auto-started are specified.
- In the "server" parameter, DB name to be auto-started is specified.

For environment setting, tools installation and interfaces installation after CUBRID installation, see Installing and Running CUBRID on Linux.

Installing CUBRID with zip on Windows

Checklist before Installing

Check below list before installing CUBRID database of Windows version.

- 64bit

  CUBRID supports only 64-bit Windows. You can check the version by selecting [My Computer] > [System Properties]. Make sure to install a CUBRID 64-bit version on 64-bit Windows.

> **Warning**
>
> 10.1 would be the last release of 32-bit Windows.

Installation Process

Specifying the Directory to Install

- Decompress the compressed file to the directory to install.

```
C:\CUBRID
```

- You may have to create a directory for **$CUBRID_DATABASES**. You can designate any directory you have enough permission.

Specifying Environment Variables

1. Select [Start button] > [Computer] > (click right mouse button) > [Properties] > [Advanced system settings] > [Environment Variables].

2. Click [New ...] under the system variables and add system variables as below.

```
CUBRID = C:\CUBRID
CUBRID_DATABASES = %CUBRID%\databases
```

3. Add CUBRID JDBC library name to **CLASSPATH** system variable.

```
%CUBRID%\jdbc\cubrid_jdbc.jar
```

4. Add CUBRID bin and ccibin directory to **Path** system variable.

```
%CUBRID%\cci\bin;%CUBRID%\bin
```

Note

Setting environment variables for using CUBRID can be conveniently configured by using the batch file (cubrid_env.bat) below. C:CUBRIDshardwindows_scriptscubrid_env.bat

Creating DB

- Run **cmd** command and open the console; move to the directory to create DB and create DB.

```
cd C:\CUBRID\databases
md testdb
cd testdb
c:\CUBRID\databases\testdb>cubrid createdb --db-volume-
size=128M --log-volume-size=128M testdb en_US
```

Auto-starting when Booting

- To start CUBRID automatically when booting the Windows system, CUBRID Service should be registered to Windows Service.

  1. Register CUBRID Service to Windows Service.

```
C:\CUBRID\bin\ctrlService.exe -i C:\CUBRID\bin
```

2. The below shows how to start/stop CUBRID Service.

```
C:\CUBRID\bin\ctrlService.exe -start/-stop
```

Auto-starting DB

- To start DB when booting on Windows, change below in C:\CUBRID\conf\cubrid.conf.

```
[service]
service=server, broker, manager
server=testdb
```

  - Specify the processes to start automatically on the “service” parameter.
  - Specify the DB name to start automatically on the “server” parameter.

Removing from Service

- To remove registered CUBRID Service, run the following.

```
C:\CUBRID\bin\ctrlService.exe -u
```

Registering CUBRID Service Tray

Since CUBRID Service Tray is not automatically registered when installing CUBRID with zip file, it is required to register manually if you want CUBRID Service Tray.

1. Create a link of C:\CUBRID\bin\CUBRID_Service_Tray.exe in [Start button] > [All Programs] > [Startup].
2. Input “regedit” in [Start button] > [Accessories] > [Run] to run a registry editor.
3. Create CUBRID folder under [Computer] > [HKEY_LOCAL_MACHINE] > [SOFTWARE].
4. Create [cmclient] folder under [CUBRID] folder(Edit > New > Key) and add below items(Edit > New > String Value).

```
Name          Type       Data

ROOT_PATH     REG_SZ     C:\CUBRID\cubridmanager
```

5. Create [cmserver] folder under [CUBRID] folder(Edit > New > Key) and add below items(Edit > New > String Value).

```
Name            Type        Data

ROOT_PATH       REG_SZ      C:\CUBRID
```

6. Create [CUBRID] folder under [CUBRID] folder(Edit > New > Key) and add below items(Edit > New > String Value).

```
Name            Type        Data

ROOT_PATH       REG_SZ      C:\CUBRID
```

7. When rebooting Windows, CUBRID Service Tray is created under right side.

Checklist After Installation

- Whether the start of CUBRID Service Tray or not

  If CUBRID Service Tray is not automatically started when starting a system, confirm the following.

  - Check if Task Scheduler is started in [Start button] > [Control panel] > [Administrative Tools] > [Services]; if not, start Task Scheduler.
  - Check if CUBRID Service Tray is registered in [Start button] > [All Programs] > [Startup]; if not, register CUBRID Service Tray.

For environment setting, tools installation and interfaces installation after CUBRID installation, see Installing and Running CUBRID on Windows.

Note

If CUBRID is installed as a zip file, it may not be executed because there is no DLL required to execute CUBRID. In this case, you need to install Microsoft Visual C++ Redistributable.

- **Microsoft Visual C++ Redistributable**

  https://visualstudio.microsoft.com/downloads/

Configuring Environment Variables

The following environment variables need to be set in order to use the CUBRID. The necessary environment variables are automatically set when the CUBRID system is installed or can be changed, as needed, by the user.

CUBRID Environment Variables

- **CUBRID**: The default environment variable that designates the location where the CUBRID is installed. This variable must be set accurately since all programs included in the CUBRID system uses this environment variable as reference.
- **CUBRID_DATABASES**: The environment variable that designates the location of the **databases.txt** file. The CUBRID system stores the absolute path of database volumes in the **$CUBRID_DATABASES/databases.txt** file. See databases.txt File.
- **CUBRID_MSG_LANG**: The environment variable that specifies the language used for usage and error messages in CUBRID. The initial value upon start is undefined, and if not defined, it is set to **en_US**. Available values for the environment variable are en_US, en_US.utf8, ko_KR.euckr, and ko_KR.utf8.

Note

- A user of CUBRID Manager should specify **CUBRID_MSG_LANG**, an environment variable of DB server node into **en_US** to print out messages normally after running database related features. However, database related features are run normally and just the output messages are broken when **CUBRID_MSG_LANG** is not **en_US**.
- To apply the changed **CUBRID_MSG_LANG**, CUBRID system of DB server node should be restarted(cubrid service stop, cubrid service start).

- **CUBRID_TMP**: The environment variable that specifies the location where CUBRID stores temporary files. There are two types of temporary files: regular files and UNIX domain sockets. If specified, temporary files are created in the directory specified in the environment variable CUBRID_TMP.
  - **Regular files**: Temporary files are created by utilities such as **plandump** in addition to the database server and csql. If this environment variable is not set, the following directory is used.
    - Linux: /tmp

        - Windows: C:\Windows\TEMP

    - **UNIX Domain Socket files**: If this environment variable is not set, UNIX Domain sockets will be created in the following directories for each process (Applicable to **Linux** only and does not apply to **Windows**):

        - cub_master: **/tmp**
        - cub_broker process: **$CUBRID/var/CUBRID_SOCK**
        - cub_pl process: **$CUBRID/var/CUBRID_SOCK**

Note

- The **java.io.tmpdir** option within the **stored_procedure_vm_options** specifies the directory where temporary files for the cub_pl process (the built-in Java VM) are created. If the **CUBRID_TMP** environment variable is set, the value of **java.io.tmpdir** is ignored, and the directory specified by CUBRID_TMP will be used instead.
- To set this environment variable in Windows, you must add the CUBRID_TMP key to the registry (refer **%CUBRID%**\share\windows_scripts\cubrid_env.bat).

CUBRID_TMP value has some constraints, which are as follows:

- Since the maximum length of the UNIX socket path is 108, when a path longer than 108 is entered in **$CUBRID_TMP**, an error is displayed.

```
$ export CUBRID_TMP=/home1/testusr/cubrid=/tmp/
12345678901234567890123456789012345678901234567890123456789012345678
901234567890123456789012345678
$ cubrid server start apricot

The $CUBRID_TMP is too long. (/home1/testusr/cubrid=/tmp/
12345678901234567890123456789012345678901234567890123456789012345678
901234567890123456789012345678)
```

- When the relative path is entered, an error is displayed.

```
$ export CUBRID_TMP=./var
$ cubrid server start testdb

The $CUBRID_TMP should be an absolute path. (./var)
```

CUBRID_TMP can be used to avoid the following problems that can occur at the default path of the UNIX domain socket that CUBRID uses.

- **/tmp** is used to store the temporary files in Linux. If the system administrator periodically and voluntarily cleans the space, the UNIX domain socket may be removed. In this case, configure **$CUBRID_TMP** to another path, not **/tmp**.
- The maximum length of the UNIX socket path is 108. When the installation path of CUBRID is too long and the **$CUBRID/var/CUBRID_SOCK** path that store the UNIX socket path for cub_broker exceeds 108 characters, the broker cannot be executed. Therefore, the path of **$CUBRID_TMP** must not exceed 108 characters.

The above mentioned environment variables are set when the CUBRID is installed. However, the following commands can be used to verify the setting.

- Linux

```
% printenv CUBRID
% printenv CUBRID_DATABASES
% printenv CUBRID_MSG_LANG
% printenv CUBRID TMP
```

- Windows

```
C:\> set CUBRID
```

OS Environment

- PATH: In the Linux environment, the directory **$CUBRID/bin**, which includes a CUBRID system executable file, must be included in the PATH environment variable.
- LD_LIBRARY_PATH: In the Linux environment, **$CUBRID/lib** and **$CUBRID/cci/lib**, which is the CUBRID system's dynamic library file (libjvm.so), must be included in the **LD_LIBRARY_PATH** (or **SHLIB_PATH** or **LIBPATH**) environment variable.
- Path: In the Windows environment, the **%CUBRID%\bin** and **%CUBRID%\cci\bin**, which is a directory that contains CUBRID system's execution file, must be included in the **Path** environment variable.

Note

- In the Linux environment, you can control the heap memory allocation behavior of **CUBRID's memory manager (GLIB memory manager)** through the following environment variable

settings. For more details, refer to: https://www.gnu.org/software/libc/manual/html_node/Malloc-Tunable-Parameters.

Environment variables	Descriptions	Default
MALLOC_MMAP_MAX_	Specifies the maximum number of memory blocks that may be allocated via the mmap().	65536
MALLOC_MMAP_THRESHOLD_	Specifies the size threshold for deciding whether to use the mmap() system call for memory allocation.	128 KiB
MALLOC_TOP_PAD_	Specifies the size of additional memory padding to be maintained at the top of the heap.	0
MALLOC_TRIM_THRESHOLD_	Specifies the threshold for deciding whether freed memory is released back to the kernel. Released if the threshold is exceeded.	128 KiB
MALLOC_ARENA_MAX	Specifies the maximum number of memory arenas that a process is allowed to create.	8 times of core

- In the installation package, .cubrid.sh (or .cubrid.csh) sets the **MALLOC_TRIM_THRESHOLD_** value to 0, while the other environment variables are commented out.

Note

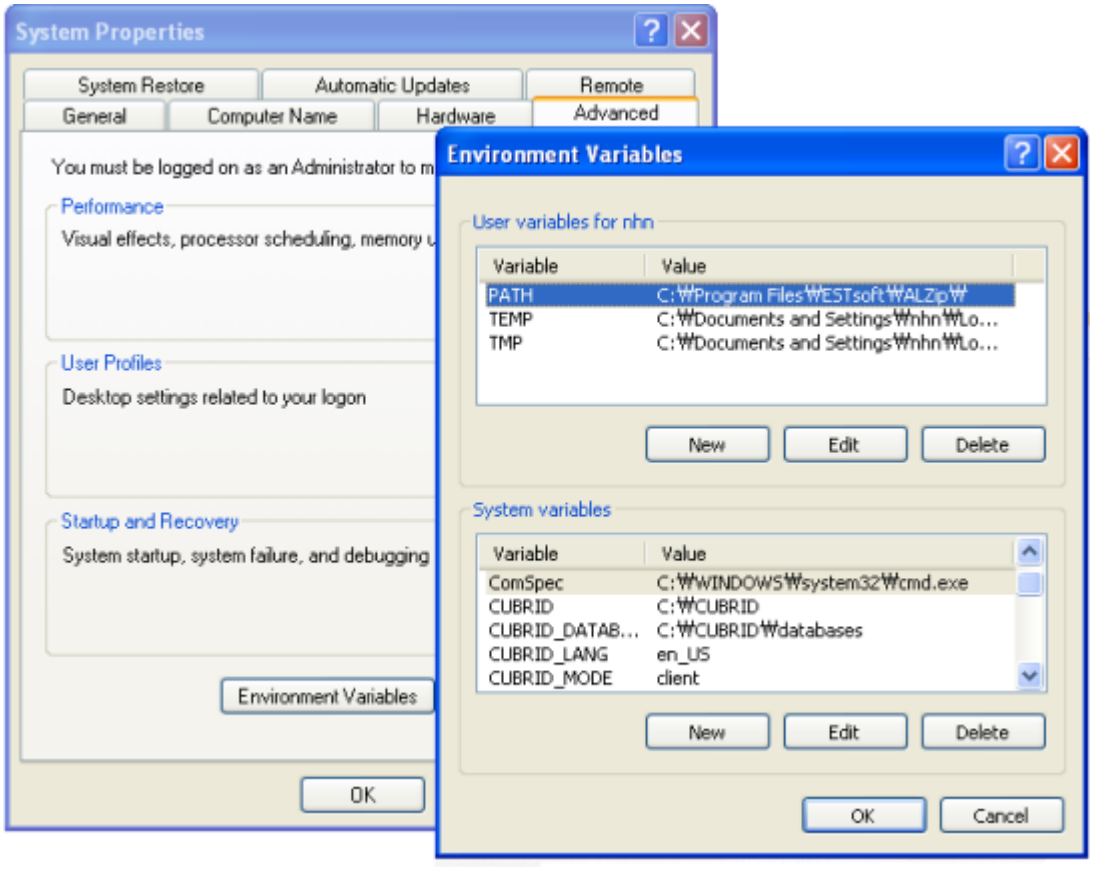
- In Linux environments, the **LD_PRELOAD** environment variable can be used to override the default memory manager used by CUBRID.
- However, replacing the memory manager may cause unexpected behavior during CUBRID operation, so it should be done with careful consideration.
- The scripts in the installation package (.cubrid.sh and .cubrid.csh) include the **LD_PRELOAD** environment variable, but it is commented out by default.

```
예) export LD_PRELOAD=/usr/lib64/jemalloc.so.1
```

Configuring the Environment Variable

For Windows

If the CUBRID system has been installed on Windows, then the installation program automatically sets the necessary environment variable. Select [Systems Properties] in [My Computer] and select the [Advanced] tab. Click the [Environment Variable] button and check the setting in the [System Variable]. The settings can be changed by clicking on the [Edit] button. See the Windows help for more information on how to change the environment variable on Windows.



For Linux

If the CUBRID system has been installed on Linux, the installation program automatically creates the **.cubrid.sh** or **.cubrid.csh** file and makes configurations so that the files are automatically called from the installation account's shell login script. The following is the contents of . **cubrid.sh** environment variable configuration that was created in an environment that uses sh, bash, etc.

```
CUBRID=/home1/cub_user/CUBRID
CUBRID_DATABASES=/home1/cub_user/CUBRID/databases
ld_lib_path=`printenv LD_LIBRARY_PATH`

if [ "$ld_lib_path" = "" ]
then
    LD_LIBRARY_PATH=$CUBRID/lib:$CUBRID/cci/lib
else
    LD_LIBRARY_PATH=$CUBRID/lib:$CUBRID/cci/lib:$LD_LIBRARY_PATH
fi

SHLIB_PATH=$LD_LIBRARY_PATH
LIBPATH=$LD_LIBRARY_PATH
PATH=$CUBRID/bin:$CUBRID/cubridmanager:$PATH

export CUBRID
export CUBRID_DATABASES
export LD_LIBRARY_PATH
export SHLIB_PATH
```

```
export LIBPATH
export PATH
```

Language & Charset Setting

The language and the charset that will be used in the CUBRID DBMS is specified after the database name when DB is created(e.g. cubrid createdb testdb ko_KR.utf8). The following are examples of values that can currently be set as a language and a charset.

- **en_US.iso88591**: English ISO-88591 encoding(.iso88591 can be omitted)
- **ko_KR.euckr**: Korean EUC-KR encoding
- **ko_KR.utf8**: Korean UTF-8 encoding(.utf8 can be omitted)
- **de_DE.utf8**: German UTF-8 encoding
- **es_ES.utf8**: Spanish UTF-8 encoding
- **fr_FR.utf8**: French UTF-8 encoding
- **it_IT.utf8**: Italian UTF-8 encoding
- **ja_JP.utf8**: Japanese UTF-8 encoding
- **km_KH.utf8**: Cambodian UTF-8 encoding
- **tr_TR.utf8**: Turkish UTF-8 encoding(.utf8 can be omitted)
- **vi_VN.utf8**: Vietnamese UTF-8 encoding
- **zh_CN.utf8**: Chinese UTF-8 encoding
- **ro_RO.utf8**: Romanian UTF-8 encoding

Language and charset setting of CUBRID affects read and write data. The language is used for messages displayed by the program.

For more details related to charset, locale and collation settings, see An Overview of Globalization.

Port Setting

If ports are closed, the ports used by CUBRID should be opened.

The following table summarizes the ports used by CUBRID. Each port on the listener that waits for connection from the opposite side should be opened.

To open the ports for a specific process on the Linux firewall, follow the guide described for the corresponding firewall program.

If available ports for Windows are used, you cannot know which port will be opened. In this case, enter “firewall” in the “Control Panel” of the Windows menu and then choose “Windows Firewall> Allow a program or functionality through Windows Firewall” and then add the program for which port should be opened.

This method can be used for the case that it is difficult to specify a specific port in Windows. This method is recommended since it is safer to add a program to the Allowed programs list than to open a port without specifying a program on the Windows firewall.

- Add “%CUBRID%\bin\cub_broker.exe” to open all ports for cub_broker.
- Add “%CUBRID%\bin\cub_cas.exe” to open all ports for CAS.
- Add “%CUBRID%\bin\cub_master.exe” to open all ports for cub_master.
- Add “%CUBRID%\bin\cub_server.exe” to open all ports for cub_server.
- **Add “%CUBRID%\bin\cub_cmserver.exe” to open all ports for the CUBRID Manager.**
    - Add “%CUBRID%\bin\cub_pl.exe” to open all ports for the CUBRID PL server (cub_pl).

If you use CUBRID for Linux at the broker machine or the DB server machine, all of Linux ports should be opened. If you use CUBRID for Windows at the broker machine or the DB server machine, all of Linux ports should be opened or the related processes should be added to the program list allowed for the Windows firewall.

Label	Listener	Requester	Linux Port	Windows Port	Firewall Port Setting	Description
Default use	cub_broker	application	BROKER_PORT	BROKER_PORT	Open	One-time connection
	CAS	application	BROKER_PORT	APPL_SERVER_PORT ~ (APP_SERVER_PORT + # of CAS - 1)	Open	Keep connected

Label	Listener	Requester	Linux Port	Windows Port	Firewall Port Setting	Description
	cub_master	CAS	cubrid_port_id	cubrid_port_id	Open	One-time connection
	cub_server	CAS	cubrid_port_id	A random available port	Linux: Open Windows: Program	Keep connected
	Client machine(*)	cub_server	ECHO(7)	ECHO(7)	Open	Periodical connection
	Server machine(**)	CAS, CSQL	ECHO(7)	ECHO(7)	Open	Periodical connection
HA use	cub_broker	application	BROKER_PORT	Not supported	Open	One-time connection
	CAS	application	BROKER_PORT	Not supported	Open	Keep connected
	cub_master	CAS	cubrid_port_id	Not supported	Open	One-time connection
	cub_master (slave)	cub_master (master)	ha_port_id	Not supported	Open	Periodical connection, check

Label	Listener	Requester	Linux Port	Windows Port	Firewall Port Setting	Description
						the heartbeat
	cub_master (master)	cub_master (slave)	ha_port_id	Not supported	Open	Periodical connection, check the heartbeat
	cub_server	CAS	cubrid_port_id	Not supported	Open	Keep connected
	Client machine(*)	cub_server	ECHO(7)	Not supported	Open	Periodical connection
	Server machine(**)	CAS, CSQL, copylogdb, applylogdb	ECHO(7)	Not supported	Open	Periodical connection
Manager use	Manager server	application	8001	8001	Open	
PL use	cub_pl	cub_server	stored_procedure_port	stored_procedure_port	Open	Keep connected

(*): The machine which has the CAS, CSQL, copylogdb, or applylogdb process

(**): The machine which has the cub_server

The detailed description on each classification is given as follows.

Default Ports for CUBRID

The following table summarizes the ports required for each OS, based on the listening processes. Each port on the listener should be opened.

Listener	Requester	Linux port	Windows port	Firewall Port Setting	Description
cub_broker	application	BROKER_PORT	BROKER_PORT	Open	One-time connection
CAS	application	BROKER_PORT	APPL_SERVER_PORT ~ (APP_SERVER_PORT + # of CAS - 1)	Open	Keep connected
cub_master	CAS	cubrid_port_id	cubrid_port_id	Open	One-time connection
cub_server	CAS	cubrid_port_id	A random available port	Linux: Open Windows: Program	Keep connected
Client machine(*)	cub_server	ECHO(7)	ECHO(7)	Open	Periodical connection
Server machine(**)	CAS, CSQL	ECHO(7)	ECHO(7)	Open	Periodical connection

(*): The machine which has the CAS or CSQL process

(**): The machine which has the cub_server

Note

In Windows, you cannot specify the ports to open because CAS randomly specifies the ports as accessing the cub_server. In this case, add "%CUBRID%\bin\cub_server.exe" to "Windows Firewall > Allowed programs".

As the server process (cub_server) and the client processes (CAS, CSQL) cross-check if the opposite node is normally running or not by using the ECHO(7) port, you should open the ECHO(7) port if there is a firewall. If the ECHO port cannot be opened for both the server and the client, set the check_peer_alive parameter value of the cubrid.conf to none.

The relation of connection between processes is as follows:

```
application - cub_broker
            -> CAS  -  cub_master
                    -> cub_server
```

- application: The application process
- cub_broker: The broker server process. It selects CAS to connect with the application.
- CAS: The broker application server process. It relays the application and the cub_server.
- cub_master: The master process. It selects the cub_server to connect with the CAS.
- cub_server: The database server process

The symbols of relation between processes and the meaning are as follows:

- - : Indicates that the connection is made only once for the initial.
- ->, <- : Indicates that the connection is maintained. The right side of -> or the left side of <- is the party that the arrow symbol indicates. The party that the arrow symbol indicates is the listener which listens to the opposite process.
- (master): Indicates the master node in the HA configuration.
- (slave): Indicates the slave node in the HA configuration.

The connection process between the application and the DB is as follows:

1. The application tries to connect to the cub_broker through the broker port (BROKER_PORT) set in the cubrid_broker.conf.
2. The cub_broker selects a connectable CAS.

3. The application and CAS are connected.

   In Linux, established TCP connection between the **BROKER** and the client will be passed to the **CAS**. Therefore, there is no need for an additional network port for the application to connect to the CAS. However, in **Windows**, when an application connects to a BROKER, the BROKER delivers the network port number to connect to the available CAS to the application. After the client closes the current connection with the BROKER, it connects to the CAS with the received network port number from the BROKER. If the **APPL_SERVER_PORT** parameter is not set, the network port used by the first CAS becomes BROKER_PORT + 1.

   For example, if the BROKER_PORT is 33000 and the APPL_SERVER_PORT value has not been set in Windows, the ports used between the application and CAS are as follows:

   - The port used to connect the application to the CAS(1): 33001
   - The port used to connect the application to the CAS(2): 33002
   - The port used to connect the application to the CAS(3): 33003

4. CAS sends a request of connecting with the cub_server to the cub_master through the cubrid_port_id port set in the cubrid.conf.

5. CAS and the cub_server are connected.

   In Linux, you should use the cubrid_port_id port as CAS is connected to the cub_server through the Unix domain socket. In Windows, CAS is connected to the cub_server through a random available port as the Unix domain socket cannot be used. If the DB server is running in Windows, a random available port is used between the broker machine and the DB server machine. In this case, note that the operation may not be successful if a firewall blocks the port for the process between the two machines.

6. After that, CAS keeps connection with the cub_server even if the application is terminated until the CAS restarts.

Ports for CUBRID HA

The CUBRID HA is supported in Linux only.

The following table summarizes the ports required for each OS, based on the listening processes. Each port on the listener should be opened.

Listener	Requester	Linux port	Firewall Port Setting	Description
cub_broker	application	BROKER_PORT	Open	One-time connection
CAS	application	BROKER_PORT	Open	Keep connected
cub_master	CAS	cubrid_port_id	Open	One-time connection
cub_master (slave)	cub_master (master)	ha_port_id	Open	Periodical connection, check the heartbeat
cub_master (master)	cub_master (slave)	ha_port_id	Open	Periodical connection, check the heartbeat
cub_server	CAS	cubrid_port_id	Open	Keep connected
Client machine(*)	cub_server	ECHO(7)	Open	Periodical connection
Server machine(**)	CAS, CSQL, copylogdb, applylogdb	ECHO(7)	Open	Periodical connection

(*): The machine which has the CAS, CSQL, copylogdb, or applylogdb process

(**): The machine which has the cub_server

As the server process (cub_server) and the client processes (CAS, CSQL, copylogdb, applylogdb, etc.) cross-check if the opposite node is normally running or not by using the ECHO(7) port, you should open the ECHO(7) port if there is a firewall. If the ECHO port cannot be opened for both

the server and the client, set the ref:*check_peer_alive <check_peer_alive>* parameter value of the cubrid.conf to none.

The relation of connection between processes is as follows:

```
application - cub_broker
            -> CAS  -  cub_master(master) <-> cub_master(slave)
                    -> cub_server(master)     cub_server(slave) <-
applylogdb(slave)
                                          <-----------------------
copylogdb(slave)
```

- cub_master(master): the master process on the master node in the CUBRID HA configuration. It checks if the peer node is alive.
- cub_master(slave): the master process on the slave node in the CUBRID HA configuration. It checks if the peer node is alive.
- copylogdb(slave): the process which copies the replication log on the slave node in the CUBRID HA configuration
- applylogdb(slave): the process which applies the replication log on the slave node in the CUBRID HA configuration

For easy understanding for the replication process from the master node to the slave node, the applylogdb and copylogdb on the master node and CAS on the slave node have been omitted.

The symbols of relation between processes and the meaning are as follows:

- - : Indicates that the connection is made only once for the initial.
- ->, <- : Indicates that the connection is maintained. The right side of -> or the left side of <- is the party that the arrow symbol indicates. The party that the arrow symbol indicates is the listener which listens to the opposite process.
- (master): Indicates the master node in the HA configuration.
- (slave): Indicates the slave node in the HA configuration.

The connection process between the application and the DB is identical with Default Ports for CUBRID. This section describes the connection process between the master node and the slave node when the master DB and the slave DB are configured 1:1 by the CUBRID HA.

1. The ha_port_id set in the cubrid_ha.conf is used between the cub_master(master) and the cub_master(slave).

2. The copylogdb(slave) sends a request for connecting with the master DB to the cub_master(master) through the port set in the cubrid_port_id of the cubrid.conf on the slave node. Finally, the copylogdb(slave) is connected with the cub_server(master).

3. The applylogdb(slave) sends a request for connecting with the slave DB to the cub_master(slave) through the port set in the cubrid_port_id of the cubrid.conf on the slave node. Finally, the applylogdb(slave) is connected with the cub_server(slave).

On the master node, the applylogdb and the copylogdb run for the case that the master node is switched to the slave node.

Ports for CUBRID Manager Server

The following table summarizes the ports, based on the listening processes, used for CUBRID Manager server. The ports are identical regardless of the OS type.

Listener	Requester	Port	Firewall Port Setting
Manager server	application	8001	Open

- The port used when the CUBRID Manager client accesses the CUBRID Manager server process is **cm_port** of the cm.conf. The default value is 8001.

Ports for CUBRID Procdedure Language (PL) Server

The following table summarizes the ports, based on the listening processes, used for CUBRID Procedural Language (PL) server. The ports are identical regardless of the OS type.

Listener	Requester	Port	Firewall Port Setting
cub_pl	cub_server	stored_procedure_port	Open

Behavior according to the OS type and the **stored_procedure_uds** setting in **cubrid.conf**

- Linux
    - If **stored_procedure_uds** is set to **yes**, a UNIX domain socket is used and a TCP socket is not used.

    - If **stored_procedure_uds** is set to **no**, a TCP socket is used, and the port is determined by the **stored_procedure_port** setting.

- Windows

    - Windows does not support UNIX domain sockets.

    - Therefore, regardless of the **stored_procedure_uds** setting, a TCP socket is used, and the port is determined by the **stored_procedure_port** setting.

Note

The port used for communication between the CUBRID Procedural Langauge server (cub_pl) and cub_server is specified by **stored_procedure_port** in **cubrid.conf**. The default value is 0, which means that a random available port will be assigned.

Upgrade

Cautions during upgrade

Changes

Please confirm 11.4 Changes in the release notes.

Saving the Existing Configuration File

- Save the configuration files in the **$CUBRID/conf** directory (**cubrid.conf**, **cubrid_broker.conf** and **cm.conf**) and the DB location file (**databases.txt**) in the **$CUBRID_DATABASES** directory.

Checking New Reserved Words

- You can check whether reserved words are being used or not by applying the CUBRID 11.4 reserved word detection script, check_reserved.sql, which is distributed through the CUBRID installation package or http://ftp.cubrid.org/CUBRID_Engine/11.4/. If the reserved words are being used as identifiers, the identifiers must be modified. See Identifier.

Configuring environment variables of CUBRID_MSG_LANG

- **CUBRID_LANG** and **CUBRID_CHARSET** environment variables are no more used, and the language and the charset should be configured during creating DB. **CUBRID_MSG_LANG** is used when displaying the messages of utilities or errors. If **CUBRID_MSG_LANG** is not configured, it follows the language and the charset specified when creating DB.

Changing schema

- 9.0 Beta or earlier version user which had used not ISO-8859-1 charset but EUC-KR charset or UTF-8 charset, should change the schema. In the earlier version of 9.0 Beta, the precision of **CHAR** or **VARCHAR** was specified as byte size. From 9.0 Beta, the precision is specified as character length.

Adding/Keeping locales

- If you have locales to want to add, add them into **$CUBRID/conf/cubrid_locales.txt** file and run **make_locale** script. For more details, see Locale Setting.
- If you want to keep the old version's locale, add the old version's locale to **$CUBRID/conf/cubrid_locales.txt** file and run **make_locale** script, and specify the old locale when running **"cubrid createdb"** command.

DB migration

- Since the DB volume of CUBRID 11.3 and earlier versions are not compatible with the DB volume of CUBRID 11.4, it should be migrated with cubrid unloaddb/loaddb utility. For more detail procedure, see DB migration.
- CUBRID 2008 R3.1 and later don't support GLO and the LOB type replaces the GLO feature. For this reason, applications or schemas that use GLO must be modified to be compatible with LOB.

Note

In 2008 R4.0 or before, TIMESTAMP '1970-01-01 00:00:00'(GMT) is the minimum value of TIMESTAMP, but in 2008 4.1 or later, it is recognized as zerodate and TIMESTAMP '1970-01-01 00:00:01'(GMT) is the minimum value of TIMESTAMP.

Compatibility table

Vesion	11.4	11.3	11.2	11.0	10.2	10.1	10.0
11.4	O						
11.3		O	O				
11.2		O	O				

Vesion	11.4	11.3	11.2	11.0	10.2	10.1	10.0
11.0				O			
10.2					O		
10.1						O	
10.0							O

Reconfiguring environments for replication or HA

- From 2008 R4.0, the replication feature is no longer supported; therefore, it is recommended to reconfigure the DB migration and HA environment for systems in which previous replication versions are used. In addition, for systems that use Linux Heartbeat-based HA feature, which is provided in CUBRID 2008 R2.0 and 2008 R2.1, you must reconfigure to DB migration and the CUBRID Heartbeat-based HA environment for better operational stability(see Database Migration under HA Environment).
- To reconfigure the HA environment configuration, see CUBRID HA in the manual.

Stored Function/Procedure

- To use stored functions/procedures, the CUBRID procedural langauge server (cub_pl) is started by default and is managed by the database server. Therefore, even if the procedural langauge server is stopped, it will be automatically restarted by the database server.
- If you do not want to run the CUBRID procedural langauge server, set the **stored_procedure** parameter in **cubrid.conf** to **no** and restart. In this case, the PL/CSQL and Java stored procedure functions cannot be used.

Upgrading from CUBRID 9.2/9.3/10.0/10.1/10.2/11.0/11.2/11.3 to CUBRID 11.4

Users who are using versions CUBRID 9.2/9.3/10.0/10.1/10.2/11.0/11.2/11.3 should install 11.4 in the different directory, migrate the databases to 11.4 and modify parameter values in the previous environment configuration file.

DB migration

The following table shows how to perform the migration using the reserved word detection script, check_reserved.sql, which is separately distributed from http://ftp.cubrid.org/CUBRID_Engine/11.4/ and the cubrid unloaddb/loaddb utilities. (See unloaddb and loaddb)

Step	Linux Environment	Windows Environment
Step C1: Stop CUBRID Service	% cubrid service stop	Stop CUBRID Service Tray.
Step C2: Execute the reserved words detection script	Execute the following command in the directory where the reserved word detection script is located. Execute migration or identifier modification by checking the detection result (For the allowable identifier). % csql -S -u dba -i check_reserved.sql testdb	
Step C3: Unload the earlier version of the DB	Store the databases.txt file and the configuration files under the conf directory of the earlier version in a separate directory (C3a). Execute the cubrid unloaddb utility and store the file generated at this point in a separate directory (C3b). % cubrid unloaddb -S -u dba testdb Delete the existing database (C3c). % cubrid deletedb testdb	
		Uninstall the earlier version of CUBRID.
Step C4: Install new version	See Installing and Running CUBRID	
Step C5: Database creation and data loading	Go to the directory where you want to create a database, and create one. At this time, be cautious about locale setting(*). (C5a) % cd $CUBRID/databases/testdb	

Step	Linux Environment	Windows Environment
	% cubrid createdb testdb en_US Execute the cubrid loaddb utility with the stored files in (C3b). (C5b) % cubrid loaddb -u dba -s testdb_schema -d testdb_objects -i testdb_indexes testdb	
Step C6: Back up the new version of the DB	% cubrid backupdb -S testdb	
Step C7: Configure the CUBRID environment and start the CUBRID Service	Modify the configuration file. At this point, partially modify the configuration files from the earlier version stored in step (C3a) to fit the new version. (For configuring system parameter, see Parameter configuration and System Parameters) % cubrid service start % cubrid server start testdb	Start the service by selecting CUBRID Service Tray > [Service Start]. Start the database server from the command prompt. % cubrid server start testdb

Parameter configuration

cubrid.conf

- The minimum size of **log_buffer_size** is changed from 48KB(3*1page, 16KB=1page) into 2MB(128*1page, 16KB=1page); therefore, this value should be larger than the changed minimum size.

Upgrading from CUBRID 9.1 to CUBRID 11.4

Users who are using versions CUBRID 9.1 should install 11.4 in the different directory, migrate databases to 11.4 and modify parameter values in the previous environment configuration file.

DB migration

Please refer DB migration for migration steps.

Parameter configuration

cubrid.conf

- The minimum size of **log_buffer_size** is changed from 48KB(3*1page, 16KB=1page) into 2MB(128*1page, 16KB=1page); therefore, this value should be larger than the changed minimum size.
- The value of **sort_buffer_size** should be configured as 2G or less since the maximum value of sort_buffer_size is 2G.
- In the following parameters, the old parameters will be deprecated and the new parameters are recommended to use. The value in the parenthesis is the unit of the value when the unit is omitted, and the new parameters can specify the unit after the value. For details, see each parameter's explanation in System Parameters

Old parameters(unit)	New parameters(unit)
lock_timeout_in_secs(sec)	lock_timeout(msec)
checkpoint_every_npages(page_count)	checkpoint_every_size(byte)
checkpoint_interval_in_mins(min)	checkpoint_interval(msec)
max_flush_pages_per_second(page_count)	max_flush_size_per_second(byte)
sync_on_nflush(page_count)	sync_on_flush_size(byte)
sql_trace_slow_msecs(msec)	sql_trace_slow(msecs)

cubrid_broker.conf

- In **KEEP_CONNECTION** parameter, OFF value should be changed as **ON** or **AUTO** since **OFF** setting value is no longer used.
- **SELECT_AUTO_COMMIT** should be deleted since this parameter is no longer used.
- The value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** should be 2,097,151 or less since the maximum value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is 2,097,151.

Environment variable

- **CUBRID_CHARSET** is removed, and now **CUBRID_CHARSET** is used for configuring the charset of database and **CUBRID_MSG_LANG** is used for configuring the charset of messages for utilities and errors.

> **Warning**
>
> When you create database, a language and a charset must be specified. It affects the length of string type, string comparison operation, etc. The specified charset when creating database cannot be changed later, so you should be careful when specifying it.
>
> For charset, locale and collation setting, see An Overview of Globalization.

Upgrading From CUBRID 2008 R4.1/R4.3/R4.4 To CUBRID 11.4

Users who are using a version of CUBRID 2008 R4.1, R4.3 or R4.4 should install 11.4 in the different directory, migrate databases to 11.4 and modify parameter values in the existing environment configuration file.

DB migration

Please refer DB migration for migration steps.

(*): The user which uses CUBRID 2008 R4.x or before should be cautious for determining a locale(language and charset). For example, when the user which used the language as ko_KR(Korean) and the charset as utf8 processes DB migration, the locale should be set as “cubrid createdb testdb ko_KR.utf8”. If the locale is not built-in locale, you should run make_locale(.sh) command first. For more details, see Locale Setting.

- You should be careful about the change of the space for storing about the multibyte character. For example, in 2008 R4.3, **CHAR(6)** means **CHAR** type with 6 bytes size, but from 9.3, **CHAR(6)**

means **CHAR** type with 6 characters. In utf8 charset, Korean uses 3 bytes per 1 character, so **CHAR(6)** has 18 bytes. Therefore, more disk space is required.

- If you used utf8 charset in CUBRID 2008 R4.x or before, you should set the charset as utf8 when you run "cubrid createdb". If not, retrieval queries or string functions are unable to work properly.

Parameter configuration

cubrid.conf

- The minimum size of **log_buffer_size** is changed from 48KB(3*1page, 16KB=1page) into 2MB(128*1page, 16KB=1page); therefore, this value should be larger than the changed minimum size.
- The value of **sort_buffer_size** should be configured as 2G or less since the maximum value of **sort_buffer_size** is 2G.
- **single_byte_compare** should be deleted since this parameter is no longer used.
- **intl_mbs_support** should be deleted since this parameter is no longer used.
- **lock_timeout_message_type** should be deleted since this parameter is no longer used.
- In the following parameters, the old parameters will be deprecated and the new parameters are recommended to use. the value in the parenthesis is the unit of the value when the unit is omitted, and the new parameters can specify the unit after the value. For details, see each parameter's explanation in System Parameters

Old parameters(unit)	New parameters(unit)
lock_timeout_in_secs(sec)	lock_timeout(msec)
checkpoint_every_npages(page_count)	checkpoint_every_size(byte)
checkpoint_interval_in_mins(min)	checkpoint_interval(msec)
max_flush_pages_per_second(page_count)	max_flush_size_per_second(byte)
sync_on_nflush(page_count)	sync_on_flush_size(byte)

Old parameters(unit)	New parameters(unit)
sql_trace_slow_msecs(msec)	sql_trace_slow(msecs)

cubrid_broker.conf

- In **KEEP_CONNECTION** parameter, **OFF** value should be changed as **ON** or **AUTO** since **OFF** setting value is no longer used.
- **SELECT_AUTO_COMMIT** should be deleted since this parameter is no longer used.
- The value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** should be 2,097,151 or less since the maximum value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is 2,097,151.

Environment variable

- **CUBRID_LANG** is removed; now the language and the charset of database is set when creating DB, and **CUBRID_MSG_LANG** is used for configuring the charset of messages for utilities and errors.

> **Warning**
>
> When you create database, the language and the charset of database should be specified. It affects the length of string type, string comparison operation, etc. The specified charset when creating database cannot be changed later, so you should be careful when specifying it.
>
> For charset, locale and collation setting, see An Overview of Globalization.

Upgrading From CUBRID 2008 R4.0 or Earlier Versions To CUBRID 11.4

Users who are using versions CUBRID 2008 R4.0 or earlier should install 11.4 in the different directory, migrate databases to 11.4 and modify parameter values in the existing environment configuration file.

DB migration

Do the same procedures with DB migration. If you use GLO classes, you must modify applications and schema in order to use **BLOB** or **CLOB** types, since GLO classes are not supported in 2008 R3.1. If this modification is not easy, it is not recommended to perform the migration.

Parameter configuration

cubrid.conf

- The minimum size of **log_buffer_size** is changed from 48KB(3*1page, 16KB=1page) into 2MB(128*1page, 16KB=1page); therefore, this value should be larger than the changed minimum size.
- The value of **sort_buffer_size** should be configured as 2G or less since the maximum valur of **sort_buffer_size** is 2G.
- **single_byte_compare** should be deleted since this parameter is no longer used.
- **intl_mbs_support** should be deleted since this parameter is no longer used.
- **lock_timeout_message_type** should be deleted since this parameter is no longer used.
- Because the default value of **thread_stacksize** has been changed from 100K to 1M, it is recommended that users who have not configured this value check memory usage of CUBRID-associative processes.
- Because the minimum value of **data_buffer_size** has been changed from 64K to 16M, users who have configured this value less than 16M must change the value equal to or greater than 16M.
- In the following parameters, the old parameters will be deprecated and the new parameters are recommended to use. the value in the parenthesis is the unit of the value when the unit is omitted, and the new parameters can specify the unit after the value. For details, see each parameter's explanation in System Parameters

Old parameters(unit)	New parameters(unit)
lock_timeout_in_secs(sec)	lock_timeout(msec)
checkpoint_every_npages(page_count)	checkpoint_every_size(byte)
checkpoint_interval_in_mins(min)	checkpoint_interval(msec)
max_flush_pages_per_second(page_count)	max_flush_size_per_second(byte)
sync_on_nflush(page_count)	sync_on_flush_size(byte)

cubrid_broker.conf

- In **KEEP_CONNECTION** parameter, **OFF** value should be changed as **ON** or **AUTO** since **OFF** setting value is no longer used.
- **SELECT_AUTO_COMMIT** should be deleted since this parameter is no longer used.
- The value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** should be 2,097,151 or less since the maximum value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is 2,097,151.
- The minimum value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is 1024M. It is recommended that users who configure **APPL_SERVER_MAX_SIZE** configure this value less than the value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT**.
- Because the default value of **CCI_DEFAULT_AUTOCOMMIT** has been changed to **ON**, users who have not configured this value should change it to **OFF** if they want to keep autocommit mode.

cubrid_ha.conf

- Users who have configured the **ha_apply_max_mem_size** parameter value more than 500 must the value to 500 or less.

Environment variable

- **CUBRID_LANG** is removed; now the language and the charset of database is set when creating DB, and **CUBRID_MSG_LANG** is used for configuring the charset of messages for utilities and errors.

> **Warning**
>
> When you create database, the language and the charset of database should be specified. It affects the length of string type, string comparison operation, etc. The specified charset when creating database cannot be changed later, so you should be careful when specifying it.
>
> For charset, locale and collation setting, see An Overview of Globalization.

Database Migration under HA Environment

HA migration from CUBRID 2008 R2.2 or higher to CUBRID 11.4

In the scenario described below, the current service is stopped to perform an upgrade in an environment in which a broker, a master DB and a slave DB are operating on different servers.

Step	Description
Steps C1-C6: Perform DB migration	Run the CUBRID upgrade and database migration in the master node, and back up the new version's database on the master node.
Step C7: Install new version in the slave node	Delete the previous version of the database from the slave node and install a new version. For more information, see Installing and Running CUBRID.
Step C8: Restore the backup copy of the master node in the slave node	Restore the new database backup copy (testdb_bk*) of the master node, which is created in step H6 , to the slave node. % scp user1@master:$CUBRID/databases/databases.txt $CUBRID/databases/. % cd ~/DB/testdb % scp user1@master:~/DB/testdb/testdb_bk0v000 . % scp user1@master:~/DB/testdb/testdb_bkvinf . % cubrid restoredb testdb
Step C9: Reconfigure HA environment and start HA mode	In the master node and the slave node, set the CUBRID environment configuration file (cubrid.conf) and the HA environment configuration file (cubrid_ha.conf) See Creating Databases and Configuring Servers.

Step	Description
Step C10: Install new version in the broker server, and start the broker	For more information about installation, see Installing and Running CUBRID. Start the broker in the Broker server. See Configuring and Starting Broker, and Verifying the Broker Status. % cubrid broker start

HA Migration from CUBRID 2008 R2.0/R2.1 to CUBRID 11.4

If you are using the HA feature of CUBRID 2008 R2.0 or 2008 R2.1, you must upgrade the server version, migrate the database, set up a new HA environment, and then change the Linux Heartbeat auto start setting used in 2008 R2.0 or 2008 R2.1. If the Linux Heartbeat package is not needed, delete it.

Perform steps C1~C10 above, then perform step C11 below:

Step	Description
Step C11: Change the previous Linux heartbeat auto start settings	Perform the following task in the master and slave nodes from a root account. [root@master ~]# chkconfig --del heartbeat // Performing the same job in the slave node

Uninstalling CUBRID

Uninstalling CUBRID in Linux

Uninstalling CUBRID of SH package or compressed package

To uninstall SH package(file extension of installing package is .sh) or compressed package(file extension of installing package is .tar.gz), proceed the following steps.

- Remove databases after stopping CUBRID service.

Remove all created databases after stopping CUBRID service.

For example, if there are testdb1, testdb2 as databases, do the following commands.

```
$ cubrid service stop
$ cubrid deletedb testdb1
$ cubrid deletedb testdb2
```

- Remove CUBRID engine.

  Remove $CUBRID and its subdirectories.

```
$ echo $CUBRID
    /home1/user1/CUBRID

$ rm -rf /home1/user1/CUBRID
```

- Remove auto-starting script.

  If you stored cubrid script in /etc/init.d directory, remove this file.

```
cd /etc/init.d
rm cubrid
```

Uninstalling CUBRID of RPM package

If you have installed CUBRID in RPM package, you can uninstall CUBRID by “rpm” command.

- Remove databases after stopping CUBRID service.

  Remove all created databases after stopping CUBRID service.

  For example, if there are testdb1, testdb2 as databases, do the following commands.

```
$ cubrid service stop
$ cubrid deletedb testdb1
$ cubrid deletedb testdb2
```

- Remove CUBRID engine.

```
$ rpm -q cubrid
cubrid-9.2.0.0123-el5.x86_64

$ rpm -e cubrid
```

```
warning: /opt/cubrid/conf/cubrid.conf saved as /opt/cubrid/conf/
cubrid.conf.rpmsave
```

- Remove auto-starting script.

  If you stored cubrid script in /etc/init.d directory, remove this file.

```
cd /etc/init.d
rm cubrid
```

Uninstalling CUBRID in Windows

Choose “CUBRID” in “Control panel > Uninstall a program” and uninstall it.

Getting Started

This chapter contains useful information on starting CUBRID; also it provides brief instructions on how to use the CSQL Interpreter and GUI tools.

CUBRID provides various tools for you to use easily through GUI. You can download them from https://www.cubrid.org/downloads.

If you use the following tools which CUBRID provides, you can use CUBRID features easily through GUI.

- CUBRID Manager
- CUBRID Migration Toolkit

CUBRID also provides various drivers such as JDBC, CCI, PHP, PDO, ODBC, OLE DB, ADO.NET, Perl, Python, and Ruby (see API Reference).

Starting the CUBRID Service

Configure environment variables and language, and then start the CUBRID service. For more information on configuring environment variables and language, see CUBRID Services.

Shell Command

The following shell command can be used to start the CUBRID service and the *demodb* included in the installation package.

```
% cubrid service start

@ cubrid master start
++ cubrid master start: success

@ cubrid broker start
++ cubrid broker start: success

@ cubrid manager server start
++ cubrid manager server start: success

% cubrid server start demodb

@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success

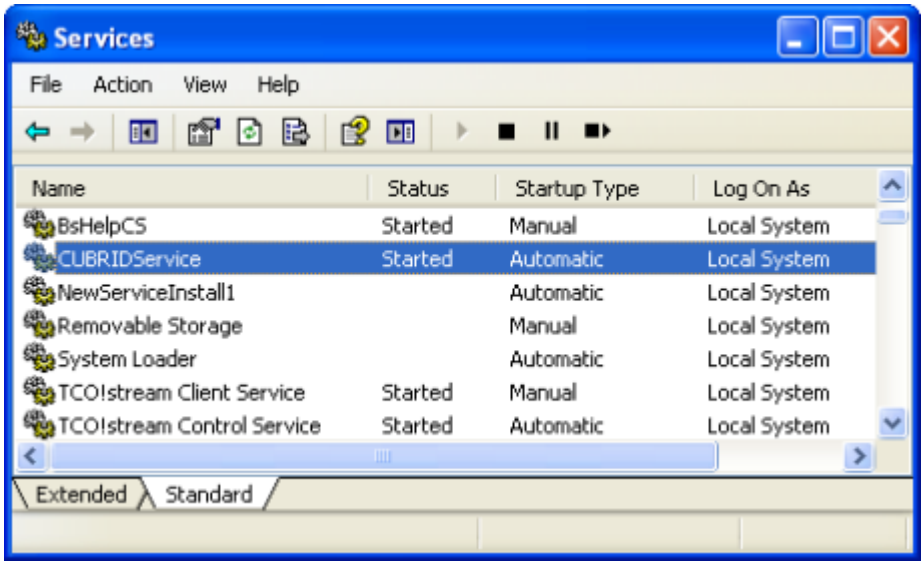
@ cubrid server status

Server demodb (rel 11.4, pid 31322)
```

CUBRIDService or CUBRID Service Tray

On the Windows environment, you can start or stop a service as follows:

- Go to [Control Panel] > [Performance and Maintenance] > [Administrator Tools] > [Services] and select the CUBRIDService to start or stop the service.



- In the system tray, right-click the CUBRID Service Tray. To start CUBRID, select [Service Start]; to stop it, select [Service Stop].

  Selecting [Service Start] or [Service Stop] menu would be like executing cubrid service start or cubrid service stop in a command prompt; this command runs or stops the processes configured in service parameters of cubrid.conf.

- If you click [Exit] while CUBRID is running, all the services and process in the server stop.

Note

An administrator level (SYSTEM) authorization is required to start/stop CUBRID processes through the CUBRID Service tray; a login level user authorization is required to start/stop them with shell commands. If you cannot control the CUBRID processes on the Windows Vista or later version environment, select [Execute as an administrator (A)] in the [Start] > [All Programs] > [Accessories] > [Command Prompt]) or execute it by using the CUBRID Service Tray. When all processes of CUBRID Server stops, an icon on the CUBRID Service tray turns out gray.

Creating Databases

You can create databases by using the **cubrid createdb** utility and execute it where database volumes and log volumes are located. If you do not specify additional options such as **--db-volume-size** or **--log-volume-size**, 1.5 GB volume files are created by default (data volume is set to 512 MB, active log is set to 512 MB, and background archive log is set to 512 MB).

```
% cd testdb
% cubrid createdb testdb en_US
% ls -l

-rw------- 1 cubrid dbms 536870912 Jan 11 15:04 testdb
-rw------- 1 cubrid dbms 536870912 Jan 11 15:04 testdb_lgar_t
-rw------- 1 cubrid dbms 536870912 Jan 11 15:04 testdb_lgat
-rw------- 1 cubrid dbms       176 Jan 11 15:04 testdb_lginf
-rw------- 1 cubrid dbms       183 Jan 11 15:04 testdb_vinf
```

In the above, *testdb* represents a data volume file, testdb_lgar_t represents a background archive log file, testdb_lgat represents an active log file, testdb_lginf represents a log information file, and testdb_vinf represents a volume information file.

For details on volumes, see Database Volume Structure . For details on creating volumes, see createdb. It is recommended to classify and add volumes based on purpose by using the **cubrid addvoldb** utility. For details, see addvoldb.

Starting Database

You can start a database process by using the **cubrid server** utility.

```
% cubrid server start testdb
```

To have *testdb* started upon startup of the CUBRID service (cubrid service start), configure *testdb* in the **server** parameter of the **cubrid.conf** file.

```
% vi cubrid.conf

[service]

service=server,broker,manager
server=testdb

...
```

Query Tools

CSQL Interpreter

The CSQL Interpreter is a program used to execute the SQL statements and retrieve results in a way that CUBRID supports. The entered SQL statements and the results can be stored as a file. For more information, see Introduction to the CSQL Interpreter and CSQL Execution Mode.

CUBRID also provides a GUI-based program called "CUBRID Manager". You can execute all SQL statements and retrieve results in the query editor of CUBRID Manager. For more information, see Management Tools.

This section describes how to use the CSQL Interpreter on the Linux environment

Starting the CSQL Interpreter

You can start the CSQL program in the shell as shown below. At the initial installation, **PUBLIC** and **DBA** users are provided and the passwords of the users not set. If no user is specified while the CSQL Interpreter is executed, **PUBLIC** is used for login.

```
% csql demodb

CUBRID SQL Interpreter

Type `;help' for help messages.
```

```
csql> ;help

=== <Help: Session Command Summary> ===


   All session commands should be prefixed by `;' and only blanks/
tabs
   can precede the prefix. Capitalized characters represent the
minimum
   abbreviation that you need to enter to execute the specified
command.

   ;REAd   [<file-name>]       - read a file into command buffer.
   ;Write  [<file-name>]       - (over)write command buffer into a
file.
   ;APpend [<file-name>]       - append command buffer into a file.
   ;PRINT                      - print command buffer.
   ;SHELL                      - invoke shell.
   ;CD                         - change current working directory.
   ;EXit                       - exit program.

   ;CLear                      - clear command buffer.
   ;EDIT   [format/fmt]        - invoke system editor [after
formatter] with command buffer.
   ;LISt                       - display the content of command
buffer.

   ;RUn                        - execute sql in command buffer.
   ;Xrun                       - execute sql in command buffer,
                                 and clear the command buffer.
   ;COMmit                     - commit the current transaction.
   ;ROllback                   - roll back the current transaction.
   ;AUtocommit [ON|OFF]        - enable/disable auto commit mode.
   ;REStart                    - reconnect to the current database
in a CSQL session.
   ;CHeckpoint                 - execute the checkpoint(CSQL with --
sysadm only).
   ;Killtran                   - check transaction status
information or end a specific transaction.(CSQL with --sysadm only).

   ;SHELL_Cmd  [shell-cmd]     - set default shell, editor, print,
pager and formatter
   ;EDITOR_Cmd [editor-cmd]       command to new one, or display the
current
   ;PRINT_Cmd  [print-cmd]        one, respectively.
   ;PAger_cmd  [pager-cmd]
   ;FOrmatter_cmd [formatter-cmd]

   ;DATE                       - display the local time, date.
   ;DATAbase                   - display the name of database being
accessed.
   ;SChema class-name          - display schema information of a
```

```
class.
   ;TRIgger [`*'|trigger-name] - display trigger definition.
   ;Get system_parameter       - get the value of a system parameter.
   ;SET system_parameter=value - set the value of a system parameter.
   ;STring-width [width]       - set width that each column which is
a string type is displayed.
   ;COLumn-width [name]=[width]- set width that a specific column is
displayed.
   ;PLan [simple/detail/off]   - show query execution plan.
   ;Info <command>             - display internal information.
   ;TIme [ON/OFF]              - enable/disable to display the query
                                 execution time.
   ;LINe-output [ON/OFF]       - enable/disable to display each
value in a line
   ;HISTORYList                - display list of the executed
queries.
   ;HISTORYRead <history_num>  - read entry on the history number
into command buffer.
   ;TRAce [ON/OFF] [text/json] - enable/disable sql auto trace.
   ;SIngleline [ON|OFF]        - enable/disable single-line mode.
   ;CONnect username [dbname | dbname@hostname]
                               - connect to the current or other
databases as a username.
   ;.Hist [ON/OFF]             - start/stop collecting statistics
information in CSQL(available DBA only).
   ;.Clear_hist                - clear the CSQL statistics
information in the buffer.
   ;.Dump_hist                 - display the CSQL statistics
information in CSQL.
   ;.X_hist                    - display the CSQL statistics
information in CSQL with statistics data initialized.
   ;HElp                       - display this help message.
```

Running SQL with CSQL

After the CSQL has been executed, you can enter the SQL into the CSQL prompt. Each SQL statement must end with a semicolon (;). Multiple SQL statements can be entered in a single line. You can find the simple usage of the session commands with the ;help command. For more information, see Session Commands.

```
% csql demodb

csql> SELECT SUM(n) FROM (SELECT gold FROM participant WHERE
nation_code='KOR'
csql> UNION ALL SELECT silver FROM participant WHERE
nation_code='JPN') AS t(n);

=== <Result of SELECT Command in Line 2> ===

       sum(n)
```

```
=============
           82

1 rows selected. (0.106504 sec) Committed.

csql> ;exit
```

Management Tools

	Summary of features	Downloads of the recent files
CUBRID Manager	Java client tool for SQL execution & DB operation. 1. Java-based management tool (JRE 1.6 or higher required) 2. Auto upgrade after the initial download 3. Useful to manage multiple hosts 4. DB access via CUBRID Manager server	CUBRID Manager Download
CUBRID Migration Toolkit	Java-based client tool to migrate schema and data from source DB (MySQL, Oracle, CUBRID) to CUBRID. 1. Java-based management tool (JRE 1.6 or higher required) 2. Auto upgrade after the initial download 3. Available migration only for multiple queries	CUBRID Migration Toolkit Download

	Summary of features	Downloads of the recent files
	results, the reuse of migration scenario; good to batch job 4. Direct DB access with JDBC	

Running SQL with CUBRID Manager

CUBRID Manager is the client tool that you should download and run. It is a Java application which requires JRE or JDK 1.6 or higher.

1. Download and install the latest CUBRID Manager file. CUBRID Manager is compatible with CUBRID DB engine 2008 R2.2 or higher version. It is recommended to upgrade to the latest version periodically; it supports the auto-update feature. (CUBRID FTP: http://ftp.cubrid.org/CUBRID_Tools/CUBRID_Manager)

2. Start CUBRID service on the server. CUBRID Manager server should be started for CUBRID Manager client to access to DB. For more information, see CUBRID Manager Server.

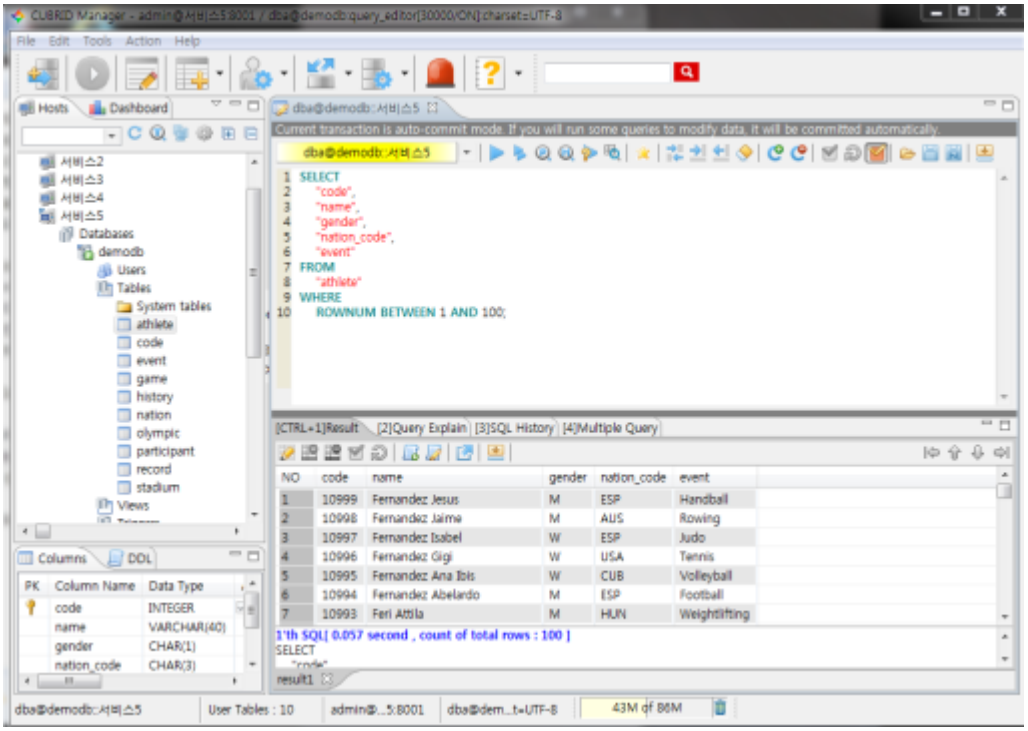
```
C:\CUBRID>cubrid service start
++ cubrid service is running.
```

3. After the installation of CUBRID Manager, register host information on the [File > Add Host] menu. To register the host, you should enter host address, connection port (default: 8001), and CUBRID Manager username/password and install the JDBC driver of the same version with DB engine (supporting auto-driver-search/auto-update).

4. Choose the host on the left tress and perform the CUBRID Manager user (=host user) authentication. The default ID/password is admin/admin.

5. Create a new database as clicking the right mouse button on the database node, or try to connect as choosing the existing database on the bottom of the host node. At this time, do the DB user's login. The default db user is "dba", and there is no password.

6. Run SQL on the access DB and confirm the result. The host, DB and table list are displayed on the left side, and the query editor and the result window is shown on the right side. You can reuse the SQLs which have been succeeded with [SQL History] tab and compare the multiple results of several DBs as adding the DBs for the comparison of the result with [Multiple Query] tab.

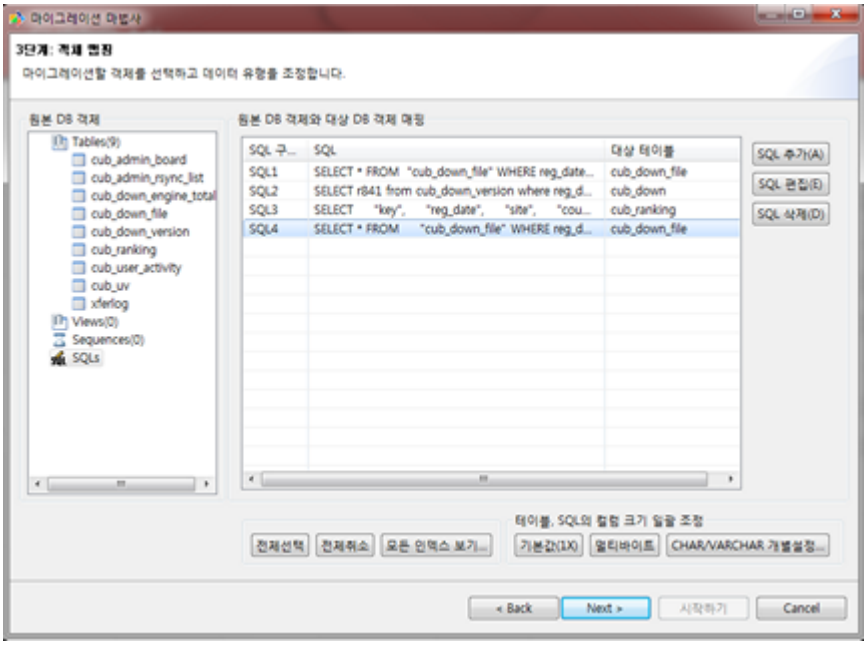


Migrating schema/data with CUBRID Migration Toolkit

CUBRID Migration Toolkit is a tool to migrate the data and the schema from the source DB (MySQL, Oracle, and CUBRID) to the target DB (CUBRID). It is also Java applications which require JRE or JDK 1.6 or later. You should download separately. (CUBRID FTP: http://ftp.cubrid.org/CUBRID_Tools/CUBRID_Migration_Toolkit)

It is useful in case of switching from other DB into CUBRID, in case of migrating into other hardware, in case of migrating some schema and data from the operating DB, in case of upgrading CUBRID version, and in case of running the batch jobs. The main features are as follows:

- Supports the tools/some schema and data migration
- Available to migrate only the desired data as running several SQLs
- Executable without suspending of operation as supporting online migration through JDBC
- Available offline migration with CSV, SQL, CUBRID loaddb format data
- Available to run directly on the target server as extracting the run-script of migration
- Shorten the batch job time as reusing the migration run-script.



Drivers

The drivers supported by CUBRID are as follows:

- CUBRID JDBC driver (Downloads JDBC)
- CUBRID CCI driver (Downloads CCI)
- CUBRID PHP driver (Downloads PHP)
- CUBRID PDO driver (Downloads PDO)
- CUBRID ODBC driver (Downloads ODBC)
- CUBRID ADO.NET driver (Downloads ADO.NET)
- CUBRID Perl driver (Downloads Perl)
- CUBRID Python driver (Downloads Python)
- CUBRID Ruby driver (Downloads Ruby)
- CUBRID Node.js driver (Downloads Node.js)

Among above drivers, JDBC and CCI drivers are automatically downloaded while CUBRID is being installed. Thus, you do not have to download them manually.

CSQL Interpreter

To execute SQL statements in CUBRID, you need to use either a Graphical User Interface(GUI)-based CUBRID Manager or a console-based CSQL Interpreter.

CSQL is an application that allows users to use SQL statements through a command-driven interface. This section briefly explains how to use the CSQL Interpreter and associated commands.

Introduction to the CSQL Interpreter

A Tool for SQL

The CSQL Interpreter is an application installed with CUBRID that allows you to execute in an interactive or batch mode and viewing query results. The CSQL Interpreter has a command-line interface. With this, you can store SQL statements together with their results to a file for a later use.

The CSQL Interpreter provides the best and easiest way to use CUBRID. You can develop database applications with various APIs (e.g. JDBC, ODBC, PHP, CCI, etc.; you can use the CUBRID Manager, which is a management and query tool provided by CUBRID. With the CSQL Interpreter, users can create and retrieve data in a terminal-based environment.

The CSQL Interpreter directly connects to a CUBRID database and executes various tasks using SQL statements. Using the CSQL Interpreter, you can:

- Retrieve, update and delete data in a database by using SQL statements
- Execute external shell commands
- Store or display query results
- Create and execute SQL script files
- Select table schema
- Retrieve or modify parameters of the database server system
- Retrieve database information (e.g. schema, triggers, queued triggers, workspaces, locks, and statistics)

A Tool for DBA

A database administrator(**DBA**) performs administrative tasks by using various administrative utilities provided by CUBRID; a terminal-based interface of CSQL Interpreter is an environment where **DBA** executes administrative tasks.

It is also possible to run the CSQL Interpreter in standalone mode. In this mode, the CSQL Interpreter directly accesses database files and executes commands including server process properties. That is, SQL statements can be executed to a database without running a separate database server process. The CSQL Interpreter is a powerful tool that allows you to use the database only with a **csql** utility, without any other applications such as the database server or the brokers.

Executing CSQL

CSQL Execution Mode

Interactive Mode

With CSQL Interpreter, you can enter and execute SQL statements to handle schema and data in the database. Enter statements in a prompt that appears when running the **csql** utility. After executing the statements, the results are listed in the next line. This is called the interactive mode.

Batch Mode

You can store SQL statements in a file and execute them later to have the **csql** utility read the file. This is called the batch mode..

Standalone Mode

In the standalone mode, CSQL Interpreter directly accesses database files and executes commands including server process functions. That is, SQL statements can be sent and executed to a database without a separate database server process running for the task. Since the standalone mode allows only one user access at a given time, it is suitable for management tasks by Database Administrators (**DBAs**).

Client/Server Mode

CSQL Interpreter usually operates as a client process and accesses the server process.

System Administration Mode

You can use this mode when you run checkpoint through CSQL interpreter or exit the transaction monitoring. Also, it allows one connection on CSQL interpreter even if the server access count exceeds the value of **max_clients** system parameter. In this mode, allowed connection count by CSQL interpreter is only one.

```
csql -u dba --sysadm demodb
```

Using CSQL (Syntax)

Connecting to Local Host

Execute the CSQL Interpreter using the **csql** utility. You can set options as needed. To set the options, specify the name of the database to connect to as a parameter. The following is a **csql** utility statement to access the database on a local server:

```
csql [options] database_name
```

Connecting to Remote Host

The following is a **csql** utility syntax to access the database on a remote host:

```
csql [options] database_name@remote_host_name
```

Make sure that the following conditions are met before you run the CSQL Interpreter on a remote host.

- The CUBRID installed on the remote host must be the same version as the one on the local host.
- The port number used by the master process on the remote host must be identical to the one on the local host.
- You must access the remote host in client/server mode using the **-C** option.

Example

The following example shows how to access the **demodb** database on the remote host with the IP address 192.168.1.3 and calls the **csql** utility.

```
csql -C demodb@192.168.1.3
```

CSQL Options

To display the option list in the prompt, execute the **csql** utilities without specifying the database name as follows:

```
$ csql
A database-name is missing.
interactive SQL utility, version 11.4
usage: csql [OPTION] database-name[@host]

valid options:
```

```
  -S, --SA-mode                standalone mode execution
  -C, --CS-mode                client-server mode execution
  -u, --user=ARG               alternate user name
  -p, --password=ARG           password string, give "" for none
  -e, --error-continue         don't exit on statement error
  -i, --input-file=ARG         input-file-name
  -o, --output-file=ARG        output-file-name
  -s, --single-line            single line oriented execution
  -c, --command=ARG            CSQL-commands
  -l, --line-output            display each value in a line
  -r, --read-only              read-only mode
  -t, --plain-output           display results in a script-friendly
format (only works with -c and -i)
  -q, --query-output           display results in a query-friendly
format (only work with -c and -i)
  -d, --loaddb-output          display results in a loaddb-friendly
format (only work with -c and -i)
  -N, --skip-column-names      do not display column names in
results (only works with -c and -i)
      --string-width           display each column which is a string
type in this width
      --no-auto-commit         disable auto commit mode execution
      --no-pager               do not use pager
      --no-single-line         turn off single line oriented
execution
      --no-trigger-action      disable trigger action
      --delimiter=ARG          delimiter between columns (only work
with -q)
      --enclosure=ARG          enclosure for a result string (only
work with -q)

For additional information, see http://www.cubrid.org
```

Options

-S, --SA-mode

The following example shows how to connect to a database in standalone mode and execute the **csql** utility. If you want to use the database exclusively, use the **-S** option. If **csql** is running in standalone mode, it is impossible to use another **csql** or utility. If both **-S** and **-C** options are omitted, the **-C** option will be specified.

```
csql -S demodb
```

-C, --CS-mode

The following example shows how to connect to a database in client/server mode and execute the **csql** utility. In an environment where multiple clients connect to the database, use the **-C** option. Even when you connect to a database on a remote host in client/server

mode, the error log created during **csql** execution will be stored in the **cub.err** file on the local host.

```
csql -C demodb
```

-i, --input-file=ARG

The following example shows how to specify the name of the input file that will be used in a batch mode with the **-i** option. In the **infile** file, more than one SQL statement is stored. Without the **-i** option specified, the CSQL Interpreter will run in an interactive mode.

```
csql -i infile demodb
```

-o, --output-file=ARG

The following example shows how to store the execution results to the specified file instead of displaying on the screen. It is useful to retrieve the results of the query performed by the CSQL Interpreter afterwards.

```
csql -o outfile demodb
```

-u, --user=ARG

The following example shows how to specify the name of the user that will connect to the specified database with the **-u** option. If the **-u** option is not specified, **PUBLIC** that has the lowest level of authorization will be specified as a user. If the username is not valid, an error message is displayed and the **csql** utility is terminated. If there is a password for the username you specify, you will be prompted to enter the password.

```
csql -u DBA demodb
```

-p, --password=ARG

The following example shows how to enter the password of the user specified with the **-p** option. Especially since there is no prompt to enter a password for the user you specify in a batch mode, you must enter the password using the **-p** option. When you enter an incorrect password, an error message is displayed and the **csql** utility is terminated.

```
csql -u DBA -p *** demodb
```

-s, --single-line

As an option used with the **-i** option, it executes multiple SQL statement one by one in a file with the **-s** option. This option is useful to allocate less memory for query execution and each SQL statement is separated by semicolons (;). If it is not specified, multiple SQL statements are retrieved and executed at once.

```
csql -s -i infile demodb
```

-c, --command=ARG

The following example shows how to execute more than one SQL statement from the shell with the **-c** option. Multiple statements are separated by semicolons (;).

```
csql -c "select * from olympic;select * from stadium" demodb
```

-l, --line-output

With **-l** option, you can display the values of SELECT lists by line. If **-l** option is omitted, all SELECT lists of the result record are displayed in one line.

```
csql -l demodb
```

-e, --error-continue

The following example shows how to ignore errors and keep execution even though semantic or runtime errors occur with the **-e** option. However, if any SQL statements have syntax errors, query execution stops after errors occur despite specifying the **-e** option.

```
$ csql -e demodb

csql> SELECT * FROM aaa;SELECT * FROM athlete WHERE code=10000;

In line 1, column 1,

ERROR: before ' ;SELECT * FROM athlete WHERE code=10000; '
Unknown class "aaa".


=== <Result of SELECT Command in Line 1> ===

         code  name                  gender
nation_code           event
============================================================================
====================================
        10000  'Aardewijn Pepijn'    'M'
'NED'                 'Rowing'


1 rows selected. (0.006433 sec) Committed.
```

-r, --read-only

You can connect to the read-only database with the **-r** option. Retrieving data is only allowed in the read-only database; creating databases and entering data are not allowed.

```
csql -r demodb
```

-t, --plain-output

It only shows column names and values and works with **-c** or **-i** option. Each column and value is separated by a tab and a new line, a tab and a backslash which are included in results are replaced by 'n', 't' and '\' for each. This option is ignored when it is given with **-l** option.

```
$ csql demodb -c "select * from athlete where code between 12762
and 12765" -t

code    name    gender  nation_code event
12762   O'Brien Dan M   USA Athletics
12763   O'Brien Leah    W   USA Softball
12764   O'Brien Shaun William   M   AUS Cycling
12765   O'Brien-Amico Leah  W   USA Softball
```

-q, --query-output

This option displays the result for easy use in insert queries, only show column names and values, and works with **-c** or **-i** option. Each column name and value are separated by a comma or a single character of the **--delimiter** option; and all results except for numeric types are enclosed by a single quote or a single character of the **--enclosure** option. If the enclosure is a single quote, the single quote in the results is replaced with two ones. It is ignored when it is given with **-l** option.

```
$ csql demodb -c "select * from athlete where code between 12762
and 12765" -q

code,name,gender,nation_code,event
12762,'O''Brien Dan','M','USA','Athletics'
12763,'O''Brien Leah','W','USA','Softball'
12764,'O''Brien Shaun William','M','AUS','Cycling'
12765,'O''Brien-Amico Leah','W','USA','Softball'
```

```
$ csql demodb -c "select * from athlete where code between 12762
and 12765" -q --delimiter="" --enclosure="\""

code,name,gender,nation_code,event
12762,"O'Brien Dan","M","USA","Athletics"
12763,"O'Brien Leah","W","USA","Softball"
12764,"O'Brien Shaun William","M","AUS","Cycling"
12765,"O'Brien-Amico Leah","W","USA","Softball"
```

-d, --loaddb-output

This option displays the result for easy use in loaddb utility, only show column names and values, and works with **-c** or **-i** option. Each column name and value are separated by a space; and all results except for numeric types are enclosed by a single quote. The single quote in the results is replaced with two ones and the result of the enum type is outputted by an index instead of a value. This opton is ignored when it is given with **-l** option.

```
$ csql demodb -c "select * from athlete where code between 12762
and 12765" -d

%class [ ] ([code] [name] [gender] [nation_code] [event])
12762 'O''Brien Dan' 'M' 'USA' 'Athletics'
12763 'O''Brien Leah' 'W' 'USA' 'Softball'
12764 'O''Brien Shaun William' 'M' 'AUS' 'Cycling'
12765 'O''Brien-Amico Leah' 'W' 'USA' 'Softball'
```

-N, --skip-column-names

It will hide column names from the results. It only works with **-c** or **-i** option and is usually used with **-t -q** or **-d** option. This option is ignored when it is given with **-l** option.

```
$ csql demodb -c "select * from athlete where code between 12762
and 12765" -d -N

12762 'O''Brien Dan' 'M' 'USA' 'Athletics'
12763 'O''Brien Leah' 'W' 'USA' 'Softball'
12764 'O''Brien Shaun William' 'M' 'AUS' 'Cycling'
12765 'O''Brien-Amico Leah' 'W' 'USA' 'Softball'
```

--no-auto-commit

The following example shows how to stop the autocommit mode with the **--no-auto-commit** option. If you do not configure **--no-auto-commit** option, the CSQL Interpreter runs in an autocommit mode by default, and the SQL statement is committed automatically at every execution. Executing the **;AUtocommit** session command after starting the CSQL Interpreter will also have the same result.

```
csql --no-auto-commit demodb
```

--no-pager

The following example shows how to display the execution results by the CSQL Interpreter at once instead of page-by-page with the **--no-pager** option. The results will be output page-by-page if **--no-pager** option is not specified.

```
csql --no-pager demodb
```

--no-single-line

The following example shows how to keep storing multiple SQL statements and execute them at once with the **;xr** or **;ru** session command. If you do not specify this option, SQL statements are executed without **;xr** or **;ru** session command. ;Singleline command will also have the same result.

```
csql --no-single-line demodb
```

--sysadm

This option should be used together with **-u dba**. It is specified when you want to run CSQL in a system administrator's mode.

```
csql -u dba --sysadm demodb
```

--write-on-standby

This option should be used together with a system administrator's mode option(**--sysadm**). dba which run CSQL with this option can write directly to the standby DB (slave DB or replica DB). However, the data to be written directly to the replica DB are not replicated.

```
csql --sysadm --write-on-standby -u dba testdb@localhost
```

Note

Please note that replication mismatch occurs when you write the data directly to the replica DB.

--no-trigger-action

If you specify this option, triggers of the queries executed in this CSQL are not triggered.

--delimiter=ARG

This option should be used together with **-q** and a single character is specified in the argument to separate the column name and value. If multiple characters are specified, the first character is used without displaying an error. (include special characters such as \t and \n)

--enclosure=ARG

This option should be used together with **-q** and a single character is specified in the argument to enclose all values except for numeric types. If multiple characters are specified, the first character is used without displaying an error.

Session Commands

In addition to SQL statements, CSQL Interpreter provides special commands allowing you to control the Interpreter. These commands are called session commands. All the session commands must start with a semicolon (;).

Enter the **;help** command to display a list of the session commands available in the CSQL Interpreter. You don't need to type the full session command; the CSQL Interpreter recognizes commands as long as you input the minimum number of characters needed to uniquely identify the command from others. Session commands are not case-sensitive.

CSQL recognizes SQL string literals, comments, and identifiers (see: lexical rules). Specifically:

- String literals enclosed in single quotes
- Comments marked by –, //, or /* */
- Identifiers enclosed in double quotes, brackets, or backticks

Strings within these are recognized only as strings and lose any special meaning they might have outside, and this applies to session commands as well. For example, ';exit' within a string literal, comment, or identifier below is not recognized as a session command but only as a string.

```
csql> select * from table_a where col = ';exit';
...
csql> select * from table_b /* ;exit on no results */;
...
csql> create table table_c([;exit] int);
...
```

Except in interactive mode CSQL, if the first character of a command line is a semicolon, it is recognized as a session command even if it is within a string literal, comment, or identifier.

```
csql> select * from table_a where col = '
csql> ;exit    <-- CSQL exit
...
csql> select * from table_b /*
csql> ;exit    <-- CSQL exit
...
csql> create table table_c([
csql> ;exit    <-- CSQL exit
...
```

This exception is necessary to allow operations such as exiting CSQL (;exit), continuing to edit in an editor (;edit), or clearing the query buffer (;clear) to be performed at any edit position.

"Query buffer" is a buffer to store the query before running it. If you run CSQL as giving the **--no-single-line** option, the query string is kept on the buffer until running **;xr** command.

Session command reference guide(;HElp) The **;HElp** command displays help information for all session commands and can be used to check the session commands available in csql.

```
csql> ;help

=== <Help: Session Command Summary> ===


All session commands should be prefixed by ';' and only blanks/tabs
can precede the prefix. Capitalized characters represent the minimum
abbreviation that you need to enter to execute the specified command.

;REAd   [<file-name>]        - read a file into command buffer.
;Write  [<file-name>]        - (over)write command buffer into a
file.
;APpend [<file-name>]        - append command buffer into a file.
;PRINT                       - print command buffer.
;SHELL                       - invoke shell.
;CD                          - change current working directory.
;EXit (or Ctrl+d)            - exit program.

;CLear                       - clear command buffer.
;EDIT   [format/fmt]         - invoke system editor [after
formatter] with command buffer.
;LISt                        - display the content of command buffer.

;RUn                         - execute sql in command buffer.
;Xrun                        - execute sql in command buffer,
                              and clear the command buffer.
;COMmit                      - commit the current transaction.
;ROllback                    - roll back the current transaction.
;AUtocommit [ON|OFF]         - enable/disable auto commit mode.
;REStart                     - reconnect to the current database in
a CSQL session.
;CHeckpoint                  - execute the checkpoint(CSQL with --
sysadm only).
;Killtran                    - check transaction status information
or end a specific transaction.(CSQL with --sysadm only).

;SHELL_Cmd     [shell-cmd]   - set default shell, editor, print,
pager and formatter
;EDITOR_Cmd    [editor-cmd]    command to new one, or display the
current
;PRINT_Cmd     [print-cmd]     one, respectively.
;PAger_cmd     [pager-cmd]
;FOrmatter_cmd [formatter-cmd]

;DATE                        - display the local time, date.
```

```
;DATAbase                    - display the name of database being
accessed.
;SChema class-name           - display schema information of a class.
;TRIgger [`*'|trigger-name]  - display trigger definition.
;Get system_parameter        - get the value of a system parameter.
;SET system_parameter=value  - set the value of a system parameter.
;STring-width [width]        - set width that each column which is a
string type is displayed.
;COLumn-width [name]=[width] - set width that a specific column is
displayed.
;PLan [simple/detail/off]    - show query execution plan.
;Info <command>              - display internal information.
;TIme [ON/OFF]               - enable/disable to display the query
execution time.
;SERver-output [ON/OFF]      - enable/disable displaying server
messages stored in DBMS_OUTPUT buffer.
;LINe-output [ON/OFF]        - enable/disable to display each value
in a line
;HISTORYList                 - display list of the executed queries.
;HISTORYRead <history_num>   - read entry on the history number into
command buffer.
;TRAce [ON/OFF] [text/json]  - enable/disable sql auto trace.
;SIngleline [ON|OFF]         - enable/disable single-line mode.
;CONnect username [dbname | dbname@hostname]
                            - connect to the current or other
databases as a username.
;.Hist [ON/OFF]              - start/stop collecting statistics
information in CSQL(available DBA only).
;.Clear_hist                 - clear the CSQL statistics information
in the buffer.
;.Dump_hist                  - display the CSQL statistics
information in CSQL.
;.X_hist                     - display the CSQL statistics
information in CSQL with statistics data initialized.
;HElp                        - display this help message.
```

Reading SQL statements from a file (;READ)

The **;READ** command reads the contents of a file into the buffer. This command is used to execute SQL commands stored in the specified file. To view the contents of the file loaded into the buffer, use the **;List** command.

```
csql> ;read nation.sql
The file has been read into the command buffer.
csql> ;list
insert into "sport_event" ("event_code", "event_name",
"gender_type", "num_player") values
(20001, 'Archery Individual', 'M', 1);
insert into "sport_event" ("event_code", "event_name",
"gender_type", "num_player") values
```

```
20002, 'Archery Individual', 'W', 1);
....
```

Storing SQL statements into a file (;Write)

The **;Write** command stores the contents of the query buffer into a file. This command is used to store queries that you entered or modified in the CSQL Interpreter.

```
csql> ;write outfile
Command buffer has been saved.
```

Appending to a file (;APpend)

This command appends the contents of the current query buffer to an **outfile** file.

```
csql> ;append outfile
Command buffer has been saved.
```

Executing a shell command (;SHELL)

The **;SHELL** session command calls an external shell. Starts a new shell in the environment where the CSQL Interpreter is running. It returns to the CSQL Interpreter when the shell terminates. If the shell command to execute with the **;SHELL_Cmd** command has been specified, it starts the shell, executes the specified command, and returns to the CSQL Interpreter.

```
csql> ;shell
% ls -al
total 2088
drwxr-xr-x 16 DBA cubrid   4096 Jul 29 16:51 .
drwxr-xr-x  6 DBA cubrid   4096 Jul 29 16:17 ..
drwxr-xr-x  2 DBA cubrid   4096 Jul 29 02:49 audit
drwxr-xr-x  2 DBA cubrid   4096 Jul 29 16:17 bin
drwxr-xr-x  2 DBA cubrid   4096 Jul 29 16:17 conf
drwxr-xr-x  4 DBA cubrid   4096 Jul 29 16:14 cubridmanager
% exit
csql>
```

Registering a shell command (;SHELL_Cmd)

The **;SHELL_Cmd** command registers a shell command to execute with the **SHELL** session command. As shown in the example below, enter the **;shell** command to execute the registered command.

```
csql> ;shell_c ls -la
csql> ;shell
```

```
total 2088
drwxr-xr-x 16 DBA cubrid   4096 Jul 29 16:51 .
drwxr-xr-x  6 DBA cubrid   4096 Jul 29 16:17 ..
drwxr-xr-x  2 DBA cubrid   4096 Jul 29 02:49 audit
drwxr-xr-x  2 DBA cubrid   4096 Jul 29 16:17 bin
drwxr-xr-x  2 DBA cubrid   4096 Jul 29 16:17 conf
drwxr-xr-x  4 DBA cubrid   4096 Jul 29 16:14 cubridmanager
csql>
```

The OS environment variable(CUBRID_CSQL_SHELL or SHELL; CUBRID_CSQL_SHELL is used if both environment variables are set) may be registered and used in advance.

```
$ export CUBRID_CSQL_SHELL='ls -la'
or
$ export SHELL='ls -la'
```

Registering a pager command (;PAger_cmd)

The ;PAger_cmd command registers a pager command to display the query result. The way of displaying is decided by the registered command. The default is **more**. Also **cat** and **less** can be used. But ;Pager_cmd command works well only on Linux.

When you register pager command as more, the query result shows by page and wait until you press the space key.

```
csql>;pager more
```

When you register pager command as cat, the query result shows all in one display without paging.

```
csql>;pager cat
```

When you redirect the output with a file, the total query result will be written on the file.

```
csql>;pager cat > output.txt
```

If you register pager command as less, you can forward, backward the query result. Also pattern matching on the query result is possible.

```
csql>;pager less
```

The keyboard commands used on the **less** are as follows.

- Page UP, b: go up to one page. (backwording)
- Page Down, Space: go down to one page (forwarding)
- /string: find a sting on the query results
- n: find the next string
- N: find the previous string
- q: quit the paging mode.

Changing the current working directory (;CD)

This command changes the current working directory where the CSQL Interpreter is running to the specified directory. If you don't specify the path, the directory will be changed to the home directory.

```
csql> ;cd /home1/DBA/CUBRID
Current directory changed to  /home1/DBA/CUBRID.
```

Exiting the CSQL Interpreter (;EXit)

This command exits the CSQL Interpreter.

```
csql> ;ex
```

Clearing the query buffer (;CLear)

The **;CLear** session command clears the contents of the query buffer.

```
csql> ;clear
csql> ;list
```

Displaying the contents of the query buffer (;List)

The **;List** session command lists the contents of the query buffer that have been entered or modified. The query buffer can be modified by **;READ** or **;Edit** command.

```
csql> ;list
```

Executing SQL statements (;RUn)

This command executes SQL statements in the query buffer. Unlike the **;Xrun** session command described below, the buffer will not be cleared even after the query execution.

```
csql> ;run
```

Clearing the query buffer after executing the SQL statement (;Xrun)

This command executes SQL statements in the query buffer. The buffer will be cleared after the query execution.

```
csql> ;xrun
```

Committing transaction (;COmmit)

This command commits the current transaction. You must enter a commit command explicitly if it is not in autocommit mode. In autocommit mode, transactions are automatically committed whenever SQL is executed.

```
csql> ;commit
Execute OK. (0.000192 sec)
```

Rolling back transaction (;ROllback)

This command rolls back the current transaction. Like a commit command (**;COmmit**), it must enter a rollback command explicitly if it is not in autocommit mode (**OFF**).

```
csql> ;rollback
Execute OK. (0.000166 sec)
```

Setting the autocommit mode (;AUtocommit)

This command sets autocommit mode to **ON** or **OFF**. If any value is not specified, current configured value is applied by default. The default value is **ON**.

```
csql> ;autocommit off
AUTOCOMMIT IS OFF
```

CHeckpoint Execution (;CHeckpoint)

This command executes the checkpoint within the CSQL session. This command can only be executed when a DBA group member, who is specified for the custom option (**-u** *user_name*), connects to the CSQL Interpreter in system administrator mode (**--sysadm**).

Checkpoint is an operation of flushing all dirty pages except for temp pages within the current data buffer to disks. You can also change the checkpoint interval using a command (**;set** *parameter_name* value) to set the parameter values in the CSQL session. You can see the examples of the parameter related to the checkpoint execution interval (**checkpoint_interval** and **checkpoint_every_size**). For more information, see Logging-Related Parameters.

```
sysadm> ;checkpoint
Checkpoint has been issued.
```

Transaction Monitoring Or Termination (;Killtran)

This command checks the transaction status information or terminates a specific transaction in the CSQL session. This command prints out the status information of all transactions on the screen if a parameter is omitted it terminates the transaction if a specific transaction ID is specified for the parameter. It can only be executed when a DBA group member, who is specified for the custom option (**-u** *user_name*), connects to the CSQL Interpreter in system administrator mode (**--sysadm**).

```
sysadm> ;killtran
Tran index      User name      Host name      Process id
Program name
------------------------------------------------------------------------
----------
      1(+)            dba      myhost             664
cub_cas
      2(+)            dba      myhost            6700
csql
      3(+)            dba      myhost            2188
cub_cas
      4(+)            dba      myhost             696
csql
      5(+)         public      myhost            6944
csql

sysadm> ;killtran 3
The specified transaction has been killed.
```

Restarting database (;REStart)

A command that tries to reconnect to the target database in a CSQL session. Note that when you execute the CSQL Interpreter in CS (client/server) mode, it will be disconnected from the server. When the connection to the server is lost due to a HA failure and failover to another server occurs, this command is particularly useful in connecting to the switched server while maintaining the current session.

```
csql> ;restart
The database has been restarted.
```

Displaying the current date (;DATE)

The **;DATE** command displays the current date and time in the CSQL Interpreter.

```
csql> ;date
     Tue July 29 18:58:12 KST 2008
```

Displaying the database information (;DATAbase)

This command displays the database name and hostname where the CSQL Interpreter is working. If the database is running, the HA mode (one of those following: active, standby, or maintenance) will be displayed as well.

```
csql> ;database
     demodb@cubridhost (active)
```

Displaying schema information of a class (;SChema)

The **;SChema** session command displays schema information of the specified table. The information includes the table name, its column name and constraints.

```
csql> ;schema event
=== <Help: Schema of a Class> ===
 <Class Name>
     event
 <Attributes>
     code            INTEGER NOT NULL
     sports          CHARACTER VARYING(50)
     name            CHARACTER VARYING(50)
     gender          CHARACTER(1)
     players         INTEGER
 <Constraints>
     PRIMARY KEY pk_event_event_code ON event (code)
```

Displaying the trigger (;TRigger)

This command searches and displays the trigger specified. If there is no trigger name specified, all the triggers defined will be displayed.

```
csql> ;trigger
=== <Help: All Triggers> ===
    trig_delete_contents
```

Checking the parameter value(;Get)

You can check the parameter value currently set in the CSQL Interpreter using the **;Get** session command. An error occurs if the parameter name specified is incorrect.

```
csql> ;get isolation_level
=== Get Param Input ===
isolation_level="tran_rep_class_commit_instance"
```

Setting the parameter value (;SET)

You can use the **;SET** session command to set a specific parameter value. Note that changeable parameters can have their values modified. To change the server parameter values, you must have DBA authorization. For information on list of changeable parameters, see Broker Configuration.

```
csql> ;set block_ddl_statement=1
=== Set Param Input ===
block_ddl_statement=1

-- Dynamically change the log_max_archives value in the csql
accessed by dba account
csql> ;set log_max_archives=5
```

Setting the output width of string (;STring-width)

You can use the **;STring-width** command to set the output width of character string or BIT string.

;string-width session command without a length shows the current setting length. When it is set to 0, the columns will be displayed as it is. If it sets greater than 0, the string typed columns will be displayed with the specified length.

```
csql> SELECT name FROM NATION WHERE NAME LIKE 'Ar%';
  'Arab Republic of Egypt'
  'Aruba'
  'Armenia'
  'Argentina'

csql> ;string-width 5
csql>  SELECT name FROM NATION WHERE NAME LIKE 'Ar%';
  'Arab '
  'Aruba'
```

```
  'Armen'
  'Argen'

csql> ;string-width
STRING-WIDTH : 5
```

Setting the output width of the column (;COLumn-width)

You can use the **;COLumn-width** command to set the output width regardless of its data types. If you don't give a value after **;COL** command, it shows the current setting length. When it sets to 0, the columns will be displayed as it is. If it sets to greater than 0, the columns will be displayed with the specified length.

```
csql> CREATE TABLE tbl(a BIGINT, b BIGINT);
csql> INSERT INTO tbl VALUES(12345678890, 1234567890)
csql> ;column-width a=5
csql> SELECT * FROM tbl;
      12345            1234567890
csql> ;column-width
COLUMN-WIDTH a : 5
```

Setting the view level of executing query plan (;PLan)

You can use the **;PLan** session command to set the view level of executing query plan the level is composed of **simple**, **detail**, and **off**. Each command refers to the following:

- **off**: Not displaying the query execution plan (OPT LEVEL=1)
- **simple**: Displaying the query execution plan in simple version (OPT LEVEL=257)
- **detail**: Displaying the query execution plan in detailed version (OPT LEVEL=513)

For more information. see Viewing Query Plan.

Setting SQL trace(;trace)

The **;trace** session command specifies if SQL trace result is printed out together with query result or not. When you set SQL trace ON by using this command, the result of query profiling is automatically shown even if you do not run "**SHOW TRACE**;" syntax.

For more information, see Query Profiling.

The command format is as follows.

```
;trace {on | off} [{text | json}]
```

- **on**: set on SQL trace.

- **off**: set off SQL trace.
- **text**: print out as a general TEXT format. If you omit OUTPUT clause, TEXT format is specified.
- **json**: print out as a JSON format.

> **Note**
>
> CSQL interpreter which is run in the standalone mode(use -S option) does not support SQL trace feature.

Displaying information (;Info)

The **;Info** session command allows you to check information such as schema, triggers, the working environment, locks and statistics.

```
csql> ;info lock
*** Lock Table Dump ***
 Lock Escalation at = 100000, Run Deadlock interval = 1
Transaction (index  0, unknown, unknown@unknown|-1)
Isolation COMMITTED READ
State TRAN_ACTIVE
Timeout_period -1
......
```

Dumping CSQL execution statistics information(;.Hist)

This command is a CSQL session command for starting to collect the statistics information in CSQL. The information is collected only for the currently connected CSQL after "**;.Hist on**" command is entered. Following options are provided for this session command.

- **on**: Starts collecting statistics information for the current connection.
- **off**: Stops collecting statistics information of server.

This command is executable while the **communication_histogram** parameter in the **cubrid.conf** file is set to **yes**. You can also view this information by using the **cubrid statdump** utility.

After running "**;.Hist on**", the execution commands such as **;.dump_hist** or **;.x** must be entered to output the statistics information. After **;.dump_hist** or **;.x**, all accumulated data are dumped and initiated.

As a reference, you should use **cubrid statdump** utility to check all queries' statistics information of a database server.

This example shows the server statistics information for current connection. For information on dumped specific items or **cubrid statdump** command, see statdump.

```
csql> ;set communication_histogram=yes
csql> ;.hist on
csql> SELECT name FROM nation LIMIT 5;
csql> ;.x
Histogram of client requests:
Name                              Rcount   Sent size  Recv size ,
Server time
LC_FETCHALL                          2 X         72+         0 b,
0.000000 s
LC_DOESEXIST                         3 X         84+        36 b,
0.000142 s
LC_FETCH_LOCKHINT_CLASSES            1 X         56+         0 b,
0.000000 s
BTREE_GET_KEY_TYPE                   5 X        180+        60 b,
0.000111 s
QM_QUERY_DROP_ALL_PLANS              1 X        224+        40 b,
0.000080 s
QM_QUERY_DUMP_PLANS                  1 X         72+       308 b,
0.000355 s
------------------------------------------------------------------------
------
Totals:                             13 X        688+       444 b
0.001415 s

Average server response time = 0.000109 secs
Average time between client requests = 0.000000 secs

*** CLIENT EXECUTION STATISTICS ***
System CPU (sec)                =          0
User CPU (sec)                  =          0
Elapsed (sec)                   =         20

*** SERVER EXECUTION STATISTICS ***
Num_file_creates                =          0
Num_file_removes                =          0
Num_file_ioreads                =          0
Num_file_iowrites               =          0
Num_file_iosynches              =          0
The timer values for file_iosync_all are:
Num_file_iosync_all             =          0
Total_time_file_iosync_all      =          0
Max_time_file_iosync_all        =          0
Avg_time_file_iosync_all        =          0
Num_file_page_allocs            =          0
Num_file_page_deallocs          =          0
Num_data_page_fetches           =         58
Num_data_page_dirties           =          0
Num_data_page_ioreads           =          0
```

```
Num_data_page_iowrites        =          0
Num_data_page_flushed         =          0
Num_data_page_private_quota   =        551
Num_data_page_private_count   =          8
Num_data_page_fixed           =          1
Num_data_page_dirty           =          0
Num_data_page_lru1            =         22
Num_data_page_lru2            =         14
Num_data_page_lru3            =          8
Num_data_page_victim_candidate =          8
Num_log_page_fetches          =          0
Num_log_page_ioreads          =          0
Num_log_page_iowrites         =          0
Num_log_append_records        =          0
Num_log_archives              =          0
Num_log_start_checkpoints     =          0
Num_log_end_checkpoints       =          0
Num_log_wals                  =          0
Num_log_page_iowrites_for_replacement =          0
Num_log_page_replacements     =          0
Num_page_locks_acquired       =          0
Num_object_locks_acquired     =          4
Num_page_locks_converted      =          0
Num_object_locks_converted    =          2
Num_page_locks_re-requested   =          0
Num_object_locks_re-requested =         11
Num_page_locks_waits          =          0
Num_object_locks_waits        =          0
Num_object_locks_time_waited_usec =          0
Num_tran_commits              =          1
Num_tran_rollbacks            =          0
Num_tran_savepoints           =          0
Num_tran_start_topops         =          0
Num_tran_end_topops           =          0
Num_tran_interrupts           =          0
Num_tran_postpone_cache_hits  =          0
Num_tran_postpone_cache_miss  =          0
Num_tran_topop_postpone_cache_hits =          0
Num_tran_topop_postpone_cache_miss =          0
Num_btree_inserts             =          0
Num_btree_deletes             =          0
Num_btree_updates             =          0
Num_btree_covered             =          0
Num_btree_noncovered          =          0
Num_btree_resumes             =          0
Num_btree_multirange_optimization =          0
Num_btree_splits              =          0
Num_btree_merges              =          0
Num_btree_get_stats           =          0
The timer values for btree_online_load are:
Num_btree_online_load         =          0
Total_time_btree_online_load  =          0
```

```
Max_time_btree_online_load    =          0
Avg_time_btree_online_load    =          0
The timer values for btree_online_insert_task are:
Num_btree_online_insert_task  =          0
Total_time_btree_online_insert_task =          0
Max_time_btree_online_insert_task =          0
Avg_time_btree_online_insert_task =          0
The timer values for btree_online_prepare_task are:
Num_btree_online_prepare_task =          0
Total_time_btree_online_prepare_task =          0
Max_time_btree_online_prepare_task =          0
Avg_time_btree_online_prepare_task =          0
The timer values for btree_online_insert_leaf are:
Num_btree_online_insert_leaf  =          0
Total_time_btree_online_insert_leaf =          0
Max_time_btree_online_insert_leaf =          0
Avg_time_btree_online_insert_leaf =          0
Num_btree_online_inserts      =          0
Num_btree_online_inserts_same_page_hold =          0
Num_btree_online_inserts_retry =          0
Num_btree_online_inserts_retry_nice =          0
Num_query_selects             =          1
Num_query_inserts             =          0
Num_query_deletes             =          0
Num_query_updates             =          0
Num_query_sscans              =          1
Num_query_iscans              =          0
Num_query_lscans              =          0
Num_query_setscans            =          0
Num_query_methscans           =          0
Num_query_nljoins             =          0
Num_query_mjoins              =          0
Num_query_objfetches          =          0
Num_query_holdable_cursors    =          0
Num_sort_io_pages             =          0
Num_sort_data_pages           =          0
Num_network_requests          =         14
Num_adaptive_flush_pages      =          0
Num_adaptive_flush_log_pages  =          0
Num_adaptive_flush_max_pages  =          0
Num_prior_lsa_list_size       =          0
Num_prior_lsa_list_maxed      =          0
Num_prior_lsa_list_removed    =          0
Time_ha_replication_delay     =          0
Num_plan_cache_add            =          0
Num_plan_cache_lookup         =          2
Num_plan_cache_hit            =          2
Num_plan_cache_miss           =          0
Num_plan_cache_full           =          0
Num_plan_cache_delete         =          0
Num_plan_cache_invalid_xasl_id =          0
Num_plan_cache_entries        =          1
```

```
Num_vacuum_log_pages_vacuumed =          0
Num_vacuum_log_pages_to_vacuum =          0
Num_vacuum_prefetch_requests_log_pages =          0
Num_vacuum_prefetch_hits_log_pages =          0
Num_heap_home_inserts         =          0
Num_heap_big_inserts          =          0
Num_heap_assign_inserts       =          0
Num_heap_home_deletes         =          0
Num_heap_home_mvcc_deletes    =          0
Num_heap_home_to_rel_deletes  =          0
Num_heap_home_to_big_deletes  =          0
Num_heap_rel_deletes          =          0
Num_heap_rel_mvcc_deletes     =          0
Num_heap_rel_to_home_deletes  =          0
Num_heap_rel_to_big_deletes   =          0
Num_heap_rel_to_rel_deletes   =          0
Num_heap_big_deletes          =          0
Num_heap_big_mvcc_deletes     =          0
Num_heap_home_updates         =          0
Num_heap_home_to_rel_updates  =          0
Num_heap_home_to_big_updates  =          0
Num_heap_rel_updates          =          0
Num_heap_rel_to_home_updates  =          0
Num_heap_rel_to_rel_updates   =          0
Num_heap_rel_to_big_updates   =          0
Num_heap_big_updates          =          0
Num_heap_home_vacuums         =          0
Num_heap_big_vacuums          =          0
Num_heap_rel_vacuums          =          0
Num_heap_insid_vacuums        =          0
Num_heap_remove_vacuums       =          0
The timer values for heap_insert_prepare are:
Num_heap_insert_prepare       =          0
Total_time_heap_insert_prepare =          0
Max_time_heap_insert_prepare  =          0
Avg_time_heap_insert_prepare  =          0
The timer values for heap_insert_execute are:
Num_heap_insert_execute       =          0
Total_time_heap_insert_execute =          0
Max_time_heap_insert_execute  =          0
Avg_time_heap_insert_execute  =          0
The timer values for heap_insert_log are:
Num_heap_insert_log           =          0
Total_time_heap_insert_log    =          0
Max_time_heap_insert_log      =          0
Avg_time_heap_insert_log      =          0
The timer values for heap_delete_prepare are:
Num_heap_delete_prepare       =          0
Total_time_heap_delete_prepare =          0
Max_time_heap_delete_prepare  =          0
Avg_time_heap_delete_prepare  =          0
The timer values for heap_delete_execute are:
```

```
Num_heap_delete_execute       =          0
Total_time_heap_delete_execute =          0
Max_time_heap_delete_execute  =          0
Avg_time_heap_delete_execute  =          0
The timer values for heap_delete_log are:
Num_heap_delete_log           =          0
Total_time_heap_delete_log    =          0
Max_time_heap_delete_log      =          0
Avg_time_heap_delete_log      =          0
The timer values for heap_update_prepare are:
Num_heap_update_prepare       =          0
Total_time_heap_update_prepare =          0
Max_time_heap_update_prepare  =          0
Avg_time_heap_update_prepare  =          0
The timer values for heap_update_execute are:
Num_heap_update_execute       =          0
Total_time_heap_update_execute =          0
Max_time_heap_update_execute  =          0
Avg_time_heap_update_execute  =          0
The timer values for heap_update_log are:
Num_heap_update_log           =          0
Total_time_heap_update_log    =          0
Max_time_heap_update_log      =          0
Avg_time_heap_update_log      =          0
The timer values for heap_vacuum_prepare are:
Num_heap_vacuum_prepare       =          0
Total_time_heap_vacuum_prepare =          0
Max_time_heap_vacuum_prepare  =          0
Avg_time_heap_vacuum_prepare  =          0
The timer values for heap_vacuum_execute are:
Num_heap_vacuum_execute       =          0
Total_time_heap_vacuum_execute =          0
Max_time_heap_vacuum_execute  =          0
Avg_time_heap_vacuum_execute  =          0
The timer values for heap_vacuum_log are:
Num_heap_vacuum_log           =          0
Total_time_heap_vacuum_log    =          0
Max_time_heap_vacuum_log      =          0
Avg_time_heap_vacuum_log      =          0
The timer values for heap_stats_sync_bestspace are:
Num_heap_stats_sync_bestspace =          0
Total_time_heap_stats_sync_bestspace =          0
Max_time_heap_stats_sync_bestspace =          0
Avg_time_heap_stats_sync_bestspace =          0
Num_heap_stats_bestspace_entries =          0
Num_heap_stats_bestspace_maxed =          0
The timer values for bestspace_add are:
Num_bestspace_add             =          0
Total_time_bestspace_add      =          0
Max_time_bestspace_add        =          0
Avg_time_bestspace_add        =          0
The timer values for bestspace_del are:
```

```
Num_bestspace_del             =          0
Total_time_bestspace_del      =          0
Max_time_bestspace_del        =          0
Avg_time_bestspace_del        =          0
The timer values for bestspace_find are:
Num_bestspace_find            =          0
Total_time_bestspace_find     =          0
Max_time_bestspace_find       =          0
Avg_time_bestspace_find       =          0
The timer values for heap_find_page_bestspace are:
Num_heap_find_page_bestspace  =          0
Total_time_heap_find_page_bestspace =          0
Max_time_heap_find_page_bestspace =          0
Avg_time_heap_find_page_bestspace =          0
The timer values for heap_find_best_page are:
Num_heap_find_best_page       =          0
Total_time_heap_find_best_page =          0
Max_time_heap_find_best_page  =          0
Avg_time_heap_find_best_page  =          0
The timer values for bt_fix_ovf_oids are:
Num_bt_fix_ovf_oids           =          0
Total_time_bt_fix_ovf_oids    =          0
Max_time_bt_fix_ovf_oids      =          0
Avg_time_bt_fix_ovf_oids      =          0
The timer values for bt_unique_rlocks are:
Num_bt_unique_rlocks          =          0
Total_time_bt_unique_rlocks   =          0
Max_time_bt_unique_rlocks     =          0
Avg_time_bt_unique_rlocks     =          0
The timer values for bt_unique_wlocks are:
Num_bt_unique_wlocks          =          0
Total_time_bt_unique_wlocks   =          0
Max_time_bt_unique_wlocks     =          0
Avg_time_bt_unique_wlocks     =          0
The timer values for bt_leaf are:
Num_bt_leaf                   =          0
Total_time_bt_leaf            =          0
Max_time_bt_leaf              =          0
Avg_time_bt_leaf              =          0
The timer values for bt_traverse are:
Num_bt_traverse               =          5
Total_time_bt_traverse        =        162
Max_time_bt_traverse          =         51
Avg_time_bt_traverse          =         32
The timer values for bt_find_unique are:
Num_bt_find_unique            =          0
Total_time_bt_find_unique     =          0
Max_time_bt_find_unique       =          0
Avg_time_bt_find_unique       =          0
The timer values for bt_find_unique_traverse are:
Num_bt_find_unique_traverse   =          5
Total_time_bt_find_unique_traverse =        165
```

```
Max_time_bt_find_unique_traverse =         53
Avg_time_bt_find_unique_traverse =         33
The timer values for bt_range_search are:
Num_bt_range_search           =          0
Total_time_bt_range_search    =          0
Max_time_bt_range_search      =          0
Avg_time_bt_range_search      =          0
The timer values for bt_range_search_traverse are:
Num_bt_range_search_traverse  =          0
Total_time_bt_range_search_traverse =          0
Max_time_bt_range_search_traverse =          0
Avg_time_bt_range_search_traverse =          0
The timer values for bt_insert are:
Num_bt_insert                 =          0
Total_time_bt_insert          =          0
Max_time_bt_insert            =          0
Avg_time_bt_insert            =          0
The timer values for bt_insert_traverse are:
Num_bt_insert_traverse        =          0
Total_time_bt_insert_traverse =          0
Max_time_bt_insert_traverse   =          0
Avg_time_bt_insert_traverse   =          0
The timer values for bt_delete_obj are:
Num_bt_delete_obj             =          0
Total_time_bt_delete_obj      =          0
Max_time_bt_delete_obj        =          0
Avg_time_bt_delete_obj        =          0
The timer values for bt_delete_obj_traverse are:
Num_bt_delete_obj_traverse    =          0
Total_time_bt_delete_obj_traverse =          0
Max_time_bt_delete_obj_traverse =          0
Avg_time_bt_delete_obj_traverse =          0
The timer values for bt_mvcc_delete are:
Num_bt_mvcc_delete            =          0
Total_time_bt_mvcc_delete     =          0
Max_time_bt_mvcc_delete       =          0
Avg_time_bt_mvcc_delete       =          0
The timer values for bt_mvcc_delete_traverse are:
Num_bt_mvcc_delete_traverse   =          0
Total_time_bt_mvcc_delete_traverse =          0
Max_time_bt_mvcc_delete_traverse =          0
Avg_time_bt_mvcc_delete_traverse =          0
The timer values for bt_mark_delete are:
Num_bt_mark_delete            =          0
Total_time_bt_mark_delete     =          0
Max_time_bt_mark_delete       =          0
Avg_time_bt_mark_delete       =          0
The timer values for bt_mark_delete_traverse are:
Num_bt_mark_delete_traverse   =          0
Total_time_bt_mark_delete_traverse =          0
Max_time_bt_mark_delete_traverse =          0
Avg_time_bt_mark_delete_traverse =          0
```

```
The timer values for bt_undo_insert are:
Num_bt_undo_insert            =          0
Total_time_bt_undo_insert     =          0
Max_time_bt_undo_insert       =          0
Avg_time_bt_undo_insert       =          0
The timer values for bt_undo_insert_traverse are:
Num_bt_undo_insert_traverse   =          0
Total_time_bt_undo_insert_traverse =          0
Max_time_bt_undo_insert_traverse =          0
Avg_time_bt_undo_insert_traverse =          0
The timer values for bt_undo_delete are:
Num_bt_undo_delete            =          0
Total_time_bt_undo_delete     =          0
Max_time_bt_undo_delete       =          0
Avg_time_bt_undo_delete       =          0
The timer values for bt_undo_delete_traverse are:
Num_bt_undo_delete_traverse   =          0
Total_time_bt_undo_delete_traverse =          0
Max_time_bt_undo_delete_traverse =          0
Avg_time_bt_undo_delete_traverse =          0
The timer values for bt_undo_mvcc_delete are:
Num_bt_undo_mvcc_delete       =          0
Total_time_bt_undo_mvcc_delete =          0
Max_time_bt_undo_mvcc_delete  =          0
Avg_time_bt_undo_mvcc_delete  =          0
The timer values for bt_undo_mvcc_delete_traverse are:
Num_bt_undo_mvcc_delete_traverse =          0
Total_time_bt_undo_mvcc_delete_traverse =          0
Max_time_bt_undo_mvcc_delete_traverse =          0
Avg_time_bt_undo_mvcc_delete_traverse =          0
The timer values for bt_vacuum are:
Num_bt_vacuum                 =          0
Total_time_bt_vacuum          =          0
Max_time_bt_vacuum            =          0
Avg_time_bt_vacuum            =          0
The timer values for bt_vacuum_traverse are:
Num_bt_vacuum_traverse        =          0
Total_time_bt_vacuum_traverse =          0
Max_time_bt_vacuum_traverse   =          0
Avg_time_bt_vacuum_traverse   =          0
The timer values for bt_vacuum_insid are:
Num_bt_vacuum_insid           =          0
Total_time_bt_vacuum_insid    =          0
Max_time_bt_vacuum_insid      =          0
Avg_time_bt_vacuum_insid      =          0
The timer values for bt_vacuum_insid_traverse are:
Num_bt_vacuum_insid_traverse  =          0
Total_time_bt_vacuum_insid_traverse =          0
Max_time_bt_vacuum_insid_traverse =          0
Avg_time_bt_vacuum_insid_traverse =          0
The timer values for vacuum_master are:
Num_vacuum_master             =          0
```

```
Total_time_vacuum_master      =          0
Max_time_vacuum_master        =          0
Avg_time_vacuum_master        =          0
The timer values for vacuum_job are:
Num_vacuum_job                =          0
Total_time_vacuum_job         =          0
Max_time_vacuum_job           =          0
Avg_time_vacuum_job           =          0
The timer values for vacuum_worker_process_log are:
Num_vacuum_worker_process_log =          0
Total_time_vacuum_worker_process_log =          0
Max_time_vacuum_worker_process_log =          0
Avg_time_vacuum_worker_process_log =          0
The timer values for vacuum_worker_execute are:
Num_vacuum_worker_execute     =          0
Total_time_vacuum_worker_execute =          0
Max_time_vacuum_worker_execute =          0
Avg_time_vacuum_worker_execute =          0
Time_get_snapshot_acquire_time =          7
Count_get_snapshot_retry      =          0
Time_tran_complete_time       =          7
The timer values for compute_oldest_visible are:
Num_compute_oldest_visible    =          0
Total_time_compute_oldest_visible =          0
Max_time_compute_oldest_visible =          0
Avg_time_compute_oldest_visible =          0
Count_get_oldest_mvcc_retry   =          0
Data_page_buffer_hit_ratio    =     100.00
Log_page_buffer_hit_ratio     =       0.00
Vacuum_data_page_buffer_hit_ratio =       0.00
Vacuum_page_efficiency_ratio  =       0.00
Vacuum_page_fetch_ratio       =       0.00
Data_page_fix_lock_acquire_time_msec =       0.00
Data_page_fix_hold_acquire_time_msec =       0.00
Data_page_fix_acquire_time_msec =       0.25
Data_page_allocate_time_ratio =     100.00
Data_page_total_promote_success =       0.00
Data_page_total_promote_fail  =       0.00
Data_page_total_promote_time_msec =       0.00
Num_unfix_void_to_private_top =          0
Num_unfix_void_to_private_mid =          0
Num_unfix_void_to_shared_mid  =          0
Num_unfix_lru1_private_to_shared_mid =          0
Num_unfix_lru2_private_to_shared_mid =          0
Num_unfix_lru3_private_to_shared_mid =          0
Num_unfix_lru2_private_keep   =          0
Num_unfix_lru2_shared_keep    =          0
Num_unfix_lru2_private_to_top =          0
Num_unfix_lru2_shared_to_top  =          0
Num_unfix_lru3_private_to_top =          4
Num_unfix_lru3_shared_to_top  =          0
Num_unfix_lru1_private_keep   =          3
```

```
Num_unfix_lru1_shared_keep    =         36
Num_unfix_void_to_private_mid_vacuum =          0
Num_unfix_lru1_any_keep_vacuum =          0
Num_unfix_lru2_any_keep_vacuum =          0
Num_unfix_lru3_any_keep_vacuum =          0
Num_unfix_void_aout_found     =          0
Num_unfix_void_aout_not_found =          0
Num_unfix_void_aout_found_vacuum =          0
Num_unfix_void_aout_not_found_vacuum =          0
Num_data_page_hash_anchor_waits =          0
Time_data_page_hash_anchor_wait =          0
The timer values for flush_collect are:
Num_flush_collect             =          0
Total_time_flush_collect      =          0
Max_time_flush_collect        =          0
Avg_time_flush_collect        =          0
The timer values for flush_flush are:
Num_flush_flush               =          0
Total_time_flush_flush        =          0
Max_time_flush_flush          =          0
Avg_time_flush_flush          =          0
The timer values for flush_sleep are:
Num_flush_sleep               =          0
Total_time_flush_sleep        =          0
Max_time_flush_sleep          =          0
Avg_time_flush_sleep          =          0
The timer values for flush_collect_per_page are:
Num_flush_collect_per_page    =          0
Total_time_flush_collect_per_page =          0
Max_time_flush_collect_per_page =          0
Avg_time_flush_collect_per_page =          0
The timer values for flush_flush_per_page are:
Num_flush_flush_per_page      =          0
Total_time_flush_flush_per_page =          0
Max_time_flush_flush_per_page =          0
Avg_time_flush_flush_per_page =          0
Num_data_page_writes          =          0
Num_data_page_dirty_to_post_flush =          0
Num_data_page_skipped_flush   =          0
Num_data_page_skipped_flush_need_wal =          0
Num_data_page_skipped_flush_already_flushed =          0
Num_data_page_skipped_flush_fixed_or_hot =          0
The timer values for compensate_flush are:
Num_compensate_flush          =          0
Total_time_compensate_flush   =          0
Max_time_compensate_flush     =          0
Avg_time_compensate_flush     =          0
The timer values for assign_direct_bcb are:
Num_assign_direct_bcb         =          0
Total_time_assign_direct_bcb  =          0
Max_time_assign_direct_bcb    =          0
Avg_time_assign_direct_bcb    =          0
```

```
The timer values for wake_flush_waiter are:
Num_wake_flush_waiter         =          0
Total_time_wake_flush_waiter  =          0
Max_time_wake_flush_waiter    =          0
Avg_time_wake_flush_waiter    =          0
The timer values for alloc_bcb are:
Num_alloc_bcb                 =          0
Total_time_alloc_bcb          =          0
Max_time_alloc_bcb            =          0
Avg_time_alloc_bcb            =          0
The timer values for alloc_bcb_search_victim are:
Num_alloc_bcb_search_victim   =          0
Total_time_alloc_bcb_search_victim =          0
Max_time_alloc_bcb_search_victim =          0
Avg_time_alloc_bcb_search_victim =          0
The timer values for alloc_bcb_cond_wait_high_prio are:
Num_alloc_bcb_cond_wait_high_prio =          0
Total_time_alloc_bcb_cond_wait_high_prio =          0
Max_time_alloc_bcb_cond_wait_high_prio =          0
Avg_time_alloc_bcb_cond_wait_high_prio =          0
The timer values for alloc_bcb_cond_wait_low_prio are:
Num_alloc_bcb_cond_wait_low_prio =          0
Total_time_alloc_bcb_cond_wait_low_prio =          0
Max_time_alloc_bcb_cond_wait_low_prio =          0
Avg_time_alloc_bcb_cond_wait_low_prio =          0
Num_alloc_bcb_prioritize_vacuum =          0
Num_victim_use_invalid_bcb    =          0
The timer values for alloc_bcb_get_victim_search_own_private_list
are:
Num_alloc_bcb_get_victim_search_own_private_list =          0
Total_time_alloc_bcb_get_victim_search_own_private_list =          0
Max_time_alloc_bcb_get_victim_search_own_private_list =          0
Avg_time_alloc_bcb_get_victim_search_own_private_list =          0
The timer values for alloc_bcb_get_victim_search_others_private_list
are:
Num_alloc_bcb_get_victim_search_others_private_list =          0
Total_time_alloc_bcb_get_victim_search_others_private_list
=          0
Max_time_alloc_bcb_get_victim_search_others_private_list =          0
Avg_time_alloc_bcb_get_victim_search_others_private_list =          0
The timer values for alloc_bcb_get_victim_search_shared_list are:
Num_alloc_bcb_get_victim_search_shared_list =          0
Total_time_alloc_bcb_get_victim_search_shared_list =          0
Max_time_alloc_bcb_get_victim_search_shared_list =          0
Avg_time_alloc_bcb_get_victim_search_shared_list =          0
Num_victim_assign_direct_vacuum_void =          0
Num_victim_assign_direct_vacuum_lru =          0
Num_victim_assign_direct_flush =          0
Num_victim_assign_direct_panic =          0
Num_victim_assign_direct_adjust_lru =          0
Num_victim_assign_direct_adjust_lru_to_vacuum =          0
Num_victim_assign_direct_search_for_flush =          0
```

```
Num_victim_shared_lru_success =          0
Num_victim_own_private_lru_success =          0
Num_victim_other_private_lru_success =          0
Num_victim_shared_lru_fail    =          0
Num_victim_own_private_lru_fail =          0
Num_victim_other_private_lru_fail =          0
Num_victim_all_lru_fail       =          0
Num_victim_get_from_lru       =          0
Num_victim_get_from_lru_was_empty =          0
Num_victim_get_from_lru_fail  =          0
Num_victim_get_from_lru_bad_hint =          0
Num_lfcq_prv_get_total_calls  =          0
Num_lfcq_prv_get_empty        =          0
Num_lfcq_prv_get_big          =          0
Num_lfcq_shr_get_total_calls  =          0
Num_lfcq_shr_get_empty        =          0
The timer values for DWB_flush_block are:
Num_DWB_flush_block           =          0
Total_time_DWB_flush_block    =          0
Max_time_DWB_flush_block      =          0
Avg_time_DWB_flush_block      =          0
The timer values for DWB_file_sync_helper are:
Num_DWB_file_sync_helper      =          0
Total_time_DWB_file_sync_helper =          0
Max_time_DWB_file_sync_helper =          0
Avg_time_DWB_file_sync_helper =          0
The timer values for DWB_flush_block_cond_wait are:
Num_DWB_flush_block_cond_wait =          0
Total_time_DWB_flush_block_cond_wait =          0
Max_time_DWB_flush_block_cond_wait =          0
Avg_time_DWB_flush_block_cond_wait =          0
The timer values for DWB_flush_block_sort are:
Num_DWB_flush_block_sort      =          0
Total_time_DWB_flush_block_sort =          0
Max_time_DWB_flush_block_sort =          0
Avg_time_DWB_flush_block_sort =          0
The timer values for DWB_decache_pages_after_write are:
Num_DWB_decache_pages_after_write =          0
Total_time_DWB_decache_pages_after_write =          0
Max_time_DWB_decache_pages_after_write =          0
Avg_time_DWB_decache_pages_after_write =          0
The timer values for DWB_wait_flush_block are:
Num_DWB_wait_flush_block      =          0
Total_time_DWB_wait_flush_block =          0
Max_time_DWB_wait_flush_block =          0
Avg_time_DWB_wait_flush_block =          0
The timer values for DWB_wait_file_sync_helper are:
Num_DWB_wait_file_sync_helper =          0
Total_time_DWB_wait_file_sync_helper =          0
Max_time_DWB_wait_file_sync_helper =          0
Avg_time_DWB_wait_file_sync_helper =          0
The timer values for DWB_flush_force are:
```

```
Num_DWB_flush_force           =           0
Total_time_DWB_flush_force    =           0
Max_time_DWB_flush_force      =           0
Avg_time_DWB_flush_force      =           0
The timer values for Log_LZ4_compress are:
Num_Log_LZ4_compress          =           0
Total_time_Log_LZ4_compress   =           0
Max_time_Log_LZ4_compress     =           0
Avg_time_Log_LZ4_compress     =           0
The timer values for Log_LZ4_decompress are:
Num_Log_LZ4_decompress        =           0
Total_time_Log_LZ4_decompress =           0
Max_time_Log_LZ4_decompress   =           0
Avg_time_Log_LZ4_decompress   =           0
Num_alloc_bcb_wait_threads_high_priority =           0
Num_alloc_bcb_wait_threads_low_priority =           0
Num_flushed_bcbs_wait_for_direct_victim =           0
Num_lfcq_big_private_lists    =           0
Num_lfcq_private_lists        =           1
Num_lfcq_shared_lists         =           0
Num_data_page_avoid_dealloc   =           0
Num_data_page_avoid_victim    =           0
Num_data_page_fix_ext:
WORKER,PAGE_HEAP     ,OLD_PAGE_IN_PB    ,READ ,UNCOND
=           8
WORKER,PAGE_VOLHEADER,OLD_PAGE_IN_PB    ,READ ,UNCOND      =
30
WORKER,PAGE_VOLBITMAP,OLD_PAGE_IN_PB    ,READ ,UNCOND      =
15
WORKER,PAGE_BTREE_R  ,OLD_PAGE_IN_PB    ,READ ,UNCOND
=           5
Num_data_page_promote_ext:
Num_data_page_promote_time_ext:
Num_data_page_unfix_ext:
WORKER,PAGE_HEAP     ,BUF_NON_DIRTY,HOLDER_NON_DIRTY,READ
=           8
WORKER,PAGE_VOLHEADER,BUF_NON_DIRTY,HOLDER_NON_DIRTY,READ  =
30
WORKER,PAGE_VOLBITMAP,BUF_NON_DIRTY,HOLDER_NON_DIRTY,READ  =
15
WORKER,PAGE_BTREE_R  ,BUF_NON_DIRTY,HOLDER_NON_DIRTY,READ
=           5
Time_data_page_lock_acquire_time:
Time_data_page_hold_acquire_time:
Time_data_page_fix_acquire_time:
WORKER,PAGE_HEAP     ,OLD_PAGE_IN_PB    ,READ ,UNCOND
=              27
WORKER,PAGE_VOLHEADER,OLD_PAGE_IN_PB    ,READ ,UNCOND
=             167
WORKER,PAGE_VOLBITMAP,OLD_PAGE_IN_PB    ,READ ,UNCOND
=              40
WORKER,PAGE_BTREE_R  ,OLD_PAGE_IN_PB    ,READ ,UNCOND
```

```
=               23
Num_mvcc_snapshot_ext:
DELETE  ,INS_VACUUMED      ,VISIBLE   =               5
SNAPSHOT,INS_VACUUMED      ,VISIBLE   =               5
Time_obj_lock_acquire_time:
Thread_stats_counters_timers:
Thread_pgbuf_daemon_stats_counters_timers:
Num_dwb_flushed_block_volumes:
Thread_loaddb_stats_counters_timers:

csql> ;.hist off
```

Displaying query execution time (;TIme)

The **;TIme** session command can be set to display the elapsed time to execute the query. It can be set to **ON** or **OFF**. The current setting is displayed if there is no value specified. The default value is **ON**.

The **SELECT** query includes the time of outputting the fetched records. Therefore, to check the execution time of complete output of all records in the **SELECT** query, use the **--no-pager** option while executing the CSQC interpreter.

```
$ csql -u dba --no-pager demodb
csql> ;time ON
csql> ;time
TIME IS ON
```

Displaying Server Stored Messages (;SERver-output)

If this value is set to ON, it will output messages stored in the server's DBMS_OUTPUT buffer. The default value is OFF. The DBMS_OUTPUT buffer stores messages through calls to DBMS_OUTPUT.put_line() or DBMS_OUTPUT.put() in PL/CSQL stored procedures/functions. After executing an SQL statement, DBMS_OUTPUT messages are displayed under the '<DBMS_OUTPUT>' label. If an error occurs during the execution of a stored procedure/function, DBMS_OUTPUT messages are displayed after all CSQL's default messages, including error messages, regardless of the execution order, so caution is needed. For example, in the case below, the DBMS_OUTPUT.put_line() statement is executed before the RETURN statement that causes a ZERO_DIVIDE exception, but in the output, the DBMS_OUTPUT message appears after the error message.

```
csql> ;server-output on
SERVER OUTPUT IS ON
csql>
csql> create or replace function late_message return integer as
csql> begin
csql>     DBMS_OUTPUT.put_line('Hello world');
```

```
csql>     return (1/0);   -- ZERO_DIVIDE exception
csql> end;
Execute OK. (0.024973 sec) Committed. (0.000000 sec)

1 command(s) successfully processed.
csql>
csql> select late_message() from dual;

In line 2, column 23,

ERROR: Stored procedure execute error:
  (line 4, column 13) division by zero

<DBMS_OUTPUT>
====
Hello world

0 command(s) successfully processed.
```

Displaying a column of result record in one line(;LINe-output)

If this value is set to ON, it makes the record display in several lines by column. The default value is OFF, which makes one record display in one line.

```
csql> ;line-output ON
csql> select * from athlete;

=== <Result of SELECT Command in Line 1> ===

<00001> code        : 10999
        name        : 'Fernandez Jesus'
        gender      : 'M'
        nation_code: 'ESP'
        event       : 'Handball'
<00002> code        : 10998
        name        : 'Fernandez Jaime'
        gender      : 'M'
        nation_code: 'AUS'
        event       : 'Rowing'
...
```

Displaying query history (;HISTORYList)

This command displays the list that contains previously executed commands (input) and their history numbers.

```
csql> ;historylist
----< 1 >----
select * from nation;
```

```
----< 2 >----
select * from athlete;
```

Reading input with the specified history number into the buffer (;HISTORYRead)

You can use **;HISTORYRead** session command to read input with history number in the **;HISTORYList** list into the command buffer. You can enter **;run** or **;xrun** directly because it has the same effect as when you enter SQL statements directly.

```
csql> ;historyread 1
```

Calling the default editor (;EDIT)

This command calls the specified editor. The default editor is **vi** on Linux **Notepad** on Windows environment. Use **;EDITOR_Cmd** command to specify a different editor.

format / fmt

The **format** or **fmt** option allows you to edit after formatting the SQL statement. Formatter registration uses the **;FOrmatter_cmd** session command.

```
csql> ;edit
SELECT * FROM t1

csql> ;edit format
SELECT
 *
FROM
 t1
```

Specifying the formatter (;FOrmatter_cmd)

This command specifies the formatter to be used with **;EDIT** session command. The user can set it up and use it as shown in the example, and the OS environment variable(CUBRID_CSQL_FORMATTER or FORMATTER; CUBRID_CSQL_FORMATTER is used if both environment variables are set) can be configured in advance for use.

```
csql> ;formatter_cmd /usr/local/bin/fsqlf

$ export CUBRID_CSQL_FORMATTER=/home/cubrid/bin/fsqlf
or
$ export FORMATTER=/home/cubrid/bin/fsqlf
```

Note

The use of Free SQL Formatter is recommended.

Download URL: https://github.com/CUBRID/fsqlf/releases

Specifying the editor (;EDITOR_Cmd)

This command specifies the editor to be used with **;EDIT** session command. As shown in the example below, you can specify other editor (ex: emacs) which is installed in the system.

```
csql> ;editor_cmd emacs
csql> ;edit
```

The OS environment variable(CUBRID_CSQL_EDITOR or EDITOR; CUBRID_CSQL_EDITOR is used if both environment variables are set) may be registered and used in advance.

```
export CUBRID_CSQL_EDITOR=emacs
or
export EDITOR=emacs
```

Specifying the single line mode (;SIngleline)

This command sets single line mode to **ON** or **OFF** (default value is **ON**). If you type semi-colon(;) and ENTER key in the single line ON mode, SQL statements are executed. If single line mode is set to **OFF**, multiple SQL statements are retrieved and executed at once with the **;xr** or **;ru** session command. If any value is not specified, current configured value is applied by default.

```
csql> ;singleline off
SINGLE IS OFF
csql> ;singleline
SINGLE IS OFF
```

Switching csql session(;Connect)

This is a session command to switch the connection to another user without terminating CSQL. The command format is as follows.

- user: Username to connect
- database: Database name to connect (if omitted, currently connected database)
- host: Hostname to connect to (if omitted, localhost)

```
csql> ;connect public
csql> ;connect dba

csql> ;connect public testdb
csql> ;connect dba demodb

csql> ;connect Peter testdb@192.168.0.1
csql> ;connect public demodb@localhost
```

Warning

1. When ';connect' session command executes, the current CSQL session connection is disconnect regardless of session transition success or not.
2. When CSQL is in system administrator mode, ';connect' session command cannot be used (When entering CSQL with an csql -u dba --sysadm demodb form command).

CUBRID SQL

This chapter describes SQL syntax such as data types, functions and operators, data retrieval or table manipulation. You can also find SQL statements used for index, trigger, partition, serial and changing user information.

The main topics covered in this chapter are as follows:

- Writing Rules
  - Identifier: Describes how to write, the identifier, string allowed to be used as a name of a table, index, and column.
  - Reserved words: Lists reserved words in CUBRID. To use a reserved word as an identifier, enclose the identifier by using double quotes, backticks (`), or brackets ([]).
  - Comment
  - Literal: Describes how to write constant values.
- Data types: Describes the data types, the format to store data.
- Data Definition Statements: Describes how to create, alter, drop, and rename a table, an index, a view and a serial.

- Operators and Functions: Describes the operators and functions used for query statements.
- Data Manipulation Statements: Describes the SELECT, INSERT, UPDATE, and DELETE statements.
- Query Optimization: Describes the query optimization by using the index, hint, and the index hint syntax.
- Partitioning: Describes how to partition one table into several independent logical units.
- Trigger: Describes how to create, alter, drop, and rename a trigger that is automatically executed in response to certain events.
- Class Inheritance: Describes how to inherit the attribute from the parent to the child table (class).
- Database Administration: Describes about user management, SET and SHOW statements.
- User Schema: Describes a logical collection of objects in a database that is created with users.
- System Catalog: Describes the CUBRID system catalog, the internal information of the CUBRID database.

Writing Rules

Identifier

Guidelines for Creating Identifiers

Table name, index name, view name, column name, username etc. are included in identifier. The guidelines for creating identifiers are as follows:

- An identifier must begin with a letter; it must not begin with a number or a symbol.
- It is not case-sensitive.
- Reserved Words are not allowed.

```
identifier        ::= identifier_letter [ { other_identifier }; ]
identifier_letter ::= upper_case_letter | lower_case_letter
other_identifier  ::= identifier_letter | digit | _ | #
digit             ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
upper_case_letter ::= A | B | C | D | E | F | G | H | I | J | K | L | M | N | O |
lower_case_letter ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o |
```

Legal Identifiers

Beginning with a Letter

An identifier must begin with a letter. All other special characters except operator characters are allowed. The following are examples of legal identifiers.

```
a
a_b
ssn#
this_is_an_example_#
```

Enclosing in Double Quotes, Square Brackets, or Backtick Symbol

Identifiers or reserved keywords shown as below are not allowed. However, if they are enclosed in double quotes, square brackets, or backtick symbol, they are allowed as an exception. Especially, the double quotations can be used as a symbol enclosing identifiers when the **ansi_quotes** parameter is set to **yes**. If this value is set to **no** double quotations are used as a symbol enclosing character strings. The following are examples of legal identifiers.

```
"select"
"@lowcost"
"low cost"
"abc" "def"
[position]
```

⚠ Warning

Even though identifiers are enclosed in double quotation marks, square brackets, or backticks, they cannot contain square brackets ([ ]) or dots (.).

- Square brackets are used by default to process reserved words when parsing a query inside.
- Dots are used in Path Expressions and JSON, and they are also used to separate schema names and object names.

The following identifiers are not allowed even though they are enclosed in double quotation marks (" ").

```
"he[[o"
"wor]d"
"hello[]world"
"hello.world"
```

Note

From 10.0 version, if a column name which is a reserved word is used together with a "table name (or alias).", you don't need to wrap up a column name with a double quotes.

```
CREATE TABLE tbl ("int" int, "double" double);

SELECT t.int FROM tbl t;
```

In the above SELECT query, "int" is a column name which is used together with "t.".

Illegal Identifiers

Beginning with special characters or numbers

An identifier starting with a special character or a number is not allowed. As an exception, a underline (_) and a sharp symbol (#) are allowed for the first character.

```
_a
#ack
%nums
2fer
88abs
```

An identifier containing a space

An identifier that a space within characters is not allowed.

```
col1 t1
```

An identifier containing operator special characters

An identifier which contains operator special characters (+, -, *, /, %, ||, !, < , > , =, |, ^, & , ~) is not allowed.

```
col+
col~
col& &
```

The maximum length of an identifier name

The following table summarizes the maximum byte length allowable for each identifier name. Note that the unit is byte and the number of characters and the bytes are different by the character set used (for example, the length of one Korean character in UTF-8 is 3 bytes).

Identifier	Maximum Bytes
Database	17
User	31
Table	222
Column	254
Index	254
Constraint	254
Java Stored Procedure	254
Trigger	222
View	222
Serial	222

Note

Prior to version 11.2, the maximum length of table, trigger, and serial names were 254 bytes. Since version 11.2, table, trigger, and serial names contain schema names. Therefore, identifier

names cannot exceed 222 bytes except for 31 bytes, which is the length of the owner name, and 1 byte, which is the length of the delimiter (.).

Note

Automatically created constraint name like a name of primary key(pk_<table_name>_<column_name>) or foreign key(fk_<table_name>_<column_name>) also does not allow over the maximum name length of the identifier, 254 bytes.

Reserved Words

The following keywords are previously reserved as a command, a function name or a type name in CUBRID. You are restricted to use these words for a class name, an attribute name, a variable name. Note than these reserved keywords can be used an identifier when they are enclosed in double quotes, square brackets, or backtick symbol (`).

ABSOLUTE	ACTION	ACCESS
ADD	ADD_MONTHS	AFTER
ALL	ALLOCATE	ALTER
AND	ANY	ARE
AS	ASC	ASSERTION
AT	ATTACH	ATTRIBUTE
AVG		
BEFORE	BETWEEN	BIGINT

BINARY	BIT	BIT_LENGTH
BLOB	BOOLEAN	BOTH
BREADTH	BY	
CALL	CASCADE	CASCADED
CASE	CAST	CATALOG
CHANGE	CHAR	CHARACTER
CHECK	CLASS	CLASSES
CLOB	COALESCE	COLLATE
COLLATION	COLUMN	COMMIT
CONNECT	CONNECT_BY_ISCYCLE	CONNECT_BY_ISLEAF
CONNECT_BY_ROOT	CONNECTION	CONSTRAINT
CONSTRAINTS	CONTINUE	CONVERT
CORRESPONDING	COUNT	CREATE
CROSS	CURRENT	CURRENT_DATE
CURRENT_DATETIME	CURRENT_TIME	CURRENT_TIMESTAMP
CURRENT_USER	CURSOR	CYCLE

DATA	DATA_TYPE	DATABASE
DATE	DATETIME	DAY
DAY_HOUR	DAY_MILLISECOND	DAY_MINUTE
DAY_SECOND	DEALLOCATE	DEC
DECIMAL	DECLARE	DEFAULT
DEFERRABLE	DEFERRED	DELETE
DEPTH	DESC	DESCRIBE
DESCRIPTOR	DIAGNOSTICS	DIFFERENCE
DISCONNECT	DISTINCT	DISTINCTROW
DIV	DO	DOMAIN
DOUBLE	DUPLICATE	DROP
EACH	ELSE	ELSEIF
END	EQUALS	ESCAPE
EVALUATE	EXCEPT	EXCEPTION
EXEC	EXECUTE	EXISTS
EXTERNAL	EXTRACT	

FALSE	FETCH	FILE
FIRST	FLOAT	FOR
FOREIGN	FOUND	FROM
FULL	FUNCTION	
GENERAL	GET	GLOBAL
GO	GOTO	GRANT
GROUP		
HAVING	HOUR	HOUR_MILLISECOND
HOUR_MINUTE	HOUR_SECOND	
IDENTITY	IF	IGNORE
IMMEDIATE	IN	INDEX
INDICATOR	INHERIT	INITIALLY
INNER	INOUT	INPUT
INSERT	INT	INTEGER
INTERSECT	INTERSECTION	INTERVAL
INTO	IS	ISOLATION

JOIN		
KEY		
LANGUAGE	LAST	LEADING
LEAVE	LEFT	LESS
LEVEL	LIKE	LIMIT
LIST	LOCAL	LOCAL_TRANSACTION_ID
LOCALTIME	LOCALTIMESTAMP	LOOP
LOWER		
MATCH	MAX	METHOD
MILLISECOND	MIN	MINUTE
MINUTE_MILLISECOND	MINUTE_SECOND	MOD
MODIFY	MODULE	MONTH
MULTISET	MULTISET_OF	
NA	NAMES	NATIONAL
NATURAL	NCHAR	NEXT
NO	NONE	NOT

NULL	NULLIF	NUMERIC
OBJECT	OCTET_LENGTH	OF
OFF	ON	ONLY
OPTIMIZATION	OPTION	OR
ORDER	OUT	OUTER
OUTPUT	OVERLAPS	
PARAMETERS	PARTIAL	POSITION
PRECISION	PREPARE	PRESERVE
PRIMARY	PRIOR	PRIVILEGES
PROCEDURE		
QUERY		
READ	REAL	RECURSIVE
REF	REFERENCES	REFERENCING
RELATIVE	RENAME	REPLACE
RESIGNAL	RESTRICT	RETURN
RETURNS	REVOKE	RIGHT

ROLE	ROLLBACK	ROLLUP
ROUTINE	ROW	ROWNUM
ROWS		
SAVEPOINT	SCHEMA	SCOPE
SCROLL	SEARCH	SECOND
SECOND_MILLISECOND	SECTION	SELECT
SENSITIVE	SEQUENCE	SEQUENCE_OF
SERIALIZABLE	SESSION	SESSION_USER
SET	SET_OF	SETEQ
SHARED	SIBLINGS	SIGNAL
SIMILAR	SIZE	SMALLINT
SOME	SQL	SQLCODE
SQLERROR	SQLEXCEPTION	SQLSTATE
SQLWARNING	STATISTICS	STRING
SUBCLASS	SUBSET	SUBSETEQ
SUBSTRING	SUM	SUPERCLASS

SUPERSET	SUPERSETEQ	SYS_CONNECT_BY_PATH
SYS_DATE	SYS_DATETIME	SYS_TIME
SYS_TIMESTAMP	SYSDATE	SYSDATETIME
SYSTEM_USER	SYSTIME	
TABLE	TEMPORARY	THEN
TIME	TIMESTAMP	TIMEZONE_HOUR
TIMEZONE_MINUTE	TO	TRAILING
TRANSACTION	TRANSLATE	TRANSLATION
TRIGGER	TRIM	TRUE
TRUNCATE		
UNDER	UNION	UNIQUE
UNKNOWN	UPDATE	UPPER
USAGE	USE	USER
USING	UTIME	
VALUE	VALUES	VARCHAR
VARIABLE	VARYING	VCLASS

VIEW		
WHEN	WHENEVER	WHERE
WHILE	WITH	WITHOUT
WORK	WRITE	
XOR		
YEAR	YEAR_MONTH	
ZONE		

Comment

There are 3 types of comments. One is a SQL-style which starts with '--', the other is C++ style starts with '//'. Both regards the entire line starts with the comment symbols is a comment line. The C style starts with '/*' and ends with '*/'

The following are examples of comments.

- How to use --

```
-- This is a SQL-style comment.
```

- How to use //

```
// This is a C++ style comment.
```

- How to use /* */

```
/* This is a C-style comment.*/

/* This is an example to use two lines
as comment by using the C-style. */
```

Literal

This section describes how to write a literal value in CUBRID.

Number

There are two ways of writing a number; how to write an exact value and how to write an approximate value.

- An exact number is written as a serial numbers and a dot (.); this input literal is translated as an INT, BIGINT or NUMERIC typed value based on its range.

```
10, 123456789012, 1234234324234.23
```

- An approximate number is written as a serial numbers, a dot (.) and E (scientific notation, multiples of 10); this input literal is translated as a DOUBLE typed value.

```
1.2345E15, 12345E5
```

- ◦ or - symbol can be written in front of a number, and this can be written between E which indicates multiples of 10 and a number.

```
+10.2345, -1.2345E-15
```

Date/Time

For representing data and time, there are DATE, TIME, DATETIME and TIMESTAMP types; these values can be represented as adding date, time, datetime and timestamp literals (case-insensitive) in front of their strings.

If you use date/time literals, you don't need to use converting functions such as `TO_DATE()`, `TO_TIME()`, `TO_DATETIME()` and `TO_TIMESTAMP()`. However, the writing order of a string which indicates date or time.

- The date literal only allows 'YYYY-MM-DD' or 'MM/DD/YYYY'.

```
date'1974-12-31', date'12/31/1974'
```

- The time literal only allows 'HH:MI:SS', 'HH:MI:SS AM' or 'HH:MI:SS PM'.

```
time'12:13:25', time'12:13:25 AM', time'12:13:25 PM'
```

- The date/time literal used in DATETIME type allows 'YYYY-MM-DD HH:MI:SS[.msec AM|PM]' or 'MM/DD/YYYY HH:MI:SS[.msec AM|PM]'. msec means milliseconds, which can be written until 3 digits.

```
datetime'1974-12-31 12:13:25.123 AM', datetime'12/31/1974
12:13:25.123 AM'
```

- The date/time literal used in TIMESTAMP type allows 'YYYY-MM-DD HH:MI:SS[AM|PM]' or 'MM/DD/YYYY HH:MI:SS[AM|PM]'.

```
timestamp'1974-12-31 12:13:25 AM', timestamp'12/31/1974 12:13:25 AM'
```

- The literal of date/time with timezone type has the same format as the above, and add an offset or a region name which indicates a timezone information.
  - Add datetimetz, datetimeltz, timestamptz or timestampltz literal at the front of a string to represent each type's value.

```
datetimetz'10/15/1986 5:45:15.135 am +02:30:20';
datetimetz'10/15/1986 5:45:15.135 am +02:30';
datetimetz'10/15/1986 5:45:15.135 am +02';
datetimeltz'10/15/1986 5:45:15.135 am Europe/Bucharest'
datetimetz'2001-10-11 02:03:04 AM Europe/Bucharest EEST';
timestampltz'10/15/1986 5:45:15 am Europe/Bucharest'
timestamptz'10/15/1986 5:45:15 am Europe/Bucharest'
```

  - The literal at the front of a string can be replaced with "<date/time type> WITH TIMEZONE" or <date/time type> WITH LOCAL TIME ZONE.

    ::

    DATETIME WITH TIMEZONE = datetimetz DATETIME WITH LOCAL TIMEZONE = datetimeltz TIMESTAMP WITH TIMEZONE = timestamptz TIMESTAMP WITH LOCAL TIMEZONE = timestampltz

```
DATETIME WITH TIME ZONE'10/15/1986 5:45:15.135 am +02';
DATETIME WITH LOCAL TIME ZONE'10/15/1986 5:45:15.135 am +02';
```

Note

- <date/time type> WITH LOCAL TIME ZONE: internally stores UTC time; it is converted as a local (current session) timezone when it is output.
- <date/time type> WITH TIME ZONE: internally stores UTC time and timezone information (decided by a user or a session timezone) when this value is created.

Bit String

Bit string uses two formats of binary format and hexadecimal format.

Binary format is written as adding B or 0b in front of a number; the input value is a string with 0 and 1 after B, and a number with 0 and 1 after 0b.

```
B'10100000'
0b10100000
```

Binary number is written by 8 digits; if the input digits are not divided into 8, this value is saved as 0s are attached. For example, B'1' is saved as B'10000000'.

Hexadecimal format is written as adding X or 0x in front of a number; the input value is a string with hexadecimal after X, and a number with hexadecimal after 0x.

```
X'a0'
0xA0
```

Hexadecimal number is written by 2 digits; if the input digits are not divided into 2, this value is saved as 0s are attached. For example, X'a' is saved as X'a0'.

Character String

Character string is written as wrapped in single quotes.

- If you want to include a single quote in a string, input it twice serially.

```
SELECT 'You''re welcome.';
```

- An escape using a backslash can be used if you set **no_backslash_escapes** in **cubrid.conf** as no. But this default value is yes.

For details, see Escape Special Characters.

- Charset introducer can be located in front of a string, and COLLATE modifier can be located after a string.

  For details, see Charset Introducer.

Collection

In collection types, there are SET, MULTISET and LIST; their values are written as elements are wrapped in braces ({, }).

```
{'c','c','c','b','b','a'}
```

For details, see Collection Types.

NULL

NULL value means there is no data. NULL is case-insensitive, so it also can be written as null. Please note that NULL value is not 0 in a number type or an empty string ('') in a string type.

Data Types

Data Types

Numeric Types

CUBRID supports the following numeric data types to store integers or real numbers.

Type	Bytes	Min	Max	Exact/approx.
SHORT, SMALLINT	2	-32,768	32,767	exact numeric
INTEGER, INT	4	-2,147,483,648	+2,147,483,647	exact numeric
BIGINT	8			exact numeric

Type	Bytes	Min	Max	Exact/approx.
		-9,223,372,036,854,775,808	+9,223,372,036,854,775,807	
NUMERIC, DECIMAL	16	precision *p*: 1 scale *s*: 0	precision *p*: 38 scale *s*: 38	exact numeric
FLOAT, REAL	4	-3.402823466E+38 (ANSI/IEEE 754-1985 standard)	+3.402823466E+38 (ANSI/IEEE 754-1985 standard)	approximate numeric floating point : 7
DOUBLE, DOUBLE PRECISION	8	-1.7976931348623157E+308 (ANSI/IEEE 754-1985 standard)	+1.7976931348623157E+308 (ANSI/IEEE 754-1985 standard)	approximate numeric floating point : 15

Numeric data types are divided into exact and approximate types. Exact numeric data types (**SMALLINT**, **INT**, **BIGINT**, **NUMERIC**) are used for numbers whose values must be precise and consistent, such as the numbers used in financial accounting. Note that even when the literal values are equal, approximate numeric data types (**FLOAT**, **DOUBLE**) can be interpreted differently depending on the system.

CUBRID does not support the UNSIGNED type for numeric data types.

On the above table, two types on the same cell are identical types but it always prints the above type name when you execute SHOW COLUMNS statement. For example, you can use both **SHORT** and **SMALLINT** when you create a table, but it prints "SHORT" when you execute SHOW COLUMNS statement.

Precision and Scale

The precision of numeric data types is defined as the number of significant figures. This applies to both exact and approximate numeric data types.

The scale represents the number of digits following the decimal point. It is significant only in exact numeric data types. Attributes declared as exact numeric data types always have fixed precision and scale. **NUMERIC** (or **DECIMAL**) data type always has at

least one-digit precision, and the scale should be between 0 and the precision declared. Scale cannot be greater than precision. For **INTEGER**, **SMALLINT**, or **BIGINT** data types, the scale is 0 (i.e. no digits following the decimal point), and the precision is fixed by the system.

Numeric Literals

Special signs can be used to input numeric values. The plus sign (+) and minus sign (-) are used to represent positive and negative numbers respectively. You can also use scientific notations. In addition, you can use currency signs specified in the system to represent currency values. The maximum precision that can be expressed by a numeric literal is 255.

Numeric Coercions

All numeric data type values can be compared with each other. To do this, automatic coercion to the common numeric data type is performed. For explicit coercion, use the **CAST** operator. When different data types are sorted or calculated in a numerical expression, the system performs automatic coercion. For example, when adding a **FLOAT** attribute value to an **INTEGER** attribute value, the system automatically coerces the **INTEGER** value to the most approximate **FLOAT** value before it performs the addition operation.

The following is an example to print out the value of **FLOAT** type when adding the value of **INTEGER** type to the value of **FLOAT** type.

```
CREATE TABLE tbl (a INT, b FLOAT);
INSERT INTO tbl VALUES (10, 5.5);
SELECT a + b FROM tbl;
```

```
1.550000e+01
```

This is an example of overflow error occurred when adding two integer values, the following can be an **INTEGER** type value for the result.

```
SELECT 100000000*1000000;
```

```
ERROR: Data overflow on data type integer.
```

In the above case, if you specify one of two integers as the **BIGINT** type, it will determine the result value into the **BIGINT** type, and then output the normal result.

```
SELECT CAST(100000000 AS BIGINT)*1000000;
```

```
100000000000000
```

> **Warning**
>
> Earlier version than CUBRID 2008 R2.0, the input constant value exceeds **INTEGER**, it is handled as **NUMERIC**. However, 2008 R2.0 or later versions, it is handled as **BIGINT** .

INT/INTEGER

The **INTEGER** data type is used to represent integers. The value range is available is from -2,147,483,648 to +2,147,483,647. **SMALLINT** is used for small integers, and **BIGINT** is used for big integers.

- If a real number is entered for an **INT** type, the number is rounded to zero decimal place and the integer value is stored.
- **INTEGER** and **INT** are used interchangeably.
- **DEFAULT** constraint can be specified in a column of this type.

```
If you specify 8934 as INTEGER, 8934 is stored.
If you specify 7823467 as INTEGER, 7823467 is stored.
If you specify 89.8 to an INTEGER, 90 is stored (all digits after
the decimal point are rounded).
If you specify 3458901122 as INTEGER, an error occurs (if the
allowable limit is exceeded).
```

SHORT/SMALLINT

The **SMALLINT** data type is used to represent a small integer type. The value range is available is from -32,768 to +32,767.

- If a real number is entered for an **SMALLINT** type, the number is rounded to zero decimal place and the integer value is stored.
- **SMALLINT** and **SHORT** are used interchangeably.
- **DEFAULT** constraint can be specified in a column of this type.

```
If you specify 8934 as SMALLINT, 8934 is stored.
If you specify 34.5 as SMALLINT, 35 is stored (all digits after the
decimal point are rounded).
If you specify 23467 as SMALLINT, 23467 is stored.
If you specify 89354 as SMALLINT, an error occurs (if the allowable
limit is exceeded).
```

BIGINT

The **BIGINT** data type is used to represent big integers. The value range is available from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.

- If a real number is entered for a **BIG** type, the number is rounded to zero decimal place and the integer value is stored.
- Based on the precision and the range of representation, the following order is applied.

  SMALLINT ??**INTEGER** ??**BIGINT** ??**NUMERIC**
- **DEFAULT** constraint can be specified in a column of this type.

```
If you specify 8934 as BIGINT, 8934 is stored.
If you specify 89.1 as BIGINT, 89 is stored.
If you specify 89.8 as BIGINT, 90 is stored (all digits after the
decimal point are rounded).
If you specify 3458901122 as BIGINT, 3458901122 is stored.
```

NUMERIC/DECIMAL

NUMERIC or **DECIMAL** data types are used to represent fixed-point numbers. As an option, the total number of digits (precision) and the number of digits after the decimal point (scale) can be specified for definition. The minimum value for the precision *p* is 1. When the precision *p* is omitted, you cannot enter data whose integer part exceeds 15 digits because the default value is 15. If the scale *s* is omitted, an integer rounded to the first digit after the decimal point is returned because the default value is 0.

```
NUMERIC [(p[, s])]
```

- Precision must be equal to or greater than scale.
- Precision must be equal to or greater than the number of integer digits + scale.
- **NUMERIC**, **DECIMAL**, and **DEC** are used interchangeably.

- To check how the precision and the scale became changed when you operate with **NUMERIC** typed values, see Arithmetic Operations and Type Casting of Numeric Data Types.

- **DEFAULT** constraint can be specified in a column of this type.

```
If you specify 12345.6789 as NUMERIC, 12346 is stored (it rounds to
the first place after the decimal point since 0 is the default value
of scale).
If you specify 12345.6789 as NUMERIC(4), an error occurs (precision
must be equal to or greater than the number of integer digits).
If you declare NUMERIC(3,4), an error occurs (precision must be
equal to or greater than the scale).
If you specify 0.12345678 as NUMERIC(4,4), .1235 is stored (it
rounds to the fifth place after the decimal point).
If you specify -0.123456789 as NUMERIC(4,4), -.1235 is stored (it
rounds to the fifth place after decimal point and then prefixes a
minus (-) sign).
```

FLOAT/REAL

The **FLOAT** (or **REAL**) data type represents floating point numbers.

The ranges of values that can be described as normalized values are from -3.402823466E+38 to -1.175494351E-38, 0, and from +1.175494351E-38 to +3.402823466E+38, whereas the values other than normalized values, which are closer to 0, are described as de-normalized values. It conforms to the ANSI/IEEE 754-1985 standard.

The minimum value for the precision *p* is 1 and the maximum value is 38. When the precision *p* is omitted or it is specified as seven or less, it is represented as single precision (in 7 significant figures). If the precision *p* is greater than 7 and equal to or less than 38, it is represented as double precision (in 15 significant figures) and it is converted into **DOUBLE** data type.

FLOAT data types must not be used if you want to store a precise value that exceeds the number of significant figures, as they only store the approximate value of any input value over 7 significant figures.

```
FLOAT[(p)]
```

- **FLOAT** is in 7 significant figures.

- Extra cautions are required when comparing data because the **FLOAT** type stores approximate numeric.

- **FLOAT** and **REAL** are used interchangeably.

- **DEFAULT** constraint can be specified in a column of this type.

```
If you specify 16777217 as FLOAT, 16777216 is stored and
1.677722e+07 is displayed (if precision is omitted, 8-th digit is
rounded up because it is represented as 7 significant figures).
If you specify 16777217 as FLOAT(5), 16777216 is stored and
1.677722e+07 is displayed (if precision is in seven or less, 8-th
digit is rounded up because it is represented as 7 significant
figures).
If you specify 16777.217 as FLOAT(5), 16777.216 is stored and
1.677722e+04 is displayed (if precision is in seven or less, 8-th
digit is rounded up because it is represented as 7 significant
figures).
If you specify 16777.217 as FLOAT(10), 16777.217 is stored and
1.6777217000000000e+04 is displayed (if precision is greater than 7
and less than or equal to 38, zeroes are added because it is
represented as 15 significant figures).
```

DOUBLE/DOUBLE PRECISION

The **DOUBLE** data type is used to represent floating point numbers.

The ranges of values that can be described as normalized values are from -1.7976931348623157E+308 to -2.2250738585072014E-308, 0, and from 2.2250738585072014E-308 to 1.7976931348623157E+308, whereas the values other than normalized values, which are closer to 0, are described as de-normalized values. It conforms to the ANSI/IEEE 754-1985 standard.

The precision *p* is not specified. The data specified as this data type is represented as double precision (in 15 significant figures).

DOUBLE data types must not be used if you want to store a precise value that exceeds the number of significant figures, as they only store the approximate value of any input value over 15 significant figures.

- **DOUBLE** is in 15 significant figures.
- Extra caution is required when comparing data because the **DOUBLE** type stores approximate numeric.
- **DOUBLE** and **DOUBLE PRECISION** are used interchangeably.
- **DEFAULT** constraint can be specified in a column of this type.

```
If you specify 1234.56789 as DOUBLE, 1234.56789 is stored and
1.2345678900000000e+03 is displayed.
If you specify 9007199254740993 as DOUBLE, 9007199254740992 is
stored and 9.007199254740992e+15 is displayed.
```

Note

MONETARY type is deprecated, and it is not recommended anymore.

Date/Time Types

Date/time data types are used to represent the date or time (or both together). CUBRID supports the following data types:

Type	bytes	Min.	Max.	Note
DATE	4	0001-01-01	9999-12-31	As an exception, DATE '0000-00-00' format is allowed.
TIME	4	00:00:00	23:59:59	
TIMESTAMP	4	1970-01-01 00:00:01 (GMT) 1970-01-01 09:00:01 (KST)	2038-01-19 03:14:07 (GMT) 2038-01-19 12:14:07 (KST)	As an exception, TIMESTAMP '0000-00-00 00:00:00' format is allowed.
DATETIME	8	0001-01-01 00:00:0.000	9999-12-31 23:59:59.999	As an exception, DATETIME '0000-00-00 00:00:00' format is allowed.

Type	bytes	Min.	Max.	Note
TIMESTAMPLTZ	4	Depends on timezone 1970-01-01 00:00:01 (GMT)	Depends on timezone 2038-01-19 03:14:07 (GMT)	Timestamp with local timezone. As an exception, TIMESTAMPLTZ'0000-00-00 00:00:00' format is allowed.
TIMESTAMPTZ	8	Depends on timezone 1970-01-01 00:00:01 (GMT)	Depends on timezone 2038-01-19 03:14:07 (GMT)	Timestamp with timezone. As an exception, TIMESTAMPTZ '0000-00-00 00:00:00' format is allowed.
DATETIMELTZ	8	Depends on timezone 0001-01-01 00:00:0.000 UTC	Depends on timezone 9999-12-31 23:59:59.999	Datetime with local timezone. As an exception, DATETIMELTZ '0000-00-00 00:00:00' format is allowed.
DATETIMETZ	12	Depends on timezone 0001-01-01 00:00:0.000 UTC	Depends on timezone 9999-12-31 23:59:59.999	Datetime with timezone. As an exception, DATETIMETZ '0000-00-00 00:00:00'

Type	bytes	Min.	Max.	Note
				format is allowed.

Range and Resolution

- By default, the range of a time value is represented by the 24-hour system. Dates follow the Gregorian calendar. An error occurs if a value that does not meet these two constraints is entered as a date or time.
- The range of year in **DATE** is 0001 - 9999 AD.
- From the CUBRID 2008 R3.0 version, if time value is represented with two-digit numbers, a number from 00 to 69 is converted into a number from 2000 to 2069; a number from 70 to 99 is converted into a number from 1970 to 1999. In earlier than CUBRID 2008 R3.0 version, if time value is represented with two-digit numbers, a number from 01 to 99 is converted into a number from 0001 to 0099.
- The range of **TIMESTAMP** is between 1970-01-01 00:00:01 and 2038-01-19 03 03:14:07 (GMT). For KST (GMT+9), values from 1970-01-01 09:00:01 to 2038-01-19 12:14:07 can be stored. timestamp'1970-01-01 00:00:00' (GMT) is the same as timestamp'0000-00-00 00:00:00'.
- The range of **TIMESTAMPLTZ**, **TIMESTAMPTZ** varies with timezone, but the value converted to UTC should be between 1970-01-01 00:00:01 and 2038-01-19 03 03:14:07.
- The range of **DATETIMELTZ**, **DATETIMETZ** varies with timezone, but the value converted to UTC should be between 0001-01-01 00:00:0.000 and 9999-12-31 23:59:59.999. A value stored in database may no longer be valid if session timezone changes.
- The results of date, time and timestamp operations may depend on the rounding mode. In these cases, for Time and Timestamp, the most approximate second is used as the minimum resolution; for Date, the most approximate date is used as the minimum resolution.

Coercions

The **Date** / **Time** types can be cast explicitly using the **CAST** operator only when they have the same field. For implicit coercion, see Implicit Type Conversion. The following table shows types that allows explicit coercions. For implicit coercion, see Arithmetic Operations and Type Casting of DATE/TIME Data Types.

Explicit Coercions

FROM \ TO	DATE	TIME	DATETIME	TIMESTAMP
DATE	-	X	O	O
TIME	X	-	X	X
DATETIME	O	O	-	O
TIMESTAMP	O	O	O	-

In general, zero is not allowed in **DATE**, **DATETIME**, and **TIMESTAMP** types. However, if both date and time values are 0, it is allowed as an exception. This is useful in terms that this value can be used if an index exists upon query execution of a column corresponding to the type.

- Some functions in which the **DATE**, **DATETIME**, and **TIMESTAMP** types are specified as an argument return different value based on the **return_null_on_function_errors** system parameter if every input argument value for date and time is 0. If **return_null_on_function_errors** is yes, **NULL** is returned; if no, an error is returned. The default value is **no**.
- The functions that return **DATE**, **DATETIME**, and **TIMESTAMP** types can return a value of 0 for date and time. However, these values cannot be stored in Date objects in Java applications. Therefore, it will be processed with one of the following based on the configuration of zeroDateTimeBehavior, the connection URL property: being handled as an exception, returning **NULL**, or returning a minimum value (see Configuration Connection).
- If the **intl_date_lang** system is configured, input string of `TO_DATE()`, `TO_TIME()`, `TO_DATETIME()`, `TO_TIMESTAMP()`, `DATE_FORMAT()`, `TIME_FORMAT()`, `TO_CHAR()` and `STR_TO_DATE()` functions follows the corresponding locale date format. For details, see Statement/Type-Related Parameters and the description of each function.
- Types with timezone follow the same conversion rules as their parent type.

Note

For literals of date/time types and date/time types with timezone, see Date/Time.

DATE

The **DATE** data type is used to represent the year (yyyy), month (mm) and day (dd). Supported range is "01/01/0001" to "12/31/9999." The year can be omitted. If it is, the year value of the current system is specified automatically. The specified input/output types are as follows:

```
date'mm/dd[/yyyy]'
date'[yyyy-]mm-dd'
```

- All fields must be entered as integer.
- The date value is displayed in the type of 'MM/DD/YYYY' in CSQL, and it is displayed in the type of 'YYYY-MM-DD' in JDBC application programs and the CUBRID Manager.
- You can store the date value of the system in the **DATE** type by using the `SYS_DATE` (or `SYSDATE`, `CURRENT_DATE`).
- The `TO_DATE()` function is used to convert a character string type into a **DATE** type.
- 0 is not allowed to input in year, month, and day; however, '0000-00-00', which every digit consisting of year, month, and day is 0, is allowed as an exception.
- If the year is two digits and the value is greater than or equal to 70, it is recognized as the 1900s, otherwise it is recognized as the 2000s.
- **DEFAULT** constraint can be specified in a column of this type.

```
DATE'2008-10-31' is displayed as '10/31/2008'.
DATE'10/31' is displayed as '10/31/2011'(if a value for year is
omitted, the current year is automatically specified).
DATE'00-10-31' is displayed as '10/31/2000'.
DATE'0000-10-31' is displayed as an error (a year value should be at
least 1).
DATE'70-10-31' is displayed as '10/31/1970'.
DATE'0070-10-31' is displayed as '10/31/0070'.
```

TIME

The **TIME** data type is used to represent the hour (hh), minute (mm) and second (ss). Supported range is "00:00:00" to "23:59:59." Second can be omitted; if it is, 0 seconds is specified. Both 12-hour and 24-hour notations are allowed as an input format. The input format of **TIME** is as follows:

```
time'hh:mi[:ss] [am | pm]'
```

- All items must be entered as integer.

- AM/PM time notation is used to display time in the CSQL; while the 24-hour notation is used in the CUBRID Manager.
- AM/PM can be specified in the 24-hour notation. An error occurs if the time specified does not follow the AM/PM format.
- Every time value is stored in the 24-hour notation.
- You can store the time value of the system in the **TIME** type by using the `SYS_TIME` (or `SYSTIME`, `CURRENT_TIME`).
- The `TO_TIME()` function is used to return a character string type into a TIME type.
- **DEFAULT** constraint can be specified in a column of this type.

```
TIME'00:00:00' is outputted as '12:00:00 AM'.
TIME'1:15' is regarded as '01:15:00 AM'.
TIME'13:15:45' is regarded as '01:15:45 PM'.
TIME'13:15:45 pm' is stored normally.
TIME'13:15:45 am' is an error (an input value does not match the AM/
PM format).
```

TIMESTAMP

The **TIMESTAMP** data type is used to represent a data value in which the date (year, month, date) and time (hour, minute, second) are combined. The range of representable value is between GMT '1970-01-01 00:00:01' and '2038-01-19 03:14:07'. The **DATETIME** type can be used if the value is out of range or data in milliseconds is stored. The input format of **TIMESTAMP** is as follows:

```
timestamp'hh:mi[:ss] [am|pm] mm/dd[/yyyy]'
timestamp'hh:mi[:ss] [am|pm] [yyyy-]mm-dd'

timestamp'mm/dd[/yyyy] hh:mi[:ss] [am|pm]'
timestamp'[yyyy-]mm-dd hh:mi[:ss] [am|pm]'
```

- All fields must be entered in integer format.
- If the year is omitted, the current year is specified by default. If the time value (hour/minute/second) is omitted, 12:00:00 AM is specified.
- You can store the timestamp value of the system in the **TIMESTAMP** type by using the `SYS_TIMESTAMP` (or `SYSTIMESTAMP`, `CURRENT_TIMESTAMP`).
- The `TIMESTAMP()` or `TO_TIMESTAMP()` function is used to cast a character string type into a **TIMESTAMP** type.

- 0 is not allowed to input in year, month, and day; however, '0000-00-00 00:00:00', which every digit consisting of year, month, day, hour, minute, and second is 0, is allowed as an exception. GMT timestamp'1970-01-01 12:00:00 AM' or KST timestamp'1970-01-01 09:00:00 AM' is translated into timestamp'0000-00-00 00:00:00'.

- **DEFAULT** constraint can be specified in a column of this type.

```
TIMESTAMP'10/31' is outputted as '12:00:00 AM 10/31/2011' (if the
value for year/time is omitted, a default value is outputted ).
TIMESTAMP'10/31/2008' is outputted as '12:00:00 AM 10/31/2008' (if
the value for time is omitted, a default value is outputted ).
TIMESTAMP'13:15:45 10/31/2008' is outputted as '01:15:45 PM
10/31/2008'.
TIMESTAMP'01:15:45 PM 2008-10-31' is outputted as '01:15:45 PM
10/31/2008'.
TIMESTAMP'13:15:45 2008-10-31' is outputted as '01:15:45 PM
10/31/2008'.
TIMESTAMP'10/31/2008 01:15:45 PM' is outputted as '01:15:45 PM
10/31/2008'.
TIMESTAMP'10/31/2008 13:15:45' is outputted as '01:15:45 PM
10/31/2008'.
TIMESTAMP'2008-10-31 01:15:45 PM' is outputted as '01:15:45 PM
10/31/2008'.
TIMESTAMP'2008-10-31 13:15:45' is outputted as '01:15:45 PM
10/31/2008'.
An error occurs on TIMESTAMP '2099-10-31 01:15:45 PM' (out of range
to represent TIMESTAMP).
```

DATETIME

The **DATETIME** data type is used to represent a data value in which the data (year, month, date) and time (hour, minute, second) are combined. The range of representable value is between 0001-01-01 00:00:00.000 and 9999-12-31 23:59:59.999 (GMT). The input format of **TIMESTAMP** is as follows:

```
datetime'hh:mi[:ss[.msec]] [am|pm] mm/dd[/yyyy]'
datetime'hh:mi[:ss[.msec]] [am|pm] [yyyy-]mm-dd'
datetime'mm/dd[/yyyy] hh:mi[:ss[.ff]] [am|pm]'
datetime'[yyyy-]mm-dd hh:mi[:ss[.ff]] [am|pm]'
```

- All fields must be entered as integer.

- If you year is omitted, the current year is specified by default. If the value (hour, minute/second) is omitted, 12:00:00.000 AM is specified.

- You can store the timestamp value of the system in the **DATETIME** type by using the `SYS_DATETIME` (or `SYSDATETIME`, `CURRENT_DATETIME`, `CURRENT_DATETIME()`, `NOW()`) function.
- The `TO_DATETIME()` function is used to convert a string type into a **DATETIME** type.
- 0 is not allowed to input in year, month, and day; however, '0000-00-00 00:00:00', which every digit consisting of year, month, day, hour, minute, and second is 0, is allowed as an exception.
- **DEFAULT** constraint can be specified in a column of this type.

```
DATETIME'10/31' is outputted as '12:00:00.000 AM 10/31/2011' (if the
value for year/time is omitted, a default value is outputted).
DATETIME'10/31/2008' is outputted as '12:00:00.000 AM 10/31/2008'.
DATETIME'13:15:45 10/31/2008' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'01:15:45 PM 2008-10-31' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'13:15:45 2008-10-31' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'10/31/2008 01:15:45 PM' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'10/31/2008 13:15:45' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'2008-10-31 01:15:45 PM' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'2008-10-31 13:15:45' is outputted as '01:15:45.000 PM
10/31/2008'.
DATETIME'2099-10-31 01:15:45 PM' is outputted as '01:15:45.000 PM
10/31/2099'.
```

CASTing a String to Date/Time Type

Recommended Format for Strings in Date/Time Type

When you casting a string to Date/Time type by using the `CAST()` function, it is recommended to write the string in the following format: Note that date/time string formats used in the `CAST()` function are not affected by locale which is specified by creating DB.

Also, in `TO_DATE()`, `TO_TIME()`, `TO_DATETIME()`, `TO_TIMESTAMP()` functions, when date/time format is omitted, write the date/time string in the following format.

- **DATE** Type

```
YYYY-MM-DD
MM/DD/YYYY
```

- **TIME** Type

```
HH:MI:SS [AM|PM]
```

- **DATETIME** Type

```
YYYY-MM-DD HH:MI:SS[.msec] [AM|PM]
HH:MI:SS[.msec] [AM|PM] YYYY-MM-DD

MM/DD/YYYY HH:MI:SS[.msec] [AM|PM]
HH:MI:SS[.msec] [AM|PM] MM/DD/YYYY
```

- **TIMESTAMP** Type

```
YYYY-MM-DD HH:MI:SS [AM|PM]
HH:MI:SS [AM|PM] YYYY-MM-DD

MM/DD/YYYY HH:MI:SS [AM|PM]
HH:MI:SS [AM|PM] MM/DD/YYYY
```

Available Format for Strings in Date/Time Type

`CAST()` function allows the below format for date/time strings.

Available DATE String Format

```
[year sep] month sep day
```

- 2011-04-20: April 20th, 2011
- 04-20: April 20th of this year

If a separator (*sep*) is a slash (/), strings are recognized in the following order:

```
month/day[/year]
```

- 04/20/2011: April 20th, 2011
- 04/20: April 20th of this year

If you do not use a separator (*sep*), strings are recognized in the following format. It is allowed to use 1, 2, and 4 digits for years and 1 and 2 digits for months. For day, you should always enter 2 digits.

```
YYYYMMDD
YYMMDD
YMMDD
MMDD
MDD
```

- 20110420: April 20th, 2011
- 110420: April 20th, 2011
- 420: April 20th of this year

Available TIME String Format

```
[hour]:min[:[sec]][.[msec]] [am|pm]
```

- 09:10:15.359 am: 9 hours 10 minutes 15 seconds AM (0.359 seconds will be truncated)
- 09:10:15: 9 hours 10 minutes 15 seconds AM
- 09:10: 9 hours 10 minutes AM
- :10: 12 hours 10 minutes AM

```
[[[[[[Y]Y]Y]Y]M]MDD]HHMISS[.[msec]] [am|pm]
```

- 20110420091015.359 am: 9 hours 10 minutes 15 seconds AM
- 0420091015: 9 hours 10 minutes 15 seconds AM

```
[H]HMMSS[.[msec]] [am|pm]
```

- 091015.359 am: 9 hours 10 minutes 15 seconds AM
- 91015: 9 hours 10 minutes 15 seconds AM

```
[M]MSS[.[msec]] [am|pm]
```

- 1015.359 am: 12 hours 10 minutes 15 seconds AM
- 1015: 12 hours 10 minutes 15 seconds AM

```
[S]S[.[msec]] [am|pm]
```

- 15.359 am: 12 hours 15 seconds AM
- 15: 12 hours 15 seconds AM

> **Note**
>
> : The [H]H format was allowed in CUBRID 2008 R3.1 and the earlier versions. That is, the string '10' was converted to **TIME** '10:00:00' in the R3.1 and the earlier versions, and will be converted to **TIME** '00:00:10' in version R4.0 and later.

Available DATETIME String Format

```
[year sep] month sep day [sep] [sep] hour [sep min[sep sec[.
[msec]]]]
```

- 04-20 09: April 20th of this year, 9 hours AM

```
month/day[/year] [sep] hour [sep min [sep sec[.[msec]]]]
```

- 04/20 09: April 20th of this year, 9 hours AM

```
year sep month sep day sep hour [sep min[sep sec[.[msec]]]]
```

- 2011-04-20 09: April 20th, 2011, 9 hours AM

```
month/day/year sep hour [sep min[sep sec [.[msec]]]]
```

- 04/20/2011 09: April 20th, 2011, 9 hours AM

```
YYMMDDH (It is allowed only when time format is one digit.)
```

- 1104209: April 20th, 2011, 9 hours AM

```
YYMMDDHHMI[SS[.msec]]
```

- 1104200910.359: April 20th, 2011, 9 hours 10 minutes AM (0.359 seconds will be truncated)
- 110420091000.359: April 20th, 2011, 9 hours 10 minutes 0.359 seconds AM

```
YYYYMMDDHHMISS[.msec]
```

- 201104200910.359: November 4th, 2020 8 hours 9 minutes 10.359 seconds PM
- 20110420091000.359: April 20th, 2011, 9 hours 10 minutes 0.359 seconds AM

Available Time-Date String Format

```
[hour]:min[:sec[.msec]] [am|pm] [year-]month-day
```

- 09:10:15.359 am 2011-04-20: April 20th, 2011, 9 hours 10 minutes 15.359 seconds AM
- :10 04-20: April 20th of this year, 12 hours 10 minutes AM

```
[hour]:min[:sec[.msec]] [am|pm] month/day[/[year]]
```

- 09:10:15.359 am 04/20/2011: April 20th, 2011, 9 hours 10 minutes 15.359 seconds AM
- :10 04/20: April 20th of this year, 12 hours 10 minutes AM

```
hour[:min[:sec[.[msec]]]] [am|pm] [year-]month-day
```

- 09:10:15.359 am 04-20: April 20th of this year, 9 hours 10 minutes 15.359 seconds AM
- 09 04-20: April 20th of this year, 9 hours AM

```
hour[:min[:sec[.[msec]]]] [am|pm] month/day[/[year]]
```

- 09:10:15.359 am 04/20: April 20th of this year, 9 hours 10 minutes, 15.359 seconds AM
- 09 04/20: April 20th of this year, 9 hours AM

Rules

msec is a series of numbers representing milliseconds. The numbers after the fourth digit will be ignored. The rules for the separator string are as follows:

- You should always use one colon (:) as a separator for the **TIME** separator.
- **DATE** and **DATETIME** strings can be represented as a series of numbers without the separator sep), and non-alphanumeric characters can be used as separators. The **DATETIME** string can be divided into Time and Date with a space.
- Separators should be identical in the input string.
- For the Time-Date string, you can only use colon (:) for a Time separator and hyphen (-) or slash (/) for a Date separator. If you use a hyphen when entering date, you should enter like yyyy-mm-dd; in case of a slash, enter like mm/dd/yyyy.

The following rules will be applied in the part of date.

- You can omit the year as long as the syntax allows it.
- If you enter the year as two digits, it represents the range from 1970-2069. That is, if YY<70, it is treated as 2000+YY; if YY>=70, it is treated as 1900+YY. If you enter one, three or four digit numbers for the year, the numbers will be represented as they are.
- A space before and after a string and the string next to the space are ignored. The am/pm identifier for the **DATETIME** and **TIME** strings can be recognized as part of TIME value, but are not recognized as the am/pm identifier if non-space characters are added to it.

The **TIMESTAMP** type of CUBRID consists of **DATE** type and **TIME** type, and **DATETIME** type consists of **DATE** type and **TIME** type with milliseconds being added to them. Input strings can include Date (**DATE** string), Time (**TIME** string), or both (**DATETIME** strings). You can convert a string including a specific type of data to another type, and the following rules will be applied for the conversion.

- If you convert the **DATE** string to the **DATETIME** type, the time value will be '00:00:00.'
- If you convert the **TIME** string to the **DATETIME** type, colon (:) is recognized as a date separator, so that the **TIME** string can be recognized as a date string and the time value will be '00:00:00.'
- If you convert the **DATETIME** string to the **DATE** type, the time part will be ignored from the result but the time input value format should be valid.

- You can covert the **DATETIME** string to the **TIME** type, and you must follow the following rules.
  - The date and time in the string must be divided by at least one blank.
  - The date part of the result value is ignored but the date input value format should be valid.
  - The year in the date part must be over 4 digits (available to start with 0) or the time part must include hours and minutes ([H]H:[M]M) at least. Otherwise the date pate are recognized as the TIME type of the [MM]SS format, and the following string will be ignored.
- If the one of the units (year, month, date, hour, minute and second) of the **DATETIME** string is greater than 999999, it is not recognized as a number, so the string including the corresponding unit will be ignored. For example, in '2009-10-21 20:9943:10', an error occurs because the value in minutes is out of the range. However, if '2009-10-21 20:1000123:10' is entered,'2009' is recognized as the **TIME** type of the MMSS format, so that **TIME** '00:20:09' will be returned.
- If you convert the time-date sting to the **TIME** type, the date part of the string is ignored but the date part format must be valid.
- All input strings including the time part allow *[.msec]* on conversion, but only the **DATETIME** type can be maintained. If you convert this to a type such as **DATE**, **TIMESTAMP** or **TIME**, the *msec* value is discarded.
- All conversions in the **DATETIME**, **TIME** string allow English locale following after time value or am/pm specifier written in the current locale of a server.

```
SELECT CAST('420' AS DATE);
```

```
   cast('420' as date)
======================
  04/20/2012
```

```
SELECT CAST('91015' AS TIME);
```

```
   cast('91015' as time)
========================
  09:10:15 AM
```

```
SELECT CAST('110420091035.359' AS DATETIME);
```

```
   cast('110420091035.359' as datetime)
=======================================
  09:10:35.359 AM 04/20/2011
```

```
SELECT CAST('110420091035.359' AS TIMESTAMP);
```

```
   cast('110420091035.359' as timestamp)
=======================================
  09:10:35 AM 04/20/2011
```

Date/Time Types with Timezone

Date/Time types with timezone are date/time types which can be input or output by specifying timezone. There are two ways of specifying timezone; specifying the name of local zone and specifying the offset of time.

Timezone information are considered in the Date/Time types if TZ or LTZ is followed after the existing Date/Time types; TZ means timezone, and LTZ means local timezone.

- TZ type can be represented as <date/time type> WITH TIME ZONE. This stores UTC time and timezone information (decided by a user or session timezone) when this is created. TZ type requires 4 bytes more to store timezone information.
- LTZ type can be represented as <date/time type> WITH LOCAL TIME ZONE. This stores UTC time internally; when this value is output, this is converted as a value of a local (current session) timezone.

This table describes date/time types to compare date/time types with timezone together.

UTC in the table means Coordinated Universal Time.

Category	Type	Input	Store	Output	Description
DATE	DATE	Without timezone	Input value	Absolute (the same as input)	Date

Category	Type	Input	Store	Output	Description
DATETIME	DATETIME	Without timezone	Input value	Absolute (the same as input)	Date/time including milliseconds
	DATETIMETZ	With timezone	UTC + timezone(region or offset)	Absolute (keep input timezone)	Date/time + timezone
	DATETIMELTZ	With timezone	UTC	Relative (transformed by session timezone)	Date/time in the session timezone
TIME	TIME	Without timezone	Input value	Absolute (the same as input)	Time
TIMESTAMP	TIMESTAMP	Without timezone	UTC	Relative (transformed by session timezone)	Input value is translated as a session timezone's value.
	TIMESTAMPTZ	With timezone	UTC + timezone(region or offset)	Absolute (keep input timezone)	UTC + timestamp with timezone
	TIMESTAMPLTZ	With timezone	UTC	Relative (transformed by	Session timezone. Same as TIMESTAMP's value, but

Category	Type	Input	Store	Output	Description
				session timezone)	timezone specifier is output when this is printed out.

The other features of date/time types with timezone (e.g. maximum/minimum value, range, resolution) are the same with the features of general date/time types.

Note

- On CUBRID, TIMESTAMP is stored as second unit, after Jan. 1, 1970 UTC (UNIX epoch).
- Some DBMS's TIMESTAMP is similar to CUBRID's DATETIME as the respect of saving milliseconds.

To see examples of functions using timezone types, see Date/Time Functions and Operators.

The following shows that the output values are different among DATETIME, DATETIMETZ and DATETIMELTZ when session timezone is changed.

```
--  csql> ;set timezone="+09"

CREATE TABLE tbl (a DATETIME, b DATETIMETZ,  c DATETIMELTZ);
INSERT INTO tbl VALUES (datetime'2015-02-24 12:30',
datetimetz'2015-02-24 12:30', datetimeltz'2015-02-24 12:30');

SELECT * FROM tbl;
```

```
12:30:00.000 PM 02/24/2015     12:30:00.000 PM 02/24/2015 +09:
00                12:30:00.000 PM 02/24/2015 +09:00
```

```
-- csql> ;set timezone="+07"

SELECT * FROM tbl;
```

```
12:30:00.000 PM 02/24/2015     12:30:00.000 PM 02/24/2015 +09:
00                10:30:00.000 AM 02/24/2015 +07:00
```

The following shows that the output values are different among TIMESTAMP, TIMESTAMPTZ and TIMESTAMPLTZ when session timezone is changed.

```
-- ;set timezone="+09"

CREATE TABLE tbl (a TIMESTAMP, b TIMESTAMPTZ,  c TIMESTAMPLTZ);
INSERT INTO tbl VALUES (timestamp'2015-02-24 12:30',
timestamptz'2015-02-24 12:30', timestampltz'2015-02-24 12:30');

SELECT * FROM tbl;
```

```
12:30:00 PM 02/24/2015     12:30:00 PM 02/24/2015 +09:
00                12:30:00 PM 02/24/2015 +09:00
```

```
-- csql> ;set timezone="+07"

SELECT * FROM tbl;
```

```
10:30:00 AM 02/24/2015     12:30:00 PM 02/24/2015 +09:
00                10:30:00 AM 02/24/2015 +07:00
```

Conversion from string to timestamp types

Conversion from string to timestamp/timestampltz/timestamptz are performed in context for creating timestamp objects from literals.

From/to	Timestamp	Timestampltz	Timestamptz
String (without timezone)	Interpret the date/time parts in session timezone. Convert to UTC, encode and store the Unix epoch.	Interpret the date/time parts in session timezone. Convert to UTC, encode and store the Unix epoch.	Interpret the date/time parts in session timezone. Convert to UTC, encode and store the Unix epoch and TZ_ID of session

From/to	Timestamp	Timestampltz	Timestamptz
String (with timezone)	Error (timezone part is not supported for timestamp).	Convert from value's timezone to UTC. Encode and store the Unix epoch.	Convert from value's timezone to UTC. Encode and store the Unix epoch and TZ_ID of value's timezone.

Conversion from string to datetime types

Conversion from string to datetime/datetimeltz/datetimetz are performed in context for creating datetime objects from literals.

From/to	Datetime	Datetimeltz	Datetimetz
String (without timezone)	Store the parsed values from string.	Interpret the date/time parts in session timezone. Convert to UTC and store the new values.	Interpret the date/time parts in session timezone. Convert to UTC and store the new values and TZ_ID of session
String (with timezone)	Error (timezone part is not supported for datetime).	Convert from value's timezone to UTC. Store the new values in UTC reference.	Convert from value's timezone to UTC. Store the new values in UTC reference TZ_ID of string's timezone.

Conversion of datetime and timestamp types to string (printing of values)

From/to	String (timezone printing not allowed)	String (timezone force print)	String (no requirement for timezone - free choice)
TIMESTAMP	Decode Unix epoch to session timezone and print	Decode Unix epoch to session timezone and print with session timezone.	Decode Unix epoch to session timezone and print. Do not print timezone string
TIMESTAMPLTZ	Decode Unix epoch to session timezone and print	Decode Unix epoch to session timezone and print with session timezone.	Decode Unix epoch to session timezone and print. Print session timezone.
TIMESTAMPTZ	Decode Unix epoch to timezone from value and print it.	Decode Unix epoch to timezone from value and print it; print timezone from value.	Decode Unix epoch to timezone from value and print it; print timezone from value.
DATETIME	Print the stored values.	Print the stored value and session timezone.	Print the stored value. Do not print any timezone.
DATETIMELTZ	Convert from UTC to session timezone and print the new value.	Convert from UTC to session timezone and print it. Print session timezone	Convert from UTC to session timezone and print it. Print session timezone.
DATETIMELTZ	Convert from UTC to value's timezone and print the new value.	Convert from UTC to value's timezone and print it. Print value's timezone	Convert from UTC to value's timezone and print it. Print value's timezone.

Timezone Configuration

The below shows the timezone related parameters configured in cubrid.conf. For parameter's configuration, see cubrid.conf Configuration File and Default Parameters.

- **timezone**

  Specifies a timezone for a session. The default is a value of **server_timezone**.

- **server_timezone**

  Specifies a timezone for a server. The default is a timezone of OS.

- **tz_leap_second_support**

  Sets for support for leap second as yes or no. The default is no.

Timezone Function

The following are timezone related functions. For each function's detail usage, click each function's name.

- `DBTIMEZONE()`
- `SESSIONTIMEZONE()`
- `FROM_TZ()`
- `NEW_TIME()`
- `TZ_OFFSET()`

Functions with a Timezone Type

All functions which use DATETIME, TIMESTAMP or TIME typed value in their input value, can use timezone typed value.

The below is an example of using timezone typed values, it works the same as the case without timezone. Exceptionally, if the type name ends with LTZ, the output value of this type follows the local timezone's setting (timezone parameter).

On the below example, the default unit of a number is millisecond, which is the minimum unit of DATETIME type.

```
SELECT datetimeltz '09/01/2009 03:30:30 pm' + 1;
```

```
03:30:30.001 PM 09/01/2009 Asia/Seoul
```

```
SELECT datetimeltz '09/01/2009 03:30:30 pm' - 1;
```

```
03:30:29.999 PM 09/01/2009 Asia/Seoul
```

On the below example, the default unit of a number is second, which is the minimum unit of TIMESTAMP type.

```
SELECT timestamptz '09/01/2009 03:30:30 pm' + 1;
```

```
03:30:31 PM 09/01/2009 Asia/Seoul
```

```
SELECT timestamptz '09/01/2009 03:30:30 pm' - 1;
```

```
03:30:29 PM 09/01/2009 Asia/Seoul
```

```
SELECT EXTRACT (hour from datetimetz'10/15/1986 5:45:15.135 am
Europe/Bucharest');

5
```

A type which the name ends with LTZ follows the setting of local timezone. Therefore, if the value of timezone parameter is set to 'Asia/Seoul', EXTRACT function returns hour value of this timezone.

```
-- csql> ;set timezone='Asia/Seoul'

SELECT EXTRACT (hour from datetimeltz'10/15/1986 5:45:15.135 am
Europe/Bucharest');
```

```
12
```

Conversion Functions for Timezone Types

The following are functions converting a string to a date/time typed value, or date/time typed value to a string; The value can include an information like an offset, a zone and a daylight saving.

- `DATE_FORMAT()`
- `STR_TO_DATE()`
- `TO_CHAR()`
- `TO_DATETIME_TZ()`
- `TO_TIMESTAMP_TZ()`

For each function's usage, see the each function's explanation by clicking the function name.

```
SELECT DATE_FORMAT (datetimetz'2012-02-02 10:10:10 Europe/Zurich
CET', '%TZR %TZD %TZH %TZM');
SELECT STR_TO_DATE ('2001-10-11 02:03:04 AM Europe/Bucharest EEST',
'%Y-%m-%d %h:%i:%s %p %TZR %TZD');
SELECT TO_CHAR (datetimetz'2001-10-11 02:03:04 AM Europe/Bucharest
EEST');
SELECT TO_DATETIME_TZ ('2001-10-11 02:03:04 AM Europe/Bucharest
EEST');
SELECT TO_TIMESTAMP_TZ ('2001-10-11 02:03:04 AM Europe/Bucharest');
```

Note

`TO_TIMESTAMP_TZ()` and `TO_DATETIME_TZ()` functions do the same behaviors with `TO_TIMESTAMP()` and `TO_DATETIME()` functions except that they can have TZR, TZD, TZH and TZM information in their date/time argument.

CUBRID uses the region name of timezone in the IANA(Internet Assigned Numbers Authority) timezone database region; for IANA timezone, see http://www.iana.org/time-zones.

IANA Timezone

In IANA(Internet Assigned Numbers Authority) timezone database, there are lots of codes and data which represent the history of localtime for many representative locations around the globe.

This database is periodically updated to reflect changes made by political bodies to timezone boundaries, UTC offsets, and daylight-saving rules. Its management procedure is described in BCP 175: Procedures for Maintaining the Time Zone Database. For more details, see http://www.iana.org/time-zones.

CUBRID supports IANA timezone, and a user can use the IANA timezone library in the CUBRID installation package as it is. If you want to update as the recent timezone, update timezone first, compile timezone library, and restart the database.

Regarding this, see Compiling Timezone Library.

Bit Strings

A bit string is a sequence of bits (1's and 0's). Images (bitmaps) displayed on the computer screen can be stored as bit strings. CUBRID supports the following two types of bit strings:

- Fixed-length bit string (**BIT**)
- Variable-length bit string (**BIT VARYING**)

A bit string can be used as a method argument or an attribute type. Bit string literals are represented in a binary or hexadecimal format. For binary format, append the string consisting of 0's and 1's to the letter **B** or append a value to the **0b** as shown example below.

```
B'1010'
0b1010
```

For hexadecimal format, append the string consisting of the numbers 0 - 9 and the letters A - F to the uppercase letter **X** or append a value to the **0x** . The following is hexadecimal representation of the same number that was represented above in binary format.

```
X'a'
0xA
```

The letters used in hexadecimal numbers are not case-sensitive. That is, X'4f' and X'4F' are considered as the same value.

Length

If a bit string is used in table attributes or method declarations, you must specify the maximum length. The maximum length for a bit string is 1,073,741,823 bits.

Bit String Coercion

Automatic coercion is performed between a fixed-length and a variable-length bit string for comparison. For explicit coercion, use the `CAST()` operator.

BIT(n)

Fixed-length binary or hexadecimal bit strings are represented as **BIT** (*n*), where *n* is the maximum number of bits. If *n* is not specified, the length is set to 1. If *n* is not specified, the length is set to 1. The bit string is filled with 8-bit unit from the left side. For example, the value of B'1' is the same as the value of B'10000000'. Therefore, it is recommended to declare a length by 8-bit unit, and input a value by 8-bit unit.

Note

If you input B'1' to the BIT(4) column, it is printed out X'8' on CSQL, X'80' on CUBRID Manager or application program.

- *n* must be a number greater than 0.
- If the length of the string exceeds *n*, it is truncated and filled with 0s.
- If a bit string smaller than *n* is stored, the remainder of the string is filled with 0s.
- **DEFAULT** constraint can be specified in a column of this type.

```
CREATE TABLE bit_tbl(a1 BIT, a2 BIT(1), a3 BIT(8), a4 BIT VARYING);
INSERT INTO bit_tbl VALUES (B'1', B'1', B'1', B'1');
INSERT INTO bit_tbl VALUES (0b1, 0b1, 0b1, 0b1);
INSERT INTO bit_tbl(a3,a4) VALUES (B'1010', B'1010');
INSERT INTO bit_tbl(a3,a4) VALUES (0xaa, 0xaa);
SELECT * FROM bit_tbl;
```

```
  a1                    a2                    a3
a4

======================================================================
====
  X'8'                  X'8'                  X'80'
X'8'
  X'8'                  X'8'                  X'80'
X'8'
  NULL                  NULL                  X'a0'
X'a'
  NULL                  NULL                  X'aa'
X'aa'
```

BIT VARYING(n)

A variable-length bit string is represented as **BIT VARYING** (*n*), where *n* is the maximum number of bits. If *n* is not specified, the length is set to 1,073,741,823 (maximum value). *n* is the maximum number of bits. If *n* is not specified, the maximum length is set to 1,073,741,823. The bit string is filled with 8-bit values from the left side. For example, the value of B'1' is the same as the value of B'10000000'. Therefore, it is recommended to declare a length by 8-bit unit, and input a value by 8-bit unit.

Note

If you input B'1' to the BIT VARYING(4) column, it is printed out X'8' on CSQL, X'80' on CUBRID Manager or application program.

- If the length of the string exceeds *n*, it is truncated and filled with 0s.
- The remainder of the string is not filled with 0s even if a bit string smaller than *n* is stored.
- *n* must be a number greater than 0.
- **DEFAULT** constraint can be specified in a column of this type.

```
CREATE TABLE bitvar_tbl(a1 BIT VARYING, a2 BIT VARYING(8));
INSERT INTO bitvar_tbl VALUES (B'1', B'1');
INSERT INTO bitvar_tbl VALUES (0b1010, 0b1010);
INSERT INTO bitvar_tbl VALUES (0xaa, 0xaa);
INSERT INTO bitvar_tbl(a1) VALUES (0xaaa);
INSERT INTO bitvar_tbl(a2) VALUES (0xaaa);
SELECT * FROM bitvar_tbl;
```

```
  a1                    a2
============================================
  X'8'                  X'8'
  X'a'                  X'a'
  X'aa'                 X'aa'
  X'aaa'                NULL
  NULL                  X'aa'
```

Character Strings

CUBRID supports the following two types of character strings:

- Fixed-length character string: **CHAR** (*n*)

- Variable-length character string: **VARCHAR** (*n*)

> **Note**
>
> From CUBRID 9.0 version, **NCHAR** and **NCHAR VARYING** is no more supported. Instead, please use **CHAR** and **VARCHAR**.

The following are the rules that are applied when using the character string types.

- In general, single quotations are used to enclose character string. Double quotations may be used as well depending on the value of **ansi_quotes**, which is a parameter related to SQL statement. If the **ansi_quotes** value is set to **no**, character string enclosed by double quotations is handled as character string, not as an identifier. The default value is **yes**. For details, Statement/Type-Related Parameters.

- If there are characters that can be considered to be blank (e.g. spaces, tabs, or line breaks) between two character strings, these two character strings are treated as one according to ANSI standard. For example, the following example shows that a line break exists between two character strings.

```
'abc'
'def'
```

  The above two strings and the below string are considered identical.

```
'abcdef'
```

- If you want to include a single quote as part of a character string, enter two single quotes in a row. For example, the character string on the left is stored as the one on the right.

```
'''abcde''fghij'        'abcde'fghij
```

- The maximum size of the token for all the character strings is 16 KB.

- To enter the language of a specific country, we recommend that you to specify the locale when creating DB, then you can change locale by the introducer **CHARSET** (or **COLLATE** modifier). For more information, see An Overview of Globalization.

Length

Specify the number of a character string.

When the length of the character string entered exceeds the length specified, the excess characters may be truncated in the insert/update operation if the allow_truncated_string configuration value is "yes" otherwise error occurs.

For a fixed-length character string type such as **CHAR**, the length is fixed at the declared length. Therefore, the right part (trailing space) of the character string is filled with space characters when the string is stored. For a variable-length character string type such as **VARCHAR**, only the entered character string is stored, and the space is not filled with space characters.

The maximum length of a **CHAR** type to be specified is 268,435,455. The maximum length of a **VARCHAR** type to be specified is 1,073,741,823.

Also, the maximum length that can be input or output in a CSQL statement is 8,192 KB.

Note

In the CUBRID version less than 9.0, the length of **CHAR** or **VARCHAR** was not the number of characters, but the byte size.

Character Set, charset

A character set (charset) is a set in which rules are defined that relate to what kind of codes can be used for encoding when specified characters (symbols) are stored in the computer. The character used by CUBRID can be configured as the **CUBRID_CHARSET** environment variable. For details, see An Overview of Globalization.

Collating Character Sets

A collation is a set of rules used for comparing characters to search or sort values stored in the database when a certain character set is specified. For details, see An Overview of Globalization.

Character String Coercion

Automatic coercion takes place between a fixed-length and a variable-length character string for the comparison of two characters, applicable only to characters that belong to the same character set.

For example, when you extract a column value from a **CHAR** (5) data type and insert it into a column with a **CHAR** (10) data type, the data type is automatically coerced to **CHAR** (10). If you want to coerce a character string explicitly, use the **CAST** operator (See `CAST()`).

String compression

Variable character type values (VARCHAR(n)) may be compressed (using LZO1X algorithm) before being stored in database (heap file, index file or list file). Compression is attempted if size in bytes is at least 255 bytes (this value is predefined and cannot be changed). If the compression is not efficient (compressed value size and its overhead is equal or greater than the original uncompressed value), the value is stored uncompressed. Compression is activated by default and may be disabled by setting the system parameter enable_string_compression. The overhead of compression is eight bytes : four for size of compressed buffer and four for the size of expected uncompressed string. Compressed strings are decompressed when they are read from database. To determine if a value is compressed or not, one may use the DISK_SIZE function result and compare it with the result of OCTET_LENGTH function on the same argument. A smaller value for DISK_SIZE (ignoring the value overhead) indicates that compression is used.

CHAR(n)

A fixed-length character string is represented as **CHAR** *(n)*, in which *n* represents the number of characters. If *n* is not specified, the value is specified as 1, default value.

When the length of a character string exceeds *n*, they may be truncated in the insert/update operation if the allow_truncated_string configuration value is "yes" otherwise error occurs. When character string which is shorter than *n* is stored, whitespace characters are used to fill up the trailing space.

CHAR (*n*) and **CHARACTER** (*n*) are used interchangeably.

Note

In the earlier versions of CUBRID 9.0, *n* represents byte length, not the number of characters.

- *n* specifies the number of bytes, ranging from 1 to 2048, and defines a fixed-length character type that allocates storage of the specified size.
- Empty quotes (' ') are used to represent a blank string. In this case, the return value of the **LENGTH** function is not 0, but is the fixed length defined in **CHAR** (*n*). That is, if you enter a blank string into a column with **CHAR** (10), the **LENGTH** is 10; if you enter a blank value into a **CHAR** with no length specified, the **LENGTH** is the default value 1.
- Space characters used as filling characters are considered to be smaller than any other characters, including special characters.

```
If you specify 'pacesetter' as CHAR(12), 'pacesetter ' is stored (a
10-character string plus two white space characters).
```

```
If you specify 'pacesetter ' as CHAR(10), 'pacesetter' is stored (a
10-character string; two white space characters are truncated).
If you specify 'pacesetter' as CHAR(4), 'pace' may be stored or
error occurs depending on the configuration value of
allow_truncated_string (truncated as the length of the character
string is greater than 4).
If you specify 'p ' as CHAR, 'p' is stored (if n is not specified,
the length is set to the default value 1).
```

- **DEFAULT** constraint can be specified in a column of this type.

VARCHAR(n)/CHAR VARYING(n)

Variable-length character strings are represented as **VARCHAR** (*n*), where *n* represents the number of characters. If *n* is not specified, the value is specified as 1,073,741,823, the maximum length.

When the length of a character string exceeds *n*, they may be truncated in the insert/update operation if the allow_truncated_string configuration value is **yes** or error occurs if not. When character string which is shorter than *n* is stored, for **VARCHAR** (*n*), the length of string used are stored without any trailing spaces.

VARCHAR (*n*), **CHARACTER, VARYING** (*n*), and **CHAR VARYING** (*n*) are used interchangeably.

Note

In the earlier versions of CUBRID 9.0, *n* represents byte length, not the number of characters.

- **STRING** is the same as the **VARCHAR** (maximum length).
- *n* is an integer between 1 and 1,073,741,823 (1G).
- Empty quotes (' ') are used to represent a blank string. In this case, the return value of the **LENGTH** function is not 0.

```
If you specify 'pacesetter' as VARCHAR(4), 'pace' may be stored or
error occurs depending on the configuration value of
allow_truncated_string (truncated as the length of the character
string is greater than 4).
If you specify 'pacesetter' as VARCHAR(12), 'pacesetter' is stored
(a 10-character string).
If you specify 'pacesetter ' as VARCHAR(12), 'pacesetter ' is stored
(a 11-character string).
If you specify 'pacesetter ' as VARCHAR(10), 'pacesetter' may be
stored or error occurs depending on the configuration value of
```

```
allow_truncated_string (a 10-character string; two white space
characters can be truncated).
If you specify 'p ' as VARCHAR, 'p ' is stored (if n is not
specified, the default value 1,073,741,823 is used, and the trailing
space is not filled with white space characters).
```

- **DEFAULT** constraint can be specified in a column of this type.

STRING

STRING is a variable-length character string data type. **STRING** is the same as the VARCHAR with the length specified as the maximum value. That is, **STRING** and **VARCHAR** (1,073,741,823) have the same value.

Escape Special Characters

CUBRID supports two kinds of methods to escape special characters. One is using quotes and the other is using backslash (\).

- Escape with Quotes

  If you set **no** for the system parameter **ansi_quotes** in the **cubrid.conf** file, you can use both double quotes (") and singe quotes (') to wrap strings. The default value for the **ansi_quotes** parameter is **yes**, and you can use only single quotes to wrap the string.

  - You should use two single quotes ('') for the single quotes included in the strings wrapped in single quotes.
  - You should use two double quotes ("") for the double quotes included in the strings wrapped in double quotes. (when **ansi_quotes** = **no**)
  - You don't need to escape the single quotes included in the string wrapped in double quotes. (when **ansi_quotes** = **no**)
  - You don't need to escape the double quotes included in the string wrapped in single quotes.

- Escape with Backslash

  You can use escape using backslash (\) only if you set no for the system parameter **no_backslash_escapes** in the **cubrid.conf** file. The default value for the **no_backslash_escapes** parameter is **yes**. If the value of **no_backslash_escapes** is **no**, the following are the special characters.

  - \' : Single quotes (')
  - \" : Double quotes (")

- \n : Newline, linefeed character
- \r : Carriage return character
- \t : Tab character
- \\ : Backslash
- \% : Percent sign (%). For details, see the following description.
- _ : Underbar (_). For details, see the following description.

For all other escapes, the backslash will be ignored. For example, "x" is the same as entering only "x".

\% and **_** are used in the pattern matching syntax such as **LIKE** to search percent signs and underbars and are used as a wildcard character if there is no backslash. Outside of the pattern matching syntax, "\%"and "_" are recognized as normal strings not wildcard characters. For details, see LIKE.

The following is the result of executing Escape if a value for the system parameter **ansi_quotes** in the **cubrid.conf** file is yes(default), and a value for **no_backslash_escapes** is no.

```
-- ansi_quotes=yes, no_backslash_escapes=no
SELECT STRCMP('single quotes test('')', 'single quotes test(\')');
```

If you run the above query, backslash is regarded as an escape character. Therefore, above two strings are the same.

```
   strcmp('single quotes test('')', 'single quotes test('')')
=============================================================
                                                           0
```

```
SELECT
STRCMP('\a\b\c\d\e\f\g\h\i\j\k\l\m\n\o\p\q\r\s\t\u\v\w\x\y\z',
'a\bcdefghijklm\nopq\rs\tuvwxyz');
```

If you run the above query, backslash is regarded as an escape character. Therefore, above two strings are the same.

```
   strcmp('abcdefghijklm
s       uvwxyz', 'abcdefghijklm
s       uvwxyz')
=============================================================
                                                           0
```

```
SELECT LENGTH('\\');
```

If you run the above query, backslash is regarded as an escape character. Therefore, the length of above string is 1.

```
   char_length('\')
====================
                   1
```

The following is the result of executing Escape if a value for the system parameter **ansi_quotes** in the **cubrid.conf** file is yes(default), and a value for **no_backslash_escapes** is yes(default). Backslash character is regarded as a general character.

```
-- ansi_quotes=yes, no_backslash_escapes=yes

SELECT STRCMP('single quotes test('')', 'single quotes test(\')');
```

If you run the above query, the quotation mark is regarded as opened, so the below error occurs. If you input this query on the CSQL interpreter's console, it waits the next quotation mark's input.

```
ERROR: syntax error, unexpected UNTERMINATED_STRING, expecting
SELECT or VALUE or VALUES or '('
```

```
SELECT
STRCMP('\a\b\c\d\e\f\g\h\i\j\k\l\m\n\o\p\q\r\s\t\u\v\w\x\y\z',
'a\bcdefghijklm\nopq\rs\tuvwxyz');
```

If you run the above query, backslash is regarded as a general character. Therefore, the result of the comparison between the above two strings shows different.

```
   strcmp('\a\b\c\d\e\f\g\h\i\j\k\l\m\n\o\p\q\r\s\t\u\v\w\x\y\z',
'a\bcdefghijklm\nopq\rs\tuvwxyz')
============================================================================
==============================

                           -1
```

```
SELECT LENGTH('\\');
```

If you run the above query, backslash is regarded as a general character. Therefore, the length of above string is 2.

```
   char_length('\\')
====================
                   2
```

The following shows the result of executing Escape about the LIKE clause when **ansi_quotes** is yes and **no_backslash_escapes** is no.

```
-- ansi_quotes=yes, no_backslash_escapes=no

CREATE TABLE t1 (a VARCHAR(200));
INSERT INTO t1 VALUES ('aaabbb'), ('aaa%');

SELECT a FROM t1 WHERE a LIKE 'aaa\%' ESCAPE '\\';
```

```
  a
======================
  'aaa%'
```

If you run above query, it returns only one row because '%' character is regarded as a general character.

In the string of LIKE clause, backslash is always regarded as a general character. Therefore, if you want to make the '%' character as a general character, not as an pattern matching character, you should specify that '%' is an escape character by using ESCAPE clause. In the ESCAPE clause, backslash is regarded as an escape character. Therefore, we used two backslashes.

If you want use other character than a backslash as an escape character, you can write the query as follows.

```
SELECT a FROM t1 WHERE a LIKE 'aaa#%' ESCAPE '#';
```

Comparison Rules

When two string values are compared, the followings are the comparison rules about the behavior of trailing spaces:

- Comparison for trailing space insensitive
- Comparison for trailing space sensitive

Trailing space insensitive

If the two string values are both fixed-length types (CHAR-type), the comparison ignores trailing spaces as below example. comparing 'abc ' with 'abc ' results equal

Trailing space sensitive

If two string values are both of variable-length types (VARCHAR-type), the comparison does not ignore trailing spaces as below example. comparing 'abc ' with 'abc ' results 'abc ' greater than 'abc '

Exceptions

When comparing two string values, if one is a fixed-type and the other is a variable-type, CUBRID follows "trailing space sensitive" rule.

ENUM Data Type

The **ENUM** type is a data type consisting of an ordered set of distinct constant char literals called enum values. The syntax for creating an enum column is:

```
<enum_type>
    : ENUM '(' <char_string_literal_list> ')'

<char_string_literal_list>
    : <char_string_literal_list> ',' CHAR_STRING
    | CHAR_STRING
```

The following example shows the definition of an **ENUM** column.

```
CREATE TABLE tbl (
    color ENUM ('red', 'yellow', 'blue', 'green')
);
```

- **DEFAULT** constraint can be specified in a column of this type.

An index is associated to each element of the enum set, according to the order in which elements are defined in the enum type. For example, the *color* column can have one of the following values (assuming that the column allows NULL values) :

Value	Index Number
NULL	NULL
'red'	1

Value	Index Number
‘yellow’	2
‘blue’	3
‘green’	4

The set of values of an **ENUM** type must not exceed 512 elements and each element of the set must be unique. CUBRID allocates two bytes of storage for each **ENUM** type value because it only stores the index of each value. This reduces the storage space needed which may improve performance.

Either the enum value or the value index can be used when working with **ENUM** types. For example, to insert values into an **ENUM** type column, users can use either the value or the index of the **ENUM** type:

```
-- insert enum element 'yellow' with index 2
INSERT INTO tbl (color) VALUES ('yellow');
-- insert enum element 'red' with index 1
INSERT INTO tbl (color) VALUES (1);
```

When used in expressions, the **ENUM** type behaves either as a **CHAR** type or as a number, depending on the context in which it is used:

```
-- the first result column has ENUM type, the second has INTEGER
type and the third has VARCHAR type
SELECT color, color + 0, CONCAT(color, '') FROM tbl;
```

```
  color                 color+0   concat(color, '')
=======================================================
  'yellow'                    2   'yellow'
  'red'                       1   'red'
```

When used in type contexts other than **CHAR** or numbers, the enum is coerced to that type using either the index or the enum value. The table below shows which part of an **ENUM** type is used in the coercion:

Type	Enum type (Index/Value)
SHORT	Index
INTEGER	Index
BIGINT	Index
FLOAT	Index
DOUBLE	Index
NUMERIC	Index
TIME	Value
DATE	Value
DATETIME	Value
TIMESTAMP	Value
CHAR	Value
VARCHAR	Value
BIT	Value
VARBIT	Value

ENUM Type Comparisons

When used in **=** or **IN** predicates of the form (<enum_column> <operator> <constant>), CUBRID tries to convert the constant to the **ENUM** type. If the coercion fails, CUBRID does not return an error but considers the comparison to be false. This is implemented like this in order to allow index scan plans to be generated on these two operators.

For all other comparison operators, the **ENUM** type is converted to the type of the other operand. If a comparison is performed on two **ENUM** types, both arguments are converted to **CHAR** type and the comparison follows **CHAR** type rules. Except for **=** and **IN**, predicates on **ENUM** columns cannot be used in index scan plans.

To understand these rules, consider the following table:

```
CREATE TABLE tbl (
    color ENUM ('red', 'yellow', 'blue', 'green')
);

INSERT INTO tbl (color) VALUES (1), (2), (3), (4);
```

The following query will convert the constant 'red' to the enum value 'red' with index 1

```
SELECT color FROM tbl WHERE color = 'red';
```

```
  color
=======================
  'red'
```

```
SELECT color FROM tbl WHERE color = 1;
```

```
  color
=======================
  'red'
```

The following queries will not return a conversion error but will not return any results:

```
SELECT color FROM tbl WHERE color = date'2010-01-01';
SELECT color FROM tbl WHERE color = 15;
SELECT color FROM tbl WHERE color = 'asdf';
```

In the following queries the **ENUM** type will be converted to the type of the other operand:

```
-- CHAR comparison using the enum value
SELECT color FROM tbl WHERE color < 'pink';
```

```
  color
=======
  'blue'
  'green'
```

```
-- INTEGER comparison using the enum index
SELECT color FROM tbl WHERE color > 3;
```

```
  color
=======
  'green'
```

```
-- Conversion error
SELECT color FROM tbl WHERE color > date'2012-01-01';
```

```
ERROR: Cannot coerce value of domain "enum" to domain "date".
```

ENUM Type Ordering

Values of the **ENUM** type are ordered by value index, not by enum value. When defining a column with **ENUM** type, users also define the ordering of the enum values.

```
SELECT color FROM tbl ORDER BY color ASC;
```

```
  color
=======
  'red'
  'yellow'
  'blue'
  'green'
```

To order the values stored in an **ENUM** type column as **CHAR** values, users can cast the enum value to the **CHAR** type:

```
SELECT color FROM tbl ORDER BY CAST (color AS VARCHAR) ASC;
```

```
  color
=======================
  'blue'
  'green'
  'red'
  'yellow'
```

Notes

The **ENUM** type is not a reusable type. If several columns require the same set of values, an **ENUM** type must be defined for each one. When comparing two columns of **ENUM** type, the comparison is performed as if the columns were coerced to **CHAR** type even if the two **ENUM** types define the same set of values.

Using the **ALTER ... CHANGE** statement to modify the set of values of an **ENUM** type is only allowed if the value of the system parameter **alter_table_change_type_strict** is set to yes. In this case, CUBRID uses enum value (the char-literal) to convert values to the new domain. If a value is outside of the new **ENUM** type values set, it is automatically mapped to the empty string(‘’).

```
CREATE TABLE tbl(color ENUM ('red', 'green', 'blue'));
INSERT INTO tbl VALUES('red'), ('green'), ('blue');
```

The following statement will extend the **ENUM** type with the value ‘yellow’:

```
ALTER TABLE tbl CHANGE color color ENUM ('red', 'green', 'blue',
'yellow');
INSERT into tbl VALUES(4);
SELECT color FROM tbl;
```

```
  color
=======================
  'red'
  'green'
  'blue'
  'yellow'
```

The following statement will change all tuples with value ‘green’ to value ‘red’ because the value ‘green’ cannot be converted the new **ENUM** type:

```
ALTER TABLE tbl CHANGE color color enum ('red', 'yellow', 'blue');
SELECT color FROM tbl;
```

```
  color
=======================
  'red'
  ''
  'blue'
  'yellow'
```

The **ENUM** type is mapped to char-string types in CUBRID drivers. The following example shows how to use the **ENUM** type in a JDBC application:

```
Statement stmt = connection.createStatement("SELECT color FROM tbl");
ResultSet rs = stmt.executeQuery();

while(rs.next()) {
   System.out.println(rs.getString());
}
```

The following example shows how to use the **ENUM** type in a CCI application.

```
req_id = cci_prepare (conn, "SELECT color FROM tbl", 0, &err);
error = cci_execute (req_id, 0, 0, &err);
if (error < CCI_ER_NO_ERROR)
{
    /* handle error */
}

error = cci_cursor (req_id, 1, CCI_CURSOR_CURRENT, &err);
if (error < CCI_ER_NO_ERROR)
{
    /* handle error */
}

error = cci_fetch (req_id, &err);
if (error < CCI_ER_NO_ERROR)
{
    /* handle error */
}

cci_get_data (req, idx, CCI_A_TYPE_STR, &data, 1);
```

BLOB/CLOB Data Types

An External **LOB** type is data to process Large Object, such as text or images. When LOB-type data is created and inserted, it will be stored in a file to an external storage, and the location information of the relevant file (**LOB** Locator) will be stored in the CUBRID database. If the **LOB**

Locator is deleted from the database, the relevant file that was stored in the external storage will be deleted as well. CUBRID supports the following two types of **LOB** :

- Binary Large Object (**BLOB**)
- Character Large Object (**CLOB**)

Note

Terminologies

- **LOB** (Large Object): Large-sized objects such as binaries or text.
- **FBO** (File Based Object): An object that stores data of the database in an external file.
- **External LOB**: An object better known as FBO, which stores **LOB** data in a file into an external DB. It is supported by CUBRID. Internal **LOB** is an object that stores **LOB** data inside the DB.
- **External Storage**: An external storage to store LOB (example : POSIX file system).
- **LOB Locator**: The path name of a file stored in external storage.
- **LOB Data**: Details of a file in a specific location of LOB Locator.

When storing LOB data in external storage, the following naming convention will be applied:

```
{table_name}_{unique_name}
```

- *table_name* : It is inserted as a prefix and able to store the **LOB** data of many tables in one external storage.
- *unique_name* : The random name created by the DB server.

LOB data is stored in the local file system of the DB server. LOB data is stored in the path specified in the **-lob-base-path option** value of **cubrid createdb**; if this value is omitted, the data will be stored in the [db-vol path]/lob path where the database volume will be created. For more details, see createdb and To create and manage LOB storage.

If a user change any **LOB** file without using CUBRID API or CUBRID tools, data consistency is not guaranteed.

If a **LOB** data file path that was registered to the database directory file(**databases.txt**) is deleted, please note that database server (**cub_server**) and standalone utilities will not correctly work.

BLOB

A type that stores binary data outside the database. The maximum length of **BLOB** data is the maximum file size creatable in an external storage. In SQL statements, the **BLOB** type expresses the input and output value in a bit string. That is, it is compatible with the **BIT** (n) and **BIT VARYING** (n) types, and only an explicit type change is allowed. If data lengths differ from one another, the maximum length is truncated to fit the smaller one. When converting the **BLOB** type value to a binary value, the length of the converted data cannot exceed 1GB. When converting binary data to the **BLOB** type, the size of the converted data cannot exceed the maximum file size provided by the **BLOB** storage.

CLOB

A type that stores character string data outside the database. The maximum length of **CLOB** data is the maximum file size creatable in an external storage. In SQL statements, the CLOB type expresses the input and output value in a character string. That is, it is compatible with the **CHAR** (n), **VARCHAR** (n) types. However, only an explicit type change is allowed, and if data lengths are different from one another, the maximum length is truncated to fit to the smaller one. When converting the **CLOB** type value to a character string, the length of the converted data cannot exceed 1 GB. When converting a character string to the **CLOB** type, the size of the converted data cannot exceed the maximum file size provided by the **CLOB** storage.

To Create and alter LOB

BLOB / **CLOB** type columns can be created/added/deleted by using a **CREATE TABLE** statement or an **ALTER TABLE** statement.

- You cannot create the index file for a **LOB** type column.
- You cannot define the **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL** constraints for a **LOB** type column. However, **SHARED** property cannot be defined and **DEFAULT** property can only be defined by the **NULL** value.
- **LOB** type column/data cannot be the element of collection type.
- If you are deleting a record containing a **LOB** type column, all files located inside a **LOB** column value (Locator) and the external storage will be deleted. When a record containing a LOB type column is deleted in a basic key table, and a record of a foreign key table that refers to the foregoing details is deleted at once, all **LOB** files located in a **LOB** column value (Locator) and the external storage will be deleted. However, if the relevant table is deleted by using a **DROP TABLE** statement, or a **LOB** column is deleted by using an **ALTER TABLE...DROP** statement, only a **LOB** column value (**LOB** Locator) is deleted, and the **LOB** files inside the external storage which a **LOB** column refers to will not be deleted.

```
-- creating a table and CLOB column
CREATE TABLE doc_t (doc_id VARCHAR(64) PRIMARY KEY, content CLOB);

-- an error occurs when UNIQUE constraint is defined on CLOB column
ALTER TABLE doc_t ADD CONSTRAINT content_unique UNIQUE(content);

-- an error occurs when creating an index on CLOB column
CREATE INDEX i_doc_t_content ON doc_t (content);

-- creating a table and BLOB column
CREATE TABLE image_t (image_id VARCHAR(36) PRIMARY KEY, doc_id
VARCHAR(64) NOT NULL, image BLOB);

-- an error occurs when adding a BOLB column with NOT NULL constraint
ALTER TABLE image_t ADD COLUMN thumbnail BLOB NOT NULL;

-- an error occurs when adding a BLOB column with DEFAULT attribute
ALTER TABLE image_t ADD COLUMN thumbnail2 BLOB DEFAULT
BIT_TO_BLOB(X'010101');
```

To store and update LOB

In a **BLOB** / **CLOB** type column, each **BLOB** / **CLOB** type value is stored, and if binary or character string data is input, you must explicitly change the types by using each `BIT_TO_BLOB()` and `CHAR_TO_CLOB()` function.

If a value is input in a **LOB** column by using an **INSERT** statement, a file is created in an external storage internally and the relevant data is stored; the relevant file path (Locator) is stored in an actual column value.

If a record containing a **LOB** column uses a **DELETE** statement, a file to which the relevant **LOB** column refers will be deleted simultaneously.

If a **LOB** column value is changed using an **UPDATE** statement, the column value will be changed following the operation below, according to whether a new value is **NULL** or not.

- If a **LOB** type column value is changed to a value that is not **NULL** : If a Locator that refers to an external file is already available in a **LOB** column, the relevant file will be deleted. A new file is created afterwards. After storing a value that is not **NULL**, a Locator for a new file will be stored in a **LOB** column value.
- If changing a **LOB** type column value to **NULL** : If a Locator that refers to an external file is already available in a **LOB** column, the relevant file will be deleted. And then **NULL** is stored in a **LOB** column value.

```
-- inserting data after explicit type conversion into CLOB type
column
```

```
INSERT INTO doc_t (doc_id, content) VALUES ('doc-1',
CHAR_TO_CLOB('This is a Dog'));
INSERT INTO doc_t (doc_id, content) VALUES ('doc-2',
CHAR_TO_CLOB('This is a Cat'));

-- inserting data after explicit type conversion into BLOB type
column
INSERT INTO image_t VALUES ('image-0', 'doc-0',
BIT_TO_BLOB(X'000001'));
INSERT INTO image_t VALUES ('image-1', 'doc-1',
BIT_TO_BLOB(X'000010'));
INSERT INTO image_t VALUES ('image-2', 'doc-2',
BIT_TO_BLOB(X'000100'));

-- inserting data from a sub-query result
INSERT INTO image_t SELECT 'image-1010', 'doc-1010', image FROM
image_t WHERE image_id = 'image-0';

-- updating CLOB column value to NULL
UPDATE doc_t SET content = NULL WHERE doc_id = 'doc-1';

-- updating CLOB column value
UPDATE doc_t SET content = CHAR_TO_CLOB('This is a Dog') WHERE
doc_id = 'doc-1';

-- updating BLOB column value
UPDATE image_t SET image = (SELECT image FROM image_t WHERE image_id
= 'image-0') WHERE image_id = 'image-1';

-- deleting BLOB column value and its referencing files
DELETE FROM image_t WHERE image_id = 'image-1010';
```

To access LOB

When you get a **LOB** type column, the data stored in a file to which the column refers will be displayed. You can execute an explicit type change by using `CAST()` operator, `CLOB_TO_CHAR()` and `BLOB_TO_BIT()` function.

- If the query is executed in CSQL, a column value (Locator) will be displayed, instead of the data stored in a file. To display the data to which a **BLOB** / **CLOB** column refers, it must be changed to strings by `CLOB_TO_CHAR()` function.
- To use the string process function, the strings need to be converted by `CLOB_TO_CHAR()` function.
- You cannot specify a **LOB** column in ** GROUP BY** clause and **ORDER BY** clause.
- Comparison operators, relational operators, **IN**, **NOT IN** operators cannot be used to compare **LOB** columns. However, **IS NULL** expression can be used to compare whether it is a **LOB** column

value (Locator) or **NULL**. This means that **TRUE** will be returned when a column value is **NULL**, and if a column value is **NULL**, there is no file to store **LOB** data.

- When a **LOB** column is created, and the file is deleted after data input, a **LOB** column value (Locator) will become a state that is referring to an invalid file. As such, using `CLOB_TO_CHAR()`, `BLOB_TO_BIT()`, `CLOB_LENGTH()` and `BLOB_LENGTH()` functions on the columns that have mismatching **LOB** Locator and a **LOB** data file enables them to display **NULL**.

```
-- displaying locator value when selecting CLOB and BLOB column in
CSQL interpreter
SELECT doc_t.doc_id, content, image FROM doc_t, image_t WHERE
doc_t.doc_id = image_t.doc_id;
```

```
  doc_id                content                image
=============================================================================
  'doc-1'               file:ces_658/doc_t.
00001282208855807171_7329  file:ces_318/image_t.
00001282208855809474_7474
  'doc-2'               file:ces_180/doc_t.
00001282208854194135_5598  file:ces_519/image_t.
00001282208854205773_1215

2 rows selected.
```

```
-- using string functions after coercing its type by CLOB_TO_CHAR( )
SELECT CLOB_TO_CHAR(content), SUBSTRING(CLOB_TO_CHAR(content), 10)
FROM doc_t;
```

```
   clob_to_char(content)  substring( clob_to_char(content) from 10)
=========================================
  'This is a Dog'        ' Dog'
  'This is a Cat'        ' Cat'

2 rows selected.
```

```
SELECT CLOB_TO_CHAR(content) FROM doc_t WHERE CLOB_TO_CHAR(content)
LIKE '%Dog%';
```

```
   clob_to_char(content)
======================
  'This is a Dog'
```

```
SELECT CLOB_TO_CHAR(content) FROM doc_t ORDER BY
CLOB_TO_CHAR(content);
```

```
   clob_to_char(content)
======================
  'This is a Cat'
  'This is a Dog'
```

```
SELECT * FROM doc_t WHERE content LIKE 'This%';
```

```
  doc_id                content
=============================================
  'doc-1'               file:ces_004/doc_t.00001366272829040346_0773
  'doc-2'               file:ces_256/doc_t.00001366272815153996_1229
```

```
-- an error occurs when LOB column specified in ORDER BY/GROUP BY
clauses
SELECT * FROM doc_t ORDER BY content;
```

```
ERROR: doc_t.content can not be an ORDER BY column
```

Note

- From CUBRID 11.4, the locator, which represents the LOB file path, **changed from an absolute file path to a relative one**. The relative path starts from '**lob-base-path**' in databases.txt. With this change, LOB file path management has become more flexible, and LOB file path problems have been reduced during database moves, backups, and restores.
- For example, if the '**lob-base-path**' path defined in databases.txt is /home1/data1, an example of the locator for each CUBRID version is as follows.
  - **CUBRID 11.4** (**relative**): *file:ces_004/doc_t.00001366272829040346_0773*
  - up to CUBRID 11.3 (**absolute**): *file:/home1/data1/ces_004/doc_t.00001366272829040346_0773*
- Due to the locator being stored as a relative path, if '**lob-base-path**' in databases.txt is changed,
  - Previously created LOB data cannot be retrieved.

    - To retrieve previously created LOB data, it should be moved to the updated '**lob-base-path**' directory.

```
# example:
% cp -r /home1/data1/* /home2/data2
```

- LOB data created in CUBRID 11.3 or earlier, even though they store the locator as an absolute file path, can be retrieved in CUBRID 11.4 without any changes.
- Advantages during database moves and restores: Even if the database is moved to another location, there is no need to manually modify the LOB file path because the path is managed based on **lob-base-path**.
- The adoption of the **relative LOB file path** is one of the notable improvements in CUBRID 11.4, and it significantly enhances database management efficiency.

Functions and Operators for LOB

You can explicitly cast bit/string type to **BLOB**/**CLOB** type and **BLOB**/**CLOB** type to bit/string type with `CAST()` operator. For more details, see `CAST()` operator.

```
CAST (<bit_type_column_or_value> AS { BLOB | CLOB })
CAST (<char_type_column_or_value> AS { BLOB | CLOB })
```

These are the functions for BLOB/CLOB types. For more details, refer LOB Functions.

- `CLOB_TO_CHAR()`
- `BLOB_TO_BIT()`
- `CHAR_TO_CLOB()`
- `BIT_TO_BLOB()`
- `CHAR_TO_BLOB()`
- `CLOB_FROM_FILE()`
- `BLOB_FROM_FILE()`
- `CLOB_LENGTH()`
- `BLOB_LENGTH()`

Note

" <*blob_or_clob_column* **IS NULL** ": using **IS NULL** condition, it compares the value of **LOB** column(Locator) if it's **NULL** or not. If it's **NULL**, this condition returns **TRUE**.

To create and manage LOB storage

By default, the **LOB** data file is stored in the <db-volume-path>/lob directory where database volume is created. However, if the lob base path is specified with `createdb -B` option when creating the database, **LOB** data files will be stored in the directory designated. However, if the specified directory does not exist, CUBRID tries to create the directory and display an error message when it fails to create it. For more details, see `createdb -B` option.

```
# image_db volume is created in the current work directory, and a
LOB data file will be stored.
% cubrid createdb image_db en_US

# LOB data file is stored in the "/home1/data1" path within a local
file system.
% cubrid createdb --lob-base-path="file:/home1/data1" image_db en_US
```

You can identify a directory where a LOB file will be stored by executing the cubrid spacedb utility.

```
% cubrid spacedb image_db

Space description for database 'image_db' with pagesize 16.0K. (log
pagesize: 16.0K)

Volid  Purpose  total_size  free_size  Vol Name

    0  GENERIC      512.0M     510.1M  /home1/data1/image_db

Space description for temporary volumes for database 'image_db' with
pagesize 16.0K.

Volid  Purpose  total_size  free_size  Vol Name

LOB space description file:/home1/data1
```

To expand or change the **lob-base-path** of the database, change its **lob-base-path** of **databases.txt** file. Restart the database server to apply the changes made to **databases.txt**.

However, even if you change the **lob-base-path** of **databases.txt**, access to the **LOB** data stored in a previous storage is possible.

```
# You can change to a new directory from the lob-base-path of
databases.txt file.
% cat $CUBRID_DATABASES/databases.txt

#db-name     vol-path           db-host       log-path
lob-base-path
image_db     /home1/data1       localhost     /home1/data1
file:/home1/data2
```

Backup/recovery for data files of **LOB** type columns are not supported, while those for meta data(Locator) are supported.

If you are copying a database by using **copydb** utility, you must configure the **databases.txt** additionally, as the **LOB** file directory path will not be copied if the related option is not specified. For more details, see the `copydb -B` and `copydb --copy-lob-path` options.

Transaction and Recovery

Commit/Rollback for **LOB** data changes are supported. That is, it ensures the validation of mapping between **LOB** Locator and actual **LOB** data within transactions, and it supports recovery during DB errors. This means that an error will be displayed in case of mapping errors between **LOB** Locator and **LOB** data due to the rollback of the relevant transactions, as the database is terminated during transactions. See the example below.

```
-- csql> ;AUTOCOMMIT OFF

CREATE TABLE doc_t (doc_id VARCHAR(64) PRIMARY KEY, content CLOB);
INSERT INTO doc_t VALUES ('doc-10', CHAR_TO_CLOB('This is content'));
COMMIT;
UPDATE doc_t SET content = CHAR_TO_CLOB('This is content 2') WHERE
doc_id = 'doc-10';
ROLLBACK;
SELECT doc_id, CLOB_TO_CHAR(content) FROM doc_t WHERE doc_id =
'doc-10';
```

```
  doc_id   content
============================================================
  'doc-10'  'This is content'
```

```
-- csql> ;AUTOCOMMIT OFF
```

```
INSERT INTO doc_t VALUES ('doc-11', CHAR_TO_CLOB ('This is
content'));
COMMIT;
UPDATE doc_t SET content = CHAR_TO_CLOB('This is content 3') WHERE
doc_id = 'doc-11';

-- system crash occurred and then restart server
SELECT doc_id, CLOB_TO_CHAR(content) FROM doc_t WHERE doc_id =
'doc-11';
```

```
-- Error : LOB Locator references to the previous LOB data because
only LOB Locator is rollbacked.
```

Note

- When selecting **LOB** data in an application through a driver such as JDBC, the driver can get **ResultSet** from DB server and fetch the record while changing the cursor location on **Resultset**. That is, only Locator, the meta data of a **LOB** column, is stored at the time when **ResultSet** is imported, and **LOB** data that is referred by a File Locator will be fetched from the file Locator at the time when a record is fetched. Therefore, if **LOB** data is updated between two different points of time, there could be an error, as the mapping of **LOB** Locator and actual **LOB** data will be invalid.
- Since backup/recovery is supported only for meta data (Locator) of the **LOB** type columns, an error is likely to occur, as the mapping of **LOB** Locator and LOB data is invalid if recovery is performed based on a specific point of time.
- TO execute **INSERT** the **LOB** data into other device, LOB data referred by the meta data (Locator) of a **LOB** column must be read.
- In a CUBRID HA environment, the meta data (Locator) of a **LOB** column is replicated and data of a **LOB** type is not replicated. Therefore, if storage of a **LOB** type is located on the local machine, no tasks on the columns in a slave node or a master node after failover are allowed.

Warning

Up to CUBRID 2008 R3.0, Large Objects are processed by using **glo** (Generalized Large Object) classes. However, the **glo** classes has been deprecated since the CUBRID 2008 R3.1. Instead of

it, **LOB** / **CLOB** data type is supported. Therefore, both DB schema and application must be modified when upgrading CUBRID in an environment using the previous version of **glo** classes.

Collection Types

Allowing multiple data values to be stored in a single attribute is an extended feature of relational database. Each element of a collection is possible to have different data type each other except View. Rest types except BLOB and CLOB can be an element of collection types.

Type	Description	Definition	Input Data	Stored Data
SET	A union which does not allow duplicates	col_name SET VARCHAR(20) or col_name SET (VARCHAR(20))	{'c','c','c','b','b','a'}	{'a','b','c'}
MULTISET	A union which allows duplicates	col_name MULTISET VARCHAR(20) or col_name MULTISET (VARCHAR(20))	{'c','c','c','b','b','a'}	{'a','b','b','c','c','c'}
LIST or **SEQUENCE**	A union which allows duplicates and stores data in the order of input	col_name LIST VARCHAR(20) or col_name LIST (VARCHAR(20))	{'c','c','c','b','b','a'}	{'c','c','c','b','b','a'}

As you see the table above, the value specified as a collection type can be inputted with curly braces ('{', '}') each value is separated with a comma (,).

If the specified collection types are identical, the collection types can be cast explicitly by using the **CAST** operator. The following table shows the collection types that allow explicit coercions.

FROM \ TO	SET	MULTISET	LIST
SET	-	Yes	Yes
MULTISET	Yes	-	No
LIST	Yes	Yes	-

Collection Types do not support collations. Therefore, Below query returns error.

```
CREATE TABLE tbl (str SET (string) COLLATE utf8_en_ci);
```

```
Syntax error: unexpected 'COLLATE', expecting ',' or ')'
```

SET

SET is a collection type in which each element has different values. Elements of a **SET** are allowed to have only one data type. It can have records of other tables.

```
CREATE TABLE set_tbl (col_1 SET (CHAR(1)));
INSERT INTO set_tbl VALUES ({'c','c','c','b','b','a'});
INSERT INTO set_tbl VALUES ({NULL});
INSERT INTO set_tbl VALUES ({''});
SELECT * FROM set_tbl;
```

```
  col_1
======================
{'a', 'b', 'c'}
{NULL}
{' '}
```

```
SELECT CAST (col_1 AS MULTISET), CAST (col_1 AS LIST) FROM set_tbl;
```

```
   cast(col_1 as multiset)   cast(col_1 as sequence)
============================================
  {'a', 'b', 'c'}  {'a', 'b', 'c'}
```

```
  {NULL}  {NULL}
  {' '}  {' '}
```

```
INSERT INTO set_tbl VALUES ('');
```

```
ERROR: Casting '' to type set is not supported.
```

MULTISET

MULTISET is a collection type in which duplicated elements are allowed. Elements of a **MULTISET** are allowed to have only one data type. It can have records of other tables.

```
CREATE TABLE multiset_tbl (col_1 MULTISET (CHAR(1)));
INSERT INTO multiset_tbl VALUES ({'c','c','c','b','b', 'a'});
SELECT * FROM multiset_tbl;
```

```
  col_1
======================
  {'a', 'b', 'b', 'c', 'c', 'c'}
```

```
SELECT CAST(col_1 AS SET), CAST(col_1 AS LIST) FROM multiset_tbl;
```

```
   cast(col_1 as set)   cast(col_1 as sequence)
============================================
  {'a', 'b', 'c'}  {'c', 'c', 'c', 'b', 'b', 'a'}
```

LIST/SEQUENCE

LIST (= **SEQUENCE**) is a collection type in which the input order of elements is preserved, and duplications are allowed. Elements of a **LIST** are allowed to have only one data type. It can have records of other tables.

```
CREATE TABLE list_tbl (col_1 LIST (CHAR(1)));
INSERT INTO list_tbl VALUES ({'c','c','c','b','b', 'a'});
SELECT * FROM list_tbl;
```

```
  col_1
======================
  {'c', 'c', 'c', 'b', 'b', 'a'}
```

```
SELECT CAST(col_1 AS SET), CAST(col_1 AS MULTISET) FROM list_tbl;
```

```
   cast(col_1 as set)  cast(col_1 as multiset)
============================================
  {'a', 'b', 'c'}  {'a', 'b', 'b', 'c', 'c', 'c'}
```

JSON Data Type

From CUBRID 10.2, native **JSON** data type is supported as defined by RFC 7159. **JSON** data type offers automatic validation and allows fast access and operations on JSON data.

> **Note**
>
> Old driver versions connecting to CUBRID 10.2 server interpret a JSON type column as Varchar.

Creating JSON data

JSON values are automatically converted (parsed) from string format when they're assigned to JSON data type columns.

```
-- assign a string to JSON type column
CREATE TABLE t (id int, j JSON);
INSERT INTO t VALUES (1, '{"a":1}');
SELECT j, TYPEOF(j) FROM t;
```

```
  j                    typeof(j)
=============================================
  {"a":1}              'json'
```

Conversions to JSON can also be forced through CAST or by using json keyword before strings.

```
-- cast string to json
SELECT CAST('{"a":1}' as JSON);
```

```
  cast('{"a":1}' as json)
======================
  {"a":1}
```

```
-- use json keyword
SELECT json'{"a":1}', TYPEOF (json'{"a":1}');
```

```
  json '{"a":1}'          typeof(json '{"a":1}')
=============================================
  {"a":1}                'json'
```

JSON data type may also be created using JSON_OBJECT and JSON_ARRAY functions.

JSON Validation

Conversion to JSON data does built-in validation and reports an error if the string is not a valid JSON.

```
-- non-quoted string is not a valid json
SELECT json'abc';
```

```
In line 1, column 8,

ERROR: before ' ; '
Invalid JSON: 'abc'.
```

JSON type columns with stricter validation rules can be defined using the draft JSON Schema standard. If you are not familiar with JSON Schema, you may refer to Understanding JSON Schema.

A simple example of how schema can be used:

```
-- set j column to accept only string type JSON's
CREATE TABLE t (id int, j JSON ('{"type": "string"}'));
```

```
-- inserting string type JSON passes schema validation
INSERT into t values (1, '"abc"');
```

```
1 command(s) successfully processed.
```

```
-- inserting object type JSON does not pass schema validation
INSERT into t values (2, '{"a":1}');
```

```
ERROR: before ' ); '
The provided JSON has been invalidated by the JSON schema (Invalid
schema path: #, Keyword: type, Invalid provided JSON path: #)
```

JSON Value Types

A JSON value must be an object, an array or a scalar (string, number, boolean or null), as defined by RFC 7159.

A table of JSON value types:

Type		CUBRID JSON type	Description
Object		JSON_OBJECT	A set of key-value pairs
Array		JSON_ARRAY	An array of JSON values
Scalar	String	STRING	A quoted string
	Number	INTEGER	32-bit signed integer
		BIGINT	64-bit signed integer
		DOUBLE	Non-integer number or integer bigger than 2^{63}- 1
	true	BOOLEAN	True boolean value
	false	BOOLEAN	False boolean value

Type	CUBRID JSON type	Description
null	JSON_NULL	Null value

The CUBRID JSON type of a JSON value can be obtained with JSON_TYPE function.

JSON Data Conversions

JSON data types can be obtained by explicit or implicit casting from and to other types.

Casting a JSON value to JSON type may fail if desired type has a schema and converted value does not pass schema validation.

Converting other types to JSON is explained by next table:

Original type	CUBRID JSON type
Any string	String is parsed as JSON data. The result may be of any type. **Note** If string codeset is not UTF8, string is first converted to UTF8 and then parsed.
Short, Integer	INTEGER
Bigint	BIGINT
Float, Double	DOUBLE
Numeric	DOUBLE

Converting JSON data type to other types is explained by next table:

CUBRID JSON Type	Other accepted types
JSON_OBJECT	String with printed JSON
JSON_ARRAY	String with printed JSON
STRING	Any type that a string can be converted to
INTEGER	Any type that an integer can be converted to
BIGINT	Any type that a bigint can be converted to
DOUBLE	Any type that a double can be converted to
BOOLEAN	“true” or “false” if converted to string
	0 or 1 if converted to a numeric type
JSON_NULL	String with printed JSON ‘null’

JSON Paths

JSON Paths provide ways of addressing json elements inside a JSON. Many of the JSON functions require a JSON Path or JSON Pointer argument to define the location inside the JSON where operations are performed. JSON Paths always start with ‘$’ and may be followed by array indexes, object key tokens and wildcards. If ‘$’ is followed by no other tokens, then path points to JSON data root.

```
<json_path>::=
   <start_token> [<path_token>] ...

<start_token>::=
   $

<path_token>::=
   <array_access_token> | <object_key_access_token> |
```

```
<wildcard_token>

<array_access_token>::=
   [idx]

<object_key_access_token>::=
   .[key_identifier | "key_str"]

<wildcard_token>::=
   .*|[*]|**path_token
```

As an example, relative to '{"a":[0,1,2,{"b":5}]}' '$.a[3].b' would mean: "The member having key 'b' of the element at index 3 of the member having key 'a' of the root" and would address the json value '5'; Object_key_access_tokens as key string can be used to express the same key_identifiers and can also enable using characters that need escaping, e.g. '$."\"' can be used to refer to a member having a double quote as a key.

JSON wildcards can be one of three types:

- .* , object member access matching wildcards
- [*], array index access matching wildcards
- **, matching a sequence of object keys and array indexes. ** wildcards must be suffixed by a token

Path expressions, like JSON Pointers and JSON text, should be encoded using ASCII or UTF-8 character set. If other character sets are used, a coercion will be done to UTF-8.

JSON Pointers

JSON Pointers, as defined by https://datatracker.ietf.org/doc/html/rfc6901 provide an alternative to JSON paths. JSON Pointers, like JSON Paths and JSON text, should be encoded using ASCII or UTF-8 character set. If other character sets are used, a coercion will be done to UTF-8.

```
<json_pointer>::=
   [/path_token] ... [/-]
```

```
'$.a[10].bb' is equivalent to '/a/10/bb'
'$' is equivalent to ''
```

The special character '-' can be used exclusively as a last path_token and can be used to address the end of a json_array.

JSON pointers can be used to address the same path as their corresponding no-wildcards JSON paths.

Implicit Type Conversion

An implicit type conversion represents an automatic conversion of a type of expression to a corresponding type.

SET, **MULTISET**, **LIST** and **SEQUENCE** should be converted explicitly.

If you convert the **DATETIME** and the **TIMESTAMP** types (including types having timezone) to the **DATE** type or the **TIME** type, data loss may occur. If you convert the **DATE** type to the **DATETIME** type or the **TIMESTAMP** type (or types with timezone), the time will be set to '12:00:00 AM.'

Timezone part of values with timezone types has only a reference purpose, their absolute value is stored as UTC reference. When converting from a value of type with timezone to a type without timezone, a conversion is operated as if session timezone is used. When converting from a value of type without timezone to a type with timezone, the conversion takes place considering the session timezone. For more details on converting to/from value with timezone type see Date/Time Types.

If you convert a string type or an exact numeric type to a floating-point numeric type, the value may not be accurate. Because a string type and an exact type use a decimal precision to represent the value, but a floating-point numeric type uses a binary precision.

The implicit type conversion executed by CUBRID is as follows:

Implicit Type Conversion Table 1

From \ To	DATETIME	DATETIMELTZ	DATETIMETZ	DATE	TIME	TIMESTAMP	TIMESTAMPLTZ	TIMESTAMPTZ
DATETIME	-	O	O	O	O	O	O	O
DATETIMELTZ	O	-	O	O	O	O	O	O
DATETIMETZ	O	O	-	O	O	O	O	O

From \ To	DATETIME	DATETIMELTZ	DATETIMETZ	DATE	TIME	TIMESTAMP	TIMESTAMPLTZ	TIMESTAMPTZ
DATE	O	O	O	-		O	O	O
TIME					-			
TIMESTAMP	O	O	O	O	O	-	O	O
TIMESTAMPLTZ	O	O	O	O	O	O	-	O
TIMESTAMPTZ	O	O	O	O	O	O	O	-
DOUBLE					O	O	O	O
FLOAT					O	O	O	O
NUMERIC						O	O	O
BIGINT					O	O	O	O
INT					O	O	O	O
SHORT					O	O	O	O
BIT								

From \ To	DATETIME	DATETIMELTZ	DATETIMETZ	DATE	TIME	TIMESTAMP	TIMESTAMPLTZ	TIMESTAMPTZ
VARBIT								
CHAR	O	O	O	O	O	O	O	O
VARCHAR	O	O	O	O	O	O	O	O

Limitations when numeric value is changed as TIME or TIMESTAMP (TIMESTAMPLTZ, TIMESTAMPTZ)

- All numeric types except for **NUMERIC** type can be converted into **TIME** type; at this time, it represents a value of the remainder which is calculated by dividing the input number into 86,400 seconds(1 day), and the remainder is calculated as seconds.
- All numeric types including **NUMERIC** can be converted into **TIMESTAMP**, **TIMESTAMPLTZ**, **TIMESTAMPTZ** types ; at this time, the input number cannot exceed 2,147,483,647 as the maximum.

Implicit Type Conversion Table 2

From \ To	INT	SHORT	BIT	VARBIT	CHAR	VARCHAR	DOUBLE	FLOAT	NUMERIC	BIGINT
DATETIME					O	O				
DATETIMELTZ					O	O				
					O	O				

From \ To	INT	SHORT	BIT	VARBIT	CHAR	VARCHAR	DOUBLE	FLOAT	NUMERIC	BIGINT
DATETIMETZ										
DATE					O	O				
TIME					O	O				
TIMESTAMP					O	O				
TIMESTAMPLTZ					O	O				
TIMESTAMPTZ					O	O				
DOUBLE	O	O			O	O	-	O	O	O
FLOAT	O	O			O	O	O	-	O	O
NUMERIC	O	O			O	O	O	O	-	O
BIGINT	O	O			O	O	O	O	O	-

From \ To	INT	SHORT	BIT	VARBIT	CHAR	VARCHAR	DOUBLE	FLOAT	NUMERIC	BIGINT
INT	-	O			O	O	O	O	O	O
SHORT	O	-			O	O	O	O	O	O
BIT			-	O	O	O				
VARBIT			O	-	O	O				
CHAR	O	O	O	O	-	O	O	O	O	O
VARCHAR	O	O	O	O	O	-	O	O	O	O

Conversion Rules

INSERT and UPDATE

The type will be converted to the type of the column affected.

```
CREATE TABLE t(i INT);
INSERT INTO t VALUES('123');

SELECT * FROM t;
```

```
             i
=============
           123
```

Function

If the parameter value entered in the function can be converted to the specified type, the parameter type will be converted. The strings are converted to numbers because the input parameter expected in the following function is a number.

```
SELECT MOD('123','2');
```

```
           mod('123', '2')
==========================
     1.000000000000000e+00
```

You can enter multiple type values in the function. If the type value not specified in the function is delivered, the type will be converted depending on the following priority order.

- Date/Time Type (**DATETIME** > **TIMESTAMP** > **DATE** > **TIME**)
- Approximate Numeric Type (**DOUBLE** > **FLOAT**)
- Exact Numeric Type (**NUMERIC** > **BIGINT** > **INT** > **SHORT**)
- String Type (**CHAR** > **VARCHAR**)

Comparison Operation

The following are the conversion rules according to an operand type of the comparison operator.

operand1 Type	operand2 Type	Conversion	Comparison
Numeric Type	Numeric Type	None	NUMERIC
	String Type	Converts operand2 to **DOUBLE**	NUMERIC
	Date/Time Type	Converts operand1 to Date/Time	TIME/TIMESTAMP
String Type	Numeric Type	Converts operand1 to **DOUBLE**	NUMERIC

operand1 Type	operand2 Type	Conversion	Comparison
	String Type	None	String
	Date/Time Type	Converts operand1 to Date/Time type	Date/Time
Date/Time Type	Numeric Type	Converts operand2 to Date/Time	TIME/TIMESTAMP
	String Type	Converts operand2 to Date/Time type	Date/Time
	Date/Time Type	Converts it to the type with higher priority	Date/Time

When Date/Time type and numeric type are compared, see Limitations when numeric value is changed as TIME or TIMESTAMP of the above table.

There are exceptions when operand1 is string type and operand2 is a value.

operand1 Type	operand2 Type	Conversion	Comparison
String type	Numeric type	Converts operand2 to the string type	String
	Date/Time type	Converts operand2 to the string type	String

If operand2 is a set operator(**IS IN**, **IS NOT IN**, **= ALL**, **= ANY**, **< ALL**, **< ANY**, **<= ALL**, **<= ANY**, **>= ALL**, **>= ANY**), the exception above is not applied.

The following is examples of implicit type conversion in comparison operations.

- **Numeric Type & String Type Operands**

The string type operand will be converted to **DOUBLE**.

```
CREATE TABLE t1(i INT, s STRING);
INSERT INTO t1 VALUES(1,'1'),(2,'2'),(3,'3'),(4,'4'), (12,'12');

SELECT i FROM t1 WHERE i < '11.3';
```

```
           i
=============
           1
           2
           3
           4
```

```
SELECT ('2' <= 11);
```

```
     ('2'<11)
=============
           1
```

- **String Type & Date/Time Type Operands**

The string type operand will be converted to the date/time type.

```
SELECT ('2010-01-01' < date'2010-02-02');
```

```
   ('2010-01-01'<date '2010-02-02')
===================================
                                  1
```

```
SELECT (date'2010-02-02' >= '2010-01-01');
```

```
  (date '2010-02-02'>='2010-01-01')
===================================
                                  1
```

- **String Type & Numeric Type Host Variable Operands**

The numeric type host variable will be converted to the string type.

```
PREPARE s FROM 'SELECT s FROM t1 WHERE s < ?';
EXECUTE s USING 11;
```

```
       s
==================
     '1'
```

- **String Type & Numeric Type value Operands**

  The numeric type value will be converted to the string type.

```
SELECT s FROM t1 WHERE s > 11;
```

```
       s
==================
     '2'
     '3'
     '4'
     '12'
```

```
SELECT s FROM t1 WHERE s BETWEEN 11 AND 33;
```

```
        s
======================
      '2'
      '3'
      '12'
```

- **String Type Column & Date/Time Type Value Operands**

  The date/time type value will be converted to the string type.

```
CREATE TABLE t2 (s STRING);
INSERT INTO t2 VALUES ('01/01/1998'), ('01/01/1999'),
('01/01/2000');
SELECT s FROM t2;
```

```
           s
======================
    '01/01/1998'
```

```
    '01/01/1999'
    '01/01/2000'
```

```
SELECT s FROM t2 WHERE s <= date'02/02/1998';
```

In the above query, comparison operation is performed by converting date'02/02/1998' into string '02/02/1998'.

```
           s
======================
    '01/01/1998'
    '01/01/1999'
    '01/01/2000'
```

Range Operation

- **Numeric Type and String Type Operands**

  The string type operand will be converted to **DOUBLE**.

```
CREATE TABLE t3 (i INT);
INSERT INTO t3 VALUES (1), (2), (3), (4);
SELECT i FROM t3 WHERE i <= ALL {'11','12'};
```

```
           i
=============
           1
           2
           3
           4
```

- **String Type and Date/Time Type Operands**

  The string type operand will be converted to the date/time type.

```
SELECT s FROM t2;
```

```
  s
=================
  '01/01/1998'
```

```
  '01/01/1999'
  '01/01/2000'
```

```
SELECT s FROM t2 WHERE s <= ALL {date'02/02/1998',date'01/01/2000'};
```

```
  s
================
 '01/01/1998'
```

An error will be returned if it cannot be converted to the corresponding type.

Arithmetic Operation

- **Date/Time Type Operand**

  If the date/time type operands are given to '-' operator and the types are different from each other, it will be converted to the type with a higher priority. The following example shows that the operand data type on the left is converted from **DATE** to **DATETIME** so that the result of '-' operation of **DATETIME** can be outputted in milliseconds.

```
SELECT date'2002-01-01' - datetime'2001-02-02 12:00:00 am';
```

```
   date '2002-01-01'- datetime '2001-02-02 12:00:00 am'
=======================================================
                                          28771200000
```

- **Numeric Type Operand**

  If the numeric type operands are given and the types are different from each other, it will be converted to the type with the higher priority.

- **Date/Time Type & Numeric Type Operands**

  If the date/time type and the numeric type operands are given to '+' or '-' operator, the numeric type operand is converted to either **BIGINT**, **INT** or **SHORT**.

- **Date/Time Type & String Type Operands**

If a date/time type and a string type are operands, only '+' and '-' operators are allowed. If the '+' operator is used, it will be applied according to the following rules.

- The string type will be converted to **BIGINT** with an interval value. The interval is the smallest unit for operands in the Date/Time type, and the interval for each type is as follows:
  - **DATE** : Days
  - **TIME**, **TIMESTAMP** : Seconds
  - **DATETIME** : Milliseconds
- Floating-point numbers are rounded.
- The result type is the type of an date/time operand.

```
SELECT date'2002-01-01' + '10';
```

```
  date '2002-01-01'+'10'
========================
  01/11/2002
```

If the date/time type and a string type are operands and the '-' operator is used, they will be applied according to the following rules.

- If the date/time type operands are **DATE**, **DATETIME** and **TIMESTAMP**, the string will be converted to **DATETIME**; if the date/time operand is **TIME**, the string is converted to **TIME**.
- The result type is always **BIGINT**.

```
SELECT date'2002-01-01'-'2001-01-01';
```

```
  date '2002-01-01'-'2001-01-01'
================================
                   31536000000

-- this causes an error
```

```
SELECT date'2002-01-01'-'10';
```

```
ERROR: Cannot coerce '10' to type datetime.
```

· **Numeric Type & String Type Operands**

If a numeric type and a string type are operands, they will be applied according to the following rules.

◦ Strings will be converted to **DOUBLE** when possible.

◦ The result type is **DOUBLE** and depends on the type of the numeric operand.

```
SELECT 4 + '5.2';
```

```
               4+'5.2'
==========================
  9.199999999999999e+00
```

Unlike CUBRID 2008 R3.1 and the earlier versions, the string in the date/time format, that is, the string such as '2010-09-15' is not converted to the date/time type. You can use a literal DATE'2010-09-15' with the date/time type for addition and subtraction operations.

```
SELECT '2002-01-01'+1;
```

```
ERROR: Cannot coerce '2002-01-01' to type double.
```

```
SELECT DATE'2002-01-01'+1;
```

```
  date '2002-01-01'+1
======================
  01/02/2002
```

· **String Type Operand**

If you multiply, divide or subtract both strings, the result returns a **DOUBLE** type value.

```
SELECT '3'*'2';
```

```
                    '3'*'2'
============================
       6.000000000000000e+00
```

The '+' operator action depends on how to set the system parameter **plus_as_concat** in the **cubrid.conf** file. For details, see Statement/Type-Related Parameters.

- If a value for **plus_as_concat** is yes (default value), the concatenation of two strings will be returned.

```
SELECT '1'+'1';
```

```
              '1'+'1'
======================
                '11'
```

- If a value for **plus_as_concat** is no and two strings can be converted to numbers, the **DOUBLE** type value will be returned by adding the two numbers.

```
SELECT '1'+'1';
```

```
                   '1'+'1'
==========================
     2.000000000000000e+00
```

An error will be returned if it cannot be converted to the corresponding type.

Data Definition Statements

TABLE DEFINITION STATEMENTS

CREATE TABLE

Table Definition

Use **CREATE TABLE** statement to create a table. A table with the same name as an existing view or synonym cannot be created. Regarding writing table name, see Identifier.

```
CREATE {TABLE | CLASS} [IF NOT EXISTS] [schema_name.]table_name
[<subclass_definition>]
[CLASS ATTRIBUTE (<column_definition>, ...)]
```

```
[([{<table_constraint>}... ,] <column_definition> [{,
{<column_definition> | <table_constraint>}}...])]
[INHERIT <resolution>, ...]
[<table_options>]

    <subclass_definition> ::= {UNDER | AS SUBCLASS OF}
[schema_name.]superclass_name, ...

    <column_definition> ::=
        column_name <data_type> [{<default_or_shared_or_ai> |
<on_update> | <column_constraint>}] [COMMENT 'column_comment_string']

        <data_type> ::= <column_type> [<charset_modifier_clause>]
[<collation_modifier_clause>]

            <charset_modifier_clause> ::= {CHARACTER_SET|CHARSET}
{<char_string_literal>|<identifier>}

            <collation_modifier_clause> ::= COLLATE
{<char_string_literal>|<identifier>}

        <default_or_shared_or_ai> ::=
            SHARED <value_specification> |
            DEFAULT <value_specification>  |
            AUTO_INCREMENT [(seed, increment)]

        <on_update> ::= [ON UPDATE <value_specification>]

        <column_constraint> ::= [CONSTRAINT constraint_name] { NOT
NULL | UNIQUE | PRIMARY KEY | [FOREIGN KEY] [WITH
<index_with_option>] <referential_definition> }

            <referential_definition> ::=
                REFERENCES [schema_name.]referenced_table_name
(column_name, ...) [<referential_triggered_action> ...]

                <referential_triggered_action> ::=
                    ON UPDATE <referential_action> |
                    ON DELETE <referential_action>

                    <referential_action> ::= CASCADE | RESTRICT | NO
ACTION | SET NULL

    <table_constraint> ::=
        {
            {KEY|INDEX} index_name <index_col_desc> |
            [CONSTRAINT [constraint_name]]
               {
                  UNIQUE [KEY|INDEX](column_name, ...) |
                  PRIMARY KEY (column_name, ...) |
                  <referential_constraint>
               }
```

```
        } COMMENT 'index_comment_string'

        <index_col_desc> ::=
          { ( {column_name | function_name (argument_list)} [ASC |
DESC] [{, {column_name | function_name (argument_list)} [ASC |
DESC]} ...] ) }
          [WHERE <filter_predicate>]
          [WITH <index_with_option>]
          [INVISIBLE]
          [COMMENT 'index_comment_string’]

        <referential_constraint> ::= FOREIGN KEY [<foreign_key_name>]
(column_name, ...) [WITH <index_with_option>]
<referential_definition>

            <referential_definition> ::=
                REFERENCES [schema_name.]referenced_table_name
(column_name, ...) [<referential_triggered_action> ...]

                <referential_triggered_action> ::=
                    ON UPDATE <referential_action> |
                    ON DELETE <referential_action>

                    <referential_action> ::= CASCADE | RESTRICT | NO
ACTION | SET NULL

    <resolution> ::= [CLASS] {column_name} OF
[schema_name.]superclass_name [AS alias]
    <table_options> ::= <table_option> [[,] <table_option> ...]
        <table_option> ::= REUSE_OID | DONT_REUSE_OID |
                           COMMENT [=] 'table_comment_string' |
                           [CHARSET charset_name] [COLLATE
collation_name] |
                           ENCRYPT [=] [AES | ARIA] |
                           AUTO_INCREMENT = initial_value

   <index_with_option> ::= {DEDUPLICATE ‘=‘ deduplicate_level }
```

- IF NOT EXISTS: If an identically named table already exists, a new table will not be created without an error.
- *schema_name*: Specifies the schema name(maximum: 31 bytes). If omitted, the schema name of the current session is used.
- *table_name*: specifies the name of the table to be created (maximum: 222 bytes).
- *column_name*: specifies the name of the column to be created (maximum: 254 bytes).
- *column_type*: specifies the data type of the column.
- [**SHARED** *value* | **DEFAULT** *value*]: specifies the initial value of the column.

- **ON UPDATE** specifies an expression to update the field when the field's ROW gets updated. For details, see ON UPDATE.
- <*column_constraint*>: specifies the constraint of the column. Available constraints are **NOT NULL**, **UNIQUE**, **PRIMARY KEY** and **FOREIGN KEY**.
- <*default_or_shared_or_ai*>: only one of DEFAULT, SHARED, AUTO_INCREMENT can be used. When AUTO_INCREMENT is specified, "(seed, increment)" and "AUTO_INCREMENT = initial_value" cannot be defined at the same time.
- *table_comment_string*: specifies a table's comment
- *column_comment_string*: specifies a column's comment.
- *index_comment_string*: specifies an index's comment.
- *deduplicate_level*: specifies the deduplicate level (0 to 14). For details, see DEDUPLICATE.

Note

- **DBA** and **DBA** members can create tables in different schemas. If a user is neither **DBA** nor **DBA** member, tables can only be created in the schema of that user.

Note

- *deduplicate_level* is an integer from 0 to 14.
- When this parameter is set to 0, it indicates an index configuration where data deduplication within indexes is not performed. This configuration is the same as that in CUBRID 11.2 or an earlier version.

```
CREATE TABLE olympic2 (
    host_year        INT     NOT NULL PRIMARY KEY,
    host_nation      VARCHAR(40) NOT NULL,
    host_city        VARCHAR(20) NOT NULL,
    opening_date     DATE        NOT NULL,
    closing_date     DATE        NOT NULL,
    mascot           VARCHAR(20),
    slogan           VARCHAR(40),
    introduction     VARCHAR(1500)
);
```

The below adds a comment of a table with ALTER statement.

```
ALTER TABLE olympic2 COMMENT = 'this is new comment for olympic2';
```

The below includes an index comment when you create a table.

CREATE TABLE tbl (a INT, index i_t_a (a) COMMENT 'index comment');

Note

A CHECK constraint in the table schema

A CHECK constraint defined in the table schema is parsed, but ignored. The reason of being parsed is to support the compatibility when DB migration from other DBMS is done.

```
CREATE TABLE tbl (
    id INT PRIMARY KEY,
    CHECK (id > 0)
)
```

Column Definition

A column is a set of data values of a particular simple type, one for each row of the table.

```
<column_definition> ::=
    column_name <data_type> [[<default_or_shared_or_ai>] |
[<on_update>] | [<column_constraint>]] ... [COMMENT 'comment_string']

    <data_type> ::= <column_type> [<charset_modifier_clause>]
[<collation_modifier_clause>]

        <charset_modifier_clause> ::= {CHARACTER_SET|CHARSET}
{<char_string_literal>|<identifier>}

        <collation_modifier_clause> ::= COLLATE
{<char_string_literal>|<identifier>}

    <default_or_shared_or_ai> ::=
        SHARED <value_specification> |
        DEFAULT <value_specification>  |
        AUTO_INCREMENT [(seed, increment)]

    <on_update> ::= [ON UPDATE <value_specification>]
```

```
    <column_constraint> ::= [CONSTRAINT constraint_name] {NOT NULL |
UNIQUE | PRIMARY KEY | [FOREIGN KEY] [WITH <index_with_option>]
<referential_definition>}
```

Column Name

How to create a column name, see Identifier. You can alter created column name by using the RENAME COLUMN Clause of the **ALTER TABLE** statement.

The following example shows how to create the *manager2* table that has the following two columns: *full_name* and *age*.

```
CREATE TABLE manager2 (full_name VARCHAR(40), age INT );
```

Note

- The first character of a column name must be an alphabet.
- The column name must be unique in the table.

Setting the Column Initial Value (SHARED, DEFAULT)

An attribute in a table can be created with an initial **SHARED** or **DEFAULT** value. You can change the value of **SHARED** and **DEFAULT** in the **ALTER TABLE** statement.

- **SHARED** : Column values are identical in all rows. If a value different from the initial value is **INSERT**ed, the column value is updated to a new one in every row.
- **DEFAULT** : The initial value set when the **DEFAULT** attribute was defined is stored even if the column value is not specified when a new row is inserted.

The pseudocolumn allows for the **DEFAULT** value as follows.

DEFAULT Value	Data Type
SYS_TIMESTAMP	TIMESTAMP
UNIX_TIMESTAMP()	INTEGER

DEFAULT Value	Data Type
CURRENT_TIMESTAMP	TIMESTAMP
SYS_DATETIME	DATETIME
CURRENT_DATETIME	DATETIME
SYS_DATE	DATE
CURRENT_DATE	DATE
SYS_TIME	TIME
CURRENT_TIME	TIME
USER, USER()	STRING
TO_CHAR(date_time[, format])	STRING
TO_CHAR(number[, format])	STRING

Note

When using a function not specified in the table above as a **DEFAULT** value, it is evaluated at the time of table creation and stored as the **DEFAULT** value. Therefore, during an **INSERT**, the default value used for the column is not the function result at the time of **INSERT**, but the value evaluated at the time the table was created. Stored functions cannot be used as **DEFAULT** values, and doing so results in an error.

```
CREATE TABLE default_val_func_test (id INT DEFAULT RAND ());
INSERT INTO default_val_func_test (id) VALUES (1);
INSERT INTO default_val_func_test (id) VALUES (DEFAULT);
```

```
INSERT INTO default_val_func_test (id) VALUES (DEFAULT);

SELECT * FROM default_val_func_test;
```

```
           id
=============
            1
   1572882525
   1572882525
```

```
CREATE FUNCTION fn_int (a int) return int as begin return a; end;
CREATE TABLE default_val_sp_test (id INT DEFAULT fn_int (1));
```

```
ERROR: before ' ); '
'fn_int(1)' function can not be used in DEFAULT clause.
```

Note

In version lower than CUBRID 9.0, the value at the time of **CREATE TABLE** has been saved when the **DATE** value of the **DATE**, **DATETIME**, **TIME**, **TIMESTAMP** column has been specified as **SYS_DATE**, **SYS_DATETIME**, **SYS_TIME**, **SYS_TIMESTAMP** while creating a table. Therefore, to enter the value at the time of data **INSERT** in version lower than CUBRID 9.0, the function should be entered to the **VALUES** clause of the **INSERT** syntax.

```
CREATE TABLE colval_tbl
(id INT, name VARCHAR SHARED 'AAA', phone VARCHAR DEFAULT
'000-0000');
INSERT INTO colval_tbl (id) VALUES (1), (2);
SELECT * FROM colval_tbl;
```

```
           id  name                  phone
============================================================
            1  'AAA'                 '000-0000'
            2  'AAA'                 '000-0000'
```

```
--updating column values on every row
INSERT INTO colval_tbl(id, name) VALUES (3,'BBB');
```

```
INSERT INTO colval_tbl(id) VALUES (4),(5);
SELECT * FROM colval_tbl;
```

```
           id  name                  phone
===============================================
            1  'BBB'                 '000-0000'
            2  'BBB'                 '000-0000'
            3  'BBB'                 '000-0000'
            4  'BBB'                 '000-0000'
            5  'BBB'                 '000-0000'
```

```
--changing DEFAULT value in the ALTER TABLE statement
ALTER TABLE colval_tbl MODIFY phone VARCHAR DEFAULT '111-1111';
INSERT INTO colval_tbl (id) VALUES (6);
SELECT * FROM colval_tbl;
```

```
           id  name                  phone
===============================================
            1  'BBB'                 '000-0000'
            2  'BBB'                 '000-0000'
            3  'BBB'                 '000-0000'
            4  'BBB'                 '000-0000'
            5  'BBB'                 '000-0000'
            6  'BBB'                 '111-1111'
```

```
--use DEFAULT TO_CHAR in CREATE TABLE statement
CREATE TABLE t1(id1 INT, id2 VARCHAR(20) DEFAULT
TO_CHAR(12345,'S999999'));
INSERT INTO t1 (id1) VALUES (1);
SELECT * FROM t1;
```

```
          id1  id2
=================================
            1  ' +12345'
```

The **DEFAULT** value of the pseudocolumn can be specified as one or more columns.

```
CREATE TABLE tbl (date1 DATE DEFAULT SYSDATE, date2 DATE DEFAULT
SYSDATE);
CREATE TABLE tbl (date1 DATE DEFAULT SYSDATE,
                  ts1   TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE t1(id1 INT, id2 VARCHAR(20) DEFAULT
TO_CHAR(12345,'S999999'), id3 VARCHAR(20) DEFAULT TO_CHAR(SYS_TIME,
```

```
'HH24:MI:SS'));
ALTER TABLE t1 add column id4 varchar (20) default
TO_CHAR(SYS_DATETIME, 'yyyy/mm/dd hh:mi:ss'), id5 DATE DEFAULT
SYSDATE;
```

AUTO INCREMENT

You can define the **AUTO_INCREMENT** attribute for the column to automatically give serial numbers to column values. This can be defined only for **SMALLINT**, **INTEGER**, **BIGINT** and **NUMERIC**(*p*, 0) types.

DEFAULT, **SHARED** and **AUTO_INCREMENT** cannot be defined for the same column. Make sure the value entered directly by the user and the value entered by the auto increment attribute do not conflict with each other.

You can change the initial value of **AUTO_INCREMENT** by using the **ALTER TABLE** statement. For details, see AUTO_INCREMENT Clause of **ALTER TABLE**.

```
CREATE TABLE [schema_name.]table_name (id INT AUTO_INCREMENT[(seed,
increment)]);

CREATE TABLE [schema_name.]table_name (id INT AUTO_INCREMENT)
AUTO_INCREMENT = seed ;
```

- *seed* : The initial value from which the number starts. All integers (positive, negative, and zero) are allowed. The default value is **1**.
- *increment* : The increment value of each row. Only positive integers are allowed. The default value is **1**.

When you use the **CREATE TABLE** *[schema_name.]table_name* (id INT **AUTO_INCREMENT**) **AUTO_INCREMENT** = *seed*; statement, the constraints are as follows:

- You should define only one column with the **AUTO_INCREMENT** attribute.
- Don't use (*seed*, *increment*) and AUTO_INCREMENT = *seed* together.

```
CREATE TABLE auto_tbl (id INT AUTO_INCREMENT, name VARCHAR);
INSERT INTO auto_tbl VALUES (NULL, 'AAA'), (NULL, 'BBB'), (NULL,
'CCC');
INSERT INTO auto_tbl (name) VALUES ('DDD'), ('EEE');
SELECT * FROM auto_tbl;
```

```
           id  name
=================================
```

```
            1  'AAA'
            2  'BBB'
            3  'CCC'
            4  'DDD'
            5  'EEE'
```

```
CREATE TABLE tbl (id INT AUTO_INCREMENT, val string) AUTO_INCREMENT
= 3;
INSERT INTO tbl VALUES (NULL, 'cubrid');

SELECT * FROM tbl;
```

```
          id  val
=====================================
            3  'cubrid'
```

```
CREATE TABLE t (id INT AUTO_INCREMENT, id2 int AUTO_INCREMENT)
AUTO_INCREMENT = 5;
```

```
ERROR: To avoid ambiguity, the AUTO_INCREMENT table option requires
the table to  have exactly one AUTO_INCREMENT column and no seed/
increment specification.
```

```
CREATE TABLE t (i INT AUTO_INCREMENT(100, 2)) AUTO_INCREMENT = 3;
```

```
ERROR: To avoid ambiguity, the AUTO_INCREMENT table option requires
the table to  have exactly one AUTO_INCREMENT column and no seed/
increment specification.
```

Note

- Even if a column has auto increment, the **UNIQUE** constraint is not satisfied.
- If **NULL** is specified in the column where auto increment is defined, the value of auto increment is stored.
- Even if a value is directly specified in the column where auto increment is defined, AUTO_INCREMENT value is not changed.

- **SHARED** or **DEFAULT** attribute cannot be specified in the column in which AUTO_INCREMENT is defined.
- The initial value and the final value obtained by auto increment cannot exceed the minimum and maximum values allowed in the given type.
- Because auto increment has no cycle, an error occurs when the maximum value of the type exceeds, and no rollback is executed. Therefore, you must delete and recreate the column in such cases.

  For example, if a table is created as below, the maximum value of A is 32767. Because an error occurs if the value exceeds 32767, you must make sure that the maximum value of the column A does not exceed the maximum value of the type when creating the initial table.

```
CREATE TABLE tb1(A SMALLINT AUTO_INCREMENT, B CHAR(5));
```

ON UPDATE

An attribute in a table can be created with an automatic update when another attribute of the row is updated. You can change the value of **ON UPDATE** in the **ALTER TABLE** statement. The pseudocolumn allows for the **ON UPDATE** value as follows. Including the attribute in the updated fields will not trigger an update with the specified **ON UPDATE** value.

DEFAULT Value	Data Type
SYS_TIMESTAMP	TIMESTAMP
UNIX_TIMESTAMP()	INTEGER
CURRENT_TIMESTAMP	TIMESTAMP
SYS_DATETIME	DATETIME
CURRENT_DATETIME	DATETIME

DEFAULT Value	Data Type
SYS_DATE	DATE

CURRENT_DATE | DATE

```
CREATE TABLE sales (sales_cnt INTEGER, last_sale TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP, product VARCHAR(100), product_id INTEGER);
INSERT INTO sales VALUES (0, NULL, 'bicycle', 1);

UPDATE sales set sales_cnt = sales_cnt + 1
WHERE product_id = 1;
```

```
ALTER TABLE sales MODIFY last_sale TIMESTAMP; -- removes ON UPDATE
UPDATE sales set sales_cnt = sales_cnt + 1
WHERE product_id = 1; -- last_sale will remain unupdated
```

Constraint Definition

You can define **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY** as the constraints. You can also create an index by using **INDEX** or **KEY**.

```
<column_constraint> ::= [CONSTRAINT constraint_name] { NOT NULL |
UNIQUE | PRIMARY KEY | [FOREIGN KEY] [WITH <index_with_option>]
<referential_definition> }

<table_constraint> ::=
    {
        {KEY|INDEX} index_name <index_col_desc> |
        [CONSTRAINT [constraint_name]]
            {
                  UNIQUE [KEY|INDEX](column_name, ...) |
                  PRIMARY KEY (column_name, ...) |
                  <referential_constraint>
            }
    }

    <referential_constraint> ::= FOREIGN KEY [<foreign_key_name>]
(column_name, ...) [WITH <index_with_option>]
<referential_definition>

        <referential_definition> ::=
            REFERENCES [schema_name.]referenced_table_name
```

```
(column_name, ...) [<referential_triggered_action> ...]

            <referential_triggered_action> ::=
                ON UPDATE <referential_action> |
                ON DELETE <referential_action>

                <referential_action> ::= CASCADE | RESTRICT | NO
ACTION | SET NULL

    <index_with_option> ::= {DEDUPLICATE ‘=‘ deduplicate_level}
```

NOT NULL Constraint

A column for which the **NOT NULL** constraint has been defined must have a certain value that is not **NULL**. The **NOT NULL** constraint can be defined for all columns. An error occurs if you try to insert a **NULL** value into a column with the **NOT NULL** constraint by using the **INSERT** or **UPDATE** statement.

In the following example, if you input NULL value on the *id* column, it occurs an error because *id* column cannot have NULL value.

```
CREATE TABLE const_tbl1(id INT NOT NULL, INDEX i_index(id ASC),
phone VARCHAR);

CREATE TABLE const_tbl2(id INT NOT NULL PRIMARY KEY, phone VARCHAR);
INSERT INTO const_tbl2 VALUES (NULL,'000-0000');
```

```
Putting value 'null' into attribute 'id' returned: Attribute "id"
cannot be made NULL.
```

UNIQUE Constraint

The **UNIQUE** constraint enforces a column to have a unique value. An error occurs if a new record that has the same value as the existing one is added by this constraint.

You can place a **UNIQUE** constraint on either a column or a set of columns. If the **UNIQUE** constraint is defined for multiple columns, the uniqueness is ensured not for each column, but the combination of multiple columns.

In the following example, the second INSERT statement fails because the value of *id* column is the same as 1 with the value of *id* column in the first INSERT statement.

```
-- UNIQUE constraint is defined on a single column only
CREATE TABLE const_tbl5(id INT UNIQUE, phone VARCHAR);
```

```
INSERT INTO const_tbl5(id) VALUES (NULL), (NULL);
INSERT INTO const_tbl5 VALUES (1, '000-0000');
SELECT * FROM const_tbl5;
```

```
   id  phone
=================
 NULL  NULL
 NULL  NULL
    1  '000-0000'
```

```
INSERT INTO const_tbl5 VALUES (1, '111-1111');
```

```
ERROR: Operation would have caused one or more unique constraint
violations.
```

In the following example, if a **UNIQUE** constraint is defined on several columns, this ensures the uniqueness of the values in all the columns.

```
-- UNIQUE constraint is defined on several columns
CREATE TABLE const_tbl6(id INT, phone VARCHAR, CONSTRAINT UNIQUE
(id, phone));
INSERT INTO const_tbl6 VALUES (1, NULL), (2, NULL), (1, '000-0000'),
(1, '111-1111');
SELECT * FROM const_tbl6;
```

```
   id  phone
====================
    1  NULL
    2  NULL
    1  '000-0000'
    1  '111-1111'
```

PRIMARY KEY Constraint

A key in a table is a set of column(s) that uniquely identifies each row. A candidate key is a set of columns that uniquely identifies each row of the table. You can define one of such candidate keys a primary key. That is, the column defined as a primary key is uniquely identified in each row.

By default, the index created by defining the primary key is created in ascending order, and you can define the order by specifying **ASC** or **DESC** keyword next to the column.

```
CREATE TABLE pk_tbl (a INT, b INT, PRIMARY KEY (a, b DESC));

CREATE TABLE const_tbl7 (
    id INT NOT NULL,
    phone VARCHAR,
    CONSTRAINT pk_id PRIMARY KEY (id)
);

-- CONSTRAINT keyword
CREATE TABLE const_tbl8 (
    id INT NOT NULL PRIMARY KEY,
    phone VARCHAR
);

-- primary key is defined on multiple columns
CREATE TABLE const_tbl8 (
    host_year    INT NOT NULL,
    event_code   INT NOT NULL,
    athlete_code INT NOT NULL,
    medal        CHAR (1)  NOT NULL,
    score        VARCHAR (20),
    unit         VARCHAR (5),
    PRIMARY KEY (host_year, event_code, athlete_code, medal)
);
```

FOREIGN KEY Constraint

A foreign key is a column or a set of columns that references the primary key in other tables in order to maintain reference relationship. The foreign key and the referenced primary key must have the same data type. Consistency between two tables is maintained by the foreign key referencing the primary key, which is called referential integrity.

```
[CONSTRAINT constraint_name] FOREIGN KEY [foreign_key_name]
(<column_name_comma_list1>) [WITH <index_with_option>] REFERENCES
[schema_name.]referenced_table_name (<column_name_comma_list2>)
[<referential_triggered_action> ...]

    <referential_triggered_action> ::= ON UPDATE
<referential_action> | ON DELETE <referential_action>

        <referential_action> ::= CASCADE | RESTRICT | NO ACTION  |
SET NULL

    <index_with_option> ::= {DEDUPLICATE ‘=‘ deduplicate_level}
```

- *constraint_name*: Specifies the name of the table to be created.

- *foreign_key_name*: Specifies a name of the **FOREIGN KEY** constraint. You can skip the name specification. However, if you specify this value, *constraint_name* will be ignored, and the specified value will be used.

- <*column_name_comma_list1*>: Specifies the name of the column to be defined as a foreign key after the **FOREIGN KEY** keyword. The column number of foreign keys defined and primary keys must be same.

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.

- *referenced_table_name*: Specifies the name of the table to be referenced.

- <*column_name_comma_list2*>: Specifies the name of the referred primary key column after the **FOREIGN KEY** keyword.

- <*referential_triggered_action*>: Specifies the trigger action that responds to a certain operation in order to maintain referential integrity. **ON UPDATE** or **ON DELETE** can be specified. Each action can be defined multiple times, and the definition order is not significant.

  - **ON UPDATE**: Defines the action to be performed when attempting to update the primary key referenced by the foreign key. You can use either **NO ACTION**, **RESTRICT**, or **SET NULL** option. The default is **RESTRICT**.

  - **ON DELETE**: Defines the action to be performed when attempting to delete the primary key referenced by the foreign key. You can use **NO ACTION**, **RESTRICT**, **CASCADE**, or **SET NULL** option. The default is **RESTRICT**.

- <*referential_action*>: You can define an option that determines whether to maintain the value of the foreign key when the primary key value is deleted or updated.

  - **CASCADE**: If the primary key is deleted, the foreign key is deleted as well. This option is supported only for the **ON DELETE** operation.

  - **RESTRICT**: Prevents the value of the primary key from being deleted or updated, and rolls back any transaction that has been attempted.

  - **SET NULL**: When a specific record is being deleted or updated, the column value of the foreign key is updated to **NULL**.

  - **NO ACTION**: Its behavior is the same as that of the **RESTRICT** option.

- *deduplicate_level*: specifies the deduplicate level (0 to 14). For details, see DEDUPLICATE.

For each row R1 of the referencing table, there should be some row R2 of the referenced table such that the value of each referencing column in R1 is either **NULL** or is equal to the value of the corresponding referenced column in R2.

```
-- creating two tables where one is referring to the other
CREATE TABLE a_tbl (
    id INT NOT NULL DEFAULT 0 PRIMARY KEY,
    phone VARCHAR(10)
);

CREATE TABLE b_tbl (
    id INT NOT NULL,
    name VARCHAR (10) NOT NULL,
    CONSTRAINT pk_id PRIMARY KEY (id),
    CONSTRAINT fk_id FOREIGN KEY (id) REFERENCES a_tbl (id)
    ON DELETE CASCADE ON UPDATE RESTRICT
);

INSERT INTO a_tbl VALUES (1,'111-1111'), (2,'222-2222'), (3,
'333-3333');
INSERT INTO b_tbl VALUES (1,'George'),(2,'Laura'), (3,'Max');
SELECT a.id, b.id, a.phone, b.name FROM a_tbl a, b_tbl b WHERE a.id
= b.id;
```

```
   id          id                  phone                name
=============================================================
=
    1           1                  '111-1111'           'George'
    2           2                  '222-2222'           'Laura'
    3           3                  '333-3333'           'Max'
```

```
-- when deleting primary key value, it cascades foreign key value
DELETE FROM a_tbl WHERE id=3;
```

```
1 row affected.
```

```
SELECT a.id, b.id, a.phone, b.name FROM a_tbl a, b_tbl b WHERE a.id
= b.id;
```

```
   id          id                  phone                name
=============================================================
=
    1           1                  '111-1111'           'George'
    2           2                  '222-2222'           'Laura'
```

```
-- when attempting to update primary key value, it restricts the
operation
UPDATE  a_tbl SET id = 10 WHERE phone = '111-1111';
```

```
ERROR: Update/Delete operations are restricted by the foreign key
'fk_id'.
```

Note

- In a referential constraint, the name of the primary key table to be referenced and the corresponding column names are defined. If the list of column names are is not specified, the primary key of the primary key table is specified in the defined order.
- The number of primary keys in a referential constraint must be identical to that of foreign keys. The same column name cannot be used multiple times for the primary key in the referential constraint.
- The actions cascaded by reference constraints do not activate the trigger action.
- It is not recommended to use *referential_triggered_action* in the CUBRID HA environment. In the CUBRID HA environment, the trigger action is not supported. Therefore, if you use *referential_triggered_action*, the data between the master database and the slave database can be inconsistent. For details, see CUBRID HA.

KEY or INDEX

KEY and **INDEX** are used interchangeably. They create an index that uses the corresponding column as a key.

```
CREATE TABLE const_tbl4(id INT, phone VARCHAR, KEY i_key(id DESC,
phone ASC));
```

Note

In versions lower than CUBRID 9.0, index name can be omitted; however, in version of CUBRID 9.0 or higher, it is no longer allowed.

Column Option

You can specify options such as **ASC** or **DESC** after the column name when defining **UNIQUE** or **INDEX** for a specific column. This keyword is specified as store the index value in ascending or descending order.

```
column_name [ASC | DESC]
```

```
CREATE TABLE const_tbl(
    id VARCHAR,
    name VARCHAR,
    CONSTRAINT UNIQUE INDEX(id DESC, name ASC)
);

INSERT INTO const_tbl VALUES('1000', 'john'), ('1000','johnny'),
('1000', 'jone');
INSERT INTO const_tbl VALUES('1001', 'johnny'), ('1001','john'),
('1001', 'jone');

SELECT * FROM const_tbl WHERE id > '100';
```

```
  id    name
================
  1001     john
  1001     johnny
  1001     jone
  1000     john
  1000     johnny
  1000     jone
```

Table Option

REUSE_OID and **DONT_REUSE_OID** are options that specify whether to be referable when creating a table. The two options cannot be used together and can be used with other options. When creating a table without the option, the **REUSE_OID** table option is used. To change the default option to **DONT_REUSE_OID**, you should change the system parameter **create_table_reuseoid** to **no**. For detail, see Statement/Type-Related Parameters .

```
<table_options> ::= <table_option> [[,] <table_option> ...]
    <table_option> ::= REUSE_OID | DONT_REUSE_OID |
                       COMMENT [=] 'table_comment_string' |
                       [CHARSET charset_name] [COLLATE
collation_name]
```

REUSE_OID

You can specify the **REUSE_OID** option when creating a table, so that OIDs that have been deleted due to the deletion of records (**DELETE**) can be reused when a new record is inserted (**INSERT**). Such a table is called an OID reusable or a non-referable table.

OID (Object Identifier) is an object identifier represented by physical location information such as the volume number, page number and slot number. By using such OIDs, CUBRID manages the reference relationships of objects and searches, stores or deletes them. When an OID is used, accessibility is improved because the object in the heap file can be directly accessed without referring to the table. However, the problem of decreased reusability of the storage occurs when there are many **DELETE/ INSERT** operations because the object's OID is kept to maintain the reference relationship with the object even if it is deleted.

If you specify the **REUSE_OID** option when creating a table, the OID is also deleted when data in the table is deleted, so that another **INSERT**ed data can use it. OID reusable tables cannot be referred to by other tables, and OID values of the objects in the OID reusable tables cannot be viewed.

```
-- creating table with REUSE_OID option specified
CREATE TABLE reuse_tbl (a INT PRIMARY KEY) REUSE_OID, COMMENT =
'reuse oid table';
INSERT INTO reuse_tbl VALUES (1);
INSERT INTO reuse_tbl VALUES (2);
INSERT INTO reuse_tbl VALUES (3);

-- an error occurs when column type is a OID reusable table itself
CREATE TABLE tbl_1 (a reuse_tbl);
```

```
ERROR: The class 'reuse_tbl' is marked as REUSE_OID and is non-
referable. Non-referable classes can't be the domain of an attribute
and their instances' OIDs cannot be returned.
```

If you specify REUSE_OID together with the collation of table, it can be placed on before or after **COLLATE** syntax.

```
CREATE TABLE t3(a VARCHAR(20)) REUSE_OID, COMMENT = 'reuse oid
table', COLLATE euckr_bin;
CREATE TABLE t4(a VARCHAR(20)) COLLATE euckr_bin REUSE_OID;
```

Note

- OID reusable tables cannot be referred to by other tables.

- Updatable views cannot be created for OID reusable tables.
- OID reusable tables cannot be specified as table column type.
- OID values of the objects in the OID reusable tables cannot be read.
- Instance methods cannot be called from OID reusable tables. Also, instance methods cannot be called if a sub class inherited from the class where the method is defined is defined as an OID reusable table.
- OID reusable tables are supported only by CUBRID 2008 R2.2 or above, and backward compatibility is not ensured. That is, the database in which the OID reusable table is located cannot be accessed from a lower version database.
- OID reusable tables can be managed as partitioned tables and can be replicated.
- The updating query with a view can be faced with error when the view includes any OID reusable table.

```
CREATE TABLE t1 (c1 INT) REUSE_OID;
CREATE VIEW v3 AS SELECT c1 FROM t1;

insert into v3(c1) values (1);
```

```
ERROR: Vclass v3 is not updatable. Please check if any of its
related classes are marked as REUSE_OID.
```

DONT_REUSE_OID

Specifying the **DONT_REUSE_OID** option when creating the table will create a referable table as opposite to **REUSE_OID**.

Charset and Collation

The charset and collation of the table and column can be designated in the **CREATE TABLE** statement. This example specifies the character set and collation for **CREATE TABLE**.

```
CREATE TABLE tblutf8(a int, b int) charset utf8;
CREATE TABLE tblutf8enci(a int, b int) collate utf8_en_ci;

CREATE TABLE tbleuckr(a int, b int) charset euckr collate euckr_bin;
```

```
CREATE TABLE tblcoleuckr(a int, c varchar(100) collate euckr_bin)
charset euckr;

CREATE TABLE tblcol (s1 STRING COLLATE utf8_en_cs,s2 STRING COLLATE
utf8_bin);
CREATE TABLE tblcharcol (col STRING CHARSET utf8) COLLATE utf8_en_ci;
```

Please see Charset and Collation of Tables or Charset and Collation of String Literals for details.

Table's COMMENT

You can write a table's comment as following.

```
CREATE TABLE tbl (a INT, b INT) COMMENT = 'this is comment for table
tbl';
```

To see the table's comment, run the below syntax.

```
SHOW CREATE TABLE table_name;
SELECT class_name, comment from db_class;
SELECT class_name, comment from _db_class;
```

Or you can see the table's comment with ;sc command in the CSQL interpreter.

```
$ csql -u dba demodb

csql> ;sc tbl
```

Table Encryption (TDE)

You can encrypt a table as follows. For more information on TDE encryption, see TDE (Transparent Data Encryption).

```
CREATE TABLE enc_tbl (a INT, b INT) ENCRYPT = AES;
```

You can specify **AES** or **ARIA** as the encryption algorithm. If omitted as follows, the encryption algorithm specified by the system parameter **tde_default_algorithm** is used. The default value is **AES**.

```
CREATE TABLE enc_tbl (a INT, b INT) ENCRYPT;
```

The encryption information is not inherited.

CREATE TABLE LIKE

You can create a table with the same schema as an existing table by using the **CREATE TABLE ... LIKE** statement. Column attribute, table constraint, index, and encryption information are replicated from the existing table. An index name created from the existing table changes according to a new table name, but an index name defined by a user is replicated as it is. Therefore, you should be careful with a query statement that is supposed to use a specific index created by using the index hint syntax(see Index Hint).

You cannot create the column definition because the **CREATE TABLE ... LIKE** statement replicates the schema only.

```
CREATE {TABLE | CLASS} [schema_name.]new_table_name LIKE
[schema_name.]source_table_name;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *new_table_name*: A table name to be created
- *source_table_name*: The name of the original table that already exists in the database. The following tables cannot be specified as original tables in the **CREATE TABLE ... LIKE** statement.
  - Partition table
  - Table that contains an **AUTO_INCREMENT** column
  - Table that uses inheritance or methods

```
CREATE TABLE a_tbl (
  id INT NOT NULL DEFAULT 0 PRIMARY KEY,
  phone VARCHAR(10)
);
INSERT INTO a_tbl VALUES (1,'111-1111'), (2,'222-2222'), (3,
'333-3333');

-- creating an empty table with the same schema as a_tbl
CREATE TABLE new_tbl LIKE a_tbl;
SELECT * FROM new_tbl;
```

```
There are no results.

csql> ;schema a_tbl
```

```
=== <Help: Schema of a Class> ===


 <Class Name>

     a_tbl

 <Attributes>

     id                    INTEGER DEFAULT 0 NOT NULL
     phone                 CHARACTER VARYING(10)

 <Constraints>

     PRIMARY KEY pk_a_tbl_id ON a_tbl (id)

csql> ;schema new_tbl

=== <Help: Schema of a Class> ===


 <Class Name>

     new_tbl

 <Attributes>

     id                    INTEGER DEFAULT 0 NOT NULL
     phone                 CHARACTER VARYING(10)

 <Constraints>

     PRIMARY KEY pk_new_tbl_id ON new_tbl (id)
```

CREATE TABLE AS SELECT

You can create a new table that contains the result records of the **SELECT** statement by using the **CREATE TABLE...AS SELECT** statement. You can define column and table constraints for the new table. The following rules are applied to reflect the result records of the **SELECT** statement.

- If *col_1* is defined in the new table and the same column *col_1* is specified in *select_statement*, the result record of the **SELECT** statement is stored as *col_1* value in the new table. Type casting is attempted if the column names are identical but the columns types are different.

- If *col_1* and *col_2* are defined in the new table, *col_1*, col_2 and *col_3* are specified in the column list of the *select_statement* and there is a containment relationship between all of them, *col_1*, *col_2* and *col_3* are created in the new table and the result data of the **SELECT** statement is stored as values for all columns. Type casting is attempted if the column names are identical but the columns types are different.

- If columns *col_1* and *col_2* are defined in the new table and *col_1* and *col_3* are defined in the column list of *select_statement* without any containment relationship between them, *col_1*, *col_2* and *col_3* are created in the new table, the result data of the **SELECT** statement is stored only for *col_1* and *col_3* which are specified in *select_statement*, and **NULL** is stored as the value of *col_2*.

- Column aliases can be included in the column list of *select_statement*. In this case, new column alias is used as a new table column name. It is recommended to use an alias because invalid column name is created, if an alias does not exist when a function calling or an expression is used.

- The **REPLACE** option is valid only when the **UNIQUE** constraint is defined in a new table column (*col_1*). When duplicate values exist in the result record of *select_statement*, a **UNIQUE** value is stored for *col_1* if the **REPLACE** option has been defined, or an error message is displayed if the **REPLACE** option is omitted due to the violation of the **UNIQUE** constraint.

```
CREATE {TABLE | CLASS} [schema_name.]table_name
[([{<table_constraint>}... ,] <column_definition> [{,
{<column_definition> | <table_constraint>}...])] [REPLACE] AS
<select_statement>;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.

- *table_name*: a name of the table to be created.

- <*column_definition*>: defines a column. If this is omitted, the column schema of **SELECT** statement is replicated; however, the constraint or the **AUTO_INCREMENT** attribute is not replicated.

- <*table_constraint*>: defines table constraint.

- <*select_statement*>: a **SELECT** statement targeting a source table that already exists in the database.

- <*select statement*> can include remote tables like tbl@srver1.

```
CREATE TABLE a_tbl (
  id INT NOT NULL DEFAULT 0 PRIMARY KEY,
  phone VARCHAR(10)
);
INSERT INTO a_tbl VALUES (1,'111-1111'), (2,'222-2222'), (3,
'333-3333');

-- creating a table without column definition
```

```
CREATE TABLE new_tbl1 AS SELECT * FROM a_tbl;
SELECT * FROM new_tbl1;
```

```
   id  phone
=================================
    1  '111-1111'
    2  '222-2222'
    3  '333-3333'
```

```
-- all of column values are replicated from a_tbl
CREATE TABLE new_tbl2 (
  id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  phone VARCHAR
) AS SELECT * FROM a_tbl;

SELECT * FROM new_tbl2;
```

```
   id  phone
=================================
    1  '111-1111'
    2  '222-2222'
    3  '333-3333'
```

```
-- some of column values are replicated from a_tbl and the rest is
NULL
CREATE TABLE new_tbl3 (
  id INT,
  name VARCHAR
) AS SELECT id, phone FROM a_tbl;

SELECT * FROM new_tbl3
```

```
  name                            id  phone
=========================================================
  NULL                             1  '111-1111'
  NULL                             2  '222-2222'
  NULL                             3  '333-3333'
```

```
-- column alias in the select statement should be used in the column
definition
CREATE TABLE new_tbl4 (
  id1 INT,
  id2 INT
```

```
) AS SELECT t1.id id1, t2.id id2 FROM new_tbl1 t1, new_tbl2 t2;

SELECT * FROM new_tbl4;
```

```
  id1          id2
============================
    1            1
    1            2
    1            3
    2            1
    2            2
    2            3
    3            1
    3            2
    3            3
```

```
-- REPLACE is used on the UNIQUE column
CREATE TABLE new_tbl5 (id1 int UNIQUE) REPLACE AS SELECT * FROM
new_tbl4;

SELECT * FROM new_tbl5;
```

```
  id1          id2
============================
    1            3
    2            3
    3            3
```

You can create a table by selecting data from a remote table as below statement.

```
-- column values are replicated from remote_tbl
CREATE TABLE new_tbl_rem (
  id INT NON NULL;
  phone VARCHAR;
  localtion VARCHAR;
) AS SELECT id, phone, 'remote' FROM a_tbl@server1 WHERE id < 3;

SELECT * FROM new_tbl_rem;
```

```
  id  phone       location
==================================
   1  '111-1111'  'remote'
   2  '222-2222'  'remote'
```

ALTER TABLE

You can modify the structure of a table by using the **ALTER** statement. You can perform operations on the target table such as adding/deleting columns, creating/deleting indexes, and type casting existing columns as well as changing table names, column names and constraints. You can also change the initial value of **AUTO_INCREMENT**. **TABLE** and **CLASS** are used interchangeably **VIEW** and **VCLASS**, and **COLUMN** and **ATTRIBUTE** as well.

```
ALTER [TABLE | CLASS] [schema_name.]table_name <alter_clause> [,
<alter_clause>] ... ;

    <alter_clause> ::=
        ADD <alter_add> [INHERIT <resolution>, ...]  |
        ADD {KEY | INDEX} <index_name> <index_col_desc> |
        ALTER [COLUMN] column_name SET DEFAULT <value_specification>
|
        DROP <alter_drop> [INHERIT <resolution>, ...] |
        DROP {KEY | INDEX} index_name |
        DROP FOREIGN KEY constraint_name |
        DROP PRIMARY KEY |
        RENAME <alter_rename> [INHERIT <resolution>, ...] |
        CHANGE <alter_change> |
        MODIFY <alter_modify> |
        INHERIT <resolution>, ... |
        AUTO_INCREMENT = <initial_value> |
        COMMENT [=] 'table_comment_string' |
        COMMENT ON {COLUMN | CLASS ATTRIBUTE}
<column_comment_definition> [, <column_comment_definition>] ;

        <alter_add> ::=
            [ATTRIBUTE|COLUMN] [(]<class_element>, ...[)] [FIRST|
AFTER old_column_name] |
            CLASS ATTRIBUTE <column_definition>, ... |
            CONSTRAINT <constraint_name> <column_constraint>
(column_name) |
            QUERY <select_statement> |
            SUPERCLASS [schema_name.]superclass_name, ...

            <class_element> ::= <column_definition> |
<table_constraint>

            <column_constraint> ::= UNIQUE [KEY] | PRIMARY KEY |
FOREIGN KEY

        <alter_drop> ::=
            [ATTRIBUTE | COLUMN]
            {
                column_name, ... |
                QUERY [<unsigned_integer_literal>] |
                SUPERCLASS [schema_name.]superclass_name, ... |
                CONSTRAINT constraint_name
```

```
            }

        <alter_rename> ::=
            [ATTRIBUTE | COLUMN]
            {
                old_column_name AS new_column_name |
                FUNCTION OF column_name AS function_name
            }

        <alter_change> ::=
            [COLUMN | CLASS ATTRIBUTE] old_col_name new_col_name
<column_definition>
                [FIRST | AFTER col_name]

        <alter_modify> ::=
            [COLUMN | CLASS ATTRIBUTE] col_name <column_definition>
                [FIRST | AFTER col_name2]

        <table_option> ::=
            CHANGE [COLUMN | CLASS ATTRIBUTE] old_col_name
new_col_name <column_definition>
                [FIRST | AFTER col_name2]
          | MODIFY [COLUMN | CLASS ATTRIBUTE] col_name
<column_definition>
                [FIRST | AFTER col_name2]

        <resolution> ::= column_name OF
[schema_name.]superclass_name [AS alias]

        <index_col_name> ::= column_name [(length)] [ASC | DESC]

        <column_comment_definition> ::= column_name [=]
'column_comment_string'
```

Note

A column's comment is specified in *<column_definition>* or *<column_comment_definition>*. For *<column_definition>*, see the above CREATE TABLE syntax.

Warning

The table's name can be changed only by the table owner, **DBA** and **DBA** members. The other users must be granted to change the name by the owner or **DBA** (see GRANT For details on authorization).

ADD COLUMN Clause

You can add a new column by using the **ADD COLUMN** clause. You can specify the location of the column to be added by using the **FIRST** or **AFTER** keyword.

```
ALTER [TABLE | CLASS] [schema_name.]table_name
ADD [COLUMN | ATTRIBUTE] [(] <column_definition> [FIRST | AFTER
old_column_name] [)];

    <column_definition> ::=
        column_name <data_type> [[<default_or_shared_or_ai>] |
[<on_update>] | [<column_constraint>]] [COMMENT 'comment_string']

        <data_type> ::= <column_type> [<charset_modifier_clause>]
[<collation_modifier_clause>]

            <charset_modifier_clause> ::= {CHARACTER_SET|CHARSET}
{<char_string_literal>|<identifier>}

            <collation_modifier_clause> ::= COLLATE
{<char_string_literal>|<identifier>}

        <default_or_shared_or_ai> ::=
            SHARED <value_specification> |
            DEFAULT <value_specification>  |
            AUTO_INCREMENT [(seed, increment)]

        <on_update> ::= [ON UPDATE <value_specification>]

        <column_constraint> ::= [CONSTRAINT constraint_name] {NOT
NULL | UNIQUE | PRIMARY KEY | [FOREIGN KEY] [WITH
<index_with_option>] <referential_definition>}

            <referential_definition> ::=
                REFERENCES [schema_name.]referenced_table_name
(column_name, ...) [<referential_triggered_action> ...]

                <referential_triggered_action> ::=
                    ON UPDATE <referential_action> |
                    ON DELETE <referential_action>

                    <referential_action> ::= CASCADE | RESTRICT | NO
ACTION | SET NULL

            <index_with_option> ::= {DEDUPLICATE '='
deduplicate_level}
```

- *schema_name*: specifies the schema name. If omitted, the schema name of the current session is used.

- *table_name*: specifies the name of a table that has a column to be added.
- <*column_definition*>: specifies the name(max 254 bytes), data type, and constraints of a column to be added.
- **AFTER** *oid_column_name*: specifies the name of an existing column before the column to be added.
- *comment_string*: specifies a column's comment.
- *deduplicate_level*: Specifies the deduplicate level (0 to 14). For details, see DEDUPLICATE.

```
CREATE TABLE a_tbl;
ALTER TABLE a_tbl ADD COLUMN age INT DEFAULT 0 NOT NULL COMMENT 'age
comment';
ALTER TABLE a_tbl ADD COLUMN name VARCHAR FIRST;
ALTER TABLE a_tbl ADD COLUMN id INT NOT NULL AUTO_INCREMENT UNIQUE
FIRST;
INSERT INTO a_tbl(age) VALUES(20),(30),(40);

ALTER TABLE a_tbl ADD COLUMN phone VARCHAR(13) DEFAULT
'000-0000-0000' AFTER name;
ALTER TABLE a_tbl ADD COLUMN birthday VARCHAR(20) DEFAULT
TO_CHAR(SYSDATE,'YYYY-MM-DD');

SELECT * FROM a_tbl;
```

```
       id  name                 phone                          age
birthday
=============================================================================
=======================
        1  NULL                 '000-0000-0000'                 20
'2017-05-24'
        2  NULL                 '000-0000-0000'                 30
'2017-05-24'
        3  NULL                 '000-0000-0000'                 40
'2017-05-24'

--adding multiple columns
ALTER TABLE a_tbl ADD COLUMN (age1 int, age2 int, age3 int);
```

The result when you add a new column depends on what constraints are added.

- If there is a **DEFAULT** constraint on the newly added column, **DEFAULT** value is inserted.
- If there is no **DEFAULT** constraint and there is a **NOT NULL** constraint, hard default value is inserted when a value of system parameter **add_column_update_hard_default** is **yes**; however, it returns an error when a value of **add_column_update_hard_default** is **no**.

The default of add_column_update_hard_default is **no**.

Depending on **DEFAULT** constraint and **add_column_update_hard_default**'s value, if they do not violate their constraints, it is possible to add **PRIMARY KEY** constraint or **UNIQUE** constraint.

- If the newly added column when there is no data on the table, or the newly added column with **NOT NULL** and **UNIQUE** data can have **PRIMARY KEY** constraint.
- If you try to add a new column with **PRIMARY KEY** constraint when there is data on the table, it returns an error.

```
CREATE TABLE tbl (a INT);
INSERT INTO tbl VALUES (1), (2);
ALTER TABLE tbl ADD COLUMN (b int PRIMARY KEY);
```

```
ERROR: NOT NULL constraints do not allow NULL value.
```

- If there is data and **UNIQUE** constraint is specified on the newly added data, **NULL** is inserted when there is no **DEFAULT** constraint.

```
ALTER TABLE tbl ADD COLUMN (b int UNIQUE);
SELECT * FROM tbl;
```

```
    a            b
==================
    1         NULL
    2         NULL
```

- If there is data on the table and **UNIQUE** constraint is specified on the newly added column, unique violation error is returned when there is **DEFAULT** constraint.

```
ALTER TABLE tbl ADD COLUMN (c int UNIQUE DEFAULT 10);
```

```
ERROR: Operation would have caused one or more unique constraint
violations.
```

- If there is data on the table and **UNIQUE** constraint is specified on the newly added column, unique violation error is returned when there is **NOT NULL** constraint and the value of system parameter add_column_update_hard_default is yes.

```
SET SYSTEM PARAMETERS 'add_column_update_hard_default=yes';
ALTER TABLE tbl ADD COLUMN (c int UNIQUE NOT NULL);
```

```
ERROR: Operation would have caused one or more unique constraint
violations.
```

For **add_column_update_hard_default** and the hard default, see CHANGE/MODIFY Clauses.

ADD CONSTRAINT Clause

You can add a new constraint by using the **ADD CONSTRAINT** clause.

By default, the index created when you add **PRIMARY KEY** constraints is created in ascending order, and you can define the key sorting order by specifying the **ASC** or **DESC** keyword next to the column name.

```
ALTER [ TABLE | CLASS | VCLASS | VIEW ] [schema_name.]table_name
ADD <table_constraint> ;

    <table_constraint> ::=
        {
            {KEY|INDEX} index_name (column_name, ...) |
            [CONSTRAINT [constraint_name]]
               {
                  UNIQUE [KEY|INDEX](column_name, ...) |
                  PRIMARY KEY (column_name, ...) |
                  <referential_constraint>
               }
        }

        <referential_constraint> ::= FOREIGN KEY [foreign_key_name]
(column_name, ...) <referential_definition>

            <referential_definition> ::=
                REFERENCES [schema_name.]referenced_table_name
(column_name, ...) [<referential_triggered_action> ...]

                <referential_triggered_action> ::=
                    ON UPDATE <referential_action> |
                    ON DELETE <referential_action>

                    <referential_action> ::= CASCADE | RESTRICT | NO
ACTION | SET NULL
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.

- *table_name*: Specifies the name of a table that has a constraint to be added.
- *constraint_name*: Specifies the name of a constraint to be added, or it can be omitted. If omitted, a name is automatically assigned(maximum: 254 bytes).
- *foreign_key_name*: Specifies a name of the **FOREIGN KEY** constraint. You can skip the name specification. However, if you specify this value, *constraint_name* will be ignored, and the specified value will be used.
- <*table_constraint*>: Defines a constraint for the specified table. For details, see ON UPDATE.

```
ALTER TABLE a_tbl ADD CONSTRAINT pk_a_tbl_id PRIMARY KEY(id);
ALTER TABLE a_tbl DROP CONSTRAINT pk_a_tbl_id;
ALTER TABLE a_tbl ADD CONSTRAINT pk_a_tbl_id PRIMARY KEY(id, name
DESC);
ALTER TABLE a_tbl ADD CONSTRAINT u_key1 UNIQUE (id);
```

ADD INDEX Clause

You can define the index attributes for a specific column by using the **ADD INDEX** clause.

```
ALTER [TABLE | CLASS] [schema_name.]table_name ADD {KEY | INDEX}
index_name (<index_col_name>) ;

    <index_col_name> ::= column_name [(length)] [ ASC | DESC ]
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table to be modified.
- *index_name* : Specifies the name of an index(maximum: 254 bytes). If omitted, a name is automatically assigned.
- *index_col_name* : Specifies the column that has an index to be defined. **ASC** or **DESC** can be specified for a column option.

```
ALTER TABLE a_tbl ADD INDEX i1(age ASC), ADD INDEX i2(phone DESC);
```

```
csql> ;schema a_tbl

=== <Help: Schema of a Class> ===

 <Class Name>
```

```
     a_tbl

<Attributes>

     name                 CHARACTER VARYING(1073741823) DEFAULT ''
     phone                CHARACTER VARYING(13) DEFAULT '111-1111'
     age                  INTEGER
     id                   INTEGER AUTO_INCREMENT  NOT NULL

 <Constraints>

     UNIQUE u_a_tbl_id ON a_tbl (id)
     INDEX i1 ON a_tbl (age)
     INDEX i2 ON a_tbl (phone DESC)
```

The below is an example to include an index's comment when you add an index with ALTER statement.

```
ALTER TABLE tbl ADD index i_t_c (c) COMMENT 'index comment c';
```

ALTER COLUMN ... SET DEFAULT Clause

You can specify a new default value for a column that has no default value or modify the existing default value by using the **ALTER COLUMN** ... **SET DEFAULT**. You can use the **CHANGE** clause to change the default value of multiple columns with a single statement. For details, see the CHANGE/MODIFY Clauses.

```
ALTER [TABLE | CLASS] [schema_name.]table_name ALTER [COLUMN]
column_name SET DEFAULT value ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table that has a column whose default value is to be modified.
- *column_name* : Specifies the name of a column whose default value is to be modified.
- *value* : Specifies a new default value.

```
csql> ;schema a_tbl

=== <Help: Schema of a Class> ===


 <Class Name>
```

```
    a_tbl

 <Attributes>

    name                CHARACTER VARYING(1073741823)
    phone               CHARACTER VARYING(13) DEFAULT
'000-0000-0000'
    age                 INTEGER
    id                  INTEGER AUTO_INCREMENT  NOT NULL

 <Constraints>

    UNIQUE u_a_tbl_id ON a_tbl (id)
```

```
ALTER TABLE a_tbl ALTER COLUMN name SET DEFAULT '';
ALTER TABLE a_tbl ALTER COLUMN phone SET DEFAULT '111-1111';
```

```
csql> ;schema a_tbl

=== <Help: Schema of a Class> ===


 <Class Name>

    a_tbl

 <Attributes>

    name                CHARACTER VARYING(1073741823) DEFAULT ''
    phone               CHARACTER VARYING(13) DEFAULT '111-1111'
    age                 INTEGER
    id                  INTEGER AUTO_INCREMENT  NOT NULL

 <Constraints>

    UNIQUE u_a_tbl_id ON a_tbl (id)
```

```
CREATE TABLE t1(id1 VARCHAR(20), id2 VARCHAR(20) DEFAULT '');
ALTER TABLE t1 ALTER COLUMN id1 SET DEFAULT TO_CHAR(SYS_DATETIME,
'yyyy/mm/dd hh:mi:ss');
```

```
csql> ;schema t1

=== <Help: Schema of a Class> ===
```

```
<Class Name>

     t1

<Attributes>

     id1                  CHARACTER VARYING(20) DEFAULT
TO_CHAR(SYS_DATETIME, 'yyyy/mm/dd hh:mi:ss')
     id2                  CHARACTER VARYING(20) DEFAULT ''
```

AUTO_INCREMENT Clause

The **AUTO_INCREMENT** clause can change the initial value of the increment value that is currently defined. However, there should be only one **AUTO_INCREMENT** column defined.

```
ALTER TABLE [schema_name.]table_name AUTO_INCREMENT = initial_value ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Table name
- *initial_value* : Initial value to alter

```
CREATE TABLE t (i int AUTO_INCREMENT);
ALTER TABLE t AUTO_INCREMENT = 5;

CREATE TABLE t (i int AUTO_INCREMENT, j int AUTO_INCREMENT);

-- when 2 AUTO_INCREMENT constraints are defined on one table, below
query returns an error.
ALTER TABLE t AUTO_INCREMENT = 5;
```

```
ERROR: To avoid ambiguity, the AUTO_INCREMENT table option requires
the table to have exactly one AUTO_INCREMENT column and no seed/
increment specification.
```

⚠ Warning

You must be careful not to violate constraints (such as a **PRIMARY KEY** or **UNIQUE**) due to changing the initial value of **AUTO_INCREMENT**.

Note

If you change the type of **AUTO_INCREMENT** column, the maximum value is changed, too. For example, if you change INT to BIGINT, the maximum value of **AUTO_INCREMENT** is changed from the maximum INT into the maximum BIGINT.

CHANGE/MODIFY Clauses

The **CHANGE** clause changes column name, type, size, and attribute. If the existing column name and a new column name are the same, types, size, and attribute will be changed.

The **MODIFY** clause can modify type, size, and attribute of a column but cannot change its name.

If you set the type, size, and attribute to apply to a new column with the **CHANGE** clause or the **MODIFY** clause, the attribute that is currently defined will not be passed to the attribute of the new column.

When you change data types using the **CHANGE** clause or the **MODIFY** clause, the data can be modified. For example, if you shorten the length of a column, the character string may be truncated if the value of configuration parameter alter_table_change_type_strict is set to **no**. But if the parameter value is set to **yes**, the change or modify is not allowed and it returns an error. the configuration parameter allow_truncated_string also affect the similar as alter_table_change_type_strict.

When changing the type of the column specified AUTO_INCREMENT, it cannot be changed to a column type that is not allowed to used with the AUTO_INCREMENT. For example, if you try to change the column type of “a” from INT to VARCHAR with the ALTER statement as shown below, an error occurs.

```
CREATE a_tbl (a int AUTO_INCREMENT, b VARCHAR);
ALTER TABLE a_tbl MODIFY COLUMN a VARCHAR;
```

```
ERROR: before '  varchar; '
The domain of the attribute 'a' having an auto increment constraint
is invalid.
```

When changing the type of a column specified a default value, if the default value cannot be coerced to the changed type, an error occurs as shown in the example below.

```
CREATE TABLE t_def (a bigint default 123456789012, b varchar(20));
ALTER TABLE t_def MODIFY a int;
```

```
ERROR: A domain conflict exists on attribute "a".
```

⚠ Warning

- **ALTER TABLE** *[schema_name.]table_name* **CHANGE** *column_name* **DEFAULT** *default_value* syntax supported in CUBRID 2008 R3.1 or earlier version is no longer supported.
- When converting a number type to character type, if alter_table_change_type_strict=no and the length of the string is shorter than that of the number, the string is truncated and saved according to the length of the converted character type. If alter_table_change_type_strict=yes, it returns an error.
- If the column attributes like a type, a collation, etc. are changed, the changed attributes are not applied into the view created with the table before the change. Therefore, if you change the attributes of a table, it is recommended to recreate the related views.

```
ALTER [/*+ SKIP_UPDATE_NULL */] TABLE [schema_name.]tbl_name
<table_options> ;

    <table_options> ::=
        <table_option>[, <table_option>, ...]

        <table_option> ::=
            CHANGE [COLUMN | CLASS ATTRIBUTE] old_col_name
new_col_name <column_definition>
                    [FIRST | AFTER col_name]
          | MODIFY [COLUMN | CLASS ATTRIBUTE] col_name
<column_definition>
                    [FIRST | AFTER col_name]
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *tbl_name*: specifies the name of the table including the column to change.
- *old_col_name*: specifies the existing column name.

- *new_col_name*: specifies the column name to change
- <*column_definition*>: specifies the type, the size, the attribute, and the comment of the column to be changed.
- *col_name*: specifies the location where the column to change exists.
- **SKIP_UPDATE_NULL**: If this hint is added, CUBRID does not check the previous NULLs even if NOT NULL constraint is added. See SKIP_UPDATE_NULL.

```
CREATE TABLE t1 (a INTEGER);

-- changing column a's name into a1
ALTER TABLE t1 CHANGE a a1 INTEGER;

-- changing column a1's constraint
ALTER TABLE t1 CHANGE a1 a1 INTEGER NOT NULL;
---- or
ALTER TABLE t1 MODIFY a1 INTEGER NOT NULL;

-- changing column col1's type - "DEFAULT 1" constraint is removed.
CREATE TABLE t1 (col1 INT DEFAULT 1);
ALTER TABLE t1 MODIFY col1 BIGINT;

-- changing column col1's type - "DEFAULT 1" constraint is kept.
CREATE TABLE t1 (col1 INT DEFAULT 1, b VARCHAR(10));
ALTER TABLE t1 MODIFY col1 BIGINT DEFAULT 1;

-- changing column b's size
ALTER TABLE t1 MODIFY b VARCHAR(20);

-- changing the name and position of a column
CREATE TABLE t1 (i1 INT, i2 INT);
INSERT INTO t1 VALUES (1,11), (2,22), (3,33);

SELECT * FROM t1 ORDER BY 1;
```

```
            i1           i2
=========================
             1           11
             2           22
             3           33
```

```
ALTER TABLE t1 CHANGE i2 i0 INTEGER FIRST;
SELECT * FROM t1 ORDER BY 1;
```

```
            i0           i1
=========================
            11            1
            22            2
            33            3
```

```
ALTER TABLE t1 MODIFY i1 VARCHAR (200) DEFAULT TO_CHAR (SYS_DATE);
INSERT INTO t1(i0) VALUES (17);
SELECT * FROM t1 ORDER BY 1;
```

```
           i0  i1
=====================================
           11  '1'
           17  '05/24/2017'
           22  '2'
           33  '3'
```

```
-- adding NOT NULL constraint (strict)
SET SYSTEM PARAMETERS 'alter_table_change_type_strict=yes';

CREATE TABLE t1 (i INT);
INSERT INTO t1 VALUES (11), (NULL), (22);

ALTER TABLE t1 CHANGE i i1 INTEGER NOT NULL;
```

```
ERROR: Cannot add NOT NULL constraint for attribute "i1": there are
existing NULL values for this attribute.
```

```
-- adding NOT NULL constraint
SET SYSTEM PARAMETERS 'alter_table_change_type_strict=no';

CREATE TABLE t1 (i INT);
INSERT INTO t1 VALUES (11), (NULL), (22);

ALTER TABLE t1 CHANGE i i1 INTEGER NOT NULL;

SELECT * FROM t1;
```

```
           i1
=============
           22
```

```
            0
           11
```

```
-- change the column's data type (no errors)

CREATE TABLE t1 (i1 INT);
INSERT INTO t1 VALUES (1), (-2147483648), (2147483647);

ALTER TABLE t1 CHANGE i1 s1 CHAR(11);
SELECT * FROM t1;
```

```
  s1
======================
  '2147483647 '
  '-2147483648'
  '1          '
```

```
-- change the column's data type (errors), strict mode
SET SYSTEM PARAMETERS 'alter_table_change_type_strict=yes';

CREATE TABLE t1 (i1 INT);
INSERT INTO t1 VALUES (1), (-2147483648), (2147483647);

ALTER TABLE t1 CHANGE i1 s1 CHAR(4);
```

```
ERROR: ALTER TABLE .. CHANGE : changing to new domain : cast failed,
current configuration doesn't allow truncation or overflow.
```

```
-- change the column's data type (errors)
SET SYSTEM PARAMETERS 'alter_table_change_type_strict=no';

CREATE TABLE t1 (i1 INT);
INSERT INTO t1 VALUES (1), (-2147483648), (2147483647);

ALTER TABLE t1 CHANGE i1 s1 CHAR(4);
SELECT * FROM t1;
```

```
-- hard default values have been placed instead of signaling overflow

  s1
======================
  '1   '
```

```
  '-214'
  '2147'
```

Note

When you change NULL constraint into NOT NULL, it takes a long time by the time updating values into **hard default**; to resolve this problem, CUBRID can skip updating values which already exists by using **SKIP_UPDATE_NULL**. However, you should consider that NULL values which do not match with the NOT NULL constraints, can exists.

```
ALTER /*+ SKIP_UPDATE_NULL */ TABLE foo MODIFY col INT NOT NULL;
```

Changes of Table Attributes based on Changes of Column Type

- Type Change : If the value of the system parameter **alter_table_change_type_strict** is set to no, then changing values to other types is allowed, but if it is set to yes then changing is not allowed. The default value of the parameter is **yes**. You can change values to all types allowed by the **CAST** operator. Changing object types is allowed only by the upper classes (tables) of the objects. Also, if the system parameter **allow_truncated_string** is **no** when changing to a string such as **char** or **varchar**, the overflowed string cannot be changed. The default value for the parameter is **no**.

- **NOT NULL**

  - Even though the **NOT NULL** constraint is not specified, it will not be removed from a new table if the constraint is present in the existing table. If you want to remove the constraint **NOT NULL**, specify the **NULL** in the statement.

  - If the **NOT NULL** constraint is specified in the column to change, the result varies depending on the configuration of the system parameter, **alter_table_change_type_strict**.

    - If **alter_table_change_type_strict** is set to yes, the column values will be checked. If **NULL** exists, an error will occur, and the change will not be executed.

    - If the **alter_table_change_type_strict** is set to no, every existing **NULL** value will be changed to a hard default value of the type to change.

- **DEFAULT** : Even though the **DEFAULT** attribute is not specified in the column to change, it will not be removed from a new table if the attribute is present in the existing table. If you want to remove the attribute **DEFAULT**, specify the **DEFAULT NULL** in the statement.

- **COMMENT** : Even though the **COMMENT** attribute is not specified in the column to change, it will not be removed from a new table if the attribute is present in the existing table. If you want to remove the attribute **COMMENT**, specify the **COMMENT ''** in the statement.

- **AUTO_INCREMENTEven though the AUTO_INCREMENT attribute is not specified in the column to change, it will not be removed from a new table if the attribute is present in the existing table.**
    - caution) The **AUTO_INCREMENT** attribute cannot be removed from a new table once the attribute is set by CREATE or ALTER.

- **ON UPDATEEven though the ON UPDATE attribute is not specified in the column to change, it will not be removed from a new table if the attribute is present in the existing table.**
    - caution) The **ON UPDATE** attribute cannot be removed from a new table once the attribute is set by CREATE or ALTER.

- **FOREIGN KEY** : You cannot change the column with the foreign key constraint that is referred to or refers to.

- Single Column **PRIMARY KEY**
    - If the **PRIMARY KEY** constraint is specified in the column to change, a **PRIMARY KEY** is re-created only in which a **PRIMARY KEY** constraint exists in the existing column and the type is upgraded.
    - If the **PRIMARY KEY** constraint is specified in the column to change but doesn't exist in the existing column, a **PRIMARY KEY** will be created.
    - If a **PRIMARY KEY** constraint exists but is not specified in the column to change, the **PRIMARY KEY** will be maintained.

- Multicolumn **PRIMARY KEY**: If the **PRIMARY KEY** constraint is specified and the type is upgraded, a **PRIMARY KEY** will be re-created.

- Single Column **UNIQUE KEY**
    - If the type is upgraded, a **UNIQUE KEY** will be re-created.
    - If a **UNIQUE KEY** exists in the existing column and it is not specified in the column to change, it will be maintained.
    - If a **UNIQUE KEY** exists in the existing column to change, it will be created.

- Multicolumn **UNIQUE KEY**: If the column type is changed, an index will be re-created.

- Column with a Non-unique Index : If the column type is changed, an index will be re-created.

- Partition Column: If a table is partitioned by a column, the column cannot be changed. Partitions cannot be added.

- Column with a Class Hierarchy : You can only change the tables that do not have a lower class. You cannot change the lower class that inherits from an upper class. You cannot change the inherited attributes.
- Trigger and View : You must redefine triggers and views directly because they are not changed according to the definition of the column to change.
- Column Sequence : You can change the sequence of columns.
- Name Change : You can change names as long as they do not conflict.

Changes of Values based on Changes of Column Type

The **alter_table_change_type_strict** parameter determines whether the value conversion is allowed according to the type change. If the value is no, it can be changed when you change a column type or add a **NOT NULL** constraint. The default value is **yes**.

When the value of the parameter, **alter_table_change_type_strict** is no, it will operate depending on the conditions as follows:

- Overflow occurred while converting numbers or character strings to Numbers: It is determined based on symbol of the result type. If it is negative value, it is specified as a minimum value or positive value, specified as the maximum value and a warning message for records where overflow occurred is recorded in the log. For strings, it will follow the rules stated above after it is converted to **DOUBLE** type. Overflow can also be returned by the parameter **allow_truncated_string** setting to **no** if the converted string does not fit the length of the target string type.
- Character strings to convert to shorter ones: The record will be updated to the hard default value of the type that is defined and the warning message will be recorded in a log. Converting to shorter ones is not allowed when the **allow_truncated_string** is set to **no**.
- Conversion failure due to other reasons: The record will be updated to the hard default value of the type that is defined and the warning message will be recorded in a log.

If the value of the **alter_table_change_type_strict** parameter is yes or **allow_truncated_string** is set to no, an error message will be displayed and the changes will be rolled back.

The **ALTER CHANGE** statement checks the possibility of type conversion before updating a record but the type conversion of specific values may fail. For example, if the value format is not correct when you convert **VARCHAR** to **DATE**, the conversion may fail. In this case, the hard default value of the **DATE** type will be assigned.

The hard default value is a value that will be used when you add columns with the **ALTER TABLE ... ADD COLUMN** statement, add or change by converting types with the **ALTER TABLE ... CHANGE/**

MODIFY statement. The operation will vary depending on the system parameter, **add_column_update_hard_default** in the **ADD COLUMN** statement.

Hard Default Value by Type

Type	Existence of Hard Default Value	Hard Default Value
INTEGER	Yes	0
FLOAT	Yes	0
DOUBLE	Yes	0
SMALLINT	Yes	0
DATE	Yes	date'01/01/0001'
TIME	Yes	time'00:00'
DATETIME	Yes	datetime'01/01/0001 00:00'
TIMESTAMP	Yes	timestamp'00:00:01 AM 01/01/1970' (GMT)
NUMERIC	Yes	0
CHAR	Yes	''
VARCHAR	Yes	''
SET	Yes	{}

Type	Existence of Hard Default Value	Hard Default Value
MULTISET	Yes	{}
SEQUENCE	Yes	{}
BIGINT	Yes	0
BIT	NO	
VARBIT	No	
OBJECT	No	
BLOB	No	
CLOB	No	

Column's COMMENT

A column's comment is specified in <*column_definition*> or <*column_comment_definition*>. <*column_definition*> is located at the end of ADD/MODIFY/CHANGE syntax and <*column_comment_definition*> is located at the end of COMMENT ON COLUMN syntax. To see the meaning of <*column_definition*>, refer to CREATE TABLE syntax on the above.

In the COMMENT ON COLUMN syntax, you can change column comments by specifying one or more columns. The following example shows how to change a column comment using the COMMENT ON COLUMN statement.

```
ALTER TABLE t1 COMMENT ON COLUMN c1 = 'changed table column c1
comment';
ALTER TABLE t1 COMMENT ON COLUMN c2 = 'changed table column c2
comment', c3 = 'changed table column c3 comment';
```

The below is a syntax to show a column's comment.

```
SHOW CREATE TABLE t1 /* table_name */ ;

SELECT attr_name, class_name, comment
FROM db_attribute
WHERE class_name = t1 /* lowercase_table_name */ ;

SHOW FULL COLUMNS FROM t1 /* table_name */ ;
```

You can see this comment with the ";sc table_name" command in the CSQL interpreter.

```
$ csql -u dba demodb

csql> ;sc table_name
```

RENAME COLUMN Clause

You can change the name of the column by using the **RENAME COLUMN** clause.

```
ALTER [TABLE | CLASS | VCLASS | VIEW] [schema_name.]table_name
RENAME [COLUMN | ATTRIBUTE] old_column_name {AS | TO}
new_column_name ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table that has a column to be renamed.
- *old_column_name* : Specifies the name of a column.
- *new_column_name* : Specifies a new column name after the **AS** keyword(maximum: 254 bytes).

```
CREATE TABLE a_tbl (id INT, name VARCHAR(50));
ALTER TABLE a_tbl RENAME COLUMN name AS name1;
```

DROP COLUMN Clause

You can delete a column in a table by using the **DROP COLUMN** clause. You can specify multiple columns to delete simultaneously by separating them with commas (,).

```
ALTER [TABLE | CLASS | VCLASS | VIEW] [schema_name.]table_name
DROP [COLUMN | ATTRIBUTE] column_name, ... ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table that has a column to be deleted.
- *column_ name* : Specifies the name of a column to be deleted. Multiple columns can be specified by separating them with commas (,).

```
ALTER TABLE a_tbl DROP COLUMN age1,age2,age3;
```

DROP CONSTRAINT Clause

You can drop the constraints pre-defined for the table, such as **UNIQUE**, **PRIMARY KEY** and **FOREIGN KEY** by using the **DROP CONSTRAINT** clause. In this case, you must specify a constraint name. You can check these names by using the CSQL command (**;schema table_name**).

```
ALTER [TABLE | CLASS] [schema_name.]table_name
DROP CONSTRAINT constraint_name ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table that has a constraint to be dropped.
- *constraint_name* : Specifies the name of a constraint to be dropped.

```
CREATE TABLE a_tbl (
  id INT NOT NULL DEFAULT 0 PRIMARY KEY,
  phone VARCHAR(10)
);

CREATE TABLE b_tbl (
  ID INT NOT NULL,
  name VARCHAR (10) NOT NULL,
  CONSTRAINT u_name UNIQUE (name),
  CONSTRAINT pk_id PRIMARY KEY (id),
  CONSTRAINT fk_id FOREIGN KEY (id) REFERENCES a_tbl (id)
  ON DELETE CASCADE ON UPDATE RESTRICT
);

ALTER TABLE b_tbl DROP CONSTRAINT pk_id;
```

```
ALTER TABLE b_tbl DROP CONSTRAINT fk_id;
ALTER TABLE b_tbl DROP CONSTRAINT u_name;
```

DROP INDEX Clause

You can delete an index defined for a column by using the **DROP INDEX** clause. A unique index can be dropped with a **DROP CONSTRAINT** clause.

```
ALTER [TABLE | CLASS] [schema_name.]table_name DROP INDEX
index_name ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table of which constraints will be deleted.
- *index_name* : Specifies the name of an index to be deleted.

```
ALTER TABLE a_tbl DROP INDEX i_a_tbl_age;
```

DROP PRIMARY KEY Clause

You can delete a primary key constraint defined for a table by using the **DROP PRIMARY KEY** clause. You do have to specify the name of the primary key constraint because only one primary key can be defined by table.

```
ALTER [TABLE | CLASS] [schema_name.]table_name DROP PRIMARY KEY ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table that has a primary key constraint to be deleted.

```
ALTER TABLE a_tbl DROP PRIMARY KEY;
```

DROP FOREIGN KEY Clause

You can drop a foreign key constraint defined for a table using the **DROP FOREIGN KEY** clause.

```
ALTER [TABLE | CLASS] [schema_name.]table_name DROP FOREIGN KEY
constraint_name ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of a table whose constraint is to be deleted.
- *constraint_name* : Specifies the name of foreign key constraint to be deleted.

```
ALTER TABLE b_tbl ADD CONSTRAINT fk_id FOREIGN KEY (id) REFERENCES
a_tbl (id);
ALTER TABLE b_tbl DROP FOREIGN KEY fk_id;
```

DROP TABLE

You can drop an existing table by the **DROP** statement. Multiple tables can be dropped with a single **DROP** statement. All rows of table are also dropped. If you also specify **IF EXISTS** clause, no error will be happened even if a target table does not exist.

```
DROP [TABLE | CLASS] [IF EXISTS] <table_specification_comma_list>
[CASCADE CONSTRAINTS] ;

    <table_specification_comma_list> ::=
        <single_table_spec> | (<table_specification_comma_list>)

        <single_table_spec> ::=
            | [ONLY] [schema_name.]table_name
            | ALL [schema_name.]table_name [( EXCEPT
[schema_name.]table_name, ... )]
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table to be dropped. You can delete multiple tables simultaneously by separating them with commas.
- If a super class name is specified after the **ONLY** keyword, only the super class, not the sub classes inheriting from it, is deleted. If a super class name is specified after the **ALL** keyword, the super classes as well as the sub classes inheriting from it are all deleted. You can specify the list of sub classes not to be deleted after the **EXCEPT** keyword.
- If sub classes that inherit from the super class specified after the **ALL** keyword are specified after the **EXCEPT** keyword, they are not deleted.

- Specifies the list of subclasses which are not to be deleted after the **EXCEPT** keyword.
- **CASCADE CONSTRAINTS**: The table is dropped and also foreign keys of other tables which refer this table are dropped.

```
CREATE TABLE b_tbl (i INT);
CREATE TABLE a_tbl (i INT);

-- DROP TABLE IF EXISTS
DROP TABLE IF EXISTS b_tbl, a_tbl;

SELECT * FROM a_tbl;
```

```
ERROR: Unknown class "a_tbl".
```

- If **CASCADE CONSTRAINTS** is specified, the specified table is dropped even if some tables refer the dropping table's primary key; foreign keys of other tables which refer this table are also dropped. However, the data of tables which are referred are not deleted.

The below shows to drop a_parent table which b_child table refers. A foreign key of b_child table also dropped, and the data of b_child table are kept.

```
CREATE TABLE a_parent (
    id INTEGER PRIMARY KEY,
    name VARCHAR(10)
);
CREATE TABLE b_child (
    id INTEGER PRIMARY KEY,
    parent_id INTEGER,
    CONSTRAINT fk_parent_id FOREIGN KEY(parent_id) REFERENCES
a_parent(id) ON DELETE CASCADE ON UPDATE RESTRICT
);

DROP TABLE a_parent CASCADE CONSTRAINTS;
```

RENAME TABLE

You can change the name of a table by using the **RENAME TABLE** statement and specify a list of the table name to change the names of multiple tables.

```
RENAME  [TABLE | CLASS] [schema_name.]old_table_name {AS | TO}
[schema_name.]new_table_name [{, [schema_name.]old_table_name {AS |
TO} [schema_name.]new_table_name}];
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used. The schema of the table to be changed and the schema of the new table must be the same.
- *old_table_name* : Specifies the old table name to be renamed.
- *new_table_name* : Specifies a new table name(maximum: 254 bytes).

```
RENAME TABLE a_tbl AS aa_tbl;
RENAME TABLE aa_tbl TO a1_tbl, b_tbl TO b1_tbl;
```

Note

The table name can be changed only by the table owner, **DBA** and **DBA** members. The other users must be granted to change the name by the owner or **DBA** (see GRANT for details about authorization).

INDEX DEFINITION STATEMENTS

CREATE INDEX

Creates an index to a specified table by using the **CREATE INDEX** statement. Regarding writing index name, see Identifier.

For how to use indexes on the **SELECT** statement like Using SQL Hint, Descending Index, Covering Index, Index Skip Scan, **ORDER BY** Optimization and **GROUP BY** Optimization, and how to create Filtered Index and Function-based Index, see Join Methods.

```
CREATE [UNIQUE] INDEX index_name ON [schema_name.]table_name
<index_col_desc2> ;

    <index_col_desc2> ::=
              {
                 (  {column_name | function_name (argument_list)}
[ASC | DESC]
                   [{, {column_name | function_name (argument_list)}
```

```
[ASC | DESC]} ...] )
              }
              [WHERE <filter_predicate> ]
              [WITH <index_with_clause> [{,
<index_with_clause>]} ...]
              [INVISIBLE]
              [COMMENT 'index_comment_string']

         <index_with_clause> ::= {ONLINE [PARALLEL parallel_count]}
| <index_with_option>
         <index_with_option> ::= {DEDUPLICATE '=' deduplicate_level}
```

- **UNIQUE**: creates an index with unique values.
- *index_name*: specifies the name of the index to be created. The index name must be unique in the table.
- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*: specifies the name of the table where the index is to be created.
- *column_name*: specifies the name of the column where the index is to be applied. To create a composite index, specify two or more column names.
- **ASC** | **DESC**: specifies the sorting order of columns.
- *deduplicate_level*: specifies the deduplicate level (0 to 14). For details, see DEDUPLICATE.

Note

- *deduplicate_level* is an integer from 0 to 14.
- When this parameter is set to 0, it indicates an index configuration where data deduplication within indexes is not performed. This configuration is the same as that in CUBRID 11.2 or an earlier version.

- <*filter_predicate*>: defines the conditions to create filtered indexes. When there are several comparison conditions between a column and a constant, filtering is available only when the conditions are connected by using **AND**. Refer to Filtered Index for more details.
- *function_name* (*argument_list*): Defines the conditions to create function-based indexes. Regarding this, definitely watch Function-based Index.
- **WITH ONLINE**: creates the index while allowing changes of table data from other transactions. If **PARALLEL** is not specified, the index is created using the transaction thread. <*parallel_count*> is

the number of threads to be used for creating the index and it must be a integer between 1 and 16.

- **INVISIBLE**: creates the index with its status set to **INVISIBLE**, meaning queries executed will not take into account the index. If **INVISIBLE** is omitted, the index will be created with its status set to **NORMAL_INDEX**.

- *index_comment_string*: specifies a comment of an index.

Note

- From CUBRID 9.0, the index name should not be omitted in the CREATE/INDEX statement.

- Prefix index feature is deprecated, so it is not recommended anymore.

- The session and server timezone (Timezone Parameter) should not be changed if database contains indexes or function index on columns of type TIMESTAMP, TIMESTAMP WITH LOCAL TIME ZONE or DATETIME WITH LOCAL TIME ZONE.

- The leap second support parameter (Timezone Parameter) should not be changed if database contains indexes or function index on columns of type TIMESTAMP or TIMESTAMP WITH LOCAL TIME ZONE.

- The number of threads from PARALLEL option applies only to index loading step. The heap scan step is performed by the transaction thread (single thread). During this process, the rows are collected into batches; when a batch is full (reached 16M in size), it is dispatched to a index loading thread (or pushed in the queue), the collecting process continues. Batch size computing consists of index key size and the row identifier size (8 bytes).

- Under stand-alone mode, the WITH ONLINE option is ignored (normal index is created).

- The creation of online index is performed in three stages:

  - Adding index into schema with status INDEX IS IN ONLINE BUILDING. This is performed with a SCH_M_LOCK (like all schema changes) on table. After this, the lock is demoted to IX_LOCK.

  - Populating the index : scanning the heap file into batches, sorting the batches and adding the keys to index. During this step, other transactions can modify table data (index is also updated with data changes from other committed transactions).

  - Updating the index status as NORMAL INDEX; this is performed after promoting the table lock back to SCH_M_LOCK (promotion is guaranteed).

- The online index being built is displayed by SHOW statements from other transactions. It is not visible from other transactions in _db_index system table due to MVCC snapshot (other transactions can only see committed entries in this table).
- Transactions running in parallel with online index building which performs operations causing unique violations in index are allowed to commit. The online index will continue to progress and check before final step (setting NORMAL INDEX status in schema) the validity of unique constraint. The index creation will be aborted in case of unique violation. The user needs to restart the operation after making sure the unique constraint is ensured.

The following example shows how to create a descending index.

```
CREATE INDEX gold_index ON participant(gold DESC);
```

The following example shows how to create a multiple column index.

```
CREATE INDEX name_nation_idx ON athlete(name, nation_code) COMMENT
'index comment';
```

COMMENT of Index

You can write a comment of an index as following.

```
CREATE TABLE tbl (a int default 0, b int, c int);

CREATE INDEX i_tbl_b on tbl (b) COMMENT 'index comment for i_tbl_b';

CREATE TABLE tbl2 (a INT, index i_tbl_a (a) COMMENT 'index comment',
b INT);

ALTER TABLE tbl2 ADD INDEX i_tbl2_b (b) COMMENT 'index comment b';
```

A specified comment of an index can be shown by running these statements.

```
SHOW CREATE TABLE table_name;
SELECT index_name, class_name, comment from db_index where
class_name ='classname';
SHOW INDEX FROM table_name;
```

Or you can see the index comments with ;sc command in the CSQL interpreter.

```
$ csql -u dba demodb

csql> ;sc tbl
```

Online index creation

You can create the index while still allowing other transactions to insert or update the table.

```
CREATE TABLE t1 (i1 int, i2 int);

CREATE INDEX i_t1_i1 on t1 (i1) WITH ONLINE PARALLEL 10;
```

Displaying online index from other transactions

Other transactions may see the online index with schema related statements:

```
csql> show index in t1;

=== <Result of SELECT Command in Line 1> ===

  Table                 Non_unique  Key_name
Seq_in_index  Column_name           Collation
Cardinality     Sub_part  Packed
Null                  Index_type            Func
Comment               Visible
=============================================================================
=============================================================================
=============================================================================
=============================================================
  't1'                            1
'i_t1'                          1  'i1'
'A'                            0         NULL
NULL                  'YES'                 'BTREE'
NULL                  NULL                  'NO'

1 row selected. (0.020779 sec) Committed.

1 command(s) successfully processed.
csql> desc t1;

=== <Result of SELECT Command in Line 1> ===

  Field                 Type                  Null
Key                   Default               Extra
=============================================================================
=============================================================
  'i1'                  'INTEGER'             'YES'
```

```
'MUL'                  NULL                   ''
  'i2'                   'INTEGER'              'YES'
''                     NULL                   ''

csql> ;schema t1

=== <Help: Schema of a Class> ===


 <Class Name>

     t1

 <Attributes>

     i1                     INTEGER
     i2                     INTEGER

 <Constraints>

     INDEX i_t1 ON t1 (i1) IN PROGRESS
```

Online unique index while other transactions inserts violates uniqueness

session 1	session 2

```
CREATE TABLE t1 (i1 int, i2
int);

COMMIT WORK;
```

```
INSERT INTO t1 VALUES (1, 10);

CREATE UNIQUE INDEX i_t1_i1 on
t1 (i1) WITH ONLINE;
```

```
csql> ;schema t1

=== <Help: Schema of a Class>
===
```

session 1	session 2
	<Class Name> t1 <Attributes> i1 INTEGER i2 INTEGER <Constraints> UNIQUE i_t1 ON t1 (i1) IN PROGRESS
	INSERT INTO t1 VALUES (1, 20); COMMIT WORK;
COMMIT WORK; ERROR: Operation would have caused one or more unique constraint violations. INDEX i_t1(B+tree: 0|3456|3457) ON CLASS t1(CLASS_OID: 0|202|7). key: *UNKNOWN-KEY*.	

DEDUPLICATE

If you use the **DEDUPLICATE** option, you can improve performance of modifying skewed index. The value of this option can be adjusted to mitigate the overflow page's linked list for a specific

key value from being made too long, thereby improving insert/delete/update and vacuum performance. However, since an index column hidden by the system is added, the number of leaf nodes in the index and the height of the b-tree may increase, increasing the size of the index and affecting search performance. In particular, if the index data is uniformly distributed with respect to key values, you are careful to use this option because only the size of the index may increase without improving performance.

If the value of *deduplicate level* is specified as 1 or higher, a hidden index column used internally by the system is added when creating the index. The value of column is used to reduce redundancy so that it is not biased toward a specific key value. A higher *deduplicate level* value, it will be mitigated redundancy and a smaller overflow page for a specific key.

Note

- The overflow page have a disadvantage in add, delete, and update performance, but are beneficial in search performance. Therefore, it is better to adjust the length of the overflow page for one key to be within tens or hundreds of pages, rather than setting the *deduplication level* high so that overflow pages are not created when creating an index with that option.

There are two ways to specify *deduplicate level* when creating an index.

Implicit method

This method automatically specifies *deduplicate level* when there is no explicit DEDUPLICATE option specified in the SQL statement. This method is affected by the set value of the system parameter **deduplicate_key_level**. If **deduplicate_key_level** is greater than or equal to 1, *deduplicate level* is automatically set to **deduplicate_key_level**.

Explicit method

This is a method in which the user explicitly specifies the **DEDUPLICATE** option in the SQL statement. Regardless of the **deduplicate_key_level** set value, the *deduplicate level* specified by the user is applied. As in the example below, specify the DEDUPLICATE statement directly.

```
CREATE TABLE tbl (a int default 0, b int, c int);
CREATE INDEX i_tbl_b on tbl (b) WITH DEDUPLICATE=3 COMMENT
'for deduplicate level 3';
CREATE INDEX i_tbl_b_c on tbl (b,c) WITH DEDUPLICATE=7
COMMENT 'for deduplicate level 1';
```

⚠ Warning

- If **deduplicate_key_level** is **-1**, it is internally ignored and not applied even if it is specified explicitly. That is, in this case, all indexes are created with *deduplicate level* as **0**.
- A column name that begins with "**_dedup_**" cannot be created.

ⓘ Note

- **When creating an index, if the composition of the key field is guaranteed to be UNIQUE, the DEDUPLICATE specified by the user is ignored and *deduplicate level* is created as 0.**
  - If the columns of index includes all columns of Primary Key or Unique Index.
  - However, except if columns of Primary key or Unique index used as an argument of a function index.

Allowing multiple indexes on the same column in the same order

Multiple indexes can be defined with all key fields and filter conditions identical except for *deduplicate level*.

```
CREATE TABLE tbl (a int primary key, b int, c int);
CREATE INDEX idx1 ON tbl(b, c) WITH DEDUPLICATE=3;
CREATE INDEX idx2 ON tbl(b, c) WITH DEDUPLICATE=5;
CREATE UNIQUE INDEX idx_uk ON tbl(b);
CREATE INDEX idx3 ON tbl(b, c) WITH DEDUPLICATE=7;
```

In the example above, idx1 and idx2 have the specified *deduplicate level*. However, since idx3 is guaranteed by idx_uk that column b is unique, it is created *duplicate level* as **0** and ignore user's specified option.

ⓘ Note

- If an FK already exists with the same index column that differs only in *deduplication level*, it cannot be duplicated.
- If columns of index includes a primary key or unique index, indexes that differ only in the deduplicate level cannot be created. (Even if duplicate indexes are allowed.)

- You cannot change the *deduplicate level* of an index with the ALTER INDEX REBUILD statement. If necessary, delete the index and recreate it.

OVERFLOW PAGE

- **What is the overflow page?**

  An index consists of non-leaf nodes and leaf nodes, and a leaf node is composed of a set of index key information. At this time, one index key information is a pair of a key value and a set of record's OIDs corresponding to the key value. If there are many records with a specific key value, the OIDs of all the records cannot be stored in the leaf node, so they are separated and managed in a separate page, which is called an overflow page. Also, when even overflow pages are full, new overflow pages are created and these pages are maintained as a linked list.

ALTER INDEX

The **ALTER INDEX** statement changes the properties of an index. Index is rebuilt unless only comment or status is changed. Rebuilding an index is a job which drops and recreates an index.

The following is a syntax of rebuilding an index.

```
ALTER INDEX index_name ON [schema_name.]table_name {{[COMMENT
index_comment_string] REBUILD} | {COMMENT index_comment_string} |
VISIBLE | INVISIBLE};
```

- *index_name*: specifies the name of the index to be altered or rebuilt. The index name must be unique in the table.
- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*: specifies the name of the table where the index is rebuilt.
- **REBUILD**: rebuild an index with the same structure as the one already created.
- *index_comment_string*: specifies a comment of an index.

Note

- From CUBRID 9.0, the index name should not be omitted.
- From CUBRID 10.0, table name should not be omitted.
- From CUBRID 10.0, even if you add column names at the end of a table name, these will be ignored and recreated with the same columns with the previous index.
- From CUBRID 11.4, an error occurs if you add a column name after the table name.

The following is an example of recreating index.

```
CREATE INDEX i_game_medal ON game(medal);
ALTER INDEX i_game_medal ON game COMMENT 'rebuild index comment'
REBUILD ;
```

If you want to add or change a comment of the index without rebuilding an index, add a **COMMENT** clause and remove **REBUILD** keyword as follows:

```
ALTER INDEX index_name ON table_name COMMENT 'index_comment_string' ;
```

The below is a syntax to only add or change a comment without rebuilding an index.

```
ALTER INDEX i_game_medal ON game COMMENT 'change index comment' ;
```

The following is a syntax to change the status of an index to **INVISIBLE**/**VISIBLE**. When an index is set as **INVISIBLE**, queries will be executed as like the index does not exist. In this way, the performance of the index may be tested and the impact of its removal be evaluated without actually dropping the index.

```
CREATE INDEX i_game_medal ON game(medal);
ALTER INDEX i_game_medal ON game VISIBLE;
ALTER INDEX i_game_medal ON game INVISIBLE;
```

DROP INDEX

Use the **DROP INDEX** statement to drop an index. An index also can be dropped with **DROP CONSTRAINT** clause.

```
DROP INDEX index_name ON [schema_name.]table_name ;
```

- *index_name*: specifies the name of the index to be dropped.
- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*: specifies the name of the table whose index is dropped.

> **Warning**
>
> From the CUBRID 10.0 version, table name cannot be omitted.

The following is an example of dropping an index:

```
DROP INDEX i_game_medal ON game;
```

VIEW DEFINITION STATEMENTS

CREATE VIEW

A view is a virtual table that does not exist physically. You can create a view by using an existing table or a query. **VIEW** and **VCLASS** are used interchangeably.

Use **CREATE VIEW** statement to create a view. A view with the same name as an existing table or synonym cannot be created. Regarding writing view name, see Identifier.

```
CREATE [OR REPLACE] {VIEW | VCLASS} [schema_name.]view_name
[<subclass_definition>]
[(view_column_name [COMMENT 'column_comment_string'], ...)]
[INHERIT <resolution>, ...]
[AS <select_statement>]
[WITH CHECK OPTION]
[COMMENT [=] 'view_comment_string'];

    <subclass_definition> ::= {UNDER | AS SUBCLASS OF}
[schema_name.]superclass_name, ...
```

```
    <resolution> ::= [CLASS | TABLE] {column_name} OF
[schema_name.]superclass_name [AS alias]
```

- **OR REPLACE**: If the keyword **OR REPLACE** is specified after **CREATE**, the existing view is replaced by a new one without displaying any error message, even when the *view_name* overlaps with the existing view name.
- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *view_name*: specifies the name of a view to be created. It must be unique in a database.
- *view_column_name*: defines the column of a view.
- **AS** <*select_statement*>: A valid **SELECT** statement must be specified. A view is created based on this.
- **WITH CHECK OPTION**: If this option is specified, the update or insert operation is possible only when the condition specified in the **WHERE** clause of the <*select_statement*> is satisfied. Therefore, this option is used to disallow the update of a virtual table that violates the condition.
- *view_comment_string*: specifies a view's comment.
- *column_comment_string*: specifies a column's comment.

```
CREATE TABLE a_tbl (
    id INT NOT NULL,
    phone VARCHAR(10)
);
INSERT INTO a_tbl VALUES(1,'111-1111'), (2,'222-2222'), (3,
'333-3333'), (4, NULL), (5, NULL);

--creating a new view based on AS select_statement from a_tbl
CREATE VIEW b_view AS SELECT * FROM a_tbl WHERE phone IS NOT NULL
WITH CHECK OPTION;
SELECT * FROM b_view;
```

```
          id  phone
==================================
           1  '111-1111'
           2  '222-2222'
           3  '333-3333'
```

```
--WITH CHECK OPTION doesn't allow updating column value which
violates WHERE clause
UPDATE b_view SET phone=NULL;
```

```
ERROR: Check option exception on view b_view.
```

The below updates the old view's definition, In addition, this adds a view's comment.

```
--creating view which name is as same as existing view name
CREATE OR REPLACE VIEW b_view AS SELECT * FROM a_tbl ORDER BY id
DESC COMMENT 'changed view';

--the existing view has been replaced as a new view by OR REPLACE
keyword
SELECT * FROM b_view;
```

```
           id  phone
=====================================
            5  NULL
            4  NULL
            3  '333-3333'
            2  '222-2222'
            1  '111-1111'
```

The below adds a comment to a view's columns.

```
CREATE OR REPLACE VIEW b_view(a COMMENT 'column id', b COMMENT
'column phone') AS SELECT * FROM a_tbl ORDER BY id DESC;
```

Condition for Creating Updatable VIEW

A virtual table is updatable if it satisfies the following conditions:

- The **FROM** clause must include the updatable table or view only.

  In versions lower than CUBRID 9.0, only one updatable table can be included to the **FROM** clause it requires. However, two tables in parentheses like FROM (class_x, class_y) can be updated since the two were expressed as one table. In version of CUBRID 9.0 or higher, more than one updatable table is allowed. The **FROM** clause must include only one table or updatable view. However, two tables included in parentheses as in **FROM** (class_x, class_y) can be updated because they represent one table.

- A **JOIN** syntax can be included.

Note

In versions lower than CUBRID 10.0, you cannot update a view which is created with a **JOIN** syntax.

- The **DISTINCT** or **UNIQUE** statement should not be included.
- The **GROUP BY... HAVING** statement should not be included.
- Aggregate functions such as **SUM** or **AVG** should not be included.
- The entire query must consist of queries that can be updated by **UNION ALL**, not by **UNION**. However, the table should exist only in one of the queries that constitute **UNION ALL**.
- If a record is inserted into a view created by using the **UNION ALL** statement, the system determines into which table the record will be inserted. This cannot be done by the user. To control this, the user must manually insert the row or create a separate view for insertion.

Even when all rules above are satisfied, columns that contains following contents cannot be updated.

- Path expressions (example: *tbl_name.col_name*)
- Numeric type column that includes an arithmetic operator

Even though the column defined in the view is updatable, a view can be updated only when an appropriate update authorization is granted on the table included in the **FROM** clause. Also there must be an access authorization to a view. The way to grant an access authorization to a view is the same to grant an access authorization to a table. For details on granting authorization, see GRANT.

View's COMMENT

You can specify a view's comment as follows.

```
CREATE OR REPLACE VIEW b_view AS SELECT * FROM a_tbl ORDER BY id
DESC COMMENT 'changed view';
```

You can see the specified comment of a view by running this syntax.

```
SHOW CREATE VIEW view_name;
SELECT vclass_name, comment from db_vclass;
```

Or you can see the view's comment with ;sc command which displays the schema in the CSQL interpreter.

```
$ csql -u dba demodb

csql> ;sc b_view
```

Also, you can add a comment for each column of the view.

```
CREATE OR REPLACE VIEW b_view (a COMMENT 'a comment', b COMMENT 'b
comment')
AS SELECT * FROM a_tbl ORDER BY id DESC COMMENT 'view comment';
```

To see how to change a comment of a view, refer to ALTER VIEW syntax on the below.

ALTER VIEW

ADD QUERY Clause

You can add a new query to a query specification by using the **ADD QUERY** clause of the **ALTER VIEW** statement. 1 is assigned to the query defined when a virtual table was created, and 2 is assigned to the query added by the **ADD QUERY** clause.

```
ALTER [VIEW | VCLASS] [schema_name.]view_name
ADD QUERY <select_statement>
[INHERIT <resolution> , ...] ;

    <resolution> ::= {column_name} OF [schema_name.]superclass_name
[AS alias]
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *view_name*: specifies the name of a view where the query to be added.
- *<select_statement>*: specifies the query to be added.

```
SELECT * FROM b_view;
```

```
           id  phone
=================================
            1  '111-1111'
```

```
            2  '222-2222'
            3  '333-3333'
            4  NULL
            5  NULL
```

```
ALTER VIEW b_view ADD QUERY SELECT * FROM a_tbl WHERE id IN (1,2);
SELECT * FROM b_view;
```

```
           id  phone
=====================================
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
            4  NULL
            5  NULL
            1  '111-1111'
            2  '222-2222'
```

AS SELECT Clause

You can change the **SELECT** query defined in the virtual table by using the **AS SELECT** clause in the **ALTER VIEW** statement. This function is working like the **CREATE OR REPLACE** statement. You can also change the query by specifying the query number 1 in the **CHANGE QUERY** clause of the **ALTER VIEW** statement.

```
ALTER [VIEW | VCLASS] [schema_name.]view_name AS <select_statement> ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *view_name*: specifies the name of a view to be modified.
- <*select_statement*>: specifies the new query statement to replace the **SELECT** statement defined when a view is created.

```
ALTER VIEW b_view AS SELECT * FROM a_tbl WHERE phone IS NOT NULL;
SELECT * FROM b_view;
```

```
           id  phone
=====================================
            1  '111-1111'
```

```
            2  '222-2222'
            3  '333-3333'
```

CHANGE QUERY Clause

You can change the query defined in the query specification by using the **CHANGE QUERY** clause reserved word of the **ALTER VIEW** statement.

```
ALTER [VIEW | VCLASS] [schema_name.]view_name
CHANGE QUERY [integer] <select_statement> ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *view_name*: specifies the name of a view to be modified.
- *integer*: specifies the number value of the query to be modified. The default value is 1.
- <*select_statement*>: specifies the new query statement to replace the query whose query number is *integer*.

```
--adding select_statement which query number is 2 and 3 for each
ALTER VIEW b_view ADD QUERY SELECT * FROM a_tbl WHERE id IN (1,2);
ALTER VIEW b_view ADD QUERY SELECT * FROM a_tbl WHERE id = 3;
SELECT * FROM b_view;
```

```
            id  phone
==================================
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
            4  NULL
            5  NULL
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
```

```
--altering view changing query number 2
ALTER VIEW b_view CHANGE QUERY 2 SELECT * FROM a_tbl WHERE phone IS
NULL;
SELECT * FROM b_view;
```

```
           id  phone
=====================================
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
            4  NULL
            5  NULL
            4  NULL
            5  NULL
            3  '333-3333'
```

DROP QUERY Clause

You can drop a query defined in the query specification by using the **DROP QUERY** of the **ALTER VIEW** statement.

```
ALTER VIEW b_view DROP QUERY 2,3;
SELECT * FROM b_view;
```

```
           id  phone
=====================================
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
            4  NULL
            5  NULL
```

COMMENT Clause

You can change a view's comment, columns' comment, or attributes' comment with **COMMENT** clause of **ALTER VIEW** syntax.

```
ALTER [VIEW | VCLASS] [schema_name.]view_name
COMMENT [=] 'view_comment_string' |
COMMENT ON {COLUMN | CLASS ATTRIBUTE} <column_comment_definition> [,
<column_comment_definition>] ;

    <column_comment_definition> ::= column_name [=]
'column_comment_string'
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *view_name*: Specifies the name of a view to be modified.

- *column_name*: Specifies the name of a column to be modified.
- *view_comment_string*: Specifies a view's comment.
- *column_comment_string*: Specifies a column's comment.

The following example shows how to change a view's comments.

```
ALTER VIEW v1 COMMENT = 'changed view v1 comment';
```

You can change the column comment by specifying one or more columns after the ON COLUMN keyword. The following example shows how to change a column's comments.

```
ALTER VIEW v1 COMMENT ON COLUMN c1 = 'changed view column c1
comment';
ALTER VIEW v1 COMMENT ON COLUMN c2 = 'changed view column c2
comment', c3 = 'changed view column c3 comment';
```

Below is a syntax to show a column's comment. But the SHOW CREATE VIEW statement shows only view comments.

```
SHOW CREATE VIEW v1 /* view_name */ ;

SELECT attr_name, class_name, comment
FROM db_attribute
WHERE class_name = 'v1' /* lowercase_view_name */ ;

SHOW FULL COLUMNS FROM v1 /* view_name */ ;
```

You can see this comment with the ";sc view_name" command in the CSQL interpreter.

```
$ csql -u dba demodb

csql> ;sc v1
```

DROP VIEW

You can drop a view by using the **DROP VIEW** clause. The way to drop a view is the same as to drop a regular table. If you also specify IF EXISTS clause, no error will be happened even if a target view does not exist.

```
DROP [VIEW | VCLASS] [IF EXISTS] [schema_name.]view_name [{,
[schema_name.]view_name}] ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *view_name*: specifies the name of a view to be dropped.

```
DROP VIEW b_view;
```

RENAME VIEW

You can change the view name by using the **RENAME VIEW** statement.

```
RENAME [VIEW | VCLASS] [schema_name.]old_view_name {AS | TO}
[schema_name.]new_view_name [{, [schema_name.]old_view_name {AS |
TO} [schema_name.]new_view_name}] ;
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used. The schema of the view to be changed and the schema of the new view must be the same.
- *old_view_name*: specifies the name of a view to be modified.
- *new_view_name*: specifies the new name of a view.

The following example shows how to rename a view name to *game_2004*.

```
RENAME VIEW game_2004 AS info_2004;
```

SERIAL DEFINITION STATEMENTS

CREATE SERIAL

Serial is an object that creates a unique sequence number, and has the following characteristics.

- The serial is useful in creating a unique sequence number in multi-user environment.
- Generated serial numbers are not related with table so, you can use the same serial in multiple tables.

- All users including **PUBLIC** can create a serial object. Once it is created, all users can get the number by using **CURRENT_VALUE(equivalent to CURRVAL)** and **NEXT_VALUE(equivalent to NEXTVAL)**.

- Only owner of a created serial object and **DBA** can update or delete a serial object. If an owner is **PUBLIC**, all users can update or delete it.

You can create a serial object in the database by using the **CREATE SERIAL** statement. Regarding writing serial name, Identifier.

```
CREATE SERIAL [schema_name.]serial_name
[START WITH initial]
[INCREMENT BY interval]
[MINVALUE min | NOMINVALUE]
[MAXVALUE max | NOMAXVALUE]
[CYCLE | NOCYCLE]
[CACHE cached_num | NOCACHE]
[COMMENT 'comment_string'];
```

- *schema_name*: Specifies the schema name of the serial(maximum: 31 bytes). If omitted, the schema name of the current session is used.

- *serial_name*: specifies the name of the serial to be generated(maximum: 222 bytes).

- **START WITH** *initial*: Specifies the initial value of serial. The range of this value is between -1,000,000,000,000,000,000,000,000,000,000,000,000(-10^36) and 9,999,999,999,999,999,999,999,999,999,999,999,999(10^37-1). The default value of ascending serial is 1 and that of descending serial is -1.

- **INCREMENT BY** *interval*: Specifies the increment of the serial. You can specify any integer between -9,999,999,999,999,999,999,999,999,999,999,999,999(-10^37+1) and 9,999,999,999,999,999,999,999,999,999,999,999,999(10^37-1) except zero at *interval*. The absolute value of the *interval* must be smaller than the difference between **MAXVALUE** and **MINVALUE**. If a negative number is specified, the serial is in descending order otherwise, it is in ascending order. The default value is **1**.

- **MINVALUE**: Specifies the minimum value of the serial. The range of this value is between -1,000,000,000,000,000,000,000,000,000,000,000,000(-10^36) and 9,999,999,999,999,999,999,999,999,999,999,999,999(10^37-1). **MINVALUE** must be smaller than or equal to the initial value and smaller than the maximum value.

- **NOMINVALUE**: 1 is set automatically as a minimum value for the ascending serial, -1,000,000,000,000,000,000,000,000,000,000,000,000(-10^36) for the descending serial.

- **MAXVALUE**: Specifies the maximum number of the serial. The range of this value is between -999,999,999,999,999,999,999,999,999,999,999,999,999(-10^36+1) and

10,000,000,000,000,000,000,000,000,000,000,000,000(10^37). **MAXVALUE** must be greater than or equal to the initial value and greater than the minimum value.

- **NOMAXVALUE**: 10,000,000,000,000,000,000,000,000,000,000,000,000(10^37) is set automatically as a maximum value for the ascending serial, -1 for the descending serial.
- **CYCLE**: Specifies that the serial will be generated continuously after reaching the maximum or minimum value. When a serial in ascending order reaches the maximum value, the minimum value is created as the next value; when a serial in descending order reaches the minimum value, the maximum value is created as the next value.
- **NOCYCLE**: Specifies that the serial will not be generated anymore after reaching the maximum or minimum value. The default value is **NOCYCLE**.
- **CACHE**: Stores as many serials as the number specified by "cached_num" in the cache to improve the performance of the serials and fetches a serial value when one is requested. If all cached values are used up, as many serials as "cached_num" are fetched again from the disk to the memory. If the database server stops accidentally, all cached serial values are deleted. For this reason, the serial values before and after the restart of the database server may be discontinuous. Because the transaction rollback does not affect the cached serial values, the request for the next serial will return the next value of the value used (or fetched) lastly when the transaction is rolled back. The "cached_num" after the **CACHE** keyword cannot be omitted. If the "cached_num" is equal to or smaller than 1, the serial cache is not applied.
- **NOCACHE**: Specifies that the serial cache is not used, and serial value is updated for each time. The default value is **NOCACHE**.
- *comment_string*: specifies a comment of a serial.

```
--creating serial with default values
CREATE SERIAL order_no;

--creating serial within a specific range
CREATE SERIAL order_no START WITH 10000 INCREMENT BY 2 MAXVALUE
20000;

--creating serial with specifying the number of cached serial values
CREATE SERIAL order_no START WITH 10000 INCREMENT BY 2 MAXVALUE
20000 CACHE 3;

--selecting serial information from the db_serial class
SELECT * FROM db_serial;
```

```
  name            current_val      increment_val
max_val         min_val         cyclic      started
cached_num        att_name
=========================================================================
```

```
=================================================================
==========
'order_no'      10006           2
20000           10000               0           1
3           NULL
```

The following example shows how to create the *athlete_idx* table to store athlete codes and names and then create an instance by using the *order_no*. NEXT_VALUE increases the serial number and returns its value.

```
CREATE TABLE athlete_idx( code INT, name VARCHAR(40) );
CREATE SERIAL order_no START WITH 10000 INCREMENT BY 2 MAXVALUE
20000;
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Park');
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Kim');
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Choo');
INSERT INTO athlete_idx VALUES (order_no.CURRENT_VALUE, 'Lee');

SELECT * FROM athlete_idx;
```

```
         code  name
=====================================
        10000  'Park'
        10002  'Kim'
        10004  'Choo'
        10004  'Lee'
```

COMMENT of Serial

The below adds a comment when you create a serial.

```
CREATE SERIAL order_no
START WITH 100 INCREMENT BY 2 MAXVALUE 200
COMMENT 'from 100 to 200 by 2';
```

To see a comment of the serial, run the below syntax.

```
SELECT name, comment FROM db_serial;
```

To change a comment of a serial, see ALTER SERIAL syntax.

ALTER SERIAL

With the **ALTER SERIAL** statement, you can update the increment of the serial value, set or delete its initial or minimum/maximum values, and set its cycle attribute.

```
ALTER SERIAL [schema_name.]serial_name
[INCREMENT BY interval]
[START WITH initial_value]
[MINVALUE min | NOMINVALUE]
[MAXVALUE max | NOMAXVALUE]
[CYCLE | NOCYCLE]
[CACHE cached_num | NOCACHE]
[COMMENT 'comment_string'];
```

- *schema_name*: Specifies the schema name of the serial(maximum: 31 bytes). If omitted, the schema name of the current session is used.
- *serial_name*: specifies the name of the serial to be created(maximum: 222 bytes).
- **INCREMENT BY** *interval*: specifies the increment of the serial. For the *interval*, you can specify any integer with 38 digits or less except zero. The absolute value of the *interval* must be smaller than the difference between **MAXVALUE** and **MINVALUE**. If a negative number is specified, the serial is in descending order; otherwise, it is in ascending order. The default value is **1**.
- **START WITH** *initial_value*: changes the initial value of Serial.
- **MINVALUE**: specifies the minimum value of the serial with 38 digits or less. **MINVALUE** must be smaller than or equal to the initial value and smaller than the maximum value.
- **NOMINVALUE**: 1 is set automatically as a minimum value for the ascending serial; -(10) 36 for the descending serial.
- **MAXVALUE**: specifies the maximum number of the serial with 38 digits or less. **MAXVALUE** must be larger than or equal to the initial value and greater than the minimum value.
- **NOMAXVALUE**: (10) 37 is set automatically as a maximum value for the ascending serial; -1 for the descending serial.
- **CYCLE**: specifies that the serial will be generated continuously after reaching the maximum or minimum value. If the ascending serial reaches the maximum value, the minimum value is generated as the next value. If the descending serial reaches the minimum value, the maximum value is generated as the next value.
- **NOCYCLE**: specifies that the serial will not be generated anymore after reaching the maximum or minimum value. The default is **NOCYCLE**.

- **CACHE**: stores as many serials as the number specified by *integer* in the cache to improve the performance of the serials and fetches a serial value when one is requested. The *integer* after the **CACHE** keyword cannot be omitted. If a number equal to or smaller than 1 is specified, the serial cache is not applied.
- **NOCACHE**: It does not use the serial cache feature. The serial value is updated every time and a new serial value is fetched from the disk upon each request. The default is **NOCACHE**.
- *comment_string*: specifies a comment of a serial.

```
--altering serial by changing start and incremental values
ALTER SERIAL order_no START WITH 100 MINVALUE 100 INCREMENT BY 2;

--altering serial to operate in cache mode
ALTER SERIAL order_no CACHE 5;

--altering serial to operate in common mode
ALTER SERIAL order_no NOCACHE;
```

⚠ Warning

In CUBRID 2008 R1.x version, the serial value can be modified by updating the db_serial table, a system catalog. However, in CUBRID 2008 R2.0 version or above, the modification of the db_serial table is not allowed but use of the **ALTER SERIAL** statement is allowed. Therefore, if an **ALTER SERIAL** statement is included in the data exported (unloaddb) from CUBRID 2008 R2.0 or above, it is not allowed to import (loaddb) the data in CUBRID 2008 R1.x or below.

⚠ Warning

When you get the value of **NEXT_VALUE** after running **ALTER SERIAL**, in the version lower than CUBRID 9.0, the next value of the initial value was returned. From CUBRID 9.0, the setting value of **ALTER_SERIAL** is returned.

```
CREATE SERIAL s1;
SELECT s1.NEXTVAL;

ALTER SERIAL s1 START WITH 10;

SELECT s1.NEXTVAL;
```

```
-- From 9.0, above query returns 10
-- In the version less than 9.0, above query returns 11
```

The below changes the comment of the serial.

```
ALTER SERIAL order_no COMMENT 'new comment';
```

DROP SERIAL

With the **DROP SERIAL** statement, you can drop a serial object from the database. If you also specify **IF EXISTS** clause, no error will be happened even if a target serial does not exist.

```
DROP SERIAL [ IF EXISTS ] [schema_name.]serial_name ;
```

- *schema_name*: Specifies the schema name of the serial. If omitted, the schema name of the current session is used.
- *serial_name*: Specifies the name of the serial to be dropped.

The following example shows how to drop the *order_no* serial.

```
DROP SERIAL order_no;
DROP SERIAL IF EXISTS order_no;
```

Accessing Serial

Pseudocolumns

You can access and update a serial by serial name and a pseudocolumn pair.

```
[schema_name.]serial_name.CURRENT_VALUE
[schema_name.]serial_name.CURRVAL

[schema_name.]serial_name.NEXT_VALUE
[schema_name.]serial_name.NEXTVAL
```

- *schema_name*: Specifies the schema name of the serial. If omitted, the schema name of the current session is used.

- *[schema_name.]serial_name*.**CURRENT_VALUE**, *[schema_name.]serial_name*.**CURRVAL**: Returns the current serial value.
- *[schema_name.]serial_name*.**NEXT_VALUE**, *[schema_name.]serial_name*.**NEXTVAL**: Increments the serial value and returns the result.

The following example shows how to create a table *athlete_idx* where athlete numbers and names are stored and how to create the instances by using a serial *order_no*.

```
CREATE TABLE athlete_idx (code INT, name VARCHAR (40));
CREATE SERIAL order_no START WITH 10000 INCREMENT BY 2 MAXVALUE
20000;
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Park');
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Kim');
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Choo');
INSERT INTO athlete_idx VALUES (order_no.NEXT_VALUE, 'Lee');
SELECT * FROM athlete_idx;
SELECT order_no.CURRENT_VALUE;
```

```
          code  name
=====================================
         10000  'Park'
         10002  'Kim'
         10004  'Choo'
         10006  'Lee'

         serial_current_value(order_no)
=======================
         10006
```

Note

When you use a serial for the first time after creating it, **NEXT_VALUE** returns the initial value. Subsequently, the sum of the current value and the increment are returned.

Functions

SERIAL_CURRENT_VALUE(*[schema_name.]serial_name*)

SERIAL_NEXT_VALUE(*[schema_name.]serial_name, number*)

The **Serial** function consists of the **SERIAL_CURRENT_VALUE** and **SERIAL_NEXT_VALUE** functions.

Parameters:

- **schema_name** – The schema name of the serial
- **serial_name** – Serial name
- **number** – The number of serials to be obtained

Return type:

NUMERIC(38,0)

The **SERIAL_CURRENT_VALUE** function returns the current serial value, which is the same value as *serial_name* **.current_value**.

This function returns as much added value as interval specified. The serial interval is determined by the value of a **CREATE SERIAL ... INCREMENT BY** statement. **SERIAL_NEXT_VALUE** (*serial_name*, 1) returns the same value as *serial_name* **.next_value**.

To get a large amount of serials at once, specify the desired number as an argument to call the **SERIAL_NEXT_VALUE** function only once; which has an advantage over calling repeatedly *serial_name* **.next_value** in terms of performance.

Assume that an application process is trying to get the number of n serials at once. To perform it, call **SERIAL_NEXT_VALUE** (*serial_name*, N) one time to store a return value and calculate a serial value between (a serial start value) and (the return value). (Serial value at the point of function call) is equal to the value of (return value) - (desired number of serials) * (serial interval).

For example, if you create a serial starting 101 and increasing by 1 and call **SERIAL_NEXT_VALUE** (*serial_name*, 10), it returns 110. The start value at the point is 110-(10-1)*1 = 101. Therefore, 10 serial values such as 101, 102, 103, ... 110 can be used by an application process. If **SERIAL_NEXT_VALUE** (*serial_name*, 10) is called in succession, 120 is returned; the start value at this point is 120-(10-1)*1 = 111.

```
CREATE SERIAL order_no START WITH 101 INCREMENT BY 1 MAXVALUE 20000;
SELECT SERIAL_CURRENT_VALUE(order_no);
```

```
101
```

```
-- At first, the first serial value starts with the initial serial
value, 10000. So the l0th serial value will be 10009.
SELECT SERIAL_NEXT_VALUE(order_no, 10);
```

```
110
```

```
SELECT SERIAL_NEXT_VALUE(order_no, 10);
```

```
120
```

Note

If you create a serial and calls the **SERIAL_NEXT_VALUE** function for the first time, a value of (serial interval) * (desired number of serials - 1) added to the current value is returned. If you call the **SERIAL_NEXT_VALUE** function in succession, a value of (serial interval) * (desired number of serials) added to the current is returned (see the example above).

SYNONYM DEFINITION STATEMENTS

CREATE SYNONYM

Create another new name to use for an object that already exists in the database. The user can use the object with the name specified when creating the database object, and can also use the object as a synonym.

1. The user can only create private synonym, and private synonym can only be used in the schema for the user.
2. The information of the created synonym can be found in the DB_SYNONYM system virtual class.
3. If a table or view already exists with the same name, a synonym cannot be created with that name.
4. When creating a synonym, it checks whether the schema of the target object exists, but it does not check whether the target object exists. If the target object does not exist, an error occurs when using a synonym.
5. The user can create a synonym only in the schema of the user, but **DBA** and members of **DBA** can create synonyms by specifying schema.

```
CREATE [OR REPLACE] [PRIVATE] SYNONYM
[schema_name_of_synonym.]synonym_name
FOR [schema_name_of_target.]target_name
[COMMENT 'synonym_comment_string'] ;
```

- **OR REPLACE**: The existing synonym is replaced by a new one without error, even if *synonym_name* already exists in the schema.
  - The existing synonym are dropped before creating a new one.
- **PRIVATE**: Specifies to create a private synonym. Even if omitted, a private synonym is created by default.
- *schema_name_of_synonym*: Specifies the schema name of the synonym. If omitted, the schema name of the current session is used.
- *synonym_name*: Specifies the name of the synonym.
- *schema_name_of_target*: Specifies the schema name of the target object. If omitted, the schema name of the current session is used.
- *target_name*: Specifies the name of the target object.
- *synonym_comment_string*: Specifies a comment of a synonym.

⚠ Warning

The target object of a synonym supports only tables and views. It is possible to use another object as the target object, but an error occurs when using a synonym.

⚠ Warning

It does not support public synonym yet.

1. Private synonym

In the example below, user u1 and user u2 have created a synonym with the same name but use different target objects.

```
call login ('dba') on class db_user;

/* current_user: dba */
```

```
create user u1;
create user u2;

create table t1 (c1 varchar(100));
insert into t1 values ('private synonym for user u1.');
grant select on t1 to u1;
grant select on t1 to u2;

create table t2 (c1 varchar(100));
insert into t2 values ('private synonym for user u2.');
grant select on t2 to u1;
grant select on t2 to u2;
```

```
call login ('u1') on class db_user;

/* current_user: u1 */
create synonym s1 for dba.t1;
select * from s1;
```

```
  c1
======================
  'private synonym for user u1.'
```

```
call login ('u2') on class db_user;

/* current_user: u2 */
create synonym s1 for dba.t2;
select * from s1;
```

```
  c1
======================
  'private synonym for user u2.'
```

2. Synonym information

In the example below, the user can find the information of a synonym in the DB_SYNONYM system virtual class.

```
/* There should be the result of example 1. */

call login ('dba') on class db_user;
```

```
/* current_user: dba */
select * from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=============================================================================
=============================================================
  's1'                    'U1'                    'NO'
't1'                    'DBA'                   NULL
  's1'                    'U2'                    'NO'
't2'                    'DBA'                   NULL
```

```
call login ('u1') on class db_user;

/* current_user: u1 */
select * from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=============================================================================
=============================================================
  's1'                    'U1'                    'NO'
't1'                    'DBA'                   NULL
```

```
call login ('u2') on class db_user;

/* current_user: u2 */
select * from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=============================================================================
=============================================================
  's1'                    'U2'                    'NO'
't2'                    'DBA'                   NULL
```

```
/* clean */

call login ('dba') on class db_user;

/* current_user: dba */
drop synonym if exists u1.s1;
drop synonym if exists u2.s1;
```

```
drop user u1;
drop user u2;

drop table if exists t1;
drop table if exists t2;
```

3. Synonym name

If a table or view already exists with the same name, a synonym cannot be created with that name.

```
call login ('public') on class db_user;

/* current_user: public */
create table t1 (c1 varchar(100));
insert into t1 values ('table for user public.');

create table s1 (c1 varchar(100));
create view s2 as select * from t1;

/* Already used as a table name. */
create synonym s1 for t1;
```

```
ERROR: before ' ; '
Class public.s1 already exists.
```

```
/* Already used as a view name. */
create synonym s2 for t1;
```

```
ERROR: before ' ; '
Class public.s2 already exists.
```

```
create synonym s3 for t1;
select * from s3;
```

```
  c1
======================
  'table for user public.'
```

```
/* clean */
```

```
call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop synonym if exists s2;
drop synonym if exists s3;
drop view if exists s2;
drop table if exists t1;
drop table if exists s1;
```

4. Synonym for synonym

When creating a synonym, it does not check whether the target object exists, so the user can create a new synonym by specifying an existing synonym as the target object. However, if a synonym is used, the target object of the synonym, which is the target object, is not found again.

```
call login ('public') on class db_user;

/* current_user: public */
create table t1 (c1 varchar(100));
insert into t1 values ('synonym for synonym.');

create synonym s1 for t1;
create synonym s2 for s1;
select * from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
========================================================================
========================================================================
  's1'                    'PUBLIC'              'NO'
't1'                    'PUBLIC'              NULL
  's2'                    'PUBLIC'              'NO'
's1'                    'PUBLIC'              NULL
```

```
select * from s2;
```

```
ERROR: before ' ; '
Unknown class "public.s1".
```

```
select * from s1;
```

```
  c1
======================
  'synonym for synonym.'
```

```
/* clean */

call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop synonym if exists s2;
drop table if exists t1;
```

5. Creating schema-specified synonyms

When **DBA** and members of **DBA** create a synonym by specifying a schema, the synonym is created in the specified schema.

```
call login ('dba') on class db_user;

/* current_user: dba */
create user u1;
create user u2;

create table t1 (c1 varchar(100));
insert into t1 values ('private synonym for user u1.');
grant select on t1 to u1;
grant select on t1 to u2;

create table t2 (c1 varchar(100));
insert into t2 values ('private synonym for user u2.');
grant select on t2 to u1;
grant select on t2 to u2;

create synonym u1.s1 for dba.t1;
create synonym u2.s1 for dba.t2;

select * from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
============================================================================
============================================================
  's1'                    'U1'                    'NO'
't1'                    'DBA'                   NULL
```

```
  's1'                  'U2'                  'NO'
't2'                  'DBA'                 NULL
```

```
call login ('u1') on class db_user;

/* current_user: u1 */
select * from s1;
```

```
  c1
======================
  'private synonym for user u1.'
```

```
call login ('u2') on class db_user;

/* current_user: u2 */
select * from s1;
```

```
  c1
======================
  'private synonym for user u2.'
```

The user can create a synonym only in the schema of the user.

```
call login ('u1') on class db_user;

/* current_user: u1 */
create synonym u2.s2 for dba.t1;
```

```
ERROR: before ' ; '
DBA, members of DBA group, and owner can perform CREATE SYNONYM.
```

```
/* clean */

call login ('dba') on class db_user;

/* current_user: dba */
drop synonym if exists u1.s1;
drop synonym if exists u2.s1;
drop user u1;
drop user u2;
```

```
drop table if exists t1;
drop table if exists t2;
```

ALTER SYNONYM

Change the target object or comment of a synonym. The synonym in use cannot be changed.

```
ALTER [PRIVATE] SYNONYM [schema_name_of_synonym.]synonym_name
{
    FOR [<schema_name_of_target>.]<target_name> [COMMENT
'comment_string'] |
    COMMENT 'synonym_comment_string'
} ;
```

- **PRIVATE**: Specifies to alter a private synonym. Even if omitted, a private synonym is created by default.
- *schema_name_of_synonym*: Specifies the schema name of the synonym. If omitted, the schema name of the current session is used.
- *synonym_name*: Specifies the name of the synonym.
- *schema_name_of_target*: Specifies the schema name of the target object. If omitted, the schema name of the current session is used.
- *target_name*: Specifies the name of the target object.
- *synonym_comment_string*: Specifies a comment of a synonym.

⚠ Warning

When the **ALTER**, **DROP**, **RENAME** statements for a synonym are executed, be careful because all query plans using the target object are deleted from the query plan cache.

However, when the **ALTER** statement is executed, the query plan is not deleted when changing to the same target object or only changing comments.

Change target object

In the example below, the target object is changed.

```
call login ('public') on class db_user;

/* current_user: public */
```

```
create table t1 (c1 varchar(100));
insert into t1 values ('target table before change.');

create table t2 (c1 varchar(100));
insert into t2 values ('target table after change.');

create synonym s1 for t1;
select * from db_synonym;
select * from s1;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=========================================================================
=========================================================================
  's1'                    'PUBLIC'              'NO'
't1'                    'PUBLIC'              NULL

  c1
======================
  'target table before change.'
```

```
alter synonym s1 for t2;
select * from db_synonym;
select * from s1;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=========================================================================
=========================================================================
  's1'                    'PUBLIC'              'NO'
't2'                    'PUBLIC'              NULL

  c1
======================
  'target table after change.'
```

```
/* clean */

call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop table if exists t1;
drop table if exists t2;
```

Change comment

In the example below, the user changes the comment of a synonym.

```
call login ('public') on class db_user;

/* current_user: public */
create table t1 (c1 varchar(100));
insert into t1 values ('change comment.');

create synonym s1 for t1 comment 'It is a synonym for the t1 table.';
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
comment
======================================================================
====================
  's1'                  'PUBLIC'              'NO'
'It is a synonym for the t1 table.'
```

It is possible to change the comment without specifying the target object.

```
alter synonym s1 comment 'the comment was changed.';
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
comment
======================================================================
====================
  's1'                  'PUBLIC'              'NO'
'the comment was changed.'
```

If both the target object and comment are not specified, an error occurs.

```
alter synonym s1;
```

```
ERROR: No options specified for ALTER SYNONYM.
```

To change a comment to **NULL**, change the comment to an empty string.

```
alter synonym s1 comment '';
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
comment
================================================================================
==================
  's1'                  'PUBLIC'              'NO'
NULL
```

```
/* clean */

call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop table if exists t1;
```

DROP SYNONYM

Drop the synonym. The synonym in use cannot be dropped. Even if a synonym is dropped, the target object is not dropped.

```
DROP [PRIVATE] SYNONYM [IF EXISTS] [schema_name.]synonym_name ;
```

- **PRIVATE**: Specifies to drop a private synonym. Even if omitted, a private synonym is created by default.
- **IF EXISTS**: No error occurs, Even if *synonym_name* does not exist in the schema.
- *schema_name*: Specifies the schema name of the synonym. If omitted, the schema name of the current session is used.
- *synonym_name*: Specifies the name of the synonym.

⚠ Warning

When the **ALTER**, **DROP**, **RENAME** statements for a synonym are executed, be careful because all query plans using the target object are deleted from the query plan cache.

```
call login ('public') on class db_user;

/* current_user: public */
create table t1 (c1 varchar(100));
insert into t1 values ('The target object of the to-be-deleted
synonym.');

create synonym s1 for t1;
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
select * from s1;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
comment
=========================================================================
====================
  's1'                  'PUBLIC'              'NO'
NULL

  c1
======================
  'The target object of the to-be-deleted synonym.'
```

```
drop synonym s1;
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
```

```
There are no results.
0 row selected.
```

```
select * from s1;
```

```
ERROR: before ' ; '
Unknown class "public.s1".
```

```
select * from t1;
```

```
  c1
======================
  'The target object of the to-be-deleted synonym.'
```

```
/* clean */

call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop table if exists t1;
```

RENAME SYNONYM

Change the name of the synonym. The name of the synonym in use cannot be changed.

1. The user cannot change the schema of the synonym when renaming a synonym.
2. If a table, view, or synonym already exists with the name to be changed, the name cannot be changed.

```
RENAME [PRIVATE] SYNONYM
[schema_name_of_old_synonym.]old_synonym_name
{AS | TO} [schema_name_of_new_synonym.]new_synonym_name ;
```

- **PRIVATE**: Specifies to rename a private synonym. Even if omitted, a private synonym is created by default.
- *schema_name_of_old_synonym*: Specifies the schema name of the to-be-renamed synonym. If omitted, the schema name of the current session is used.
- *old_synonym_name*: Specifies the name of the to-be-renamed synonym.
- *schema_name_of_new_synonym*: Specifies the schema name of synonym for new name. If omitted, the schema name of the current session is used.
- *new_synonym_name*: Specifies the new name of the new-named synonym.

⚠ Warning

When the **ALTER**, **DROP**, **RENAME** statements for a synonym are executed, be careful because all query plans using the target object are deleted from the query plan cache.

1. Cannot change schema

In the example below, an error occurs when a schema name is specified differently when renamed.

```
call login ('dba') on class db_user;

/* current_user: dba */
create user u1;
create user u2;
```

```
call login ('u1') on class db_user;

/* current_user: u1 */
create table t1 (c1 varchar(100));
insert into t1 values ('private synonym for user u1.');

create synonym s1 for t1;
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
select * from s1;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
comment
============================================================================
====================
  's1'                  'U1'                  'NO'
NULL

  c1
======================
  'private synonym for user u1.'
```

```
rename synonym s1 as u2.s2;
rename synonym u1.s1 as u2.s2;
```

```
ERROR: before ' ; '
Rename cannot change owner.
```

```
call login ('dba') on class db_user;

/* current_user: dba */
rename synonym u1.s1 as u2.s2;
```

```
ERROR: before ' ; '
Rename cannot change owner.
```

```
call login ('u1') on class db_user;

/* current_user: u1 */
rename synonym s1 as s2;
select synonym_name, synonym_owner_name, is_public_synonym, comment
from db_synonym;
select * from s2;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
comment
========================================================================
====================
  's2'                  'U1'                  'NO'
NULL

  c1
======================
  'private synonym for user u1.'
```

```
/* clean */

call login ('dba') on class db_user;

/* current_user: dba */
drop synonym if exists u1.s1;
drop synonym if exists u1.s2;
drop synonym if exists u2.s2;
drop table if exists u1.t1;
drop user u1;
drop user u2;
```

2. Name already in use

In the example below, an error occurs because the name to be changed is already in use.

```
call login ('public') on class db_user;

/* current_user: public */
create table t1 (c1 varchar(100));
insert into t1 values ('first table for user u1.');

create table t2 (c1 varchar(100));
insert into t2 values ('second table for user u1.');

create table s_t1 (c1 varchar(100));
create table s_v1 as select * from t1;
```

```
create synonym s_s1 for t2;

create synonym s1 for t1;
select * from db_synonym;
select * from s1;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
============================================================================
============================================================
  's_s1'                  'PUBLIC'              'NO'
't2'                    'PUBLIC'              NULL
  's1'                    'PUBLIC'              'NO'
't1'                    'PUBLIC'              NULL

  c1
======================
  'first table for user u1.'
```

```
rename synonym s1 as s_t1;
```

```
ERROR: before ' ; '
Class dba.s_t1 already exists.
```

```
rename synonym s1 as s_v1;
```

```
ERROR: before ' ; '
Class dba.s_v1 already exists.
```

```
rename synonym s1 as s_s1;
```

```
ERROR: before ' ; '
Synonym "dba.s_s1" already exists.
```

```
rename synonym s1 as s2;
select * from db_synonym;
select * from s2;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=========================================================================
=========================================================
  's_s1'                'PUBLIC'              'NO'
't2'                  'PUBLIC'              NULL
  's2'                  'PUBLIC'              'NO'
't1'                  'PUBLIC'              NULL

  c1
======================
  'first table for user u1.'
```

```
/* clean */

call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop synonym if exists s2;
drop synonym if exists s_s1;
drop table if exists t1;
drop table if exists t2;
drop table if exists s_t1;
drop table if exists s_v1;
```

USE SYNONYM

The synonym can be used only if table names and view names are available. When using synonyms, the user must have access authorization to the target object.

1. **GRANT** and **REVOKE** cannot be done for synonyms. If the schema name of a synonym is specified, a synonym existing in another schema can be used.
2. The synonym cannot be used in **ALTER**, **DROP**, **RENAME** statements and **TRUNCATE** statements that change the target object.

1. Use synonyms from other schemas

```
call login ('dba') on class db_user;

/* current_user: dba */
create user u1;
create user u2;
```

```
call login ('u1') on class db_user;

/* current_user: u1 */
create table t1 (c1 varchar(100));
insert into t1 values ('first table for user u1.');
grant select on t1 to u2;

create synonym s1 for t1;
select * from s1;
```

```
  c1
======================
  'first table for user u1.'
```

```
call login ('u2') on class db_user;

/* current_user: u2 */
select * from s1;
select * from u1.s1;
```

```
ERROR: before ' ; '
Unknown class "u2.s1".

  c1
======================
  'first table for user u1.'
```

```
/* clean */

call login ('dba') on class db_user;

/* current_user: dba */
drop synonym if exists u1.s1;
drop table if exists u1.t1;
drop user u1;
drop user u2;
```

2. Statements where synonyms cannot be used

```
call login ('public') on class db_user;

/* current_user: public */
```

```
create table t1 (c1 varchar(100));
insert into t1 values ('first table for user public.');

create synonym s1 for t1;
select * from db_synonym;
select * from s1;
```

```
  synonym_name          synonym_owner_name    is_public_synonym
target_name           target_owner_name     comment
=========================================================================
=========================================================
  's1'                  'PUBLIC'              'NO'
't1'                  'PUBLIC'              NULL

  c1
======================
  'first table for user public.'
```

```
alter table s1 add column c2 int;
```

```
ERROR: before '  add column c2 int; '
Class public.s1 does not exist.
```

```
drop table s1;
```

```
ERROR: before ' ; '
Class public.s1 does not exist.
```

```
rename table s1 to s2;
```

```
ERROR: before ' ; '
Class public.s1 does not exist.
```

```
truncate s1;
```

```
ERROR: before ' ; '
Class public.s1 does not exist.
```

```
select * from s1;
```

```
  c1
======================
  'first table for user public.'
```

```
/* clean */

call login ('public') on class db_user;

/* current_user: public */
drop synonym if exists s1;
drop table if exists t1;
drop table if exists s2;
```

SERVER DEFINITION STATEMENTS

CREATE SERVER

Server Definition

Create remote access information using the **CREATE SERVER** statement. The created server is used to designate a remote server when executing a SELECT query using **DBLINK**. For how to use the prepared server, refer to Remote Table.

```
CREATE SERVER <dblink_server_name> (<connect_info>) ;

    <dblink_server_name> ::= [owner_name.] server_name

    <connect_info> ::=
            <connect_info>, <connect_item>
            | <connect_item>
    <connect_item> ::=
            HOST = host_string
            | PORT = port_number
            | DBNAME = db_name
            | USER = user_name
            | PASSWORD = [password_string]
```

```
                | PROPERTIES = [properties_string]
                | COMMENT = [server_comment_string]
```

- *owner_name*: Specifies the name of the owner of the server to be created.
- *server_name*: Specifies the name of the server to be created. (up to 254 bytes)
- <*connect_info*>: **HOST**, **PORT**, **DBNAME**, and **USER** are mandatory items in the <connect_item> list of access information.
- <*connect_item*>: It consists of HOST, PORT, DBNAME, USER, PASSWOED, PROPERTIES, and COMMENT items, and the same item cannot be duplicated.
  - *host_string*: It is the hose name or IP address of the broker server that has DBMS information to be accessed remotely.
  - *port_number*: The port number of the broker server that has DBMS information to be accessed remotely.
  - *db_name*: The database name to connect to remotely.
  - *user_name*: The username to use when connecting to the database to be accessed remotely.
  - *password_string*: Password string for *user_name* used to connect to the database to be accessed remotely.
  - *properties_string*: A string of property information (up to 2047 bytes) used when connecting to a broker (or gateway) for remote database usage. For detailed attribute information, see cci_connect_with_url
  - *server_comment_string*: Specifies comments about server information. (up to 1023 bytes)

Note

db_name and user_name can be written in the form of identifier or string literal.

- **Example of identifier format**

  t123db, “123db”, *123db*, [124db]

- **Example of string format**

  ‘t123db’, ‘123db’

Example 1 The following basic example has the required fields (HOST,PORT,DBNAME,USER) and also contains the PROPERTIES and COMMENT fields.

```
CREATE SERVER dblink_srv1 (
       HOST='192.168.1.8',
       PORT=3300,
       DBNAME=demodb,
       USER=dba,
       PASSWORD='password1234',
       PROPERTIES='?rcTime=600',
       COMMENT='this is dblink_srv1'
);
```

Example 2 The following is an example that includes minimal information when creating a server. It indicates that the remote demodb will be connected to the dev1 account without a password. srv1, srv2, and srv3 have the same information.

```
 CREATE SERVER srv1 (
        HOST='192.168.1.8',
        PORT=3300,
        DBNAME=demodb,
        USER=dev1
 );

CREATE SERVER srv2 (
        HOST='192.168.1.8',
        PORT=3300,
        DBNAME=demodb,
        USER=dev1,
        PASSWORD=
 );

 CREATE SERVER srv3 (
        HOST='192.168.1.8',
        PORT=3300,
        DBNAME=demodb,
        USER=dev1,
        PASSWORD=''
 );
```

Example 3 The following example specifies the owner when creating the server. In case of CREATE without designating the owner, the current user becomes the owner. Later, you can change the owner using the ALTER SERVER statement. In the example below, the two servers have the same name as *srv2*, but have different owners as *dba* and *cub*, respectively.

```
 -- When the current account is dba
 CREATE SERVER srv2 (
        HOST='192.168.1.8',
        PORT=3300,
        DBNAME=demodb,
```

```
        USER=dev1,
        PASSWORD='dev1-password',
        COMMENT='The owner of this server is dba'
 );

CREATE SERVER cub.srv2 (
        HOST='192.168.1.8',
        PORT=3300,
        DBNAME=demodb,
        USER=dev2,
        PASSWORD='dev2-password',
        COMMENT='The owner of this server is cub.'
 );
```

Example 4 The following is an example of setting up for failover of a remote database by using the altHosts attribute in PROPERTIES when using a remote database configured in an HA environment. For more information, see cci_connect_with_url

```
CREATE SERVER dblink_srv1 (
   HOST='192.168.1.8',
   PORT=3300,
   DBNAME=demodb,
   USER=dba,
   PASSWORD='password1234',
   PROPERTIES='?altHosts=192.168.1.9:33000',
   COMMENT='this is dblink_srv1'
);
```

Example 5 The following is an example of a poor use of the server name (<dblink_server_name>).

```
CREATE SERVER srv1 ( HOST='localhost', PORT=3300, DBNAME=demodb,
USER=dev1 );
CREATE SERVER "srv 1" ( HOST='localhost', PORT=3300, DBNAME=demodb,
USER=dev1 );
CREATE SERVER "srv.1" ( HOST='localhost', PORT=3300, DBNAME=demodb,
USER=dev1 );
CREATE SERVER cub.srv1 ( HOST='localhost', PORT=3300, DBNAME=demodb,
USER=dev1 );

CREATE SERVER "cub"."srv 2" ( HOST='localhost', PORT=3300,
DBNAME=demodb, USER=dev1 );
CREATE SERVER [cub].[srv.2] ( HOST='localhost', PORT=3300,
DBNAME=demodb, USER=dev1 );
```

In the above example, creation with *“srv.1”* and *[cub].[srv.2]* names fails. In the case of *cub.srv1*, *cub* will be recognized as the username and the server name will be recognized as *srv1*.

Note

A dot (.) cannot be used for the server name. You cannot use a dot (.) in the server name, even if it is enclosed in quotation marks or [].

RENAME SERVER

You can change the server name using the **RENAME SERVER** syntax.

```
RENAME SERVER [owner_name.] old_server_name {AS | TO}
new_server_name ;
```

- *owner_name*: Specify the owner name of the target server to be renamed.
- *old_server_name*: Specify the name of the server to be renamed.
- *new_server_name*: Specifies the name of the server to be newly assigned. (up to 254 bytes)

Note

Only the owner of the server or members of the ownership group can change the information. In particular, **DBA** or members of **DBA** can change all server information.

Even after the change, the owners remain the same. To change the owner, refer to ALTER SERVER syntax.

```
-- When the current account is dba
RENAME SERVER srv1 AS srv2;
RENAME SERVER dev1.srv1 AS srv3;
```

Even if the above example is performed under the *dba* account, the owner of *srv2* is not changed and is maintained as *cub*, the owner of the *srv1* server. Also, the owner of the *srv3* server remains *dev1*.

DROP SERVER

Existing servers can be removed using **DROP SERVER** syntax. If the **IF EXISTS** clause is used together, no error occurs even if the server does not exist.

```
DROP SERVER [IF EXISTS] [owner_name.] server_name  ;
```

- *owner_name*: Specify the owner name of the server to be removed..
- *server_name*: Specify the name of the server to be removed.

```
DROP SERVER srv1;
DROP SERVER cub.srv1;
DROP SERVER IF EXISTS srv2;
```

⚠ Warning

When deleting a user with the DROP USER statement, if there is a server owned by the user, an error is processed and the user is not deleted. First, remove the server owned by the user with the DROP SERVER statement, and then delete the user account.

```
-- When the current account is dba
csql> create user cub;
Execute OK. (0.000371 sec) Committed.

1 command(s) successfully processed.
csql> create server cub.tsrv (HOST='localhost', PORT=3300,
DBNAME=demdb, USER=dev1);
Execute OK. (0.000761 sec) Committed.

1 command(s) successfully processed.
csql> drop user cub;

In the command from line 1,

ERROR: Cannot drop the user who owns database objects(class/trigger/
serial/server etc).

0 command(s) successfully processed.
csql> drop server cub.tsrv;
Execute OK. (0.000761 sec) Committed.

1 command(s) successfully processed.
csql> drop user cub;
Execute OK. (0.001650 sec) Committed.

1 command(s) successfully processed.
csql>
```

In the example above, you can see that the drop user *cub* statement is failing while the *tsrv* server owned by the *cub* account is created. After removing the *cub.tsrv* server, you can see that the *cub* account could be deleted normally..

ALTER SERVER

You can change the server information by using the **ALTER** statement. You can change the owner of the target server, or update information about HOST, PORT, DBNAME, USER, PASSWOED, PROPERTIES, and COMMENT.

```
ALTER SERVER <dblink_server_name> <alter_server_list> ;

    <dblink_server_name> ::=  [owner_name.] server_name

    <alter_server_list> ::=
            <alter_server_list>, <alter_server_item>
            | <alter_server_item>
    <alter_server_item> ::=
            OWNER TO owner_name
            | CHANGE <connect_item>
    <connect_item> ::=
            HOST = host_string
            | PORT = port_number
            | DBNAME = db_name
            | USER = user_name
            | PASSWORD = [password_string]
            | PROPERTIES = [properties_string]
            | COMMENT = [server_comment_string]
```

Note

Only the owner of the server or members of the ownership group can change the information. In particular, **DBA** or **DBA** members can change all server information.

Warning

It is not possible to update to remove values for HOST, PORT, DBNAME, and USER.

OWNER TO clause

You can change the owner of the server using the **OWNER TO** clause.

```
ALTER SERVER [owner_name.] server_name  OWNER TO new_owner_name ;
```

- *owner_name*: Specifies the owner name of the target server whose owner is to be changed.
- *server_name*: Specifies the name of the target server whose owner is to be changed.
- *new_owner_name*: Specifies the new owner name.

⚠ Warning

- There is no OWNER TO clause in an ALTER SERVER clause, or it must be specified only once.

```
CREATE SERVER srv1 (HOST='broker-server-name', PORT=3300,
DBNAME=demodb, USER=dev1);
ALTER SERVER srv1 OWNER TO usr1;
ALTER SERVER usr1.srv1 OWNER TO usr2;
```

CHANGE clause

The **CHANGE** section is used to change the values of HOST, PORT, DBNAME, USER, PASSWOED, PROPERTIES, and COMMENT items.

⚠ Warning

- Multiple CHANGE clauses in one ALTER SERVER statement can be listed by separating them with commas (,). However, at this time, only one CHANGE clause for the same item should be specified.
- Items that were not mentioned when performing the ALTER SERVER syntax do not initialize or delete the value, but retain the existing value.

```
ALTER SERVER  [owner_name.] server_name CHANGE <connect_item> [,
CHANGE <connect_item>] ... ;

   <connect_item> ::=
           HOST = host_string
```

```
            | PORT = port_number
            | DBNAME = db_name
            | USER = user_name
            | PASSWORD = [password_string]
            | PROPERTIES = [properties_string]
            | COMMENT = [server_comment_string]
```

- *owner_name*: Specifies the username of the server to be created.
- *server_name*: Specifies the name of the server to be created. (up to 254 bytes)
- *host_string*: It is the hose name or IP address of the broker server that has DBMS information to be accessed remotely.
- *port_number*: The port number of the broker server that has DBMS information to be accessed remotely.
- *db_name*: The database name to connect to remotely.
- *user_name*: he username to use when connecting to the database to be accessed remotely.
- *password_string*: Password string for *user_name* used to connect to the database to be accessed remotely.
- *properties_string*: Property information string used when connecting to the database to be accessed remotely. (up to 2047 bytes)
- *server_comment_string*: Specifies comments about server information. (up to 1023 bytes)

```
CREATE SERVER srv1 ( HOST='localhost', PORT=3300, DBNAME=demodb,
USER=dev1 );

ALTER SERVER srv1 CHANGE HOST='127.0.0.1';
ALTER SERVER srv1 CHANGE HOST='127.0.0.1', OWNER TO usr1;
ALTER SERVER srv1 CHANGE USER=dev2, CHANGE PASSWORD='dev2-pawword',
CHANGE PORT=3500;
```

The above example shows that the **CHANGE** clause can be used to list multiple items at once, or it can be used together with the **OWNER TO** clause.

```
ALTER SERVER srv1 CHANGE PORT=;
ALTER SERVER srv1 CHANGE DBNAME=;
ALTER SERVER srv1 CHANGE USER=;
ALTER SERVER srv1 CHANGE HOST=;
ALTER SERVER srv1 CHANGE HOST='';
```

All of the above examples are examples of not supported cases. Since HOST, PORT, DBNAME, and USER, which are essential elements in the configuration of server information, must have values, setting changes that delete values are not supported. In particular, in the case of HOST, setting it to an empty string is also not allowed.

```
ALTER SERVER srv1 CHANGE PASSWORD=;
ALTER SERVER srv1 CHANGE PASSWORD='';

ALTER SERVER srv1 CHANGE PROPERTIES=;
ALTER SERVER srv1 CHANGE PROPERTIES='';

ALTER SERVER srv1 CHANGE COMMENT=;
ALTER SERVER srv1 CHANGE COMMENT='';
```

The above examples are all supported examples. PASSWORD, PROPERTIES, COMMENT, which are not essential elements in the configuration of server information, do not necessarily have a value, so it is possible to change the setting to delete the value.

STORED FUNCTION/PROCEDURE DEFINITION STATEMENTS

CREATE PROCEDURE

Create stored procedure using the **CREATE PROCEDURE** statement.

```
CREATE [OR REPLACE] PROCEDURE [schema_name.]procedure_name
([<parameter_definition> [, <parameter_definition>] ...])
[<procedure_properties>]
{IS | AS} LANGUAGE <lang>
  <body>
COMMENT 'procedure_comment';

    <parameter_definition> ::= parameter_name [mode] sql_type [ {
DEFAULT | = } <default_expr> ] [COMMENT 'parameter_comment_string']
    <lang> ::= [PLCSQL | JAVA]
    <mode> ::= IN | OUT | IN OUT | INOUT
    <procedure_properties> ::= <authid>
        <authid> ::= AUTHID {DEFINER | OWNER | CALLER | CURRENT_USER}
```

You can use the **OR REPLACE** clause to replace or create a new stored function/procedure.

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.

- *procedure_name*: Specifies the name of the stored procedure to be created (maximum 222 bytes).
- *parameter_name*: Specifies the name of the parameter (maximum 254 bytes).
- *sql_type*: Specifies the data type of the parameter. For available data types, refer to Supported Data Types of Arguments and Return.
- *default_arg*: Specifies the default value of the parameter. Refer to Using Default Arguments.
- *authid*: Specifies the execution authority of the stored procedure. For more details, refer to The Difference Between Owner's Rights and Caller's Rights.
- *parameter_comment_string*: Specifies the comment string for the parameter.
- *body*: Specifies the body of the stored procedure.
- *procedure_comment*: Specifies the comment string for the stored procedure.

COMMENT of Stored Procedure

A comment of stored function/procedure can be written at the end of the statement as follows.

```
CREATE FUNCTION Hello() RETURN VARCHAR
AS LANGUAGE JAVA
NAME 'SpCubrid.HelloCubrid() return java.lang.String'
COMMENT 'function comment';
```

A comment of a paramenter can be written as follows.

```
CREATE OR REPLACE FUNCTION test(i in int COMMENT 'arg i')
RETURN int AS LANGUAGE JAVA NAME 'SpTest.testInt(int) return int'
COMMENT 'function test';
```

A comment of a stored function/procedure can be shown by running the following syntax.

```
SELECT sp_name, comment FROM db_stored_procedure;
```

A comment for a parameter of a function can be shown by running the following syntax.

```
SELECT sp_name, arg_name, comment FROM db_stored_procedure_args;
```

Checking the Published Java Stored Procedure Information

You can check the information on the published Java stored procedure. The **db_stored_procedure** system virtual table provides the information on stored names and types, return types, number of parameters, Java class specifications, and the owner. The **db_stored_procedure_args** system virtual table provides the information on parameters used in the stored function/procedure.

```
SELECT * FROM db_stored_procedure WHERE sp_type = 'PROCEDURE';
```

```
sp_name               sp_type               return_type
arg_count  lang target                owner
=========================================================================
===========
'athlete_add'         'PROCEDURE'
'void'                          4
'JAVA''Athlete.Athlete(java.lang.String, java.lang.String,
java.lang.String, java.lang.String)'  'DBA'
```

```
SELECT * FROM db_stored_procedure_args WHERE sp_name = 'athlete_add';
```

```
sp_name   index_of  arg_name  data_type      mode
=============================================
 'athlete_add'                  0  'name'
'STRING'              'IN'
 'athlete_add'                  1  'gender'
'STRING'              'IN'
 'athlete_add'                  2  'nation_code'
'STRING'              'IN'
 'athlete_add'                  3  'event'
'STRING'              'IN'
```

CREATE FUNCTION

Create stored function using the **CREATE FUNCTION** statement.

```
CREATE [OR REPLACE] FUNCTION [schema_name.]function_name
([<parameter_definition> [, <parameter_definition>] ...])
RETURN sql_type
[<function_properties>]
{IS | AS} LANGUAGE <lang>
  <body>
COMMENT 'function_comment';

    <parameter_definition> ::= parameter_name [mode] sql_type
```

```
[<default_arg>] [COMMENT 'param_comment_string']
        <default_arg> ::= { DEFAULT | = } <default_expr>
    <function_properties> ::= [<authid>] [<deterministic>]
        <authid> ::= AUTHID {DEFINER | OWNER | CALLER | CURRENT_USER}
        <deterministic> ::= [NOT DETERMINISTIC | DETERMINISTIC]
    <lang> ::= [PLCSQL | JAVA]
    <mode> ::= IN | OUT | IN OUT | INOUT
```

- *schema_name*: Specifies the schema name (up to 31 bytes). If omitted, the schema name of the current session is used.
- *function_name*: Specifies the name of the stored function to be created (up to 222 bytes).
- *parameter_name*: Specifies the name of the parameter (up to 254 bytes).
- *sql_type*: Specifies the data type of the parameter or return value. For available data types, refer to Supported Data Types of Arguments and Return.
- *default_arg*: Specifies the default value of the parameter. Refer to Using Default Arguments.
- *param_comment_string*: Specifies the comment string for the parameter.
- *authid*: Specifies the execution authority of the stored function. For more details, refer to The Difference Between Owner's Rights and Caller's Rights.
- *deterministic*: Specifies whether the stored function is deterministic. For more details, refer to Use Deterministic Functions.
- *body*: Specifies the body of the stored function.
- *function_comment*: Specifies the comment string for the stored function.

COMMENT of Stored Function

A comment of stored function/procedure can be written at the end of the statement as follows.

```
CREATE FUNCTION Hello() RETURN VARCHAR
AS LANGUAGE JAVA
NAME 'SpCubrid.HelloCubrid() return java.lang.String'
COMMENT 'function comment';
```

A comment of a paramenter can be written as follows.

```
CREATE OR REPLACE FUNCTION test(i in int COMMENT 'arg i')
RETURN int AS LANGUAGE JAVA NAME 'SpTest.testInt(int) return int'
COMMENT 'function test';
```

A comment of a stored function/procedure can be shown by running the following syntax.

```
SELECT sp_name, comment FROM db_stored_procedure;
```

A comment for a parameter of a function can be shown by running the following syntax.

```
SELECT sp_name, arg_name, comment FROM db_stored_procedure_args;
```

Checking the Published Java Stored Function Information

You can check the information on the published Java stored function. The **db_stored_procedure** system virtual table provides the information on stored names and types, return types, number of parameters, Java class specifications, and the owner. The **db_stored_procedure_args** system virtual table provides the information on parameters used in the stored function/procedure.

```
SELECT * FROM db_stored_procedure WHERE sp_type = 'FUNCTION';
```

```
sp_name               sp_type               return_type
arg_count  lang target                owner
=============================================================================
===========
'hello'               'FUNCTION'
'STRING'                          0  'JAVA''SpCubrid.HelloCubrid()
return java.lang.String'  'DBA'

'sp_int'              'FUNCTION'
'INTEGER'                         1  'JAVA''SpCubrid.SpInt(int) return
int'  'DBA'
```

```
SELECT * FROM db_stored_procedure_args WHERE sp_name = 'sp_int';
```

```
sp_name   index_of  arg_name  data_type      mode
=============================================
 'sp_int'                          0  'i'
'INTEGER'             'IN'
```

DROP PROCEDURE

In CUBRID, A stored proceudre can be deleted using the **DROP PROCEDURE** statement. Also, you can delete multiple stored procedures at the same time with several *procedure_name*s separated by a comma (,).

```
DROP PROCEDURE procedure_name [{ , procedure_name , ... }];
```

- *procedure_name*: Specifies the name of procedure to delete

```
DROP PROCEDURE hello, sp_int;
```

A stored procedure can be deleted only by the user who published it or by DBA members. For example, if a **PUBLIC** user published the 'sp_int' stored procedure, only the **PUBLIC** or **DBA** members can delete it.

DROP FUNCTION

In CUBRID, A stored function can be deleted using the **DROP FUNCTION** statement. You can also delete multiple stored functions at the same time by specifying several *function_name*s separated by commas (,).

```
DROP FUNCTION function_name [{ , function_name , ... }];
```

- *function_name*: Specifies the name of function to delete

```
DROP FUNCTION hello, sp_int;
```

A stored function can be deleted only by the user who published it or by DBA members. For example, if a **PUBLIC** user published the 'sp_int' stored function, only the **PUBLIC** or **DBA** members can delete it.

Operators and Functions

Logical Operators

For logical operators, boolean expressions are specified as operands; **TRUE**, **FALSE** or **NULL** is returned as a result. If a boolean value is used in an arithmetic expression, **TRUE** is evaluated to 1 and **FALSE** is evaluated to 0.

Note

As of version 11.2, using a non-logical value (such as an integer or column) as an operand raises the error "operand must be logical expression." Such a condition must be rewritten with

an explicit comparison: a constant condition like **WHERE 1** as **WHERE 1=1**, and a column condition like **WHERE column_name** as **WHERE column_name <> 0**.

The following table shows the logical operators.

Logical Operators

Logical Operator	Description	Condition
AND, **&&**	If all operands are **TRUE**, it returns **TRUE**.	a **AND** b
OR, **||**	If none of operands is **NULL** and one or more operands are **TRUE**, it returns **TRUE**. If **pipes_as_concat** is no that is a parameter related to SQL statements, a double pipe symbol can be used as **OR** operator.	a **OR** b
XOR	If none of operand is **NULL** and each of operand has a different value, it returns **TRUE**.	a **XOR** b
NOT, **!**	A unary operator. If a operand is **FALSE** , it returns **TRUE**. If it is **TRUE** , returns **FALSE**.	**NOT** a

Truth Table of Logical Operators

a	b	a AND b	a OR b	NOT a	a XOR b
TRUE	**TRUE**	TRUE	TRUE	FALSE	FALSE

a	b	a AND b	a OR b	NOT a	a XOR b
TRUE	**FALSE**	FALSE	TRUE	FALSE	TRUE
TRUE	**NULL**	NULL	TRUE	FALSE	NULL
FALSE	**TRUE**	FALSE	TRUE	TRUE	TRUE
FALSE	**FALSE**	FALSE	FALSE	TRUE	FALSE
FALSE	**NULL**	FALSE	NULL	TRUE	NULL

Comparison Operators

Contents

Comparison Operators 333

The comparison operators compare the operand on the left and on the right, and they return 1 or 0. The operands of comparison operations must have the same data type. Therefore, implicit type casting by the system or implicit type casting by the user is required.

Syntax 1

```
<expression> <comparison_operator> <expression>

    <expression> ::=
        bit_string |
        character_string |
        numeric_value |
        date-time_value |
        collection_value |
        NULL

    <comparison_operator> ::=
        = |
        <=> |
        <> |
        != |
```

```
        > |
        < |
        >= |
        <=
```

Syntax 2

```
<expression> IS [NOT] NULL
<logical_expression> IS [NOT] <truth_value>

    <expression> ::=
        bit_string |
        character_string |
        numeric_value |
        date-time_value |
        collection_value |
        NULL

    <truth_value> ::=
        TRUE |
        FALSE |
        UNKNOWN

    <logical_expression> ::=
        <expression> <comparison_operator> <expression> |
        <logical_expression> { AND | OR | XOR } <logical_expression>
|
        NOT <logical_expression>

    <comparison_operator> ::=
        = |
        <=> |
        <> |
        != |
        > |
        < |
        >= |
        <=
```

- <*expression*>: Declares an expression to be compared.
  - *bit_string*: A Boolean operation can be performed on bit strings, and all comparison operators can be used for comparison between bit strings. If you compare two expressions with different lengths, 0s are padded at the end of the shorter one.
  - *character_string*: When compared by a comparison operator, two character strings must have the same character sets. The comparison is determined by the collation sequence of the character code set. If you compare two character strings with different lengths, blanks

are padded at the end of the shorter one before comparison so that they have the same length.

- *numeric_value*: The Boolean operator can be performed for all numeric values and any types of comparison operator can be used. When two different numeric types are compared, the system implicitly performs type casting. For example, when an **INTEGER** value is compared with a **DECIMAL** value, the system first casts **INTEGER** to **DECIMAL** before it performs comparison. When you compare a **FLOAT** value, you must specify the range instead of an exact value because the processing of **FLOAT** is dependent on the system.
- *date-time_value*: If two date-time values with the same type are compared, the order is determined in time order. That is, when comparing two date-time values, the earlier date is considered to be smaller than the later date. You cannot compare date-time values with different type by using a comparison operator; therefore, you must explicitly convert it. However, comparison operation can be performed between DATE, TIMESTAMP, and DATETIME because they are implicitly converted.
- *collection_value*: When comparing two LISTs (SEQUENCE), comparison is performed between the two elements by user-specified order when LIST was created. Comparison including SET and MULTISET is overloaded to an appropriate operator. You can perform comparison operations on SET, MULTISET, LIST, or SEQUENCE by using a containment operator explained later in this chapter. For details, see Containment Operators.
- **NULL**: The **NULL** value is not included in the value range of any data type. Therefore, comparison between **NULL** values is only allowed to determine if the given value is **NULL** or not. An implicit type cast does not take place when a **NULL** value is assigned to a different data type. For example, when an attribute of **INTEGER** type has a **NULL** and is compared with a floating point type, the **NULL** value is not coerced to **FLOAT** before comparison is made. A comparison operation on the **NULL** value does not return a result.

- <*logical_expression*>: An expression that evaluates to a boolean (logical) value, such as the result of a comparison (e.g., a = 1) or a logical operation (e.g., (a = 1) AND (b = 2)).

The following table shows the comparison operators supported by CUBRID and their return values.

Comparison Operators

Comparison Operator	Description	Predicate	Return Value
=	A general equal sign. It compares whether the values of the left and right	1=2 1=NULL	0 NULL

Comparison Operator	Description	Predicate	Return Value
	operands are the same. Returns **NULL** if one or more operands are NULL.		
<=>	A NULL-safe equal sign. It compares whether the values of the left and right operands are the same including **NULL**. Returns 1 if both operands are **NULL**.	1<=>2 1<=> NULL	0 0
<>, !=	The value of left operand is not equal to that of right operand. If any operand value is **NULL**, **NULL** is returned.	1<>2	1
>	The value of left operand is greater than that of right operand. If any operand value is **NULL**, **NULL** is returned.	1>2	0
<	The value of left operand is less than that of right operand. If any operand value is	1<2	1

Comparison Operator	Description	Predicate	Return Value
	NULL, **NULL** is returned.		
>=	The value of left operand is greater than or equal to that of right operand. If any operand value is **NULL**, **NULL** is returned.	1>=2	0
<=	The value of left operand is less than or equal to that of right operand. If any operand value is **NULL**, **NULL** is returned.	1<=2	1
IS *truth_value*	Compares whether the value of the left operand is the same as the truth value of the right. The truth value may be **TRUE**, **FALSE**, or **UNKNOWN**.	(1=1) IS FALSE	0

Comparison Operator	Description	Predicate	Return Value
IS NOT *truth_value*	Compares whether the value of the left operand differs from the truth value of the right. The truth value may be **TRUE**, **FALSE**, or **UNKNOWN**.	(1=1) IS NOT FALSE	1

Note

As of version 11.2, the left operand of **IS [NOT] { TRUE | FALSE | UNKNOWN }** must be a logical expression; otherwise the error "operand must be logical expression." is raised. For example, **1 IS FALSE** must be written as **(1=1) IS FALSE**. For a logical expression, the UNKNOWN truth value is represented as NULL, so **IS [NOT] UNKNOWN** and **IS [NOT] NULL** return the same result.

The following are the examples which use comparison operators.

```
SELECT (1 <> 0); -- 1 is displayed because it is TRUE.
SELECT (1 != 0); -- 1 is displayed because it is TRUE.
SELECT (0.01 = '0.01'); -- An error occurs because a numeric data
type is compared with a character string type.
SELECT (1 = NULL); -- NULL is displayed.
SELECT (1 <=> NULL); -- 0 is displayed because it is FALSE.
SELECT (1.000 = 1); -- 1 is displayed because it is TRUE.
SELECT ('cubrid' = 'CUBRID'); -- 0 is displayed because it is case
sensitive.
SELECT ('cubrid' = 'cubrid'); -- 1 is displayed because it is TRUE.
SELECT (SYSTIMESTAMP = CAST(SYSDATETIME AS TIMESTAMP)); -- 1 is
displayed after casting the type explicitly and then performing
comparison operator.
SELECT (SYSTIMESTAMP = SYSDATETIME); --0 is displayed after casting
the type implicitly and then performing comparison operator.
SELECT (SYSTIMESTAMP <> NULL); -- NULL is returned without
performing comparison operator.
SELECT (SYSTIMESTAMP IS NOT NULL); -- 1 is returned because it is
not NULL.
SELECT ((1 > 0) IS TRUE); -- 1 is displayed because (1 > 0) is TRUE.
SELECT ((1 > 0) IS FALSE); -- 0 is displayed because (1 > 0) is not
```

```
FALSE.
SELECT ((1 > 0) IS UNKNOWN); -- 0 is displayed because (1 > 0) is
not UNKNOWN.
SELECT ((NULL = 1) IS UNKNOWN); -- 1 is displayed because (NULL = 1)
is UNKNOWN.
SELECT ((1 > 0) IS NOT FALSE); -- 1 is displayed because (1 > 0) is
not FALSE.
```

Arithmetic Operators

Contents

Arithmetic Operators 339

- Arithmetic Operations and Type Casting of Numeric Data Types 341
- Arithmetic Operations and Type Casting of DATE/TIME Data Types 345
  - Behavior related to timezone parameters 348

For arithmetic operators, there are binary operators for addition, subtraction, multiplication, or division, and unary operators to represent whether the number is positive or negative. The unary operators to represent the numbers' positive/negative status have higher priority over the binary operators.

```
<expression>  <mathematical_operator>  <expression>

    <expression> ::=
        bit_string |
        character_string |
        numeric_value |
        date-time_value |
        collection_value |
        NULL

    <mathematical_operator> ::=
        <set_arithmetic_operator> |
        <arithmetic_operator>

            <arithmetic_operator> ::=
                + |
                - |
                * |
                { / | DIV } |
```

```
                { % | MOD }

            <set_arithmetic_operator> ::=
                UNION |
                DIFFERENCE |
                { INTERSECT | INTERSECTION }
```

- <*expression*>: Declares the mathematical operation to be calculated.
- <*mathematical_operator*>: A operator that performs an operation the arithmetic and the set operators are applicable.
  - <*set_arithmetic_operator*>: A set arithmetic operator that performs operations such as union, difference and intersection on collection type operands.
  - <*arithmetic_operator*>: An operator to perform the four fundamental arithmetic operations.

The following table shows the arithmetic operators supported by CUBRID and their return values.

Arithmetic Operators

Arithmetic Operator | Description		Operator	Return Value
+ | Addition		1+2	3
-	Subtraction	1-2	-1
*****	Multiplication	1*2	2
/	Division. Returns quotient.	1/2.0	0.500000000
DIV	Division. Returns quotient.	1 DIV 2	0
% , MOD	Division. Returns quotient. An operator must	1 % 2 1 MOD 2	1

Arithmetic Operator | Description		Operator	Return Value
	be an integer type, and it always returns integer. If an operand is real number, the **MOD** function can be used.		

Arithmetic Operations and Type Casting of Numeric Data Types

All numeric data types can be used for arithmetic operations. The result type of the operation differs depending on the data types of the operands and the type of the operation. The following table shows the result data types of addition/subtraction/multiplication for each operand type.

Result Data Type by Operand Type

	INT	NUMERIC	FLOAT	DOUBLE
INT	INT or BIGINT	NUMERIC	FLOAT	DOUBLE
NUMERIC	NUMERIC	NUMERIC (p and s are also converted)	DOUBLE	DOUBLE
FLOAT	FLOAT	DOUBLE	FLOAT	DOUBLE
DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE

Note that the result type of the operation does not change if all operands are of the same data type but type casting occurs exceptionally in division operations. An error occurs when a denominator, i.e. a divisor, is 0.

The following table shows the total number of digits (*p*) and the number of digits after the decimal point (*s*) of the operation results when all operands are of the **NUMERIC** type.

Result of NUMERIC Type Operation

Operation	Maximum Precision	Maximum Scale
N(p1, s1) + N(p2, s2)	max(p1 - s1, p2 - s2) + max(s1, s2) + 1	max(s1, s2)
N(p1, s1) - N(p2, s2)	max(p1 - s1, p2 - s2) + max(s1, s2)	max(s1, s2)
N(p1, s1) * N(p2, s2)	p1 + p2 + 1	s1 + s2
N(p1, s1) / N(p2, s2)	Pt = (p1 - s1) + s2 + max(9, max(s1, s2)) Pt = (Pt > 38) ? 38 : Pt	St = max(9, max(s1, s2)) St = (Pt > 38) ? min(9, St - (Pt - 38)) : St

When the arithmetic operator is '/', the result type differs depending on the setting of the system parameter **oracle_compat_number_behavior**. That is, in case of integer/integer, if **oracle_compat_number_behavior** is set to yes, the result type is NUMERIC. Otherwise, it follows the **result data type for each operand type** rule.

Example

```
--int * int
SELECT 123*123;
```

```
      123*123
=============
        15129
```

```
-- int * int returns overflow error
SELECT (1234567890123*1234567890123);
```

```
ERROR: Data overflow on data type bigint.
```

```
-- int * numeric returns numeric type
SELECT (1234567890123*CAST(1234567890123 AS NUMERIC(15,2)));
```

```
 (1234567890123* cast(1234567890123 as numeric(15,2)))
=======================
  1524157875322755800955129.00
```

```
-- int * float returns float type
SELECT (1234567890123*CAST(1234567890123 AS FLOAT));
```

```
 (1234567890123* cast(1234567890123 as float))
================================================
                                   1.524158e+024
```

```
-- int * double returns double type
SELECT (1234567890123*CAST(1234567890123 AS DOUBLE));
```

```
 (1234567890123* cast(1234567890123 as double))
================================================
                           1.5241578753227 56e+024
```

```
-- numeric * numeric returns numeric type
SELECT (CAST(1234567890123 AS NUMERIC(15,2))*CAST(1234567890123 AS
NUMERIC(15,2)));
```

```
 ( cast(1234567890123 as numeric(15,2))* cast(1234567890123 as
numeric(15,2)))
=======================
  1524157875322755800955129.0000
```

```
-- numeric * float returns double type
SELECT (CAST(1234567890123 AS NUMERIC(15,2))*CAST(1234567890123 AS
FLOAT));
```

```
 ( cast(1234567890123 as numeric(15,2))* cast(1234567890123 as
float))
=========================================================================
==

1.524157954716582e+024
```

```
-- numeric * double returns double type
SELECT (CAST(1234567890123 AS NUMERIC(15,2))*CAST(1234567890123 AS
DOUBLE));
```

```
 ( cast(1234567890123 as numeric(15,2))* cast(1234567890123 as
double))
=========================================================================
===

1.524157875322756e+024
```

```
-- float * float returns float type
SELECT (CAST(1234567890123 AS FLOAT)*CAST(1234567890123 AS FLOAT));
```

```
 ( cast(1234567890123 as float)* cast(1234567890123 as float))
=============================================================
                                                1.524158e+024
```

```
-- float * double returns double type
SELECT (CAST(1234567890123 AS FLOAT)*CAST(1234567890123 AS DOUBLE));
```

```
 ( cast(1234567890123 as float)* cast(1234567890123 as double))
==============================================================
                                       1.524157954716582e+024
```

```
-- double * double returns double type
SELECT (CAST(1234567890123 AS DOUBLE)*CAST(1234567890123 AS DOUBLE));
```

```
 ( cast(1234567890123 as double)* cast(1234567890123 as double))
===============================================================
                                        1.524157875322756e+024
```

```
csql> ;get oracle_compat_number_behavior
oracle_compat_number_behavior=n

-- int / int returns int type without type conversion or rounding
SELECT 100100/100000;
```

```
  100100/100000
================
               1
```

```
csql> ;get oracle_compat_number_behavior
oracle_compat_number_behavior=n

-- int / int returns int type without type conversion or rounding
SELECT 100100/200200;
```

```
  100100/200200
================
               0
```

```
csql> ;get oracle_compat_number_behavior
oracle_compat_number_behavior=y

-- int / int returns numeric type with oracle_compat_number_behavior
SELECT 1/2;
```

```
             1/2
================
             0.5
```

```
-- int / zero returns error
SELECT 100100/(100100-100100);
```

```
ERROR: Attempt to divide by zero.
```

Arithmetic Operations and Type Casting of DATE/TIME Data Types

If all operands are date/time type, only a subtraction operation is allowed and its return value is **BIGINT** . Note that the unit of the operation differs depending on the types of the operands. Both addition and subtraction operations are allowed in case of date/time and integer types In this case, operation units and return values are date/time data type.

The following table shows operations allowed for each operand type, and their result types.

Allowable Operation and Result Data Type by Operand Type

	TIME (in seconds)	DATE (in day)	TIMESTAMP (in seconds)	DATETIME (in milliseconds)	INT
TIME	A subtraction is allowed. **BIGINT**	X	X	X	An addition and a subtraction are allowed. **TIME**
DATE	X	A subtraction is allowed. **BIGINT**	A subtraction is allowed. **BIGINT**	A subtraction is allowed. **BIGINT**	An addition and a subtraction are allowed. **DATE**
TIMESTAMP	X	A subtraction is allowed. **BIGINT**	A subtraction is allowed. **BIGINT**	A subtraction is allowed. **BIGINT**	An addition and a subtraction are allowed. **TIMESTAMP**
DATETIME	X	A subtraction is allowed. **BIGINT**	A subtraction is allowed. **BIGINT**	A subtraction is allowed. **BIGINT**	An addition and a subtraction are allowed. **DATETIME**
INT	An addition and a subtraction are allowed. **TIME**	An addition and a subtraction are allowed. **DATE**	An addition and a subtraction are allowed. **TIMESTAMP**	An addition and a subtraction are allowed. **DATETIME**	All operations are allowed.

Note

If any of the date/time arguments contains **NULL**, **NULL** is returned.

Example

```
-- initial systimestamp value
SELECT SYSDATETIME;
```

```
  SYSDATETIME
==============================
  07:09:52.115 PM 01/14/2010
```

```
-- time type + 10(seconds) returns time type
SELECT (CAST (SYSDATETIME AS TIME) + 10);
```

```
 ( cast( SYS_DATETIME  as time)+10)
====================================
  07:10:02 PM
```

```
-- date type + 10 (days) returns date type
SELECT (CAST (SYSDATETIME AS DATE) + 10);
```

```
 ( cast( SYS_DATETIME  as date)+10)
====================================
  01/24/2010
```

```
-- timestamp type + 10(seconds) returns timestamp type
SELECT (CAST (SYSDATETIME AS TIMESTAMP) + 10);
```

```
 ( cast( SYS_DATETIME  as timestamp)+10)
=========================================
  07:10:02 PM 01/14/2010
```

```
-- systimestamp type + 10(milliseconds) returns systimestamp type
SELECT (SYSDATETIME  + 10);
```

```
 ( SYS_DATETIME +10)
================================
  07:09:52.125 PM 01/14/2010
```

```
SELECT DATETIME '09/01/2009 03:30:30.001 pm'- TIMESTAMP '08/31/2009
03:30:30 pm';
```

```
 datetime '09/01/2009 03:30:30.001 pm'-timestamp '08/31/2009
03:30:30 pm'
=======================================
  86400001
```

```
SELECT TIMESTAMP '09/01/2009 03:30:30 pm'- TIMESTAMP '08/31/2009
03:30:30 pm';
```

```
 timestamp '09/01/2009 03:30:30 pm'-timestamp '08/31/2009 03:30:30
pm'
=======================================
  86400
```

Behavior related to timezone parameters

TIMESTAMP and TIMESTAMP WITH LOCAL TIME ZONE data types stores internally UNIX epoch values (number of secons elapsed from 1970). When leap second is used (tz_leap_second_support is set to yes, see Timezone Parameter), they may contain virtual date-time values.

```
Virtual date-time       Unix timestamp
2008-12-31 23:59:58  -> 79399951
2008-12-31 23:59:59  -> 79399952
2008-12-31 23:59:60  -> 79399953    -> not real date (introduced by
leap second)
2009-01-01 00:00:00  -> 79399954
2009-01-01 00:00:01  -> 79399955
```

Arithmetic operations with TIMESTAMP and TIMESTAMPLTZ values are performed directly on Unix epoch values. Unix epoch values coresponding to non-exising date/time values are allowed. For this reason, the comparison:

```
SELECT TIMESTAMPLTZ'2008-12-31 23:59:59 UTC'=TIMESTAMPLTZ'2008-12-31
23:59:59 UTC'+1;
```

```
timestampltz '2008-12-31 23:59:59 UTC'=timestampltz '2008-12-31
23:59:59 UTC'+1
================================================================================
============

       0
```

is equivalent to comparing the Unix timestamps : 79399952 and 79399953. But when same values are used as TIMESTAMPTZ, there is equality:

```
SELECT TIMESTAMPTZ'2008-12-31 23:59:59 UTC'=TIMESTAMPTZ'2008-12-31
23:59:59 UTC'+1;
```

```
timestamptz '2008-12-31 23:59:59 UTC'=timestamptz '2008-12-31
23:59:59 UTC'+1
================================================================================
==========

       1
```

The inconsistency arise at display :

```
SELECT TIMESTAMPLTZ'2008-12-31 23:59:59 UTC'+1;
```

```
timestampltz '2008-12-31 23:59:59 UTC'+1
============================================
11:59:59 PM 12/31/2008 Etc/UTC UTC
```

Since '2008-12-31 23:59:60 UTC' corresponding to Unix timestamp value 79399953 is not a real date, the immediately preceding value is used. Internally, it is equivalent to the value ('2008-12-31 23:59:60 UTC').

TIMESTAMP WITH TIME ZONE data type contains both a UNIX timestamp and a timezone identifier. Arithmetic on TIMESTAMPTZ is also performed on UNIX timestamp part value, but is followed by an automatic adjusting operation. The presence of timezone identifier (which includes region, offset and daylight saving), requires the TIMESTAMPTZ object to be a valid date-time. The operation timestamptz'2008-12-31 23:59:59 UTC'+1 implies an automatic validation-conversion: instead of (79399953, UTC) which is not a valid date-time the value is automatically converted to (79399952,UTC) which coresponds to '2008-12-31 23:59:59 UTC'.

After each arithmetic operation implying DATETIMETZ and TIMESTAMPTZ, CUBRID performs an automatic adjustment of result value which involves:

- adjusting the timezone identifier : adding a number of seconds to a date with timezone may lead to change of internally stored offset rule, daylight saving rule, hence the timezone identifier must be updated
- adjusting the Unix timestamp (only for TIMESTAMPTZ): virtual date-time values (when leap-second is enabled) are always converted to the immediately preceding Unix timestamp value.

Set Arithmetic Operators

SET, MULTISET, LIST

To compute union, difference or intersection of collections types (**SET**, **MULTISET**, and **LIST (SEQUENCE)**), you can use +, -, or * operators, respectively.

```
<value_expression>  <set_arithmetic_operator> <value_expression>

    <value_expression> ::=
        collection_value |
        NULL

    <set_arithmetic_operator> ::=
        + (union) |
        - (difference) |
        * (intersection)
```

The following table shows a result data type by the operator if collection type is an operand.

Result Data Type by Operand Type

	SET	MULTISET	LIST
SET	**+ , - , * : SET**	**+ , - , * : MULTISET**	**+ , - , * : MULTISET**
MULTISET	**+ , - , * : MULTISET**	**+ , - , * : MULTISET**	**+ , - , * : MULTISET**
LIST (=SEQUENCE)	**+, -, * : MULTISET**	**+, -, * : MULTISET**	**+ : LIST -, * : MULTISET**

The following are the examples which execute arithmetic operations with collection types.

```
SELECT ((CAST ({3,3,3,2,2,1} AS SET))+(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as set))+( cast({4, 3, 3, 2} as
multiset)))
======================
  {1, 2, 2, 3, 3, 3, 4}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS MULTISET))+(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as multiset))+( cast({4, 3, 3, 2} as
multiset)))
======================
  {1, 2, 2, 2, 3, 3, 3, 3, 3, 4}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS LIST))+(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as sequence))+( cast({4, 3, 3, 2} as
multiset)))
======================
  {1, 2, 2, 2, 3, 3, 3, 3, 3, 4}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS SET))-(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as set))-( cast({4, 3, 3, 2} as
multiset)))
======================
  {1}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS MULTISET))-(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as multiset))-( cast({4, 3, 3, 2} as
multiset)))
=======================
  {1, 2, 3}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS LIST))-(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as sequence))-( cast({4, 3, 3, 2} as
multiset)))
=======================
  {1, 2, 3}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS SET))*(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as set))*( cast({4, 3, 3, 2} as
multiset)))
=======================
  {2, 3}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS MULTISET))*(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as multiset))*( cast({4, 3, 3, 2} as
multiset)))
=======================
  {2, 3, 3}
```

```
SELECT ((CAST ({3,3,3,2,2,1} AS LIST))*(CAST ({4,3,3,2} AS
MULTISET)));
```

```
 (( cast({3, 3, 3, 2, 2, 1} as sequence))*( cast({4, 3, 3, 2} as
multiset)))
=======================
{2, 3, 3}
```

Assigning Collection Value to Variable

For a collection value to be assigned to a variable, the outer query must return a single row as a result.

The following example shows how to assign a collection value to a variable. The outer query must return only a single row as follows:

```
CREATE TABLE people (
    ssn VARCHAR(10),
    name VARCHAR(255)
);

INSERT INTO people
VALUES ('1234', 'Ken'), ('5678', 'Dan'), ('9123', 'Jones');

SELECT SET(SELECT name
FROM people
WHERE ssn in {'1234', '5678'})
TO :name_group;
```

Statement Set Operators

UNION, DIFFERENCE, INTERSECTION

Statement set operators are used to get union, difference or intersection on the result of more than one query statement specified as an operand. Note that the data types of the data to be retrieved from the target tables of the two query statements must be identical or implicitly castable.

```
query_term statement_set_operator [qualifier] <query_term>
[{statement_set_operator [qualifier] <query_term>}];

    <query_term> ::=
        query_specification
        subquery
```

- *qualifier*
  - DISTINCT, DISTINCTROW or UNIQUE(A returned instance is a distinct value.)
  - ALL (All instances are returned. Duplicates are allowed.)

- *statement_set_operator*
  - UNION (union)
  - DIFFERENCE (difference)
  - INTERSECT | INTERSECTION (intersection)

The following table shows statement set operators supported by CUBRID.

Statement Set Operators

Statement Set Operator	Description	Note
UNION	Union Duplicates are not allowed.	Outputs all instance results containing duplicates with **UNION ALL**
DIFFERENCE	Difference Duplicates are not allowed.	Same as the **EXCEPT** operator. Outputs all instance results containing duplicates with **DIFFERENCE ALL**.
INTERSECTION	Intersection Duplicates are not allowed.	Same as the **INTERSECTION** operator. Outputs all instance results containing duplicates with **INTERSECTION ALL**.

The following are the examples which execute queries with statement set operators.

```
CREATE TABLE nojoin_tbl_1 (ID INT, Name VARCHAR(32));

INSERT INTO nojoin_tbl_1 VALUES (1,'Kim');
INSERT INTO nojoin_tbl_1 VALUES (2,'Moy');
INSERT INTO nojoin_tbl_1 VALUES (3,'Jonas');
INSERT INTO nojoin_tbl_1 VALUES (4,'Smith');
INSERT INTO nojoin_tbl_1 VALUES (5,'Kim');
INSERT INTO nojoin_tbl_1 VALUES (6,'Smith');
INSERT INTO nojoin_tbl_1 VALUES (7,'Brown');

CREATE TABLE nojoin_tbl_2 (id INT, Name VARCHAR(32));
```

```
INSERT INTO nojoin_tbl_2 VALUES (5,'Kim');
INSERT INTO nojoin_tbl_2 VALUES (6,'Smith');
INSERT INTO nojoin_tbl_2 VALUES (7,'Brown');
INSERT INTO nojoin_tbl_2 VALUES (8,'Lin');
INSERT INTO nojoin_tbl_2 VALUES (9,'Edwin');
INSERT INTO nojoin_tbl_2 VALUES (10,'Edwin');

--Using UNION to get only distinct rows
SELECT id, name FROM nojoin_tbl_1
UNION
SELECT id, name FROM nojoin_tbl_2;
```

```
           id  name
=================================
            1  'Kim'
            2  'Moy'
            3  'Jonas'
            4  'Smith'
            5  'Kim'
            6  'Smith'
            7  'Brown'
            8  'Lin'
            9  'Edwin'
           10  'Edwin'
```

```
--Using UNION ALL not eliminating duplicate selected rows
SELECT id, name FROM nojoin_tbl_1
UNION ALL
SELECT id, name FROM nojoin_tbl_2;
```

```
           id  name
=================================
            1  'Kim'
            2  'Moy'
            3  'Jonas'
            4  'Smith'
            5  'Kim'
            6  'Smith'
            7  'Brown'
            5  'Kim'
            6  'Smith'
            7  'Brown'
            8  'Lin'
            9  'Edwin'
           10  'Edwin'
```

```
--Using DEFFERENCE to get only rows returned by the first query but
not by the second
SELECT id, name FROM nojoin_tbl_1
DIFFERENCE
SELECT id, name FROM nojoin_tbl_2;
```

```
           id  name
================================
            1  'Kim'
            2  'Moy'
            3  'Jonas'
            4  'Smith'
```

```
--Using INTERSECTION to get only those rows returned by both queries
SELECT id, name FROM nojoin_tbl_1
INTERSECT
SELECT id, name FROM nojoin_tbl_2;
```

```
           id  name
================================
            5  'Kim'
            6  'Smith'
            7  'Brown'
```

Containment Operators

Contents

Containment Operators 356

- SETEQ 361
- SETNEQ 362
- SUPERSET 363
- SUPERSETEQ 365
- SUBSET 366

- SUBSETEQ 367

Containment operators are used to check the containment relationship by performing comparison operation on operands of the collection data type. Collection data types or subqueries can be specified as operands. The operation returns **TRUE** or **FALSE** if there is a containment relationship between the two operands of identical/different/subset/proper subset.

```
<collection_operand> <containment_operator> <collection_operand>

    <collection_operand> ::=
        <set> |
        <multiset> |
        <sequence> (or <list>) |
        <subquery> |
        NULL

    <containment_operator> ::=
        SETEQ |
        SETNEQ |
        SUPERSET |
        SUBSET |
        SUPERSETEQ |
        SUBSETEQ
```

- <*collection_operand*>: This expression that can be specified as an operand is a single SET-valued attribute, an arithmetic expression containing a SET operator or a SET value enclosed in braces. If the type is not specified, the SET value enclosed in braces is treated as a **LIST** type by default.

  Subqueries can be specified as operands. If a column which is not a collection type is searched, a collection data type keyword is required for the subquery like **SET** (*subquery*). The column retrieved by a subquery must return a single set so that it can be compared with the set of the other operands.

  If the element type of collection is an object, the OIDs, not its contents, are compared. For example, two objects with different OIDs are considered to be different even though they have the same attribute values.

  - **NULL**: Any of operands to be compared is **NULL**, **NULL** is returned.

The description and return values about the containment operators supported by CUBRID are as follows:

Containment Operators Supported by CUBRID

Containment Operator	Description	Predicates	Return Value
A **SETEQ** B	A = B: Elements in A and B are same each other.	{1,2} SETEQ {1,2,2}	0
A **SETNEQ** B	A <> B: Elements in A and B are not same each other.	{1,2} SETNEQ {1,2,3}	1
A **SUPERSET** B	A > B: B is a proper subset of A.	{1,2} SUPERSET {1,2,3}	0
A **SUBSET** B	A < B: A is a proper subset of B.	{1,2} SUBSET {1,2,3}	1
A **SUPERSETEQ** B	A >= B: B is a subset of A.	{1,2} SUPERSETEQ {1,2,3}	0
A **SUBSETEQ** B	A <= B: A is a subset of B.	{1,2} SUBSETEQ {1,2,3}	1

The following table shows than possibility of operation by operand and type conversion if a containment operator is used.

Possibility of Operation by Operand

	SET	MULTISET	LIST(=SEQUENCE)
SET	Operation possible	Operation possible	Operation possible
MULTISET	Operation possible	Operation possible	Operation possible (**LIST** is converted into **MULTISET**)

	SET	MULTISET	LIST(=SEQUENCE)
LIST(=SEQUENCE)	Operation possible	Operation possible (**LIST** is converted into **MULTISET**)	Some operation possible (**SETEQ**, **SETNEQ**) Error occurs for the rest of operators.

```
--empty set is a subset of any set
SELECT ({} SUBSETEQ (CAST ({3,1,2} AS SET)));
```

```
       Result
=============
            1
```

```
--operation between set type and null returns null
SELECT ((CAST ({3,1,2} AS SET)) SUBSETEQ NULL);
```

```
       Result
=============
         NULL
```

```
--{1,2,3} seteq {1,2,3} returns true
SELECT ((CAST ({3,1,2} AS SET)) SETEQ (CAST ({1,2,3,3} AS SET)));
```

```
       Result
=============
            1
```

```
--{1,2,3} seteq {1,2,3,3} returns false
SELECT ((CAST ({3,1,2} AS SET)) SETEQ (CAST ({1,2,3,3} AS
MULTISET)));
```

```
       Result
=============
            0
```

```
--{1,2,3} setneq {1,2,3,3} returns true
SELECT ((CAST ({3,1,2} AS SET)) SETNEQ (CAST ({1,2,3,3} AS
MULTISET)));
```

```
       Result
=============
            1
```

```
--{1,2,3} subseteq {1,2,3,4} returns true
SELECT ((CAST ({3,1,2} AS SET)) SUBSETEQ (CAST ({1,2,4,4,3} AS
SET)));
```

```
       Result
=============
            1
```

```
--{1,2,3} subseteq {1,2,3,4,4} returns true
SELECT ((CAST ({3,1,2} AS SET)) SUBSETEQ (CAST ({1,2,4,4,3} AS
MULTISET)));
```

```
       Result
=============
            1
```

```
--{1,2,3} subseteq {1,2,4,4,3} returns true
SELECT ((CAST ({3,1,2} AS SET)) SUBSETEQ (CAST ({1,2,4,4,3} AS
LIST)));
```

```
       Result
=============
            0
```

```
--{1,2,3} subseteq {1,2,3,4,4} returns true
SELECT ((CAST ({3,1,2} AS SET)) SUBSETEQ (CAST ({1,2,3,4,4} AS
LIST)));
```

```
       Result
=============
            1
```

```
--{3,1,2} seteq {3,1,2} returns true
SELECT ((CAST ({3,1,2} AS LIST)) SETEQ (CAST ({3,1,2} AS LIST)));
```

```
       Result
=============
            1
```

```
--error occurs because LIST subseteq LIST is not supported
SELECT ((CAST ({3,1,2} AS LIST)) SUBSETEQ (CAST ({3,1,2} AS LIST)));
```

```
ERROR: ' subseteq ' operator is not defined on types sequence and
sequence.
```

SETEQ

The **SETEQ** operator returns **TRUE** if the first operand is the same as the second one. It can perform comparison operator for all collection data type.

```
collection_operand SETEQ collection_operand
```

```
--creating a table with SET type address column and LIST type
zip_code column

CREATE TABLE contain_tbl (id INT PRIMARY KEY, name CHAR(10), address
SET VARCHAR(20), zip_code LIST INT);
INSERT INTO contain_tbl VALUES(1, 'Kim', {'country', 'state'},{1, 2,
3});
INSERT INTO contain_tbl VALUES(2, 'Moy', {'country', 'state'},{3, 2,
1});
INSERT INTO contain_tbl VALUES(3, 'Jones', {'country', 'state',
'city'},{1,2,3,4});
INSERT INTO contain_tbl VALUES(4, 'Smith', {'country', 'state',
'city', 'street'},{1,2,3,4});
INSERT INTO contain_tbl VALUES(5, 'Kim', {'country', 'state',
'city', 'street'},{1,2,3,4});
INSERT INTO contain_tbl VALUES(6, 'Smith', {'country', 'state',
'city', 'street'},{1,2,3,5});
INSERT INTO contain_tbl VALUES(7, 'Brown', {'country', 'state',
```

```
'city', 'street'},{});

--selecting rows when two collection_operands are same in the WHERE
clause
SELECT id, name, address, zip_code FROM contain_tbl WHERE address
SETEQ {'country','state', 'city'};
```

```
           id  name                  address               zip_code
==============================================================================
==========
            3  'Jones     '          {'city', 'country', 'state'}
{1, 2, 3, 4}

1 row selected.
```

```
--selecting rows when two collection_operands are same in the WHERE
clause
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SETEQ {1,2,3};
```

```
           id  name                  address               zip_code
==============================================================================
==========
            1  'Kim       '          {'country', 'state'}  {1, 2, 3}

1 rows selected.
```

SETNEQ

The **SETNEQ** operator returns **TRUE** (1) if a first operand is different from a second operand. A comparable operation can be performed for all collection data types.

```
collection_operand SETNEQ collection_operand
```

```
--selecting rows when two collection_operands are not same in the
WHERE clause
SELECT id, name, address, zip_code FROM contain_tbl WHERE address
SETNEQ {'country','state', 'city'};
```

```
           id  name                  address               zip_code
==============================================================================
==========
```

```
           1  'Kim       '          {'country', 'state'}  {1, 2, 3}
           2  'Moy       '          {'country', 'state'}  {3, 2, 1}
           4  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
           5  'Kim       '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
           6  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 5}
           7  'Brown     '          {'city', 'country', 'state',
'street'}  {}

6 rows selected.
```

```
--selecting rows when two collection_operands are not same in the
WHERE clause
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SETNEQ {1,2,3};
```

```
          id  name                  address               zip_code
==========================================================================
==========
           2  'Moy       '          {'country', 'state'}  {3, 2, 1}
           3  'Jones     '          {'city', 'country', 'state'}
{1, 2, 3, 4}
           4  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
           5  'Kim       '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
           6  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 5}
           7  'Brown     '          {'city', 'country', 'state',
'street'}  {}
```

SUPERSET

The **SUPERSET** operator returns **TRUE** (1) when a second operand is a proper subset of a first operand; that is, the first one is larger than the second one. If two operands are identical, **FALSE** (0) is returned. Note that **SUPERSET** is not supported if all operands are **LIST** type.

```
collection_operand SUPERSET collection_operand
```

```
--selecting rows when the first operand is a superset of the second
operand and they are not same
```

```
SELECT id, name, address, zip_code FROM contain_tbl WHERE address
SUPERSET {'country','state','city'};
```

```
           id  name                    address                 zip_code
==========================================================================
==========
            4  'Smith     '            {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
            5  'Kim       '            {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
            6  'Smith     '            {'city', 'country', 'state',
'street'}  {1, 2, 3, 5}
            7  'Brown     '            {'city', 'country', 'state',
'street'}  {}
```

```
--SUPERSET operator cannot be used for comparison between LIST and
LIST type values
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUPERSET {1,2,3};
```

```
ERROR: ' superset ' operator is not defined on types sequence and
sequence.
```

```
--Comparing operands with a SUPERSET operator after casting LIST
type as SET type
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUPERSET (CAST ({1,2,3} AS SET));
```

```
           id  name                    address                 zip_code
==========================================================================
==========
            3  'Jones     '            {'city', 'country', 'state'}
{1, 2, 3, 4}
            4  'Smith     '            {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
            5  'Kim       '            {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
            6  'Smith     '            {'city', 'country', 'state',
'street'}  {1, 2, 3, 5}
```

SUPERSETEQ

The **SUPERSETEQ** operator returns **TRUE** (1) when a second operand is a subset of a first operand; that is, the first one is identical to or larger than the second one. Note that **SUPERSETEQ** is not supported if an operand is **LIST** type.

```
collection_operand SUPERSETEQ collection_operand
```

```
--selecting rows when the first operand is a superset of the second
operand
SELECT id, name, address, zip_code FROM contain_tbl WHERE address
SUPERSETEQ {'country','state','city'};
```

```
           id  name                  address                zip_code
==========================================================================
==========
            3  'Jones     '          {'city', 'country', 'state'}
{1, 2, 3, 4}
            4  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
            5  'Kim       '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
            6  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 5}
            7  'Brown     '          {'city', 'country', 'state',
'street'}  {}
```

```
--SUPERSETEQ operator cannot be used for comparison between LIST and
LIST type values
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUPERSETEQ {1,2,3};
```

```
ERROR: ' superseteq ' operator is not defined on types sequence and
sequence.
```

```
--Comparing operands with a SUPERSETEQ operator after casting LIST
type as SET type
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUPERSETEQ (CAST ({1,2,3} AS SET));
```

```
           id  name                  address                zip_code
==========================================================================
==========
```

```
           1  'Kim       '          {'country', 'state'}  {1, 2, 3}
           3  'Jones     '          {'city', 'country', 'state'}
{1, 2, 3, 4}
           4  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
           5  'Kim       '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 4}
           6  'Smith     '          {'city', 'country', 'state',
'street'}  {1, 2, 3, 5}
```

SUBSET

The **SUBSET** operator returns **TRUE** (1) if the second operand contains all elements of the first operand. If the first and the second collection have the same elements, **FALSE** (0) is returned. Note that both operands are the **LIST** type, the **SUBSET** operation is not supported.

```
collection_operand SUBSET collection_operand
```

```
--selecting rows when the first operand is a subset of the second
operand and they are not same
SELECT id, name, address, zip_code FROM contain_tbl WHERE address
SUBSET {'country','state','city'};
```

```
          id  name                address              zip_code
==============================================================
==========
           1  'Kim       '          {'country', 'state'}  {1, 2, 3}
           2  'Moy       '          {'country', 'state'}  {3, 2, 1}

--SUBSET operator cannot be used for comparison between LIST and
LIST type values
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUBSET {1,2,3};
```

```
ERROR: ' subset ' operator is not defined on types sequence and
sequence.

--Comparing operands with a SUBSET operator after casting LIST type
as SET type
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUBSET (CAST ({1,2,3} AS SET));
```

```
          id  name                address              zip_code
==============================================================
```

```
==========
           7  'Brown     '          {'city', 'country', 'state',
'street'}  {}
```

SUBSETEQ

The **SUBSETEQ** operator returns **TRUE** (1) when a first operand is a subset of a second operand; that is, the second one is identical to or larger than the first one. Note that **SUBSETEQ** is not supported if an operand is **LIST** type.

```
collection_operand SUBSETEQ collection_operand
```

```
--selecting rows when the first operand is a subset of the second
operand
SELECT id, name, address, zip_code FROM contain_tbl WHERE address
SUBSETEQ {'country','state','city'};
```

```
           id  name                  address               zip_code
=============================================================================
==========
            1  'Kim       '          {'country', 'state'}  {1, 2, 3}
            2  'Moy       '          {'country', 'state'}  {3, 2, 1}
            3  'Jones     '          {'city', 'country', 'state'}
{1, 2, 3, 4}
```

```
--SUBSETEQ operator cannot be used for comparison between LIST and
LIST type values
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUBSETEQ {1,2,3};
```

```
ERROR: ' subseteq ' operator is not defined on types sequence and
sequence.
```

```
--Comparing operands with a SUBSETEQ operator after casting LIST
type as SET type
SELECT id, name, address, zip_code FROM contain_tbl WHERE zip_code
SUBSETEQ (CAST ({1,2,3} AS SET));
```

```
           id  name                  address               zip_code
=============================================================================
==========
```

```
           1  'Kim       '           {'country', 'state'}  {1, 2, 3}
           7  'Brown     '           {'city', 'country', 'state',
'street'}  {}
```

BIT Functions and Operators

Contents

BIT Functions and Operators 368

- Bitwise Operator 368
- BIT_AND 370
- BIT_OR 370
- BIT_XOR 371
- BIT_COUNT 371

Bitwise Operator

A **Bitwise** operator performs operations in bits, and can be used in arithmetic operations. An integer type is specified as the operand and the **BIT** type cannot be specified. An integer of **BIGINT** type (64-bit integer) is returned as a result of the operation. If one or more operands are **NULL**, **NULL** is returned.

The following table shows the bitwise operators supported by CUBRID.

The bitwise operators

Bitwise operator	Description	Expression	Return Value
&	Performs AND operation in bits and returns a BIGINT integer.	17 & 3	1
|		17 | 3	19

Bitwise operator	Description	Expression	Return Value
	Performs OR operation in bits and returns a BIGINT integer.		
^	Performs XOR operation in bits and returns a BIGINT integer.	17 ^ 3	18
~	A unary operator. It performs complementary operation that reverses (INVERT) the bit order of the operand and returns a BIGINT integer.	~17	-18
<<	Performs the operation to shift bits of the left operand as far to the left as the value of the right operand, and returns a BIGINT integer.	17 << 3	136
>>	Performs the operation to shift bits of the left operand as far to the right as the value of the right operand, and	17 >> 3	2

Bitwise operator	Description	Expression	Return Value
	returns a BIGINT integer.		

BIT_AND

BIT_AND(*expr*)

As an aggregate function, it performs **AND** operations in bits on every bit of *expr*. The return value is a **BIGINT** type. If there is no row that satisfies the expression, **NULL** is returned.

Parameters:

expr – An expression of integer type

Return type:

BIGINT

```
CREATE TABLE bit_tbl(id int);
INSERT INTO bit_tbl VALUES (1), (2), (3), (4), (5);
SELECT 1&3&5, BIT_AND(id) FROM bit_tbl WHERE id in(1,3,5);
```

```
     1&3&5           bit_and(id)
=============================================
         1                     1
```

BIT_OR

BIT_OR(*expr*)

As an aggregate function, it performs **OR** operations in bits on every bit of *expr*. The return value is a **BIGINT** type. If there is no row that satisfies the expression, **NULL** is returned.

Parameters:

expr – An expression of integer type

Return type:

BIGINT

```
SELECT 1|3|5, BIT_OR(id) FROM bit_tbl WHERE id in(1,3,5);
```

```
    1|3|5           bit_or(id)
==========================================
          7                     7
```

BIT_XOR

BIT_XOR(*expr*)

As an aggregate function, it performs **XOR** operations in bits on every bit of *expr*. The return value is a **BIGINT** type. If there is no row that satisfies the expression, **NULL** is returned.

Parameters:

expr – An expression of integer type

Return type:

BIGINT

```
SELECT 1^3^5, BIT_XOR(id) FROM bit_tbl WHERE id in(1,3,5);
```

```
    1^3^5           bit_xor(id)
==========================================
          7                     7
```

BIT_COUNT

BIT_COUNT(*expr*)

The **BIT_COUNT** function returns the number of bits of *expr* that have been set to 1; it is not an aggregate function. The return value is a **BIGINT** type.

Parameters:

expr – An expression of integer type

Return type:

BIGINT

```
SELECT BIT_COUNT(id) FROM bit_tbl WHERE id in(1,3,5);
```

```
   bit_count(id)
================
           1
           2
           2
```

String Functions and Operators

Contents

String Functions and Operators 372

- Concatenation Operator 374
- ASCII 375
- BIN 376
- BIT_LENGTH 377
- CHAR_LENGTH, CHARACTER_LENGTH, LENGTHB, LENGTH 379
- CHR 380
- CONCAT 382
- CONCAT_WS 383
- ELT 384

- FIELD 385
- FIND_IN_SET 387
- FROM_BASE64 388
- INSERT 388
- INSTR 390
- LCASE, LOWER 392
- LEFT 393
- LOCATE 394
- LPAD 396
- LTRIM 398
- MID 399
- OCTET_LENGTH 400
- POSITION 402
- REPEAT 404
- REPLACE 405
- REVERSE 407
- RIGHT 407
- RPAD 408
- RTRIM 410
- SPACE 411
- STRCMP 412

- SUBSTR 414
- SUBSTRING 416
- SUBSTRING_INDEX 417
- TO_BASE64 419
- TRANSLATE 420
- TRIM 421
- UCASE, UPPER 423

Note

In the string functions, if the value of **oracle_style_empty_string** parameter is yes, CUBRID does not separate an empty string and NULL; according to each function, CUBRID regards all of them as NULL or an empty string. For the detail description, see oracle_style_empty_string.

Concatenation Operator

A concatenation operator gets a character string or bit string data type as an operand and returns a concatenated string. The plus sign (+) and double pipe symbol (||) are provided as concatenation operators for character string data. If **NULL** is specified as an operand, a **NULL** value is returned.

If **pipes_as_concat** that is a parameter related to SQL statement is set to **no** (default value: yes), a double pipe (||) symbol is interpreted as an **OR** operator. If plus_as_concat is set to no (default value: yes), a plus (+) symbol is interpreted as a plus (+) operator. In such case, It is recommended to concatenate strings or bit strings, by using the **CONCAT** function.

```
<concat_operand1> +  <concat_operand2>
<concat_operand1> || <concat_operand2>

    <concat_operand1> ::=
        bit string |
        character string
        NULL

    <concat_operand2> ::=
        bit string |
```

```
        character string
        NULL
```

- <*concat_operand1*>: Left string after concatenation. String or bit string can be specified.
- <*concat_operand2*>: Right string after concatenation. String or bit string can be specified.

```
SELECT 'CUBRID' || ',' + '2008';
```

```
 'CUBRID'||','+'2008'
======================
  'CUBRID,2008'
```

```
SELECT 'cubrid' || ',' || B'0010' ||B'0000' ||B'0000' ||B'1000';
```

```
 'cubrid'||','||B'0010'||B'0000'||B'0000'||B'1000'
======================
  'cubrid,2008'
```

```
SELECT ((EXTRACT(YEAR FROM SYS_TIMESTAMP))||(EXTRACT(MONTH FROM
SYS_TIMESTAMP)));
```

```
 (( extract(year  from  SYS_TIMESTAMP ))||( extract(month  from
SYS_TIMESTAMP )))
======================
  '200812'
```

```
SELECT 'CUBRID' || ',' + NULL;
```

```
 'CUBRID'||','+null
======================
  NULL
```

ASCII

ASCII(*str*)

The **ASCII** function returns the ASCII code of the most left character in numeric value. If an input string is **NULL**, **NULL** is returned. This **ASCII** function supports single-byte character

sets only. If a numeric value is entered, it is converted into character string and then the ASCII code of the most left character is returned.

Parameters:

str – Input string

Return type:

STRING

```
SELECT ASCII('5');
```

```
53
```

```
SELECT ASCII('ab');
```

```
97
```

BIN

BIN(*n*)

The **BIN** function converts a **BIGINT** type number into binary string. If an input string is **NULL**, **NULL** is returned. When you input the string which cannot be transformed into **BIGINT**, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

n – A **BIGINT** type number

Return type:

STRING

```
SELECT BIN(12);
```

```
'1100'
```

BIT_LENGTH

BIT_LENGTH(*string*)

The **BIT_LENGTH** function returns the length (bits) of a character string or bit string as an integer value. The return value of the **BIT_LENGTH** function may depend on the character set, because for the character string, the number of bytes taken up by a single character is different depending on the character set of the data input environment (e.g., UTF-8 Korean characters: one Korean character is 3*8 bits). For details about character sets supported by CUBRID, see Character Strings. When you input the invalid value, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

string – Specifies the character string or bit string whose number of bits is to be calculated. If this value is **NULL**, **NULL** is returned.

Return type:

INT

```
SELECT BIT_LENGTH('');
```

```
   bit_length('')
================
              0
```

```
SELECT BIT_LENGTH('CUBRID');
```

```
   bit_length('CUBRID')
======================
                    48
```

```
-- UTF-8 Korean character
SELECT BIT_LENGTH('큐브리드');
```

```
     bit_length('큐브리드')
=========================
                       96
```

```
SELECT BIT_LENGTH(B'010101010');
```

```
   bit_length(B'010101010')
===========================
                          9
```

```
CREATE TABLE bit_length_tbl (char_1 CHAR, char_2 CHAR(5), varchar_1
VARCHAR, bit_var_1 BIT VARYING);
INSERT INTO bit_length_tbl VALUES('', '', '', B''); --Length of
empty string
INSERT INTO bit_length_tbl VALUES('a', 'a', 'a', B'010101010'); --
English character
INSERT INTO bit_length_tbl VALUES(NULL, '큐', '큐', B'010101010'); --
UTF-8 Korean character and NULL
INSERT INTO bit_length_tbl VALUES(' ', ' 큐', ' 큐', B'010101010'); --
UTF-8 Korean character and space

SELECT BIT_LENGTH(char_1), BIT_LENGTH(char_2),
BIT_LENGTH(varchar_1), BIT_LENGTH(bit_var_1) FROM bit_length_tbl;
```

```
bit_length(char_1)  bit_length(char_2)      bit_length(varchar_1)
bit_length(bit_var_1)
=============================================================================
===========
8                   40                       0
0
8                   40                       8
9
NULL                56                      24
9
8                   40                      32
9
```

CHAR_LENGTH, CHARACTER_LENGTH, LENGTHB, LENGTH

CHAR_LENGTH(*string*)

CHARACTER_LENGTH(*string*)

LENGTHB(*string*)

LENGTH(*string*)

CHAR_LENGTH, **CHARACTER_LENGTH**, **LENGTHB**, and **LENGTH** are used interchangeably. The number of characters is returned as an integer. For details on character set supported by CUBRID, see An Overview of Globalization.

Parameters:

string – Specifies the string whose length will be calculated according to the number of characters. If the character string is **NULL**, **NULL** is returned.

Return type:

INT

Note

- In versions lower than CUBRID 9.0, the multibyte string returns the number of bytes in the string. Therefore, the length of one character is calculated as 2- or 3-bytes according to the charset.
- The length of each space character that is included in a character string is one byte.
- The length of empty quotes (”) to represent a space character is 0. Note that in a **CHAR** (*n*) type, the length of a space character is *n*, and it is specified as 1 if n is omitted.

```
--character set is UTF-8 for Korean characters
SELECT LENGTH('');
```

```
char length('')
===============
              0
```

```
SELECT LENGTH('CUBRID');
```

```
char length('CUBRID')
======================
                     6
```

```
SELECT LENGTH('큐브리드');
```

```
char length('큐브리드')
======================
                     4
```

```
CREATE TABLE length_tbl (char_1 CHAR, char_2 CHAR(5), varchar_1
VARCHAR, varchar_2 VARCHAR);
INSERT INTO length_tbl VALUES('', '', '', ''); --Length of empty
string
INSERT INTO length_tbl VALUES('a', 'a', 'a', 'a'); --English
character
INSERT INTO length_tbl VALUES(NULL, '큐', '큐', '큐'); --Korean
character and NULL
INSERT INTO length_tbl VALUES(' ', ' 큐', ' 큐', ' 큐'); --Korean
character and space

SELECT LENGTH(char_1), LENGTH(char_2), LENGTH(varchar_1),
LENGTH(varchar_2) FROM length_tbl;
```

```
char_length(char_1) char_length(char_2) char_length(varchar_1)
char_length(varchar_2)
=======================================================================
===========
1                     5                     0             0
1                     5                     1             1
NULL                  5                     1             1
1                     5                     2             2
```

CHR

CHR(*number_operand [USING charset_name]*)

The **CHR** function returns a character that corresponds to the return value of the expression specified as an argument. When you input the code value within invalid ranges, it returns

an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

- **number_operand** – Specifies an expression that returns a numeric value.
- **charset_name** – Characterset name. It supports utf8 and iso88591.

Return type:

STRING

```
SELECT CHR(68) || CHR(68-2);
```

```
   chr(68)|| chr(68-2)
======================
  'DB'
```

If you want to get a multibyte character with the **CHR** function, input a number with the valid range of the charset.

```
SELECT CHR(14909886 USING utf8);
-- Below query's result is the same as above.
SET NAMES utf8;
SELECT CHR(14909886);
```

```
   chr(14909886 using utf8)
======================
  'ま'
```

If you want to get the hexadecimal string from a character, use **HEX** function.

```
SET NAMES utf8;
SELECT HEX('ま');
```

```
   hex(_utf8'ま')
======================
  'E381BE'
```

If you want to get the decimal string from a hexadecimal string, use **CONV** function.

```
SET NAMES utf8;
SELECT CONV('E381BE',16,10);
```

```
   conv(_utf8'E381BE', 16, 10)
======================
  '14909886'
```

CONCAT

CONCAT(*string1, string2 [,string3 [, ... [, stringN]...]]*)

The **CONCAT** function has at least one argument specified for it and returns a string as a result of concatenating all argument values. The number of parameters that can be specified is unlimited. Automatic type casting takes place if a non-string type is specified as the argument. If any of the arguments is specified as **NULL**, **NULL** is returned.

If you want to insert separators between strings specified as arguments for concatenation, use the `CONCAT_WS()` Function.

Parameters:

strings – character string

Return type:

STRING

```
SELECT CONCAT('CUBRID', '2008' , 'R3.0');
```

```
   concat('CUBRID', '2008', 'R3.0')
======================
'CUBRID2008R3.0'
```

```
--it returns null when null is specified for one of parameters
SELECT CONCAT('CUBRID', '2008' , 'R3.0', NULL);
```

```
   concat('CUBRID', '2008', 'R3.0', null)
=======================
  NULL
```

```
--it converts number types and then returns concatenated strings
SELECT CONCAT(2008, 3.0);
```

```
   concat(2008, 3.0)
=======================
  '20083.0'
```

CONCAT_WS

CONCAT_WS(*string1, string2 [,string3 [, ... [, stringN]...]]*)

The **CONCAT_WS** function has at least two arguments specified for it. The function uses the first argument value as the separator and returns the result.

Parameters:

strings – character string

Return type:

STRING

```
SELECT CONCAT_WS(' ', 'CUBRID', '2008' , 'R3.0');
```

```
concat_ws(' ', 'CUBRID', '2008', 'R3.0')
=======================
  'CUBRID 2008 R3.0'
```

```
--it returns strings even if null is specified for one of parameters
SELECT CONCAT_WS(' ', 'CUBRID', '2008', NULL, 'R3.0');
```

```
concat_ws(' ', 'CUBRID', '2008', null, 'R3.0')
=======================
  'CUBRID 2008 R3.0'
```

```
--it converts number types and then returns concatenated strings
with separator
SELECT CONCAT_WS(' ',2008, 3.0);
```

```
concat_ws(' ', 2008, 3.0)
=======================
  '2008 3.0'
```

ELT

ELT(*N*, *string1*, *string2*, *...*)

If *N* is 1, the **ELT** function returns *string1* and if *N* is 2, it returns *string2*. The return value is a **VARCHAR** type. You can add conditional expressions as needed.

The maximum byte length of the character string is 33,554,432 and if this length is exceeded, **NULL** will be returned.

If *N* is 0 or a negative number, an empty string will be returned. If *N* is greater than the number of this input character string, **NULL** will be returned as it is out of range. If *N* is a type that cannot be converted to an integer, an error will be returned.

Parameters:

- **N** – A position of a string to return among the list of strings
- **strings** – The list of strings

Return type:

STRING

```
SELECT ELT(3,'string1','string2','string3');
```

```
  elt(3, 'string1', 'string2', 'string3')
======================
  'string3'
```

```
SELECT ELT('3','1/1/1','23:00:00','2001-03-04');
```

```
  elt('3', '1/1/1', '23:00:00', '2001-03-04')
======================
  '2001-03-04'
```

```
SELECT ELT(-1, 'string1','string2','string3');
```

```
  elt(-1, 'string1','string2','string3')
======================
  NULL
```

```
SELECT ELT(4,'string1','string2','string3');
```

```
  elt(4, 'string1', 'string2', 'string3')
======================
  NULL
```

```
SELECT ELT(3.2,'string1','string2','string3');
```

```
  elt(3.2, 'string1', 'string2', 'string3')
======================
  'string3'
```

```
SELECT ELT('a','string1','string2','string3');
```

```
ERROR: Cannot coerce 'a' to type bigint.
```

FIELD

FIELD(*search_string, string1 [,string2 [, ... [, stringN]...]]*)

The **FIELD** function returns the location index value (position) of a string of *string1*, *string2*. The function returns 0 if it does not have a parameter value which is the same as *search_string*. It returns 0 if *search_string* is **NULL** because it cannot perform the comparison operation with the other arguments.

If all arguments specified for **FIELD** function are of string type, string comparison operation is performed: if all of them are of number type, numeric comparison operation is performed. If the type of one argument is different from that of another, a comparison operation is performed by casting each argument to the type of the first argument. If type casting fails during the comparison operation with each argument, the function considers the result of the comparison operation as **FALSE** and resumes the other operations.

Parameters:

- **search_string** – A string pattern to search
- **strings** – The list of strings to be searched

Return type:

INT

```
SELECT FIELD('abc', 'a', 'ab', 'abc', 'abcd', 'abcde');
```

```
   field('abc', 'a', 'ab', 'abc', 'abcd', 'abcde')
=================================================
                                               3
```

```
--it returns 0 when no same string is found in the list
SELECT FIELD('abc', 'a', 'ab', NULL);
```

```
   field('abc', 'a', 'ab', null)
================================
                               0
```

```
--it returns 0 when null is specified in the first parameter
SELECT FIELD(NULL, 'a', 'ab', NULL);
```

```
   field(null, 'a', 'ab', null)
================================
                              0
```

```
SELECT FIELD('123', 1, 12, 123.0, 1234, 12345);
```

```
   field('123', 1, 12, 123.0, 1234, 12345)
==========================================
                                        0
```

```
SELECT FIELD(123, 1, 12, '123.0', 1234, 12345);
```

```
   field(123, 1, 12, '123.0', 1234, 12345)
==============================================
                                          3
```

FIND_IN_SET

FIND_IN_SET(*str*, *strlist*)

The **FIND_IN_SET** function looks for the string *str* in the string list *strlist* and returns a position of *str* if it exists. A string list is a string composed of substrings separated by a comma (,). If *str* is not in *strlist* or *strlist* is an empty string, 0 is returned. If either argument is **NULL**, **NULL** is returned. This function does not work properly if *str* contains a comma (,).

Parameters:

- **str** – A string to be searched
- **strlist** – A group of strings separated by a comma

Return type:

INT

```
SELECT FIND_IN_SET('b','a,b,c,d');
```

```
2
```

FROM_BASE64

FROM_BASE64(*str*)

FROM_BASE64 function returns the the decoded result as binary string from the input string encoded as base-64 rule, which is used in **TO_BASE64** function. If the input value is **NULL**, it returns **NULL. When you input the invalid base-64 string, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default); NULL if this value is yes. See `TO_BASE64()` for more details on base-64 encoding rules.

Parameters:

str – Input string

Return type:

STRING

```
SELECT TO_BASE64('abcd'), FROM_BASE64(TO_BASE64('abcd'));
```

```
   to_base64('abcd') from_base64( to_base64('abcd'))
================================================
  'YWJjZA==' 'abcd'
```

See also

`TO_BASE64()`

INSERT

INSERT(*str*, *pos*, *len*, *string*)

The **INSERT** function inserts a partial character string as long as the length from the specific location of the input character string. The return value is a **VARCHAR** type. The maximum length of the character string is 33,554,432 and if this length is exceeded, **NULL** will be returned.

Parameters:

- **str** – Input character string

- **pos** – *str* location. Starts from 1. If *pos* is smaller than 1 or greater than the length of *string* + 1, the *string* will not be inserted and the *str* will be returned instead.
- **len** – Length of *string* to insert *pos* of *str*. If *len* exceeds the length of the partial character string, insert as many values as *string* in the *pos* of the *str* . If *len* is a negative number, *str* will be the end of the character string.
- **string** – Partial character string to insert to *str*

Return type:

STRING

```
SELECT INSERT('cubrid',2,2,'dbsql');
```

```
  insert('cubrid', 2, 2, 'dbsql')
======================
  'cdbsqlrid'
```

```
SELECT INSERT('cubrid',0,3,'db');
```

```
  insert('cubrid', 0, 3, 'db')
======================
  'cubrid'
```

```
SELECT INSERT('cubrid',-3,3,'db');
```

```
  insert('cubrid', -3, 3, 'db')
======================
  'cubrid'
```

```
SELECT INSERT('cubrid',3,100,'db');
```

```
  insert('cubrid', 3, 100, 'db')
=======================
  'cudb'
```

```
SELECT INSERT('cubrid',7,100,'db');
```

```
  insert('cubrid', 7, 100, 'db')
=======================
  'cubriddb'
```

```
SELECT INSERT('cubrid',3,-1,'db');
```

```
  insert('cubrid', 3, -1, 'db')
=======================
  'cudb'
```

INSTR

INSTR(*string*, *substring*[, *position*])

The **INSTR** function, similarly to the **POSITION**, returns the position of a *substring* within *string*; the position. For the **INSTR** function, you can specify the starting position of the search for *substring* to make it possible to search for duplicate *substring*.

Parameters:

- **string** – Specifies the input character string.
- **substring** – Specifies the character string whose position is to be returned.
- **position** – Optional. Represents the position of a *string* where the search begins in character unit. If omitted, the default value 1 is applied. The first position of the *string* is specified as 1. If the value is negative, the system counts backward from the end of the *string*.

Return type:

INT

Note

In the earlier versions of CUBRID 9.0, position value is returned in byte unit, not character unit. When a multibyte character set is used, the number of bytes representing one character is different; so the return value may not the same.

```
--character set is UTF-8 for Korean characters
--it returns position of the first 'b'
SELECT INSTR ('12345abcdeabcde','b');
```

```
   instr('12345abcdeabcde', 'b', 1)
===================================
                                  7
```

```
-- it returns position of the first '나' on UTF-8 Korean charset
SELECT INSTR ('12345가나다라마가나다라마', '나' );
```

```
   instr('12345가나다라마가나다라마', '나', 1)
=================================
                                7
```

```
-- it returns position of the second '나' on UTF-8 Korean charset
SELECT INSTR ('12345가나다라마가나다라마', '나', 11 );
```

```
   instr('12345가나다라마가나다라마', '나', 11)
=================================
                               12
```

```
--it returns position of the 'b' searching from the 8th position
SELECT INSTR ('12345abcdeabcde','b', 8);
```

```
   instr('12345abcdeabcde', 'b', 8)
===================================
                                  12
```

```
--it returns position of the 'b' searching backwardly from the end
SELECT INSTR ('12345abcdeabcde','b', -1);
```

```
   instr('12345abcdeabcde', 'b', -1)
====================================
                                  12
```

```
--it returns position of the 'b' searching backwardly from a
specified position
SELECT INSTR ('12345abcdeabcde','b', -8);
```

```
   instr('12345abcdeabcde', 'b', -8)
====================================
                                   7
```

LCASE, LOWER

LCASE(*string*)

LOWER(*string*)

The functions **LCASE** and **LOWER** are used interchangeably. They convert uppercase characters included in string to lowercase characters.

Parameters:

string – Specifies the string in which uppercase characters are to be converted to lowercase. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

```
SELECT LOWER('');
```

```
  lower('')
======================
  ''
```

```
SELECT LOWER(NULL);
```

```
  lower(null)
======================
  NULL
```

```
SELECT LOWER('Cubrid');
```

```
  lower('Cubrid')
======================
  'cubrid'
```

Note that the **LOWER** function may not work properly by specified collation. For example, when you try to change character Ă used in Romanian as lower character, this function works as follows by collation.

If collation is utf8_bin, this character is not changed.

```
SET NAMES utf8 COLLATE utf8_bin;
SELECT LOWER('Ă');

   lower(_utf8'Ă')
======================
  'Ă'
```

If collation is utf8_ro_RO, 'Ă' can be changed.

```
SET NAMES utf8 COLLATE utf8_ro_cs;
SELECT LOWER('Ă');

   lower(_utf8'Ă' COLLATE utf8_ro_cs)
======================
  'ă'
```

For supporting collations in CUBRID, see CUBRID Collation.

LEFT

LEFT(*string*, *length*)

The **LEFT** function returns a length number of characters from the leftmost *string*. If any of the arguments is **NULL**, **NULL** is returned. If a value greater than the *length* of the *string* or

a negative number is specified for a length, the entire string is returned. To extract a length number of characters from the rightmost string, use the `RIGHT()`.

Parameters:

- **string** – Input string
- **length** – The length of a string to be returned

Return type:

STRING

```
SELECT LEFT('CUBRID', 3);
```

```
 left('CUBRID', 3)
======================
  'CUB'
```

```
SELECT LEFT('CUBRID', 10);
```

```
  left('CUBRID', 10)
======================
  'CUBRID'
```

LOCATE

LOCATE(*substring*, *string*[, *position*])

The **LOCATE** function returns the location index value of a *substring* within a character string. The third argument *position* can be omitted. If this argument is specified, the function searches for *substring* from the given position and returns the location index value of the first occurrence. If the *substring* cannot be found within the string, 0 is returned. The **LOCATE** function behaves like the `POSITION()`, but you cannot use **LOCATE** for bit strings.

Parameters:

- **substring** – A string pattern to search
- **string** – A whole string to be searched

- **position** – Starting position of a whole string to be searched

Return type:

INT

```
--it returns 1 when substring is empty space
SELECT LOCATE ('', '12345abcdeabcde');
```

```
 locate('', '12345abcdeabcde')
================================
                              1
```

```
--it returns position of the first 'abc'
SELECT LOCATE ('abc', '12345abcdeabcde');
```

```
 locate('abc', '12345abcdeabcde')
==================================
                               6
```

```
--it returns position of the second 'abc'
SELECT LOCATE ('abc', '12345abcdeabcde', 8);
```

```
 locate('abc', '12345abcdeabcde', 8)
=====================================
                                 11
```

```
--it returns 0 when no substring found in the string
SELECT LOCATE ('ABC', '12345abcdeabcde');
```

```
 locate('ABC', '12345abcdeabcde')
==================================
                               0
```

LPAD

LPAD(*char1*, *n*[, *char2*])

The **LPAD** function pads the left side of a string until the string length reaches the specified value.

Parameters:

- **char1** – Specifies the string to pad characters to. If *n* is smaller than the length of *char1*, padding is not performed, and *char1* is truncated to length n and then returned. If the value is **NULL**, **NULL** is returned.
- **n** – Specifies the total length of *char1* in bytes. If the value is **NULL**, **NULL** is returned.
- **char2** – Specifies the string to pad to the left until the length of *char1* reaches *n*. If it is not specified, empty characters (' ') are used as a default. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

Note

In versions lower than CUBRID 9.0, a single character is processed as 2 or 3 bytes in a multibyte character set environment. If *n* is truncated up to the first byte representing a character according to a value of *char1*, the last byte is removed and a space character (1 byte) is added to the left because the last character cannot be represented normally. When the value is **NULL**, **NULL** is returned as its result.

```
--character set is UTF-8 for Korean characters

--it returns only 3 characters if not enough length is specified
SELECT LPAD ('CUBRID', 3, '?');
```

```
  lpad('CUBRID', 3, '?')
======================
  'CUB'
```

```
SELECT LPAD ('큐브리드', 3, '?');
```

```
 lpad('큐브리드', 3, '?')
======================
  '큐브리'
```

```
--padding spaces on the left till char_length is 10
SELECT LPAD ('CUBRID', 10);
```

```
 lpad('CUBRID', 10)
======================
  '    CUBRID'
```

```
--padding specific characters on the left till char_length is 10
SELECT LPAD ('CUBRID', 10, '?');
```

```
 lpad('CUBRID', 10, '?')
======================
  '????CUBRID'
```

```
--padding specific characters on the left till char_length is 10
SELECT LPAD ('큐브리드', 10, '?');
```

```
 lpad('큐브리드', 10, '?')
======================
  '??????큐브리드'
```

```
--padding 4 characters on the left
SELECT LPAD ('큐브리드', LENGTH('큐브리드')+4, '?');
```

```
 lpad('큐브리드',  char_length('큐브리드')+4, '?')
======================
  '????큐브리드'
```

LTRIM

LTRIM(*string*[, *trim_string*])

The **LTRIM** function removes all specified characters from the left-hand side of a string.

Parameters:

- **string** – Enters a string or string-type column to trim. If this value is **NULL**, **NULL** is returned.
- **trim_string** – You can specify a specific string to be removed in the left side of *string*. If it is not specified, empty characters (' ') is automatically specified so that the empty characters in the left side are removed.

Return type:

STRING

```
--trimming spaces on the left
SELECT LTRIM ('     Olympic     ');
```

```
  ltrim('     Olympic     ')
=======================
  'Olympic     '
```

```
--If NULL is specified, it returns NULL
SELECT LTRIM ('iiiiiOlympiciiiii', NULL);
```

```
  ltrim('iiiiiOlympiciiiii', null)
=======================
  NULL
```

```
-- trimming specific strings on the left
SELECT LTRIM ('iiiiiOlympiciiiii', 'i');
```

```
  ltrim('iiiiiOlympiciiiii', 'i')
=======================
  'Olympiciiiii'
```

MID

MID(*string*, *position*, *substring_length*)

The **MID** function extracts a string with the length of *substring_length* from a *position* within the *string* and then returns it. If a negative number is specified as a *position* value, the *position* is calculated in a reverse direction from the end of the *string*. **substring_length** cannot be omitted. If a negative value is specified, the function considers this as 0 and returns an empty string.

The **MID** function is working like the `SUBSTR()`, but there are differences in that it cannot be used for bit strings, that the *substring_length* argument must be specified, and that it returns an empty string if a negative number is specified for *substring_length*.

Parameters:

- **string** – Specifies an input character string. If this value is **NULL**, **NULL** is returned.
- **position** – Specifies the starting position from which the string is to be extracted. The position of the first character is 1. It is considered to be 1 even if it is specified as 0. If the input value is **NULL**, **NULL** is returned.
- **substring_length** – Specifies the length of the string to be extracted. If 0 or a negative number is specified, an empty string is returned; if **NULL** is specified, **NULL** is returned.

Return type:

STRING

```
CREATE TABLE mid_tbl(a VARCHAR);
INSERT INTO mid_tbl VALUES('12345abcdeabcde');

--it returns empty string when substring_length is 0
SELECT MID(a, 6, 0), SUBSTR(a, 6, 0), SUBSTRING(a, 6, 0) FROM
mid_tbl;
```

```
  mid(a, 6, 0)          substr(a, 6, 0)       substring(a from 6 for
0)
================================================================
  ''                    ''                    ''
```

```
--it returns 4-length substrings counting from the 6th position
SELECT MID(a, 6, 4), SUBSTR(a, 6, 4), SUBSTRING(a, 6, 4) FROM
mid_tbl;
```

```
  mid(a, 6, 4)          substr(a, 6, 4)       substring(a from 6 for
4)
=======================================================================
  'abcd'                'abcd'                'abcd'
```

```
--it returns an empty string when substring_length < 0
SELECT MID(a, 6, -4), SUBSTR(a, 6, -4), SUBSTRING(a, 6, -4) FROM
mid_tbl;
```

```
  mid(a, 6, -4)         substr(a, 6, -4)      substring(a from 6 for
-4)
=======================================================================
  ''                    NULL                  'abcdeabcde'
```

```
--it returns 4-length substrings at 6th position counting backward
from the end
SELECT MID(a, -6, 4), SUBSTR(a, -6, 4), SUBSTRING(a, -6, 4) FROM
mid_tbl;
```

```
  mid(a, -6, 4)         substr(a, -6, 4)      substring(a from -6
for 4)
=======================================================================
  'eabc'                'eabc'                '1234'
```

OCTET_LENGTH

OCTET_LENGTH(*string*)

The **OCTET_LENGTH** function returns the length (byte) of a character string or bit string as an integer. Therefore, it returns 1 (byte) if the length of the bit string is 8 bits, but 2 (bytes) if the length is 9 bits.

Parameters:

string – Specifies the character or bit string whose length is to be returned in bytes. If the value is **NULL**, **NULL** is returned.

Return type:

INT

```
--character set is UTF-8 for Korean characters

SELECT OCTET_LENGTH('');
```

```
 octet_length('')
==================
                 0
```

```
SELECT OCTET_LENGTH('CUBRID');
```

```
 octet_length('CUBRID')
==================
                 6
```

```
SELECT OCTET_LENGTH('큐브리드');
```

```
 octet_length('큐브리드')
==================
                12
```

```
SELECT OCTET_LENGTH(B'010101010');
```

```
 octet_length(B'010101010')
==================
                 2
```

```
CREATE TABLE octet_length_tbl (char_1 CHAR, char_2 CHAR(5),
varchar_1 VARCHAR, bit_var_1 BIT VARYING);
INSERT INTO octet_length_tbl VALUES('', '', '', B''); --Length of
```

```
empty string
INSERT INTO octet_length_tbl VALUES('a', 'a', 'a', B'010101010'); --
English character
INSERT INTO octet_length_tbl VALUES(NULL, '큐', '큐', B'010101010');
--Korean character and NULL
INSERT INTO octet_length_tbl VALUES(' ', ' 큐', ' 큐', B'010101010');
--Korean character and space

SELECT OCTET_LENGTH(char_1), OCTET_LENGTH(char_2),
OCTET_LENGTH(varchar_1), OCTET_LENGTH(bit_var_1) FROM
octet_length_tbl;
```

```
octet_length(char_1) octet_length(char_2) octet_length(varchar_1)
octet_length(bit_var_1)
================================================================================
===========
1                    5
0                     0
1                    5
1                     2
NULL                 7
3                     2
1                    7
4                     2
```

POSITION

POSITION(*substring IN string*)

The **POSITION** function returns the position of a character string corresponding to *substring* within a character string corresponding to *string*.

An expression that returns a character string or a bit string can be specified as an argument of this function. The return value is an integer greater than or equal to 0. This function returns the position value in character unit for a character string, and in bits for a bit string.

The **POSITION** function is occasionally used in combination with other functions. For example, if you want to extract a certain string from another string, you can use the result of the **POSITION** function as an input to the **SUBSTRING** function.

Note

The location is returned in the unit of byte, not the character, in version lower than CUBRID 9.0. The multibyte charset uses different numbers of bytes to express one character, so the result value may differ.

Parameters:

substring – Specifies the character string whose position is to be returned. If the value is an empty character, 1 is returned. If the value is **NULL**, **NULL** is returned.

Return type:

INT

```
--character set is UTF-8 for Korean characters

--it returns 1 when substring is empty space
SELECT POSITION ('' IN '12345abcdeabcde');
```

```
  position('' in '12345abcdeabcde')
===================================
                                  1
```

```
--it returns position of the first 'b'
SELECT POSITION ('b' IN '12345abcdeabcde');
```

```
  position('b' in '12345abcdeabcde')
====================================
                                   7
```

```
-- it returns position of the first '나'
SELECT POSITION ('나' IN '12345가나다라마가나다라마');
```

```
  position('나' in '12345가나다라마가나다라마')
==========================================
                                         7
```

```
--it returns 0 when no substring found in the string
SELECT POSITION ('f' IN '12345abcdeabcde');
```

```
  position('f' in '12345abcdeabcde')
====================================
                                   0
```

```
SELECT POSITION (B'1' IN B'000011110000');
```

```
  position(B'1' in B'000011110000')
===================================
                                  5
```

REPEAT

REPEAT(*string*, *count*)

The **REPEAT** function returns the character string with a length equal to the number of repeated input character strings. The return value is a **VARCHAR** type. The maximum length of the character string is 33,554,432 and if it this length is exceeded, **NULL** will be returned. If one of the parameters is **NULL**, **NULL** will be returned.

Parameters:

- **substring** – Character string
- **count** – Repeat count. If you enter 0 or a negative number, an empty string will be returned and if you enter a non-numeric data type, an error will be returned.

Return type:

STRING

```
SELECT REPEAT('cubrid',3);
```

```
   repeat('cubrid', 3)
======================
  'cubridcubridcubrid'
```

```
SELECT REPEAT('cubrid',32000000);
```

```
   repeat('cubrid', 32000000)
======================
  NULL
```

```
SELECT REPEAT('cubrid',-1);
```

```
   repeat('cubrid', -1)
======================
  ''
```

```
SELECT REPEAT('cubrid','a');
```

```
ERROR: Cannot coerce 'a' to type integer.
```

REPLACE

REPLACE(*string*, *search_string*[, *replacement_string*])

The **REPLACE** function searches for a character string, *search_string*, within a given character string, *string*, and replaces it with a character string, *replacement_string*. If the string to be replaced, *replacement_string* is omitted, all *search_strings* retrieved from *string* are removed. If **NULL** is specified as an argument, **NULL** is returned.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.
- **search_string** – Specifies the string to be searched. If the value is **NULL**, **NULL** is returned
- **replacement_string** – Specifies the string to replace the *search_string*. If this value is omitted, *string* is returned with the *search_string* removed. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

```
--it returns NULL when an argument is specified with NULL value
SELECT REPLACE('12345abcdeabcde','abcde',NULL);
```

```
replace('12345abcdeabcde', 'abcde', null)
======================
  NULL
```

```
--not only the first substring but all substrings into 'ABCDE' are
replaced
SELECT REPLACE('12345abcdeabcde','abcde','ABCDE');
```

```
replace('12345abcdeabcde', 'abcde', 'ABCDE')
======================
  '12345ABCDEABCDE'
```

```
--it removes all of substrings when replace_string is omitted
SELECT REPLACE('12345abcdeabcde','abcde');
```

```
replace('12345abcdeabcde', 'abcde')
======================
  '12345'
```

The following shows how to print out the newline as “\n”.

```
-- no_backslash_escapes=yes (default)

CREATE TABLE tbl (cmt_no INT PRIMARY KEY, cmt VARCHAR(1024));
INSERT INTO tbl VALUES (1234,
'This is a test for

 new line.');

SELECT REPLACE(cmt, CHR(10), '\n')
FROM tbl
WHERE cmt_no=1234;
```

```
This is a test for\n\n new line.
```

REVERSE

REVERSE(*string*)

The **REVERSE** function returns *string* converted in the reverse order.

Parameters:

string – Specifies an input character string. If the value is an empty string, empty value is returned. If the value is NULL, NULL is returned.

Return type:

STRING

```
SELECT REVERSE('CUBRID');
```

```
 reverse('CUBRID')
======================
  'DIRBUC'
```

RIGHT

RIGHT(*string*, *length*)

The **RIGHT** function returns a *length* number of characters from the rightmost *string*. If any of the arguments is **NULL**, **NULL** is returned. If a value greater than the length of the *string* or a negative number is specified for a *length*, the entire string is returned. To extract a length number of characters from the leftmost string, use the `LEFT()`.

Parameters:

- **string** – Input string
- **length** – The length of a string to be returned

Return type:

STRING

```
SELECT RIGHT('CUBRID', 3);
```

```
 right('CUBRID', 3)
=====================
  'RID'
```

```
SELECT RIGHT ('CUBRID', 10);
```

```
 right('CUBRID', 10)
======================
  'CUBRID'
```

RPAD

RPAD(*char1*, *n*[, *char2*])

The **RPAD** function pads the right side of a string until the string length reaches the specified value.

Parameters:

- **char1** – Specifies the string to pad characters to. If *n* is smaller than the length of *char1*, padding is not performed, and *char1* is truncated to length *n* and then returned. If the value is **NULL**, **NULL** is specified.
- **n** – Specifies the total length of *char1*. If the value is **NULL**, **NULL** is specified.
- **char2** – Specifies the string to pad to the right until the length of *char1* reaches *n*. If it is not specified, empty characters (' ') are used as a default. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

Note

In versions lower than CUBRID 9.0, a single character is processed as 2 or 3 bytes in a multibyte character set environment. If *n* is truncated up to the first byte representing a character according to a value of *char1*, the last byte is removed and a space character (1 byte) is added to the right because the last character cannot be represented normally. When the value is **NULL**, **NULL** is returned as its result.

```
--character set is UTF-8 for Korean characters

--it returns only 3 characters if not enough length is specified
SELECT RPAD ('CUBRID', 3, '?');
```

```
 rpad('CUBRID', 3, '?')
=======================
  'CUB'
```

```
--on multi-byte charset, it returns the first character only with a
right-padded space
SELECT RPAD ('큐브리드', 3, '?');
```

```
 rpad('큐브리드', 3, '?')
=======================
  '큐브리'
```

```
--padding spaces on the right till char_length is 10
SELECT RPAD ('CUBRID', 10);
```

```
 rpad('CUBRID', 10)
=======================
  'CUBRID    '
```

```
--padding specific characters on the right till char_length is 10
SELECT RPAD ('CUBRID', 10, '?');
```

```
 rpad('CUBRID', 10, '?')
=======================
  'CUBRID????'
```

```
--padding specific characters on the right till char_length is 10
SELECT RPAD ('큐브리드', 10, '?');
```

```
 rpad('큐브리드', 10, '?')
======================
  '큐브리드??????'
```

```
--padding 4 characters on the right
SELECT RPAD ('큐브리드', LENGTH('큐브리드')+4, '?');
```

```
 rpad('',  char_length('')+4, '?')
======================
  '큐브리드????'
```

RTRIM

RTRIM(*string*[, *trim_string*])

The **RTRIM** function removes specified characters from the right-hand side of a string.

Parameters:

- **string** – Enters a string or string-type column to trim. If this value is **NULL**, **NULL** is returned.
- **trim_string** – You can specify a specific string to be removed in the right side of *string*. If it is not specified, empty characters (' ') is automatically specified so that the empty characters in the right side are removed.

Return type:

STRING

```
SELECT RTRIM ('     Olympic     ');
```

```
 rtrim('     Olympic     ')
======================
  '     Olympic'
```

```
--If NULL is specified, it returns NULL
SELECT RTRIM ('iiiiiOlympiciiiii', NULL);
```

```
 rtrim('iiiiiOlympiciiiii', null)
======================
  NULL
```

```
-- trimming specific strings on the right
SELECT RTRIM ('iiiiiOlympiciiiii', 'i');
```

```
 rtrim('iiiiiOlympiciiiii', 'i')
======================
  'iiiiiOlympic'
```

SPACE

SPACE(*N*)

The **SPACE** function returns as many empty strings as the number specified. The return value is a **VARCHAR** type.

Parameters:

N – Space count. It cannot be greater than the value specified in the system parameter, **string_max_size_bytes** (default 1048576). If it exceeds the specified value, **NULL** will be returned. The maximum value is 33,554,432; if this length is exceeded, **NULL** will be returned. If you enter 0 or a negative number, an empty string will be returned; if you enter a type that cannot be converted to a numeric value, an error will be returned.

Return type:

STRING

```
SELECT SPACE(8);
```

```
   space(8)
======================
  '        '
```

```
SELECT LENGTH(space(1048576));
```

```
   char_length( space(1048576))
================================
                         1048576
```

```
SELECT LENGTH(space(1048577));
```

```
   char_length( space(1048577))
================================
                           NULL
```

```
-- string_max_size_bytes=33554432
SELECT LENGTH(space('33554432'));
```

```
   char_length( space('33554432'))
==================================
                          33554432
```

```
SELECT SPACE('aaa');
```

```
ERROR: Cannot coerce 'aaa' to type bigint.
```

STRCMP

STRCMP(*string1*, *string2*)

The **STRCMP** function compares two strings, *string1* and *string2*, and returns 0 if they are identical, 1 if *string1* is greater, or -1 if *string1* is smaller. If any of the parameters is **NULL**, **NULL** is returned.

Parameters:

- **string1** – A string to be compared
- **string2** – A string to be compared

Return type:

INT

```
SELECT STRCMP('abc', 'abc');
```

```
0
```

```
SELECT STRCMP ('acc', 'abc');
```

```
1
```

Note

Until the previous version of 9.0, STRCMP did not distinguish an uppercase and a lowercase. From 9.0, it compares the strings case-sensitively. To make STRCMP case-insensitive, you should use case-insensitive collation(e.g.: utf8_en_ci).

```
-- In previous version of 9.0 STRCMP works case-insensitively
SELECT STRCMP ('ABC','abc');
```

```
0
```

```
-- From 9.0 version, STRCMP distinguish the uppercase and the
lowercase when the collation is case-sensitive.
-- charset is en_US.iso88591

SELECT STRCMP ('ABC','abc');
```

```
-1
```

```
-- If the collation is case-insensitive, it does not distinguish
the uppercase and the lowercase.
-- charset is en_US.iso88591

SELECT STRCMP ('ABC' COLLATE utf8_en_ci ,'abc' COLLATE utf8_en_ci);
```

```
0
```

SUBSTR

SUBSTR(*string*, *position*[, *substring_length*])

The **SUBSTR** function extracts a character string with the length of *substring_length* from a position, *position*, within character string, *string*, and then returns it.

Note

In the previous versions of CUBRID 9.0, the starting position and string length are calculated in byte unit, not in character unit; therefore, in a multibyte character set, you must specify the parameter in consideration of the number of bytes representing a single character.

Parameters:

- **string** – Specifies the input character string. If the input value is **NULL**, **NULL** is returned.
- **position** – Specifies the position from where the string is to be extracted in bytes. Even though the position of the first character is specified as 1 or a negative number, it is considered as 1. If a value greater than the string length or **NULL** is specified, **NULL** is returned.
- **substring_length** – Specifies the length of the string to be extracted in bytes. If this argument is omitted, character strings between the given position, *position*, and the end of them are extracted. **NULL** cannot be specified as an argument value of this function. If 0 is specified, an empty string is returned; if a negative value is specified, **NULL** is returned.

Return type:

STRING

```
--character set is UTF-8 for Korean characters

--it returns empty string when substring_length is 0
SELECT SUBSTR('12345abcdeabcde',6, 0);
```

```
 substr('12345abcdeabcde', 6, 0)
=================================
  ''
```

```
--it returns 4-length substrings counting from the position
SELECT SUBSTR('12345abcdeabcde', 6, 4), SUBSTR('12345abcdeabcde',
-6, 4);
```

```
 substr('12345abcdeabcde', 6, 4)   substr('12345abcdeabcde', -6, 4)
=========================================================
  'abcd'                 'eabc'
```

```
--it returns substrings counting from the position to the end
SELECT SUBSTR('12345abcdeabcde', 6), SUBSTR('12345abcdeabcde', -6);
```

```
 substr('12345abcdeabcde', 6)   substr('12345abcdeabcde', -6)
==============================================
  'abcdeabcde'           'eabcde'
```

```
-- it returns 4-length substrings counting from 11th position
SELECT SUBSTR ('12345가나다라마가나다라마', 11 , 4);
```

```
 substr('12345가나다라마가나다라마', 11 , 4)
======================
  '가나다라'
```

SUBSTRING

SUBSTRING (string, position [, substring_length]),

SUBSTRING(*string FROM position [FOR substring_length]*)

The **SUBSTRING** function, operating like **SUBSTR**, extracts a character string having the length of *substring_length* from a position, *position*, within character string, *string*, and returns it. If a negative number is specified as the *position* value, the **SUBSTRING** function calculates the position from the beginning of the string. And **SUBSTR** function calculates the position from the end of the string. If a negative number is specified as the *substring_length* value, the **SUBSTRING** function handles the argument is omitted, but the **SUBSTR** function returns **NULL**.

Parameters:

- **string** – Specifies the input character string. If the input value is **NULL**, **NULL** is returned.
- **position** – Specifies the position from where the string is to be extracted. If the position of the first character is specified as 0 or a negative number, it is considered as 1. If a value greater than the string length is specified, an empty string is returned. If **NULL**, **NULL** is returned.
- **substring_length** – Specifies the length of the string to be extracted. If this argument is omitted, character strings between the given position, *position*, and the end of them

are extracted. **NULL** cannot be specified as an argument value of this function. If 0 is specified, an empty string is returned; if a negative value is specified, **NULL** is returned.

Return type:

STRING

```
SELECT SUBSTRING('12345abcdeabcde', -6 ,4),
SUBSTR('12345abcdeabcde', -6 ,4);
```

```
  substring('12345abcdeabcde' from -6 for 4)
substr('12345abcdeabcde', -6, 4)
============================================
  '1234'                 'eabc'
```

```
SELECT SUBSTRING('12345abcdeabcde', 16), SUBSTR('12345abcdeabcde',
16);
```

```
  substring('12345abcdeabcde' from 16)   substr('12345abcdeabcde',
16)
============================================
  ''                     NULL
```

```
SELECT SUBSTRING('12345abcdeabcde', 6, -4),
SUBSTR('12345abcdeabcde', 6, -4);
```

```
  substring('12345abcdeabcde' from 6 for -4)
substr('12345abcdeabcde', 6, -4)
============================================
  'abcdeabcde'           NULL
```

SUBSTRING_INDEX

SUBSTRING_INDEX(*string*, *delim*, *count*)

The **SUBSTRING_INDEX** function counts the separators included in the partial character string and will return the partial character string before the *count*-th separator. The return value is a **VARCHAR** type.

Parameters:

- **string** – Input character string. The maximum length is 33,554,432 and if this length is exceeded, **NULL** will be returned.
- **delim** – Delimiter. It is case-sensitive.
- **count** – Delimiter occurrence count. If you enter a positive number, it counts the character string from the left and if you enter a negative number, it counts it from the right. If it is 0, an empty string will be returned. If the type cannot be converted, an error will be returned.

Return type:

STRING

```
SELECT SUBSTRING_INDEX('www.cubrid.org','.','2');
```

```
  substring_index('www.cubrid.org', '.', '2')
======================
  'www.cubrid'
```

```
SELECT SUBSTRING_INDEX('www.cubrid.org','.','2.3');
```

```
  substring_index('www.cubrid.org', '.', '2.3')
======================
  'www.cubrid'
```

```
SELECT SUBSTRING_INDEX('www.cubrid.org',':','2.3');
```

```
  substring_index('www.cubrid.org', ':', '2.3')
======================
  'www.cubrid.org'
```

```
SELECT SUBSTRING_INDEX('www.cubrid.org','cubrid',1);
```

```
  substring_index('www.cubrid.org', 'cubrid', 1)
========================
  'www.'
```

```
SELECT SUBSTRING_INDEX('www.cubrid.org','.',100);
```

```
  substring_index('www.cubrid.org', '.', 100)
========================
  'www.cubrid.org'
```

TO_BASE64

TO_BASE64(*str*)

Returns the result as the transformed base-64 string. If the input argument is not a string, it is changed into a string before it is transformed. If the input argument is **NULL**, it returns **NULL**. The base-64 encoded string can be decoded with `FROM_BASE64()` function.

Parameters:

str – Input string

Return type:

STRING

```
SELECT TO_BASE64('abcd'), FROM_BASE64(TO_BASE64('abcd'));
```

```
   to_base64('abcd') from_base64( to_base64('abcd'))
============================================
  'YWJjZA==' 'abcd'
```

The following is rules for `TO_BASE64()` function and `FROM_BASE64()`.

- The encoded character for the alphabet value 62 is '+'.
- The encoded character for the alphabet value 63 is '/'.
- The encoded result consists of character groups, and each group has 4 characters which can be printed out. The 3 bytes of the input data are encoded into 4 bytes. If the last group are not filled with 4 characters, '=' character is padded into that group and 4 characters are made.

- To divide the long output into the several lines, a newline is added into each 76 encoded output characters.
- Decoding process indicates newline, carriage return, tab, and space and ignore them.

See also

`FROM_BASE64()`

TRANSLATE

TRANSLATE(*string*, *from_substring*, *to_substring*)

The **TRANSLATE** function replaces a character into the character specified in *to_substring* if the character exists in the specified *string*. Correspondence relationship is determined based on the order of characters specified in *from_substring* and *to_substring*. Any characters in *from_substring* that do not have one on one relationship to *to_substring* are all removed. This function is working like the `REPLACE()` but the argument of *to_substring* cannot be omitted in this function.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.
- **from_substring** – Specifies the string to be retrieved. If the value is **NULL**, **NULL** is returned.
- **to_substring** – Specifies the character string in the *from_substring* to be replaced. It cannot be omitted. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

```
--it returns NULL when an argument is specified with NULL value
SELECT TRANSLATE('12345abcdeabcde','abcde', NULL);
```

```
  translate('12345abcdeabcde', 'abcde', null)
======================
  NULL
```

```
--it translates 'a','b','c','d','e' into '1', '2', '3', '4', '5'
respectively
SELECT TRANSLATE('12345abcdeabcde', 'abcde', '12345');
```

```
  translate('12345abcdeabcde', 'abcde', '12345')
======================
  '123451234512345'
```

```
--it translates 'a','b','c' into '1', '2', '3' respectively and
removes 'd's and 'e's
SELECT TRANSLATE('12345abcdeabcde','abcde', '123');
```

```
  translate('12345abcdeabcde', 'abcde', '123')
======================
  '12345123123'
```

```
--it removes 'a's,'b's,'c's,'d's, and 'e's in the string
SELECT TRANSLATE('12345abcdeabcde','abcde', '');
```

```
  translate('12345abcdeabcde', 'abcde', '')
======================
  '12345'
```

```
--it only translates 'a','b','c' into '3', '4', '5' respectively
SELECT TRANSLATE('12345abcdeabcde','ABabc', '12345');
```

```
  translate('12345abcdeabcde', 'ABabc', '12345')
======================
  '12345345de345de'
```

TRIM

TRIM(*[[LEADING | TRAILING | BOTH] [trim_string] FROM] string*)

The **TRIM** function removes specific characters which are located before and after the string.

Parameters:

- **trim_string** – Specifies a specific string to be removed that is in front of or at the back of the target string. If it is not specified, an empty character (' ') is automatically specified so that spaces in front of or at the back of the target string are removed.
- **string** – Enters a string or string-type column to trim. If this value is **NULL**, **NULL** is returned.

Return type:

STRING

- **[LEADING|TRAILING|BOTH]** : You can specify an option to trim a specified string that is in a certain position of the target string. If it is **LEADING**, trimming is performed in front of a character string if it is **TRAILING**, trimming is performed at the back of a character string if it is **BOTH**, trimming is performed in front and at the back of a character string. If the option is not specified, **BOTH** is specified by default.
- The character string of *trim_string* and *string* should have the same character set.

```
--trimming NULL returns NULL
SELECT TRIM (NULL);
```

```
 trim(both  from null)
======================
  NULL
```

```
--trimming spaces on both leading and trailing parts
SELECT TRIM ('     Olympic     ');
```

```
 trim(both  from '     Olympic     ')
======================
  'Olympic'
```

```
--trimming specific strings on both leading and trailing parts
SELECT TRIM ('i' FROM 'iiiiiOlympiciiiii');
```

```
 trim(both 'i' from 'iiiiiOlympiciiiii')
=======================
  'Olympic'
```

```
--trimming specific strings on the leading part
SELECT TRIM (LEADING 'i' FROM 'iiiiiOlympiciiiii');
```

```
 trim(leading 'i' from 'iiiiiOlympiciiiii')
=======================
  'Olympiciiiii'
```

```
--trimming specific strings on the trailing part
SELECT TRIM (TRAILING 'i' FROM 'iiiiiOlympiciiiii');
```

```
 trim(trailing 'i' from 'iiiiiOlympiciiiii')
=======================
  'iiiiiOlympic'
```

UCASE, UPPER

UCASE(*string*)

UPPER(*string*)

The function **UCASE** or **UPPER** converts lowercase characters that are included in a character string to uppercase characters.

Parameters:

string – Specifies the string in which lowercase characters are to be converted to uppercase. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

```
SELECT UPPER('');
```

```
 upper('')
======================
  ''
```

```
SELECT UPPER(NULL);
```

```
 upper(null)
======================
  NULL
```

```
SELECT UPPER('Cubrid');
```

```
 upper('Cubrid')
======================
  'CUBRID'
```

Note that the **UPPER** function may not work properly by specified collation. For example, when you try to change character 'ă' used in Romanian as upper character, this function works as follows by collation.

If collation is utf8_bin, it is not changed.

```
SET NAMES utf8 COLLATE utf8_bin;
SELECT UPPER('ă');

   upper(_utf8'ă')
======================
  'ă'
```

If collation is utf8_ro_RO, this can be changed.

```
SET NAMES utf8 COLLATE utf8_ro_cs;
SELECT UPPER('ă');

   upper(_utf8'ă' COLLATE utf8_ro_cs)
======================
  'Ă'
```

Regarding collations which CUBRID supports, see CUBRID Collation.

Regular Expressions Functions and Operators

A regular expression is a powerful way to specify a pattern for a complex search. The functions and operators described in this section performs regular expression matching on a string.

Contents

Regular Expressions Functions and Operators 425

- Regular Expressions Pattern Syntax 426
  - Special Pattern Characters 427
  - Quantifiers 430
  - Groups 432
  - Assertions 433
  - Alternatives 435
  - Character classes 436
    - Individual characters 436
    - Ranges 437
    - POSIX-based character classes 437
    - Unicode Character Class 439
- REGEXP, RLIKE 451
- REGEXP_COUNT 452
- REGEXP_INSTR 454
- REGEXP_LIKE 456

- REGEXP_REPLACE 458
- REGEXP_SUBSTR 460

Regular Expressions Pattern Syntax

To implement regular expressions, CUBRID uses the Google RE2 library and the C++ <regex> standard library. You can select the regular expression library to use by setting the **regexp_engine** system parameter, and the Google RE2 library is used as a default.

The following sub-sections describes supported regular expression grammars with several examples.

Note

Compatibility Considerations

- In the prior version of CUBRID 11, CUBRID used Henry Spencer's implementation of regular expressions.
- From the CUBRID 11.0, CUBRID removes the Henry Spencer library and uses C++ <regex> standard library to support regular expression functions and operators.
- From the CUBRID 11.2, CUBRID has added the Google RE2 library. Either C++ <regex> or Google RE2 can be used by setting the system parameter.

1. The Henry Spencer's implementation of regular expressions operates in byte-wise fashion. So the REGEXP and RLIKE were not multibyte safe, they only worked as ASCII encoding without considering the collation of operands.

2. The Spencer library supports the POSIX *collating sequence* expressions (*[.character.]*). But it does not support anymore. Also, *character equivalents* (*[=word=]*) does not support. CUBRID occurs an error when these collating element syntax is given.

3. The Spencer library matches line-terminator characters for the dot operator (.). But it does not.

4. The word-beginning and word-end boundary ([[:<:]] and [[:>:]]) doesn't support anymore. Instead, the word boundary notation (\b) can be used.

Warning

C++ <regex> library caveats

There is an issue with the C++ <regex> library that can cause excessive recursive calls when the input string is long or the regular expression pattern is complex. Therefore, it is recommended to use Google RE2 instead of C++ <regex>. C++ <regex> is reserved for backwards compatibility and has been deprecated.

Note

Multibyte Character Comparision Considerations

C++ <regex> performs multibyte comparision by C++ <locale> standard dependent on system-supplied locales. Therefore, system locale should be installed on your system for locale-sensitive functions.

Special Pattern Characters

Special pattern characters are characters (or sequences of characters) that have a special meaning when they appear in a regular expression pattern, either to represent a character that is difficult to express in a string, or to represent a category of characters. Each of these special pattern characters is matched in a string against a single character (unless a quantifier specifies otherwise).

Characters	Description
.	Any character except line terminators (LF, CR, LS, PS).
\t	A horizontal tab character (same as \u0009).
\n	A newline (line feed) character (same as \u000A).

Characters	Description
\v	A vertical tab character (same as \u000B).
\f	A form feed character (same as \u000C).
\r	A carriage return character (same as \u000D)
\c*letter*	A control code character whose code unit value is the same as the remainder of dividing the code unit value of *letter* by 32.
\x*hh*	A a character whose code unit value has an hex value equivalent to the two hex digits *hh*.
\u*hhhh*	A character whose code unit value has an hex value equivalent to the four hex digits *hhhh*. *Only supported in C++ <regex>*
\x{*digits*}	A hex character code corresponds to the value of *digits Only supported in Google RE2*
\0	A null character (same as \u0000). *Only supported in C++ <regex>*
num	The result of the submatch whose opening parenthesis is the *num*-th. See groups below for more info.
\d	A decimal digit character (same as [[:digit:]]).
\D	Any character that is not a decimal digit character (same as [^[:digit:]]).

Characters	Description
\s	A whitespace character (same as [[:space:]]).
\S	Any character that is not a whitespace character (same as [^[:space:]]).
\w	An alphanumeric or underscore character (same as [_[:alnum:]]).
\W	Any character that is not an alphanumeric or underscore character (same as [^_[:alnum:]]).
character	The *character* character as it is, without interpreting its special meaning within a regex expression. Any character can be escaped except those which form any of the special character sequences above. Needed for: ^ $ \ . * + ? ( ) [ ] { } |
[*class*]	A string is part of the *class*. see POSIX-based character classes.
[^*class*]	A string is not part of the *class*. see POSIX-based character classes.
\p{*class*}	A string is range of Unicode range of *class*. see Unicode Character Class.

```
-- .: match any character
SELECT ('cubrid dbms' REGEXP '^c.*$');
```

```
('cubrid dbms' regexp '^c.*$')
================================
  1
```

To match special characters such as “\n”, “\t”, “\r”, and “\\”, some must be escaped with the backslash (\) by specifying the value of **no_backslash_escapes** (default: yes) to **no**. For details on **no_backslash_escapes**, see Escape Special Characters.

```
-- \n : match a special character, when no_backslash_escapes=yes
(default)
SELECT ('new\nline' REGEXP 'new\\nline');
```

```
('new\nline' REGEXP 'new\\nline');
====================================
  1
```

```
-- \n : match a special character, when no_backslash_escapes=no
SELECT ('new\nline' REGEXP 'new
line');
```

```
('new
line' regexp 'new
line')
====================================
  0
```

Quantifiers

Quantifiers follow a character or a special pattern character. They can modify the amount of times that character is repeated in the match:

Characters	Description
*	The preceding is matched 0 or more times.
+	The preceding is matched 1 or more times.
?	

Characters	Description
	The preceding is optional (matched either 0 times or once).
{*num*}	The preceding is matched exactly *num* times.
{*num*,}	The preceding is matched *num* or more times.
{*min*,*max*}	The preceding is matched at least *min* times, but not more than *max*.

```
-- a+ : match any sequence of one or more a characters. case
insensitive.
SELECT ('Aaaapricot' REGEXP '^A+pricot');
```

```
('Aaaapricot' regexp '^A+pricot')
=================================
  1
```

```
-- a? : match either zero or one a character.
SELECT ('Apricot' REGEXP '^Aa?pricot');
```

```
('Apricot' regexp '^Aa?pricot')
===============================
  1
```

```
SELECT ('Aapricot' REGEXP '^Aa?pricot');
```

```
('Aapricot' regexp '^Aa?pricot')
================================
  1
```

```
SELECT ('Aaapricot' REGEXP '^Aa?pricot');
```

```
('Aaapricot' regexp '^Aa?pricot')
=============================
  0
```

```
-- (cub)* : match zero or more instances of the sequence abc.
SELECT ('cubcub' REGEXP '^(cub)*$');
```

```
('cubcub' regexp '^(cub)*$')
============================
  1
```

By default, all these quantifiers perform in a *greedy* way which takes as many characters that meet the condition as possible. And this behavior can be overridden to *non-greedy* by adding a question mark (?) after the quantifier.

```
-- (a+), (a+?) : match with quantifiers performs greedy and ungreedy
respectively.
SELECT REGEXP_SUBSTR ('aardvark', '(a+)'), REGEXP_SUBSTR
('aardvark', '(a+?)');
```

```
regexp_substr('aardvark', '(a+)')  regexp_substr('aardvark', '(a+?)')
=============================================
  'aa'                  'a'
```

Groups

Groups allow to apply quantifiers to a sequence of characters (instead of a single character). There are two kinds of groups:

> **⚠ Warning**
>
> a backreference syntax using $int is supported only in C++ <regex>.

Characters	Description
(*subpattern*)	Group which creates a backreference.

Characters	Description
(?:*subpattern*)	Passive group which does not create a backreference.

```
-- The captured group can be referenced with $int
SELECT REGEXP_REPLACE ('hello cubrid','([[:alnum:]]+)','$1!');
```

```
regexp_replace('hello cubrid','([[:alnum:]]+)','$1!')
============================
  'hello! cubrid!'
```

When a group creates a backreference, the characters that represent the subpattern in a string are stored as a submatch. Each submatch is numbered after the order of appearance of their opening parenthesis (the first submatch is number 1, the second is number 2, and so on...). These submatches can be used in the regular expression itself to specify that the entire subpattern should appear again somewhere else (see int in the special characters list). They can also be used in the replacement string in the REGEXP_REPLACE function.

```
-- performs regexp_substr without groups. the following is the case
that fully matched.
SELECT REGEXP_SUBSTR ('abckabcjabc', '[a-c]{3}k[a-c]{3}j[a-c]{3}');

-- ([a-c]{3}) creates a backreference, \1
SELECT REGEXP_SUBSTR ('abckabcjabc', '([a-c]{3})k\1j\1');
```

```
regexp_substr('abckabcjabc', '[a-c]{3}k[a-c]{3}j[a-c]{3}')
======================
  'abckabcjabc'

regexp_substr('abckabcjabc', '([a-c]{3})k\1j\1')
======================
  'abckabcjabc'
```

Assertions

Assertions are conditions that do not consume characters in a string: they do not describe a character, but a condition that must be fulfilled before or after a character.

Characters	Description
^	The beginning of a string, or follows a line terminator
$	The end of a string, or precedes a line terminator

Note

The following syntax is supported only in C++ <regex>.

Characters	Description
\b	The previous character is a word character and the next is a non-word character (or vice-versa).
\B	The previous and next characters are both word characters or both are non-word characters.
(?=subpattern)	Positive lookahead. The characters following the character must match subpattern, but no characters are consumed.
(?!subpattern)	Negative lookahead. The characters following the assertion must not match subpattern, but no characters are consumed.

```
-- ^ : match the beginning of a string
SELECT ('cubrid dbms' REGEXP '^cub');
```

```
('cubrid dbms' regexp '^cub')
================================
  1
```

```
-- $ : match the end of a string
SELECT ('this is cubrid dbms' REGEXP 'dbms$');
```

```
('this is cubrid dbms' regexp 'dbms$')
=======================================
  1
```

```
-- (?=subpattern): positive lookahead
SELECT REGEXP_REPLACE ('cubrid dbms cubrid sql cubrid rdbms',
'cubrid(?= sql)', 'CUBRID');

-- (?!subpattern): nagative lookahead
SELECT REGEXP_REPLACE ('cubrid dbms cubrid sql cubrid rdbms',
'cubrid(?! sql)', 'CUBRID');
```

```
regexp_replace('cubrid dbms cubrid sql cubrid rdbms', 'cubrid(?=
sql)', 'CUBRID')
======================
  'cubrid dbms CUBRID sql cubrid rdbms'

regexp_replace('cubrid dbms cubrid sql cubrid rdbms', 'cubrid(?!
sql)', 'CUBRID')
======================
  'CUBRID dbms cubrid sql CUBRID rdbms'
```

Alternatives

A pattern can include different alternatives:

Characters	Description
|	Separates two alternative patterns or subpatterns.

```
-- a|b : matches any character that is either a or b.
SELECT ('a' REGEXP 'a|b');
SELECT ('d' REGEXP 'a|b');
```

```
('a' regexp 'a|b')
==============================
  1

('d' regexp 'a|b')
==============================
  0
```

A regular expression can contain multiple alternative patterns simply by separating them with the separator operator (|): The regular expression will match if any of the alternatives match, and as soon as one does. Subpatterns (in groups or assertions) can also use the separator operator to separate different alternatives.

```
-- a|b|c : matches any character that is either a, b or c.
SELECT ('a' REGEXP 'a|b|c');
SELECT ('d' REGEXP 'a|b|c');
```

```
('a' regexp 'a|b|c')
==============================
  1

('d' regexp 'a|b|c')
==============================
  0
```

Character classes

Character classes syntax matches one of characters or a category of characters within square brackets.

Individual characters

Any character specified is considered part of the class (except the characters \, [,]).

```
-- [abc] : matches any character that is either a, b or c.
SELECT ('a' REGEXP '[abc]');
SELECT ('d' REGEXP '[abc]');
```

```
('a' regexp '[abc]')
==============================
  1

('d' regexp '[abc]')
==============================
  0
```

Ranges

To represent a range of characters, use the dash character (-) between two valid characters. For example, “[a-z]” matches any alphabet letter whereas “[0-9]” matches any single number.

```
SELECT ('adf' REGEXP '[a-f]');
SELECT ('adf' REGEXP '[g-z]');
```

```
('adf' regexp '[a-f]')
================================
  1

('adf' regexp '[g-z]')
================================
  0
```

```
-- [0-9]+: matches number sequence in a string
SELECT REGEXP_SUBSTR ('aas200gjb', '[0-9]+');
```

```
regexp_substr('aas200gjb', '[0-9]+')
======================
  '200'
```

```
SELECT ('strike' REGEXP '^[^a-dXYZ]+$');
```

```
('strike' regexp '^[^a-dXYZ]+$')
================================
  1
```

POSIX-based character classes

The POSIX-based character class (*[:classname:]*) defines categories of characters as shown below.

> **Note**
>
> Google RE2 matches only ASCII characters, and C++ <regex> matches Unicode characters as well.

Class	Description
[:alnum:]	Alpha-numerical character
[:alpha:]	Alphabetic character
[:blank:]	Blank character
[:cntrl:]	Control character
[:digit:]	Decimal digit character
[:graph:]	Character with graphical representation
[:lower:]	Lowercase letter
[:print:]	Printable character
[:punct:]	Punctuation mark character
[:space:]	Whitespace character
[:upper:]	Uppercase letter

Class	Description
[:xdigit:]	Hexadecimal digit character

[:d:], [:w:] and [:s:] are an extension to the ECMAScript grammar. Only available in C++ <regex>.

Class	Description
[:d:]	Decimal digit character (0-9)
[:w:]	Word character
[:s:]	Whitespace character

```
SELECT REGEXP_SUBSTR ('Samseong-ro 86-gil, Gangnam-gu, Seoul 06178',
'[[:digit:]]{5}');
```

```
regexp_substr('Samseong-ro 86-gil, Gangnam-gu, Seoul 06178',
'[[:digit:]]{5}')
======================
  '06178'
```

```
-- ;set regexp_engine=cppstd
SET NAMES utf8 COLLATE utf8_ko_cs;
SELECT REGEXP_REPLACE ('가나다 가나 가나다라', '\b([[:alpha:]]{2})\b',
'#');
```

```
regexp_replace('가나다 가나 가나다라' collate utf8_ko_cs,
'\b([[:alpha:]]{2})\b' collate utf8_ko_cs, '#' collate utf8_ko_cs)
======================
'가나다 # 가나다라'
```

Unicode Character Class

Unicode character classes (*\p{classname}*) are only supported by Google RE2. The Unicode class name can be specified as shown in the table below. For example, to match Korean characters, you can use **\p{Hangul}**.

Unicode Character Class
Adlam
Ahom
Anatolian_Hieroglyphs
Arabic
Armenian
Avestan
Balinese
Bamum
Bassa_Vah
Batak
Bengali
Bhaiksuki
Bopomofo

Unicode Character Class
Brahmi
Braille
Buginese
Buhid
Canadian_Aboriginal
Carian
Caucasian_Albanian
Chakma
Cham
Cherokee
Chorasmian
Common
Coptic
Cuneiform
Cypriot

Unicode Character Class
Cypro_Minoan
Cyrillic
Deseret
Devanagari
Dives_Akuru
Dogra
Duployan
Egyptian_Hieroglyphs
Elbasan
Elymaic
Ethiopic
Georgian
Glagolitic
Gothic
Grantha

Unicode Character Class
Greek
Gujarati
Gunjala_Gondi
Gurmukhi
Han
Hangul
Hanifi_Rohingya
Hanunoo
Hatran
Hebrew
Hiragana
Imperial_Aramaic
Inherited
Inscriptional_Pahlavi
Inscriptional_Parthian

Unicode Character Class
Javanese
Kaithi
Kannada
Katakana
Kawi
Kayah_Li
Kharoshthi
Khitan_Small_Script
Khmer
Khojki
Khudawadi
Lao
Latin
Lepcha
Limbu

Unicode Character Class
Linear_A
Linear_B
Lisu
Lycian
Lydian
Mahajani
Makasar
Malayalam
Mandaic
Manichaean
Marchen
Masaram_Gondi
Medefaidrin
Meetei_Mayek
Mende_Kikakui

Unicode Character Class
Meroitic_Cursive
Meroitic_Hieroglyphs
Miao
Modi
Mongolian
Mro
Multani
Myanmar
Nabataean
Nag_Mundari
Nandinagari
New_Tai_Lue
Newa
Nko
Nushu

Unicode Character Class
Nyiakeng_Puachue_Hmong
Ogham
Ol_Chiki
Old_Hungarian
Old_Italic
Old_North_Arabian
Old_Permic
Old_Persian
Old_Sogdian
Old_South_Arabian
Old_Turkic
Old_Uyghur
Oriya
Osage
Osmanya

Unicode Character Class
Pahawh_Hmong
Palmyrene
Pau_Cin_Hau
Phags_Pa
Phoenician
Psalter_Pahlavi
Rejang
Runic
Samaritan
Saurashtra
Sharada
Shavian
Siddham
SignWriting
Sinhala

Unicode Character Class
Sogdian
Sora_Sompeng
Soyombo
Sundanese
Syloti_Nagri
Syriac
Tagalog
Tagbanwa
Tai_Le
Tai_Tham
Tai_Viet
Takri
Tamil
Tangsa
Tangut

Unicode Character Class
Telugu
Thaana
Thai
Tibetan
Tifinagh
Tirhuta
Toto
Ugaritic
Vai
Vithkuqi
Wancho
Warang_Citi
Yezidi
Yi
Zanabazar_Square

```
-- ;set regexp_engine=re2
SELECT REGEXP_COUNT('가나 가나다라 마바사아 자차카타 파하', '\p{Hangul}+');
```

```
regexp_count(_utf8'가나 가나다라 마바사아 자차카타 파하' collate
utf8_ko_cs, _utf8'\p{Hangul}+' collate utf8_ko_cs)
==============================
5
```

REGEXP, RLIKE

The **REGEXP** and **RLIKE** are used interchangeably. It performs a regular expression matcinh of a string. In the below syntax, if *expression* matches *pattern*, 1 is returned; otherwise, 0 is returned. If either *expression* or *pattern* is **NULL**, **NULL** is returned. The second syntax has the same meaning as the third syntax, which both syntaxes are using **NOT**.

```
expression REGEXP | RLIKE [BINARY] pattern
expression NOT REGEXP | RLIKE pattern
NOT (expression REGEXP | RLIKE pattern)
```

- *expression* : Column or input expression
- *pattern* : Pattern used in regular expressions

The difference between **REGEXP** and **LIKE** are as follows:

- The **LIKE** operator succeeds only if the pattern matches the entire value.
- The **REGEXP** operator succeeds if the pattern matches anywhere in the value. To match the entire value, you should use "^" at the beginning and "$" at the end.
- The **LIKE** operator is case sensitive, but patterns of regular expressions in **REGEXP** is not case sensitive. To enable case sensitive, you should use **REGEXP BINARY** statement.

```
-- [a-dX] : matches any character that is either a, b, c, d or X.
SELECT ('aXbc' REGEXP '[a-dX]');
```

```
('aXbc' regexp '[a-dX]')
==============================
1
```

```
-- When REGEXP is used in SELECT list, enclosing this with
parentheses is required.
-- But used in WHERE clause, no need parentheses.
-- case insensitive, except when used with BINARY.
SELECT name FROM public.athlete where name REGEXP '^[a-d]';
```

```
name
======================
'Dziouba Irina'
'Dzieciol Iwona'
'Dzamalutdinov Kamil'
'Crucq Maurits'
'Crosta Daniele'
'Bukovec Brigita'
'Bukic Perica'
'Abdullayev Namik'
```

REGEXP_COUNT

REGEXP_COUNT(*string*, *pattern_string*[, *position*[, *match_type*]])

The **REGEXP_COUNT** function returns the number of occurrences of the regular expression pattern, *pattern_string*, within a given character string, *string*. If **NULL** is specified as an argument, **NULL** is returned.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.
- **pattern_string** – Specifies the regular expression pattern string to be searched. If the value is **NULL**, **NULL** is returned.
- **position** – Specifies the position of the *string* to start the search. If the value is omitted, the default value 1 is applied. If the value is negative or zero, an error will be returned. If the value is **NULL**, **NULL** is returned
- **match_type** – Specifies the string to change default matching behavior of the function. If the value is omitted, the default value 'i' is applied. If the value is other than 'c' or 'i', an error will be returned. If the value is **NULL**, **NULL** is returned.

Return type:

INT

```
-- it returns NULL when an argument is specified with NULL value
SELECT REGEXP_COUNT('ab123ab111a','[a-d]+',NULL);
```

```
regexp_count('ab123ab111a','[a-d]+',NULL)
======================
  NULL
```

```
-- an empty string pattern doesn't match with any string
SELECT REGEXP_COUNT('ab123ab111a','');
```

```
regexp_count('ab123ab111a','')
======================
  0
```

```
SELECT REGEXP_COUNT('ab123Ab111aAA','[a-d]', 3);
```

```
regexp_count('ab123Ab111aAA', '[a-d]', 3)
==========================================
                                         5
```

```
-- case insensitive ('i') is the default value
SELECT REGEXP_COUNT('ab123Ab111aAA','[a-d]', 3, 'i');

-- If case sensitive ('c') is specified as match_type, A is not
matched.
SELECT REGEXP_COUNT('ab123Ab111aAA','[a-d]', 3, 'c');
```

```
regexp_count('ab123Ab111aAA', '[a-d]', 3, 'i')
==============================================
                                             5

regexp_count('ab123Ab111aAA', '[a-d]', 3, 'c')
==============================================
                                             2
```

```
SET NAMES utf8 COLLATE utf8_ko_cs;
SELECT REGEXP_COUNT('가나123abc가다abc가가','[가-나]+');
```

```
regexp_count('가나123abc가다abc가가' collate utf8_ko_cs, '[가-나]+'
collate utf8_ko_cs)
==========================================================================
============================

                           3
```

REGEXP_INSTR

REGEXP_INSTR(*string*, *pattern_string*[, *position*[, *occurrence*[, *return_option*[, *match_type*]]]])

The **REGEXP_INSTR** function returns the beginning or ending position by searching for a regular expression pattern, *pattern_string*, within a given character string, *string*. If **NULL** is specified as an argument, **NULL** is returned.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.

- **pattern_string** – Specifies the regular expression pattern string to be searched. If the value is **NULL**, **NULL** is returned.
- **position** – Specifies the position of the *string* to start the search. If the value is omitted, the default value 1 is applied. If the value is negative or zero, an error will be returned. If the value is **NULL**, **NULL** is returned
- **occurrence** – Specifies the occurrence of the match to use. If the value is omitted, the default value 1 is applied. If the value is negative, an error will be returned. If the value is **NULL**, **NULL** is returned.
- **return_option** – Specifies whether to return the position of the start or end of the matched string. If the value is 0, the position of the first character of the match is returned. If the value is 0, the starting position of the matched string is returned. If the value is 1, the end position of the matched string is returned. If the value is other than 0 or 1, an error will be returned. If the value is **NULL**, **NULL** is returned.
- **match_type** – Specifies the string to change default matching behavior of the function. If the value is omitted, the default value 'i' is applied. If the value is other than 'c' or 'i', an error will be returned. If the value is **NULL**, **NULL** is returned.

Return type:

INT

```
-- it returns NULL when an argument is specified with NULL value
SELECT REGEXP_INSTR('12345abcdeabcde','[abc]',NULL);
```

```
regexp_instr('12345abcdeabcde', '[abc]', null)
======================
  NULL
```

```
-- an empty string pattern doesn't match with any string
SELECT REGEXP_INSTR('12345abcdeabcde','');
```

```
regexp_instr('12345abcdeabcde', '')
=======================
  0
```

```
-- it returns the position of the first character of the match.
SELECT REGEXP_INSTR('12354abc5','[:alpha:]+',1,1,0);
```

```
regexp_instr('12354abc5','[:alpha:]+', 1, 1, 0);
=======================
  6
```

```
-- it returns the position of the character following the match.
SELECT REGEXP_INSTR('12354abc5','[:alpha:]+',1,1,1);
```

```
regexp_instr('12354abc5','[:alpha:]+', 1, 1, 1);
=======================
  7
```

```
SET NAMES utf8 COLLATE utf8_ko_cs;
SELECT REGEXP_INSTR('12345가나다라마가나다라마바','[가-다]+');
```

```
regexp_instr('12345가나다라마가나다라마바' collate utf8_ko_cs, '[가-다]+'
collate utf8_ko_cs)
=============================================================================
=======================================

                                     6
```

REGEXP_LIKE

REGEXP_LIKE(*string*, *pattern_string*[, *match_type*])

The **REGEXP_LIKE** function searches for a regular expression pattern, *pattern_string*, within a given character string, *string*. If the pattern matched anywhere in the *string*, 1 is returned. Otherwise, 0 is returned. If **NULL** is specified as an argument, **NULL** is returned.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.
- **pattern_string** – Specifies the regular expression pattern string to be searched. If the value is **NULL**, **NULL** is returned.
- **match_type** – Specifies the string to change default matching behavior of the function. If the value is omitted, the default value 'i' is applied. If the value is other than 'c' or 'i', an error will be returned. If the value is **NULL**, **NULL** is returned.

Return type:

INT

```
SELECT REGEXP_LIKE('abbbbc','ab+c');
```

```
regexp_like('abbbbc', 'ab+c');
======================
  1
```

```
-- an empty string pattern doesn't match with any string
SELECT REGEXP_LIKE('abbbbc','');
```

```
regexp_like('abbbbc', '');
======================
  0
```

```
SELECT REGEXP_LIKE('abbbbc','AB+C', 'c');
```

```
regexp_like('abbbbc', 'AB+C');
======================
  0
```

```
SET NAMES utf8 COLLATE utf8_ko_cs;
SELECT REGEXP_LIKE('가나다','가나?다');
SELECT REGEXP_LIKE('가나라다','가나?다');
```

```
regexp_like('가나다' collate utf8_ko_cs, '가나?다' collate utf8_ko_cs)
=======================================================================
=========

       1

regexp_like('가나라다' collate utf8_ko_cs, '가나?다' collate utf8_ko_cs)
=======================================================================
============

           0
```

REGEXP_REPLACE

REGEXP_REPLACE(*string*, *pattern_string*, *replacement_string*[, *position*[, *occurrence*[, *match_type*]]])

The **REGEXP_REPLACE** function searches for a regular expression pattern, *pattern_string*, within a given character string, *string*, and replaces it with a character string, *replacement_string*. If **NULL** is specified as an argument, **NULL** is returned.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.
- **pattern_string** – Specifies the regular expression pattern string to be searched. If the value is **NULL**, **NULL** is returned.
- **replacement_string** – Specifies the string to replace the matched string by *pattern_string*. If the value is **NULL**, **NULL** is returned.
- **position** – Specifies the position of the *string* to start the search. If the value is omitted, the default value 1 is applied. If the value is negative or zero, an error will be returned. If the value is **NULL**, **NULL** is returned

- **occurrence** – Specifies the occurrence of the pattern to use. If the value is omitted, the default value 0 is applied. If the value is negative, an error will be returned. If the value is **NULL**, **NULL** is returned.
- **match_type** – Specifies the string to change default matching behavior of the function. If the value is omitted, the default value 'i' is applied. If the value is other than 'c' or 'i', an error will be returned. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

```
-- it returns NULL when an argument is specified with NULL value
SELECT REGEXP_REPLACE('12345abcdeabcde','[a-d]',NULL);
```

```
regexp_replace('12345abcdeabcde', '[a-d]', null)
=======================
NULL
```

```
-- an empty string pattern doesn't match with any string
SELECT REGEXP_REPLACE('12345abcdeabcde','','#');
```

```
regexp_replace('12345abcdeabcde', '', '#')
=======================
  '12345abcdeabcde'
```

```
SELECT REGEXP_REPLACE('12345abDEKBcde','[a-d]','#');
```

```
regexp_replace('12345abDEKBcde', '[a-d]', '#')
=======================
  '12345###EK###e'
```

```
-- case insensitive ('i') is the default value
SELECT REGEXP_REPLACE('12345abDEKBcde','[a-d]','#', 1, 0, 'i');

-- match_type is specified as case sensitive ('c'). 'B' and 'D' are
```

```
not matched.
SELECT REGEXP_REPLACE('12345abDEKBcde','[a-d]','#', 1, 0, 'c');
```

```
regexp_replace('12345abDEKBcde', '[a-d]', '#', 1, 0, 'i')
======================
  '12345###EK###e'


regexp_replace('12345abDEKBcde', '[a-d]', '#', 1, 0, 'c')
======================
  '12345##DEKB##e'
```

```
SET NAMES utf8 COLLATE utf8_ko_cs;
SELECT REGEXP_REPLACE('a1가b2나다라','[가-다]','#',6);
```

```
regexp_replace('a1가b2나다라' collate utf8_ko_cs, '[가-다]' collate
utf8_ko_cs, '#' collate utf8_ko_cs, 6)
======================
'a1가b2##라'
```

REGEXP_SUBSTR

REGEXP_SUBSTR(*string*, *pattern_string*[, *position*[, *occurrence*[, *match_type*]]])

The **REGEXP_SUBSTR** function extracts a character string matched for a regular expression pattern, *pattern_string*, within a given character string, *string*. If **NULL** is specified as an argument, **NULL** is returned.

Parameters:

- **string** – Specifies the original string. If the value is **NULL**, **NULL** is returned.
- **pattern_string** – Specifies the regular expression pattern string to be searched. If the value is **NULL**, **NULL** is returned.
- **position** – Specifies the position of the *string* to start the search. If the value is omitted, the default value 1 is applied. If the value is negative or zero, an error will be returned. If the value is **NULL**, **NULL** is returned

- **occurrence** – Specifies the occurrence of the pattern to use. If the value is omitted, the default value 0 is applied. If the value is negative, an error will be returned. If the value is **NULL**, **NULL** is returned.
- **match_type** – Specifies the string to change default matching behavior of the function. If the value is omitted, the default value 'i' is applied. If the value is other than 'c' or 'i', an error will be returned. If the value is **NULL**, **NULL** is returned.

Return type:

STRING

```
-- if pattern is not matched, null is returned
SELECT REGEXP_SUBSTR('12345abcdeabcde','[k-z]+');
```

```
regexp_substr('12345abcdeabcde','[k-z]+');
======================
  NULL
```

```
-- an empty string pattern doesn't match with any string
SELECT REGEXP_SUBSTR('12345abcdeabcde','');
```

```
regexp_substr('12345abcdeabcde', '')
======================
  NULL
```

```
SELECT REGEXP_SUBSTR('Samseong-ro, Gangnam-gu, Seoul',',[^,]+,');
```

```
regexp_substr('Samseong-ro, Gangnam-gu, Seoul', ',[^,]+,')
======================
  ', Gangnam-gu,'
```

```
SET NAMES utf8 COLLATE utf8_ko_cs;
SELECT REGEXP_SUBSTR('삼성로, 강남구, 서울특별시','\p{Hangul}+',1,2);
```

```
regexp_substr('삼성로, 강남구, 서울특별시' collate utf8_ko_cs,
'\p{Hangul}+' collate utf8_ko_cs, 1, 2)
======================
'강남구'
```

Numeric/Mathematical Functions

Contents

Numeric/Mathematical Functions 462

- ABS 463
- ACOS 464
- ASIN 465
- ATAN 465
- ATAN2 466
- CEIL 467
- CONV 467
- COS 468
- COT 469
- CRC32 469
- DEGREES 470
- DRANDOM, DRAND 471
- EXP 473
- FLOOR 473

- HEX 474
- LN 475
- LOG 475
- LOG2 476
- LOG10 477
- MOD 477
- PI 479
- POW, POWER 479
- RADIANS 480
- RANDOM, RAND 481
- ROUND 483
- SIGN 484
- SIN 484
- SQRT 485
- TAN 486
- TRUNC, TRUNCATE 486
- WIDTH_BUCKET 488

ABS

ABS(*number_expr*)

The **ABS** function returns the absolute value of a given number. The data type of the return value is the same as that of the argument. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

number_expr – An operator which returns a numeric value

Return type:

same as that of the argument

```
--it returns the absolute value of the argument
SELECT ABS(12.3), ABS(-12.3), ABS(-12.3000), ABS(0.0);
```

```
  abs(12.3)             abs(-12.3)            abs(-12.3000)
abs(0.0)
=============================================================================
===========
  12.3                  12.3                  12.3000                .
0
```

ACOS

ACOS(*x*)

The **ACOS** function returns an arc cosine value of the argument. That is, it returns a value whose cosine is *x* in radian. The return value is a **DOUBLE** type. x must be a value between -1 and 1, inclusive. Otherwise, **NULL** is returned. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT ACOS(1), ACOS(0), ACOS(-1);
```

```
  acos(1)                  acos(0)                  acos(-1)
=============================================================================
```

```
=============
  0.000000000000000e+00     1.570796326794897e+00
3.141592653589793e+00
```

ASIN

ASIN(*x*)

The **ASIN** function returns an arc sine value of the argument. That is, it returns a value whose sine is *x* in radian. The return value is a **DOUBLE** type. x must be a value between -1 and 1, inclusive. Otherwise, **NULL** is returned. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT ASIN(1), ASIN(0), ASIN(-1);
```

```
  asin(1)                  asin(0)                  asin(-1)
============================================================================
=========
  1.570796326794897e+00     0.000000000000000e+00
-1.570796326794897e+00
```

ATAN

ATAN([*y*,]*x*)

The **ATAN** function returns a value whose tangent is *x* in radian. The argument *y* can be omitted. If *y* is specified, the function calculates the arc tangent value of *y*/*x*. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x,y – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT ATAN(1), ATAN(-1), ATAN(1,-1);
```

```
                  atan(1)                  atan(-1)
atan2(1, -1)
===========================================================================
=========
     7.853981633974483e-01    -7.853981633974483e-01
2.356194490192345e+00
```

ATAN2

ATAN2(*y*, *x*)

The **ATAN2** function returns the arc tangent value of *y*/*x* in radian. This function is working like the `ATAN()`. Arguments *x* and *y* must be specified. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x,y – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT ATAN2(1,1), ATAN2(-1,-1), ATAN2(Pi(),0);
```

```
atan2(1, 1)              atan2(-1, -1)           atan2( pi(), 0)
===========================================================================
=========
 7.853981633974483e-01    -2.356194490192345e+00
1.570796326794897e+00
```

CEIL

CEIL(*number_operand*)

The **CEIL** function returns the smallest integer that is not less than its argument. The return value is determined based on the valid number of digits that are specified as the *number_operand* argument. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

number_operand – An expression that returns a numeric value

Return type:

INT

```
SELECT CEIL(34567.34567), CEIL(-34567.34567);
```

```
  ceil(34567.34567)     ceil(-34567.34567)
============================================
  34568.00000           -34567.00000

SELECT CEIL(34567.1), CEIL(-34567.1);
```

```
  ceil(34567.1)         ceil(-34567.1)
============================
  34568.0         -34567.0
```

CONV

CONV(*number*, *from_base*, *to_base*)

The **CONV** function converts numbers between different number bases. This function returns a string representation of a converted number. The minimum value is 2 and the maximum value is 36. If *to_base* (representing the base to be returned) is negative, *number* is regarded as a signed number. Otherwise, it regarded as a unsigned number. When you input the string which cannot be transformed into the number to *from_base* or *to_base*, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

- **number** – An input number
- **from_base** – The base of an input number
- **to_base** – The base of an returned value

Return type:

STRING

```
SELECT CONV('f',16,2);
```

```
'1111'
```

```
SELECT CONV('6H',20,8);
```

```
'211'
```

```
SELECT CONV(-30,10,-20);
```

```
'-1A'
```

COS

COS(*x*)

The **COS** function returns a cosine value of the argument. The argument *x* must be a radian value. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT COS(pi()/6), COS(pi()/3), COS(pi());
```

```
  cos( pi()/6)              cos( pi()/3)               cos( pi())
=============================================================================
==========
  8.660254037844387e-01     5.000000000000001e-01
-1.000000000000000e+00
```

COT

COT(*x*)

The **COT** function returns the cotangent value of the argument *x*. That is, it returns a value whose tangent is *x* in radian. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT COT(1), COT(-1), COT(0);
```

```
  cot(1)                    cot(-1)   cot(0)
=============================================================================
======
  6.420926159343306e-01     -6.420926159343306e-01  NULL
```

CRC32

CRC32(*string*)

The **CRC32** function returns a cyclic redundancy check value as 32-bit integer. When NULL is given as input, it returns NULL.

Parameters:

string – An expression that returns a string value

Return type:

INTEGER

```
SELECT CRC32('cubrid');
```

```
   crc32('cubrid')
==================
         908740081
```

DEGREES

DEGREES(*x*)

The **DEGREES** function returns the argument *x* specified in radian converted to a degree value. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT DEGREES(pi()/6), DEGREES(pi()/3), DEGREES (pi());
```

```
  degrees( pi()/6)          degrees( pi()/3)            degrees(
pi())
=========================================================================
=========
```

```
  3.000000000000000e+01     5.999999999999999e+01
1.800000000000000e+02
```

DRANDOM, DRAND

DRANDOM([*seed*])

DRAND([*seed*])

The function **DRANDOM** or **DRAND** returns a random double-precision floating point value in the range of between 0.0 and 1.0. A *seed* argument that is **INTEGER** type can be specified. It rounds up real numbers and an error is returned when it exceeds the range of **INTEGER**.

When *seed* value is not given, the **DRAND** function performs the operation only once to produce only one random number regardless of the number of rows where the operation is output, but the **DRANDOM** function performs the operation every time the statement is repeated to produce a different random value for each row. Therefore, to output rows in a random order, you must use the **DRANDOM** function in the **ORDER BY** clause. To obtain a random integer value, use the `RANDOM()`.

Parameters:

seed – seed value

Return type:

DOUBLE

```
SELECT DRAND(), DRAND(1), DRAND(1.4);
```

```
                  drand()                  drand(1)
drand(1.4)
================================================================================
    2.849646518006921e-001    4.163034446537495e-002
4.163034446537495e-002
```

```
CREATE TABLE rand_tbl (
    id INT,
    name VARCHAR(255)
);

INSERT INTO rand_tbl VALUES
```

```
    (1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'),
    (6, 'f'), (7, 'g'), (8, 'h'), (9, 'i'), (10, 'j');

SELECT * FROM rand_tbl;
```

```
           id  name
=======================================
            1  'a'
            2  'b'
            3  'c'
            4  'd'
            5  'e'
            6  'f'
            7  'g'
            8  'h'
            9  'i'
           10  'j'
```

```
--drandom() returns random values on every row
SELECT DRAND(), DRANDOM() FROM rand_tbl;
```

```
   drand()                  drandom()
=========================================================================
=========
   7.638782921842098e-001    1.018707846308786e-001
   7.638782921842098e-001    3.191320535905026e-001
   7.638782921842098e-001    3.461714529862361e-001
   7.638782921842098e-001    6.791894283883175e-001
   7.638782921842098e-001    4.533829767754143e-001
   7.638782921842098e-001    1.714224677266762e-001
   7.638782921842098e-001    1.698049867244484e-001
   7.638782921842098e-001    4.507583849604786e-002
   7.638782921842098e-001    5.279091769157994e-001
   7.638782921842098e-001    7.021088290047914e-001
```

```
--selecting rows in random order
SELECT * FROM rand_tbl ORDER BY DRANDOM();
```

```
           id  name
=======================================
            6  'f'
            2  'b'
            7  'g'
            8  'h'
            1  'a'
```

```
             4  'd'
            10  'j'
             9  'i'
             5  'e'
             3  'c'
```

EXP

EXP(*x*)

The **EXP** function returns e x (the base of natural logarithm) raised to a power. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An operator which returns a numeric value

Return type:

DOUBLE

```
SELECT EXP(1), EXP(0);
```

```
  exp(1)                    exp(0)
=======================================================
  2.718281828459045e+000 1.000000000000000e+000
```

```
SELECT EXP(-1), EXP(2.00);
```

```
  exp(-1)                  exp(2.00)
=======================================================
  3.678794411714423e-001 7.389056098930650e+000
```

FLOOR

FLOOR(*number_operand*)

The **FLOOR** function returns the largest integer that is not greater than its argument. The data type of the return value is the same as that of the argument. When you input the

string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

number_operand – An operator which returns a numeric value

Return type:

same as that of the argument

```
--it returns the largest integer less than or equal to the arguments
SELECT FLOOR(34567.34567), FLOOR(-34567.34567);
```

```
  floor(34567.34567)    floor(-34567.34567)
=============================================
  34567.00000           -34568.00000
```

```
SELECT FLOOR(34567), FLOOR(-34567);
```

```
  floor(34567)   floor(-34567)
==============================
         34567         -34567
```

HEX

HEX(*n*)

The **HEX** function returns a hexadecimal string about the string which is specified as an argument; it returns a hexadecimal string of the number if a number is specified as an argument. If a number is specified as an argument, it returns a value like CONV(num, 10, 16).

Parameters:

n – A string or a number

Return type:

STRING

```
SELECT HEX('ab'), HEX(128), CONV(HEX(128), 16, 10);
```

```
hex('ab')             hex(128)              conv(hex(128), 16, 10)
=============================================================
  '6162'                 '80'                   '128'
```

LN

LN(*x*)

The **LN** function returns the natural log value (base = e) of an antilogarithm *x*. The return value is a **DOUBLE** type. If the antilogarithm is 0 or a negative number, an error is returned. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a positive number

Return type:

DOUBLE

```
SELECT ln(1), ln(2.72);
```

```
     ln(1)                     ln(2.72)
=======================================================
     0.000000000000000e+00     1.000631880307906e+00
```

LOG

LOG(*x*, *y*)

The **LOG** function returns the logarithm of *x* with base *y*, and its return type is **DOUBLE**. If either *x* or *y* is 0 or a negative number, the function returns an error. When a non-numeric string is provided as input, the function's behavior depends on the value of the **return_null_on_function_errors** parameter in **cubrid.conf**.

Parameters:

- **x** – The base of the logarithm; it must be an expression that evaluates to a positive value other than 1.
- **y** – The antilogarithm of the logarithm expression; it must be an expression that evaluates to a positive value.

Return type:

DOUBLE

```
SELECT log(2,1), log(2,8), log(10,1), log(10,1000);
```

```
     log(2,1)                   log(2,8)                   log(10,
1)                log(10, 1000)
=============================================================================
==================================
     0.000000000000000e+00      3.000000000000000e+00
0.000000000000000e+00      3.000000000000000e+00
```

LOG2

LOG2(*x*)

The **LOG2** function returns a log value whose antilogarithm is *x* and base is 2. The return value is a **DOUBLE** type. If the antilogarithm is 0 or a negative number, an error is returned. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a positive number

Return type:

DOUBLE

```
SELECT log2(1), log2(8);
```

```
     log2(1)                   log2(8)
============================================================
     0.000000000000000e+00     3.000000000000000e+00
```

LOG10

LOG10(*x*)

The **LOG10** function returns the common log value of an antilogarithm *x*. The return value is a **DOUBLE** type. If the antilogarithm is 0 or a negative number, an error is returned. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a positive number

Return type:

DOUBLE

```
SELECT log10(1), log10(1000);
```

```
     log10(1)                  log10(1000)
============================================================
     0.000000000000000e+00     3.000000000000000e+00
```

MOD

MOD(*m*, *n*)

The **MOD** function returns the remainder of the first parameter *m* divided by the second parameter *n*. If *n* is 0 or greater than *m*, *m* is returned without the division operation being performed. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Note that if the dividend, the parameter m of the **MOD** function, is a negative number, the function operates differently from a typical operation (classical modulus) method.

Result of MOD

m	n	MOD(m, n)	Classical Modulus m-n*FLOOR(m/n)
11	4	3	3
11	-4	3	-1
-11	4	-3	1
-11	-4	-3	-3
11	0	11	Divided by 0 error

Parameters:

- **m** – Represents a dividend. It is an expression that returns a numeric value.
- **n** – Represents a divisor. It is an expression that returns a numeric value.

Return type:

Result type of m/n

```
--it returns the reminder of m divided by n
SELECT MOD(11, 4), MOD(11, -4), MOD(-11, 4), MOD(-11, -4), MOD(11,0);
```

```
    mod(11, 4)   mod(11, -4)   mod(-11, 4)   mod(-11, -4)   mod(11,
0)
=============================================================================
            3             3            -3             -3            11
```

```
SELECT MOD(11.0, 4), MOD(11.000, 4), MOD(11, 4.0), MOD(11, 4.000);
```

```
  mod(11.0, 4)          mod(11.000, 4)        mod(11, 4.0)
mod(11, 4.000)
=============================================================================
====
  3.0                   3.000                 3.0
3.000
```

PI

PI()

The **PI** function returns the ? value of type **DOUBLE**.

Return type:

DOUBLE

```
SELECT PI(), PI()/2;
```

```
     pi()                    pi()/2
=======================================================
     3.141592653589793e+00     1.570796326794897e+00
```

POW, POWER

POW(*x*, *y*)

POWER(*x*, *y*)

The **POW** function returns *x* to the power of *y*. The functions **POW** and **POWER** are used interchangeably. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

- **x** – It represents the base. It is an expression that returns a numeric value. An expression that returns a numeric value.

- **y** – It represents the exponent. An expression that returns a numeric value. If the base is a negative number, an integer must specified as the exponent.

Return type:

DOUBLE

```
SELECT POWER(2, 5), POWER(-2, 5), POWER(0, 0), POWER(1,0);
```

```
 power(2, 5)              power(-2, 5)              power(0,
0)               power(1, 0)
================================================================================
================================
 3.200000000000000e+01    -3.200000000000000e+01
1.000000000000000e+00     1.000000000000000e+00
```

```
--it returns an error when the negative base is powered by a non-int
exponent
SELECT POWER(-2, -5.1), POWER(-2, -5.1);
```

```
ERROR: Argument of power() is out of range.
```

RADIANS

RADIANS(*x*)

The **RADIANS** function returns the argument *x* specified in degrees converted to a radian value. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT RADIANS(90), RADIANS(180), RADIANS(360);
```

```
     radians(90)              radians(180)             radians(360)
=============================================================================
=========
     1.570796326794897e+00     3.141592653589793e+00
6.283185307179586e+00
```

RANDOM, RAND

RANDOM([*seed*])

RAND([*seed*])

The function **RANDOM** or **RAND** returns any integer value, which is greater than or equal to 0 and less than 2^31, and a *seed* argument that is **INTEGER** type can be specified. It rounds up real numbers and an error is returned when it exceeds the range of **INTEGER**.

When *seed* value is not given, the **RAND** function performs the operation only once to produce only one random number regardless of the number of rows where the operation is output, but the **RANDOM** function performs the operation every time the statement is repeated to produce a different random value for each row. Therefore, to output rows in a random order, you must use the **RANDOM** function.

To obtain a random real number, use the `DRANDOM()`.

Parameters:

seed

Return type:

INT

```
SELECT RAND(), RAND(1), RAND(1.4);
```

```
       rand()      rand(1)    rand(1.4)
=======================================
   1526981144     89400484     89400484
```

```
--creating a new table
SELECT * FROM rand_tbl;
```

```
           id  name
=================================
            1  'a'
            2  'b'
            3  'c'
            4  'd'
            5  'e'
            6  'f'
            7  'g'
            8  'h'
            9  'i'
           10  'j'
```

```
--random() returns random values on every row
SELECT RAND(),RANDOM() FROM rand_tbl;
```

```
       rand()       random()
=============================
   2078876566     1753698891
   2078876566     1508854032
   2078876566      625052132
   2078876566      279624236
   2078876566     1449981446
   2078876566     1360529082
   2078876566     1563510619
   2078876566     1598680194
   2078876566     1160177096
   2078876566     2075234419
```

```
--selecting rows in random order
SELECT * FROM rand_tbl ORDER BY RANDOM();
```

```
           id  name
=================================
            6  'f'
            1  'a'
            5  'e'
            4  'd'
            2  'b'
            7  'g'
           10  'j'
```

```
             9  'i'
             3  'c'
             8  'h'
```

ROUND

ROUND(*number_operand*, *integer*)

The **ROUND** function returns the specified argument, *number_operand*, rounded to the number of places after the decimal point specified by the *integer*. If the *integer* argument is a negative number, it rounds to a place before the decimal point, that is, at the integer part. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

- **number_operand** – An expression that returns a numeric value
- **integer** – Specifies the place to round to. If a positive integer *n* is specified, the number is represented to the nth place after the decimal point; if a negative integer *n* is specified, the number is rounded to the *n* th place before the decimal point.

Return type:

same type as the *number_operand*

```
--it rounds a number to one decimal point when the second argument
is omitted
SELECT ROUND(34567.34567), ROUND(-34567.34567);
```

```
  round(34567.34567, 0)   round(-34567.34567, 0)
========================================
  34567.00000           -34567.00000
```

```
--it rounds a number to three decimal point
SELECT ROUND(34567.34567, 3), ROUND(-34567.34567, 3)  FROM db_root;
```

```
 round(34567.34567, 3)   round(-34567.34567, 3)
=============================================
  34567.34600           -34567.34600
```

```
--it rounds a number three digit to the left of the decimal point
SELECT ROUND(34567.34567, -3), ROUND(-34567.34567, -3);
```

```
 round(34567.34567, -3)   round(-34567.34567, -3)
=============================================
  35000.00000           -35000.00000
```

SIGN

SIGN(*number_operand*)

The **SIGN** function returns the sign of a given number. It returns 1 for a positive value, -1 for a negative value, and 0 for zero. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

number_operand – An operator which returns a numeric value

Return type:

INT

```
--it returns the sign of the argument
SELECT SIGN(12.3), SIGN(-12.3), SIGN(0);
```

```
    sign(12.3)   sign(-12.3)      sign(0)
=========================================
             1            -1            0
```

SIN

SIN(*x*)

The **SIN** function returns a sine value of the parameter. The argument *x* must be a radian value. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT SIN(pi()/6), SIN(pi()/3), SIN(pi());
```

```
     sin( pi()/6)              sin( pi()/3)              sin( pi())
=============================================================================
=========
     4.999999999999999e-01     8.660254037844386e-01
1.224646799147353e-16
```

SQRT

SQRT(*x*)

The **SQRT** function returns the square root of *x* as a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value. An error is returned if this value is a negative number.

Return type:

DOUBLE

```
SELECT SQRT(4), SQRT(16.0);
```

```
     sqrt(4)                  sqrt(16.0)
=============================================================
     2.000000000000000e+00    4.000000000000000e+00
```

TAN

TAN(*x*)

The **TAN** function returns a tangent value of the argument. The argument *x* must be a radian value. The return value is a **DOUBLE** type. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

x – An expression that returns a numeric value

Return type:

DOUBLE

```
SELECT TAN(pi()/6), TAN(pi()/3), TAN(pi()/4);
```

```
     tan( pi()/6)             tan( pi()/3)             tan( pi()/4)
=============================================================================
=========
     5.773502691896257e-01    1.732050807568877e+00
9.999999999999999e-01
```

TRUNC, TRUNCATE

TRUNC(*x*[, *dec*])

TRUNCATE(*x*, *dec*)

The function **TRUNC** or **TRUNCATE** truncates the numbers of the specified argument *x* to the right of the *dec* position. If the *dec* argument is a negative number, it displays 0s to the *dec*-th position left to the decimal point. Note that the *dec* argument of the **TRUNC** function can be omitted, but that of the **TRUNCATE** function cannot be omitted. If the *dec* argument is a negative number, it displays 0s to the *dec* -th position left to the decimal point. The number of digits of the return value to be represented follows the argument *x*. When you input the string which cannot be transformed into the number, it returns an error if the

value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

Parameters:

- **x** – An expression that returns a numeric value
- **dec** – The place to be truncated is specified. If a positive integer *n* is specified, the number is represented to the *n*-th place after the decimal point; if a negative integer *n* is specified, the number is truncated to the *n*-th place before the decimal point. It truncates to the first place after the decimal point if the *dec* argument is 0 or omitted. Note that the *dec* argument cannot be omitted in the **TRUNCATE** function.

Return type:

same type as the *x*

```
--it returns a number truncated to 0 places
SELECT TRUNC(34567.34567), TRUNCATE(34567.34567, 0);
```

```
  trunc(34567.34567, 0)   trunc(34567.34567, 0)
========================================
  34567.00000             34567.00000
```

```
--it returns a number truncated to three decimal places
SELECT TRUNC(34567.34567, 3), TRUNC(-34567.34567, 3);
```

```
  trunc(34567.34567, 3)   trunc(-34567.34567, 3)
========================================
  34567.34500            -34567.34500
```

```
--it returns a number truncated to three digits left of the decimal
point
SELECT TRUNC(34567.34567, -3), TRUNC(-34567.34567, -3);
```

```
  trunc(34567.34567, -3)   trunc(-34567.34567, -3)
=========================================
  34000.00000            -34000.00000
```

WIDTH_BUCKET

WIDTH_BUCKET(*expression*, *from*, *to*, *num_buckets*)

WIDTH_BUCKET distributes the rows in an ordered partition into a specified number of buckets. The buckets are numbered, starting from one. That is, **WIDTH_BUCKET** function creates an equi-width histogram. The return value is an integer. When you input the string which cannot be transformed into the number, it returns an error if the value of **return_null_on_function_errors** in **cubrid.conf** is no(the default), or returns NULL if it is yes.

This function equally divides the range by the given number of buckets and assigns the bucket number to each bucket. That is, every interval (bucket) has the identical size.

Note that `NTILE()` function equally divides the number of rows by the given number of buckets and assigns the bucket number to each bucket. That is, every bucket has the same number of rows.

Parameters:

- **expression** – an input value to assign the bucket number. It specifies a certain expression which returns the number.
- **from** – a start value of the range, which is given to *expression*. It is included in the entire range.
- **to** – an end value of the range, which is given to *expression*. It is not included in the entire range.
- **num_buckets** – the number of buckets. The #0 bucket and the #(*num_buckets* + 1) bucket are created to include the contents beyond the range.

Return type:

INT

expression is an input value to assign the bucket number. *from* and *to* should be numeric values, date/time values, or the string which can be converted to date/time value. *from* is included in the acceptable range, but *to* is beyond the range.

For example, WIDTH_BUCKET (score, 80, 50, 3) returns

- 0 when the score is larger than 80,
- 1 for [80, 70),
- 2 for [70, 60),
- 3 for [60, 50),
- and 4 when the score is 50 or smaller.

The following example divides the range equal to 80 or smaller and larger than 50 into the score range that has the identical score range from 1 to 3. If any score is beyond the range, 0 is given for the score larger than 80 and 4 is given for the score of 50 or smaller than 50.

```
CREATE TABLE t_score (name VARCHAR(10), score INT);
INSERT INTO t_score VALUES
    ('Amie', 60),
    ('Jane', 80),
    ('Lora', 60),
    ('James', 75),
    ('Peter', 70),
    ('Tom', 50),
    ('Ralph', 99),
    ('David', 55);

SELECT name, score, WIDTH_BUCKET (score, 80, 50, 3) grade
FROM t_score
ORDER BY grade ASC, score DESC;
```

```
  name                          score        grade
================================================
  'Ralph'                          99            0
  'Jane'                           80            1
  'James'                          75            1
  'Peter'                          70            2
  'Amie'                           60            3
  'Lora'                           60            3
  'David'                          55            3
  'Tom'                            50            4
```

In the following example, **WIDTH_BUCKET** function evenly divides the birthdate range into buckets and assigns the bucket number based on the range. It divides the range of eight customers from '1950-01-01' to '1999-12-31' into five buckets based on their dates of birth. If the birthdate value is beyond the range, 0 or 6 (*num_buckets* + 1) is returned.

```
CREATE TABLE t_customer (name VARCHAR(10), birthdate DATE);
INSERT INTO t_customer VALUES
    ('Amie', date'1978-03-18'),
    ('Jane', date'1983-05-12'),
    ('Lora', date'1987-03-26'),
    ('James', date'1948-12-28'),
    ('Peter', date'1988-10-25'),
    ('Tom', date'1980-07-28'),
    ('Ralph', date'1995-03-17'),
    ('David', date'1986-07-28');

SELECT name, birthdate, WIDTH_BUCKET (birthdate, date'1950-01-01',
date'2000-1-1', 5) age_group
FROM t_customer
ORDER BY birthdate;
```

```
  name                  birthdate     age_group
=============================================
  'James'               12/28/1948            0
  'Amie'                03/18/1978            4
  'Tom'                 07/28/1980            4
  'Jane'                05/12/1983            5
  'David'               07/28/1986            5
  'Lora'                03/26/1987            5
  'Peter'               10/25/1988            5
  'Ralph'               03/17/1995            6
```

Date/Time Functions and Operators

Contents

Date/Time Functions and Operators 490

- ADDDATE, DATE_ADD 493
- ADDTIME 497
- ADD_MONTHS 498
- CURDATE, CURRENT_DATE 500
- CURRENT_DATETIME, NOW 501

- CURTIME, CURRENT_TIME 503
- CURRENT_TIMESTAMP, LOCALTIME, LOCALTIMESTAMP 504
- DATE 146
- DATEDIFF 506
- DATE_SUB, SUBDATE 507
- DAY, DAYOFMONTH 509
- DAYOFWEEK 510
- DAYOFYEAR 511
- EXTRACT 512
- FROM_DAYS 514
- FROM_TZ 515
- FROM_UNIXTIME 516
- HOUR 517
- LAST_DAY 518
- MAKEDATE 519
- MAKETIME 520
- MINUTE 521
- MONTH 522
- MONTHS_BETWEEN 523
- NEW_TIME 524
- QUARTER 525

- ROUND 483
- SEC_TO_TIME 529
- SECOND 530
- SYS_DATE, SYSDATE 531
- SYS_DATETIME, SYSDATETIME 532
- SYS_TIME, SYSTIME 534
- SYS_TIMESTAMP, SYSTIMESTAMP 535
- TIME 146
- TIME_TO_SEC 538
- TIMEDIFF 539
- TIMESTAMP 147
- TO_DAYS 541
- TRUNC 543
- TZ_OFFSET 545
- UNIX_TIMESTAMP 545
- UTC_DATE 547
- UTC_TIME 547
- WEEK 547
- WEEKDAY 550
- YEAR 551

ADDDATE, DATE_ADD

ADDDATE(*date*, *INTERVAL expr unit*)

ADDDATE(*date*, *days*)

DATE_ADD(*date*, *INTERVAL expr unit*)

The **ADDDATE** function performs an addition or subtraction operation on a specific **DATE** value; **ADDDATE** and **DATE_ADD** are used interchangeably. The return value is a **DATE** or **DATETIME** type. The **DATETIME** type is returned in the following cases.

- The first argument is a **DATETIME** or **TIMESTAMP** type
- The first argument is a **DATE** type and the unit of **INTERVAL** value specified is less than the unit of day

Therefore, to return value of **DATETIME** type, you should convert the value of first argument by using the **CAST** function. Even though the date resulting from the operation exceeds the last day of the month, the function returns a valid **DATE** value considering the last date of the month.

If every input argument value of date and time is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

If the calculated value is between '0000-00-00 00:00:00' and '0001-01-01 00:00:00', a value having 0 for all arguments is returned in **DATE** or **DATETIME** type. Note that operation in JDBC program is determined by the configuration of zeroDateTimeBehavior, connection URL property. For more information about JDBC connection URL, please see Configuration Connection.

Parameters:

- **date** – It is a **DATE**, **TIMETIME**, or **TIMESTAMP** expression that represents the start date. If an invalid **DATE** value such as '2006-07-00' is specified, an error is returned.
- **expr** – It represents the interval value to be added to the start date. If a negative number is specified next to the **INTERVAL** keyword, the interval value is subtracted from the start date.
- **unit** – It represents the unit of the interval value specified in the *expr* expression. See the following table to specify the format for the interpretation of the interval value. If the value of *expr* unit is less than the number requested in the *unit*, it is specified from the smallest unit. For

example, if it is HOUR_SECOND, three values such as 'HOURS:MINUTES:SECONDS' are required. In the case, if only two values such as "1:1" are given, it is regarded as 'MINUTES:SECONDS'.

Return type:

DATE or **DATETIME**

Format of expr for unit

unit Value	expr Format	Example
MILLISECOND	MILLISECONDS	ADDDATE(SYSDATE, INTERVAL 123 MILLISECOND)
SECOND	SECONDS	ADDDATE(SYSDATE, INTERVAL 123 SECOND)
MINUTE	MINUTES	ADDDATE(SYSDATE, INTERVAL 123 MINUTE)
HOUR	HOURS	ADDDATE(SYSDATE, INTERVAL 123 HOUR)
DAY	DAYS	ADDDATE(SYSDATE, INTERVAL 123 DAY)
WEEK	WEEKS	ADDDATE(SYSDATE, INTERVAL 123 WEEK)
MONTH	MONTHS	ADDDATE(SYSDATE, INTERVAL 12 MONTH)
QUARTER	QUARTERS	ADDDATE(SYSDATE, INTERVAL 12 QUARTER)

unit Value	expr Format	Example
YEAR	YEARS	ADDDATE(SYSDATE, INTERVAL 12 YEAR)
SECOND_MILLISECOND	‘SECONDS.MILLISECONDS’	ADDDATE(SYSDATE, INTERVAL ‘12.123’ SECOND_MILLISECOND)
MINUTE_MILLISECOND	‘MINUTES:SECONDS.MILLISECONDS’	ADDDATE(SYSDATE, INTERVAL ‘12:12.123’ MINUTE_MILLISECOND)
MINUTE_SECOND	‘MINUTES:SECONDS’	ADDDATE(SYSDATE, INTERVAL ‘12:12’ MINUTE_SECOND)
HOUR_MILLISECOND	‘HOURS:MINUTES:SECONDS.MILLISECONDS’	ADDDATE(SYSDATE, INTERVAL ‘12:12:12.123’ HOUR_MILLISECOND)
HOUR_SECOND	‘HOURS:MINUTES:SECONDS’	ADDDATE(SYSDATE, INTERVAL ‘12:12:12’ HOUR_SECOND)
HOUR_MINUTE	‘HOURS:MINUTES’	ADDDATE(SYSDATE, INTERVAL ‘12:12’ HOUR_MINUTE)
DAY_MILLISECOND	‘DAYS HOURS:MINUTES:SECONDS.MILLISECONDS’	ADDDATE(SYSDATE, INTERVAL ‘12 12:12:12.123’ DAY_MILLISECOND)

unit Value	expr Format	Example
DAY_SECOND	'DAYS HOURS:MINUTES:SECONDS'	ADDDATE(SYSDATE, INTERVAL '12 12:12:12' DAY_SECOND)
DAY_MINUTE	'DAYS HOURS:MINUTES'	ADDDATE(SYSDATE, INTERVAL '12 12:12' DAY_MINUTE)
DAY_HOUR	'DAYS HOURS'	ADDDATE(SYSDATE, INTERVAL '12 12' DAY_HOUR)
YEAR_MONTH	'YEARS-MONTHS'	ADDDATE(SYSDATE, INTERVAL '12-13' YEAR_MONTH)

```
SELECT SYSDATE, ADDDATE(SYSDATE,INTERVAL 24 HOUR), ADDDATE(SYSDATE,
1);
```

```
03/30/2010  12:00:00.000 AM 03/31/2010              03/31/2010
```

```
--it subtracts days when argument < 0
SELECT SYSDATE, ADDDATE(SYSDATE,INTERVAL -24 HOUR), ADDDATE(SYSDATE,
-1);
```

```
03/30/2010  12:00:00.000 AM 03/29/2010              03/29/2010
```

```
--when expr is not fully specified for unit
SELECT SYS_DATETIME, ADDDATE(SYS_DATETIME, INTERVAL '1:20'
HOUR_SECOND);
```

```
06:18:24.149 PM 06/28/2010     06:19:44.149 PM 06/28/2010
```

```
SELECT ADDDATE('0000-00-00', 1 );
```

```
ERROR: Conversion error in date format.
```

```
SELECT ADDDATE('0001-01-01 00:00:00', -1);
```

```
'12:00:00.000 AM 00/00/0000'
```

ADDTIME

ADDTIME(*expr1*, *expr2*)

The **ADDTIME** function adds or subtracts a value of specific time. The first argument is **DATE**, **DATETIME**, **TIMESTAMP**, or **TIME** type and the second argument is **TIME**, **DATETIME**, or **TIMESTAMP** type. Time should be included in the second argument, and the date of the second argument is ignored. The return type for each argument type is follows:

First Argument Type	Second Argument Type	Return Type	Note
TIME	TIME, DATETIME, TIMESTAMP	TIME	The result value must be equal to or less than 24 hours.
DATE	TIME, DATETIME, TIMESTAMP	DATETIME	
DATETIME	TIME, DATETIME, TIMESTAMP	DATETIME	
date/time string	TIME, DATETIME, TIMESTAMP or time string	VARCHAR	The result string includes time.

Parameters:

- **expr1** – **DATE**, **DATETIME**, **TIME** or **TIMESTAMP** type
- **expr2** – **DATETIME**, **TIMESTAMP**, **TIME** type or date/time string

```
SELECT ADDTIME(datetime'2007-12-31 23:59:59', time'1:1:2');
```

```
01:01:01.000 AM 01/01/2008
```

```
SELECT ADDTIME(time'01:00:00', time'02:00:01');
```

```
03:00:01 AM
```

ADD_MONTHS

ADD_MONTHS(*date_argument*, *month*)

The **ADD_MONTHS** function adds a *month* value to the expression *date_argument* of **DATE** type, and it returns a **DATE** type value. If the day (*dd*) of the value specified as an argument exists within the month of the result value of the operation, it returns the given day (*dd*); otherwise returns the last day of the given month (*dd*). If the result value of the operation exceeds the expression range of the **DATE** type, it returns an error.

Parameters:

- **date_argument** – Specifies an expression of **DATE** type. To specify a **TIMESTAMP** or **DATETIME** value, an explicit casting to **DATE** type is required. If the value is **NULL**, **NULL** is returned.
- **month** – Specifies the number of the months to be added to the *date_argument*. Both positive and negative values can be specified. If the given value is not an integer type, conversion to an integer type by an implicit casting (rounding to the first place after the decimal point) is performed. If the value is **NULL**, **NULL** is returned.

```
--it returns DATE type value by adding month to the first argument
SELECT ADD_MONTHS(DATE '2008-12-25', 5), ADD_MONTHS(DATE
'2008-12-25', -5);
```

```
05/25/2009                          07/25/2008
```

```
SELECT ADD_MONTHS(DATE '2008-12-31', 5.5), ADD_MONTHS(DATE
'2008-12-31', -5.5);
```

```
06/30/2009                            06/30/2008
```

```
SELECT ADD_MONTHS(CAST (SYS_DATETIME AS DATE), 5), ADD_MONTHS(CAST
(SYS_TIMESTAMP AS DATE), 5);
```

```
07/03/2010                                    07/03/2010
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT ADD_MONTHS (datetimeltz'2001-10-11 10:11:12', 1);
```

```
11/11/2001
```

```
SELECT ADD_MONTHS (datetimetz'2001-10-11 10:11:12 Europe/Paris', 1);
```

```
11/11/2001
```

```
SELECT ADD_MONTHS (timestampltz'2001-10-11 10:11:12', 1);
```

```
11/11/2001
```

```
SELECT ADD_MONTHS (timestamptz'2001-10-11 10:11:12 Europe/Paris', 1);
```

```
11/11/2001
```

CURDATE, CURRENT_DATE

CURDATE()

CURRENT_DATE()

CURRENT_DATE

CURDATE (), **CURRENT_DATE** and **CURRENT_DATE** () are used interchangeably and they return the current date of session as the **DATE** type (*MM*/*DD*/*YYYY* or *YYYY-MM-DD*). The unit is day. When the timezone of the current session is same as that of server, these functions are same as `SYS_DATE`, `SYSDATE`. Please refer `SYS_DATE`, `SYSDATE` and the following examples to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

If input every argument value of year, month, and day is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Return type:

DATE

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current date in DATE type

SELECT CURDATE(), CURRENT_DATE(), CURRENT_DATE, SYS_DATE, SYSDATE;

   CURRENT_DATE    CURRENT_DATE    CURRENT_DATE    SYS_DATE
SYS_DATE
============================================================================
===
  02/05/2016      02/05/2016      02/05/2016      02/05/2016  02/05/
2016
```

```
-- it returns the date 60 days added to the current date

SELECT CURDATE()+60;
```

```
   CURRENT_DATE +60
==================
  04/05/2016
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

SET TIME ZONE 'America/Los_Angeles';

SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that CURDATE() and SYS_DATE returns different results

SELECT CURDATE(), CURRENT_DATE(), CURRENT_DATE, SYS_DATE, SYSDATE;

   CURRENT_DATE    CURRENT_DATE    CURRENT_DATE    SYS_DATE
SYS_DATE
============================================================================
===
  02/04/2016      02/04/2016      02/04/2016      02/05/2016  02/05/
2016
```

⚠ Warning

As 10.0, **CURDATE** (), **CURRENT_DATE**, **CURRENT_DATE** () are different from **SYS_DATE** and **SYSDATE**. They are synonym for 9.x and lower.

CURRENT_DATETIME, NOW

CURRENT_DATETIME()

CURRENT_DATETIME

NOW()

CURRENT_DATETIME, **CURRENT_DATETIME** () and **NOW** () are used interchangeably, and they return the current date and time of session in **DATETIME** type. The unit is millisecond. When the timezone of the current session is same as that of server, these functions are same as `SYS_DATETIME`, `SYSDATETIME`. Please also refer `SYS_DATETIME`, `SYSDATETIME` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

Return type:

DATETIME

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current date and time in DATETIME type

SELECT NOW(), SYS_DATETIME;

   CURRENT_DATETIME               SYS_DATETIME
=================================================================
  04:05:09.292 PM 02/05/2016     04:05:09.292 PM 02/05/2016
```

```
-- it returns the timestamp value 1 hour added to the current
sys_datetime value

SELECT TO_CHAR(SYSDATETIME+3600*1000, 'YYYY-MM-DD HH24:MI');

   to_char( SYS_DATETIME +3600*1000, 'YYYY-MM-DD HH24:MI')
=======================
  '2016-02-05 17:05'
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

set time zone 'America/Los_Angeles';

SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that NOW() and SYS_DATETIME return different results

SELECT NOW(), SYS_DATETIME;

   CURRENT_DATETIME               SYS_DATETIME
=================================================================
  11:08:57.041 PM 02/04/2016     04:08:57.041 PM 02/05/2016
```

> **⚠ Warning**
>
> As 10.0, **CURRENT_DATETIME** (), **NOW** () are different from **SYS_DATEIME**, **SYSDATETIME**. They are synonym for 9.x and lower.

CURTIME, CURRENT_TIME

CURTIME()

CURRENT_TIME

CURRENT_TIME()

CURTIME (), **CURRENT_TIME** and **CURRENT_TIME** () are used interchangeably and they return the current time of session as **TIME** type (*HH:MI:SS*). The unit is second. When the timezone of the current session is same as that of server, these functions are same as `SYS_TIME`, `SYSTIME`. Please also refer `SYS_TIME`, `SYSTIME` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

Return type:

TIME

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current time in TIME type

SELECT CURTIME(), CURRENT_TIME(), CURRENT_TIME, SYS_TIME, SYSTIME;

   CURRENT_TIME    CURRENT_TIME    CURRENT_TIME    SYS_TIME
SYS_TIME
========================================================================
=====
  04:22:54 PM     04:22:54 PM     04:22:54 PM     04:22:54 PM  04:22:
54 PM
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

SET TIME ZONE 'AMERICA/LOS_ANGELES';
```

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
=============================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that CURTIME() and SYS_TIME return different results

SELECT CURTIME(), CURRENT_TIME(), CURRENT_TIME, SYS_TIME, SYSTIME;

   CURRENT_TIME     CURRENT_TIME     CURRENT_TIME     SYS_TIME
SYS_TIME
=========================================================================
======
  11:23:16 PM      11:23:16 PM      11:23:16 PM      04:23:16 PM  04:23:
16 PM
```

⚠ Warning

As 10.0, **CURTIME** (), **CURRENT_TIME** () are different from **SYS_TIME**, **SYSTIME**. They are synonym for 9.x and lower.

CURRENT_TIMESTAMP, LOCALTIME, LOCALTIMESTAMP

CURRENT_TIMESTAMP

CURRENT_TIMESTAMP()

LOCALTIME

LOCALTIME()

LOCALTIMESTAMP

LOCALTIMESTAMP()

CURRENT_TIMESTAMP, **CURRENT_TIMESTAMP** (), **LOCALTIME**, **LOCALTIME** (), **LOCALTIMESTAMP** and **LOCALTIMESTAMP** () are used interchangeably and they return the current date and time of session as **TIMESTAMP** type. The unit is second. When the timezone of the current session is same as that of server, these functions are same as `SYS_TIMESTAMP`, `SYSTIMESTAMP`. Please also refer `SYS_TIMESTAMP`, `SYSTIMESTAMP` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

Return type:

TIMESTAMP

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
=============================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current date and time in TIMESTAMP type of session
and server timezones.

SELECT LOCALTIME, SYS_TIMESTAMP;

   CURRENT_TIMESTAMP          SYS_TIMESTAMP
=======================================================
  04:34:16 PM 02/05/2016     04:34:16 PM 02/05/2016
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

SET TIME ZONE 'America/Los_Angeles';

SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
=============================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that LOCALTIME() and SYS_TIMESTAMP return different results

SELECT LOCALTIME, SYS_TIMESTAMP;

   CURRENT_TIMESTAMP          SYS_TIMESTAMP
=======================================================
  11:34:37 PM 02/04/2016     04:34:37 PM 02/05/2016
```

⚠ Warning

As 10.0, **CURRENT_TIMESTAMP**, **CURRENT_TIMESTAMP** (), **LOCALTIME**, **LOCALTIME** (), **LOCALTIMESTAMP** and **LOCALTIMESTAMP** () are different from **SYS_TIMESTAMP** (), **SYSTIMESTAMP**. They are synonym for 9.x and lower.

DATE

DATE(*date*)

The **DATE** function extracts the date part from specified argument, and returns it as '*MM*/*DD*/*YYYY*' format string. Arguments that can be specified are **DATE**, **TIMESTAMP** and **DATETIME** types. The return value is a **VARCHAR** type.

0 is not allowed in the argument value corresponding to year, month, and day; however, if 0 is inputted in every argument value corresponding to date and time, string where 0 is specified for year, month, and day is returned.

Parameters:

date – **DATE**, **TIMESTAMP** or **DATETIME** can be specified.

Return type:

STRING

```
SELECT DATE('2010-02-27 15:10:23');
```

```
'02/27/2010'
```

```
SELECT DATE(NOW());
```

```
'04/01/2010'
```

```
SELECT DATE('0000-00-00 00:00:00');
```

```
'00/00/0000'
```

DATEDIFF

DATEDIFF(*date1*, *date2*)

The **DATEDIFF** function returns the difference between two arguments as an integer representing the number of days. Arguments that can be specified are **DATE**, **TIMESTAMP** and **DATETIME** types and its return value is only **INTEGER** type.

If every input argument value of date and time is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Parameters:

date1,date2 – Specifies the types that include date (**DATE**, **TIMESTAMP** or **DATETIME**) type or string that represents the value of corresponding type. If invalid string is specified, an error is returned.

Return type:

INT

```
SELECT DATEDIFF('2010-2-28 23:59:59','2010-03-02');
```

```
-2
```

```
SELECT DATEDIFF('0000-00-00 00:00:00', '2010-2-28 23:59:59');
```

```
ERROR: Conversion error in date format.
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT IF(DATEDIFF('2002-03-03 12:00:00 AM','1990-01-01 11:59:59
PM') = DATEDIFF(timestampltz'2002-03-03 12:00:00
AM',timestampltz'1990-01-01 11:59:59 PM'),'ok','nok');
```

```
'ok'
```

DATE_SUB, SUBDATE

DATE_SUB(*date*, *INTERVAL expr unit*)

SUBDATE(*date*, *INTERVAL expr unit*)

SUBDATE(*date*, *days*)

The functions **DATE_SUB** and **SUBDATE** () are used interchangeably and they perform an addition or subtraction operation on a specific **DATE** value. The value is returned in **DATE** or **DATETIME** type. If the date resulting from the operation exceeds the last day of the month, the function returns a valid **DATE** value considering the last date of the month.

If every input argument value of date and time is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

If the calculated value is between '0000-00-00 00:00:00' and '0001-01-01 00:00:00', a value having 0 for all arguments is returned in **DATE** or **DATETIME** type. Note that operation in JDBC program is determined by the configuration of zeroDateTimeBehavior, connection URL property (see Configuration Connection for details).

param date:

It is a **DATE** or **TIMESTAMP** expression that represents the start date. If an invalid **DATE** value such as '2006-07-00' is specified, **NULL** is returned.

param expr:

It represents the interval value to be subtracted from the start date. If a negative number is specified next to the **INTERVAL** keyword, the interval value is added to the start date.

param unit:

It represents the unit of the interval value specified in the *exp* expression. To check the expr argument for the unit value, see the table of `ADDDATE()`.

rtype:

DATE or DATETIME

```
SELECT SYSDATE, SUBDATE(SYSDATE,INTERVAL 24 HOUR), SUBDATE(SYSDATE,
1);
```

```
03/30/2010  12:00:00.000 AM 03/29/2010              03/29/2010
```

```
--it adds days when argument < 0
SELECT SYSDATE, SUBDATE(SYSDATE,INTERVAL -24 HOUR), SUBDATE(SYSDATE,
-1);
```

```
03/30/2010  12:00:00.000 AM 03/31/2010              03/31/2010
```

```
SELECT SUBDATE('0000-00-00 00:00:00', -50);
```

```
ERROR: Conversion error in date format.
```

```
SELECT SUBDATE('0001-01-01 00:00:00', 10);
```

```
'12:00:00.000 AM 00/00/0000'
```

DAY, DAYOFMONTH

DAY(*date*)

DAYOFMONTH(*date*)

The function **DAY** or **DAYOFMONTH** returns day in the range of 1 to 31 from the specified parameter. You can specify the **DATE**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

0 is not allowed in the argument value corresponding to year, month, and day; however, if 0 is inputted in every argument value corresponding to year, month, and day, 0 is returned as an exception.

Parameters:

date – Date

Return type:

INT

```
SELECT DAYOFMONTH('2010-09-09');
```

```
9
```

```
SELECT DAY('2010-09-09 19:49:29');
```

```
9
```

```
SELECT DAYOFMONTH('0000-00-00 00:00:00');
```

```
0
```

DAYOFWEEK

DAYOFWEEK(*date*)

The **DAYOFWEEK** function returns a day in the range of 1 to 7 (1: Sunday, 2: Monday, ..., 7: Saturday) from the specified parameters. The day index is same as the ODBC standards. You can specify the **DATE**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

If every input argument value of year, month, and day is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Parameters:

date – Date

Return type:

INT

```
SELECT DAYOFWEEK('2010-09-09');
```

```
5
```

```
SELECT DAYOFWEEK('2010-09-09 19:49:29');
```

```
5
```

```
SELECT DAYOFWEEK('0000-00-00');
```

```
ERROR: Conversion error in date format.
```

DAYOFYEAR

DAYOFYEAR(*date*)

The **DAYOFYEAR** function returns the day of a year in the range of 1 to 366. You can specify the **DATE**, **TIMESTAMP** or **DATETIME** types; the value is returned in **INTEGER** type.

If every input argument value of year, month, and day is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Parameters:

date – Date

Return type:

INT

```
SELECT DAYOFYEAR('2010-09-09');
```

```
252
```

```
SELECT DAYOFYEAR('2010-09-09 19:49:29');
```

```
252
```

```
SELECT DAYOFYEAR('0000-00-00');
```

```
ERROR: Conversion error in date format.
```

EXTRACT

EXTRACT(*field FROM date-time_argument*)

The **EXTRACT** operator extracts the values from *date-time_argument* and then converts the value type into **INTEGER**.

0 is not allowed in the input argument value corresponding to year, month, and day; however, if 0 is inputted in every argument value corresponding to date and time, 0 is returned as an exception.

Parameters:

- **field** – Specifies a value to be extracted from date-time expression.
- **date-time_argument** – An expression that returns a value of date-time. This expression must be one of **TIME**, **DATE**, **TIMESTAMP**, or **DATETIME** types. If the value is **NULL**, **NULL** is returned.

Return type:

INT

```
SELECT EXTRACT(MONTH FROM DATETIME '2008-12-25 10:30:20.123' );
```

```
12
```

```
SELECT EXTRACT(HOUR FROM DATETIME '2008-12-25 10:30:20.123' );
```

```
10
```

```
SELECT EXTRACT(MILLISECOND FROM DATETIME '2008-12-25 10:30:20.123' );
```

```
123
```

```
SELECT EXTRACT(MONTH FROM '0000-00-00 00:00:00');
```

```
0
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT EXTRACT (MONTH FROM datetimetz'10/15/1986 5:45:15.135 am
Europe/Bucharest');
```

```
10
```

```
SELECT EXTRACT (MONTH FROM datetimeltz'10/15/1986 5:45:15.135 am
Europe/Bucharest');
```

```
10
```

```
SELECT EXTRACT (MONTH FROM timestampltz'10/15/1986 5:45:15 am Europe/
Bucharest');
```

```
10
```

```
SELECT EXTRACT (MONTH FROM timestamptz'10/15/1986 5:45:15 am Europe/
Bucharest');
```

```
10
```

FROM_DAYS

FROM_DAYS(*N*)

The **FROM_DAYS** function returns a date value in **DATE** type if **INTEGER** type is inputted as an argument.

It is not recommended to use the **FROM_DAYS** function for dates prior to the year 1582 because the function does not take dates prior to the introduction of the Gregorian Calendar into account.

If a value in the range of 0 to 3,652,424 can be inputted as an argument. If a value in the range of 0 to 365 is inputted, 0 is returned. 3,652,424, which is the maximum value, means the last day of year 9999.

Parameters:

N – Integer in the range of 0 to 3,652,424

Return type:

DATE

```
SELECT FROM_DAYS(719528);
```

```
01/01/1970
```

```
SELECT FROM_DAYS('366');
```

```
01/03/0001
```

```
SELECT FROM_DAYS(3652424);
```

```
12/31/9999
```

```
SELECT FROM_DAYS(0);
```

```
00/00/0000
```

FROM_TZ

FROM_TZ(*datetime*, *timezone_string*)

Converts date/time type without timezone as date/time type with timezone by adding timezone to DATETIME typed value. Input value's type is DATETIME, and the result value's type is DATETIMETZ.

Parameters:

- **datetime** – DATETIME
- **timezone_string** – String representing a timezone name or and offset '+05:00', 'Asia/Seoul'.

Return type:

DATETIMETZ

```
SELECT FROM_TZ(datetime '10/10/2014 00:00:00 AM', 'Europe/Vienna');
```

```
12:00:00.000 AM 10/10/2014 Europe/Vienna CEST
```

```
SELECT FROM_TZ(datetime '10/10/2014 23:59:59 PM', '+03:25:25');
```

```
11:59:59.000 PM 10/10/2014 +03:25:25
```

For timezone related description, see Date/Time Types with Timezone.

See also

`DBTIMEZONE()`, `SESSIONTIMEZONE()`, `NEW_TIME()`, `TZ_OFFSET()`

FROM_UNIXTIME

FROM_UNIXTIME(*unix_timestamp*[, *format*])

The **FROM_UNIXTIME** function returns the string of the specified format in **VARCHAR** type if the argument *format* is specified; if the argument *format* is omitted, it returns a value of **TIMESTAMP** type. Specify the arguement *unix_timestamp* as an **INTEGER** type that corresponds to the UNIX timestamp. The returned value is displayed in the current timezone.

It displays the result according to the format that you specified, and the date/time format, *format* follows the Date/Time Format 2 table of `DATE_FORMAT()`.

The relation is not one of one-to-one correspondence between **TIMESTAMP** and UNIX timestamp so if you use `UNIX_TIMESTAMP()` or **FROM_UNIXTIME** function, partial value could be lost. For details, see `UNIX_TIMESTAMP()`.

0 is not allowed in the argument value corresponding to year, month, and day; however, if 0 is inputted in every argument value corresponding to date and time, string where 0 is specified for every date and time value is returned. Note that operation in JDBC program is determined by the configuration of zeroDateTimeBehavior, connection URL property (see Configuration Connection for details).

Parameters:

- **unix_timestamp** – Positive integer
- **format** – Time format. Follows the date/time format of the `DATE_FORMAT()`.

Return type:

STRING, INT

```
SELECT FROM_UNIXTIME(1234567890);
```

```
01:31:30 AM 02/14/2009
```

```
SELECT FROM_UNIXTIME('1000000000');
```

```
04:46:40 AM 09/09/2001
```

```
SELECT FROM_UNIXTIME(1234567890,'%M %Y %W');
```

```
'February 2009 Saturday'
```

```
SELECT FROM_UNIXTIME('1234567890','%M %Y %W');
```

```
'February 2009 Saturday'
```

```
SELECT FROM_UNIXTIME(0);
```

```
12:00:00 AM 00/00/0000
```

HOUR

HOUR(*time*)

The **HOUR** function extracts the hour from the specified parameter and then returns the value in integer. The type **TIME**, **TIMESTAMP** or **DATETIME** can be specified and a value is returned in the **INTEGER** type.

Parameters:

time – Time

Return type:

INT

```
SELECT HOUR('12:34:56');
```

```
12
```

```
SELECT HOUR('2010-01-01 12:34:56');
```

```
12
```

```
SELECT HOUR(datetime'2010-01-01 12:34:56');
```

```
12
```

LAST_DAY

LAST_DAY(*date_argument*)

The **LAST_DAY** function returns the last day of the given month as **DATE** type.

If every input argument value of year, month, and day is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Parameters:

date_argument – Specifies an expression of **DATE** type. To specify a **TIMESTAMP** or **DATETIME** value, explicit casting to **DATE** is required. If the value is **NULL**, **NULL** is returned.

Return type:

DATE

```
--it returns last day of the month in DATE type
SELECT LAST_DAY(DATE '1980-02-01'), LAST_DAY(DATE '2010-02-01');
```

```
02/28/1980                    02/28/2010
```

```
--it returns last day of the month when explicitly casted to DATE
type
SELECT LAST_DAY(CAST (SYS_TIMESTAMP AS DATE)), LAST_DAY(CAST
(SYS_DATETIME AS DATE));
```

```
02/28/2010                                  02/28/2010
```

```
SELECT LAST_DAY('0000-00-00');
```

```
ERROR: Conversion error in date format.
```

MAKEDATE

MAKEDATE(*year*, *dayofyear*)

The **MAKEDATE** function returns a date from the specified parameter. You can specify an **INTEGER** type corresponding to the day of the year in the range of 1 to 9999 as an argument; the value in the range of 1/1/1 to 12/31/9999 is returned in **DATE** type. If the day of the year has passed the corresponding year, it will become the next year. For example, MAKEDATE(1999, 366) will return 2000-01-01. However, if you input a value in the range of 0 to 69 as the year, it will be processed as the year 2000-2069, if it is a value in the range of 70 to 99, it will be processed as the year 1970-1999.

If every value specified in *year* and *dayofyear* is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Parameters:

- **year** – Year in the range of 1 to 9999
- **dayofyear** – If you input a value in the range of 0 to 99 in the argument, it is handled as an exception; *dayofyear* must be equal to or less than 3,615,902 and the return value of MAKEDATE(100, 3615902) is 9999/12/31.

Return type:

DATE

```
SELECT MAKEDATE(2010,277);
```

```
10/04/2010
```

```
SELECT MAKEDATE(10,277);
```

```
10/04/2010
```

```
SELECT MAKEDATE(70,277);
```

```
10/04/1970
```

```
SELECT MAKEDATE(100,3615902);
```

```
12/31/9999
```

```
SELECT MAKEDATE(9999,365);
```

```
12/31/9999
```

```
SELECT MAKEDATE(0,0);
```

```
ERROR: Conversion error in date format.
```

MAKETIME

MAKETIME(*hour*, *min*, *sec*)

The **MAKETIME** function returns the hour from specified argument in the AM/PM format. You can specify the **INTEGER** types corresponding hours, minutes and seconds as arguments; the value is returned in **DATETIME**.

Parameters:

- **hour** – An integer representing the hours in the range of 0 to 23
- **min** – An integer representing the minutes in the range of 0 to 59
- **sec** – An integer representing the minutes in the range of 0 to 59

Return type:

DATETIME

```
SELECT MAKETIME(13,34,4);
```

```
01:34:04 PM
```

```
SELECT MAKETIME('1','34','4');
```

```
01:34:04 AM
```

```
SELECT MAKETIME(24,0,0);
```

```
ERROR: Conversion error in time format.
```

MINUTE

MINUTE(*time*)

The **MINUTE** function returns the minutes in the range of 0 to 59 from specified argument. You can specify the **TIME** , **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

Parameters:

time – Time

Return type:

INT

```
SELECT MINUTE('12:34:56');
```

```
34
```

```
SELECT MINUTE('2010-01-01 12:34:56');
```

```
34
```

```
SELECT MINUTE('2010-01-01 12:34:56.7890');
```

```
34
```

MONTH

MONTH(*date*)

The **MONTH** function returns the month in the range of 1 to 12 from specified argument. You can specify the **DATE**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

0 is not allowed in the argument value corresponding to year, month, and day; however, if 0 is inputted in every argument value corresponding to date, 0 is returned as an exception.

Parameters:

date – Date

Return type:

INT

```
SELECT MONTH('2010-01-02');
```

```
1
```

```
SELECT MONTH('2010-01-02 12:34:56');
```

```
1
```

```
SELECT MONTH('2010-01-02 12:34:56.7890');
```

```
1
```

```
SELECT MONTH('0000-00-00');
```

```
0
```

MONTHS_BETWEEN

MONTHS_BETWEEN(*date_argument*, *date_argument*)

The **MONTHS_BETWEEN** function returns the difference between the given **DATE** value. The return value is **DOUBLE** type. An integer value is returned if the two dates specified as arguments are identical or are the last day of the given month; otherwise, a value obtained by dividing the day difference by 31 is returned.

Parameters:

date_argument – Specifies an expression of **DATE** type. **TIMESTAMP** or **DATETIME** value also can be used. If the value is **NULL**, **NULL** is returned.

Return type:

DOUBLE

```
--it returns the negative months when the first argument is the
previous date
SELECT MONTHS_BETWEEN(DATE '2008-12-31', DATE '2010-6-30');
```

```
-1.800000000000000e+001
```

```
--it returns integer values when each date is the last date of the
month
SELECT MONTHS_BETWEEN(DATE '2010-6-30', DATE '2008-12-31');
```

```
1.800000000000000e+001
```

```
--it returns months between two arguments when explicitly casted to
DATE type
SELECT MONTHS_BETWEEN(CAST (SYS_TIMESTAMP AS DATE), DATE
'2008-12-25');
```

```
1.332258064516129e+001
```

```
--it returns months between two arguments when explicitly casted to
DATE type
SELECT MONTHS_BETWEEN(CAST (SYS_DATETIME AS DATE), DATE
'2008-12-25');
```

```
1.332258064516129e+001
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT MONTHS_BETWEEN(datetimetz'2001-10-11 10:11:12 +02:00',
datetimetz'2001-05-11 10:11:12 +03:00');
```

```
5.000000000000000e+00
```

NEW_TIME

NEW_TIME(*src_datetime*, *src_timezone*, *dst_timezone*)

Moves a date value from a timezone to the other timezone. The type of *src_datetime* is DATETIME or TIME, and the return value's type is the same with the *src_datetime*'s value.

Parameters:

- **src_datetime** – input value of DATETIME or TIME
- **src_timezone** – a region name of a source timezone
- **dst_timezion** – a region name of a target timezone

Return type:

the same type with a *src_datetime*s type

```
SELECT NEW_TIME(datetime '10/10/2014 10:10:10 AM', 'Europe/Vienna',
'Europe/Bucharest');
```

```
11:10:10.000 AM 10/10/2014
```

TIME type only accept an offset timezone as an input argument; a region name is not allowed.

```
SELECT NEW_TIME(time '10:10:10 PM', '+03:00', '+10:00');
```

```
05:10:10 AM
```

To see the timezone related description, see Date/Time Types with Timezone.

See also

`DBTIMEZONE()`, `SESSIONTIMEZONE()`, `FROM_TZ()`, `TZ_OFFSET()`

QUARTER

QUARTER(*date*)

The **QUARTER** function returns the quarter in the range of 1 to 4 from specified argument. You can specify the **DATE**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

Parameters:

date – Date

Return type:

INT

```
SELECT QUARTER('2010-05-05');
```

```
2
```

```
SELECT QUARTER('2010-05-05 12:34:56');
```

```
2
```

```
SELECT QUARTER('2010-05-05 12:34:56.7890');
```

```
2
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT QUARTER('2008-04-01 01:02:03 Asia/Seoul');
```

```
2
```

```
SELECT QUARTER(datetimetz'2003-12-31 01:02:03.1234 Europe/Paris');
```

```
4
```

ROUND

ROUND(*date*, *fmt*)

This function rounds date to the unit specified by the format string, *fmt*. It returns a value of DATE type.

param date:

The value of **DATE**, **TIMESTAMP** or **DATETIME**

param fmt:

Specifies the format for the truncating unit. If omitted, “dd” is default.

rtype:

DATE

The format and its unit and the return value are as follows:

Format	Unit	Return value
‘yyyy’ or ‘yy’	year	a value rounded to year
‘mm’ or ‘month’	month	a value rounded to month
‘q’	quarter	a value rounded to quarter, one of 1/1, 4/1, 7/1, 10/1
‘day’	week	a value rounded to week, this Sunday of *date* week or the next Sunday of *date* week
‘dd’	day	a value rounded to day
‘hh’	hour	a value rounded to hour
‘mi’	minute	a value rounded to minute
‘ss’	second	a value rounded to second

```
SELECT ROUND(date'2012-10-26', 'yyyy');
```

```
01/01/2013
```

```
SELECT ROUND(timestamp'2012-10-26 12:10:10', 'mm');
```

```
11/01/2012
```

```
SELECT ROUND(datetime'2012-12-26 12:10:10', 'dd');
```

```
12/27/2012
```

```
SELECT ROUND(datetime'2012-12-26 12:10:10', 'day');
```

```
12/30/2012
```

```
SELECT ROUND(datetime'2012-08-26 12:10:10', 'q');
```

```
10/01/2012
```

```
SELECT TRUNC(datetime'2012-08-26 12:10:10', 'q');
```

```
07/01/2012
```

```
SELECT ROUND(datetime'2012-02-28 23:10:00', 'hh');
```

```
02/28/2012
```

```
SELECT ROUND(datetime'2012-02-28 23:58:59', 'hh');
```

```
02/29/2012
```

```
SELECT ROUND(datetime'2012-02-28 23:59:59', 'mi');
```

```
02/29/2012
```

```
SELECT ROUND(datetime'2012-02-28 23:59:59.500', 'ss');
```

```
02/29/2012
```

In order to truncate date instead of rounding, please see TRUNC(date, fmt).

SEC_TO_TIME

SEC_TO_TIME(*second*)

The **SEC_TO_TIME** function returns the time including hours, minutes and seconds from specified argument. You can specify the **INTEGER** type in the range of 0 to 86,399; the value is returned in **TIME** type.

Parameters:

second – Seconds in the range of 0 to 86,399

Return type:

TIME

```
SELECT SEC_TO_TIME(82800);
```

```
  sec_to_time(82800)
======================
  11:00:00 PM
```

```
SELECT SEC_TO_TIME('82800.3');
```

```
  sec_to_time('82800.3')
=========================
  11:00:00 PM
```

```
SELECT SEC_TO_TIME(86399);
```

```
  sec_to_time(86399)
=====================
  11:59:59 PM
```

SECOND

SECOND(*time*)

The **SECOND** function returns the seconds in the range of 0 to 59 from specified argument. You can specify the **TIME**, **TIMESTAMP** or **DATETIME**; the value is returned in **INTEGER** type.

Parameters:

time – Time

Return type:

INT

```
SELECT SECOND('12:34:56');
```

```
   second('12:34:56')
====================
                    56
```

```
SELECT SECOND('2010-01-01 12:34:56');
```

```
   second('2010-01-01 12:34:56')
================================
                              56
```

```
SELECT SECOND('2010-01-01 12:34:56.7890');
```

```
   second('2010-01-01 12:34:56.7890')
=====================================
                                   56
```

SYS_DATE, SYSDATE

SYS_DATE

SYSDATE

SYS_DATE and **SYSDATE** are used interchangeably and they return the current date of server as the **DATE** type (*MM*/*DD*/*YYYY* or *YYYY*-*MM*-*DD*). The unit is day. When the timezone of the current session is same as that of server, these functions are same as `CURDATE()`, `CURRENT_DATE()` and `CURRENT_DATE`. Please also refer `CURDATE()`, `CURRENT_DATE()` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

If input every argument value of year, month, and day is 0, the return value is determined by the **return_null_on_function_errors** system parameter; if it is set to yes, then **NULL** is returned; if it is set to no, an error is returned. The default value is **no**.

Return type:

DATE

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
=========================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current date in DATE type

SELECT CURDATE(), CURRENT_DATE(), CURRENT_DATE, SYS_DATE, SYSDATE;

   CURRENT_DATE    CURRENT_DATE    CURRENT_DATE    SYS_DATE
SYS_DATE
=========================================================================
===
  02/05/2016      02/05/2016      02/05/2016      02/05/2016  02/05/
2016
```

```
-- it returns the date 60 days added to the current date

SELECT CURDATE()+60;

   CURRENT_DATE +60
==================
  04/05/2016
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

SET TIME ZONE 'America/Los_Angeles';

SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
=========================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that CURDATE() and SYS_DATE returns different results

SELECT CURDATE(), CURRENT_DATE(), CURRENT_DATE, SYS_DATE, SYSDATE;

   CURRENT_DATE    CURRENT_DATE    CURRENT_DATE    SYS_DATE
SYS_DATE
=========================================================================
===
  02/04/2016      02/04/2016      02/04/2016      02/05/2016  02/05/
2016
```

⚠ Warning

As 10.0, **SYS_DATE** and **SYSDATE** are different from **CURDATE** (), **CURRENT_DATE**, **CURRENT_DATE** (). They are synonym for 9.x and lower.

SYS_DATETIME, SYSDATETIME

SYS_DATETIME

SYSDATETIME

SYS_DATETIME and **SYSDATETIME** are used interchangeably, and they return the current date and time of server in **DATETIME** type. The unit is millisecond. When the timezone of the current session is same as that of server, these functions are same as `CURRENT_DATETIME()`,

`CURRENT_DATETIME`, `NOW()`. Please also refer `CURRENT_DATETIME()`, `NOW()` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

Return type:

DATETIME

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current date and time in DATETIME type

SELECT NOW(), SYS_DATETIME;

   CURRENT_DATETIME               SYS_DATETIME
=================================================================
  04:05:09.292 PM 02/05/2016     04:05:09.292 PM 02/05/2016
```

```
-- it returns the timestamp value 1 hour added to the current
sys_datetime value

SELECT TO_CHAR(SYSDATETIME+3600*1000, 'YYYY-MM-DD HH24:MI');

   to_char( SYS_DATETIME +3600*1000, 'YYYY-MM-DD HH24:MI')
======================
  '2016-02-05 17:05'
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

SET TIME ZONE 'America/Los_Angeles';

SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that NOW() and SYS_DATETIME return different results

SELECT NOW(), SYS_DATETIME;

   CURRENT_DATETIME               SYS_DATETIME
```

```
=========================================================
  11:08:57.041 PM 02/04/2016     04:08:57.041 PM 02/05/2016
```

> **⚠ Warning**
>
> As 10.0, **SYS_DATEIME**, **SYSDATETIME** are different from **CURRENT_DATETIME** (), **NOW** (). They are synonym for 9.x and lower.

SYS_TIME, SYSTIME

SYS_TIME

SYSTIME

SYS_TIME and **SYSTIME** are used interchangeably and they return the current time of server as **TIME** type (*HH:MI:SS*). The unit is second. When the timezone of the current session is same as that of server, these functions are same as `CURTIME()`, `CURRENT_TIME`, `CURRENT_TIME()`. Please also refer `CURTIME()`, `CURRENT_TIME()` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

Return type:

TIME

```
select dbtimezone(), sessiontimezone();

  dbtimezone            sessiontimezone
=============================================
  'Asia/Seoul'          'Asia/Seoul'

-- it returns the current time in TIME type

SELECT CURTIME(), CURRENT_TIME(), CURRENT_TIME, SYS_TIME, SYSTIME;

   CURRENT_TIME    CURRENT_TIME    CURRENT_TIME    SYS_TIME
SYS_TIME
=========================================================================
=====
  04:22:54 PM     04:22:54 PM     04:22:54 PM     04:22:54 PM  04:22:
54 PM
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'
```

```
SET TIME ZONE 'America/Los_Angeles';

select dbtimezone(), sessiontimezone();

  dbtimezone            sessiontimezone
=========================================
  'Asia/Seoul'          'America/Los_Angeles'

-- Note that CURTIME() and SYS_TIME return different results

SELECT CURTIME(), CURRENT_TIME(), CURRENT_TIME, SYS_TIME, SYSTIME;

   CURRENT_TIME    CURRENT_TIME    CURRENT_TIME    SYS_TIME
SYS_TIME
=========================================================================
======
  11:23:16 PM     11:23:16 PM     11:23:16 PM     04:23:16 PM  04:23:
16 PM
```

⚠ Warning

As 10.0, **SYS_TIME**, **SYSTIME** are different from **CURTIME** (), **CURRENT_TIME** (). They are synonym for 9.x and lower.

SYS_TIMESTAMP, SYSTIMESTAMP

SYS_TIMESTAMP

SYSTIMESTAMP

SYS_TIMESTAMP and **SYSTIMESTAMP** are used interchangeably and they return the current date and time of server as **TIMESTAMP** type. The unit is second. When the timezone of the current session is same as that of server, these functions are same as `CURRENT_TIMESTAMP`, `CURRENT_TIMESTAMP()`, `LOCALTIME`, `LOCALTIME()`, `LOCALTIMESTAMP`, `LOCALTIMESTAMP()`. Please also refer `CURRENT_TIMESTAMP`, `CURRENT_TIMESTAMP()`, `LOCALTIME`, `LOCALTIME()`, `LOCALTIMESTAMP`, `LOCALTIMESTAMP()` to find a difference and `DBTIMEZONE()`, `SESSIONTIMEZONE()` for details of the functions.

Return type:

TIMESTAMP

```
SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
```

```
==========================================
  'Asia/Seoul'           'Asia/Seoul'

-- it returns the current date and time in TIMESTAMP type of session
and server timezones.

SELECT LOCALTIME, SYS_TIMESTAMP;

   CURRENT_TIMESTAMP          SYS_TIMESTAMP
=====================================================
  04:34:16 PM 02/05/2016     04:34:16 PM 02/05/2016
```

```
-- change session time from 'Asia/Seoul' to 'America/Los_Angeles'

SET TIME ZONE 'America/Los_Angeles';

SELECT DBTIMEZONE(), SESSIONTIMEZONE();

  dbtimezone            sessiontimezone
============================================
  'Asia/Seoul'           'America/Los_Angeles'

-- Note that LOCALTIME() and SYS_TIMESTAMP return different results

SELECT LOCALTIME, SYS_TIMESTAMP;

   CURRENT_TIMESTAMP          SYS_TIMESTAMP
=====================================================
  11:34:37 PM 02/04/2016     04:34:37 PM 02/05/2016
```

⚠ Warning

As 10.0, **SYS_TIMESTAMP** (), **SYSTIMESTAMP** are different from **CURRENT_TIMESTAMP**, **CURRENT_TIMESTAMP** (), **LOCALTIME**, **LOCALTIME** (), **LOCALTIMESTAMP** and **LOCALTIMESTAMP** (). They are synonym for 9.x and lower.

TIME

TIME(*time*)

The **TIME** function extracts the time part from specified argument and returns the **VARCHAR** type string in the 'HH:MI:SS' format. You can specify the **TIME**, **TIMESTAMP** and **DATETIME** types.

Parameters:

time – Time

Return type:

STRING

```
SELECT TIME('12:34:56');
```

```
   time('12:34:56')
======================
  '12:34:56'
```

```
SELECT TIME('2010-01-01 12:34:56');
```

```
   time('2010-01-01 12:34:56')
======================
  '12:34:56'
```

```
SELECT TIME(datetime'2010-01-01 12:34:56');
```

```
   time(datetime '2010-01-01 12:34:56')
======================
  '12:34:56'
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT TIME(datetimetz'1996-02-03 02:03:04 AM America/Lima PET');
```

```
'02:03:04'
```

```
SELECT TIME(datetimeltz'1996-02-03 02:03:04 AM America/Lima PET');
```

```
'16:03:04'
```

```
SELECT TIME(datetimeltz'2000-12-31 17:34:23.1234 -05:00');
```

```
'07:34:23.123'
```

```
SELECT TIME(datetimetz'2000-12-31 17:34:23.1234 -05:00');
```

```
'17:34:23.123'
```

TIME_TO_SEC

TIME_TO_SEC(*time*)

The **TIME_TO_SEC** function returns the seconds in the range of 0 to 86,399 from specified argument. You can specify the **TIME**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

Parameters:

time – Time

Return type:

INT

```
SELECT TIME_TO_SEC('23:00:00');
```

```
82800
```

```
SELECT TIME_TO_SEC('2010-10-04 23:00:00');
```

```
82800
```

```
SELECT TIME_TO_SEC('2010-10-04 23:00:00.1234');
```

```
82800
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT TIME_TO_SEC(datetimeltz'1996-02-03 02:03:04 AM America/Lima
PET');

57784
```

```
SELECT TIME_TO_SEC(datetimetz'1996-02-03 02:03:04 AM America/Lima
PET');

7384
```

TIMEDIFF

TIMEDIFF(*expr1*, *expr2*)

The **TIMEDIFF** function returns the time difference between the two specified time arguments. You can enter a date/time type, the **TIME**, **DATE**, **TIMESTAMP** or **DATETIME** type and the data types of the two arguments must be identical. The **TIME** will be returned and the time difference between the two arguments must be in the range of 00:00:00 -23:59:59. If it exceeds the range, an error will be returned.

Parameters:

expr2 (*expr1,*) – Time. The data types of the two arguments must be identical.

Return type:

TIME

```
SELECT TIMEDIFF(time '17:18:19', time '12:05:52');
```

```
05:12:27 AM
```

```
SELECT TIMEDIFF('17:18:19','12:05:52');
```

```
05:12:27 AM
```

```
SELECT TIMEDIFF('2010-01-01 06:53:45', '2010-01-01 03:04:05');
```

```
03:49:40 AM
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT TIMEDIFF (datetimeltz'2013-10-28 03:11:12 AM Asia/Seoul',
datetimeltz'2013-10-27 04:11:12 AM Asia/Seoul');
```

```
11:00:00 PM
```

TIMESTAMP

TIMESTAMP(*date*[, *time*])

The **TIMESTAMP** function converts a **DATE** or **TIMESTAMP** type expression to **DATETIME** type.

If the **DATE** format string ('*YYYY-MM-DD*' or '*MM/DD/YYYY*') or **TIMESTAMP** format string ('*YYYY-MM-DD HH:MI:SS*' or '*HH:MI:SS MM/DD/ YYYY*') is specified as the first argument, the function returns it as **DATETIME**.

If the **TIME** format string ('*HH:MI:SS*.*FF*') is specified as the second, the function adds it to the first argument and returns the result as a **DATETIME** type. If the second argument is not specified, **12:00:00.000 AM** is specified by default.

Parameters:

- **date** – The format strings can be specified as follows: '*YYYY-MM-DD*', '*MM/DD/YYYY*', '*YYYY-MM-DD HH:MI:SS*.*FF*', '*HH:MI:SS*.*FF* *MM/DD/YYYY*'.
- **time** – The format string can be specified as follows: '*HH:MI:SS*[.*FF*]'.

Return type:

DATETIME

```
SELECT TIMESTAMP('2009-12-31'), TIMESTAMP('2009-12-31','12:00:00');
```

```
12:00:00.000 AM 12/31/2009     12:00:00.000 PM 12/31/2009
```

```
SELECT TIMESTAMP('2010-12-31 12:00:00','12:00:00');
```

```
12:00:00.000 AM 01/01/2011
```

```
SELECT TIMESTAMP('13:10:30 12/25/2008');
```

```
01:10:30.000 PM 12/25/2008
```

The following are examples of using timezone type values. For timezone related description, see Date/Time Types with Timezone.

```
SELECT TIMESTAMP(datetimetz'2010-12-31 12:00:00 America/New_York',
'03:00');
```

```
03:00:00.000 PM 12/31/2010
```

```
SELECT TIMESTAMP(datetimeltz'2010-12-31 12:00:00 America/New_York',
'03:00');
```

```
05:00:00.000 AM 01/01/2011
```

TO_DAYS

TO_DAYS(*date*)

The **TO_DAYS** function returns the number of days after year 0 in the rage of 366 to 3652424 from specified argument. You can specify **DATE** type; the value is returned in **INTEGER** type.

It is not recommended to use the **TO_DAYS** function for dates prior to the year 1582, as the function does not take dates prior to the introduction of the Gregorian Calendar into account.

Parameters:

date – Date

Return type:

INT

```
SELECT TO_DAYS('2010-10-04');
```

```
   to_days('2010-10-04')
=========================
                  734414
```

```
SELECT TO_DAYS('2010-10-04 12:34:56');
```

```
   to_days('2010-10-04 12:34:56')
=================================
                          734414
```

```
SELECT TO_DAYS('2010-10-04 12:34:56.7890');
```

```
   to_days('2010-10-04 12:34:56.7890')
======================================
                             734414
```

```
SELECT TO_DAYS('1-1-1');
```

```
   to_days('1-1-1')
====================
                 366
```

```
SELECT TO_DAYS('9999-12-31');
```

```
   to_days('9999-12-31')
=========================
                 3652424
```

TRUNC

TRUNC(*date*[, *fmt*])

This function truncates date to the unit specified by the format string, *fmt*. It returns a value of DATE type.

param date:

The value of **DATE**, **TIMESTAMP** or **DATETIME**

param fmt:

Specifies the format for the truncating unit. If omitted, “dd” is default.

rtype:

DATE

The format and its unit and the return value are as follows:

Format	Unit	Return value
‘yyyy’ or ‘yy’	year	the same year with Jan. 1st
‘mm’ or ‘month’	month	the same month with 1st
‘q’	quarter	

Format	Unit	Return value
		the same quarter with one of Jan. 1st, Apr. 1st, Jul. 1st, Oct. 1st
‘day’	week	Sunday of the same week(starting date of the week including *date*)
‘dd’	day	the same date with *date*

```
SELECT TRUNC(date'2012-12-26', 'yyyy');
```

```
01/01/2012
```

```
SELECT TRUNC(timestamp'2012-12-26 12:10:10', 'mm');
```

```
12/01/2012
```

```
SELECT TRUNC(datetime'2012-12-26 12:10:10', 'q');
```

```
10/01/2012
```

```
SELECT TRUNC(datetime'2012-12-26 12:10:10', 'dd');
```

```
12/26/2012
```

```
-- It returns the date of Sunday of the week which includes
date'2012-12-26'
SELECT TRUNC(datetime'2012-12-26 12:10:10', 'day');
```

```
12/23/2012
```

In order to round date instead of truncation, please see ROUND(date, fmt).

TZ_OFFSET

TZ_OFFSET(*timezone_string*)

This returns a timezone offset from a timezone offset or timezone region name (e.g. '-05:00', or 'Europe/Vienna').

Parameters:

timezone_string – timezone offset of timezone region name.

Return type:

STRING

```
SELECT TZ_OFFSET('+05:00');
```

```
'+05:00'
```

```
SELECT TZ_OFFSET('Asia/Seoul');
```

```
'+09:00'
```

For timezone related description, see Date/Time Types with Timezone.

See also

`DBTIMEZONE()`, `SESSIONTIMEZONE()`, `FROM_TZ()`, `NEW_TIME()`

UNIX_TIMESTAMP

UNIX_TIMESTAMP([*date*])

The argument of the **UNIX_TIMESTAMP** function can be omitted. If it is omitted, the function returns the interval between '1970-01-01 00:00:00' UTC and the current system date/time in seconds as **INTEGER** type. If the date argument is specified, the function returns the interval between '1970-01-01 00:00:00' UTC and the specified date/time in seconds.

0 is not allowed in the argument value corresponding to year, month, and day; however, if 0 is given in every argument value corresponding to date and time, 0 is returned as an exception.

Argument of DATETIME type is considered in session timezone.

Parameters:

date – **DATE** type, **TIMESTAMP** type, **TIMESTAMPTZ** type, **TIMESTAMPLTZ** type, **DATETIME** type, **DATETIMETZ** type, **DATETIMELTZ** type, **DATE** format string ('*YYYY-MM-DD*' or '*MM/DD/YYYY*'), **TIMESTAMP** format string ('*YYYY-MM-DD HH:MI:SS*', '*HH:MI:SS MM/DD/YYYY*') or '*YYYYMMDD*' format string can be specified.

Return type:

INT

```
SELECT UNIX_TIMESTAMP('1970-01-02'), UNIX_TIMESTAMP();
```

```
   unix_timestamp('1970-01-02')   unix_timestamp()
================================================
                          54000         1270196737
```

```
SELECT UNIX_TIMESTAMP ('0000-00-00 00:00:00');
```

```
   unix_timestamp('0000-00-00 00:00:00')
========================================
                                      0
```

```
 -- when used without argument, it returns the exact value at the
moment of execution of each occurence
 SELECT  UNIX_TIMESTAMP(), SLEEP(1), UNIX_TIMESTAMP();

   unix_timestamp()     sleep(1)   unix_timestamp()
```

```
==================================================
         1454661297            0         1454661298
```

UTC_DATE

UTC_DATE()

The **UTC_DATE** function returns the UTC date in 'YYYY-MM-DD' format.

Return type:

STRING

```
SELECT UTC_DATE();
```

```
  utc_date()
==============
  01/12/2011
```

UTC_TIME

UTC_TIME()

The **UTC_TIME** function returns the UTC time in 'HH:MI:SS' format.

Return type:

STRING

```
SELECT UTC_TIME();
```

```
  utc_time()
==============
  10:35:52 AM
```

WEEK

WEEK(*date*[, *mode*])

The **WEEK** function returns the week in the range of 0 to 53 from specified argument. You can specify the **DATE**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

Parameters:

- **date** – Date
- **mode** – Value in the range of 0 to 7

Return type:

INT

You can omit the second argument, *mode* and must input a value in the range of 0 to 7. You can set that a week starts from Sunday or Monday and the range of the return value is from 0 to 53 or 1 to 53 with this value. If you omit the *mode*, the system parameter, **default_week_format** value(default: 0) will be used. The *mode* value means as follows:

mode	Start Day of the Week	Range	The First Week of the Year
0	Sunday	0~53	The first week that Sunday is included in the year
1	Monday	0~53	The first week that more than three days are included in the year
2	Sunday	1~53	The first week in the year that includes a Sunday
3	Monday	1~53	The first week in the year that includes more than three days

mode	Start Day of the Week	Range	The First Week of the Year
4	Sunday	0~53	The first week in the year that includes more than three days
5	Monday	0~53	The first week in the year that includes Monday
6	Sunday	1~53	The first week in the year that includes more than three days
7	Monday	1~53	The first week in the year that includes Monday

If the *mode* value is one of 0, 1, 4 or 5, and the date corresponds to the last week of the previous year, the **WEEK** function will return 0. The purpose is to see what nth of the year the week is so it returns 0 for the 52th week of the year 1999.

```
SELECT YEAR('2000-01-01'), WEEK('2000-01-01',0);
```

```
   year('2000-01-01')   week('2000-01-01', 0)
=============================================
                 2000                        0
```

To see what n-th the week is based on the year including the start day of the week, use 0, 2, 5 or 7 as the *mode* value.

```
SELECT WEEK('2000-01-01',2);
```

```
    week('2000-01-01', 2)
========================
                      52
```

```
SELECT WEEK('2010-04-05');
```

```
   week('2010-04-05', 0)
========================
                      14
```

```
SELECT WEEK('2010-04-05 12:34:56',2);
```

```
   week('2010-04-05 12:34:56',2)
================================
                              14
```

```
SELECT WEEK('2010-04-05 12:34:56.7890',4);
```

```
   week('2010-04-05 12:34:56.7890',4)
=====================================
                                 14
```

WEEKDAY

WEEKDAY(*date*)

The **WEEKDAY** function returns the day of week in the range of 0 to 6 (0: Monday, 1: Tuesday, ..., 6: Sunday) from the specified parameter. You can specify **DATE**, **TIMESTAMP**, **DATETIME** types as parameters and an **INTEGER** type will be returned.

Parameters:

date – Date

Return type:

INT

```
SELECT WEEKDAY('2010-09-09');
```

```
   weekday('2010-09-09')
=========================
                        3
```

```
SELECT WEEKDAY('2010-09-09 13:16:00');
```

```
   weekday('2010-09-09 13:16:00')
=================================
                              3
```

YEAR

YEAR(*date*)

The **YEAR** function returns the year in the range of 1 to 9,999 from the specified parameter. You can specify **DATE**, **TIMESTAMP** or **DATETIME** type; the value is returned in **INTEGER** type.

Parameters:

date – Date

Return type:

INT

```
SELECT YEAR('2010-10-04');
```

```
   year('2010-10-04')
======================
                  2010
```

```
SELECT YEAR('2010-10-04 12:34:56');
```

```
   year('2010-10-04 12:34:56')
==============================
                          2010
```

```
SELECT YEAR('2010-10-04 12:34:56.7890');
```

```
   year('2010-10-04 12:34:56.7890')
===================================
                               2010
```

JSON functions

Introduction to JSON functions

The functions described in this section perform operations on JSON data. All supported JSON functions can be found in next table:

->	JSON_CONTAINS_PATH	JSON_MERGE_PATCH	JSON_REPLACE
->>	JSON_DEPTH	JSON_MERGE_PRESERVE	JSON_SEARCH
JSON_ARRAY	JSON_EXTRACT	JSON_OBJECT	JSON_SET
JSON_ARRAYAGG	JSON_INSERT	JSON_OBJECTAGG	JSON_TABLE
JSON_ARRAY_APPEND	JSON_KEYS	JSON_PRETTY	JSON_TYPE
JSON_ARRAY_INSERT	JSON_LENGTH	JSON_QUOTE	JSON_UNQUOTE
JSON_CONTAINS	JSON_MERGE	JSON_REMOVE	JSON_VALID

They have in common several types of input arguments:

- *json_doc*: a JSON or string that is parsed as JSON
- *val*: a JSON or a value that can be interpreted as one of supported JSON scalar types
- *json key*: a string as key name
- *json path/pointer*: a string that follows the rules explained in JSON Paths and JSON Pointers.

Note

UTF8 is expected to be the codeset of JSON functions string arguments. Inputs with different codesets are implicitly converted to UTF8. One consequence is that searching a case insensitive collation string with a codeset other than UTF8 may not provide expected results.

The next table shows the differences between *json_doc* and *val* when accepting input arguments:

Input type	*json_doc*	*val*
JSON	Input is unchanged	Input is unchanged
String	JSON value is parsed from input	Input is converted to JSON STRING
Short, Integer	Conversion error	Input is converted to JSON INTEGER
Bigint	Conversion error	Input is converted to JSON BIGINT
Float, Double, Numeric	Conversion error	Input is converted to JSON DOUBLE
NULL	NULL	Input is converted to JSON_NULL

Input type	*json_doc*	*val*
Other	Conversion error	Conversion error

JSON_ARRAY

JSON_ARRAY(*[val1 [, val2] ...]*)

The **JSON_ARRAY** function returns a json array containing the given list (possibly empty) of values.

```
SELECT JSON_ARRAY();
```

```
  json_array()
======================
  []
```

```
SELECT JSON_ARRAY(1, '1', json '{"a":4}', json '[1,2,3]');
```

```
  json_array(1, '1', json '{"a":4}', json '[1,2,3]')
======================
  [1,"1",{"a":4},[1,2,3]]
```

JSON_OBJECT

JSON_OBJECT(*[key1, val1 [, key2, val2] ...]*)

The **JSON_OBJECT** function returns a json object containing the given list (possibly empty) of key-value pairs.

```
SELECT JSON_OBJECT();
```

```
  json_object()
======================
  {}
```

```
SELECT JSON_OBJECT('a', 1, 'b', '1', 'c', json '{"a":4}', 'd', json
'[1,2,3]');
```

```
  json_object('a', 1, 'b', '1', 'c', json '{"a":4}', 'd', json
'[1,2,3]')
======================
  {"a":1,"b":"1","c":{"a":4},"d":[1,2,3]}
```

JSON_KEYS

JSON_KEYS(*json_doc*[, *json path*])

The **JSON_KEYS** function returns a json array of all the object keys of the json object at the given path. Json null is returned if the path addresses a json element that is not a json object. If json path argument is missing, the keys are gathered from json root element. An error occurs if *json path* does not exist. Returns **NULL** if *json_doc* argument is **NULL**.

```
SELECT JSON_KEYS('{}');
```

```
  json_keys('{}')
======================
  []
```

```
SELECT JSON_KEYS('"non-object"');
```

```
  json_keys('"non-object"')
======================
  null
```

```
SELECT JSON_KEYS('{"a":1, "b":2, "c":{"d":1}}');
```

```
  json_keys('{"a":1, "b":2, "c":{"d":1}}')
======================
  ["a","b","c"]
```

JSON_DEPTH

JSON_DEPTH(*json_doc*)

The **JSON_DEPTH** function returns the maximum depth of the json. Depth count starts at 1. The depth level is increased by one by non-empty json arrays or by non-empty json objects. Returns **NULL** if argument is **NULL**.

```
SELECT JSON_DEPTH('"scalar"');
```

```
  json_depth('"scalar"')
======================
  1
```

```
SELECT JSON_DEPTH('[{"a":4}, 2]');
```

```
  json_depth('[{"a":4}, 2]')
======================
  3
```

Example of a deeper json:

```
SELECT JSON_DEPTH('[{"a":[1,2,3,{"k":[4,5]}]},2,3,4,5,6,7]');
```

```
  json_depth('[{"a":[1,2,3,{"k":[4,5]}]},2,3,4,5,6,7]')
======================
  6
```

JSON_LENGTH

JSON_LENGTH(*json_doc*[, *json path*])

The **JSON_LENGTH** function returns the length of the json element at the given path. If no path argument is given, the returned value is the length of the root json element. Returns **NULL** if any argument is **NULL** or if no element exists at the given path.

```
SELECT JSON_LENGTH('"scalar"');
```

```
  json_length('"scalar"')
======================
  1
```

```
SELECT JSON_LENGTH('[{"a":4}, 2]', '$.a');
```

```
  json_length('[{"a":4}, 2]', '$.a')
======================
  NULL
```

```
SELECT JSON_LENGTH('[2, {"a":4, "b":4, "c":4}]', '$[1]');
```

```
  json_length('[2, {"a":4, "b":4, "c":4}]', '$[1]')
======================
  3
```

```
SELECT JSON_LENGTH('[{"a":[1,2,3,{"k":[4,5,6,7,8]}]},2]');
```

```
  json_length('[{"a":[1,2,3,{"k":[4,5,6,7,8]}]},2]')
======================
  2
```

JSON_VALID

JSON_VALID(*val*)

The **JSON_VALID** function returns 1 if the given *val* argument is a valid json_doc, 0 otherwise. Returns **NULL** if argument is **NULL**.

```
SELECT JSON_VALID('[{"a":4}, 2]');
1
SELECT JSON_VALID('{"wrong json object":');
0
```

JSON_TYPE

JSON_TYPE(*json_doc*)

The **JSON_TYPE** function returns the type of the *json_doc* argument as a string.

```
SELECT JSON_TYPE ('[{"a":4}, 2]');
'JSON_ARRAY'
SELECT JSON_TYPE ('{"a":4}');
'JSON_OBJECT'
SELECT JSON_TYPE ('"aaa"');
'STRING'
```

JSON_QUOTE

JSON_QUOTE(*str*)

Escapes quotes and special characters and surrounds the resulting string in quotes. Returns result as a json_string. Returns **NULL** if *str* argument is **NULL**.

```
SELECT JSON_QUOTE ('simple');
```

```
  json_unquote('simple')
======================
  '"simple"'
```

```
SELECT JSON_QUOTE ('"');
```

```
  json_unquote('"')
======================
  '"\""'
```

JSON_UNQUOTE

JSON_UNQUOTE(*json_doc*)

Unquotes a json_value's json string and returns the resulting string. Returns **NULL** if *json_doc* argument is **NULL**.

```
SELECT JSON_UNQUOTE ('"\\u0032"');
```

```
  json_unquote('"\u0032"')
======================
  '2'
```

```
SELECT JSON_UNQUOTE ('"\\""');
```

```
  json_unquote('"\""')
======================
  '"'
```

JSON_PRETTY

JSON_PRETTY(*json_doc*)

Returns a string containing the *json_doc* pretty-printed. Returns **NULL** if *json_doc* argument is **NULL**.

```
SELECT JSON_PRETTY('[{"a":"val1", "b":"val2", "c": [1, "elem2", 3,
4, {"key":"val"}]}]');
```

```
  json_pretty('[{"a":"val1", "b":"val2", "c": [1, "elem2", 3, 4,
{"key":"val"}]}]')
======================
  '[
  {
    "a": "val1",
    "b": "val2",
    "c": [
      1,
      "elem2",
      3,
      4,
      {
        "key": "val"
      }
    ]
  }
]'
```

JSON_SEARCH

JSON_SEARCH(*json_doc, one/all, search_str [, escape_char [, json path] ...]*)

Returns a json array of json paths or a single json path which contain json strings matching the given *search_str*. The matching is performed by applying the **LIKE** operator on internal json strings and *search_str*. Same rules apply for the *escape_char* and *search_str* of **JSON_SEARCH** as for their counter-parts from the **LIKE** operator. For further description of **LIKE**-related arguments rules refer to LIKE.

Using 'one' as one/all argument will cause the **JSON_SEARCH** to stop after the first match is found. On the other hand, 'all' will force **JSON_SEARCH** to gather all paths matching the given *search_str*.

The given json paths determine filters on the returned paths, the resulting json paths's prefixes need to match at least one given json path argument. If no json path argument is given, **JSON_SEARCH** will execute the search starting from the root element.

```
SELECT JSON_SEARCH('{"a":["a","b"],"b":"a","c":"a"}', 'one', 'a');
```

```
  json_search('{"a":["a","b"],"b":"a","c":"a"}', 'one', 'a')
======================
  "$.a[0]"
```

```
SELECT JSON_SEARCH('{"a":["a","b"],"b":"a","c":"a"}', 'all', 'a');
```

```
  json_search('{"a":["a","b"],"b":"a","c":"a"}', 'all', 'a')
======================
  "["$.a[0]","$.b","$.c"]"
```

```
SELECT JSON_SEARCH('{"a":["a","b"],"b":"a","c":"a"}', 'all', 'a',
NULL, '$.a', '$.b');
```

```
  json_search('{"a":["a","b"],"b":"a","c":"a"}', 'all', 'a', null,
'$.a', '$.b')
======================
  "["$.a[0]","$.b"]"
```

Wildcards can be used to define path filters as more general formats. Accepting only json paths that start with object key identifier:

```
SELECT JSON_SEARCH('{"a":["a","b"],"b":"a","c":"a"}', 'all', 'a',
NULL, '$.*');
```

```
  json_search('{"a":["a","b"],"b":"a","c":"a"}', 'all', 'a', null,
'$.*')
======================
  "["$.a[0]","$.b","$.c"]"
```

Accepting only json paths that start with object key identifier and follow immediately with a json array index will filter out '$.b', '$.d.e[0]' matches:

```
SELECT JSON_SEARCH('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', 'all', 'a', NULL, '$.*[*]');
```

```
  json_search('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'all', 'a', null, '$.*[*]')
======================
  "["$.a[0]","$.c[0]"]"
```

Accepting any paths that contain json array indexes will filter out '$.b'

```
SELECT JSON_SEARCH('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', 'all', 'a', NULL, '$**[*]');
```

```
  json_search('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'all', 'a', null, '$**[*]')
======================
  "["$.a[0]","$.c[0]","$.d.e[0]"]"
```

JSON_EXTRACT

JSON_EXTRACT(*json_doc, json path [, json path] ...*)

Returns json elements from the *json_doc*, that are addressed by the given paths. If json path arguments contain wildcards, all elements that are addressed by a path compatible with the wildcards-containing json path are gathered in a resulting json array. A single json element is returned if no wildcards are used in the given json paths and a single element is found, otherwise the json elements found are wrapped in a json array. Raises an error if a json path is **NULL** or invalid or if *json_doc* argument is invalid. Returns **NULL** if no elements are found or if json_doc is **NULL**.

```
SELECT JSON_EXTRACT('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', '$.a');
```

```
  json_extract('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'$.a')
======================
  "["a","b"]" -- at '$.a' we have the json array ["a","b"]
```

```
SELECT JSON_EXTRACT('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', '$.a[*]');
```

```
  json_extract('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'$.a[*]')
======================
  "["a","b"]" -- '$.a[0]' and '$.a[1]' wrapped in a json array,
forming ["a","b"]
```

Changing '.a' from previous query with '.*' wildcards will also match '$.c[0]'. This will match any json path that is exactly an object key identifier followed by an array index.

```
SELECT JSON_EXTRACT('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', '$.*[*]');
```

```
  json_extract('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'$.*[*]')
======================
  "["a","b","a"]"
```

The following json path will match all json paths that end with a json array index (matches all previous matched paths and, in addition, '$.d.e[0]') :

```
SELECT JSON_EXTRACT('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', '$**[*]');
```

```
  json_extract('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'$**[*]')
======================
  "["a","b","a","a"]"
```

```
SELECT JSON_EXTRACT('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":
["a"]}}', '$.d**[*]');
```

```
  json_extract('{"a":["a","b"],"b":"a","c":["a"], "d":{"e":["a"]}}',
'$d**[*]')
======================
  "["a"]" -- '$.d.e[0]' is the only path matching the given argument
path family - paths that start with '.d' and end with an array index
```

->

json_doc -> json path

Alias operator for **JSON_EXTRACT** with two arguments, having the *json_doc* argument constrained to be a column. Raises an error if the json path is **NULL** or invalid. Returns **NULL** if it is applied on a **NULL** *json_doc* argument.

```
CREATE TABLE tj (a json);
INSERT INTO tj values ('{"a":1}'), ('{"a":2}'), ('{"a":3}'), (NULL);

SELECT a->'$.a' from tj;
```

```
  json_extract(a, '$.a')
======================
  1
  2
  3
  NULL
```

->>

json_doc ->> json path

Alias for **JSON_UNQUOTE** (json_doc->json path). Operator can be applied only on *json_doc* arguments that are columns. Raises an error if the json path is **NULL** or invalid. Returns **NULL** if it is applied on a **NULL** *json_doc* argument.

```
CREATE TABLE tj (a json);
INSERT INTO tj values ('{"a":1}'), ('{"a":2}'), ('{"a":3}'), (NULL);

SELECT a->>'$.a' from tj;
```

```
  json_unquote(json_extract(a, '$.a'))
======================
  '1'
  '2'
  '3'
  NULL
```

JSON_CONTAINS_PATH

JSON_CONTAINS_PATH(*json_doc, one/all, json path [, json path] ...*)

The **JSON_CONTAINS_PATH** function verifies whether the given paths exist inside the *json_doc*.

When one/all argument is 'all', all given paths must exist to return 1. Returns 0 otherwise.

When one/all argument is 'one', it returns 1 if any given path exists. Returns 0 otherwise.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid.

```
SELECT JSON_CONTAINS_PATH ('[{"0":0},1,"2",{"three":3}]', 'all',
'$[0]', '$[0]."0"', '$[1]', '$[2]', '$[3]');
```

```
  json_contains_path('[{"0":0},1,"2",{"three":3}]', 'all', '$[0]',
'$[0]."0"', '$[1]', '$[2]', '$[3]')
=====================================================================
==================================

                                  1
```

```
SELECT JSON_CONTAINS_PATH ('[{"0":0},1,"2",{"three":3}]', 'all',
'$[0]', '$[0]."0"', '$[1]', '$[2]', '$[3]', '$.inexistent');
```

```
  json_contains_path('[{"0":0},1,"2",{"three":3}]', 'all', '$[0]',
'$[0]."0"', '$[1]', '$[2]', '$[3]', '$.inexistent')
=====================================================================
================================================

                                                  0
```

The **JSON_CONTAINS_PATH** function supports wildcards inside json paths.

```
SELECT JSON_CONTAINS_PATH ('[{"0":0},1,"2",{"three":3}]', 'one',
'$.inexistent', '$[*]."three"');
```

```
 json_contains_path('[{"0":0},1,"2",{"three":3}]', 'one',
'$.inexistent', '$[*]."three"')
================================================================
=====

    1
```

JSON_CONTAINS

JSON_CONTAINS(*json_doc doc1*, *json_doc doc2*[, *json path*])

The **JSON_CONTAINS** function verifies whether the *doc2* is contained inside the *doc1* at the optionally specified path. A json element contains another json element if the following recursive rules are satisfied:

- A json scalar contains another json scalar if they have the same type (their **JSON_TYPE** () are equal) and are equal. As an exception, json integer can be compared and equal to json double (even if their **JSON_TYPE** () evaluation are different).
- A json array contains a json scalar or a json object if any of json array's elements contains the json_nonarray.
- A json array contains another json array if all the second json array's elements are contained in the first json array.
- A json object contains another json object if, for every (*key2*, *value2*) pair in the second object, there exists a (*key1*, *value1*) pair in the first object with *key1* = *key2* and *value2* contained in *value1*.
- Otherwise the json element is not contained.

Returns whether *doc2* is contained in root json element of *doc1* if no json path argument is given. Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid.

```
SELECT JSON_CONTAINS ('"simple"','"simple"');
```

```
  json_contains('"simple"', '"simple"')
=======================================
                                      1
```

```
SELECT JSON_CONTAINS ('["a", "b"]','"b"');
```

```
  json_contains('["a", "b"]', '"b"')
====================================
                                   1
```

```
SELECT JSON_CONTAINS ('["a", "b1", ["a", "b2"]]','["b1", "b2"]');
```

```
  json_contains('["a", "b1", ["a", "b2"]]','["b1", "b2"]')
==========================================================
                                                         1
```

```
SELECT JSON_CONTAINS ('{"k1":["a", "b1"], "k2": ["a",
"b2"]}','{"k1":"b1", "k2":"b2"}');
```

```
  json_contains('{"k1":["a", "b1"], "k2": ["a", "b2"]}','{"k1":"b1",
"k2":"b2"}')
==========================================================================
============

           1
```

Note that json objects do not check containment the same way json arrays do. It is impossible to have a json element that is not a descendent of a json object contained in a sub-element of a json object.

```
SELECT JSON_CONTAINS ('["a", "b1", ["a", {"k":"b2"}]]','["b1",
"b2"]');
```

```
  json_contains('["a", "b1", ["a", {"k":"b2"}]]','["b1", "b2"]')
================================================================
                                                             0
```

```
SELECT JSON_CONTAINS ('["a", "b1", ["a", {"k":["b2"]}]]','["b1",
{"k":"b2"}]');
```

```
  json_contains('["a", "b1", ["a", {"k":["b2"]}]]','["b1",
{"k":"b2"}]')
```

```
=========================================================================
===

  1
```

JSON_MERGE_PATCH

JSON_MERGE_PATCH(*json_doc, json_doc [, json_doc] ...*)

The **JSON_MERGE_PATCH** function merges two or more json docs and returns the resulting merged json. **JSON_MERGE_PATCH** differs from **JSON_MERGE_PRESERVE** in that it will take the second argument when encountering merging conflicts. **JSON_MERGE_PATCH** is compliant with RFC 7396.

The merging of two json documents is performed with the following rules, recursively:

- when two non-object jsons are merged, the result of the merge is the second value.
- when a non-object json is merged with a json object, the result is the merge of an empty object with the second merging argument.
- when two objects are merged, the resulting object consists of the following members:
  - All members from the first object that have no corresponding member with the same key in the second object.
  - All members from the second object that have no corresponding members with equal keys in the first object, having values not null. Members with null values from second object are ignored.
  - One member for each member in the first object that has a corresponding non-null valued member in the second object with the same key. Same key members that appear in both objects and the second object's member value is null, are ignored. The values of these pairs become the results of merging operations performed on the values of the members from the first and second object.

Merge operations are executed serially when there are more than two arguments: the result of merging first two arguments is merged with third, this result is then merged with fourth and so on.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is not valid.

```
SELECT JSON_MERGE_PATCH ('["a","b","c"]', '"scalar"');
```

```
  json_merge_patch('["a","b","c"]', '"scalar"')
======================
  "scalar"
```

The exception to the merge-patching, when the first argument is non-object and the second is an object. A merge operation is performed between an empty object and the second object argument.

```
SELECT JSON_MERGE_PATCH ('["a"]', '{"a":null}');
```

```
  json_merge_patch('["a"]', '{"a":null}')
======================
  {}
```

Objects merging example, exemplifying the described object merging rules:

```
SELECT JSON_MERGE_PATCH ('{"a":null,"c":["elem"]}','{"b":null,"c":
{"k":null},"d":"elem"}');
```

```
  json_merge_patch('{"a":null,"c":["elem"]}', '{"b":null,"c":
{"k":null},"d":"elem"}')
======================
  {"a":null,"c":{},"d":"elem"}
```

JSON_MERGE_PRESERVE

JSON_MERGE_PRESERVE(*json_doc, json_doc [, json_doc] ...*)

The **JSON_MERGE_PRESERVE** function merges two or more json docs and returns the resulting merged json. **JSON_MERGE_PRESERVE** differs from **JSON_MERGE_PATCH** in that it preserves both json elements on merging conflicts.

The merging of two json documents is performed after the following rules, recursively:

- when two json arrays are merged, they are concatenated.
- when two non-array (scalar/object) json elements are merged and at most one of them is a json object, the result is an array containing the two json elements.
- when a non-array json element is merged with a json array, the non-array is wrapped as a single element json array and then merged with the json array according to json array merging rules.

- when two json objects are merged, all pairs that do not have a corresponding pair in the other json object are preserved. For matching keys, the values are always merged by applying the rules recursively.

Merge operations are executed serially when there are more than two arguments: the result of merging first two arguments is merged with third, this result is then merged with fourth and so on.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is not valid.

```
SELECT JSON_MERGE_PRESERVE ('"a"', '"b"');
```

```
  json_merge('"a"', '"b"')
======================
  ["a","b"]
```

```
SELECT JSON_MERGE_PRESERVE ('["a","b","c"]', '"scalar"');
```

```
  json_merge('["a","b","c"]', '"scalar"')
======================
  ["a","b","c","scalar"]
```

JSON_MERGE_PRESERVE, as opposed to **JSON_MERGE_PATCH**, will not drop and patch first argument's elements during merges and will gather them together.

```
SELECT JSON_MERGE_PRESERVE ('{"a":null,"c":["elem"]}','{"b":null,"c":
{"k":null},"d":"elem"}');
```

```
  json_merge('{"a":null,"c":["elem"]}','{"b":null,"c":
{"k":null},"d":"elem"}')
======================
  {"a":null,"c":["elem",{"k":null}],"b":null,"d":"elem"}
```

JSON_MERGE

JSON_MERGE(*json_doc, json_doc [, json_doc] ...*)

JSON_MERGE is an alias for **JSON_MERGE_PRESERVE**.

JSON_ARRAY_APPEND

JSON_ARRAY_APPEND(*json_doc, json path, json_val [, json path, json_val] ...*)

The **JSON_ARRAY_APPEND** function returns a modified copy of the first argument. For each given <*json path*, *json_val*> pair, the function appends the value to the json array addressed by the corresponding path.

The (*json path*, *json_val*) pairs are evaluated one by one, from left to right. The document produced by evaluating one pair becomes the new value against which the next pair is evaluated.

If the json path points to a json array inside the *json_doc*, the *json_val* is appended at the end of the array. If the json path points to a non-array json element, the non-array gets wrapped as a single element json array containing the referred non-array element followed by the appending of the given *json_val*.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid.

```
SELECT JSON_ARRAY_APPEND ('{"a":[1,2]}','$.a','b');
```

```
  json_array_append('{"a":[1,2]}', '$.a', 'b')
======================
  {"a":[1,2,"b"]}
```

```
SELECT JSON_ARRAY_APPEND ('{"a":1}','$.a','b');
```

```
  json_array_append('{"a":1}', '$.a', 'b')
======================
  {"a":[1,"b"]}
```

```
SELECT JSON_ARRAY_APPEND ('{"a":[1,2]}', '$.a[0]', '1');
```

```
  json_array_append('{"a":[1,2]}', '$.a[0]', '1')
======================
  {"a":[[1,"1"],2]}
```

JSON_ARRAY_INSERT

JSON_ARRAY_INSERT(*json_doc, json path, json_val [, json path, json_val] ...*)

The **JSON_ARRAY_INSERT** function returns a modified copy of the first argument. For each given <*json path*, *json_val*> pair, the function inserts the value in the json array addressed by the corresponding path.

The (*json path*, *json_val*) pairs are evaluated one by one, from left to right. The document produced by evaluating one pair becomes the new value against which the next pair is evaluated.

The rules of the **JSON_ARRAY_INSERT** operation are the following:

- if a json path addresses an element of a json_array, the given *json_val* is inserted at the specified index, shifting any following elements to the right.
- if the json path points to an array index after the end of an array, the array is filled with nulls after end of the array until the specified index and the json_val is inserted at the specified index.
- if the json path does not exist inside the *json_doc*, the last token of the json path is an array index and the json path without the last array index token would have pointed to an element inside the *json_doc*, the element found by the stripped json path is replaced with single element json array and the **JSON_ARRAY_INSERT** operation is performed with the original json path.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid or if a *json_path* does not address a cell of an array inside the *json_doc*.

```
SELECT JSON_ARRAY_INSERT ('[0,1,2]', '$[0]', '1');
```

```
  json_array_insert('[0,1,2]', '$[0]', '1')
======================
  ["1",0,1,2]
```

```
SELECT JSON_ARRAY_INSERT ('[0,1,2]', '$[5]', '1');
```

```
  json_array_insert('[0,1,2]', '$[5]', '1')
======================
  [0,1,2,null,null,"1"]
```

Examples for **JSON_ARRAY_INSERT's** third rule.

```
SELECT JSON_ARRAY_INSERT ('{"a":4}', '$[5]', '1');
```

```
  json_array_insert('{"a":4}', '$[5]', '1')
======================
  [{"a":4},null,null,null,null,"1"]
```

```
SELECT JSON_ARRAY_INSERT ('"a"', '$[5]', '1');
```

```
  json_array_insert('"a"', '$[5]', '1')
======================
  ["a",null,null,null,null,"1"]
```

JSON_INSERT

JSON_INSERT(*json_doc, json path, json_val [, json path, json_val] ...*)

The **JSON_INSERT** function returns a modified copy of the first argument. For each given <*json path*, *json_val*> pair, the function inserts the value if no other value exists at the corresponding path.

The insertion rules for **JSON_INSERT** are the following:

The *json_val* is inserted if the json path addresses one of the following json values inside the *json_doc*:

- An inexistent object member of an existing json object. A (*key*, *value*) pair is added to the json object with the key being json path's last element and the value being the *json_val*.
- An array index past of an existing json array's end. The array is filled with nulls after the initial end of the array and the *json_val* is inserted at the specified index.

The document produced by evaluating one pair becomes the new value against which the next pair is evaluated.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid.

Paths to existing elements inside the *json_doc* are ignored:

```
SELECT JSON_INSERT ('{"a":1}','$.a','b');
```

```
  json_insert('{"a":1}', '$.a', 'b')
======================
  {"a":1}
```

```
SELECT JSON_INSERT ('{"a":1}','$.b','1');
```

```
  json_insert('{"a":1}', '$.b', '1')
======================
  {"a":1,"b":"1"}
```

```
SELECT JSON_INSERT ('[0,1,2]','$[4]','1');
```

```
  json_insert('[0,1,2]', '$[4]', '1')
======================
  [0,1,2,null,"1"]
```

JSON_SET

JSON_SET(*json_doc, json path, json_val [, json path, json_val] ...*)

The **JSON_SET** function returns a modified copy of the first argument. For each given <*json path*, *json_val*> pair, the function inserts or replaces the value at the corresponding path. Otherwise, the *json_val* is inserted if the json path addresses one of the following json values inside the *json_doc*:

- An inexistent object member of an existing json object. A (*key*, *value*) pair is added to the json object with the key deduced from the json path and the value being the *json_val*.
- An array index past of an existing json array's end. The array is filled with nulls after the initial end of the array and the *json_val* is inserted at the specified index.

The document produced by evaluating one pair becomes the new value against which the next pair is evaluated.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid.

```
SELECT JSON_SET ('{"a":1}','$.a','b');
```

```
  json_set('{"a":1}', '$.a', 'b')
======================
  {"a":"b"}
```

```
SELECT JSON_SET ('{"a":1}','$.b','1');
```

```
  json_set('{"a":1}', '$.b', '1')
======================
  {"a":1,"b":"1"}
```

```
SELECT JSON_SET ('[0,1,2]','$[4]','1');
```

```
  json_set('[0,1,2]', '$[4]', '1')
======================
  [0,1,2,null,"1"]
```

JSON_REPLACE

JSON_REPLACE(*json_doc, json path, json_val [, json path, json_val] ...*)

The **JSON_REPLACE** function returns a modified copy of the first argument. For each given <*json path*, *json_val*> pair, the function replaces the value only if another value is found at the corresponding path.

If the *json_path* does not exist inside the *json_doc*, the (*json path*, *json_val*) pair is ignored and has no effect.

The document produced by evaluating one pair becomes the new value against which the next pair is evaluated.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid.

```
SELECT JSON_REPLACE ('{"a":1}','$.a','b');
```

```
  json_replace('{"a":1}', '$.a', 'b')
======================
  {"a":"b"}
```

No replacement is done if the *json path*` does not exist inside the *json_doc.*

```
SELECT JSON_REPLACE ('{"a":1}','$.b','1');
```

```
  json_replace('{"a":1}', '$.b', '1')
======================
  {"a":1}
```

```
SELECT JSON_REPLACE ('[0,1,2]','$[4]','1');
```

```
  json_replace('[0,1,2]', '$[4]', '1')
======================
  [0,1,2]
```

JSON_REMOVE

JSON_REMOVE(*json_doc, json path [, json path] ...*)

The **JSON_REMOVE** function returns a modified copy of the first argument, by removing values from all given paths.

The json path arguments are evaluated one by one, from left to right. The result produced by evaluating a json path becomes the value against which the next json path is evaluated.

Returns **NULL** if any argument is **NULL**. An error occurs if any argument is invalid or if a path points to the root or if a path does not exist.

```
SELECT JSON_REMOVE ('[0,1,2]','$[1]');
```

```
  json_remove('[0,1,2]','$[1]')
======================
  [0,2]
```

```
SELECT JSON_REMOVE ('{"a":1,"b":2}','$.a');
```

```
  json_remove('{"a":1,"b":2}','$.a')
======================
  {"b":2}
```

JSON_TABLE

JSON_TABLE function facilitates transforming jsons into a table-like structures that can be queried similarly as regular tables. The transformation generates a single row or multiple rows, by expanding for example the elements of a JSON_ARRAY.

The full syntax of **JSON_TABLE**:

```
JSON_TABLE(
    expr,
    path COLUMNS (column_list)
)   [AS] alias


<column_list>::=
    <column> [, <column>] ...

<column>::=
    name FOR ORDINALITY
    |  name type PATH string_path <on_empty> <on_error>
    |  name type EXISTS PATH string_path
    |  NESTED [PATH] string_path COLUMNS <column_list>

<on_empty>::=
    NULL | ERROR | DEFAULT value ON EMPTY

<on_error>::=
    NULL | ERROR | DEFAULT value ON ERROR
```

The *json_doc* expr must be an expression that results in a json_doc. This can be a constant json, a table's column or the result of a function or operator. The *json path* must be a valid path and is used to extract json data to be evaluated in the **COLUMNS** clause. The **COLUMNS** clause defines output column types and operations performed to get the output. The [**AS**] *alias* clause is required.

JSON_TABLE supports four types of columns:

- *name* **FOR ORDINALITY**: this type keeps track of a row's number inside a **COLUMNS** clause. The column's type is **INTEGER**.
- *name type* **PATH** *json path* [**on empty**] [**on error**]: Columns of this type are used to extract json_values from the specified json paths. The extracted json data is then coerced to the specified type. If the path does not exist, json value triggers the **on empty** clause. The **on error** clause is triggered if the extracted json value is not coercible to the target type.
    - **on empty** determines the behavior of **JSON_TABLE** in case the path does not exist. **on empty** can have one of the following values:
        - **NULL ON EMPTY**: the column is set to **NULL**. This is the default behavior.
        - **ERROR ON EMPTY**: an error is thrown
        - **DEFAULT** *value* **ON EMPTY**: *value* will be used instead of the missing value.

    - **on error** can have one of the following values:
        - **NULL ON ERROR**: the column is set to **NULL**. This is the default behavior.
        - **ERROR ON ERROR**: an error is thrown.
        - **DEFAULT** *value* **ON ERROR**: *value* will be used instead of the array/object/json scalar that failed coercion to desired column type.

- *name type* **EXISTS PATH** *json path*: this returns 1 if any data is present at the json path location, 0 otherwise.

- **NESTED** [**PATH**] *json path* **COLUMNS** (*column list*) generates from json data found at path a separate subset of rows and columns that are combined with the results of parent. Results are combined similarly as "for each" loops. The json path is relative to the parent's path. Same rules for **COLUMNS** clause are applied recursively.

```
SELECT * FROM JSON_TABLE (
        '{"a":[1,[2,3]]}',
        '$.a[*]' COLUMNS ( col INT PATH '$')
    )   AS jt;
```

```
                    col
=======================
                      1 -- first value found at '$.a[*]' is 1 json
scalar, which is coercible to 1
                   NULL -- second value found at '$.a[*]' is [2,3]
json array which cannot be coerced to int, triggering NULL ON ERROR
default behavior
```

Overriding the default on_error behavior, results in a different output from previous example:

```
SELECT * FROM JSON_TABLE (
        '{"a":[1,[2,3]]}',
        '$.a[*]' COLUMNS ( col INT PATH '$' DEFAULT '-1' ON ERROR)
    )   AS jt;
```

```
                    col
=======================
                      1 -- first value found at '$.a[*]' is '1' json
scalar, which is coercible to 1
                     -1 -- second value found at '$.a[*]' is '[2,3]'
json array which cannot be coerced to int, triggering ON ERROR
```

ON EMPTY example:

```
SELECT * FROM JSON_TABLE (
        '{"a":1}',
        '$' COLUMNS ( col1 INT PATH '$.a',
                      col2 INT PATH '$.b',
                      col3 INT PATH '$.c' DEFAULT '0' ON EMPTY)
    )   AS jt;
```

```
         col1         col2         col3
=========================================
            1         NULL            0
```

In the example below, '$.*' path will be used to make the parent columns receive root json object's member values one by one. Column a shows what is processed. Each member's value of the root object will then be processed further by the **NESTED** [**PATH**] clause. **NESTED PATH** uses path '$[*]' take each element of the array to be further processed by its columns. **FOR ORDINALITY** columns track the count of the current processed element. In the example's result we can see that for each new element in a column, the *ord* column's value also gets incremented. **FOR ORDINALITY** *nested_ord* column also acts as a counter of the number of elements processed by sibling columns. The nested **FOR ORDINALITY** column gets reset after finishing each processing batch. The third member's value, 6 cannot be treated as an array and therefore cannot be processed by the nested columns. Nested columns will yield **NULL** values.

```
SELECT * FROM JSON_TABLE (
        '{"a":[1,2],"b":[3,4,5],"d":6,"c":[7]}',
        '$.*' COLUMNS ( ord FOR ORDINALITY,
                        col JSON PATH '$',
                        NESTED PATH '$[*]' COLUMNS ( nested_ord FOR
ORDINALITY,
                                                     nested_col JSON
PATH '$'))
    )   AS jt;
```

```
         ord  col                       nested_ord  nested_col
=============================================================
           1  [1,2]                            1  1
           1  [1,2]                            2  2
           2  [3,4,5]                          1  3
           2  [3,4,5]                          2  4
           2  [3,4,5]                          3  5
           3  6                             NULL  NULL
           4  [7]                              1  7
```

The following example showcases how multiple same-level **NESTED** [**PATH**] clauses are treated by the **JSON_TABLE**. The value to be processed gets passed once, one by one and in order, to each of

the **NESTED** [**PATH**] clauses. During processing of a value by a **NESTED** [**PATH**] clause, any sibling **NESTED** [**PATH**] clauses will fill their column with **NULL** values.

```
SELECT * FROM JSON_TABLE (
        '{"a":{"key1":[1,2], "key2":[3,4,5]},"b":{"key1":6, "key2":
[7]}}',
        '$.*' COLUMNS ( ord FOR ORDINALITY,
                        col JSON PATH '$',
                        NESTED PATH '$.key1[*]' COLUMNS (
nested_ord1 FOR ORDINALITY,

nested_col1 JSON PATH '$'),
                        NESTED PATH '$.key2[*]' COLUMNS (
nested_ord2 FOR ORDINALITY,

nested_col2 JSON PATH '$'))
    )   AS jt;
```

```
          ord  col                              nested_ord1
nested_col1           nested_ord2  nested_col2
=============================================================================
=============================================
            1  {"key1":[1,2],"key2":[3,4,5]}            1
1                          NULL  NULL
            1  {"key1":[1,2],"key2":[3,4,5]}            2
2                          NULL  NULL
            1  {"key1":[1,2],"key2":[3,4,5]}         NULL
NULL                          1  3
            1  {"key1":[1,2],"key2":[3,4,5]}         NULL
NULL                          2  4
            1  {"key1":[1,2],"key2":[3,4,5]}         NULL
NULL                          3  5
            2  {"key1":6,"key2":[7]}                 NULL
NULL                          1  7
```

An example for multiple layers **NESTED** [**PATH**] clauses:

```
SELECT * FROM JSON_TABLE (
        '{"a":{"key1":[1,2], "key2":[3,4,5]},"b":{"key1":6, "key2":
[7]}}',
        '$.*' COLUMNS ( ord FOR ORDINALITY,
                        col JSON PATH '$',
                        NESTED PATH '$.*' COLUMNS ( nested_ord1 FOR
ORDINALITY,
                                                    nested_col1 JSON
PATH '$',
                                                    NESTED PATH '$
[*]' COLUMNS ( nested_ord11 FOR ORDINALITY,
```

```
            nested_col11 JSON PATH '$')),
                        NESTED PATH '$.key2[*]' COLUMNS (
nested_ord2 FOR ORDINALITY,

nested_col2 JSON PATH '$'))
    )   AS jt;
```

```
          ord  col                             nested_ord1
nested_col1           nested_ord11  nested_col11
nested_ord2  nested_col2
================================================================================
================================================================================
=============
            1  {"key1":[1,2],"key2":[3,4,5]}            1  [1,
2]                           1  1                            NULL
NULL
            1  {"key1":[1,2],"key2":[3,4,5]}            1  [1,
2]                           2  2                            NULL
NULL
            1  {"key1":[1,2],"key2":[3,4,5]}            2  [3,4,
5]                         1  3                             NULL
NULL
            1  {"key1":[1,2],"key2":[3,4,5]}            2  [3,4,
5]                         2  4                             NULL
NULL
            1  {"key1":[1,2],"key2":[3,4,5]}            2  [3,4,
5]                         3  5                             NULL
NULL
            1  {"key1":[1,2],"key2":[3,4,5]}         NULL
NULL                         NULL  NULL
1  3
            1  {"key1":[1,2],"key2":[3,4,5]}         NULL
NULL                         NULL  NULL
2  4
            1  {"key1":[1,2],"key2":[3,4,5]}         NULL
NULL                         NULL  NULL
3  5
            2  {"key1":6,"key2":[7]}                    1
6                            NULL  NULL
NULL  NULL
            2  {"key1":6,"key2":[7]}                    2
[7]                            1  7
NULL  NULL
            2  {"key1":6,"key2":[7]}                 NULL
NULL                         NULL  NULL
1  7
```

LOB Functions

Contents

LOB Functions 581

- BIT_TO_BLOB 581
- BLOB_FROM_FILE 582
- BLOB_LENGTH 582
- BLOB_TO_BIT 582
- CHAR_TO_BLOB 583
- CHAR_TO_CLOB 583
- CLOB_FROM_FILE 583
- CLOB_LENGTH 584
- CLOB_TO_CHAR 584

BIT_TO_BLOB

BIT_TO_BLOB(*blob_type_column_or_value*)

This function converts **BIT** or **VARYING BIT** type into **BLOB** type.

Parameters:

blob_type_column_or_value – Target column or value to convert

Return type:

BLOB

BLOB_FROM_FILE

BLOB_FROM_FILE(*file_pathname*)

This function read the contents from the file with **VARCHAR** type data and returns **BLOB** type data.

Parameters:

file_pathname – the path on the server which DB clients like CAS or CSQL are started

Return type:

BLOB

```
SELECT CAST(BLOB_FROM_FILE('local:/home/cubrid/demo/lob/ces_622/
image_t.00001352246696287352_4131')
   AS BIT VARYING) result;

SELECT CAST(BLOB_FROM_FILE('file:/home/cubrid/demo/lob/ces_622/
image_t.00001352246696287352_4131')
   AS BIT VARYING) result;
```

BLOB_LENGTH

BLOB_LENGTH(*blob_column*)

The length of **LOB** data stored in **BLOB** file is returned.

Parameters:

blob_column – The column to get the length of **BLOB**

Return type:

INT

BLOB_TO_BIT

BLOB_TO_BIT(*blob_type_column*)

This function converts **BLOB** type to **VARYING BIT** type.

Parameters:

blob_type_column – Target column to convert

Return type:

VARYING BIT

CHAR_TO_BLOB

CHAR_TO_BLOB(*char_type_column_or_value*)

This function converts **CHAR** or **VARCHAR** type into **BLOB** type.

Parameters:

char_type_column_or_value – Target column or value to convert

Return type:

BLOB

CHAR_TO_CLOB

CHAR_TO_CLOB(*char_type_column_or_value*)

This function converts **CHAR** or **VARCHAR** type into **CLOB** type.

Parameters:

char_type_column_or_value – Target column or value to convert

Return type:

CLOB

CLOB_FROM_FILE

CLOB_FROM_FILE(*file_pathname*)

This function read the contents from the file with **VARCHAR** type data and returns **CLOB** type data.

Parameters:

file_pathname – the path on the server which DB clients like CAS or CSQL are started

Return type:

CLOB

If you specify the *file_pathname* as the relative path, the parent path will be the current working directory.

For the statement including this function, the query plan is not cached.

```
SELECT CAST(CLOB_FROM_FILE('local:/home/cubrid/demo/lob/ces_622/
image_t.00001352246696287352_4131')
   AS VARCHAR) result;

SELECT CAST(CLOB_FROM_FILE('file:/home/cubrid/demo/lob/ces_622/
image_t.00001352246696287352_4131')
   AS VARCHAR) result;
```

CLOB_LENGTH

CLOB_LENGTH(*clob_column*)

The length of **LOB** data stored in **CLOB** file is returned.

Parameters:

clob_column – The column to get the length of **CLOB**

Return type:

INT

CLOB_TO_CHAR

CLOB_TO_CHAR(*clob_type_column [USING charset]*)

This function converts **CLOB** type into **VARCHAR** type.

Parameters:

- **clob_type_column** – Target column to convert
- **charset** – The character set of string to convert. It can be utf8, euckr or iso88591.

Return type:

STRING

Data Type Casting Functions and Operators

Contents

Data Type Casting Functions and Operators 585

- CAST 586
- DATE_FORMAT 593
- FORMAT 599
- STR_TO_DATE 600
- TIME_FORMAT 602
- TO_CHAR(date_time) 604
- TO_CHAR(number) 614
- TO_DATE 620
- TO_DATETIME 622
- TO_DATETIME_TZ 624
- TO_NUMBER 625

- TO_TIME 626
- TO_TIMESTAMP 629
- TO_TIMESTAMP_TZ 631

CAST

CAST(*cast_operand AS cast_target*)

The **CAST** operator can be used to explicitly cast one data type to another in the **SELECT** statement. A query list or a value expression in the **WHERE** clause can be cast to another data type.

Parameters:

- **cast_operand** – Declares the value to cast to a different data type.
- **cast_target** – Specifies the type to cast to.

Return type:

cast_target

Depending on the situation, data type can be automatically converted without using the **CAST** operator. For details, see Implicit Type Conversion.

See CASTing a String to Date/Time Type regarding to convert the string of date/time type into date/time type.

The following table shows a summary of explicit type conversions (casts) using the **CAST** operator in CUBRID.

From \ To	ENUM	ANUM	VCHAR	FCHAR	VBIT	FBIT	ENUM	BLOB	CLOB	DATE	TIME	UTIME	UTIMEZ	DATETIME	DATETIMEZ	SET	MSET	SEQ
ENUM					No	No	No	No	No	No	No			No	No	No	No	No

	Yes	Yes	Yes	Yes								Yes	Yes					
AN	Yes	Yes	Yes	Yes	No	No	No	No	No	No	No	Yes	Yes	No	No	No	No	No
VC	Yes	Yes	Yes(*)	Yes(*)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
FC	Yes	Yes	Yes(*)	Yes(*)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
VB	No	No	Yes	Yes	Yes	Yes	No	Yes	Yes	No	No	No	No	No	No	No	No	No	No
FB	No	No	Yes	Yes	Yes	Yes	No	Yes	Yes	No	No	No	No	No	No	No	No	No	No
ENUM	No	No	Yes	Yes	No	No	Yes	No	No	No	No	No	No	No	No	No	No	No	No
BLOB	No	No	No	No	Yes	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	No
CLOB	No	No	Yes	Yes	No	No	Yes	No	Yes	No	No	No	No	No	No	No	No	No	No

D	No	No	Yes	Yes	No	No	No	No	No	Yes	No	Yes	Yes	Yes	Yes	No	No	No
T	No	No	Yes	Yes	No	No	No	No	No	No	Yes	No	No	No	No	No	No	No
UT	No	No	Yes	Yes	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
UTZ	No	No	Yes	Yes	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
DT	No	No	Yes	Yes	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
DTZ	No	No	Yes	Yes	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
S	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes
MS	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes
SQ	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes

(*): The **CAST** operation is allowed only when the value expression and the data type to be cast have the same character set.

- **Data Type Key**
  - **EN**: Exact numeric data type (**INTEGER**, **SMALLINT**, **BIGINT**, **NUMERIC**, **DECIMAL**)
  - **AN**: Approximate numeric data type (**FLOAT/REAL**, **DOUBLE**)
  - **VC**: Variable-length character string (**VARCHAR** (*n*))
  - **FC**: Fixed-length character string (**CHAR** (*n*))
  - **VB**: Variable-length bit string (**BIT VARYING** (*n*))
  - **FB**: Fixed-length bit string (**BIT** (*n*))
  - **ENUM**: **ENUM** type
  - **BLOB**: Binary data that is stored outside DB
  - **CLOB**: String data that is stored inside DB
  - **D**: **DATE**
  - **T**: **TIME**
  - **DT**: **DATETIME**
  - **DTZ**: **DATETIME WITH TIME ZONE** and **DATETIME WITH LOCAL TIME ZONE** data types
  - **UT**: **TIMESTAMP**
  - **UTZ**: **TIMESTAMP WITH TIME ZONE** and **TIMESTAMP WITH LOCAL TIME ZONE** data types
  - **S**: **SET**
  - **MS**: **MULTISET**
  - **SQ**: **LIST** (= **SEQUENCE**)

```
--operation after casting character as INT type returns 2
SELECT (1+CAST ('1' AS INT));
```

```
2
```

```
--cannot cast the string which is out of range as SMALLINT
SELECT (1+CAST('1234567890' AS SMALLINT));
```

```
ERROR: Cannot coerce value of domain "character" to domain
"smallint".
```

```
--operation after casting returns 1+1234567890
SELECT (1+CAST('1234567890' AS INT));
```

```
1234567891
```

```
--'1234.567890' is casted to 1235 after rounding up
SELECT (1+CAST('1234.567890' AS INT));
```

```
1236
```

```
--'1234.567890' is casted to string containing only first 5 letters.
SELECT (CAST('1234.567890' AS CHAR(5)));
```

```
'1234.'
```

```
--numeric type can be casted to CHAR type only when enough length is
specified
SELECT (CAST(1234.567890 AS CHAR(5)));
```

```
ERROR: Cannot coerce value of domain "numeric" to domain "character".
```

```
--numeric type can be casted to CHAR type only when enough length is
specified
SELECT (CAST(1234.567890 AS CHAR(11)));
```

```
'1234.567890'
```

```
--numeric type can be casted to CHAR type only when enough length is
specified
SELECT (CAST(1234.567890 AS VARCHAR));
```

```
'1234.567890'
```

```
--string can be casted to time/date types only when its literal is
correctly specified
SELECT (CAST('2008-12-25 10:30:20' AS TIMESTAMP));
```

```
10:30:20 AM 12/25/2008
```

```
SELECT (CAST('10:30:20' AS TIME));
```

```
10:30:20 AM
```

```
--string can be casted to TIME type when its literal is same as
TIME's.
SELECT (CAST('2008-12-25 10:30:20' AS TIME));
```

```
10:30:20 AM
```

```
--string can be casted to TIME type after specifying its type of the
string
SELECT (CAST(TIMESTAMP'2008-12-25 10:30:20' AS TIME));
```

```
10:30:20 AM
```

```
SELECT CAST('abcde' AS BLOB);
```

```
file:/home1/user1/db/tdb/lob/ces_743/ces_temp.
000012832320243091 72_1342
```

```
SELECT CAST(B'11010000' as varchar(10));
```

```
'd0'
```

```
SELECT CAST('1A' AS BLOB);
```

```
X'1a00'
```

```
--numbers can be casted to TIMESTAMP type
SELECT CAST (1 AS TIMESTAMP), CAST (1.2F AS TIMESTAMP);
```

```
09:00:01 AM 01/01/1970     09:00:01 AM 01/01/1970
```

```
--numbers cannot be casted to DATETIME type
SELECT CAST (1 AS DATETIME);
```

```
Cannot coerce 1 to type datetime
```

```
--TIMESTAMP cannot be casted to numbers
SELECT CAST (TIMESTAMP'09:00:01 AM 01/01/1970' AS INT)
```

```
Cannot coerce timestamp '09:00:01 AM 01/01/1970' to type integer.
```

Note

- **CAST** is allowed only between data types having the same character set.
- If you cast an approximate data type(FLOAT, DOUBLE) to integer type, the number is rounded to zero decimal places.
- If you cast an exact numeric data type(NUMERIC) to integer type, the number is rounded to zero decimal places.
- If you cast a numeric data type to string character type, it should be longer than the length of significant figures + decimal point. An error occurs otherwise.

- If you cast a character string type *A* to a character string type *B*, B should be longer than the *A*. The end of character string is truncated otherwise.
- If you cast a character string type *A* to a date-time date type *B*, it is converted only when literal of *A* and *B* type match one another. An error occurs otherwise.
- You must explicitly do type casting for numeric data stored in a character string so that an arithmetic operation can be performed.

DATE_FORMAT

DATE_FORMAT(*date*, *format*)

The **DATE_FORMAT** function converts the value of date/time data type which include a date to specified date/time format and then return the value with the **VARCHAR** data type. For the format parameter to assign, refer to Date/Time Format 2 table of the `DATE_FORMAT()`. The Date/Time Format 2 table is used in `DATE_FORMAT()`, `TIME_FORMAT()`, and `STR_TO_DATE()` functions.

Parameters:

- **date** – A value of DATE, TIMESTAMP, DATETIME, DATETIMETZ, DATETIMELTZ, TIMESTAMPTZ, or TIMESTAMP.
- **format** – Specifies the output format. The format specifier starting with '%' is used.

Return type:

STRING

When the *format* argument is assigned, the string is interpreted according to the specified language. At that time, the language specified as the **intl_date_lang** system parameter is applied. If the **intl_date_lang** value is not set, the language specified when creating DB is applied.

For example, when the language is "de_DE" and the format is "%d %M %Y", the string "3 Oktober 2009" is interpreted as the DATE type of "2009-10-03". When the specified *format* argument does not correspond to the given string, an error is returned.

In the following Date/Time Format 2 table, the month/day, date, and AM/PM in characters are different by language.

Date/Time Format 2

format Value	Meaning
%a	Weekday, English abbreviation (Sun, ... , Sat)
%b	Month, English abbreviation (Jan, ... , Dec)
%c	Month (1, ... , 12)
%D	Day of the month, English ordinal number (1st, 2nd, 3rd, ...)
%d	Day of the month, two-digit number (01, ... , 31)
%e	Day of the month (1, ... , 31)
%f	Microseconds, three-digit number (000, ... , 999)
%H	Hour, 24-hour based, number with at least two-digit (00, ... , 23, ... , 100, ...)
%h	Hour, 12-hour based two-digit number (01, ... , 12)
%I	Hour, 12-hour based two-digit number (01, ... , 12)
%i	Minutes, two-digit number (00, ... , 59)
%j	Day of year, three-digit number (001, ... , 366)
%k	

format Value	Meaning
	Hour, 24-hour based, number with at least one-digit (0, ... , 23, ... , 100, ...)
%l	Hour, 12-hour based (1, ... , 12)
%M	Month, English string (January, ... , December)
%m	Month, two-digit number (01, ... , 12)
%p	AM or PM
%r	Time, 12-hour based, hour:minute:second (hh:mi:ss AM or hh:mi:ss PM)
%S	Seconds, two-digit number (00, ... , 59)
%s	Seconds, two-digit number (00, ... , 59)
%T	Time, 24-hour based, hour:minute:second (hh:mi:ss)
%U	Week, two-digit number, week number of the year with Sunday being the first day Week (00, ... , 53)
%u	Week, two-digit number, week number of the year with Monday being the first day (00, ... , 53)
%V	Week, two-digit number, week number of the year with Sunday being the first day

format Value	Meaning
	Week (00, ... , 53) (Available to use in combination with %X)
%v	Week, two-digit number, week number of the year with Monday being the first day (00, ... , 53) (Available to use in combination with %x)
%W	Weekday, English string (Sunday, ... , Saturday)
%w	Day of the week, number index (0=Sunday, ... , 6=Saturday)
%X	Year, four-digit number calculated as the week number with Sunday being the first day of the week (0000, ... , 9999) (Available to use in combination with %V)
%x	Year, four-digit number calculated as the week number with Monday being the first day of the week (0000, ... , 9999) (Available to use in combination with %v)
%Y	Year, four-digit number (0001, ... , 9999)
%y	Year, two-digit number (00, 01, ... , 99)
%%	Output the special character “%” as a string
%x	Output an arbitrary character x as a string out of English letters that are not used as format specifiers.

format Value	Meaning
%TZR	Timezone region information. e.g. US/Pacific.
%TZD	Daylight saving information. e.g. KST, KT, EET
%TZH	Timezone hour offset. e.g. +09, -09
%TZM	Timezone minute offset. e.g. +00, +30

Note

%TZR, %TZD, %TZH, %TZM can be used only in timezone types.

Note

A format specifying a number after TZD

See A format specifying a number after TZD.

The following example shows the case when the system parameter **intl_date_lang** is “en_US”.

```
SELECT DATE_FORMAT(datetime'2009-10-04 22:23:00', '%W %M %Y');
```

```
'Sunday October 2009'
```

```
SELECT DATE_FORMAT(datetime'2007-10-04 22:23:00', '%H:%i:%s');
```

```
'22:23:00'
```

```
SELECT DATE_FORMAT(datetime'1900-10-04 22:23:00', '%D %y %a %d %m %b
%j');
```

```
'4th 00 Thu 04 10 Oct 277'
```

```
SELECT DATE_FORMAT(date'1999-01-01', '%X %V');
```

```
'1998 52'
```

The following example shows the case when the system parameter **intl_date_lang** is “de_DE”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT DATE_FORMAT(datetime'2009-10-04 22:23:00', '%W %M %Y');
```

```
'Sonntag Oktober 2009'
```

```
SELECT DATE_FORMAT(datetime'2007-10-04 22:23:00', '%H:%i:%s %p');
```

```
'22:23:00 Nachm.'
```

```
SELECT DATE_FORMAT(datetime'1900-10-04 22:23:00', '%D %y %a %d %m %b
%j');
```

```
'4 00 Do. 04 10 Okt 277'
```

Note

When the charset is ISO-8859-1, the language that can be changed by the system parameter **intl_date_lang** is “ko_KR” and “tr_TR” except “en_US”. If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`).

The following example outputs the value of DATETIMETZ type which includes timezone information as the desired format.

```
SELECT DATE_FORMAT(datetimetz'2012-02-02 10:10:10 Europe/Zurich
CET', '%TZR %TZD %TZH %TZM');
```

::

'Europe/Zurich CET 01 00'

FORMAT

FORMAT(*x*, *dec*)

The **FORMAT** function displays the number *x* by using digit grouping symbol as a thousands separator, so that its format becomes '#,###,###.#####' and performs rounding after the decimal point symbol to express as many as *dec* digits after it. The return value is a **VARCHAR** type.

Parameters:

- **x** – An expression that returns a numeric value
- **dec** – the number of digits of fractional parts

Return type:

STRING

Thousands separator symbol and decimal point symbol is output in the format according to the specified language. The language is specified by the **intl_number_lang** system parameter. If the value of **intl_number_lang** is not set, the language specified when creating DB is applied.

For example, when the language is one of the European languages, such as "de_DE" or "fr_FR", "." is interpreted as the thousands separator and "," as the decimal point symbol (see Default output of number by language of the `TO_CHAR()`).

The following example shows command execution by setting the value of the **intl_number_lang system** parameter to "en_US".

```
SET SYSTEM PARAMETERS 'intl_number_lang="en_US"';
SELECT FORMAT(12000.123456,3), FORMAT(12000.123456,0);
```

```
'12,000.123'          '12,000'
```

The following example shows command execution on the database by setting the value of the **intl_number_lang** system parameter to “de_DE”. In the number output format of most European countries, such as Germany and France, “.” is the cipher identifier and “,” is the decimal point symbol.

```
SET SYSTEM PARAMETERS 'intl_number_lang="de_DE"';
SELECT FORMAT(12000.123456,3), FORMAT(12000.123456,0);
```

```
'12.000,123'          '12.000'
```

STR_TO_DATE

STR_TO_DATE(*string*, *format*)

The **STR_TO_DATE** function converts the given character string to a date/time value by interpreting it according to the specified format and operates in the opposite way to the `DATE_FORMAT()` function. The return value is determined by the date/time part included in the character string.

Parameters:

- **string** – String.
- **format** – Specifies the format to interpret the character string. You should use character strings including % for the format specifiers. See Date/Time Format 2 table of `DATE_FORMAT()` function.

Return type:

DATETIME, DATE, TIME, DATETIMETZ

For the *format* argument to assign, see Date/Time Format 2 table of the `DATE_FORMAT()`.

If *string* is invalid date/time value or *format* is invalid, it returns an error.

When the *format* argument is assigned, the *string* is interpreted according to the specified language. At that time, the language specified as the **intl_date_lang** system parameter is applied. If the **intl_date_lang** value is not set, the language specified when creating DB is applied.

For example, when the language is “de_DE” and the *format* is “%d %M %Y”, the string “3 Oktober 2009” is interpreted as the **DATE** type of “2009-10-03”. If the *format* argument does not correspond to the given *string*, an error is returned.

0 is not allowed in the argument value corresponding to year, month, and day; however, if 0 is inputted in every argument value corresponding to date and time, the value of **DATE** or **DATETIME** type that has 0 for every date and time value is returned as an exception. Note that operation in JDBC program is determined by the configuration of zeroDateTimeBehavior, connection URL property. For more information about zeroDateTimeBehavior, please see Configuration Connection.

The following example shows the case when the system parameter **intl_date_lang** is “en_US”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="en_US"';
SELECT STR_TO_DATE('01,5,2013','%d,%m,%Y');
```

```
05/01/2013
```

```
SELECT STR_TO_DATE('May 1, 2013','%M %d,%Y');
```

```
05/01/2013
```

```
SELECT STR_TO_DATE('13:30:17','%H:%i');
```

```
01:30:00 PM
```

```
SELECT STR_TO_DATE('09:30:17 PM','%r');
```

```
09:30:17 PM
```

```
SELECT STR_TO_DATE('0,0,0000','%d,%m,%Y');
```

```
00/00/0000
```

The following example shows the case when the system parameter **intl_date_lang** is “de_DE”. The German Oktober is interpreted to 10.

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT STR_TO_DATE('3 Oktober 2009', '%d %M %Y');
```

```
10/03/2009
```

Note

When the charset is ISO-8859-1, the language that can be changed by the system parameter **intl_date_lang** is “ko_KR” and “tr_TR” except “en_US”. If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`).

The following example shows which converts a date/time string with timezone information into DATETIMETZ type value.

```
SELECT STR_TO_DATE('2001-10-11 02:03:04 AM Europe/Bucharest EEST',
'%Y-%m-%d %h:%i:%s %p %TZR %TZD');
```

```
02:03:04.000 AM 10/11/2001 Europe/Bucharest EEST
```

TIME_FORMAT

TIME_FORMAT(*time*, *format*)

The **TIME_FORMAT** function converts the date/time data type value including time value into a string of specified date/time format, and returns the value with the **VARCHAR** data type.

Parameters:

- **time** – A value of a type with time. (TIME, TIMESTAMP, DATETIME, TIMESTAMPTZ or DATETIMETZ)
- **format** – Specifies the output format. Use a string that contains ‘%’ as a specifier. See the table Date/Time Format 2 of `DATE_FORMAT()` function.

Return type:

STRING

When the *format* argument is assigned, the time is output according to the specified language. At this time, the language specified as the **intl_date_lang** system parameter is applied. If **intl_date_lang** system parameter is not set, the language specified when creating DB is applied.

For example, when the language is set to “de_DE” and the format is “%h:%i:%s %p”, “08:46:53 PM” is output as “08:46:53 Nachm.”. When the *format* argument specified does not correspond to the given string, an error is returned.

The following example shows the case when the system parameter **intl_date_lang** is “en_US”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="en_US"';
SELECT TIME_FORMAT(time'22:23:00', '%H %i %s');
```

```
'22 23 00'
```

```
SELECT TIME_FORMAT(time'23:59:00', '%H %h %i %s %f');
```

```
'23 11 59 00 000'
```

```
SELECT SYSTIME, TIME_FORMAT(SYSTIME, '%p');
```

```
08:46:53 PM  'PM'
```

The following example shows the case when the system parameter **intl_date_lang** is “de_DE”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT SYSTIME, TIME_FORMAT(SYSTIME, '%p');
```

```
08:46:53 PM  'Nachm.'
```

Note

When the charset is ISO-8859-1, the language that can be changed by the system parameter **intl_date_lang** is “ko_KR” and “tr_TR” except “en_US”. If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`).

The following outputs the value with a timezone information into a specified format string.

```
SELECT TIME_FORMAT(datetimetz'2001-10-11 02:03:04 AM Europe/
Bucharest EEST', '%h:%i:%s %p %TZR %TZD');
```

```
'02:03:04 AM Europe/Bucharest EEST'
```

TO_CHAR(date_time)

TO_CHAR(*date_time*[, *format*[, *date_lang_string_literal*]])

The **TO_CHAR** (date_time) function converts the value of date/time types (**TIME**, **DATE**, **TIMESTAMP**, **DATETIME**) to a string depending on the table Date/Time Format 1 and then returns the value. The type of the return value is **VARCHAR**.

param date_time:

A value of date/time type. (TIME, DATE, TIMESTAMP, DATETIME, DATETIMETZ, DATETIMELTZ, TIMESTAMPTZ, TIMESTAMPLTZ)

param format:

A format of return value.

param
date_lang_string_literal:

Specifies a language applied to a return value.

rtype:

STRING

When the *format* argument is specified, the *date_time* is output according to the specified language (see the Date/Time Format 1 table). A language is defined by the *date_lang_string_literal*. If *date_lang_string_literal* is omitted, the language specified by the *intl_date_lang* parameter is applied; if the value of *intl_date_lang* is not specified, the language specified when creating DB is applied.

For example, when the language is set to “de_DE” and the format is “HH:MI:SS:AM”, “08:46:53 PM” is output as “08:46:53 Nachm.”. When the *format* argument specified does not correspond to the given *string*, an error is returned.

When the *format* argument is omitted, the *date_time* is output as a string according to the default output format of the “en_US”(see the following table Default Date/Time Output Format for Each Language).

Note

The **CUBRID_DATE_LANG** environment used in earlier version of CUBRID 9.0 is no longer supported.

Default Date/Time Output Format for Each Language

LANG	DATE	TIME	TIMESTAMP	DATETIME	TIMESTAMP WITH TIME ZONE	DATETIME WITH TIME ZONE
en_US	‘MM/DD/YYYY’	‘HH:MI:SS AM’	‘HH:MI:SS AM MM/DD/YYYY’	‘HH:MI:SS.FF AM MM/DD/YYYY’	‘HH:MI:SS AM MM/DD/YYYY TZR’	‘HH:MI:SS.FF AM MM/DD/YYYY TZR’
de_DE	‘DD.MM.YYYY’	‘HH24:MI:SS’	‘HH24:MI:SS DD.MM.YYYY’	‘HH24:MI:SS.FF DD.MM.YYYY’	‘HH24:MI:SS DD.MM.YYYY TZR’	‘HH24:MI:SS.FF DD.MM.YYYY TZR’
es_ES	‘DD.MM.YYYY’	‘HH24:MI:SS’	‘HH24:MI:SS DD.MM.YYYY’	‘HH24:MI:SS.FF DD.MM.YYYY’	‘HH24:MI:SS DD/MM/YYYY TZR’	‘HH24:MI:SS.FF DD/MM/YYYY TZR’
fr_FR	‘DD.MM.YYYY’	‘HH24:MI:SS’	‘HH24:MI:SS	‘HH24:MI:SS.FF	‘HH24:MI:SS DD/MM/YYYY TZR’	‘HH24:MI:SS.FF DD/

LANG	DATE	TIME	TIMESTAMP	DATETIME	TIMESTAMP WITH TIME ZONE	DATETIME WITH TIME ZONE
			DD.MM.YYYY'	DD.MM.YYYY'		MM/YYYY TZR'
it_IT	'DD.MM.YYYY'	'HH24:MI:SS'	'HH24:MI:SS DD.MM.YYYY'	'HH24:MI:SS.FF DD.MM.YYYY'	'HH24:MI:SS DD/MM/YYYY TZR'	'HH24:MI:SS.FF DD/MM/YYYY TZR'
ja_JP	'YYYY/MM/DD'	'HH24:MI:SS'	'HH24:MI:SS YYYY/MM/DD'	'HH24:MI:SS.FF YYYY/MM/DD'	'HH24:MI:SS YYYY/MM/DD TZR'	'HH24:MI:SS.FF YYYY/MM/DD TZR'
km_KH	'DD/MM/YYYY'	'HH24:MI:SS'	'HH24:MI:SS DD/MM/YYYY'	'HH24:MI:SS.FF DD/MM/YYYY'	'HH24:MI:SS DD/MM/YYYY TZR'	'HH24:MI:SS.FF DD/MM/YYYY TZR'
ko_KR	'YYYY.MM.DD'	'HH24:MI:SS'	'HH24:MI:SS YYYY.MM.DD'	'HH24:MI:SS.FF YYYY.MM.DD'	'HH24:MI:SS YYYY.MM.DD TZR'	'HH24:MI:SS.FF YYYY.MM.DD TZR'
tr_TR	'DD.MM.YYYY'	'HH24:MI:SS'	'HH24:MI:SS DD.MM.YYYY'	'HH24:MI:SS.FF DD.MM.YYYY'	'HH24:MI:SS DD.MM.YYYY TZR'	'HH24:MI:SS.FF DD.MM.YYYY TZR'
vi_VN	'DD/MM/YYYY'	'HH24:MI:SS'	'HH24:MI:SS DD/MM/YYYY'	'HH24:MI:SS.FF DD/MM/YYYY'	'HH24:MI:SS DD/MM/YYYY TZR'	'HH24:MI:SS.FF DD/MM/YYYY TZR'

LANG	DATE	TIME	TIMESTAMP	DATETIME	TIMESTAMP WITH TIME ZONE	DATETIME WITH TIME ZONE
zh_CN	'YYYY-MM-DD'	'HH24:MI:SS'	'HH24:MI:SS YYYY-MM-DD'	'HH24:MI:SS.FF YYYY-MM-DD'	'HH24:MI:SS YYYY-MM-DD TZR'	'HH24:MI:SS.FF YYYY-MM-DD TZR'
ro_RO	'DD.MM.YYYY'	'HH24:MI:SS'	'HH24:MI:SS DD.MM.YYYY'	'HH24:MI:SS.FF DD.MM.YYYY'	'HH24:MI:SS DD.MM.YYYY TZR'	'HH24:MI:SS.FF DD.MM.YYYY TZR'

Date/Time Format 1

Format Element	Description
CC	Century
YYYY , YY	Year with 4 numbers, Year with 2 numbers
Q	Quarter (1, 2, 3, 4; January - March = 1)
MM	Month (01-12; January = 01) *Note: MI represents the minute of hour.*
MONTH	Month in characters
MON	Abbreviated month name
DD	Day (1 - 31)

Format Element	Description
DAY	Day of the week in characters
DY	Abbreviated day of the week
D or **d**	Day of the week in numbers (1 - 7)
AM or **PM**	AM/PM
A.M. or **P.M.**	AM/PM with periods
HH or **HH12**	Hour (1 -12)
HH24	Hour (0 - 23)
MI	Minute (0 - 59)
SS	Second (0 - 59)
FF	Millisecond (0-999)
- / , . ; : “text”	Punctuation and quotation marks are represented as they are in the result
TZD	Daylight saving information. e.g. KST, KT, EET
TZH	Timezone hour offset. e.g. +09, -09
TZM	Timezone minute offset. e.g. +00, +30

Note

TZR, TZD, TZH, TZM can be used only in timezone types.

Note

A format to specify a number after "TZD"

A number can be added after "TZD". This format is from TZD2 to TZD11; When you use a general character as a separator of a string, this format can be used.

```
SELECT STR_TO_DATE('09:30:17 20140307XEESTXEurope/
Bucharest','%h:%i:%s %Y%d%mX%TZD4X%TZR');
```

```
09:30:17.000 AM 07/03/2014 Europe/Bucharest EEST
```

When you use a general character, 'X', as a separator to separate each value, TZD value's string length is variable; therefore, it is confused to separate TZD value and a separator. In this case, TZD values' length should be specified.

Example of date_lang_string_literal

	date_lang_string_literal	
Format Element	**'en_US'**	**'ko_KR'**
MONTH	JANUARY	1월
MON	JAN	1
DAY	MONDAY	월요일
DY	MON	월

	date_lang_string_literal	
Format Element	**'en_US'**	**'ko_KR'**
Month	January	1월
Mon	Jan	1
Day	Monday	월요일
Dy	Mon	월
month	january	1월
mon	jan	1
day	monday	월요일
Dy	mon	월
AM	AM	오전
Am	Am	오전
am	am	오전
A.M.	A.M.	오후
A.m.	A.m.	오전
a.m.	a.m.	오전

	date_lang_string_literal	
Format Element	**'en_US'**	**'ko_KR'**
PM	PM	오후
Pm	Pm	오후
pm	pm	오후
P.M.	P.M.	오후
P.m.	P.m.	오후
p.m.	p.m.	오후

Example of Format Digits of Return Value

Format Element	en_US Digits | ko_KR Digits	
MONTH(Month, month)	9	4
MON(Mon, mon)	3	2
DAY(Day, day)	9	6
DY(Dy, dy)	3	2
HH12, HH24	2	2
"text"	The length of the text	The length of the text

Format Element	en_US Digits | ko_KR Digits	
Other formats	Same as the length of the format	Same as the length of the format

The following example shows the query executed by setting the language and charset to “en_US.iso88591”.

```
-- create database testdb en_US.iso88591

--creating a table having date/time type columns
CREATE TABLE datetime_tbl(a TIME, b DATE, c TIMESTAMP, d DATETIME);
INSERT INTO datetime_tbl VALUES(SYSTIME, SYSDATE, SYSTIMESTAMP,
SYSDATETIME);

--selecting a VARCHAR type string from the data in the specified
format
SELECT TO_CHAR(b, 'DD, DY , MON, YYYY') FROM datetime_tbl;
```

```
'20, TUE , AUG, 2013'
```

```
SELECT TO_CHAR(c, 'HH24:MI, DD, MONTH, YYYY') FROM datetime_tbl;
```

```
'17:00, 20, AUGUST   , 2013'
```

```
SELECT TO_CHAR(d, 'HH12:MI:SS:FF pm, YYYY-MM-DD-DAY') FROM
datetime_tbl;
```

```
'05:00:58:358 pm, 2013-08-20-TUESDAY  '
```

```
SELECT TO_CHAR(TIMESTAMP'2009-10-04 22:23:00', 'Day Month yyyy');
```

```
'Sunday    October   2009'
```

The following example shows an additional language parameter given to the **TO_CHAR** function in the database created above. When the charset is ISO-8859-1, setting the language parameter of

the **TO_CHAR** function to “tr_TR” or “ko_KR” is allowed, but the other languages are not allowed. To use all languages by setting the language parameter of **TO_CHAR**, the charset when creating DB should be UTF-8.

```
SELECT TO_CHAR(TIMESTAMP'2009-10-04 22:23:00', 'Day Month
yyyy','ko_KR');
```

```
'Iryoil    10wol 2009'
```

```
SELECT TO_CHAR(TIMESTAMP'2009-10-04 22:23:00', 'Day Month
yyyy','tr_TR');
```

```
'Pazar     Ekim    2009'
```

Note

- In the function that interprets the month/day in characters and AM/PM differently by language, if the charset is ISO-8859-1, the language can be changed to “ko_KR” or “tr_TR” only by using the **intl_date_lang** except “en_US” (see the above example). If the charset is UTF-8, the language can be changed to any language supported by CUBRID. By setting the intl_date_lang system parameter or by specifying the language parameter of the **TO_CHAR** function, the language can be changed to one of all the languages supported by CUBRID (see *date_lang_string_literal* of “Syntax” above). For a list of functions that interpret the date/time differently by language, see the description of the **intl_date_lang** system parameter.

```
-- change date locale as "de_DE" and run the below query.
-- This case is failed because database locale, en_US's charset
is ISO-8859-1
-- and 'de_DE' only supports UTF-8 charset.

SELECT TO_CHAR(TIMESTAMP'2009-10-04 22:23:00', 'Day Month
yyyy','de_DE');
```

```
ERROR: before ' , 'Day Month yyyy','de_DE'); '
Locales for language 'de_DE' are not available with charset
'iso8859-1'.
```

The following example shows how to set the language parameter of the **TO_CHAR** function to “de_DE” when you created DB with the locale “en_US.utf8”.

```
SELECT TO_CHAR(TIMESTAMP'2009-10-04 22:23:00', 'Day Month
yyyy','de_DE');
```

```
'Sonntag   Oktober 2009'
```

- If the first argument is zerodate and the second argument has a literal like ‘Month’, ‘Day’, then TO_CHAR function returns NULL.

```
SELECT TO_CHAR(timestamp '0000-00-00 00:00:00', 'Month Day
YYYY');
```

```
NULL
```

The following is an example to output date/time type with timezone in TO_CHAR function.

If you don’t define a format, it outputs as the following format

```
SELECT TO_CHAR(datetimetz'2001-10-11 02:03:04 AM Europe/Bucharest
EEST');
```

```
'02:03:04.000 AM 10/11/2001 Europe/Bucharest EEST'
```

If you define a format, it outputs as the defined format.

```
SELECT TO_CHAR(datetimetz'2001-10-11 02:03:04 AM Europe/Bucharest
EEST', 'MM/DD/YYYY HH24:MI TZR TZD TZH TZM');
```

```
'10/11/2001 02:03 Europe/Bucharest EEST +03 +00'
```

TO_CHAR(number)

TO_CHAR(*number*[, *format*[, *number_lang_string_literal*]])

The **TO_CHAR** function converts a numeric data type to a character string according to Number Format and returns it. The type of the return value is **VARCHAR** .

Parameters:

- **number** – Specifies an expression that returns numeric data type string. If the input value is **NULL**, **NULL** is returned. If the input value is character type, the character itself is returned.
- **format** – Specifies a format of return value. If format is not specified, all significant figures are returned as character string by default. If the value is **NULL**, **NULL** is returned.
- **number_lang_string_literal** – Specifies the language to be applied to the input value.

Return type:

STRING

If *format* argument is specified, *number* is converted into a character string according to a specified language. At this time, the language is defined by the *number_lang_string_literal* argument. If *number_lang_string_literal* is omitted, the language specified by **intl_number_lang** system parameter is applied; if **intl_number_lang** is not set, the language specified when creating DB is applied.

For example, if the language is one of the European languages, such as “de_DE” or “fr_FR”, “.” is printed out as a thousands separator and “,” is printed out as a decimal point. If the *format* argument does not correspond to the given string, the function returns an error.

If *format* argument is omitted, *number* is converted into a character string according to the default format of a specified language(see the table Default Output of Number for Each Language).

Number Format

Format Element	Example	Description
9	9999	The number of 9’s represents the number of significant figures to be

Format Element	Example	Description
		returned. If the number of significant figures specified in the format is not sufficient, only the decimal part is rounded. If it is less than the number of digits in an integer, # is outputted. If the number of significant figures specified in the format is sufficient, the part preceding the integer part is filled with space characters and the decimal part is filled with 0.
0	0999	If the number of significant figures specified in the format is sufficient, the part preceding the integer part is filled with 0, not space characters before the value is returned.
S	S9999	Outputs the negative/ positive sign in the specified position. These signs can be used only at the beginning of character string.
C	C9999	Returns the ISO currency code at the specified position.
, (comma)	9,999	Returns a comma (“,”) at the specified position. Multiple

Format Element	Example	Description
		commas are allowed in the format.
. (decimal point)	9.999	Returns a decimal point (“.”) which distinguishes between a decimal and an at the specified position. Only one decimal point is allowed in the format. see the table, Default Output of Number for Each Language
EEEE	9.99EEEE	Returns a scientific notation number.

Default Output of Number for Each Language

Language	Locale	Number of Digits	Decimal Symbol	Example of Number Usage
English	en_US	,(comma)	.(period)	123,456,789.012
German	de_DE	.(period)	,(comma)	123.456.789,012
Spanish	es_ES	.(period)	,(comma)	123.456.789,012
French	fr_FR	.(period)	,(comma)	123.456.789,012
Italian	it_IT	.(period)	,(comma)	

Language	Locale	Number of Digits	Decimal Symbol	Example of Number Usage
				123.456.789,012
Japanese	ja_JP	,(comma)	.(period)	123,456,789.012
Cambodian	km_KH	.(period)	,(comma)	123.456.789,012
Korean	ko_KR	,(comma)	.(period)	123,456,789.012
Turkish	tr_TR	.(period)	,(comma)	123.456.789,012
Vietnamese	vi_VN	.(period)	,(comma)	123.456.789,012
Chinese	zh_CN	,(comma)	.(period)	123,456,789.012
Romanian	ro_RO	.(period)	,(comma)	123.456.789,012

The following example shows execution of the database by the locale specified when creating DB to “en_US.utf8”.

```
--selecting a string casted from a number in the specified format
SELECT TO_CHAR(12345,'S999999'), TO_CHAR(12345,'S099999');
```

```
' +12345'              '+012345'
```

```
SELECT TO_CHAR(1234567,'9,999,999,999');
```

```
'    1,234,567'
```

```
SELECT TO_CHAR(1234567,'9.999.999.999');
```

```
'##############'
```

```
SELECT TO_CHAR(123.4567,'99'), TO_CHAR(123.4567,'999.99999'),
TO_CHAR(123.4567,'99999.999');
```

```
'##'                  '123.45670'           '  123.457'
```

The following example shows command execution by setting the value of the **intl_number_lang** system parameter to “de_DE”. In the number output format of most European countries such as Germany and France, “.” is the cipher identifier and “,” is the decimal point symbol.

```
SET SYSTEM PARAMETERS 'intl_number_lang="de_DE"';

--selecting a string casted from a number in the specified format
SELECT TO_CHAR(12345,'S999999'), TO_CHAR(12345,'S099999');
```

```
' +12345'             '+012345'
```

```
SELECT TO_CHAR(1234567,'9,999,999,999');
```

```
'##############'
```

```
SELECT TO_CHAR(1234567,'9.999.999.999');
```

```
'    1.234.567'
```

```
SELECT TO_CHAR(123.4567,'99'), TO_CHAR(123.4567,'999,99999'),
TO_CHAR(123.4567,'99999,999');
```

```
'##'                    '123,45670'             '  123,457'
```

```
SELECT TO_CHAR(123.4567,'99','en_US'), TO_CHAR(123.
4567,'999.99999','en_US'), TO_CHAR(123.4567,'99999.999','en_US');
```

```
'##'                    '123.45670'             '  123.457'
```

```
SELECT TO_CHAR(1.234567,'99.999EEEE','en_US'), TO_CHAR(1.
234567,'99,999EEEE','de_DE'), to_char(123.4567);
```

```
'1.235E+00'             '1,235E+00'             '123,4567'
```

TO_DATE

TO_DATE(*string*[, *format*[, *date_lang_string_literal*]])

The **TO_DATE** function interprets a character string based on the date format given as an argument, converts it to a **DATE** type value, and returns it. For the format, see Date/Time Format 1.

Parameters:

- **string** – A character string
- **format** – Specifies a format of return value to be converted as **DATE** type. See Date/Time Format 1. If the value is **NULL**, **NULL** is returned.
- **date_lang_string_literal** – Specifies the language for the input value to be applied.

Return type:

DATE

When the *format* argument is specified, the *string* is interpreted according to the specified language. At this time, the language is set by *date_lang_string_literal* argument. If *date_lang_string_literal* argument is not set, the language is specified by the **intl_date_lang** system parameter; if the value of **intl_date_lang** is not set, the language is applied by the language specified when creating DB.

For example, when a language is "de_DE" and *string* is "12.mai.2012", and *format* is "DD.mon.YYYY", it is interpreted as May 12th, 2012.When the *format* parameter specified does not correspond to the given *string*, an error is returned.

When the *format* argument is omitted, *string* is interpreted as the CUBRID default format (refer to Recommended Format for Strings in Date/Time Type) and if it fails, *string* is interpreted as the language format (see the table Default Output Format of Language in the `TO_CHAR()`) by **intl_date_lang**. If the value of **intl_date_lang** is not set, the language is applied by the language specified when creating DB.

For example, when a language is "de_DE", the acceptable strings for **DATE** type are "MM/DD/YYYY", CUBRID default format and "DD.MM.YYYY", "de_DE" default format.

The following example shows the query executed by the locale specified when creating DB to "en_US.utf8".

```
--selecting a date type value casted from a string in the specified
format

SELECT TO_DATE('12/25/2008');
```

```
12/25/2008
```

```
SELECT TO_DATE('25/12/2008', 'DD/MM/YYYY');
```

```
12/25/2008
```

```
SELECT TO_DATE('081225', 'YYMMDD');
```

```
12/25/2008
```

```
SELECT TO_DATE('2008-12-25', 'YYYY-MM-DD');
```

```
12/25/2008
```

The following example shows the query executed when the system parameter **intl_date_lang** is "de_DE".

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT TO_DATE('25.12.2012');
```

```
12/25/2012
```

```
SELECT TO_DATE('12/mai/2012','dd/mon/yyyy', 'de_DE');
```

```
05/12/2012
```

Note

When the charset is ISO-8859-1, the language that can be changed by the system parameter **intl_date_lang** is "ko_KR" and "tr_TR" except "en_US". If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`).

TO_DATETIME

TO_DATETIME(*string*[, *format*[, *date_lang_string_literal*]])

The **TO_DATETIME** function interprets a character string based on the date-time format given as an argument, converts it to a **DATETIME** type value, and returns it. For the format, see Date/Time Format 1.

Parameters:

- **string** – A character string
- **format** – Specifies a format of return value to be converted as **DATETIME** type. See the table, Date/Time Format 1. If the value is **NULL**, **NULL** is returned.
- **date_lang_string_literal** – Specifies the language for the input value to be applied.

Return type:

DATETIME

When the *format* argument is specified, the *string* is interpreted according to the specified language.

For example, when a language is "de_DE" and *string* is "12/mai/2012 12:10:00 Nachm." and *format* is "DD/MON/YYYY HH:MI:SS AM", it is interpreted as May 12th, 2012 12:10:00 PM. At this time, the language is set by *date_lang_string_literal* argument. If *date_lang_string_literal* argument is not set, the language is specified by the **intl_date_lang** system parameter; if the value of **intl_date_lang** is not set, the language is specified by the language specified when creating DB. When the *format* parameter specified does not correspond to the given *string*, an error is returned.

When the *format* argument is omitted, *string* is interpreted as the CUBRID default format (refer to Recommended Format for Strings in Date/Time Type) and if it fails, *string* is interpreted as the language format (see the table Default Output Format of Language in the `TO_CHAR()`) by **intl_date_lang**. If the value of **intl_date_lang** is not set, the language is applied by the language specified when creating DB.

For example, when a language is "de_DE", the acceptable strings for **DATETIME** type are "HH:MI:SS.FF AM MM/DD/YYYY", CUBRID default format and "HH24:MI:SS.FF DD.MM.YYYY", "de_DE" default format.

Note

The **CUBRID_DATE_LANG** environment used in earlier version of CUBRID 9.0 is no longer supported.

The following example shows execution of the database by setting the environment variable **CUBRID_CHARSET** to "en_US".

```
--selecting a datetime type value casted from a string in the
specified format

SELECT TO_DATETIME('13:10:30 12/25/2008');
```

```
01:10:30.000 PM 12/25/2008
```

```
SELECT TO_DATETIME('08-Dec-25 13:10:30.999', 'YY-Mon-DD
HH24:MI:SS.FF');
```

```
01:10:30.999 PM 12/25/2008
```

```
SELECT TO_DATETIME('DATE: 12-25-2008 TIME: 13:10:30.999', '"DATE:"
MM-DD-YYYY "TIME:" HH24:MI:SS.FF');
```

```
01:10:30.999 PM 12/25/2008
```

The following example shows the case when the system parameter **intl_date_lang** is “de_DE”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT TO_DATETIME('13:10:30.999 25.12.2012');
```

```
01:10:30.999 PM 12/25/2012
```

```
SELECT TO_DATETIME('12/mai/2012 12:10:00 Nachm.','DD/MON/YYYY
HH:MI:SS AM', 'de_DE');
```

```
12:10:00.000 PM 05/12/2012
```

Note

When the charset is ISO-8859-1, the language that can be changed in **TO_DATETIME** function is “ko_KR” and “tr_TR” except “en_US”. If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`.

TO_DATETIME_TZ

TO_DATETIME_TZ(*string*[, *format*[, *date_lang_string_literal*]])

TO_DATETIME_TZ function is the same as `TO_DATETIME()` function except that this function can include a timezone information on this input string.

Return type:

DATETIMETZ

```
SELECT TO_DATETIME_TZ('13:10:30 Asia/Seoul 12/25/2008', 'HH24:MI:SS
TZR MM/DD/YYYY');
```

```
01:10:30.000 PM 12/25/2008 Asia/Seoul
```

TO_NUMBER

TO_NUMBER(*string*[, *format*])

The **TO_NUMBER** function interprets a character string based on the number format given as an argument, converts it to a **NUMERIC** type value, and returns it.

Parameters:

- **string** – Specifies an expression that returns character string. If the value is **NULL**, **NULL** is returned.
- **format** – Specifies a format of return value to be converted as **NUMBER** type. See Number Format. If it is omitted, NUMERIC(38,0) value is returned.

Return type:

NUMERIC

When the *format* argument is assigned, the string is interpreted according to the specified language. At this time, the language is specified by the **intl_number_lang** system parameter. If the value of **intl_number_lang** is not set, the language specified when creating DB is applied.

For example, when the language is one of the European languages, such as “de_DE” and “fr_FR”, “.” is interpreted as the cipher identifier and “,” as the decimal point symbol. When the format parameter specified does not correspond to the given string, an error is returned.

If the *format* argument is omitted, string is interpreted according to the default output format set by **intl_number_lang** (see Default Output of Number for Each Language). When the **intl_number_lang** is not set, the language specified when creating DB is applied.

The following example shows execution of the database by setting the value of system parameter **intl_number_lang** as “en_US”.

```
SET SYSTEM PARAMETERS 'intl_number_lang="en_US"';

--selecting a number casted from a string in the specified format
SELECT TO_NUMBER('-1234');
```

```
-1234
```

```
SELECT TO_NUMBER('12345','999999');
```

```
12345
```

```
SELECT TO_NUMBER('12,345.67','99,999.999');
```

```
12345.670
```

```
SELECT TO_NUMBER('12345.67','99999.999');
```

```
12345.670
```

The following example shows command execution on the database by setting the value of the **intl_number_lang** system parameter to “de_DE”. In the number output format of most European countries, such as Germany and France, “.” is the cipher identifier and “,” is the decimal point symbol.

```
SET SYSTEM PARAMETERS 'intl_number_lang="de_DE"';
SELECT TO_NUMBER('12.345,67','99.999,999');
```

```
12345.670
```

TO_TIME

TO_TIME(*string*[, *format*[, *date_lang_string_literal*]])

The **TO_TIME** function interprets a character string based on the time format given as an argument, converts it to a **TIME** type value, and returns it. For the format, see Date/Time Format 1.

Parameters:

- **string** – Specifies an expression that returns character string. If the value is **NULL**, **NULL** is returned.
- **format** – Specifies a format of return value to be converted as **TIME** type. See Date/Time Format 1. If the value is **NULL**, **NULL** is returned.
- **date_lang_string_literal** – Specifies the language for the input value to be applied.

Return type:

TIME

When the *format* argument is specified, the *string* is interpreted according to the specified language. At this time, the language is set by *date_lang_string_literal* argument. If *date_lang_string_literal* argument is not set, the language is specified by the **intl_date_lang** system parameter; if the value of **intl_date_lang** is not set, the language specified when creating DB is applied. When the *format* parameter does not correspond to the given *string*, an error is returned.

For example, when a language is “de_DE” and *string* is “10:23:00 Nachm.”, and *format* is “HH/MI/SS/AM, it is interpreted as 10:23:00 PM.

When the *format* argument is omitted, *string* is interpreted as the CUBRID default format (refer to Recommended Format for Strings in Date/Time Type) and if it fails, *string* is interpreted as the language format (see the table Default Output Format of Language in the `TO_CHAR()`) by **intl_date_lang**. If the value of **intl_date_lang** is not set, the language is applied by the language specified when creating DB.

For example, when a language is “de_DE”, the acceptable strings for **TIME** type are “HH:MI:SS AM”, CUBRID default format and “HH24:MI:SS”, “de_DE” default format.

Note

The **CUBRID_DATE_LANG** environment used in earlier version of CUBRID 9.0 is no longer supported.

The following example shows execution of the database by setting the value of system parameter **intl_date_lang** as “en_US”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="en_US"';

--selecting a time type value casted from a string in the specified
format

SELECT TO_TIME ('13:10:30');
```

```
01:10:30 PM
```

```
SELECT TO_TIME('HOUR: 13 MINUTE: 10 SECOND: 30', '"HOUR:" HH24
"MINUTE:" MI "SECOND:" SS');
```

```
01:10:30 PM
```

```
SELECT TO_TIME ('13:10:30', 'HH24:MI:SS');
```

```
01:10:30 PM
```

```
SELECT TO_TIME ('13:10:30', 'HH12:MI:SS');
```

```
ERROR: Conversion error in date format.
```

The following example shows the case when the system parameter **intl_date_lang** is “de_DE”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT TO_TIME('13:10:30');
```

```
01:10:30 PM
```

```
SELECT TO_TIME('10:23:00 Nachm.', 'HH:MI:SS AM');
```

```
10:23:00 PM
```

Note

When the charset is ISO-8859-1, the language that can be changed by the system parameter **intl_date_lang** is “ko_KR” and “tr_TR” except “en_US”. If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`).

TO_TIMESTAMP

TO_TIMESTAMP(*string*[, *format*[, *date_lang_string_literal*]])

The **TO_TIMESTAMP** function interprets a character string based on the time format given as an argument, converts it to a **TIMESTAMP** type value, and returns it. For the format, see Date/Time Format 1.

Parameters:

- **string** – Specifies an expression that returns character string. If the value is **NULL**, **NULL** is returned.
- **format** – Specifies a format of return value to be converted as **TIMESTAMP** type. See Date/Time Format 1. If the value is **NULL**, **NULL** is returned.
- **date_lang_string_literal** – Specifies the language for the input value to be applied.

Return type:

TIMESTAMP

When the *format* argument is specified, the *string* is interpreted according to the specified language. The language is set by *date_lang_string_literal* argument. If *date_lang_string_literal* argument is not set, the language is specified by the **intl_date_lang** system parameter; if the value of **intl_date_lang** is not set, the language specified when creating DB is applied.

For example, when a language is “de_DE” and *string* is “12/mai/2012 12:10:00 Nachm.”, and *format* is “DD/MON/YYYY HH:MI:SS AM”, it is interpreted as May 12th, 2012, 12:10:00 AM. When the *format* parameter specified does not correspond to the given string, an error is returned.

When the *format* argument is omitted, *string* is interpreted as the CUBRID default format(refer to Recommended Format for Strings in Date/Time Type) and if it fails, *string* is interpreted as the language format (see the table Default Output Format of Language in the `TO_CHAR()`) by **intl_date_lang**. If the value of **intl_date_lang** is not set, the language is applied by the language specified when creating DB.

For example, when a language is “de_DE”, the acceptable strings for **TIMESTAMP** type are “HH:MI:SS AM MM/DD/YYYY”, CUBRID default format and “HH24:MI:SS DD.MM.YYYY”, “de_DE” default format.

The following example shows execution of the database by setting the value of system parameter **intl_date_lang** as “en_US”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="en_US"';

--selecting a timestamp type value casted from a string in the
specified format

SELECT TO_TIMESTAMP('13:10:30 12/25/2008');
```

```
01:10:30 PM 12/25/2008
```

```
SELECT TO_TIMESTAMP('08-Dec-25 13:10:30', 'YY-Mon-DD HH24:MI:SS');
```

```
01:10:30 PM 12/25/2008
```

```
SELECT TO_TIMESTAMP('YEAR: 2008 DATE: 12-25 TIME: 13:10:30',
'"YEAR:" YYYY "DATE:" MM-DD "TIME:" HH24:MI:SS');
```

```
01:10:30 PM 12/25/2008
```

The following example shows the case when the system parameter **intl_date_lang** is “de_DE”.

```
SET SYSTEM PARAMETERS 'intl_date_lang="de_DE"';
SELECT TO_TIMESTAMP('13:10:30 25.12.2008');
```

```
01:10:30 PM 12/25/2008
```

```
SELECT TO_TIMESTAMP('10:23:00 Nachm.', 'HH12:MI:SS AM');
```

```
10:23:00 PM 08/01/2012
```

Note

When the charset is ISO-8859-1, the language that can be changed by the system parameter **intl_date_lang** is “ko_KR” and “tr_TR” except “en_US”. If the charset is UTF-8, it can be changed to any language supported by CUBRID. For details, see Note in the `TO_CHAR()`).

TO_TIMESTAMP_TZ

TO_TIMESTAMP_TZ(*string*[, *format*[, *date_lang_string_literal*]])

TO_TIMESTAMP_TZ function is the same as `TO_TIMESTAMP()` function except that this function can include a timezone information on this input string.

rtype:

TIMESTAMPTZ

```
SELECT TO_TIMESTAMP_TZ('13:10:30 Asia/Seoul 12/25/2008',
'HH24:MI:SS TZR MM/DD/YYYY');
```

```
01:10:30 PM 12/25/2008 Asia/Seoul
```

Aggregate/Analytic Functions

Contents

Aggregate/Analytic Functions 631

- Overview 633
- Aggregate vs. Analytic 636
- Analytic functions which “ORDER BY” clause must be specified in OVER function 637
- AVG 638
- COUNT 640
- CUME_DIST 642
- DENSE_RANK 646
- FIRST_VALUE 648
- GROUP_CONCAT 650
- LAG 652
- LAST_VALUE 653
- LEAD 655
- MAX 657
- MEDIAN 659
- MIN 660
- NTH_VALUE 661
- NTILE 663
- PERCENT_RANK 665
- PERCENTILE_CONT 670
- PERCENTILE_DISC 673

- RANK 675
- ROW_NUMBER 677
- STDDEV, STDDEV_POP 678
- STDDEV_SAMP 681
- SUM 683
- VARIANCE, VAR_POP 685
- VAR_SAMP 688
- JSON_ARRAYAGG 690
- JSON_OBJECTAGG 690

Overview

Aggregate/Analytic function is used when you want to analyze data and extract some results.

- Aggregate function returns the grouped results and it returns only the columns which are grouped.
- Analytic function returns the grouped results, but it includes the ungrouped columns, so it can return multiple rows for one group.

For examples, aggregate/analytic function can be used to get the answers as follows.

1. What is the total sales amount per year?
2. How can you display the sales amount from the highest one's month by grouping each year?
3. How can you display the accumulated sales amount as annual and monthly order by grouping each year?

You can answer for No. 1 with aggregate function. for No. 2 and No. 3, you can answer with analytic function. Above questions can be written as following SQL statements.

Below is the table which stores the sales amounts per month of each year.

```
CREATE TABLE sales_mon_tbl (
    yyyy INT,
    mm INT,
```

```
    sales_sum INT
);

INSERT INTO sales_mon_tbl VALUES
    (2000, 1, 1000), (2000, 2, 770), (2000, 3, 630), (2000, 4, 890),
    (2000, 5, 500), (2000, 6, 900), (2000, 7, 1300), (2000, 8, 1800),
    (2000, 9, 2100), (2000, 10, 1300), (2000, 11, 1500), (2000, 12,
1610),
    (2001, 1, 1010), (2001, 2, 700), (2001, 3, 600), (2001, 4, 900),
    (2001, 5, 1200), (2001, 6, 1400), (2001, 7, 1700), (2001, 8,
1110),
    (2001, 9, 970), (2001, 10, 690), (2001, 11, 710), (2001, 12,
880),
    (2002, 1, 980), (2002, 2, 750), (2002, 3, 730), (2002, 4, 980),
    (2002, 5, 1110), (2002, 6, 570), (2002, 7, 1630), (2002, 8,
1890),
    (2002, 9, 2120), (2002, 10, 970), (2002, 11, 420), (2002, 12,
1300);
```

1. What is the total sales amount per year?

```
SELECT yyyy, sum(sales_sum)
FROM sales_mon_tbl
GROUP BY yyyy;
```

```
         yyyy  sum(sales_sum)
==============================
         2000           14300
         2001           11870
         2002           13450
```

1. How can you display the sales amount from the highest one's month by grouping each year?

```
SELECT yyyy, mm, sales_sum, RANK() OVER (PARTITION BY yyyy ORDER BY
sales_sum DESC) AS rnk
FROM sales_mon_tbl;
```

```
         yyyy           mm    sales_sum          rnk
=============================================================
         2000            9         2100            1
         2000            8         1800            2
         2000           12         1610            3
         2000           11         1500            4
         2000            7         1300            5
         2000           10         1300            5
         2000            1         1000            7
```

```
        2000            6          900            8
        2000            4          890            9
        2000            2          770           10
        2000            3          630           11
        2000            5          500           12
        2001            7         1700            1
        2001            6         1400            2
        2001            5         1200            3
        2001            8         1110            4
        2001            1         1010            5
        2001            9          970            6
        2001            4          900            7
        2001           12          880            8
        2001           11          710            9
        2001            2          700           10
        2001           10          690           11
        2001            3          600           12
        2002            9         2120            1
        2002            8         1890            2
        2002            7         1630            3
        2002           12         1300            4
        2002            5         1110            5
        2002            1          980            6
        2002            4          980            6
        2002           10          970            8
        2002            2          750            9
        2002            3          730           10
        2002            6          570           11
        2002           11          420           12
```

1. How can you display the accumulated sales amount as annual and monthly order by grouping each year?

```
SELECT yyyy, mm, sales_sum, SUM(sales_sum) OVER (PARTITION BY yyyy
ORDER BY yyyy, mm) AS a_sum
FROM sales_mon_tbl;
```

```
        yyyy           mm    sales_sum        a_sum
=============================================================
        2000            1         1000         1000
        2000            2          770         1770
        2000            3          630         2400
        2000            4          890         3290
        2000            5          500         3790
        2000            6          900         4690
        2000            7         1300         5990
        2000            8         1800         7790
        2000            9         2100         9890
        2000           10         1300        11190
```

```
2000          11          1500        12690
2000          12          1610        14300
2001           1          1010         1010
2001           2           700         1710
2001           3           600         2310
2001           4           900         3210
2001           5          1200         4410
2001           6          1400         5810
2001           7          1700         7510
2001           8          1110         8620
2001           9           970         9590
2001          10           690        10280
2001          11           710        10990
2001          12           880        11870
2002           1           980          980
2002           2           750         1730
2002           3           730         2460
2002           4           980         3440
2002           5          1110         4550
2002           6           570         5120
2002           7          1630         6750
2002           8          1890         8640
2002           9          2120        10760
2002          10           970        11730
2002          11           420        12150
2002          12          1300        13450
```

Aggregate vs. Analytic

Aggregate function returns one result based on the group of rows. When the **GROUP BY** clause is included, a one-row aggregate result per group is returned. When the **GROUP BY** clause is omitted, a one-row aggregate result for all rows is returned. The **HAVING** clause is used to add a condition to the query which contains the **GROUP BY** clause.

Most aggregate functions can use **DISTINCT**, **UNIQUE** constraints. For the **GROUP BY ... HAVING** clause, see GROUP BY ... HAVING Clause.

Analytic function calculates the aggregate value based on the result of rows. The analytic function is different from the aggregate function since it can return one or more rows based on the groups specified by the <*partition_by_clause*> after the **OVER** clause (when this clause is omitted, all rows are regarded as a group).

The analytic function is used along with a new analytic clause, **OVER**, for the existing aggregate functions to allow a variety of statistics for a group of specific rows.

```
function_name ([<argument_list>]) OVER (<analytic_clause>)

<analytic_clause>::=
```

```
     [<partition_by_clause>] [<order_by_clause>]

<partition_by_clause>::=
    PARTITION BY value_expr[, value_expr]...

<order_by_clause>::=
    ORDER BY { expression | position | column_alias } [ ASC | DESC ]
        [, { expression | position | column_alias } [ ASC |
DESC ] ] ...
```

- <*partition_by_clause*>: Groups based on one or more *value_expr*. It uses the **PARTITION BY** clause to partition the query result.
- <*order_by_clause*>: defines the data sorting method in the partition made by <*partition_by_clause*>. The result can be sorted with several keys. When <*partition_by_clause*> is omitted, the data is sorted within the overall result sets. Based on the sorting order, the function is applied to the column values of accumulated records, including the previous values.

The behavior of a query with the expression of ORDER BY/PARTITION BY clause which is used together after the OVER clause is as follows.

- ORDER BY/PARTITION BY <expression with non-constant> (ex: i, sin(i+1)): The expression is used to do ordering/partitioning.
- ORDER BY/PARTITION BY <constant> (ex: 1): Constant is considered as the column position of SELECT list.
- ORDER BY/PARTITION BY <constant expression> (ex: 1+0): Constant is ignored and it is not used to do ordering/partitioning.

Analytic functions which “ORDER BY” clause must be specified in OVER function

The below functions require ordering; therefore, “ORDER BY” clause must be specified inside OVER function. In the case of omitting “ORDER BY” clause, please note that an error occurs or proper ordering is not guaranteed.

- `CUME_DIST()`
- `DENSE_RANK()`
- `LAG()`
- `LEAD()`
- `NTILE()`
- `PERCENT_RANK()`

- `RANK()`
- `ROW_NUMBER()`

AVG

AVG(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

AVG(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The **AVG** function is used as an aggregate function or an analytic function. It calculates the arithmetic average of the value of an expression representing all rows. Only one *expression* is specified as a parameter. You can get the average without duplicates by using the **DISTINCT** or **UNIQUE** keyword in front of the expression or the average of all values by omitting the keyword or by using **ALL**.

param expression:

Specifies an expression that returns a numeric value. An expression that returns a collection-type data is not allowed.

param ALL:

Calculates an average value for all data (default).

param
DISTINCT,DISTINCTROW,UNIQUE:

Calculates an average value without duplicates.

rtype:

DOUBLE

The following example shows how to retrieve the average number of gold medals that Korea won in Olympics in the *demodb* database.

```
SELECT AVG(gold)
FROM participant
WHERE nation_code = 'KOR';
```

```
                avg(gold)
==========================
     9.600000000000000e+00
```

The following example shows how to output the number of gold medals by year and the average number of accumulated gold medals in history, acquired whose nation_code starts with 'AU'.

```
SELECT host_year, nation_code, gold,
    AVG(gold) OVER (PARTITION BY nation_code ORDER BY host_year)
avg_gold
FROM participant WHERE nation_code like 'AU%';
```

```
    host_year  nation_code                gold
avg_gold
==========================================================================
==
         1988  'AUS'                       3
3.000000000000000e+00
         1992  'AUS'                       7
5.000000000000000e+00
         1996  'AUS'                       9
6.333333333333333e+00
         2000  'AUS'                      16
8.750000000000000e+00
         2004  'AUS'                      17
1.040000000000000e+01
         1988  'AUT'                       1
1.000000000000000e+00
         1992  'AUT'                       0
5.000000000000000e-01
         1996  'AUT'                       0
3.333333333333333e-01
         2000  'AUT'                       2
7.500000000000000e-01
         2004  'AUT'                       2
1.000000000000000e+00
```

The following example is removing the "ORDER BY host_year" clause under the **OVER** analysis clause from the above example. The avg_gold value is the average of gold medals for all years, so the value is identical for every year by nation_code.

```
SELECT host_year, nation_code, gold, AVG(gold) OVER (PARTITION BY
nation_code) avg_gold
FROM participant WHERE nation_code LIKE 'AU%';
```

```
    host_year  nation_code                 gold
avg_gold
================================================================================
=====
         2004  'AUS'                        17
1.0400000000000000e+01
         2000  'AUS'                        16
1.0400000000000000e+01
         1996  'AUS'                         9
1.0400000000000000e+01
         1992  'AUS'                         7
1.0400000000000000e+01
         1988  'AUS'                         3
1.0400000000000000e+01
         2004  'AUT'                         2
1.0000000000000000e+00
         2000  'AUT'                         2
1.0000000000000000e+00
         1996  'AUT'                         0
1.0000000000000000e+00
         1992  'AUT'                         0
1.0000000000000000e+00
         1988  'AUT'                         1
1.0000000000000000e+00
```

COUNT

COUNT(*)

COUNT(*) OVER (<analytic_clause>)

COUNT([DISTINCT | DISTINCTROW | UNIQUE | ALL] expression)

COUNT([DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>)

The **COUNT** function is used as an aggregate function or an analytic function. It returns the number of rows returned by a query. If an asterisk (*) is specified, the number of all rows satisfying the condition (including the rows with the **NULL** value) is returned. If the **DISTINCT** or **UNIQUE** keyword is specified in front of the expression, only the number of rows that have a unique value (excluding the rows with the **NULL** value) is returned after duplicates have been removed. Therefore, the value returned is always a big integer and **NULL** is never returned.

param expression:

Specifies an expression.

param ALL:

Gets the number of rows given in the *expression* (default).

param DISTINCT,DISTINCTROW,UNIQUE:

Gets the number of rows without duplicates.

rtype:

BIGINT

A column that has collection type and object domain (user-defined class) can also be specified in the *expression*.

The following example shows how to retrieve the number of Olympic Games that have a mascot in the *demodb* database.

```
SELECT COUNT(*)
FROM olympic
WHERE mascot IS NOT NULL;
```

```
              count(*)
======================
                     9
```

The following example shows how to output the number of players whose nation_code is 'AUT' in *demodb* by accumulating the number of events when the event is changed. The last row shows the number of all players.

```
SELECT nation_code, event, name, COUNT(*) OVER (ORDER BY event) co
FROM athlete WHERE nation_code='AUT';
```

```
  nation_code           event
name                                      co
=============================================================================
==================
  'AUT'                 'Athletics'           'Kiesl
```

```
Theresia'                          2
  'AUT'                 'Athletics'          'Graf
Stephanie'                          2
  'AUT'                 'Equestrian'         'Boor
Boris'                              6
  'AUT'                 'Equestrian'         'Fruhmann
Thomas'                            6
  'AUT'                 'Equestrian'         'Munzner
Joerg'                             6
  'AUT'                 'Equestrian'         'Simon
Hugo'                              6
  'AUT'                 'Judo'               'Heill
Claudia'                           9
  'AUT'                 'Judo'               'Seisenbacher
Peter'                             9
  'AUT'                 'Judo'               'Hartl
Roswitha'                          9
  'AUT'                 'Rowing'             'Jonke
Arnold'                           11
  'AUT'                 'Rowing'             'Zerbst
Christoph'                        11
  'AUT'                 'Sailing'            'Hagara
Roman'                            15
  'AUT'                 'Sailing'            'Steinacher Hans
Peter'                           15
  'AUT'                 'Sailing'            'Sieber
Christoph'                        15
  'AUT'                 'Sailing'            'Geritzer
Andreas'                         15
  'AUT'                 'Shooting'           'Waibel Wolfram
Jr.'                             17
  'AUT'                 'Shooting'           'Planer
Christian'                        17
  'AUT'                 'Swimming'           'Rogan
Markus'                           18
```

CUME_DIST

CUME_DIST(*expression[, expression] ...) WITHIN GROUP (<order_by_clause>*)

CUME_DIST() *OVER ([<partition_by_clause>] <order_by_clause>*)

> **CUME_DIST** function is used as an aggregate function or an analytic function. It returns the value of cumulated distribution about the specified value within the group. The range of a return value by CUME_DIST is 0> and 1<=. The return value of **CUME_DIST** about the same input argument is evaluated as the same cumulated distribution value.

param expression:

an expression which returns the number or string. This should not be a column.

param order_by_clause:

column names followed by ORDER BY clause should be matched to the number of expressions

rtype:

DOUBLE

See also

`PERCENT_RANK()` , CUME_DIST vs. PERCENT_RANK

If it is used as an aggregate function, **CUME_DIST** sorts the data by the order specified in **ORDER BY** clause; then it returns the relative position of a hypothetical row in the rows of aggregate group. At this time, the position is calculated as if a hypothetical row is newly inserted. That is, **CUME_DIST** returns ("cumulated RANK of a hypothetical row" + 1)/("the number of total rows in an aggregate group").

If it is used as an analytic function, **CUME_DIST** returns the relative position of a value in the group after sorting each row (**ORDER BY**) within each partitioned group (**PARTITION BY**). The relative position is calculated by dividing the number of rows with values less than or equal to the input argument by the total number of rows in the group (rows grouped by the partition_by_clause or all rows if no partition is specified). That is, it returns (cumulative rank of a certain row)/(number of rows within the group). For example, if the number of rows with RANK 1 is 2, the **CUME_DIST** values of the first and second rows will be "2/10 = 0.2".

The following is a schema and data to use in the example of this function.

```
CREATE TABLE scores(id INT PRIMARY KEY AUTO_INCREMENT, math INT,
english INT, pe CHAR, grade INT);

INSERT INTO scores(math, english, pe, grade)
       VALUES(60, 70, 'A', 1),
       (60, 70, 'A', 1),
       (60, 80, 'A', 1),
       (60, 70, 'B', 1),
       (70, 60, 'A', 1) ,
```

```
       (70, 70, 'A', 1) ,
       (80, 70, 'C', 1) ,
       (70, 80, 'C', 1),
       (85, 60, 'C', 1),
       (75, 90, 'B', 1);
INSERT INTO scores(math, english, pe, grade)
       VALUES(95, 90, 'A', 2),
       (85, 95, 'B', 2),
       (95, 90, 'A', 2),
       (85, 95, 'B', 2),
       (75, 80, 'D', 2),
       (75, 85, 'D', 2),
       (75, 70, 'C', 2),
       (65, 95, 'A', 2),
       (65, 95, 'A', 2),
       (65, 95, 'A', 2);
```

The following is an example to be used as an aggregate function; it returns the result that the sum of each cumulated distribution about each column - *math*, *english* and *pe* - is divided by 3.

```
SELECT CUME_DIST(60, 70, 'D')
WITHIN GROUP(ORDER BY math, english, pe) AS cume
FROM scores;
```

```
1.904761904761905e-01
```

The following is an example to be used as an analytic function; it returns the cumulated distributions of each row about the 3 columns - *math*, *english* and *pe*.

```
SELECT id, math, english, pe, grade, CUME_DIST() OVER(ORDER BY math,
english, pe) AS cume_dist
FROM scores
ORDER BY cume_dist;
```

```
          id         math      english  pe
grade                cume_dist
================================================================================
================================
           1           60           70
'A'                             1     1.000000000000000e-01
           2           60           70
'A'                             1     1.000000000000000e-01
           4           60           70
'B'                             1     1.500000000000000e-01
           3           60           80
'A'                             1     2.000000000000000e-01
```

```
          18           65           95
'A'                          2     3.500000000000000e-01
          19           65           95
'A'                          2     3.500000000000000e-01
          20           65           95
'A'                          2     3.500000000000000e-01
           5           70           60
'A'                          1     4.000000000000000e-01
           6           70           70
'A'                          1     4.500000000000000e-01
           8           70           80
'C'                          1     5.000000000000000e-01
          17           75           70
'C'                          2     5.500000000000000e-01
          15           75           80
'D'                          2     6.000000000000000e-01
          16           75           85
'D'                          2     6.500000000000000e-01
          10           75           90
'B'                          1     7.000000000000000e-01
           7           80           70
'C'                          1     7.500000000000000e-01
           9           85           60
'C'                          1     8.000000000000000e-01
          12           85           95
'B'                          2     9.000000000000000e-01
          14           85           95
'B'                          2     9.000000000000000e-01
          11           95           90
'A'                          2     1.000000000000000e+00
          13           95           90
'A'                          2     1.000000000000000e+00
```

The following is an example to be used as an analytic function; it returns the cumulated distributions of each row about the 3 columns - *math*, *english* and *pe* - by grouping as *grade* column.

```
SELECT id, math, english, pe, grade, CUME_DIST() OVER(PARTITION BY
grade ORDER BY math, english, pe) AS cume_dist
FROM scores
ORDER BY grade, cume_dist;
```

```
   id         math      english  pe
grade                 cume_dist
============================================================================
========================
    1           60           70  'A'
1     2.000000000000000e-01
    2           60           70  'A'
```

```
1     2.000000000000000e-01
    4           60           70  'B'
1     3.000000000000000e-01
    3           60           80  'A'
1     4.000000000000000e-01
    5           70           60  'A'
1     5.000000000000000e-01
    6           70           70  'A'
1     6.000000000000000e-01
    8           70           80  'C'
1     7.000000000000000e-01
   10           75           90  'B'
1     8.000000000000000e-01
    7           80           70  'C'
1     9.000000000000000e-01
    9           85           60  'C'
1     1.000000000000000e+00
   18           65           95  'A'
2     3.000000000000000e-01
   19           65           95  'A'
2     3.000000000000000e-01
   20           65           95  'A'
2     3.000000000000000e-01
   17           75           70  'C'
2     4.000000000000000e-01
   15           75           80  'D'
2     5.000000000000000e-01
   16           75           85  'D'
2     6.000000000000000e-01
   12           85           95  'B'
2     8.000000000000000e-01
   14           85           95  'B'
2     8.000000000000000e-01
   11           95           90  'A'
2     1.000000000000000e+00
   13           95           90  'A'
2     1.000000000000000e+00
```

In the above result, the row that *id* is 1, is located at the first and the second on the total 10 rows, and the value of CUME_DIST is 2/10, that is, 0.2.

The row that *id* is 5, is located at the fifth on the total 10 rows, and the value of **CUME_DIST** is 5/10, that is, 0.5.

DENSE_RANK

DENSE_RANK() *OVER ([<partition_by_clause>] <order_by_clause>*)

DENSE_RANK function is used as an analytic function only. The rank of the value in the column value group made by the **PARTITION BY** clause is calculated and output as **INTEGER**.

Even when there is the same rank, 1 is added to the next rank value. For example, when there are three rows of Rank 13, the next rank is 14, not 16. On the contrary, the `RANK()` function calculates the next rank by adding the number of same ranks.

Return type:

INT

The following example shows output of the number of Olympic gold medals of each country and the rank of the countries by year: The number of the same rank is ignored and the next rank is calculated by adding 1 to the rank.

```
SELECT host_year, nation_code, gold,
DENSE_RANK() OVER (PARTITION BY host_year ORDER BY gold DESC) AS
d_rank
FROM participant;
```

```
host_year  nation_code                gold       d_rank
============================================================
     1988  'URS'                        55            1
     1988  'GDR'                        37            2
     1988  'USA'                        36            3
     1988  'KOR'                        12            4
     1988  'HUN'                        11            5
     1988  'FRG'                        11            5
     1988  'BUL'                        10            6
     1988  'ROU'                         7            7
     1988  'ITA'                         6            8
     1988  'FRA'                         6            8
     1988  'KEN'                         5            9
     1988  'GBR'                         5            9
     1988  'CHN'                         5            9
...
     1988  'CHI'                         0           14
     1988  'ARG'                         0           14
     1988  'JAM'                         0           14
     1988  'SUI'                         0           14
     1988  'SWE'                         0           14
     1992  'EUN'                        45            1
     1992  'USA'                        37            2
     1992  'GER'                        33            3
...
     2000  'RSA'                         0           15
     2000  'NGR'                         0           15
     2000  'JAM'                         0           15
     2000  'BRA'                         0           15
     2004  'USA'                        36            1
```

```
      2004  'CHN'                       32            2
      2004  'RUS'                       27            3
      2004  'AUS'                       17            4
      2004  'JPN'                       16            5
      2004  'GER'                       13            6
      2004  'FRA'                       11            7
      2004  'ITA'                       10            8
      2004  'UKR'                        9            9
      2004  'CUB'                        9            9
      2004  'GBR'                        9            9
      2004  'KOR'                        9            9
...
      2004  'EST'                        0           17
      2004  'SLO'                        0           17
      2004  'SCG'                        0           17
      2004  'FIN'                        0           17
      2004  'POR'                        0           17
      2004  'MEX'                        0           17
      2004  'LAT'                        0           17
      2004  'PRK'                        0           17
```

FIRST_VALUE

FIRST_VALUE(*expression) [{RESPECT|IGNORE} NULLS] OVER (<analytic_clause>*)

FIRST_VALUE function is used as an analytic function only. It returns **NULL** if the first value in the set is null. But, if you specify **IGNORE NULLS**, the first value will be returned as excluding null or **NULL** will be returned if all values are null.

Parameters:

expression – a column or an expression which returns a number or a string. **FIRST_VALUE** function or other analytic function cannot be included.

Return type:

a type of an expression

See also

`LAST_VALUE()`, `NTH_VALUE()`

The following is schema and data to run the example.

```
CREATE TABLE test_tbl(groupid int,itemno int);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,1);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,2);
INSERT INTO test_tbl VALUES(1,3);
INSERT INTO test_tbl VALUES(1,4);
INSERT INTO test_tbl VALUES(1,5);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,6);
INSERT INTO test_tbl VALUES(2,7);
```

The following is a query and a result to run **FIRST_VALUE** function.

```
SELECT groupid, itemno, FIRST_VALUE(itemno) OVER(PARTITION BY
groupid ORDER BY itemno) AS ret_val
FROM test_tbl;
```

```
      groupid       itemno      ret_val
=======================================
            1         NULL         NULL
            1         NULL         NULL
            1         NULL         NULL
            1            1         NULL
            1            2         NULL
            1            3         NULL
            1            4         NULL
            1            5         NULL
            2         NULL         NULL
            2         NULL         NULL
            2         NULL         NULL
            2            6         NULL
            2            7         NULL
```

Note

CUBRID sorts **NULL** value as first order than other values. The below SQL1 is interpreted as SQL2 which includes **NULLS FIRST** in ORDER BY clause.

```
SQL1: FIRST_VALUE(itemno) OVER(PARTITION BY groupid ORDER BY
itemno) AS ret_val
```

```
SQL2: FIRST_VALUE(itemno) OVER(PARTITION BY groupid ORDER BY
itemno NULLS FIRST) AS ret_val
```

The following is an example to specify **IGNORE NULLS**.

```
SELECT groupid, itemno, FIRST_VALUE(itemno) IGNORE NULLS
OVER(PARTITION BY groupid ORDER BY itemno) AS ret_val
FROM test_tbl;
```

```
      groupid       itemno      ret_val
=======================================
            1         NULL         NULL
            1         NULL         NULL
            1         NULL         NULL
            1            1            1
            1            2            1
            1            3            1
            1            4            1
            1            5            1
            2         NULL         NULL
            2         NULL         NULL
            2         NULL         NULL
            2            6            6
            2            7            6
```

GROUP_CONCAT

GROUP_CONCAT([*DISTINCT] expression [ORDER BY {column | unsigned_int} [ASC | DESC]] [SEPARATOR str_val*])

The **GROUP_CONCAT** function is used as an aggregate function only. It connects the values that are not **NULL** in the group and returns the character string in the **VARCHAR** type. If there are no rows of query result or there are only **NULL** values, **NULL** will be returned.

Parameters:

- **expression** – Column or expression returning numerical values or character strings
- **str_val** – Character string to use as a separator
- **DISTINCT** – Removes duplicate values from the result.
- **ORDERBY** – Specifies the order of result values.

- **SEPARATOR** – Specifies the separator to divide the result values. If it is omitted, the default character, comma (,) will be used as a separator.

Return type:

STRING

The maximum size of the return value follows the configuration of the system parameter, **group_concat_max_len**. The default is **1024** bytes, the minimum value is 4 bytes and the maximum value is IN_MAX(about 2G) bytes.

This function is affected by **string_max_size_bytes** parameter; if the value of **group_concat_max_len** is larger than the value **string_max_size_bytes** and the result size of **GROUP_CONCAT** exceeds the value of **string_max_size_bytes**, an error occurs.

To remove the duplicate values, use the **DISTINCT** clause. The default separator for the group result values is comma (,). To represent the separator explicitly, add the character string to use as a separator in the **SEPARATOR** clause and after that. If you want to remove separators, enter empty strings after the **SEPARATOR** clause.

If the non-character string type is passed to the result character string, an error will be returned.

To use the **GROUP_CONCAT** function, you must meet the following conditions.

- Only one expression (or a column) is allowed for an input parameter.
- Sorting with **ORDER BY** is available only in the expression used as a parameter.
- The character string used as a separator allows not only character string type but also allows other types.

```
SELECT GROUP_CONCAT(s_name) FROM code;
```

```
  group_concat(s_name)
======================
  'X,W,M,B,S,G'
```

```
SELECT GROUP_CONCAT(s_name ORDER BY s_name SEPARATOR ':') FROM code;
```

```
  group_concat(s_name order by s_name separator ':')
======================
  'B:G:M:S:W:X'
```

```
CREATE TABLE t(i int);
INSERT INTO t VALUES (4),(2),(3),(6),(1),(5);

SELECT GROUP_CONCAT(i*2+1 ORDER BY 1 SEPARATOR '') FROM t;
```

```
  group_concat(i*2+1 order by 1 separator '')
======================
  '35791113'
```

Note

GROUP_CONCAT performance for processing large numbers of rows has been improved starting from CUBRID 11.3 through optimization of internal string operations.

LAG

LAG(*expression[, offset[, default]]*) *OVER ([<partition_by_clause>] <order_by_clause>*)

LAG is an analytic function that returns the *expression* value from a previous row, before *offset* that comes before the current row. It can be used to access several rows simultaneously without making any self join.

Parameters:

- **expression** – a column or an expression that returns a number or a string
- **offset** – an integer which indicates the offset position. If not specified, the default is 1
- **default** – a value to return when an *expression* value before *offset* is NULL. If a default value is not specified, NULL is returned

Return type:

NUMBER or STRING

The following example shows how to sort employee numbers and output the previous employee number on the same row:

```
CREATE TABLE t_emp (name VARCHAR(10), empno INT);
INSERT INTO t_emp VALUES
    ('Amie', 11011),
    ('Jane', 13077),
    ('Lora', 12045),
    ('James', 12006),
    ('Peter', 14006),
    ('Tom', 12786),
    ('Ralph', 23518),
    ('David', 55);

SELECT name, empno, LAG (empno, 1) OVER (ORDER BY empno) prev_empno
FROM t_emp;
```

```
  name                          empno    prev_empno
================================================
  'David'                          55          NULL
  'Amie'                        11011            55
  'James'                       12006         11011
  'Lora'                        12045         12006
  'Tom'                         12786         12045
  'Jane'                        13077         12786
  'Peter'                       14006         13077
  'Ralph'                       23518         14006
```

On the contrary, `LEAD()` function returns the expression value from a subsequent row, after *offset* that follows the current row.

LAST_VALUE

LAST_VALUE(*expression) [{RESPECT|IGNORE} NULLS] OVER (<analytic_clause>*)

LAST_VALUE function is used as an analytic function only. It returns **NULL** if the last value in the set is null. But, if you specify **IGNORE NULLS**, the last value will be returned as excluding null or **NULL** will be returned if all values are null.

Parameters:

expression – a column or an expression which returns a number or a string. **LAST_VALUE** function or other analytic function cannot be included.

Return type:

a type of an *expression*

See also

`FIRST_VALUE()`, `NTH_VALUE()`

The following is schema and data to run the example.

```
CREATE TABLE test_tbl(groupid int,itemno int);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,1);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,2);
INSERT INTO test_tbl VALUES(1,3);
INSERT INTO test_tbl VALUES(1,4);
INSERT INTO test_tbl VALUES(1,5);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,6);
INSERT INTO test_tbl VALUES(2,7);
```

The following is a query and a result to run **LAST_VALUE** function.

```
SELECT groupid, itemno, LAST_VALUE(itemno) OVER(PARTITION BY groupid
ORDER BY itemno) AS ret_val
FROM test_tbl;
```

```
      groupid       itemno      ret_val
=======================================
            1         NULL         NULL
            1         NULL         NULL
            1         NULL         NULL
            1            1            1
            1            2            2
            1            3            3
```

```
1           4           4
1           5           5
2        NULL        NULL
2        NULL        NULL
2        NULL        NULL
2           6           6
2           7           7
```

LAST_VALUE function is calculated by the current row. That is, values which are not binded are not included on the calculation. For example, on the above result, the value of **LAST_VALUE** is 1 when "(groupid, itemno) = (1, 1)"; 2 when "(groupid, itemno) = (1, 2)

Note

CUBRID sorts **NULL** value as first order than other values. The below SQL1 is interpreted as SQL2 which includes **NULLS FIRST** in **ORDER BY** clause.

```
SQL1: LAST_VALUE(itemno) OVER(PARTITION BY groupid ORDER BY
itemno) AS ret_val
SQL2: LAST_VALUE(itemno) OVER(PARTITION BY groupid ORDER BY itemno
NULLS FIRST) AS ret_val
```

LEAD

LEAD(*expression, offset[, default]) OVER ([<partition_by_clause>] <order_by_clause>*)

LEAD is an analytic function that returns the *expression* value from a subsequent row, after *offset* that follows the current row. It can be used to access several rows simultaneously without making any self join.

Parameters:

- **expression** – a column or an expression which returns a number or a string.
- **offset** – the number which indicates the offset location. If it's omitted, the default is 1.
- **default** – the output value when the *expression* value located before *offset* is **NULL**. The default is **NULL**.

Return type:

NUMBER or STRING

The following example shows how to sort employee numbers and output the next employee number on the same row:

```
CREATE TABLE t_emp (name VARCHAR(10), empno INT);
INSERT INTO t_emp VALUES
('Amie', 11011), ('Jane', 13077), ('Lora', 12045), ('James', 12006),
('Peter', 14006), ('Tom', 12786), ('Ralph', 23518), ('David', 55);

SELECT name, empno, LEAD (empno, 1) OVER (ORDER BY empno) next_empno
FROM t_emp;
```

```
  name                          empno   next_empno
================================================
  'David'                          55        11011
  'Amie'                        11011        12006
  'James'                       12006        12045
  'Lora'                        12045        12786
  'Tom'                         12786        13077
  'Jane'                        13077        14006
  'Peter'                       14006        23518
  'Ralph'                       23518         NULL
```

The following example shows how to output the title of the previous row and the title of the next row along with the title of the current row on the tbl_board table:

```
CREATE TABLE tbl_board (num INT, title VARCHAR(50));
INSERT INTO tbl_board VALUES
(1, 'title 1'), (2, 'title 2'), (3, 'title 3'), (4, 'title 4'), (5,
'title 5'), (6, 'title 6'), (7, 'title 7');

SELECT num, title,
    LEAD (title,1,'no next page') OVER (ORDER BY num) next_title,
    LAG (title,1,'no previous page') OVER (ORDER BY num) prev_title
FROM tbl_board;
```

```
  num  title                 next_title            prev_title
=================================================================
==========
    1  'title 1'             'title 2'             NULL
    2  'title 2'             'title 3'             'title 1'
    3  'title 3'             'title 4'             'title 2'
    4  'title 4'             'title 5'             'title 3'
    5  'title 5'             'title 6'             'title 4'
```

```
    6  'title 6'             'title 7'             'title 5'
    7  'title 7'             NULL                  'title 6'
```

The following example shows how to output the title of the previous row and the title of the next row along with the title of a specified row on the tbl_board table. If a WHERE condition is enclosed in parentheses, the values of next_title and prev_title are **NULL** as only one row is selected but the previous row and the subsequent row.

```
SELECT * FROM
(
    SELECT num, title,
        LEAD(title,1,'no next page') OVER (ORDER BY num) next_title,
        LAG(title,1,'no previous page') OVER (ORDER BY num)
prev_title
    FROM tbl_board
)
WHERE num=5;
```

```
  num  title                 next_title            prev_title
=================================================================
==========
    5  'title 5'             'title 6'             'title 4'
```

MAX

MAX(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

MAX(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The **MAX** function is used as an aggregate function or an analytic function. It gets the greatest value of expressions of all rows. Only one *expression* is specified. For expressions that return character strings, the string that appears later in alphabetical order becomes the maximum value; for those that return numbers, the greatest value becomes the maximum value.

param expression:

Specifies an expression that returns a numeric or string value. An expression that returns a collection-type data is not allowed.

param ALL:

Gets the maximum value for all data (default).

param
DISTINCT,DISTINCTROW,UNIQUE:

Gets the maximum value without duplicates.

rtype:

same type as that the expression

The following example shows how to retrieve the maximum number of gold (*gold*) medals that Korea won in the Olympics in the *demodb* database.

```
SELECT MAX(gold) FROM participant WHERE nation_code = 'KOR';
```

```
    max(gold)
==============
           12
```

The following example shows how to output the number of gold medals by year and the maximum number of gold medals in history, acquired by the country whose nation_code code starts with 'AU'.

```
SELECT host_year, nation_code, gold,
    MAX(gold) OVER (PARTITION BY nation_code) mx_gold
FROM participant
WHERE nation_code LIKE 'AU%'
ORDER BY nation_code, host_year;
```

```
    host_year  nation_code                gold      mx_gold
=============================================================
         1988  'AUS'                         3           17
         1992  'AUS'                         7           17
         1996  'AUS'                         9           17
         2000  'AUS'                        16           17
         2004  'AUS'                        17           17
         1988  'AUT'                         1            2
         1992  'AUT'                         0            2
         1996  'AUT'                         0            2
         2000  'AUT'                         2            2
         2004  'AUT'                         2            2
```

MEDIAN

MEDIAN(*expression*)

MEDIAN(*expression) OVER ([<partition_by_clause>]*)

> **MEDIAN** function is used as an aggregate function or an analytic function. It returns the median value. The median value is the value which is located on the middle between the minimum value and the maximum value.
>
> param expression:
>
> column with value or expression which can be converted as number or date
>
> rtype:
>
> **DOUBLE** or **DATETIME**

The following is a schema and data to run examples.

```
CREATE TABLE tbl (col1 int, col2 double);
INSERT INTO tbl VALUES(1,2), (1,1.5), (1,1.7), (1,1.8), (2,3), (2,
4), (3,5);
```

The following is an example to be used as an aggregate function. It returns the median values of aggregated col2 by each group of col1.

```
SELECT col1, MEDIAN(col2)
FROM tbl GROUP BY col1;
```

```
         col1  median(col2)
====================================
            1  1.750000000000000e+00
            2  3.500000000000000e+00
            3  5.000000000000000e+00
```

The following is an example to be used as an analytic function. It returns the median values of col2 by each group of col1.

```
SELECT col1, MEDIAN(col2) OVER (PARTITION BY col1)
FROM tbl;
```

```
     col1  median(col2) over (partition by col1)
==================================================
        1  1.750000000000000e+00
        1  1.750000000000000e+00
        1  1.750000000000000e+00
        1  1.750000000000000e+00
        2  3.500000000000000e+00
        2  3.500000000000000e+00
        3  5.000000000000000e+00
```

MIN

MIN(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

MIN(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The **MIN** function is used as an aggregate function or an analytic function. It gets the smallest value of expressions of all rows. Only one *expression* is specified. For expressions that return character strings, the string that appears earlier in alphabetical order becomes the minimum value; for those that return numbers, the smallest value becomes the minimum value.

param expression:

Specifies an expression that returns a numeric or string value. A collection expression cannot be specified.

param ALL:

Gets the minimum value for all data (default).

param
DISTINCT,DISTINCTROW,UNIQUE:

Gets the maximum value without duplicates.

rtype:

same type as the *expression*

The following example shows how to retrieve the minimum number of gold (*gold*) medals that Korea won in the Olympics in the *demodb* database.

```
SELECT MIN(gold) FROM participant WHERE nation_code = 'KOR';
```

```
    min(gold)
=============
            7
```

The following example shows how to output the number of gold medals by year and the maximum number of gold medals in history, acquired by the country whose nation_code code starts with 'AU'.

```
SELECT host_year, nation_code, gold,
MIN(gold) OVER (PARTITION BY nation_code) mn_gold
FROM participant WHERE nation_code like 'AU%' ORDER BY nation_code,
host_year;
```

```
    host_year  nation_code                  gold      mn_gold
=============================================================
         1988  'AUS'                           3            3
         1992  'AUS'                           7            3
         1996  'AUS'                           9            3
         2000  'AUS'                          16            3
         2004  'AUS'                          17            3
         1988  'AUT'                           1            0
         1992  'AUT'                           0            0
         1996  'AUT'                           0            0
         2000  'AUT'                           2            0
         2004  'AUT'                           2            0
```

NTH_VALUE

NTH_VALUE(*expression*, *N) [{RESPECT|IGNORE} NULLS] OVER (<analytic_clause>*)

NTH_VALUE is used as an analytic function only. It returns an *expression* value of *N*-th row in the set of sorted values.

Parameters:

- **expression** – a column or an expression which returns a number or a string
- **N** – a constant, a binding variable, a column or an expression which can be interpreted as a positive integer

Return type:

a type of an *expression*

See also

`FIRST_VALUE()`, `LAST_VALUE()`

{RESPECT|IGNORE} NULLS syntax decides if null value of *expression* is included in the calculation or not. The default is **RESPECT NULLS**.

The following is a schema and data to run examples.

```
CREATE TABLE test_tbl(groupid int,itemno int);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,1);
INSERT INTO test_tbl VALUES(1,null);
INSERT INTO test_tbl VALUES(1,2);
INSERT INTO test_tbl VALUES(1,3);
INSERT INTO test_tbl VALUES(1,4);
INSERT INTO test_tbl VALUES(1,5);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,null);
INSERT INTO test_tbl VALUES(2,6);
INSERT INTO test_tbl VALUES(2,7);
```

The following is a query and results to run **NTH_VALUE** function by the value of *N* as 2.

```
SELECT groupid, itemno, NTH_VALUE(itemno, 2) IGNORE NULLS
OVER(PARTITION BY groupid ORDER BY itemno NULLS FIRST) AS ret_val
FROM test_tbl;
```

```
      groupid       itemno      ret_val
=======================================
            1         NULL         NULL
            1         NULL         NULL
            1         NULL         NULL
            1            1         NULL
            1            2            2
            1            3            2
            1            4            2
            1            5            2
```

```
          2         NULL         NULL
          2         NULL         NULL
          2         NULL         NULL
          2            6         NULL
          2            7            7
```

Note

CUBRID sorts **NULL** value as first order than other values. The below SQL1 is interpreted as SQL2 which includes **NULLS FIRST** in **ORDER BY** clause.

```
SQL1: NTH_VALUE(itemno) OVER(PARTITION BY groupid ORDER BY itemno)
AS ret_val
SQL2: NTH_VALUE(itemno) OVER(PARTITION BY groupid ORDER BY itemno
NULLS FIRST) AS ret_val
```

NTILE

NTILE(*expression) OVER ([<partition_by_clause>] <order_by_clause>*)

NTILE is an analytic function. It divides an ordered data set into a number of buckets indicated by the input parameter value and assigns the appropriate bucket number from 1 to each row.

Parameters:

expression – the number of buckets. It specifies a certain expression which returns a number value.

Return type:

INT

NTILE function equally divides the number of rows by the given number of buckets and assigns the bucket number to each bucket. That is, **NTILE** function creates an equi-height histogram. The number of rows in the buckets can differ by at most 1. The remainder values (the remainder number of rows divided by buckets number) are distributed one for each bucket, starting with #1 Bucket.

On the contrary, `WIDTH_BUCKET()` function equally divides the range by the given number of buckets and assigns the bucket number to each bucket. That is, every interval (bucket) has the identical size.

The following example divides rows into five buckets of eight customers based on their dates of birth. Because the total number of rows is not divisible by the number of buckets, the first three buckets have two rows and the remaining groups have one row each.

```
CREATE TABLE t_customer(name VARCHAR(10), birthdate DATE);
INSERT INTO t_customer VALUES
    ('Amie', date'1978-03-18'),
    ('Jane', date'1983-05-12'),
    ('Lora', date'1987-03-26'),
    ('James', date'1948-12-28'),
    ('Peter', date'1988-10-25'),
    ('Tom', date'1980-07-28'),
    ('Ralph', date'1995-03-17'),
    ('David', date'1986-07-28');

SELECT name, birthdate, NTILE(5) OVER (ORDER BY birthdate) age_group
FROM t_customer;
```

```
  name                  birthdate     age_group
================================================
  'James'               12/28/1948            1
  'Amie'                03/18/1978            1
  'Tom'                 07/28/1980            2
  'Jane'                05/12/1983            2
  'David'               07/28/1986            3
  'Lora'                03/26/1987            3
  'Peter'               10/25/1988            4
  'Ralph'               03/17/1995            5
```

The following example divides eight students into five buckets that have the identical number of rows in the order of score and outputs in the order of the name. As the score column of the t_score table has eight rows, the remaining three rows are assigned to buckets from #1 Bucket. The first three buckets have one more row than the remaining groups. The NTILE function equally divides the grade based on the number of rows, regardless the range of the score.

```
CREATE TABLE t_score(name VARCHAR(10), score INT);
INSERT INTO t_score VALUES
    ('Amie', 60),
    ('Jane', 80),
    ('Lora', 60),
    ('James', 75),
    ('Peter', 70),
    ('Tom', 30),
    ('Ralph', 99),
    ('David', 55);

SELECT name, score, NTILE(5) OVER (ORDER BY score DESC) grade
```

```
FROM t_score
ORDER BY name;
```

```
  name                          score        grade
================================================
  'Ralph'                          99            1
  'Jane'                           80            1
  'James'                          75            2
  'Peter'                          70            2
  'Amie'                           60            3
  'Lora'                           60            3
  'David'                          55            4
  'Tom'                            30            5
```

PERCENT_RANK

PERCENT_RANK(*expression[, expression] ...) WITHIN GROUP (<order_by_clause>*)

PERCENT_RANK(*) OVER ([<partition_by_clause>] <order_by_clause>*)

PERCENT_RANK function is used as an aggregate function or an analytic function. It returns the relative position of the row in the group as a ranking percent. It is similar to **CUME_DIST** function(returns cumulated distribution value). The range of this function is from 0 to 1. The first value of **PERCENT_RANK** is always 0.

param expression:

an expression which returns a number or a string. It should not be a column.

rtype:

DOUBLE

See also

`CUME_DIST()`, `RANK()`

If it is an aggregate function, it returns the value that the RANK minus 1 of a hypothetical row selected in the whole aggregated rows is divided by the number of rows in the aggregated group. That is, (“RANK of a hypothetical row” - 1)/(the number or rows in the aggregated group).

If it is an analytic function, it returns ("RANK per group" -1)/("the number of rows in the group" -1) when each row with the group divided by **PARTITION BY** is sorted by the specified order by the **ORDER BY** clause. For example, if the number of rows appeared as the first rank(RANK=1) in the total 10 rows is 2, **PERCENT_RANK** values of the first and second rows are (1-1)/(10-1)=0.

The following is a table which compares the return values of **CUME_DIST** and **PERCENT_RANK** which are used as aggregate functions when there are input arguments *VAL*.

VAL	RANK()	DENSE_RANK()	CUME_DIST(VAL)	PERCENT_RANK (VAL)
100	1	1	0.33 => (1+1)/(5+1)	0 => (1-1)/5
200	2	2	0.67 => (2+1)/(5+1)	0.2 => (2-1)/5
200	2	2	0.67 => (2+1)/(5+1)	0.2 => (2-1)/5
300	4	3	0.83 => (4+1)/(5+1)	0.6 => (4-1)/5
400	5	4	1 => (5+1)/(5+1)	0.8 => (5-1)/5

The following is a table which compares the return values of **CUME_DIST** and **PERCENT_RANK** which are used as analytic functions when there are input arguments *VAL*.

VAL	RANK()	DENSE_RANK()	CUME_DIST()	PERCENT_RANK ()
100	1	1	0.2 => 1/5	0 => (1-1)/(5-1)
200	2	2	0.6 => 3/5	0.25 => (2-1)/(5-1)

VAL	RANK()	DENSE_RANK()	CUME_DIST()	PERCENT_RANK()
200	2	2	0.6 => 3/5	0.25 => (2-1)/(5-1)
300	4	3	0.8 => 4/5	0.75 => (4-1)/(5-1)
400	5	4	1 => 5/5	1 => (5-1)/(5-1)

The following is a schema and examples of queries which are related to the above tables.

```
CREATE TABLE test_tbl(VAL INT);
INSERT INTO test_tbl VALUES (100), (200), (200), (300), (400);


SELECT CUME_DIST(100) WITHIN GROUP (ORDER BY val) AS cume FROM
test_tbl;
SELECT PERCENT_RANK(100) WITHIN GROUP (ORDER BY val) AS pct_rnk FROM
test_tbl;

SELECT CUME_DIST() OVER (ORDER BY val) AS cume FROM test_tbl;
SELECT PERCENT_RANK() OVER (ORDER BY val) AS pct_rnk FROM test_tbl;
```

The following is a schema and data which will be used in the below.

```
CREATE TABLE scores(id INT PRIMARY KEY AUTO_INCREMENT, math INT,
english INT, pe CHAR, grade INT);

INSERT INTO scores(math, english, pe, grade)
       VALUES(60, 70, 'A', 1),
       (60, 70, 'A', 1),
       (60, 80, 'A', 1),
       (60, 70, 'B', 1),
       (70, 60, 'A', 1) ,
       (70, 70, 'A', 1) ,
       (80, 70, 'C', 1) ,
       (70, 80, 'C', 1),
       (85, 60, 'C', 1),
       (75, 90, 'B', 1);
INSERT INTO scores(math, english, pe, grade)
       VALUES(95, 90, 'A', 2),
```

```
        (85, 95, 'B', 2),
        (95, 90, 'A', 2),
        (85, 95, 'B', 2),
        (75, 80, 'D', 2),
        (75, 85, 'D', 2),
        (75, 70, 'C', 2),
        (65, 95, 'A', 2),
        (65, 95, 'A', 2),
        (65, 95, 'A', 2);
```

The following is an example of aggregate function. It displays the result that each **PERCENT_RANK** about three columns, *math*, *english* and *pe* are added and divided by 3.

```
SELECT PERCENT_RANK(60, 70, 'D')
WITHIN GROUP(ORDER BY math, english, pe) AS percent_rank
FROM scores;
```

```
1.500000000000000e-01
```

The following is an example of analytic function. It returns the **PERCENT_RANK** values of the entire rows based on three columns, *math*, *english* and **pe**.

```
SELECT id, math, english, pe, grade, PERCENT_RANK() OVER(ORDER BY
math, english, pe) AS percent_rank
FROM scores
ORDER BY percent_rank;
```

```
          id         math      english  pe
grade              percent_rank
================================================================================
================================
           1           60           70
'A'                           1     0.000000000000000e+00
           2           60           70
'A'                           1     0.000000000000000e+00
           4           60           70
'B'                           1     1.052631578947368e-01
           3           60           80
'A'                           1     1.578947368421053e-01
          18           65           95
'A'                           2     2.105263157894737e-01
          19           65           95
'A'                           2     2.105263157894737e-01
          20           65           95
'A'                           2     2.105263157894737e-01
           5           70           60
```

```
'A'                             1     3.684210526315789e-01
           6            70           70
'A'                             1     4.210526315789473e-01
           8            70           80
'C'                             1     4.736842105263158e-01
          17            75           70
'C'                             2     5.263157894736842e-01
          15            75           80
'D'                             2     5.789473684210527e-01
          16            75           85
'D'                             2     6.315789473684210e-01
          10            75           90
'B'                             1     6.842105263157895e-01
           7            80           70
'C'                             1     7.368421052631579e-01
           9            85           60
'C'                             1     7.894736842105263e-01
          12            85           95
'B'                             2     8.421052631578947e-01
          14            85           95
'B'                             2     8.421052631578947e-01
          11            95           90
'A'                             2     9.473684210526315e-01
          13            95           90
'A'                             2     9.473684210526315e-01
```

The following is an example of analytic function. It returns the **PERCENT_RANK** values grouped by *grade* column, based on three columns, *math*, *english* and **pe**.

```
SELECT id, math, english, pe, grade, RANK(), PERCENT_RANK()
OVER(PARTITION BY grade ORDER BY math, english, pe) AS percent_rank
FROM scores
ORDER BY grade, percent_rank;
```

```
           id         math      english  pe
grade              percent_rank
================================================================================
================================
           1            60           70
'A'                             1     0.000000000000000e+00
           2            60           70
'A'                             1     0.000000000000000e+00
           4            60           70
'B'                             1     2.222222222222222e-01
           3            60           80
'A'                             1     3.333333333333333e-01
           5            70           60
'A'                             1     4.444444444444444e-01
           6            70           70
```

```
'A'                             1     5.555555555555556e-01
          8           70            80
'C'                             1     6.666666666666666e-01
         10           75            90
'B'                             1     7.777777777777778e-01
          7           80            70
'C'                             1     8.888888888888888e-01
          9           85            60
'C'                             1     1.000000000000000e+00
         18           65            95
'A'                             2     0.000000000000000e+00
         19           65            95
'A'                             2     0.000000000000000e+00
         20           65            95
'A'                             2     0.000000000000000e+00
         17           75            70
'C'                             2     3.333333333333333e-01
         15           75            80
'D'                             2     4.444444444444444e-01
         16           75            85
'D'                             2     5.555555555555556e-01
         12           85            95
'B'                             2     6.666666666666666e-01
         14           85            95
'B'                             2     6.666666666666666e-01
         11           95            90
'A'                             2     8.888888888888888e-01
         13           95            90
'A'                             2     8.888888888888888e-01
```

In the above result, the rows with *id* 1 are located at the first and the second in the 10 rows whose *grade* is 1, and the values of **PERCENT_RANK** will be (1-1)/(10-1)=0. A row whose *id* is 5 is located at the fifth in the 10 rows whose *grade* is 1, and the value of **PERCENT_RANK** will be (5-1)/(10-1)=0.44.

PERCENTILE_CONT

PERCENTILE_CONT(expression1) WITHIN GROUP (ORDER BY expression2 [ASC | DESC]) [OVER (<partition_by_clause>)]

PERCENTILE_CONT is used as an aggregate or analytic function, and is a reverse distribution function to assume a continuous distribution model. This takes a percentile value and returns a interpolated value within a set of sorted values. NULLs are ignored when calculating.

This function's input value is a number or a string which can be converted into a number, and the type of returned value is DOUBLE.

Parameters:

- **expression1** – Percentile value. This must be between 0 and 1.
- **expression2** – The column names followed by an **ORDER BY** clause. The number of columns should be the same with the number of columns in *expression1*.

Return type:

DOUBLE

See also

Difference between PERCENTILE_DISC and PERCENTILE_CONT

When this is an aggregate function, this sorts results by the order specified by the **ORDER BY** clause; then this returns an interpolation value belongs to the percentile value from the rows in the aggregate group.

When this is an analytic function, this sorts each row divided by **PARTITION BY** clause, by the order specified by the **ORDER BY** clause; then this returns an interpolation value belongs to the percentile value from the rows in the group.

Note

Difference between PERCENTILE_CONT and PERCENTILE_DISC

PERCENTILE_CONT and PERCENTILE_DISC can return different results.

PERCENTILE_CONT operates continuous interpolation; then it returns the calculated result.

PERCENTILE_DISC returns a value from the set of aggregated values.

In the below examples, when a percentile value is 0.5 and the group has even items, PERCENTILE_CONT returns the average of the two values from the medium position; however, PERCENTILE_DISC returns the first value between the two values from the medium position. If the group has odd items, both of them returns the value of a centered item.

In fact, the MEDIAN function is a particular case of PERCENTILE_CONT with the default of percentile value(0.5). Please also see `MEDIAN()` for more details.

The below shows the schema and the data on the next examples.

```
CREATE TABLE scores([id] INT PRIMARY KEY AUTO_INCREMENT, [math] INT,
english INT, [class] CHAR);

INSERT INTO scores VALUES
       (1, 30, 70, 'A'),
       (2, 40, 70, 'A'),
       (3, 60, 80, 'A'),
       (4, 70, 70, 'A'),
       (5, 72, 60, 'A') ,
       (6, 77, 70, 'A') ,
       (7, 80, 70, 'C') ,
       (8, 70, 80, 'C'),
       (9, 85, 60, 'C'),
       (10, 78, 90, 'B'),
       (11, 95, 90, 'D'),
       (12, 85, 95, 'B'),
       (13, 95, 90, 'B'),
       (14, 85, 95, 'B'),
       (15, 75, 80, 'D'),
       (16, 75, 85, 'D'),
       (17, 75, 70, 'C'),
       (18, 65, 95, 'C'),
       (19, 65, 95, 'D'),
       (20, 65, 95, 'D');
```

The below is an example of an aggregate function; it returns a median value for the *math* column.

```
SELECT PERCENTILE_CONT(0.5)
WITHIN GROUP(ORDER BY math) AS pcont
FROM scores;
```

```
  pcont
======================
  7.500000000000000e+01
```

The below is an example of an analytic function; it returns a median value for the *math* column within the set grouped by which the values of *class* column are the same.

```
SELECT math, [class], PERCENTILE_CONT(0.5)
WITHIN GROUP(ORDER BY math)
```

```
OVER (PARTITION BY [class]) AS pcont
FROM scores;
```

```
     math  class                 pcont
==========================================================
       30  'A'                   6.500000000000000e+01
       40  'A'                   6.500000000000000e+01
       60  'A'                   6.500000000000000e+01
       70  'A'                   6.500000000000000e+01
       72  'A'                   6.500000000000000e+01
       77  'A'                   6.500000000000000e+01
       78  'B'                   8.500000000000000e+01
       85  'B'                   8.500000000000000e+01
       85  'B'                   8.500000000000000e+01
       95  'B'                   8.500000000000000e+01
       65  'C'                   7.500000000000000e+01
       70  'C'                   7.500000000000000e+01
       75  'C'                   7.500000000000000e+01
       80  'C'                   7.500000000000000e+01
       85  'C'                   7.500000000000000e+01
       65  'D'                   7.500000000000000e+01
       65  'D'                   7.500000000000000e+01
       75  'D'                   7.500000000000000e+01
       75  'D'                   7.500000000000000e+01
       95  'D'                   7.500000000000000e+01
```

In class 'A', the number of 'math' is totally 6; PERCENTILE_CONT assumes that continuous values exist from the discrete values; therefore, the median value is 65, an average of the 3rd value (60) and the 4th value (70).

PERCENTILE_CONT assumes the continuous value; therefore, it returns DOUBLE type value which can show the continuous representation.

PERCENTILE_DISC

PERCENTILE_DISC(expression1) WITHIN GROUP (ORDER BY expression2 [ASC | DESC]) [OVER (<partition_by_clause>)]

PERCENTILE_DISC is used as an aggregate or analytic function, and is a reverse distribution function to assume a discrete distribution model. This takes a percentile value and returns a discrete value within a set of sorted values. NULLs are ignored when calculating.

This function's input value is a number or a string which can be converted into a number, and the type of returned value is the same as the type of input value.

Parameters:

- **expression1** – Percentile value. This must be between 0 and 1.
- **expression2** – The column names followed by an **ORDER BY** clause. The number of columns should be the same with the number of columns in *expression1*.

Return type:

the same with the *expression2*'s type.

See also

Difference between PERCENTILE_DISC and PERCENTILE_CONT

When this is an aggregate function, this sorts results by the order specified by the **ORDER BY** clause; then this returns an interpolation value located to the percentile value from the rows in the aggregate group.

When this is an analytic function, this sorts each row divided by **PARTITION BY** clause, by the order specified by the **ORDER BY** clause; then this returns an interpolation value located to the percentile value from the rows in the group.

The schema and the data used in this function's example are the same with them in `PERCENTILE_CONT()`.

The below is an example of an aggregate function; it returns a median value for the *math* column.

```
SELECT PERCENTILE_DISC(0.5)
WITHIN GROUP(ORDER BY math) AS pdisc
FROM scores;
```

```
  pdisc
=======================
  75
```

The below is an example of an analytic function; it returns a median value for the *math* column within the set grouped by which the values of *class* column are the same.

```
SELECT math, [class], PERCENTILE_DISC(0.5)
WITHIN GROUP(ORDER BY math)
OVER (PARTITION BY [class]) AS pdisc
FROM scores;
```

```
 math  class                 pdisc
============================================================
   30  'A'                   60
   40  'A'                   60
   60  'A'                   60
   70  'A'                   60
   72  'A'                   60
   77  'A'                   60
   78  'B'                   85
   85  'B'                   85
   85  'B'                   85
   95  'B'                   85
   65  'C'                   75
   70  'C'                   75
   75  'C'                   75
   80  'C'                   75
   85  'C'                   75
   65  'D'                   75
   65  'D'                   75
   75  'D'                   75
   75  'D'                   75
   95  'D'                   75
```

In class 'A', the number of 'math' is totally 6; PERCENTILE_DISC outputs the first one if the medium values are the two; therefore, the median value is 60, between the 3rd value (60) and the 4th value (70).

RANK

RANK() *OVER ([<partition_by_clause>] <order_by_clause>*)

RANK function is used as an analytic function only. The rank of the value in the column value group made by the **PARTITION BY** clause is calculated and output as **INTEGER**. When there is another identical rank, the next rank is the number adding the number of the same ranks. For example, when there are three rows of Rank 13, the next rank is 16, not 14. On the contrary, the `DENSE_RANK()` function calculates the next rank by adding 1 to the rank.

rtype:

INT

The following example shows output of the number of Olympic gold medals of each country and the rank of the countries by year. The next rank of the same rank is calculated by adding the number of the same ranks.

```
SELECT host_year, nation_code, gold,
RANK() OVER (PARTITION BY host_year ORDER BY gold DESC) AS g_rank
FROM participant;
```

```
    host_year  nation_code                  gold       g_rank
==============================================================
         1988  'URS'                          55            1
         1988  'GDR'                          37            2
         1988  'USA'                          36            3
         1988  'KOR'                          12            4
         1988  'HUN'                          11            5
         1988  'FRG'                          11            5
         1988  'BUL'                          10            7
         1988  'ROU'                           7            8
         1988  'ITA'                           6            9
         1988  'FRA'                           6            9
         1988  'KEN'                           5           11
         1988  'GBR'                           5           11
         1988  'CHN'                           5           11
...
         1988  'CHI'                           0           32
         1988  'ARG'                           0           32
         1988  'JAM'                           0           32
         1988  'SUI'                           0           32
         1988  'SWE'                           0           32
         1992  'EUN'                          45            1
         1992  'USA'                          37            2
         1992  'GER'                          33            3
...
         2000  'RSA'                           0           52
         2000  'NGR'                           0           52
         2000  'JAM'                           0           52
         2000  'BRA'                           0           52
         2004  'USA'                          36            1
         2004  'CHN'                          32            2
         2004  'RUS'                          27            3
         2004  'AUS'                          17            4
         2004  'JPN'                          16            5
         2004  'GER'                          13            6
         2004  'FRA'                          11            7
         2004  'ITA'                          10            8
         2004  'UKR'                           9            9
         2004  'CUB'                           9            9
         2004  'GBR'                           9            9
         2004  'KOR'                           9            9
```

```
...
         2004  'EST'                          0           57
         2004  'SLO'                          0           57
         2004  'SCG'                          0           57
         2004  'FIN'                          0           57
         2004  'POR'                          0           57
         2004  'MEX'                          0           57
         2004  'LAT'                          0           57
         2004  'PRK'                          0           57
```

ROW_NUMBER

ROW_NUMBER() *OVER ([<partition_by_clause>] <order_by_clause>*)

ROW_NUMBER function is used as an analytic function only. The rank of a row is one plus the number of distinct ranks that come before the row in question by using the **PARTITION BY** clause and outputs as **INTEGER**.

Return type:

INT

The following example shows output of the serial number according to the number of Olympic gold medals of each country by year. If the number of gold medals is the same, the sorting follows the alphabetic order of the nation_code.

```
SELECT host_year, nation_code, gold,
ROW_NUMBER() OVER (PARTITION BY host_year ORDER BY gold DESC) AS
r_num
FROM participant;
```

```
    host_year  nation_code                gold       r_num
==========================================================
         1988  'URS'                        55           1
         1988  'GDR'                        37           2
         1988  'USA'                        36           3
         1988  'KOR'                        12           4
         1988  'FRG'                        11           5
         1988  'HUN'                        11           6
         1988  'BUL'                        10           7
         1988  'ROU'                         7           8
         1988  'FRA'                         6           9
         1988  'ITA'                         6          10
         1988  'CHN'                         5          11
...
         1988  'YEM'                         0         152
```

```
          1988  'YMD'                 0           153
          1988  'ZAI'                 0           154
          1988  'ZAM'                 0           155
          1988  'ZIM'                 0           156
          1992  'EUN'                45             1
          1992  'USA'                37             2
          1992  'GER'                33             3
...
          2000  'VIN'                 0           194
          2000  'YEM'                 0           195
          2000  'ZAM'                 0           196
          2000  'ZIM'                 0           197
          2004  'USA'                36             1
          2004  'CHN'                32             2
          2004  'RUS'                27             3
          2004  'AUS'                17             4
          2004  'JPN'                16             5
          2004  'GER'                13             6
          2004  'FRA'                11             7
          2004  'ITA'                10             8
          2004  'CUB'                 9             9
          2004  'GBR'                 9            10
          2004  'KOR'                 9            11
...
          2004  'UGA'                 0           195
          2004  'URU'                 0           196
          2004  'VAN'                 0           197
          2004  'VEN'                 0           198
          2004  'VIE'                 0           199
          2004  'VIN'                 0           200
          2004  'YEM'                 0           201
          2004  'ZAM'                 0           202
```

STDDEV, STDDEV_POP

STDDEV(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

STDDEV_POP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

STDDEV(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

STDDEV_POP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The functions **STDDEV** and **STDDEV_POP** are used interchangeably and they are used as an aggregate function or an analytic function. They return a standard variance of the values calculated for all rows. The **STDDEV_POP** function is a standard of the SQL:1999. Only one *expression* is specified as a parameter. If the **DISTINCT** or **UNIQUE** keyword is inserted before the expression, they calculate

the sample standard variance after deleting duplicates; if keyword is omitted or **ALL**, they it calculate the sample standard variance for all values.

param expression:

Specifies an expression that returns a numeric value.

param ALL:

Calculates the standard variance for all data (default).

param DISTINCT,DISTINCTROW,UNIQUE:

Calculates the standard variance without duplicates.

rtype:

DOUBLE

The return value is the same with the square root of its variance (the return value of `VAR_POP()` and it is a **DOUBLE** type. If there are no rows that can be used for calculating a result, **NULL** is returned.

The following is a formula that is applied to the function.

$$\mathrm{STDDEV_{POP}} = \left[\left(\frac{1}{N} \right) * \mathrm{SUM}\left(\{ x_I - \mathrm{AVG}(x) \}^2 \right) \right]^{\frac{1}{2}}$$

Note

In CUBRID 2008 R3.1 or earlier, the **STDDEV** function worked the same as the `STDDEV_SAMP()` .

The following example shows how to output the population standard variance of all students for all subjects.

```
CREATE TABLE student (name VARCHAR(32), subjects_id INT, score
DOUBLE);
INSERT INTO student VALUES
('Jane',1, 78), ('Jane',2, 50), ('Jane',3, 60),
```

```
('Bruce', 1, 63), ('Bruce', 2, 50), ('Bruce', 3, 80),
('Lee', 1, 85), ('Lee', 2, 88), ('Lee', 3, 93),
('Wane', 1, 32), ('Wane', 2, 42), ('Wane', 3, 99),
('Sara', 1, 17), ('Sara', 2, 55), ('Sara', 3, 43);

SELECT STDDEV_POP (score) FROM student;
```

```
         stddev_pop(score)
===========================
     2.329711474744362e+01
```

The following example shows how to output the score and population standard variance of all students by subject (subjects_id).

```
SELECT subjects_id, name, score,
STDDEV_POP(score) OVER(PARTITION BY subjects_id) std_pop
FROM student
ORDER BY subjects_id, name;
```

```
  subjects_id  name
score                   std_pop
=============================================================================
==================
            1  'Bruce'                  6.300000000000000e+01
2.632869157402243e+01
            1  'Jane'                   7.800000000000000e+01
2.632869157402243e+01
            1  'Lee'                    8.500000000000000e+01
2.632869157402243e+01
            1  'Sara'                   1.700000000000000e+01
2.632869157402243e+01
            1  'Wane'                   3.200000000000000e+01
2.632869157402243e+01
            2  'Bruce'                  5.000000000000000e+01
1.604992211819110e+01
            2  'Jane'                   5.000000000000000e+01
1.604992211819110e+01
            2  'Lee'                    8.800000000000000e+01
1.604992211819110e+01
            2  'Sara'                   5.500000000000000e+01
1.604992211819110e+01
            2  'Wane'                   4.200000000000000e+01
1.604992211819110e+01
            3  'Bruce'                  8.000000000000000e+01
2.085185843036539e+01
            3  'Jane'                   6.000000000000000e+01
2.085185843036539e+01
```

```
            3  'Lee'                    9.300000000000000e+01
2.085185843036539e+01
            3  'Sara'                   4.300000000000000e+01
2.085185843036539e+01
            3  'Wane'                   9.900000000000000e+01
2.085185843036539e+01
```

STDDEV_SAMP

STDDEV_SAMP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

STDDEV_SAMP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The **STDDEV_SAMP** function is used as an aggregate function or an analytic function. It calculates the sample standard variance. Only one *expression* is specified as a parameter. If the **DISTINCT** or **UNIQUE** keyword is inserted before the expression, it calculates the sample standard variance after deleting duplicates; if a keyword is omitted or **ALL**, it calculates the sample standard variance for all values.

param expression:

An expression that returns a numeric value

param ALL:

Used to calculate the standard variance for all values. It is the default value.

param DISTINCT,DISTINCTROW,UNIQUE:

Used to calculate the standard variance for the unique values without duplicates.

rtype:

DOUBLE

The return value is the same as the square root of its sample variance (`VAR_SAMP()`) and it is a **DOUBLE** type. If there are no rows that can be used for calculating a result, **NULL** is returned.

The following are the formulas applied to the function.

$$\mathrm{STDDEV_{SAMP}} = \left[\left\{\frac{1}{(N-1)}\right\} * \mathrm{SUM}(\{x_I - \mathrm{mean}(x)\}^2)\right]^{\frac{1}{2}}$$

The following example shows how to output the sample standard variance of all students for all subjects.

```
CREATE TABLE student (name VARCHAR(32), subjects_id INT, score
DOUBLE);
INSERT INTO student VALUES
('Jane',1, 78), ('Jane',2, 50), ('Jane',3, 60),
('Bruce', 1, 63), ('Bruce', 2, 50), ('Bruce', 3, 80),
('Lee', 1, 85), ('Lee', 2, 88), ('Lee', 3, 93),
('Wane', 1, 32), ('Wane', 2, 42), ('Wane', 3, 99),
('Sara', 1, 17), ('Sara', 2, 55), ('Sara', 3, 43);

SELECT STDDEV_SAMP (score) FROM student;
```

```
        stddev_samp(score)
============================
     2.411480477888654e+01
```

The following example shows how to output the sample standard variance of all students for all subjects.

```
SELECT subjects_id, name, score,
STDDEV_SAMP(score) OVER(PARTITION BY subjects_id) std_samp
FROM student
ORDER BY subjects_id, name;
```

```
  subjects_id  name
score                  std_samp
================================================================================
==================
            1  'Bruce'                6.300000000000000e+01
2.943637205907005e+01
            1  'Jane'                 7.800000000000000e+01
2.943637205907005e+01
            1  'Lee'                  8.500000000000000e+01
2.943637205907005e+01
            1  'Sara'                 1.700000000000000e+01
2.943637205907005e+01
            1  'Wane'                 3.200000000000000e+01
2.943637205907005e+01
            2  'Bruce'                5.000000000000000e+01
1.794435844492636e+01
            2  'Jane'                 5.000000000000000e+01
```

```
1.794435844492636e+01
           2  'Lee'                    8.800000000000000e+01
1.794435844492636e+01
           2  'Sara'                   5.500000000000000e+01
1.794435844492636e+01
           2  'Wane'                   4.200000000000000e+01
1.794435844492636e+01
           3  'Bruce'                  8.000000000000000e+01
2.331308645374953e+01
           3  'Jane'                   6.000000000000000e+01
2.331308645374953e+01
           3  'Lee'                    9.300000000000000e+01
2.331308645374953e+01
           3  'Sara'                   4.300000000000000e+01
2.331308645374953e+01
           3  'Wane'                   9.900000000000000e+01
2.331308645374953e+01
```

SUM

SUM(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

SUM(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The **SUM** function is used as an aggregate function or an analytic function. It returns the sum of expressions of all rows. Only one *expression* is specified as a parameter. You can get the sum without duplicates by inserting the **DISTINCT** or **UNIQUE** keyword in front of the expression, or get the sum of all values by omitting the keyword or by using **ALL**.

param expression:

Specifies an expression that returns a numeric value.

param ALL:

Gets the sum for all data (default).

param DISTINCT,DISTINCTROW,UNIQUE:

Gets the sum of unique values without duplicates

rtype:

same type as that the expression

The following is an example that outputs the top 10 countries and the total number of gold medals based on the sum of gold medals won in the Olympic Games in *demodb*.

```
SELECT nation_code, SUM(gold)
FROM participant
GROUP BY nation_code
ORDER BY SUM(gold) DESC
LIMIT 10;
```

```
  nation_code             sum(gold)
=====================================
  'USA'                         190
  'CHN'                          97
  'RUS'                          85
  'GER'                          79
  'URS'                          55
  'FRA'                          53
  'AUS'                          52
  'ITA'                          48
  'KOR'                          48
  'EUN'                          45
```

The following example shows how to output the number of gold medals by year and the average sum of the accumulated gold medals to the year acquired by the country whose nation_code code starts with 'AU' in *demodb*.

```
SELECT host_year, nation_code, gold,
    SUM(gold) OVER (PARTITION BY nation_code ORDER BY host_year)
sum_gold
FROM participant
WHERE nation_code LIKE 'AU%';
```

```
    host_year  nation_code                  gold     sum_gold
=============================================================
         1988  'AUS'                           3            3
         1992  'AUS'                           7           10
         1996  'AUS'                           9           19
         2000  'AUS'                          16           35
         2004  'AUS'                          17           52
         1988  'AUT'                           1            1
         1992  'AUT'                           0            1
         1996  'AUT'                           0            1
```

```
          2000  'AUT'                          2            3
          2004  'AUT'                          2            5
```

The following example is removing the "ORDER BY host_year" clause under the **OVER** analysis clause from the above example. The avg_gold value is the average of gold medals for all years, so the value is identical for every year by nation_code.

```
SELECT host_year, nation_code, gold, SUM(gold) OVER (PARTITION BY
nation_code) sum_gold
FROM participant
WHERE nation_code LIKE 'AU%';
```

```
    host_year  nation_code                  gold     sum_gold
=============================================================
          2004  'AUS'                        17           52
          2000  'AUS'                        16           52
          1996  'AUS'                         9           52
          1992  'AUS'                         7           52
          1988  'AUS'                         3           52
          2004  'AUT'                         2            5
          2000  'AUT'                         2            5
          1996  'AUT'                         0            5
          1992  'AUT'                         0            5
          1988  'AUT'                         1            5
```

VARIANCE, VAR_POP

VARIANCE(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

VAR_POP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

VARIANCE(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

VAR_POP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

> The functions **VARPOP** and **VARIANCE** are used interchangeably and they are used as an aggregate function or an analytic function. They return a variance of expression values for all rows. Only one *expression* is specified as a parameter. If the **DISTINCT** or **UNIQUE** keyword is inserted before the expression, they calculate the population variance after deleting duplicates; if the keyword is omitted or **ALL**, they calculate the sample population variance for all values.
>
> param expression:

Specifies an expression that returns a numeric value.

param ALL:

Gets the variance for all values (default).

param DISTINCT,DISTINCTROW,UNIQUE:

Gets the variance of unique values without duplicates.

rtype:

DOUBLE

The return value is a **DOUBLE** type. If there are no rows that can be used for calculating a result, **NULL** will be returned.

The following is a formula that is applied to the function.

$$\mathrm{VAR_{POP}} = \left(\frac{1}{\mathrm{N}}\right) * \mathrm{SUM}\left(\{\mathrm{x_I} - \mathrm{AVG(x)}\}^2\right)$$

Note

In CUBRID 2008 R3.1 or earlier, the **VARIANCE** function worked the same as the `VAR_SAMP()`.

The following example shows how to output the population variance of all students for all subjects

```
CREATE TABLE student (name VARCHAR(32), subjects_id INT, score
DOUBLE);
INSERT INTO student VALUES
('Jane',1, 78), ('Jane',2, 50), ('Jane',3, 60),
('Bruce', 1, 63), ('Bruce', 2, 50), ('Bruce', 3, 80),
('Lee', 1, 85), ('Lee', 2, 88), ('Lee', 3, 93),
('Wane', 1, 32), ('Wane', 2, 42), ('Wane', 3, 99),
('Sara', 1, 17), ('Sara', 2, 55), ('Sara', 3, 43);

SELECT VAR_POP(score) FROM student;
```

```
           var_pop(score)
===========================
     5.427555555555550e+02
```

The following example shows how to output the score and population variance of all students by subject (subjects_id).

```
SELECT subjects_id, name, score, VAR_POP(score) OVER(PARTITION BY
subjects_id) v_pop
FROM student
ORDER BY subjects_id, name;
```

```
  subjects_id  name
score                    v_pop
=============================================================================
==================
            1  'Bruce'                 6.300000000000000e+01
6.931999999999998e+02
            1  'Jane'                  7.800000000000000e+01
6.931999999999998e+02
            1  'Lee'                   8.500000000000000e+01
6.931999999999998e+02
            1  'Sara'                  1.700000000000000e+01
6.931999999999998e+02
            1  'Wane'                  3.200000000000000e+01
6.931999999999998e+02
            2  'Bruce'                 5.000000000000000e+01
2.575999999999999e+02
            2  'Jane'                  5.000000000000000e+01
2.575999999999999e+02
            2  'Lee'                   8.800000000000000e+01
2.575999999999999e+02
            2  'Sara'                  5.500000000000000e+01
2.575999999999999e+02
            2  'Wane'                  4.200000000000000e+01
2.575999999999999e+02
            3  'Bruce'                 8.000000000000000e+01
4.348000000000002e+02
            3  'Jane'                  6.000000000000000e+01
4.348000000000002e+02
            3  'Lee'                   9.300000000000000e+01
4.348000000000002e+02
            3  'Sara'                  4.300000000000000e+01
4.348000000000002e+02
            3  'Wane'                  9.900000000000000e+01
4.348000000000002e+02
```

VAR_SAMP

VAR_SAMP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression*)

VAR_SAMP(*[DISTINCT | DISTINCTROW | UNIQUE | ALL] expression) OVER (<analytic_clause>*)

The **VAR_SAMP** function is used as an aggregate function or an analytic function. It returns the sample variance. The denominator is the number of all rows - 1. Only one *expression* is specified as a parameter. If the **DISTINCT** or **UNIQUE** keyword is inserted before the expression, it calculates the sample variance after deleting duplicates and if the keyword is omitted or **ALL**, it calculates the sample variance for all values.

param expression:

Specifies one expression to return the numeric.

param ALL:

Is used to calculate the sample variance of unique values without duplicates. It is the default value.

param DISTINCT,DISTINCTROW,UNIQUE:

Is used to calculate the sample variance for the unique values without duplicates.

rtype:

DOUBLE

The return value is a **DOUBLE** type. If there are no rows that can be used for calculating a result, **NULL** is returned.

The following are the formulas applied to the function.

$$\mathrm{VAR_{SAMP}} = \left\{\frac{1}{N-1}\right\} * \mathrm{SUM}\left(\{x_I - \mathrm{mean}(x)\}^2\right)$$

The following example shows how to output the sample variance of all students for all subjects.

```
CREATE TABLE student (name VARCHAR(32), subjects_id INT, score
DOUBLE);
```

```
INSERT INTO student VALUES
('Jane',1, 78), ('Jane',2, 50), ('Jane',3, 60),
('Bruce', 1, 63), ('Bruce', 2, 50), ('Bruce', 3, 80),
('Lee', 1, 85), ('Lee', 2, 88), ('Lee', 3, 93),
('Wane', 1, 32), ('Wane', 2, 42), ('Wane', 3, 99),
('Sara', 1, 17), ('Sara', 2, 55), ('Sara', 3, 43);

SELECT VAR_SAMP(score) FROM student;
```

```
           var_samp(score)
==========================
     5.815238095238092e+02
```

The following example shows how to output the score and sample variance of all students by subject (subjects_id).

```
SELECT subjects_id, name, score, VAR_SAMP(score) OVER(PARTITION BY
subjects_id) v_samp
FROM student
ORDER BY subjects_id, name;
```

```
  subjects_id  name
score                    v_samp
=============================================================================
==================
            1  'Bruce'                 6.300000000000000e+01
8.665000000000000e+02
            1  'Jane'                  7.800000000000000e+01
8.665000000000000e+02
            1  'Lee'                   8.500000000000000e+01
8.665000000000000e+02
            1  'Sara'                  1.700000000000000e+01
8.665000000000000e+02
            1  'Wane'                  3.200000000000000e+01
8.665000000000000e+02
            2  'Bruce'                 5.000000000000000e+01
3.220000000000000e+02
            2  'Jane'                  5.000000000000000e+01
3.220000000000000e+02
            2  'Lee'                   8.800000000000000e+01
3.220000000000000e+02
            2  'Sara'                  5.500000000000000e+01
3.220000000000000e+02
            2  'Wane'                  4.200000000000000e+01
3.220000000000000e+02
            3  'Bruce'                 8.000000000000000e+01
5.435000000000000e+02
```

```
            3  'Jane'                    6.000000000000000e+01
5.435000000000000e+02
            3  'Lee'                     9.300000000000000e+01
5.435000000000000e+02
            3  'Sara'                    4.300000000000000e+01
5.435000000000000e+02
            3  'Wane'                    9.900000000000000e+01
5.435000000000000e+02
```

JSON_ARRAYAGG

JSON_ARRAYAGG(*json_val*)

Aggregate function that builds a json array out of the evaluated rows.

```
CREATE TABLE t_score(name VARCHAR(10), score INT);
INSERT INTO t_score VALUES
    ('Amie', 60),
    ('Jane', 80),
    ('Lora', 60),
    ('James', 75),
    ('Peter', 70),
    ('Tom', 30),
    ('Ralph', 99),
    ('David', 55),
    ('Amie', 65);

SELECT JSON_ARRAYAGG (name) AS test_takers from t_score;
```

```
  test_takers
======================
  ["Amie","Jane","Lora","James","Peter","Tom","Ralph","David","Amie"]
```

JSON_OBJECTAGG

JSON_OBJECTYAGG(*key*, *json_val expr*)

Creates a json object out of the (key, json_val) expressions gathered from each row evaluation.

```
CREATE TABLE t_score(name VARCHAR(10), score INT);
INSERT INTO t_score VALUES
    ('Amie', 60),
    ('Jane', 80),
```

```
    ('Lora', 60),
    ('James', 75),
    ('Peter', 70),
    ('Tom', 30),
    ('Ralph', 99),
    ('David', 55);

SELECT JSON_OBJECTAGG (name, score) AS test_scores from t_score;
```

```
  test_scores
======================
  {"Amie":60,"Jane":80,"Lora":60,"James":75,"Peter":70,"Tom":
30,"Ralph":99,"David":55}
```

Click Counter Functions

Contents

Click Counter Functions 691

- INCR, DECR 691

INCR, DECR

INCR(*column_name*)

DECR(*column_name*)

The **INCR** function increases the column's value given as a parameter of the **SELECT** statement by 1. The **DECR** function decreases the value of the column by 1.

Parameters:

column – the name of column defined with SMALLINT, INT or BIGINT type

Return type:

SMALLINT, INT or BIGINT

The **INCR** and **DECR** functions are called “click counters” and can be effectively used to increase the number of post views for a Bulletin Board System (BBS) type of web service. In a scenario where you want to **SELECT** a post and immediately increase the number of views by 1 using an **UPDATE** statement, you can view the post and increment the number at once by using the **INCR** function in a single **SELECT** statement.

The **INCR** function increments the column value specified as an argument. Only integer type numbers can be used as arguments. If the value is **NULL**, the **INCR** function returns the **NULL**. That is, a value must be valid in order to be incremented by the **INCR** function. The **DECR** function decrements the column value specified as a parameter.

If an **INCR** function is specified in the **SELECT** statement, the **COUNTER** value is incremented by 1 and the query result is displayed with the values before the increment. Furthermore, the **INCR** function does not increment the value of the row(tuple) affected by the query process but rather the one affected by the final result.

If you want to increase or decrease the click counter without specifying **INCR** or **DECR** on the **SELECT** list, specify **WITH INCREMENT FOR** *column* or **WITH DECREMENT FOR** *column* after the WHERE clause.

```
CREATE TABLE board (id INT, cnt INT, content VARCHAR(8096));
SELECT content FROM board WHERE id=1 WITH INCREMENT FOR cnt;
```

Note

- The **INCR/DECR** functions execute independent of user-defined transactions and is applied automatically to the database by the top operation internally used in the system, apart from the transaction’s **COMMIT/ROLLBACK**.
- When multiple **INCR/DECR** functions are specified in a single **SELECT** statement, the failure of any of the **INCR/DECR** functions leads to the failure of all of them.
- The **INCR/DECR** functions apply only to top-level **SELECT** statements. **SUB SELECT** statements such as **INSERT** … **SELECT** … statement and **UPDATE** table **SET** col = **SELECT** … statement are not supported. The following example shows where the **INCR** function is not allowed.

  ```
  SELECT b.content, INCR(b.read_count) FROM (SELECT * FROM board
  WHERE id = 1) AS b
  ```

- If the **SELECT** statement with **INCR/DECR** functions returns more than one row as a result, it is treated as an error. The final result where only one row exists is valid.

- The **INCR/DECR** function can be used only in numerical type. Applicable domains are limited to integer data types such as **SMALLINT**, **INTEGER** and **BIGINT**. They cannot be used in other types.
- When the **INCR** function is called, the value to be returned will be the current value, while the value to be stored will be the current value + 1. Execute the following statement to select the value to be stored as a result :

```
SELECT content, INCR(read_count) + 1 FROM board WHERE id = 1;
```

- If the defined maximum value of the type is exceeded, the **INCR** function initializes the column value to 0. Likewise, the column value is also initialized to 0 when the **DECR** function applies to the minimum value.
- Data inconsistency can occur because the **INCR/DECR** functions are executed regardless of **UPDATE** trigger. The following example shows the database inconsistency in that situation.

```
CREATE TRIGGER event_tr BEFORE UPDATE ON event EXECUTE REJECT;
SELECT INCR(players) FROM event WHERE gender='M';
```

- The **INCR** / **DECR** functions returns an error in the write-protected broker mode such as slave mode of HA configuration, CSQL Interpreter (csql -r) of read-only, Read Only or Standby Only mode(ACCESS_MODE=RO or SO in cubrid_broker.conf).

Example

Suppose that the following three rows of data are inserted into the 'board' table.

```
CREATE TABLE board (
  id  INT,
  title  VARCHAR(100),
  content  VARCHAR(4000),
  read_count  INT
);
INSERT INTO board VALUES (1, 'aaa', 'text...', 0);
INSERT INTO board VALUES (2, 'bbb', 'text...', 0);
INSERT INTO board VALUES (3, 'ccc', 'text...', 0);
```

The following example shows how to increment the value of the 'read_count' column in data whose 'id' value is 1 by using the **INCR** function.

```
SELECT content, INCR(read_count) FROM board WHERE id = 1;
```

```
  content                read_count
==================================
  'text...'                       0
```

In the example, the column value becomes read_count + 1 as a result of the **INCR** function in the **SELECT** statement. You can check the result using the following **SELECT** statement.

```
SELECT content, read_count FROM board WHERE id = 1;
```

```
  content                read_count
==================================
  'text...'                       1
```

ROWNUM Functions

ROWNUM, INST_NUM

ROWNUM

INST_NUM()

The **ROWNUM** function returns the number representing the order of the records that will be generated by the query result. The first result record is assigned 1, and the second result record is assigned 2.

Return type:

BIGINT

ROWNUM and **INST_NUM()** can be used in the **SELECT** statement; **ORDERBY_NUM()** can be used in the **SELECT** statement with **ORDER BY** clauses, and **GROUPBY_NUM()** can be used in the **SELECT** statement with **GROUP BY** clauses. The **ROWNUM** function can be used to limit the number of result records of the query in several ways. For example, it can be used to search only the first 10 records or to return even or odd number records.

The **ROWNUM** function has a result value as a big integer, and can be used wherever an expression is valid such as the **SELECT** or **WHERE** clause. However, it is not allowed to compare the result of the **ROWNUM** function with the attribute or the correlated subquery.

Note

- The **ROWNUM** function specified in the **WHERE** clause works the same as the **INST_NUM()** function.
- The **ROWNUM** function belongs to each **SELECT** statement. That is, if a **ROWNUM** function is used in a subquery, it returns the sequence of the subquery result while it is being executed. Internally, the result of the **ROWNUM** function is generated right before the searched record is written to the query result set. At this moment, the counter value that generates the serial number of the result set records increases.
- If you want to add a sequence on the **SELECT** result; use **ROWNUM** when there is no sorting process; use **ORDERBY_NUM()** when there is an ORDER BY clause; use **GROUPBY_NUM()** function when there is a GROUP BY clause.
- If an **ORDER BY** clause is included in the **SELECT** statement, the value of the **ORDERBY_NUM()** function specified in the **WHERE** clause is generated after sorting for the **ORDER BY** clause.
- If a **GROUP BY** clause is included in the **SELECT** statement, the value of the **GROUPBY_NUM()** function specified in the **HAVING** clause is calculated after the query results are grouped.
- For the purpose of limiting the sorted result rows, instead of **FOR ORDERBY_NUM()** or **HAVING GROUPBY_NUM()**, **LIMIT** clause can be used.
- The **ROWNUM** function can also be used in SQL statements such as **INSERT**, **DELETE** and **UPDATE** in addition to the **SELECT** statement. For example, as in the query **INSERT INTO** *table_name* **SELECT** ... **FROM** ... **WHERE** ..., you can search for part of the row from one table and then insert it into another by using the **ROWNUM** function in the **WHERE** clause.

The following example shows how to retrieve country names ranked first to fourth based on the number of gold (*gold*) medals in the 1988 Olympics in the *demodb* database.

```
--Limiting 4 rows using ROWNUM in the WHERE condition
SELECT  * FROM
(SELECT nation_code FROM participant WHERE host_year = 1988
```

```
     ORDER BY gold DESC) AS T
WHERE ROWNUM <5;
```

```
  nation_code
======================
  'URS'
  'GDR'
  'USA'
  'KOR'
```

LIMIT clause limits the sorted result rows; therefore, the below result is the same as the above.

```
--Limiting 4 rows using LIMIT
SELECT ROWNUM, nation_code FROM participant WHERE host_year = 1988
ORDER BY gold DESC
LIMIT 4;
```

```
                rownum  nation_code
=============================================
                   143  'URS'
                    51  'GDR'
                   145  'USA'
                    78  'KOR'
```

ROWNUM condition in the below limits before sorting; therefore, the result is different from the above.

```
--Unexpected results : ROWNUM operated before ORDER BY
SELECT ROWNUM, nation_code FROM participant
WHERE host_year = 1988 AND ROWNUM < 5
ORDER BY gold DESC;
```

```
                rownum  nation_code
=============================================
                     1  'AFG'
                     2  'AHO'
                     3  'AND'
                     4  'ANG'
```

ORDERBY_NUM

ORDERBY_NUM()

The **ORDERBY_NUM()** function is used with the **ROWNUM()** or **INST_NUM()** function to limit the number of result rows. The difference is that the **ORDERBY_NUM()** function is combined after the ORDER BY clause to give order to a result that has been already sorted. That is, when retrieving only some of the result rows by using **ROWNUM** in a condition clause of the **SELECT** statement that includes the **ORDER BY** clause, **ROWNUM** is applied first and then group sorting by **ORDER BY** is performed. On the other hand, when retrieving only some of the result rows by using the **ORDER_NUM()** function, **ROWNUM** is applied to the result of sorting by **ORDER BY**.

Return type:

BIGINT

The following example shows how to retrieve athlete names ranked 3rd to 5th and their records in the *history* table in the *demodb* database.

```
--Ordering first and then limiting rows using FOR ORDERBY_NUM()
SELECT ORDERBY_NUM(), athlete, score FROM history
ORDER BY score FOR ORDERBY_NUM() BETWEEN 3 AND 5;
```

```
         orderby_num()  athlete               score
=======================================================================
                     3  'Luo Xuejuan'         '01:07.0'
                     4  'Rodal Vebjorn'       '01:43.0'
                     5  'Thorpe Ian'          '01:45.0'
```

The following query using a LIMIT clause outputs the same result with the above query.

```
SELECT ORDERBY_NUM(), athlete, score FROM history
ORDER BY score LIMIT 2, 3;
```

The following query using ROWNUM limits the result rows before sorting; then ORDER BY sorting is operated.

```
--Limiting rows first and then Ordering using ROWNUM
SELECT athlete, score FROM history
WHERE ROWNUM BETWEEN 3 AND 5 ORDER BY score;
```

```
  athlete               score
=============================================
  'Thorpe Ian'          '01:45.0'
```

```
  'Thorpe Ian'          '03:41.0'
  'Hackett Grant'       '14:43.0'
```

GROUPBY_NUM

GROUPBY_NUM()

The **GROUPBY_NUM()** function is used with the **ROWNUM** or **INST_NUM()** function to limit the number of result rows. The difference is that the **GROUPBY_NUM()** function is combined after the **GROUP BY ... HAVING** clause to give order to a result that has been already sorted. In addition, while the **INST_NUM()** function is a scalar function, the **GROUPBY_NUM()** function is kind of an aggregate function.

That is, when retrieving only some of the result rows by using **ROWNUM** in a condition clause of the **SELECT** statement that includes the **GROUP BY** clause, **ROWNUM** is applied first and then group sorting by **GROUP BY** is performed. On the other hand, when retrieving only some of the result rows by using the **GROUPBY_NUM()** function, **ROWNUM** is applied to the result of group sorting by **GROUP BY**.

Return type:

BIGINT

The following example shows how to retrieve the fastest record in the previous five Olympic Games from the *history* table in the *demodb* database.

```
--Group-ordering first and then limiting rows using GROUPBY_NUM()
SELECT  GROUPBY_NUM(), host_year, MIN(score) FROM history
GROUP BY host_year HAVING GROUPBY_NUM() BETWEEN 1 AND 5;
```

```
         groupby_num()    host_year  min(score)
=======================================================
                     1         1968  '8.9'
                     2         1980  '01:53.0'
                     3         1984  '13:06.0'
                     4         1988  '01:58.0'
                     5         1992  '02:07.0'
```

The following query using a LIMIT clause outputs the same result with the above query.

```
SELECT  GROUPBY_NUM(), host_year, MIN(score) FROM history
GROUP BY host_year LIMIT 5;
```

The following query using ROWNUM limits the result rows before grouping; then GROUP BY operation is performed.

```
--Limiting rows first and then Group-ordering using ROWNUM
SELECT host_year, MIN(score) FROM history
WHERE ROWNUM BETWEEN 1 AND 5 GROUP BY host_year;
```

```
    host_year  min(score)
==================================
         2000  '03:41.0'
         2004  '01:45.0'
```

Information Functions

Contents

Information Functions 699

- CHARSET 700
- COERCIBILITY 701
- COLLATION 701
- CURRENT_USER, USER 702
- DATABASE, SCHEMA 703
- DBTIMEZONE 703
- DEFAULT 704
- DISK_SIZE 705
- INDEX_CARDINALITY 706
- INET_ATON 708
- INET_NTOA 708

- LAST_INSERT_ID 709
- LIST_DBS 712
- ROW_COUNT 713
- SESSIONTIMEZONE 714
- USER, SYSTEM_USER 714
- VERSION 715

CHARSET

CHARSET(*expr*)

This function returns the character set of *expr*.

Parameters:

expr – Target expression to get the character set.

Return type:

STRING

```
SELECT CHARSET('abc');
```

```
'iso88591'
```

```
SELECT CHARSET(_utf8'abc');
```

```
'utf8'
```

```
SET NAMES utf8;
SELECT CHARSET('abc');
```

```
'utf8'
```

COERCIBILITY

COERCIBILITY(*expr*)

This function returns the collation coercibility level of *expr*. The collation coercibility level determines which collation or charset should be used when each column(expression) has different collation or charset. For more details, please see Collation Coercibility.

Parameters:

expr – Target expression to get the collation coercibility level.

Return type:

INT

```
SELECT COERCIBILITY(USER());
```

```
7
```

```
SELECT COERCIBILITY(_utf8'abc');
```

```
10
```

COLLATION

COLLATION(*expr*)

This function returns the collation of *expr*.

Parameters:

expr – Target expression to get the collation.

Return type:

STRING

```
SELECT COLLATION('abc');
```

```
'iso88591_bin'
```

```
SELECT COLLATION(_utf8'abc');
```

```
'utf8_bin'
```

CURRENT_USER, USER

CURRENT_USER

USER

CURRENT_USER and **USER** are pseudo-columns and can be used interchangeably. They return the username that is currently logged in to the database as a string.

Please note that `SYSTEM_USER()` and `USER()` functions return the username with a hostname where CSQL or CAS was executed.

Return type:

STRING

```
--selecting the current user on the session
SELECT USER;
```

```
   CURRENT_USER
======================
  'PUBLIC'
```

```
SELECT USER(), CURRENT_USER;
```

```
   user()                CURRENT_USER
=============================================
  'PUBLIC@cdbs006.cub'   'PUBLIC'
```

```
--selecting all users of the current database from the system table
SELECT name, id, password FROM db_user;
```

```
  name                            id   password
=============================================================
  'DBA'                          NULL  NULL
  'PUBLIC'                       NULL  NULL
  'SELECT_ONLY_USER'             NULL  db_password
  'ALMOST_DBA_USER'              NULL  db_password
  'SELECT_ONLY_USER2'            NULL  NULL
```

DATABASE, SCHEMA

DATABASE()

SCHEMA()

The functions **DATABASE** and **SCHEMA** are used interchangeably. They return the name of currently-connected database as a **VARCHAR** type.

Return type:

STRING

```
SELECT DATABASE(), SCHEMA();
```

```
   database()            schema()
=============================================
  'demodb'              'demodb'
```

DBTIMEZONE

DBTIMEZONE()

Prints out a timezone of database server (offset or region name) as a string. (e.g. '-05:00', or 'Europe/Vienna').

```
SELECT DBTIMEZONE();
```

```
  dbtimezone
======================
  'Asia/Seoul'
```

See also

`SESSIONTIMEZONE()` , `FROM_TZ()` , `NEW_TIME()` , `TZ_OFFSET()`

DEFAULT

DEFAULT(*column_name*)

DEFAULT

The **DEFAULT** and the **DEFAULT** function returns a default value defined for a column. If a default value is not specified for the column, **NULL** or an error is output. **DEFAULT** has no parameter, however, the **DEFAULT** function uses the column name as the input parameter. **DEFAULT** can be used for the input data of the **INSERT** statement and the **SET** clause of the **UPDATE** statement and the **DEFAULT** function can be used anywhere.

If any of constraints is not defined or the **UNIQUE** constraint is defined for the column where a default value is not defined, **NULL** is returned. If **NOT NULL** or **PRIMARY KEY** constraint is defined, an error is returned.

```
CREATE TABLE info_tbl(id INT DEFAULT 0, name VARCHAR);
INSERT INTO info_tbl VALUES (1,'a'),(2,'b'),(NULL,'c');

SELECT id, DEFAULT(id) FROM info_tbl;
```

```
           id   default(id)
============================
            1             0
            2             0
         NULL             0
```

```
UPDATE info_tbl SET id = DEFAULT WHERE id IS NULL;
DELETE FROM info_tbl WHERE id = DEFAULT(id);
INSERT INTO info_tbl VALUES (DEFAULT,'d');
```

Note

In version lower than CUBRID 9.0, the value at the time of CREATE TABLE has been saved when the value of the DATE, DATETIME, TIME, TIMESTAMP column has been specified as SYS_DATE, SYS_DATETIME, SYS_TIME, SYS_TIMESTAMP while creating a table. Therefore, to enter the value at the time of data INSERT in version lower than CUBRID 9.0, the function should be entered to the VALUES clause of the INSERT syntax.

DISK_SIZE

DISK_SIZE(*expr*)

This function returns the size in bytes required to store the value of *expr* after evaluation. Main usage is to get necessary size for storing values in database heap file.

Parameters:

expr – Target expression to get the size.

Return type:

INTEGER

```
SELECT DISK_SIZE('abc'), DISK_SIZE(1);
```

```
   disk_size('abc')   disk_size(1)
====================================
                  7               4
```

The size depends on the actual content of value, string compression is also taken into account:

```
CREATE TABLE t1(s1 VARCHAR(10), s2 VARCHAR(300), c1 CHAR(10), c2
CHAR(300));
INSERT INTO t1 VALUES(REPEAT('a', 10), REPEAT('b', 300), REPEAT('c',
```

```
10), REPEAT('d', 300));
INSERT INTO t1 VALUES('a', 'b', 'c', 'd');
SELECT DISK_SIZE(s1), DISK_SIZE(s2), DISK_SIZE(c1), DISK_SIZE(c2)
FROM t1;
```

```
   disk_size(s1)   disk_size(s2)   disk_size(c1)   disk_size(c2)
=================================================================
              12              24              10             300
               4               4              10             300
```

INDEX_CARDINALITY

INDEX_CARDINALITY(*table*, *index*, *key_pos*)

The **INDEX_CARDINALITY** function returns the index cardinality in a table. The index cardinality is the number of unique values defining the index. The index cardinality can be applied even to the partial key of the multiple column index and displays the number of the unique value for the partial key by specifying the column location with the third parameter. Note that this value is an approximate value.

If you want the updated result from this function, you should run **UPDATE STATISTICS** statement.

Parameters:

- **table** – Table name
- **index** – Index name that exists in the *table*
- **key_pos** –

  Partial key location. It *key_pos* starts from 0 and has a range that is smaller than the number of columns consisting of keys; that is, the *key_pos* of the first column is 0. For the single column index, it is 0. It can be one of the following types.

  - Character string that can be converted to a numeric type.
  - Numeric type that can be converted to an integer type. The **FLOAT** or the **DOUBLE** types will be the value converted by the **ROUND** function.

Return type:

INT

The return value is 0 or a positive integer and if any of the input parameters is **NULL**, **NULL** is returned. If tables or indexes that are input parameters are not found, or *key_pos* is out of range, **NULL** is returned.

```
CREATE TABLE t1( i1 INTEGER ,
i2 INTEGER not null,
i3 INTEGER unique,
s1 VARCHAR(10),
s2 VARCHAR(10),
s3 VARCHAR(10) UNIQUE);

CREATE INDEX i_t1_i1 ON t1(i1 DESC);
CREATE INDEX i_t1_s1 ON t1(s1(7));
CREATE INDEX i_t1_i1_s1 on t1(i1,s1);
CREATE UNIQUE INDEX i_t1_i2_s2 ON t1(i2,s2);

INSERT INTO t1 VALUES (1,1,1,'abc','abc','abc');
INSERT INTO t1 VALUES (2,2,2,'zabc','zabc','zabc');
INSERT INTO t1 VALUES (2,3,3,'+abc','+abc','+abc');

UPDATE STATISTICS ON t1;
SELECT INDEX_CARDINALITY('t1','i_t1_i1_s1',0);
```

```
   index_cardinality('t1', 'i_t1_i1_s1', 0)
============================================
                                           2
```

```
SELECT INDEX_CARDINALITY('t1','i_t1_i1_s1',1);
```

```
   index_cardinality('t1', 'i_t1_i1_s1', 1)
============================================
                                           3
```

```
SELECT INDEX_CARDINALITY('t1','i_t1_i1_s1',2);
```

```
   index_cardinality('t1', 'i_t1_i1_s1', 2)
============================================
                                        NULL
```

```
SELECT INDEX_CARDINALITY('t123','i_t1_i1_s1',1);
```

```
  index_cardinality('t123', 'i_t1_i1_s1', 1)
============================================
                                        NULL
```

INET_ATON

INET_ATON(*ip_string*)

The **INET_ATON** function receives the string of an IPv4 address and returns a number. When an IP address string such as 'a.b.c.d' is entered, the function returns "a * 256 ^ 3 + b * 256 ^ 2 + c * 256 + d". The return type is **BIGINT**.

Parameters:

ip_string – IPv4 address string

Return type:

BIGINT

In the following example, 192.168.0.10 is calculated as "192 * 256 ^ 3 + 168 * 256 ^ 2 + 0 * 256 + 10".

```
SELECT INET_ATON('192.168.0.10');
```

```
   inet_aton('192.168.0.10')
============================
                  3232235530
```

INET_NTOA

INET_NTOA(*expr*)

The **INET_NTOA** function receives a number and returns an IPv4 address string. The return type is VARCHAR.

Parameters:

expr – Numeric expression

Return type:

STRING

```
SELECT INET_NTOA(3232235530);
```

```
   inet_ntoa(3232235530)
======================
  '192.168.0.10'
```

LAST_INSERT_ID

LAST_INSERT_ID()

The **LAST_INSERT_ID** function returns the value that has been most recently inserted to the **AUTO_INCREMENT** column by a single **INSERT** statement.

Return type:

BIGINT

The value returned by the **LAST_INSERT_ID** function has the following characteristics.

- The latest **LAST_INSERT_ID** value which was INSERTed successfully will be maintained. If it fails to INSERT, there is no change for **LAST_INSERT_ID**() value, but **AUTO_INCREMENT** value is internally increased. Therefore, **LAST_INSERT_ID**() value after the next **INSERT** statement's success reflects the internally increased **AUTO_INCREMENT** value.

```
CREATE TABLE tbl(a INT PRIMARY KEY AUTO_INCREMENT, b INT UNIQUE);
INSERT INTO tbl VALUES (null, 1);
INSERT INTO tbl VALUES (null, 1);
```

```
ERROR: Operation would have caused one or more unique constraint
violations.
```

```
INSERT INTO tbl VALUES (null, 1);
```

```
ERROR: Operation would have caused one or more unique constraint
violations.
```

```
SELECT LAST_INSERT_ID();
```

```
1

-- In 2008 R4.x or before, above value is 3.
```

```
INSERT INTO tbl VALUES (null, 2);
SELECT LAST_INSERT_ID();
```

```
4
```

- In the Multiple-rows **INSERT** statement(INSERT INTO tbl VALUES (), (), ..., ()), **LAST_INSERT_ID**() returns the firstly inserted **AUTO_INCREMENT** value. In other words, from the second row, there is no change on **LAST_INSERT_ID**() value even if the next rows are inserted.

```
INSERT INTO tbl VALUES (null, 11), (null, 12), (null, 13);
SELECT LAST_INSERT_ID();
```

```
5
```

```
INSERT INTO tbl VALUES (null, 21);
SELECT LAST_INSERT_ID();
```

```
8
```

- If **INSERT** statement succeeds to execute, **LAST_INSERT_ID** () value is not recovered to its previous value even if the transaction is rolled back.

```
-- csql> ;autocommit off
CREATE TABLE tbl2(a INT PRIMARY KEY AUTO_INCREMENT, b INT UNIQUE);
INSERT INTO tbl2 VALUES (null, 1);
```

```
COMMIT;

SELECT LAST_INSERT_ID();
```

```
1
```

```
INSERT INTO tbl2 VALUES (null, 2);
INSERT INTO tbl2 VALUES (null, 3);

ROLLBACK;

SELECT LAST_INSERT_ID();
```

```
3
```

- **LAST_INSERT_ID**() value used from the inside of a trigger cannot be identified from the outside of the trigger.
- **LAST_INSERT_ID**is independently kept by a session of each application.

```
CREATE TABLE ss (id INT AUTO_INCREMENT NOT NULL PRIMARY KEY, text
VARCHAR(32));
INSERT INTO ss VALUES (NULL, 'cubrid');
SELECT LAST_INSERT_ID ();
```

```
     last_insert_id()
======================
                     1
```

```
INSERT INTO ss VALUES (NULL, 'database'), (NULL, 'manager');
SELECT LAST_INSERT_ID ();
```

```
     last_insert_id()
======================
                     2
```

```
CREATE TABLE tbl (id INT AUTO_INCREMENT);
INSERT INTO tbl values (500), (NULL), (NULL);
SELECT LAST_INSERT_ID();
```

```
     last_insert_id()
======================
                     1
```

```
INSERT INTO tbl VALUES (500), (NULL), (NULL);
SELECT LAST_INSERT_ID();
```

```
     last_insert_id()
======================
                     3
```

```
SELECT * FROM tbl;
```

```
                    id
=======================
                   500
                     1
                     2
                   500
                     3
                     4
```

LIST_DBS

LIST_DBS()

The **LIST_DBS** function outputs the list of all databases in the directory file(**$CUBRID_DATABASES/databases.txt**), separated by blanks.

Return type:

STRING

```
SELECT LIST_DBS();
```

```
  list_dbs()
======================
  'testdb demodb'
```

ROW_COUNT

ROW_COUNT()

The **ROW_COUNT** function returns the number of rows updated (**UPDATE**, **INSERT**, **DELETE**, **REPLACE**) by the previous statement.

ROW_COUNT returns 1 for each inserted row and 2 for each updated row for **INSERT ON DUPLICATE KEY UPDATE** statement. It returns the sum of number of deleted and inserted rows for **REPLACE** statement.

Statements triggered by trigger will not affect the ROW_COUNT for the statement.

Return type:

INT

```
CREATE TABLE rc (i int);
INSERT INTO rc VALUES (1),(2),(3),(4),(5),(6),(7);
SELECT ROW_COUNT();
```

```
   row_count()
===============
             7
```

```
UPDATE rc SET i = 0 WHERE i >  3;
SELECT ROW_COUNT();
```

```
   row_count()
===============
             4
```

```
DELETE FROM rc WHERE i = 0;
SELECT ROW_COUNT();
```

```
   row_count()
===============
             4
```

SESSIONTIMEZONE

SESSIONTIMEZONE()

Prints out a timezone of session (offset or region name) as a string. (e.g. '-05:00', or 'Europe/Vienna').

```
SELECT SESSIONTIMEZONE();
```

```
  sessiontimezone
======================
  'Asia/Seoul'
```

See also

`DBTIMEZONE()`, `FROM_TZ()`, `NEW_TIME()`, `TZ_OFFSET()`

USER, SYSTEM_USER

USER()

SYSTEM_USER()

The functions **USER** and **SYSTEM_USER** are identical and they return the username together with the hostname where CSQL or CAS was executed.

The `USER` and `CURRENT_USER` pseudo-columns return the usernames who has logged on to the current database as character strings.

Return type:

STRING

```
--selecting the current user on the session
SELECT SYSTEM_USER ();
```

```
   user()
======================
  'PUBLIC@cubrid_host'
```

```
SELECT USER(), CURRENT_USER;
```

```
   user()                CURRENT_USER
============================================
  'PUBLIC@cubrid_host'   'PUBLIC'
```

```
--selecting all users of the current database from the system table
SELECT name, id, password FROM db_user;
```

```
  name                            id  password
=============================================================
  'DBA'                         NULL  NULL
  'PUBLIC'                      NULL  NULL
  'SELECT_ONLY_USER'            NULL  db_password
  'ALMOST_DBA_USER'             NULL  db_password
  'SELECT_ONLY_USER2'           NULL  NULL
```

VERSION

VERSION()

The **VERSION** function returns the version character string representing the CUBRID server version.

Return type:

STRING

```
SELECT VERSION();
```

```
   version()
=====================
  '9.1.0.0203'
```

Encryption Function

Contents

Encryption Function 716

- MD5 716
- SHA1 717
- SHA2 718

MD5

MD5(*string*)

The **MD5** function returns the MD5 128-bit checksum for the input character string. The result value is displayed as a character string that is expressed in 32 hexadecimals, which you can use to create hash keys, for example.

Parameters:

string – Input string. If a value that is not a **VARCHAR** type is entered, it will be converted to **VARCHAR**.

Return type:

STRING

The return value is a **VARCHAR** (32) type and if an input parameter is **NULL**, **NULL** will be returned.

```
SELECT MD5('cubrid');
```

```
   md5('cubrid')
======================
  '685c62385ce717a04f909047d0a55a16'
```

```
SELECT MD5(255);
```

```
   md5(255)
======================
  'fe131d7f5a6b38b23cc967316c13dae2'
```

```
SELECT MD5('01/01/2010');
```

```
   md5('01/01/2010')
======================
  '4a2f373c30426a1b8e9cf002ef0d4a58'
```

```
SELECT MD5(CAST('2010-01-01' as DATE));
```

```
   md5( cast('2010-01-01' as date))
======================
  '4a2f373c30426a1b8e9cf002ef0d4a58'
```

SHA1

SHA1(*string*)

The **SHA1** function calculates an SHA-1 160-bit checksum for the string, as described in RFC 3174 (Secure Hash Algorithm).

Parameters:

string – target string to be encrypted

Return type:

STRING

The value is returned as a string of 40 hex digits, or NULL if the argument is NULL.

```
SELECT SHA1('cubrid');
```

```
      sha1('cubrid')
======================
  '0562A8E9C814E660F5FFEB0DAC739ABFBBB1CB69'
```

SHA2

SHA2(*string*, *hash_length*)

The **SHA2** function calculates the SHA-2 family of hash functions (SHA-224, SHA-256, SHA-384, and SHA-512). The first argument is the cleartext string to be hashed. The second argument indicates the desired bit length of the result, which must have a value of 224, 256, 384, 512, or 0 (which is equivalent to 256).

Parameters:

string – target string to be encrypted

Return type:

STRING

If either argument is NULL or the hash length is not one of the permitted values, the return value is NULL. Otherwise, the function result is a hash value containing the desired number of bits.

```
SELECT SHA2('cubrid', 256);
```

```
   sha2('cubrid', 256)
======================
  'D14DA17F2C492114F4A57D9F7BED908FD3A351B40CD59F0F79413687E4CA85A5'
```

```
SELECT SHA2('cubrid', 224);
```

```
   sha2('cubrid', 224)
======================
  '8E5E18B5B47646C31CCEA98A87B19CBEF084036716FBD13D723AC9B2'
```

Comparison Expression

Simple Comparison Expression

A comparison expression is an expression that is included in the **WHERE** clause of the **SELECT**, **UPDATE** and **DELETE** statements, and in the **HAVING** clause of the **SELECT** statement. There are simple comparison, **ANY** / **SOME** / **ALL**, **BETWEEN**, **EXISTS**, **IN** / **NOT IN**, **LIKE** and **IS NULL** comparison expressions, depending on the kinds of the operators combined.

A simple comparison expression compares two comparable data values. Expressions or subqueries are specified as operands, and the comparison expression always returns **NULL** if one of the operands is **NULL**. The following table shows operators that can be used in the simple comparison expressions. For details, see Comparison Operators.

Comparison Operators

Comparison Operator	Description	Comparison Expression	Return Value
=	A value of left operand is the same as that of right operand.	1=2	0
<> , !=	A value of left operand is not the same as that of right operand.	1<>2	1
>	A value of left operand is greater than that of right operand.	1>2	0
<	A value of left operand is less than that of right operand.	1<2	1
>=	A value of left operand is equal to or greater than that of right operand.	1>=2	0
<=	A value of left operand is equal to	1<=2	1

Comparison Operator	Description	Comparison Expression	Return Value
	or less than that of right operand.		

ANY/SOME/ALL quantifiers

A comparison expression that includes quantifiers such as **ANY/SOME/ALL** performs comparison operation on one data value and on some or all values included in the list. A comparison expression that includes **ANY** or **SOME** returns **TRUE** if the value of the data on the left satisfies simple comparison with at least one of the values in the list specified as an operand on the right. A comparison expression that includes **ALL** returns **TRUE** if the value of the data on the left satisfies simple comparison with all values in the list on the right.

When a comparison operation is performed on **NULL** in a comparison expression that includes **ANY** or **SOME**, **UNKNOWN** or **TRUE** is returned as a result; when a comparison operation is performed on **NULL** in a comparison expression that includes **ALL**, **UNKNOWN** or **FALSE** is returned.

```
expression comp_op SOME expression
expression comp_op ANY expression
expression comp_op ALL expression
```

- *comp_op* : A comparison operator >, = or <= can be used.
- *expression* (left): A single-value column, path expression (ex.: *tbl_name.col_name*), constant value or arithmetic function that produces a single value can be used.
- *expression* (right): A column name, path expression, list (set) of constant values or subquery can be used. A list is a set represented within braces ({}). If a subquery is used, *expression* (left) and comparison operation on all results of the subquery execution is performed.

```
--creating a table

CREATE TABLE condition_tbl (id int primary key, name char(10),
dept_name VARCHAR, salary INT);
INSERT INTO condition_tbl VALUES(1, 'Kim', 'devel', 4000000);
INSERT INTO condition_tbl VALUES(2, 'Moy', 'sales', 3000000);
INSERT INTO condition_tbl VALUES(3, 'Jones', 'sales', 5400000);
INSERT INTO condition_tbl VALUES(4, 'Smith', 'devel', 5500000);
INSERT INTO condition_tbl VALUES(5, 'Kim', 'account', 3800000);
INSERT INTO condition_tbl VALUES(6, 'Smith', 'devel', 2400000);
```

```
INSERT INTO condition_tbl VALUES(7, 'Brown', 'account', NULL);

--selecting rows where department is sales or devel
SELECT * FROM condition_tbl WHERE dept_name = ANY{'devel','sales'};
```

```
           id  name                 dept_name
salary
=============================================================================
=
            1  'Kim       '         'devel'
4000000
            2  'Moy       '         'sales'
3000000
            3  'Jones     '         'sales'
5400000
            4  'Smith     '         'devel'
5500000
            6  'Smith     '         'devel'
2400000
```

```
--selecting rows comparing NULL value in the ALL group conditions
SELECT * FROM condition_tbl WHERE salary > ALL{3000000, 4000000,
NULL};
```

```
There are no results.
```

```
--selecting rows comparing NULL value in the ANY group conditions
SELECT * FROM condition_tbl WHERE salary > ANY{3000000, 4000000,
NULL};
```

```
           id  name                 dept_name
salary
=============================================================================
=
            1  'Kim       '         'devel'
4000000
            3  'Jones     '         'sales'
5400000
            4  'Smith     '         'devel'
5500000
            5  'Kim       '         'account'
3800000
```

```
--selecting rows where salary*0.9 is less than those salary in devel
department
SELECT * FROM condition_tbl WHERE (
  (0.9 * salary) < ALL (SELECT salary FROM condition_tbl
  WHERE dept_name = 'devel')
);
```

```
           id  name                 dept_name
salary
=============================================================================
=
            6  'Smith      '        'devel'
2400000
```

BETWEEN

The **BETWEEN** makes a comparison to determine whether the data value on the left exists between two data values specified on the right. It returns **TRUE** even when the data value on the left is the same as a boundary value of the comparison target range. If **NOT** comes before the **BETWEEN** keyword, the result of a **NOT** operation on the result of the **BETWEEN** operation is returned.

i **BETWEEN** *g* **AND** *m* and the compound condition *i* **>= g AND** *i* <= *m* have the same effect.

```
expression [ NOT ] BETWEEN expression AND expression
```

- *expression* : A column name, path expression (ex.: *tbl_name.col_name*), constant value, arithmetic expression or aggregate function can be used. For a character string expression, the conditions are evaluated in alphabetical order. If **NULL** is specified for at least one of the expressions, the **BETWEEN** predicate returns **UNKNOWN** as a result.

```
--selecting rows where 3000000 <= salary <= 4000000
SELECT * FROM condition_tbl WHERE salary BETWEEN 3000000 AND 4000000;
SELECT * FROM condition_tbl WHERE (salary >= 3000000) AND (salary <=
4000000);
```

```
           id  name                 dept_name
salary
=============================================================================
=
            1  'Kim        '        'devel'
4000000
```

```
            2  'Moy       '           'sales'
3000000
            5  'Kim       '           'account'
3800000
```

```
--selecting rows where salary < 3000000 or salary > 4000000
SELECT * FROM condition_tbl WHERE salary NOT BETWEEN 3000000 AND
4000000;
```

```
           id  name                 dept_name
salary
=============================================================
=
            3  'Jones     '         'sales'
5400000
            4  'Smith     '         'devel'
5500000
            6  'Smith     '         'devel'
2400000
```

```
--selecting rows where name starts from A to E
SELECT * FROM condition_tbl WHERE name BETWEEN 'A' AND 'E';
```

```
           id  name                 dept_name
salary
=============================================================
=
            7  'Brown     '         'account'
NULL
```

EXISTS

The **EXISTS** returns **TRUE** if one or more results of the execution of the subquery specified on the right exist, and returns **FALSE** if the result of the operation is an empty set.

```
EXISTS expression
```

- *expression* : Specifies a subquery and compares to determine whether the result of the subquery execution exists. If the subquery does not produce any result, the result of the conditional expression is **FALSE**.

```
--selecting rows using EXISTS and subquery
SELECT 'raise' FROM db_root WHERE EXISTS(
SELECT * FROM condition_tbl WHERE salary < 2500000);
```

```
  'raise'
======================
  'raise'
```

```
--selecting rows using NOT EXISTS and subquery
SELECT 'raise' FROM db_root WHERE NOT EXISTS(
SELECT * FROM condition_tbl WHERE salary < 2500000);
```

```
There are no results.
```

IN

The **IN** compares to determine whether the single data value on the left is included in the list specified on the right. That is, the predicate returns **TRUE** if the single data value on the left is an element of the expression specified on the right. If **NOT** comes before the **IN** keyword, the result of a **NOT** operation on the result of the **IN** operation is returned.

```
expression [ NOT ] IN expression
```

- *expression* (left): A single-value column, path expression (ex.: *tbl_name.col_name*), constant value or arithmetic function that produces a single value can be used.
- *expression* (right): A column name, path expression, list (set) of constant values or subquery can be used. A list is a set represented within parentheses (()) or braces ({}). If a subquery is used, comparison with expression(left) is performed for all results of the subquery execution.

```
--selecting rows where department is sales or devel
SELECT * FROM condition_tbl WHERE dept_name IN {'devel','sales'};
SELECT * FROM condition_tbl WHERE dept_name = ANY{'devel','sales'};
```

```
           id  name                 dept_name
salary
=============================================================================
=
            1  'Kim       '         'devel'
4000000
```

```
            2  'Moy       '           'sales'
3000000
            3  'Jones     '           'sales'
5400000
            4  'Smith     '           'devel'
5500000
            6  'Smith     '           'devel'
2400000
```

```
--selecting rows where department is neither sales nor devel
SELECT * FROM condition_tbl WHERE dept_name NOT IN {'devel','sales'};
```

```
           id  name                 dept_name
salary
=============================================================================
=
            5  'Kim       '           'account'
3800000
            7  'Brown     '           'account'
NULL
```

IS NULL

The **IS NULL** compares to determine whether the expression specified on the left is **NULL**, and if it is **NULL**, returns **TRUE** and it can be used in the conditional expression. If **NOT** comes before the **NULL** keyword, the result of a **NOT** operation on the result of the **IS NULL** operation is returned.

expression IS [NOT] NULL

- *expression* : A single-value column, path expression (ex.: *tbl_name.col_name*), constant value or arithmetic function that produces a single value can be used.

```
--selecting rows where salary is NULL
SELECT * FROM condition_tbl WHERE salary IS NULL;
```

```
           id  name                 dept_name
salary
=============================================================================
=
            7  'Brown     '           'account'
NULL
```

```
--selecting rows where salary is NOT NULL
SELECT * FROM condition_tbl WHERE salary IS NOT NULL;
```

```
           id  name                 dept_name
salary
=============================================================================
=
            1  'Kim       '         'devel'
4000000
            2  'Moy       '         'sales'
3000000
            3  'Jones     '         'sales'
5400000
            4  'Smith     '         'devel'
5500000
            5  'Kim       '         'account'
3800000
            6  'Smith     '         'devel'
2400000
```

```
--simple comparison operation returns NULL when operand is NULL
SELECT * FROM condition_tbl WHERE salary = NULL;
```

```
There are no results.
```

LIKE

The **LIKE** compares patterns between character string data, and returns **TRUE** if a character string whose pattern matches the search word is found. Pattern comparison target types are **CHAR**, **VARCHAR** and **STRING**. The **LIKE** search cannot be performed on an **BIT** type. If **NOT** comes before the **LIKE** keyword, the result of a **NOT** operation on the result of the **LIKE** operation is returned.

A wild card string corresponding to any character or character string can be included in the search word on the right of the **LIKE** operator. % (percent) and _ (underscore) can be used. .% corresponds to any character string whose length is 0 or greater, and _ corresponds to one character. An escape character is a character that is used to search for a wild card character itself, and can be specified by the user as another character (**NULL**, alphabet, or number whose length is 1. See below for an example of using a character string that includes wild card or escape characters.

```
expression [ NOT ] LIKE pattern [ ESCAPE char ]
```

- *expression*: Specifies the data type column of the character string. Pattern comparison, which is case-sensitive, starts from the first character of the column.
- *pattern*: Enters the search word. A character string with a length of 0 or greater is required. Wild card characters (% or _) can be included as the pattern of the search word. The length of the character string is 0 or greater.
- **ESCAPE** *char* : **NULL**, alphabet, or number is allowed for *char*. If the string pattern of the search word includes "_" or "%" itself, an ESCAPE character must be specified. For example, if you want to search for the character string "10%" after specifying backslash (\) as the ESCAPE character, you must specify "10%" for *pattern*. If you want to search for the character string "C:\", you can specify "C:\" for *pattern*.

For details about character sets supported in CUBRID, see Character Strings.

Whether to detect the escape characters of the LIKE conditional expression is determined depending on the configuration of **no_backslash_escapes** and **require_like_escape_character** in the **cubrid.conf** file. For details, see Statement/Type-Related Parameters.

Note

- To execute string comparison operation for data entered in the multibyte charset environment such as UTF-8, the parameter setting (**single_byte_compare** = yes) which compares strings by 1 byte should be added to the **cubrid.conf** file for a successful search result.
- Versions after CUBRID 9.0 support Unicode charset, so the **single_byte_compare** parameter is no longer used.

```
--selection rows where name contains lower case 's', not upper case
SELECT * FROM condition_tbl WHERE name LIKE '%s%';
```

```
           id  name                 dept_name
salary
=============================================================================
=
            3  'Jones     '         'sales'
5400000
```

```
--selection rows where second letter is 'O' or 'o'
SELECT * FROM condition_tbl WHERE UPPER(name) LIKE '_O%';
```

```
           id  name                 dept_name
salary
=============================================================================
=
            2  'Moy       '         'sales'
3000000
            3  'Jones     '         'sales'
5400000
```

```
--selection rows where name is 3 characters
SELECT * FROM condition_tbl WHERE name LIKE '___';
```

```
           id  name                 dept_name
salary
=============================================================================
=
            1  'Kim       '         'devel'
4000000
            2  'Moy       '         'sales'
3000000
            5  'Kim       '         'account'
3800000
```

CASE

The **CASE** expression uses the SQL statement to perform an **IF** ... **THEN** statement. When a result of comparison expression specified in a **WHEN** clause is true, a value specified in **THEN** clause is returned. A value specified in an **ELSE** clause is returned otherwise. If no **ELSE** clause exists, **NULL** is returned.

```
CASE control_expression simple_when_list
[ else_clause ]
END

CASE searched_when_list
[ else_clause ]
END

simple_when :
WHEN expression THEN result
```

```
searched_when :
WHEN search_condition THEN result

else_clause :
ELSE result

result :
expression | NULL
```

The CASE expression must end with the END keyword. A *control_expression* argument and an *expression argument* in *simple_when* expression should be comparable data types. The data types of *result* specified in the **THEN** ... **ELSE** statement should all same, or they can be convertible to common data type.

The data type for a value returned by the **CASE** expression is determined based on the following rules.

- If data types for result specified in the **THEN** statement are all same, a value with the data type is returned.
- If data types can be convertible to common data type even though they are not all same, a value with the data type is returned.
- If any of values for *result* is a variable length string, a value data type is a variable length string. If values for *result* are all a fixed length string, the longest character string or bit string is returned.
- If any of values for result is an approximate numeric data type, a value with a numeric data type is returned. The number of digits after the decimal point is determined to display all significant figures.

```
--creating a table
CREATE TABLE case_tbl( a INT);
INSERT INTO case_tbl VALUES (1);
INSERT INTO case_tbl VALUES (2);
INSERT INTO case_tbl VALUES (3);
INSERT INTO case_tbl VALUES (NULL);

--case operation with a search when clause
SELECT a,
       CASE WHEN a=1 THEN 'one'
            WHEN a=2 THEN 'two'
            ELSE 'other'
       END
FROM case_tbl;
```

```
            a  case when a=1 then 'one' when a=2 then 'two' else
'other' end
====================================
            1  'one'
            2  'two'
            3  'other'
         NULL  'other'
```

```
--case operation with a simple when clause
SELECT a,
       CASE a WHEN 1 THEN 'one'
              WHEN 2 THEN 'two'
              ELSE 'other'
       END
FROM case_tbl;
```

```
            a  case a when 1 then 'one' when 2 then 'two' else
'other' end
====================================
            1  'one'
            2  'two'
            3  'other'
         NULL  'other'
```

```
--result types are converted to a single type containing all of
significant figures
SELECT a,
       CASE WHEN a=1 THEN 1
            WHEN a=2 THEN 1.2345
            ELSE 1.234567890
       END
FROM case_tbl;
```

```
            a  case when a=1 then 1 when a=2 then 1.2345 else
1.234567890 end
====================================
            1  1.000000000
            2  1.234500000
            3  1.234567890
         NULL  1.234567890
```

```
--an error occurs when result types are not convertible
SELECT a,
       CASE WHEN a=1 THEN 'one'
```

```
            WHEN a=2 THEN 'two'
            ELSE 1.2345
       END
FROM case_tbl;
```

```
ERROR: Cannot coerce 'one' to type double.
```

Comparison Functions

Contents

Comparison Functions 731

- COALESCE 731
- DECODE 733
- GREATEST 734
- IF 735
- IFNULL, NVL 737
- ISNULL 739
- LEAST 739
- NULLIF 740
- NVL2 742

COALESCE

COALESCE(*expression [, expression] ...*)

The **COALESCE** function has more than one expression as an argument. If the first argument is non-**NULL**, the corresponding value is returned if it is **NULL**, the second argument is returned. If all expressions which have an argument are **NULL**, **NULL** is returned. Therefore, this function is generally used to replace **NULL** with other default value.

Parameters:

expression – Specifies more than one expression. Their types must be comparable each other.

Return type:

determined with the type of the arguments

Operation is performed by converting the type of every argument into that with the highest priority. If there is an argument whose type cannot be converted, the type of every argument is converted into a **VARCHAR** type. The following list shows priority of conversion based on input argument type.

- **CHAR** < **VARCHAR**
- **BIT** < **VARBIT**
- **SHORT** < **INT** < **BIGINT** < **NUMERIC** < **FLOAT** < **DOUBLE**
- **DATE** < **TIMESTAMP** < **DATETIME**

For example, if a type of a is **INT**, b, **BIGINT**, c, **SHORT**, and d, **FLOAT**, then **COALESCE** (a, b, c, d) returns a **FLOAT** type. If a type of a is **INTEGER**, b, **DOULBE** , c, **FLOAT**, and d, **TIMESTAMP**, then **COALESCE** (a, b, c, d) returns a **VARCHAR** type.

COALESCE (*a, b*) works the same as the **CASE** expression as follows:

```
CASE WHEN a IS NOT NULL
THEN a
ELSE b
END
```

```
SELECT * FROM case_tbl;
```

```
            a
=============
            1
            2
            3
         NULL
```

```
--substituting a default value 10.0000 for a NULL value
SELECT a, COALESCE(a, 10.0000) FROM case_tbl;
```

```
            a  coalesce(a, 10.0000)
===================================
            1  1.0000
            2  2.0000
            3  3.0000
         NULL  10.0000
```

DECODE

DECODE(*expression, search, result [, search, result]* [, default]*)

As well as a **CASE** expression, the **DECODE** function performs the same functionality as the **IF** ... **THEN** ... **ELSE** statement. It compares the *expression* argument with *search* argument, and returns the *result* corresponding to *search* that has the same value. It returns *default* if there is no *search* with the same value, and returns **NULL** if *default* is omitted. An expression argument and a search argument to be comparable should be same or convertible each other. The number of digits after the decimal point is determined to display all significant figures including valid number of all *result*.

Parameters:

- **expression,search** – expressions that are comparable with each other
- **result** – the value to be returned when matched
- **default** – the value to be retuned when no match is found

Return type:

determined with the type of *result* and *default*

DECODE(*a*, *b*, *c*, *d*, *e*, *f*) has the same meaning as the **CASE** expression below.

```
CASE WHEN a = b THEN c
WHEN a = d THEN e
ELSE f
END
```

```
SELECT * FROM case_tbl;
```

```
             a
==============
             1
             2
             3
          NULL
```

```
--Using DECODE function to compare expression and search values one
by one
SELECT a, DECODE(a, 1, 'one', 2, 'two', 'other') FROM case_tbl;
```

```
             a  decode(a, 1, 'one', 2, 'two', 'other')
===================================
             1  'one'
             2  'two'
             3  'other'
          NULL  'other'
```

```
--result types are converted to a single type containing all of
significant figures
SELECT a, DECODE(a, 1, 1, 2, 1.2345, 1.234567890) FROM case_tbl;
```

```
             a  decode(a, 1, 1, 2, 1.2345, 1.234567890)
=================================
             1  1.000000000
             2  1.234500000
             3  1.234567890
          NULL  1.234567890
```

```
--an error occurs when result types are not convertible
SELECT a, DECODE(a, 1, 'one', 2, 'two', 1.2345) FROM case_tbl;
```

```
ERROR: Cannot coerce 'one' to type double.
```

GREATEST

GREATEST(*expression [, expression] ...*)

The **GREATEST** function compares more than one expression specified as parameters and returns the greatest value. If only one expression has been specified, the expression is returned because there is no expression to be compared with.

Therefore, more than one expression that is specified as parameters must be of the type that can be compared with each other. If the types of the specified parameters are identical, so are the types of the return values; if they are different, the type of the return value becomes a convertible common data type.

That is, the **GREATEST** function compares the values of column 1, column 2 and column 3 in the same row and returns the greatest value while the **MAX** function compares the values of column in all result rows and returns the greatest value.

Parameters:

expression – Specifies more than one expression. Their types must be comparable each other. One of the arguments is **NULL**, **NULL** is returned.

Return type:

same as that of the argument

The following example shows how to retrieve the number of every medals and the highest number that Korea won in the *demodb* database.

```
SELECT gold, silver , bronze, GREATEST (gold, silver, bronze)
FROM participant
WHERE nation_code = 'KOR';
```

```
         gold       silver       bronze  greatest(gold, silver,
bronze)
=============================================================================
==
            9           12            9
12
            8           10           10
10
            7           15            5
15
           12            5           12
12
           12           10           11
12
```

IF

IF(*expression1*, *expression2*, *expression3*)

The **IF** function returns *expression2* if the value of the arithmetic expression specified as the first parameter is **TRUE**, or *expression3* if the value is **FALSE** or **NULL**. *expression2* and *expression3* which are returned as a result must be the same or of a convertible common type. If one is explicitly **NULL**, the result of the function follows the type of the non-**NULL** parameter.

Parameters:

- **expression1** – comparison expression
- **expression2** – the value to be returned when *expression1* is true
- **expression3** – the value to be returned when *expression1* is not true

Return type:

type of *expression2* or *expression3*

IF(*a*, *b*, *c*) has the same meaning as the **CASE** expression in the following example:

```
CASE WHEN a IS TRUE THEN b
ELSE c
END
```

```
SELECT * FROM case_tbl;
```

```
             a
==============
             1
             2
             3
          NULL
```

```
--IF function returns the second expression when the first is TRUE
SELECT a, IF(a=1, 'one', 'other') FROM case_tbl;
```

```
           a   if(a=1, 'one', 'other')
=====================================
           1  'one'
           2  'other'
           3  'other'
        NULL  'other'
```

```
--If function in WHERE clause
SELECT * FROM case_tbl WHERE IF(a=1, 1, 2) = 1;
```

```
           a
=============
           1
```

IFNULL, NVL

IFNULL(*expr1*, *expr2*)

NVL(*expr1*, *expr2*)

The **IFNULL** function is working like the **NVL** function; however, only the **NVL** function supports collection type as well. The **IFNULL** function (which has two arguments) returns *expr1* if the value of the first expression is not **NULL** or returns *expr2*, otherwise.

Parameters:

- **expr1** – expression
- **expr2** – the value to be returned when *expr1* is **NULL**

Return type:

determined with the type of *expr1* and *expr2*

Operation is performed by converting the type of every argument into that with the highest priority. If there is an argument whose type cannot be converted, the type of every argument is converted into a **VARCHAR** type. The following list shows priority of conversion based on input argument type.

- **CHAR** < **VARCHAR**
- **BIT** < **VARBIT**

- **SHORT** < **INT** < **BIGINT** < **NUMERIC** < **FLOAT** < **DOUBLE**
- **DATE** < **TIMESTAMP** < **DATETIME**

For example, if a type of a is **INT** and b is **BIGINT**, then **IFNULL** (a, b) returns a **BIGINT** type. If a type of a is **INTEGER** and b is **TIMESTAMP**, then **IFNULL** (a, b) returns a **VARCHAR** type.

IFNULL(*a*, *b*) or **NVL**(*a*, *b*) has the same meaning as the **CASE** expression below.

```
CASE WHEN a IS NULL THEN b
ELSE a
END
```

```
SELECT * FROM case_tbl;
```

```
             a
==============
             1
             2
             3
          NULL
```

```
--returning a specific value when a is NULL
SELECT a, NVL(a, 10.0000) FROM case_tbl;
```

```
             a  nvl(a, 10.0000)
====================================
             1  1.0000
             2  2.0000
             3  3.0000
          NULL  10.0000
```

```
--IFNULL can be used instead of NVL and return values are converted
to the string type
SELECT a, IFNULL(a, 'UNKNOWN') FROM case_tbl;
```

```
             a   ifnull(a, 'UNKNOWN')
====================================
             1  '1'
             2  '2'
             3  '3'
          NULL  'UNKNOWN'
```

ISNULL

ISNULL(*expression*)

The **ISNULL** function performs a comparison to determine if the result of the expression specified as an argument is **NULL**. The function returns 1 if it is **NULL** or 0 otherwise. You can check if a certain value is **NULL**. This function is working like the **ISNULL** expression.

Parameters:

expression – An arithmetic function that has a single-value column, path expression (ex.: *tbl_name.col_name*), constant value is specified.

Return type:

INT

```
--Using ISNULL function to select rows with NULL value
SELECT * FROM condition_tbl WHERE ISNULL(salary);
```

```
           id  name                 dept_name
salary
=============================================================================
=
            7  'Brown     '         'account'
NULL
```

LEAST

LEAST(*expression [, expression] ...*)

The **LEAST** function compares more than one expression specified as parameters and returns the smallest value. If only one expression has been specified, the expression is returned because there is no expression to be compared with.

Therefore, more than one expression that is specified as parameters must be of the type that can be compared with each other. If the types of the specified parameters are identical, so are the types of the return values; if they are different, the type of the return value becomes a convertible common data type.

That is, the **LEAST** function compares the values of column 1, column 2 and column 3 in the same row and returns the smallest value while the `MIN()` compares the values of column in all result rows and returns the smallest value.

Parameters:

expression – Specifies more than one expression. Their types must be comparable each other. One of the arguments is **NULL**, **NULL** is returned.

Return type:

same as that of the argument

The following example shows how to retrieve the number of every medals and the lowest number that Korea won in the *demodb* database.

```
SELECT gold, silver , bronze, LEAST(gold, silver, bronze) FROM
participant
WHERE nation_code = 'KOR';
```

```
         gold       silver       bronze  least(gold, silver, bronze)
=============================================================================
            9           12            9                            9
            8           10           10                            8
            7           15            5                            5
           12            5           12                            5
           12           10           11                           10
```

NULLIF

NULLIF(*expr1*, *expr2*)

The **NULLIF** function returns **NULL** if the two expressions specified as the parameters are identical, and returns the first parameter value otherwise.

Parameters:

- **expr1** – expression to be compared with *expr2*
- **expr2** – expression to be compared with *expr1*

Return type:

type of *expr1*

NULLIF (*a*, *b*) is the same of the **CASE** expression.

```
CASE
WHEN a = b THEN NULL
ELSE a
END
```

```
SELECT * FROM case_tbl;
```

```
            a
=============
            1
            2
            3
         NULL
```

```
--returning NULL value when a is 1
SELECT a, NULLIF(a, 1) FROM case_tbl;
```

```
            a  nullif(a, 1)
============================
            1          NULL
            2             2
            3             3
         NULL          NULL
```

```
--returning NULL value when arguments are same
SELECT NULLIF (1, 1.000)  FROM db_root;
```

```
  nullif(1, 1.000)
======================
  NULL
```

```
--returning the first value when arguments are not same
SELECT NULLIF ('A', 'a')  FROM db_root;
```

```
  nullif('A', 'a')
======================
  'A'
```

NVL2

NVL2(*expr1*, *expr2*, *expr3*)

Three parameters are specified for the **NVL2** function. The second expression (*expr2*) is returned if the first expression (*expr1*) is not **NULL**; the third expression (*expr3*) is returned if it is **NULL**.

Parameters:

- **expr1** – expression
- **expr2** – the value to be returned when *expr1* is not **NULL**
- **expr3** – the value to be returned when *expr1* is **NULL**

Return type:

determined with the type of *expr1*, *expr2* and *expr3*

Operation is performed by converting the type of every argument into that with the highest priority. If there is an argument whose type cannot be converted, the type of every argument is converted into a **VARCHAR** type. The following list shows priority of conversion based on input argument type.

- **CHAR** < **VARCHAR**
- **BIT** < **VARBIT**
- **SHORT** < **INT** < **BIGINT** < **NUMERIC** < **FLOAT** < **DOUBLE**
- **DATE** < **TIMESTAMP** < **DATETIME**

For example, if a type of a is **INT**, b, **BIGINT**, and c, **SHORT**, then **NVL2** (a, b, c) returns a **BIGINT** type. If a type of a is **INTEGER**, b, **DOUBLE**, and c, **TIMESTAMP**, then **NVL2** (a, b, c) returns a **VARCHAR** type.

```
SELECT * FROM case_tbl;
```

```
             a
==============
             1
             2
             3
          NULL
```

```
--returning a specific value of INT type
SELECT a, NVL2(a, a+1, 10.5678) FROM case_tbl;
```

```
            a  nvl2(a, a+1, 10.5678)
=======================================
            1                       2
            2                       3
            3                       4
         NULL                      11
```

Other functions

Contents

Other functions 743

- SLEEP 743
- SYS_GUID 744

SLEEP

SLEEP(*sec*)

This function pauses for the specified time then resumes the operations.

Parameters:

sec – sleep time. The unit is second and inputs double type value.

Return type:

INT

```
SELECT SLEEP(3);
```

It pauses for 3 seconds.

SYS_GUID

SYS_GUID()

It returns the unique hexadecimal string of 32 characters randomly.

```
SELECT SYS_GUID();
```

```
sys_guid()
==================================
'938210043A7B4455927A8697FB2571FF'
```

Data Manipulation Statements

SELECT

The **SELECT** statement specifies columns that you want to retrieve from a table.

```
SELECT [ <qualifier> ] <select_expressions>
    [{TO | INTO} <variable_comma_list>]
    [FROM <extended_table_specification_comma_list>]
    [WHERE <search_condition>]
    [GROUP BY {col_name | expr} [ASC | DESC], ...[WITH ROLLUP]]
    [HAVING  <search_condition> ]
    [USING INDEX { index_name [,index_name, ...] | NONE }]
    [ORDER BY {col_name | expr} [ASC | DESC], ... [NULLS {FIRST |
LAST}]
    [LIMIT [offset,] row_count]
    [FOR UPDATE [OF <spec_name_comma_list>]]

    <qualifier> ::= ALL | DISTINCT | DISTINCTROW | UNIQUE

    <select_expressions> ::= * | <expression_comma_list> | *,
<expression_comma_list>

    <variable_comma_list> ::= [:] identifier, [:] identifier, ...

    <extended_table_specification_comma_list> ::=
        <table_specification>   [
                                    {, <table_specification> } ... |
                                    <join_table_specification> ... |
                                    <join_table_specification2> ...
                                ]
```

```
<table_specification> ::=
    <single_table_spec> [<correlation>] |
    <metaclass_specification> [ <correlation> ] |
    <subquery> <correlation> |
    TABLE ( <expression> ) <correlation>

<correlation> ::= [AS] <identifier> [(<identifier_comma_list>)]

<single_table_spec> ::= [ONLY] [schema_name.]table_name |
                        ALL [schema_name.]table_name [ EXCEPT
[schema_name.]table_name ]

<metaclass_specification> ::= CLASS [schema_name.]class_name

<join_table_specification> ::=
    {
        [INNER | {LEFT | RIGHT} [OUTER]] JOIN

    } <table_specification> ON <search_condition>

<join_table_specification2> ::=
    {
        CROSS JOIN |
        NATURAL [ LEFT | RIGHT ] JOIN
    } <table_specification>
```

- *qualifier*: A qualifier. When omitted, it is set to **ALL**.
  - **ALL**: Retrieves all records of the table.
  - **DISTINCT**: Retrieves only records with unique values without allowing duplicates. **DISTINCT**, **DISTINCTROW**, and **UNIQUE** are used interchangeably.
- <*select_expressions*>
  - *: By using **SELECT** * statement, you can retrieve all columns from the table specified in the **FROM** clause.
  - *expression_comma_list*: *expression* can be a path expression (ex.: *tbl_name.col_name*), variable or table name. All general expressions including arithmetic operations can also be used. Use a comma (,) to separate each expression in the list. You can specify aliases by using the **AS** keyword for columns or expressions to be queried. Specified aliases are used as column names in **GROUP BY**, **HAVING** and **ORDER BY** clauses. The position index of a column is assigned based on the order in which the column was specified. The starting value is 1.

    As **AVG**, **COUNT**, **MAX**, **MIN**, or **SUM**, an aggregate function that manipulates the retrieved data can also be used in the *expression*.

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*.*: Specifies the table name and using * has the same effect as specifying all columns for the given table.
- *variable_comma_list*: The data retrieved by the *select_expressions* can be stored in more than one variable.
- [:]*identifier*: By using the *:identifier* after **TO** (or **INTO**), you can store the data to be retrieved in the ':identifier' variable.
- <*single_table_spec*>
  - If a superclass name is specified after the **ONLY** keyword, only the superclass, not the subclass inheriting from it, is selected.
  - If a superclass name is specified after the **ALL** keyword, the superclass as well as the subclass inheriting from it are both selected.
  - You can define the list of subclass not to be selected after the **EXCEPT** keyword.

The following example shows how to retrieve host countries of the Olympic Games without any duplicates. This example is performed on the *olympic* table of *demodb*. The **DISTINCT** or **UNIQUE** keyword makes the query result unique. For example, when there are multiple *olympic* records of which each *host_nation* value is 'Greece', you can use such keywords to display only one value in the query result.

```
SELECT DISTINCT host_nation
FROM olympic;
```

```
  host_nation
======================
  'Australia'
  'Belgium'
  'Canada'
  'Finland'
  'France'
...
```

The following example shows how to define an alias to a column to be queried and sort the result record by using the column alias in the **ORDER BY** clause. At this time, the number of the result records is limited to 5 by using the **LIMIT** clause.

```
SELECT host_year as col1, host_nation as col2
FROM olympic
ORDER BY col2 LIMIT 5;
```

```
         col1  col2
=====================================
         2000  'Australia'
         1956  'Australia'
         1920  'Belgium'
         1976  'Canada'
         1948  'England'
```

```
SELECT CONCAT(host_nation, ', ', host_city) AS host_place
FROM olympic
ORDER BY host_place LIMIT 5;
```

```
  host_place
======================
  'Australia,  Melbourne'
  'Australia,  Sydney'
  'Belgium,  Antwerp'
  'Canada,  Montreal'
  'England,  London'
```

FROM Clause

The **FROM** clause specifies the table in which data is to be retrieved in the query. If no table is referenced, the **FROM** clause can be omitted. Retrieval paths are as follows:

- Single table
- Subquery
- Derived table
- Remote table

```
SELECT [<qualifier>] <select_expressions>
[
    FROM <table_specification> [ {, <table_specification> |
<join_table_specification> }... ]
]

<select_expressions> ::= * | <expression_comma_list> | *,
```

```
<expression_comma_list>

<table_specification> ::=
    <single_table_spec> [<correlation>] |
    <metaclass_specification> [<correlation>] |
    <subquery> <correlation> |
    TABLE (<expression>) <correlation> |
    DBLINK (<dblink_expr>) <dblink_identifier_col_attrs>

<correlation> ::= [AS] <identifier> [(<identifier_comma_list>)]

<single_table_spec> ::= [ONLY] [schema_name.]table_name |
                        ALL [schema_name.]table_name [EXCEPT
[schema_name.]table_name]

<metaclass_specification> ::= CLASS [schema_name.]class_name
```

- <*select_expressions*>: One or more columns or expressions to query is specified. Use * to query all columns in the table. You can also specify an alias for a column or an expression to be queried by using the AS keyword. This keyword can be used in **GROUP BY**, **HAVING** and **ORDER BY** clauses. The position index of the column is given according to the order in which the column was specified. The starting value is 1.

- <*table_specification*>: At least one table name is specified after the **FROM** clause. Subqueries and derived tables can also be used in the **FROM** clause. For details on subquery derived tables, see Subquery Derived Table.

```
--FROM clause can be omitted in the statement
SELECT 1+1 AS sum_value;
```

```
    sum_value
=============
            2
```

```
SELECT CONCAT('CUBRID', '2008' , 'R3.0') AS db_version;
```

```
  db_version
======================
  'CUBRID2008R3.0'
```

Derived Table

In the query statement, subqueries can be used in the table specification of the **FROM** clause. Such subqueries create derived tables where subquery results are treated as tables. A correlation specification must be used when a subquery that creates a derived table is used.

Derived tables are also used to access the individual element of an attribute that has a set value. In this case, an element of the set value is created as an instance in the derived table.

Subquery Derived Table

Each instance in the derived table is created from the result of the subquery in the **FROM** clause. A derived table created form a subquery can have any number of columns and records.

```
FROM (subquery) [AS] [derived_table_name [(column_name [{,
column_name } ... ])]]
```

- The number of *column_name* and the number of columns created by the *subquery* must be identical.
- *derived_table_name* can be omitted.

The following example shows how to retrieve the sum of the number of gold (*gold*) medals won by Korea and that of silver medals won by Japan. This example shows a way of getting an intermediate result of the subquery and processing it as a single result, by using a derived table. The query returns the sum of the *gold* values whose *nation_code* is 'KOR' and the *silver* values whose *nation_code* column is 'JPN'.

```
SELECT SUM (n)
FROM (SELECT gold FROM participant WHERE nation_code = 'KOR'
      UNION ALL
      SELECT silver FROM participant WHERE nation_code = 'JPN') AS
t(n);
```

Subquery derived tables can be useful when combined with outer queries. For example, a derived table can be used in the **FROM** clause of the subquery used in the **WHERE** clause. The following example shows *nation_code*, *host_year* and *gold* records whose number of gold medals is greater than average sum of the number of silver and bronze medals when one or more silver or bronze medals were won. In this example, the query (the outer **SELECT** clause) and the subquery (the inner **SELECT** clause) share the *nation_code* attribute.

```
SELECT nation_code, host_year, gold
FROM participant p
WHERE gold > (SELECT AVG(s)
```

```
              FROM (SELECT silver + bronze
                    FROM participant
                    WHERE nation_code = p.nation_code
                    AND silver > 0
                    AND bronze > 0)
                   AS t(s));
```

```
  nation_code      host_year      gold
=======================================
  'JPN'                2004         16
  'CHN'                2004         32
  'DEN'                1996          4
  'ESP'                1992         13
```

Remote Table

A remote table can be specified in the FROM clause, and when specifying a remote table, '@' is used and a table extension such as table_name@server_name is used. The remote server can use not only CUBRID but also other DBMS (ORACLE, MySQL, and MariaDB). The query for remote table is recreated and executed using the DBLINK statement while going through an optimization step.

```
-- at remote-side, "remote_server"
CREATE TABLE remote_tbl (
  id INT,
  name VARCHAR(32)
);

INSERT INTO remote_tbl VALUES (1, 'Kim');
INSERT INTO remote_tbl VALUES (2, 'Lee');
INSERT INTO remote_tbl VALUES (3, 'Park');
```

```
-- at local-side
SELECT *
FROM remote_tbl@remote_server rem
WHERE id < 3;
```

```
    id       name
==================
     1       Kim
     2       Lee
```

The query rewritten with the DBLINK statement for the remote table in the optimization stage is as follows.

```
SELECT *
FROM DBLINK(remote_server, 'SELECT id, name FROM remote_tbl WHERE id
< 3') AS dbl (id INT, name VARCHAR(32));
```

Note

Objects allowed for table extensions include general tables, synonyms, and views. The example below shows three types of table extensions.

```
-- at remote-side
CREATE TABLE remote_table (
  id INT,
  phone VARCHAR(12)
};

CREATE SYNONYM a_remote_tbl FOR user_a.remote_table
CREATE VIEW v_remote_tbl(r_phone) AS SELECT phone FROM remote_tble
WHERE id > 10;

-- at local-side

-- remote-table
SELECT phone FROM user_a.remote_table@server1 WHERE id > 10;

-- remote-synonym
SELECT phone FROM a_remote_tbl@server1 WHERE id > 10;

-- remote-view
SELECT r_phone FROM v_remote_tbl@server1;
```

All three queries above return the same result.

DBLINK

The result can be obtained by executing a query in a separate DBMS located at a remote location. The result is a kind of subquery that is created as Subquery Derived Table.

```
FROM DBLINK (<dblink_expr>) [AS] <dblink_identifier_col_attrs>

    <dblink_expr> ::= <dblink_conn>,  remote_query_sting
    <dblink_conn> ::= server_name | dblink_conn_string
```

```
    <dblink_identifier_col_attrs> ::= dblink_table_alias (
<dblink_column_definition_list> )
    <dblink_column_definition_list> ::= dblink_column_alias
<primitive_type> [{, dblink_column_alias <primitive_type>} ...]
```

- *remote_query_sting*: Only SELECT query can be specified as the query to be transmitted to the remote DBMS.
- *server_name*: The name of the server created using:doc:*/sql/schema/server_stmt*.
- *dblink_conn_string*: Remote access information expressed as a string.
- *dblink_table_alias*: The name of a derived table created using DBLINK.
- *dblink_column_alias*: Virtual column name corresponding to the select list of *remote_query_string* in DBLINK.

Note

The attributes of the column supported by DBLINK are as follows.

- INT, BIGINT, SHORT, FLOAT, DOUBLE, MONETARY, NUMERIC
- VARCHAR, CHAR
- DATE, TIME, TIMESTAMP, DATETIME
- DATETIMETZ, DATETIMELTZ, TIMESTAMPTZ, TIMESTAMPLTZ

Warning

DBLINK does not support columns with the following data types.

- COLLECTION TYPE (SET, MULTISET, SEQUENCE)
- OBJECT
- CLOB / BLOB
- ENUM
- BIT / BIT VARYING

- JSON

Note

dblink_conn_string is composed of the following structure. Each content is information corresponding to HOST, PORT, DBNAME, USER, PASSWOED, and PROPERTIES in the SERVER DEFINITION STATEMENTS syntax. Each item is separated by the character ':'.

<broker-host>:<port#>:<db_name>:<db_user>:<db_password>:[?<properties>]

To prevent password exposure, it is recommended to use *server_name* rather than *dblink_conn_string*.

```
CREATE SERVER remote_srv1 ( HOST='127.0.0.1', PORT=3300,
DBNAME=demodb, USER=cub, PASSWORD='cub-password');
SELECT * FROM DBLINK (remote_srv1, 'SELECT col1 FROM remote_t') AS
t(col1 int);

SELECT * FROM DBLINK ('127.0.0.1:3300:demodb:cub:cub-
password:','SELECT col1, col2 FROM remote_t') AS t(col1 int, col2
varchar(32));
```

In the example above, the two SELECT statements perform the same function.

WHERE Clause

In a query, a column can be processed based on conditions. The **WHERE** clause specifies a search condition for data.

```
WHERE <search_condition>

    <search_condition> ::=
        <comparison_predicate>
        <between_predicate>
        <exists_predicate>
        <in_predicate>
        <null_predicate>
        <like_predicate>
        <quantified_predicate>
        <set_predicate>
```

The **WHERE** clause specifies a condition that determines the data to be retrieved by *search_condition* or a query. Only data for which the condition is true is retrieved for the query results. (**NULL** value is not retrieved for the query results because it is evaluated as unknown value.)

- *search_condition*: It is described in detail in the following sections.
  - Simple Comparison Expression
  - BETWEEN
  - EXISTS
  - IN
  - IS NULL
  - LIKE
  - ANY/SOME/ALL quantifiers

The logical operator **AND** or **OR** can be used for multiple conditions. If **AND** is specified, all conditions must be true. If **OR** is specified, only one needs to be true. If the keyword **NOT** is preceded by a condition, the meaning of the condition is reserved. The following table shows the order in which logical operators are evaluated.

Priority	Operator	Function
1	**()**	Logical expressions in parentheses are evaluated first.
2	**NOT**	Negates the result of the logical expression.
3	**AND**	All conditions in the logical expression must be true.
4	**OR**	One of the conditions in the logical expression must be true.

GROUP BY ... HAVING Clause

The **GROUP BY** clause is used to group the result retrieved by the **SELECT** statement based on a specific column. This clause is used to sort by group or to get the aggregation by group using the aggregation function. Herein, a group consists of records that have the same value for the column specified in the **GROUP BY** clause.

You can also set a condition for group selection by including the **HAVING** clause after the **GROUP BY** clause. That is, only groups satisfying the condition specified by the **HAVING** clause are queried out of all groups that are grouped by the **GROUP BY** clause.

By SQL standard, you cannot specify a column (hidden column) not defined in the **GROUP BY** clause to the SELECT column list. However, by using extended CUBRID grammars, you can specify the hidden column to the SELECT column list. If you do not use the extended CUBRID grammars, the **only_full_group_by** parameter should be set to **yes**. For details, see Statement/Type-Related Parameters.

```
SELECT ...
GROUP BY {col_name | expr | position} [ASC | DESC], ...
         [WITH ROLLUP] [HAVING <search_condition>]
```

- *col_name* | *expr* | *position*: Specifies one or more column names, expressions, aliases or column location. Items are separated by commas. Columns are sorted on this basis.
- [**ASC** | **DESC**]: Specifies the **ASC** or **DESC** sorting option after the columns specified in the **GROUP BY** clause. If the sorting option is not specified, the default value is **ASC**.
- <*search_condition*>: Specifies the search condition in the **HAVING** clause. In the **HAVING** clause, you can refer to columns and aliases specified in the **GROUP BY** clause, or columns used in aggregate functions.

> **Note**
>
> Even the hidden columns not specified in the **GROUP BY** clause can be referred to, if the value of the **only_full_group_by** parameter is set to **yes**. At this time, the HAVING condition does not affect to the query result.

- **WITH ROLLUP**: If you specify the **WITH ROLLUP** modifier in the **GROUP BY** clause, the aggregate information of the result value of each GROUPed BY column is displayed for each group, and the total of all result rows is displayed at the last row. When a **WITH ROLLUP** modifier is defined in the **GROUP BY** clause, the result value for all rows of the group is additionally displayed. In other words, total aggregation is made for the value aggregated by group. When there are two columns for Group By, the former is considered as a large unit and the latter is

considered as a small unit, so the total aggregation row for the small unit and the total aggregation row for the large unit are added. For example, you can check the aggregation of the sales result per department and salesperson through one query.

```
-- creating a new table
CREATE TABLE sales_tbl
(dept_no INT, name VARCHAR(20), sales_month INT, sales_amount INT
DEFAULT 100, PRIMARY KEY (dept_no, name, sales_month));

INSERT INTO sales_tbl VALUES
(201, 'George' , 1, 450), (201, 'George' , 2, 250), (201, 'Laura'  ,
1, 100), (201, 'Laura'  , 2, 500),
(301, 'Max'    , 1, 300), (301, 'Max'    , 2, 300),
(501, 'Stephan', 1, 300), (501, 'Stephan', 2, DEFAULT), (501,
'Chang'  , 1, 150),(501, 'Chang'  , 2, 150),
(501, 'Sue'    , 1, 150), (501, 'Sue'    , 2, 200);

-- selecting rows grouped by dept_no
SELECT dept_no, avg(sales_amount)
FROM sales_tbl
GROUP BY dept_no;
```

```
      dept_no         avg(sales_amount)
=======================================
          201     3.250000000000000e+02
          301     3.000000000000000e+02
          501     1.750000000000000e+02
```

```
-- conditions in WHERE clause operate first before GROUP BY
SELECT dept_no, avg(sales_amount)
FROM sales_tbl
WHERE sales_amount > 100
GROUP BY dept_no;
```

```
      dept_no         avg(sales_amount)
=======================================
          201     4.000000000000000e+02
          301     3.000000000000000e+02
          501     1.900000000000000e+02
```

```
-- conditions in HAVING clause operate last after GROUP BY
SELECT dept_no, avg(sales_amount)
FROM sales_tbl
```

```
WHERE sales_amount > 100
GROUP BY dept_no HAVING avg(sales_amount) > 200;
```

```
      dept_no         avg(sales_amount)
=======================================
          201     4.000000000000000e+02
          301     3.000000000000000e+02
```

```
-- selecting and sorting rows with using column alias
SELECT dept_no AS a1, avg(sales_amount) AS a2
FROM sales_tbl
WHERE sales_amount > 200 GROUP
BY a1 HAVING a2 > 200
ORDER BY a2;
```

```
           a1                         a2
=======================================
          301     3.000000000000000e+02
          501     3.000000000000000e+02
          201     4.000000000000000e+02
```

```
-- selecting rows grouped by dept_no, name with WITH ROLLUP modifier
SELECT dept_no AS a1, name AS a2, avg(sales_amount) AS a3
FROM sales_tbl
WHERE sales_amount > 100
GROUP BY a1, a2 WITH ROLLUP;
```

```
           a1  a2                                        a3
=============================================================
          201  'George'                  3.500000000000000e+02
          201  'Laura'                   5.000000000000000e+02
          201  NULL                      4.000000000000000e+02
          301  'Max'                     3.000000000000000e+02
          301  NULL                      3.000000000000000e+02
          501  'Chang'                   1.500000000000000e+02
          501  'Stephan'                 3.000000000000000e+02
          501  'Sue'                     1.750000000000000e+02
          501  NULL                      1.900000000000000e+02
         NULL  NULL                      2.750000000000000e+02
```

ORDER BY Clause

The **ORDER BY** clause sorts the query result set in ascending or descending order. If you do not specify a sorting option such as **ASC** or **DESC**, the result set in ascending order by default. If you do not specify the **ORDER BY** clause, the order of records to be queried may vary depending on query.

```
SELECT ...
ORDER BY {col_name | expr | position} [ASC | DESC], ...] [NULLS
{FIRST | LAST}]
```

- *col_name* | *expr* | *position*: Specifies a column name, expression, alias, or column location. One or more column names, expressions or aliases can be specified. Items are separated by commas. A column that is not specified in the list of **SELECT** columns can be specified.
- [**ASC** | **DESC**]: **ASC** means sorting in ascending order, and **DESC** is sorting in descending order. If the sorting option is not specified, the default value is **ASC**.
- [**NULLS** {**FIRST** | **LAST**}]: **NULLS FIRST** sorts NULL at first, **NULLS LAST** sorts NULL at last. If this syntax is omitted, **ASC** sorts NULL at first, **DESC** sorts NULL at last.

```
-- selecting rows sorted by ORDER BY clause
SELECT *
FROM sales_tbl
ORDER BY dept_no DESC, name ASC;
```

```
      dept_no  name                  sales_month  sales_amount
==============================================================
          501  'Chang'                         1           150
          501  'Chang'                         2           150
          501  'Stephan'                       1           300
          501  'Stephan'                       2           100
          501  'Sue'                           1           150
          501  'Sue'                           2           200
          301  'Max'                           1           300
          301  'Max'                           2           300
          201  'George'                        1           450
          201  'George'                        2           250
          201  'Laura'                         1           100
          201  'Laura'                         2           500
```

```
-- sorting reversely and limiting result rows by LIMIT clause
SELECT dept_no AS a1, avg(sales_amount) AS a2
FROM sales_tbl
GROUP BY a1
```

```
ORDER BY a2 DESC
LIMIT 3;
```

```
          a1           a2
=======================================
         201     3.250000000000000e+02
         301     3.000000000000000e+02
         501     1.750000000000000e+02
```

The following is an example how to specify the NULLS FIRST or NULLS LAST after ORDER BY clause.

```
CREATE TABLE tbl (a INT, b VARCHAR);

INSERT INTO tbl VALUES
(1,NULL), (2,NULL), (3,'AB'), (4,NULL), (5,'AB'),
(6,NULL), (7,'ABCD'), (8,NULL), (9,'ABCD'), (10,NULL);
```

```
SELECT * FROM tbl ORDER BY b NULLS FIRST;
```

```
           a  b
=================================
           1  NULL
           2  NULL
           4  NULL
           6  NULL
           8  NULL
          10  NULL
           3  'ab'
           5  'ab'
           7  'abcd'
           9  'abcd'
```

```
SELECT * FROM tbl ORDER BY b NULLS LAST;
```

```
           a  b
=================================
           3  'ab'
           5  'ab'
           7  'abcd'
           9  'abcd'
           1  NULL
           2  NULL
           4  NULL
```

```
           6  NULL
           8  NULL
          10  NULL
```

Note

Translation of GROUP BY alias

```
CREATE TABLE t1(a INT, b INT, c INT);
INSERT INTO t1 VALUES(1,1,1);
INSERT INTO t1 VALUES(2,NULL,2);
INSERT INTO t1 VALUES(2,2,2);

SELECT a, NVL(b,2) AS b
FROM t1
GROUP BY a, b;  -- Q1
```

When you run the above SELECT query, “GROUP BY a, b” is translated as:

- “GROUP BY a, NVL(b, 2)”(alias name b) in 9.2 or before. The result is the same as Q2’s result as below.

```
SELECT a, NVL(b,2) AS bxxx
FROM t1
GROUP BY a, bxxx;  -- Q2
```

```
        a            b
======================
        1            1
        2            2
```

  - “GROUP BY a, b”(column name b) in 9.3 or higher. The result is the same as Q3’s result as below.

```
SELECT a, NVL(b,2) AS bxxx
FROM t1
GROUP BY a, b;  -- Q3
```

```
        a            b
======================
        1            1
```

```
          2             2
          2             2
```

LIMIT Clause

The **LIMIT** clause can be used to limit the number of records displayed. You can specify a very big integer for *row_count* to display to the last row, starting from a specific row. The **LIMIT** clause can be used as a prepared statement. In this case, the bind parameter (?) can be used instead of an argument.

INST_NUM () and **ROWNUM** cannot be included in the **WHERE** clause in a query that contains the **LIMIT** clause. Also, **LIMIT** cannot be used together with **HAVING GROUPBY_NUM** ().

```
LIMIT {[offset,] row_count | row_count [OFFSET offset]}

<offset> ::= <limit_expression>
<row_count> ::= <limit_expression>

<limit_expression> ::= <limit_term> | <limit_expression> +
<limit_term> | <limit_expression> - <limit_term>
<limit_term> ::= <limit_factor> | <limit_term> * <limit_factor> |
<limit_term> / <limit_factor>
<limit_factor> ::= <unsigned int> | <input_hostvar> | (
<limit_expression> )
```

- *offset*: Specifies the offset of the starting row to be displayed. The offset of the starting row of the result set is 0; it can be omitted and the default value is **0**. It can be one of unsigned int, a host variable or a simple expression.
- *row_count*: Specifies the number of records to be displayed. It can be one of unsigned integer, a host variable or a simple expression.

```
-- LIMIT clause can be used in prepared statement
PREPARE stmt FROM 'SELECT * FROM sales_tbl LIMIT ?, ?';
EXECUTE stmt USING 0, 10;
```

```
-- selecting rows with LIMIT clause
SELECT *
FROM sales_tbl
WHERE sales_amount > 100
LIMIT 5;
```

```
      dept_no  name                  sales_month  sales_amount
=============================================================
          201  'George'                        1           450
          201  'George'                        2           250
          201  'Laura'                         2           500
          301  'Max'                           1           300
          301  'Max'                           2           300
```

```
-- LIMIT clause can be used in subquery
SELECT t1.*
FROM (SELECT * FROM sales_tbl AS t2 WHERE sales_amount > 100 LIMIT
5) AS t1
LIMIT 1,3;

-- above query and below query shows the same result
SELECT t1.*
FROM (SELECT * FROM sales_tbl AS t2 WHERE sales_amount > 100 LIMIT
5) AS t1
LIMIT 3 OFFSET 1;
```

```
      dept_no  name                  sales_month  sales_amount
=============================================================
          201  'George'                        2           250
          201  'Laura'                         2           500
          301  'Max'                           1           300
```

```
-- LIMIT clause allows simple expressions for both offset and
row_count
SELECT *
FROM sales_tbl
WHERE sales_amount > 100
LIMIT ? * ?, (? * ?) + ?;
```

Join Query

A join is a query that combines the rows of two or more tables or virtual tables (views). In a join query, a condition that compares the columns that are common in two or more tables is called a join condition. Rows are retrieved from each joined table, and are combined only when they satisfy the specified join condition.

A join query using an equality operator (=) is called an equi-join, and one without any join condition is called a cartesian product. Meanwhile, joining a single table is called a self join. In a

self join, table **ALIAS** is used to distinguish columns, because the same table is used twice in the **FROM** clause.

A join that outputs only rows that satisfy the join condition from a joined table is called an inner or a simple join, whereas a join that outputs both rows that satisfy and do not satisfy the join condition from a joined table is called an outer join.

An outer join is divided into a left outer join which outputs all rows of the left table as a result(outputs NULL when the right table's columns don't match conditions), a right outer join which outputs all rows of the right table as a result(outputs NULL when the left table's columns don't match conditions) and a full outer join which outputs all rows of both tables. If there is no column value that corresponds to a table on one side in the result of an outer join query, all rows are returned as **NULL**.

```
FROM <table_specification> [{, <table_specification>
    | { <join_table_specification> |
<join_table_specification2> } ...]

<table_specification> ::=
    <single_table_spec> [<correlation>] |
    <metaclass_specification> [<correlation>] |
    <subquery> <correlation> |
    TABLE (<expression>) <correlation>

<join_table_specification> ::=
    {
        [INNER | {LEFT | RIGHT} [OUTER]] JOIN

     } <table_specification> ON <search_condition>

<join_table_specification2> ::=
            {
        CROSS JOIN |
        NATURAL [ LEFT | RIGHT ] JOIN
    } <table_specification>
```

- *<join_table_specification>*
  - [**INNER**] **JOIN**: Used for inner join and requires join conditions.
  - {**LEFT** | **RIGHT**} [**OUTER**] **JOIN**: **LEFT** is used for a left outer join query, and **RIGHT** is for a right outer join query.
- *<join_table_specification2>*
  - **CROSS JOIN**: Used for cross join and requires no join conditions.

- **NATURAL** [**LEFT** | **RIGHT**] **JOIN**: Used for natural join and join condition is not used. It operates in the equivalent same way to have a condition between columns equivalent of the same name.

Inner Join

The inner join requires join conditions. The **INNER JOIN** keyword can be omitted. When it is omitted, the table is separated by a comma (,). The **ON** join condition can be replaced with the **WHERE** condition.

The following example shows how to retrieve the years and host countries of the Olympic Games since 1950 where a world record has been set. The following query retrieves instances whose values of the *host_year* column in the *history* table are greater than 1950. The following two queries output the same result.

```
SELECT DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o ON h.host_year = o.host_year AND
o.host_year > 1950;

SELECT DISTINCT h.host_year, o.host_nation
FROM history h, olympic o
WHERE h.host_year = o.host_year AND o.host_year > 1950;
```

```
    host_year  host_nation
=====================================
         1968  'Mexico'
         1980  'U.S.S.R.'
         1984  'United States of America'
         1988  'Korea'
         1992  'Spain'
         1996  'United States of America'
         2000  'Australia'
         2004  'Greece'
```

Outer Join

CUBRID does not support full outer joins; it supports only left and right joins. Path expressions that include subqueries and sub-columns cannot be used in the join conditions of an outer join.

Join conditions of an outer join are specified in a different way from those of an inner join. In an inner join, join conditions can be expressed in the **WHERE** clause; in an outer join, they appear after the **ON** keyword within the **FROM** clause. Other retrieval conditions can be used in the **WHERE** or **ON** clause, but the retrieval result depends on whether the condition is used in the **WHERE** or **ON** clause.

The table execution order is fixed according to the order specified in the **FROM** clause. Therefore, when using an outer join, you should create a query statement in consideration of the table order. It is recommended to use standard statements using { **LEFT** | **RIGHT** } [**OUTER**] **JOIN**, because using an Oracle-style join query statements by specifying an outer join operator **(+)** in the **WHERE** clause, even if possible, might lead the execution result or plan in an unwanted direction.

The following example shows how to retrieve the years and host countries of the Olympic Games since 1950 where a world record has been set, but including the Olympic Games where any world records haven't been set in the result. This example can be expressed in the following right outer join query. In this example, all instances whose values of the *host_year* column in the *history* table are not greater than 1950 are also retrieved. All instances of *host_nation* are included because this is a right outer join. *host_year* that does not have a value is represented as **NULL**.

```
SELECT DISTINCT h.host_year, o.host_year, o.host_nation
FROM history h RIGHT OUTER JOIN olympic o ON h.host_year =
o.host_year
WHERE o.host_year > 1950;
```

```
    host_year    host_year  host_nation
=============================================
         NULL         1952  'Finland'
         NULL         1956  'Australia'
         NULL         1960  'Italy'
         NULL         1964  'Japan'
         NULL         1972  'Germany'
         NULL         1976  'Canada'
         1968         1968  'Mexico'
         1980         1980  'USSR'
         1984         1984  'USA'
         1988         1988  'Korea'
         1992         1992  'Spain'
         1996         1996  'USA'
         2000         2000  'Australia'
         2004         2004  'Greece'
```

A right outer join query can be converted to a left outer join query by switching the position of two tables in the **FROM** clause. The right outer join query in the previous example can be expressed as a left outer join query as follows:

```
SELECT DISTINCT h.host_year, o.host_year, o.host_nation
FROM olympic o LEFT OUTER JOIN history h ON h.host_year = o.host_year
WHERE o.host_year > 1950;
```

```
    host_year    host_year  host_nation
=============================================
```

```
          NULL          1952  'Finland'
          NULL          1956  'Australia'
          NULL          1960  'Italy'
          NULL          1964  'Japan'
          NULL          1972  'Germany'
          NULL          1976  'Canada'
          1968          1968  'Mexico'
          1980          1980  'USSR'
          1984          1984  'USA'
          1988          1988  'Korea'
          1992          1992  'Spain'
          1996          1996  'USA'
          2000          2000  'Australia'
          2004          2004  'Greece'
```

Outer joins can also be represented by using **(+)** in the **WHERE** clause. The above example is a query that has the same meaning as the example using the **LEFT OUTER JOIN**. The **(+)** syntax is not ISO/ANSI standard, so it can lead to ambiguous situations. It is recommended to use the standard syntax **LEFT OUTER JOIN** (or **RIGHT OUTER JOIN**) if possible.

```
SELECT DISTINCT h.host_year, o.host_year, o.host_nation
FROM history h, olympic o
WHERE o.host_year = h.host_year(+) AND o.host_year > 1950;
```

```
    host_year    host_year  host_nation
=============================================
          NULL          1952  'Finland'
          NULL          1956  'Australia'
          NULL          1960  'Italy'
          NULL          1964  'Japan'
          NULL          1972  'Germany'
          NULL          1976  'Canada'
          1968          1968  'Mexico'
          1980          1980  'USSR'
          1984          1984  'USA'
          1988          1988  'Korea'
          1992          1992  'Spain'
          1996          1996  'USA'
          2000          2000  'Australia'
          2004          2004  'Greece'
```

In the above examples, *h.host_year=o.host_year* is an outer join condition, and *o.host_year > 1950* is a search condition. If the search condition is not written in the **WHERE** clause but in the **ON** clause, the meaning and the result will be different. The following query also includes instances whose values of *o.host_year* are not greater than 1950.

```
SELECT DISTINCT h.host_year, o.host_year, o.host_nation
FROM olympic o LEFT OUTER JOIN history h ON h.host_year =
o.host_year AND o.host_year > 1950;
```

```
    host_year    host_year  host_nation
=============================================
         NULL         1896  'Greece'
         NULL         1900  'France'
         NULL         1904  'USA'
         NULL         1908  'United Kingdom'
         NULL         1912  'Sweden'
         NULL         1920  'Belgium'
         NULL         1924  'France'
         NULL         1928  'Netherlands'
         NULL         1932  'USA'
         NULL         1936  'Germany'
         NULL         1948  'England'
         NULL         1952  'Finland'
         NULL         1956  'Australia'
         NULL         1960  'Italy'
         NULL         1964  'Japan'
         NULL         1972  'Germany'
         NULL         1976  'Canada'
         1968         1968  'Mexico'
         1980         1980  'USSR'
         1984         1984  'USA'
         1988         1988  'Korea'
         1992         1992  'Spain'
         1996         1996  'USA'
         2000         2000  'Australia'
         2004         2004  'Greece'
```

In the above example, **LEFT OUTER JOIN** should attach all rows to the result rows even if the left table's rows do not match to the condition; therefore, the left table's condition, "AND o.host_year > 1950" is ignored. But "WHERE o.host_year > 1950" is applied after the join operation is completed. Please consider that a condition after **ON** clause and a condition after **WHERE** clause can be applied differently in **OUTER JOIN**.

Cross Join

The cross join is a cartesian product, meaning that it is a combination of two tables, without any condition. For the cross join, the **CROSS JOIN** keyword can be omitted. When it is omitted, the table is separated by a comma (,).

The following example shows how to write cross join.

```
SELECT DISTINCT h.host_year, o.host_nation
FROM history h CROSS JOIN olympic o;

SELECT DISTINCT h.host_year, o.host_nation
FROM history h, olympic o;
```

The above two queries output the same results.

```
    host_year  host_nation
=====================================
         1968  'Australia'
         1968  'Belgium'
         1968  'Canada'
         1968  'England'
         1968  'Finland'
         1968  'France'
         1968  'Germany'
...
         2004  'Spain'
         2004  'Sweden'
         2004  'USA'
         2004  'USSR'
         2004  'United Kingdom'

144 rows selected. (1.283548 sec) Committed.
```

Natural Join

When column names to be joined to each table are the same, that is, when you want to grant equivalent conditions between each column with the same name, a natural join, which can replace inner/outer join, can be used.

```
CREATE TABLE t1 (a int, b1 int);
CREATE TABLE t2 (a int, b2 int);

INSERT INTO t1 values(1,1);
INSERT INTO t1 values(3,3);
INSERT INTO t2 values(1,1);
INSERT INTO t2 values(2,2);
```

The below is an example of running **NATURAL JOIN**.

```
SELECT /** RECOMPILE*/ *
FROM t1 NATURAL JOIN t2;
```

Running the above query is the same as running the below query, and they display the same result.

```
SELECT /*+ RECOMPILE*/ *
FROM t1 INNER JOIN t2 ON t1.a=t2.a;
```

```
        a           b1           a           b2
=============================================
        1            1           1            1
```

The below is an example of running **NATURAL LEFT JOIN**.

```
SELECT /*+ RECOMPILE*/ *
FROM t1 NATURAL LEFT JOIN t2;
```

Running the above query is the same as running the below query, and they display the same result.

```
SELECT /*+ RECOMPILE*/ *
FROM t1 LEFT JOIN t2 ON t1.a=t2.a;
```

```
           a           b1           a           b2
=====================================================
           1            1           1            1
           3            3        NULL         NULL
```

The below is an example of running **NATURAL RIGHT JOIN**.

```
SELECT /*+ RECOMPILE*/ *
FROM t1 NATURAL RIGHT JOIN t2;
```

Running the above query is the same as running the below query, and they display the same result.

```
SELECT /*+ RECOMPILE*/ *
FROM t1 RIGHT JOIN t2 ON t1.a=t2.a;
```

```
           a           b1           a           b2
=====================================================
```

```
           1           1           1           1
        NULL        NULL           2           2
```

Subquery

A subquery can be used wherever expressions such as **SELECT** or **WHERE** clause can be used. If the subquery is represented as an expression, it must return a single column; otherwise it can return multiple rows. Subqueries can be divided into single-row subquery and multiple-row subquery depending on how they are used.

Single-Row Subquery

A single-row subquery outputs a row that has a single column. If no row is returned by the subquery, the subquery expression has a **NULL** value. If the subquery is supposed to return more than one row, an error occurs.

The following example shows how to retrieve the *history* table as well as the host country where a new world record has been set. This example shows a single-row subquery used as an expression. In this example, the subquery returns *host_nation* values for the rows whose values of the *host_year* column in the *olympic* table are the same as those of the *host_year* column in the *history* table. If there are no values that meet the condition, the result of the subquery is **NULL**.

```
SELECT h.host_year, (SELECT host_nation FROM olympic o WHERE
o.host_year=h.host_year) AS host_nation,
       h.event_code, h.score, h.unit
FROM history h;
```

```
    host_year  host_nation           event_code
score                 unit
=========================================================================
=======================
    2004       'Greece'              20283
'07:53.0'             'time'
    2004       'Greece'              20283
'07:53.0'             'time'
    2004       'Greece'              20281
'03:57.0'             'time'
    2004       'Greece'              20281
'03:57.0'             'time'
    2004       'Greece'              20281
'03:57.0'             'time'
    2004       'Greece'              20281
'03:57.0'             'time'
    2004       'Greece'              20326
```

```
'210'                 'kg'
    2000        'Australia'             20328
'225'                 'kg'
    2004        'Greece'                20331
'237.5'               'kg'
...
```

Multiple-Row Subquery

The multiple-row subquery returns one or more rows that contain the specified column. The result of the multiple-row subquery can create **SET**, **MULTISET** and **LIST**) by using an appropriate keyword.

The following example shows how to retrieve nations, capitals and host cities for Olympic Game all together in the *nation* table. In this example, the subquery result is used to create a **List** from the values of the *host_city* column in the *olympic* table. This query returns *name* and *capital* value for *nation* table, as well as a set that contains *host_city* values of the *olympic* table with *host_nation* value. If the *name* value is an empty set in the query result, it is excluded. If there is no *olympic* table that has the same value as the *name*, an empty set is returned.

```
SELECT name, capital, list(SELECT host_city FROM olympic WHERE
host_nation = name) AS host_cities
FROM nation;
```

```
  name                      capital                 host_cities
=========================================================================
  'Somalia'                   'Mogadishu'           {}
  'Sri Lanka'                 'Sri Jayewardenepura Kotte' {}
  'Sao Tome & Principe'       'Sao Tome'            {}
  ...
  'U.S.S.R.'                  'Moscow'              {'Moscow'}
  'Uruguay'                   'Montevideo'          {}
  'United States of America'  'Washington.D.C'      {'Atlanta ',
'St. Louis', 'Los Angeles', 'Los Angeles'}
  'Uzbekistan'                'Tashkent'            {}
  'Vanuatu'                   'Port Vila'           {}
```

Such multiple-row subquery expressions can be used anywhere a collection-type value expression is allowed. However, they cannot be used where a collection-type constant value is required as in the **DEFAULT** specification in the class attribute definition.

If the **ORDER BY** clause is not used explicitly in the subquery, the order of the multiple-row query result is not set. Therefore, the order of the multiple-row subquery result that creates **LIST** must be specified by using the **ORDER BY** clause.

VALUES

The **VALUES** clause prints out the values of rows defined in the expression. In most cases, the **VALUES** clause is used for creating a constant table, however, the clause itself can be used. When one or more rows are specified in the **VALUES** clause, all rows should have the same number of the elements.

```
VALUES (expression[, ...])[, ...]
```

- *expression* : An expression enclosed within parentheses stands for one row in a table.

The **VALUES** clause can be used to express the **UNION ALL** query, which consists of constant values in a simpler way. For example, the following query can be executed.

```
VALUES (1 AS col1, 'first' AS col2), (2, 'second'), (3, 'third'),
(4, 'fourth');
```

The above query prints out the following result.

```
SELECT 1 AS col1, 'first' AS col2
UNION ALL
SELECT 2, 'second'
UNION ALL
SELECT 3, 'third'
UNION ALL
SELECT 4, 'fourth';
```

The following example shows use of the **VALUES** clause with multiple rows in the **INSERT** statement.

```
INSERT INTO athlete (code, name, gender, nation_code, event)
VALUES ('21111', 'Jang Mi-Ran ', 'F', 'KOR', 'Weight-lifting'),
       ('21112', 'Son Yeon-Jae ', 'F', 'KOR', 'Rhythmic gymnastics');
```

The following example shows how to use subquery in the **FROM** statement.

```
SELECT a.*
FROM athlete a, (VALUES ('Jang Mi-Ran', 'F'), ('Son Yeon-Jae', 'F'))
AS t(name, gender)
WHERE a.name=t.name AND a.gender=t.gender;
```

```
         code  name                gender   nation_code        event
==========================================================================
================================
        21111  'Jang Mi-Ran'       'F'      'KOR'
```

```
'Weight-lifting'
        21112  'Son Yeon-Jae'       'F'       'KOR'
'Rhythmic gymnastics'
```

FOR UPDATE

The **FOR UPDATE** clause can be used in **SELECT** statements for locking rows returned by the statement for a later **UPDATE/DELETE**.

```
SELECT ... [FOR UPDATE [OF <spec_name_comma_list>]]

    <spec_name_comma_list> ::= <spec_name> [, <spec_name>, ... ]
        <spec_name> ::= [schema_name.]table_name |
[schema_name.]view_name
```

- <*spec_name_comma_list*>: A list of table/view names referenced from the **FROM** clause.

Only table/view referenced in <*spec_name_comma_list*> will be locked. If the <*spec_name_comma_list*> is missing but **FOR UPDATE** is present then we assume that all tables/views from the **FROM** clause of the **SELECT** statement are referenced. Rows are locked using **X_LOCK**.

Note

Restrictions

- It cannot be used in subqueries (but it can reference subqueries).
- It cannot be used in a statement that has **GROUP BY**, **DISTINCT** or aggregate functions.
- It cannot reference **UNION**s.

The following shows how to use **SELECT ... FOR UPDATE** statements.

```
CREATE TABLE t1(i INT);
INSERT INTO t1 VALUES (1), (2), (3), (4), (5);

CREATE TABLE t2(i INT);
INSERT INTO t2 VALUES (1), (2), (3), (4), (5);
CREATE INDEX idx_t2_i ON t2(i);

CREATE VIEW v12 AS SELECT t1.i AS i1, t2.i AS i2 FROM t1 INNER JOIN
t2 ON t1.i=t2.i;
```

```
SELECT * FROM t1 ORDER BY 1 FOR UPDATE;
SELECT * FROM t1 ORDER BY 1 FOR UPDATE OF t1;
SELECT * FROM t1 INNER JOIN t2 ON t1.i=t2.i ORDER BY 1 FOR UPDATE OF
t1, t2;

SELECT * FROM t1 INNER JOIN (SELECT * FROM t2 WHERE t2.i > 0) r ON
t1.i=r.i WHERE t1.i > 0 ORDER BY 1 FOR UPDATE;

SELECT * FROM v12 ORDER BY 1 FOR UPDATE;
SELECT * FROM t1, (SELECT * FROM v12, t2 WHERE t2.i > 0 AND
t2.i=v12.i1) r WHERE t1.i > 0 AND t1.i=r.i ORDER BY 1 FOR UPDATE OF
r;
```

Hierarchical Query

Hierarchical Query is used to obtain a set of data organized in a hierarchy. The **START WITH ... CONNECT BY** clause is used in combination with the **SELECT** clause in the following form.

You can execute the queries by changing the order of two clauses like **CONNECT BY ... START WITH**.

```
SELECT column_list
FROM table_joins | tables
[WHERE join_conditions and/or filtering_conditions]
[hierarchical_clause]

hierarchical_clause :
    [START WITH condition] CONNECT BY [NOCYCLE] condition
    | CONNECT BY [NOCYCLE] condition [START WITH condition]
```

START WITH Clause

The **START WITH** clause will filter the rows from which the hierarchy will start. The rows that satisfy the **START WITH** condition will be the root nodes of the hierarchy. If **START WITH** is omitted, then all the rows will be considered as root nodes.

Note

If **START WITH** clause is omitted or the rows that satisfy the **START WITH** condition does not exist, all of rows in the table are considered as root nodes; which means that hierarchy relationship of sub rows which belong each root is searched. Therefore, some of results can be duplicate.

CONNECT BY Clause

- **PRIOR** : The **CONNECT BY** condition is tested for a pair of rows. If it evaluates to true, the two rows satisfy the parent-child relationship of the hierarchy. We need to specify the columns that are used from the parent row and the columns that are used from the child row. We can use the **PRIOR** operator when applied to a column, which will refer to the value of the parent row for that column. If **PRIOR** is not used for a column, the value in the child row is used.

- **NOCYCLE** : In some cases, the resulting rows of the table joins may contain cycles, depending on the **CONNECT BY** condition. Because cycles cause an infinite loop in the result tree construction, CUBRID detects them and either returns an error doesn't expand the branches beyond the point where a cycle is found (if the **NOCYCLE** keyword is specified). This keyword may be specified after the **CONNECT BY** keywords. It makes CUBRID run a statement even if the processed data contains cycles.

  If a **CONNECT BY** statement causes a cycle at runtime and the **NOCYCLE** keyword is not specified, CUBRID will return an error and the statement will be canceled. When specifying the **NOCYCLE** keyword, if CUBRID detects a cycle while processing a hierarchy node, it will set the **CONNECT_BY_ISCYCLE** attribute for that node to the value of 1 and it will stop further expansion of that branch.

The following example shows how to execute hierarchical query.

```
-- Creating tree table and then inserting data
CREATE TABLE tree(ID INT, MgrID INT, Name VARCHAR(32), BirthYear
INT);

INSERT INTO tree VALUES (1,NULL,'Kim', 1963);
INSERT INTO tree VALUES (2,NULL,'Moy', 1958);
INSERT INTO tree VALUES (3,1,'Jonas', 1976);
INSERT INTO tree VALUES (4,1,'Smith', 1974);
INSERT INTO tree VALUES (5,2,'Verma', 1973);
INSERT INTO tree VALUES (6,2,'Foster', 1972);
INSERT INTO tree VALUES (7,6,'Brown', 1981);

-- Executing a hierarchical query with CONNECT BY clause
SELECT id, mgrid, name
FROM tree
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
   id        mgrid  name
=======================================
    1         NULL  'Kim'
    2         NULL  'Moy'
    3            1  'Jonas'
    3            1  'Jonas'
```

```
     4            1  'Smith'
     4            1  'Smith'
     5            2  'Verma'
     5            2  'Verma'
     6            2  'Foster'
     6            2  'Foster'
     7            6  'Brown'
     7            6  'Brown'
     7            6  'Brown'
```

```
-- Executing a hierarchical query with START WITH clause
SELECT id, mgrid, name
FROM tree
START WITH mgrid IS NULL
CONNECT BY prior id=mgrid
ORDER BY id;
```

```
   id        mgrid  name
=======================================
     1         NULL  'Kim'
     2         NULL  'Moy'
     3            1  'Jonas'
     4            1  'Smith'
     5            2  'Verma'
     6            2  'Foster'
     7            6  'Brown'
```

Hierarchical Query Execution

Hierarchical Query for Table Join

When target table is joined in **SELECT** statement, **WHERE** clause can include not only searching conditions but also joining conditions. At this time, CUBRID applies the joining conditions in **WHERE** clause at first, then conditions in **CONNECT BY** clause; at last, the left searching conditions.

When specifying joining conditions and searching conditions together in **WHERE** clause, joining conditions can be applied as searching conditions even if there was no intention; so the operating order can be different; therefore, we recommend that you specify the table joining conditions in **FROM** clause, not in **WHERE** conditions.

Query Results

The resulting rows of the table joins are filtered according to the **START WITH** condition to obtain the root nodes for the hierarchy. If no **START WITH** condition is specified, then all the rows

resulting from the table joins will be considered as root nodes. After the root nodes are obtained, CUBRID will select the child rows for the root nodes. These are all nodes from the table joins that respect the **CONNECT BY** condition. This step will be repeated for the child nodes to determine their child nodes and so on until no more child nodes can be added.

In addition, CUBRID evaluates the **CONNECT BY** clause first and all the rows of the resulting hierarchy tress by using the filtering condition in the **WHERE** clause.

The example illustrates how joins can be used in **CONNECT BY** queries.

```
-- Creating tree2 table and then inserting data
CREATE TABLE tree2(id int, treeid int, job varchar(32));

INSERT INTO tree2 VALUES(1,1,'Partner');
INSERT INTO tree2 VALUES(2,2,'Partner');
INSERT INTO tree2 VALUES(3,3,'Developer');
INSERT INTO tree2 VALUES(4,4,'Developer');
INSERT INTO tree2 VALUES(5,5,'Sales Exec.');
INSERT INTO tree2 VALUES(6,6,'Sales Exec.');
INSERT INTO tree2 VALUES(7,7,'Assistant');
INSERT INTO tree2 VALUES(8,null,'Secretary');

-- Executing a hierarchical query onto table joins
SELECT t.id,t.name,t2.job,level
FROM tree t INNER JOIN tree2 t2 ON t.id=t2.treeid
START WITH t.mgrid is null
CONNECT BY prior t.id=t.mgrid
ORDER BY t.id;
```

```
   id  name                  job                           level
=======================================================================
    1  'Kim'                 'Partner'                         1
    2  'Moy'                 'Partner'                         1
    3  'Jonas'               'Developer'                       2
    4  'Smith'               'Developer'                       2
    5  'Verma'               'Sales Exec.'                     2
    6  'Foster'              'Sales Exec.'                     2
    7  'Brown'               'Assistant'                       3
```

Ordering Data with the Hierarchical Query

The **ORDER SIBLINGS BY** clause will cause the ordering of the rows while preserving the hierarchy ordering so that the child nodes with the same parent will be stored according to the column list.

```
ORDER SIBLINGS BY col_1 [ASC|DESC] [, col_2 [ASC|DESC] [...[, col_n
[ASC|DESC]]...]]
```

The following example shows how to display information about seniors and subordinates in a company in the order of birth year.

The result with hierarchical query shows parent and child nodes in a row according to the column list specified in **ORDER SIBLINGS BY** statement by default. Sibling nodes that share the same parent node have outputted in a specified order.

```
-- Outputting a parent node and its child nodes, which sibling nodes
that share the same parent are sorted in the order of birthyear.
SELECT id, mgrid, name, birthyear, level
FROM tree
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER SIBLINGS BY birthyear;
```

```
   id        mgrid  name                    birthyear        level
=========================================================================
    2         NULL  'Moy'                        1958            1
    6            2  'Foster'                     1972            2
    7            6  'Brown'                      1981            3
    5            2  'Verma'                      1973            2
    1         NULL  'Kim'                        1963            1
    4            1  'Smith'                      1974            2
    3            1  'Jonas'                      1976            2
```

The following example shows how to display information about seniors and subordinates in a company in the order of joining. For the same level, the employee ID numbers are assigned in the order of joining. *id* indicates employee ID numbers (parent and child nodes) and *mgrid* indicates the employee ID numbers of their seniors.

```
-- Outputting a parent node and its child nodes, which sibling nodes
that share the same parent are sorted in the order of id.
SELECT id, mgrid, name, LEVEL
FROM tree
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER SIBLINGS BY id;
```

```
   id        mgrid  name                  level
=================================================
    1         NULL  'Kim'                     1
    3            1  'Jonas'                   2
    4            1  'Smith'                   2
    2         NULL  'Moy'                     1
    5            2  'Verma'                   2
```

```
    6            2  'Foster'                        2
    7            6  'Brown'                         3
```

Pseudo Columns for Hierarchical Query

LEVEL

LEVEL is a pseudocolumn representing depth of hierarchical queries. The **LEVEL** of root node is 1 and the LEVEL of its child node is 2.

The **LEVEL** (pseudocolumn) can be used in the **WHERE** clause, **ORDER BY** clause, and **GROUP BY ... HAVING** clause of the **SELECT** statement. And it can also be used in the statement using aggregate functions.

The following example shows how to retrieve the **LEVEL** value to check level of node.

```
-- Checking the LEVEL value
SELECT id, mgrid, name, LEVEL
FROM tree
WHERE LEVEL=2
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
   id        mgrid  name                          level
=======================================================
    3            1  'Jonas'                         2
    4            1  'Smith'                         2
    5            2  'Verma'                         2
    6            2  'Foster'                        2
```

The following example shows how to add **LEVEL** conditions after the **CONNECT BY** statement.

```
SELECT LEVEL FROM db_root CONNECT BY LEVEL <= 10;
```

```
        level
=============
            1
            2
            3
            4
            5
            6
```

```
             7
             8
             9
            10
```

Note that the format of “CONNECT BY expr(LEVEL) < expr”, for example “CONNECT BY LEVEL +1 < 5”) is not supported.

CONNECT_BY_ISLEAF

CONNECT_BY_ISLEAF is a pseudocolumn representing that the result of hierarchical query is leaf node. If the current row is a leaf node, it returns 1; otherwise, it returns 0.

The following example shows how to retrieve the **CONNECT_BY_ISLEAF** value to check whether it is a leaf node or not.

```
-- Checking a CONNECT_BY_ISLEAF value
SELECT id, mgrid, name, CONNECT_BY_ISLEAF
FROM tree
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
  id        mgrid  name                  connect_by_isleaf
==========================================================
   1         NULL  'Kim'                                 0
   2         NULL  'Moy'                                 0
   3            1  'Jonas'                               1
   4            1  'Smith'                               1
   5            2  'Verma'                               1
   6            2  'Foster'                              0
   7            6  'Brown'                               1
```

CONNECT_BY_ISCYCLE

CONNECT_BY_ISCYCLE is a pseudocolumn representing that a cycle was detected while processing the node, meaning that a child was also found to be an ancestor. A value of 1 for a row means a cycle was detected; the pseudo-column’s value is 0, otherwise.

The **CONNECT_BY_ISCYCLE** pseudo-column can be used in the **WHERE**, **ORDER BY** and **GROUP BY** ... **HAVING** clauses of the **SELECT** statement. It can also be used in aggregate functions.

Note

This pseudocolumn is available only when the **NOCYCLE** keyword is used in the statement.

The following example shows how to retrieve the **CONNECT_BY_ISCYCE** value to check a row that occurs loop.

```
-- Creating a tree_cycle table and inserting data
CREATE TABLE tree_cycle(ID INT, MgrID INT, Name VARCHAR(32));

INSERT INTO tree_cycle VALUES (1,NULL,'Kim');
INSERT INTO tree_cycle VALUES (2,11,'Moy');
INSERT INTO tree_cycle VALUES (3,1,'Jonas');
INSERT INTO tree_cycle VALUES (4,1,'Smith');
INSERT INTO tree_cycle VALUES (5,3,'Verma');
INSERT INTO tree_cycle VALUES (6,3,'Foster');
INSERT INTO tree_cycle VALUES (7,4,'Brown');
INSERT INTO tree_cycle VALUES (8,4,'Lin');
INSERT INTO tree_cycle VALUES (9,2,'Edwin');
INSERT INTO tree_cycle VALUES (10,9,'Audrey');
INSERT INTO tree_cycle VALUES (11,10,'Stone');

-- Checking a CONNECT_BY_ISCYCLE value
SELECT id, mgrid, name, CONNECT_BY_ISCYCLE
FROM tree_cycle
START WITH name in ('Kim', 'Moy')
CONNECT BY NOCYCLE PRIOR id=mgrid
ORDER BY id;
```

```
id        mgrid  name        connect_by_iscycle
================================================
 1         NULL  'Kim'                       0
 2           11  'Moy'                       0
 3            1  'Jonas'                     0
 4            1  'Smith'                     0
 5            3  'Verma'                     0
 6            3  'Foster'                    0
 7            4  'Brown'                     0
 8            4  'Lin'                       0
 9            2  'Edwin'                     0
10            9  'Audrey'                    0
11           10  'Stone'                     1
```

Operators for Hierarchical Query

CONNECT_BY_ROOT

The **CONNECTION_BY_ROOT** operator returns the value of a root row as a column value. This operator can be used in the **WHERE** and **ORDER BY** clauses of the **SELECT** statement.

The following example shows how to retrieve the root row's *id* value.

```
-- Checking the id value of a root row for each row
SELECT id, mgrid, name, CONNECT_BY_ROOT id
FROM tree
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
   id        mgrid  name                    connect_by_root id
=============================================================
    1         NULL  'Kim'                                  1
    2         NULL  'Moy'                                  2
    3            1  'Jonas'                                1
    4            1  'Smith'                                1
    5            2  'Verma'                                2
    6            2  'Foster'                               2
    7            6  'Brown'                                2
```

PRIOR

The PRIOR operator returns the value of a parent row as a column value and returns NULL for the root row. This operator can be used in the **WHERE**, **ORDER BY** and **CONNECT BY** clauses of the **SELECT** statement.

The following example shows how to retrieve the parent row's *id* value.

```
-- Checking the id value of a parent row for each row
SELECT id, mgrid, name, PRIOR id as "prior_id"
FROM tree
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
   id        mgrid  name                   prior_id
=================================================
```

```
    1          NULL  'Kim'                          NULL
    2          NULL  'Moy'                          NULL
    3             1  'Jonas'                           1
    4             1  'Smith'                           1
    5             2  'Verma'                           2
    6             2  'Foster'                          2
    7             6  'Brown'                           6
```

Functions for Hierarchical Query

SYS_CONNECT_BY_PATH

The **SYS_CONNECT_BY_PATH** function returns the hierarchical path from a root to the specified row in string. The column and separator specified as an argument must be a character type. Each path separated by specified separator will be displayed consecutively. This function can be used in the **WHERE** and **ORDER BY** clauses of the **SELECT** statement.

```
SYS_CONNECT_BY_PATH (column_name, separator_char)
```

The following example shows how to retrieve path from a root to the specified row.

```
-- Executing a hierarchical query with SYS_CONNECT_BY_PATH function
SELECT id, mgrid, name, SYS_CONNECT_BY_PATH(name,'/') as [hierarchy]
FROM tree
START WITH mgrid IS NULL
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
   id         mgrid  name                    hierarchy
=============================================================
    1          NULL  'Kim'                   '/Kim'
    2          NULL  'Moy'                   '/Moy'
    3             1  'Jonas'                 '/Kim/Jonas'
    4             1  'Smith'                 '/Kim/Smith'
    5             2  'Verma'                 '/Moy/Verma'
    6             2  'Foster'                '/Moy/Foster'
    7             6  'Brown'                 '/Moy/Foster/Brown'
```

Examples of Hierarchical Query

The examples in this page shows how to write hierarchical queries by specifying the **CONNECT BY** clause within the **SELECT** statement.

A table that have relationship with recursive reference is create and the table consists of two columns named *ID* and *ParentID*; assume that *ID* is a primary key for the table and *ParentID* is a foreign key for the same table. In this context, the root node will have a *ParentID* value of **NULL**.

Once a table is create, you can get the entire data with hierarchical structure and a value of **LEVEL** by using the **UNION ALL** as shown below.

```
CREATE TABLE tree_table (ID int PRIMARY KEY, ParentID int, name
VARCHAR(128));

INSERT INTO tree_table VALUES (1,NULL,'Kim');
INSERT INTO tree_table VALUES (2,1,'Moy');
INSERT INTO tree_table VALUES (3,1,'Jonas');
INSERT INTO tree_table VALUES (4,1,'Smith');
INSERT INTO tree_table VALUES (5,3,'Verma');
INSERT INTO tree_table VALUES (6,3,'Foster');
INSERT INTO tree_table VALUES (7,4,'Brown');
INSERT INTO tree_table VALUES (8,4,'Lin');
INSERT INTO tree_table VALUES (9,2,'Edwin');
INSERT INTO tree_table VALUES (10,9,'Audrey');
INSERT INTO tree_table VALUES (11,10,'Stone');

SELECT L1.ID, L1.ParentID, L1.name, 1 AS [Level]
    FROM tree_table AS L1
    WHERE L1.ParentID IS NULL
UNION ALL
SELECT L2.ID, L2.ParentID, L2.name, 2 AS [Level]
    FROM tree_table AS L1
        INNER JOIN tree_table AS L2 ON L1.ID=L2.ParentID
    WHERE L1.ParentID IS NULL
UNION ALL
SELECT L3.ID, L3.ParentID, L3.name, 3 AS [Level]
    FROM tree_table AS L1
        INNER JOIN tree_table AS L2 ON L1.ID=L2.ParentID
        INNER JOIN tree_table AS L3 ON L2.ID=L3.ParentID
    WHERE L1.ParentID IS NULL
UNION ALL
SELECT L4.ID, L4.ParentID, L4.name, 4 AS [Level]
    FROM tree_table AS L1
        INNER JOIN tree_table AS L2 ON L1.ID=L2.ParentID
        INNER JOIN tree_table AS L3 ON L2.ID=L3.ParentID
        INNER JOIN tree_table AS L4 ON L3.ID=L4.ParentID
    WHERE L1.ParentID IS NULL;
```

```
   ID     ParentID  name                          Level
=======================================================
    1         NULL  'Kim'                             1
    2            1  'Moy'                             2
    3            1  'Jonas'                           2
```

```
     4            1  'Smith'                        2
     9            2  'Edwin'                        3
     5            3  'Verma'                        3
     6            3  'Foster'                       3
     7            4  'Brown'                        3
     8            4  'Lin'                          3
    10            9  'Audrey'                       4
```

Because you do not know how many levels exist in the data, you can rewrite the query above as a stored procedure that loops until no new row is retrieved.

However, the hierarchical structure should be checked every step while looping, specify the **CONNECT BY** clause within the **SELECT** statement as follows; the example below shows how to get the entire data with hierarchical structure and the level of each row in the hierarchy.

```
SELECT ID, ParentID, name, Level
FROM tree_table
START WITH ParentID IS NULL
CONNECT BY ParentID=PRIOR ID;
```

```
   ID     ParentID  name                          level
=======================================================
    1         NULL  'Kim'                           1
    2            1  'Moy'                           2
    9            2  'Edwin'                         3
   10            9  'Audrey'                        4
   11           10  'Stone'                         5
    3            1  'Jonas'                         2
    5            3  'Verma'                         3
    6            3  'Foster'                        3
    4            1  'Smith'                         2
    7            4  'Brown'                         3
    8            4  'Lin'                           3
```

You can specify **NOCYCLE** to prevent an error from occurring as follows. When you run the following query, a loop does not appear; therefore, the result is the same as the above.

```
SELECT ID, ParentID, name, Level
FROM tree_table
START WITH ParentID IS NULL
CONNECT BY NOCYCLE ParentID=PRIOR ID;
```

CUBRID judges that this hierarchical query has a loop if the same row is found during searching process on the query. The below is an example that the loop exists; we define **NOCYCLE** to end the additional searching process if a loop exists.

```
CREATE TABLE tbl(seq INT, id VARCHAR(10), parent VARCHAR(10));

INSERT INTO tbl VALUES (1, 'a', null);
INSERT INTO tbl VALUES (2, 'b', 'a');
INSERT INTO tbl VALUES (3, 'b', 'c');
INSERT INTO tbl VALUES (4, 'c', 'b');
INSERT INTO tbl VALUES (5, 'c', 'b');

SELECT seq, id, parent, LEVEL,
  CONNECT_BY_ISCYCLE AS iscycle,
  CAST(SYS_CONNECT_BY_PATH(id,'/') AS VARCHAR(10)) AS idpath
FROM tbl
START WITH PARENT is NULL
CONNECT BY NOCYCLE PARENT = PRIOR id;
```

```
    seq  id           parent       level      iscycle  idpath
=========================================================================
========
      1  'a'          NULL             1            0  '/a'
      2  'b'          'a'              2            0  '/a/b'
      4  'c'          'b'              3            0  '/a/b/c'
      3  'b'          'c'              4            1  '/a/b/c/b'
      5  'c'          'b'              5            1  '/a/b/c/b/c'
      5  'c'          'b'              3            0  '/a/b/c'
      3  'b'          'c'              4            1  '/a/b/c/b'
      4  'c'          'b'              5            1  '/a/b/c/b/c'
```

The below shows to output dates of March, 2013(201303) with a hierarchical query.

```
SELECT TO_CHAR(base_month + lvl -1, 'YYYYMMDD') h_date
FROM (
    SELECT LEVEL lvl, base_month
    FROM (
            SELECT TO_DATE('201303', 'YYYYMM') base_month FROM
db_root
    )
    CONNECT BY LEVEL <= LAST_DAY(base_month) - base_month + 1
);
```

```
h_date
======================
  '20130301'
  '20130302'
  '20130303'
  '20130304'
  '20130305'
  '20130306'
```

```
  '20130307'
  '20130308'
  '20130309'
  '20130310'
  '20130311'
  '20130312'
  '20130313'
  '20130314'
  '20130315'
  '20130316'
  '20130317'
  '20130318'
  '20130319'
  '20130320'
  '20130321'
  '20130322'
  '20130323'
  '20130324'
  '20130325'
  '20130326'
  '20130327'
  '20130328'
  '20130329'
  '20130330'
  '20130331'

31 rows selected. (0.066175 sec) Committed.
```

Performance of Hierarchical Query

Hierarchical query with single-table

A single-table hierarchical query can improve performance by adding an index to the columns corresponding to the child in the **CONNECT BY** clause. The following example shows how to improve performance by adding an index to the *mgrid* column.

```
 -- Creating tree table and then inserting data
CREATE TABLE tree(ID INT, MgrID INT, Name VARCHAR(32), BirthYear
INT);

INSERT INTO tree VALUES (1,NULL,'Kim', 1963);
INSERT INTO tree VALUES (2,NULL,'Moy', 1958);
INSERT INTO tree VALUES (3,1,'Jonas', 1976);
INSERT INTO tree VALUES (4,1,'Smith', 1974);
INSERT INTO tree VALUES (5,2,'Verma', 1973);
INSERT INTO tree VALUES (6,2,'Foster', 1972);
INSERT INTO tree VALUES (7,6,'Brown', 1981);
```

```
-- Copy data
INSERT INTO tree select * from tree;
INSERT INTO tree select * from tree;
INSERT INTO tree select * from tree;
INSERT INTO tree select * from tree;
INSERT INTO tree select * from tree;

-- Executing a single table hierarchical query
SELECT count(*)
FROM tree
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
Trace Statistics:
  SELECT (time: 1686, fetch: 155971, fetch_time: 11, ioread: 0)
    SCAN (table: dba.tree), (heap time: 1, fetch: 1, ioread: 0,
readrows: 224, rows: 224)
    CONNECTBY (time: 1685, fetch: 155970, fetch_time: 0, ioread: 0)
      SCAN (table: dba.tree), (heap time: 1607, fetch: 38112,
ioread: 0, readrows: 8537088, rows: 37888)
      ORDERBY (time: 11, sort: true, page: 105, ioread: 0)
```

```
-- Create index
CREATE INDEX idx ON tree(mgrid);

-- Executing a single table hierarchical query with index
SELECT count(*)
FROM tree
CONNECT BY PRIOR id=mgrid
ORDER BY id;
```

```
Trace Statistics:
  SELECT (time: 128, fetch: 195206, fetch_time: 9, ioread: 0)
    SCAN (table: dba.tree), (heap time: 0, fetch: 1, ioread: 0,
readrows: 224, rows: 224)
      CONNECTBY (time: 128, fetch: 195203, fetch_time: 0, ioread: 0)
        SCAN (index: dba.tree.idx), (btree time: 35, fetch: 77344,
ioread: 0, readkeys: 1120, filteredkeys: 0, rows: 37888) (lookup
time: 10, rows: 37888)
        ORDERBY (time: 11, sort: true, page: 105, ioread: 0)
```

Hierarchical query with join-table

A hierarchical query for table join is optimized using a hash list scan when applying the condition in the **CONNECT BY** clause after performing the table join operation. The following example shows the difference in performance between a hash list scan and a list scan.

```
-- Creating tree2 table and then inserting data
CREATE TABLE tree2(id int, treeid int, job varchar(32));

INSERT INTO tree2 VALUES(1,1,'Partner');
INSERT INTO tree2 VALUES(2,2,'Partner');
INSERT INTO tree2 VALUES(3,3,'Developer');
INSERT INTO tree2 VALUES(4,4,'Developer');
INSERT INTO tree2 VALUES(5,5,'Sales Exec.');
INSERT INTO tree2 VALUES(6,6,'Sales Exec.');
INSERT INTO tree2 VALUES(7,7,'Assistant');
INSERT INTO tree2 VALUES(8,null,'Secretary');

-- Copy data
INSERT INTO tree2 select * from tree2;

-- Executing a hierarchical query onto table joins
SELECT count(*)
FROM tree t INNER JOIN tree2 t2 ON t.id=t2.treeid
START WITH t.mgrid is null
CONNECT BY prior t.id=t.mgrid
ORDER BY t.id;
```

```
Trace Statistics:
  SELECT (time: 663, fetch: 877290, fetch_time: 54, ioread: 0)
    SCAN (table: dba.tree), (heap time: 0, fetch: 225, ioread: 0,
readrows: 224, rows: 224)
      SCAN (table: dba.tree2), (heap time: 0, fetch: 224, ioread: 0,
readrows: 3584, rows: 448)
    CONNECTBY (time: 662, fetch: 876841, fetch_time: 0, ioread: 0)
      SCAN (hash temp(m), build time: 0, time: 53, fetch: 0, ioread:
0, readrows: 278976, rows: 278528)
      ORDERBY (time: 92, sort: true, page: 1560, ioread: 0)
```

```
-- Executing a hierarchical query onto table joins without hash list
scan
SELECT /*+ NO_HASH_LIST_SCAN */ count(*)
FROM tree t INNER JOIN tree2 t2 ON t.id=t2.treeid
START WITH t.mgrid is null
CONNECT BY prior t.id=t.mgrid
ORDER BY t.id;
```

```
Trace Statistics:
  SELECT (time: 8977, fetch: 877335, fetch_time: 57, ioread: 0)
    SCAN (table: dba.tree), (heap time: 0, fetch: 225, ioread: 0,
readrows: 224, rows: 224)
      SCAN (table: dba.tree2), (heap time: 0, fetch: 224, ioread: 0,
readrows: 3584, rows: 448)
```

```
    CONNECTBY (time: 8977, fetch: 876886, fetch_time: 0, ioread: 0)
      SCAN (temp time: 8353, fetch: 0, ioread: 0, readrows:
124837888, rows: 278528)
      ORDERBY (time: 91, sort: true, page: 1560, ioread: 0)
```

Note

HASH LIST SCAN optimization for a hierarchical query with join table is supported starting from CUBRID 11.0.

INSERT

You can insert a new record into a table in a database by using the **INSERT** statement. CUBRID supports **INSERT ... VALUES**, **INSERT ... SET** and **INSERT ... SELECT** statements.

INSERT ... VALUES and **INSERT ... SET** statements are used to insert a new record based on the value that is explicitly specified while the **INSERT ... SELECT** statement is used to insert query result records obtained from different tables. Use the **INSERT VALUES** or **INSERT ... SELECT** statement to insert multiple rows by using the single **INSERT** statement.

```
<INSERT ... VALUES statement>
INSERT [INTO] <table_specification> [(column_name, ...)]
    {VALUES | VALUE}({expr | DEFAULT}, ...)[,({expr |
DEFAULT}, ...),...]
    [ON DUPLICATE KEY UPDATE column_name = expr, ... ]
INSERT [INTO] <table-specification> DEFAULT [ VALUES ]

<INSERT ... SET statement>
INSERT [INTO] <table_specification>
    SET column_name = {expr | DEFAULT}[, column_name = {expr |
DEFAULT},...]
    [ON DUPLICATE KEY UPDATE column_name = expr, ... ]

<INSERT ... SELECT statement>
INSERT [INTO] <table_specification> [(column_name, ...)]
    SELECT...
    [ON DUPLICATE KEY UPDATE column_name = expr, ... ]

<table_specification> ::= [schema_name.]table_name |
<remote_table_spec>
```

```
<remote_table_spec> ::=
[schema_name.]table_name@[schema_name.]server_name
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*: Specifies the name of the target table into which you want to insert a new record.
- *server_name*: Specifies the name of the remote server to connect to with dblink.
- *column_name*: Specifies the name of the column into which you want to insert the value. If you omit to specify the column name, it is considered that all columns defined in the table have been specified. Therefore, you must specify the values for all columns next to the **VALUES** keyword. If you do not specify all the columns defined in the table, a **DEFAULT** value is assigned to the non-specified columns; if the **DEFAULT** value is not defined, a **NULL** value is assigned.
- *expr* | **DEFAULT**: Specifies values that correspond to the columns next to the **VALUES** keyword. Expressions or the **DEFAULT** keyword can be specified as a value. At this time, the order and number of the specified column list must correspond to the column value list. The column value list for a single record is described in parentheses.
- **DEFAULT**: You can use the **DEFAULT** keyword to specify a default value as the column value. If you specify **DEFAULT** in the column value list next to the **VALUES** keyword, a default value column is stored for the given column: if you specify **DEFAULT** before the **VALUES** keyword, default values are stored for all columns in the table. **NULL** is stored for the column whose default value has not been defined.
- **ON DUPLICATE KEY UPDATE**: In case constraints are violated because a duplicated value for a column where **PRIMARY KEY** or **UNIQUE** attribute is defined is inserted, the value that makes constraints violated is changed into a specific value by performing the action specified in the **ON DUPLICATE KEY UPDATE** statement.

```
CREATE TABLE a_tbl1(
    id INT UNIQUE,
    name VARCHAR,
    phone VARCHAR DEFAULT '000-0000'
);

--insert default values with DEFAULT keyword before VALUES
INSERT INTO a_tbl1 DEFAULT VALUES;

--insert multiple rows
INSERT INTO a_tbl1 VALUES (1,'aaa', DEFAULT),(2,'bbb', DEFAULT);

--insert a single row specifying column values for all
INSERT INTO a_tbl1 VALUES (3,'ccc', '333-3333');
```

```
--insert two rows specifying column values for only
INSERT INTO a_tbl1(id) VALUES (4), (5);

--insert a single row with SET clauses
INSERT INTO a_tbl1 SET id=6, name='eee';
INSERT INTO a_tbl1 SET id=7, phone='777-7777';

SELECT * FROM a_tbl1;
```

```
          id  name                 phone
=============================================================
        NULL  NULL                 '000-0000'
           1  'aaa'                '000-0000'
           2  'bbb'                '000-0000'
           3  'ccc'                '333-3333'
           4  NULL                 '000-0000'
           5  NULL                 '000-0000'
           6  'eee'                '000-0000'
           7  NULL                 '777-7777'
```

```
INSERT INTO a_tbl1 SET id=6, phone='000-0000'
ON DUPLICATE KEY UPDATE phone='666-6666';
SELECT * FROM a_tbl1 WHERE id=6;
```

```
          id  name                 phone
=============================================================
           6  'eee'                '666-6666'
```

```
INSERT INTO a_tbl1 SELECT * FROM a_tbl1 WHERE id=7 ON DUPLICATE KEY
UPDATE name='ggg';
SELECT * FROM a_tbl1 WHERE id=7;
```

```
          id  name                 phone
=============================================================
           7  'ggg'                '777-7777'
```

In **INSERT ... SET** syntax, the evaluation of an assignment expression is performed from left to right. If the column value is not specified, then the default value is assigned. If there is no default value, **NULL** is assigned.

```
CREATE TABLE tbl (a INT, b INT, c INT);
INSERT INTO tbl SET a=1, b=a+1, c=b+2;
SELECT * FROM tbl;
```

```
        a            b            c
====================================
        1            2            4
```

In the above example, b's value will be 2 and c's value will be 4 since a's value is 1.

```
CREATE TABLE tbl2 (a INT, b INT, c INT);
INSERT INTO tbl2 SET a=b+1, b=1, c=b+2;
```

In the above example, a's value will be **NULL** since b's value is not specified yet when assigning a's value.

```
SELECT * FROM tbl2;
```

```
        a            b            c
====================================
     NULL            1            3
```

```
CREATE TABLE tbl3 (a INT, b INT default 10, c INT);
INSERT INTO tbl3 SET a=b+1, b=1, c=b+2;
```

In the above example, a's value will be 11 since b's value is not specified yet and b's default is 10.

```
SELECT * FROM tbl3;
```

```
        a            b            c
====================================
       11            1            3
```

INSERT ... SELECT Statement

If you use the **SELECT** query in the **INSERT** statement, you can insert query results which satisfy the specified retrieval condition from one or many tables to the target table.

```
INSERT [INTO] [schema_name.]table_name [(column_name, ...)]
    SELECT...
    [ON DUPLICATE KEY UPDATE column_name = expr, ... ]
```

The **SELECT** statement can be used in place of the **VALUES** keyword, or be included as a subquery in the column value list next to **VALUES**. If you specify the **SELECT** statement in place of the **VALUES** keyword, you can insert multiple query result records into the column of the table at once. However, there should be only one query result record if the **SELECT** statement is specified in the column value list.

```
--creating an empty table which schema replicated from a_tbl1
CREATE TABLE a_tbl2 LIKE a_tbl1;

--inserting multiple rows from SELECT query results
INSERT INTO a_tbl2 SELECT * FROM a_tbl1 WHERE id IS NOT NULL;

--inserting column value with SELECT subquery specified in the value
list
INSERT INTO a_tbl2 VALUES(8, SELECT name FROM a_tbl1 WHERE name
<'bbb', DEFAULT);

SELECT * FROM a_tbl2;
```

```
           id  name                  phone
=============================================================
            1  'aaa'                 '000-0000'
            2  'bbb'                 '000-0000'
            3  'ccc'                 '333-3333'
            4  NULL                  '000-0000'
            5  NULL                  '000-0000'
            6  'eee'                 '000-0000'
            7  NULL                  '777-7777'
            8  'aaa'                 '000-0000'
```

INSERT INTO <remote-table-spec>... SELECT statement

If a remote table of the same remote server used in the SELECT query is used in the INSERT statement, query results that satisfy specific search conditions from one or more remote tables can be inserted into the remote table. However, an error occurs when the remote table of the INSERT statement among the tables specified in the SELECT statement and the table of another server (local table or table of another remote server) are used.

```
--at remote-side
--creating an empty table which schema replicated from a_tbl1
```

```
CREATE TABLE a_tbl2 LIKE a_tbl1;

--at local-side
--inserting multiple rows from SELECT query results
INSERT INTO a_tbl2@server1 SELECT * FROM a_tbl1@server1 WHERE id IS
NOT NULL;

--inserting column value with SELECT subquery specified in the value
list
INSERT INTO a_tbl2@server1 VALUES(8, SELECT name FROM a_tbl1@server1
WHERE name <'bbb', DEFAULT);

SELECT * FROM a_tbl2@server1;
```

```
           id  name                  phone
=============================================================
            1  'aaa'                 '000-0000'
            2  'bbb'                 '000-0000'
            3  'ccc'                 '333-3333'
            4  NULL                  '000-0000'
            5  NULL                  '000-0000'
            6  'eee'                 '000-0000'
            7  NULL                  '777-7777'
            8  'aaa'                 '000-0000'
```

Data from a remote table can be inserted into a local table, but queries that insert data from a local table into a remote table cannot be used, so care must be taken when using it. The query below throws an error.

```
--inserting multiple rows from SELECT query results
INSERT INTO a_tbl2@server1 SELECT * FROM a_tbl1 WHERE id IS NOT NULL;
```

```
dblink: local mixed remote DML is not allowed
```

Also, if the server of the remote table used in the INSERT statement and the SELECT statement are different, query execution is not allowed. The query below throws an error.

```
--inserting multiple rows from SELECT query results
INSERT INTO a_tbl2@server1 SELECT * FROM a_tbl1@server2 WHERE id IS
NOT NULL;
```

```
dblink: multi-remote DML is not allowed
```

ON DUPLICATE KEY UPDATE Clause

In a situation in which a duplicate value is inserted into a column for which the **UNIQUE** index or the **PRIMARY KEY** constraint has been set, you can update to a new value by specifying the **ON DUPLICATE KEY UPDATE** clause in the **INSERT** statement.

Note

- If **PRIMARY KEY** and **UNIQUE** or multiple **UNIQUE** constraints exist on a table together, constraint violation can happen by one of them; so in this case, **ON DUPLICATE KEY UPDATE** clause is not recommended.
- Even if **UPDATE** is executed after failing executing **INSERT**, **AUTO_INCREMENT** value which is increased once cannot be rolled back into the previous value.

```
<INSERT ... VALUES statement>
<INSERT ... SET statement>
<INSERT ... SELECT statement>
    INSERT ...
    [ON DUPLICATE KEY UPDATE column_name = expr, ... ]
```

- *column_name* = *expr*: Specifies the name of the column whose value you want to change next to **ON DUPLICATE KEY UPDATE** and a new column value by using the equal sign.

```
--creating a new table having the same schema as a_tbl1
CREATE TABLE a_tbl3 LIKE a_tbl1;
INSERT INTO a_tbl3 SELECT * FROM a_tbl1 WHERE id IS NOT NULL and
name IS NOT NULL;
SELECT * FROM a_tbl3;
```

```
          id  name                 phone
============================================================
           1  'aaa'                '000-0000'
           2  'bbb'                '000-0000'
           3  'ccc'                '333-3333'
           6  'eee'                '000-0000'
```

```
--insert duplicated value violating UNIQUE constraint
INSERT INTO a_tbl3 VALUES(2, 'bbb', '222-2222');
```

```
ERROR: Operation would have caused one or more unique constraint
violations.
```

With ON DUPLICATE KEY UPDATE, “affected rows” value per row will be 1 if a new row is inserted, and 2 if an existing row is updated.

```
--insert duplicated value with specifying ON DUPLICATE KEY UPDATE
clause
INSERT INTO a_tbl3 VALUES(2, 'ggg', '222-2222')
ON DUPLICATE KEY UPDATE name='ggg', phone = '222-2222';

SELECT * FROM a_tbl3 WHERE id=2;
```

```
           id  name                 phone
============================================================
            2  'ggg'                '222-2222'

2 rows affected.
```

UPDATE

You can update the column value of a record stored in the target table or view to a new one by using the **UPDATE** statement. Specify the name of the column to update and the new value in the **SET** clause, and specify the condition to extract the record to be updated in the WHERE Clause. You can update one or more tables with a single **UPDATE** statement.

```
<UPDATE single table>
UPDATE [schema_name.]table_name | <remote_table_spec> | view_name
SET column_name = {<expr> | DEFAULT} [, column_name = {<expr> |
DEFAULT} ...]
    [WHERE <search_condition>]
    [ORDER BY {col_name | <expr>}]
    [LIMIT row_count]

<remote_table_spec> ::=
[schema_name.]table_name@[schema.name.]server_name [correlation>]
<UPDATE multiple tables>
UPDATE <table_specifications> SET column_name = {<expr> | DEFAULT}
[, column_name = {<expr> | DEFAULT} ...]
    [WHERE <search_condition>]
```

- *<table_specifications>*: You can specify the statement such as **FROM** clause of the **SELECT** statement and one or more tables can be specified.

- *server_name*: Used when specifying a table of a remote server connected by dblink, not the current server.
- *column_name*: Specifies the column name to be updated. Columns for one or more tables can be specified.
- <*expr*> | **DEFAULT**: Specifies a new value for the column and expression or **DEFAULT** keyword can be specified as a value. The **SELECT** statement returning result record also can be specified.
- <*search_condition*>: Update only data that meets the <*search_condition*> if conditions are specified in the WHERE Clause.
- *col_name* | <*expr*>: Specifies base column to be updated.
- *row_count*: Specifies the number of records to be updated after the LIMIT Clause. It can be one of unsigned integer, a host variable or a simple expression.

The following are allowed only when a single table is specified in <*table_specifications*>:

- The ORDER BY Clause can be specified. If the ORDER BY Clause is specified, records are updated in the order of the specified column. This is useful for maintaining the order of trigger execution and the order of locking.
- The LIMIT Clause can be specified. If the LIMIT Clause is specified, the number of records to be updated can be limited.
- Analytic functions can be used in the <*expr*> of the **SET** clause. However, if a **SELECT** query is specified in <*expr*>, analytic functions can be used in the **SELECT** query regardless of the number of tables specified in <*table_specifications*>.

Note

In CUBRID versions prior to 9.0, only a single table can be specified in <*table_specifications*>.

Note

From CUBRID 10.0 onward, updates to views containing **JOIN** clauses are possible.

Example 1. Single Table Update Using ORDER BY Clause

```
DROP TABLE IF EXISTS t1;

CREATE TABLE t1 (id INT);
```

```
INSERT INTO t1 VALUES (11), (1), (12), (13), (2), (3), (14), (4);

CREATE TRIGGER trigger1 BEFORE UPDATE ON t1 IF NEW.id < 10 EXECUTE
PRINT 'trigger1 executed';
CREATE TRIGGER trigger2 BEFORE UPDATE ON t1 IF NEW.id > 10 EXECUTE
PRINT 'trigger2 executed';

UPDATE t1  SET id = id + 1;

    trigger2 executed
    trigger1 executed
    trigger2 executed
    trigger2 executed
    trigger1 executed
    trigger1 executed
    trigger2 executed
    trigger1 executed
```

Using the ORDER BY Clause can change the order in which records are updated.

```
TRUNCATE TABLE t1;
INSERT INTO t1 VALUES (11), (1), (12), (13), (2), (3), (14), (4);

UPDATE t1  SET id = id + 1 ORDER BY id;

    trigger1 executed
    trigger1 executed
    trigger1 executed
    trigger1 executed
    trigger2 executed
    trigger2 executed
    trigger2 executed
    trigger2 executed
```

Example 2. Single Table Update Using LIMIT Clause

This example uses the *LIMIT 3* clause to update only up to 3 records with *name IS NULL*.

```
DROP TABLE IF EXISTS t1;

CREATE TABLE t1 (id int, name varchar, phone varchar);
INSERT INTO t1 VALUES (NULL,  NULL, '000-0000');
INSERT INTO t1 VALUES (   1, 'aaa', '111-1111');
INSERT INTO t1 VALUES (   2, 'bbb', '222-2222');
INSERT INTO t1 VALUES (   3, 'ccc', '333-3333');
INSERT INTO t1 VALUES (   4,  NULL, '000-0000');
INSERT INTO t1 VALUES (   5,  NULL, '000-0000');
INSERT INTO t1 VALUES (   6, 'ddd', '000-0000');
INSERT INTO t1 VALUES (   7,  NULL, '777-7777');
```

```
SELECT * FROM t1 WHERE name IS NULL;

              id  name                  phone
    ============================================================
            NULL  NULL                  '000-0000'
               4  NULL                  '000-0000'
               5  NULL                  '000-0000'
               7  NULL                  '777-7777'

UPDATE t1 SET name='update', phone='999-9999' WHERE name IS NULL
LIMIT 3;

SELECT * FROM t1;

              id  name                  phone
    ============================================================
            NULL  'update'              '999-9999'
               1  'aaa'                 '111-1111'
               2  'bbb'                 '222-2222'
               3  'ccc'                 '333-3333'
               4  'update'              '999-9999'
               5  'update'              '999-9999'
               6  'ddd'                 '000-0000'
               7  NULL                  '777-7777'
```

Example 3. Update Using Joins

When a record in table **A** is joined with multiple records in table **B** in an **UPDATE** statement, the record in **A** is updated using only the value from the first matching record in **B**.

In this example, a record in table *t1* with an *id* of 3 is joined with two records in table *t2* where *rate_id* is 3. However, the *charge* column in the *t1* table is updated using only the *rate* value from the first matching record in the *t2* table.

```
DROP TABLE IF EXISTS t1, t2;

CREATE TABLE t1 (id INT PRIMARY KEY, charge DOUBLE);
INSERT INTO t1 VALUES (1, 100.0);
INSERT INTO t1 VALUES (2, 100.0);
INSERT INTO t1 VALUES (3, 100.0);

CREATE TABLE t2 (rate_id INT, rate DOUBLE);
INSERT INTO t2 VALUES (1, 0.1);
INSERT INTO t2 VALUES (2, 0.2);
INSERT INTO t2 VALUES (3, 0.3);
INSERT INTO t2 VALUES (3, 0.5);

UPDATE t1 a INNER JOIN t2 b ON a.id = b.rate_id
SET a.charge = a.charge * (1 + b.rate);
```

```
SELECT id, TO_CHAR (charge) AS charge FROM t1;

               id  charge
    ===================================
                1  '110'
                2  '120'
                3  '150'
```

For details, see Join Query.

Example 4. View Update

```
DROP TABLE IF EXISTS t1, t2;
DROP VIEW v1;

CREATE TABLE t1 (id INT, val INT) DONT_REUSE_OID;
INSERT INTO t1 VALUES (1, 1);
INSERT INTO t1 VALUES (2, 2);
INSERT INTO t1 VALUES (3, 3);
INSERT INTO t1 VALUES (4, 4);
INSERT INTO t1 VALUES (5, 5);

CREATE TABLE t2 (id INT, val INT) DONT_REUSE_OID;
INSERT INTO t2 VALUES (1, 1);
INSERT INTO t2 VALUES (2, 2);
INSERT INTO t2 VALUES (3, 3);
INSERT INTO t2 VALUES (4, 4);
INSERT INTO t2 VALUES (6, 6);

CREATE VIEW v1 AS
  SELECT b.id, b.val FROM t2 b LEFT JOIN t1 a ON b.id = a.id WHERE
b.id <= 3;

UPDATE v1 SET val = -1;

SELECT * from v1;

               id          val
    ============================
                1           -1
                2           -1
                3           -1

SELECT * from t2;

               id          val
    ============================
                1           -1
                2           -1
```

```
                3            -1
                4             4
                6             6
```

⚠ Warning

Records in views that include tables with REUSE OID cannot be updated.

For details, see REUSE_OID and DONT_REUSE_OID.

Example 5. Update Using the update_use_attribute_references Parameter

The result of this example depends on the value of the update_use_attribute_references parameter.

- If the value of this parameter is yes, *c2* is updated to 10, influenced by c1 = 10.
- If the value of this parameter is no, *c2* is not influenced by *c1 = 10* and is updated to 1, based on the value of c1 stored in the record.

```
DROP TABLE IF EXISTS t1;

CREATE TABLE t1 (c1 INT, c2 INT);
INSERT INTO t1 values (1, NULL);

SET SYSTEM PARAMETERS 'update_use_attribute_references=yes';

UPDATE t1 SET c1 = 10, c2 = c1;

SELECT * FROM t1;

              c1           c2
    ==========================
              10           10
```

```
TRUNCATE TABLE t1;
INSERT INTO t1 values (1, NULL);

SET SYSTEM PARAMETERS 'update_use_attribute_references=no';

UPDATE t1 SET c1 = 10, c2 = c1;

SELECT * FROM t1;

              c1           c2
```

```
    ==========================
               10             1
```

Example 6. Single Table Update Using Analytic Functions

When only one table is specified in the **UPDATE** statement, analytic functions can be used in the **SET** clause.

```
DROP TABLE IF EXISTS t1, t2;

CREATE TABLE t1 (id INT);
INSERT INTO t1 VALUES (1), (2), (3), (4), (5);

CREATE TABLE t2 (id INT, val INT, update_val DOUBLE, join_update_val
DOUBLE);
INSERT INTO t2 (id, val) SELECT a.id, b.id FROM t1 a, t1 b WHERE
b.id <= a.id;

UPDATE t2 SET update_val = AVG (val) OVER (PARTITION BY id);

SELECT DISTINCT id, TO_CHAR (update_val) AS update_val FROM t2;

               id  update_val
    ===================================
                1  '1'
                2  '1.5'
                3  '2'
                4  '2.5'
                5  '3'
```

Example 7. Multiple Tables Update Using Analytic Functions

This example continues from Example 6. Single Table Update Using Analytic Functions.

When multiple tables are specified in the **UPDATE** statement, analytic functions cannot be used in the **SET** clause. However, if a **SELECT** query is specified in the **SET** clause, analytic functions can be used within that **SELECT** query, regardless of the number of tables specified.

```
UPDATE t1 a, t2 b SET b.join_update_val = AVG (b.val) OVER
(PARTITION BY b.id) WHERE a.id = b.id;

    ERROR: before '  where a.id = b.id; '
    Nested analytic functions are not allowed.
```

```
UPDATE t2 c SET c.join_update_val = (SELECT AVG (b.val) OVER
(PARTITION BY b.id) FROM t1 a, t2 b WHERE a.id = b.id AND a.id =
c.id LIMIT 1);
```

```
SELECT DISTINCT id, TO_CHAR (join_update_val) AS join_update_val
FROM t2;

               id  join_update_val
    ==================================
                1  '1'
                2  '1.5'
                3  '2'
                4  '2.5'
                5  '3'
```

Example 8. Remote Table Update

By using table extension names, it is possible to update tables not only on the local server but also on remote servers.

```
/**
 * on the remote server (e.g., 192.168.6.21)
 */

-- DROP SERVER origin;
CREATE SERVER origin (HOST='localhost', PORT=33000, DBNAME=demodb,
USER=public);

DROP TABLE IF EXISTS t1;

CREATE TABLE t1 (id int, name varchar, phone varchar);
INSERT INTO t1@origin VALUES (NULL,  NULL, '000-0000');
INSERT INTO t1@origin VALUES (   1, 'aaa', '111-1111');
INSERT INTO t1@origin VALUES (   2, 'bbb', '222-2222');
INSERT INTO t1@origin VALUES (   3, 'ccc', '333-3333');
INSERT INTO t1@origin VALUES (   4,  NULL, '000-0000');
INSERT INTO t1@origin VALUES (   5,  NULL, '000-0000');
INSERT INTO t1@origin VALUES (   6, 'ddd', '000-0000');
INSERT INTO t1@origin VALUES (   7,  NULL, '777-7777');

SELECT * FROM t1;

               id  name                 phone
    ============================================================
             NULL  NULL                 '000-0000'
                1  'aaa'                '111-1111'
                2  'bbb'                '222-2222'
                3  'ccc'                '333-3333'
                4  NULL                 '000-0000'
                5  NULL                 '000-0000'
                6  'ddd'                '000-0000'
                7  NULL                 '777-7777'
```

```
/**
 * on the local server (e.g., 192.168.6.22)
 */

-- DROP SERVER remote;
CREATE SERVER remote (HOST='192.168.6.21', PORT=33000,
DBNAME=demodb, USER=public);

SELECT * FROM t1@remote WHERE name IS NULL;

               id  name                  phone
    ============================================================
             NULL  NULL                  '000-0000'
                4  NULL                  '000-0000'
                5  NULL                  '000-0000'
                7  NULL                  '777-7777'

UPDATE t1@remote SET name='update', phone='999-9999' WHERE name IS
NULL LIMIT 3;

SELECT * FROM t1@remote;

               id  name                  phone
    ============================================================
             NULL  'update'              '999-9999'
                1  'aaa'                 '111-1111'
                2  'bbb'                 '222-2222'
                3  'ccc'                 '333-3333'
                4  'update'              '999-9999'
                5  'update'              '999-9999'
                6  'ddd'                 '000-0000'
                7  NULL                  '777-7777'
```

Example 9. Local Table Update Using Joins with Remote Tables

```
/**
 * on the remote server (e.g., 192.168.6.21)
 */

DROP TABLE IF EXISTS t2;

CREATE TABLE t2 (rate_id INT, rate DOUBLE);
```

```
/**
 * on the local server (e.g., 192.168.6.22)
 */

-- DROP SERVER remote;
```

```
CREATE SERVER remote (HOST='192.168.6.21', PORT=33000,
DBNAME=demodb, USER=public);

DROP TABLE IF EXISTS t1;

CREATE TABLE t1 (id INT PRIMARY KEY, charge DOUBLE);
INSERT INTO t1 VALUES (1, 100.0);
INSERT INTO t1 VALUES (2, 100.0);
INSERT INTO t1 VALUES (3, 100.0);

INSERT INTO t2@remote VALUES (1, 0.1);
INSERT INTO t2@remote VALUES (2, 0.2);
INSERT INTO t2@remote VALUES (3, 0.3);
INSERT INTO t2@remote VALUES (3, 0.5);

UPDATE t1 a INNER JOIN t2@remote b ON a.id = b.rate_id
SET a.charge = a.charge * (1 + b.rate);

SELECT id, TO_CHAR (charge) AS charge FROM t1;

               id  charge
    ==================================
                1  '110'
                2  '120'
                3  '150'
```

⚠ Warning

In **UPDATE** statements that join local and remote tables, records in the remote table cannot be updated.

```
UPDATE t1 a INNER JOIN t2@remote b ON a.id = b.rate_id
SET b.rate = b.rate + 0.1;

    ERROR: before '  a INNER JOIN t2@remote b ON a.id = b.rate_id
    SET b.rate = b....'
    dblink: local mixed remote DML is not allowed
```

REPLACE

The **REPLACE** statement works like INSERT, but the difference is that it inserts a new record after deleting the existing record without displaying the error when a duplicate value is to be inserted into a column for which **PRIMARY KEY** or **UNIQUE** constraints have defined. You must have both

INSERT and **DELETE** authorization to use the **REPLACE** statement, because it performs insertion or insertion after deletion operations. Please see GRANT for more information about authorization.

```
<REPLACE ... VALUES statement>
REPLACE [INTO] <table_specification> [(column_name, ...)]
    {VALUES | VALUE}({expr | DEFAULT}, ...)[,({expr |
DEFAULT}, ...),...]

<REPLACE ... SET statement>
REPLACE [INTO] <table_specification>
    SET column_name = {expr | DEFAULT}[, column_name = {expr |
DEFAULT},...]

<REPLACE ... SELECT statement>
REPLACE [INTO] <table_specification> [column_name, ...)]
    SELECT...

<table_specfication> ::= [schema_name.]table_name
                       |
[schema_name.]table_name@[schema_name.]server_name
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*: Specifies the name of the target table into which you want to insert a new record.
- *server_name*: Specifies the remote server name of the target table. For local tables, do not specify *server_name*.
- *column_name*: Specifies the name of the column into which you want to insert the value. If you omit to specify the column name, it is considered that all columns defined in the table have been specified. Therefore, you must specify the value for the column next to **VALUES**. If you do not specify all the columns defined in the table, a **DEFAULT** value is assigned to the non-specified columns; if the **DEFAULT** value is not defined, a **NULL** value is assigned.
- *expr* | **DEFAULT**: Specifies values that correspond to the columns after **VALUES**. Expressions or the **DEFAULT** keyword can be specified as a value. At this time, the order and number of the specified column list must correspond to the column value list. The column value list for a single record is described in parentheses.

The **REPLACE** statement determines whether a new record causes the duplication of **PRIMARY KEY** or **UNIQUE** index column values. Therefore, for performance reasons, it is recommended to use the **INSERT** statement for a table for which a **PRIMARY KEY** or **UNIQUE** index has not been defined.

```
--creating a new table having the same schema as a_tbl1
CREATE TABLE a_tbl4 LIKE a_tbl1;
INSERT INTO a_tbl4 SELECT * FROM a_tbl1 WHERE id IS NOT NULL and
```

```
name IS NOT NULL;

SELECT * FROM a_tbl4;
```

```
           id  name                 phone
============================================================
            1  'aaa'                '000-0000'
            2  'bbb'                '000-0000'
            3  'ccc'                '333-3333'
            6  'eee'                '000-0000'
```

```
--insert duplicated value violating UNIQUE constraint
REPLACE INTO a_tbl4 VALUES(1, 'aaa', '111-1111'),(2, 'bbb',
'222-2222');
REPLACE INTO a_tbl4 SET id=6, name='fff', phone=DEFAULT;

SELECT * FROM a_tbl4;
```

```
           id  name                 phone
============================================================
            3  'ccc'                '333-3333'
            1  'aaa'                '111-1111'
            2  'bbb'                '222-2222'
            6  'fff'                '000-0000'
```

DELETE

You can delete records in the table by using the **DELETE** statement. You can specify delete conditions by combining the statement with the WHERE Clause. You can delete one or more tables with one **DELETE** statement.

```
<DELETE single table>
DELETE [FROM] [schema_name.]table_name[@[schema_name.]server_name]
[<correlation>] WHERE <search_condition> ] [LIMIT row_count]

<DELETE multiple tables FROM ...>
DELETE [schema_name.]table_name[@[schema_name.]server_name] |
<correlation> [{,
[schema_name.]table_name[@[schema_name.]server_name] |
<correlation>}]
  FROM <table_specifications> [ WHERE <search_condition> ]

<DELETE FROM multiple tables USING ...>
DELETE FROM [schema_name.]table_name[@[schema_name.]server_name] |
```

```
<correlation> [{,
[schema_name.]table_name[@[schema_name.]server_name] |
<correlation>}]
  USING <table_specifications> [ WHERE <search_condition> ]
```

- *<table_specifications>*: You can specify the same syntax as the **FROM** clause of the **SELECT** statement, and you can specify one or more tables. Starting with version 11.3, you can specify not only local tables but also remote tables.
- *server_name*: Used when specifying a table of a remote server connected by dblink, not the current server.
- *correlation*: alias of the specified local table or remote table
- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name*: Specifies the name of a table where the data to be deleted is contained. If the number of table is one, the **FROM** keyword can be omitted.
- *search_condition*: Deletes only data that meets *search_condition* by using WHERE Clause. If it is specified, all data in the specified tables will be deleted.
- *row_count*: Specifies the number of records to be deleted in the LIMIT Clause. It can be one of unsigned integer, a host variable or a simple expression.

When a table to delete records is only one, LIMIT Clause can be specified. You can limit the number of records by specifying the LIMIT Clause. If the number of records satisfying the WHERE Clause exceeds *row_count*, only the number of records specified in *row_count* will be deleted.

Note

- On the **DELETE** statement with multiple tables, the table alias can be defined within *<table_specifications>* only. At the outside of *<table_specifications>*, the table alias defined in *<table_specifications>* can be used.
- Previous versions of CUBRID 9.0 allow only one table for *<table_specifications>*.

```
CREATE TABLE a_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));
INSERT INTO a_tbl VALUES(1,'111-1111'), (2,'222-2222'), (3,
'333-3333'), (4, NULL), (5, NULL);

--delete one record only from a_tbl
```

```
DELETE FROM a_tbl WHERE phone IS NULL LIMIT 1;
SELECT * FROM a_tbl;
```

```
           id  phone
================================
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
            5  NULL
```

```
--delete all records from a_tbl
DELETE FROM a_tbl;
```

Below tables are created to explain **DELETE JOIN**.

```
CREATE TABLE a_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));
CREATE TABLE b_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));
CREATE TABLE c_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));

INSERT INTO a_tbl VALUES(1,'111-1111'), (2,'222-2222'), (3,
'333-3333'), (4, NULL), (5, NULL);
INSERT INTO b_tbl VALUES(1,'111-1111'), (2,'222-2222'), (3,
'333-3333'), (4, NULL);
INSERT INTO c_tbl VALUES(1,'111-1111'), (2,'222-2222'), (10,
'333-3333'), (11, NULL), (12, NULL);
```

The below queries delete rows after joining multiple tables. They show the same result.

```
-- Below four queries show the same result.
--  <DELETE multiple tables FROM ...>

DELETE a, b FROM a_tbl a, b_tbl b, c_tbl c
WHERE a.id=b.id AND b.id=c.id;

DELETE a, b FROM a_tbl a INNER JOIN b_tbl b ON a.id=b.id
INNER JOIN c_tbl c ON b.id=c.id;

-- <DELETE FROM multiple tables USING ...>

DELETE FROM a, b USING a_tbl a, b_tbl b, c_tbl c
```

```
WHERE a.id=b.id AND b.id=c.id;

DELETE FROM a, b USING a_tbl a INNER JOIN b_tbl b ON a.id=b.id
INNER JOIN c_tbl c ON b.id=c.id;
```

For more details on join syntax, see Join Query.

You can also delete remote table data using the extended table name. The query below is for deleting remote table data.

```
--at remote srv1
CREATE TABLE a_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));
INSERT INTO a_tbl VALUES(1,'111-1111'), (2,'222-2222'), (3,
'333-3333'), (4, NULL), (5, NULL);

--at local
--delete one record only from remote a_tbl
DELETE FROM a_tbl@srv1 WHERE phone IS NULL LIMIT 1;
SELECT * FROM a_tbl@srv1;
           id  phone
==================================
            1  '111-1111'
            2  '222-2222'
            3  '333-3333'
            5  NULL
--delete all records from remote a_tbl
DELETE FROM a_tbl@srv1;
```

The tables below were created to explain remote DELETE JOIN.

```
--at remote srv1
CREATE TABLE a_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));

CREATE TABLE b_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));

CREATE TABLE c_tbl(
    id INT NOT NULL,
    phone VARCHAR(10));

--at local
INSERT INTO a_tbl@srv1 VALUES(1,'111-1111'), (2,'222-2222'), (3,
'333-3333'), (4, NULL), (5, NULL);
INSERT INTO b_tbl@srv1 VALUES(1,'111-1111'), (2,'222-2222'), (3,
```

```
'333-3333'), (4, NULL);
INSERT INTO c_tbl@srv1 VALUES(1,'111-1111'), (2,'222-2222'), (10,
'333-3333'), (11, NULL), (12, NULL);
```

The following queries perform deletion after joining tables including remote tables (2 local tables, 1 remote table), and all show the same result.

```
-- Below four queries show the same result.
--  <DELETE multiple tables FROM ...>

DELETE a, b FROM a_tbl@srv1 a, b_tbl@srv1 b, c_tbl@srv1 c
WHERE a.id=b.id AND b.id=c.id;

DELETE a, b FROM a_tbl@srv1 a INNER JOIN b_tbl@srv1 b ON a.id=b.id
INNER JOIN c_tbl@srv1 c ON b.id=c.id;

-- <DELETE FROM multiple tables USING ...>

DELETE FROM a, b USING a_tbl@srv1 a, b_tbl@srv1 b, c_tbl@srv1 c
WHERE a.id=b.id AND b.id=c.id;

DELETE FROM a, b USING a_tbl@srv1 a INNER JOIN b_tbl@srv1 b ON
a.id=b.id
INNER JOIN c_tbl@srv1 c ON b.id=c.id;
```

Caution

As shown below, DELETE JOIN queries that include local tables and remote tables and delete remote tables are not allowed. This is because it does not send local data to the remote.

```
DELETE c FROM a_tbl a, b_tbl b, c_tbl@srv1 c
WHERE a.id=b.id AND b.id=c.id;

DELETE a, b, c FROM a_tbl a, b_tbl b, c_tbl@srv1 c
WHERE a.id=b.id AND b.id=c.id;
```

MERGE

The **MERGE** statement is used to select rows from a source and to update or to insert the rows onto one table or view. You can specify the condition whether to update or to insert the rows onto the target table or view. The **MERGE** statement is a deterministic statement, so you cannot update the same rows on the target table several times within one sentence.

```
MERGE [<merge_hint>] INTO <target> [[AS] <alias>]
USING <source> [[AS] <alias>]
ON <join_condition>
[ <merge_update_clause> ]
[ <merge_insert_clause> ]

<merge_update_clause> ::=
WHEN MATCHED THEN UPDATE
SET <col = expr> [,<col = expr>,...] [WHERE <update_condition>]
[DELETE WHERE <delete_condition>]

<merge_insert_clause> ::=
WHEN NOT MATCHED THEN INSERT [(<attributes_list>)] VALUES
(<expr_list>) [WHERE <insert_condition>]

<merge_hint> ::=
/*+ [ USE_UPDATE_IDX (<update_index_list>) ] [ USE_INSERT_IDX
(<insert_index_list>) ] */
```

- <*target*>: The target table to update or insert into. It can be multiple tables or views, and you can also specify remote tables like table-name@server-name.
- <*source*>: The source table from which to import data. It can be a single table or view, and a subquery is also possible. If *target* is a remote table, *source* must also be a remote table or view specified on the same server as *target*. Or, it must be a subquery that includes the server specified in the target.
- <*join_condition*>: Specifies the updated conditions
- <*merge_update_clause*>: If <*join_condition*> is TRUE, the new column value of a target table will be specified.
  - When the **UPDATE** clause is executed, all **UPDATE** triggers defined in <*target*> are enabled.
  - <*col*>: The column to be updated should be the column of the target table.
  - <*expr*>: Allows the expression including columns of <*source*> and <*target*>. Or it can be the **DEFAULT** keyword. <*expr*> cannot include the aggregate functions.
  - <*update_condition*>: Performs the **UPDATE** operation only when a specific condition is TRUE. For the condition, both of the source tables and the target tables can be referenced. The rows which do not satisfy the condition are not updated.
  - <*delete_condition*>: Specifies a target to be deleted after update. At this time, the **DELETE** condition is executed with the updated result value. When **DELETE** statement is executed, the **DELETE** trigger defined in <*target*> will be enabled.
  - The column referred by the **ON** condition clause cannot be updated.

  - When you update view, you cannot specify **DEFAULT**.

- <*merge_insert_clause*>: If the <*join_condition*> condition is FALSE, the column value of the target table will be inserted.
  - When the **INSERT** clause is executed, all **INSERT** triggers defined in the target table are enabled.
  - <*insert_condition*>: Performs the INSERT operation only when the specified condition is TRUE. <Columns of <*source*> can be included in the condition.
  - <*attribute_list*>: Columns to be inserted in <*target*>.
  - <*expr_list*>: Integer filter condition can be used to insert all source rows to the target table. ON (0=1) is an example of integer filter condition.
  - This clause can be specified as it is or as <*merge_update_clause*>. If both of two are defined, the order does not matter.
- <*merge_hint*>: Index hint of **MERGE** statement
  - **USE_UPDATE_IDX** (<*update_index_list*>): Index hint used in **UPDATE** clause of **MERGE** statement. Index names are listed on the *update_index_list* when **UPDATE** clause is run. This hint is applied to <*join_condition*> and <*update_condition*>.
  - **USE_INSERT_IDX** (<*insert_index_list*>): Index hint used in **INSERT** clause of **MERGE** statement. Index names are listed on the *insert_index_list* when **INSERT** clause is run. This hint is applied to <*join_condition*>.

To execute the **MERGE** statement, the **SELECT** authorization for the source table should be granted. When the **UPDATE** clause is included in the target table, the **UPDATE** authorization should be granted. When the **DELETE** clause is included in the target table, the **DELETE** should be granted. When the **INSERT** clause is included in the target table, the **INSERT** should be granted.

The following example shows how to merge the value of *source_table* to *target_table*.

```
-- source_table
CREATE TABLE source_table (a INT, b INT, c INT);
INSERT INTO source_table VALUES (1, 1, 1);
INSERT INTO source_table VALUES (1, 3, 2);
INSERT INTO source_table VALUES (2, 4, 5);
INSERT INTO source_table VALUES (3, 1, 3);

-- target_table
CREATE TABLE target_table (a INT, b INT, c INT);
INSERT INTO target_table VALUES (1, 1, 4);
INSERT INTO target_table VALUES (1, 2, 5);
INSERT INTO target_table VALUES (1, 3, 2);
INSERT INTO target_table VALUES (3, 1, 6);
```

```
INSERT INTO target_table VALUES (5, 5, 2);

MERGE INTO target_table tt USING source_table st
ON (st.a=tt.a AND st.b=tt.b)
WHEN MATCHED THEN UPDATE SET tt.c=st.c
     DELETE WHERE tt.c = 1
WHEN NOT MATCHED THEN INSERT VALUES (st.a, st.b, st.c);

-- the result of above query
SELECT * FROM target_table;
```

```
           a            b            c
=======================================
           1            2            5
           1            3            2
           3            1            3
           5            5            2
           2            4            5
```

In the above example, when column A and B of source_table are identical with the values of column A and B in *target_table*, column C of *target_table* is updated with the column C of *source_table*. Otherwise, the records in *source_table* are inserted to *target_table*. However, if the value of column C in *target_table* is 1 in the updated record, the record is deleted.

The following example shows how to use the **MERGE** statement to arrange the records of *bonus* score table to give to students.

```
CREATE TABLE bonus (std_id INT, addscore INT);
CREATE INDEX i_bonus_std_id ON bonus (std_id);

INSERT INTO bonus VALUES (1,10);
INSERT INTO bonus VALUES (2,10);
INSERT INTO bonus VALUES (3,10);
INSERT INTO bonus VALUES (4,10);
INSERT INTO bonus VALUES (5,10);
INSERT INTO bonus VALUES (6,10);
INSERT INTO bonus VALUES (7,10);
INSERT INTO bonus VALUES (8,10);
INSERT INTO bonus VALUES (9,10);
INSERT INTO bonus VALUES (10,10);

CREATE TABLE std (std_id INT, score INT);
CREATE INDEX i_std_std_id  ON std (std_id);
CREATE INDEX i_std_std_id_score ON std (std_id, score);

INSERT INTO std VALUES (1,60);
INSERT INTO std VALUES (2,70);
INSERT INTO std VALUES (3,80);
```

```
INSERT INTO std VALUES (4,35);
INSERT INTO std VALUES (5,55);
INSERT INTO std VALUES (6,30);
INSERT INTO std VALUES (7,65);
INSERT INTO std VALUES (8,65);
INSERT INTO std VALUES (9,70);
INSERT INTO std VALUES (10,22);
INSERT INTO std VALUES (11,67);
INSERT INTO std VALUES (12,20);
INSERT INTO std VALUES (13,45);
INSERT INTO std VALUES (14,30);

MERGE INTO bonus t USING (SELECT * FROM std WHERE score < 40) s
ON t.std_id = s.std_id
WHEN MATCHED THEN
UPDATE SET t.addscore = t.addscore + s.score * 0.1
WHEN NOT MATCHED THEN
INSERT (t.std_id, t.addscore) VALUES (s.std_id, 10 + s.score * 0.1)
WHERE s.score <= 30;

SELECT * FROM bonus ORDER BY 1;
```

```
std_id     addscore
==========================
     1           10
     2           10
     3           10
     4           14
     5           10
     6           13
     7           10
     8           10
     9           10
    10           12
    12           12
    14           13
```

In the above example, the source table is a set of *std* table records, where the score is less than 40 and the target table is *bonus*. The student numbers (*std_id*) where the score (*std.score*) is less than 40 are 4, 6, 10, 12, and 14. Among them, for 4, 6, and 10 on the *bonus* table, the **UPDATE** clause adds 10% of the score of their own to the existing bonus. For 12 and 14 which are not on the *bonus* table, the **INSERT** clause adds 10 scores and 10% of the score of their own.

The following shows how to use index hints in **MERGE** statement.

```
CREATE TABLE target (i INT, j INT);
CREATE TABLE source (i INT, j INT);
```

```
INSERT INTO target VALUES (1,1),(2,2),(3,3);
INSERT INTO source VALUES (1,11),(2,22),(4,44),(5,55),(7,77),(8,88);

CREATE INDEX i_t_i ON target(i);
CREATE INDEX i_t_ij ON target(i,j);
CREATE INDEX i_s_i ON source(i);
CREATE INDEX i_s_ij ON source(i,j);

MERGE /*+ RECOMPILE USE_UPDATE_IDX(i_s_ij) USE_INSERT_IDX(i_t_ij,
i_t_i) */
INTO target t USING source s ON t.i=s.i
WHEN MATCHED THEN UPDATE SET t.j=s.j WHERE s.i <> 1
WHEN NOT MATCHED THEN INSERT VALUES (i,j);
```

To execute the MERGE statement, you must have SELECT privilege on the source table, UPDATE privilege if the UPDATE clause is included, DELETE privilege if the DELETE clause is included, and INSERT privilege if the INSERT clause is included on the target table.

The following is an example of combining source_table values of the remote server with target_table of the remote server.

```
-- at remote server
-- source_table
CREATE TABLE source_table (a INT, b INT, c INT);
CREATE TABLE target_table (a INT, b INT, c INT);

-- at local server
-- source table
INSERT INTO source_table@srv1 VALUES (1, 1, 1);
INSERT INTO source_table@srv1 VALUES (1, 3, 2);
INSERT INTO source_table@srv1 VALUES (2, 4, 5);
INSERT INTO source_table@srv1 VALUES (3, 1, 3);

-- target_table
INSERT INTO target_table@srv1 VALUES (1, 1, 4);
INSERT INTO target_table@srv1 VALUES (1, 2, 5);
INSERT INTO target_table@srv1 VALUES (1, 3, 2);
INSERT INTO target_table@srv1 VALUES (3, 1, 6);
INSERT INTO target_table@srv1 VALUES (5, 5, 2);

MERGE INTO target_table@srv1 tt USING source_table@srv1 st
ON (st.a=tt.a AND st.b=tt.b)
WHEN MATCHED THEN UPDATE SET tt.c=st.c
     DELETE WHERE tt.c = 1
WHEN NOT MATCHED THEN INSERT VALUES (st.a, st.b, st.c);

-- the result of above query
SELECT * FROM target_table@srv1;
        a           b           c
=======================================
```

```
        1           2           5
        1           3           2
        3           1           3
        5           5           2
        2           4           5
```

The following is an example of combining source_table values of the remote server with target_table of the local server.

```
-- at remote server
-- source_table
CREATE TABLE source_table (a INT, b INT, c INT);

-- at local server
CREATE TABLE target_table (a INT, b INT, c INT);

-- source table insert
INSERT INTO source_table@srv1 VALUES (1, 1, 1);
INSERT INTO source_table@srv1 VALUES (1, 3, 2);
INSERT INTO source_table@srv1 VALUES (2, 4, 5);
INSERT INTO source_table@srv1 VALUES (3, 1, 3);

-- target_table insert
INSERT INTO target_table VALUES (1, 1, 4);
INSERT INTO target_table VALUES (1, 2, 5);
INSERT INTO target_table VALUES (1, 3, 2);
INSERT INTO target_table VALUES (3, 1, 6);
INSERT INTO target_table VALUES (5, 5, 2);

MERGE INTO target_table tt USING source_table@srv1 st
ON (st.a=tt.a AND st.b=tt.b)
WHEN MATCHED THEN UPDATE SET tt.c=st.c
     DELETE WHERE tt.c = 1
WHEN NOT MATCHED THEN INSERT VALUES (st.a, st.b, st.c);

-- the result of above query
SELECT * FROM target_table;
        a           b           c
=========================================
        1           2           5
        1           3           2
        3           1           3
        5           5           2
        2           4           5
```

⚠ Warning

As in the example below, if the target is remote and the source is local, an error occurs.

```
MERGE INTO target_table@srv1 tt USING source_table st
ON (st.a=tt.a AND st.b=tt.b)
WHEN MATCHED THEN UPDATE SET tt.c=st.c
     DELETE WHERE tt.c = 1
WHEN NOT MATCHED THEN INSERT VALUES (st.a, st.b, st.c);
```

Also, an error occurs when the target and source are different servers, as in the example below.

```
MERGE INTO target_table@srv1 tt USING source_table@srv2 st
ON (st.a=tt.a AND st.b=tt.b)
WHEN MATCHED THEN UPDATE SET tt.c=st.c
     DELETE WHERE tt.c = 1
WHEN NOT MATCHED THEN INSERT VALUES (st.a, st.b, st.c);
```

TRUNCATE

You can delete all records in the specified table by using the **TRUNCATE** statement.

It has the following advantages over using the **DELETE FROM** *[schema_name.]table_name* statement without a **WHERE** clause.

- It's way faster due to deleting all indexes and constraints in advance and deleting records at once.
- There is no vacuum cost.
- It generates way lesser log records so that it is better in terms of HA replication, Recovery, and Rollback.

Note

- The **DELETE** trigger is disabled while deleting records with the **TRUNCATE** statement.
- The **TRUNCATE** statement initializes the **AUTO INCREMENT** column of the table. Therefore, if data is inserted, the **AUTO INCREMENT** column value increases from the initial value.
- To execute the **TRUNCATE** statement, the authorization of **ALTER**, **INDEX**, and **DELETE** is required on the table. For granting authorization, see GRANT.

- If a table is created with **DONT_REUSE_OID** option (DONT_REUSE_OID), other tables can refer to it as a column. When the specified table is referred to like this, it is impossible to truncate the table and it's done with **DELETE FROM** statement internally.

```
TRUNCATE [ TABLE ] [schema_name.]table_name [ CASCADE ]
```

- *schema_name*: Specifies the schema name. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table that contains the data to be deleted.
- **CASCADE** : Deletes all records in all tables referring to the specified table with a foreign key. This is propagated to all tables in the foreign key relationship. A **PRIMARY KEY** constraint has to be defined in the table and this has to be referred to by one or more **FOREIGN KEY**, and the foreign key action has to be **ON DELETE**. It fails without this option when a foreign key referring to the specified table is defined. It also fails when even one **ON DELETE** action is not **CASCADE** in all foreign key relationships. See FOREIGN KEY Constraint for more information about the foreign key constraint.

```
CREATE TABLE a_tbl(A INT AUTO_INCREMENT(3,10) PRIMARY KEY);
INSERT INTO a_tbl VALUES (NULL),(NULL),(NULL);
SELECT * FROM a_tbl;
```

```
            a
==============
            3
            13
            23
```

```
--AUTO_INCREMENT column value increases from the initial value after
truncating the table
TRUNCATE TABLE a_tbl;
INSERT INTO a_tbl VALUES (NULL);
SELECT * FROM a_tbl;
```

```
            a
==============
            3
```

The following example uses the **CASCADE** option. Note that even a record refers to nothing, but the propagated **TRUNCATE** deletes all records.

```
-- a_tbl <- b_tbl <- c_tbl
--       <- d_tbl
CREATE TABLE a_tbl(a1 INT PRIMARY KEY);
CREATE TABLE b_tbl(b1 INT PRIMARY KEY, b2 INT FOREIGN KEY REFERENCES
a_tbl (a1) ON DELETE CASCADE);
CREATE TABLE c_tbl(c1 INT PRIMARY KEY, c2 INT FOREIGN KEY REFERENCES
b_tbl (b1) ON DELETE CASCADE);
CREATE TABLE d_tbl(d1 INT PRIMARY KEY, d2 INT FOREIGN KEY REFERENCES
a_tbl (a1) ON DELETE CASCADE);
INSERT INTO a_tbl VALUES (1);
INSERT INTO b_tbl VALUES (2, 1);
INSERT INTO c_tbl VALUES (3, 2);
INSERT INTO d_tbl VALUES (5, 1);
INSERT INTO d_tbl VALUES (4, NULL); -- not refer to any record in
a_tbl, but will be deleted.
SELECT * FROM a_tbl;
SELECT * FROM b_tbl;
SELECT * FROM c_tbl;
SELECT * FROM d_tbl;

TRUNCATE a_tbl CASCADE;
SELECT * FROM a_tbl, b_tbl, c_tbl, d_tbl;
```

```
          a1
=============
           1

          b1           b2
==========================
           2            1

          c1           c2
==========================
           3            2

          d1           d2
==========================
           5            1
           4         NULL

-- after TRUNCATE a_tbl CASCADE;
There are no results.
```

PREPARED STATEMENT

In general, the prepared statement is executed through the interface functions of JDBC, PHP, or ODBC; it can also be executed in the SQL level. The following SQL statements are provided for execution of prepared statement.

- Prepare the SQL statement to execute.

```
PREPARE stmt_name FROM preparable_stmt
```

- Execute the prepared statement.

```
EXECUTE stmt_name [USING value [, value] ...]
```

- Drop the prepared statement.

```
{DEALLOCATE | DROP} PREPARE stmt_name
```

Note

- In SQL level, PREPARE statement is recommended to use only in CSQL interpreter. If it is used in the application program, it is not guaranteed to work normally.
- In SQL level, the number of PREPARE statements is limited to 20 per DB connection. It is limited to protect abusing DB server memory, because PREPARE statement in SQL level uses the memory of DB server.
- In the interface function, the number of prepared statements is limited to MAX_PREPARED_STMT_COUNT of broker parameter per DB connection.

PREPARE Statement

The **PREPARE** statement prepares the query specified in *preparable_stmt* of the **FROM** clause and assigns the name to be used later when the SQL statement is referenced to *stmt_name*. See EXECUTE Statement for example.

```
PREPARE stmt_name FROM preparable_stmt
```

- *stmt_name* : The prepared statement is specified. If an SQL statement with the same *stmt_name* exists in the given client session, clear the existing prepared statement and prepare a new SQL

statement. If the **PREPARE** statement is not executed properly due to an error in the given SQL statement, it is processed as if the *stmt_name* assigned to the SQL statement does not exist.

- *preparable_stmt* : You must use only one SQL statement. Multiple SQL statements cannot be specified. You can use a question mark (?) as a bind parameter in the *preparable_stmt* statement and it should not be enclosed with quotes.

Remark

The **PREPARE** statement starts by connecting an application to a server and will be maintained until the application terminates the connection. The connection maintained during this period is called a session. You can set the session time with the **session_state_timeout** parameter of **cubrid.conf**; the default value is **21600** seconds (=6 hours).

The data managed by the session includes the **PREPARE** statement, user-defined variables, the last ID inserted (**LAST_INSERT_ID**), and the number of rows affected by the statement (**ROW_COUNT**) that you execute at the end.

EXECUTE Statement

The **EXECUTE** statement executes the prepared statement. You can bind the data value after the **USING** clause if a bind parameter (?) is included in the prepared statement. You cannot specify user-defined variables like an attribute in the **USING** clause. A value such as literal and an input parameter only can be specified.

```
EXECUTE stmt_name [USING value [, value] ...]
```

- *stmt_name* : The name given to the prepared statement to be executed is specified. An error message is displayed if the *stmt_name* is not valid, or if the prepared statement does not exist.
- *value* : The data to bind is specified if there is a bind parameter in the prepared statement. The number and the order of the data must correspond to that of the bind parameter. If it does not, an error message is displayed.

```
PREPARE st FROM 'SELECT 1 + ?';
EXECUTE st USING 4;
```

```
   1+ ?:0
=========================
   5
```

```
PREPARE st FROM 'SELECT 1 + ?';
SET @a=3;
EXECUTE st USING @a;
```

```
   1+ ?:0
==========================
   4
```

```
PREPARE st FROM 'SELECT ? + ?';
EXECUTE st USING 1,3;
```

```
   ?:0 + ?:1
==========================
   4
```

```
PREPARE st FROM 'SELECT ? + ?';
EXECUTE st USING 'a','b';
```

```
   ?:0 + ?:1
==========================
   'ab'
```

```
PREPARE st FROM 'SELECT FLOOR(?)';
EXECUTE st USING '3.2';
```

```
   floor( ?:0 )
==========================
   3.000000000000000e+000
```

DEALLOCATE PREPARE/DROP PREPARE Statements

The statements **DEALLOCATE PREPARE** and **DROP PREPARE** are used interchangeably and they clear the prepared statement. All prepared statements are cleared automatically by the server when the client session is terminated even if the **DEALLOCATE PREPARE** or **DROP PREPARE** statement is not executed.

```
{DEALLOCATE | DROP} PREPARE stmt_name
```

- *stmt_name* : The name given to the prepared statement to be cleared is specified. An error message is displayed if the *stmt_name* is not valid, or if the prepared statement does not exist.

```
DEALLOCATE PREPARE stmt1;
```

DO

The **DO** statement executes the specified expression, but does not return the result. This can be used to determine whether or not the syntax of the expression is correct because an error is returned when a specified expression does not comply with the syntax. In general, the execution speed of the **DO** statement is higher than that of the **SELECT** statement because the database server does not return the operation result or errors.

```
DO expression
```

- *expression* : Specifies an expression.

```
DO 1+1;
DO SYSDATE + 1;
DO (SELECT count(*) FROM athlete);
```

CTE

Common Table Expressions (CTEs) are temporary tables (list of results) associated with a statement. A CTE can be referenced multiple times within the statement, and is visible only within the statement scope. It enables better separation of statement logic and may enhance execution performance. Moreover, recursive CTEs can be used to generate a hierarchical statement based on parent-child relationships, being able to reproduce **CONNECT BY** statements and other more complex queries.

A CTE is introduced using the **WITH** clause. A list of subqueries is expected, and final query which uses the subqueries. Each subquery (table expression) has a name and a query definition. A table expression may refer another table expression which previously defined in the same statement. The general syntax is:

```
WITH
  [RECURSIVE <recursive_cte_name> [ (<recursive_column_names>) ] AS
<recursive_sub-query>]
```

```
  <cte_name1> [ (<cte1_column_names>) ] AS <sub-query1>
  <cte_name2> [ (<cte2_column_names>) ] AS <sub-query2>
  ...
<final_query>
```

- *recursive_cte_name*, *cte_name1*, *cte_name2* : identifiers for the table expressions (subqueries)
- *recursive_column_names*, *cte1_column_names*, *cte2_column_names* : identifiers for the columns of the results of each table expression
- *sub-query1*, *sub-query2* : subqueries which define each table expression.
- *final_query* : query using table expression previously defined. Usually, the **FROM** clause of this will contain the CTEs identifiers.

Simplest usage is to combine result lists of table expressions:

```
CREATE TABLE products (id INTEGER PRIMARY KEY, parent_id INTEGER,
item VARCHAR(100), price INTEGER);
INSERT INTO products VALUES (1, -1, 'Drone', 2000);
INSERT INTO products VALUES (2, 1, 'Blade', 10);
INSERT INTO products VALUES (3, 1, 'Brushless motor', 20);
INSERT INTO products VALUES (4, 1, 'Frame', 50);
INSERT INTO products VALUES (5, -1, 'Car', 20000);
INSERT INTO products VALUES (6, 5, 'Wheel', 100);
INSERT INTO products VALUES (7, 5, 'Engine', 4000);
INSERT INTO products VALUES (8, 5, 'Frame', 4700);

WITH
 of_drones AS (SELECT item, 'drones' FROM products WHERE parent_id =
1),
 of_cars AS (SELECT item, 'cars' FROM products WHERE parent_id = 5)
SELECT * FROM of_drones UNION ALL SELECT * FROM of_cars ORDER BY 1;
```

```
  item                'drones'
=======================================
  'Blade'             'drones'
  'Brushless motor'   'drones'
  'Car'               'cars'
  'Drone'             'drones'
  'Engine'            'cars'
  'Frame'             'drones'
  'Frame'             'cars'
  'Wheel'             'cars'
```

A subquery of one CTE may be referenced by other subquery of another CTE (the referenced CTE needs to be defined before):

```
WITH
 of_drones AS (SELECT item FROM products WHERE parent_id = 1),
 filter_common_with_cars AS (SELECT * FROM of_drones INTERSECT
SELECT item FROM products WHERE parent_id = 5)
SELECT * FROM filter_common_with_cars ORDER BY 1;
```

```
  item
=======================
  'Frame'
```

Error will be prompted if:

- More than one CTE uses the same identifier name.
- using nested **WITH** clauses.

```
WITH
 my_cte AS (SELECT item FROM products WHERE parent_id = 1),
 my_cte AS (SELECT * FROM my_cte INTERSECT SELECT item FROM products
WHERE parent_id = 5)
SELECT * FROM my_cte ORDER BY 1;
```

```
before '
    SELECT * FROM my_cte ORDER BY 1;
'
CTE name ambiguity, there are more than one CTEs with the same name:
'my_cte'.
```

```
WITH
 of_drones AS (SELECT item FROM products WHERE parent_id = 1),
 of_cars1 AS (WITH
                of_cars2 AS (SELECT item FROM products WHERE
parent_id = 5)
              SELECT * FROM of_cars2
              )
SELECT * FROM of_drones, of_cars1 ORDER BY 1;
```

```
before '
    SELECT * FROM of_drones, of_cars1 ORDER BY 1;
'
Nested WITH clauses are not supported.
```

CTE column names

The column names of each CTE result may be specified after the CTE name. The number of elements in the CTE column list must match the number of columns in the CTE subquery.

```
WITH
 of_drones (product_name, product_type, price) AS (SELECT item,
'drones', price FROM products WHERE parent_id = 1),
 of_cars (product_name, product_type, price) AS (SELECT item,
'cars', price FROM products WHERE parent_id = 5)
SELECT * FROM of_drones UNION ALL SELECT * FROM of_cars ORDER BY
product_type, price;

WITH
 of_drones (product_name, product_type, price) AS (SELECT item,
'drones' as type, MAX(price) FROM products WHERE parent_id = 1 GROUP
BY type),
 of_cars (product_name, product_type, price) AS (SELECT item,
'cars'  as type, MAX (price) FROM products WHERE parent_id = 5 GROUP
BY type)
SELECT * FROM of_drones UNION ALL SELECT * FROM of_cars ORDER BY
product_type, price;
```

```
  product_name          product_type                price
===============================================================
  'Wheel'               'cars'                        100
  'Engine'              'cars'                       4000
  'Frame'               'cars'                       4700
  'Blade'               'drones'                       10
  'Brushless motor'     'drones'                       20
  'Frame'               'drones'                       50

 product_name          product_type                price
===============================================================
 'Wheel'               'cars'                       4700
 'Blade'               'drones'                       50
```

If no column names are given in the CTE, the column names are extracted from the first inner select list of the CTE. The expressions result columns will be named according to their original text.

```
WITH
 of_drones AS (SELECT item, 'drones', MAX(price) FROM products WHERE
parent_id = 1 GROUP BY 2),
 of_cars AS (SELECT item, 'cars', MAX (price) FROM products WHERE
parent_id = 5 GROUP BY 2)
SELECT * FROM of_drones UNION ALL SELECT * FROM of_cars ORDER BY 1;
```

```
 item                 'drones'              max(products.price)
=========================================================================
 'Blade'              'drones'                                50
 'Wheel'              'cars'                                4700
```

RECURSIVE clause

The **RECURSIVE** keyword allows construction recurrent queries (the table expression subqueries definition contains its own name). A recursive table expression is composed of the non-recursive part and a recursive part (which references the subqueries by its CTE name). The recursive and non-recursive parts **must** be combined using the **UNION ALL** query operator. The recursive part should be defined in such way, that no cycle will be generated. Also if the recursive part contains aggregate functions, it should also contain a **GROUP BY** clause, because aggregate functions will return always a tuple and the recursive iterations will never stop. The recursive part will stop iterating when the conditions from **WHERE** clause are no longer true, and the current iteration return no results.

```
WITH
 RECURSIVE cars (id, parent_id, item, price) AS (
                   SELECT id, parent_id, item, price
                       FROM products WHERE item LIKE 'Car%'
                   UNION ALL
                   SELECT p.id, p.parent_id, p.item, p.price
                       FROM products p
                   INNER JOIN cars rec_cars ON p.parent_id =
rec_cars.id)
SELECT item, price FROM cars ORDER BY 1;
```

```
  item                          price
=====================================
  'Car'                         20000
  'Engine'                       4000
  'Frame'                        4700
  'Wheel'                         100
```

Recursive CTEs may fall into an infinite loop. To avoid such case, set the system parameter **cte_max_recursions** to a desired threshold. Its default value is 2000 recursive iterations, maximum is 1000000 and minimum 2.

```
SET SYSTEM PARAMETERS 'cte_max_recursions=2';
WITH
 RECURSIVE cars (id, parent_id, item, price) AS (
                   SELECT id, parent_id, item, price
                       FROM products  WHERE item LIKE 'Car%'
```

```
                    UNION ALL
                    SELECT p.id, p.parent_id, p.item, p.price
                        FROM products p
                    INNER JOIN cars rec_cars ON p.parent_id =
rec_cars.id)
SELECT item, price FROM cars ORDER BY 1;
```

```
In the command from line 9,
Maximum recursions 2 reached executing CTE.
```

⚠ Warning

- Depending on the complexity of the CTE subqueries, the result set can grow very large for subqueries which produces large amount of data. Even the default value of **cte_max_recursions** may not be enough to avoid starvation of disk space.

The execution algorithm of a recursive CTE may be summarized as:

- execute the non recursive part of CTE and add its results to then final result set
- execute the recursive part using the result set obtained by the non recursive part, add its results to the final result set and memorize the start and end of the current iteration within the result set.
- repeat the non recursive part execution using the result set from previous iteration and add its results to the final result set
- if a recursive iteration produces no results, then stop
- if the configured maximum number of iterations is reached, also stop

The recursive CTE must be referenced directly in the **FROM** clause, referencing it in subquery will prompt an error:

```
WITH
 RECURSIVE cte1(x) AS SELECT c FROM t1 UNION ALL SELECT * FROM
(SELECT cte1.x + 1 FROM cte1 WHERE cte1.x < 5)
SELECT * FROM cte1;
```

```
before '
SELECT * FROM cte1;
'
```

```
Recursive CTE 'cte1' must be referenced directly in its recursive
query.
```

CTE Usage in DMLs and CREATE

Besides their use for **SELECT** statements, CTEs can also be used for other statements. CTEs can be used in **CREATE TABLE** *table_name* **AS SELECT**:

```
CREATE TABLE inc AS
    WITH RECURSIVE cte (n) AS (
        SELECT 1
        UNION ALL
        SELECT n + 1
        FROM cte
        WHERE n < 3)
    SELECT n FROM cte;

SELECT * FROM inc;
```

```
            n
=============
            1
            2
            3
```

Also, **INSERT**/**REPLACE INTO** *table_name* **SELECT** can use CTE:

```
INSERT INTO inc
    WITH RECURSIVE cte (n) AS (
        SELECT 1
        UNION ALL
        SELECT n + 1
        FROM cte
        WHERE n < 3)
    SELECT * FROM cte;

REPLACE INTO inc
   WITH cte AS (SELECT * FROM inc)
   SELECT * FROM cte;
```

Also, in subclauses of **UPDATE** statement:

```
CREATE TABLE green_products (producer_id INTEGER, sales_n INTEGER,
product VARCHAR, product_type INTEGER, price INTEGER);
INSERT INTO green_products VALUES (1, 99, 'bicycle', 1, 99);
```

```
INSERT INTO green_products VALUES (2, 337, 'bicycle', 1, 129);
INSERT INTO green_products VALUES (3, 5012, 'bicycle', 1, 199);
INSERT INTO green_products VALUES (1, 989, 'scooter', 2, 899);
INSERT INTO green_products VALUES (3, 3211, 'scooter', 2, 599);
INSERT INTO green_products VALUES (4, 2312, 'scooter', 2, 1009);

WITH price_increase_th AS (
    SELECT SUM (sales_n) * 7 / 10 AS threshold, product_type
    FROM green_products
    GROUP BY product_type
)
    UPDATE green_products gp JOIN price_increase_th th ON
gp.product_type = th.product_type
    SET price = price + (price / 10)
    WHERE sales_n >= threshold;
```

And also, in subclauses of **DELETE** statement:

```
WITH product_removal_th AS (
    SELECT SUM (sales_n) / 20 AS threshold, product_type
    FROM green_products
    GROUP BY product_type
)
    DELETE
    FROM green_products gp
    WHERE sales_n < (select threshold from product_removal_th WHERE
product_type = gp.product_type);
```

CALL

The **CALL** statement invokes a stored routine defined in the database. The stored routines supported by CUBRID are as follows.

- CREATE PROCEDURE
- CREATE FUNCTION
- Method

```
CALL <routine_clause> ;

    <routine_clause> ::=
        routine_name ([<arg_value> [{, <arg_value> } ...]])
[<method_call_target>] [<to_variable>]

    <arg_value>
        literal |
```

```
        :host_variable |
        any CSQL expression

    <method_call_target> ::=
        ON [CLASS] object-valued-expression

    <to_variable> ::=
        INTO :host_variable |
        TO :host_variable
```

- *routine_name*: Specifies the method name or stored function/procedure name. It is not case-sensitive.
- <*arg_value*>: Specifies the arguments of the stored routine. The argument can be a literal value, host variable, and any expression.
- <*method_call_target*>: Specifies only when using a method written in C. Specifies an object-valued-expression containing a class name, a variable, or another method call (which returns an object). To call a class method for a class object, you must place the **CLASS** keyword before the <*call_target*>. In this case, the table name must be the name of the class where the table method is defined. To call a record method, you must specify the expression representing the record object.
- <*to_variable*>: The value returned by the stored routine can be stored in a host variable.

For *routine_name* of the **CALL** statement, CUBRID searches a stored routine as follows.

1 It is processed as a method if there is a target class in the **CALL** statement. 2 If there is no target class in the **CALL** statement, it is checked whether a Java stored function/procedure is executed or not; a Java stored function/procedure will be executed if one exists. 3 If no Java stored function/procedure exists in step 2 above, it is checked whether a method is executed or not; a method will be executed if one with the same name exists.

```
CALL Hello() INTO :HELLO;
CALL Sp_int(3) INTO :i;
CALL phone_info('Tom','016-111-1111');
```

The following error occurs if you call a Java stored function/procedure that does not exist.

- If there is **no argument** in the **CALL** statement, a message “ERROR: Stored procedure/function ‘deposit’ does not exist.” appears because it can be distinguished from a method.
- if there is **an argument** in the **CALL** statement, a message “ERROR: Methods require an object as their target.” appears because it cannot be distinguished from a method.

```
// there is not argument in the CALL statement
CALL deposit();
```

```
ERROR: Stored procedure/function 'deposit' does not exist.
```

```
// there are arguments in the CALL statement
CALL deposit('Tom', 3000000);
```

```
ERROR: Methods require an object as their target.
```

If the **CALL** statement is nested within another **CALL** statement calling a Java stored function/procedure, or if a subquery is used in calling the Java function/procedure, the **CALL** statement is not executed.

```
CALL phone_info('Tom', CALL sp_int(999));
CALL phone_info((SELECT * FROM Phone WHERE id='Tom'));
```

Query Optimization

Join Methods

A Relational database stores data by dividing it into multiple tables through normalization. This approach effectively reduces data redundancy and ensures data integrity. However, to query the divided data in an integrated manner, relationships between tables must be combined. A join operation combines these relationships and returns results as if they were from a single table.

Join Type

The records included in the join results depend on the specified type of join. For more details, refer to Join Query.

Join Method

The way records are combined between tables depends on the join method used. CUBRID supports the following join methods:

- Nested Loop Join
- Sort-Merge Join

- Hash Join

Join methods are determined by the query optimizer based on a cost comparison, though users can also specify them manually. The query optimizer calculates the cost of each join method using statistics collected by the database. It then selects the method with the lowest cost. When a query includes two or more joins, different join methods can be applied to each join. Each join method operates on only two tables at a time and is executed sequentially. In other words, the next join is performed only after the previous one has been completed.

By default, CUBRID does not generate execution plans for **Sort-Merge Joins** or **Hash Joins**. SQL hints are required for the optimizer to consider these join methods. The **optimizer_enable_merge_join** parameter can be set to **yes** (default is **no**) to enable a **Sort-Merge Join** without requiring the **USE_MERGE** hint. For more details, refer to Other Parameters.

Note

The term **table** here is used broadly to include not only physical tables but also views (virtual tables) and derived tables.

Nested Loop Join

A nested loop join reads records from the outer table one at a time and combines each record with matching records from the inner table. This approach is similar to the concept of nested loops in programming. The table used in the outer loop is referred to as the **driving table**, while the table in the inner loop is called the **driven table**.

Driving Table

When selecting a driving table from two tables, it is generally more efficient to choose the table with fewer records. Fewer records in the driving table result in fewer join operations, reducing the overall cost of the join. Since random access costs are typically constant, fewer join operations on the driven table lead to lower join costs and better performance. The query optimizer automatically selects the driving table by comparing join costs, but users can specify the driving table explicitly using **ORDERED** or **LEADING** hints.

Driven Table

A nested loop join performs efficiently when random access to the driven table is possible. For random access, a join-compatible index must be created on the driven table. If no such index exists, all records in the driven table must be read sequentially, which degrades join performance. When random access to the driven table is enabled through a join-compatible index, the nested loop join is displayed as an **Index Join (idx-join)** in the execution plan.

Partial Range Processing

Nested loop joins can perform join operations by reading just one record from each table, and they maintain join results that can be returned immediately, even if the join is not completed for all records. This behavior enables partial range processing: if only the first N results are needed, they are returned as soon as those N results are ready. Partial range processing improves overall join performance by avoiding unnecessary join operations.

Hash List Scan

If the driven table is a view or a derived table, the index cannot be used, so the join is performed using the list scan method, which sequentially reads records from the temporary result. Hash list scan is a type of list scan that allows random access to records in the temporary result that satisfy the join condition. Since list scan reads records from a temporary result, it cannot be used for physical tables. Therefore, hash list scan can only be used when the driven table is a view or a derived table. Hash list scan operates similarly to a hash join and can be used only when all join conditions are equality conditions. The use of hash list scan can be verified through query profiling.

The differences between hash list scan and hash join are as follows:

1. Hash list scan builds the hash table using only the driven table.

2. Hash list scan is part of a nested loop join, enabling partial range processing.

The query optimizer does not consider whether a hash list scan is performed when calculating the cost of a join.

Example 1. Nested Loop Join without Index Scans

In this example, the driven table does not utilize an index scan during the nested loop join. The **ORDERED** hint is applied to enforce the join order, making the nation table the driving table and the athlete table the driven table. Since the driven table lacks an index appropriate for the join condition, all records in the athlete table are read sequentially for each record in the nation table.

```
select /*+ recompile ordered use_nl */
    n.name, a.name
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
where
    n.continent = 'Asia'
    and a.event = 'Baseball';
```

```
Query plan:

nl-join (inner join)
```

```
    edge:  term[0]
    outer: sscan
               class: n node[0]
               sargs: term[2]
               cost:  2 card 31
    inner: sscan
               class: a node[1]
               sargs: term[0] AND term[1]
               cost:  49 card 202
    cost:  4697 card 29
```

```
Trace Statistics:
  SELECT (time: 92, fetch: 1720, fetch_time: 0, ioread: 0)
    SCAN (table: public.nation), (heap time: 0, fetch: 216,
ioread: 0, readrows: 215, rows: 47)
      SCAN (table: public.athlete), (heap time: 92, fetch:
1504, ioread: 0, readrows: 313819, rows: 103)
```

Execution Steps:

1. Read a record from the nation table that satisfies the filter condition (`n.continent = 'Asia'`).
2. Read a record from the athlete table that satisfies both the filter condition (`a.event = 'Baseball'`) and join condition (`n.code = a.nation_code`).
3. If the condition is met, combine the two records and add them to the result.
4. Repeat **step 2** for the next record in the athlete table.
5. Repeat **steps 1-3** for the next record in the nation table.
6. Continue until all records from both tables are processed.

Example 2. Nested Loop Join with Index Scans

This example demonstrates a nested loop join where the driven table uses an index scan. An index suitable for the join condition is created on the driven table (athlete). As a result, not all records in the athlete table need to be scanned for each record in the nation table, enabling random access.

```
create index i_athlete_nation_code on athlete (nation_code);

select /*+ recompile ordered use_nl */
    n.name, a.name
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
```

```
where
    n.continent = 'Asia'
    and a.event = 'Baseball';

drop index i_athlete_nation_code on athlete;
```

```
Query plan:

idx-join (inner join)
    outer: sscan
                class: n node[0]
                sargs: term[2]
                cost:  2 card 31
    inner: iscan
                class: a node[1]
                index: i_athlete_nation_code term[0]
                sargs: term[1]
                cost:  4 card 202
    cost:  68 card 29
```

```
Trace Statistics:
  SELECT (time: 1, fetch: 1265, fetch_time: 0, ioread: 0)
    SCAN (table: public.nation), (heap time: 1, fetch: 216,
ioread: 0, readrows: 215, rows: 47)
      SCAN (index: public.athlete.i_athlete_nation_code),
(btree time: 0, fetch: 1045, ioread: 0, readkeys: 24,
filteredkeys: 0, rows: 948) (lookup time: 0, rows: 103)
```

Compared to Example 1, the number of records read from the athlete table before comparing the join conditions is significantly reduced (`readrows: 313,819` -> `rows: 948`).

Example 3. Performance Differences Depending on the Driving Table

This example changes the driving table to athlete, unlike previous examples where nation was the driving table. The **LEADING** hint is used to specify the join order, making athlete the driving table and nation the driven table.

```
select /*+ recompile leading(a, n) use_nl */
    n.name, a.name
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
where
    n.continent = 'Asia'
    and a.event = 'Baseball';
```

```
Query plan:

idx-join (inner join)
    outer: sscan
                class: a node[0]
                sargs: term[1]
                cost:  49 card 202
    inner: iscan
                class: n node[1]
                index: pk_nation_code term[0]
                sargs: term[2]
                cost:  2 card 31
    cost:  161 card 29
```

```
Trace Statistics:
  SELECT (time: 5, fetch: 7168, fetch_time: 0, ioread: 0)
    SCAN (table: public.athlete), (heap time: 5, fetch:
6709, ioread: 0, readrows: 6677, rows: 229)
      SCAN (index: public.nation.pk_nation_code), (btree
time: 0, fetch: 458, ioread: 0, readkeys: 229,
filteredkeys: 0, rows: 229) (lookup time: 0, rows: 103)
```

Using athlete as the driving table leads to a decrease in performance due to its higher number of records.

Example 4. Nested Loop Join with Hash List Scan

This example demonstrates a nested loop join where a hash list scan is performed on the driven table. By combining the **ORDERED** and **NO_MERGE** hints, an inline view that cannot perform an index scan is used as the driven table. Additionally, the query retrieves only the first 20 results, allowing partial range processing to be combined with the hash list scan.

```
select /*+ recompile ordered use_nl */
    n.name, v.name
from
    nation as n
    inner join (
            select /*+ no_merge */
                a.name, a.nation_code
            from
                athlete as a
            where
                a.event = 'Baseball'
        ) as v on n.code = v.nation_code
where
    n.continent = 'Asia'
limit 20;
```

```
Query plan:

nl-join (inner join)
    edge:  term[0]
    outer: sscan
                class: n node[0]
                sargs: term[1] AND term[2]
                cost:  2 card 31
    inner: sscan
                class: v node[1]
                sargs: term[0]
                cost:  2 card 202
    sargs: term[1]
    cost:  148 card 29
```

```
Trace Statistics:
  SELECT (time: 2, fetch: 33, fetch_time: 0, ioread: 0)
    SCAN (table: public.nation), (heap time: 0, fetch: 1,
ioread: 0, readrows: 26, rows: 8)
      SCAN (hash temp(m), build time: 0, time: 0, fetch: 0,
ioread: 0, readrows: 249, rows: 20)
    SUBQUERY (uncorrelated)
      SELECT (time: 2, fetch: 32, fetch_time: 0, ioread: 0)
        SCAN (table: public.athlete), (heap time: 2, fetch:
32, ioread: 0, readrows: 6677, rows: 229)
```

Hash list scans can be confirmed through query profiling but are not explicitly shown in the execution plan.

Sort-Merge Join

A sort merge join combines records from two tables by first sorting them based on the join condition and then sequentially comparing the sorted records. This method can only be performed when the join condition is an equality condition.

The process of sorting data from two tables before joining is similar to creating an index in real time. However, the sorting process is not skipped even if a join-compatible index exists. The process of combining the sorted data is similar to a nested loop join in that it performs the join operation by reading one record from each of the two sorted temporary results. The buffer size used during the sorting process can be set with the **sort_buffer_size** parameter. For more details, refer to Memory-Related Parameters.

Join Cost

Sort-merge joins can become inefficient when the sorting step incurs a high cost. When the query optimizer compares the costs of join methods, nested loop joins are often evaluated as having relatively low cost because they do not require preprocessing, Hash joins also tend to have lower cost since they build a hash table using data from only one of the two tables. In contrast, sort-merge joins are evaluated as having higher cost because they need to sort the data from both tables.

Partial Range Processing

Sort-merge joins cannot perform partial range processing because they can only proceed after the sorting of both tables is complete. As a result, even if only the first N results are needed, the query will return results only after the entire join process has finished.

Example 1. Execution Process of a Sort-Merge Join

This example demonstrates the steps involved in executing a sort-merge join. Although the **ORDERED** hint does not directly impact join performance, it is included here to ensure the join is performed in the specified order.

```
create index i_nation_continent_code on nation (continent,
code);
create index i_athlete_event_nation_code on athlete (event,
nation_code);

select /*+ recompile ordered use_merge */
    n.name, a.name
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
where
    n.continent = 'Asia'
    and a.event = 'Baseball';

drop index i_nation_continent_code on nation;
drop index i_athlete_event_nation_code on athlete;
```

```
Query plan:

temp
    order: UNORDERED
    subplan: m-join (inner join)
                edge:  term[0]
                outer: temp
                           order: code[0]
                           subplan: iscan
                                       class: n node[0]
                                       index:
i_nation_continent_code term[1]
```

```
                                        cost:  2 card 1
                          cost:  8 card 1
                inner: temp
                          order: nation_code[1]
                          subplan: iscan
                                        class: a node[1]
                                        index:
i_athlete_event_nation_code term[2]
                                        cost:  2 card 7
                          cost:  8 card 7
                cost:  17 card 1
    cost:  23 card 1
```

```
Trace Statistics:
  SELECT (time: 1, fetch: 287, fetch_time: 1, ioread: 0)
    SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 103,
rows: 103)
      MERGELIST
        SELECT (time: 0, fetch: 52, fetch_time: 0, ioread:
0)
          SCAN (index:
public.nation.i_nation_continent_code), (btree time: 0,
fetch: 49, ioread: 0, readkeys: 47, filteredkeys: 0, rows:
47) (lookup time: 0, rows: 47)
          ORDERBY (time: 0, sort: true, page: 0, ioread: 0)
        SELECT (time: 1, fetch: 235, fetch_time: 1, ioread:
0)
          SCAN (index:
public.athlete.i_athlete_event_nation_code), (btree time:
1, fetch: 231, ioread: 0, readkeys: 6, filteredkeys: 0,
rows: 229) (lookup time: 1, rows: 229)
          ORDERBY (time: 0, sort: true, page: 0, ioread: 0)
```

Execution Steps:

1. Retrieve records from the nation table that meet the filter condition (`n.continent = 'Asia'`) and sort them by the code column.

```
select n.code, n.name from nation as n where n.continent =
'Asia' order by n.code;
```

1. Retrieve records from the athlete table that meet the filter condition (`a.event = 'Baseball'`) and sort them by the nation_code column.

```
select a.nation_code, a.name from athlete as a where
a.event = 'Baseball' order by a.nation_code;
```

1. Sequentially read records from the two sorted results, compare the join condition (`n.code = a.nation_code`), and if the condition is satisfied, combine the two records and add them to the result.

2. Repeat this process until all records have been processed and the join is complete.

Hash Join

A hash join combines records from two tables by using a hash function to match records with the same hash value. To efficiently compare records with the same hash value, a hash table is used. One of the two tables is used to **build** the hash table, while the other is used to **probe** it. The hash value is generated by applying a hash function to the join column values. When different input values produce the same hash value, a hash collision occurs. In such cases, records with the same hash value are sequentially examined, and the join condition is checked. A hash join can only be performed when all join conditions are equality conditions.

SQL Hints

There are two hints associated with hash joins:

- **USE_HASH**: Allows the query optimizer to generate an execution plan using a hash join.
- **NO_USE_HASH**: Prevents the query optimizer from generating an execution plan using a hash join.

Both hints can specify one or more tables. The hint applies to the inner table (driven table) if a table is explicitly specified. If no table is specified, the hint applies to all tables. If the same table is specified in both the **USE_HASH** and **NO_USE_HASH** hints, the **NO_USE_HASH** hint takes precedence.

Cost Calculation

Each table in a hash join can either be used to build the hash table or to probe it. To determine the most efficient approach, the costs of both options are calculated and compared, with the lower cost being selected as the hash join cost. For example, the following scenarios are compared, and the one with the lower cost is selected:

- Building the hash table with the outer table (driving table) and probing it with the inner table (driven table).

- Building the hash table with the inner table (driven table) and probing it with the outer table (driving table).

Table used to build the hash table

The table used to build the hash table cannot be explicitly specified using **ORDERED** or **LEADING** hints. The table with fewer records is typically chosen for building the hash table. However, in the case of outer joins, the table used to build the hash table is fixed based on the join direction:

- **Left Outer Join**: The right table is used to build the hash table.
- **Right Outer Join**: The left table is used to build the hash table.

How to build the hash table

The method used to build the hash table depends on the size of the temporary results and the maximum size of the hash table. The maximum hash table size can be configured using the **max_hash_list_scan_size** parameter. For more details, refer to Memory-Related Parameters.

CUBRID supports the following hashing methods:

- **Memory Hash**: Used when all temporary results can fit in the hash table.

```
Number of temporary result pages * Page size <= Maximum
hash table size
```

- **Hybrid Hash**: Used when the hash table can store the positions of all temporary result records.

```
Number of temporary result records * (Hash entry size +
Position information size <= Maximum hash table size
```

- **File Hash**: Used when neither of the above methods is applicable.
- **Skip Hash**: If either of the two temporary results is empty or unnecessary (in the case of an outer join), building the hash table is skipped.

Hybrid Hash and **File Hash** store record positions in the hash table, requiring an additional step to locate records before reading them.

Memory Hash and **Hybrid Hash** use memory pages to build the hash table, whereas **File Hash** uses temporary volume pages. Accessing pages in a temporary volume incurs additional logical I/O, which can lead to performance degradation.

During query execution, once the join is complete, the hash table is deleted, and the memory it occupied is released. The specific hash method used during a hash join can be identified through query profiling. For details, refer to Query Profiling.

Partial Range Processing

Hash joins cannot perform partial range processing because the join can only proceed after the hash table has been fully built.

Example 1. Execution Process of a Hash Join

This example explains the steps involved in performing a hash join. Even with the **ORDERED** hint, the table used to build the hash table cannot be explicitly specified.

```
select /*+ recompile ordered use_hash */
    n.name, a.name
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
where
    n.continent = 'Asia'
    and a.event = 'Baseball';
```

```
Query plan:

hash-join (inner join)
    edge:  term[0]
    outer: sscan
                class: n node[0]
                sargs: term[2]
                cost:  2 card 31
    inner: sscan
                class: a node[1]
                sargs: term[1]
                cost:  49 card 202
    cost:  63 card 29
```

```
Trace Statistics:
  SELECT (time: 2, fetch: 33, fetch_time: 0, ioread: 0)
    SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 103,
rows: 103)
      HASHJOIN (time: 2, fetch: 33, fetch_time: 0, ioread:
0)
        BUILD (time: 0, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: memory)
          SELECT (time: 0, fetch: 1, fetch_time: 0, ioread:
0)
```

```
            SCAN (table: public.nation), (heap time: 0,
fetch: 1, ioread: 0, readrows: 215, rows: 47)
        PROBE (time: 2, probe_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, readkeys: 103, rows: 103,
max_collisions: 1)
          SELECT (time: 2, fetch: 32, fetch_time: 0,
ioread: 0)
            SCAN (table: public.athlete), (heap time: 1,
fetch: 32, ioread: 0, readrows: 6677, rows: 229)
```

The nation table contains 47 records that satisfy the filter condition (`n.continent = 'Asia'`), and the athlete table contains 229 records that satisfy the filter condition (`a.event = 'Baseball'`). Thus, the nation table with fewer records is selected to build the hash table.

Query Profiling Results:

This section describes only the query profiling items related to hash joins. For details on other items, refer to Query Profiling.

1. **HASHJOIN**
    - **time**: Total execution time for hash join (ms)
    - **fetch**: Pages fetched during the hash join
    - **fetch_time**: Time spent fetching pages for the hash join (ms)
    - **ioread**: Number of physical I/O reads during page fetches for the hash join
1. **BUILD**: Except for **time**, the remaining items do not include cumulative information when querying temporary results.
    - **time**: Total time for building the hash table, including temporary results (ms)
    - **build_time**: : Time specifically spent building the hash table (ms)
    - **fetch**: Pages fetched during the hash table build
    - **fetch_time**: Time spent fetching pages for the hash table build (ms)
    - **ioread**: Physical I/O reads during the hash table build
    - **hash_method**: Method used to build the hash table (**memory**, **hybrid**, **file**, or **skip**)
        - **memory**: Use the memory hashing method
        - **hybrid**: Use the hybrid hashing method

- **file**: Use the file hashing method
- **skip**: Skip building the hash table

1. **PROBE**: Except for **time**, the remaining items do not include cumulative information when querying temporary results.
   - **time**: Total time for probing the hash table, including temporary results (ms)
   - **probe_time**: Time specifically spent probing the hash table (ms)
   - **fetch**: Pages fetched during the hash table probe
   - **fetch_time**: Time spent fetching pages for the hash table probe (ms)
   - **ioread**: Physical I/O reads during the hash table probe
   - **readkeys**: Number of records with matching hash values during the probe
   - **rows**: Number of records satisfying the join condition (**readkeys** >= **rows**)
   - **max_collisions**: Maximum number of records read for a single hash value during probing

Execution Steps:

1. Retrieve records from the nation table that satisfy the filter condition (`n.continent = 'Asia'`) and create a temporary result.

```
select n.code, n.name from nation as n where
n.continent = 'Asia';
```

1. Retrieve records from the athlete table that satisfy the filter condition (`a.event = 'Baseball'`) and create a temporary result.

```
select a.nation_code, a.name from athlete as a
where a.event = 'Baseball';
```

1. Read the temporary result from the nation table and build a hash table.
2. For each record in the temporary result from the athlete table, probe the hash table of the nation table.

3. Match records(`readkeys`) in the hash table with the same hash value and compare them against the join condition (`n.code = a.nation_code`). If the condition is satisfied(`rows`), combine the records and add them to the result.

4. Repeat **step 5** for subsequent records with the same hash value.

5. Repeat **steps 4-6** for the next record in the athlete table.

6. Continue until all join operations are complete.

예시 2. Hash join using hints

This example demonstrates how to use the **USE_HASH** and **NO_USE_HASH** hints to control which tables perform a hash join. The **ORDERED** hint is used to fix the inner table (driven table) for applying the hint.

```
select /*+ recompile ordered use_hash no_use_hash(r) */
    n.name, a.name, a.code, r.medal
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
    inner join record as r on a.code = r.athlete_code
where
    n.continent = 'Asia'
    and a.event = 'Shooting'
    and r.host_year = 1992;
```

```
Query plan:

idx-join (inner join)
    outer: hash-join (inner join)
                edge:  term[0]
                outer: sscan
                           class: n node[0]
                           sargs: term[3]
                           cost:  2 card 31
                inner: sscan
                           class: a node[1]
                           sargs: term[2]
                           cost:  49 card 202
                cost:  63 card 29
    inner: iscan
                class: r node[2]
                index:
pk_record_host_year_event_code_athlete_code_medal term[4]
(covers)
                filtr: term[1]
```

```
                cost:  5 card 400
    cost:  109 card 2
```

The **USE_HASH** hint specifies that a hash join is performed for the nation and athlete tables.

The **NO_USE_HASH** hint ensures that a hash join is not performed for the record table.

```
Trace Statistics:
  SELECT (time: 5, fetch: 139, fetch_time: 0, ioread: 0)
    SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 35,
rows: 35)
      SCAN (index:
public.record.pk_record_host_year_event_code_athlete_code_me
dal), (btree time: 3, fetch: 105, ioread: 0, readkeys: 7,
filteredkeys: 0, rows: 7, covered: true)
      HASHJOIN (time: 2, fetch: 33, fetch_time: 0, ioread:
0)
        BUILD (time: 1, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: memory)
          SELECT (time: 1, fetch: 1, fetch_time: 0, ioread:
0)
            SCAN (table: public.nation), (heap time: 1,
fetch: 1, ioread: 0, readrows: 215, rows: 47)
        PROBE (time: 1, probe_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, readkeys: 35, rows: 35,
max_collisions: 1)
          SELECT (time: 1, fetch: 32, fetch_time: 0,
ioread: 0)
            SCAN (table: public.athlete), (heap time: 1,
fetch: 32, ioread: 0, readrows: 6677, rows: 173)
```

Example 3. Hash join with index skip scan

This example demonstrates using an index skip scan to retrieve temporary results during a hash join.

```
/* for nested loop join */
create index i_athlete_nation_code_event on athlete
(nation_code, event);

/* for hash join */
create index i_nation_continent_code on nation (continent,
code);

select /*+ recompile use_hash index_ss */
    n.name, a.name
from
    nation as n
    inner join athlete as a on n.code = a.nation_code
```

```
where
    n.continent = 'Asia'
    and a.event = 'Baseball';

drop index i_athlete_nation_code_event on athlete;
drop index i_nation_continent_code on nation;
```

```
Query plan:

hash-join (inner join)
    edge:  term[0]
    outer: iscan
                class: n node[0]
                index: i_nation_continent_code term[1]
                cost:  2 card 1
    inner: iscan
                class: a node[1]
                index: i_athlete_nation_code_event term[2]
(index skip scan)
                cost:  13 card 202
    cost:  25 card 1
```

```
Trace Statistics:
  SELECT (time: 1, fetch: 748, fetch_time: 0, ioread: 0)
    SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 103,
rows: 103)
      HASHJOIN (time: 1, fetch: 748, fetch_time: 0, ioread:
0)
        BUILD (time: 0, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: memory)
          SELECT (time: 0, fetch: 52, fetch_time: 0,
ioread: 0)
            SCAN (index:
public.nation.i_nation_continent_code), (btree time: 0,
fetch: 49, ioread: 0, readkeys: 47, filteredkeys: 0, rows:
47) (lookup time: 0, rows: 47)
        PROBE (time: 1, probe_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, readkeys: 103, rows: 103,
max_collisions: 1)
          SELECT (time: 1, fetch: 696, fetch_time: 0,
ioread: 0)
            SCAN (index:
public.athlete.i_athlete_nation_code_event), (btree time:
1, fetch: 692, ioread: 0, readkeys: 121, filteredkeys: 0,
rows: 229) (lookup time: 0, rows: 229)
```

Example 4. Hash Right Outer Join

This example illustrates how the direction of an outer join determines which table is used to build the hash table.

```
select /*+ recompile use_hash */
    n.name, p.host_year, p.gold
from
    participant as p
    right outer join nation as n on n.code = p.nation_code
and p.gold > 0
where
    n.continent = 'Asia';
```

```
Query plan:

hash-join (right outer join)
    edge:  term[0]
    outer: sscan
                class: p node[0]
                sargs: term[1]
                cost:  5 card 92
    inner: sscan
                class: n node[1]
                sargs: term[2]
                cost:  2 card 31
    cost:  15 card 31
```

```
Trace Statistics:
  SELECT (time: 1, fetch: 4, fetch_time: 0, ioread: 0)
    SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 70,
rows: 70)
      HASHJOIN (time: 0, fetch: 4, fetch_time: 0, ioread: 0)
        BUILD (time: 0, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: memory)
          SELECT (time: 0, fetch: 3, fetch_time: 0, ioread:
0)
            SCAN (table: public.participant), (heap time:
0, fetch: 3, ioread: 0, readrows: 916, rows: 228)
        PROBE (time: 0, probe_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, readkeys: 36, rows: 70,
max_collisions: 5)
          SELECT (time: 0, fetch: 1, fetch_time: 0, ioread:
0)
            SCAN (table: public.nation), (heap time: 0,
fetch: 1, ioread: 0, readrows: 215, rows: 47)
```

The participant table contains 228 records that satisfy the filter condition (`p.gold > 0`), while the nation table contains 47 records that meet the filter condition (`n.continent = 'Asia'`). Even though

the participant table has more records, in a right outer join, the left table (participant) is used to build the hash table.

Example 5. Hash Outer Join Skipping Hash Table Build

This example demonstrates a hash outer join where the hash table build is skipped. In a left outer join, the right table is typically used to build the hash table. However, since no records in the participant table satisfy the filter condition (`p.gold > 100`), the hash table build is skipped. Instead, records from the nation table that satisfy the filter condition (`n.continent = 'Asia'`) are directly returned as the query result.

```
select /*+ recompile use_hash */
    n.name, p.host_year, p.gold
from
    nation as n
    left outer join participant as p on n.code =
p.nation_code and p.gold > 100
where
    n.continent = 'Asia';
```

```
Query plan:

hash-join (left outer join)
    edge:  term[0]
    outer: sscan
                class: n node[0]
                sargs: term[2]
                cost:  2 card 31
    inner: sscan
                class: p node[1]
                sargs: term[1]
                cost:  5 card 92
    cost:  15 card 31
```

```
Trace Statistics:
  SELECT (time: 1, fetch: 4, fetch_time: 0, ioread: 0)
    SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 47,
rows: 47)
      HASHJOIN (time: 1, fetch: 4, fetch_time: 0, ioread: 0)
        BUILD (time: 1, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: skip)
          SELECT (time: 1, fetch: 3, fetch_time: 0, ioread:
0)
            SCAN (table: public.participant), (heap time:
1, fetch: 3, ioread: 0, readrows: 916, rows: 0)
        PROBE (time: 0, probe_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, readkeys: 0, rows: 47,
```

```
max_collisions: 0)
          SELECT (time: 0, fetch: 1, fetch_time: 0, ioread:
0)
            SCAN (table: public.nation), (heap time: 0,
fetch: 1, ioread: 0, readrows: 215, rows: 47)
```

In the **BUILD** section, the hash table build method is displayed as `hash_method: skip`. This indicates that the hash table was not built. Instead, the records from the nation table satisfying the filter condition (`n.continent = 'Asia'`) constitute the query result.

Example 6. Hash Join Using the Hybrid Hash Method

This example demonstrates how to perform a hash join using the hybrid hash method. Because the data set is small, the hash table's maximum size must be reduced to use the hybrid hash method. To do this, set the **max_hash_list_scan_size** parameter to **12K** (default: **8M**).

```
set system parameters 'max_hash_list_scan_size=12k';

select /*+ recompile use_hash */
    count (*)
from
    nation as n
    inner join athlete as a on n.code = a.nation_code;

set system parameters 'max_hash_list_scan_size=8M';
```

```
Query plan:

hash-join (inner join)
    edge:  term[0]
    outer: sscan
               class: n node[0]
               cost:  2 card 215
    inner: sscan
               class: a node[1]
               cost:  49 card 6677
    cost:  400 card 6677
```

```
Trace Statistics:
  SELECT (time: 5, fetch: 126, fetch_time: 0, ioread: 0)
    SCAN (temp time: 0, fetch: 19, ioread: 0, readrows:
6677, rows: 6677)
      HASHJOIN (time: 5, fetch: 105, fetch_time: 0, ioread:
0)
        BUILD (time: 0, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: hybrid)
          SELECT (time: 0, fetch: 1, fetch_time: 0, ioread:
```

```
0)
            SCAN (table: public.nation), (heap time: 0,
fetch: 1, ioread: 0, readrows: 215, rows: 215)
        PROBE (time: 5, probe_time: 2, fetch: 49,
fetch_time: 0, ioread: 0, readkeys: 6692, rows: 6677,
max_collisions: 2)
          SELECT (time: 3, fetch: 53, fetch_time: 0,
ioread: 0)
            SCAN (table: public.athlete), (heap time: 2,
fetch: 32, ioread: 0, readrows: 6677, rows: 6677)
```

When more than one join hint is used on the same table, the query optimizer compares the costs of the specified join methods and selects the one with the lower cost. The selected join method can be verified through the generated execution plan. The following is an example where the **USE_HASH** and **USE_NL** hints are used together, and the query optimizer selects the hybrid hash method, generating and executing the execution plan as a hash join.

```
set system parameters 'max_hash_list_scan_size=12k';

select /*+ recompile use_hash use_nl */
    count (*)
from
    nation as n
    inner join athlete as a on n.code = a.nation_code;

set system parameters 'max_hash_list_scan_size=8M';
```

```
Query plan:

hash-join (inner join)
    edge:  term[0]
    outer: sscan
               class: n node[0]
               cost:  2 card 215
    inner: sscan
               class: a node[1]
               cost:  49 card 6677
    cost:  400 card 6677
```

```
Trace Statistics:
  SELECT (time: 5, fetch: 126, fetch_time: 0, ioread: 0)
    SCAN (temp time: 1, fetch: 19, ioread: 0, readrows:
6677, rows: 6677)
      HASHJOIN (time: 4, fetch: 105, fetch_time: 0, ioread:
0)
        BUILD (time: 0, build_time: 0, fetch: 0,
fetch_time: 0, ioread: 0, hash_method: hybrid)
```

```
          SELECT (time: 0, fetch: 1, fetch_time: 0, ioread:
0)
            SCAN (table: public.nation), (heap time: 0,
fetch: 1, ioread: 0, readrows: 215, rows: 215)
        PROBE (time: 4, probe_time: 2, fetch: 49,
fetch_time: 0, ioread: 0, readkeys: 6692, rows: 6677,
max_collisions: 2)
          SELECT (time: 2, fetch: 53, fetch_time: 0,
ioread: 0)
            SCAN (table: public.athlete), (heap time: 2,
fetch: 32, ioread: 0, readrows: 6677, rows: 6677)
```

Example 7. Hash Join Using the File Hash Method

This example demonstrates how to perform a hash join using the file hash method. Because the data set is small, the hash table's maximum size must be reduced to below **12K** to use the file hash method. To do this, set the **max_hash_list_scan_size** parameter to **8K** (default: **8M**).

```
set system parameters 'max_hash_list_scan_size=8k';

select /*+ recompile use_hash */
    count (*)
from
    nation as n
    inner join athlete as a on n.code = a.nation_code;

set system parameters 'max_hash_list_scan_size=8M';
```

```
Query plan:

hash-join (inner join)
    edge:  term[0]
    outer: sscan
               class: n node[0]
               cost:  2 card 215
    inner: sscan
               class: a node[1]
               cost:  49 card 6677
    cost:  3846 card 6677
```

The query optimizer evaluates whether memory hashing or hybrid hashing can be used by calculating based on statistics collected by the database:

```
Temporary result record count * (Hash entry size + Location
info size) <= Maximum hash table size
                                          215 * (32 byte
* 16 byte) <= 8,192 byte
```

```
    10,320 <= 8192 byte (False)
```

Since the calculated size exceeds the maximum size of the hash table, the file hash method is chosen instead of the memory or hybrid method. File hash methods (cost: 3,846) are more expensive than other hash methods (cost: 400) due to the use of temporary volume pages.

```
Trace Statistics:
  SELECT (time: 10, fetch: 27510, fetch_time: 3, ioread: 0)
    SCAN (temp time: 0, fetch: 19, ioread: 0, readrows:
6677, rows: 6677)
      HASHJOIN (time: 10, fetch: 27489, fetch_time: 3,
ioread: 0)
        BUILD (time: 1, build_time: 1, fetch: 645,
fetch_time: 1, ioread: 0, hash_method: file)
          SELECT (time: 0, fetch: 1, fetch_time: 0, ioread:
0)
            SCAN (table: public.nation), (heap time: 0,
fetch: 1, ioread: 0, readrows: 215, rows: 215)
        PROBE (time: 9, probe_time: 7, fetch: 26772,
fetch_time: 2, ioread: 0, readkeys: 6692, rows: 6677,
max_collisions: 2)
          SELECT (time: 2, fetch: 53, fetch_time: 0,
ioread: 0)
            SCAN (table: public.athlete), (heap time: 1,
fetch: 32, ioread: 0, readrows: 6677, rows: 6677)
```

When compared with the query profiling results from Example 6, It can be seen that the execution time of the file hash method is slow in proportion to the cost.

When more than one join hint is used on the same table, the query optimizer compares the costs of the specified join methods and selects the one with the lower cost. The selected join method can be verified through the generated execution plan. The following is an example where the **USE_HASH** and **USE_NL** hints are used together, and the query optimizer generates and executes a plan with a nested loop join (cost: 3,405) because the cost of a hash join (cost: 3,846) using the file hash method) is high.

```
set system parameters 'max_hash_list_scan_size=8k';

select /*+ recompile use_hash use_nl */
    count (*)
from
    nation as n
    inner join athlete as a on n.code = a.nation_code;

set system parameters 'max_hash_list_scan_size=8M';
```

```
Query plan:

idx-join (inner join)
    outer: sscan
                class: a node[1]
                cost:  49 card 6677
    inner: iscan
                class: n node[0]
                index: pk_nation_code term[0] (covers)
                cost:  2 card 215
    cost:  3405 card 6677
```

```
Trace Statistics:
  SELECT (time: 19, fetch: 13387, fetch_time: 1, ioread: 0)
    SCAN (table: public.athlete), (heap time: 4, fetch:
6709, ioread: 0, readrows: 6677, rows: 6677)
      SCAN (index: public.nation.pk_nation_code), (btree
time: 10, fetch: 6677, ioread: 0, readkeys: 6677,
filteredkeys: 0, rows: 0, covered: true, count_only: true)
```

Updating Statistics

Statistics for tables and indexes enables queries of the database system to process efficiently. Statistics are not updated automatically for DDL statements such as **CREATE INDEX**, **CREATE TABLE** and DML statements such as **INSERT** and **DELETE**. **UPDATE STATISTICS** statement is the only way to update statistics. So it is necessary to update the statistics by **UPDATE STATISTICS** statement(See Checking Statistics Information).

UPDATE STATISTICS statement is recommended to be executed periodically. It is also recommended to execute when a new index is added or when a mass of **INSERT** or **DELETE** statements make the big difference between the statistics and the actual information.

When updating statistics, the execution plan cache related to those statistics is not deleted. The execution plan is regenerated when the following two criteria are met during query execution.

1. 6 minutes have passed since the execution plan cache creation or regeneration check
2. The page size increases or decreases by more than 10 times, and statistics are updated

The user can delete plan cache using the **PLANDUMP** utility. For more information about **PLANDUMP**, see plandump.

```
UPDATE STATISTICS ON [schema_name.]class-name [{,
[schema_name.]class-name}] [WITH FULLSCAN];

UPDATE STATISTICS ON ALL CLASSES [WITH FULLSCAN];

UPDATE STATISTICS ON CATALOG CLASSES [WITH FULLSCAN];
```

- **WITH FULLSCAN**: It updates the statistics with all the data in the specified table. If this is omitted, it updates the statistics with sampling data. The sampling data is 5000 pages regardless of total pages of table.
- **ALL CLASSES**: If the **ALL CLASSES** keyword is specified, the statistics on all the tables existing in the database are updated.
- **CATALOG CLASSES**: It updates the statistics of the catalog tables.

```
CREATE TABLE foo (a INT, b INT);
CREATE INDEX idx1 ON foo (a);
CREATE INDEX idx2 ON foo (b);

UPDATE STATISTICS ON foo;
UPDATE STATISTICS ON foo WITH FULLSCAN;

UPDATE STATISTICS ON ALL CLASSES;
UPDATE STATISTICS ON ALL CLASSES WITH FULLSCAN;

UPDATE STATISTICS ON CATALOG CLASSES;
UPDATE STATISTICS ON CATALOG CLASSES WITH FULLSCAN;
```

When starting and ending an update of statistics information, NOTIFICATION message is written on the server error log. You can check the updating term of statistics information by these two messages.

```
Time: 05/07/13 15:06:25.052 - NOTIFICATION *** file ../../src/
storage/statistics_sr.c, line 123  CODE = -1114 Tran = 1, CLIENT =
testhost:csql(21060), EID = 4
Started to update statistics (class "code", oid : 0|522|3).

Time: 05/07/13 15:06:25.053 - NOTIFICATION *** file ../../src/
storage/statistics_sr.c, line 330  CODE = -1115 Tran = 1, CLIENT =
testhost:csql(21060), EID = 5
Finished to update statistics (class "code", oid : 0|522|3, error
code : 0).
```

Note

From version 10.0 of CUBRID, on the HA environment, **UPDATE STATISTICS** on the master node is replicated to the slave/replica node.

Note

From version 11.3 of CUBRID, synonyms cannot be used when executing **UPDATE STATISTICS** statement.

```
/* CURRENT_USER: PUBLIC */
CREATE TABLE t (c int);
CREATE SYNONYM s for t;

UPDATE STATISTICS ON t;
/* Execute OK. */

UPDATE STATISTICS ON s;
/* ERROR: before ' ; '
 * Class public.s does not exist. */
```

Note

From version 11.4 of CUBRID, **SELECT** authorization is required when executing **UPDATE STATISTICS** statement.

Checking Statistics Information

You can check the statistics Information with the session command of the CSQL Interpreter.

```
csql> ;info stats table_name
```

- *schema_name*: Specifies the schema of the table. If omitted, the schema name of the current session is used.
- *table_name*: Table name to check the statistics Information

The following shows the statistical information of *t1* table in CSQL interpreter.

```
CREATE TABLE t1 (code INT, name VARCHAR(20));
INSERT INTO t1 VALUES(1,'Park'),(2,'Park'),(3,'Joo'),(4,'Joo'),
(5,'Song');
CREATE INDEX i_t1_code ON t1(code,name);
UPDATE STATISTICS ON t1;
;info stats t1
```

```
CLASS STATISTICS
*****************
 Class name: t1 Timestamp: Thu Dec 19 15:15:10 2024
 Total pages in class heap: 1
 Total objects: 5
 Number of attributes: 2
 Attribute: code (integer)
    Number of Distinct Values: 5
    B+tree statistics:
        BTID: { 0 , 5760 }
        Cardinality: 5 (5,5) , Total pages: 3 , Leaf pages: 1 ,
Height: 2

 Attribute: name (character varying)
    Number of Distinct Values: 3
```

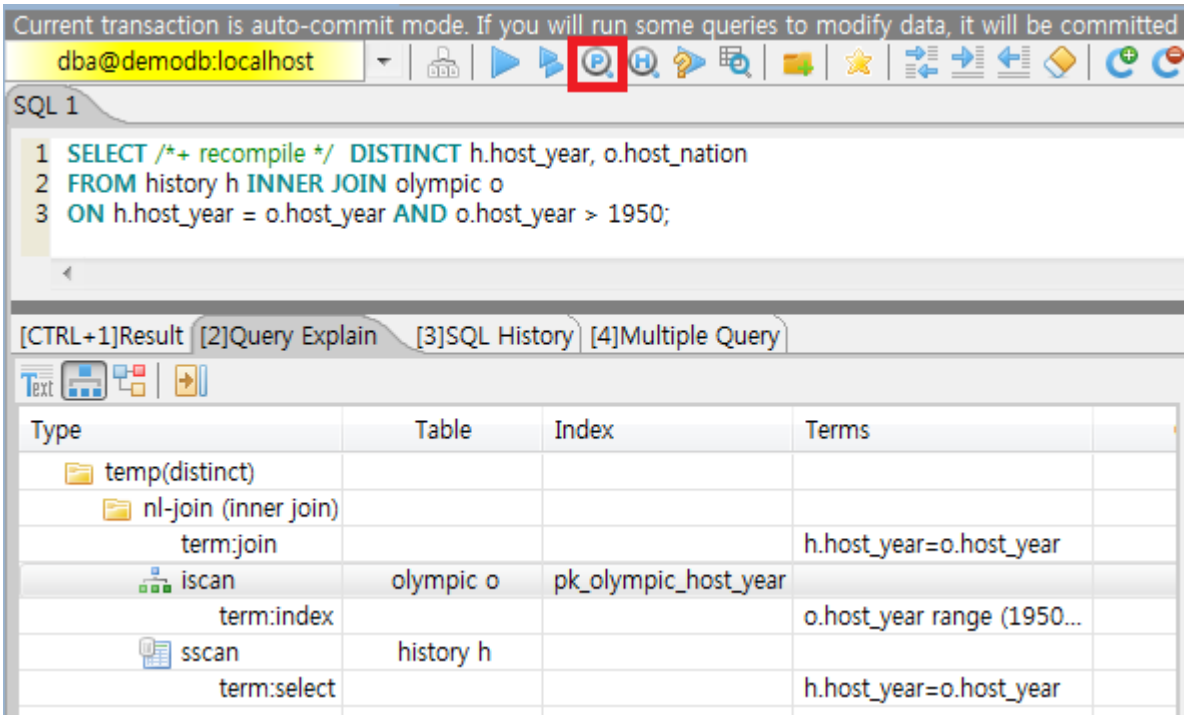
- *Timestamp*: The time when the statistics were updated.
- *Total pages*: The number of pages in the table.
- *Total objects*: The total number of rows in the table.
- *Number of Distinct Values*: The number of values, with duplicates removed. The column **code** consists of all different values, so the NDV is 5. If type of the column is **LOB** or **VARCHAR** exceeding 4,000 characters, NDV is not generated. NDV is used to calculate **SELECTIVITY** in the optimizer. For more information, see Optimizer Principle.
- B+tree statistics: Index statistics
  - *B+tree Cardinality*: The number of values from which the accumulated duplicates of the index key values have been removed. In the example above, **(5,5)** matches the index column **(code,name)**. The first **5** is the number of values from which the duplicates of the **code** column have been removed, and the second **5** is the number of values from which the duplicates of the two columns **code,name** have been removed.
  - *Total pages*: The total number of index pages.
  - *Leaf pages*: The number of pages in the index leaf block.

- *Height*: The height of the B+tree index including the leaf block.

Viewing Query Plan

To view a query plan for a CUBRID SQL query, you can use following methods.

- Press “show plan” button on CUBRID Manager.



- Change the value of the optimization level by running “;plan simple” or “;plan detail” on CSQL interpreter, or by using the **SET OPTIMIZATION** statement. You can get the current optimization level value by using the **GET OPTIMIZATION** statement. For details on CSQL Interpreter, see Session Commands.

SET OPTIMIZATION or **GET OPTIMIZATION LEVEL** syntax is as following.

```
SET OPTIMIZATION LEVEL opt-level [;]
GET OPTIMIZATION LEVEL [ { TO | INTO } variable ] [;]
```

- *opt-level* : This value specifies the optimization level and is associated with the following settings. Any other value will result in an error.
  - 1: Creates a query plan by performing query optimization and executes the query. This is a default value used in CUBRID, and does not have to be changed in most cases.
  - 2: Creates a query plan by performing query optimization. However, the query itself is not executed. In general, this value is not used; it is used together with the following values to be set for viewing query plans.

- 257: Performs query optimization and outputs the created query plan. This value works for displaying the query plan by internally interpreting the value as 256+1 related with the value 1.
- 258: Performs query optimization and outputs the created query plan, but does not execute the query. That is, this value works for displaying the query plan by internally interpreting the value as 256+2 related with the value 2. This setting is useful to examine the query plan but not to intend to see the query results.
- 513: Performs query optimization and outputs the detailed query plan. This value works for displaying more detailed query plan than the value 257 by internally interpreting the value as 512+1.
- 514: Performs query optimization and outputs the detailed query plan. However, the query is not executed. This value works for displaying more detailed query plan than the value 258 by internally interpreting the value as 512+2.

Note

If you configure the optimization level as not executing the query like 2, 258, or 514, all queries(not only SELECT, but also INSERT, UPDATE, DELETE, REPLACE, TRIGGER, SERIAL, etc.) are not executed.

The CUBRID query optimizer determines whether to perform query optimization and output the query plan by referring to the optimization level value set by the user.

The following shows the result which ran the query after inputting ";plan simple" or "SET OPTIMIZATION LEVEL 257;" in CSQL.

```
SET OPTIMIZATION LEVEL 257;
--  csql> ;plan simple
SELECT /*+ RECOMPILE */  DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o
ON h.host_year = o.host_year AND o.host_year > 1950;
```

```
Query plan:

 Sort(distinct)
    Nested-loop join(h.host_year=o.host_year)
        Index scan(olympic o, pk_olympic_host_year, (o.host_year> ?:
```

```
0 ))
        Sequential scan(history h)
```

- Sort(distinct): Perform DISTINCT.
- Nested-loop join: Join method is Nested-loop.
- Index scan: Perform index-scan by using pk_olympic_host_year index about olympic table. At that time, the condition which used this index is “o.host_year > ?”.

The following shows the result which ran the query after inputting “;plan detail” or “SET OPTIMIZATION LEVEL 513;” in CSQL.

```
SET OPTIMIZATION LEVEL 513;
--  csql> ;plan detail

SELECT /*+ RECOMPILE */  DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o
ON h.host_year = o.host_year AND o.host_year > 1950;
```

```
Join graph segments (f indicates final):
seg[0]: [0]
seg[1]: host_year[0] (f)
seg[2]: [1]
seg[3]: host_nation[1] (f)
seg[4]: host_year[1]
Join graph nodes:
node[0]: history h(147/1)
node[1]: olympic o(25/1) (sargs 1)
Join graph equivalence classes:
eqclass[0]: host_year[0] host_year[1]
Join graph edges:
term[0]: h.host_year=o.host_year (sel 0.04) (join term) (mergeable)
(inner-join) (indexable host_year[1]) (loc 0)
Join graph terms:
term[1]: o.host_year range (1950 gt_inf max) (sel 0.1) (rank 2)
(sarg term) (not-join eligible) (indexable host_year[1]) (loc 0)

Query plan:

temp(distinct)
    subplan: nl-join (inner join)
                edge:  term[0]
                outer: iscan
                           class: o node[1]
                           index: pk_olympic_host_year term[1]
                           cost:  1 card 2
                inner: sscan
                           class: h node[0]
```

```
                            sargs: term[0]
                            cost:  1 card 147
                  cost:  3 card 15
    cost:  9 card 15

Query stmt:

select distinct h.host_year, o.host_nation from history h, olympic o
where h.host_year=o.host_year and (o.host_year> ?:0 )
```

On the above output, the information which is related to the query plan is “Query plan:”. Query plan is performed sequentially from the inside above line. In other words, “outer: iscan -> inner:scan” is repeatedly performed and at last, “temp(distinct)” is performed. “Join graph segments” is used for checking more information on “Query plan:”. For example, “term[0]” in “Query plan:” is represented as “term[0]: h.host_year=o.host_year (sel 0.04) (join term) (mergeable) (inner-join) (indexable host_year[1]) (loc 0)” in “Join graph segments”.

The following shows the explanation of the above items of “Query plan:”.

- temp(distinct): (distinct) means that CUBRID performs DISTINCT query. temp means that it saves the result to the temporary space.
  - nl-join: “nl-join” means nested loop join.
  - (inner join): join type is “inner join”.
    - outer: iscan: performs iscan(index scan) in the outer table.
      - class: o node[1]: It uses o table. For details, see node[1] of “Join graph segments”.
      - index: pk_olympic_host_year term[1]: use pk_olympic_host_year index and for details, see term[1] of “Join graph segments”.
      - cost: a cost to perform this syntax.
        - card: It means cardinality. Note that this is an approximate value.
    - inner: sscan: It performs sscan(sequential scan) in the inner table.
      - class: h node[0]: It uses h table. For details, see node[0] of “Join graph segments”.
      - sargs: term[0]: sargs represent data filter(WHERE condition which does not use an index); it means that term[0] is the condition used as data filter.
      - cost: A cost to perform this syntax.
        - card: It means cardinality. Note that this is an approximate value.

    - cost: A cost to perform all syntaxes. It includes the previously performed cost.
        - card: It means cardinality. Note that this is an approximate value.

Query Plan Related Terms

The following show the meaning for each term which is printed as a query plan.

- Join method: It is printed as “nl-join” on the above. The following are the join methods which are printed on the query plan.
    - nl-join: Nested loop join
    - m-join: Sort merge join
    - idx_join: Nested loop join, and it is a join which uses an index in the inner table as reading rows of the outer table.
    - hash-join: Hash join
- Join type: It is printed as “(inner join)” on the above. The following are the join types which are printed on the query plan.
    - inner join
    - left outer join
    - right outer join: On the query plan, the different “outer” direction with the query’s direction can be printed. For example, even if you specified “right outer” on the query, but “left outer” can be printed on the query plan.
    - cross join
- Types of join tables: It is printed as outer or inner on the above. They are separated as outer table and inner table which are based on the position on either side of the loop, on the nested loop join.
    - outer table: The first base table to read when joining.
    - inner table: The target table to read later when joining.
- Scan method: It is printed as iscan or sscan. You can judge that if the query uses index or not.
    - sscan: sequential scan. Also it can be called as full table scan; it scans all of the table without using an index.
    - iscan: index scan. It limits the range to scan by using an index.
- cost: It internally calculate the cost related to CPU, IO etc., mainly the use of resources.

- card: It means cardinality. It is a number of rows which are predicted as selected.

The following is an example of performing m-join(sort merge join) as specifying USE_MERGE hint. In general, sort merge join is used when sorting and merging an outer table and an inner table is judged as having an advantage than performing nested loop join. In most cases, it is desired that you do not perform sort merge join.

Note

From 9.3 version, if USE_MERGE hint is not specified or the **optimizer_enable_merge_join** parameter of cubrid.conf is not specified as yes, sort merge join will not be considered to be applied.

```
SET OPTIMIZATION LEVEL 513;
-- csql> ;plan detail

SELECT /*+ RECOMPILE USE_MERGE*/  DISTINCT h.host_year, o.host_nation
FROM history h LEFT OUTER JOIN olympic o ON h.host_year =
o.host_year AND o.host_year > 1950;
```

```
Query plan:

temp(distinct)
    subplan: temp
                order: host_year[0]
                subplan: m-join (left outer join)
                             edge:  term[0]
                             outer: temp
                                        order: host_year[0]
                                        subplan: sscan
                                                    class: h
node[0]
                                                    cost:  1 card
147
                                        cost:  10 card 147
                             inner: temp
                                        order: host_year[1]
                                        subplan: iscan
                                                    class: o
node[1]
                                                    index:
pk_olympic_host_year term[1]
                                                    cost:  1 card 2
                                        cost:  7 card 2
                             cost:  18 card 147
```

```
                cost:  24 card 147
    cost:  30 card 147
```

The following performs the idx-join(index join). If performing join by using an index of inner table is judged as having an advantage, you can ensure performing idx-join by specifying **USE_IDX** hint.

```
SET OPTIMIZATION LEVEL 513;
-- csql> ;plan detail

CREATE INDEX i_history_host_year ON history(host_year);

SELECT /*+ RECOMPILE */  DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o ON h.host_year = o.host_year;
```

```
Query plan:

temp(distinct)
    subplan: idx-join (inner join)
                outer: sscan
                           class: o node[1]
                           cost:  1 card 25
                inner: iscan
                           class: h node[0]
                           index: i_history_host_year term[0]
(covers)
                           cost:  1 card 147
                cost:  2 card 147
    cost:  9 card 147
```

On the above query plan, "(covers)" is printed on the "index: i_history_host_year term[0]" of "inner: iscan", it means that Covering Index functionality is applied. In other words, it does not retrieve data storage additionally because there are required data inside the index in inner table.

If you ensure that left table's row number is a lot smaller than the right table's row number on the join tables, you can specify **ORDERED** hint. Then always the left table will be outer table, and the right table will be inner table.

```
SELECT /*+ RECOMPILE ORDERED */  DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o ON h.host_year = o.host_year;
```

You can specify the table order by specifying the **LEADING** hint. Unlike the **ORDERED** hint, it can be specified regardless of the order in which the tables are written.

```
SELECT /*+ RECOMPILE LEADING(o,h) */  DISTINCT h.host_year,
o.host_nation
FROM history h INNER JOIN olympic o ON h.host_year = o.host_year;
```

Optimizer Principle

CUBRID's optimizer performs cost-based optimization when generating a query execution plan, and calculates **selectivity** and **expected number of rows and pages** through statistics. Based on this, it calculates the cost and selects the execution plan with the lowest cost.

Selectivity

Selectivity is the ratio of data to be selected when a specific predicate is evaluated. The optimizer calculates the selectivity for each predicate in the **WHERE** clause. **CUBRID** assumes that the entire data is evenly distributed and calculates the **selectivity** using the **Number of Distinct Value** of statistics.

```
SELECTIVITY = 1 / Number of Distinct Value
```

```
CREATE TABLE t1 (code INT, name VARCHAR(20));
INSERT INTO t1 VALUES(1,'Park'),(2,'Park'),(3,'Park'),(4,'joo'),
(5,'joo'),(5,'joo');
CREATE INDEX i_t1_code ON t1(code,name);
UPDATE STATISTICS ON t1;

;info stats t1
```

```
CLASS STATISTICS
****************
 Class name: t1 Timestamp: Fri Dec 20 13:07:58 2024
 Total pages in class heap: 1
 Total objects: 6
 Number of attributes: 2
 Attribute: code (integer)
    Number of Distinct Values: 5
    B+tree statistics:
        BTID: { 0 , 5760 }
        Cardinality: 5 (5,5) , Total pages: 3 , Leaf pages: 1 ,
Height: 2

 Attribute: name (character varying)
    Number of Distinct Values: 2
```

```
;plan detail
SELECT /*+ recompile */ * FROM t1 WHERE code = 3 AND name IN
('Song', 'Ham');
```

```
Join graph terms:
term[0]: [dba.t1].code=3 (sel 0.2)
term[1]: [dba.t1].[name] range ('Ham' =  or 'Song' = ) (sel 0.75)
```

term[0]: [dba.t1].code=3 The **Number of Distinct Values** of **code** in the statistics is 5, so the selectivity is 1/5 or 0.2. **term[1]: [dba.t1].[name] range ('Ham' = or 'Song' =)** performs an **OR** operation on two values of name. Since the **Number of Distinct Values** for **name** is 2, the selectivity for each value is 1/2. Adding the two values and subtracting the intersection gives the calculation **0.5 + 0.5 - (0.5 x 0.5)**, resulting in 0.75.

Expected number of rows

The **expected number of rows** is calculated by multiplying the **selectivity** by the **total number of data rows**.

```
Number of expected rows = total rows of a table * selectivity
```

```
;plan detail
SELECT /*+ recompile */ * FROM t1 WHERE name = 'Park';
```

```
Join graph nodes:
node[0]: dba.t1 dba.t1(6/1)
Join graph terms:
term[0]: [dba.t1].[name]='Park' (sel 0.5)

Query plan:

sscan
    class: t1 node[0]
    sargs: term[0]
    cost:  1 card 3
```

dba.t1(6/1) indicates that the total number of rows in the t1 table is 6 and the number of pages is 1. Since the selectivity of **[dba.t1].[name]='Park'** is 0.5, the expected number of rows is 3, which is calculated as **6 x 0.5**. **cost: 1 card 3** in the execution plan indicates that the cost is 1 and the expected number of rows is 3.

Cost of sequential scan

When estimating costs, the optimizer considers two things:

1. The cost of reading a page
2. The CPU cost of repetitive routines

The following example shows how the cost of a sequential scan is calculated.

```
Page cost = number of pages in table
CPU cost = total number of rows in table X CPU weight
```

```
drop table if exists t1;
create table t1 (col1 int, col2 int, col3 int, col4 int);
insert into t1 select mod(rownum,2), mod(rownum,4), rownum, rownum
from dual connect by level <= 4000;
update statistics on t1;

;plan detail
select /*+ recompile */ count(*) from t1 where col2 = 2;
```

```
Join graph nodes:
node[0]: dba.t1 dba.t1(4000/9)
Join graph terms:
term[0]: [dba.t1].col2=2 (sel 0.25)

Query plan:

sscan
    class: t1 node[0]
    sargs: term[0]
    cost:  19 card 1000
```

node[0]: dba.t1 dba.t1(4000/9) shows that the total number of rows in the t1 table is 4000 and the number of pages is 9. Therefore, the page cost is 9, which is the total number of pages in the t1 table. The CPU cost is the total number of rows in the table multiplied by the CPU weight, which is **4000 x 0.0025**, resulting in a cost of 10. Adding the two values gives 19, and from **cost: 19 card 1000**, we can see that the cost is 19 and the expected number of rows to be retrieved is 1000.

Cost of index scan

Index scan is performed by searching non-leaf nodes, leaf nodes, and heap areas. The index scan starts by searching non-leaf nodes including the root node to find the starting point of the leaf node. Then, it searches the leaf nodes, looks up the satisfying key, and scans the heap area with the **OID**. The following example shows how the index scan is performed.

```
drop table if exists t2;
create table t2 (col1 int, col2 int, col3 int, col4 int);
insert into t2 select mod(rownum,20), mod(rownum,80), rownum, rownum
from dual connect by level <= 4000;
create index idx on t2(col1,col2,col3);
update statistics on t2;

;plan detail
select /*+ recompile */ count(*) from t2 where col1 = 1 and col3 = 1
and col4 = 1;
```

```
Join graph terms:
term[0]: [dba.t2].col4=1 (sel 0.00025)
term[1]: [dba.t2].col3=1 (sel 0.0125)
term[2]: [dba.t2].col1=1 (sel 0.05)

Query plan:

iscan
    class: t2 node[0]
    index: idx term[2]
    filtr: term[1]
    sargs: term[0]
    cost:  4 card 1
```

You can check the predicates in the execution plan through the information in **Join graph terms**. **index: idx term[2]** is the predicate for searching the non-leaf node during the index vertical scan. The **filtr: term[1]** predicate is for filtering the key in the leaf node during the index horizontal scan. When accessing the heap and filtering data, the **sargs: term[0]** predicate is used to filter.

```
;info stats t2
```

```
Attribute: col1 (integer)
   Number of Distinct Values: 20
   B+tree statistics:
       BTID: { 0 , 5760 }
       Cardinality: 4000 (20,4000,4000) , Total pages: 11 , Leaf
pages: 9 , Height: 2
```

The cost of an index scan is calculated as follows:

1. Cost of reading a page = predicted non-leaf node pages accessed + leaf node pages + heap pages

2. CPU cost of a repeated routine = predicted number of leaf node keys accessed

The number of pages in a non-leaf node is 1 because the index **Height** of the statistics is 2 and the number of pages to be read excluding the leaf node is **2 - 1**. The number of leaf node pages is obtained by multiplying **Leaf pages** by the selectivity of the predicate **index: idx term[2]** used to search the non-leaf node. Since it is **MAX(9 X 0.05, 1)**, the number of leaf node pages to be read is 1. The number of heap pages to be read is obtained by multiplying the total number of pages in the table by the selectivity of **index** and **filtr** conditions. Here, it is calculated as **MAX(9 X 0.05 X 0.0125, 1)**, so it is calculated that 1 page must be read. Finally, the CPU cost is obtained by multiplying the selectivity of **index: idx term[2]** by the CPU weight from the total number of rows in the table. If you calculate it as **MAX(4000 X 0.05 X 0.0025, 1)**, you get 1, and if you add up all the results so far, you can see that the cost is 4. If you look at **cost: 4 card 1** in the execution plan, you can see that the cost is 4 and the expected number of rows is 1.

Cost of join

The cost of a join is generated by calculating the cost of the preceding table and the cost of the succeeding table separately, and the cost is calculated according to the join method. The following example below shows how the cost is calculated in a nested loop join method.

```
create index idx1 on t2(col4);

--;plan detail
select /*+ recompile */ count(*)
from t1 a, t2 b
where a.col4 = b.col4
and b.col1 = 1
and b.col3 = 1
and a.col2 = 2;
```

```
Query plan:

idx-join (inner join)
    outer: sscan
                class: a node[0]
                sargs: term[3]
                cost:  19 card 1000
    inner: iscan
                class: b node[1]
                index: idx1 term[0]
                sargs: term[1] AND term[2]
                cost:  2 card 3
    cost:  572 card 1
```

The preceding table is *a* and the succeeding table is *b*. Since the execution of the succeeding table is repeated as many times as the number of rows of the preceding table due to the nature of the nested loop join, the cost of the succeeding table can be calculated by multiplying the

variable cost of the *b* table by the expected number of rows of the *a* table. **CUBRID** internally manages fixed costs and variable costs separately, and this information cannot be displayed in the execution plan. The variable cost of the succeeding table is approximately *0.553*, and the cost of the succeeding table incurred during the join is 553 when calculated by *0.553 * 1000*, and the final cost is *572* when adding the cost of the preceding table *19*.

Query Profiling

If the performance analysis of SQL is required, you can use query profiling feature. To use query profiling, specify SQL trace with **SET TRACE ON** syntax; to print out the profiling result, run **SHOW TRACE** syntax.

And if you want to always include the query plan when you run **SHOW TRACE**, you need to add /*+ RECOMPILE */ hint on the query.

The format of **SET TRACE ON** syntax is as follows.

```
SET TRACE {ON | OFF} [OUTPUT {TEXT | JSON}]
```

- ON: set on SQL trace.
- OFF: set off SQL trace.
- OUTPUT TEXT: print out as a general TEXT format. If you omit OUTPUT clause, TEXT format is specified.
- OUTPUT JSON: print out as a JSON format.

As below, if you run **SHOW TRACE** syntax, the trace result is shown.

```
SHOW TRACE;
```

Below is an example that prints out the query tracing result after setting SQL trace ON.

```
csql> SET TRACE ON;
csql> SELECT /*+ RECOMPILE */ o.host_year, o.host_nation,
o.host_city, SUM(p.gold)
        FROM OLYMPIC o, PARTICIPANT p
        WHERE o.host_year = p.host_year AND p.gold > 20
        GROUP BY o.host_nation;
csql> SHOW TRACE;
```

```
=== <Result of SELECT Command in Line 2> ===
```

```
  trace
======================
  '
Query Plan:
  SORT (group by)
    NESTED LOOPS (inner join)
      TABLE SCAN (o)
      INDEX SCAN (p.fk_participant_host_year) (key range:
o.host_year=p.host_year)

  rewritten query: select o.host_year, o.host_nation, o.host_city,
sum(p.gold) from OLYMPIC o, PARTICIPANT p where
o.host_year=p.host_year and (p.gold> ?:0 ) group by o.host_nation

Trace Statistics:
  SELECT (time: 2, fetch: 975, fetch_time: 1, ioread: 2)
    SCAN (table: olympic), (heap time: 0, fetch: 26, ioread: 0,
readrows: 25, rows: 25)
      SCAN (index: participant.fk_participant_host_year), (btree
time: 1, fetch: 941, ioread: 2, readkeys: 5, filteredkeys: 5, rows:
916) (lookup time: 0, rows: 14)
    GROUPBY (time: 0, sort: true, page: 0, ioread: 0, rows: 5)
'
```

In the above example, under lines of “Trace Statistics:” are the result of tracing. Each items of tracing result are as below.

- **SELECT** (time: 2, fetch: 975, fetch_time: 1, ioread: 2)
  - time: 2 => Total query time took 2ms.
  - fetch: 975 => 975 times were fetched regarding pages. (not the number of pages, but the count of accessing pages. even if the same pages are fetched, the count is increased.).
  - fetch_time: 1=> Total fetch time took 1ms.
  - ioread: disk accessed 2 times.

  : Total statistics regarding SELECT query. If the query is rerun, fetching count and ioread count can be shrinken because some of query result are read from buffer.

  - **SCAN** (table: olympic), (heap time: 0, fetch: 26, ioread: 0, readrows: 25, rows: 25)
    - heap time: 0 => It took less than 1ms. CUBRID rounds off a value less than millisecond, so a time value less than 1ms is displayed as 0.
    - fetch: 26 => page fetching count is 26.
    - ioread: 0 => disk accessing count is 0.

- readrows: 25 => the number of rows read when scanning is 25.
- rows: 25 => the number of rows in result is 25.

: Heap scan statistics for the olympic table.

- **SCAN** (index: participant.fk_participant_host_year), (btree time: 1, fetch: 941, ioread: 2, readkeys: 5, filteredkeys: 5, rows: 916) (lookup time: 0, rows: 14)
  - btree time: 1 => It took 1ms.
  - fetch: 941 => page fetching count is 941.
  - ioread: 2 => disk accessing count is 2.
  - readkeys: 5 => the number of keys read is 5.
  - filteredkeys: 5 => the number of keys which the key filter is applied is 5.
  - rows: 916 => the number of rows scanning is 916.
  - lookup time: 0 => It took less than 1ms when accessing data after index scan.
  - rows: 14 => the number of rows after applying data filter; in the query, the number of rows is 14 when data filter “p.gold > 20” is applied.

  : Index scanning statistics regarding participant.fk_participant_host_year index.

- **GROUPBY** (time: 0, sort: true, page: 0, ioread: 0, rows: 5)
  - time: 0 => It took less than 1ms when “group by” is applied.
  - sort: true => It’s true because sorting is applied.
  - page: 0 => the number or temporary pages used in sorting is 0.
  - ioread: 0 => It took less than 1ms to access disk.
  - rows: 5 => the number of result rows regarding “group by” is 5.

: Group by statistics.

The following is an example to join 3 tables.

```
csql> SET TRACE ON;
csql> SELECT /*+ RECOMPILE ORDERED */ o.host_year, o.host_nation,
o.host_city, n.name, SUM(p.gold), SUM(p.silver), SUM(p.bronze)
        FROM OLYMPIC o,
             (select /*+ NO_MERGE */ * from PARTICIPANT p where
p.gold > 10) p,
             NATION n
```

```
      WHERE o.host_year = p.host_year AND p.nation_code = n.code
      GROUP BY o.host_nation;
csql> SHOW TRACE;

  trace
======================
'
Query Plan:
  TABLE SCAN (p)

  rewritten query: (select p.host_year, p.nation_code, p.gold,
p.silver, p.bronze from PARTICIPANT p where (p.gold> ?:0 ))

  SORT (group by)
    NESTED LOOPS (inner join)
      NESTED LOOPS (inner join)
        TABLE SCAN (o)
        TABLE SCAN (p)
      INDEX SCAN (n.pk_nation_code) (key range: p.nation_code=n.code)

  rewritten query: select /*+ ORDERED */ o.host_year, o.host_nation,
o.host_city, n.[name], sum(p.gold), sum(p.silver), sum(p.bronze)
from OLYMPIC o, (select p.host_year, p.nation_code, p.gold,
p.silver, p.bronze from PARTICIPANT p where (p.gold> ?:0 )) p
(host_year, nation_code, gold,
silver, bronze), NATION n where o.host_year=p.host_year and
p.nation_code=n.code group by o.host_nation


Trace Statistics:
  SELECT (time: 6, fetch: 880, fetch_time: 2, ioread: 0)
    SCAN (table: olympic), (heap time: 0, fetch: 104, ioread: 0,
readrows: 25, rows: 25)
      SCAN (hash temp(m), buildtime : 0, time: 0, fetch: 0, ioread:
0, readrows: 76, rows: 38)
        SCAN (index: nation.pk_nation_code), (btree time: 2, fetch:
760, ioread: 0, readkeys: 38, filteredkeys: 0, rows: 38) (lookup
time: 0, rows: 38)
    GROUPBY (time: 0, hash: true, sort: true, page: 0, ioread: 0,
rows: 5)
    SUBQUERY (uncorrelated)
      SELECT (time: 2, fetch: 12, ioread: 0)
        SCAN (table: participant), (heap time: 2, fetch: 12, ioread:
0, readrows: 916, rows: 38)
'
```

The following are the explanation regarding items of trace statistics.

SELECT

- time: total estimated time when this query is performed(ms)

- fetch: total page fetching count about this query
- fetch_time : total fetch time when this query is performed(ms)
- ioread: total I/O read count about this query. disk access count when the data is read

FUNC: Outputs the results of the stored procedure call accumulated from the current query and subqueries. The fetch and ioread counts do not include those that occur during the execution of queries within the stored procedure.

- time: total execution time of the stored procedure (ms)
- fetch: the number of times pages are fetched during the stored procedure call
- ioread: total I/O read count during the stored procedure call; indicates the number of disk accesses when reading data
- calls: the number of stored procedure calls

SCAN

- heap: data scanning job without index
  - time, fetch, ioread: the estimated time(ms), page fetching count and I/O read count in the heap of this operation
  - readrows: the number of rows read when this operation is performed
  - rows: the number of result rows when this operation is performed
- btree: index scanning job
  - time, fetch, ioread: the estimated time(ms), page fetching count and I/O read count in the btree of this operation
  - readkeys: the number of the keys which are read in btree when this operation is performed
  - filteredkeys: the number of the keys to which the key filter is applied from the read keys
  - rows: the number of result rows when this operation is performed; the number of result rows to which key filter is applied
- temp: data scanning job with temp file
  - hash temp(m): hash list scan or not. depending on the amount of data, the IN-MEMORY(m), HYBRID(h), FILE(f) hash data structure is used.
  - buildtime: the estimated time(ms) in building hash table.
  - time: the estimated time(ms) in probing hash table.

  - fetch, ioread: page fetching count and I/O read count in the temp file of this operation
  - readrows: the number of rows read when this operation is performed
  - rows: the number of result rows when this operation is performed

- lookup: data accessing job after index scanning
  - time: the estimated time(ms) in this operation
  - rows: the number of the result rows in this operation; the number of result rows to which the data filter is applied

- noscan: An operation that uses statistical information of index headers without scanning when executing an aggregate operation. (aggregate: count, min, max)
    - agl: aggregate lookup, index list used for aggregate operation

    The following is an example for noscan and agl.

```
SET TRACE ON;
CREATE TABLE agl_tbl (id INTEGER PRIMARY KEY, phone VARCHAR(20));
INSERT INTO agl_tbl VALUES (1, '123-456-789');
INSERT INTO agl_tbl VALUES (999, '999-999-999');

SELECT count(*), min(id), max(id) FROM agl_tbl;

SHOW TRACE;
```

```
Trace Statistics:
  SELECT (time: 0, fetch: 16, fetch_time: 0, ioread: 0)
    SCAN (table: agl_tbl), (noscan time: 0, fetch: 0, ioread: 0,
readrows: 0, rows: 0, agl: pk_agl_tbl_id)
```

GROUPBY

- time: the estimated time(ms) in this operation
- sort: sorting or not
- page: the number of pages which is read in this operation; the number of used pages except the internal sorting buffer
- rows: the number of the result rows in this operation
- hash: hash aggregate evaluation or not, when sorting tuples in the aggregate function(true/false). See NO_HASH_AGGREGATE hint.

INDEX SCAN

- key range: the range of a key
- covered: covered index or not(true/false)
- loose: loose index scan or not(true/false)

The above example can be output as JSON format.

```
csql> SET TRACE ON OUTPUT JSON;
csql> SELECT n.name, a.name FROM athlete a, nation n WHERE
n.code=a.nation_code;
csql> SHOW TRACE;

  trace
======================
  '{
  "Trace Statistics": {
    "SELECT": {
      "time": 29,
      "fetch": 5836,
      "ioread": 3,
      "SCAN": {
        "access": "temp",
        "temp": {
          "time": 5,
          "fetch": 34,
          "ioread": 0,
          "readrows": 6677,
          "rows": 6677
        }
      },
      "MERGELIST": {
        "outer": {
          "SELECT": {
            "time": 0,
            "fetch": 2,
            "ioread": 0,
            "SCAN": {
              "access": "table (nation)",
              "heap": {
                "time": 0,
                "fetch": 1,
                "ioread": 0,
                "readrows": 215,
                "rows": 215
              }
            },
            "ORDERBY": {
              "time": 0,
```

```
                "sort": true,
                "page": 21,
                "ioread": 3
              }
            }
          }
        }
      }
    }
  }
}'
```

On CSQL interpreter, if you use the command to set the SQL trace on automatically, the trace result is printed out automatically after printing the query result even if you do not run **SHOW TRACE;** syntax.

For how to set the trace on automatically, see Set SQL trace.

Note

- CSQL interpreter which is run in the standalone mode(use -S option) does not support SQL trace feature.
- When multiple queries are performed at once(batch query, array query), they are not profiled.

Using SQL Hint

Using hints can affect the performance of query execution. You can allow the query optimizer to create more efficient execution plan by referring to the SQL HINT. The SQL HINTs related tale join and index are provided by CUBRID.

```
{ SELECT | UPDATE | DELETE } /*+ <hint> [ { <hint> } ... ] */ ...;

MERGE /*+ <merge_statement_hint> [ { <merge_statement_hint> } ... ]
*/ INTO ...;

<hint> ::=
USE_NL [ (<spec_name_comma_list>) ] |
USE_IDX [ (<spec_name_comma_list>) ] |
USE_MERGE [ (<spec_name_comma_list>) ] |
USE_HASH [ (<spec_name_comma_list>) ] |
NO_USE_HASH [ (<spec_name_comma_list>) ] |
ORDERED |
LEADING |
```

```
USE_DESC_IDX |
USE_SBR |
INDEX_SS [ (<spec_name_comma_list>) ] |
INDEX_LS |
NO_DESC_IDX |
NO_COVERING_IDX |
NO_MULTI_RANGE_OPT |
NO_SORT_LIMIT |
NO_SUBQUERY_CACHE |
NO_PUSH_PRED |
NO_MERGE |
NO_ELIMINATE_JOIN |
NO_HASH_AGGREGATE |
NO_HASH_LIST_SCAN |
NO_LOGGING |
RECOMPILE |
QUERY_CACHE

<spec_name_comma_list> ::= <spec_name> [, <spec_name>, ... ]
    <spec_name> ::= [schema_name.]table_name |
[schema_name.]view_name

<merge_statement_hint> ::=
USE_UPDATE_INDEX (<update_index_list>) |
USE_DELETE_INDEX (<insert_index_list>) |
RECOMPILE
```

SQL hints are specified by using a plus sign(+) to comments. To use a hint, there are three styles as being introduced on Comment. Therefore, also SQL hint can be used as three styles.

- /*+ hint */
- --+ hint
- //+ hint

The hint comment must appear after the keyword such as **SELECT**, **UPDATE** or **DELETE**, and the comment must begin with a plus sign (+), following the comment delimiter. When you specify several hints, they are separated by blanks.

The following hints can be specified in **UPDATE**, **DELETE** and **SELECT** statements.

- **USE_NL**: Related to a table join, the query optimizer creates a nested loop join execution plan with this hint.
- **USE_MERGE**: Related to a table join, the query optimizer creates a sort merge join execution plan with this hint.
- **USE_HASH**: Related to a table join, the query optimizer creates a hash join execution plan with this hint. see Hash Join.

- **NO_USE_HASH**: Related to a table join, The query optimizer does not create a hash join execution plan with this hint. see Hash Join.
- **ORDERED**: Related to a table join, the query optimizer create a join execution plan with this hint, based on the order of tables specified in the **FROM** clause. The left table in the **FROM** clause becomes the outer table; the right one becomes the inner table.
- **LEADING**: Related to a table join, the query optimizer create a join execution plan with this hint, based on the order of tables specified in the **LEADING** hint.
- **USE_IDX**: Related to an index, the query optimizer creates an index join execution plan corresponding to a specified table with this hint.
- **USE_DESC_IDX**: This is a hint for the scan in descending index. For more information, see Index Scan in Descending Order.
- **USE_SBR**: This is a hint for the statement-based replication. It supports data replication for tables without a primary key.

> **Note**
>
> The data inconsistency of a table may occur between nodes since the corresponding statement is executed when the transaction log is applied in the slave node.

- **INDEX_SS**: Consider the query plan of index skip scan. For more information, see Index Skip Scan.
- **INDEX_LS**: Consider the query plan of loose index scan. For more information, see Loose Index Scan.
- **NO_DESC_IDX**: This is a hint not to use the descending index.
- **NO_COVERING_IDX**: This is a hint not to use the covering index. For details, see Covering Index.
- **NO_MULTI_RANGE_OPT**: This is a hint not to use the multi-key range optimization. For details, see Multiple Key Ranges Optimization.
- **NO_SORT_LIMIT**: This is a hint not to use the SORT-LIMIT optimization. For more details, see SORT-LIMIT optimization.
- **NO_SUBQUERY_CACHE**: This is a hint not to use the SUBQUERY CACHE optimization. For more details, see SUBQUERY CACHE (correlated).
- **NO_PUSH_PRED**: This is a hint not to use the PREDICATE-PUSH optimization.
- **NO_MERGE**: This is a hint not to use the VIEW-MERGE optimization.

- **NO_ELIMINATE_JOIN**: This is a hint not to use join elimination optimization. For more details, see Join transforming Optimization.

- **NO_HASH_AGGREGATE**: This is a hint not to use hashing for the sorting tuples in aggregate functions. Instead, external sorting is used in aggregate functions. By using an in-memory hash table, we can reduce or even eliminate the amount of data that needs to be sorted. However, in some scenarios the user may know beforehand that hash aggregation will fail and can use the hint to skip hash aggregation entirely. For setting the memory size of hashing aggregate, see max_agg_hash_size.

  Note

  Hash aggregate evaluation will not work for functions evaluated on distinct values (e.g. AVG(DISTINCT x)) and for the GROUP_CONCAT and MEDIAN functions, since they require an extra sorting step for the tuples of each group.

- **NO_HASH_LIST_SCAN**: This is a hint not to use hash list scan for scanning subquery's result. Instead, list scan is used to scan temp file. By building and probing hash table, we can reduce the amount of data that needs to be searched. However, in some scenarios, the user may know beforehand that outer cardinality is very small and can use the hint to skip hash list scan entirely. For setting the memory size of hash scan, see max_hash_list_scan_size.

  Note

  Hash List scan only works for predicates having a equal operation and does NOT work for predicates having OID type.

- **NO_LOGGING**: This is a hint not to include the redo in the log generated when inserting, updating, or deleting records to a table.

  Note

  Currently, The NO_LOGGING hint only affects the log created from the heap file when inserting, updating, or deleting records to a table. Therefore, problems such as the inconsistency between the data of the table and the data of the index might occur after

recovery; and the situation of committed record cannot be recovered might also occur, etc. You should use it carefully.

- **RECOMPILE** : Recompiles the query execution plan. This hint is used to delete the query execution plan stored in the cache and establish a new query execution plan.
- **QUERY_CACHE**: This is a hint for caching the query with its results. This hint can be specified in **SELECT** statements only. For more information, see QUERY CACHE.

Note

If <*spec_name*> is specified together with **USE_NL**, **USE_IDX** or **USE_MERGE**, the specified join method applies only to the <*spec_name*>.

```
SELECT /*+ ORDERED USE_NL(B) USE_NL(C) USE_MERGE(D) */ *
FROM A INNER JOIN B ON A.col=B.col
INNER JOIN C ON B.col=C.col INNER JOIN D  ON C.col=D.col;
```

If you run the above query, **USE_NL** is applied when A and B are joined; **USE_NL** is applied when C is joined, too; **USE_MERGE** is applied when D is joined.

If **USE_NL** and **USE_MERGE** are specified together without <*spec_name*>, the given hint is ignored. In some cases, the query optimizer cannot create a query execution plan based on the given hint. For example, if **USE_NL** is specified for a right outer join, the query is converted to a left outer join internally, and the join order may not be guaranteed.

Note

If you specify the **ORDERED** hint, all **LEADING** hint is ignored. If you specify two or more **LEADING** hints, then only the first one is activated and all of them except the first are ignored.

```
SELECT /*+ ORDERED LEADING(b, d) */ *
FROM a INNER JOIN b ON a.col=b.col
INNER JOIN c ON b.col=c.col INNER JOIN d ON c.col=d.col;
```

If you run the above query, the **LEADING** hint is ignored, and tables a, b, c, and d are joined in the order of the **FROM** clause according to the **ORDERED** hint.

```
SELECT /*+ LEADING(b, d) LEADING(c, d) */ *
FROM a INNER JOIN b ON a.col=b.col
INNER JOIN c ON b.col=c.col INNER JOIN d ON c.col=d.col;
```

If you run the above query, the second **LEADING** hint is ignored, and join order in which tables b and d are joined first is generated.

MERGE statement can have below hints.

- **USE_INSERT_IDX** (<*insert_index_list*>): An index hint which is used in **INSERT** clause of **MERGE** statement. Lists index names to *insert_index_list* to use when executing **INSERT** clause. This hint is applied to <*join_condition*> of **MERGE** statement.
- **USE_UPDATE_IDX** (<*update_index_list*>): An index hint which is used in **UPDATE** clause of **MERGE** statement. Lists index names to *update_index_list* to use when executing **UPDATE** clause. This hint is applied to <*join_condition*> and <*update_condition*> of **MERGE** statement.
- **RECOMPILE**: See the above RECOMPILE.

Table/view names to join can be specified to the joining hint; at this time, table/view names are separated by ",".

```
SELECT /*+ USE_NL(a, b) */ *
FROM a INNER JOIN b ON a.col=b.col;
```

The following example shows how to retrieve the years when *'Sim Kwon Ho'* won medals and the types of medals. It can be expressed by the following query. The query optimizer creates a nested loop join execution plan that has the *athlete* table as an outer table and the *game* table as an inner table.

```
-- csql> ;plan_detail

SELECT /*+ USE_NL ORDERED  */ a.name, b.host_year, b.medal
FROM athlete a, game b
WHERE a.name = 'Sim Kwon Ho' AND a.code = b.athlete_code;
```

```
Query plan:

idx-join (inner join)
    outer: sscan
                class: a node[0]
                sargs: term[1]
                cost:  44 card 7
```

```
    inner: iscan
                class: b node[1]
                index: fk_game_athlete_code term[0]
                cost:  3 card 8653
    cost:  73 card 9
```

The following example shows how to specify tables when using a **USE_NL** hint.

```
-- csql> ;plan_detail

SELECT /*+ USE_NL(a,b)  */ a.name, b.host_year, b.medal
FROM athlete a, game b
WHERE a.name = 'Sim Kwon Ho' AND a.code = b.athlete_code;
```

Index Hint

The index hint syntax allows the query processor to select a proper index by specifying the index in the query. You can specify the index hint by **USING INDEX** clause or by { **USE** | **FORCE** | **IGNORE** } **INDEX** syntax after "**FROM** table" clause.

USING INDEX

USING INDEX clause should be specified after **WHERE** clause of **SELECT**, **DELETE** or **UPDATE** statement. **USING INDEX** clause forces a sequential/index scan to be used or an index that can improve the performance to be included.

If **USING INDEX** clause is specified with the list of index names, query optimizer creates optimized execution plan by calculating the query execution cost based on the specified indexes only and comparing the index scan cost and the sequential scan cost of the specified indexes(CUBRID performs cost-based query optimization to select an execution plan).

The **USING INDEX** clause is useful to get the results in the desired order without **ORDER BY**. When index scan is performed by CUBRID, the results are created in the order they were saved in the index. When there are more than one indexes in one table, you can use **USING INDEX** to get the query results in a given order of indexes.

```
SELECT ... WHERE ...
[USING INDEX { NONE | [ ALL EXCEPT ] <index_spec> [ {,
<index_spec> } ...] } ] [ ; ]

DELETE ... WHERE ...
[USING INDEX { NONE | [ ALL EXCEPT ] <index_spec> [ {,
<index_spec> } ...] } ] [ ; ]

UPDATE ... WHERE ...
```

```
[USING INDEX { NONE | [ ALL EXCEPT ] <index_spec> [ {,
<index_spec> } ...] } ] [ ; ]

<index_spec> ::=
  [table_spec.]index_name [(+) | (-)] |
  table_spec.NONE
```

- **NONE**: If **NONE** is specified, a sequential scan is used on all tables.
- **ALL EXCEPT**: All indexes except the specified indexes can be used when the query is executed.
- *index_name*(+): If (+) is specified after the index_name, it is the first priority in index selection. IF this index is not proper to run the query, it is not selected.
- *index_name*(-): If (-) is specified after the index_name, it is excluded from index selection.
- *table_spec*.**NONE**: All indexes are excluded from the selection, so sequential scan is used.

USE, FORCE, IGNORE INDEX

Index hints can be specified through **USE**, **FORCE**, **IGNORE INDEX** syntax after table specification of **FROM** clause.

```
FROM table_spec [ <index_hint_clause> ] ...

<index_hint_clause> ::=
  { USE | FORCE | IGNORE } INDEX  ( <index_spec> [,
<index_spec>  ...] )

<index_spec> ::=
  [table_spec.]index_name
```

- **USE INDEX** (<*index_spec*>): Only specified indexes are considered when choose them.
- **FORCE INDEX** (<*index_spec*>): Specified indexes are chosen as the first priority.
- **IGNORE INDEX** (<*index_spec*>): Specified indexes are excluded from the choice.

USE, **FORCE**, **IGNORE INDEX** syntax is automatically rewritten as the proper **USING INDEX** syntax by the system.

Examples of index hint

```
CREATE TABLE athlete2 (
   code            SMALLINT PRIMARY KEY,
   name            VARCHAR(40) NOT NULL,
```

```
    gender          CHAR(1),
    nation_code     CHAR(3),
    event           VARCHAR(30)
);
CREATE UNIQUE INDEX athlete2_idx1 ON athlete2 (code, nation_code);
CREATE INDEX athlete2_idx2 ON athlete2 (gender, nation_code);
```

Below two queries do the same behavior and they select index scan if the specified index, *athlete2_idx2*'s scan cost is lower than sequential scan cost.

```
SELECT /*+ RECOMPILE */ *
FROM athlete2 USE INDEX (athlete2_idx2)
WHERE gender='M' AND nation_code='USA';

SELECT /*+ RECOMPILE */ *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
USING INDEX athlete2_idx2;
```

Below two queries do the same behavior and they always use *athlete2_idx2*

```
SELECT /*+ RECOMPILE */ *
FROM athlete2 FORCE INDEX (athlete2_idx2)
WHERE gender='M' AND nation_code='USA';

SELECT /*+ RECOMPILE */ *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
USING INDEX athlete2_idx2(+);
```

Below two queries do the same behavior and they always don't use *athlete2_idx2*

```
SELECT /*+ RECOMPILE */ *
FROM athlete2 IGNORE INDEX (athlete2_idx2)
WHERE gender='M' AND nation_code='USA';

SELECT /*+ RECOMPILE */ *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
USING INDEX athlete2_idx2(-);
```

Below query always do the sequential scan.

```
SELECT *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
```

```
USING INDEX NONE;

SELECT *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
USING INDEX athlete2.NONE;
```

Below query forces to be possible to use all indexes except *athlete2_idx2* index.

```
SELECT *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
USING INDEX ALL EXCEPT athlete2_idx2;
```

When two or more indexes have been specified in the **USING INDEX** clause, the query optimizer selects the proper one of the specified indexes.

```
SELECT *
FROM athlete2 USE INDEX (athlete2_idx2, athlete2_idx1)
WHERE gender='M' AND nation_code='USA';

SELECT *
FROM athlete2
WHERE gender='M' AND nation_code='USA'
USING INDEX athlete2_idx2, athlete2_idx1;
```

When a query is run for several tables, you can specify a table to perform index scan by using a specific index and another table to perform sequential scan. The query has the following format.

```
SELECT *
FROM tab1, tab2
WHERE ...
USING INDEX tab1.idx1, tab2.NONE;
```

When executing a query with the index hint syntax, the query optimizer considers all available indexes on the table for which no index has been specified. For example, when the *tab1* table includes *idx1* and *idx2* and the *tab2* table includes *idx3*, *idx4*, and *idx5*, if indexes for only *tab1* are specified but no indexes are specified for *tab2*, the query optimizer considers the indexes of *tab2*.

```
SELECT ...
FROM tab1, tab2 USE INDEX(tab1.idx1)
WHERE ... ;

SELECT ...
FROM tab1, tab2
```

```
WHERE ...
USING INDEX tab1.idx1;
```

The above query select the scan method of table *tab1* after comparing the cost between the sequential scan of the table *tab1* and the index scan of the index *idx1*, and select the scan method of table *tab2* after comparing the cost between the sequential scan of the table *tab2* and the index scan of the indexes *idx3*, *idx4*, *idx5*.

Special Indexes

Filtered Index

The filtered index is used to sort, search, or operate a well-defined partials set for one table. It is called the partial index since only some data that satisfy the condition are kept in that index.

```
CREATE /*+ hints */ INDEX index_name
ON [schema_name.]table_name (col1, col2, ...)
WHERE <filter_predicate>;

ALTER  /*+ hints */ INDEX index_name
[ ON [schema_name.]table_name (col1, col2, ...)
[ WHERE <filter_predicate> ] ]
REBUILD;

<filter_predicate> ::= <filter_predicate> AND <expression> |
<expression>
```

- <*filter_predicate*>: Condition to compare the column and the constant. When there are several conditions, filtering is available only when they are connected by using **AND**. The filter conditions can include most of the operators and functions supported by CUBRID. However, the date/time function that shows the current date/time (ex: `SYS_DATETIME()`) or random functions (ex: `RAND()`), which outputs different results for one input are not allowed.

If you want to apply the filtered index, that filtered index must be specified by **USE INDEX** syntax or **FORCE INDEX** syntax.

- When a filtered index is specified by **USING INDEX** clause or **USE INDEX** syntax:

  If columns of which the index consists are not included on the conditions of **WHERE** clause, the filtered index is not used.

```
CREATE TABLE blogtopic
(
    blogID BIGINT NOT NULL,
    title VARCHAR(128),
```

```
    author VARCHAR(128),
    content VARCHAR(8096),
    postDate TIMESTAMP NOT NULL,
    deleted SMALLINT DEFAULT 0
);

CREATE INDEX my_filter_index ON blogtopic(postDate) WHERE deleted=0;
```

On the below query, *postDate*, a column of which *my_filter_index* consists, is included on the conditions of **WHERE** condition. Therefore, this index can be used by **USE INDEX** clause.

```
SELECT *
FROM blogtopic USE INDEX (my_filter_index)
WHERE postDate>'2010-01-01' AND deleted=0;
```

- When a filtered index is specified by **USING INDEX** <index_name>(+) clause or **FORCE INDEX** syntax:

  Even if a column of which the index consists is not included on the condition of **WHERE** clause, the filtered index is used.

  On the below query, *my_filter_index* cannot be used by **"USE INDEX"** syntax because a column of which *my_filter_index* consists is not included on the **WHERE** condition.

```
SELECT *
FROM blogtopic USE INDEX (my_filter_index)
WHERE author = 'David' AND deleted=0;
```

  Therefore, to use *my_filter_index*, it should be forced by **"FORCE INDEX"**.

```
SELECT *
FROM blogtopic FORCE INDEX (my_filter_index)
WHERE author = 'David' AND deleted=0;
```

The following example shows a bug tracking system that maintains bugs/issues. After a specified period of development, the bugs table records bugs. Most of the bugs have already been closed. The bug tracking system makes queries on the table to find new open bugs. In this case, the indexes on the bug table do not need to know the records on closed bugs. Then the filtered indexes allow indexing of open bugs only.

```
CREATE TABLE bugs
(
    bugID BIGINT NOT NULL,
    CreationDate TIMESTAMP NOT NULL,
    Author VARCHAR(255),
```

```
    Subject VARCHAR(255),
    Description VARCHAR(255),
    CurrentStatus INTEGER,
    Closed SMALLINT
);
```

Indexes for open bugs can be created by using the following sentence:

```
CREATE INDEX idx_open_bugs ON bugs(bugID) WHERE Closed = 0;
```

To process queries that are interested in open bugs, specify the index as an index hint. It will allow creating query results by accessing less index pages through filtered indexes.

```
SELECT *
FROM bugs
WHERE Author = 'madden' AND Subject LIKE '%fopen%' AND Closed = 0
USING INDEX idx_open_bugs(+);

SELECT *
FROM bugs FORCE INDEX (idx_open_bugs)
WHERE CreationDate > CURRENT_DATE - 10 AND Closed = 0;
```

On the above example, if you use "**USING INDEX** *idx_open_bugs*" or "**USE INDEX** (*idx_open_bugs*)", a query is processed without using the *idx_open_bugs* index.

Warning

If you execute queries by specifying indexes with index hint syntax even though the conditions of creating filtered indexes does not match the query conditions, CUBRID performs a query by choosing a specified index. Therefore, query results can be different with the given searching conditions.

Note

Constraints

Only generic indexes are allowed as filtered indexes. For example, the filtered unique index is not allowed. Also, it is not allowed that columns which compose an index are all NULLable. For example, below is not allowed because Author is NULLable.

```
CREATE INDEX idx_open_bugs ON bugs (Author) WHERE Closed = 0;
```

```
ERROR: before ' ; '
Invalid filter expression (bugs.Closed=0) for index.
```

However, below is allowed because Author is NULLable, but CreationDate is not NULLable.

```
CREATE INDEX idx_open_bugs ON bugs (Author, CreationDate) WHERE
Closed = 0;
```

The following cases are not allowed as filtering conditions.

- Functions, which output different results with the same input, such as date/time function or random function

```
CREATE INDEX idx ON bugs(creationdate) WHERE creationdate >
SYS_DATETIME;
```

```
ERROR: before ' ; '
'sys_datetime ' is not allowed in a filter expression for index.
```

```
CREATE INDEX idx ON bugs(bugID) WHERE bugID > RAND();
```

```
ERROR: before ' ; '
'rand ' is not allowed in a filter expression for index.
```

- In case of using the **OR** operator

```
CREATE INDEX IDX ON bugs (bugID) WHERE bugID > 10 OR bugID = 3;
```

```
ERROR: before ' ; '
' or ' is not allowed in a filter expression for index.
```

- In case of including functions like `INCR()`, `DECR()` functions, which modify the data of a table.
- In case of Serial-related functions and including pseudo columns.
- In case of including aggregate functions such as `MIN()`, `MAX()`, `STDDEV()`

- In case of using the types where indexes cannot be created
  - The operators and functions where an argument is the **SET** type
  - The functions to use LOB file(`CHAR_TO_BLOB()` , `CHAR_TO_CLOB()` , `BIT_TO_BLOB()` , `BLOB_FROM_FILE()` , `CLOB_FROM_FILE()`)
- The **IS NULL** operator can be used only when at least one column of an index is not **NULL**.

```
CREATE TABLE t (a INT, b INT);

-- IS NULL cannot be used with expressions
CREATE INDEX idx ON ta (a) WHERE (not ta.a<>0 ) IS NULL;
```

```
ERROR: before ' ; '
Invalid filter expression (( not [dba.ta].a<>0) is null ) for
index.
```

```
CREATE INDEX idx ON t (a) WHERE (a+1) IS NULL;
```

```
ERROR: before ' ; '
Invalid filter expression ((t.a+1) is null ) for index.
```

```
-- At least one attribute must not be used with IS NULL
CREATE INDEX idx ON t(a,b) WHERE a IS NULL ;
```

```
ERROR: before '  ; '
Invalid filter expression (t.a is null ) for index.
```

```
CREATE INDEX idx ON t(a,b) WHERE a IS NULL and b IS NULL;
```

```
ERROR: before ' ; '
Invalid filter expression (t.a is null  and t.b is null ) for
index.
```

```
CREATE INDEX idx ON t(a,b) WHERE a IS NULL and b IS NOT NULL;
```

- Index Skip Scan (ISS) is not allowed for the filtered indexes.

- The length of condition string used for the filtered index is limited to 128 characters.

```
CREATE TABLE t(VeryLongColumnNameOfTypeInteger INT);

CREATE INDEX idx ON t(VeryLongColumnNameOfTypeInteger)
WHERE VeryLongColumnNameOfTypeInteger > 3 AND
VeryLongColumnNameOfTypeInteger < 10 AND
SQRT(VeryLongColumnNameOfTypeInteger) < 3 AND
SQRT(VeryLongColumnNameOfTypeInteger) < 10;
```

```
ERROR: before ' ; '
The maximum length of filter predicate string must be 128.
```

Function-based Index

Function-based index is used to sort or find the data based on the combination of values of table rows by using a specific function. For example, to find the space-ignored string, it can be used to optimize the query by using the function that provides the feature. In addition, it is useful to search the non-case-sensitive names.

```
CREATE /*+ hints */ INDEX index_name
ON [schema_name.]table_name (function_name (argument_list));

ALTER /*+ hints */ INDEX index_name
[ ON [schema_name.]table_name (function_name (argument_list)) ]
REBUILD;
```

After the following indexes have been created, the **SELECT** query automatically uses the function-based index.

```
CREATE INDEX idx_trim_post ON posts_table(TRIM(keyword));

SELECT *
FROM posts_table
WHERE TRIM(keyword) = 'SQL';
```

If a function-based index is created by using the **LOWER** function, it can be used to search the non-case-sensitive names.

```
CREATE INDEX idx_last_name_lower ON clients_table(LOWER(LastName));
```

```
SELECT *
FROM clients_table
WHERE LOWER(LastName) = LOWER('Timothy');
```

To make an index selected while creating a query plan, the function used for the index should be used for the query condition in the same way. The **SELECT** query above uses the last_name_lower index created above. However, this index is not used for the following condition:

```
SELECT *
FROM clients_table
WHERE LOWER(CONCAT('Mr. ', LastName)) = LOWER('Mr. Timothy');
```

In addition, to make the function-based index used by force, use the **USING INDEX** syntax.

```
CREATE INDEX i_tbl_first_four ON tbl(LEFT(col, 4));
SELECT *
FROM clients_table
WHERE LEFT(col, 4) = 'CAT5'
USING INDEX i_tbl_first_four;
```

Functions with the function-based indexes are as follows:

ABS	ACOS	ADD_MONTHS	ADDDATE	ASIN
ATAN	ATAN2	BIT_COUNT	BIT_LENGTH	CEIL
CHAR_LENGTH	CHR	COS	COT	DATE
DATE_ADD	DATE_FORMAT	DATE_SUB	DATEDIFF	DAY
DAYOFMONTH	DAYOFWEEK	DAYOFYEAR	DEGREES	EXP
FLOOR	FORMAT	FROM_DAYS	FROM_UNIXTIME	GREATEST
HOUR	IFNULL	INET_ATON	INET_NTOA	INSTR

LAST_DAY	LEAST	LEFT	LN	LOCATE
LOG	LOG10	LOG2	LOWER	LPAD
LTRIM	MAKEDATE	MAKETIME	MD5	MID
MINUTE	MOD	MONTH	MONTHS_BETWEEN	NULLIF
NVL	NVL2	OCTET_LENGTH	POSITION	POWER
QUARTER	RADIANS	REPEAT	REPLACE	REVERSE
RIGHT	ROUND	RPAD	RTRIM	SECOND
SECTOTIME	SIN	SQRT	STR_TO_DATE	STRCMP
SUBDATE	SUBSTR	SUBSTRING	SUBSTRING_INDEX	TAN
TIME	TIME_FORMAT	TIMEDIFF	TIMESTAMP	TIMETOSEC
TO_CHAR	TO_DATE	TO_DATETIME	TO_DAYS	TO_NUMBER
TO_TIME	TO_TIMESTAMP	TRANSLATE	TRIM	TRUNC
UNIX_TIMESTAMP	UPPER	WEEK	WEEKDAY	YEAR

Arguments of functions which can be used in the function-based indexes, only column names and constants are allowed; nested expressions are not allowed. For example, a statement below will cause an error.

```
CREATE INDEX my_idx ON tbl (TRIM(LEFT(col, 3)));
CREATE INDEX my_idx ON tbl (LEFT(col1, col2 + 3));
```

However, implicit cast is allowed. In the example below, the first argument type of the **LEFT** () function should be **VARCHAR** and the second argument type should be **INTEGER**; it works normally.

```
CREATE INDEX my_idx ON tbl (LEFT(int_col, str_col));
```

Function-based indexes cannot be used with filtered indexes. The example will cause an error.

```
CREATE INDEX my_idx ON tbl (TRIM(col)) WHERE col > 'SQL';
```

Function-based indexes cannot become multiple-columns indexes. The example will cause an error.

```
CREATE INDEX my_idx ON tbl (TRIM(col1), col2, LEFT(col3, 5));
```

Optimization Using Indexes

Covering Index

The covering index is the index including the data of all columns in the **SELECT** list and the **WHERE**, **HAVING**, **GROUP BY**, and **ORDER BY** clauses.

You only need to scan the index pages, as the covering index contains all the data necessary for executing a query, and it also reduces the I/O costs as it is not necessary to scan the data storage any further. To increase data search speed, you can consider creating a covering index but you should be aware that the **INSERT** and the **DELETE** processes may be slowed down due to the increase in index size.

The rules about the applicability of the covering index are as follows:

- If the covering index is applicable, you should use the CUBRID query optimizer first.
- For the join query, if the index includes columns of the table in the **SELECT** list, use this index.

- You cannot use the covering index if an index cannot be used.

```
CREATE TABLE t (col1 INT, col2 INT, col3 INT);
CREATE INDEX i_t_col1_col2_col3 ON t (col1,col2,col3);
INSERT INTO t VALUES (1,2,3),(4,5,6),(10,8,9);
```

The following example shows that the index is used as a covering index because columns of both **SELECT** and **WHERE** condition exist within the index.

```
-- csql> ;plan simple
SELECT * FROM t WHERE col1 < 6;
```

```
Query plan:
 Index scan(t t, i_t_col1_col2_col3, [(t.col1 range (min inf_lt
t.col3))] (covers))

         col1         col2         col3
=======================================
            1            2            3
            4            5            6
```

The following example shows an optimization that reduces unnecessary scans when the **SELECT** list only contains **COUNT(*)** when using a covering index.

```
-- insert dummy data
INSERT INTO t select rownum % 8, rownum % 100, rownum % 1000 FROM
dual connect by level <= 40000;

-- csql> ;trace on

-- count(*) optimization
SELECT count(*) FROM t WHERE col1 < 9;
```

```
Trace Statistics:
  SELECT (time: 1, fetch: 66, fetch_time: 0, ioread: 0)
    SCAN (index: dba.t.i_t_col1_col2_col3), (btree time: 1, fetch:
65, ioread: 0, readkeys: 1002, filteredkeys: 0, rows: 0, covered:
true, count_only: true)
```

```
-- no count(*) optimization
SELECT count(col1) FROM t WHERE col1 < 9;
```

```
Trace Statistics:
  SELECT (time: 13, fetch: 180, fetch_time: 0, ioread: 0)
    SCAN (index: dba.t.i_t_col1_col2_col3), (btree time: 8, fetch:
179, ioread: 0, readkeys: 1002, filteredkeys: 0, rows: 40002,
covered: true)
```

Note

COUNT(*) optimization for a single table when using a covering index is supported from CUBRID 11.2, and **COUNT(*)** optimization for join tables is supported from CUBRID 11.3.

Warning

When a covering index is applied to retrieve values from a **VARCHAR** type column, any trailing empty strings will be truncated. This occurs because the empty string is stored in the index with the trailing spaces removed, affecting the query results when the index is used for query optimization.

If you don't want this, use the **NO_COVERING_IDX** hint, which does not use the covering index function. If you use the hint, you can get the result value from the data area rather than from the index area.

The following is a detailed example of the above situation. First, create a table with columns in **VARCHAR** types, and then **INSERT** the value with the same start character string value but the number of empty characters. Next, create an index in the column.

```
CREATE TABLE tab(c VARCHAR(32));
INSERT INTO tab VALUES('abcd'),('abcd    '),('abcd ');
CREATE INDEX i_tab_c ON tab(c);
```

If you must use the index (the covering index applied), the query result is as follows:

```
-- csql>;plan simple
SELECT * FROM tab WHERE c='abcd    ' USING INDEX i_tab_c(+);
```

```
Query plan:
 Index scan(tab tab, i_tab_c, (tab.c='abcd    ') (covers))

 c
======================
```

```
'abcd'
'abcd'
'abcd'
```

The following is the query result when you don't use the index.

```
SELECT * FROM tab WHERE c='abcd    ' USING INDEX tab.NONE;
```

```
Query plan:
 Sequential scan(tab tab)

 c
======================
'abcd'
'abcd    '
'abcd '
```

As you can see in the above comparison result, the value in the **VARCHAR** type retrieved from the index will appear with the following empty string truncated when the covering index has been applied.

Note

If covering index optimization is available to be applied, the I/O performance can be improved because the disk I/O is decreased. But if you don't want covering index optimization in a special condition, specify a **NO_COVERING_IDX** hint to the query. For how to add a query, see Using SQL Hint.

Optimizing ORDER BY Clause

The index including all columns in the **ORDER BY** clause is referred to as the ordered index. Optimizing the query with **ORDER BY** clause is no need for the additional sorting process(skip order by), because the query results are searched by the ordered index. In general, for an ordered index, the columns in the **ORDER BY** clause should be located at the front of the index.

```
SELECT *
FROM tab
```

```
WHERE col1 > 0
ORDER BY col1, col2;
```

- The index consisting of *tab* (*col1*, *col2*) is an ordered index.
- The index consisting of *tab* (*col1*, *col2*, *col3*) is also an ordered index. This is because the *col3*, which is not referred to by the **ORDER BY** clause, comes after *col1* and *col2* .
- The index consisting of *tab* (*col1*) is not an ordered index.
- You can use the index consisting of *tab* (*col3*, *col1*, *col2*) or *tab* (*col1*, *col3*, *col2*) for optimization. This is because *col3* is not located at the back of the columns in the **ORDER BY** clause.

Although the columns composing an index do not exist in the **ORDER BY** clause, you can use an ordered index if the column condition is a constant.

```
SELECT *
FROM tab
WHERE col2=val
ORDER BY col1,col3;
```

If the index consisting of *tab* (*col1*, *col2*, *col3*) exists and the index consisting of *tab* (*col1*, *col2*) do not exist when executing the above query, the query optimizer uses the index consisting of *tab* (*col1*, *col2*, *col3*) as an ordered index. You can get the result in the requested order when you execute an index scan, so you don't need to sort records.

If you can use the sorted index and the covering index, use the latter first. If you use the covering index, you don't need to retrieve additional data, because the data result requested is included in the index page, and you won't need to sort the result if you are satisfied with the index order.

If the query doesn't include any conditions and uses an ordered index, the ordered index will be used under the condition that the first column meets the **NOT NULL** condition.

```
CREATE TABLE tab (i INT, j INT, k INT);
CREATE INDEX i_tab_j_k on tab (j,k);
INSERT INTO tab VALUES (1,2,3),(6,4,2),(3,4,1),(5,2,1),(1,5,5),(2,6,
6),(3,5,4);
```

The following example shows that indexes consisting of *tab* (*j*, *k*) become sorted indexes and no separate sorting process is required because **GROUP BY** is executed by *j* and *k* columns.

```
SELECT i,j,k
FROM tab
WHERE j > 0
ORDER BY j,k;
```

```
--  the  selection from the query plan dump shows that the ordering
index i_tab_j_k was used and sorting was not necessary
--  (/* --> skip ORDER BY */)
Query plan:
iscan
    class: tab node[0]
    index: i_tab_j_k term[0]
    sort:  2 asc, 3 asc
    cost:  1 card 0
Query stmt:
select tab.i, tab.j, tab.k from tab tab where ((tab.j> ?:0 )) order
by 2, 3
/* ---> skip ORDER BY */

            i            j            k
=======================================
            5            2            1
            1            2            3
            3            4            1
            6            4            2
            3            5            4
            1            5            5
            2            6            6
```

The following example shows that *j* and *k* columns execute **ORDER BY** and the index including all columns are selected so that indexes consisting of *tab* (*j*, *k*) are used as covering indexes; no separate process is required because the value is selected from the indexes themselves.

```
SELECT /*+ RECOMPILE */ j,k
FROM tab
WHERE j > 0
ORDER BY j,k;
```

```
--  in this case the index i_tab_j_k is a covering index and also
respects the ordering index property.
--  Therefore, it is used as a covering index and sorting is not
performed.

Query plan:
iscan
    class: tab node[0]
    index: i_tab_j_k term[0] (covers)
    sort:  1 asc, 2 asc
    cost:  1 card 0

Query stmt: select tab.j, tab.k from tab tab where ((tab.j> ?:0 ))
order by 1, 2
/* ---> skip ORDER BY */
```

```
            j            k
=========================
            2            1
            2            3
            4            1
            4            2
            5            4
            5            5
            6            6
```

The following example shows that *i* column exists, **ORDER BY** is executed by *j* and *k* columns, and columns that perform **SELECT** are *i*, *j*, and *k*. Therefore, indexes consisting of *tab* (*i*, *j*, *k*) are used as covering indexes; separate sorting process is required for **ORDER BY** *j*, *k* even though the value is selected from the indexes themselves.

```
CREATE INDEX i_tab_j_k ON tab (i,j,k);
SELECT /*+ RECOMPILE */ i,j,k
FROM tab
WHERE i > 0
ORDER BY j,k;
```

```
-- since an index on (i,j,k) is now available, it will be used as
covering index. However, sorting the results according to
-- the ORDER BY  clause is needed.
Query plan:
temp(order by)
    subplan: iscan
                class: tab node[0]
                index: i_tab_i_j_k term[0] (covers)
                sort:  1 asc, 2 asc, 3 asc
                cost:  1 card 1
    sort:  2 asc, 3 asc
    cost:  7 card 1

Query stmt: select tab.i, tab.j, tab.k from tab tab where ((tab.i> ?:
0 )) order by 2, 3

            i            j            k
=======================================
            5            2            1
            1            2            3
            3            4            1
            6            4            2
            3            5            4
            1            5            5
            2            6            6
```

Note

Even if the type of a column in the **ORDER BY** clause is converted by using `CAST()`, **ORDER BY** optimization is executed when the sorting order is the same as before.

Before	After
numeric type	numeric type
string type	string type
DATETIME	TIMESTAMP
TIMESTAMP	DATETIME
DATETIME	DATE
TIMESTAMP	DATE
DATE	DATETIME

Index Scan in Descending Order

When a query is executed by sorting in descending order as follows, it usually creates a descending index. In this way, you do not have to go through addition procedure.

```
SELECT *
FROM tab
[WHERE ...]
ORDER BY a DESC;
```

However, if you create an ascending index and an descending index in the same column, the possibility of deadlock increases. In order to decrease the possibility of such case, CUBRID

supports the descending scan only with ascending index. Users can use the **USE_DESC_IDX** hint to specify the use of the descending scan. If the hint is not specified, the following three query executions should be considered, provided that the columns listed in the **ORDER BY** clause can use the index.

- Sequential scan + Sort in descending order
- Scan in general ascending order + sort in descending
- Scan in descending order that does not require a separate scan

Although the **USE_DESC_IDX** hint is omitted for the scan in descending order, the query optimizer decides the last execution plan of the three listed for an optimal plan.

Note

The **USE_DESC_IDX** hint is not supported for the join query.

```
CREATE TABLE di (i INT);
CREATE INDEX i_di_i on di (i);
INSERT INTO di VALUES (5),(3),(1),(4),(3),(5),(2),(5);
```

The query will be executed as an ascending scan without **USE_DESC_IDX** hint.

```
-- The query will be executed with an ascending scan.

SELECT  *
FROM di
WHERE i > 0
LIMIT 3;
```

```
Query plan:

Index scan(di di, i_di_i, (di.i range (0 gt_inf max) and inst_num()
range (min inf_le 3)) (covers))

             i
=============
             1
             2
             3
```

If you add **USE_DESC_IDX** hint to the above query, a different result will be shown by descending scan.

```
-- We now run the following query, using the ''use_desc_idx'' SQL
hint:

SELECT /*+ USE_DESC_IDX */ *
FROM di
WHERE i > 0
LIMIT 3;
```

```
Query plan:
 Index scan(di di, i_di_i, (di.i range (0 gt_inf max) and inst_num()
range (min inf_le 3)) (covers) (desc_index))

            i
=============
            5
            5
            5
```

The following example requires descending **ORDER BY** clause. In this case, there is no **USE_DESC_IDX** but do the descending scan.

```
-- We also run the same query, this time asking that the results are
displayed in descending order.
-- However, no hint is given.
-- Since ORDER BY...DESC clause exists, CUBRID will use descending
scan, even though the hint is not given,
-- thus avoiding to sort the records.

SELECT *
FROM di
WHERE i > 0
ORDER BY i DESC LIMIT 3;
```

```
Query plan:
 Index scan(di di, i_di_i, (di.i range (0 gt_inf max)) (covers)
(desc_index))

            i
=============
            5
            5
            5
```

Optimizing GROUP BY Clause

GROUP BY clause optimization works on the premise that if all columns in the **GROUP BY** clause are included in an index, CUBRID can use the index upon executing a query, so CUBRID don't execute a separate sorting job. The columns in the **GROUP BY** clause must exist in front side of the column forming the index.

```
SELECT *
FROM tab
WHERE col1 > 0
GROUP BY col1,col2;
```

- You can use the index consisting of *tab* (*col1*, *col2*) for optimization.
- The index consisting of *tab* (*col1*, *col2*, *col3*) can be used because *col3* which is not referred to by **GROUP BY** comes after *col1* and *col2*.
- You cannot use the index consisting of *tab* (*col1*) for optimization.
- You also cannot use the index consisting of *tab* (*col3*, *col1*, *col2*) or *tab* (*col1*, *col3*, *col2*), because *col3* is not located at the back of the column in the **GROUP BY** clause.

You can use the index if the column condition is a constant although the column consisting of the index doesn't exist in the **GROUP BY** clause.

```
SELECT *
FROM tab
WHERE col2=val
GROUP BY col1,col3;
```

If there is any index that consists of *tab* (*col1*, *col2*, *col3*) in the above example, use the index for optimizing **GROUP BY**.

Row sorting by **GROUP BY** is not required, because you can get the result as the requested order on the index scan.

If the index consisting of the **GROUP BY** column and the first column of the index is **NOT NULL**, even though there is no **WHERE** clause, the **GROUP BY** optimization will be applied.

If there is an index made of **GROUP BY** columns even when using aggregate functions, **GROUP BY** optimization is applied.

```
CREATE INDEX i_T_a_b_c ON T(a, b, c);
SELECT a, MIN(b), c, MAX(b) FROM T WHERE a > 18 GROUP BY a, b;
```

Note

When a column of **DISTINCT** or a **GROUP BY** clause contains the subkey of a index, loose index scan adjusts the scope dynamically to unique values of the each columns constituting the partial key, and starts the search of a B-tree. Regarding this, see Loose Index Scan.

Example

```
CREATE TABLE tab (i INT, j INT, k INT);
CREATE INDEX i_tab_j_k ON tab (j, k);
INSERT INTO tab VALUES (1,2,3),(6,4,2),(3,4,1),(5,2,1),(1,5,5),(2,6,
6),(3,5,4);

UPDATE STATISTICS on tab;
```

The following example shows that indexes consisting of *tab* (*j*, *k*) are used and no separate sorting process is required because **GROUP BY** is executed by *j* and *k* columns.

```
SELECT /*+ RECOMPILE */ j,k
FROM tab
WHERE j > 0
GROUP BY j,k;

--  the  selection from the query plan dump shows that the index
i_tab_j_k was used and sorting was not necessary
--  (/* ---> skip GROUP BY */)
```

```
Query plan:
iscan
    class: tab node[0]
    index: i_tab_j_k term[0]
    sort:  2 asc, 3 asc
    cost:  1 card 0

Query stmt:
select tab.i, tab.j, tab.k from tab tab where ((tab.j> ?:0 )) group
by tab.j, tab.k
/* ---> skip GROUP BY */
           i            j            k
           5            2            1
           1            2            3
           3            4            1
           6            4            2
           3            5            4
```

```
            1            5            5
            2            6            6
```

The following example shows that an index consisting of *tab* (*j*, *k*) is used and no separate sorting process is required while **GROUP BY** is executed by *j* and *k* columns, no condition exists for *j*, and *j* column has **NOT NULL** attribute.

```
ALTER TABLE tab CHANGE COLUMN j j INT NOT NULL;

SELECT *
FROM tab
GROUP BY j,k;
```

```
--  the  selection from the query plan dump shows that the index
i_tab_j_k was used (since j has the NOT NULL constraint )
--  and sorting was not necessary (/* ---> skip GROUP BY */)
Query plan:
iscan
    class: tab node[0]
    index: i_tab_j_k
    sort:  2 asc, 3 asc
    cost:  1 card 0

Query stmt: select tab.i, tab.j, tab.k from tab tab group by tab.j,
tab.k
/* ---> skip GROUP BY */
=== <Result of SELECT Command in Line 1> ===
            i            j            k
=======================================
            5            2            1
            1            2            3
            3            4            1
            6            4            2
            3            5            4
            1            5            5
            2            6            6
```

```
CREATE TABLE tab (k1 int, k2 int, k3 int, v double);
INSERT INTO tab
    SELECT
        RAND(CAST(UNIX_TIMESTAMP() AS INT)) MOD 5,
        RAND(CAST(UNIX_TIMESTAMP() AS INT)) MOD 10,
        RAND(CAST(UNIX_TIMESTAMP() AS INT)) MOD 100000,
        RAND(CAST(UNIX_TIMESTAMP() AS INT)) MOD 100000
    FROM db_class a, db_class b, db_class c, db_class d LIMIT 20000;
CREATE INDEX idx ON tab(k1, k2, k3);
```

If you create tables and indexes of the above, the following example runs the **GROUP BY** with *k1*, *k2* columns and performs an aggregate function in *k3*; therefore, the index which consists of *tab* (*k1*, *k2*, *k3*) is used and no sort processing is required. In addition, because all columns of *k1*, *k2*, *k3* of **SELECT** list are present in the index configured in the *tab* (*k1*, *k2*, *k3*), covering index is applied.

```
SELECT /*+ RECOMPILE INDEX_SS */ k1, k2, SUM(DISTINCT k3)
FROM tab
WHERE k2 > -1 GROUP BY k1, k2;
```

```
Query plan:

iscan
    class: tab node[0]
    index: idx term[0] (covers) (index skip scan)
    sort:  1 asc, 2 asc
    cost:  85 card 2000

Query stmt:

select tab.k1, tab.k2, sum(distinct tab.k3) from tab tab where
(tab.k2> ?:0 ) group by tab.k1, tab.k2

/* ---> skip GROUP BY */
```

The following example performs **GROUP BY** clause with *k1*, *k2* columns; therefore, the index composed with *tab* (*k1*, *k2*, *k3*) is used and no sort processing is required. However, *v* column in the **SELECT** list is not present in the index composed of *tab* (*k1*, *k2*, *k3*); therefore, it does not apply covering index.

```
SELECT /*+ RECOMPILE INDEX_SS */ k1, k2, stddev_samp(v)
FROM tab
WHERE k2 > -1 GROUP BY k1, k2;
```

```
Query plan:

iscan
    class: tab node[0]
    index: idx term[0] (index skip scan)
    sort:  1 asc, 2 asc
    cost:  85 card 2000

Query stmt:

select tab.k1, tab.k2, stddev_samp(tab.v) from tab tab where
(tab.k2> ?:0 ) group by tab.k1, tab.k2
```

```
/* ---> skip GROUP BY */
```

Multiple Key Ranges Optimization

Optimizing the **LIMIT** clause is crucial for performance because the most queries have limit filter. A representative optimization of this case is Multiple Key Ranges Optimization.

Multiple Key Ranges Optimization generate the query result with Top N Sorting to scan only some key ranges in an index rather than doing a full index scan. Top N Sorting always keeps the best N tuples sorted rather than selecting all tuples and then sorting. Therefore, it shows the outstanding performance.

For example, when you search only the recent 10 posts which your friends wrote, CUBRID which applied Multiple KEY Ranges Optimization finds the result not by sorting after finding all your friends' posts, but by scanning the index which keeps the recent 10 sorted posts of each friends.

An example of Multiple Key Ranges Optimization is as follows.

```
CREATE TABLE t (a int, b int);
CREATE INDEX i_t_a_b ON t (a,b);

-- Multiple key range optimization
SELECT *
FROM t
WHERE a IN (1,2,3)
ORDER BY b
LIMIT 2;
```

```
Query plan:
iscan
class: t node[0]
index: i_t_a_b term[0] (covers) (multi_range_opt)
sort: 1 asc, 2 asc
cost: 1 card 0
```

On a single table, multiple key range optimization can be applied if below conditions are satisfied.

```
SELECT /*+ hints */ ...
FROM table
WHERE col_1 = ? AND col_2 = ? AND ... AND col(j-1) = ?
AND col_(j) IN (?, ?, ...)
AND col_(j+1) = ? AND ... AND col_(p-1) = ?
AND key_filter_terms
ORDER BY col_(p) [ASC|DESC], col_(p+1) [ASC|DESC], ... col_(p+k-1)
```

```
[ASC|DESC]
LIMIT n;
```

Firstly, upper limit(*n*) for **LIMIT** should be less than or equal to the value of **multi_range_optimization_limit** system parameter.

And you need the proper index to the multiple key range optimization, this index should cover all *k* columns specified in the **ORDER BY** clause. In other words, this index should include all *k* columns specified in the **ORDER BY** clause and the sorting order should be the same as the columns' order. Also this index should include all columns used in **WHERE** clause.

Among columns that comprise the index,

- Columns in front of range condition(e.g. IN condition) are represented as equivalent condition(=).
- Only one column with range condition exists.
- Columns after range condition exist as key filters.
- There should be no data filtering condition. In other words, the index should include all columns used in **WHERE** clause.
- Columns after the key filter exist in **ORDER BY** clause.
- Columns of key filter condition always should not the column of **ORDER BY** clause.
- If key filter condition with correlated subquery exists, related columns to this should be included into **WHERE** clause with no range condition.

On the below example, Multiple Key Ranges Optimization can be applied.

```
CREATE TABLE t (a INT, b INT, c INT, d INT, e INT);
CREATE INDEX i_t_a_b ON t (a,b,c,d,e);

SELECT *
FROM t
WHERE a = 1 AND b = 3 AND c IN (1,2,3) AND d = 3
ORDER BY e
LIMIT 2;
```

Queries with multiple joined tables can also support Multiple Key Ranges Optimization:

```
SELECT /*+ hints */ ...
FROM table_1, table_2, ... table_(sort), ...
WHERE col_1 = ? AND col_2 = ? AND ...
AND col_(j) IN (?, ?, ... )
AND col_(j+1) = ? AND ... AND col_(p-1) = ?
```

```
AND key_filter_terms
AND join_terms
ORDER BY col_(p) [ASC|DESC], col_(p+1) [ASC|DESC], ... col_(p+k-1)
[ASC|DESC]
LIMIT n;
```

If queries with multiple joined tables can support Multiple Key Ranges Optimization, below conditions should be satisfied:

- Columns in **ORDER BY** clause only exist on one table, and this table should satisfy all required conditions by Multiple Key Ranges Optimization on a single table query. Let the “sort table” be the table that contains all sorting columns.
- All columns of “sort table” specified in a JOIN condition between “sort table” and “outer tables” should be included on an index. In other words, there should be no data filtering condition.
- All columns of “sort table” specified in a JOIN condition between “sort table” and “outer tables” should be included on the **WHERE** clause with no range condition.

Note

In most cases available to apply Multiple Key Ranges Optimization, this optimization shows the best performance. However, if you do not want this optimization on the special case, specify **NO_MULTI_RANGE_OPT** hint to the query. For details, see Using SQL Hint.

Index Skip Scan

Index Skip Scan (here after ISS) is an optimization method that allows ignoring the first column of an index when the first column of the index is not included in the condition but the following column is included in the condition (in most cases, =).

Applying ISS is considered when **INDEX_SS** for specific tables is specified through a query hint and the below cases are satisfied.

1. The query condition should be specified from the second column of the composite index.
2. The used index should not be a filtered index.
3. The first column of an index should not be a range filter or key filter.
4. A hierarchical query is not supported.
5. A query which an aggregate function is included is not supported.

In a **INDEX_SS** hint, a list of table to consider applying ISS, can be input; if a list of table is omitted, applying ISS for all tables can be considered.

```
/*+ INDEX_SS */
/*+ INDEX_SS(tbl1) */
/*+ INDEX_SS(tbl1, tbl2) */
```

Note

When “INDEX_SS” is input, the ISS hint is applied to all tables; when “INDEX_SS()” is input, this hint is ignored.

```
CREATE TABLE t1 (id INT PRIMARY KEY, a INT, b INT, c INT);
CREATE TABLE t2 (id INT PRIMARY KEY, a INT, b INT, c INT);
CREATE INDEX i_t1_ac ON t1(a,c);
CREATE INDEX i_t2_ac ON t2(a,c);

INSERT INTO t1 SELECT rownum, rownum, rownum, rownum
FROM db_class x1, db_class x2, db_class LIMIT 10000;

INSERT INTO t2 SELECT id, a%5, b, c FROM t1;


SELECT /*+ INDEX_SS */ *
FROM t1, t2
WHERE t1.b<5 AND t1.c<5 AND t2.c<5
USING INDEX i_t1_ac, i_t2_ac limit 1;

SELECT /*+ INDEX_SS(t1) */ *
FROM t1, t2
WHERE t1.b<5 AND t1.c<5 AND t2.c<5
USING INDEX i_t1_ac, i_t2_ac LIMIT 1;

SELECT /*+ INDEX_SS(t1, t2) */ *
FROM t1, t2
WHERE t1.b<5 AND t1.c<5 AND t2.c<5
USING INDEX i_t1_ac, i_t2_ac LIMIT 1;
```

Generally, ISS should consider several columns (C1, C2, ..., Cn). Here, a query has the conditions for the consecutive columns and the conditions are started from the second column (C2) of the index.

```
INDEX (C1, C2, ..., Cn);

SELECT ... WHERE C2 = x AND C3 = y AND ... AND Cp = z; -- p <= n
```

```
SELECT ... WHERE C2 < x AND C3 >= y AND ... AND Cp BETWEEN (z AND
w); -- other conditions than equal
```

The query optimizer eventually determines whether ISS is the most optimum access method based on the cost. ISS is applied under very specific situations, such as when the first column of an index has a very small number of **DISTINCT** values compared to the number of records. In this case, ISS provides higher performance compared to Index Full Scan. For example, when the first column of index columns has very low cardinality, such as the value of men/women or millions of records with the value of 1~100, it may be inefficient to perform index scan by using the first column value. So ISS is useful in this case.

ISS skips reading most of the index pages in the disk and uses range search which is dynamically readjusted. Generally, ISS can be applied to a specific scenario when the number of **DISTINCT** values in the first column is very small. If ISS is applied to this case, ISS provides significantly higher performance than the index full scan. However, it means improper index creation that ISS is applied to a lot queries. So DBA should consider whether readjusting the indexes or not.

```
CREATE TABLE tbl (name STRING, gender CHAR (1), birthday DATETIME);

INSERT INTO tbl
SELECT ROWNUM, CASE (ROWNUM MOD 2) WHEN 1 THEN 'M' ELSE 'F' END,
SYSDATETIME
FROM db_class a, db_class b, db_class c, db_class d, db_class e
LIMIT 360000;

CREATE INDEX idx_tbl_gen_name ON tbl (gender, name);
-- Note that gender can only have 2 values, 'M' and 'F' (low
cardinality)

UPDATE STATISTICS ON ALL CLASSES;
```

```
-- csql>;plan simple
-- this will qualify to use Index Skip Scanning
SELECT /*+ RECOMPILE INDEX_SS */ *
FROM tbl
WHERE name = '1000';
```

```
Query plan:

 Index scan(tbl tbl, idx_tbl_gen_name, tbl.[name]= ?:0  (index skip
scan))
```

```
-- csql>;plan simple
-- this will qualify to use Index Skip Scanning
```

```
SELECT /*+ RECOMPILE INDEX_SS */ *
FROM tbl
WHERE name between '1000' and '1050';
```

```
Query plan:

 Index scan(tbl tbl, idx_tbl_gen_name, (tbl.[name]>= ?:0  and tbl.
[name]<= ?:1 ) (index skip scan))
```

Loose Index Scan

When **GROUP BY** clause or **DISTINCT** column includes a subkey of a index, loose index scan starts B-tree search by adjusting the range dynamically for unique value of each of the columns that make up the subkey. Therefore, it is possible to significantly reduce the scanning area of B-tree.

Applying loose index scan is advantageous when the cardinality of the grouped column is very small, compared to the total data amount.

Loose index scan optimization is considered to be applied when **INDEX_LS** is input as a hint and the below cases are satisfied:

1. when an index covers all **SELECT** list, that is, covered index is applied.
2. when the statement is **SELECT DISTINCT**, **SELECT** ... **GROUP BY** statement or a single tuple **SELECT**.
3. all aggregate functions (with the exception of **MIN**/**MAX**) must have **DISTINCT** input
4. **COUNT(*)** should not be used
5. when cardinality of the used subkey is 100 times smaller than the cardinality of the whole index

a subkey is a prefix part in a composite index; e.g. when there is INDEX(a, b, c, d), (a), (a, b) or (a, b, c) belongs to the subkey.

When you run the below query regarding the above table,

```
SELECT /*+ INDEX_LS */ a, b FROM tbl GROUP BY a;
```

CUBRID cannot use a subkey because there is no condition for the column a. However, if the condition of the subkey is specified as follows, loose index scan can be applied.

```
SELECT /*+ INDEX_LS */ a, b FROM tbl WHERE a > 10 GROUP BY a;
```

As follows, a subkey can be used when the grouped column is on the first and the WHERE-condition column is on the following position; therefore, also in this case, loose index scan can be applied.

```
SELECT /*+ INDEX_LS */ a, b FROM tbl WHERE b > 10 GROUP BY a;
```

The following shows the cases when loose index scan optimization is applied.

```
CREATE TABLE tbl1 (
    k1 INT,
    k2 INT,
    k3 INT,
    k4 INT
);

INSERT INTO tbl1
SELECT ROWNUM MOD 2, ROWNUM MOD 400, ROWNUM MOD 80000, ROWNUM
FROM db_class a, db_class b, db_class c, db_class d, db_class e
LIMIT 360000;

CREATE INDEX idx ON tbl1 (k1, k2, k3);

CREATE TABLE tbl2 (
    k1 INT,
    k2 INT
);

INSERT INTO tbl2 VALUES (0, 0), (1, 1), (0, 2), (1, 3), (0, 4), (0,
100), (1000, 1000);

UPDATE STATISTICS ON ALL CLASSES;
```

```
-- csql>;plan simple
-- add a condition to the grouped column, k1 to enable loose index
scan
SELECT /*+ RECOMPILE INDEX_LS */ DISTINCT k1
FROM tbl1
WHERE k1 > -1000000 LIMIT 20;
```

```
Query plan:

 Sort(distinct)
    Index scan(tbl1 tbl1, idx, (tbl1.k1> ?:0 ) (covers) (loose index
scan on prefix 1))
```

```
-- csql>;plan simple
-- different key ranges/filters
SELECT /*+ RECOMPILE INDEX_LS */ DISTINCT k1
FROM tbl1
WHERE k1 >= 0 AND k1 <= 1;
```

```
Query plan:

 Sort(distinct)
    Index scan(tbl1 tbl1, idx, (tbl1.k1>= ?:0  and tbl1.k1<= ?:1 )
(covers) (loose index scan on prefix 1))
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE INDEX_LS */ DISTINCT k1, k2
FROM tbl1
WHERE k1 >= 0 AND k1 <= 1 AND k2 > 3 AND k2 < 11;
```

```
Query plan:

 Sort(distinct)
    Index scan(tbl1 tbl1, idx, (tbl1.k1>= ?:0  and tbl1.k1<= ?:1 ),
[(tbl1.k2> ?:2  and tbl1.k2< ?:3 )] (covers) (loose index scan on
prefix 2))
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE INDEX_LS */ DISTINCT k1, k2
FROM tbl1
WHERE k1 >= 0 AND k1 + k2 <= 10;
```

```
Query plan:

 Sort(distinct)
    Index scan(tbl1 tbl1, idx, (tbl1.k1>= ?:0 ),
[tbl1.k1+tbl1.k2<=10] (covers) (loose index scan on prefix 2))
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE INDEX_LS */ tbl1.k1, tbl1.k2
FROM tbl2 INNER JOIN tbl1
ON tbl2.k1 = tbl1.k1 AND tbl2.k2 = tbl1.k2
GROUP BY tbl1.k1, tbl1.k2;
```

```
Sort(group by)
   Nested loops
       Sequential scan(tbl2 tbl2)
       Index scan(tbl1 tbl1, idx, tbl2.k1=tbl1.k1 and
tbl2.k2=tbl1.k2 (covers) (loose index scan on prefix 2))
```

```
SELECT /*+ RECOMPILE INDEX_LS */ MIN(k2), MAX(k2)
FROM tbl1;
```

```
Query plan:

 Index scan(tbl1 tbl1, idx (covers) (loose index scan on prefix 2))
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE INDEX_LS */ SUM(DISTINCT k1), SUM(DISTINCT k2)
FROM tbl1;
```

```
Query plan:

 Index scan(tbl1 tbl1, idx (covers) (loose index scan on prefix 2))
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE INDEX_LS */ DISTINCT k1
FROM tbl1
WHERE k2 > 0;
```

```
Query plan:

 Sort(distinct)
    Index scan(tbl1 tbl1, idx, [(tbl1.k2> ?:0 )] (covers) (loose
index scan on prefix 2))
```

The following shows the cases when loose index scan optimization is not applied.

```
-- csql>;plan simple
-- not enabled when full key is used
SELECT /*+ RECOMPILE INDEX_LS */ DISTINCT k1, k2, k3
FROM tbl1
ORDER BY 1, 2, 3 LIMIT 10;
```

```
Query plan:

 Sort(distinct)
    Sequential scan(tbl1 tbl1)
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE INDEX_LS */ k1, k2, k3
FROM tbl1
WHERE k1 > -10000 GROUP BY k1, k2, k3 LIMIT 10;
```

```
Query plan:

 Index scan(tbl1 tbl1, idx, (tbl1.k1> ?:0 ) (covers))
 skip GROUP BY
```

```
-- csql>;plan simple
-- not enabled when using count star
SELECT /*+ RECOMPILE INDEX_LS */ COUNT(*), k1
FROM tbl1
WHERE k1 > -10000 GROUP BY k1;
```

```
Query plan:

 Index scan(tbl1 tbl1, idx, (tbl1.k1> ?:0 ) (covers))
 skip GROUP BY
```

```
-- csql>;plan simple
-- not enabled when index is not covering
SELECT /*+ RECOMPILE INDEX_LS */ k1, k2, SUM(k4)
FROM tbl1
WHERE k1 > -1 AND k2 > -1 GROUP BY k1, k2 LIMIT 10;
```

```
Query plan:

 Index scan(tbl1 tbl1, idx, (tbl1.k1> ?:0 ), [(tbl1.k2> ?:1 )])
 skip GROUP BY
```

```
-- csql>;plan simple
-- not enabled for non-distinct aggregates
SELECT /*+ RECOMPILE INDEX_LS */ k1, SUM(k2)
```

```
FROM tbl1
WHERE k1 > -1 GROUP BY k1;
```

```
Query plan:

 Index scan(tbl1 tbl1, idx, (tbl1.k1> ?:0 ) (covers))
 skip GROUP BY
```

```
-- csql>;plan simple
SELECT /*+ RECOMPILE */ SUM(k1), SUM(k2)
FROM tbl1;
```

```
Query plan:

 Sequential scan(tbl1 tbl1)
```

In Memory Sort

The "in memory sort(IMS)" feature is an optimization applied to the **LIMIT** queries specifying **ORDER BY**. Normally, when executing a query which specifies **ORDER BY** and **LIMIT** clauses, CUBRID generates the full sorted result set and then applies the **LIMIT** operator to this result set. With the IMS optimization, instead of generating the whole result set, CUBRID uses an in-memory binary heap in which only tuples satisfying the **ORDER BY** and **LIMIT** clauses are allowed. This optimization improves performance by eliminating the need for a full unordered result set.

Whether this optimization is applied or not is not transparent to users. CUBRID decides to use in memory sort in the following situation:

- The query specifies **ORDER BY** and **LIMIT** clauses.
- The size of the final result (after applying the **LIMIT** clause) is less than the amount of memory used by external sort (see **sort_buffer_size** in Memory-Related Parameters).

Note that IMS considers the actual size of the result and not the count of tuples the result contains. For example, for the default sort buffer size (two megabytes), this optimization will be applied for a **LIMIT** value of 524,288 tuples consisting of one 4 byte **INTEGER** type but only for ~2,048 tuples of **CHAR** (1024) values. This optimization is not applied to queries requiring **DISTINCT** ordered result sets.

SORT-LIMIT optimization

The SORT-LIMIT optimization applies to queries specifying **ORDER BY** and **LIMIT** clauses. The idea behind it is to evaluate the **LIMIT** operator as soon as possible in the query plan in order to benefit from the reduced cardinality during joins.

A SORT-LIMIT plan can be generated when the following conditions are met:

- All referred tables in the **ORDER BY** clause belong to the SORT-LIMIT plan.
- A table belonging to a SORT-LIMIT plan is either:
  - The owner of a foreign key from a fk->pk join
  - The left side of a **LEFT JOIN**.
  - The right side of a **RIGHT JOIN**.
- **LIMIT** rows should be specified as less rows than the value of **sort_limit_max_count** system parameter(default: 1000).
- Query does not have cross joins.
- Query joins at least two relations.
- Query does not have a **GROUP BY** clause.
- Query does not specify **DISTINCT**.
- **ORDER BY** expressions can be evaluated during scan.

  For example, the below query cannot apply SORT-LIMIT plan because **SUM** cannot be evaluated during scan.

```
SELECT SUM(u.i) FROM u, t where u.i = t.i ORDER BY 1 LIMIT 5;
```

The below is an example of planning SORT-LIMIT.

```
CREATE TABLE t(i int PRIMARY KEY, j int, k int);
CREATE TABLE u(i int, j int, k int);
ALTER TABLE u ADD constraint fk_t_u_i FOREIGN KEY(i) REFERENCES t(i);
CREATE INDEX i_u_j ON u(j);

INSERT INTO t SELECT ROWNUM, ROWNUM, ROWNUM FROM _DB_CLASS a,
_DB_CLASS b LIMIT 1000;
INSERT INTO u SELECT 1+(ROWNUM % 1000), RANDOM(1000), RANDOM(1000)
FROM _DB_CLASS a, _DB_CLASS b, _DB_CLASS c LIMIT 5000;
```

```
SELECT /*+ RECOMPILE */ * FROM u, t WHERE u.i = t.i AND u.j > 10
ORDER BY u.j LIMIT 5;
```

The above **SELECT** query's plan is printed out as below; we can see "(sort limit)".

```
Query plan:

idx-join (inner join)
    outer: temp(sort limit)
                subplan: iscan
                             class: u node[0]
                             index: i_u_j term[1]
                             cost:  1 card 0
                cost:  1 card 0
    inner: iscan
                class: t node[1]
                index: pk_t_i term[0]
                cost:  6 card 1000
    sort:  2 asc
    cost:  7 card 0
```

Optimization Using Rewrite

Join transforming Optimization

The join transforming optimization is a method to reduce join operations and improve the query performance by transforming the joins with the tables that do not affect the query results.

In the join transforming optimization, there are three operations:

1. Transforming **OUTER JOIN** to **INNER JOIN**
2. Eliminating **INNER JOIN**
3. Eliminating **LEFT OUTER JOIN**

To disable the join elimination optimization, use the **NO_ELIMINATE_JOIN** hint.

Transforming **OUTER JOIN** to **INNER JOIN**

OUTER JOIN has a predefined join order with the leading table and trailing table, which restricts join order optimization. For performance improvement, **CUBRID** converts **OUTER JOIN** to **INNER**

JOIN if there is a predicate which is not nullable on the trailing table. A nullable predicate is excluded from this transformation and satisfy the following conditions:

1. Predicates written in the **ON** clause.
2. Using **NULL** transformation functions. This includes **COALESCE (), NVL (), NVL2 (), DECODE (), IF (), IFNULL (), CONCAT_WS ()**.
3. **IS NULL ,CASE** statements are also not targets for **Predicate Push**.

The following example shows an optimization that transform **OUTER JOIN**to **INNER JOIN**.

```
--create table
CREATE TABLE t1 (col1 int, col2 int);
INSERT INTO t1 values (1,1),(2,2),(3,2);

-- csql> ;plan simple
SELECT /*+ recompile */ *
FROM t1 a LEFT OUTER JOIN t1 b on a.col1 = b.col2
WHERE b.col1 = 2;
```

```
Query Plan:
  NESTED LOOPS (inner join)
    TABLE SCAN (a)
    TABLE SCAN (b)
```

The following example demonstrates that the transformation from **OUTER JOIN** to **INNER JOIN** is not possible due to the nullable predicate.

```
-- csql> ;plan simple
SELECT /*+ recompile */ *
FROM t1 a LEFT OUTER JOIN t1 b on a.col1 = b.col2
WHERE nvl(b.col1,2) = 2;
```

```
Query Plan:
  NESTED LOOPS (left outer join)
    TABLE SCAN (a)
    TABLE SCAN (b)
```

Eliminating **INNER JOIN**

The relationship between a table with a primary key (**PK**) and a table with a foreign key (**FK**) that refers to that primary key (**PK**) is called a parent-child relationship. In **INNER JOIN** of a parent-

child relationship, if there are no references to the parent table other than the join conditions, eliminating the join with the parent table does not affect the query results. In this case, by elimination the join with the parent table, the join operation is reduced and the query performance is improved.

To eliminate the **INNER JOIN**, the following conditions must be satisfied:

1. Perform the **INNER JOIN** of the parent-child relationship.
2. All columns of the primary key (**PK**) and the foreign key (**FK**) must be used in the join condition.
3. The primary key (**PK**) column must be used in the same join condition as a foreign key (**FK**) column that refers to the corresponding primary key (**PK**) column.
4. All join conditions must use the equality (=) comparison.
5. There must be no references to the parent table other than the join conditions.
6. The **ALL** keyword must not be used when querying the tables that have an inheritance relationship.

When eliminating the join with the parent table, if the foreign key (**FK**) column in the child table does not have the **NOT NULL** constraint, the **IS NOT NULL** condition is added to that column.

The following is an example of eliminating the **INNER JOIN**.

```
call login ('public') on class db_user;

/* current_user: public */
drop table if exists child_tbl, parent_tbl;

create table parent_tbl (
    id int,
    sub_id int,
    name varchar (100),
    filter int,
    primary key (id, sub_id)
);

insert into parent_tbl
select
    ((rownum - 1) / 10) + 1 as id,
    (((rownum - 1) % 10) + 1) * 10 as sub_id,
    'Parent-' || lpad (rownum, 3) as name,
    ((rownum - 1) % 10) + 1 as filter
from db_root
connect by level <= 100;

create table child_tbl (
```

```
    id int,
    name varchar (100),
    filter int,
    parent_id int not null,
    parent_sub_id int,
    primary key (id),
    foreign key (parent_id, parent_sub_id) references parent_tbl
(id, sub_id)
);

insert into child_tbl
select
    rownum as id,
    'Child-' || lpad (rownum, 5) as name,
    ((rownum - 1) % 100) + 1 as filter,
    id as parent_id,
    sub_id as parent_sub_id
from parent_tbl, (select level from db_root connect by level <= 100);

update statistics on parent_tbl, child_tbl with fullscan;
```

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 1 and c.parent_id = 1
order by
    c.name;
```

The join with the *parent_tbl* table has been eliminated, and the **IS NOT NULL** condition for the *parent_sub_id* column of the *child_tbl* table has been added.

```
Query plan:

temp(order by)
    subplan: iscan
                class: c node[0]
                index: fk_child_tbl_parent_id_parent_sub_id term[1]
                filtr: term[2]
                sargs: term[0]
                sort:  5 asc
                cost:  8 card 1
    sort:  2 asc
```

```
    cost:  14 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
child_tbl c where c.parent_id= ?:0  and c.filter= ?:1  and
c.parent_sub_id is not null  order by 2
```

```
           id  name                      filter    parent_id
parent_sub_id
=============================================================================
=======
            1  'Child-    1'                  1
1             10
          101  'Child-  101'                  1
1             20
          201  'Child-  201'                  1
1             30
          301  'Child-  301'                  1
1             40
          401  'Child-  401'                  1
1             50
          501  'Child-  501'                  1
1             60
          601  'Child-  601'                  1
1             70
          701  'Child-  701'                  1
1             80
          801  'Child-  801'                  1
1             90
          901  'Child-  901'                  1
1            100
```

The following is an example of using the **NO_ELIMINATE_JOIN** hint.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile no_eliminate_join */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 1 and c.parent_id = 1
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated because the **NO_ELIMINATE_JOIN** hint was used.

```
Query plan:

temp(order by)
    subplan: idx-join (inner join)
                outer: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[4]
(covers)
                           cost:  1 card 10
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[1] AND term[3]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  4 card 1
    sort:  2 asc
    cost:  10 card 1

Query stmt:

select /*+ NO_ELIMINATE_JOIN */ c.id, c.[name], c.filter,
c.parent_id, c.parent_sub_id from parent_tbl p, child_tbl c where
p.sub_id=c.parent_sub_id and p.id= ?:0  and c.parent_id= ?:1  and
c.filter= ?:2  and p.id=c.parent_id order by 2
```

```
           id  name                      filter    parent_id
parent_sub_id
=============================================================================
=======
            1  'Child-    1'                  1
1             10
          101  'Child-  101'                  1
1             20
          201  'Child-  201'                  1
1             30
          301  'Child-  301'                  1
1             40
          401  'Child-  401'                  1
1             50
          501  'Child-  501'                  1
1             60
          601  'Child-  601'                  1
1             70
          701  'Child-  701'                  1
1             80
```

```
          801  'Child-  801'                  1
1           90
          901  'Child-  901'                  1
1          100
```

All columns of the primary key (**PK**) and the foreign key (**FK**) must be used in the join condition.

The following is an example where only the *id* column of the *parent_tbl* table and the *parent_id* column of the *child_tbl* table are used as the join conditions.

```
/* current_user: public */
set optimization level 513;

select /** recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id
where
    c.filter = 2 and c.parent_id = 2 and c.parent_sub_id = 20
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated because the *sub_id* column of the *parent_tbl* table and the *parent_sub_id* column of the *child_tbl* table were not used as join conditions.

```
Query plan:

temp(order by)
    subplan: nl-join (cross join)
                outer: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[2] AND term[3]
                           sargs: term[1]
                           cost:  3 card 1
                inner: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[4]
(covers)
                           cost:  1 card 10
                cost:  4 card 10
    sort:  2 asc
    cost:  10 card 10
```

```
Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
parent_tbl p, child_tbl c where p.id= ?:0  and c.parent_sub_id= ?:1
and c.parent_id= ?:2  and c.filter= ?:3  and p.id=c.parent_id order
by 2
```

```
           id  name                          filter    parent_id
parent_sub_id
=============================================================================
=======
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
         1102  'Child- 1102'                      2
2             20
```

The primary key (**PK**) column must be used in the same join condition as the foreign key (**FK**) column that refers to the corresponding primary key (**PK**).

The following is an example where the join conditions compare the *id* column with the *parent_sub_id* column, and the *sub_id column* with the *parent_id* column.

```
/* current_user: public */
set optimization level 513;

select /** recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_sub_id and p.sub_id =
```

```
c.parent_id
where
    c.filter = 3 and c.parent_id = 3
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated because the *id* column was not compared to the *parent_id* column, and the *sub_id* column was not compared to the *parent_sub_id* column.

```
Query plan:

temp(order by)
    subplan: idx-join (inner join)
                outer: sscan
                           class: p node[0]
                           sargs: term[3]
                           cost:  1 card 1
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[0] AND term[4]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  5 card 1
    sort:  2 asc
    cost:  11 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
parent_tbl p, child_tbl c where p.id=c.parent_sub_id and p.sub_id= ?:
0  and c.parent_id= ?:1  and c.filter= ?:2  and p.sub_id=c.parent_id
order by 2
```

```
There are no results.
0 row selected.
```

All join conditions must use the equality (=) comparison.

The following is an example where the join conditions do not use the equality (=) comparison.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
```

```
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id <
c.parent_sub_id
where
    c.filter = 4 and c.parent_id = 4 and c.parent_sub_id = 40
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated because the comparison between the *sub_id* column of the *parent_tbl* table and the *parent_sub_id* column of the *child_tbl* table in the join condition does not use the equality (=) comparison.

```
Query plan:

temp(order by)
    subplan: nl-join (cross join)
                outer: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[5]
AND term[6] (covers)
                           cost:  1 card 1
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[3] AND term[4]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  4 card 1
    sort:  2 asc
    cost:  10 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
parent_tbl p, child_tbl c where (p.sub_id< ?:0 ) and p.id= ?:1  and
c.parent_sub_id= ?:2  and c.parent_id= ?:3  and c.filter= ?:4  and
p.id=c.parent_id and (p.sub_id<c.parent_sub_id) order by 2
```

```
           id  name                      filter    parent_id
parent_sub_id
=============================================================================
=======
         3304  'Child- 3304'                  4
4             40
         3304  'Child- 3304'                  4
4             40
```

```
          3304  'Child- 3304'                    4
4             40
```

There must be no references to the parent table other than the join conditions.
The following is an example that retrieves the *name* column from the *parent_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /** recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id, p.name
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 5 and c.parent_id = 5
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated because it is retrieving the values from the *name* column of the *parent_tbl* table.

```
Query plan:

temp(order by)
    subplan: idx-join (inner join)
                outer: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[4]
                           cost:  1 card 10
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[1] AND term[3]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  4 card 1
    sort:  2 asc
    cost:  10 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id, p.
[name] from parent_tbl p, child_tbl c where p.sub_id=c.parent_sub_id
```

```
and p.id= ?:0  and c.parent_id= ?:1  and c.filter= ?:2  and
p.id=c.parent_id order by 2
```

```
           id  name                          filter    parent_id
parent_sub_id  name
================================================================================
==============================
         4005  'Child- 4005'                      5
5             10  'Parent- 41'
         4105  'Child- 4105'                      5
5             20  'Parent- 42'
         4205  'Child- 4205'                      5
5             30  'Parent- 43'
         4305  'Child- 4305'                      5
5             40  'Parent- 44'
         4405  'Child- 4405'                      5
5             50  'Parent- 45'
         4505  'Child- 4505'                      5
5             60  'Parent- 46'
         4605  'Child- 4605'                      5
5             70  'Parent- 47'
         4705  'Child- 4705'                      5
5             80  'Parent- 48'
         4805  'Child- 4805'                      5
5             90  'Parent- 49'
         4905  'Child- 4905'                      5
5            100  'Parent- 50'
```

The following is an example with a condition on the *filter* column of the *parent_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 5 and c.parent_id = 5 and p.filter = 5
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated because it is retrieving the records where the *filter* column of the *parent_tbl* table has a value of 5.

```
Query plan:

temp(order by)
    subplan: idx-join (inner join)
                outer: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[5]
                           sargs: term[3]
                           cost:  1 card 1
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[1] AND term[4]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  4 card 1
    sort:  2 asc
    cost:  10 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
parent_tbl p, child_tbl c where p.sub_id=c.parent_sub_id and p.id= ?:
0  and p.filter= ?:1  and c.parent_id= ?:2  and c.filter= ?:3  and
p.id=c.parent_id order by 2
```

```
           id  name                        filter    parent_id
parent_sub_id
=============================================================================
=======
         4405  'Child- 4405'                    5
5             50
```

The following is an example that sorts the results based on the *name* column of the *parent_tbl* table.

```
select /** recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 5 and c.parent_id = 5
order by
    p.name desc;
```

The join with the *parent_tbl* table was not eliminated because it is sorting the results based on the *name* column of the *parent_tbl* table in descending order.

```
Query plan:

temp(order by)
    subplan: idx-join (inner join)
                outer: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[4]
                           cost:  1 card 10
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[1] AND term[3]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  4 card 1
    sort:  6 desc
    cost:  10 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id, p.
[name] from parent_tbl p, child_tbl c where p.sub_id=c.parent_sub_id
and p.id= ?:0  and c.parent_id= ?:1  and c.filter= ?:2  and
p.id=c.parent_id order by 6 desc
```

```
           id  name                       filter    parent_id
parent_sub_id
=============================================================================
=======
         4905  'Child- 4905'                   5
5            100
         4805  'Child- 4805'                   5
5             90
         4705  'Child- 4705'                   5
5             80
         4605  'Child- 4605'                   5
5             70
         4505  'Child- 4505'                   5
5             60
         4405  'Child- 4405'                   5
5             50
         4305  'Child- 4305'                   5
5             40
         4205  'Child- 4205'                   5
5             30
         4105  'Child- 4105'                   5
```

```
5             20
         4005  'Child- 4005'                    5
5             10
```

Implicit Equality (=)

Even if there are conditions related to the parent table in the **WHERE** clause other than the join conditions, if the column used in the join conditions is also used in those conditions, the join with the parent table can be eliminated based on implicit equality (=). That condition must use an equality (=) comparison and be compared to a constant value.

The following is an example with an equality (=) comparison condition on the *sub_id* column of the *parent_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 6 and c.parent_id = 6 and p.sub_id = 60
order by
    c.name;
```

The join with the *parent_tbl* table was eliminated because the condition being queried, which retrieves records where the *sub_id* column of the *parent_tbl* table has a value of 60, is equivalent to the condition where the *parent_sub_id* column of the *child_tbl* table has a value of 60.

```
Query plan:

temp(order by)
    subplan: iscan
                class: c node[0]
                index: fk_child_tbl_parent_id_parent_sub_id term[1]
AND term[2]
                filtr: term[3]
                sargs: term[0]
                cost:  3 card 1
    sort:  2 asc
    cost:  9 card 1

Query stmt:
```

```
select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
child_tbl c where c.parent_sub_id= ?:0  and c.parent_id= ?:1  and
c.filter= ?:2  and c.parent_sub_id is not null  order by 2
```

```
           id  name                        filter    parent_id
parent_sub_id
=============================================================================
=======
         5506  'Child- 5506'                    6
6             60
```

The **ALL** keyword must not be used when querying the tables that have an inheritance relationship.

If the **ALL** keyword is used on either the parent or the child table that has an inheritance relationship, the join with the parent table cannot be eliminated.

The following is an example where the **ALL** keyword is used on the *child_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    parent_tbl p
    inner join all child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 7 and c.parent_id = 7
order by
    c.name;
```

The join with the *parent_tbl* table was not eliminated.

```
Query plan:

temp(order by)
    subplan: idx-join (inner join)
                outer: iscan
                           class: p node[0]
                           index: pk_parent_tbl_id_sub_id term[4]
(covers)
                           cost:  1 card 10
```

```
                inner: iscan
                           class: c node[1]
                           index:
fk_child_tbl_parent_id_parent_sub_id term[1] AND term[3]
                           sargs: term[2]
                           cost:  3 card 1
                cost:  4 card 1
    sort:  2 asc
    cost:  10 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
parent_tbl p,  all child_tbl c where p.sub_id=c.parent_sub_id and
p.id= ?:0  and c.parent_id= ?:1  and c.filter= ?:2  and
p.id=c.parent_id order by 2
```

```
           id  name                      filter    parent_id
parent_sub_id
=============================================================================
=======
         6007  'Child- 6007'                  7
7             10
         6107  'Child- 6107'                  7
7             20
         6207  'Child- 6207'                  7
7             30
         6307  'Child- 6307'                  7
7             40
         6407  'Child- 6407'                  7
7             50
         6507  'Child- 6507'                  7
7             60
         6607  'Child- 6607'                  7
7             70
         6707  'Child- 6707'                  7
7             80
         6807  'Child- 6807'                  7
7             90
         6907  'Child- 6907'                  7
7            100
```

The following is an example where the **ONLY** keyword is used on both the *parent_tbl* table and the *child_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
```

```
    c.id, c.name, c.filter, c.parent_id, c.parent_sub_id
from
    only parent_tbl p
    inner join only child_tbl c on p.id = c.parent_id and p.sub_id =
c.parent_sub_id
where
    c.filter = 7 and c.parent_id = 7
order by
    c.name;
```

The join with the *parent_tbl* table was eliminated.

```
Query plan:

temp(order by)
    subplan: iscan
                class: c node[0]
                index: fk_child_tbl_parent_id_parent_sub_id term[1]
                filtr: term[2]
                sargs: term[0]
                sort:  5 asc
                cost:  8 card 1
    sort:  2 asc
    cost:  14 card 1

Query stmt:

select c.id, c.[name], c.filter, c.parent_id, c.parent_sub_id from
child_tbl c where c.parent_id= ?:0  and c.filter= ?:1  and
c.parent_sub_id is not null  order by 2
```

```
          id  name                      filter    parent_id
parent_sub_id
=============================================================================
=======
        6007  'Child- 6007'                   7
7             10
        6107  'Child- 6107'                   7
7             20
        6207  'Child- 6207'                   7
7             30
        6307  'Child- 6307'                   7
7             40
        6407  'Child- 6407'                   7
7             50
        6507  'Child- 6507'                   7
7             60
        6607  'Child- 6607'                   7
7             70
```

```
         6707  'Child- 6707'                  7
7             80
         6807  'Child- 6807'                  7
7             90
         6907  'Child- 6907'                  7
7            100
```

Eliminating **LEFT OUTER JOIN**

In **LEFT OUTER JOIN** of an N:1 relationship, if there are no references to the right table other than the join conditions, eliminating the join with the right table does not affect the query result. In this case, by elimination the join with the right table, the join operation is reduced and the query performance is improved.

To eliminate the **LEFT OUTER JOIN**, the following conditions must be satisfied:

1. Perform the **LEFT OUTER JOIN**.
2. The right table must have the primary key (**PK**) or the **UNIQUE** constraint.
3. All columns of the primary key (**PK**) or the **UNIQUE** constraint must be used in the join conditions.
4. All join conditions must use the equality (=) comparison.
5. There must be no references to the right table ther than the join conditions.
6. The **ALL** keyword must not be used when querying the tables that have an inheritance relationship.

The following is an example of eliminating the **LEFT OUTER JOIN**.

```
call login ('public') on class db_user;

/* current_user: public */
drop table if exists left_tbl, right_tbl;

create table left_tbl (
    id int,
    sub_id int,
    other_id int,
    name varchar (100),
    filter int,
    primary key (id, sub_id, other_id)
);

insert into left_tbl
select
```

```
    ((rownum - 1) / 100) + 1 as id,
    (((rownum - 1) % 10) + 1) * 10 as sub_id,
    (((rownum - 1) % 100) + 1) * 100 as other_id,
    'Left-' || lpad (rownum, 4) as name,
    ((rownum - 1) % 100) + 1 as filter
from db_root
connect by level <= 1000;

create table right_tbl (
    id int,
    sub_id int,
    name varchar (100),
    filter int,
    primary key (id, sub_id)
);

insert into right_tbl
select
    id as id,
    sub_id as sub_id,
    'Right-' || lpad (rownum, 3) as name,
    ((rownum - 1) % 10) + 1 as filter
from (select distinct id, sub_id from left_tbl);

update statistics on left_tbl, right_tbl with fullscan;
```

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 1
order by
    l.name;
```

The join with the *right_tbl* table was eliminated.

```
Query plan:

temp(order by)
    subplan: sscan
                class: l node[0]
                sargs: term[0]
                cost:  6 card 1
```

```
    sort:  4 asc
    cost:  13 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
=============================================================================
=====
            1           10          100  'Left-
1'                     1
            2           10          100  'Left-
101'                    1
            3           10          100  'Left-
201'                    1
            4           10          100  'Left-
301'                    1
            5           10          100  'Left-
401'                    1
            6           10          100  'Left-
501'                    1
            7           10          100  'Left-
601'                    1
            8           10          100  'Left-
701'                    1
            9           10          100  'Left-
801'                    1
           10           10          100  'Left-
901'                    1
```

The following is an example of using the **NO_ELIMINATE_JOIN** hint.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile no_eliminate_join */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 1
order by
    l.name;
```

The join with the *right_tbl* table was not eliminated because the **NO_ELIMINATE_JOIN** hint was used.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                  outer: sscan
                             class: l node[0]
                             sargs: term[2]
                             cost:  6 card 1
                  inner: iscan
                             class: r node[1]
                             index: pk_right_tbl_id_sub_id term[0]
AND term[1] (covers)
                             cost:  1 card 100
                  cost:  9 card 1
    sort:  4 asc
    cost:  15 card 1

Query stmt:

select /*+ NO_ELIMINATE_JOIN */ l.id, l.sub_id, l.other_id, l.
[name], l.filter from left_tbl l left outer join right_tbl r on
l.sub_id=r.sub_id and l.id=r.id where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
=============================================================================
=====
            1           10          100  'Left-
1'                        1
            2           10          100  'Left-
101'                      1
            3           10          100  'Left-
201'                      1
            4           10          100  'Left-
301'                      1
            5           10          100  'Left-
401'                      1
            6           10          100  'Left-
501'                      1
            7           10          100  'Left-
601'                      1
            8           10          100  'Left-
701'                      1
            9           10          100  'Left-
801'                      1
```

```
           10           10          100  'Left-
901'                     1
```

The right table must have the primary key (**PK**) or the **UNIQUE** constraint.

If the right table does not have the primary key (**PK**) or the **UNIQUE** constraint, it is not an N:1 relationship for the **LEFT OUTER JOIN**, and therefore the join with the right table cannot be eliminated.

The following is an example where the primary key (**PK**) of the *right_tbl* table is removed, and a general index is created on the *id* and the *sub_id* columns.

```
/* current_user: public */
set optimization level 513;

alter table right_tbl drop primary key;
alter table right_tbl add index i_right_id_sub_id (id, sub_id);
update statistics on right_tbl with fullscan;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 2
order by
    l.name;

drop index i_right_id_sub_id on right_tbl;
alter table right_tbl add primary key (id, sub_id);
update statistics on right_tbl with fullscan;
```

The join with the *right_tbl* table was not eliminated because the *right_tbl* table does not have the primary key (**PK**) or the **UNIQUE** constraint.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                outer: sscan
                           class: l node[0]
                           sargs: term[2]
                           cost:  6 card 1
                inner: iscan
                           class: r node[1]
```

```
                            index: i_right_id_sub_id term[0] AND
term[1] (covers)
                            cost:  1 card 100
                cost:  9 card 1
    sort:  4 asc
    cost:  15 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l left outer join right_tbl r on l.sub_id=r.sub_id and l.id=r.id
where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
============================================================================
=====
            1           20          200  'Left-
2'                     2
            2           20          200  'Left-
102'                    2
            3           20          200  'Left-
202'                    2
            4           20          200  'Left-
302'                    2
            5           20          200  'Left-
402'                    2
            6           20          200  'Left-
502'                    2
            7           20          200  'Left-
602'                    2
            8           20          200  'Left-
702'                    2
            9           20          200  'Left-
802'                    2
           10           20          200  'Left-
902'                    2
```

The following is an example of removing the primary key (**PK**) of the *right_tbl* table and creating the **UNIQUE** constraint consisting of the *id* column and the *sub_id* column.

```
/* current_user: public */
set optimization level 513;

alter table right_tbl drop primary key;
alter table right_tbl add constraint unique (id, sub_id);
update statistics on right_tbl with fullscan;

select /*+ recompile */
```

```
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 2
order by
    l.name;

alter table right_tbl drop constraint u_right_tbl_id_sub_id;
alter table right_tbl add primary key (id, sub_id);
update statistics on right_tbl with fullscan;
```

The join with the *right_tbl* table was eliminated because the *right_tbl* table does not have the primary key (**PK**) or the **UNIQUE** constraint.

```
Query plan:

temp(order by)
    subplan: sscan
                class: l node[0]
                sargs: term[0]
                cost:  6 card 1
    sort:  4 asc
    cost:  13 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
=============================================================================
=====
            1           20          200  'Left-
2'                     2
            2           20          200  'Left-
102'                    2
            3           20          200  'Left-
202'                    2
            4           20          200  'Left-
302'                    2
            5           20          200  'Left-
402'                    2
            6           20          200  'Left-
502'                    2
            7           20          200  'Left-
```

```
602'                    2
           8           20          200  'Left-
702'                    2
           9           20          200  'Left-
802'                    2
          10           20          200  'Left-
902'                    2
```

All columns of the primary key (**PK**) or the **UNIQUE** constraint must be used in the join conditions.

The following is an example where only the *id* column of the *left_tbl* table and the *id* column of the *right_tbl* table are used as the join conditions.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id
where
    l.filter = 3 and l.id = 3
order by
    l.name;
```

The join with the *right_tbl* table was not eliminated because the *sub_id* column of the *left_tbl* table and the *sub_id* column of the *right_tbl* table were not used as the join conditions.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                outer: iscan
                           class: l node[0]
                           index: pk_left_tbl_id_sub_id_other_id
term[2]
                           sargs: term[1]
                           cost:  3 card 1
                inner: iscan
                           class: r node[1]
                           index: pk_right_tbl_id_sub_id term[0]
(covers)
                           filtr: term[3]
                           cost:  1 card 10
                sort:  2 asc, 3 asc
```

```
                cost:  4 card 1
    sort:  4 asc
    cost:  10 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l left outer join right_tbl r on l.id=r.id and r.id= ?:0  where
l.id= ?:1  and l.filter= ?:2  order by 4
```

```
           id       sub_id     other_id  name
filter
============================================================================
======
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
            3           30          300  'Left-
203'                     3
```

All join conditions must use the equality (=) comparison.

The following is an example where the join conditions do not use the equality (=) comparison.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id <
```

```
r.sub_id
where
    l.filter = 4 and l.id = 4
order by
    l.name;
```

The join with the *right_tbl* table was not eliminated because the comparison between the *sub_id* column of the *left_tbl* table and the *sub_id* column of the *right_tbl* table in the join conditions does not use the equality (=) comparison.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                outer: iscan
                           class: l node[0]
                           index: pk_left_tbl_id_sub_id_other_id
term[3]
                           sargs: term[2]
                           cost:  3 card 1
                inner: iscan
                           class: r node[1]
                           index: pk_right_tbl_id_sub_id term[0]
(covers)
                           filtr: term[1] AND term[4]
                           cost:  1 card 10
                sort:  2 asc, 3 asc
                cost:  4 card 1
    sort:  4 asc
    cost:  10 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l left outer join right_tbl r on l.id=r.id and r.id= ?:0  and
(l.sub_id<r.sub_id) where l.id= ?:1  and l.filter= ?:2  order by 4
```

```
          id       sub_id     other_id  name
filter
=============================================================================
=====
           4           40          400  'Left-
304'                     4
           4           40          400  'Left-
304'                     4
           4           40          400  'Left-
304'                     4
           4           40          400  'Left-
304'                     4
```

```
             4            40           400  'Left-
304'                       4
             4            40           400  'Left-
304'                       4
```

There must be no references to the right table ther than the join conditions.
The following is an example that retrieves the *name* column from the *right_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /** recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter, r.name
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 5
order by
    l.name;
```

The join with the *right_tbl* table was not eliminated because it is retrieving the values from the *name* column of the *right_tbl* table.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                outer: sscan
                           class: l node[0]
                           sargs: term[2]
                           cost:  6 card 1
                inner: iscan
                           class: r node[1]
                           index: pk_right_tbl_id_sub_id term[0]
AND term[1]
                           cost:  1 card 100
                cost:  9 card 1
    sort:  4 asc
    cost:  15 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter, r.[name] from
```

```
left_tbl l left outer join right_tbl r on l.sub_id=r.sub_id and
l.id=r.id where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter  name
=============================================================================
============================
            1           50          500  'Left-
5'                     5  'Right-  5'
            2           50          500  'Left-
105'                    5  'Right- 15'
            3           50          500  'Left-
205'                    5  'Right- 25'
            4           50          500  'Left-
305'                    5  'Right- 35'
            5           50          500  'Left-
405'                    5  'Right- 45'
            6           50          500  'Left-
505'                    5  'Right- 55'
            7           50          500  'Left-
605'                    5  'Right- 65'
            8           50          500  'Left-
705'                    5  'Right- 75'
            9           50          500  'Left-
805'                    5  'Right- 85'
           10           50          500  'Left-
905'                    5  'Right- 95'
```

The following is an example where there are conditions on the *filter* column of the *right_tbl* table before performing the **LEFT OUTER JOIN**.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id and r.filter = 5
where
    l.filter = 5
order by
    l.name;
```

The join with the *right_tbl* table was eliminated because the query is filtering the records to retrieve only those where the *filter* column of the *right_tbl* table has a value of 5 before performing the **LEFT OUTER JOIN**.

```
Query plan:

temp(order by)
    subplan: sscan
                class: l node[0]
                sargs: term[0]
                cost:  6 card 1
    sort:  4 asc
    cost:  13 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
=============================================================================
=====
            1           50          500  'Left-
5'                     5
            2           50          500  'Left-
105'                    5
            3           50          500  'Left-
205'                    5
            4           50          500  'Left-
305'                    5
            5           50          500  'Left-
405'                    5
            6           50          500  'Left-
505'                    5
            7           50          500  'Left-
605'                    5
            8           50          500  'Left-
705'                    5
            9           50          500  'Left-
805'                    5
           10           50          500  'Left-
905'                    5
```

The following is an example that sorts the results based on the *name* column of the *right_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter, r.name
```

```
from
    left_tbl as l
    left outer join right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 5
order by
    r.name desc;
```

The join with the *right_tbl* table was not eliminated because it is sorting the results based on the *name* column of the *right_tbl* table in descending order.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                outer: sscan
                           class: l node[0]
                           sargs: term[2]
                           cost:  6 card 1
                inner: iscan
                           class: r node[1]
                           index: pk_right_tbl_id_sub_id term[0]
AND term[1]
                           cost:  1 card 100
                cost:  9 card 1
    sort:  6 desc
    cost:  15 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter, r.[name] from
left_tbl l left outer join right_tbl r on l.sub_id=r.sub_id and
l.id=r.id where l.filter= ?:0  order by 6 desc
```

```
          id       sub_id     other_id  name
filter  name
=============================================================================
============================
          10           50          500  'Left-
905'                    5  'Right- 95'
           9           50          500  'Left-
805'                    5  'Right- 85'
           8           50          500  'Left-
705'                    5  'Right- 75'
           7           50          500  'Left-
605'                    5  'Right- 65'
           6           50          500  'Left-
505'                    5  'Right- 55'
```

```
            5           50          500  'Left-
405'                     5  'Right- 45'
            4           50          500  'Left-
305'                     5  'Right- 35'
            3           50          500  'Left-
205'                     5  'Right- 25'
            2           50          500  'Left-
105'                     5  'Right- 15'
            1           50          500  'Left-
5'                      5  'Right-  5'
```

The **ALL** keyword must not be used when querying the tables that have an inheritance relationship.

If the **ALL** keyword is used on the right table that has an inheritance relationship, the join with the right table cannot be eliminated.

The following is an example where the **ALL** keyword is used on the *right_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    left_tbl as l
    left outer join all right_tbl as r on l.id = r.id and l.sub_id =
r.sub_id
where
    l.filter = 6
order by
    l.name;
```

The join with the *right_tbl* table was not eliminated.

```
Query plan:

temp(order by)
    subplan: idx-join (left outer join)
                outer: sscan
                           class: l node[0]
                           sargs: term[2]
                           cost:  6 card 1
                inner: iscan
                           class: r node[1]
                           index: pk_right_tbl_id_sub_id term[0]
AND term[1] (covers)
```

```
                            cost:  1 card 100
                cost:  9 card 1
    sort:  4 asc
    cost:  15 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l left outer join  all right_tbl r on l.sub_id=r.sub_id and
l.id=r.id where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
================================================================================
=====
            1           60          600  'Left-
6'                     6
            2           60          600  'Left-
106'                    6
            3           60          600  'Left-
206'                    6
            4           60          600  'Left-
306'                    6
            5           60          600  'Left-
406'                    6
            6           60          600  'Left-
506'                    6
            7           60          600  'Left-
606'                    6
            8           60          600  'Left-
706'                    6
            9           60          600  'Left-
806'                    6
           10           60          600  'Left-
906'                    6
```

The following is an example where the **ONLY** keyword is used on both the *left_tbl* table and the *right_tbl* table.

```
/* current_user: public */
set optimization level 513;

select /*+ recompile */
    l.id, l.sub_id, l.other_id, l.name, l.filter
from
    only left_tbl as l
    left outer join only right_tbl as r on l.id = r.id and l.sub_id
= r.sub_id
where
```

```
    l.filter = 6
order by
    l.name;
```

The join with the *right_tbl* table was eliminated.

```
Query plan:

temp(order by)
    subplan: sscan
                class: l node[0]
                sargs: term[0]
                cost:  6 card 1
    sort:  4 asc
    cost:  13 card 1

Query stmt:

select l.id, l.sub_id, l.other_id, l.[name], l.filter from left_tbl
l where l.filter= ?:0  order by 4
```

```
           id       sub_id     other_id  name
filter
================================================================================
            1           60          600  'Left-
6'                     6
            2           60          600  'Left-
106'                    6
            3           60          600  'Left-
206'                    6
            4           60          600  'Left-
306'                    6
            5           60          600  'Left-
406'                    6
            6           60          600  'Left-
506'                    6
            7           60          600  'Left-
606'                    6
            8           60          600  'Left-
706'                    6
            9           60          600  'Left-
806'                    6
           10           60          600  'Left-
906'                    6
```

View Merging Optimization

View Merging is an optimization for reducing overhead that occur during the processing of view or inline view. When a query includes a view, there is an overhead of creating a temporary table for that view. And it's impossible to perform index scan on temporary tables, leading to performance degradation. But, by using **View Merging** to merge the view with table in main query, performance can be improved because index scan is available.

Like *Query 1* below, which use inline views, make it easier to understand the intent of the query.

```
/* Query 1 */
SELECT *
FROM (SELECT * FROM athlete WHERE nation_code = 'USA') a,
(SELECT * FROM record WHERE medal = 'G') b
WHERE a.code = b.athlete_code;
```

If the query is executed without **View Merging** optimization, the join is performed with the temporary results of executing each of the inline views *a* and *b*. Temporary results cannot use indexes, resulting in performance degradation.

```
/* Query 2 */
SELECT *
FROM athlete a, record b
WHERE a.code = b.athlete_code
AND a.nation_code = 'USA'
AND b.medal = 'G';
```

When **View Merging** optimization is performed, *Query 1* is transformed into *Query 2*.

In the following cases, **View Merging** is not performed:

1. Using **NO_MERGE** hint on a view.

2. The view includes a **CONNECT BY** clause.

3. The view includes a **DISTINCT** clause.

4. The view is **CTE** (Common Table Expressions).

5. Using **OUTER JOIN** with a view.

6. The view using aggregate or analytic functions.

7. Using **ROWNUM, LIMIT**, or **GROUPBY_NUM(), INST_NUM(), ORDERBY_NUM()**.

8. The view using **Correlated Subquery**.

9. The view includes methods.

10. The view includes **RANDOM(), DRANDOM(), SYS_GUID()**.

The following is an example that uses **NO_MERGE** hint on a view.

```
SELECT *
FROM (SELECT /*+ NO_MERGE*/ * FROM athlete WHERE nation_code =
'USA') a,
(SELECT /*+ NO_MERGE*/ * FROM record WHERE medal = 'G') b
WHERE a.code = b.athlete_code;
```

If the **NO_MERGE** hint is used on a view, **View Merging** is not performed.

The following is an example that includes the **CONNECT BY** clause.

```
-- Creating a tree table and then inserting data
CREATE TABLE tree(id INT, mgrid INT, name VARCHAR(32), birthyear
INT);

INSERT INTO tree VALUES (1,NULL,'Kim', 1963);
INSERT INTO tree VALUES (2,NULL,'Moy', 1958);
INSERT INTO tree VALUES (3,1,'Jonas', 1976);
INSERT INTO tree VALUES (4,1,'Smith', 1974);
INSERT INTO tree VALUES (5,2,'Verma', 1973);
INSERT INTO tree VALUES (6,2,'Foster', 1972);
INSERT INTO tree VALUES (7,6,'Brown', 1981);

-- Executing a hierarchical query with CONNECT BY clause
SELECT *
FROM (SELECT * FROM tree WHERE birthyear = 1973) t
CONNECT BY PRIOR t.id=t.mgrid;
```

Due to the use of the **CONNECT BY** in the above query, **View Merging** cannot be performed.

The following is an example where a view includes the **DISTINCT** clause.

```
SELECT * FROM (SELECT DISTINCT host_year FROM record) T;
```

Due to the **DISTINCT** clause used within the view in the above query, **View Merging** cannot be performed.

The following is an example that the view is **CTE** (Common Table Expressions).

```
WITH cte AS (SELECT * FROM athlete WHERE gender = 'M')
SELECT * FROM cte WHERE cte.nation_code = 'USA';
```

When using **CTE** as above, **View Merging** optimization cannot be performed.

The following is an example performing an **OUTER JOIN** with a view.

```
SELECT *
FROM athlete a
LEFT OUTER JOIN (SELECT * FROM record WHERE host_year = 2020) b
ON a.code = b.athlete_code;
```

In cases where an **OUTER JOIN** is performed as above, **View Merging** cannot be performed.

The following is an example using aggregate or analytic functions.

```
SELECT *
FROM (SELECT AVG(host_year) FROM record WHERE medal = 'G') a;
```

When using aggregate or analytic functions in views as above, **View Merging** optimization cannot be performed.

The following is an example using **ROWNUM, LIMIT** or **GROUPBY_NUM(), INST_NUM(), ORDERBY_NUM()**.

```
SELECT *
FROM (SELECT gender, rownum FROM athlete WHERE rownum < 15) a
WHERE gender = 'M';
```

When using **ROWNUM, LIMIT** or **GROUPBY_NUM(), INST_NUM(), ORDERBY_NUM()** in views as above, **View Merging** optimization cannot be performed. However, when **ROWNUM** is used for partial range processing, **View Merging** is performed even if **ROWNUM** exists. This is the case when the following two conditions are satisfied.

1. The **WHERE** clause includes only predicates involving **ROWNUM**
2. There is only one subquery in the **FROM** clause

```
--csql> ;plan detail
SELECT *
FROM (SELECT name, rownum rn
        FROM (SELECT name FROM athlete WHERE nation_code = 'KOR') a
       WHERE rownum < 15) b
WHERE rn > 5;
```

```
Query stmt:

select b.[name], (rownum) from athlete b where ((rownum)> ?:0  and
(rownum)< ?:1 ) and (b.nation_code= ?:2 )
```

The following is an example using **Correlated Subquery**.

```
SELECT COUNT(*)
FROM athlete a,
(SELECT * FROM record r WHERE a.code = r.athlete_code) b;
```

When using **Correlated Subquery** as above, **View Merging** optimization cannot be performed.

The following is an example where a view includes **RANDOM(), DRANDOM(), SYS_GUID()**.

```
SELECT *
FROM    (SELECT RANDOM (), code FROM athlete WHERE nation_code =
'USA') a,
        (SELECT SYS_GUID (), athlete_code FROM record WHERE medal =
'G') b
WHERE a.code = b.athlete_code;
```

When using **RANDOM(), DRANDOM(), SYS_GUID()** in views as above, **View Merging** optimization cannot be performed.

If **View Merging** optimization cannot be performed, **CUBRID** optimizes the **SELECT-LIST** item of the subquery. The following example shows how the **SELECT-LIST** of the subquery is optimized.

```
--csql> ;plan detail
SELECT /*+ recompile */ aa.host_year, aa.rn
FROM (select host_year,
            (select name from event where code = a.event_code)
event_name,
            (select name from athlete where code = a.athlete_code)
athlete_name,
            rownum rn
     from game a
     where medal = 'G') aa
WHERE host_year = '2004';
```

```
Query stmt:

select aa.host_year, aa.rn from (select a.host_year, (rownum) as
[rn] from game a where a.medal= ?:1 ) aa (host_year, rn) where
aa.host_year= ?:0
```

Predicate Push

Predicate Push is an optimization that pushes the predicate of the main query into the view.

By applying **Predicate Push** to the subquery, the amount of data queried in the subquery is reduced.

For instance, when performing a join using the join condition *a.code = r.athlete_code* as in the query below, if the external predicate can be pushed into the query block, the amount of data to be joined can be reduced.

```
SELECT a.name, r.score
FROM (SELECT name, nation_code, code, count(*) cnt FROM athlete
GROUP BY name, nation_code) a, record r
WHERE a.code = r.athlete_code
AND a.nation_code = 'KOR';
```

There are no predicates in the inline-view of the query above. If the query rewrite doesn't work, the *athlete* table would have been fully scanned, creating a temporary result, joined, and filtering with the condition *a.nation_code = 'KOR'*.

However, if the query is rewritten as follows by using **Predicate Push**, it can be optimized to reduce the amount of data being queried.

```
SELECT a.name, r.score
FROM (SELECT name, nation_code, code, count(*) cnt FROM athlete
WHERE nation_code = 'KOR' GROUP BY name, nation_code ) a, record r
WHERE a.code = r.athlete_code;
```

The following example shows how **Predicate Push** works in a situation where there are multiple subqueries.

```
--csql> ;plan detail
SELECT a.host_year
FROM (select distinct host_year, event_code, athlete_code from game
where host_year = 2004) a,
     (select distinct host_year, event_code, athlete_code from game
where event_code = 20021) b
WHERE a.host_year = b.host_year
AND a.event_code = b.event_code
AND a.athlete_code = b.athlete_code;
```

```
Query stmt:

select a.host_year
 from (select distinct game.host_year, game.event_code,
game.athlete_code from game game where game.host_year= ?:0  and
game.event_code=20021) a (host_year, event_code, athlete_code),
      (select distinct game.host_year, game.event_code,
```

```
game.athlete_code from game game where game.event_code= ?:1  and
game.host_year=2004) b (host_year, event_code, athlete_code)
where a.athlete_code=b.athlete_code and a.host_year=b.host_year and
a.event_code=b.event_code
```

The predicates **game.event_code=20021** and **game.host_year=2004** are added to each subquery according to the join condition of the main query.

In the following cases, **Predicate Push** is not performed:

1. Using **NO_PUSH_PRED** hint on main query.
2. Contains **CONNECT BY** on main query.
3. Using aggregate or analytic functions on view.
4. Using **ROWNUM, LIMIT**, or **GROUPBY_NUM(), INST_NUM(), ORDERBY_NUM()** on view.
5. Using **Correlated Subquery**.
6. Using subqueries in predicates.
7. The view includes methods.
8. When the predicate to be pushed or the target for **Predicate Push** within the view uses **RANDOM (), DRANDOM (), SYS_GUID ()**.
9. When performing an **OUTER JOIN** and either the predicate to be pushed or the target for **Predicate Push** within the view uses:
    - Predicates written in the **ON** clause.
    - Using **NULL** transformation functions. This includes **COALESCE (), NVL (), NVL2 (), DECODE (), IF (), IFNULL (), CONCAT_WS ()**.
    - **IS NULL ,CASE** statements are also not targets for **Predicate Push**.

Note

NULL transformation function includes the following functions:

- **COALESCE ()**
- **NVL ()**
- **NVL2 ()**
- **DECODE ()**

- **IF ()**
- **IFNULL ()**
- **CONCAT_WS ()**

Furthermore, **IS NULL ,CASE** statements are also not targets for **Predicate Push**.

The following is an example that uses **NO_PUSH_PRED** hint on main query.

```
SELECT /*+ NO_PUSH_PRED*/ a.name, r.score
FROM (SELECT name, nation_code, code, count(*) cnt FROM athlete
GROUP BY name, nation_code) a, record r
WHERE a.code = r.athlete_code
AND a.nation_code = 'KOR';
```

If the **NO_PUSH_PRED** hint is used on main query, **Predicate Push** is not performed.

The following is an example that performs an **OUTER JOIN** with the predicate to be pushed in the **ON** clause condition.

```
SELECT a.name, r.score
FROM (SELECT * FROM athlete WHERE gender = 'M') a
    LEFT OUTER JOIN record r ON a.code = r.athlete_code AND
a.nation_code = 'KOR';
```

In this case, *a.nation_code = 'KOR'* exists in the **ON** clause during the **LEFT OUTER JOIN**. Such a predicate within the **ON** clause is not a target for **Predicate Push**.

The following is an example that performs an **OUTER JOIN** where either the predicate to be pushed or the target for **Predicate Push** within the view uses the **NULL** transformation function.

```
SELECT a.name, r.score
FROM athlete a
        LEFT OUTER JOIN (SELECT * FROM record WHERE medal = 'G') r
ON a.code = r.athlete_code
WHERE NVL(r.score, '0') = '0';
```

When performing an **OUTER JOIN** and either the predicate to be pushed or the target for **Predicate Push** within the view uses a **NULL** transformation function, it's not a target for **Predicate Push**.

Subquery unnest

Subquery unnest improves performance by converting the filtering method, which executes the subquery in the **WHERE** clause repeatedly, into a join method that guarantees the same result. Subqueries in the **WHERE** clause can have different characteristics depending on the operator. Subqueries with operators like **IN** or **EXISTS**, which can return multiple rows, are the main targets for optimization. **CUBRID** partially supports this optimization only for the **IN** operator.

The following example shows converting an IN operator into a join query.

```
--csql> set optimization level 513;
SELECT game_date
  FROM game
 WHERE (host_year,event_code,athlete_code) IN (select
host_year,event_code,athlete_code
                                                 from game
                                                where nation_code =
'KOR' and medal = 'G');
```

```
Query plan:

idx-join (inner join)
    outer: sscan
                class: av1861 node[1]
                cost:  1 card 25
    inner: iscan
                class: game node[0]
                index: pk_game_host_year_event_code_athlete_code
term[0] AND term[1] AND term[2]
                cost:  3 card 8653
    cost:  17 card 1

Query stmt:

select game.game_date from game game, (select distinct
game.host_year, game.event_code, game.athlete_code from game game
where game.medal= ?:0  and game.nation_code= ?:1 ) av1861 (av_1,
av_2, av_3) where game.host_year=av1861.av_1 and
game.event_code=av1861.av_2 and game.athlete_code=av1861.av_3
```

Note

Starting from CUBRID 11.0, subqueries that retrieve multiple columns are allowed in the arguments of the **IN** operator.

Transitive predicate

Transitive predicate improves query performance by adding conditional clauses that can be logically constructed.

The following example shows that the **b.col1 = 3** predicate is added with **Transitive predicate** optimization.

```
CREATE TABLE t1 (col1 int, col2 int, col3 int);
CREATE TABLE t2 (col1 int, col2 int, col3 int);

--csql> ;plan detail
SELECT /*+ recompile */ *
FROM t1 a, t2 b
WHERE a.col1 = b.col1
AND a.col1 = 3;
```

```
Query stmt:

select a.col1, a.col2, a.col3, b.col1, b.col2, b.col3 from t1 a, t2
b where b.col1= ?:0  and a.col1= ?:1  and a.col1=b.col1
```

QUERY CACHE

The **QUERY_CACHE** hint can be used to enhance the performance for the query which is executed repeatedly. The query is cached in dedicated memory area and its results are also cached at the separated disk space. The hint is applied to SELECT query only; however, for the following cases, the hint is not applicable to the query and the hint is meaningless:

- a system time or date related attribute in the query as below ex) SELECT SYSDATE, ADDDATE(SYSDATE,INTERVAL -24 HOUR), ADDDATE(SYSDATE, -1);
- a SERIAL related attribute is in the query
- a column-path related attribute is in the query
- a method is in the query
- a stored procedure or a stored function is in the query
- a system tables like dual, _db_attribute, and so on, is in the query
- a system function like sys_guid() is in the query

When the hint is set and a new SELECT query is processed, the query cache is looked up if the query appears in the query cache. The queries are considered identical in case they use the same

query text and the same bind values under the same database. If the cached query is not found, the query will be processed and then cached newly with its result. If the query is found from the cache, the results will be fetched from the cached area. AT the CSQL, we can measure the enhancement easily to execute the query repeatedly using the COUNT function as below example. The query and its results will be cached at the first appearance, so the response time is slower than the next same query. The second query's result is fetched from the cached area, so the response time is much faster than the prior same query's one.

```
csql> SELECT /*+ QUERY_CACHE */ count(*) FROM game;

=== <Result of SELECT Command in Line 1> ===

     count(*)
==============
     8653

1 row selected. (0.107082 sec) Committed.

1 command(s) successfully processed.

csql> SELECT /*+ QUERY_CACHE */ count(*) FROM game;

=== <Result of SELECT Command in Line 1> ===

     count(*)
==============
     8653

1 row selected. (0.003932 sec) Committed.

1 command(s) successfully processed.
```

The user can check the query to be cached or not by putting the session command *;info qcache*' in CSQL as follows:

```
csql> ;info qcache

LIST_CACHE {
  n_hts 1010
  n_entries 1  n_pages 1
  lookup_counter 1
  hit_counter 1
  miss_counter 0
  full_counter 0
}

list_hts[0] 0x6a74d10
HTABLE NAME = list file cache (DB_VALUE list), SIZE = 211, REHASH_AT
```

```
= 147,
NENTRIES = 1, NPREALLOC = 0, NCOLLISIONS = 0

HASH AT 0
LIST_CACHE_ENTRY (0x6c46d18) {
  param_values = [ ]
  list_id = { type_list { 1 integer/1 } tuple_cnt 1 page_cnt 1
first_vpid { 65 32766 } last_vpid { 65 32766 } lasttpl_len 24
query_id 2
  temp_vfid { 64 32766 } }
  uncommitted_marker = false
  tran_isolation = 4
  tran_index_array = [ ]
  last_ta_idx = 0
  query_string = select /*+ QUERY_CACHE */ count(*) from [game]
[game]?193="en_US";194="en_US";249="Asia/Seoul";user=0|833|1
  time_created = 11/23/20 16:07:12.779703
  time_last_used = 11/23/20 16:07:22.772330
  ref_count = 1
  deletion_marker = false
}
```

The cached query is shown as **query_string** in the middle of the result screen. **n_entries** and **n_pages** represent the number of cached queries and the number of pages in the cached results, respectively. **n_entries** is limited by the value of the configuration parameter **max_QUERY_CACHE_entries**, and **n_pages** is limited by the value of the configuration parameter **QUERY_CACHE_size_in_pages**. If **n_entries** or **n_pages** overflow, some candidates among the cache entries are selected and uncached. The number of candidates is approximately 20% of the **max_QUERY_CACHE_entries** value and the **QUERY_CACHE_size_in_pages** value.

Since version 11.4, subqueries can also be cached, and the subqueries that can be cached are as follows.

1) CTE query When the QUERY_CACHE hint is specified in a non-recursive query included in the WITH clause

```
WITH
 of_drones AS (SELECT /*+ QUERY_CACHE */ item, 'drones' FROM
products WHERE parent_id = 1),
 of_cars AS (SELECT /*+ QUERY_CACHE */ item, 'cars' FROM products
WHERE parent_id = 5)
SELECT * FROM of_drones UNION ALL SELECT * FROM of_cars ORDER BY 1;
```

When the QUERY_CACHE hint is specified in a recursive query included in the WITH clause and no other CTE is referenced.

```
WITH
 RECURSIVE cars (id, parent_id, item, price) AS (
                    SELECT /*+ QUERY_CACHE */ id, parent_id, item,
price
                        FROM products WHERE item LIKE 'Car%'
                    UNION ALL
                    SELECT /*+ QUERY_CACHE */ p.id, p.parent_id,
p.item, p.price
                        FROM products p
                    INNER JOIN cars rec_cars ON p.parent_id =
rec_cars.id)
SELECT item, price FROM cars ORDER BY 1;
```

In the UNION query above, the first query is cached, but the second query is not cached because it references another table.

1. When the QUERY_CACHE hint is specified in a subquery that does not refer to another table

```
SELECT h.host_year, (SELECT /*+ QUERY_CACHE */ host_nation FROM
olympic o WHERE o.host_year > 1994 limit 1) AS host_nation,
h.event_code, h.score, h.unit
FROM history h;

SELECT name, capital, list(SELECT /*+ QUERY_CACHE */ host_city FROM
olympic WHERE host_nation like 'K%') AS host_cities
FROM nation;
```

However, if another table is referenced within a subquery as shown below, the query is not cached even if the QUERY_CACHE hint is specified.

```
SELECT h.host_year, (SELECT /*+ QUERY_CACHE */ host_nation FROM
olympic o WHERE o.host_year=h.host_year) AS host_nation,
h.event_code, h.score, h.unit
FROM history h;

SELECT name, capital, list(SELECT /*+ QUERY_CACHE */ host_city FROM
olympic WHERE host_nation = name) AS host_cities
FROM nation;
```

The cached query is shown as **query_string** in the middle of the result screen. Each of the **n_entries** and **n_pages** represents the number of cached queries and the number of pages in the cached results. The **n_entries** is limited to the value of configuration parameter **max_query_cache_entries** and the **n_pages** is limited to the value of **query_cache_size_in_pages**. If the **n_entries** is overflown or the **n_pages** is overflown, some victims among the cache entries are selected and they are uncached. The number of victims is about 20% of **max_query_cache_entries** value and of the **query_cache_size_in_pages** value.

When executing a query using query profiling, profiling information for the uncorrelated subquery cache is displayed as sub-information of the subquery to which the uncorrelated subquery cache is applied.

The following is an example of subquery cache profiling information displayed when executing an uncorrelated subquery.

```
SELECT name, capital, list (
            SELECT /*+ QUERY_CACHE */ host_city
            FROM olympic
            WHERE host_nation like 'K%'
) AS host_cities
FROM nation;

Trace Statistics:
  SELECT (time: 13, fetch: 2584, fetch_time: 0, ioread: 0)
    SCAN (table: public.nation), (heap time: 2, fetch: 864, ioread:
0, readrows: 215, rows: 215)
    SUBQUERY (uncorrelated)
      SELECT (time: 0, fetch: 0, fetch_time: 0, ioread: 0)
        RESULT CACHE (reference count: 1)
```

The description of the profiling items for the uncorrelated subquery cache is as follows:

- **reference count**: The number of references to the cache referenced by the subquery.

SUBQUERY CACHE (correlated)

Subquery cache optimization can enhance the performance of queries including correlated subqueries by caching each subquery's results in private memory space per subquery. This optimization is enabled by default. To disable it during query execution, use the NO_SUBQUERY_CACHE hint on the target subquery.

When the correlated subquery is in the SELECT clause, subquery cache is enabled. The processing mechanism of the cache is as follows: If the column values of the search conditions of the re-executed correlated subquery are the same, the cached results are used instead of executing the subquery. If the cached value cannot be found in the subquery cache, after executing the subquery, the retrieved column value and results are stored in the subquery cache.

When executing a query using query profiling, the profile results for the subquery cache are displayed as part of the profile for correlated subqueries.

The following example displays subquery cache profiling information, when performing a correlated subquery.

```
csql> SELECT /*+ RECOMPILE NO_MERGE */
        (SELECT t1_pk FROM t1 b WHERE b.t1_pk = a.c3)
      FROM t1 a
      WHERE a.c2 >= 1;

Trace Statistics:
  SELECT (time: 108, fetch: 103639, fetch_time: 13, ioread: 0)
    SCAN (table: dba.t1), (heap time: 70, fetch: 100384, ioread: 0,
readrows: 100000, rows: 99000)
    SUBQUERY (correlated)
      SELECT (time: 4, fetch: 2970, fetch_time: 0, ioread: 0)
        SCAN (index: dba.t1.pk_t1), (btree time: 2, fetch: 1980,
ioread: 0, readkeys: 990, filteredkeys: 0, rows: 990, covered: true)
        SUBQUERY_CACHE (hit: 98010, miss: 990, size: 269384, status:
enabled)
```

Descriptions for each item are as follows:

- **hit**: The number of retrieves from the cache without executing the query.
- **miss**: The number of times the result of a query execution is stored into the cache (the number of times a query result is not found in the cache).
- **size**: The memory size used by the subquery cache.
- **status**: The activation status of the subquery cache at the end of executing the query.

If the **size** exceeds the set value during query execution, the subquery cache is disabled and the **status** in the SQL trace information is output as disabled. Additionally, if the **hit** ratio is less than 90%, even when the memory size of the subquery cache does not exceed the set value, the cache may be disabled during query execution because the cache miss rate is considered high.

Note

The **hit** ratio is calculated as follows:

$$\text{hit ratio} = \frac{\text{hit}}{\text{hit} + \text{miss}}$$

The following example contains queries that count the results of a correlated subquery that is executed repeatedly to check for performance improvements depending on whether subquery caching is used. The following query creates data to perform the example queries.

```
# Prepare data
csql> DROP TABLE IF EXISTS t1;
```

```
csql> CREATE TABLE t1 AS
        SELECT
                ROWNUM AS t1_pk,
                MOD(ROWNUM, 10) AS c1,
                MOD(ROWNUM, 100) AS c2,
                MOD(ROWNUM, 1000) AS c3
        FROM
                db_class a, db_class b, db_class c, db_class d,
db_class e, db_class f
        LIMIT 100000;

csql> ALTER TABLE t1 ADD CONSTRAINT PRIMARY KEY pk_t1 (t1_pk);

csql> CREATE TABLE t2 AS
        SELECT
                ROWNUM as c1,
                1 as c2,
                TO_CHAR(ROWNUM * 1000, '0999') as code
        FROM
                db_class a, db_class b
        LIMIT 10;

csql> update statistics on t1 with fullscan;

csql> ;trace on
```

Query #1 is executed using subquery cache optimization, while Query #2 is executed without subquery cache optimization by using the NO_SUBQUERY_CACHE hint. A comparison of the results from the two queries reveals that subquery cache optimization improves response performance.

Query #1

```
csql> SELECT COUNT(*)
       FROM (
              SELECT /*+
RECOMPILE NO_MERGE */
                     (SELECT
t1_pk FROM t1 b WHERE b.t1_pk
= a.c3)
                FROM t1 a
               WHERE a.c2 >= 1
            );

=== <Result of SELECT Command
in Line 2> ===

             count(*)
```

Query #2

```
csql> SELECT COUNT(*)
       FROM (
              SELECT /*+
RECOMPILE NO_MERGE */
                     (SELECT /
*+ NO_SUBQUERY_CACHE */ t1_pk
FROM t1 b WHERE b.t1_pk =
a.c3)
                FROM t1 a
               WHERE a.c2 >= 1
            );

=== <Result of SELECT Command
in Line 2> ===
```

Query #1

```
======================
                99000

1 row selected. (0.128251 sec)
Committed. (0.000010 sec)

1 command(s) successfully
processed.
```

Query #2

```
             count(*)
======================
                99000

1 row selected. (0.626199
sec) Committed. (0.000011 sec)

1 command(s) successfully
processed.
```

Subquery cache optimization does not work in the following scenarios:

- When the correlated subquery contains another correlated subquery. (However, subquery cache optimization is performed for the innermost correlated subquery that does not contain other correlated subqueries.)
- When the subquery is not scalar subquery.
- When the subquery includes a CONNECT BY clause.
- When the subquery includes OID-related features.
- When the subquery includes the **NO_SUBQUERY_CACHE** hint.
- When the subquery cache memory size exceeds the configured size (default: 2MB).
- When the subquery contains functions that produce different results with each execution, such as random() or sys_guid().

The following example shows that subquery caching is enabled only for the innermost correlated subquery and disabled for the outer correlated subquery.

```
csql> SELECT /*+ recompile */
        (
            SELECT
                (
                    SELECT c.code
                    FROM t2 c
                    WHERE c.c1 = b.c1
                )
            FROM t1 b
            WHERE b.t1_pk = a.c1
        ) s
        FROM t1 a
        WHERE a.c3 = 1;
```

```
Trace Statistics:
    SELECT (time: 56, fetch: 100785, fetch_time: 10, ioread: 0)
        SCAN (table: dba.t1), (heap time: 55, fetch: 100384, ioread:
0, readrows: 100000, rows: 100)
        SUBQUERY (correlated)
        SELECT (time: 0, fetch: 401, fetch_time: 0, ioread: 0)
            SCAN (index: dba.t1.pk_t1), (btree time: 0, fetch: 300,
ioread: 0, readkeys: 100, filteredkeys: 0, rows: 100) (lookup time:
0, rows: 100)
            SUBQUERY (correlated)
            SELECT (time: 0, fetch: 1, fetch_time: 0, ioread: 0)
                SCAN (table: dba.t2), (heap time: 0, fetch: 1,
ioread: 0, readrows: 10, rows: 1)
                SUBQUERY_CACHE (hit: 99, miss: 1, size: 150704,
status: enabled)
```

The following example shows that subquery cache optimization is disabled when the correlated subquery includes random() functions, which yield different results with each execution.

```
csql> WITH cte_1 AS
        (SELECT
            DISTINCT (SELECT random(1) FROM t2 b WHERE b.c1 = a.c1
AND b.c2 = 1) v
            FROM t1 a
            WHERE a.c2 = 1
        ) SELECT count(*) FROM cte_1;

Trace Statistics:
    SELECT (time: 65, fetch: 101384, fetch_time: 9, ioread: 0)
        SCAN (temp time: 0, fetch: 0, ioread: 0, readrows: 1000,
rows: 1000)
        SUBQUERY (uncorrelated)
        CTE (non_recursive_part)
            SELECT (time: 65, fetch: 101384, fetch_time: 9, ioread:
0)
            SCAN (table: dba.t1), (heap time: 59, fetch: 100384,
ioread: 0, readrows: 100000, rows: 1000)
            ORDERBY (time: 0, sort: true, page: 0, ioread: 0)
            SUBQUERY (correlated)
                SELECT (time: 4, fetch: 1000, fetch_time: 0, ioread:
0)
                SCAN (table: dba.t2), (heap time: 3, fetch: 1000,
ioread: 0, readrows: 10000, rows: 1000)
```

Partitioning

Partitioning is a method by which a table is divided into multiple independent physical units called partitions. In CUBRID, each partition is a table implemented as a subclass of the partitioned table. Each partition holds a subset of the partitioned table data defined by a Partitioning key and a partitioning method. Users can access data stored in partitions by executing statements on the partitioned table. This means that users can partition tables without modifying statements or code that is used to access these tables (benefiting from the advantages of partitioning almost without modifying the user application).

Partitioning can enhance manageability, performance and availability. Some advantages of partitioning a table are:

- Improved management of large capacity tables
- Improved performance by narrowing the range of access when retrieving data
- Improved performance and decreased physical loads by distributing disk I/O
- Decreased possibility of data corruption and improved availability by partitioning a table into multiple chunks
- Optimized storage cost

Partitioned data is auto-managed by CUBRID. INSERT and UPDATE statements executed on partitioned tables perform an additional step during execution to identify the partition in which a tuple must be placed. During UPDATE statement execution, CUBRID identifies situations in which the modified tuple should be moved to another partition and performs this operation keeping the partitioning definition consistent. Inserting tuples for which there is no valid partition will return an error.

When executing SELECT statements, CUBRID applies a procedure called Partition Pruning to narrow the search space to only those partitions for which the search predicates will produce results. Pruning (eliminating) most of the partitions during a SELECT statement greatly improves performance.

Table partitioning is most effective when applied to large tables. Exactly what a “large” table means is dependent on the user application and on the way in which the table is used in queries. Which is the best partitioning method (range, list or hash) for a table, also depends on how the table is used in queries and how data will be distributed between partitions. Even though partitioned tables can be used just like normal tables, there are some Notes on Partitioning which should be taken into consideration.

Partitioning

Partitioning key

The partitioning key is an expression which is used by the partitioning method to distribute data across defined partitions. The following data types are supported for the partitioning key:

- **CHAR**
- **VARCHAR**
- **SMALLINT**
- **INT**
- **BIGINT**
- **DATE**
- **TIME**
- **TIMESTAMP**
- **DATETIME**

The following restrictions apply to the partitioning key:

- The partitioning key must use exactly one column from the partitioned table.
- Aggregate functions, analytic functions, logical operators and comparison operators are not allowed in the partitioning key expression.
- The following functions and expressions are not allowed in the partitioning key expression:
  - CASE
  - `CHARSET()`
  - `CHR()`
  - `COALESCE()`
  - `SERIAL_CURRENT_VALUE()`
  - `SERIAL_NEXT_VALUE()`
  - `DECODE()`
  - `DECR()`
  - `INCR()`
  - `DRAND()`

- `DRANDOM()`
- `GREATEST()`
- `LEAST()`
- `IF()`
- `IFNULL()`
- `INSTR()`
- `NVL()`
- `NVL2()`
- `ROWNUM`
- `INST_NUM()`
- `USER`
- PRIOR
- `WIDTH_BUCKET()`

- The partitioning key needs to be present in the key of each unique index (including primary keys). For more information on this aspect, please see here.
- The partitioning expression's length must not exceed 1024 bytes.

Range Partitioning

Range partitioning is a partitioning method in which a table is partitioned using a user specified range of values of the partitioning key for each partition. Ranges are defined as continuous non-overlapping intervals. This partitioning method is most useful when table data can be divided into range intervals (e.g. order placement date for an orders table or age intervals for a user's table). Range partitioning is the most versatile partitioning method in terms of Partition Pruning because almost all search predicates can be used to identify matching ranges.

Tables can be partitioned by range by using the **PARTITION BY RANGE** clause in **CREATE** or **ALTER** statements.

```
CREATE TABLE [schema_name.]table_name (
    ...
)
PARTITION BY RANGE ( <partitioning_key> ) (
    PARTITION partition_name VALUES LESS THAN ( <range_value> )
```

```
[COMMENT 'comment_string'] ,
    PARTITION partition_name VALUES LESS THAN ( <range_value> )
[COMMENT 'comment_string'] ,
    ...
)

ALTER TABLE [schema_name.]table_name
PARTITION BY RANGE ( <partitioning_key> ) (
    PARTITION partition_name VALUES LESS THAN ( <range_value> )
[COMMENT 'comment_string'] ,
    PARTITION partition_name VALUES LESS THAN ( <range_value> )
[COMMENT 'comment_string'] ,
    ...
)
```

- *partitioning_key* : specifies the Partitioning key.
- *partition_name* : specifies the partition name.
- *range_value* : specifies the upper limit of the partitioning key value. All tuples for which the evaluation of partitioning key is less than (but not equal to) the *range_value* will be stored in this partition.
- *comment_string*: specifies a comment for each partition.

The following example shows how to create the *participant2* table which holds countries participating at the Olympics and partition this table into partitions holding participants before year 2000(*before_2000* partition) and participants before year 2008(*before_2008* partition):

```
CREATE TABLE participant2 (
    host_year INT,
    nation CHAR(3),
    gold INT,
    silver INT,
    bronze INT
)
PARTITION BY RANGE (host_year) (
    PARTITION before_2000 VALUES LESS THAN (2000),
    PARTITION before_2008 VALUES LESS THAN (2008)
);
```

When creating partitions, CUBRID sorts the user supplied range values from smallest to largest and creates the non-overlapping intervals from the sorted list. In the above example, the created range intervals are [-inf, 2000) and [2000, 2008). The identifier **MAXVALUE** can be used to specify an infinite upper limit for a partition.

```
ALTER TABLE participant2 ADD PARTITION (
  PARTITION before_2012 VALUES LESS THAN (2012),
```

```
  PARTITION last_one VALUES LESS THAN MAXVALUE
);
```

When inserting a tuple into a range-partitioned table, CUBRID identifies the range to which the tuple belongs by evaluating the partitioning key. If the partitioning key value is **NULL**, the data is stored in the partition with the smallest specified range value. If there is no range which would accept the partitioning key value, CUBRID returns an error. CUBRID also returns an error when updating a tuple if the new value of the partitioning key does not belong to any of the defined ranges.

The below is an example to add a comment for each partition.

```
CREATE TABLE tbl (a int, b int) PARTITION BY RANGE(a) (
    PARTITION less_1000 VALUES LESS THAN (1000) COMMENT 'less 1000
comment',
    PARTITION less_2000 VALUES LESS THAN (2000) COMMENT 'less 2000
comment'
);

ALTER TABLE tbl PARTITION BY RANGE(a) (
    PARTITION less_1000 VALUES LESS THAN (1000) COMMENT 'new
partition comment');
```

To see a partition comment, refer to COMMENT of Partition.

Hash Partitioning

Hash partitioning is a partitioning method which is used to distribute data across a specified number of partition. This partitioning method is useful when table data contains values for which ranges or lists would be meaningless (for example, a keywords table or an users table for which user_id is the most interesting value). If the values for the partitioning key are evenly distributed across the table data, hash-partitioning technique divides table data evenly between the defined partitions. For hash partitioning, Partition Pruning can only be applied on equality predicates (e.g. predicates using **=** and IN expressions), making hash partitioning useful only if most of the queries specify such a predicate for the partitioning key.

Tables can be partitioned by hash by using the **PARTITION BY HASH** clause in **CREATE** or **ALTER** statements:

```
CREATE TABLE [schema_name.]table_name (
    ...
)
PARTITION BY HASH ( <partitioning_key> )
PARTITIONS ( number_of_partitions )
```

```
ALTER TABLE [schema_name.]table_name
PARTITION BY HASH (<partitioning_key>)
PARTITIONS (number_of_partitions)
```

- *partitioning_key* : Specifies the Partitioning key.
- *number_of_partitions* : Specifies the number of partitions to be created.

The following example shows how to create the *nation2* table with country *code* and country names, and define 4 hash partitions based on code values. Only the number of partitions, not the name, is defined in hash partitioning.

```
CREATE TABLE nation2 (
  code CHAR (3),
  name VARCHAR (50)
)
PARTITION BY HASH (code) PARTITIONS 4;
```

When a value is inserted into a hash-partitioned table, the partition to store the data is determined by the hash value of the partitioning key. If the partitioning key value is **NULL**, the data is stored in the first partition.

List Partitioning

List partitioning is a partitioning method in which a table is divided into partitions according to user specified list of values for the partitioning key. The lists of values for partitions must be disjoint sets. This partitioning method is useful when table data can be divided into lists of possible values which have a certain meaning (e.g. department id for an employees table or country code for a user's table). As for hash partitioning, Partition Pruning for list partitioned tables can only be applied on equality predicates (e.g. predicates using **=** and IN expressions).

Tables can be partitioned by list by using the **PARTITION BY LIST** clause in **CREATE** or **ALTER** statements:

```
CREATE TABLE [schema_name.]table_name (
  ...
)
PARTITION BY LIST ( <partitioning_key> ) (
  PARTITION partition_name VALUES IN ( <values_list> ) [COMMENT
'comment_string'],
  PARTITION partition_name VALUES IN ( <values_list> ) [COMMENT
'comment_string'],
  ...
)
```

```
ALTER TABLE [schema_name.]table_name
PARTITION BY LIST ( <partitioning_key> ) (
  PARTITION partition_name VALUES IN ( <values_list> ) [COMMENT
'comment_string'],
  PARTITION partition_name VALUES IN ( <values_list> ) [COMMENT
'comment_string'],
  ...
)
```

- *partitioning_key*: specifies the Partitioning key.
- *partition_name*: specifies the partition name.
- *value_list*: specifies the list of values for the partitioning key.
- *comment_string*: specifies a comment for each partition.

The following example shows how to create the *athlete2* table with athlete names and sport events, and define list partitions based on event values.

```
CREATE TABLE athlete2 (name VARCHAR (40), event VARCHAR (30))
PARTITION BY LIST (event) (
    PARTITION event1 VALUES IN ('Swimming', 'Athletics'),
    PARTITION event2 VALUES IN ('Judo', 'Taekwondo', 'Boxing'),
    PARTITION event3 VALUES IN ('Football', 'Basketball', 'Baseball')
);
```

When inserting a tuple into a list-partitioned table, the value of the partitioning key must belong to one of the value lists defined for partitions. For this partitioning model, CUBRID does not automatically assign a partition for **NULL** values of the partitioning key. To be able to store **NULL** values into a list-partitioned table, a partition which includes the **NULL** value in the values list must be created:

```
CREATE TABLE athlete2 (name VARCHAR (40), event VARCHAR (30))
PARTITION BY LIST (event) (
    PARTITION event1 VALUES IN ('Swimming', 'Athletics' ),
    PARTITION event2 VALUES IN ('Judo', 'Taekwondo', 'Boxing'),
    PARTITION event3 VALUES IN ('Football', 'Basketball',
'Baseball', NULL)
);
```

The below is examples of adding comments for each partition.

```
CREATE TABLE athlete2 (name VARCHAR (40), event VARCHAR (30))
PARTITION BY LIST (event) (
    PARTITION event1 VALUES IN ('Swimming', 'Athletics') COMMENT
'G1',
```

```
    PARTITION event2 VALUES IN ('Judo', 'Taekwondo', 'Boxing')
COMMENT 'G2',
    PARTITION event3 VALUES IN ('Football', 'Basketball',
'Baseball') COMMENT 'G3');

CREATE TABLE athlete3 (name VARCHAR (40), event VARCHAR (30));
ALTER TABLE athlete3 PARTITION BY LIST (event) (
    PARTITION event1 VALUES IN ('Handball', 'Volleyball', 'Tennis')
COMMENT 'G1');
```

COMMENT of Partition

A partition's comment can be written only for the range partition and the list partition. You cannot write the comment about the hash partition. The partition comment can be shown by running this syntax.

```
SHOW CREATE TABLE [schema_name.]table_name;
SELECT class_name, partition_name, COMMENT FROM db_partition WHERE
class_name ='table_name';
```

Or you can use CSQL interpreter by running ;sc command.

```
$ csql -u dba demodb

csql> ;sc tbl
```

Partition Pruning

Partition pruning is an optimization method, limiting the scope of a search on a partitioned table by eliminating partitions. During partition pruning, CUBRID examines the **WHERE** clause of the query to identify partitions for which this clause is always false, as considering the way partitioning was defined. In the following example, the **SELECT** query will only be applied to partitions *before_2008* and *before_2012*, since CUBRID knows that the rest of partitions hold data for which *YEAR (opening_date)* is less than 2004.

```
CREATE TABLE olympic2 (opening_date DATE, host_nation VARCHAR (40))
PARTITION BY RANGE (YEAR(opening_date)) (
    PARTITION before_1996 VALUES LESS THAN (1996),
    PARTITION before_2000 VALUES LESS THAN (2000),
    PARTITION before_2004 VALUES LESS THAN (2004),
    PARTITION before_2008 VALUES LESS THAN (2008),
    PARTITION before_2012 VALUES LESS THAN (2012)
);
```

```
SELECT opening_date, host_nation
FROM olympic2
WHERE YEAR(opening_date) > 2004;
```

Partition pruning greatly reduces the disk I/O and the amount of data which must be processed during query execution. It is important to understand when pruning is performed in order to fully benefit from it. In order for CUBRID to successfully prune partitions, the following conditions have to be met:

- Partitioning key must be used in the *WHERE* clause directly (without applying other expressions to it)
- For range-partitioning, the partitioning key must be used in range predicates (**<**, **>**, **BETWEEN**, etc) or equality predicates (**=**, **IN**, etc).
- For list and hash partitioning, the partitioning key must be used in equality predicates (**=**, **IN**, etc).

The following queries explain how pruning is performed on the *olympic2* table from the example above:

```
-- prune all partitions except before_2012
SELECT host_nation
FROM olympic2
WHERE YEAR (opening_date) >= 2008;

-- prune all partitions except before_2008
SELECT host_nation
FROM olympic2
WHERE YEAR(opening_date) BETWEEN 2005 and 2007;

-- no partition is pruned because partitioning key is not used
SELECT host_nation
FROM olympic2
WHERE opening_date = '2008-01-02';

-- no partition is pruned because partitioning key is not used
directly
SELECT host_nation
FROM olympic2
WHERE YEAR(opening_date) + 1 = 2008;

-- no partition is pruned because there is no useful predicate in
the WHERE clause
SELECT host_nation
FROM olympic2
WHERE YEAR(opening_date) != 2008;
```

Note

In versions older than CUBRID 9.0, partition pruning was performed during query compilation stage. Starting with CUBRID 9.0, partition pruning is performed during the query execution stage, because executing partition pruning during query execution allows CUBRID to apply this optimization on much more complex queries. However, pruning information is not displayed in query planning stage anymore, since query planning happens before query execution and this information is not available at that time.

Users can also access partitions directly (independent of the partitioned table) either by using the table name assigned by CUBRID to a partition or by using the *table PARTITION (name)* clause:

```
-- to specify a partition with its table name
SELECT * FROM olympic2__p__before_2008;

-- to specify a partition with PARTITION clause
SELECT * FROM olympic2 PARTITION (before_2008);
```

Both of the queries above access partition *before_2008* as if it were a normal table (not a partition). This is a very useful feature because it allows certain query optimizations to be used even though they are disabled on partitioned tables (see Notes on Partitioning for more info). Users should note that, when accessing partitions directly, the scope of the query is limited to that partition. This means that tuples from other partitions are not considered (even though the **WHERE** clause includes them) and, for **INSERT** and **UPDATE** statements, if the tuple inserted/updated does not belong to the specified partition, an error is returned.

By executing queries on a partition rather than the partitioned table, some of the benefits of partitioning are lost. For example, if users only execute queries on the partitioned table, this table can be repartitioned or partitions can be dropped without having to modify the user application. If users access partitions directly, this benefit is lost. Users should also note that, even though using partitions in **INSERT** statements is allowed (for consistency), it is discouraged because there is no performance gain from it.

Partitioning Management

Partitioned tables can be managed using partition specific clauses of the **ALTER** statement. CUBRID allows several actions to be performed on partitions:

1. Modifying a partitioned table into a regular table.

2. Partitions reorganization.

3. Adding partitions to an already partitioned table.

4. Dropping partitions.

5. Promote partitions to regular tables.

Modifying a Partitioned Table into a Regular Table

Changing a partitioned table into a regular table can be done using the **REMOVE PARTITIONING** clause of the **ALTER** statement:

```
ALTER [TABLE | CLASS] [schema_name.]table_name REMOVE PARTITIONING
```

- *schema_name*: Specifies the schema name of the table. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table to be altered.

When removing partitioning, CUBRID moves all data from partitions into the partitioned table. This is a costly operation and should be carefully planned.

Partition Reorganization

Partition reorganization is a process through which a partition can be divided into smaller partitions or a group of partitions can be merged into a single partition. For this purpose, CUBRID implements the **REORGANIZE PARTITION** clause of the **ALTER** statement:

```
ALTER [TABLE | CLASS] [schema_name.]table_name
REORGANIZE PARTITION <alter_partition_name_comma_list>
INTO "(" <partition_definition_comma_list> ")"

partition_definition_comma_list ::=
PARTITION partition_name VALUES LESS THAN "(" <range_value> ")", ...
```

- *schema_name*: Specifies the schema name of the table. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table to be redefined.
- *alter_partition_name_comma_list* : Specifies the partition to be redefined(current partitions). Multiple partitions are separated by commas (,).
- *partition_definition_comma_list* : Specifies the redefined partitions(new partitions). Multiple partitions are separated by commas (,).

This clause applies only to range and list partitioning. Since data distribution in hash-partitioning method is semantically different, hash-partitioned tables only allow adding and dropping partitions. See Hash Partitioning Reorganization for details.

The following example shows how to reorganize the *before_2000* partition of the participant2 table into the *before_1996* and *before_2000* partitions.

```
ALTER TABLE participant2
REORGANIZE PARTITION before_2000 INTO (
  PARTITION before_1996 VALUES LESS THAN (1996),
  PARTITION before_2000 VALUES LESS THAN (2000)
);
```

The following example shows how to merge the two partitions defined in the above example back into a single *before_2000* partition.

```
ALTER TABLE participant2
REORGANIZE PARTITION before_1996, before_2000 INTO (
  PARTITION before_2000 VALUES LESS THAN (2000)
);
```

The following example shows how to reorganize partitions defined on the athlete2, dividing the *event2* partition into *event2_1* (Judo) and *event2_2* (Taekwondo, Boxing).

```
ALTER TABLE athlete2
REORGANIZE PARTITION event2 INTO (
    PARTITION event2_1 VALUES IN ('Judo'),
    PARTITION event2_2 VALUES IN ('Taekwondo', 'Boxing')
);
```

The following example shows how to combine the *event2_1* and *event2_2* partitions back into a single *event2* partition.

```
ALTER TABLE athlete2
REORGANIZE PARTITION event2_1, event2_2 INTO (
    PARTITION event2 VALUES IN ('Judo', 'Taekwondo', 'Boxing')
);
```

Note

- In a range-partitioned table, only adjacent partitions can be reorganized.

- During partition reorganization, CUBRID moves data between partitions in order to reflect the new partitioning schema. Depending on the size of the reorganized partitions, this might be a time consuming operations and should be carefully planned.
- The **REORGANIZE PARTITION** clause cannot be used to change the partitioning method. For example, a range-partitioned table cannot be changed into a hash-partitioned one.
- There must be at least one partition remaining after deleting partitions.

Adding Partitions

Partitions can be added to a partitioned table by using the *ADD PARTITION* clause of the *ALTER* statement.

```
ALTER [TABLE | CLASS] [schema_name.]able_name
ADD PARTITION "(" <partition_definitions_comma_list> ")"
```

- *schema_name*: Specifies the schema name of the table. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table to which partitions are added.
- *partition_definitions_comma_list* : Specifies the partitions to be added. Multiple partitions are separated by commas (,).

The following example shows how to add the *before_2012* and *last_one* partitions to the participant2 table.

```
ALTER TABLE participant2 ADD PARTITION (
  PARTITION before_2012 VALUES LESS THAN (2012),
  PARTITION last_one VALUES LESS THAN MAXVALUE
);
```

Note

- For range-partitioned tables, range values for added partitions must be greater than the largest range value of the existing partitions.
- For range-partitioned tables, if the upper limit of the range of one of the existing partitions is specified by **MAXVALUE**, **ADD PARTITION** clause will always return an error (the REORGANIZE PARTITION clause should be used instead).

- The *ADD PARTITION* clause can only be used on already partitioned tables.
- This clause has different semantics when executed on hash-partitioned tables. See Hash Partitioning Reorganization for details.

Dropping Partitions

Partitions can be dropped from a partitioned table by using the **DROP PARTITION** clause of the **ALTER** statement.

```
ALTER [TABLE | CLASS] [schema_name.]table_name
DROP PARTITION partition_name_list
```

- *schema_name*: Specifies the schema name of the table. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the partitioned table.
- <*partition_name_list*> : Specifies the names of the partitions to be dropped, separated by comma(,).

The following example shows how to drop the *before_2000* partition in the participant2 table.

```
ALTER TABLE participant2 DROP PARTITION before_2000;
```

Note

- When dropping a partition, all stored data in the partition is deleted. If you want to change the partitioning of a table without losing data, use the **ALTER TABLE ... REORGANIZE PARTITION** statement.
- The number of rows deleted is not returned when a partition is dropped. If you want to delete the data, but want to maintain the table and partitions, use the **DELETE** statement.

This statement is not allowed on hash-partitioned tables. To drop partitions of a hash-partitioned table, use the hash partitioning specific alter clauses.

Hash Partitioning Reorganization

Because data distribution among partitions in a hash-partitioned table is controlled internally by CUBRID, hash-partitioning reorganization behaves differently for hash-partitioned tables than for list or range partitioned tables. CUBRID allows the number of partitions defined on a hash-partitioned table to be increased or reduced. When modifying the number of partitions of a hash-partitioned table, no data is lost. However, because the domain of the hashing function is modified, table data has to be redistributed between the new partitions in order to maintain hash-partitioning consistency.

The number of partitions defined on a hash-partitioned table can be reduced using the **COALESCE PARTITION** clause of the **ALTER** statement.

```
ALTER [TABLE | CLASS] [schema_name.]table_name
COALESCE PARTITION number_of_shrinking_partitions
```

- *schema_name*: Specifies the schema name of the table. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table to be redefined.
- *number_of_shrinking_partitions* : Specifies the number of partitions to be deleted.

The following example shows how to decrease the number of partitions in the nation2 table from 4 to 3.

```
ALTER TABLE nation2 COALESCE PARTITION 1;
```

The number of partitions defined on a hash partitioned table can be increased using the **ADD PARTITION** clause of the **ALTER** statement.

```
ALTER [TABLE | CLASS] [schema_name.]table_name
ADD PARTITION PARTITIONS number
```

- *schema_name*: Specifies the schema name of the table. If omitted, the schema name of the current session is used.
- *table_name* : Specifies the name of the table to be redefined.
- *number* : Specifies the number of partitions to be added.

The following example shows how to add 3 partitions to the nation2.

```
ALTER TABLE nation2 ADD PARTITION PARTITIONS 3;
```

Partition Promotion

The **PROMOTE** clause of the **ALTER** statement promotes a partition of a partitioned table to a regular table. This feature is useful when a certain partition contains historic data which is almost never used. By promoting the partition to a regular table, performance on the partitioned table is increased and the data removed from this table (contained in the promoted partition) can still be accessed. Promoting a partition is an irreversible process, promoted partitions cannot be added back to the partitioned table.

The partition **PROMOTE** statement is allowed only on range and list-partitioned tables. Since users do not control how data is distributed among hash partitions, promoting such a partition does not make sense.

When the partition is promoted to a standalone table, this table inherits the data and ordinary indexes only. The following constraints are not available on the promoted partition:

- Primary Key
- Foreign key
- Unique index
- **AUTO_INCREMENT** attribute and serial
- Triggers
- Methods
- Inheritance relationship (super-class and sub-class)

The syntax for promoting partitions is:

```
ALTER TABLE [schema_name.]table_name PROMOTE PARTITION
<partition_name_list>
```

- *partition_name_list*: The user defined names of partitions to promote separated by comma(,)

The following example creates a partitioned table, inserts some tuples into it and then promotes two of its partitions:

```
CREATE TABLE t (i INT) PARTITION BY LIST (i) (
    PARTITION p0 VALUES IN (1, 2),
    PARTITION p1 VALUES IN (3, 4),
    PARTITION p2 VALUES IN (5, 6)
);

INSERT INTO t VALUES(1), (2), (3), (4), (5), (6);
```

Schema and data of table *t* are shown below.

```
csql> ;schema t
=== <Help: Schema of a Class> ===
...
 <Partitions>
     PARTITION BY LIST ([i])
     PARTITION p0 VALUES IN (1, 2)
     PARTITION p1 VALUES IN (3, 4)
     PARTITION p2 VALUES IN (5, 6)

csql> SELECT * FROM t;

=== <Result of SELECT Command in Line 1> ===
            i
=============
            1
            2
            3
            4
            5
            6
```

The following statement promotes partitions *p0* and *p2*:

```
ALTER TABLE t PROMOTE PARTITION p0, p2;
```

After promotion, table *t* has only one partition (*p1*) and contains the following data.

```
csql> ;schema t
=== <Help: Schema of a Class> ===
 <Class Name>
     t
 ...
 <Partitions>
     PARTITION BY LIST ([i])
     PARTITION p1 VALUES IN (3, 4)

csql> SELECT * FROM t;

=== <Result of SELECT Command in Line 1> ===
            i
=============
            3
            4
```

Indexes on Partitioned Tables

All indexes created on a partitioning table are local indexes. With local indexes, data for each partition is stored in a separate(local) index. This increases concurrency on a partitioned table's indexes, since transactions access data from different partitions also do different, local, indexes.

In order to ensure local unique indexes, the following restriction must be satisfied when creating unique indexes on partitions:

- The partitioning key must be part of the primary key's and the all the unique indexes' definition.

If this is not satisfied, CUBRID will return an error:

```
csql> CREATE TABLE t(i INT , j INT) PARTITION BY HASH (i) PARTITIONS
4;
Execute OK. (0.142929 sec) Committed.

1 command(s) successfully processed.
csql> ALTER TABLE t ADD PRIMARY KEY (i);
Execute OK. (0.123776 sec) Committed.

1 command(s) successfully processed.
csql> CREATE UNIQUE INDEX idx2 ON t(j);

In the command from line 1,

ERROR: Partition key attributes must be present in the index key.


0 command(s) successfully processed.
```

It is important to understand the benefits of local indexes. In a global index scan, for each partition that was not pruned a separate index scan would have been performed. This leads to poorer performance than scanning local indexes because data from other partitions is fetched from disk and then discarded (it belongs to another partition than the one being scanned at the moment). **INSERT** statements also show better performance on local indexes since these indexes are smaller.

Notes on Partitioning

Partitioned tables normally behave like regular tables. However there are some notes that should be taken into consideration in order to fully benefit from partitioning a table.

Statistics on Partitioning Tables

Since CUBRID 9.0, the clause **ANALYZE PARTITION** of the **ALTER** statement has been deprecated. Since partition pruning happens during query execution, this statement will not produce any useful results. Since 9.0, CUBRID keeps separated statistics on each partition. The statistics on the partitioned table are computed as a mean value of the statistics of the table partitions. This is done to optimize the usual case in which, for a query, all partitions are pruned except one.

Restrictions on Partitioned Tables

The following restrictions apply to partitioned tables:

- The maximum number of partitions which can be defined on a table is 1,024.
- Partitions cannot be a part of the inheritance chain. Classes cannot inherit a partition and partitions cannot inherit other classes than the partitioned class (which it inherits by default).
- The following query optimizations are not performed on partitioned tables:
  - ORDER BY skip (for details, see Optimizing ORDER BY Clause)
  - GROUP BY skip (for details, see Optimizing GROUP BY Clause)
  - Multi-key range optimization (for details, see Multiple Key Ranges Optimization)
  - INDEX JOIN

Partitioning Key and Charset, Collation

Partitioning keys and partition definition must have the same character set. The following query will return an error:

```
CREATE TABLE t (c CHAR(50) COLLATE utf8_bin)
PARTITION BY LIST (c) (
    PARTITION p0 VALUES IN (_utf8'x'),
    PARTITION p1 VALUES IN (_iso88591'y')
);
```

```
ERROR: Invalid codeset '_iso88591' for partition value. Expecting
'_utf8' codeset.
```

CUBRID uses the collation defined on the table when performing comparisons on the partitioning key. The following example will return an error because, for utf8_en_ci collation 'test' equals 'TEST'.

```
CREATE TABLE tbl (str STRING) COLLATE utf8_en_ci
PARTITION BY LIST (str) (
    PARTITION p0 VALUES IN ('test'),
    PARTITION p1 VALUES IN ('TEST')
);
```

```
ERROR: Partition definition is duplicated. 'p1'
```

Globalization

Globalization includes internationalization and localization. Internationalization can be applied to various languages and regions. Localization fits the language and culture in a specific area as appending the language-specific components. CUBRID supports multilingual collations including Europe and Asia to facilitate the localization.

If you want to know overall information about character data setting, see Configuration Guide for Characters.

If you want to know about charset, collation and locale, see An Overview of Globalization.

For timezone type and realted functions, see Date/Time Types with Timezone. For timezone related system parameters, see Timezone Parameter. If you want to update a timezone information as a new one, timezone library should be recompiled; for details, see Compiling Timezone Library.

If you want to apply the wanted locale to the database, you have to set the locale firstly, then create the database. Regarding this setting, see Locale Setting.

If you want to change the collation or charset specified on the database, specify COLLATE modifier or CHARSET modifier to the column, table, expression, and specify COLLATE modifier or Charset Introducer to the string literal. Regarding this setting, see Collation.

The functions or operators related to strings can work differently by charset and collation. Regarding this, see Operations Requiring Collation and Charset.

An Overview of Globalization

Character data

Character data (strings) may be stored with VARCHAR(STRING), CHAR, ENUM, and they support charset and collation.

Charset(character set, codeset) controls how characters are stored (on any type of storage) as bytes, or how a series of bytes forms a character. CUBRID supports ISO-88591-1, UTF-8 and EUC-KR charsets. For UTF-8, we support only the Unicode characters up to codepoint 10FFFF (encoded on up to four bytes). For instance, the character "Ç" is encoded in codeset ISO-8859-1 using a single byte (C7), in UTF-8 is encoded with 2 bytes (C3 88), while in EUC-KR this character is not available.

Collation decides how strings compare. Most often, users require case insensitive and case sensitive comparisons. For instance, the strings "ABC" and "abc" are equal in a case insensitive collation, while in a case sensitive collation, they are not, and depending on other collation settings, the relationship can be "ABC" < "abc" , or "ABC" > "abc".

Collation means more than comparing character casing. Collation decides the relationship between two strings (greater, lower, equal), is used in string matching (LIKE), or computing boundaries in index scan.

In CUBRID, a collation implies a charset. For instance, collations "utf8_en_ci" and "iso88591_en_ci" perform case insensitive compare, but operate on different charsets. Although for ASCII range, in these particular cases the results are similar, the collation with "utf8_en_ci" is slower, since it needs to work on variable number of bytes (UTF-8 encoding).

- "'a' COLLATE iso88591_en_ci" indicates "_iso88591'a' COLLATE iso88591_en_ci".
- "'a' COLLATE utf8_en_ci" indicates "_utf8'a' COLLATE utf8_en_ci".

All string data types support precision. Special care is required with fixed characters(CHAR). The values of this types are padded to fill up the precision. For instance, inserting "abc" into a CHAR(5) column, will result in storing "abc " (2 padding spaces are added). Space (ASCII 32 decimal, Unicode 0020) is the padding character for most charsets. But, for EUC-KR charset, the padding consists of one character which is stored with two bytes (A1 A1).

Related Terms

- **Character set**: A group of encoded symbols (giving a specific number to a certain symbol)
- **Collation**: A set of rules for comparison of characters in the character set and for sorting data
- **Locale**: A set of parameters that defines any special variant preferences such as number format, calendar format (month and day in characters), date/time format, and collation

depending on the operator's language and country. Locale defines the linguistic localization. Character set of locale defines how the month in characters and other data are encoded. A locale identifier consists of at least a language identifier and a region identifier, and it is expressed as language[_territory][.codeset] (For example, Australian English using UTF-8 encoding is written as en_AU.UTF-8).

- **Unicode normalization**: The specification by the Unicode character encoding standard where some sequences of code points represent essentially the same character. CUBRID uses Normalization Form C (NFC: codepoint is decomposed and then composed) for input and Normalization Form D (NFD: codepoint is composed and then decomposed) for output. However, CUBRID does not apply the canonical equivalence rule as an exception.

  For example, canonical equivalence is applied in general NFC rule so codepoint 212A (Kelvin K) is converted to codepoint 4B (ASCII code uppercase K). Since CUBRID does not perform the conversion by using the canonical equivalence rule to make normalization algorithm quicker and easier, it does not perform reverse-conversion, too.

- **Canonical equivalence**: A basic equivalence between characters or sequences of characters, which cannot be visually distinguished when they are correctly rendered. For example, let's see 'Å' ('A' with an angstrom). 'Å' (Unicode U + 212B) and Latin 'A' (Unicode U + 00C5) have same A and different codepoints, however, the decomposed result is 'A' and U+030A, so it is canonical equivalence.

- **Compatibility equivalence**: A weaker equivalence between characters or sequences of characters that represent the same abstract character. For example, let's see number '2' (Unicode U + 0032) and superscript '²'(Unicode U + 00B2). '²' is a different format of number '2', however, it is visually distinguished and has a different meaning, so it is not canonical equivalence. When normalizing '2²' with NFC, '2²' is maintained since it uses canonical equivalence. However, with NFKC, '²' is decomposed to '2' which is compatibility equivalence and then it can be recomposed to '22'. Unicode normalization of CUBRID does not apply the compatibility equivalence rule.

For explanation on Unicode normalization, see Unicode Normalization. For more details, see http://unicode.org/reports/tr15/.

The default value of the system parameter related to Unicode normalization is unicode_input_normalization=no and unicode_output_normalization=no. For a more detailed description on parameters, see Statement/Type-Related Parameters.

Locale Attributes

Locale is defined by following attributes.

- **Charset (codeset)**: How bytes are interpreted into single characters (Unicode codepoints)

- **Collations**: Among all collations defined in locale of LDML(UNICODE Locale Data Markup Language) file, the last one is the default collation. Locale data may contain several collations.
- **Alphabet (casing rules)**: One locale data may have up 2 alphabets, one for identifier and one for user data. One locale data can have two types of alphabets.
- **Calendar**: Names of weekdays, months, day periods (AM/PM)
- **Numbering settings**: Symbols for digit grouping
- **Text conversion data**: For CSQL conversion. Option.
- **Unicode normalization data**: Data converted by normalizing several characters with the same shape into one based on a specified rule. After normalization, characters with the same shape will have the same code value even though the locale is different. Each locale can activate/deactivate the normalization functionality.

Note

Generally, locale supports a variety of character sets. However, CUBRID locale supports both ISO and UTF-8 character sets for English and Korean. The other operator-defined locales using the LDML file support the UTF-8 character set only.

Collation Properties

A collation is an assembly of information which defines an order for characters and strings. In CUBRID, collation has the following properties.

- **Strength**: This is a measure of how "different" basic comparable items (characters) are. This affects selectivity. In LDML files, collation strength is configurable and has four levels. For example a Case insensitive collation should be set with level = "secondary" (2) or "primary" (1).
- Whether it supports or not **expansions** and **contractions**

Each column has a collation, so when applying `LOWER()`, `UPPER()` functions the casing rules of locale which defines the collation's default language is used.

Depending on collation properties some CUBRID optimizations may be disabled for some collations:

- **LIKE** rewrite: is disabled for collations which maps several different character to the same weight (case insensitive collations for example) and for collations with expansions.

- Covering index scan: disabled for collations which maps several different character to the same weight (see Covering Index).

For more information, see Collation settings impacting CUBRID features .

Collation Naming Rules

The collation name in CUBRID follows the conversion:

```
<charset>_<lang specific>_<desc1>_<desc2>_...
```

- <charset>: The full charset name as used by CUBRID. iso88591, utf8, euckr.
- <lang specific>: a region/language specific. The language code is expected as two characters; en, de, es, fr, it, ja, km, ko, tr, vi, zh, ro. “gen” if it does not address a specific language, but a more general sorting rule.
- <desc1>_<desc2>_...: They have the following meaning. Most of them apply only to LDML collations.
  - ci: case insensitive In LDML, can be obtained using the settings: strength=”secondary” caseLevel=”off” caseFirst=”off”.
  - cs: case sensitive; By default all collations are case sensitive. In LDML, can be obtained using at least: strength=”tertiary”.
  - bin: it means that the sorting order under such collation is almost the same with the order of codepoints; If memory (byte) comparison is used, then almost the same result is obtained. Space character and EUC double-byte padding character are always sorted as zero in “bin” collation. No collations with such setting are currently configured in LDML (they are already available as built-in), but a similar one can be obtained using the maximum setting strength=”quaternary” or strength=”identical”.
  - ai: accent insensitive; this means that ‘Á’ is sorted the same as ‘A’. Due to particularities of the UCA based algorithms, an accent insensitive collation is also a case insensitive collation. In LDML, can be obtained using: strength=”primary”.
  - uca: this signals a UCA based collation; this is used only to differentiate such collations from similar built-in variants. All LDML collations are based on UCA, but in order to keep shorter names only two collations (‘utf8_ko_cs_uca’ , ‘utf8_tr_cs_uca’) have this description in their names, in order to differentiate them from ‘utf8_ko_cs’ and ‘utf8_tr_cs’ collations.
  - exp: this collations use a full-word matching/compare algorithm, contrary to the rest of collations which use character-by-character compare. This collation uses a more complex

algorithm, with multiple passes which is much slower, but may prove useful for alphabetical sorts. In LDML, the Expansion needs to be explicit by adding CUBRIDExpansions="use".

- ab: accent backwards; it is particularity of French-Canadian sorting, where level 2 of UCA (used to store accents weights) is compared from end of string towards the beginning. This collation setting can be used only when :ref`expansion` setting is also activated. The "backwards" setting allows for the following sorting:
  - Normal Accent Ordering: cote < coté < côte < côté
  - Backward Accent Ordering: cote < côte < coté < côté
- cbm: contraction boundary match; it is a particularity of collations with Expansion and Contraction and refers to how it behaves at string matching when a Contraction is found. Suppose the collation has defined the Contraction "ch"; then normally, the pattern "bac" will not match the string"bachxxx" But when the collation is configured to allow "matching the characters starting a contraction", the above matching will return a positive. Only one collation is configured in this manner - 'utf8_ja_exp_cbm' - Japanese sorting requires a lot of contractions.

The collation names are not dynamically generated. They are user defined (configured in LDML), and should reflect the settings of the collation.

The name of collation influences the internal numeric id of the collation. For instance, in CUBRID only 256 collations are allowed, and the numeric IDs are assigned as:

- 0 -31: built-in collations (for these collations the name and id are hard-coded)
- 32 - 46: LDML collations having "gen" as "language" part
- 47 - 255: the rest of LDML collations

If you want to include all locales into the database which CUBRID provide, first, copy cubrid_locales.all.txt of $CUBRID/conf directory into cubrid_locales.txt and next, run make_locale script(in extension, Linux is .sh, Windows is .bat). For more details on make_locale script, see Step 2: Compiling Locale.

If you want to include the newly added locale information into the existing database, run "cubrid synccolldb <dbname>". For more information, see Synchronization of Database Collations with System Collations.

If you include all locales defined in LDML files, CUBRID has the following collations.

CUBRID Collation

Collation	Locale for casing	Character range
iso88591_bin	en_US - English	ASCII + ISO88591 (C0-FE, except D7, F7)
iso88591_en_cs	en_US - English	ASCII + ISO88591 (C0-FE, except D7, F7)
iso88591_en_ci	en_US - English	ASCII + ISO88591 (C0-FE, except D7, F7)
utf8_bin	en_US - English	ASCII
euckr_bin	ko_KR - Korean, same as en_US - English	ASCII
utf8_en_cs	en_US - English	ASCII
utf8_en_ci	en_US - English	ASCII
utf8_tr_cs	tr_TR - Turkish	Turkish alphabet
utf8_ko_cs	ko_KR - Korean, same as en_US - English	ASCII
utf8_gen	de_DE - German, generic Unicode casing customized with German rules	All Unicode codepoints in range 0000-FFFF
utf8_gen_ai_ci	de_DE - German, generic Unicode casing customized with German rules	All Unicode codepoints in range 0000-FFFF

Collation	Locale for casing	Character range
utf8_gen_ci	de_DE - German, generic Unicode casing customized with German rules	All Unicode codepoints in range 0000-FFFF
utf8_de_exp_ai_ci	de_DE - German, generic Unicode casing customized with German rules	All Unicode codepoints in range 0000-FFFF
utf8_de_exp	de_DE - German, generic Unicode casing customized with German rules	All Unicode codepoints in range 0000-FFFF
utf8_ro_cs	ro_RO - Romanian, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
utf8_es_cs	es_ES - Spanish, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
utf8_fr_exp_ab	fr_FR - French, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
utf8_ja_exp	ja_JP - Japanese, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
utf8_ja_exp_cbm	ja_JP - Japanese, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
utf8_km_exp	km_KH - Cambodian, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
utf8_ko_cs_uca	ko_KR - Korean, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF

Collation	Locale for casing	Character range
utf8_tr_cs_uca	tr_TR - Turkish, generic Unicode casing customized with Turkish rules	All Unicode codepoints in range 0000-FFFF
utf8_vi_cs	vi_VN - Vietnamese, same as generic Unicode casing	All Unicode codepoints in range 0000-FFFF
binary	none (invariant to casing operations)	any byte value (zero is nul-terminator)

The Turkish casing rules changes the casing for character i,I,ı,İ. The German casing rules changes the casing for ß.

On the above collations, 9 collations like iso88591_bin, iso88591_en_cs, iso88591_en_ci, utf8_bin, euckr_bin, utf8_en_cs, utf8_en_ci, utf8_tr_cs and utf8_ko_cs, are built in the CUBRID before running make_locale script.

If a collation is included in more than one locale (.ldml) file, the locale for casing (default locale of collation) is the locale in which it is first included. The order of loading is the locales order from $CUBRID/conf/cubrid_locales.txt. The above locale casing for collations utf8_gen, utf8_gen_ci, utf8_gen_ai_ci, assumes the default order (alphabetical) in cubrid_locales.txt, so the default locale for all generic LDML collations is de_DE (German).

Files For Locale Setting

CUBRID uses following directories and files to set the locales.

- **$CUBRID/conf/cubrid_locales.txt** file: A configuration file containing the list of locales to be supported
- **$CUBRID/conf/cubrid_locales.all.txt** file: A configuration file template with the same structure as **cubrid_locales.txt**. Contains the entire list of all the locales that the current version of CUBRID is capable of supporting without any efforts from the end user's side.
- **$CUBRID/locales/data** directory: This contains files required to generate locale data.
- **$CUBRID/locales/loclib** directory: contains a C header file, **locale_lib_common.h** and OS dependent makefile which are used in the process of creating / generating locales shared libraries.

- **$CUBRID/locales/data/ducet.txt** file: Text file containing default universal collation information (codepoints, contractions and expansions, to be more specific) and their weights, as standardized by The Unicode Consortium, which is the starting point for the creation of collations. For more information, see http://unicode.org/reports/tr10/#Default_Unicode_Collation_Element_Table .
- **$CUBRID/locales/data/unicodedata.txt** file: Text file containing information about each Unicode codepoint regarding casing, decomposition, normalization etc. CUBRID uses this to determine casing. For more information, see https://docs.python.org/3/library/unicodedata.html .
- **$CUBRID/locales/data/ldml** directory: common_collations.xml and XML files, name with the convention cubrid_<*locale_name*>.xml. common_collations.xml file contains shared collation information in all locale files, and each cubrid_<*locale_name*>.xml file contains a locale information for the supported language.
- **$CUBRID/locales/data/codepages** directory: contains codepage console conversion for single byte codepages(8859-1.txt , 8859-15.txt, 8859-9.txt) and codepage console conversion for double byte codepages(CP1258.txt , CP923.txt, CP936.txt , CP949.txt).
- **$CUBRID/bin/make_locale.sh** file or **%CUBRID%\bin\make_locale.bat** file: A script file used to generate shared libraries for locale data
- **$CUBRID/lib** directory: Shared libraries for generated locales will be stored here.

Locale Setting

When you want to use a charset and collation of a specific language, the charset should be identical with a database which will be created newly. Supported CUBRID charsets are ISO-8859-1, EUC-KR and UTF-8 and the charset to be used is specified when creating a database.

For example, when you created a database with a locale ko_KR.utf8, you can use collations starting with “utf8_” like utf8_ja_exp. However, if you set the locale as ko_KR.euckr, you cannot use all collations which are related with other charset(see CUBRID Collation).

The following is an example which used utf8_ja_exp after creating a database with en_US.utf8.

1. cd $CUBRID/conf
2. cp cubrid_locales.all.txt cubrid_locales.txt
3. make_locale.sh -t64 # 64 bit locale library creation
4. cubrid createdb testdb en_US.utf8
5. cubrid server start testdb

6. csql -u dba testdb

7. run below query on csql

```
SET NAMES utf8;
CREATE TABLE t1 (i1 INT , s1 VARCHAR(20) COLLATE utf8_ja_exp, a
INT, b VARCHAR(20) COLLATE utf8_ja_exp);
INSERT INTO t1 VALUES (1, 'いイ基盤',1,'いイ 繭');
```

For more details, see the following.

Step 1: Selecting a Locale

Configure locales to use on **$CUBRID/conf/cubrid_locales.txt**. You can select all or some of locales which are supported.

CUBRID supports locales as follows: en_US, de_DE, es_ES, fr_FR, it_IT, ja_JP, km_KH, ko_KR, tr_TR, vi_VN, zh_CN, ro_RO.

The language and country for each locale are shown in the following table.

Locale Name	Language - Country
en_US	English - U.S.A.
de_DE	German - Germany
es_ES	Spanish - Spain
fr_FR	French - France
it_IT	Italian - Italy
ja_JP	Japanese - Japan
km_KH	Khmer - Cambodia

ko_KR	Korean - Korea
tr_TR	Turkish - Turkey
vi_VN	Vietnamese - Vietnam
zh_CN	Chinese - China
ro_RO	Romanian - Romania

Note

The LDML files for the supported locales are named cubrid_<*locale_name*>.xml and they can be found in the **$CUBRID/locales/data/ldml** directory. If only a subset of these locales are to be supported by CUBRID, one must make sure their corresponding LDML files are present in the **$CUBRID/locales/data/ldml** folder. A locale cannot be used by CUBRID, unless it has an entry in **cubrid_locales.txt file** and it has a corresponding cubrid_<*locale_name*>.xml.

Locale libraries are generated according to the contents of **$CUBRID/conf/cubrid_locales.txt** configuration file. This file contains the language codes of the wanted locales (all user defined locales are generated with UTF-8 charset). Also, in this file can be configured the file paths for each locale LDML file and libraries can be optionally configured.

```
<lang_name>  <LDML file>
<lib file>
ko_KR        /home/CUBRID/locales/data/ldml/cubrid_ko_KR.xml    /
home/CUBRID/lib/libcubrid_ko_KR.so
```

By default, the LDML files are found in **$CUBRID/locales/data/ldml** and the locale libraries in **$CUBRID/lib**; the filenames for LDML are formatted like: cubrid_<*lang_name*>.ldml.

The filenames for libraries: libcubrid_<*lang_name*>.dll (.so for Linux).

Step 2: Compiling Locale

Once the requirements described above are met, the locales can be compiled.

Regarding the embedded locales in CUBRID, they can be used without compiling user locale library, so they can be used by skipping the step 2. But there are differences between the embedded locale and the library locale. Regarding this, see Built-in Locale and Library Locale.

To compile the locale libraries, one must use the **make_locale** (**.bat** for Windows and **.sh** for Linux) utility script from command console. The file is delivered in **$CUBRID/bin** folder so it should be resolved by **$PATH** environment variable. Here **$CUBRID, $PATH** are the environment variables of Linux, **%CUBRID%**, **%PATH%** are the environment variables of Windows.

Note

To run a **make_locale** script in Windows, it requires Visual C++ 2005, 2008 or 2010.

Usage can be displayed by running **make_locale.sh -h**. (**make_locale /h** in Windows.)

```
make_locale.sh [options] [locale]

options ::= [-t 32|64 ] [-m debug|release]
locale ::= [de_DE|es_ES|fr_FR|it_IT|ja_JP|km_KH|ko_KR|tr_TR|vi_VN|
zh_CN|ro_RO]
```

- *options*
  - **-t**: Selects 32bit or 64bit (default value: **64**).
  - **-m**: Selects release or debug. In general, release is selected (default value: release). The debug mode is provided for developers who would like to write the locale library themselves. Selects release or debug. In general, release is selected (default value: release). The debug mode is provided for developers who would like to write the locale library themselves.
- *locale*: The locale name of the library to build. If *locale* is not specified, the build includes data from all configured locales. In this case, library file is stored in **$CUBRID/lib** directory with the name of **libcubrid_all_locales.so** (**.dll** for Windows).

To create user defined locale shared libraries, two choices are available:

- Creating a single lib with all locales to be supported.

```
make_locale.sh -t64                          # Build and pack all
locales (64/release)
```

- Creating one lib for each locale to be supported.

```
make_locale.sh -t 64 -m release ko_KR
```

The first choice is recommended. In this scenario, some data may be shared among locales. If you choose the first one, a lib supporting all locales has less than 15 MB; in the second one, consider for each locale library from 1 MB to more than 5 MB. Also the first one is recommended because it has no runtime overhead during restarting the servers when you choose the second one.

Warning

Limitations and Rules

- Do not change the contents of **$CUBRID/conf/cubrid_locales.txt** after locales generation; once the locales libraries are generated, the contents of **$CUBRID/conf/cubrid_locales.txt** should not be changed (order of languages within the file must also be preserved). During locale compiling, the generic collation uses the first one as default locale; changing the order may cause different results with casing for such collation (utf8_gen_*).
- Do not change the contents for **$CUBRID/locales/data/*.txt** files.

Note

Procedure of Executing make_locale.sh(.bat) Script

The processing in **make_locale.sh(.bat)** script

1. Reads the **.ldml** file corresponding to a language, along with some other installed common data files like **$CUBRID/locales/data/ducet.txt**, **$CUBRID/locales/data/unicodedata.txt**, and **$CUBRID/locales/data/codepages/*.txt**
2. After processing of raw data, it writes in a temporary **$CUBRID/locales/loclib/locale.c** file C constants values and arrays consisting of locales data.
3. The temporary file **locale.c** is passed to the platform compiler to build a **.dll/.so** file. This step assumes that the machines has an installed C/C++ compiler and linker. Currently, only the MS Visual Studio for Windows and gcc for Linux compilers are supported.
4. Temporary files are removed.

Step 3: Setting CUBRID to Use a Specific Locale

Only one locale can be selected as the default locale when you create DB.

In addition to the possibility of specifying a default locale, one can override the default calendar settings with the calendar settings from another locale, using the **intl_date_lang** system parameter.

- The locale will be in the format: <*locale_name*>.[**utf8** | **iso**] (e.g. tr_TR.utf8, en_EN.ISO, ko_KR.utf8)
- **intl_date_lang**: <*locale_name*>. The possible values for <*locale_name*> are listed on Step 1: Selecting a Locale.

Note

Setting the Month/Day in Characters, AM/PM, and Number Format

For the function that inputs and outputs the day/time, you can set the month/day in characters, AM/PM, and number format by the locale in the **intl_date_lang** system parameter.

Also for the function that converts a string to numbers or the numbers to a string, you can set the string format by the locale in **intl_number_lang** system parameter.

Built-in Locale and Library Locale

Regarding the embedded locales in CUBRID, they can be used without compiling user locale library, so they can be used by skipping the step 2. But there are two differences between the embedded locale and the library locale.

- Embedded(built-in) locale(and collation) are not aware of Unicode data For instance, casing (lower, upper) of (Á, á) is not available in embedded locales. The LDML locales provide data for Unicode codepoints up to 65535.
- Also, the embedded collations deals only with ASCII range, or in case of 'utf8_tr_cs' - only ASCII and letters from Turkish alphabet. Embedded UTF-8 locales are not Unicode compatible, while compiled (LDML) locales are.

Currently, the built-in locales which can be set during creating DB are as follows:

- en_US.iso88591
- en_US.utf8
- ko_KR.utf8

- ko_KR.euckr
- ko_KR.iso88591: Will have Romanized Korean names for month, day names.
- tr_TR.utf8
- tr_TR.iso88591: Will have Romanized Turkish names for month, day names.

The order stated above is important; if no charset is defined while creating DB, the charset is the charset of the locale shown first. For example, if the locale is set as ko_KR(e.g. cubrid createdb testdb ko_KR), the charset is specified as ko_KR.utf8, the first locale among the ko_KR in the above list. Locales of the other languages except the built-in locales should end with **.utf8**. For example, specify the locale as de_DE.utf8 for German.

The names of month and day for ko_KR.iso88591 and tr_TR.iso88591 should be Romanized. For example, "일요일" for Korean (Sunday in English) is Romanized to "Iryoil". Providing ISO-8859-1 characters only is required. For more information, see The Month/Day in Korean and Turkish Characters for ISO-8859-1 Charset.

The Month/Day in Korean and Turkish Characters for ISO-8859-1 Charset

In Korean or Turkish which have charset UTF-8 or in Korean which have charset EUC-KR, the month/day in characters and AM/PM are encoded according to the country. However, for ISO-8859-1 charset, if the month/day in characters and AM/PM in Korean or Turkish is used as its original encoding, an unexpected behavior may occur in the server process because of its complex expression. Therefore, the name should be Romanized. The default charset of CUBRID is ISO-8859-1 and the charset can be used for Korean and Turkish. The Romanized output format is as follows:

Day in Characters

Day in Characters Long/Short Format	Long/Short Romanized Korean	Long/Short Romanized Turkish
Sunday / Sun	Iryoil / Il	Pazar / Pz
Monday / Mon	Woryoil / Wol	Pazartesi / Pt
Tuesday / Tue	Hwayoil / Hwa	Sali / Sa

Day in Characters Long/ Short Format	Long/Short Romanized Korean	Long/Short Romanized Turkish
Wednesday / Wed	Suyoil / Su	Carsamba / Ca
Thursday / Thu	Mogyoil / Mok	Persembe / Pe
Friday / Fri	Geumyoil / Geum	Cuma / Cu
Saturday / Sat	Toyoil / To	Cumartesi / Ct

Month in Characters

Month in Characters Long/ Short Format	Long/Short Romanized Korean (Not Classified)	Long/Short Romanized Turkish
January / Jan	1wol	Ocak / Ock
February / Feb	2wol	Subat / Sbt
March / Mar	3wol	Mart / Mrt
April / Apr	4wol	Nisan / Nsn
May / May	5wol	Mayis / Mys
June / Jun	6wol	Haziran / Hzr
July / Jul	7wol	Temmuz / Tmz
August / Aug	8wol	Agustos / Ags

Month in Characters Long/ Short Format	Long/Short Romanized Korean (Not Classified)	Long/Short Romanized Turkish
September / Sep	9wol	Eylul / Eyl
October / Oct	10wol	Ekim / Ekm
November / Nov	11wol	Kasim / Ksm
December / Dec	12wol	Aralik / Arl

AM/PM in Characters

AM/PM in Characters Long/ Short Format	Romanized in Korean	Romanized in Turkish
AM	ojeon	AM
PM	ohu	PM

Step 4: Creating a Database with the Selected Locale Setting

When issuing the command "**cubrid createdb** *<db_name> <locale_name.charset>*", a database will be created using the settings in the variables described above.

Once the database is created a locale setting which was given to the database cannot be changed. The charset and locale name are stored in "**db_root**" system catalog table.

Step 5 (optional): Manually Verifying the Locale File

The contents of locales libraries may be displayed in human readable form using the **dumplocale** CUBRID utility. Execute **cubrid dumplocale -h** to output the usage. The used syntax is as follows.

```
cubrid dumplocale [options] [language-string]

options ::= -i|--input-file <shared_lib>
```

```
            -d|--calendar
            -n|--numeric
            {-a |--alphabet=}{l|lower|u|upper|both}
            -c|--codepoint-order
            -w|--weight-order
            {-s|--start-value} <starting_codepoint>
            {-e|--end-value} <ending_codepoint>
            -k
            -z

language-string ::= de_DE|es_ES|fr_FR|it_IT|ja_JP|km_KH|ko_KR|tr_TR|
vi_VN|zh_CN|ro_RO
```

- **dumplocale**: A command which dumps the contents of locale shared library previously generated using LDML input file.

- *language-string*: One of de_DE, es_ES, fr_FR, it_IT, ja_JP, km_KH, ko_KR, tr_TR, vi_VN, zh_CN and ro_RO. Configures the locale language to dump the locale shared library. If it's not set, all languages which are configured on **cubrid_locales.txt** are given.

The following are [options] for **cubrid dumplocale**.

-i, --input-file=FILE
: The name of the locale shared library file (< *shared_lib*>) created previously. It includes the directory path.

-d, --calendar
: Dumps the calendar and date/time data. Default value: No

-n, --numeric
: Dumps the number data. Default value: No

-a, --alphabet=l|lower|u|upper|both
: Dumps the alphabet and case data. Default value: No

--identifier-alphabet=l|lower|u|upper
: Dumps the alphabet and case data for the identifier. Default value: No

-c, --codepoint-order
: Dumps the collation data sorted by the codepoint value. Default value: No (displayed data: cp, char, weight, next-cp, char and weight)

-w, --weight-order
: Dumps the collation data sorted by the weight value. Default value: No (displayed data: weight, cp, char)

-s, --start-value=CODEPOINT

Specifies the dump scope. Starting codepoint for **-a, --identifier-alphabet, -c, -w** options. Default value: 0

-e, --end-value=CODEPOINT
Specifies the dump scope. Ending codepoint for **-a, --identifier-alphabet, -c, -w** options. Default value: Max value read from the locale shared library.

-k, --console-conversion
Dumps the data of console conversion. Default value: No

-z, --normalization
Dumps the normalization data. Default value: No

The following example shows how to dump the calendar, number formatting, alphabet and case data, alphabet and case data for the identifier, collation sorting based on the codepoint order, collation sorting based on the weight, and the data in ko_KR locale into ko_KR_dump.txt by normalizing:

```
% cubrid dumplocale -d -n -a both -c -w -z ko_KR > ko_KR_dump.txt
```

It is highly recommended to redirect the console output to a file, as it can be very big data, and seeking information could prove to be difficult.

Step 6: Starting CUBRID-Related Processes

All CUBRID-related processes should be started in an identical environmental setting. The CUBRID server, the broker, CAS, and CSQL should use the locale binary file of an identical version. Also CUBRID HA should use the same setting. For example, in the CUBRID HA, master server, slave server and replica server should use the same environmental variable setting.

There is no check on the compatibility of the locale used by server and CAS (client) process, so the user should make sure the LDML files used are the same.

Locale library loading is one of the first steps in CUBRID startup. Locale (collation) information is required for initializing databases structures (indexes depends on collation). This process is performed by each CUBRID process which requires locale information: server, CAS, CSQL, createdb, copydb, unload, load DB.

The process of loading a locale library is as follows.

- If no lib path is provided, CUBRID will try to load $CUBRID/lib/libcubrid_<*lang_name*>.so file; if this file is not found, then CUBRID assumes all locales are found in a single library: **$CUBRID/lib/libcubrid_all_locales.so**.
- If suitable locale library cannot be found or any other error occurs during loading, the CUBRID process stops.

- If collations between the database and the locale library are different, the CUBRID process cannot start. To include the newly changed collations of the locale library, firstly synchronize the database collation with the system collation by running **cubrid synccolldb** command. Next, update from the existing database to the wanted collations of schemas and data. For more details, see Synchronization of Database Collations with System Collations.

Synchronization of Database Collations with System Collations

CUBRID's normal operation requires that the system collation and the database collation must be the same. The system locale means that the locale which include built-in locales and library locales created through cubrid_locales.txt (see Locale Setting), and it includes the system collation information. The database collation information is stored on the **_db_collation** system catalog table.

cubrid synccolldb utility checks if the database collation is the same with the system collation, and synchronize into the system collation if they are different. However, note that this utility doesn't transform the data itself stored on the database.

This utility can be used when the existing database collation should be changed after the system locale is changed. However, there are operations which the user have to do manually.

The user should do this operations before the synchronization. These operations can be done by running CSQL with cubrid_synccolldb_<*database_name*>.sql file, which is created by **cubrid synccolldb -c**.

- Change collation using ALTER TABLE .. MODIFY statement.
- Remove any views, indexes, triggers or partitions containing the collation.

Run synchronization with **cubrid synccolldb**. After then, do the following operations.

- Recreate views, indexes, triggers, or partitions
- Update application statements to use new collations

This utility should work only in offline mode.

synccolldb syntax is as follows.

```
cubrid synccolldb [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **synccolldb**: A command to synchronize collations of a database with collations from the system(according to contents of locales libraries and $CUBRID/conf/cubrid_locales.txt).

- *database_name*: A database name to be synchronized with collations from the system.

If [options] is omitted, **synccolldb** checks the collation differences between the system and the database, synchronize the database collation with the system collation, and create the cubrid_synccolldb_<*database_name*>.sql file including the queries of objects to be dropped before the synchronization.

The following are [options] which are used on **cubrid synccolldb**.

-c, --check-only
: This option prints out the collation information which is different between the database collation and the system collation.

-f, --force-only
: This option doesn't ask when updating the database collation with the system collation.

The following shows that how it works when the system collation and the database collation are different.

Firstly, make locale library about ko_KR locale.

```
$ echo ko_KR > $CUBRID/conf/cubrid_locales.txt
$ make_locale.sh -t 64
```

Next, create the database.

```
$ cubrid createdb --db-volume-size=20M --log-volume-size=20M xdb
en_US
```

Create a schema. At this time, specify the needed collation in each table.

```
$ csql -S -u dba xdb -i in.sql
```

```
CREATE TABLE dept(depname STRING PRIMARY KEY) COLLATE utf8_ko_cs_uca;
CREATE TABLE emp(eid INT PRIMARY KEY, depname STRING,address STRING)
COLLATE utf8_ko_cs_uca;
ALTER TABLE emp ADD CONSTRAINT FOREIGN KEY (depname) REFERENCES
dept(depname);
```

Change the locale setting of the system. If you do not any values on **cubrid_locales.txt**, the database consider that only built-in locales exist

```
$ echo "" > $CUBRID/conf/cubrid_locales.txt
```

Check the difference between system and database by running **cubrid synccolldb -c** command.

```
$ cubrid synccolldb -c xdb

----------------------------------------
----------------------------------------
Collation 'utf8_ko_cs_uca' (Id: 133) not found in database or
changed in new system configuration.
----------------------------------------
----------------------------------------
Collation 'utf8_gen_ci' (Id: 44) not found in database or changed in
new system configuration.
----------------------------------------
----------------------------------------
Collation 'utf8_gen_ai_ci' (Id: 37) not found in database or changed
in new system configuration.
----------------------------------------
----------------------------------------
Collation 'utf8_gen' (Id: 32) not found in database or changed in
new system configuration.
----------------------------------------
----------------------------------------
There are 4 collations in database which are not configured or are
changed compared to system collations.
Synchronization of system collation into database is required.
Run 'cubrid synccolldb -f xdb'
```

If the indexes exist, firstly you should remove the indexes, and change the collation of each table, then recreate the indexes directly. The process to remove indexes and change the collation of tables can be executed by using cubrid_synccolldb_xdb.sql file which was created by **synccolldb** command. On the below example, a foreign key is the index which you should recreate.

```
$ cat cubrid_synccolldb_xdb.sql

ALTER TABLE [dept] COLLATE utf8_bin;
ALTER TABLE [emp] COLLATE utf8_bin;
ALTER TABLE [emp] DROP FOREIGN KEY [fk_emp_depname];
ALTER TABLE [dept] MODIFY [depname] VARCHAR(1073741823) COLLATE
utf8_bin;
ALTER TABLE [emp] MODIFY [address] VARCHAR(1073741823) COLLATE
utf8_bin;
ALTER TABLE [emp] MODIFY [depname] VARCHAR(1073741823) COLLATE
utf8_bin;

$ csql -S -u dba -i cubrid_synccolldb_xdb.sql xdb
```

Removing the obsolete collations by executing the above cubrid_synccolldb_xdb.sql script file must be performed before forcing the synchronization of system collations into database.

Run **cubrid synccolldb** command. If the option is omitted, the message is shown to ask to run this command or not; if the **-f** option is given, the synchronization is run without checking message.

```
$ cubrid synccolldb xdb
Updating system collations may cause corruption of database.
Continue (y/n) ?
Contents of '_db_collation' system table was updated with new system
collations.
```

Recreate the dropped foreign key.

```
$ csql -S -u dba xdb

ALTER TABLE emp ADD CONSTRAINT FOREIGN KEY fk_emp_depname(depname)
REFERENCES dept(depname);
```

Note

In CUBRID, collations are identified by the ID number on the CUBRID server, and its range is from 0 to 255. LDML file is compiled with shared library, which offers the mapping information between the ID and the collation(name, attribute).

- The system collation is the collation which is loaded from the locale library, by the CUBRID server and the CAS module.
- The database collation is the collation which is stored into the **_db_collation** system table.

Collation

A collation is an assembly of information which defines an order for characters and strings. One common type of collation is called alphabetization.

If not explicitly set otherwise at column creation, the charset and collation of columns are charset and collation of table. The charset and collation are taken (in order in is found first) from the client. If the result of an expression is a character data type, gets the collation and charset by the collation inference with the operands of the expression.

Note

In CUBRID, collations are supported for a number of languages, including European and Asian. In addition to the different alphabets, some of these languages may require the definition of

expansions or contractions for some characters or character groups. Most of these aspects have been put together by the Unicode Consortium into The Unicode Standard (up to version 6.1.0 in 2012). Most of the information is stored in the DUCET file http://www.unicode.org/Public/UCA/latest/allkeys.txt which contains all characters required by most languages.

Most of the codepoints represented in DUCET, are in range 0 - FFFF, but codepoints beyond this range are included. However, CUBRID will ignore the latest ones, and use only the codepoints in range 0 - FFFF (or a lower value, if configured).

Each codepoint in DUCET has one or more 'collation elements' attached to it. A collation element is a set of four numeric values, representing weights for 4 levels of comparison. Weight values are in range 0 - FFFF.

In DUCET, a character is represented on a single line, in the form:

```
< codepoint_or_multiple_codepoints >   ; [.W1.W2.W3.W4][....]....
# < readable text explanation of the symbol/character >
```

A Korean character kiyeok is represented as follows:

```
1100  ; [.313B.0020.0002.1100] # HANGUL CHOSEONG KIYEOK
```

For example, 1100 is a codepoint, [.313B.0020.0002.1100] is one collation element, 313B is the weight of Level 1, 0020 is the weight of Level 2, 0002 is the weight of Level 3, and 1100 is the weight of Level 4.

Expansion support, defined as a functional property, means supporting the interpretation of a composed character as a pair of the same characters which it's made of. A rather obvious example is interpreting the character "æ" in the same way as the two character string "ae". This is an expansion. In DUCET, expansions are represented by using more than one collation element for a codepoint or contraction. By default, CUBRID has expansions disabled. Handling collations with expansions requires when comparing two strings several passes (up to the collation strength/level).

Charset and Collation of Column

Charset and Collation apply to string data types: **VARCHAR** (**STRING**), **CHAR** and **ENUM**. By default, all string data types inherit the default database collation and character set, but CUBRID supports two modifiers which affect collation and character set.

Charset

Character set may be specified as character string literal or as non-quoted identifier. Supported character sets:

- ISO-8859-1
- UTF-8 (with maximum 4 bytes per characters, which means it supports codepoints from 0 to 0x10FFFF)
- EUC-KR (the support for this character set is only for backward compatibility reasons, its usage is not recommended)

Note

Previous versions of CUBRID 9.0 supported EUC-KR characters when ISO-8859-1 charset (the single one available) was set. From CUBRID 9.0 Beta, this is no longer available. EUC-KR characters should be used only with EUC-KR charset.

String Check

By default, all input data is assumed to be in the server character specified when creating DB. This may be overridden by **SET NAMES** or charset introducer (or **COLLATE** string literal modifier) (For more information, see Charset and Collation of String Literals.

Invalid data may lead to undefined behavior or even crashes if string checking is disabled (by default is disabled). This can be enabled by **intl_check_input_string** system parameter. However, if you are sure that only valid data is input, you can obtain better performance by disabling string check. Only UTF-8 and EUC-KR text data is checked for valid encodings. Since ISO-8859-1 is single byte encoding and all byte values are valid, there is no checking on this charset.

Charset Conversion

When **collation** / **charset** modifiers or normal collation inference requires it, character conversion may occur. Conversions are not reversible. Generally, charset conversion is character transcoding (the bytes representing a character in one charset are replaced with other bytes representing the same character but in the destination charset).

With any conversion, losses may occur. If a character from source charset cannot be encoded in destination charset, it is replaced with a '?' character. This also applies to conversions from binary charset to any other charset. The widest character support is with UTF-8 charset, and since it encodes Unicode, one expects that all character can be encoded. However, during conversion from ISO-8859-1 to UTF-8 some "losses" occur: bytes range 80-A0 are not valid ISO-8859-1

characters but may appear in strings. After conversion to UTF-8 these characters are replaced with '?'.

Rules for conversion of values from one charset to another:

Source \ Destination	Binary	ISO-8859-1	UTF-8	EUC-KR
Binary	No change	No change The byte size unchanged. Character length unchanged.	No change. Validation per character. Invalid char replace with '?'	No change. Validation per character. Invalid char replace with '?'
ISO-8859-1	No change	No change	Byte conversion. The byte size increases. No loss of useful characters.	Byte conversion. Byte size increase. No loss of useful characters.
UTF-8	No change. The byte size unchanged. Character length increases.	Byte conversion. Byte size may decrease. Expect loss of characters.	No change	Byte conversion. Byte size may decrease. Expect loss of characters.
EUC-KR	No change. The byte size unchanged. Character length increases	Byte conversion. Byte size may decrease. Expect loss of characters	Byte conversion. Byte size may increase. No loss of useful characters.	No change

Note

Previous versions of CUBRID 9.x didn't supported binary charset. The ISO-8859-1 charset had the role of existing binary charset. Conversions from UTF-8 and EUC-KR charsets to ISO-8859-1 were performed by reinterpreting the byte content of source, not by character translation.

Collation

Collation may be specified as character string literal or as non-quoted identifier.

The following is a query(SELECT * FROM db_collation WHERE is_builtin='Yes') on the **db_collation** system table.

```
coll_id  coll_name        charset_name    is_builtin
has_expansions  contractions  uca_strength
============================================================================
============================
0        'iso88591_bin'   'iso88591'     'Yes'
'No'            0            'Not applicable'
1        'utf8_bin'       'utf8'         'Yes'
'No'            0            'Not applicable'
2        'iso88591_en_cs' 'iso88591'     'Yes'
'No'            0            'Not applicable'
3        'iso88591_en_ci' 'iso88591'     'Yes'
'No'            0            'Not applicable'
4        'utf8_en_cs'     'utf8'         'Yes'
'No'            0            'Not applicable'
5        'utf8_en_ci'     'utf8'         'Yes'
'No'            0            'Not applicable'
6        'utf8_tr_cs'     'utf8'         'Yes'
'No'            0            'Not applicable'
7        'utf8_ko_cs'     'utf8'         'Yes'
'No'            0            'Not applicable'
8        'euckr_bin'      'euckr'        'Yes'
'No'            0            'Not applicable'
9        'binary'         'binary'       'Yes'
'No'            0            'Not applicable'
```

Built-in collations are available without requiring additional user locale libraries.

Each **collation** has an associated **charset**. For this reason, it is not allowed to set incompatible pair to **character** set and **collation**.

When **COLLATE** modifier is specified without **CHARSET** modifier, then the default charset of collation is set. When **CHARSET** modifier is specified without **COLLATE** modifier, then the default collation is set. The default collation for character sets are the bin collation:

- ISO-8859-1: iso88591_bin
- UTF-8: utf8_bin
- EUC-KR: euckr_bin
- Binary: binary

Binary is the name of both the collation and its associated charset.

For more information on how to determine the collation among the expression parameters (operands) with different collations (and charsets), see How to Determine Collation among Columns with Different Collation.

CHARSET and COLLATE modifier

CUBRID supports two modifiers which affect collation and character set without following the default database collation and character set.

- **CHARACTER_SET** (alias **CHARSET**) changes the columns character set
- **COLLATE** changes the collation

```
<data_type> ::= <column_type> [<charset_modifier_clause>]
[<collation_modifier_clause>]

<charset_modifier_clause> ::= {CHARACTER_SET | CHARSET}
{<char_string_literal> | <identifier> }

<collation_modifier_clause> ::= {COLLATE } {<char_string_literal> |
<identifier> }
```

The following example shows how to set the charset of the **VARCHAR** type column to UTF-8

```
CREATE TABLE t1 (s1 VARCHAR (100) CHARSET utf8);
```

The following example shows how to change the name of column s1 to c1 and the type to CHAR(10) with the collation of utf8_en_cs (the charset is the default charset of the collation, UTF-8).

```
ALTER TABLE t1 CHANGE s1 c1 CHAR(10) COLLATE utf8_en_cs;
```

The value of the c1 column is changed to the VARCHAR(5) type whose collation is iso88591_en_ci. It is performed by using the collation iso88591_en_ci for the type of column selected first or by using sorting.

```
SELECT CAST (c1 as VARCHAR(5) COLLATE 'iso88591_en_ci') FROM t1
ORDER BY 1;
```

The following query (same sorting) is similar to the above but the output column result is the original value.

```
SELECT c1 FROM t1 ORDER BY CAST (c1 as VARCHAR(5) COLLATE
iso88591_en_ci);
```

How to Determine Collation among Columns with Different Collation

```
CREATE TABLE t (
    s1 STRING COLLATE utf8_en_cs,
    s2 STRING COLLATE utf8_tr_cs
);

-- insert values into both columns

SELECT s1, s2 FROM t WHERE s1 > s2;
```

```
ERROR: '>' requires arguments with compatible collations.
```

In the above example, column *s1* and column *s2* have different collations. Comparing *s1* with *s2* means comparing the strings to determine which column value is “larger” among the records on the table t. In this case, an error will occur because the comparison between the collation utf8_en_cs and the collation utf8_tr_cs cannot be done.

The rules to determine the types of arguments for an expression are also applied to the rules to determine the collations.

1. A common collation and a characterset are determined by considering all arguments of an expression.

2. If an argument has a different collation(and a characterset) with a common collation(and a characterset) decided in No. 1., it is changed into the common collation(and a characterset).

3. To change the collation, `CAST()` operator can be used.

Collation coercibility is used to determine the result collation of comparison expression. It expresses how easily the collation can be converted to the collation of the opposite argument. High collation coercibility when comparing two operands of an expression means that the collation can be easily converted to the collation of the opposite argument. That is, an argument with high collation coercibility can be changed to the collation of an argument with lower collation coercibility.

When an expression has various arguments with different collation, a common collation is computed based on each arguments collation and coercibility. The rules for collation inference are:

1. Arguments with higher coercibility are coerced (or casted) to collation of arguments with lower coercibility.

2. When arguments have different collation but same coercibility, the expression's collation cannot be resolved and an error is returned. However, when comparing two operands of which collation coercibility level is 11(session variable, host variable) and charset is the same, one of their collation is changed as non-bin collation if one of them is bin collation(utf8_bin, iso88591_bin, euckr_bin). See Converting Collation of Session Variable and/or Host Variable.

Below table shows the collation coercibility about arguments of the expression

Collation Coercibility	Arguments of the Expression(Operands)
-1	As an expression which has arguments with only host variables, this coercibility cannot be determined before the execution step.
0	Operand having **COLLATE** modifier
1	**Columns** with non-binary and non-bin collation
2	**Columns** with binary collation and binary charset

Collation Coercibility	Arguments of the Expression(Operands)
3	**Columns** with bin collation (iso88591_bin, utf8_bin, euckr_bin)
4	**SELECT values**, **Expression** With non-binary and non-bin collation
5	**SELECT values**, **Expression** With binary collation and binary charset
6	**SELECT values**, **Expression** With bin collation (iso88591_bin, utf8_bin, euckr_bin)
7	**Special functions** (`SYSTEM_USER()`, `DATABASE()`, `SCHEMA()`, `VERSION()`)
8	**Constants(string literals)** With non-binary and non-bin collation
9	**Constants(string literals)** With binary collation and binary charset
10	**Constants(string literals)** With bin collation (iso88591_bin, utf8_bin, euckr_bin)
11	host variables, session variables

Note

In 9.x versions, the binary collation was not available. The iso85891_bin collation had the role of existing binary collation. Since version 10.0, the coercibility of columns with iso88591_bin

was demoted from 2 to 3, that of expressions with iso88591_bin from 5 to 6, and of constants with iso88591_bin from 9 to 10.

Regarding an expression which has arguments with only host variables, (e.g. UPPER(?) as the below) this coercibility can be determined on the execution step. That is, the coercibility like this expression cannot be determined on the parsing step; therefore, COERCIBILITY function returns -1.

```
SET NAMES utf8
PREPARE st FROM 'SELECT COLLATION(UPPER(?)) col1,
COERCIBILITY(UPPER(?)) col2';
EXECUTE st USING 'a', 'a';
```

```
  col1                          col2
=====================================
  'utf8_bin'                      -1
```

For expressions having all arguments with coercibility 11 and with different collations, the common collation is resolved at runtime (this is an exception from the coercibility value-based rule for inference which would require to raise an error).

```
PREPARE st1 FROM 'SELECT INSERT(?,2,2,?)';
EXECUTE st1 USING _utf8'abcd', _binary'ef';
```

```
 insert( ?:0 , 2, 2,  ?:1 )
======================
 'aefd'
```

The following shows converting two parameters with different collation to one collation.

- **Converting into the Wanted Collation**

  The **SELECT** statement, failing to execute in the above example, is successfully executed by specifying a collation on one column by using the **CAST** operator as shown in the following query; then the two operands have the same collation.

  ```
  SELECT s1, s2 FROM t WHERE s1 > CAST (s2 AS STRING COLLATE
  utf8_en_cs);
  ```

  Also, by **CAST** s2 to bin collation, the collation coercibility of CAST (6) is higher then coercibility of s1 (1).

```
SELECT s1, s2 FROM t WHERE s1 > CAST (s2 AS STRING COLLATE
utf8_bin);
```

In the following query, the second operand “CAST (s2 AS STRING COLLATE utf8_tr_cs)” is a sub-expression. The sub-expression has higher coercibility than the column (s1) so “CAST (s2 AS STRING COLLATE utf8_tr_cs)” is converted to the collation of s1.

```
SELECT s1, s2 FROM t WHERE s1 > CAST (s2 AS STRING COLLATE
utf8_tr_cs);
```

Any expression has higher coercibility than any column. So “CONCAT (s2,'')” is converted to the collation of s1 in the following query and the query is successfully performed.

```
SELECT s1, s2 FROM t WHERE s1 > CONCAT (s2,'');
```

- **Converting Collation of Constant and Column**

In the following case, comparison is made by using the collation of s1.

```
SELECT s1, s2 FROM t WHERE s1 > 'abc';
```

- **When a Column is Created with Bin Collation**

```
CREATE TABLE t (
    s1 STRING COLLATE utf8_en_cs,
    s2 STRING COLLATE utf8_bin
);
SELECT s1, s2 FROM t WHERE s1 > s2;
```

In this case, s2 column’s coercibility is 6(bin collation) and s2 can be “fully convertible” to the collation of s1. utf8_en_cs is used.

```
CREATE TABLE t (
    s1 STRING COLLATE utf8_en_cs,
    s2 STRING COLLATE iso88591_bin
);
SELECT s1, s2 FROM t WHERE s1 > s2;
```

In this case, utf8_en_cs is used as collation, too. However, some overhead occurs to convert the charset to UTF-8 since s2 is the ISO charset.

In the following query, the charset is not converted (UTF-8 byte data in s2 is easily reinterpreted to the ISO-8859-1 charset) but character comparison is made by using the iso88591_en_cs collation.

```
CREATE TABLE t (
    s1 STRING COLLATE iso88591_en_cs,
    s2 STRING COLLATE utf8_bin
);
SELECT s1, s2 FROM t WHERE s1 > s2;
```

- **Converting Collation of Sub-Expression and Column**

```
CREATE TABLE t (
    s1 STRING COLLATE utf8_en_cs,
    s2 STRING COLLATE utf8_tr_cs
);
SELECT s1, s2 FROM t WHERE s1 > s2 + 'abc';
```

In this case, the second operand is the expression, so the collation of s1 is used.

In the following example, an error occurs. An error occurs because '+' operation is tried for s2 and s3 where the collation is different.

```
CREATE TABLE t (
    s1 STRING COLLATE utf8_en_cs,
    s2 STRING COLLATE utf8_tr_cs,
    s3 STRING COLLATE utf8_en_ci
);

SELECT s1, s2 FROM t WHERE s1 > s2 + s3;
```

```
ERROR: '+' requires arguments with compatible collations.
```

In the following example, the collation of s2 and s3 is utf8_tr_cs. Therefore, the collation of '+' expression is utf8_tr_cs, too. Expressions have higher coercibility than columns. Therefore, comparison operation is made by using the utf8_en_cs collation.

```
CREATE TABLE t (
    s1 STRING COLLATE utf8_en_cs,
    s2 STRING COLLATE utf8_tr_cs,
    s3 STRING COLLATE utf8_tr_cs
);

SELECT s1, s2 FROM t WHERE s1 > s2 + s3;
```

- **Converting Collation of Number, Date**

  Number or date constant which is convertible into string during operation always coercible into the other string's collation.

- **Converting Collation of Session Variable and/or Host Variable**

  When comparing the two operands of which collation coercibility level is 11(session variable, host variable) and charset is the same, one of their collation is changed as non-bin collation.

```
SET NAMES utf8;
SET @v1='a';
PREPARE stmt FROM 'SELECT COERCIBILITY(?), COERCIBILITY(@v1),
COLLATION(?), COLLATION(@v1), ? = @v1';
SET NAMES utf8 COLLATE utf8_en_ci;
EXECUTE stmt USING 'A', 'A', 'A';
```

When comparing @v1 and 'A', @v1's collation will be changed as utf8_en_ci, non-bin collation; therefore, @v1's value and 'A' will be the same and the result of "? = @v1" will be 1 as below.

```
   coercibility( ?:0 )   coercibility(@v1)   collation( ?:1 )
collation(@v1)          ?:2 =@v1
================================================================
==============================
                    11                  11  'utf8_en_ci'
'utf8_bin'                          1
```

```
SET NAMES utf8 COLLATE utf8_en_cs;
EXECUTE stmt USING 'A', 'A', 'A';
```

When comparing @v1 and 'A', @v1's collation will be changed as utf8_en_cs, non-bin collation; therefore, @v1's value and 'A' will be different and "? = @v1"'s result will be 0 as below.

```
   coercibility( ?:0 )   coercibility(@v1)   collation( ?:1 )
collation(@v1)          ?:2 =@v1
================================================================
==============================
                    11                  11  'utf8_en_cs'
'utf8_bin'                          0
```

However, if collations of @v1 and 'A' are different as below and the two collations are different, an error occurs.

```
DEALLOCATE PREPARE stmt;
SET NAMES utf8 COLLATE utf8_en_ci;
SET @v1='a';
PREPARE stmt FROM 'SELECT COERCIBILITY(?), COERCIBILITY(@v1),
COLLATION(?), COLLATION(@v1), ? = @v1';
SET NAMES utf8 COLLATE utf8_en_cs;
EXECUTE stmt USING 'A', 'A', 'A';
```

```
ERROR: Context requires compatible collations.
```

Charset and Collation of an ENUM type column

Charset and Collation of an ENUM type column follow the locale specified when creating DB.

For example, create the below table after creating DB with en_US.iso88591.

```
CREATE TABLE tbl (e ENUM (_utf8'a', _utf8'b'));
```

a column 'e' of the above table has ISO88591 charset and iso88591_bin collation even if the charset of the element is defined as UTF8. If the user want to apply the other charset or collation, it should be specified to the column of the table.

Below is an example to specify the collation about the column of the table.

```
CREATE TABLE t (e ENUM (_utf8'a', _utf8'b') COLLATE utf8_bin);
CREATE TABLE t (e ENUM (_utf8'a', _utf8'b')) COLLATE utf8_bin;
```

Charset and Collation of Tables

The charset and the collation can be specified after the table creation syntax.

```
CREATE TABLE table_name (<column_list>)  [CHARSET charset_name]
[COLLATE collation_name]
```

If the charset and the collation of a column are omitted, the charset and the collation of a table is used. If the charset and the collation of a table are omitted, the charset and the collation of a system is used.

The following shows how to specify the collation on the table.

```
CREATE TABLE tbl(
    i1 INTEGER,
    s STRING
) CHARSET utf8 COLLATE utf8_en_cs;
```

If the charset of a column is specified and the collation of a table is specified, the collation of this column is specified as the default collation(<collation_name>_bin) about this column's charset.

```
CREATE TABLE tbl (col STRING CHARSET utf8) COLLATE utf8_en_ci;
```

On the above query, the collation of the column col becomes utf8_bin, the default collation about this column.

```
csql> ;sc tbl

 <Class Name>

  tbl                 COLLATE utf8_en_ci

 <Attributes>

  col                 CHARACTER VARYING(1073741823) COLLATE utf8_bin
```

Charset and Collation of String Literals

The charset and the collation of a string literal are determined based on the following priority.

1. Charset Introducer introducer or COLLATE modifier of string literal

2. The charset and the collation defined by the SET NAMES Statement

3. System charset and collation(Default collation by the locale specified when creating DB)

SET NAMES Statement

The **SET NAMES** statement changes the default client charset and the collation. Therefore, all sentences in the client which has executed the statement have the specified charset and collation. The syntax is as follows.

```
SET NAMES [ charset_name ] [ COLLATE collation_name]
```

- *charset_name*: Valid charset name is iso88591, utf8, euckr and binary.
- *collation_name*: Collation setting can be omitted and all available collations can be set. The collation should be compatible with the charset; otherwise, an error occurs. To find the available collation names, look up the **db_collation** catalog VIEW (see Charset and Collation of Column).

Specifying a collation with **SET NAMES** statement is the same as specifying a system parameter **intl_collation**. Therefore, the following two statements are the same behavior.

```
SET NAMES utf8;
SET SYSTEM PARAMETERS 'intl_collation=utf8_bin';
```

The following example shows how to create the string literal with the default charset and collation.

```
SELECT 'a';
```

The following example shows how to create the string literal with the utf8 charset and utf8_bin collation(the default collation is the bin collation of the charset)

```
SET NAMES utf8;
SELECT 'a';
```

Charset Introducer

In front of the constant string, the charset introducer and the **COLLATE** modifier can be positioned. The charset introducer is the charset name starting with a underscore (_), coming before the constant string. The syntax to specify the **CHARSET** introducer and the **COLLATE** modifier for a string is as follows.

```
[charset_introducer]'constant-string' [ COLLATE collation_name ]
```

- *charset_introducer*: a charset name starting with an underscore (_), can be omitted. One of _utf8, _iso88591, _euckr and _binary can be entered.
- *constant-string*: a constant string value.
- *collation_name*: the name of a collation, which can be used in the system, can be omitted.

The default charset and collation of the constant string is determined based on the current database connected (the **SET NAMES** statement executed last or the default value).

- When the string charset introducer is specified and the **COLLATE** modifier is omitted, the collation is:
  - if the charset introducer is the same as client charset (from a previous SET NAMES), then the client collation is applied.
  - if the charset introducer does not match the client charset, then the bin collation(one of euckr_bin, iso88591_bin and utf8_bin) corresponding to charset introducer is applied.
- When the charset introducer is omitted and the **COLLATE** modifier is specified, the character is determined based on collation.

The following example shows how to specify the charset introducer and the **COLLATE** modifier.

```
SELECT 'cubrid';
SELECT _utf8'cubrid';
SELECT _utf8'cubrid' COLLATE utf8_en_cs;
```

The following example shows how to create the string literal with utf8 charset and utf8_en_cs collation. The **COLLATE** modifier of **SELECT** statement overrides the collation specified by **SET NAMES** syntax.

```
SET NAMES utf8 COLLATE utf8_en_ci;
SELECT 'a' COLLATE utf8_en_cs;
```

Charset and Collation of Expressions

The charset and collation of expression's result are inferred from charset and collation of arguments in the expression. Collation inference in CUBRID is based on coercibility. For more information, see How to Determine Collation among Columns with Different Collation.

All string matching function(LIKE, REPLACE, INSTR, POSITION, LOCATE, SUBSTRING_INDEX, FIND_IN_SET, etc) and comparison operators(<, >, =, etc) take collation into account.

Charset and Collation of System Data

The system charset is taken from the locale specified when creating DB. The system collation is always the bin collation (<*charset*>_bin) of system charset. CUBRID supports three charset(iso88591, euckr, utf8), and accordingly three system collations(iso88591_bin, euckr_bin, utf8_bin).

Impact of Charset Specified When Creating DB

The locale specified when creating DB affects the following.

- Character supported in identifiers and casing rules (called “alphabet”)
- Default locale for date - string conversion functions
- Default locale for number - string conversion functions
- Console conversion in CSQL

Casing and identifiers

In CUBRID, identifiers are cases insensitive. Tables, columns, session variables, triggers, stored procedures are stored in lower case. Authentication identifiers (user and group names) are stored in uppercase.

The ISO-8859-1 charset contains only 255 characters, so the primitives are able to use built-in data. Also the EUC-KR charset, from which only the ASCII compatible characters are considered for casing (and are handled in the code), is built-in.

The UTF-8 charset is a special case: There are built-in variants of UTF-8 locales (like en_US.utf8, tr_TR.utf8 and ko_KR.utf8) and LDML locales.

The built-in variant implement only the characters specific to the locale (ASCII characters for en_US.utf8 and ko_KR.utf8, ASCII + Turkish glyphs [1] for tr_TR.utf8). This means that while all UTF-8 characters encoded on maximum 4 bytes are still supported and accepted as identifiers, most of them are not handled as letters, and treated as any normal Unicode character by casing primitives. For instance, character “È” (Unicode codepoint 00C8) is allowed, but an identifier containing it will not be normalized to “è” (lower case).

```
CREATE TABLE ÈABC;
```

Therefore, after running above query, it will have a table name with “Èabc” into the system table, **_db_class**.

Using a LDML locale (built-in variants can also be overridden with a LDML variant), extends the supported Unicode characters up to codepoint FFFF. For instance, if the locale is set by es_ES.utf8 when creating DB and the corresponding locale library is loaded, the previous statement will create a table with the name “èabc”.

As previously mentioned, a set of casing rules and supported characters (letters) forms an “alphabet” in CUBRID (this is actually a tag in LDML). Some locales, like tr_TR and de_DE have specific casing rules: - in Turkish: lower(‘I’)=’ı’ (dot-less lower i); upper (‘i’) = ‘İ’ (capital I with dot). - in German: upper (‘ß’)=’SS’ (two capital S letters).

Because of this, such locales have two sets of alphabets: one which applies to system data (identifiers) and one which applies to user data. The alphabet applying to user data include the special rules, while the system (identifiers) alphabet do not, thus making the system alphabets compatible between locales. This is required to avoid issues with identifiers (like in Turkish, where casing of the group name “public” results in errors -> “PUBLİC” != “PUBLIC”).

It also provides a compatibility between databases with different locales (should be able to export - import schema and data).

String literal input and output

String literals data may be entered to CUBRID by various ways:

- API interface (CCI)
- language dependent interface - JDBC, Perl driver, etc.
- CSQL - command line from console or input file

When receiving character data through drivers, CUBRID cannot be aware of the charset of those strings. All text data contained between quotes (string literals) are handled by CUBRID as raw bytes; the charset meta-information must be provided by client. CUBRID provides a way for the client to instruct it about which type of encoding is using for its character data. This is done with the SET NAMES statement or with charset introducer.

Text Conversion for CSQL

Text console conversion works in CSQL console interface. Most locales have associated character set (or codepage in Windows) which make it easy to write non-ASCII characters from console. For example in LDML for tr_TR.utf8 locale, there is a line:

```
<consoleconversion type="ISO88599" windows_codepage="28599"
linux_charset="iso88599,ISO_8859-9,ISO8859-9,ISO-8859-9">
```

If the user set its console in one of the above settings (chcp 28599 in Windows, or export LANG=tr_TR.iso88599 in Linux), CUBRID assumes all input is encoded in ISO-8859-9 charset, and converts all data to UTF-8. Also when printing results, CUBRID performs the reverse conversion (from UTF-8 to ISO-8859-9). In Linux, to prevent this transform, using UTF-8(ex: export LANG=tr_TR.utf8) directly is recommended.

The setting is optional in the sense that the XML tag is not required in LDML locale file. For example, the locale km_KH.utf8 does not have an associated codepage.

Example for configuring French language and inputting French characters

Enable fr_FR in cubrid_locales.txt, compile the locales(see Locale Setting) and set fr_FR.utf8 when you create DB.

In Linux:

- Set console to receive UTF-8; set LANG=fr_FR.utf8 or en_US.utf8 (any locale with UTF-8). This setting will allow to input any UTF-8 character (not only French specific)
- or, set console to receive ISO-8859-15; set LANG=fr_FR.iso885915; in LDML <consoleconversion> tag, set linux_charset="iso885915". This will receive only ISO-8859-15 characters which will be converted by CSQL to UTF-8 encoding.

In Windows:

- Set windows codepage to 28605 (chcp 28605 in a command prompt); in LDML <consoleconversion> tag, set windows_codepage="28605". Codepage 28605 is the corresponding for ISO-8859-15 charset.

Example for configuring Romanian and inputting Romanian characters

Enable ro_RO in cubrid_locales.txt, compile the locales(see Locale Setting) and set ro_RO.utf8 when you create DB.

In Linux:

- Set console to receive UTF-8; set LANG=ro_RO.utf8 or en_US.utf8 (any locale with UTF-8). This setting will allow to input any UTF-8 character (not only Romanian specific)
- or, set console to receive ISO-8859-2; set LANG=ro_RO.iso88592; in LDML <consoleconversion> tag, set linux_charset="iso88592". This will receive only ISO-8859-15 characters which will be converted by CSQL to UTF-8 encoding.

In Windows:

- Set windows codepage to 1250 (chcp 1250 in a command prompt); in LDML <consoleconversion> tag, set windows_codepage="1250". Codepage 1250 is the corresponding for ISO-8859-2 charset. Codepage 1250 contains characters specific to some Central and Eastern European languages, including Romanian. Please note that characters outside codepage 1250 will not be properly displayed.

  To use special characters which exist on Romanian alphabet(e.g. "S" and "T" with cedilla bellow), the Romanian legacy keyboard setting of "Control Panel" on Windows is required.

- ISO8859-2 contains some characters which codepage 1250 does not have, so you cannot input or output all characters of ISO8859-2 with CSQL.

At input, the console conversion process takes all input (including statements) and performs the conversion (only if it is required - if it contains characters that needs conversion). At output

(printing results, error messages), CSQL is more selective and does not convert all texts. For instance, printing of numeric values is not filtered through console conversion (since number text contains only ASCII characters).

Unicode Normalization

Glyphs [1] can be written in various forms using Unicode characters/codepoints. Most known are the decomposed and composed forms. For instance, the glyph 'Ä' is written in composed form with a single codepoint: 00C4, in UTF-8 these has two bytes: C3 84. In (fully) decomposed form, it written with two codepoints: 0041 ('A') and 0308 (COMBINING DIAERESIS), and in UTF-8 is encode using 3 bytes: 41 CC 88. Most text editors are able to handle both forms, so both encodings will appear as the same glyph: 'Ä'. Internally, CUBRID "knows" to work only with "fully composed" text.

For clients working with "fully decomposed" text, CUBRID can be configured to convert such text to "fully composed" and serve them back as "fully decomposed". Normalization is not a locale specific feature, it does not depend on locale.

unicode_input_normalization system parameter controls the composition at system level. For more details, see unicode_input_normalization.

The main use case is with both enabled (**unicode_input_normalization**, **unicode_output_normalization**): this ensures that a string from a client knowing only decomposed Unicode is still properly handled by CUBRID. A second use case is with **unicode_input_normalization** = yes and **unicode_output_normalization** = no, for a client able to handle both types of Unicode writing.

Contraction and Expansion of Collation

CUBRID supports contraction and expansion for collation. Contraction and expansion are available for UTF-8 charset collation. You can see the contraction and expansion of collation in the collation setting in the LDML file. Using contraction and expansion affects the size of locale data (shared library) and server performance.

Contraction

A contraction is a sequence consisting of two or more codepoints, considered a single letter in sorting. For example, in the traditional Spanish sorting order, "ch" is considered a single letter. All words that begin with "ch" sort after all other words beginning with "c", but before words starting with "d". Other examples of contractions are "ch" in Czech, which sorts after "h", and "lj" and "nj" in Croatian and Latin Serbian, which sort after "l" and "n" respectively. See https://unicode-org.github.io/icu/userguide/collation/concepts for additional information. There are also some contractions defined in http://www.unicode.org/Public/UCA/latest/allkeys.txt DUCET.

Contractions are supported in both collation variants: with expansions and without expansions. Contractions support requires changes in a significant number of key areas. It also involves storing a contraction table inside the collation data. The handling of contractions is controlled by LDML parameters **DUCETContractions="ignore/use" TailoringContractions="ignore/use"** in <settings> tag of collation definition. The first one controls if contractions in DUCET file are loaded into collation, the second one controls if contractions defined by rules in LDML are ignore or not (easier way then adding-deleting all rules introducing contractions).

Expansion

Expansions refer to codepoints which have more than one collation element. Enabling expansions in CUBRID radically changes the collation's behavior as described below. The CUBRIDExpansions="use" parameter controls the this behavior.

Collation without Expansion

In a collation without expansions, each codepoint is treated independently. Based on the strength of the collation, the alphabet may or may not be fully sorted. A collation algorithm will sort the codepoints by comparing the weights in a set of levels, and then will generate a single value, representing the weight of the codepoint. String comparison will be rather straight-forward. Comparing two strings in an expansion-free collation means comparing codepoint by codepoint using the computed weight values.

Collation with Expansion

In a collation with expansions, some composed characters (codepoints) are to be interpreted as an ordered list of other characters (codepoints). For example, 'æ' might require to be interpreted the same way as 'ae', or 'ä' as ''ae'' or ''aa''. In DUCET, the collation element list of 'æ' will be the concatenation of collation element lists of both 'a' and 'e', in this order. Deciding a particular order for the codepoints is no longer possible, and neither is computing new weight values for each character/codepoint.

In a collation with expansions, string comparison is done by concatenating the collation elements for the codepoints/contractions in two lists (for the two strings) and then comparing the weights in those lists for each level.

Example 1

The purpose of these examples is to show that under different collation settings (with or without expansion support), string comparison might yield different results.

Here there are the lines from DUCET which correspond to a subset of codepoints to be used for comparisons in the examples below.

```
0041  ; [.15A3.0020.0008.0041] # LATIN CAPITAL LETTER A
0052  ; [.1770.0020.0008.0052] # LATIN CAPITAL LETTER R
0061  ; [.15A3.0020.0002.0061] # LATIN SMALL LETTER A
0072  ; [.1770.0020.0002.0072] # LATIN SMALL LETTER R
00C4  ; [.15A3.0020.0008.0041][.0000.0047.0002.0308] #
LATIN CAPITAL LETTER A WITH DIAERESIS;
00E4  ; [.15A3.0020.0002.0061][.0000.0047.0002.0308] #
LATIN SMALL LETTER A WITH DIAERESIS;
```

Three types of settings for the collation will be illustrated:

- Primary strength, no casing (level 1 only)
- Secondary strength, no casing (levels 1 and 2)
- Tertiary strength, uppercase first (levels 1, 2 and 3)

From now on, sorting of the strings “Ar” and “Är” will be attempted.

Collation without Expansions Support

When expansions are disabled, each codepoint is reassigning a new single valued weight. Based on the algorithms described above the weights for A, Ä, R and their lowercase correspondents, the order of the codepoints for these characters, for each collation settings example above, will be as follows.

- Primary strength: A = Ä < R = r
- Secondary strength: A < Ä < R = r
- Tertiary strength: A < Ä < R < r

The sort order for the chosen strings is easy to decide, since there are computed weights for each codepoint.

- Primary strength: “Ar” = “Är”
- Secondary strength: “Ar” < “Är”
- Tertiary strength: “Ar” < “Är”

Collation with Expansions

The sorting order is changed for collation with expansion. Based on DUCET, the concatenated lists of collation elements for the strings from our samples are provided below:

```
Ar [.15A3.0020.0008.0041][.1770.0020.0002.0072]
Är [.15A3.0020.0008.0041][.0000.0047.0002.0308][.
1770.0020.0002.0072]
```

It is rather obvious that on the first pass, for level 1 weights, 0x15A3 will be compared with 0x15A3. In the second iteration, the 0x0000 weight will be skipped, and 0x1770 will be compared with 0x1770. Since the strings are declared identical so far, the comparison will continue on the level 2 weights, first comparing 0x0020 with 0x0020, then 0x0020 with 0x0047, yielding “Ar” < “Är”. The example above was meant to show how strings comparison is done when using a collation with expansion support.

Let us change the collation settings, and show how one may obtain a different order for the same strings when using a collation for German, where “Ä” is supposed to be interpreted as the character group “AE”. The codepoints and collation elements of the characters involved in this example are as follows.

```
0041  ; [.15A3.0020.0008.0041] # LATIN CAPITAL
LETTER A
0045  ; [.15FF.0020.0008.0045] # LATIN CAPITAL
LETTER E
0072  ; [.1770.0020.0002.0072] # LATIN SMALL
LETTER R
00C4  ; [.15A3.0020.0008.0041][.15FF.
0020.0008.0045] # LATIN CAPITAL LETTER A WITH
DIAERESIS; EXPANSION
```

When comparing the strings “Är” and “Ar”, the algorithm for string comparison when using a collation with expansion support will involve comparing the simulated concatenation of collation element lists for the characters in the two strings.

```
Ar [.15A3.0020.0008.0041][.1770.0020.0002.0072]
Är [.15A3.0020.0008.0041][.15FF.0020.0008.0045][.
1770.0020.0002.0072]
```

On the first pass, when comparing level 1 weights, 0x15A3 will be compared with 0x15A3, then 0x1770 with 0x15FF, where a difference is found. This comparison yields “Ar” > “Är”, a result completely different than the one for the previous example.

Example 2

In Canadian French sorting by the collation with expansion, accent is compared from end of string towards the beginning.

- Normal Accent Ordering: cote < coté < côte < côté
- Backward Accent Ordering: cote < côte < coté < côté

Operations Requiring Collation and Charset

Charset

Charset information is required for functions which use character primitives. There are exceptions: `OCTET_LENGTH()` and `BIT_LENGTH()` do not require charset internally to return the length in bytes and bits. However, for the same glyph (character symbol) stored in different charset, they return different values:

```
CREATE TABLE t (s_iso STRING CHARSET iso88591, s_utf8 STRING CHARSET
utf8);
SET NAMES iso88591;
INSERT INTO t VALUES ('È','È');

-- the first returns 1, while the second does 2
SELECT OCTET_LENGTH(s_iso), OCTET_LENGTH(s_utf8) FROM t;
```

The previous example should be run from console (or a client) with ISO-8859-1 charset.

Collation

Collation is required in functions and operators which involves a comparison between two strings or matching two strings. These includes functions like: `STRCMP()`, `POSITION()`, LIKE condition, and operators (<,= , >=, etc.). Also clauses like ORDER BY, GROUP BY and aggregates(`MIN()`, `MAX()`, `GROUP_CONCAT()`) use collation.

Also, collation is considered in `UPPER()` and `LOWER()` functions, in the following manner:

- Each collation has a default (parent) locale.
- UPPER and LOWER functions are performed using the user alphabet of the default locale of the collation.

For most collations, the default locale is obvious (is embedded in the name):

- utf8_tr_cs → tr_TR.utf8

- iso88591_en_ci → en_US (ISO-8859-1 charset)

The bin collations have the following default locales:

- iso88591_bin → en_US (ISO-8859-1 charset)
- utf8_bin (en_US.utf8 - built-in locale - and handles ASCII characters only)
- euckr_bin (ko_KR.euckr - built-in locale - and handles ASCII characters only)

There are some generic collations available in LDML. These collations have as default locale, the locale in which they are first found. The order of loading is the locales order from **$CUBRID/conf/cubrid_locales.txt**. Assuming the default order (alphabetical), the default locale for all generic LDML collations is de_DE (German).

Charset conversion

For the three charsets supported by CUBRID the conversion rules are:

- General rules is that character transcoding occurs (representation of bytes is changed to the destination charset) - precision is kept, while byte size may change (for variable character data). When changing charset of a column with fixed precision (ALTER..CHANGE), the size in bytes always changes (size = precision x charset multiplier).
- Exceptions are: utf8 and euckr to iso88591 - the precision is kept and data can be truncated.

The following is an example that you run queries by changing the charset as utf8 in the database that the locale specified when creating DB is en_US(.iso88591).

```
SET NAMES utf8;
CREATE TABLE t1(col1 CHAR(1));
INSERT INTO t1 VALUES ('Ç');
```

When you run above queries, the data of col1 is truncated because 'Ç' is two bytes character and col1's size is one byte. The charset of database is iso88591, and the charset of input data is utf8; it converts utf8 to iso88591.

Collation settings impacting CUBRID features

LIKE Conditional Optimization

The **LIKE** conditional expression compares patterns between string data, and returns TRUE if a string whose pattern matches the search word is found.

As already proven above, when using a “collation without expansion support”, each codepoint will receive a single integer value, representing its weight in the comparison process. This weight value is computed based on collation settings (strength, casing etc.). Due to the fact that characters can always be regarded as single entities, trying to match a string with a pattern using the **LIKE** predicate is equivalent to checking if the string can be found in a certain range of strings. For example in order to process a predicate such as ‘’s LIKE ‘abc%’ ‘’, CUBRID will first rewrite it as a range restriction for the string “s”. “s LIKE ‘abc%’” means that “s” must start with the string “abc”. In terms of string comparison, this is equivalent, in expansion-free collations, with “s” being greater than “abc”, but smaller than its successor (using the English alphabet, the successor of “abc” would be “abd”).

```
s LIKE 'abc%' → s ≥ 'abc' AND s < 'abd' (if using strictly the
English alphabet)
```

This way, the actual interpretation of **LIKE** is replaced with simple comparisons, but “Collations with expansion support” behave differently.

To compare strings when using such a collation means comparing the concatenated lists of collation elements for each codepoint or expansion, level by level. For more information about comparing strings on the collation with expansion, see Expansion.

If the **LIKE** predicate rewrite method is kept the same as in a collation with no expansion support as above example, the comparison result can be wrong. To ensure the right query result, the **LIKE** predicate rewrite method is ran differently as the below example. That is, the **LIKE** predicate is added as a filter to exclude the wrong data which can be added in a collation with expansion.

```
s LIKE 'abc%' → s ≥ 'abc' AND s < 'abd' and s LIKE 'abc%' (if using
strictly the English alphabet)
```

Index Covering

Covering index scan is query optimization, in which if all values in query can be computed using only the values found in the index, without requiring additional row lookup in heap file. For more information, see Covering Index.

In the collation without casing, for two strings values, ‘abc’ and ‘ABC’, only one value is stored in the index(this is either ‘abc’ or ‘ABC’ depending which one was inserted first). As a result, the incorrect result may happen when at least two different strings produce the same sort key in a given collation. For this reason, for all UTF-8 collations with strength level less than 4 (quaternary), the index covering query optimization is disabled.

This is controlled by strength=”tertiary/quaternary” in <strength> tag of collation definition in LDML. It should be considered to set this level as maximum strength, because the quaternary

strength level requires not only more memory space and bigger size of the shared library file, but also string-comparison time.

For more information about collations, see Collation.

Summary of CUBRID Features for Each Collation

Collation	LIKE condition kept after rewrite to range	Allows index covering
iso88591_bin	No	Yes
iso88591_en_cs	No	Yes
iso88591_en_ci	Yes	No
utf8_bin	No	Yes
euckr_bin	No	Yes
utf8_en_cs	No	Yes
utf8_en_ci	Yes	No
utf8_tr_cs	No	Yes
utf8_ko_cs	No	Yes
utf8_gen	No	Yes
utf8_gen_ai_ci	Yes	No
utf8_gen_ci	Yes	No

Collation	LIKE condition kept after rewrite to range	Allows index covering
utf8_de_exp_ai_ci	Yes	No
utf8_de_exp	Yes	No
utf8_ro_cs	No	Yes
utf8_es_cs	No	Yes
utf8_fr_exp_ab	Yes	No
utf8_ja_exp	Yes	No
utf8_ja_exp_cbm	Yes	No
utf8_km_exp	Yes	No
utf8_ko_cs_uca	No	Yes
utf8_tr_cs_uca	No	Yes
utf8_vi_cs	No	Yes
binary	No	Yes

Viewing Collation Information

To view the collation information, use `CHARSET()`, `COLLATION()` and `COERCIBILITY()` functions.

The information of the database collation can be shown on db_collation system view or SHOW COLLATION.

Using i18n characters with JDBC

CUBRID JDBC stores string type values received from server using String and CUBRIDBinaryString objects. String objects uses UTF-16 internally to store each character. It should be used to store any string DB value except those having binary charset. CUBRIDBinaryString uses byte array and should be used to store database string values of binary charset. The data buffer of each string value received from server is accompanied by the charset of the value from server. The charset is either the charset of column or expression's result or system charset for any other string without correspondent in database.

Note

In previous versions, the connection charset was used to instantiate JDBC String objects from database values.

Create table with one column having UTF-8 charset and the other of binary charset:

```
CREATE TABLE t1(col1 VARCHAR(10) CHARSET utf8, col2 VARCHAR(10)
CHARSET binary);
```

Insert one row (the second column has random value bytes):

```
Connection conn = getConn(null);
PreparedStatement st = conn.prepareStatement("insert into t1
values( ?, ? )");
byte[] b = new byte[]{(byte)161, (byte)224};
CUBRIDBinaryString cbs = new CUBRIDBinaryString(b);
String utf8_str = new String("abc");
st.setObject(1, utf8_str);
st.setObject(2, cbs);
st.executeUpdate();
```

Query the table and show contents (for binary string - we display a hex dump, for other charsets - the String value):

```
ResultSet rs = null;
Statement stmt = null;
rs = stmt.executeQuery("select col1, col2 from t1;");
ResultSetMetaData rsmd = null;
rsmd = rs.getMetaData();
int numberofColumn = rsmd.getColumnCount();
while (rs.next()) {
        for (int j = 1; j <= numberofColumn; j++) {
                String columnTypeName = rsmd.getColumnTypeName(j);
```

```
                int columnType = rsmd.getColumnType(j);
                if (((CUBRIDResultSetMetaData)
rsmd).getColumnCharset(j).equals("BINARY")) {
                        // database string with binary charset
                        Object res;
                        byte[] byte_array = ((CUBRIDBinaryString)
res).getBytes();
                        res = rs.getObject(j);
                        System.out.println(res.toString());
                } else {
                        // database string with any other charset
                        String res;
                        res = rs.getString(j);
                        System.out.print(res);
                }
        }
}
```

Timezone Setting

Timezone can be set by system parameters; a **timezone** parameter which is set on a session, and a **server_timezone** parameter which is set on a database server. For details, see Timezone Parameter.

A **timezone** parameter is a parameter about a session. This setting value can be kept by session unit.

```
SET SYSTEM PARAMETERS 'timezone=Asia/Seoul';
```

If this value is not set, it follows **server_timezone**'s setting as default.

A **server_timezone** parameter is a parameter about a database server.

```
SET SYSTEM PARAMETERS 'server_timezone=Asia/Seoul';
```

If this value is not set, it follows OS's setting.

To use timezone information, timezone type should be used. For details, see Date/Time Types with Timezone.

When timezone is set by a region name, it requires a separate timezone library, To use an updated library which has a changed timezone information, not an installed timezone information, timezone library should be compiled after timezone information is changed.

The following is an example to compile a timezone library after updating a timezone information through IANA (http://www.iana.org/time-zones).

For details, see the following description.

Compiling Timezone Library

To use a timezone by specifying a timezone region name, timezone library is required. This is provided as default when CUBRID is installed. By the way, to update a timezone region information as a latest one, timezone library should be compiled after updating a recent code which can be downloaded in IANA (http://www.iana.org/time-zones).

The following is a process to update timezone library as a recent one.

At first, stop cubrid service and operate the following process.

Windows

1. Download the recent data from http://www.iana.org/time-zones .

   Download the linked file in "Latest version"'s "Time Zone Data".

2. Decompress the compressed file to **%CUBRID%/timezones/tzdata** directory.

3. Run **%CUBRID%/bin/make_tz.bat**.

   libcubrid_timezones.dll is created at the **%CUBRID%/lib** directory.

```
make_tz.bat
```

Note

To run **make_locale** script in Windows, one of Visual C++ 2005, 2008 or 2010 should be installed.

Linux

1. Download the recent data from http://www.iana.org/time-zones .

   Download the linked file in "Latest version"'s "Time Zone Data".

2. Decompress the compressed file to **$CUBRID/timezones/tzdata** directory.

3. Run **make_tz.sh**.

libcubrid_timezones.so is created at the **$CUBRID/lib** directory.

```
make_tz.sh
```

Timezone library and database compatibility

The timezone library built by CUBRID includes an MD5 checksum of the timezone data it contains. This hash is stored in all databases which are created with that library, into column **timezone_checksum** of **db_root** system table. If timezone library is changed (recompiled with newer timezone data) and the checksum changes, the CUBRID server, nor any other CUBRID tool cannot be started with existing databases. To avoid such incompatibility, one option is to use **extend** argument of the **make_tz** tool. When this option is provided, the **timezone_checksum** value of database is also changed with the new MD5 checksum of the new timezone library. The extend feature should be used when you decide to use a different version of timezone library from the IANA site. It does two things:

- It generates a new library by merging the old timezone data with the new timezone data. After the merge, all the timezone regions that aren't present in the new timezone database are kept in order to ensure backward compatibility with the data in the database tables.
- The second thing that it does is to update the timezone data in the tables in the situation when backward compatibility could not be ensured in the first phase when the new timezone library was generated. This situation can occur when an offset rule or a daylight saving rule changes.

When you run **make_tz** with the extend option all the databases in your database directory file (**databases.txt**) are updated together with the MD5 checksum. There are some corner cases:

- There is the situation when multiple users share the same **CUBRID** installation and an extend is done by one of them. If that user doesn't have access rights on the files that contain the databases of the other users, those databases will not be updated. After that, if a different user for whom the update wasn't made will try to do an extend on his or her databases, he or she will not be able to do this because the checksum of the library will be different from the one in his or her databases.
- Also, if the **CUBRID_DATABASES** environment variable has a different value for some users, they will have different **databases.txt** files. In this situation, currently, the update of all the databases at once will not be possible.

There would be two solutions for this with the current implementation:

- Each user grants access to the folders that contain his or her databases and also the **CUBRID_DATABASES** variable must have the same value.

- If this is is not possible, then we can do the following for each user except the last one:
  - Backup the current timezone library and the **databases.txt** file
  - Delete from the **databases.txt** file all the databases except for the ones of the current user
  - Run the extend
  - Restore the **databases.txt** file and the timezone library

For the last user the single difference is that only the **databases.txt** file should be backed up and restored.

In Linux:

```
make_tz.sh -g extend
```

In Windows:

```
make_tz.bat /extend
```

Usage of timezone data types with JDBC

JDBC CUBRID driver is completely dependent on CUBRID server for timezone information. Although, Java and CUBRID uses the same primary source of information for timezone (IANA), the names of regions and timezone information should be considered as incompatible.

All CUBRID data types having timezone are mapped to **CUBRIDTimestamptz** Java objects.

Using JDBC to insert value with timezone:

```
String datetime = "2000-01-01 01:02:03.123";
String timezone = "Europe/Kiev";
CUBRIDTimestamptz dt_tz =
CUBRIDTimestamptz.valueOf(datetime, false, timezone);
PreparedStatement  pstmt = conn.prepareStatement("insert
into t values(?)");
pstmt.setObject(1, ts);
pstmt.executeUpdate();
```

Using JDBC to retrieve value with timezone:

```
String sql = "select * from tz";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

```
CUBRIDTimestamptz object1 = (CUBRIDTimestamptz)
rs.getObject(1);
System.out.println("object: " + object1.toString());
```

Internally, CUBRID JDBC stores the date/time parts of **CUBRIDTimestamptz** object into a 'long' value (inheritanted through Date object) which holds the number of milliseconds elapsed since 1st January 1970 (Unix epoch). The internal encoding is performed in UTC time reference, which is different from Timestamp objects which uses local timezone. For this reason, a **CUBRIDTimestamptz** object created with the same timezone as Java local timezone will not hold the same internal epoch value. In order to provide the Unix epoch, the **getUnixTime()** method may be used:

```
String datetime = "2000-01-01 01:02:03.123";
String timezone = "Asia/Seoul";
CUBRIDTimestamptz dt_tz =
CUBRIDTimestamptz.valueOf(datetime, false, timezone);
System.out.println("dt_tz.getTime: " + dt_tz.getTime());
System.out.println("dt_tz.getUnixTime: " +
dt_tz.getUnixTime ());
```

Configuration Guide for Characters

Database designers should take into account character data properties when designing the database structure. The following is the summarized guide when configuring aspects related to CUBRID character data.

Locale

- By default, en_US gives best performance. If you have a plan to use only English, this is recommended.
- Using UTF-8 locale will increase storage requirement of fixed char(CHAR) by 4 times; using EUC-KR increases storage 3 times.
- If user string literals have different charset and collation from system, query strings will grow as the string literals are decorated with them.
- If localized (non-ASCII) characters will be used for identifiers, then use an .utf8 locale
- Once established the UTF-8 charset for DB, it is best to use a LDML locale (this ensures that identifier names containing most Unicode characters are correctly cased) than a system locale.
- Setting a locale affects also conversion functions(intl_date_lang, intl_number_lang).

- When you set the locale during creating DB, there should be no concern on charset and collation of string-literals or user tables columns; all of them can be changed at runtime (with `CAST()` in queries) or ALTER .. CHANGE for a permanent change.

CHAR and VARCHAR

- Generally, use VARCHAR if there are large variations in actual number of characters in user data.
- CHAR type is fixed length type. Therefore, Even if you store only English character in CHAR type, it requires 4 bytes storage in UTF-8 and 3 bytes in EUC-KR.
- The precision of columns refers to the number of characters (glyphs).
- After choosing precision, charset and collation should be set according to most used scenarios.

Choosing Charset

- Even if your text contains non-ASCII character, use utf8 or euckr charsets only if application requires character counting, inserting, replacing.
- For CHAR data, the main concern should be storage requirement (4x or utf8, 3x for euckr).
- For both CHAR and VARCHAR data, there is some overhead when inserting/updating data: counting the precision (number of characters) of each instance is more consuming for non-ISO charsets.
- In queries, charset of expressions may be converted using `CAST()` operator.

Choosing Collation

- If no collation dependent operations are performed (string searching, sorting, comparisons, casing), than choose bin collation for that charset or binary collation
- Collation may be easily overridden using `CAST()` operator, and COLLATE modifier (in 9.1 version) if charset is unchanged between original charset of expression and the new collation.
- Collation controls also the casing rules of strings
- Collations with expansions are slower, but are more flexible and they perform whole-word sorting

Normalization

- If your client applications send text data to CUBRID in decomposed form, then configure **unicode_input_normalization** = yes, so that CUBRID re-composes it and handles it in composed form
- If your client "knows" to handle data only in decomposed form, than set **unicode_output_normalization** = yes, so that CUBRID always sends in decomposed form.
- If the client "knows" both forms, then leave **unicode_output_normalization** = no

CAST vs COLLATE

- When building statements, the `CAST()` operator is more costly than COLLATE modifier (even more when charset conversion occurs).
- COLLATE modifier does not add an additional execution operator; using COLLATE modifier should enhance execution speed over using `CAST()` operator.
- COLLATE modifier can be used only when charset is not changed

Remark

- Query plans printing: collation is not displayed in plans for results with late binding.
- Only the Unicode code-points in range 0000-FFFF (Basic Multilingual Plan) are normalized.
- Some locales use space character as separator for digit grouping (thousands, millions, ..). Space is allowed but not working properly in some cases of localized conversion from string to number.

Note

- In 9.2 or lower version, user defined variable cannot be changed into the different collation from the system collation. For example, "set @v1='a' collate utf8_en_cs;" syntax cannot be executed when the system collation is iso88591.
- In 9.3 or higher version, the above constraint no more exists.

Guide for Adding Locales and Collations

Most new locales and/or collations can be added by user simply by adding (or changing) a new (existing) LDML file. The LDML files format used by CUBRID are derived from generic Unicode

Locale Data Markup Language (http://www.unicode.org/reports/tr35/). The tags and attributes which are specific only to CUBRID can be easily identified (they contain a "cubrid" into the naming).

The best approach to add a new locale is to copy existing LDML file and tweak various setting until desired results are obtained. The filename must be formatted like cubrid_<*language*>.xml and be placed in the folder **$CUBRID/locales/data/ldml**. The <*language*> part should be a ASCII string (normally five characters) in IETF format (https://en.wikipedia.org/wiki/BCP_47). After creating the LDML file, the <*language*> part string must be added into CUBRID configuration file **$CUBRID/conf/cubrid_locales.txt**. Note that the order in this file is the order of generating (compiling) locale library and loading locales at startup.

The **make_locale** script must be used to compile the new added locale and add its data into the CUBRID locales library (locale in **$CUBRID/lib/**).

The LDML file is expected in UTF-8 encoding, and it is not possible to add more than one locale into the same LDML file.

Adding a new locale in LDML file requires:

- to specify calendar information (CUBRID date formats, name of months and week days in various forms, names for AM/PM day periods). CUBRID supports only Gregorian calendar (generic LDML specifies other calendar types which are not supported by CUBRID).
- to specify number settings (digit grouping symbols)
- providing an alphabet (set of rules for how letters are upper-cased and lower-cased)
  - optionally, some collations can be added
  - also optionally, console conversion rules for Windows CSQL application can be defined

LDML Calendar Information

- The first part consists in providing default CUBRID formats for **DATE**, **DATETIME**, **TIME**, **TIMESTAMP**, **DATETIME WITH TIME ZONE** and **TIMESTAMP WITH TIME ZONE** data type conversion to/from string. This formats are used by functions `TO_DATE()`, `TO_TIME()`, `TO_DATETIME()`, `TO_TIMESTAMP()`, `TO_CHAR()`, `TO_DATETIME_TZ()`, `TO_TIMESTAMP_TZ()`. The formats elements allowed depend on data type and are the ones used for `TO_CHAR()` function (Date/Time Format 1). Only ASCII characters are allowed in the format strings. The allowed size are 30 bytes (characters) for **DATE** and **TIME** formats, 48 characters for **DATETIME** and **TIMESTAMP** formats and 70 characters for **DATETIME WITH TIME ZONE** and **TIMESTAMP WITH TIME ZONE**.
- The <months> requires to specify the names for months in both long form and abbreviated form. The allowed size are 15 (or 60 bytes) for abbreviated form and 25 characters (or 100 bytes) for normal form.

- The <days> requires week day names in both long and abbreviated form. The allowed size are 10 characters (or 40 bytes) for abbreviated form and 15 characters (or 60 bytes) for full day name.
    - The <dayperiods> sub-tree requires to define the string for AM/PM format variants (according to type attribute). The allowed size is 10 characters (or 40 bytes).

The months and week-days names (in both long and abbreviated form) must be specified in Camel case format (first letter uppercase, the rest in lower case). CUBRID checks only the maximum allowed size in bytes; the size in characters is computed only for full-width UTF-8 characters (4 bytes), so it would be possible to set a month name having 100 ASCII-only characters (the 25 characters limit is when each character from month name is encoded on 4 bytes in UTF-8).

LDML Numbers information

- The <symbols> tag defines the characters used as symbols for splitting decimal part from integer part in numbers and for grouping the digits. CUBRID expects only ASCII characters for these symbols. Empty of space character is not allowed. CUBRID performs grouping for 3 digits.

LDML Alphabet

These allow to define casing rules for alphabet of the locale. The 'CUBRIDAlphabetMode' attribute defines the primary source of data for characters. Normally, this should be set to "UNICODEDATAFILE", values which instructs CUBRID to use the Unicode data file (**$CUBRID/locales/data/unicodedata.txt**).

This file must not be modified, any customization on certain characters should be done in LDML file. If such value is configured, all Unicode characters up to codepoint 65535 are loaded with casing information. The other allowed value for *CUBRIDAlphabetMode* is "ASCII" which will lead to only ASCII character can be lower case, uppercase or case-insensitive compare in matching functions.

This does not affect CUBRID's ability to support all UTF-8 4 bytes encoded Unicode characters, it just limits the casing ability for characters not included.

The casing rules are optional and apply on top of the primary source of character information (UNICODEDATAFILE or ASCII).

CUBRID allows to define upper casing rules (<u> tag) and lower casing rules (<l> tag).Each of upper and lower casing rules set consists for pairs of source-destination (<s> = source, <d> = destination). For instance, the following defines a rule that each character "A" is lower cased to "aa" (two character "a").

```
<l>
    <s>A</s>
    <d>aa</d>
</l>
```

LDML Console Conversion

In Windows, the console does not support UTF-8 encoding, so CUBRID allows to translate characters from their UTF-8 encoding to a desired encoding. After configuring console conversion for a locale, the user must set prior to starting CSQL application the codepage of the console using 'chcp' command (the codepage argument must match the 'windows_codepage' attribute in LDML). Conversion will work bidirectionally (input and output in CSQL), but is only limited to Unicode characters which can be converted in the configured codepage.

The <consoleconversion> element is optional and allows to instruct CSQL how to print (in which character encoding) the text in interactive command. The 'type' attribute defines the conversion scheme. The allowed values are:

- ISO: is a generic scheme in which the destination codepage is a single byte charset
- ISO88591: is a predefined single byte scheme for ISO-8859-1 charset (the 'file' attribute is not required, is ignored)
- ISO88599: is a predefined single byte scheme for ISO-8859-9 charset (also the 'file' attribute is not required)
- DBCS: Double Byte Code-Set; it is a generic scheme in which the destination codepage is a double byte charset

The 'windows_codepage' is the value for Windows codepage which CUBRID automatically activates console conversion. The 'linux_charset' is corresponding value for charset part in **LANG** environment variable from UNIX system. It is recommended to use native CUBRID charset in Linux console.

The 'file' attribute is required only for "ISO" and "DBCS" values of 'type' attribute and is the file containing the translation information (**$CUBRID/locales/data/codepages/**).

LDML Collation

Configuring a collation is the most complex task for adding LDML locale in CUBRID. Only collation having UTF-8 codeset can be configured. CUBRID allows to configure most constructs specified by UCA - Unicode Collation Algorithm (http://www.unicode.org/reports/tr10/) including contractions and expansions, but the properties for the collation are mostly controlled via the 'settings' attribute.

A LDML file can contain multiple collations. Collations can be included from external file using the 'include' tag. The 'validSubLocales' attribute of 'collations' tag is a filter allowing to control locale compilation when external collations (from external files) are included. Its values can be either a list of locales or "*" in which case the collations in sub-tree are added in all locales from which the file is included.

One collation is defined using the 'collation' tag and its sub-tree. The 'type' attribute indicates the name for the collation as it will be added in CUBRID. The 'settings' tag defines the properties of the collation:

- 'id' is the (internal) numeric identifier used by CUBRID. It is integer value in range (32 - 255) and is optional, but is strongly recommended that an explicit unassigned values is set. Please see Collation Naming Rules.
- 'strength' is a measure of how strings compare. See Collation Properties. The allowed values are :
  - "quaternary": different graphic symbols of the same character compare differently, but different Unicode codepoints may compare equal.
  - "tertiary": graphic symbols of the same character are equal, case-sensitive collation.
  - "secondary": case insensitive collation, characters with accents compare different
  - "primary": accents are ignored, all characters compare as the base character.
- 'caseLevel': special setting to enable case sensitive compare for collations having strength < tertiary. Valid values are "on" or "off".
- 'caseFirst': order of casing. Valid values are "lower", "upper" and "off". The "upper" values means uppercase letters are ordered before the corresponding lower case letter.
- 'CUBRIDMaxWeights': it is the number of codepoints (or last codepoint + 1) which are customized in the collation. Maximum value is 65536. Increasing this value increases the size of collation data.
- 'DUCETContractions': valid values are "use" or "ignore". When "use" - enable CUBRID to use in the collation the contractions defined by DUCET file ($CUBRID/locales/data/ducet.txt) or ignoring them.
- 'TailoringContractions': same as previous but refers to the contractions defined or derived from explicit collation rules. Enabling contractions leads to a more complex collation (slower string compares).
- 'CUBRIDExpansions': allowed values are "use" or "ignore" (default) and refers to usage of collation expansions from both the DUCET file and tailoring rules; This has the most influence on collation properties. Enabling it will result in a compare with multiple passes (up to

collation strength) when comparing strings. Also it greatly increases collation data, with the benefit of obtaining a more "natural" sort order. See Expansion.

- 'backwards': "on" or "off": used to obtain "french" order by performing an end-to-start compare on secondary level (for accents). It has effect only when 'CUBRIDExpansions' are enabled.
- 'MatchContractionBoundary': "true" or "false". This is used in collation having expansions and contractions to configure behavior at string matching when a contraction is found.

The main data for a collation is loaded from the DUCET file. After this step, the collation may be customized using "tailoring rules". These are the "<rules>" (LDML) and "<cubridrules>" (CUBRID specific).

The 'cubridrules' tag is optional and can be used to explicitly set weight values for a codepoint or a range of codepoints. The cubridrules apply after loading the primary collation data from DUCET file and before applying the UCA rules (from '<rules>' tag). Each of these rule is enclosed in '<set>' tag. If the rule refers to only one Unicode codepoint, then a '<scp>' tag is provided which contains the hexadecimal value of codepoint.

All available CUBRID collations contain this cubrid-rule:

```
<cubridrules>
    <set>
        <scp>20</scp>
        <w>[0.0.0.0]</w>
    </set>
</cubridrules>
```

This rule says that weight values (UCA defines four weight values per collation element) of the codepoints starting with 20 (which is ASCII space character) are all set to zero. Since there is no '<ecp>' tag, the only codepoint affected is 20. In CUBRID, space character compares as zero. The allowed tags inside of a '<set>' rule are:

- '<cp>': rule to set the weights for single codepoint.
- '<ch>': rule to set the weights for single character. Similar to previous one, but instead of codepoint it expects a Unicode character (in UTF-8 encoding).
- '<scp>': rule to set the weights for a range of codepoints. This is the starting codepoint.
- '<sch>': rule to set the weights for a range of characters. This is the starting character. In this context, the order of characters is given by their Unicode codepoints.
- '<ecp>': end codepoint for a range rule.
- '<ech>': end character for a range rule.

- '<w>': weight values to set (single value). The weight values are expected in hexadecimal. Each collation element has four values which are delimited by point and enclosed by square brackets([]). There can be up to 10 collation elements.

- '<wr>': starting weight values to set for a range. Optionally, there is 'step' attribute of this tag, which sets the increasing step after each codepoint. By default the step is [0001.0000.0000.0000], which means that after setting the first weight values for the starting codepoint, one value is added to primary level weight and set to the next codepoint in range, and the process is repeated until end codepoint.

Examples:

```
<cubridrules>
    <set>                                <!-- Rule 1 -->
        <scp>0</scp>
        <ecp>20</ecp>
        <w>[0.0.0.0]</w>
    </set>

    <set>                                <!-- Rule 2 -->
        <scp>30</scp>
        <ecp>39</ecp>
        <wr step="[1.0.0.0][1.0.0.0]">[30.0.0.0][30.0.0.0]</
wr>
    </set>

</cubridrules>
```

The Rule 1, sets for codepoints ranging from 0 to 20 (including) the weight values 0. The Rule 2, sets for codepoints ranging from 30 to 39 (which are the digits), a set of two collation elements with increasing weights; In this example, codepoint 39 (character "9") will have the weights with two collation elements [39.0.0.0][39.0.0.0].

The '<rules>' tag is also optional but is according to LDML and UCA specifications. The meanings of sub-ordinates tags are :

- '<reset>': anchor collation element. It defines the reference to which subsequent rules (up to next *<reset>*) are tailored. It can be a single characters or multiple characters in which case is either a contraction or an expansion. By default, all the tailoring rules after the anchor are sort "after" (element from first rule is after the anchor, element from second rule sorts after element in first rule); if the optional attribute "before" is present, then only the first rule after the <reset> sorts before the anchor, while the second and the following rules resumes the normal "after" sort (element in second rule sorts after element in first rule).

- '<p>': the character comes after (or before, if the anchor had the *before* attribute) the previously tailored one at primary level.

- '<s>': the character comes after (or before, if the anchor had the *before* attribute) the previously tailored one at secondary level.
- '<t>': the character comes after (or before, if the anchor had the *before* attribute) the previously tailored one at tertiary level.
- '<i>': the character sorts identically to previous one
- '<pc>', '<sc>', '<tc>', '<ic>': same as '<p>', '<s>', '<t>', '<i>' but applies to a range of characters
- '<x>': specifies the expansion character
- '<extend>': specifies the second character of expansion.
- '<context>': specifies the context in which a rule applies. A variant to specify contractions and expansions.

For more information on UCA tailoring with LDML rules see http://www.unicode.org/reports/tr35/tr35-collation.html.

Footnotes

[1] (1,2)
glyph: an element for the shape of a character; a graphic symbol which indicates a shape or a form for a character. Because a glyph specifies the shape which is shown, several glyphs about one character can exist.

Transaction and Lock

This chapter covers issues relating to concurrency and restore, as well as how to commit or rollback transactions.

In multi-user environment, controlling access and update is essential to protect database integrity and ensure that a user's transaction will have accurate and consistent data. Without appropriate control, data could be updated incorrectly in the wrong order.

To control parallel operations on the same data, data must be locked during transaction, and unacceptable access to the data by another transaction must be blocked until the end of the transaction. In addition, any updates to a certain class must not be seen by other users before they are committed. If updates are not committed, all queries entered after the last commit or rollback of the update can be invalidated.

CUBRID uses Multiversion Concurrency Control to optimize data access. Data being modified is not overwritten, old data is instead marked as deleted and a newer version is added elsewhere. Other transactions can continue to read the old version without locking it. Each transaction sees

its own snapshot of the database that is defined at the start of transaction or statement execution. Furthermore, the snapshot is only affected by own changes and remains consistent throughout the execution.

All examples introduced here were executed by csql.

Database Transaction

A database transaction groups CUBRID queries into a unit of consistency (for ensuring valid results in multi-user environment) and restore (for making the results of committed transactions permanent and ensuring that the aborted transactions are canceled in the database despite any failure, such as system failure). A transaction is a collection of one or more queries that access and update the database.

CUBRID allows multiple users to access the database simultaneously and manages accesses and updates to prevent inconsistency of the database. For example, if data is updated by one user, the changes made by this transaction are not seen to other users or the database until the updates are committed. This principle is important because the transaction can be rolled back without being committed.

You can delay permanent updates to the database until you are confident of the transaction result. Also, you can remove (**ROLLBACK**) all updates in the database if an unsatisfactory result or failure occurs in the application or computer system during the transaction. The end of the transaction is determined by the **COMMIT WORK** or **ROLLBACK WORK** statement. The **COMMIT WORK** statement makes all updates permanent while the **ROLLBACK WORK** statement cancels all updates entered in the transaction.

Transaction Commit

Updates that occurred in the database are not permanently stored until the **COMMIT WORK** statement is executed. “Permanently stored” means that storing the updates in the disk is completed; The **WORK** keyword can be omitted. In addition, other users of the database cannot see the updates until they are permanently applied. For example, when a new row is inserted into a class, only the user who inserted the row can access it until the database transaction is committed.

All locks obtained by the transaction are released after the transaction is committed.

```
COMMIT [WORK];
```

```
-- ;autocommit off
-- AUTOCOMMIT IS OFF
```

```
SELECT name, seats
FROM stadium WHERE code IN (30138, 30139, 30140);
```

```
    name                                    seats
=============================================
    'Athens Olympic Tennis Centre'      3200
    'Goudi Olympic Hall'                5000
    'Vouliagmeni Olympic Centre'        3400
```

The following **UPDATE** statement changes three column values of seats from the stadium. To compare the results, check the current values and names before the update is done. Since csql runs in an autocommit mode by default, the following example is executed after setting the autocommit mode to off.

```
UPDATE stadium
SET seats = seats + 1000
WHERE code IN (30138, 30139, 30140);

SELECT name, seats FROM stadium WHERE code in (30138, 30139, 30140);
```

```
    name                                    seats
=============================================
    'Athens Olympic Tennis Centre'      4200
    'Goudi Olympic Hall'                6000
    'Vouliagmeni Olympic Centre'        4400
```

If the update is properly done, the changes can be permanently fixed. In this time, use the **COMMIT WORK** as below:

```
COMMIT [WORK];
```

Note

In CUBRID, an autocommit mode is set by default for transaction management.

An autocommit mode is a mode that commits or rolls back all SQL statements. The transaction is committed automatically if the SQL is executed successfully, or is rolled back automatically if an error occurs. Such autocommit modes are supported in any interfaces.

In CCI, PHP, ODBC and OLE DB interfaces, you can configure autocommit mode by using **CCI_DEFAULT_AUTOCOMMIT** upon startup of an application. If configuration on broker parameter

is omitted, the default value is set to **ON**. To change autocommit mode, use the following functions by interface: **cci_set_autocommit** () for CCI interface and **cubrid_set_autocommit** () for PHP interface.

For session command (**;AUtocommit**) which enables autocommit configuration in CSQL Interpreter, see Session Commands.

Transaction Rollback

The **ROLLBACK WORK** statement removes all updates to the database since the last transaction. The **WORK** keyword can be omitted. By using this statement, you can cancel incorrect or unnecessary updates before they are permanently applied to the database. All locks obtained during the transaction are released.

```
ROLLBACK [WORK];
```

The following example shows two commands that modify the definition and the row of the same table.

```
-- csql> ;autocommit off
CREATE TABLE code2 (
    s_name  CHAR(1),
    f_name  VARCHAR(10)
);
COMMIT;

ALTER TABLE code2 DROP s_name;
INSERT INTO code2 (s_name, f_name) VALUES ('D','Diamond');
```

```
ERROR: s_name is not defined.
```

The **INSERT** statement fails because the *s_name* column has been dropped in the definition of *code*. The data intended to be entered to the *code* table is correct, but the *s_name* column is wrongly removed. At this point, you can use the **ROLLBACK WORK** statement to restore the original definition of the *code* table.

```
ROLLBACK WORK;
```

Later, remove the *s_name* column by entering the **ALTER TABLE** again and modify the **INSERT** statement. The **INSERT** command must be entered again because the transaction has been aborted. If the database update has been done as intended, commit the transaction to make the changes permanent.

```
ALTER TABLE code2 DROP s_name;
INSERT INTO code2 (f_name) VALUES ('Diamond');

COMMIT WORK;
```

Savepoint and Partial Rollback

A savepoint is established during the transaction so that database changes made by the transaction are rolled back to the specified savepoint. Such operation is called a partial rollback. In a partial rollback, database operations (insert, update, delete, etc.) after the savepoint are rolled back, and transaction operations before it are not rolled back. The transaction can proceed with other operations after the partial rollback is executed. Or the transaction can be terminated with the **COMMIT WORK** or **ROLLBACK WORK** statement. Note that the savepoint does not commit the changes made by the transaction.

A savepoint can be created at a certain point of the transaction, and multiple savepoints can be used for a certain point. If a partial rollback is executed to a savepoint before the specified savepoint or the transaction is terminated with the **COMMIT WORK** or **ROLLBACK WORK** statement, the specified savepoint is removed. The partial rollback after the specified savepoint can be performed multiple times.

Savepoints are useful because intermediate steps can be created and named to control long and complicated utilities. For example, if you use a savepoint during the update operation, you don't need to perform all statements again when you made a mistake.

```
SAVEPOINT <mark>;

<mark> :
  - a SQL identifier
  - a host variable (starting with :)
```

If you make *mark* all the same value when you specify multiple savepoints in a single transaction, only the latest savepoint appears in the partial rollback. The previous savepoints remain hidden until the rollback to the latest savepoint is performed and then appears when the latest savepoint disappears after being used.

```
ROLLBACK [WORK] [TO [SAVEPOINT] <mark>];

<mark>:
  - a SQL identifier
  - a host variable (starting with :)
```

Previously, the **ROLLBACK WORK** statement canceled all database changes added since the latest transaction. The **ROLLBACK WORK** statement is also used for the partial rollback that rolls back the transaction changes after the specified savepoint.

If *mark* value is not given, the transaction terminates canceling all changes including all savepoints created in the transaction. If *mark* value is given, changes after the specified savepoint are canceled and the ones before it are remained.

The below example shows how to rollback a part of a transaction. Firstly, specify savepoints *SP1*, *SP2*.

```
-- csql> ;autocommit off

CREATE TABLE athlete2 (name VARCHAR(40), gender CHAR(1), nation_code
CHAR(3), event VARCHAR(30));
INSERT INTO athlete2(name, gender, nation_code, event)
VALUES ('Lim Kye-Sook', 'W', 'KOR', 'Hockey');
SAVEPOINT SP1;

SELECT * from athlete2;
INSERT INTO athlete2(name, gender, nation_code, event)
VALUES ('Lim Jin-Suk', 'M', 'KOR', 'Handball');

SELECT * FROM athlete2;
SAVEPOINT SP2;

RENAME TABLE athlete2 AS sportsman;
SELECT * FROM sportsman;
ROLLBACK WORK TO SP2;
```

In the example above, the name change of the *athlete2* table is rolled back by the partial rollback. The following example shows how to execute the query with the original name and examining the result.

```
SELECT * FROM athlete2;
DELETE FROM athlete2 WHERE name = 'Lim Jin-Suk';
SELECT * FROM athlete2;
ROLLBACK WORK TO SP2;
```

In the example above, deleting ‘Lim Jin-Suk’ is discarded by rollback work to *SP2* command. The following example shows how to roll back to *SP1*.

```
SELECT * FROM athlete2;
ROLLBACK WORK TO SP1;
SELECT * FROM athlete2;
COMMIT WORK;
```

Cursor Holdability

Cursor holdability is when an application holds the record set of the **SELECT** query result to fetch the next record even after performing an explicit commit or an automatic commit. In each application, cursor holdability can be specified as Connection level or Statement level. If it is not specified, the cursor is held by default. Therefore, **HOLD_CURSORS_OVER_COMMIT** is the default setting.

The following code shows how to set cursor holdability in JDBC:

```
// set cursor holdability at the connection level
conn.setHoldability(ResultSet.HOLD_CURSORS_OVER_COMMIT);

// set cursor holdability at the statement level which can override
the connection
PreparedStatement pStmt = conn.prepareStatement(sql,
                                    ResultSet.TYPE_SCROLL_SENSITIVE,
                                    ResultSet.CONCUR_UPDATABLE,

ResultSet.HOLD_CURSORS_OVER_COMMIT);
```

To set cursor holdability to close the cursor when a transaction is committed, set **ResultSet.CLOSE_CURSORS_AT_COMMIT**, instead of **ResultSet.HOLD_CURSORS_OVER_COMMIT**, in the above example.

The default setting for applications that were developed based on CCI is to hold the cursor. If the cursor is set to 'not to hold a cursor' at connection level and you want to hold the cursor, define the **CCI_PREPARE_HOLDABLE** flag while preparing a query. The default setting for CCI drivers (PHP, PDO, ODBC, OLE DB, ADO.NET, Perl, Python, Ruby) is to hold the cursor. To check whether a driver supports the cursor holdability setting, refer to the **PREPARE** function of the driver.

Note

- Note that versions lower than CUBRID 9.0 do not support cursor holdability. The default setting of those versions is to close all cursors at commit.
- CUBRID currently does not support ResultSet.HOLD_CURSORS_OVER_COMMIT in java.sql.XAConnection interface.

Cursor-related Operation at Transaction Commit

When a transaction is committed, all statements and result sets that are closed are released even if you have set cursor holdability. After that, when the result sets are used for another transaction, some or all of the result sets should be closed as required.

When a transaction is rolled back, all result sets are closed. This means that all result sets held in the previous transaction are closed because you have set cursor holdability.

```
rs1 = stmt.executeQuery(sql1);
conn.commit();
rs2 = stmt.executeQuery(sql2);
conn.rollback();  // result sets rs1 and rs2 are closed and it will
not be available to use them.
```

When the Result Sets are Closed

The result sets that hold the cursor are closed in the following cases:

- Driver closes the result set, i.e. rs.close()
- Driver closes the statement, i.e. stmt.close()
- Driver disconnects the connection.
- Transaction aborts, for instance, application explicitly calls rollback(), auto rollback due to a query failure under autocommit mode.

Relationship with CAS

When the connection between an application and the CAS is closed, all result sets are automatically closed even if you have set cursor holdability in the application. The setting value of **KEEP_CONNECTION**, the broker parameter, affects cursor holdability of the result set.

- KEEP_CONNECTION = ON: Cursor holdability is not affected.
- KEEP_CONNECTION = AUTO: The CAS cannot be restarted while the result set with cursor holdability is open.

Warning

Usage of memory will increase in the status of result set opened. Thus, you should close the result set after completion.

Note

Note that CUBRID versions lower than 9.0 do not support cursor holdability and the cursor is automatically closed when a transaction is committed. Therefore, the recordset of the **SELECT** query result is not kept.

Database Concurrency

If there are multiple users with read and write authorization to a database, possibility exists that more than one user will access the database simultaneously. Controlling access and update in multi-user environment is essential to protect database integrity and ensure that users and transactions should have accurate and consistent data. Without appropriate control, data could be updated incorrectly in the wrong order.

The transaction must ensure database concurrency, and each transaction must guarantee appropriate results. When multiple transactions are being executed at once, an event in transaction *T1* should not affect an event in transaction *T2*. This means isolation. Transaction isolation level is the degree to which a transaction is separated from all other concurrent transactions. The higher isolation level means the lower interference from other transactions. The lower isolation level means the higher the concurrency. A database determines whether which lock is applied to tables and records based on these isolation levels. Therefore, can control the level of consistency and concurrency specific to a service by setting appropriate isolation level.

The read operations that allow interference between transactions with isolation levels are as follows:

- **Dirty read** : A transaction *T2* can read *D'* before a transaction *T1* updates data *D* to *D'* and commits it.
- **Non-repeatable read** : A transaction *T1* can read changed value, if a transaction *T2* updates or deletes data and commits while data is retrieved in the transaction *T1* multiple times.
- **Phantom read** : A transaction *T1* can read *E*, if a transaction *T2* inserts new record *E* and commits while data is retrieved in the transaction *T1* multiple times.

Based on these interferences, the SQL standard defines four levels of transaction isolation:

- **READ UNCOMMITTED** allows dirty read, unrepeatable read and phantom read.
- **READ COMMITTED** does not allow dirty read but allows unrepeatable read and phantom read.
- **REPEATABLE READ** does not allow dirty read and unrepeatable read but allows phantom read.
- **SERIALIZABLE** does not allow interrupts between transactions when doing read operation.

Isolation Levels Provided by CUBRID

On the below table, the number wrapped with parenthesis right after the isolation level name is a number which can be used instead of the isolation level name when setting an isolation level.

You can set an isolation level by using the SET TRANSACTION ISOLATION LEVEL statement or system parameters provided by CUBRID. For details, see Concurrency/Lock-Related Parameters.

(O: YES, X: NO)

CUBRID Isolation Level (isolation_level)	DIRTY READ	UNREPEATABLE READ	PHANTOM READ	Schema Changes of the Table Being Retrieved
SERIALIZABLE Isolation Level (6)	X	X	X	X
REPEATABLE READ Isolation Level (5)	X	X	X	X
READ COMMITTED Isolation Level (4)	X	O	O	X

The default value of CUBRID isolation level is READ COMMITTED Isolation Level.

Multiversion Concurrency Control

CUBRID previous versions managed isolation levels using the well known two phase locking protocol. In this protocol, a transaction obtains a shared lock before it reads an object, and an exclusive lock before it updates the object so that conflicting operations are not executed simultaneously. If transaction *T1* requires a lock, the system checks if the requested lock conflicts with the existing one. If it does, transaction *T1* enters a standby state and delays the lock. If another transaction *T2* releases the lock, transaction *T1* resumes and obtains it. Once the lock is released, the transaction do not require any new locks.

CUBRID 10.0 replaced the two phase locking protocol with a Multiversion Concurrency Control (MVCC) protocol. Unlike two phase locking, MVCC does not block readers to access objects being modified by concurrent transactions. MVCC duplicates rows, creating multiple versions on each update. Never blocking readers is important for workloads involving mostly value reads from the

database, commonly used in read-world scenarios. Exclusive locks are still required before updating objects.

MVCC is also known for providing point in time consistent view of the database and for being particularly adept at implementing true **snapshot isolation** with less performance costs than other methods of concurrency.

Versioning, visibility and snapshot

MVCC maintains multiple versions for each database row. Each version is marked by its inserter and deleter with MVCCID's - unique identifiers for writer transactions. These markers are useful to identify the author of a change and to place the change on a timeline.

When a transaction *T1* inserts a new row, it creates its first version and sets its unique identifier *MVCCID1* as insert id. The MVCCID is stored as metadata in record header:

OTHER META-DATA	MVCCID1	RECORD DATA

Until *T1* commits, other transactions should not see this row. The MVCCID helps identifying the authors of database changes and place them on a time line, so others can know if the change is valid or not. In this case, anyone checking this row find the *MVCCID1*, find out that the owner is still active, hence the row must be (still) invisible.

After *T1* commits, a new transaction *T2* finds the row and decides to remove it. *T2* does not remove this version, allowing others to access it, instead it gets an exclusive lock, to prevent others from changing it, and marks the version as deleted. It adds another MVCCID so others can identify the deleter:

OTHER META-DATA	MVCCID1	MVCCID2	RECORD DATA

If *T2* decides instead to update one of the record values, it must update the row to a new version and store the old version in log. The new row consists of new data, transaction MVCCID as insert MVCCID and the address of log entry storing previous version. The row representations looks like this:

HEAP file contains a single row identified by an OID:

OTHER META-DATA	MVCCID_INS1	PREV_VERSION_LSA1	RECORD DATA

LOG file has a chain of log entries, the undo part of each log entry contains the original heap record before modification:

LOG ENTRY META-DATA	OTHER META-DATA		MVCCID_INS2	PREV_VERSION_LSA2	RECORD DATA
LOG ENTRY META-DATA	OTHER META-DATA		MVCCID_INS3	NULL	RECORD DATA

Other transactions may need to walk the log chain of previous version LSA of multiple log record until one record satisfies the visibility condition, determined by the values of insert and delete MVCCID of each record.

Note

- Previous version used the heap (another OID) to store the old and new version of the updated rows. In fact, old version was the the row which remained unchanged, which was appended with and OID link to the new version. Both new version and old version were located in the heap.

Currently, only *T2* can see the updated row, while other transactions will access the row version contained on the log page and accessible through the LSA obtained from heap row. The property of a version to be seen or not to be seen by running transactions is called **visibility**. The visibility property is relative to each transaction, some can consider it true, whereas others can consider it false.

A transaction *T3* that starts after *T2* executes row update, but before *T2* commits, will not be able to see its new version, not even after *T2* commits. The visibility of one version towards *T3* depends on the state of its inserter and deleter when *T3* started and preserves its status for the lifetime of *T3*.

As a matter of fact, the visibility of all versions in database towards on transaction does not depend on the changes that occur after transaction is started. Moreover, any new version added is also ignored. Consequently, the set of all visible versions in the database remains unchanged and form the snapshot of the transaction. Hence, **snapshot isolation** is provided by MVCC and it is a guarantee that all read queries made in a transaction see a consistent view of the database.

In CUBRID 10.0, **snapshot** is a filter of all invalid MVCCID's. An MVCCID is invalid if it is not committed before the snapshot is taken. To avoid updating the snapshot filter whenever a new transaction starts, the snapshot is defined using two border MVCCID's: the lowest active MVCCID and the highest committed MVCCID. Only a list of active MVCCID values between the border is saved. Any transaction starting after snapshot is guaranteed to have an MVCCID bigger than

highest committed and is automatically considered invalid. Any MVCCID below lowest active must be committed and is automatically considered valid.

The snapshot filter algorithm that decides a version visibility queries the MVCCID markers used for insert and delete. The snapshot starts by checking the *last version* stored in heap and, based on result, it can either fetch version from heap, fetch older version from log or can ignore row:

Insert MVCCID	Previous version LSA	Delete MVCCID	Snapshot test result
Not visible	NULL	None or not visible	Version is too *new* and is not visible Row has no previous version, so it is ignored
	LSA	None or not visible	Version is too *new* and is not visible Row has previous version and snapshot must check it
None or visible	LSA or NULL	None or not visible	Version is visible and its data is fetched It does not matter if row has previous versions
		Visible	Version is too old, was deleted and is not visible It does not matter if row has previous versions

If version is too new, but it has a previous version stored in log, the same checks are repeated on previous version. The checks stop when no previous versions are found (the entire row chain is too new for this transaction), or when a visible version is found.

Let's see how snapshot works (**REPEATABLE READ** isolation will be used to keep same snapshot during entire transaction):

Example 1: Inserting a new row

session 1	session 2

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> set transaction
isolation level REPEATABLE
READ;

Isolation level set to:
REPEATABLE READ
```

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> set transaction
isolation level REPEATABLE
READ;

Isolation level set to:
REPEATABLE READ
```

```
csql> CREATE TABLE
tbl(host_year integer,
nation_code char(3));
csql> COMMIT WORK;
```

```
-- insert a row without
committing
csql> INSERT INTO tbl VALUES
(2008, 'AUS');

-- current transaction sees
its own changes
csql> SELECT * FROM tbl;

    host_year  nation_code
=============================
====
         2008  'AUS'
```

```
-- this snapshot should not
see uncommitted row
csql> SELECT * FROM tbl;
```

session 1	session 2
	There **are no** results.
csql> **COMMIT WORK**;	
	-- even though inserter did commit, this snapshot still can't see the row csql> **SELECT * FROM** tbl; There **are no** results. -- commit to start a new transaction with a new snapshot csql> **COMMIT WORK**; -- the new snapshot should see committed row csql> **SELECT * FROM** tbl; host_year nation_code ========== 2008 'AUS'

Example 2: Deleting a row

session 1	session 2
csql> ;autocommit **off** AUTOCOMMIT **IS OFF** csql> **set transaction isolation level REPEATABLE**	csql> ;autocommit **off** AUTOCOMMIT **IS OFF** csql> **set transaction isolation level REPEATABLE**

session 1	session 2

```
READ;

Isolation level set to:
REPEATABLE READ
```

```
READ;

Isolation level set to:
REPEATABLE READ
```

```
csql> CREATE TABLE
tbl(host_year integer,
nation_code char(3));
csql> INSERT INTO tbl VALUES
(2008, 'AUS');
csql> COMMIT WORK;
```

```
-- delete the row without
committing
csql> DELETE FROM tbl WHERE
nation_code = 'AUS';

-- this transaction sees its
own changes
csql> SELECT * FROM tbl;

There are no results.
```

```
-- delete was not committed,
so the row is visible to this
snapshot
csql> SELECT * FROM tbl;

    host_year  nation_code
=============================
=====
         2008  'AUS'
```

session 1	session 2

```
csql> COMMIT WORK;
```

```
-- delete was committed, but
the row is still visible to
this snapshot
csql> SELECT * FROM tbl;

    host_year  nation_code
=============================
=====
         2008  'AUS'

-- commit to start a new
transaction with a new
snapshot
csql> COMMIT WORK;

-- the new snapshot can no
longer see deleted row
csql> SELECT * FROM tbl;

There are no results.
```

Example 3: Updating a row

session 1	session 2

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> set transaction
isolation level REPEATABLE
READ;
```

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> set transaction
isolation level REPEATABLE
READ;
```

session 1	session 2

```
Isolation level set to:
REPEATABLE READ
```

```
Isolation level set to:
REPEATABLE READ
```

```
csql> CREATE TABLE
tbl(host_year integer,
nation_code char(3));
csql> INSERT INTO tbl VALUES
(2008, 'AUS');
csql> COMMIT WORK;
```

```
-- delete the row without
committing
csql> UPDATE tbl SET host_year
= 2012 WHERE nation_code =
'AUS';

-- this transaction sees new
version, host_year = 2012
csql> SELECT * FROM tbl;

    host_year  nation_code
==============================
====
         2012  'AUS'
```

```
-- update was not committed,
so this snapshot sees old
version
csql> SELECT * FROM tbl;

    host_year  nation_code
==============================
=====
         2008  'AUS'
```

session 1	session 2

```
csql> COMMIT WORK;
```

```
-- update was committed, but
this snapshot still sees old
version
csql> SELECT * FROM tbl;

    host_year  nation_code
=============================
=====
         2008  'AUS'

-- commit to start a new
transaction with a new
snapshot
csql> COMMIT WORK;

-- the new snapshot can see
new version, host_year = 2012
csql> SELECT * FROM tbl;

    host_year  nation_code
=============================
=====
         2012  'AUS'
```

Example 4: Different versions can be visible to different transactions

session 1	session 2	session 3

```
csql> ;autocommit
off

AUTOCOMMIT IS OFF

csql> set
transaction
```

```
csql> ;autocommit
off

AUTOCOMMIT IS OFF

csql> set
transaction
```

```
csql> ;autocommit
off

AUTOCOMMIT IS OFF

csql> set
transaction
```

session 1	session 2	session 3
isolation level REPEATABLE READ; Isolation level set to: REPEATABLE READ	isolation level REPEATABLE READ; Isolation level set to: REPEATABLE READ	isolation level REPEATABLE READ; Isolation level set to: REPEATABLE READ

```
csql> CREATE TABLE
tbl(host_year
integer,
nation_code
char(3));
csql> INSERT INTO
tbl VALUES (2008,
'AUS');
csql> COMMIT WORK;
```

```
-- update row
csql> UPDATE tbl
SET host_year =
2012 WHERE
nation_code =
'AUS';

csql> SELECT *
FROM tbl;

    host_year
nation_code
===================
================
         2012
'AUS'
```

```
csql> SELECT *
FROM tbl;

    host_year
nation_code
===================
================
         2008
'AUS'
```

```
csql> COMMIT WORK;
```

session 1	session 2	session 3

```
csql> UPDATE tbl
SET host_year =
2016 WHERE
nation_code =
'AUS';

csql> SELECT *
FROM tbl;

    host_year
nation_code
==========
==========
         2016
'AUS'
```

```
csql> SELECT *
FROM tbl;

    host_year
nation_code
==========
==========
         2008
'AUS'
```

```
csql> SELECT *
FROM tbl;

    host_year
nation_code
==========
==========
         2012
'AUS'
```

VACUUM

Creating new versions for each update and keeping old versions on delete could lead to unlimited database size growth, definitely a major issue for a database. Therefore, a clean up system is necessary, to remove obsolete data and reclaim the occupied space for reuse.

Each row version goes through same stages:

1. newly inserted, not committed, visible only to its inserter.
2. committed, invisible to preceding transactions, visible to future transactions.
3. deleted, not committed, visible to other transactions, invisible to the deleter.
4. committed, still visible to preceding transactions, invisible to future transactions.
5. invisible to all active transactions.
6. removed from database.

The role of the clean up system is to get versions from stage 5 to 6. This system is called **VACUUM** in CUBRID.

VACUUM system was developed with the guidance of three principles:

- **VACUUM** must be correct and complete. **VACUUM** should never remove data still visible to some and it should not miss any obsolete data.
- **VACUUM** must be discreet. Since clean-up process changes database content, there may be some interference in the activity of live transactions, but it must be kept to the minimum possible.
- **VACUUM** must be fast and efficient. If **VACUUM** is too slow and if it starts lagging, the database state can deteriorate, thus the overall performance can be affected.

With these principles in mind, **VACUUM** implementation uses existing recovery logging, because:

- The address is kept among recovery data for both heap and index changes. This allows **VACUUM** go directly to target, rather than scanning the database.
- Processing log data rarely interferes with the work of active workers.

Log recovery was adapted to **VACUUM** needs by adding MVCCID information to logged data. **VACUUM** can decide based on MVCCID if the log entry is ready to be processed. MVCCID's that are still visible to active transactions cannot be processed. In due time, each MVCCID becomes old enough and all changes using the MVCCID become invisible.

Each transaction keeps the oldest MVCCID it considers active. The oldest MVCCID considered active by all running transactions is determined by the smallest oldest MVCCID of all transactions. Anything below this value is invisible and **VACUUM** can clean.

VACUUM Parallel Execution

According to the third principle of **VACUUM** it must be fast and it should not fall behind active workers. It is obvious that one thread cannot handle all the **VACUUM** works if system workload is heavy, thus it had to be parallelized.

To achieve parallelization, the log data was split into fixed size blocks. Each block generates one vacuum job, when the time is right (the most recent MVCCID can be vacuumed, which means all logged operations in the block can be vacuumed). Vacuum jobs are picked up by multiple **VACUUM Workers** that clean the database based on relevant log entries found in the log block. The tracking of log blocks and generating vacuum jobs is done by the **VACUUM Master**.

VACUUM Data

Aggregated data on log blocks is stored in vacuum data file. Since the vacuum job generated by an operations occurs later in time, the data must be saved until the job can be executed, and it must also be persistent even if the server crashes. No operation is allowed to leak and not be

vacuumed. If the server crashes, some jobs may be executed twice, which is preferable to not being executed at all.

After a job has been successfully executed, the aggregated data on the processed log block is removed.

Aggregated log block data is not added directly to vacuum data. A latch-free buffer is used to avoid synchronizing active working threads (which generate the log blocks and their aggregated data) with the vacuum system. **VACUUM Master** wakes up periodically, dumps everything in buffer to vacuum data, removes data already process and generates new jobs (if available).

VACUUM jobs

Vacuum job execution steps:

1. **Log pre-fetch**. Vacuum master or workers pre-fetch log pages to be processed by the job.
2. **Repeat for each log record**:
    1. **Read** log record.
    2. **Check dropped file.** If the log record points to dropped file, proceed to next log record.
    3. **Execute index vacuum and collect heap OID's**
        - If log record belongs to index, execute vacuum immediately.
        - If log record belongs to heap, collect OID to be vacuumed later.
1. **Execute heap vacuum** based on collected OID's.
2. **Complete job.** Mark the job as completed in vacuum data.

Several measures were taken to ease log page reading and to optimize vacuum execution.

Tracking dropped files

When a transaction drops a table or an index, it usually locks the affected table(s) and prevents others from accessing it. Opposed to active workers, **VACUUM** Workers are not allowed to use locking system, for two reasons: interference with active workers must be kept to the minimum,

and **VACUUM** system is never supposed to stop as long as it has data to clean. Moreover, **VACUUM** is not allowed to skip any data that needs cleaning. This has two consequences:

1. **VACUUM** doesn't stop from cleaning a file belonging to a dropped table or a dropped index until the dropper commits. Even if a transaction drops a table, its file is not immediately destroyed and it can still be accessed. The actual destruction is postponed until after commit.

2. Before the actual file destruction, **VACUUM** system must be notified. The dropper sends a notification to **VACUUM** system and then waits for the confirmation. **VACUUM** works on very short iterations and it checks for new dropped files frequently, so the dropper doesn't have to wait for a long time.

After a file is dropped, **VACUUM** will ignore all found log entries that belong to the file. The file identifier, paired with an MVCCID that marks the moment of drop, is stored in a persistent file until **VACUUM** decides it is safe to remove it (the decision is based on the smallest MVCCID not yet vacuumed).

Lock Protocol

In the two-phase locking protocol, a transaction obtains a shared lock before it reads an object, and an exclusive lock before it updates the object so that conflicting operations are not executed simultaneously. The MVCC locking protocol, which is now used by CUBRID, does not require shared locks before reading rows (however intent shared lock on table object is still used to read its rows). If transaction *T1* requires a lock, CUBRID checks if the requested lock conflicts with the existing one. If it does, transaction *T1* enters a standby state and delays the lock. If another transaction *T2* releases the lock, transaction *T1* resumes and obtains it. Once the lock is released, the transaction do not acquire any new locks.

Granularity Locking

CUBRID uses a granularity locking protocol to decrease the number of locks. In the granularity locking protocol, a database can be modelled as a hierarchy of lockable units: row lock, table lock and database lock. Coarser locks have more granular locks.

If the locking granularities overlap, effects of a finer granularity are propagated in order to prevent conflicts. That is, if a shared lock is required on an instance of a table, an intent shared lock will be set on the table. If an exclusive lock is required on an instance of a table, an intent exclusive lock will be set on the table. An intent shared lock on a table means that a shared lock can be set on an instance of the table. An intent exclusive lock on a table means that a shared/exclusive lock can be set on an instance of the table. That is, if an intent shared lock on a table is allowed in one transaction, another transaction cannot obtain an exclusive lock on the table (for example, to add a new column). However, the second transaction may obtain a shared lock on the table. If an intent exclusive lock on the table is allowed in one transaction, another transaction

cannot obtain a shared lock on the table (for example, a query on an instance of the tables cannot be executed because it is being changed).

A mechanism called lock escalation is used to limit the number of locks being managed. If a transaction has more than a certain number of locks (a number which can be changed by the **lock_escalation** system parameter), the system begins to require locks at the next higher level of granularity. This escalates the locks to a coarser level of granularity. CUBRID performs lock escalation when no transactions have a higher level of granularity in order to avoid a deadlock caused by lock conversion.

Lock Mode Types And Compatibility

CUBRID determines the lock mode depending on the type of operation to be performed by the transaction, and determines whether or not to share the lock depending on the mode of the lock preoccupied by another transaction. Such decisions concerning the lock are made by the system automatically. Manual assignment by the user is not allowed. To check the lock information of CUBRID, use the **cubrid lockdb** *db_name* command. For details, see lockdb.

- **Shared lock (shared lock, S_LOCK, no longer used with MVCC protocol)**

  This lock is obtained before the read operation is executed on the object.

  It can be obtained by multiple transactions for the same object. At this time, transaction *T2* and *T3* can perform the read operation on the object concurrently, but not the update operation.

  Note

  - Shared locks are rarely used in CUBRID 10.0, because of MVCC. It is still used, mostly in internal database operations, to protect rows or index keys from being modified.

- **Exclusive lock (exclusive lock, X_LOCK)**

  This lock is obtained before the update operation is executed on the object.

  It can only be obtained by one transaction. Transaction *T1* obtains the exclusive lock first before it performs the update operation on a certain object *X*, and does not release it until transaction *T1* is committed even after the update operation is completed. Therefore, transaction *T2* and *T3* cannot perform the read operation as well on *X* before transaction *T1* releases the exclusive lock.

- **Intent lock**

This lock is set inherently in a higher-level object than a certain object to protect the lock on the object of a certain level.

For example, when a shared lock is requested for a certain row, prevent a situation from occurring in which the table is locked by another transaction by setting the intent shared lock as well on the table at the higher level in hierarchy. Therefore, the intent lock is not set on rows at the lowest level, but is set on higher-level objects. The types of intent locks are as follows:

- **Intent shared lock (IS_LOCK)**

  If the intent shared lock is set on the table, which is the higher-level object, as a result of the shared lock set on a certain row, another transaction cannot perform operations such as changing the schema of the table (e.g. adding a column or changing the table name) or updating all rows. However updating some rows or viewing all rows is allowed.

- **Intent exclusive lock (IX_LOCK)**

  If the intent exclusive lock is set on the table, which is the higher-level object, as a result of the exclusive lock set on a certain row, another transaction cannot perform operations such as changing the schema of the table, updating or viewing all rows. However updating some rows is allowed.

- **Shared with intent exclusive lock(SIX_LOCK)**

  This lock is set on the higher-level object inherently to protect the shared lock set on all objects at the lower hierarchical level and the intent exclusive lock on some object at the lower hierarchical level.

  Once the shared intent exclusive lock is set on a table, another transaction cannot change the schema of the table, update all/some rows or view all rows. However, viewing some rows is allowed.

· **Schema Lock**

A schema lock is acquired when executing DDL work.

- **Schema stability lock, SCH_S_LOCK**

  This lock is acquired during compiling a query and it guarantees that the schema which is included in this query is not changed.

- **Schema modification lock, SCH_M_LOCK**

  This lock is acquired during running DDL(**ALTER**/**CREATE**/**DROP**) and it protects that other transactions access the modified schema.

Some DDL operations like **ALTER**, **CREATE INDEX** do not acquire **SCH_M_LOCK** directly. For example, CUBRID operates type checking about filtering expression when you create a filtered

index; during this term, the lock which is kept to the target table is **SCH_S_LOCK** like other type checking operations. The lock is then upgraded to **SIX_LOCK** (other transactions are prevented from modifying target table rows, but they can continue reading them), and finally **SCH_M_LOCK** is requested to change the table schema. The method has a strength to increase the concurrency by allowing other transaction's operation during DDL operation's compilation and execution.

However, it also has a weakness not to avoid a deadlock when DDL operations are operated at the same table at the same time. A deadlock case by loading indexes is as follows.

T1	T2
`CREATE INDEX i_t_i on t(i) WHERE i > 0;`	`CREATE INDEX i_t_j on t(j) WHERE j > 0;`
SCH_S_LOCK during checking types of "i > 0" case.	
	SCH_S_LOCK during checking types of "j > 0" case."j > 0"
SIX_LOCK during index loading.	
	requesting SIX_LOCK but waiting T1's SIX_LOCK is released
requesting SCH_M_LOCK but waiting T2's SCH_S_LOCK is released	

- **Special Locks**

  CUBRID 10.2 introduces a new type of locks, which are used in a few special cases of internal operations.

  - **Bulk Update Lock, BU_LOCK**

    This lock is designed for inserting large amounts of data into the database. As of **CUBRID 10.2**, this lock is exclusively used during the loaddb process while loading rows into a table.

BU_LOCK is compatible with itself and **SCH_S_LOCK**; as a result this ensures that no other **SELECT/DML/DDL** statements are allowed on the same table. However, multiple **loaddb** may load rows into a table. The **BU_LOCK** holder doesn't require row locks.

Note

This is a summarized description about locking.

- There are row(instance) and schema(class) about objects of locking targets. The locks grouped by the type of objects they're used:
  - row locks: **S_LOCK**, **X_LOCK**
  - intention/schema locks: **IX_LOCK**, **IS_LOCK**, **SIX_LOCK**, **SCH_S_LOCK**, **SCH_M_LOCK**
  - special locks: **BU_LOCK**
- All types of locks affect each other.

The following table briefly shows the lock compatibility between the locks described above. Compatibility means that the lock requester can obtain a lock while the lock holder is keeping the lock obtained for a certain object.

Lock Compatibility

- **NULL**: The status that any lock exists.

(O: TRUE, X: FALSE)

		Lock holder								
		NULL	**SCH-S**	**IS**	**S**	**IX**	**BU**	**SIX**	**X**	**SCH-M**
Lock requester	**NULL**	O	O	O	O	O	O	O	O	O
	SCH-S	O	O	O	O	O	O	O	O	X
	IS	O	O	O	O	O	X	O	X	X

S	O	O	O	O	X	X	X	X	X
IX	O	O	O	X	O	X	X	X	X
BU	O	O	X	X	X	O	X	X	X
SIX	O	O	O	X	X	X	X	X	X
X	O	O	X	X	X	X	X	X	X
SCH-M	O	X	X	X	X	X	X	X	X

Lock Transformation Table

- **NULL**: The status that any lock exists.

		Granted lock mode								
		NULL	**SCH-S**	**IS**	**S**	**IX**	**BU**	**SIX**	**X**	**SCH-M**
Requested lock mode	**NULL**	NULL	SCH-S	IS	S	IX	BU	SIX	X	SCH-M
	SCH-S	SCH-S	SCH-S	IS	S	IX	BU	SIX	X	SCH-M
	IS	IS	IS	IS	S	IX	X	SIX	X	SCH-M
	S	S	S	S	S	SIX	X	SIX	X	SCH-M
	IX	IX	IX	IX	SIX	IX	X	SIX	X	

									SCH-M
BU	BU	BU	BU	X	BU	BU	BU	X	SCH-M
SIX	SIX	SIX	SIX	SIX	SIX	X	SIX	X	SCH-M
X	X	X	X	X	X	X	X	X	SCH-M
SCH-M	SCH-M	SCH-M	SCH-M	SCH-M	SCH-M	SCH-M	SCH-M	SCH-M	SCH-M

Examples using locks

For next few examples, REPEATABLE READ(5) isolation level will be used. READ COMMITTED has different rules for updating rows and will be presented in next section (reference here). The examples will make use of lockdb utility to show existing locks.

Locking example: For next example REPEATABLE READ(5) isolation will be used and it will prove that read and write on same row are not blocked. Also conflicting updates will be tried, where the second updater is blocked. When transaction T1 commits, T2 is unblocked but update is not permitted because of isolation level restrictions. If T1 would rollback, then T2 can proceed with its update.

T1	T2	Description
`csql> ;au off` `csql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;`	`csql> ;au off` `csql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;`	AUTOCOMMIT OFF and REPEATABLE READ

T1	T2	Description
csql> CREATE TABLE tbl(a INT PRIMARY KEY, b INT); csql> INSERT INTO tbl VALUES (10, 10), (30, 30), (50, 50), (70, 70); csql> COMMIT;		
csql> UPDATE tbl SET a = 90 WHERE a = 10;		First version of row where a = 10 is locked and updated. A new version where row has a = 90 is created and also locked. cubrid lockdb: OID = 0|623| 4 Object type: Class = tbl. Total mode of holders = IX_LOCK, Total mode of waiters = NULL_LOCK. Num holders= 1, Num blocked-holders= 0, Num waiters= 0 LOCK HOLDERS: Tran_index = 1, Granted_mode =

T1	T2	Description
		IX_LOCK OID = 0| 650| 5 Object type: Instance of class (0| 623| 4) = tbl. MVCC info: insert ID = 5, delete ID = missing. Total mode of holders = X_LOCK, Total mode of waiters = NULL_LOCK. Num holders= 1, Num blocked- holders= 0, Num waiters= 0 LOCK HOLDERS: Tran_index = 1, Granted_mode = X_LOCK OID = 0| 650| 1 Object type: Instance of class (0| 623| 4) = tbl. MVCC info: insert ID = 4, delete ID = 5. Total mode of holders = X_LOCK, Total mode of waiters = NULL_LOCK. Num holders= 1, Num blocked- holders= 0, Num waiters= 0

T1	T2	Description

```
LOCK HOLDERS:
    Tran_index
=   1,
Granted_mode =
X_LOCK
```

```
csql> SELECT *
FROM tbl WHERE a
<= 20;


a            b

==================
========

10           10
```

Transaction T2 reads all rows where a <= 20. Since T1 did not commit its update, T2 will continue to see the row with a = 10 and will not lock it.:

```
cubrid lockdb:

OID =  0|
623|   4
Object type:
Class = tbl.
Total mode of
holders =
IX_LOCK,
    Total mode of
waiters =
NULL_LOCK.
Num holders=  2,
Num blocked-
holders=  0,
    Num waiters=
0
LOCK HOLDERS:
    Tran_index
=   1,
Granted_mode =
IX_LOCK
    Tran_index
=   2,
Granted_mode =
IS_LOCK
```

Transaction T2 now tries to update all rows having a <=

T1	T2	Description

	`csql> UPDATE tbl SET a = a + 100 WHERE a <= 20;`	20. This means T2 will upgrade its lock on class to IX_LOCK and will also try to update the row = 10 by first locking it. However, T1 has locked it already, so T2 will be blocked.
		
		`cubrid lockdb: OID = 0| 623| 4 Object type: Class = tbl. Total mode of holders = IX_LOCK, Total mode of waiters = NULL_LOCK. Num holders= 2, Num blocked-holders= 0, Num waiters= 0 LOCK HOLDERS: Tran_index = 1, Granted_mode = IX_LOCK Tran_index = 2, Granted_mode = IX_LOCK`
		
		`OID = 0| 650| 5 Object type: Instance of class ( 0| 623| 4) = tbl. MVCC info: insert ID = 5, delete ID = missing. Total mode of holders = X_LOCK,`

T1	T2	Description
		Total mode of waiters = NULL_LOCK. Num holders= 1, Num blocked-holders= 0, Num waiters= 0 LOCK HOLDERS: Tran_index = 1, Granted_mode = X_LOCK OID = 0| 650| 1 Object type: Instance of class (0| 623| 4) = tbl. MVCC info: insert ID = 4, delete ID = 5. Total mode of holders = X_LOCK, Total mode of waiters = X_LOCK. Num holders= 1, Num blocked-holders= 0, Num waiters= 1 LOCK HOLDERS: Tran_index = 1, Granted_mode = X_LOCK LOCK WAITERS: Tran_index = 2, Blocked_mode = X_LOCK
		T1's locks are released.

T1	T2	Description
`csql> COMMIT;`		
	`ERROR: Serializable conflict due to concurrent updates`	T2 is unblocked and will try to update the object T1 already updated. This is however not allowed in REPEATABLE READ isolation level and an error is thrown.

Locking to protect unique constraint

Two phase locking protocol in older CUBRID versions used index key locks to protect unique constraints and higher isolation restrictions. In CUBRID 10.0, key locking was removed. Isolation level restrictions are solved by MVCC snapshot, however unique constraint still needed some type of protection.

With MVCC, unique index can keep multiple versions at the same time, similarly to rows, each visible to different transactions. One is the last version, while the other versions are kept temporarily until they become invisible and can be removed by **VACUUM**. The rule to protect unique constraint is that all transactions trying to modify a key has to lock key's last existing version.

The below example uses **REPEATABLE READ** isolation to show the way locking prevents unique constraint violations.

T1	T2	Description
`csql> ;au off` `csql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;`	`csql> ;au off` `csql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;`	AUTOCOMMIT OFF and REPEATABLE READ

T1	T2	Description

`csql> CREATE TABLE tbl(a INT PRIMARY KEY, b INT);`
 	
`csql> INSERT INTO tbl`
`VALUES (10, 10),`
`(30, 30),`
`(50, 50),`
`(70, 70);`
`csql> COMMIT;`		

`csql> INSERT INTO tbl VALUES (20, 20);`		T1 inserts a new row into table and also locks it. The key 20 is therefore protected.

	`INSERT INTO tbl VALUES (20, 120);`	T2 also inserts a new row into table and locks it. However, when it tries to insert it in primary key, it discovers key 20 already exists. T2 has to lock existing object, that T1 inserted, and is blocked until T1 commits.
		
		`cubrid lockdb:`
		
		`OID = 0|623| 4`
		`Object type: Class = tbl.`
		`Total mode of holders = IX_LOCK,`

T1	T2	Description
		Total mode of waiters = NULL_LOCK. Num holders= 2, Num blocked-holders= 0, Num waiters= 0 LOCK HOLDERS: Tran_index = 1, Granted_mode = IX_LOCK Tran_index = 2, Granted_mode = IX_LOCK OID = 0| 650| 5 Object type: Instance of class (0| 623| 4) = tbl. MVCC info: insert ID = 5, delete ID = missing. Total mode of holders = X_LOCK, Total mode of waiters = X_LOCK. Num holders= 1, Num blocked-holders= 0, Num waiters= 1 LOCK HOLDERS: Tran_index = 1, Granted_mode = X_LOCK LOCK WAITERS: Tran_index = 1, Blocked_mode = X_LOCK

T1	T2	Description
		OID = 0|650| 6 Object type: Instance of class (0| 623| 4) = tbl. MVCC info: insert ID = 6, delete ID = missing. Total mode of holders = X_LOCK, Total mode of waiters = NULL_LOCK. Num holders= 1, Num blocked-holders= 0, Num waiters= 0 LOCK HOLDERS: Tran_index = 2, Granted_mode = X_LOCK
COMMIT;		T1's locks are released.
	ERROR: Operation would have caused one or more unique constraint violations. INDEX pk_tbl_a(B+tree: 0|186|640) ON CLASS tbl(CLASS_OID: 0|623|4).	T2 is unlocked, finds the key has been committed and throws unique constraint violation error.

T1	T2	Description
	key: 20(OID: 0|650|6).	

Transaction Deadlock

A deadlock is a state in which two or more transactions wait at once for another transaction's lock to be released. CUBRID resolves the problem by rolling back one of the transactions because transactions in a deadlock state will hinder the work of another transaction. The transaction to be rolled back is usually the transaction which has made the least updates; it is usually the one that started more recently. As soon as a transaction is rolled back, the lock held by the transaction is released and other transactions in a deadlock are permitted to proceed.

It is impossible to predict such deadlocks, but it is recommended that you reduce the range to which lock is applied by setting the index, shortening the transaction, or setting the transaction isolation level as low in order to decrease such occurrences.

Note that if you configure the value of **error_log_level**, which indicates the severity level, to NOTIFICATION, information on lock is stored in error log file of server upon deadlock occurrences.

Compared to older versions, CUBRID 10.0 no longer uses index key locking to read and write in index, thus deadlock occurrences have been greatly reduced. Another reason that deadlocks do not occur as often is that reading a range in index could lock many objects with high isolation levels in previous CUBRID versions, whereas CUBRID 10.0 uses no locks.

However, deadlocks are still possible when two transaction update same objects, but in a different order.

Example

session 1	session 2
csql> ;autocommit off AUTOCOMMIT IS OFF csql> set transaction isolation level REPEATABLE READ;	csql> ;autocommit off AUTOCOMMIT IS OFF csql> set transaction isolation level REPEATABLE READ;

session 1	session 2

```
Isolation level set to:
REPEATABLE READ
```

```
Isolation level set to:
REPEATABLE READ
```

```
csql> CREATE TABLE
lock_tbl(host_year INTEGER,

nation_code CHAR(3));
csql> INSERT INTO lock_tbl
VALUES (2004, 'KOR');
csql> INSERT INTO lock_tbl
VALUES (2004, 'USA');
csql> INSERT INTO lock_tbl
VALUES (2004, 'GER');
csql> INSERT INTO lock_tbl
VALUES (2008, 'GER');
csql> COMMIT;
```

```
csql> DELETE FROM lock_tbl
WHERE nation_code = 'KOR';

/* The two transactions lock
different objects
 * and they do not block each-
other.
 */
```

```
csql> DELETE FROM lock_tbl
WHERE nation_code = 'GER';
```

```
csql> DELETE FROM lock_tbl
WHERE host_year=2008;

/* T1 want's to modify a row
locked by T2 and is blocked */
```

```
csql> DELETE FROM lock_tbl
WHERE host_year = 2004;
```

session 1	session 2
	/* T2 now want to delete the row blocked by T1 * and a deadlock is created. */
ERROR: Your transaction (index 1, dba@ 090205|4760) has been unilaterally aborted by the system. /* System rolled back the transaction 1 to resolve a deadlock */	/* T2 is unblocked and proceeds on modifying its rows. */

Transaction Lock Timeout

CUBRID provides the lock timeout feature, which sets the waiting time for the lock until the transaction lock setting is allowed.

If the lock is allowed within the lock timeout, CUBRID rolls back the transaction and outputs an error message when the timeout has passed. If a transaction deadlock occurs within the lock timeout, CUBRID rolls back the transaction whose waiting time is closest to the timeout.

Setting the Lock Timeout

The system parameter **lock_timeout** in the **$CUBRID/conf/cubrid.conf** file or the **SET TRANSACTION** statement sets the timeout (in seconds) during which the application will wait for the lock and rolls back the transaction and outputs an error message when the specified time has passed. The default value of the **lock_timeout** parameter is **-1**, which means the application will wait indefinitely until the transaction lock is allowed. Therefore, the user can change this value depending on the transaction pattern of the application. If the lock timeout value has been set to 0, an error message will be displayed as soon as a lock occurs.

```
SET TRANSACTION LOCK TIMEOUT timeout_spec [ ; ]
timeout_spec:
- INFINITE
- OFF
```

```
- unsigned_integer
- variable
```

- **INFINITE** : Wait indefinitely until the transaction lock is allowed. Has the same effect as setting the system parameter **lock_timeout** to -1.
- **OFF** : Do not wait for the lock, but roll back the transaction and display an error message. Has the same effect as setting the system parameter **lock_timeout** to 0.
- *unsigned_integer* : Set in seconds. Wait for the transaction lock for the specified time period.
- *variable* : A variable can be specified. Wait for the transaction lock for the value stored by the variable.

Example 1

```
vi $CUBRID/conf/cubrid.conf
...
lock_timeout = 10s
...
```

Example 2

```
SET TRANSACTION LOCK TIMEOUT 10;
```

Checking the Lock Timeout

You can check the lock timeout set for the current application by using the **GET TRANSACTION** statement, or store this value in a variable.

```
GET TRANSACTION LOCK TIMEOUT [ { INTO | TO } variable ] [ ; ]
```

Example

```
GET TRANSACTION LOCK TIMEOUT;

         Result
================
  1.000000e+001
```

Checking and Handling Lock Timeout Error Message

The following message is displayed if lock timeout occurs in a transaction that has been waiting for another transaction's lock to be released.

```
Your transaction (index 2, user1@host1|9808) timed out waiting on
IX_LOCK lock on class tbl. You are waiting for
user(s) user1@host1|csql(9807), user1@host1|csql(9805) to finish.
```

- Your transaction(index 2 ...): This means that the index of the transaction that was rolled back due to timeout while waiting for the lock is 2. The transaction index is a number that is sequentially assigned when the client connects to the database server. You can also check this number by executing the **cubrid lockdb** utility.
- (... user1@host1|9808): *cub_user* is the login ID of the client and the part after @ is the name of the host where the client was running. The part after| is the process ID (PID) of the client.
- IX_LOCK: This means the exclusive lock set on the object to perform data update. For details, see Lock Mode Types And Compatibility.
- user1@host1|csql(9807), user1@host1|csql(9805): Another transactions waiting for termination to lock **IX_LOCK**

That is, the above lock error message can be interpreted as meaning that "Because another client is holding **X_LOCK** on a specific row in the *participant* table, transaction 3 which running on the host *host1* waited for the lock and the timeout has passed". If you want to check the lock information of the transaction specified in the error message, you can do so by using the **cubrid lockdb** utility to search for the OID value (ex: 0|636|34) of a specific row where the **X_LOCK** is set currently to find the transaction ID currently holding the lock, the client program name and the process ID (PID). For details, see lockdb. You can also check the transaction lock information in the CUBRID Manager.

You can organize the transactions by checking uncommitted queries through the SQL log after checking the transaction lock information in the manner described above. For information on checking the SQL log, see Broker Logs.

Also, you can forcefully stop problematic transactions by using the **cubrid killtran** utility. For details, see killtran.

Transaction Isolation Level

The transaction isolation level is determined based on how much interference occurs. The higher isolation means the less interference from other transactions and more serializable. The lower isolation means the more interference from other transactions and higher level of concurrency. You can control the level of consistency and concurrency specific to a service by setting appropriate isolation level.

Note

A transaction can be restored in all supported isolation levels because updates are not committed before the end of the transaction.

SET TRANSACTION ISOLATION LEVEL

You can set the level of transaction isolation by using **isolation_level** and the **SET TRANSACTION** statement in the **$CUBRID/conf/cubrid.conf**. The level of **READ COMMITTED** is set by default, which indicates the level 4 through level 4 to 6 (levels 1 to 3 were used by older versions of CUBRID and are now obsolete). For details, see Database Concurrency.

```
SET TRANSACTION ISOLATION LEVEL isolation_level_spec ;

isolation_level_spec:
    SERIALIZABLE | 6
    REPETABLE READ | 5
    READ COMMITTED | CURSOR STABILITY | 4
```

Example 1

```
vi $CUBRID/conf/cubrid.conf
...

isolation_level = 4
...

-- or

isolation_level = "TRAN_READ_COMMITTED"
```

Example 2

```
SET TRANSACTION ISOLATION LEVEL 4;
-- or
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Levels of Isolation Supported by CUBRID

Name	Description
READ COMMITTED (4)	Another transaction T2 cannot update the schema of table A while transaction T1 is

Name	Description
	viewing table A. Transaction T1 may experience R read (non-repeatable read) that was updated and committed by another transaction T2 when it is repeatedly retrieving the record R.
REPEATABLE READ (5)	Another transaction T2 cannot update the schema of table A while transaction T1 is viewing table A.
SERIALIZABLE (6)	Temporarily disabled - details in SERIALIZABLE Isolation Level

If the transaction level is changed in an application while a transaction is executed, the new level is applied to the rest of the transaction being executed. It is recommended that to modify the transaction isolation level when a transaction starts (after commit, rollback or system restart) because an isolation level which has already been set does not apply to the entire transaction, but can be changed during the transaction.

GET TRANSACTION ISOLATION LEVEL

You can assign the current isolation level to *variable* by using the **GET TRANSACTION ISOLATION LEVEL** statement. The following is a statement that verifies the isolation level.

```
GET TRANSACTION ISOLATION LEVEL [ { INTO | TO } variable ] [ ; ]
```

```
GET TRANSACTION ISOLATION LEVEL;
```

```
       Result
=============
  READ COMMITTED
```

READ COMMITTED Isolation Level

A relatively low isolation level (4). A dirty read does not occur, but non-repeatable or phantom read may. That is, transaction *T1* can read another value because insert or update by transaction *T2* is allowed while transaction *T1* is repeatedly retrieving one object.

The following are the rules of this isolation level:

- Transaction *T1* cannot read or modify the record inserted by another transaction *T2*. The record is instead ignored.
- Transaction *T1* can read the record being updated by another transaction *T2* and it sees the record's last committed version (but it cannot see uncommitted versions).
- Transaction *T1* cannot modify the record being updated by another transaction *T2*. *T1* waits for *T2* to commit and it re-evaluates record values. If the re-evaluation test is passed, *T1* modifies the record, otherwise it ignores it.
- Transaction *T1* can modify the record being viewed by another transaction *T2*.
- Transaction *T1* can update/insert record to the table being viewed by another transaction *T2*.
- Transaction *T1* cannot change the schema of the table being viewed by another transaction *T2*.
- Transaction *T1* creates a new snapshot with each executed statement, thus phantom or non-repeatable read may occur.

This isolation level follows MVCC locking protocol for an exclusive lock. A shared lock on a row is not required; however, an intent lock on a table is released when a transaction terminates to ensure repeatable read on the schema.

Example

The following example shows that a phantom or non-repeatable read may occur because another transaction can add or update a record while one transaction is performing the object read but repeatable read for the table schema update is ensured when the transaction level of the concurrent transactions is **READ COMMITTED**.

session 1	**session 2**

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> SET TRANSACTION
ISOLATION LEVEL READ COMMITTED;
```

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> SET TRANSACTION
ISOLATION LEVEL READ
```

session 1	session 2

```
Isolation level set to:
READ COMMITTED
```

```
COMMITTED;

Isolation level set to:
READ COMMITTED
```

```
csql> CREATE TABLE
isol4_tbl(host_year integer,
nation_code char(3));

csql> INSERT INTO isol4_tbl
VALUES (2008, 'AUS');

csql> COMMIT;
```

```
csql> SELECT * FROM isol4_tbl;

    host_year  nation_code
=============================
=====
         2008  'AUS'
```

```
csql> INSERT INTO isol4_tbl
VALUES (2004, 'AUS');
csql> INSERT INTO isol4_tbl
VALUES (2000, 'NED');
csql> COMMIT;
```

```
/* phantom read occurs
because tran 1 committed */
csql> SELECT * FROM isol4_tbl;

    host_year  nation_code
=============================
=====
         2008  'AUS'
```

session 1	session 2

```
          2004   'AUS'
          2000   'NED'
```

```
csql> UPDATE isol4_tbl
csql> SET nation_code = 'KOR'
csql> WHERE host_year = 2008;
csql> COMMIT;
```

```
/* unrepeatable read occurs
because tran 1 committed */
csql> SELECT * FROM isol4_tbl;

    host_year  nation_code
=============================
=====
          2008  'KOR'
          2004  'AUS'
          2000  'NED'
```

```
csql> ALTER TABLE isol4_tbl
ADD COLUMN gold INT;

/* unable to alter the table
schema until tran 2 committed
*/
```

```
/* repeatable read is ensured
while
 * tran_1 is altering table
schema
 */

csql> SELECT * FROM isol4_tbl;

    host_year  nation_code
```

session 1	session 2
	``` ========== ===== 2008 'KOR' 2004 'AUS' 2000 'NED' ```
	``` csql> COMMIT; ```
	``` csql> SELECT * FROM isol4_tbl; /* unable to access the table until tran_1 committed */ ```
``` csql> COMMIT; ```	
	``` host_year nation_code gold ========== ===== 2008 'KOR' NULL 2004 'AUS' NULL 2000 'NED' NULL ```

READ COMMITTED UPDATE RE-EVALUATION

READ COMMITTED isolation treats concurrent row updates differently than higher isolation levels. In higher isolation levels, if *T2* tries to modify a row already updated by concurrent transaction *T1*, it is blocked until *T1* commits and rollbacks, and if *T1* commits, *T2* aborts its statement execution, throwing serialization error. Under **READ COMMITTED** isolation, after *T1* commits, *T2* does not immediately abort its statement execution and re-evaluates the new version, which is not considered committed and would not violate any restrictions for this isolation. If the predicate used to select previous version is still true for the new version, *T2* goes ahead and modifies the

new version. If the predicate is no longer true, *T2* just ignores the record as if the predicate was never satisfied.

Example:

session 1	session 2

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> SET TRANSACTION
ISOLATION LEVEL 4;

Isolation level set to:
READ COMMITTED
```

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> SET TRANSACTION
ISOLATION LEVEL 4;

Isolation level set to:
READ COMMITTED
```

```
csql> CREATE TABLE
isol4_tbl(host_year integer,
nation_code char(3));
csql> INSERT INTO isol4_tbl
VALUES (2000, 'KOR');
csql> INSERT INTO isol4_tbl
VALUES (2004, 'USA');
csql> INSERT INTO isol4_tbl
VALUES (2004, 'GER');
csql> INSERT INTO isol4_tbl
VALUES (2008, 'GER');
csql> COMMIT;
```

```
csql> UPDATE isol4_tbl
csql> SET host_year =
host_year - 4
csql> WHERE nation_code =
'GER';

/* T1 locks and modifies
(2004, 'GER') to (2000, 'GER')
*/
/* T1 locks and modifies
```

session 1	session 2
(2008, 'GER') to (2004, 'GER') */	
	csql> UPDATE isol4_tbl csql> SET host_year = host_year + 4 csql> WHERE host_year >= 2004; /* T2 snapshot will try to modify three records: * (2004, 'USA'), (2004, 'GER'), (2008, 'GER') * * T2 locks and modifies (2004, 'USA') to (2008, 'USA') * T2 is blocked on lock on (2004, 'GER'). */
csql> COMMIT; /* T1 releases locks on modified rows. */	
	/* T2 is unblocked and will do the next steps: * * T2 finds (2004, 'GER') has a new version (2000, 'GER') * that doesn't satisfy predicate anymore. * T2 releases the lock on object and ignores it. * * T2 finds (2008, 'GER') has a new version (2004, 'GER')

session 1	session 2

	`*  that still satisfies the predicate.`
	`*  T2 keeps the lock and changes row to (2008, 'GER')`
	`*/`

	`csql> SELECT * FROM isol4_tbl;`
	
	`  host_year  nation_code`
	`==========`
	`2000  'KOR'`
	`2000  'GER'`
	`2008  'USA'`
	`2008  'GER'`

REPEATABLE READ Isolation Level

A relatively high isolation level (5). Dirty, non-repeatable, and phantom reads do not occur due to **snapshot isolation**. However, it's still not truly **serializable**, transaction execution cannot be defined *as if there were no other transactions running* at the same time. More complex anomalies, like write skews, that a **serializable snapshot isolation** level should not allow still occur.

In a write skew anomaly, two transactions concurrently read overlapping data sets and make disjoint updates on the overlapped data set, neither having seen the update performed by the other. In a serializable system, such anomaly would be impossible, since one transaction must occur first and the second transaction should see the update of the first transaction.

The following are the rules of this isolation level:

- Transaction *T1* cannot read or modify the record inserted by another transaction *T2*. The record is instead ignored.
- Transaction *T1* can read the record being updated by another transaction *T2* and it will see the record's last committed version.
- Transaction *T1* cannot modify the record being updated by another transaction *T2*.

- Transaction *T1* can modify the record being viewed by another transaction *T2*.
- Transaction *T1* can update/insert record to the table being viewed by another transaction *T2*.
- Transaction *T1* cannot change the schema of the table being viewed by another transaction *T2*.
- Transaction *T1* creates a unique snapshot valid throughout the entire duration of the transaction.

Example

The following example shows that non-repeatable and phantom reads may not occur because of **snapshot isolation**. However, write skews are possible, which means the isolation level is not **serializable**.

session 1	session 2

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> SET TRANSACTION
ISOLATION LEVEL 5;

Isolation level set to:
REPEATABLE READ
```

```
csql> ;autocommit off

AUTOCOMMIT IS OFF

csql> SET TRANSACTION
ISOLATION LEVEL 5;

Isolation level set to:
REPEATABLE READ
```

```
csql> CREATE TABLE
isol5_tbl(host_year integer,
nation_code char(3));
csql> CREATE UNIQUE INDEX
isol5_u_idx
          on
isol5_tbl(nation_code,
host_year);
csql> INSERT INTO isol5_tbl
VALUES (2008, 'AUS');
csql> INSERT INTO isol5_tbl
VALUES (2004, 'AUS');

csql> COMMIT;
```

session 1	session 2

```
csql> SELECT * FROM isol5_tbl
WHERE nation_code='AUS';

    host_year  nation_code
=============================
=====
         2004  'AUS'
         2008  'AUS'
```

```
csql> INSERT INTO isol5_tbl
VALUES (2004, 'KOR');
csql> INSERT INTO isol5_tbl
VALUES (2000, 'AUS');

/* able to insert new rows  */
csql> COMMIT;
```

```
csql> SELECT * FROM isol5_tbl
WHERE nation_code='AUS';

/* phantom read cannot occur
due to snapshot isolation */

    host_year  nation_code
=============================
=====
         2004  'AUS'
         2008  'AUS'
```

```
csql> UPDATE isol5_tbl
csql> SET host_year = 2012
csql> WHERE nation_code =
'AUS' and
csql> host_year=2008;

/* able to update rows viewed
```

session 1	session 2

```
by T2 */
csql> COMMIT;
```

```
csql> SELECT * FROM isol5_tbl
WHERE nation_code = 'AUS';

/* non-repeatable read cannot
occur due to snapshot
isolation */

    host_year  nation_code
=============================
=====
         2004  'AUS'
         2008  'AUS'
```

```
csql> COMMIT;
```

```
csql> SELECT * FROM isol5_tbl
WHERE host_year >= 2004;

    host_year  nation_code
=============================
====
         2004  'AUS'
         2004  'KOR'
         2012  'AUS'

csql> UPDATE isol5_tbl
csql> SET nation_code = 'USA'
csql> WHERE nation_code =
'AUS' and
csql> host_year = 2004;

csql> COMMIT;
```

```
csql> SELECT * FROM isol5_tbl
WHERE nation_code = 'AUS';

    host_year  nation_code
=============================
=====
         2000  'AUS'
         2004  'AUS'
         2012  'AUS'

csql> UPDATE isol5_tbl
csql> SET nation_code = 'NED'
csql> WHERE nation_code =
'AUS' and
csql> host_year = 2012;

csql> COMMIT;
```

session 1 **session 2**

```
/* T1 and T2 first have selected each 3 throws and rows (2004,
'AUS'), (2012, 'AUS') overlapped.
 * Then T1 modified (2004, 'AUS'), while T2 modified (2012, 'AUS'),
without blocking each other.
 * In a serial execution, the result of select query for T1 or T2,
whichever executes last, would be different.
 */
```

```
csql> SELECT * FROM isol5_tbl
WHERE nation_code = 'AUS';

    host_year  nation_code
=============================
=====
         2000  'AUS'
```

```
csql> ALTER TABLE isol5_tbl
ADD COLUMN gold INT;

/* unable to alter the table
schema until tran 2 committed
*/
```

session 1	session 2

	`/* repeatable read is ensured while tran_1 is altering * table schema */`
	
	`csql> SELECT * FROM isol5_tbl WHERE nation_code = 'AUS';`
	
	`host_year nation_code`
	`==========`
	`2000 'AUS'`

	`csql> COMMIT;`

	`csql> SELECT * FROM isol5_tbl WHERE nation_code = 'AUS';`
	
	`/* unable to access the table until tran_1 committed */`

`csql> COMMIT;`	

	`host_year nation_code gold`
	`==========`
	`2000 'AUS' NULL`

SERIALIZABLE Isolation Level

CUBRID 10.0 **SERIALIZABLE** isolation level is identical to **REPEATABLE READ** isolation level. As explained in REPEATABLE READ Isolation Level section, even though **SNAPSHOT** isolation ensures non-repeatable read and phantom read anomalies do not happen, write skew anomalies are still possible. To protect against write skew, index key locks for read may be used. Alternatively, there are many works that describe complex systems to provide **SERIALIZABLE SNAPSHOT ISOLATION**, by aborting transactions with the potential of creating an isolation conflict. One such system will be provided in a future CUBRID version.

The keyword was not removed for backward compatibility reasons, but remember, it is similar to **REPEATABLE READ**.

How to Handle Dirty Record

CUBRID flushes dirty data (or dirty record) in the client buffers to the database (server) such as the following situations. In additions to those, there can be more situations where flushes can be performed.

- Dirty data can be flushed to server when a transaction is committed.
- Some of dirty data can be flushed to server when a lot of data is loaded into the client buffers.
- Dirty data of table *A* can be flushed to server when the schema of table *A* is updated.
- Dirty data of table *A* can be flushed to server when the table *A* is retrieved (**SELECT**)
- Some of dirty data can be flushed to server when a server function is called.

Transaction Termination and Restoration

The restore process in CUBRID makes it possible that the database is not affected even if a software or hardware error occurs. In CUBRID, all read and update commands that are made during a transaction must be atomic. This means that either all of the transaction's commands are committed to the database or none are. The concept of atomicity is extended to the set of operations that consists of a transaction. The transaction must either commit so that all effects are permanently applied to the database or roll back so that all effects are removed. To ensure transaction atomicity, CUBRID applies the effects of the committed transaction again every time an error occurs without the updates of the transaction being written to the disk. CUBRID also removes the effects of partially committed transactions in the database every time the site fails (some transactions may have not committed or applications may have requested to cancel transactions). This restore feature eases the burden for the applications of maintaining the database consistency depending on the system error. The restore process used in CUBRID is based on the undo/redo logging mechanism.

CUBRID provides an automatic restore method to maintain the transaction atomicity when a hardware or software error occurs. You do not have to take the responsibility for restore since CUBRID's restore feature always returns the database to a consistent state even when an application or computer system error occurs. For this purpose, CUBRID automatically rolls back part of committed transactions when the application fails or the user requests explicitly. For example, a system error that occurred during the execution of the **COMMIT WORK** statement must be stopped if the transaction has not committed yet (it cannot be confirmed that the user's operation has been committed). Automatic stop prevents errors causing undesired changes to the database by canceling uncommitted updates.

Restarting Database

CUBRID uses log volumes/files and database backups to restore committed or uncommitted transactions when system or media (disk) error occurs. Logs are also used to support the user-specified rollback. A log consists of a collection of sequential files created by CUBRID. The most recent log is called the active log, and the rest are called archive logs. A log file refers to both the active and archive logs.

All updates of the database are written to the log. Actually, two copies of the updates are logged. The first one is called a before image (UNDO log) and used to restore data during execution of the user-specified **ROLLBACK WORK** statement or during media or system errors. The second copy is an after image (REDO log) and used to re-apply the updates when media or system error occurs.

When the active log is full, CUBRID copies it to an archive log to store in the disk. The archive log is needed to restore the database when a system failure occurs.

Normal Termination or Error

CUBRID restores the database if it restarts due to a normal termination or a device error. The restore process re-applies the committed changes that have not been applied to the database and removes the uncommitted changes stored in the database. The general operation of the database resumes after the restore is completed. This restore process does not use any archive logs or database backup.

In a client/server environment, the database can restart by using the **cubrid server** utility.

Media Error

The user's intervention is somewhat needed to restart the database after media error occurs. The first step is to restore the database by installing a backup of a known good state. In CUBRID, the most recent log file (the one after the last backup) must be installed. This specific log (archive or active) is applied to a backup copy of the database. As with normal termination, the database can restart after restoration is committed.

Note

To minimize the possibility of losing database updates, it is recommended to create a snapshot and store it in the backup media before it is deleted from the disk. The DBA can backup and restore the database by using the **cubrid backupdb** and **cubrid restoredb** utilities. For details on these utilities, see backupdb.

Trigger

CREATE TRIGGER

Guidelines for Trigger Definition

Trigger definition provides various and powerful functionalities. Before creating a trigger, you must consider the following:

- **Does the trigger condition expression cause unexpected results (side effect)?**

  You must use the SQL statements within an expectable range.

- **Does the trigger action change the table given as its event target?**

  While this type of design is not forbidden in the trigger definition, it must be carefully applied, because a trigger can be created that falls into an infinite loop. When the trigger action modifies the event target table, the same trigger can be called again. If a trigger occurs in a statement that contains a **WHERE** clause, there is no side effect in the table affected by the **WHERE** clause.

- **Does the trigger cause unnecessary overhead?**

  If the desired action can be expressed more effectively in the source, implement it directly in the source.

- **Is the trigger executed recursively?**

  If the trigger action calls a trigger and this trigger calls the previous trigger again, a recursive loop is created in the database. If a recursive loop is created, the trigger may not be executed correctly, or the current session must be forced to terminate to break the ongoing infinite loop.

- **Is the trigger definition unique?**

A trigger defined in the same table or the one started in the same action becomes the cause of an unrecoverable error. A trigger in the same table must have a different trigger event. In addition, trigger priority must be explicitly and unambiguously defined.

Trigger Definition

A trigger is created by defining a trigger target, condition and action to be performed in the **CREATE TRIGGER** statement. A trigger is a database object that performs a defined action when a specific event occurs in the target table.

```
CREATE TRIGGER [schema_name.]trigger_name
[ STATUS { ACTIVE | INACTIVE } ]
[ PRIORITY key ]
<event_time> <event_type> [<event_target>]
[ IF condition ]
EXECUTE [ AFTER | DEFERRED ] action
[COMMENT 'trigger_comment'];

<event_time> ::=
    BEFORE |
    AFTER  |
    DEFERRED

<event_type> ::=
    INSERT |
    STATEMENT INSERT |
    UPDATE |
    STATEMENT UPDATE |
    DELETE |
    STATEMENT DELETE |
    ROLLBACK |
    COMMIT

<event_target> ::=
    ON [schema_name.]table_name |
    ON [schema_name.]table_name [ (column_name) ]

<condition> ::=
    expression

<action> ::=
    REJECT |
    INVALIDATE TRANSACTION |
    PRINT message_string |
    INSERT statement |
```

```
    UPDATE statement |
    DELETE statement
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used.
- *trigger_name*: specifies the name of the trigger to be defined.
- [**STATUS** { **ACTIVE** | **INACTIVE** }]: Defines the state of the trigger (if not defined, the default value is **ACTIVE**).
  - If **ACTIVE** state is specified, the trigger is executed every time the corresponding event occurs.
  - If **INACTIVE** state is specified, the trigger is not executed even when the corresponding event occurs. The state of the trigger can be modified. For details, see ALTER TRIGGER section.
- [**PRIORITY** *key*]: specifies a trigger priority if multiple triggers are called for an event. *key* must be a floating point value that is not negative. If the priority is not defined, the lowest priority 0 is assigned. Triggers having the same priority are executed in a random order. The priority of triggers can be modified. For details, see ALTER TRIGGER section.
- <*event_time*>: specifies the point of time when the conditions and actions are executed. **BEFORE**, **AFTER** or **DEFERRED** can be specified. For details, see the Event Time section.
- <*event_type*>: trigger types are divided into a user trigger and a table trigger. For details, see the Trigger Event Type section.
- <*event_target*>: An event target is used to specify the target for the trigger to be called. For details, see the Trigger Event Target section.
- <*condition*>: specifies the trigger condition. For details, see the Trigger Condition section.
- <*action*>: specifies the trigger action. For details, see the Trigger Action section.
- *trigger_comment*: specifies a trigger's comment.

Note

- **DBA** and **DBA** members can create triggers in different schemas. If a user is neither **DBA** nor **DBA** member, triggers can only be created in the schema of that user.

The following example shows how to create a trigger that rejects the update if the number of medals won is smaller than 0 when an instance of the *participant* table is updated. As shown

below, the update is rejected if you try to change the number of gold (*gold*) medals that Korea won in the 2004 Olympic Games to a negative number.

```
CREATE TRIGGER medal_trigger
BEFORE UPDATE ON participant
IF new.gold < 0 OR new.silver < 0 OR new.bronze < 0
EXECUTE REJECT;

UPDATE participant SET gold = -5 WHERE nation_code = 'KOR'
AND host_year = 2004;
```

```
ERROR: The operation has been rejected by trigger "medal_trigger".
```

Event Time

Specifies the point of time when trigger conditions and actions are executed. The types of event time are **BEFORE**, **AFTER** and **DEFERRED**.

- **BEFORE**: checks the condition before the event is processed.
- **AFTER**: checks the condition after the event is processed.
- **DEFERRED**: checks the condition at the end of the transaction for the event. If you specify **DEFERRED**, you cannot use **COMMIT** or **ROLLBACK** as the event type.

Trigger Type

User Trigger

- A trigger relevant to a specific user of the database is called a user trigger.
- A user trigger has no event target and is executed only by the owner of the trigger (the user who created the trigger).
- Event types that define a user trigger are **COMMIT** and **ROLLBACK**.

Table Trigger

- A trigger that has a table as the event target is called a table trigger (class trigger).
- A table trigger can be seen by all users who have the **SELECT** authorization on a target table.
- Event types that define a table trigger are instance and statement events.

Trigger Event Type

- Instance events: An event type whose unit of operation is an instance. The types of instance (record) events are as follows:
  - **INSERT**
  - **UPDATE**
  - **DELETE**
- Statement events: If you define a statement event as an event type, the trigger is called only once when the trigger starts even when there are multiple objects (instances) affected by the given statement (event). The types of statement events are as follows:
  - **STATEMENT INSERT**
  - **STATEMENT UPDATE**
  - **STATEMENT DELETE**
- Other events: **COMMIT** and **ROLLBACK** cannot be applied to individual instances.
  - **COMMIT**
  - **ROLLBACK**

The following example shows how to use an instance event. The *example* trigger is called by each instance affected by the database update. For example, if the *score* values of five instances in the *history* table are modified, the trigger is called five times.

```
CREATE TABLE update_logs(event_code INTEGER, score VARCHAR(10), dt
DATETIME);

CREATE TRIGGER example
BEFORE UPDATE ON history(score)
EXECUTE INSERT INTO update_logs VALUES (obj.event_code, obj.score,
SYSDATETIME);
```

If you want the trigger to be called only once, before the first instance of the *score* column is updated, use the **STATEMENT UPDATE** type as the following example.

The following example shows how to use a statement event. If you define a statement event, the trigger is called only once before the first instance gets updated even when there are multiple instances affected by the update.

```
CREATE TRIGGER example
BEFORE STATEMENT UPDATE ON history(score)
EXECUTE PRINT 'There was an update on history table';
```

Note

- You must specify the event target when you define an instance or statement event as the event type.
- **COMMIT** and **ROLLBACK** cannot have an event target.

Trigger Event Target

An event target specifies the target for the trigger to be called. The target of a trigger event can be specified as a table or column name. If a column name is specified, the trigger is called only when the specified column is affected by the event. If a column is not specified, the trigger is called when any column of the table is affected. Only **UPDATE** and **STATEMENT UPDATE** events can specify a column as the event target.

The following example shows how to specify the *score* column of the *history* table as the event target of the *example* trigger.

```
CREATE TABLE update_logs(event_code INTEGER, score VARCHAR(10), dt
DATETIME);

CREATE TRIGGER example
BEFORE UPDATE ON history(score)
EXECUTE INSERT INTO update_logs VALUES (obj.event_code, obj.score,
SYSDATETIME);
```

Combination of Event Type and Target

A database event calling triggers is identified by the trigger event type and event target in a trigger definition. The following table shows the trigger event type and target combinations, along with the meaning of the CUBRID database event that the trigger event represents.

Event Type	Event Target	Corresponding Database Activity
UPDATE	Table	Trigger is called when the UPDATE statement for a table is executed.
INSERT	Table	Trigger is called when the INSERT statement for a table is executed.
DELETE	Table	Trigger is called when the DELETE statement for a table is executed.
COMMIT	None	Trigger is called when database transaction is committed.
ROLLBACK	None	Trigger is called when database transaction is rolled back.

Trigger Condition

You can specify whether a trigger action is to be performed by defining a condition when defining the trigger.

- If a trigger condition is specified, it can be written as an independent compound expression that evaluates to true or false. In this case, the expression can contain arithmetic and logical operators allowed in the **WHERE** clause of the **SELECT** statement. The trigger action is performed if the condition is true; if it is false, action is ignored.
- If a trigger condition is omitted, the trigger becomes an unconditional trigger, which refers to that the trigger action is performed whenever it is called.

The following example shows how to use a correlation name in an expression within a condition. If the event type is **INSERT**, **UPDATE** or **DELETE**, the expression in the condition can refer to the

correlation names **obj**, **new** or **old** to access a specific column. This example prefixes **obj** to the column name in the trigger condition to show that the *example* trigger tests the condition based on the current value of the *record* column.

```
CREATE TRIGGER example
BEFORE UPDATE ON participant
IF new.gold < 0 OR new.silver < 0 OR new.bronze < 0
EXECUTE REJECT;
```

The following example shows how to use the **SELECT** statement in an expression within a condition. The trigger in this example uses the **SELECT** statement that contains an aggregate function **COUNT** (*) to compare the value with a constant. The **SELECT** statement must be enclosed in parentheses and must be placed at the end of the expression.

```
CREATE TRIGGER example
BEFORE INSERT ON participant
IF 1000 >  (SELECT COUNT(*) FROM participant)
EXECUTE REJECT;
```

Note

The expression given in the trigger condition may cause side effects on the database if a method is called while the condition is performed. A trigger condition must be constructed to avoid unexpected side effects in the database.

Correlation Name

You can access the column values defined in the target table by using a correlation name in the trigger definition. A correlation name is the instance that is actually affected by the database operation calling the trigger. A correlation name can also be specified in a trigger condition or action.

The types of correlation names are **new**, **old** and **obj**. These correlation names can be used only in instance triggers that have an **INSERT**, **UPDATE** or **DELETE** event.

As shown in the table below, the use of correlation names is further restricted by the event time defined for the trigger condition.

	BEFORE	AFTER or DERERRED
INSERT	**new**	**obj**
UPDATE	**obj** **new**	**obj** **old** (AFTER)
DELETE	**obj**	N/A

Correlation Name	Representative Attribute Value
obj	Refers to the current attribute value of an instance. This can be used to access attribute values before an instance is updated or deleted. It is also used to access attribute values after an instance has been updated or inserted.
new	Refers to the attribute value proposed by an insert or update operation. The new value can be accessed only before the instance is actually inserted or updated.
old	**Refers to the attribute value that existed prior to the completion of an update operation. This value is maintained** only while the trigger is being performed. Once the trigger is completed, the **old** values get lost.

Trigger Action

A trigger action describes what to be performed if the trigger condition is true or omitted. If a specific point of time (**AFTER** or **DEFERRED**) is not given in the action clause, the action is executed at once as the trigger event.

The following is a list of actions that can be used for trigger definitions.

- **REJECT**: discards the operation that initiated the trigger and keeps the former state of the database, if the condition is not true. Once the operation is performed, **REJECT** is allowed only when the action time is **BEFORE** because the operation cannot be rejected. Therefore, you must not use **REJECT** if the action time is **AFTER** or **DERERRED**.
- **INVALIDATE TRANSACTION**: allows the event operation that called the trigger, but does not allow the transaction that contains the commit to be executed. You must cancel the transaction by using the **ROLLBACK** statement if it is not valid. Such action is used to protect the database from having invalid data after a data-changing event happens.
- **PRINT**: displays trigger actions on the terminal screen in text messages, and can be used during developments or tests. The results of event operations are not rejected or discarded.
- **INSERT**: inserts one or more new instances to the table.
- **UPDATE**: updates one or more column values in the table.
- **DELETE**: deletes one or more instances from the table.

The following example shows how to define an action when a trigger is created. The *medal_trig* trigger defines **REJECT** in its action. **REJECT** can be specified only when the action time is **BEFORE**.

```
CREATE TRIGGER medal_trig
BEFORE UPDATE ON participant
IF new.gold < 0 OR new.silver < 0 OR new.bronze < 0
EXECUTE REJECT;
```

Note

- Trigger may fall into an infinite loop when you use **INSERT** in an action of a trigger where an **INSERT** event is defined.
- If a trigger where an **UPDATE** event is defined runs on a partitioned table, you must be careful because the defined partition can be broken or unintended malfunction may occur. To prevent such situation, CUBRID outputs an error so that the **UPDATE** causing changes to

the running partition is not executed. Trigger may fall into an infinite loop when you use **UPDATE** in an action of a trigger where an **UPDATE** event is defined.

Trigger's COMMENT

You can specify a trigger's comment as follows.

```
CREATE TRIGGER trg_ab BEFORE UPDATE on abc(c) EXECUTE UPDATE cube_ab
SET sumc = sumc + 1
COMMENT 'test trigger comment';
```

You can see a trigger's comment by running the below statement.

```
SELECT name, comment FROM db_trigger;
SELECT trigger_name, comment FROM db_trig;
```

Or you can see a trigger's comment with ;sc command which displays a schema in the CSQL interpreter.

```
$ csql -u dba demodb

csql> ;sc tbl
```

To change the trigger's comment, refer to **ALTER TRIGGER** syntax on the below.

ALTER TRIGGER

In the trigger definition, **STATUS** and **PRIORITY** options can be changed by using the **ALTER** statement. If you need to alter other parts of the trigger (event targets or conditional expressions), you must delete and then re-create the trigger.

```
ALTER TRIGGER [schema_name.]trigger_name <trigger_option> ;

<trigger_option> ::=
    STATUS { ACTIVE | INACTIVE } |
    PRIORITY key
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used.
- *trigger_name*: specifies the name of the trigger to be changed.

- **STATUS** { **ACTIVE** | **INACTIVE** }: changes the status of the trigger.
- **PRIORITY** *key*: changes the priority.

The following example shows how to create the medal_trig trigger and then change its state to **INACTIVE** and its priority to 0.7.

```
CREATE TRIGGER medal_trig
STATUS ACTIVE
BEFORE UPDATE ON participant
IF new.gold < 0 OR new.silver < 0 OR new.bronze < 0
EXECUTE REJECT;

ALTER TRIGGER medal_trig STATUS INACTIVE;
ALTER TRIGGER medal_trig PRIORITY 0.7;
```

Note

- Only one *trigger_option* can be specified in a single **ALTER TRIGGER** statement.
- To change a table trigger, you must be the trigger owner or granted the **ALTER** authorization on the table where the trigger belongs.
- A user trigger can only be changed by its owner. For details on *trigger_option*, see the CREATE TRIGGER section. The key specified together with the **PRIORITY** option must be a non-negative floating point value.

Trigger's COMMENT

You can change a trigger's comment by running **ALTER TRIGGER** syntax as below.

```
ALTER TRIGGER [schema_name.]trigger_name [trigger_option]
[COMMENT 'comment_string'];
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used.
- *trigger_name*: specifies the name of the trigger to be changed.
- *comment_string*: specifies a trigger's comment.

If you want to change only trigger's comment, you can omit trigger options (*trigger_option*).

For *trigger_option*, see ALTER TRIGGER on the above.

```
ALTER TRIGGER trg_ab COMMENT 'new trigger comment';
```

DROP TRIGGER

You can drop a trigger by using the **DROP TRIGGER** statement.

```
DROP TRIGGER [schema_name.]trigger_name ;
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used.
- *trigger_name*: specifies the name of the trigger to be dropped.

The following example shows how to drop the medal_trig trigger.

```
DROP TRIGGER medal_trig;
```

Note

- A user trigger (i.e. the trigger event is **COMMIT** or **ROLLBACK**) can be seen and dropped only by the owner.
- Only one trigger can be dropped by a single **DROP TRIGGER** statement. A table trigger can be dropped by a user who has an **ALTER** authorization on the table.

RENAME TRIGGER

You can change a trigger name by using the **TRIGGER** reserved word in the **RENAME** statement.

```
RENAME TRIGGER [schema_name.]old_trigger_name {AS | TO}
[schema_name.]new_trigger_name ;
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used. The schema of the current trigger and the schema of the trigger to be changed must be the same.
- *old_trigger_name*: specifies the current name of the trigger.
- *new_trigger_name*: specifies the name of the trigger to be modified.

```
RENAME TRIGGER medal_trigger AS medal_trig;
```

Note

- A trigger name must be unique among triggers owned by the user. However, it can be the same as the table name in the database or the name of a trigger owned by another owner.
- To rename a table trigger, you must be the trigger owner or granted the **ALTER** authorization on the table where the trigger belongs. A user trigger can only be renamed by its user.

Deferred Condition and Action

A deferred trigger action and condition can be executed later or canceled. These triggers include a **DEFERRED** time option in the event time or action clause. If the **DEFERRED** option is specified in the event time and the time is omitted before the action, the action is deferred automatically.

Executing Deferred Condition and Action

Executes the deferred condition or action of a trigger immediately.

```
EXECUTE DEFERRED TRIGGER <trigger_identifier> ;

<trigger_identifier> ::=
    [schema_name.]trigger_name |
    ALL TRIGGERS
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used.
- *trigger_name*: executes the deferred action of the trigger when a trigger name is specified.
- **ALL TRIGGERS**: executes all currently deferred actions.

Dropping Deferred Condition and Action

Drops the deferred condition and action of a trigger.

```
DROP DEFERRED TRIGGER trigger_identifier ;

<trigger_identifier> ::=
```

```
    [schema_name.]trigger_name |
    ALL TRIGGER
```

- *schema_name*: Specifies the schema name of the trigger. If omitted, the schema name of the current session is used.
- *trigger_name* : Cancels the deferred action of the trigger when a trigger name is specified.
- **ALL TRIGGERS** : Cancels currently deferred actions.

Granting Trigger Authorization

Trigger authorization is not granted explicitly. Authorization on the table trigger is automatically granted to the user if the authorization is granted on the event target table described in the trigger definition. In other words, triggers that have table targets (**INSERT**, **UPDATE**, etc.) are seen by all users. User triggers (**COMMIT** and **ROLLBACK**) are seen only by the user who defined the triggers. All authorizations are automatically granted to the trigger owner.

Note

- To define a table trigger, you must have an **ALTER** authorization on the table.
- To define a user trigger, the database must be accessed by a valid user.

Trigger on REPLACE and INSERT ... ON DUPLICATE KEY UPDATE

When the **REPLACE** statement and **INSERT ... ON DUPLICATE KEY UPDATE** statements are executed, the trigger is executed in CUBRID, while **DELETE**, **UPDATE**, **INSERT** jobs occur internally. The following table shows the order in which the trigger is executed in CUBRID depending on the event that occurred when the **REPLACE** or **INSERT ... ON DUPLICATE KEY UPDATE** statement is executed. Both the **REPLACE** statement and the **INSERT ... ON DUPLICATE KEY UPDATE** statement do not execute triggers in the inherited class (table).

Execution Sequence of Triggers in the REPLACE and the INSERT ... ON DUPLICATE KEY UPDATE statements

Event	Execution Sequence of Triggers
REPLACE When a record is deleted and new one is inserted	BEFORE DELETE > AFTER DELETE > BEFORE INSERT > AFTER INSERT

Event	Execution Sequence of Triggers
INSERT ... ON DUPLICATE KEY UPDATE When a record is updated	BEFORE UPDATE > AFTER UPDATE
REPLACE, INSERT ... ON DUPLCATE KEY UPDATE Only when a record is inserted	BEFORE INSERT > AFTER INSERT

The following example shows that **INSERT ... ON DUPLICATE KEY UPDATE** and **REPLACE** are executed in the *with_trigger* table and records are inserted to the *trigger_actions* table as a consequence of the execution.

```
CREATE TABLE with_trigger (id INT UNIQUE);
INSERT INTO with_trigger VALUES (11);

CREATE TABLE trigger_actions (val INT);

CREATE TRIGGER trig_1 BEFORE INSERT ON with_trigger EXECUTE INSERT
INTO trigger_actions VALUES (1);
CREATE TRIGGER trig_2 BEFORE UPDATE ON with_trigger EXECUTE INSERT
INTO trigger_actions VALUES (2);
CREATE TRIGGER trig_3 BEFORE DELETE ON with_trigger EXECUTE INSERT
INTO trigger_actions VALUES (3);

INSERT INTO with_trigger VALUES (11) ON DUPLICATE KEY UPDATE id=22;

SELECT * FROM trigger_actions;
```

```
          va
==============
            2
```

```
REPLACE INTO with_trigger VALUES (22);

SELECT * FROM trigger_actions;
```

```
          va
==============
            2
            3
            1
```

Trigger Debugging

Once a trigger is defined, it is recommended to check whether it is running as intended. Sometimes the trigger takes more time than expected in processing. This means that it is adding too much overhead to the system or has fallen into a recursive loop. This section explains several ways to debug the trigger.

The following example shows a trigger that was defined to fall into a recursive *loop_tgr* when it is called. A *loop_tgr* trigger is somewhat artificial in its purpose; it can be used as an example of debugging trigger.

```
CREATE TRIGGER loop_tgr
BEFORE UPDATE ON participant(gold)
IF new.gold > 0
EXECUTE UPDATE participant
        SET gold = new.gold - 1
        WHERE nation_code = obj.nation_code AND host_year =
obj.host_year;
```

Viewing Trigger Execution Log

You can view the execution log of the trigger from a terminal by using the **SET TRIGGER TRACE** statement.

```
SET TRIGGER TRACE <switch> ;

<switch> ::=
    ON |
    OFF
```

- **ON**: executes **TRACE** until the switch is set to **OFF** or the current database session terminates.
- **OFF**: stops the **TRACE**.

The following example shows how to execute the **TRACE** and the *loop_tgr* trigger to view the trigger execution logs. To identify the trace for each condition and action executed when the trigger is called, a message is displayed on the terminal. The following message appears 15 times because the *loop_tgr* trigger is executed until the *gold* value becomes 0.

```
SET TRIGGER TRACE ON;
UPDATE participant SET gold = 15 WHERE nation_code = 'KOR' AND
host_year = 1988;
```

```
TRACE: Evaluating condition for trigger "loop".
TRACE: Executing action for trigger "loop".
```

Limiting Nested Trigger

With the **MAXIMUM DEPTH** keyword of the **SET TRIGGER** statement, you can limit the number of triggers to be initiated at each step. By doing so, you can prevent a recursively called trigger from falling into an infinite loop.

```
SET TRIGGER [ MAXIMUM ] DEPTH count ;
```

- *count*: A positive integer value that specifies the number of times that a trigger can recursively start another trigger or itself. If the number of triggers reaches the maximum depth, the database request stops(aborts) and the transaction is marked as invalid. The specified **DEPTH** applies to all other triggers except the current session. The maximum value is 32.

The following example shows how to configure the maximum number of times of recursive trigger calling to 10. This applies to all triggers that start subsequently. In this example, the *gold* column value is updated to 15, so the trigger is called 16 times in total. This exceeds the currently set maximum depth and the following error message occurs.

```
SET TRIGGER MAXIMUM DEPTH 10;
UPDATE participant SET gold = 15 WHERE nation_code = 'KOR' AND
host_year = 1988;
```

```
ERROR: Maximum trigger depth 10 exceeded at trigger "loop_tgr".
```

Trigger Example

This section covers trigger definitions in the demo database. The triggers created in the *demodb* database are not complex, but use most of the features available in CUBRID. If you want to maintain the original state of the *demodb* database when testing such triggers, you must perform a rollback after changes are made to the data.

Triggers created by the user in the own database can be as powerful as applications created by the user.

The following trigger created in the *participant* table rejects an update to the medal column (*gold*, *silver*, *bronze*) if a given value is smaller than 0. The evaluation time must be **BEFORE** because a correlation name new is used in the trigger condition. Although not described, the action time of this trigger is also **BEFORE**.

```
CREATE TRIGGER medal_trigger
BEFORE UPDATE ON participant
IF new.gold < 0 OR new.silver < 0 OR new.bronze < 0
EXECUTE REJECT;
```

The trigger *medal_trigger* starts when the number of gold (*gold*) medals of the country whose nation code is 'BLA' is updated. Since the trigger created does not allow negative numbers, the example below will not be updated.

```
UPDATE participant
SET gold = -10
WHERE nation_code = 'BLA';
```

The following trigger has the same condition as the one above except that **STATUS ACTIVE** is added. If the **STATUS** statement is omitted, the default value is **ACTIVE**. You can change **STATUS** to **INACTIVE** by using the **ALTER TRIGGER** statement.

You can specify whether or not to execute the trigger depending on the **STATUS** value.

```
CREATE TRIGGER medal_trig
STATUS ACTIVE
BEFORE UPDATE ON participant
IF new.gold < 0 OR new.silver < 0 OR new.bronze < 0
EXECUTE REJECT;

ALTER TRIGGER medal_trig
STATUS INACTIVE;
```

The following trigger shows how integrity constraint is enforced when a transaction is committed. This example is different from the previous ones, in that one trigger can have specific conditions for multiple tables.

```
CREATE TRIGGER check_null_first
BEFORE COMMIT
IF 0 < (SELECT count(*) FROM athlete WHERE gender IS NULL)
OR 0 < (SELECT count(*) FROM game WHERE nation_code IS NULL)
EXECUTE REJECT;
```

The following trigger delays the update integrity constraint check for the *record* table until the transaction is committed. Since the **DEFERRED** keyword is given as the event time, the trigger is not executed at the time.

```
CREATE TRIGGER deferred_check_on_record
DEFERRED UPDATE ON record
```

```
IF obj.score = '100'
EXECUTE INVALIDATE TRANSACTION;
```

Once completed, the update in the *record* table can be confirmed at the last point (commit or rollback) of the current transaction. The correlation name **old** cannot be used in the conditional clause of the trigger where **DEFERRED UPDATE** is used. Therefore, you cannot create a trigger as the following.

```
CREATE TABLE foo (n int);
CREATE TRIGGER foo_trigger
    DEFERRED UPDATE ON foo
    IF old.n = 100
    EXECUTE PRINT 'foo_trigger';
```

If you try to create a trigger as shown above, an error message is displayed and the trigger fails.

```
ERROR: Error compiling condition for 'foo_trigger' : old.n is not
defined.
```

The correlation name **old** can be used only with **AFTER**.

CUBRID DBLink

Introduction to CUBRID DBLink

When retrieving information from a database, it is often necessary to retrieve information from an external database. In this way, if you use CUBRID DBLink to inquire information in an external database, you can access information in other databases. CUBRID DBLink provides a function to inquire information in the databases of homogeneous CUBRID and heterogeneous Oracle, MySQL and MariaDB.

It has the advantage of being able to directly inquire information from an external database. However, it is possible to set up multiple external databases, but when searching for information, it is possible to inquire information from only one other database.

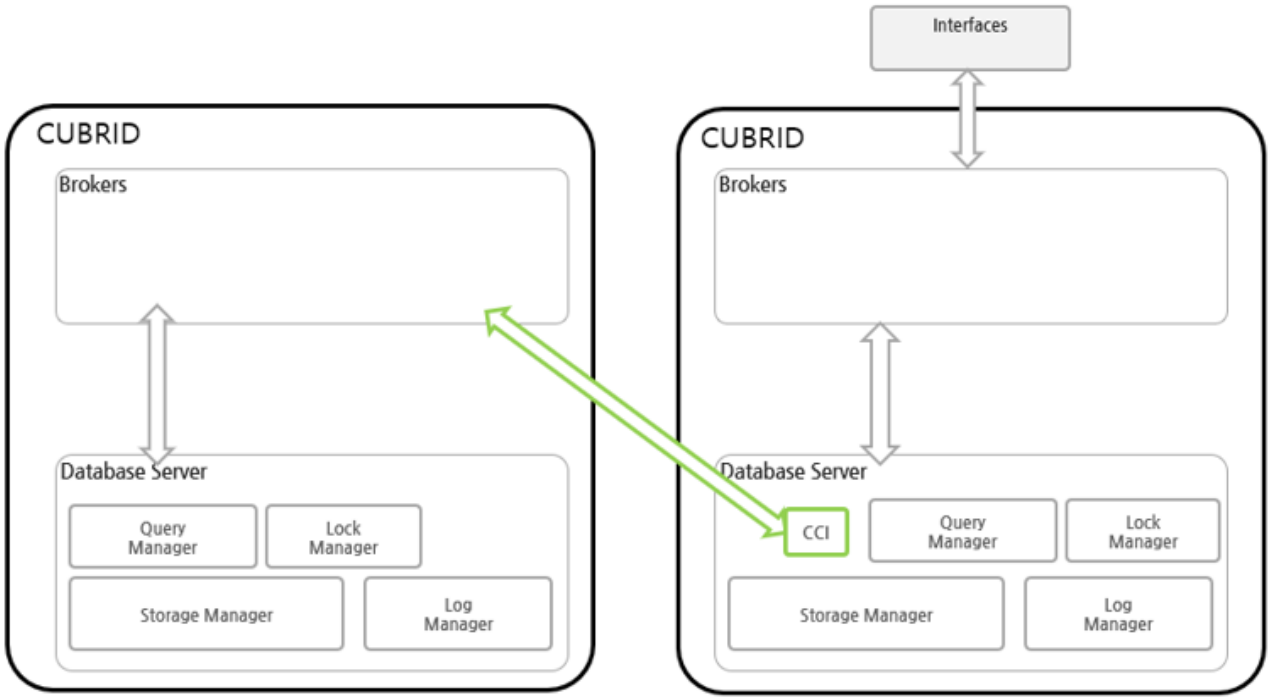
CUBRID DBLink can be used in the form of a DBLINK statement specifying the server to be connected and the query to be executed in the FROM clause of SELECT and in the form of a remote table (table extension), and only remote table forms can be used in INSERT/UPDATE/DELETE/MERGE.

CUBRID DBLink diagram

CUBRID DBLink supports DBLink between homogeneous and heterogeneous DBLinks.

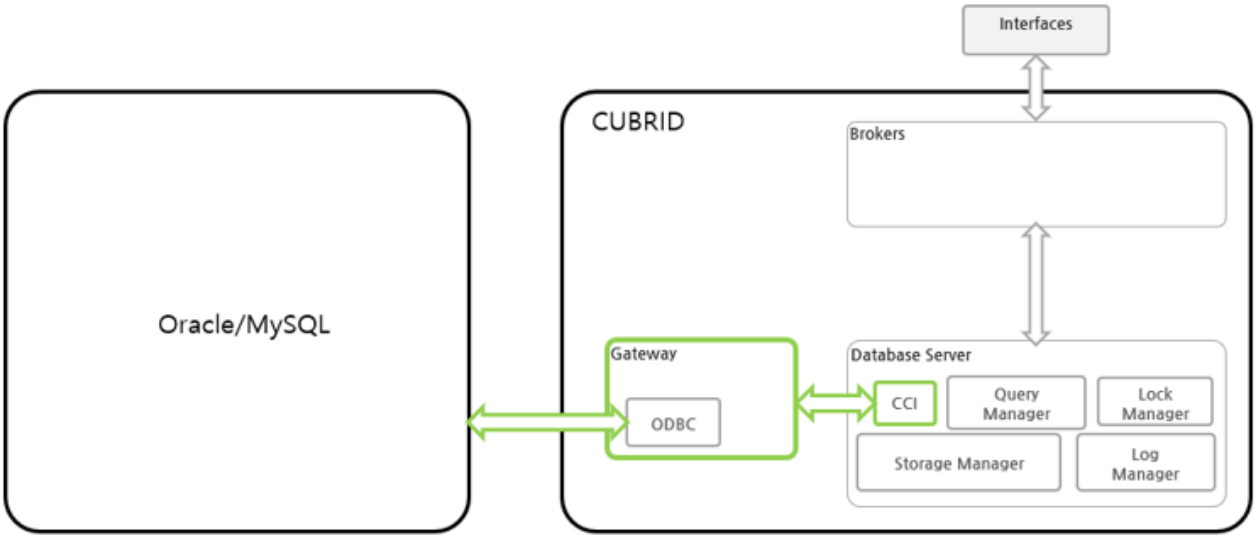
Homogeneous DBLink diagram

If you look at the configuration diagram for inquiring information of a homogeneous database, you can use CCI in Database Server to connect to homogeneous brokers and inquire information from an external database.



Heterogeneous DBLink diagram

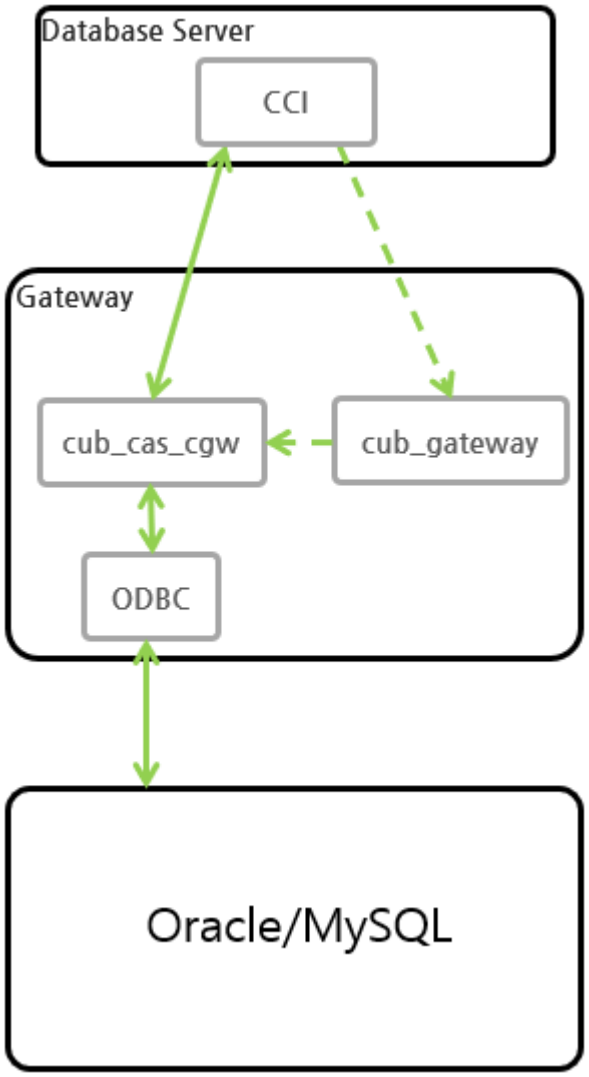
If you look at the configuration diagram for inquiring information in heterogeneous databases, you can inquire information in heterogeneous databases through GATEWAY. The gateway uses the ODBC (Open DataBase Connectivity) driver of the connecting database.



GATEWAY

A gateway is a middleware that mediates between the CUBRID database and heterogeneous databases and is similar to a broker. The gateway connects to a heterogeneous database server (Oracle/MySQL/MariaDB etc), retrieves data, and delivers it to the CUBRID database server.

A cubrid system including a gateway has a multi-hierarchical structure including cub_gateway and cub_cas_cgw as shown in the figure below.



cub_cas_cgw

cub_cas_cgw (CAS Gateway) acts as a common application server used by all the application clients that request connections. cub_cas_cgw also acts as the database server's client and provides the connection to the database server upon the client's request. The number of cub_cas_cgw(s) running in the service pool can be specified in the cubrid_broker.conf file, and this number is dynamically adjusted by cub_gateway.

cub_gateway

cub_gateway mediates the connection between the application client and the cub_cas_cgw. That is, when an application client requests access, the cub_broker checks the status of the cub_cas_cgw through the shared memory, and then delivers the request to an accessible cub_cas_cgws. It then returns the processing results of the request from the cub_cas_cgw to the application client.

The cub_gateway also manages the server load by adjusting the number of cub_cas_cgw (s) in the service pool and monitors and manages the status of the cub_cas_cgw. If the cub_gateway delivers the request to cub_cas_cgw but the connection to cub_cas_cgw 1 fails because of an abnormal termination, it sends an error message about the connection failure to the application client and restarts cub_cas_cgw 1. Restarted cub_cas_cgw 1 is now in a normal stand-by mode, and will be reconnected by a new request from a new application client.

Shared memory

The status information of the cub_cas_cgw의 is stored in the shared memory, and the cub_broker refers to this information to mediate the connection to the application client. With the status information stored in the shared memory, the system manager can identify which task the cub_cas_cgw의 is currently performing or which application client's request is currently being processed.

Start GATEWAY

Enter the command below to start the GATEWAY.

```
$ cubrid gateway start
```

The following message is returned if the GATEWAY is already running.

```
$ cubrid gateway start
@ cubrid gateway start
++ cubrid gateway is running.
```

Stopping GATEWAY

Enter the below command to stop the GATEWAY.

```
$ cubrid gateway stop
```

The following message is returned if the GATEWAY has stopped.

```
$ cubrid gateway stop
@ cubrid gateway stop
++ cubrid gateway is not running.
```

Restarting GATEWAY

Enter the below command to restart the GATEWAY.

```
$ cubrid gateway restart
```

Checking GATEWAY Status

cubrid gateway status provides various options and allows you to check the gateway status information, including the number of completed tasks for each gateway and the number of tasks waiting to be processed.

GATEWAY status is the same as broker, so refer to Checking Broker Status.

```
cubrid gateway status [options] [expr]
```

Starting the GATEWAY together when the CUBRID service starts

To have *GATEWAY* started upon startup of the CUBRID service (cubrid service start), configure *gateway* in the **service** parameter of the **cubrid.conf** file.

```
# cubrid.conf

[service]

service=server,broker,gateway,manager

...
```

CUBRID DBLINK settings

The settings for using CUBRID DBLink are different from those of homogeneous DBLink and heterogeneous DBLink.

Homogeneous DBLink Setting

If you look at the homogeneous DBlink diagram above, you need to connect to the broker in the remote database, so you need to set up the broker in the remote database. This setting is the same as the general broker setting.

Heterogeneous DBLink Setting

To connect to a heterogeneous database (Oracle/MySQL/MariaDB), cubrid_gateway.conf, unixODBC installation, and ODBC Driver information settings are required.

Gateway configuration file

The parameters used in cubrid_gateway.conf, the default gateway configuration file created when CUBRID is installed, are almost identical to the broker parameters, and some additional parameters that must be changed are included. Parameter values not included by default can be directly added/edited and used. The following is the content of the cubrid_gateway.conf file that is provided by default during installation.

The following is the content of the cubrid_gateway.conf file provided by default.

```
[gateway]
MASTER_SHM_ID           =50001
ADMIN_LOG_FILE          =log/gateway/cubrid_gateway.log

[%oracle_gateway]
SERVICE                 =OFF
SSL                     =OFF
APPL_SERVER             =CAS_CGW
BROKER_PORT             =53000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =53000
LOG_DIR                 =log/gateway/sql_log
ERROR_LOG_DIR           =log/gateway/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO
CCI_DEFAULT_AUTOCOMMIT  =ON
APPL_SERVER_MAX_SIZE    =256
CGW_LINK_SERVER         =ORACLE
CGW_LINK_SERVER_IP      =localhost
CGW_LINK_SERVER_PORT    =1521
CGW_LINK_ODBC_DRIVER_NAME   =Oracle_ODBC_Driver
CGW_LINK_CONNECT_URL_PROPERTY       =


[%mysql_gateway]
SERVICE                 =OFF
SSL                     =OFF
APPL_SERVER             =CAS_CGW
BROKER_PORT             =56000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
```

```
APPL_SERVER_SHM_ID      =56000
LOG_DIR                 =log/gateway/sql_log
ERROR_LOG_DIR           =log/gateway/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO
CCI_DEFAULT_AUTOCOMMIT  =ON
APPL_SERVER_MAX_SIZE    =256
CGW_LINK_SERVER         =MYSQL
CGW_LINK_SERVER_IP      =localhost
CGW_LINK_SERVER_PORT    =3306
CGW_LINK_ODBC_DRIVER_NAME   =MySQL_ODBC_Driver
CGW_LINK_CONNECT_URL_PROPERTY
="charset=utf8;PREFETCH=100;NO_CACHE=1"

[%mariadb_gateway]
SERVICE                 =OFF
SSL                     =OFF
APPL_SERVER             =CAS_CGW
BROKER_PORT             =59000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =59000
LOG_DIR                 =log/gateway/sql_log
ERROR_LOG_DIR           =log/gateway/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO
CCI_DEFAULT_AUTOCOMMIT  =ON
APPL_SERVER_MAX_SIZE    =256
CGW_LINK_SERVER         =MARIADB
CGW_LINK_SERVER_IP      =localhost
CGW_LINK_SERVER_PORT    =3306
CGW_LINK_ODBC_DRIVER_NAME   =MariaDB_ODBC_Driver
CGW_LINK_CONNECT_URL_PROPERTY       =
```

GATEWAY Parameter

This parameter is set to use a heterogeneous database and DBLink.

The meaning of each parameter is slightly different for each heterogeneous database.

Parameter Name	Type	Value
APPL_SERVER	string	

Parameter Name	Type	Value
CGW_LINK_SERVER	string	
CGW_LINK_SERVER_IP	string	
CGW_LINK_SERVER_PORT	int	
CGW_LINK_ODBC_DRIVER_NAME	string	
CGW_LINK_CONNECT_URL_PROPERTY	string	

APPL_SERVER

APPL_SERVER is the part that sets the application server name of the gateway and must be set as CAS_CGW.

CGW_LINK_SERVER

CGW_LINK_SERVER should set the name of the heterogeneous database to be used by connecting to CAS_CGW. Currently, supported databases are Oracle, MySQL and MariaDB.

CGW_LINK_SERVER_IP

CGW_LINK_SERVER_IP should set the IP address of the heterogeneous database to be connected with CAS_CGW.

Note

- In case of Oracle, net_service_name of tnsnames.ora is used, so this parameter is not used.
- For details, refer to Setting Connection Information for Connection to Oracle Database.

CGW_LINK_SERVER_PORT

CGW_LINK_SERVER_PORT should set the port number of databaseto be connected with CAS_CGW.

Note

- In case of Oracle, net_service_name of tnsnames.ora is used, so this parameter is not used.
- For details, refer to Setting Connection Information for Connection to Oracle Database.

CGW_LINK_ODBC_DRIVER_NAME

CGW_LINK_ODBC_DRIVER_NAME must set the ODBC Driver name provided by the heterogeneous database when connecting with CAS_CGW.

Note

- In Windows, if the ODBC driver of the heterogeneous database is installed, the driver name can be checked through the ODBC data source manager.
- In Linux, the driver name must be specified directly in odbcinit.ini.
- For details, refer to :ref: *ODBC Driver Information Settings <odbcdriver-info>*.

CGW_LINK_CONNECT_URL_PROPERTY

CGW_LINK_CONNECT_URL_PROPERTY sets the connection property used for the connection string for heterogeneous database connection.

Note

- Connection properties are different for each database, so refer to the site below.
- Oracle : https://docs.oracle.com/cd/B19306_01/server.102/b15658/app_odbc.htm#UNXAR418
- MySQL : https://dev.mysql.com/doc/connector-odbc/en/connector-odbc-configuration-connection-parameters.html#codbc-dsn-option-flags
- MariaDB : https://mariadb.com/docs/connectors/mariadb-connector-odbc/mariadb-connector-odbc-guide#general-connection-parameters

Install unixODBC

The unixODBC Driver Manager is an open source ODBC driver manager that can be used with ODBC drivers on Linux and UNIX operating systems. To use ODBC in the gateway, unixODBC must be installed.

Note

- In Windows, you can use Microsoft® ODBC Data Source Manager, which is installed by default.

How to install unixODBC

```
$ wget https://www.unixodbc.org/unixODBC-2.3.9.tar.gz
$ tar xvf unixODBC-2.3.9.tar.gz
$ cd unixODBC-2.3.9
$ ./configure
$ make
$ make install
```

Note

For instructions on how to install the unixODBC driver manager, refer to the url below.

unixODBC website: https://www.unixodbc.org/

ODBC Driver information setting

After unixODBC is installed, the ODBC Driver information of the database to be connected must be registered.

ODBC Driver information is registered by directly modifying odbcinst.ini.

The following is an example of setting MySQL, Oracle and MariaDB ODBC Driver information.

```
[MySQL ODBC 8.0 Unicode Driver]
Description = MySQL ODBC driver v8.0
Driver=/usr/lib64/libmyodbc8w.so

[Oracle 11g ODBC driver]
Description = Oracle ODBC driver v11g
Driver = /home/user/oracle/instantclient/libsqora.so.19.1
```

```
[mariadb odbc 3.1.13 driver]
Description= mariadb odbc driver 3.1.13
Driver=/home/user/mariadb-odbc-3.1.13/lib64/mariadb/libmaodbc.so
```

Note

For reference, in the example above, the driver names are “MySQL ODBC 8.0 Unicode Driver”, “Oracle 11g ODBC driver” and “mariadb odbc 3.1.13 driver” respectively.

Oracle Setting for DBLink

Oracle Environment Configuration

To use Oracle in DBLink, you must install and configure Oracle Instant Client, set connection information, set Oracle Database environment variables, and set gateway.

Install Oracle Instant Client ODBC

Download ODBC Package and Basic Package from the Oracle Instant Client download site and extract them to the same directory.

```
unzip instantclient-basic-linux.x64-19.20.0.0.0dbru.zip
unzip instantclient-odbc-linux.x64-19.20.0.0.0dbru.zip
```

Oracle Instant Client Download Site: https://www.oracle.com/database/technologies/instant-client/downloads.html

Oracle Instant Client Environment Variable Settings

```
export ORACLE_INSTANT_CLIENT=/home/user/oracle/instantclient
export PATH=$ORACLE_INSTANT_CLIENT:$PATH
export LD_LIBRARY_PATH=$ORACLE_INSTANT_CLIENT:$LD_LIBRARY_PATH
```

Set connection information for connection to Oracle Database

In order to connect to Oracle Database, the tnsnames.ora file containing connection information must be modified. Connection information should be written in these three items: HOST, PORT, and SERVICE_NAME in the basic format below. For the tnsnames.ora file in which connection information is created, the directory path must be set in the TNS_ADMIN environment variable. For how to set TNS_ADMIN, refer to “Setting TNS_ADMIN Environment Variable”.

Default format of the tnsnames.ora file

```
net_service_name =
  (DESCRIPTION=
        (ADDRESS = (PROTOCOL = TCP)(HOST = xxx.xxx.xxx.xxx)(PORT =
1521)
  )
  (CONNECT_DATA =
        (SERVICE_NAME=service_name)
  )
)
```

- net_service_name: The name of the net service for database connection, and the name used for the db_name of the connection url.
- HOST: IP address or server name to connect to the database.
- PORT: Port required for connection. In most cases, the default port is 1521.
- SERVICE_NAME: The name of the database to connect to.

Note

For reference, no error occurs even if net_service_name is duplicated. However, since it may be connected to a different server than the intended one, net_service_name must be set so that it does not overlap.

Oracle Database Environment Variable Settings

The following environment variables must be set in the Oracle database server.

```
export ORACLE_SID=XE
export ORACLE_BASE=/u01/app/oracle
export ORACLE_HOME=$ORACLE_BASE/product/11.2.0/xe
export PATH=$ORACLE_HOME/bin:$PATH
```

- ORACLE_SID is the system identifier.
- ORACLE_BASE is the Oracle base directory.
- ORACLE_HOME is the path where the Oracle database is installed.

TNS_ADMIN environment variable setting

TNS_ADMIN points to the directory path where the tnsnames.ora file is located. If there is a tnsnames.ora file in /home/user/myconfigs, you can settings it as follows.

```
export TNS_ADMIN=/home/user/myconfigs
```

Configuring cubrid_gateway.conf for Oracle

In order to connect to Oracle from the gateway, several settings are required as follows.

For details, refer to Gateway configuration file.

Because the gateway uses the information in tnsnames.ora to connect to oracle, it is not necessary to write CGW_LINK_SERVER_IP and CGW_LINK_SERVER_PORT. Even if the corresponding information is written, the gateway does not refer to it.

```
APPL_SERVER                     =CAS_CGW
                .
                .
                .
CGW_LINK_SERVER                 =ORACLE
CGW_LINK_ODBC_DRIVER_NAME   =Oracle 19c ODBC driver
CGW_LINK_CONNECT_URL_PROPERTY =
```

MySQL Configuration for DBLink

MySQL Environment Configuration

Install MySQL ODBC Driver

MySQL Unicode ODBC Driver is required to connect to MySQL from the gateway. The following is how to install MySQL ODBC Driver.

Use the MySQL Yum repository to provide the Connector/ODBC RPM package. The MySQL Yum repository should be in your system's list of repositories, If not, select the package for your platform from the MySQL Yum repository download page (https://dev.mysql.com/downloads/repo/yum/) and download it.

Install the downloaded release package.

```
$ sudo yum install mysql80-community-release-el6-{version-
number}.noarch.rpm
```

Update the repository using yum.

```
$ sudo yum update mysql-community-release
```

Install Connector/ODBC with the command below.

```
$ sudo yum install mysql-connector-odbc
```

For detailed installation instructions, refer to https://dev.mysql.com/doc/connector-odbc/en/connector-odbc-installation-binary-yum.html.

Configuring cubrid_gateway.conf for MySQL

In order to connect to MySQL from the gateway, several settings are required as below.

For details, refer to gateway configuration file.

```
APPL_SERVER                 =CAS_CGW
                .
                .
                .
CGW_LINK_SERVER                 =MYSQL
CGW_LINK_SERVER_IP          =localhost
CGW_LINK_SERVER_PORT        =3306
CGW_LINK_ODBC_DRIVER_NAME   =MySQL ODBC 8.0 Unicode Driver
CGW_LINK_CONNECT_URL_PROPERTY ="charset=utf8;PREFETCH=100;NO_CACHE=1"
```

MariaDB Configuration for DBLink

MariaDB Environment Configuration

Install MariaDB ODBC Driver

MariaDB ODBC Driver is required to connect to MariaDB from the gateway.

The following is how to install MariaDB ODBC Driver.

MariaDB Connector/ODBC package can be downloaded by selecting the version from the page below.

https://mariadb.com/downloads/connectors/

Extract the files from the downloaded tarball package. Then, install the driver's shared library in an appropriate location in the system. The installed driver must register driver information in odbcinst.ini. For settings method, refer to :ref: *ODBC Driver Information Settings <odbcdriver-info>*.

```
$ mariadb-connector-odbc-3.1.13-centos7-amd64.tar.gz -C mariadb-
odbc-3.1.13
```

For detailed installation instructions, refer to https://mariadb.com/docs/connectors/mariadb-connector-odbc/mariadb-connector-odbc-guide#installing-mariadb-connector-odbc-on-linux .

Configuring cubrid_gateway.conf for MariaDB

In order to connect to MariaDB from the gateway, several settings are required as below.

For details, refer to gateway configuration file.

```
APPL_SERVER                 =CAS_CGW
                .
                .
                .
CGW_LINK_SERVER                 =MARIADB
CGW_LINK_SERVER_IP          =localhost
CGW_LINK_SERVER_PORT        =3306
CGW_LINK_ODBC_DRIVER_NAME   =mariadb odbc 3.1.13 driver
```

How to use CUBRID DBLink

If the information on brokers of CUBRID to be connected to use DBLink has been completed or gateway settings for heterogeneous databases have been completed, let's learn how to write a query statement using DBLink.

There are two ways to write DBLINK Query statement for data inquiry.

First, how to query information from other databases by writing DBLINK syntax in the FROM clause

The Query statement below is a Query statement that inquires the remote_t table information of another database of IP 192.xxx.xxx.xxx.

```
SELECT * FROM DBLINK ('192.xxx.xxx.xxx:
53000:testdb:user:password:','SELECT col1, col2 FROM remote_t') AS
t(col1 int, col2 varchar(32));
```

Note

In case of Oracle, ip and port are gateway connection information among remote connection information, and net_service_name of tnsnames.ora must be entered in the db_name field. If net_service_name is ora_test, write as follow

SELECT * FROM DBLINK ('192.xxx.xxx.xxx:53000:ora_test:user:password:','SELECT col1, col2 FROM remote_t') AS t(col1 int, col2 varchar(32));

Second, use the CREATE SERVER statement to protect user information (id, password) and the hassle of writing connection information every time you write a Query. If the CREATE SERVER statement is used, the Query statement becomes concise and helps to protect user information.

```
CREATE SERVER remote_srv1 ( HOST='192.xxx.xxx.xxx', PORT=53000,
DBNAME=testdb, USER=user, PASSWORD='password');
SELECT * FROM DBLINK (remote_srv1, 'SELECT col1 FROM remote_t') AS
t(col1 int);
```

Note

DBLINK can set additional connection properties in the connection URL. For detailed attribute details, refer to CCI driver's cci_connect_with_url function

If the DBLINK target database is configured as an HA environment, you can use the altHosts attribute to set it as shown in the example below.

192.168.0.1:53000:testdb:user:password::?altHosts=192.168.0.2:33000,192.168.0.3:33000

In the example, if the 192.168.0.1 server is an active database and cannot connect to the server, it is the setting to request a connection to the 192.168.0.2 server. As in the example above, multiple altHosts can be specified, and connections are attempted in the order listed.

For detailed DBLink SQL syntax, refer to SELECT and SERVER DEFINITION STATEMENTS.

Notice

- Synonym creation: Local synonyms can be created for remote tables and remote synonyms. In the case of DBMSs other than CUBRID, the username or db name must be added to the original table name.

```
-- for CUBRID
create synonym synonym_1 for t1@srv1;
create synonym synonym_2 for remote_synonym@srv1;

-- for ORACLE
create synonym synonym_ora for user_ora.t1@srv_ora;

-- for MySQL and MariaDB
```

```
create synonym synonym_my for my_db.t1@srv_mysql;
create synonym synonym_maria for maria_db.t1@srv_maria;
```

- The reserved word processing character for CUBRID is the double quotation mark (" ") as like the following.

```
SELECT ["COLUMN"],["ADD"],["ALTER"] FROM ["TABLE"]@srv1 ;
SELECT * FROM dblink(srv1, 'select "COLUMN","ADD","ALTER" from
"TABLE" ') AS t(a varchar, b varchar, c varchar );
```

- The reserved word processing character for ORACLE is the double quotation mark (" ") as like the following.

```
SELECT ["COLUMN"],["ADD"],["ALTER"] FROM ["TABLE"]@srv1 ;
SELECT * FROM dblink(srv1, 'select "COLUMN","ADD","ALTER" from
"TABLE" ') AS t(a varchar, b varchar, c varchar );
```

- The reserved word processing character for MySQL and MariaDB is the backquote (` `) as like the following.

```
SELECT [`COLUMN`],[`ADD`],[`ALTER`] FROM [`TABLE`]@srv1 ;
SELECT * FROM dblink(srv1, 'select `COLUMN`,`ADD`,`ALTER` from
`TABLE` ') AS t(a varchar, b varchar, c varchar );
```

Constraints

Common Constraints

- The charset of the remote database must be unicode (utf-8).
- **Table extension style (object@server)**
    - Supports only tables, views, and synonyms
    - Serial, built-in functions, and stored functions are not supported.

      (Example: stored function sp_func() of remote server (server1) cannot be used in sp_func@server1(arg1, ...) format.)
- All functions (stored functions, built-in functions including SYSDATE), serial-related functions and system constants in the SELECT query all operate locally. (If you need to execute a function or serial in a remote DB, you should use the DBLINK statement.) For example, when the select

query for remote table is requested and the optimizer rewrite query as below, you look the queries in DBLINK() are only executed in the remote DB.

```
SELECT A.*, rownum rn, '' empty, null null_col, SYSDATE
FROM t1@srv1 A ;

-- rewritten query
SELECT A.id, A.parentid, A.[text], rownum, '', null, SYS_DATE -- at
local
FROM ( SELECT [_dbl].id, [_dbl].parentid, [_dbl].[text]
       FROM DBLINK( srv1 /* '192.168.1.125:33000:remotedb1:dba:*:'
*/ ,
                'SELECT * FROM tree A') AS [_dbl](id integer, parentid
integer, [text] varchar(32)) -- at remote
     ) A (id, parentid, [text])
```

- All functions, serial-related functions, and system constants of INSERT/UPDATE/DELETE/MERGE queries are all executed on the remote server, so be careful when using built-in functions (i.e., CUBRID's built-in functions may not be supported by the remote DBMS or have different usage instructions)

- Transaction: Transactions (commit, rollback) between local DB and remote DB are not processed as one transaction. DML (INSERT/UPDATE/DELETE/MERGE) queries in the remote DB are excuted as autocommits separately from transactions in the local DB. As in the example below, when performing a transaction, data is inserted into the remote DB, but no data is entered into the local DB, because doing rollback.

```
-- local input
INSERT INTO t1(a, b) VALUES (1, 'local');

-- remote input
INSERT INTO t2@srv1(a, b) VALUES (1, 'remote');

rollback;

SELECT a, b FROM t1, t2@srv1 t2 WHERE t1.a = t2.a;
there's no result
```

- TRUNCATE statement is not supported.

- CREATE TABLE ... LIKE table@server syntax is not supported (For reference, CREATE TABLE ... AS SELECT FROM table@server syntax is supported)

- DBLINK() and remote table (@server) are not allowed in the TRIGGER statement.

- predicate push: The SELECT statement written in table extension style (@server) is internally rewritten to DBLINK() by the optimizer. To improve performance, It push with conditional

clauses that can be performed in the remote DB. However, if a built-in function or user-defined function in a conditional clause is used, it is excluded from pushing.

- Performance notes

> **Note**
>
> When the connect by clause, group by clause, having clause, and limit clause are used in the SELECT statement in table extension style (@server), the where condition, group by clause, having clause, and limit clause are not executed in the remote DB. After executing exclude the statement in the remote DB and sending the performed results to the local DB, performance may be slowed due to execute the statement in the local DB.

Below is an example of processing the "*group by*" and "*count()*" after transferring all data from the remote DB's *tree* table to the local DB.

```
-- original query
SELECT A.parentid, count()
FROM tree@srv1 A
GROUP BY A.parentid ;

-- rewritten query
SELECT A.parentid, count()
FROM ( SELECT [_dbl].parentid
       FROM DBLINK( srv1 /* '192.168.1.125:33000:remotedb1:dba::'
*/,
                    'SELECT parentid FROM tree A'
                  ) AS [_dbl](parentid integer)
     ) A (parentid)
GROUP BY A.parentid
```

- The SYSDATE function used in the table extension style (@server) is performed in the local DB, so caution is required if the time between servers is different.

```
-- original query
SELECT * FROM tbl@srv1 WHERE col1 >= sysdate;

-- rewritten query
SELECT *
FROM ( SELECT col1, col2
       FROM DBLINK( srv1 /* '192.168.1.125:33000:remotedb1:dba::'
*/,
                    'SELECT col1, col2 FROM tbl'
                  ) AS [_dbl](col1 date, col2 varchar)
```

```
      ) tbl (col1, col2)
WHERE col1>= SYS_DATE
```

- When using a scalar subquery, subquery and EXIST clause with a co-related condition in the table extension style (@server), the remote query brings the whole data to the local DB every time and performs to find data corresponding to the join column. As a result, a rapid decrease in performance occurs. The example below uses T1.a as a condition for a scalar subquery, and all data as T1.a < 4 of svr1's tree table is sent to the local DB to find suitable data per executing scalar subquey, so it may degrade performance.

```
-- original query
SELECT T1.a,
       (SELECT A.text FROM tree@srv1 A WHERE A.id = T1.a )
remote_text
FROM hangul_t1 T1
WHERE T1.a < 4;

-- rewritten query
SELECT T1.a,
       (SELECT A.[text] from (select [_dbl].[text], [_dbl].id
        FROM DBLINK(srv1 /* '192.168.1.125:33000:remotedb1:dba::'
*/,
                    'SELECT [text], id FROM tree A'
                   ) AS [_dbl]([text] varchar(32), id integer)
        WHERE [_dbl].id=T1.a) A ([text], id))
FROM hangul_t1 T1
WERE (T1.a < ?:0)
```

CUBRID Constraints

- ENUM, BLOB, CLOB, and SET types are not supported in the select statement.
- If the system parameters of the local DB and remote DB are different, undesirable results may occur.

Common constraints of Heterogeneous DBMS.

- The gateway must use the Unicode-only ODBC Driver of the heterogeneous remote database (Oracle/MySQL/MariaDB).
- Among ODBC types, SQL_INTERVAL, SQL_GUID, SQL_BIT, SQL_BINARY, SQL_VARBINARY, SQL_LONGVARBINARY, SQL_LONGVARCHAR, and SQL_WLONGVARCHAR are not supported types.

ODBC SQL Type	Oracle Data Type	MySQL Data Type	MariaDB Data Type
SQL_LONGVARCHAR	LONG, CLOB	LONGTEXT	LONGTEXT
SQL_WLONGVARCHAR	NCLOB		
SQL_BIT		BIT	BIT
SQL_BINARY		BINARY	BINARY
SQL_VARBINARY	RAW	VARBINARY	VARBINARY
SQL_LONGVARBINARY	LONG RAW	LONG VARBINARY	LONG VARBINARY
	BLOB	BLOB	BLOB
	BFILE	TINYBLOB	TINYBLOB
		MEDIUMBLOB	MEDIUMBLOB
		LONGBLOB	LONGBLOB
		GEOMETRY	GEOMETRY
		POINT	POINT
		POLGON	POLGON

ODBC SQL Type	Oracle Data Type	MySQL Data Type	MariaDB Data Type
		GEOMETRYCOLLECTION	GEOMETRYCOLLECTION
		MULTILINESTRING	MULTILINESTRING
		MULTIPOINT	MULTIPOINT
		MULTIPOLYGON	MULTIPOLYGON
SQL_INTERVAL	INTERVAL YEAR TO MONTH		
	INTERVAL DAY TO SECOND		

- The maximum string length for one column is 16M. The table below shows the data type for each DBMS with a data length of 16M or more.

DBMS Name	Data Type
Oracle	LONG, NCLOB, CLOB
MySQL	LONGTEXT
MariaDB	LONGTEXT

- In DML statements such as INSERT, UPDATE, DELETE, and MERGE, if the built-in functions not supported by CUBRID and are not in the form of function (parameter 1, ..., parameter N) as below, the query results error.

  Example: the convert function of MySQL and MariaDB: convert('binary' using binary)

Note

Oracle Constraints

- LONG, INTERVAL DAY TO SECOND, INTERVAL YEAR TO MONTH, BLOB, and CLOB types are not supported in the select statement. For more information, refer to Unsupported Types of Common constraints of Heterogeneous DBMS
- The INTERVAL DAY TO SECOND and INTERVAL YEAR TO MONTH types are not supported by Oracle ODBC. For more information, see "Using the Oracle ODBC Driver" (https://docs.oracle.com/en/database/oracle/oracle-database/19/adfns/odbc-driver.html#GUID-3FE69BEF-F8D2-4152-9B1A-877186C47028).
- Oracle ODBC does not support the timezone type, so when SELECTing timezone data, the timezone is calculated as a local time, converted to timestamp type, and returned.

Below is an example of converting Oracle DB's timezone data to a local timezone when querying it with ODBC. The entered timezone is "+02:00", converted to local timezone "+09:00", and output as "PM 08:00".

```
-- oracle input
INSERT INTO tbl VALUES (to_timestamp_tz('2021-07-25 12:34:56
+02:00', 'yyyy-mm-dd hh24:mi:ss tzh:tzm'));

-- local
SELECT t_timestamp_timezone2 FROM tbl@server;
07:34:56.000 PM 07/25/2021

SELECT to_char(t_timestamp_timezone2, 'yyyy-mm-dd hh24:mi:ss.ff
tzh:tzm') FROM tbl@server;
2021-07-25 19:34:56.000 +09:00
```

- REPLACE syntax is not supported and an error occurs when used.
- Oracle's processing range of date and number types is larger than CUBRID's, so data that falls outside of CUBRID's type range will result in an error.

Note

MySQL/MariaDB limitations

- When using cache in Mysql, the memory usage of gateway cub_cas_cgw increases, so it is recommended to use PREFETCH, NO_CACHE=1.
- When performing a query that includes the repeat() function in MySQL/MariaDB, part of the string may be truncated or the string may not be read.
- LONGTEXT, BIT, BLOB, and LONGBLOB types are not supported in the select statement. For more information, refer to Unsupported Types of Common constraints of Heterogeneous DBMS

Class Inheritance

To explain the concept of inheritance, a table is expressed as a class, a column is expressed as an attribute, and a type is expressed as a domain.

Classes in CUBRID database can have class hierarchy. Attributes and methods can be inherited through such hierarchy. As shown in the previous section, you can create a *Manager* class by inheriting attributes from an *Employee* class. The *Manager* class is called the sub class of the *Employee* class, and the *Employee* class is called the super class of the *Manager* class. Inheritance can simplify class creation by reusing the existing class hierarchy.

CUBRID allows multiple inheritances, which means that a class can inherit attributes and methods from more than one super class. However, inheritance can cause conflicts when an attribute or method of the super class is added or deleted.

Such conflict occurs in multiple inheritance if there are attributes or methods with the same name in different super classes. For example, if it is likely that a class inherits attributes of the same name and type from more than one super class, you must specify the attributes to be inherited. In such a case, if the inherited super class is deleted, a new attribute of the same name and type must be inherited from another super class. In most cases, the database system resolves such problems automatically. However, if you don't like the way that the system resolves a problem, you can resolve it manually by using the INHERIT clause.

When attributes are inherited from more than one super class, it is possible that their names are to be the same, while their domains are different. For example, two super classes may have the same attribute, whose domain is a class. In this case, a sub class automatically inherits attributes with more specialized (a lower in the class hierarchy) domains. If such conflict occurs between basic data types (e.g. STRING or INTEGER) provided by the system, inheritance fails.

Conflicts during inheritance and their resolutions will be covered in the Resolving Class Conflicts section.

Note

The following cautions must be observed during inheritance:

- The class name must be unique in the database. An error occurs if you create a class that inherits another class that does not exist.
- The name of a method/attribute must be unique within a class. The name cannot contain spaces, and cannot be a reserved keyword of CUBRID. Alphabets as well as '_', '#', '%' are allowed in the class name, but the first character cannot be '_'. Class names are not case-sensitive. A class name will be stored in the system after being converted to lowercase characters.
- Encryption option (TDE) of the class is not inherited. For more information, see Table Encryption (TDE).

Note

A super class name can begin with the username so that the owner of the class can be easily identified.

Class Attribute and Method

You can create class attributes to store the aggregate property of all instances in the class. When you define a **CLASS** attribute or method, you must precede the attribute or method name with the keyword **CLASS**. Because a class attribute is associated with the class itself, not with an instances of the class, it has only one value. For example, a class attribute can be used to store the average value determined by a class method or the timestamp when the class was created. A class method is executed in the class object itself. It can be used to calculate the aggregate value for the instances of the class.

When a sub class inherits a super class, each class has a separate storage space for class attributes, so that two classes may have different values of class attribute. Therefore, the sub class does not change even when the attributes of the super class are changed.

The name of a class attribute can be the same as that of an instance attribute of the same class. Likewise, the name of a class method can be the same as that of an instance method of the same class.

Order Rule for Inheritance

The following rules apply to inheritance. The term class is generally used to describe the inheritance relationship between classes and virtual classes in the database.

- For an object without a super class, attributes are defined in the same order as in the **CREATE** statement (an ANSI standard).
- If there is one super class, locally created attributes are placed after the super class attributes. The order of the attributes inherited from the super class follows the one defined during the super class definition. For multiple inheritance, the order of the super class attributes is determined by the order of the super classes specified during the class definition.
- If more than one super class inherits the same class, the attribute that exists in both super classes is inherited to the sub class only once. At this time, if a conflict occurs, the attribute of the first super class is inherited.
- If a name conflict occurs in more than one super class, you can inherit only the ones you want from the super class attributes by using the **INHERIT** clause in order to resolve the conflict.
- If the name of the super class attribute is changed by the alias option of the **INHERIT** clause, its position is maintained.

INHERIT Clause

When a class is created as a sub class, the class inherits all attributes and methods of the super class. A name conflict that occurs during inheritance can be handled by either a system or a user. To resolve the name conflict directly, add the **INHERIT** clause to the **CREATE CLASS** statement.

```
CREATE CLASS
.
.
.
INHERIT resolution [{, resolution}] ;

resolution:
[column_name | method_name] OF [schema_name.]superclass_name [AS
alias]
```

In the **INHERIT** clause, specify the name of the attribute or method of the super class to inherit. With the **ALIAS** clause, you can resolve a name conflict that occurs in multiple inheritance statements by inheriting a new name.

ADD SUPERCLASS Clause

To extend class inheritance, add a super class to a class. A relationship between two classes is created when a super class is added to an existing class. Adding a super class does not mean adding a new class.

```
ALTER CLASS
.
.
.
ADD SUPERCLASS [schema_name.]superclass_name [{,
[schema_name.]superclass_name}]
[INHERIT resolution [{, resolution}]] ;

resolution:
[column_name | method_name] OF [schema_name.]superclass_name [AS
alias]
```

Specify the name of the superclass to be added in *superclass_name*. Attributes and methods of the super class can be inherited by using the syntax above.

Name conflicts can occur when adding a new super class. If a name conflict cannot be resolved by the database system, attributes or methods to inherit from the super class can be specified by using the **INHERIT** clause. You can use aliases to inherit all attributes or methods that cause the conflict. For details on super class name conflicts, see Class Conflict Resolution.

The following example shows how to create the *female_event* class by inheriting the *event* class included in *demodb*.

```
CREATE CLASS female_event UNDER event;
```

DROP SUPERCLASS Clause

Deleting a super class from a class means removing the relationship between two classes. If a super class is deleted from a class, it changes inheritance relationship of the classes as well as of all their sub classes.

```
ALTER CLASS
.
.
.
DROP SUPERCLASS [schema_name.]superclass_name [{,
[schema_name.]superclass_name}]
[INHERIT resolution [{, resolution}]] ;

resolution:
```

```
[column_name | method_name] OF [schema_name.]superclass_name [AS
alias]
```

Specify the name of the superclass to be deleted in *superclass_name*. For the second *class_name*, specify the name of the super class to be deleted. If a name conflict occurs after deleting a super class, see the Class Conflict Resolution section for the resolution.

The following example shows how to inherit the *female_event* class from the *event* class.

```
CREATE CLASS female_event UNDER event
```

The following example shows how to delete the super class *event* from the f *emale_event* class. Attributes that the *female_event* class inherited from the even *class* no longer exist.

```
ALTER CLASS female_event DROP SUPERCLASS event;
```

Class Conflict Resolution

If you modify the schema of the database, conflicts can occur between attributes or methods of inheritance classes. Most conflicts are resolved automatically by CUBRID otherwise, you must resolve the conflict manually. Therefore, you need to examine the possibility of conflicts before modifying the schema.

Two types of conflicts can cause damage to the database schema. One is conflict with a sub class when the sub class schema is modified. The other is conflict with a super class when the super class is modified. The following are operations that may cause conflicts between classes.

- Adding an attribute
- Deleting an attribute
- Adding a super class
- Deleting a super class
- Deleting a class

If a conflict occurs as a result of the above operations, CUBRID applies a basic resolution to the sub class where the conflict occurred. Therefore, the database schema can always maintain consistent state.

Resolution Specifier

Conflicts between the existing classes or attributes, and inheritance conflicts can occur if the database schema is modified. If the system fails to resolve a conflict automatically or if you don't like the way the system resolved the problem, you can suggest how to resolve the conflict by using the **INHERIT** clause of the **ALTER** statement (often referred to as resolution specifier).

When the system resolves the conflict automatically, basically, the existing inheritance is maintained (if any). If the previous resolution becomes invalid when the schema is modified, the system will arbitrarily select another one. Therefore, you must avoid excessive reuse of attributes or methods in the schema design stage because the way the system will resolve the conflict cannot always be predictable.

What will be discussed concerning conflicts is applied commonly to both attributes and methods.

```
ALTER [class_type] [schema_name.]class_name alter_clause
[INHERIT resolution [{, resolution }]] ;

resolution:
[column_name | method_name] OF [schema_name.]superclass_name [AS
alias]
```

Superclass Conflict

Adding a super class

The **INHERIT** clause of the **ALTER CLASS** statement is optional, but must be used when a conflict occurs due to class changes. You can specify more than one resolution after the **INHERIT** clause.

superclass_name specifies the name of the super class that has the new attribute(column) or method to inherit when a conflict occurs. *column_name* or *method_name* specifies the name of the attribute or method to inherit. You can use the **AS** clause when you need to change the name of the attribute or method to inherit.

The following example shows how to create the *soccer_stadium* class by inheriting the *event* and *stadium* classes in the *olympic* database of *demodb*. Because both *event* and *stadium* classes have the name and code attributes, you must specify the attributes to inherit using the **INHERIT** clause.

```
CREATE CLASS soccer_stadium UNDER event, stadium
INHERIT name OF stadium, code OF stadium;
```

When the two super classes (*event* and *stadium*) have the *name* attribute, if the *soccer_stadium* class needs to inherit both attributes, it can inherit the *name* unchanged from the *stadium* class and the *name* changed from the *event* class by using the **alias** clause of the **INHERIT**.

The following example shows in which the *name* attribute of the *stadium* class is inherited as it is, and that of the *event* class is inherited as the *purpose* alias.

```
ALTER CLASS soccer_stadium
INHERIT name OF event AS purpose;
```

Deleting a super class

A name conflict may occur again if a super class that explicitly inherited an attribute or method is dropped by using the **INHERIT**. In this case, you must specify the attribute or method to be explicitly inherited when dropping the super class.

```
CREATE CLASS a_tbl(a INT PRIMARY KEY, b INT);
CREATE CLASS b_tbl(a INT PRIMARY KEY, b INT, c INT);
CREATE CLASS c_tbl(b INT PRIMARY KEY, d INT);

CREATE CLASS a_b_c UNDER a_tbl, b_tbl, c_tbl INHERIT a OF b_tbl, b
OF b_tbl;

ALTER CLASS a_b_c
DROP SUPERCLASS b_tbl
INHERIT b OF a_tbl;
```

The above example shows how to create the *a_b_c* class by inheriting *a_tbl*, *b_tbl* and *c_tbl* classes, and delete the *b_tbl* class from the super class. Because *a* and *b* are explicitly inherited from the *b_tbl* class, you must resolve their name conflicts before deleting it from the super class. However, *a* does not need to be specified explicitly because it exists only in the *a_tbl* class except for the *b_tbl* class to be deleted.

Compatible Domains

If the conflicting attributes do not have compatible domains, the class hierarchy cannot be created.

For example, the class that inherits a super class with the *phone* attribute of integer type cannot have another super class with the *phone* attribute of string type. If the types of the *phone* attributes of the two super classes are both String or Integer, you can add a new super class by resolving the conflict with the **INHERIT** clause.

Compatibility is checked when inheriting an attribute with the same name, but with the different domain. In this case, the attribute that has a lower class in the class inheritance hierarchy as the domain is automatically inherited. If the domains of the attributes to inherit are compatible, the conflict must be resolved in the class where an inheritance relationship is defined.

Sub class Conflict

Any changes in a class will be automatically propagated to all sub classes. If a problem occurs in the sub class due to the changes, CUBRID resolves the corresponding sub class conflict and then displays a message saying that the conflict has been resolved automatically by the system.

Sub class conflicts can occur due to operations such as adding a super class, or creating/deleting a method or an attribute. Any changes in a class will affect all sub classes. Since changes are automatically propagated, harmless changes can even cause side effects in sub classes.

Adding Attributes and Methods

The simplest sub class conflict occurs when an attribute is added. A sub class conflict occurs if an attribute added to a super class has the same name as one already inherited by another super class. In such cases, CUBRID will automatically resolve the problem. That is, the added attribute will not be inherited to all sub classes that have already inherited the attribute with the same name.

The following example shows how to add an attribute to the *event* class. The super classes of the *soccer_stadium* class are the *event* and the *stadium* classes, and the *nation_code* attribute already exists in the *stadium* class. Therefore, a conflict occurs in the *soccer_stadium* class if the *nation_code* attribute is added to the *event* class. However, CUBRID resolves this conflict automatically.

```
ALTER CLASS event
ADD ATTRIBUTE nation_code CHAR(3);
```

If the *event* class is dropped from the *soccer_stadium* super class, the *cost* attribute of the *stadium* class will be inherited automatically.

Dropping Attributes and Methods

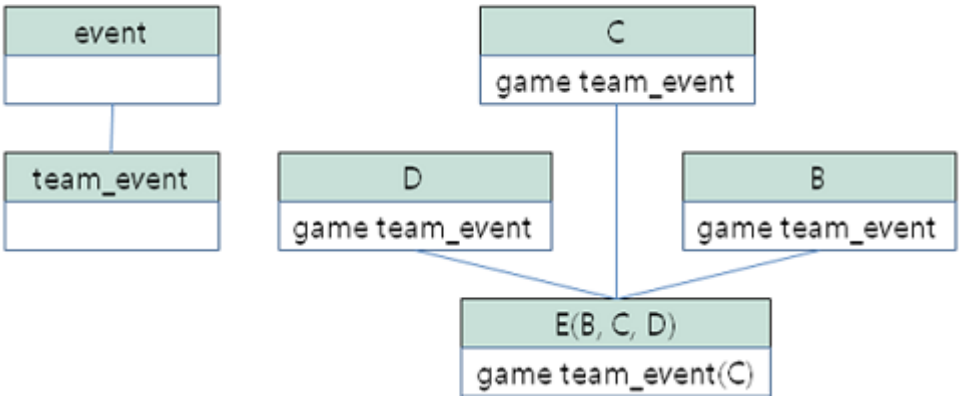
When an attribute is dropped from a class, any resolution specifiers which refer to the attribute by using the **INHERIT** clause are also removed. If a conflict occurs due to the deletion of an attribute, the system will determine a new inheritance hierarchy. If you don't like the inheritance hierarchy determined by the system, you can determine it by using the **INHERIT** clause of the **ALTER** statement. The following example shows such conflict.

Suppose there is a sub class that inherits attributes from three different super classes. If a name conflict occurs in all super classes and the explicitly inherited attribute is dropped, one of the remaining two attributes will be inherited automatically to resolve the problem.

The following example shows sub class conflict. Classes *B*, *C* and *D* are super classes of class *E*, and have an attribute whose name is *team* and the domain is *team_event*. Class *E* was created with the *place* attribute inherited from class *C* as follows:

```
create class E under B, C, D
inherit place of C;
```

In this case, the inheritance hierarchy is as follows:



Suppose that you decide to delete class *C* from the super class. This drop will require changes to the inheritance hierarchy. Because the domains of the remaining classes *B* and *D* with the *game* attribute are at the same level, the system will randomly choose to inherit from one of the two classes. If you don't want the system to make a random selection, you can specify the class to inherit from by using the **INHERIT** clause when you change the class.

```
ALTER CLASS E INHERIT game OF D;
ALTER CLASS C DROP game;
```

Note

If the domain of one *game* attribute in one super class is *event* and that of another super class is *team_event*, *team_event* is more specific than event because *team_event* is the descendant of event. Therefore, a super class that has the *team_event* attribute as a domain will be inherited; a user cannot forcefully inherit a super class that has the *event* attribute as a domain.

Schema Invariant

Invariants of a database schema are a property of the schema that must be preserved consistently (before and after the schema change). There are four types of invariants: invariants of class hierarchy, name, inheritance and consistency.

Invariant of class hierarchy

has a single root and defines a class hierarchy as a Directed Acyclic Graph (DAG) where all connected classes have a single direction. That is, all classes except the root have one or more super classes, and cannot become their own super classes. The root of DAG is “object,” a system-defined class.

Invariant of name

means that all classes in the class hierarchy and all attributes in a class must have unique names. That is, attempts to create classes with the same name or to create attributes or methods with the same name in a single class are not allowed.

Invariant of name is redefined by the ‘RENAME’ qualifier. The ‘RENAME’ qualifier allows the name of an attribute or method to be changed.

Invariant of inheritance

means that a class must inherit all attributes and methods from all super classes. This invariant can be distinguished with three qualifiers: source, conflict and domain. The names of inherited attributes and methods can be modified. For default or shared value attributes, the default or shared value can be modified. Invariant of inheritance means that such changes will be propagated to all classes that inherit these attributes and methods.

- **source qualifier**

  means that if class *C* inherits sub classes of class *S*, only one of the sub class attributes (methods) inherited from class *S* can be inherited to class *C*. That is, if an attribute (method) defined in class *S* is inherited by other classes, it is in effect a single attribute (method), even though it exists in many sub classes. Therefore, if a class multiply inherits from classes that have attributes (methods) of the same source, only one appearance of the attribute (method) is inherited.

- **conflict qualifier**

  means that if class *C* inherits from two or more classes that have attributes (methods) with the same name but of different sources, it can inherit more than one class. To inherit attributes (methods) with the same name, you must change their names so as not to violate the invariant of name.

- **domain qualifier**

  means that a domain of an inherited attribute can be converted to the domain's sub class.

Invariant of consistency

means that the database schema must always follow the invariants of a schema and all rules except when it is being changed.

Rule for Schema Changes

The Invariants of a Schema section has described the characteristics of schema that must be preserved all the time.

There are some methods for changing schemas, and all these methods must be able to preserve the invariants of a schema. For example, suppose that in a class which has a single super class, the relationship with the super class is to be removed. If the relationship with the super class is removed, the class becomes a direct sub class of the object class, or the removal attempt will be rejected if the user specified that the class should have at least one super class. To have some rules for selecting one of the methods for changing schemas, even though such selection seems arbitrary, will be definitely useful to users and database designers.

The following three types of rules apply: conflict-resolution rules, domain-change rule and class-hierarchy rule.

Seven conflict-resolution rules reinforce the invariant of inheritance. Most schema change rules are needed because of name conflicts. A domain-change rule reinforces a domain resolution of the invariant of inheritance. A class-hierarchy rule reinforces the invariant of class hierarchy.

Conflict-Resolution Rules

- **Rule 1**: If an attribute (method) name of class *C* and an attribute name of the super class *S* conflict with each other (that is, their names are same), the attribute of class *C* is used. The attribute of *S* is not inherited.

  If a class has one or more super classes, three aspects of the attribute (method) of each super class must be considered to determine whether the attributes are semantically equal and which attribute to inherit. The three aspects of the attribute (method) are the name, domain and source. The following table shows eight combinations of these three aspects that can happen with two super classes. In Case 1 (two different super classes have attributes with the same name, domain and source), only one of the two sub classes should be inherited because two attributes are identical. In Case 8 (two different super classes have attributes with different names, domains and sources), both classes should be inherited because two attributes are totally different ones.

Case	Name	Domain	Source
1	Same	Same	Same
2	Same	Same	Different
3	Same	Different	Same
4	Same	Different	Different
5	Different	Same	Same
6	Different	Same	Different
7	Different	Different	Same
8	Different	Different	Different

Five cases (1, 5, 6, 7, 8) out of eight have clear meaning. Invariant of inheritance is a guideline for resolving conflicts in such cases. In other cases (2, 3, 4), it is very difficult to resolve conflicts automatically. Rules 2 and 3 can be resolutions for these conflicts.

- **Rule 2**: When two or more super classes have attributes (methods) with different sources but the same name and domain, one or more attributes (methods) can be inherited if the conflict-resolution statement is used. If the conflict-resolution statement is not used, the system will select and inherit one of the two attributes.

  This rule is a guideline for resolving conflicts of Case 2 in the table above.

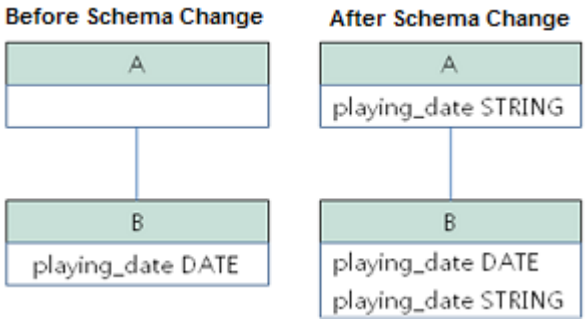
- **Rule 3**: If two or more super classes have attributes with different sources and domains but the same name, attributes (methods) with more detailed (lower in the inheritance hierarchy) domains are inherited. If there is no inheritance relationship between domains, schema change is not allowed.

  This rule is a guideline for resolving conflicts of Case 3 and 4. If Case 3 and 4 conflict with each other, Case 3 has the priority.

- **Rule 4**: The user can make any changes except the ones in Case 3 and 4. In addition, the resolution of sub class conflicts cannot cause changes in the super class.

  The philosophy of Rule 4 is that "an inheritance is a privilege that sub class has obtained from a super class, so changes in a sub class cannot affect the super class." Rule 4 means that the name of the attribute (method) included in the super class cannot be changed to resolve conflicts between class *C* and super classes. Rule 4 has an exception in cases where the schema change causes conflicts in Case 3 and 4.

  - For example, suppose that class *A* is the super class of class *B*, and class B has the playing_date attribute of **DATE** type. If an attribute of **STRING** type named *playing_date* is added to class *A*, it conflicts with the *playing_date* attribute in class *B*. This is what happens in Case 4. The precise way to resolve such conflict is for the user to specify that class *B* must inherit the *playing_date* attribute of class *A*. If a method refers to the attribute, the user of class *B* needs to modify the method properly so that the appropriate *playing_date* attribute will be referenced. Schema change of class *A* is not allowed because the schema falls into an inconsistent state if the user of class *B* does not describe an explicit statement to resolve the conflict occurring from the schema change.



- **Rule 5**: If a conflict occurs due to a schema change of the super class, the original resolution is maintained as long as the change does not violate the rules. However, if the original resolution becomes invalid due to the schema change, the system will apply another resolution.

  Rule 5 is for cases where a conflict is caused to a conflict-free class or where the original resolution becomes invalid.

  This is the case where the name or domain of an attribute (method) is modified or a super class is deleted when the attribute (method) is added to the super class or the one inherited from the super class is deleted. The philosophy of Rule 5 coincides with that of Rule 4. That is, the user can change the class freely without considering what effects the sub class that inherits from the given class will have on the inherited attribute (method).

  When you change the schema of class *C*, if you decide to inherit an attribute of the class due to an earlier conflict with another class, this may cause attribute (method) loss of class *C*. Instead, you must inherit one of the attributes (methods) that caused conflicts earlier.

  The schema change of the super class can cause a conflict between the attribute (method) of the super class and the (locally declared or inherited) attribute (method) of class *C*. In this

case, the system resolves the conflict automatically by applying Rule 2 or 3 and may inform the user.

Rule 5 cannot be applied to cases where a new conflict occurs due to the addition or deletion of the relationship with the super class. The addition/deletion of a super class must be limited to within the class. That is, the user must provide an explicit resolution.

- **Rule 6**: Changes of attributes or methods are propagated only to sub classes without conflicts.

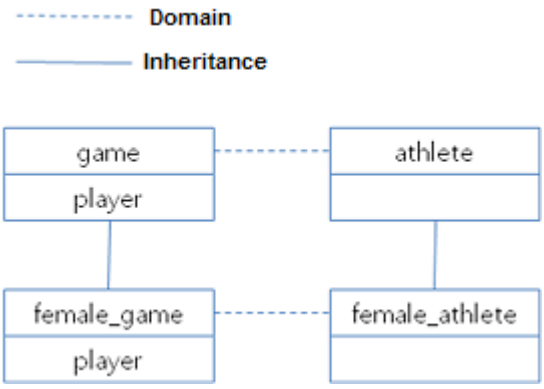
  This rule limits the application of Rule 5 and the invariant of inheritance. Conflicts can be detected and resolved by applying Rule 2 and 3.

- **Rule 7**: Class *C* can be dropped even when an attribute of class *R* uses class *C* as a domain. In this case, the domain of the attribute that uses class *C* as a domain can be changed to *object*.

Domain-Change Rules

- **Rule 8**: If the domain of an attribute of class *C* is changed from *D* to a super class of *D*, the new domain is less generic than the corresponding domain in the super class from which class *C* inherited the attribute. The following example explains the principle of this rule.

  Suppose that in the database there are the *game* class with the *player* attribute and the *female_game* class which inherits game. The domain of the player attribute of the *game* class is the *athlete* class, but the domain of the player attribute of the *female_game* class is changed to *female_athlete* which is a sub class of *athlete*. The following diagram shows such relationship. The domain of the *player* attribute of the *female_game* class can be changed back to *athlete*, which is the super class of *female_athlete*.



Class-Hierarchy Rules

- **Rule 9**: A class without a super class becomes a direct sub class of object. The class-hierarchy rule defines characteristics of classes without super classes. If you create a class without a super class, object becomes the super class. If you delete the super class *S*, which is a unique super class of class *C*, class *C* becomes a direct sub class of object.

Database Administration

User Management

Database User

To know the username's writing rule, see Identifier.

CUBRID has two types of users by default: **DBA** and **PUBLIC**. At the initial installation of the product, no passwords are set for these users.

- All database users are automatically members of **PUBLIC**, so they inherit any authorization granted to the **PUBLIC** user. Granting authorization to **PUBLIC** means granting it to all users.
- The **DBA** user has the authorization of the database administrator. The **DBA** automatically becomes the member of all users and groups. That is, the **DBA** is granted the access for all tables. Therefore, there is no need to grant authorization explicitly to the **DBA** and **DBA** members. Each database user has a unique name. The database administrator can create multiple users simultaneously using the **cubrid createdb** utility (see cubrid Utilities for details). A database user cannot have a member who already has the same authorization. If authorization is granted to a user, all members of the user is automatically granted the same authorization.

Note

From version 11.3 of CUBRID, **DBA** members can also query objects of all users in the system catalog virtual class.

CREATE USER

You can create a user using the CREATE USER statement. The default DBA, PUBLIC user is created without a password.

```
CREATE USER user_name
[PASSWORD password]
[GROUPS user_name [{, user_name } ... ]]
```

```
[MEMBERS user_name [{, user_name } ... ]]
[COMMENT 'comment_string'];
```

- *user_name*: specifies the username to create.
- *password*: specifies the user password to create.
- *comment_string*: specifies the user comment to create.

> **Note**
>
> Only **DBA** and **DBA** members can create users using the CREATE USER statement.

The following example creates a user test_user1 and specifies a password.

```
CREATE USER test_user1 PASSWORD 'password';
```

The following example shows how to create a user and add member to the user. By the following statement, *company* becomes a group that has *engineering*, *marketing* and *design* as its members. *marketing* becomes a group with members *smith* and *jones*, *design* becomes a group with a member *smith*, and *engineering* becomes a group with a member *brown*.

```
CREATE USER company;
CREATE USER engineering GROUPS company;
CREATE USER marketing GROUPS company;
CREATE USER design GROUPS company;
CREATE USER smith GROUPS design, marketing;
CREATE USER jones GROUPS marketing;
CREATE USER brown GROUPS engineering;
```

The following example shows how to create the same groups as above but use the **MEMBERS** keyword instead of **GROUPS**.

```
CREATE USER smith;
CREATE USER brown;
CREATE USER jones;
CREATE USER engineering MEMBERS brown;
CREATE USER marketing MEMBERS smith, jones;
CREATE USER design MEMBERS smith;
CREATE USER company MEMBERS engineering, marketing, design;
```

User's COMMENT

The following example creates a user test_user1 and adds a password and a comment.

```
CREATE USER test_user1 PASSWORD 'password' COMMENT 'new user';
```

You can see a comment for a user with this syntax.

```
SELECT name, comment FROM db_user;
```

To change user comment, refer to the description of the ALTER USER statement.

ALTER USER

You can use the ALTER USER statement to change the password, members and comment of a created user.

```
ALTER USER user_name
[PASSWORD password] |
[ADD MEMBERS user_name [{, user_name } ... ]] |
[DROP MEMBERS user_name [{, user_name } ... ]]
[COMMENT 'comment_string'];
```

- *user_name*: specifies the username to change.
- *password*: specifies the user password to change.
- *comment_string*: specifies the user comment to change.

Note

DBA and **DBA** members can use the ALTER USER statement to change the password, members and comment of **all users**.

General users can change **their own** password, member and comment using the ALTER USER statement.

The following example creates a user test_user1 and changes the password.

```
CREATE USER test_user1;
ALTER USER test_user1 PASSWORD '1234';
```

The following example creates a user and adds members to the created user. This example does the same thing as the example in CREATE USER .. MEMBERS ..

```
CREATE USER company;
CREATE USER engineering;
CREATE USER marketing;
CREATE USER design;
CREATE USER smith;
CREATE USER jones;
CREATE USER brown;

ALTER USER engineering ADD MEMBERS brown;
ALTER USER marketing ADD MEMBERS smith, jones;
ALTER USER design ADD MEMBERS smith;
ALTER USER company ADD MEMBERS engineering, marketing, design;
```

The following example deletes the members of a created user group. The *marketing* member is deleted from the *company* group through the following sentence and the *marketing* group deletes *smith* and *jones* from the member.

```
ALTER USER company DROP MEMBERS marketing;
ALTER USER marketing DROP MEMBERS smith, jones;
```

User's COMMENT Change

The following example changes the comment for the created user.

```
CREATE USER test_user1 COMMENT 'new user';
ALTER USER test_user1 COMMENT 'old user';
```

DROP USER

You can delete a user using the DROP USER statement. Users who own objects in table, view, trigger, stored function/procedure, serial, synonym, and server cannot be dropped.

```
DROP USER user_name;
```

- *user_name*: specifies the username to delete.

Note

Only **DBA** and **DBA** members can delete users using the DROP USER statement.

The following example shows how to create a user (*test_user1*), change a password, and delete the user.

```
CREATE USER test_user1;
ALTER USER test_user1 PASSWORD '1234';
DROP USER test_user1;
```

GRANT

In CUBRID, the following database objects can be granted permissions:

- **Table**
- **View**
- **Procedure**

To allow other users (groups) to access the database objects you created, you must grant them the appropriate permissions. All members of a group with granted permissions have the same permissions, so there is no need to grant permissions individually to each member. However, members of a group with the WITH GRANT OPTION permission, who are not part of the DBA or owner group, cannot grant the received permissions to other users.

Database objects created by the **PUBLIC** user are accessible to all users.

You can grant access permissions to a user using the following **GRANT** statement:

```
(1) Table and View:
    GRANT operation [ { ,operation } ... ] ON
[schema_name.]object_name [ { , [schema_name.]object_name } ... ]
    TO user [ { ,user } ... ] [ WITH GRANT OPTION ];

(2) Stored Procedure and Function:
```

```
GRANT EXECUTE ON PROCEDURE [schema_name.]object_name
TO user [ { ,user } ... ];
```

- *operation*: Specifies an operation that can be used when granting authorization. The following table shows operations.
    - 1. Table and View
        - **SELECT**: Allows reading the table definitions and retrieving instances. The most common type of permission.
        - **INSERT**: Allows creating instances in the table.
        - **UPDATE**: Allows modifying instances already existing in the table.
        - **DELETE**: Allows deleting instances in the table.
        - **ALTER**: Allows modifying the table definition, renaming, or deleting the table.
        - **INDEX**: Allows creating indexes on columns to improve search speed.
        - **EXECUTE**: Allows calling table methods or instance methods.
        - **ALL PRIVILEGES**: Includes all the above-mentioned permissions.
    - 1. Stored Procedure and Function
        - **EXECUTE ON PROCEDURE**: Allows calling a stored procedure or function.
- *schema_name*: Specifies the schema name of the table or virtual table. If omitted, the schema name of the current session is used.
- *table_name*: Specifies the name of a table or virtual table to be granted.
- *user*: Specifies the name of a user (group) to be granted. Enter the login name of the database user or **PUBLIC**, a system-defined user. If **PUBLIC** is specified, all database users are granted with the permission.
- **WITH GRANT OPTION**
    - The **WITH GRANT OPTION** allows the grantee of authorization to grant that same authorization to another user.
    - The **EXECUTE ON PROCEDURE** permission for stored procedures and functions does not support the **WITH GRANT OPTION**.

The following example shows how to grant the **SELECT** authorization for the *olympic* table to *smith* (including his members).

```
GRANT SELECT ON olympic TO smith;
```

The following example shows how to grant the **SELECT**, **INSERT**, **UPDATE** and **DELETE** authorization on the *nation* and *athlete* tables to *brown* and *jones* (including their members).

```
GRANT SELECT, INSERT, UPDATE, DELETE ON nation, athlete TO  brown,
jones;
```

The following example shows how to grant every authorization on the *tbl1* and *tbl2* tables to all users(public).

```
CREATE TABLE tbl1 (a INT);
CREATE TABLE tbl2 (a INT);
GRANT ALL PRIVILEGES ON tbl1, tbl2 TO public;
```

The following **GRANT** statement example shows how to grant retrieval authorization on the *record* and *history* tables to *brown*. Using **WITH GRANT OPTION** allows *brown* to grant retrieval authorization to other users. *brown* can grant authorization to others within their own authorization.

```
GRANT SELECT ON record, history TO brown WITH GRANT OPTION;
```

Note

- The grantor of authorization must be the owner of all tables listed before the grant operation or have **WITH GRANT OPTION** specified.
- Before granting **SELECT**, **UPDATE**, **DELETE** and **INSERT** authorization for a virtual table, the owner of the virtual table must have **SELECT** and **GRANT** authorization for all the tables included in the query specification. The **DBA** user and the members of the **DBA** group are automatically granted all authorization for all tables.
- To execute the **TRUNCATE** statement, the **ALTER**, **INDEX**, and **DELETE** authorization is **required**.

The following example shows how to grant the permission to execute the *my_sp* procedure to *smith*.

```
CREATE OR REPLACE PROCEDURE my_sp ()
IS
BEGIN
        DBMS_OUTPUT.put_line('grant test');
END;
GRANT EXECUTE ON PROCEDURE my_sp TO smith;
```

Note

- The **GRANT** statement can only grant permission to one object at a time for stored procedures and functions.
- The **EXECUTE ON PROCEDURE** operation is used for both stored procedures and functions.
- Currently, the **WITH GRANT OPTION** is not supported for granting permissions on stored procedures and functions, and will return a syntax error if used.

REVOKE

You can revoke authorization using the **REVOKE** statement. The authorization granted to a user can be revoked at any time. If more than one type of authorization is granted to a user, all or part of the authorization can be revoked. Additionally, even if a single **GRANT** statement is used to grant authorization on multiple database objects to multiple users, selective revocation of authorization for specific users and specific database objects is possible.

If the authorization is revoked from the grantor (**WITH GRANT OPTION**), the authorization granted to the grantee by that grantor is also revoked.

The following **REVOKE** statement can be used to revoke the authorization granted to a user.

```
(1) For tables and views:
    REVOKE operation [ { ,operation } ... ] ON
[schema_name.]object_name [ { , [schema_name.]object_name } ... ]
    FROM user [ { ,user } ... ];

(2) For stored procedures and functions:
    REVOKE EXECUTE ON PROCEDURE [schema_name.]object_name
    FROM user [ { ,user } ... ];
```

- *operation*: Indicates the type of operation that can be revoked (see GRANT for details).

- *schema_name*: Specifies the schema name of the database object. If omitted, the schema name of the current session is used.
- *object_name*: Specifies the name of the database object for which the authorization is to be revoked.
- *user*: Specifies the name of the user or group from which the authorization is to be revoked.

The following example shows how to grant **SELECT**, **INSERT**, **UPDATE** and **DELETE** authorization to *smith* and *jones* so that they can perform on the *nation* and *athlete* tables.

```
GRANT SELECT, INSERT, UPDATE, DELETE ON nation, athlete TO smith,
jones;
```

The following example shows how to execute the **REVOKE** statement; this allows *jones* to have only **SELECT** authorization. If *jones* has granted authorization to another user, the user is also allowed to execute **SELECT** only.

```
REVOKE INSERT, UPDATE, DELETE ON nation, athlete FROM jones;
```

The following example shows how to execute the **REVOKE** statement revoking all authorization that has granted to *smith*. *smith* is not allowed to execute any operations on the *nation* and *athlete* tables once this statement is executed.

```
REVOKE ALL PRIVILEGES ON nation, athlete FROM smith;
```

ALTER ... OWNER

Database Administrator (**DBA**) or a member of the **DBA** group can change the owner of table, view, trigger, stored functions/procedures, and serial by using the following query.

```
ALTER (TABLE | CLASS | VIEW | VCLASS | TRIGGER | PROCEDURE |
FUNCTION | SERIAL) [schema_name.]name OWNER TO user_id;
```

- *schema_name*: Specifies the schema of the object. If omitted, the schema name of the current session is used.
- *name*: The name of schema object of which owner is to be changed
- *user_id*: User ID

```
ALTER TABLE test_tbl OWNER TO public;
ALTER VIEW test_view OWNER TO public;
ALTER TRIGGER test_trigger OWNER TO public;
ALTER FUNCTION test_function OWNER TO public;
ALTER PROCEDURE test_procedure OWNER TO public;
ALTER SERIAL test_serial OWNER TO public;
```

⚠ Warning

When the owner changes, privileges granted by the previous owner are transferred to the new owner. **However, privileges on the object that were previously granted to the new owner by the previous owner are automatically revoked.**

User Authorization Management METHOD

The database administrator (**DBA**) can check and modify user authorization by calling authorization-related methods defined in **db_user** where information about database user is stored, or **db_authorizations** (the system authorization class). The administrator can specify **db_user** or **db_authorizations** depending on the method to be called, and store the return value of a method to a variable. In addition, some methods can be called only by **DBA** or members of **DBA** group.

ℹ Note

Note that method call made by the master node is not applied to the slave node in the HA environment.

```
CALL method_definition ON CLASS auth_class [ TO variable ] [ ; ]
CALL method_definition ON variable [ ; ]
```

login( ) method

As a class method of **db_user** class, this method is used to change the users who are currently connected to the database. The name and password of a new user to connect are given as arguments, and they must be string type. If there is no password, a blank character (‘’) can be used as the argument. **DBA** and **DBA** members can call the **login( )** method without a password.

```
-- Connect as DBA user who has no password
CALL login ('dba', '') ON CLASS db_user;

-- Connect as a user_1 whose password is cubrid
CALL login ('user_1', 'cubrid') ON CLASS db_user;
```

add_user( ) method

As a class method of **db_user** class, this method is used to add a new user. The name and password of a new user to add are given as arguments, and they must be string type. At this time, the new username should not duplicate any username already registered in a database. The **add_user( )** can be called only by **DBA** or members of **DBA** group.

```
-- Add user_2 who has no password
CALL add_user ('user_3', '') ON CLASS db_user;

-- Add user_3 who has no password, and store the return value of a
method into an admin variable
CALL add_user ('user_2', '') ON CLASS db_user to admin;
```

drop_user( ) method

As a class method of **db_user** class, this method is used to drop an existing user. Only the username to be dropped is given as an argument, and it must be a string type. However, the owner of a class cannot be dropped thus **DBA** needs to specify a new owner of the class before dropping the user. The **drop_user( )** method can be also called only by **DBA** or members of **DBA**.

```
-- Delete user_2
CALL drop_user ('user_2') ON CLASS db_user;
```

find_user( ) method

As a class method of **db_user** class, this method is used to find a user who is given as an argument. The name of a user to be found is given as an argument, and the return value of the method is stored into a variable that follows 'to'. The stored value can be used in a next query execution.

```
-- Find user_2 and store it into a variable called 'admin'
CALL find_user ('user_2') ON CLASS db_user to admin;
```

set_password( ) method

This method is an instance method that can call each user instance, and it is used to change a user's password. The new password of a specified user is given as an argument. General users other than **DBA** and **DBA** group members can only change their own passwords.

```
-- Add user_4 and store it into a variable called user_common
CALL add_user ('user_4','') ON CLASS db_user to user_common;

-- Change the password of user_4 to 'abcdef'
CALL set_password('abcdef') on user_common;
```

change_owner() method

As a class method of **db_authorizations** class, this method is used to change the owner of a class. The name of a class for which you want to change the owner, and the name of a new owner are given as arguments. The table name must be prefixed with the schema name. If omitted, the schema name of the current session is used. At this time, the class and owner that are specified as an argument must exist in a database. Otherwise, an error occurs. **change_owner( )** can be called only by **DBA** or members of **DBA** group. The **ALTER ... OWNER** query has the same role as the method. See ALTER ... OWNER.

```
-- Change the owner of table_1 owned by user_1 to user_4
CALL change_owner ('user_1.table_1', 'user_4') ON CLASS
db_authorizations;
```

The following example shows a **CALL** statement that calls the find_user method defined in the system table **db_user**. It is called to determine whether the database user entered as the **find_user** exists. The first statement calls the table method defined in the **db_user** class. The name (**db_user** in this case) is stored in x if the user is registered in the database. Otherwise, **NULL** is stored.

The second statement outputs the value stored in the variable x. In this query statement, the **DB_ROOT** is a system class that can have only one record. It can be used to output the value of sys_date or other registered variables. For this purpose, the **DB_ROOT** can be replaced by another table having only one record.

```
CALL find_user('dba') ON CLASS db_user to x;
```

```
Result
======================
db_user
```

```
SELECT x FROM db_root;
```

```
x
======================
db_user
```

With **find_user**, you can determine if the user exists in the database depending on whether the return value is **NULL** or not.

SET

The **SET** statement is the syntax that specifies a system parameter's value or user-defined variables.

System Parameter

You can define a value of a system parameter with SQL syntax by CSQL interpreter or a query editor of CUBRID Manager. However, be cautious that updatable parameters are limited. For the information of updatable parameters, see cubrid.conf Configuration File and Default Parameters.

```
SET SYSTEM PARAMETERS 'parameter_name=value [{; name=value}...]'
```

DEFAULT for *value* will reset the parameter to its default value with an exception of **call_stack_dump_activation_list** parameter.

```
SET SYSTEM PARAMETERS 'lock_timeout=DEFAULT';
```

User Variables

You can create user-defined variables in two ways. One is to use the **SET** statement and the other is to use the assignment statement of user-defined variables within SQL statements. You can delete the user-defined variables that you defined with the **DEALLOCATE** or the **DROP** statements.

The user-defined variables are also called session variables as they are used for maintaining connections within one application. The user-defined variables are used within the part of a connection session, and the user-defined variables defined by an application cannot be accessed by other applications. When an application terminates connections, all variables will be removed automatically. The user-defined variables are limited to twenty per connection session for an application. If you already have twenty user-defined variables and want to define a new user-defined variable, you must remove some variables with the **DROP VARIABLE** statement.

You can use user-defined variables in most SQL statements. If you define user-defined variables and refer to them in one statement, the sequence is not guaranteed. That is, if you refer to the

variables specified in the **SELECT** list of the **HAVING**, **GROUP BY** or **ORDER BY** clause, you may not get the values in the sequence you expect. You cannot also use user-defined variables as identifiers, such as column names or table names within SQL statements

The user-defined variables are not case-sensitive. The user-defined variable type can be one of the **SHORT**, **INTEGER**, **BIGINT**, **FLOAT**, **DOUBLE**, **NUMERIC**, **CHAR**, **VARCHAR**, **BIT** and **BIT VARYING**. Other types will be converted to the **VARCHAR** type.

```
SET @v1 = 1, @v2=CAST(1 AS BIGINT), @v3 = '123', @v4 =
DATE'2010-01-01';

SELECT typeof(@v1), typeof(@v2), typeof(@v3), typeof(@v4);
```

```
   typeof(@v1)          typeof(@v2)          typeof(@v3)
typeof(@v4)
=============================================================================
=================
  'integer'            'bigint'             'character (-1)'
'character varying (1073741823)'
```

The user-defined variables can be changed when you define values.

```
SET @v = 'a';
SET @v1 = 10;

SELECT @v := 1, typeof(@v1), @v1:='1', typeof(@v1);
```

```
  @v := 1                typeof(@v1)          @v1 := '1'
typeof(@v1)
=============================================================================
=================
  1                    'integer'            '1'
'character (-1)'
```

```
<set_statement>
        : <set_statement>, <udf_assignment>
        | SET <udv_assignment>
        ;

<udv_assignment>
        : @<name> = <expression>
        | @<name> := <expression>
        ;

{DEALLOCATE|DROP} VARIABLE <variable_name_list>
```

```
<variable_name_list>
       : <variable_name_list> ',' @<name>
```

- You must define the variable names with alphanumeric characters and underscores (_).
- When you define the variables within SQL statements, you should use the ':=' operator.

The following example shows how to define the variable a and assign a value 1 to it.

```
SET @a = 1;
SELECT @a;
```

```
  @a
======================
  1
```

The following example shows how to count the number of rows in the **SELECT** statement by using the user-defined variable.

```
CREATE TABLE t (i INTEGER);
INSERT INTO t(i) VALUES(2),(4),(6),(8);

SET @a = 0;

SELECT @a := @a+1 AS row_no, i FROM t;
```

```
  row_no                          i
 ==================================
  1                               2
  2                               4
  3                               6
  4                               8

4 rows selected.
```

The following example shows how to use the user-defined variable as the input of bind parameter specified in the prepared statement.

```
SET @a:=3;

PREPARE stmt FROM 'SELECT i FROM t WHERE i < ?';
EXECUTE stmt USING @a;
```

```
            i
=============
            2
```

The following example shows how to declare the user-defined variable by using the ':=' operator.

```
SELECT @a := 1, @user_defined_variable := 'user defined variable';
UPDATE t SET i = (@var := 1);
```

The following example shows how to delete the user-defined variable *a* and *user_defined_variable*.

```
DEALLOCATE VARIABLE @a, @user_defined_variable;
DROP VARIABLE @a, @user_defined_variable;
```

Note

The user-defined variables that are defined by the **SET** statement start by connecting an application to a server and will be maintained until the application terminates the connection. The connection maintained during this period is called a session. When an application terminates the connection or when there are no requests for a certain period of time, the session will expire, and the user-defined variables will be deleted as a result. You can set the session time with the **session_state_timeout** parameter of **cubrid.conf**; the default value is **21600** seconds (=6 hours).

The data managed by the session includes **PREPARE** statements, the user-defined variables, the last ID inserted (**LAST_INSERT_ID**) and the number of rows affected by the statement that you execute at the end (**ROW_COUNT**).

KILL

The **KILL** statement terminates transactions with an option **TRANSACTION** or **QUERY** modifier.

```
KILL [TRANSACTION | QUERY] tran_index, ... ;
```

- **KILL TRANSACTION** is default of **KILL** statement. It is the same as **KILL** without a modifier. It terminates the connection associated with the given *tran_index*.

- **KILL QUERY** terminates the statement that the transaction is executing.

DBA and a user of DBA group are able to terminate all the transactions of the system, while non-DBA users are only allowed to terminate their own transaction.

```
KILL TRANSACTION 1;
```

SHOW

Contents

SHOW 1194

- DESC, DESCRIBE 1195
- EXPLAIN 1196
- SHOW TABLES 1196
- SHOW COLUMNS 1197
- SHOW INDEX 1201
- SHOW COLLATION 1204
- SHOW TIMEZONES 1206
- SHOW GRANTS 1209
- SHOW CREATE TABLE 1209
- SHOW CREATE VIEW 1210
- SHOW ACCESS STATUS 1210
- SHOW EXEC STATISTICS 1211

- Diagnostics 1214
  - SHOW VOLUME HEADER 1214
  - SHOW LOG HEADER 1217
  - SHOW ARCHIVE LOG HEADER 1222
  - SHOW HEAP HEADER 1224
  - SHOW HEAP CAPACITY 1230
  - SHOW SLOTTED PAGE HEADER 1233
  - SHOW SLOTTED PAGE SLOTS 1235
  - SHOW INDEX HEADER 1237
  - SHOW INDEX CAPACITY 1239
  - SHOW CRITICAL SECTIONS 1244
  - SHOW TRANSACTION TABLES 1247
  - SHOW THREADS 1255
  - SHOW JOB QUEUES 1261
  - SHOW PAGE BUFFER STATUS 1261

DESC, DESCRIBE

It shows the column information of a table, and it's like a **SHOW COLUMNS** statement. For more details, see SHOW COLUMNS.

```
DESC [schema_name.]table_name;
DESCRIBE [schema_name.]table_name;
```

EXPLAIN

It shows the column information of a table, and it's like a **SHOW COLUMNS** statement. For more details, see SHOW COLUMNS.

```
EXPLAIN [schema_name.]table_name;
```

SHOW TABLES

It shows the list of all table names within a database. The name of the result column will be *tables_in_<database name>* and it will have one column. If you use the **LIKE** clause, you can search the table names matching this and if you use the **WHERE** clause, you can search table names with more general terms. **SHOW FULL TABLES** displays the *owner* column and the *table_type* column together. The *owner* column has the owner name as a value. *table_type* columns have **BASE TABLE** values for tables and **VIEW** for views.

```
SHOW [FULL] TABLES [LIKE 'pattern' | WHERE expr];
```

The following shows the examples of this syntax.

```
SHOW TABLES;
```

```
Tables_in_demodb
======================
  'athlete'
  'code'
  'event'
  'game'
  'history'
  'nation'
  'olympic'
  'participant'
  'record'
  'stadium'
```

```
SHOW FULL TABLES;
```

```
  Tables_in_demodb      Owner                 Table_type
=============================================================
  'athlete'             'PUBLIC'              'BASE TABLE'
  'code'                'PUBLIC'              'BASE TABLE'
  'event'               'PUBLIC'              'BASE TABLE'
```

```
  'game'                'PUBLIC'              'BASE TABLE'
  'history'             'PUBLIC'              'BASE TABLE'
  'nation'              'PUBLIC'              'BASE TABLE'
  'olympic'             'PUBLIC'              'BASE TABLE'
  'participant'         'PUBLIC'              'BASE TABLE'
  'record'              'PUBLIC'              'BASE TABLE'
  'stadium'             'PUBLIC'              'BASE TABLE'
```

```
SHOW FULL TABLES LIKE '%c%';
```

```
  Tables_in_demodb      Owner                 Table_type
================================================================
  'code'                'PUBLIC'              'BASE TABLE'
  'olympic'             'PUBLIC'              'BASE TABLE'
  'participant'         'PUBLIC'              'BASE TABLE'
  'record'              'PUBLIC'              'BASE TABLE'
```

```
SHOW FULL TABLES WHERE table_type = 'BASE TABLE' and
TABLES_IN_demodb LIKE '%co%';
```

```
  Tables_in_demodb      Owner                 Table_type
================================================================
  'code'                'PUBLIC'              'BASE TABLE'
  'record'              'PUBLIC'              'BASE TABLE'
```

SHOW COLUMNS

It shows the column information of a table. You can use the **LIKE** clause to search the column names matching it. If you use the **WHERE** clause, you can search column names with more general terms like, “General Considerations for All **SHOW** Statements.”.

```
SHOW [FULL] COLUMNS (FROM | IN) [schema_name.]table_name [LIKE
'pattern' | WHERE expr];
```

If a **FULL** keyword is used, it shows the additional information, **collation** and **comment**.

The **DESCRIBE** (abbreviated **DESC**) statement and the **EXPLAIN** statement provide the same information with **SHOW COLUMNS**, but they don’t support LIKE clause or WHERE clause.

This query has the following columns:

Column name	Type	Description
Field	VARCHAR	Column name
Type	VARCHAR	Column data type
Null	VARCHAR	If you can store **NULL**, the value is YES; if not, it is NO
Key	VARCHAR	**Whether a column has an index or not. If there is more than one key value in the given column of a table, this displays only the one that appears first in the order of PRI, UNI and MUL.** • If the key is a space, the column doesn't have an index, it is not the first column in the multiple column index or the index is non-unique. • If the value is PRI, it is a primary key or the primary key of multiple columns. • If the value is UNI, it is a unique index. (The unique index allows multiple NULL values but you can also set a NOT NULL constraint.) • If the value is MUL, it is the first column

Column name	Type	Description
		of the non-unique index that allows the given value to be displayed in the column several times. If the column composes a composite unique index, the value will be MUL. The combination of column values can be unique but the value of each column can appear several times.
Default	VARCHAR	Default value defined in the column
Extra	VARCHAR	Additional information available on the given column. For the column with **AUTO_INCREMENT** constraint, it shows the 'auto_increment'.

The following shows the examples of this syntax.

```
SHOW COLUMNS FROM athlete;
```

```
  Field                 Type                 Null
Key          Default               Extra
================================================================================
  'code'                'INTEGER'            'NO'
'PRI'        NULL                 'auto_increment'
  'name'                'VARCHAR(40)'        'NO'
```

```
''            NULL                    ''
  'gender'              'CHAR(1)'             'YES'
''            NULL                    ''
  'nation_code'         'CHAR(3)'             'YES'
''            NULL                    ''
  'event'               'VARCHAR(30)'         'YES'
''            NULL                    ''
```

```
SHOW COLUMNS FROM athlete WHERE field LIKE '%c%';
```

```
  Field                 Type                  Null
Key           Default               Extra
================================================================================
==========================================
  'code'                'INTEGER'             'NO'
'PRI'         NULL                  'auto_increment'
  'nation_code'         'CHAR(3)'             'YES'
''            NULL                  ''
```

```
SHOW COLUMNS FROM athlete  WHERE "type" = 'INTEGER' and "key"='PRI'
AND extra='auto_increment';
```

```
  Field                 Type                  Null
Key           Default               Extra
================================================================================
==========================================
  'code'                'INTEGER'             'NO'
'PRI'         NULL                  'auto_increment'
```

```
SHOW FULL COLUMNS FROM athlete WHERE field LIKE '%c%';
```

```
  Field                   Type                  Collation
Null                  Key                   Default
Extra                 Comment
================================================================================
================================================================================
====================================
  'code'                  'INTEGER'             NULL
'NO'                  'PRI'                 NULL
'auto_increment'      NULL
  'nation_code'           'CHAR(3)'             'iso88591_bin'
'YES'                 ''                    NULL
''                    NULL
```

SHOW INDEX

It shows the index information.

```
SHOW (INDEX | INDEXES | KEYS) (FROM | IN) [schema_name.]table_name;
```

This query has the following columns:

Column name	Type	Description
Table	VARCHAR	Table name
Non_unique	INTEGER	**Unique or not** • 0: Duplicated value is not allowed • 1: Duplicated value is allowed
Key_name	VARCHAR	Index name
Seq_in_index	INTEGER	Serial number of the column in the index. Starts from 1.
Column_name	VARCHAR	Column name
Collation	VARCHAR	Method of sorting columns in the index. 'A' means ascending and **NULL** means not sorted.
Cardinality	INTEGER	The number of values measuring the unique values in the index. The higher the cardinality, the

Column name	Type	Description
		more efficient the index search. This value is updated every time **SHOW INDEX** is executed. Note that this is an approximate value.
Sub_part	INTEGER	The number of bytes of the indexed characters if the columns are indexed partially. **NULL** if all columns are indexed.
Packed		Shows how keys are packed. If they are not packed, it will be **NULL**. Currently no support.
Null	VARCHAR	YES if a column can include **NULL**, NO if not.
Index_type	VARCHAR	Index to be used (currently, only the BTREE is supported.)
Func	VARCHAR	A function which is used in a function-based index
Comment	VARCHAR	Comment to describe the index
Visible	VARCHAR	Shows the Visibility of an index (YES/NO)

The following shows the examples of this syntax.

```
SHOW INDEX IN athlete;
```

```
  Table             Non_unique  Key_name           Seq_in_index
Column_name  Collation  Cardinality  Sub_part  Packed  Null
Index_type  func  Comment  Visible
================================================================================
================================================================================
=======================
  'public.athlete'           0  'pk_athlete_code'             1
'code'       'A'              6677      NULL  NULL    'NO'
'BTREE'     NULL  NULL     'YES'
```

```
CREATE TABLE tbl1 (i1 INTEGER , i2 INTEGER NOT NULL, i3 INTEGER
UNIQUE, s1 VARCHAR(10), s2 VARCHAR(10), s3 VARCHAR(10) UNIQUE);

CREATE INDEX i_tbl1_i1 ON tbl1 (i1 DESC);
CREATE INDEX i_tbl1_s1 ON tbl1 (s1 (7));
CREATE INDEX i_tbl1_i1_s1 ON tbl1 (i1, s1);
CREATE UNIQUE INDEX i_tbl1_i2_s2 ON tbl1 (i2, s2);

ALTER INDEX i_tbl1_s1 ON tbl1 INVISIBLE;

SHOW INDEXES FROM tbl1;
```

```
  Table          Non_unique  Key_name        Seq_in_index
Column_name  Collation  Cardinality  Sub_part  Packed  Null
Index_type  Func  Comment  Visible
================================================================================
================================================================================
===================
  'public.tbl1'           1  'i_tbl1_i1'              1
'i1'         'D'                 0      NULL  NULL    'YES'
'BTREE'     NULL  NULL     'YES'
  'public.tbl1'           1  'i_tbl1_i1_s1'            1
'i1'         'A'                 0      NULL  NULL    'YES'
'BTREE'     NULL  NULL     'YES'
  'public.tbl1'           1  'i_tbl1_i1_s1'            2
's1'         'A'                 0      NULL  NULL    'YES'
'BTREE'     NULL  NULL     'YES'
  'public.tbl1'           0  'i_tbl1_i2_s2'            1
'i2'         'A'                 0      NULL  NULL    'NO'
'BTREE'     NULL  NULL     'YES'
  'public.tbl1'           0  'i_tbl1_i2_s2'            2
's2'         'A'                 0      NULL  NULL    'YES'
'BTREE'     NULL  NULL     'YES'
  'public.tbl1'           1  'i_tbl1_s1'              1
's1'         'A'                 0         7  NULL    'YES'
```

```
'BTREE'     NULL  NULL     'NO'
  'public.tbl1'           0  'u_tbl1_i3'               1
'i3'         'A'                  0      NULL  NULL    'YES'
'BTREE'     NULL  NULL     'YES'
  'public.tbl1'           0  'u_tbl1_s3'               1
's3'         'A'                  0      NULL  NULL    'YES'
'BTREE'     NULL  NULL     'YES'
```

SHOW COLLATION

It lists collations supported by the database. If LIKE clause is present, it indicates which collation names to match.

```
SHOW COLLATION [LIKE 'pattern'];
```

This query has the following columns:

Column name	Type	Description
Collation	VARCHAR	Collation name
Charset	CHAR(1)	Charset name
Id	INTEGER	Collation ID
Built_in	CHAR(1)	Built-in collation or not. Built-in collations are impossible to add or remove because they are hard-coded.
Expansions	CHAR(1)	Collation with expansion or not. For details, see Expansion.
Strength	CHAR(1)	The number of levels to be considered in comparison, and the character order can

Column name	Type	Description
		be different by this number. For details, see Collation Properties.

The following shows the examples of this syntax.

```
SHOW COLLATION;
```

```
  Collation            Charset                        Id
Built_in             Expansions            Strength
=============================================================================
=================================================
  'euckr_bin'          'euckr'                         8
'Yes'                'No'                  'Not applicable'
  'iso88591_bin'       'iso88591'                      0
'Yes'                'No'                  'Not applicable'
  'iso88591_en_ci'     'iso88591'                      3
'Yes'                'No'                  'Not applicable'
  'iso88591_en_cs'     'iso88591'                      2
'Yes'                'No'                  'Not applicable'
  'utf8_bin'           'utf8'                          1
'Yes'                'No'                  'Not applicable'
  'utf8_de_exp'        'utf8'                         76
'No'                 'Yes'                 'Tertiary'
  'utf8_de_exp_ai_ci'  'utf8'                         72
'No'                 'Yes'                 'Primary'
  'utf8_en_ci'         'utf8'                          5
'Yes'                'No'                  'Not applicable'
  'utf8_en_cs'         'utf8'                          4
'Yes'                'No'                  'Not applicable'
  'utf8_es_cs'         'utf8'                         85
'No'                 'No'                  'Quaternary'
  'utf8_fr_exp_ab'     'utf8'                         94
'No'                 'Yes'                 'Tertiary'
  'utf8_gen'           'utf8'                         32
'No'                 'No'                  'Quaternary'
  'utf8_gen_ai_ci'     'utf8'                         37
'No'                 'No'                  'Primary'
  'utf8_gen_ci'        'utf8'                         44
'No'                 'No'                  'Secondary'
  'utf8_ja_exp'        'utf8'                        124
'No'                 'Yes'                 'Tertiary'
  'utf8_ja_exp_cbm'    'utf8'                        125
'No'                 'Yes'                 'Tertiary'
```

```
  'utf8_km_exp'         'utf8'                      132
'No'                  'Yes'                 'Quaternary'
  'utf8_ko_cs'          'utf8'                        7
'Yes'                 'No'                  'Not applicable'
  'utf8_ko_cs_uca'      'utf8'                      133
'No'                  'No'                  'Quaternary'
  'utf8_tr_cs'          'utf8'                        6
'Yes'                 'No'                  'Not applicable'
  'utf8_tr_cs_uca'      'utf8'                      205
'No'                  'No'                  'Quaternary'
  'utf8_vi_cs'          'utf8'                      221
'No'                  'No'                  'Quaternary'
```

```
SHOW COLLATION LIKE '%_ko_%';
```

```
  Collation             Charset                       Id
Built_in              Expansions            Strength
============================================================================
============================================================
  'utf8_ko_cs'          'utf8'                        7
'Yes'                 'No'                  'Not applicable'
  'utf8_ko_cs_uca'      'utf8'                      133
'No'                  'No'                  'Quaternary'
```

SHOW TIMEZONES

It shows the timezone information which the current CUBRID supports.

```
SHOW [FULL] TIMEZONES [LIKE 'pattern'];
```

If FULL is not specified, one column which has timezone's region names is displayed. The name of this column is timezone_region.

If FULL is specified, four columns which have timezone information are displayed.

If LIKE clause is present, it indicates which timezone_region names to match.

Column name	Type	Description
timezone_region	VARCHAR(32)	Timezone region name
region_offset	VARCHAR(32)	

Column name	Type	Description
		Offset of timezone (daylight saving time is not considered)
dst_offset	VARCHAR(32)	Offset of daylight saving time (applied to timezone region) which is currently considered
dst_abbreviation	VARCHAR(32)	An abbreviation of the daylight saving time which is currently applied for the region

The information listed for the second, third and fourth columns is for the current date and time.

If a timezone region doesn't have daylight saving time rules at all then the dst_offset and dst_abbreviation columns will contain NULL values.

If at the current date, there aren't daylight saving time rules that apply, then dst_offset will be set to 0 and dst_abbreviation will be the empty string.

The LIKE condition without the WHERE condition is applied on the first column. The WHERE condition may be used to filter the output.

```
SHOW TIMEZONES;
```

```
timezone_region
=======================
'Africa/Abidjan'
'Africa/Accra'
'Africa/Addis_Ababa'
'Africa/Algiers'
'Africa/Asmara'
'Africa/Asmera'
...
'US/Michigan'
'US/Mountain'
'US/Pacific'
'US/Samoa'
```

```
'UTC'
'Universal'
'W-SU'
'WET'
'Zulu'
```

```
SHOW FULL TIMEZONES;
```

```
timezone_region       region_offset         dst_offset
dst_abbreviation
================================================================================
==============
'Africa/Abidjan'      '+00:00'              '+00:00'
'GMT'
'Africa/Accra'        '+00:00'              NULL
NULL
'Africa/Addis_Ababa'  '+03:00'              '+00:00'
'EAT'
'Africa/Algiers'      '+01:00'              '+00:00'
'CET'
'Africa/Asmara'       '+03:00'              '+00:00'
'EAT'
'Africa/Asmera'       '+03:00'              '+00:00'
'EAT'
...
'US/Michigan'         '-05:00'              '+00:00'
'EST'
'US/Mountain'         '-07:00'              '+00:00'
'MST'
'US/Pacific'          '-08:00'              '+00:00'
'PST'
'US/Samoa'            '-11:00'              '+00:00'
'SST'
'UTC'                 '+00:00'              '+00:00'
'UTC'
'Universal'           '+00:00'              '+00:00'
'UTC'
'W-SU'                '+04:00'              '+00:00'
'MSK'
'WET'                 '+00:00'              '+00:00'
'WET'
'Zulu'                '+00:00'              '+00:00'
'UTC'
```

```
SHOW FULL TIMEZONES LIKE '%Paris%';
```

```
timezone_region       region_offset         dst_offset
dst_abbreviation
================================================================================
==================
'Europe/Paris'        '+01:00'              '+00:00'
'CET'
```

SHOW GRANTS

It shows the permissions associated with the database user accounts.

```
SHOW GRANTS FOR user_name;
```

The following shows the examples of this syntax.

```
CREATE TABLE testgrant (id INT);
CREATE USER user1;
GRANT INSERT,SELECT ON testgrant TO user1;

SHOW GRANTS FOR user1;
```

```
  Grants for USER1
======================
  'GRANT INSERT, SELECT ON dba.testgrant TO USER1'
```

SHOW CREATE TABLE

When a table name is specified, It shows the **CREATE TABLE** statement of the table.

```
SHOW CREATE TABLE [schema_name.]table_name;
```

```
SHOW CREATE TABLE nation;
```

```
  TABLE                 CREATE TABLE
================================================
  'public.nation'       'CREATE TABLE [nation] ([code] CHARACTER(3)
NOT NULL, [name] CHARACTER VARYING(40) NOT NULL, [continent]
CHARACTER VARYING(10), [capital] CHARACTER VARYING(30), CONSTRAINT
```

```
[pk_nation_code] PRIMARY KEY  ([code])) DONT_REUSE_OID, COLLATE
iso88591_bin'
```

SHOW CREATE TABLE statement does not display as the user's written syntax. For example, the comment that user wrote is not displayed, and table names and column names are always displayed as lower case letters.

SHOW CREATE VIEW

It shows the corresponding **CREATE VIEW** statement if view name is specified.

```
SHOW CREATE VIEW view_name;
```

The following shows the examples of this syntax.

```
SHOW CREATE VIEW db_class;
```

```
  View                  Create View
=============================================
  'db_class'            'SELECT [c].[class_name], CAST([c].[owner].
[name] AS VARCHAR(255)), CASE [c].[class_type] WHEN 0 THEN 'CLASS'
WHEN 1 THEN 'VCLASS' ELSE 'UNKNOWN' END, CASE WHEN MOD([c].
[is_system_class], 2) = 1 THEN 'YES' ELSE 'NO' END, CASE [c].
[tde_algorithm] WHEN 0 THEN 'NONE' WHEN 1 THEN 'AES' WHEN 2 THEN
'ARIA' END, CASE WHEN [c].[sub_classes] IS NULL THEN 'NO' ELSE
NVL((SELECT 'YES' FROM [_db_partition] [p] WHERE [p].[class_of] =
[c] and [p].[pname] IS NULL), 'NO') END, CASE WHEN MOD([c].
[is_system_class] / 8, 2) = 1 THEN 'YES' ELSE 'NO' END, [coll].
[coll_name], [c].[comment] FROM [_db_class] [c], [_db_collation]
[coll] WHERE [c].[collation_id] = [coll].[coll_id] AND (CURRENT_USER
= 'DBA' OR {[c].[owner].[name]} SUBSETEQ(SELECT SET{CURRENT_USER} +
COALESCE(SUM(SET{[t].[g].[name]}), SET{}) FROM [db_user] [u],
TABLE([groups]) AS [t]([g]) WHERE [u].[name] = CURRENT_USER) OR
{[c]} SUBSETEQ ( SELECT SUM(SET{[au].[class_of]}) FROM [_db_auth]
[au] WHERE {[au].[grantee].[name]} SUBSETEQ ( SELECT
SET{CURRENT_USER} + COALESCE(SUM(SET{[t].[g].[name]}), SET{}) FROM
[db_user] [u], TABLE([groups]) AS [t]([g]) WHERE [u].[name] =
CURRENT_USER) AND [au].[auth_type] = 'SELECT'))'
```

SHOW ACCESS STATUS

SHOW ACCESS STATUS statement displays login information regarding database accounts. Only database's DBA account can use this statement.

```
SHOW ACCESS STATUS [LIKE 'pattern' | WHERE expr];
```

This statement displays the following columns.

Column name	Type	Description
user_name	VARCHAR(32)	DB user's account
last_access_time	DATETIME	Last time that the database user accessed
last_access_host	VARCHAR(32)	Lastly accessed host
program_name	VARCHAR(32)	The name of client program(broker_cub_cas_1, csql ..)

The following shows the result of running this statement.

```
SHOW ACCESS STATUS;
```

```
  user_name last_access_time last_access_host program_name
========================================================================
========
  'DBA' 08:19:31.000 PM 02/10/2014 127.0.0.1 'csql'
  'PUBLIC' NULL NULL NULL
```

Note

The above login information which **SHOW ACCESS STATUS** shows is initialized when the database is restarted, and this query is not replication in HA environment; therefore, each node shows the different result.

SHOW EXEC STATISTICS

It shows statistics information of executing query.

- To start collecting **@collect_exec_stats** statistics information, configure the value of session variable **@collect_exec_stats** to 1; to stop, configure it to 0.
- It outputs the result of collecting statistics information.
  - The **SHOW EXEC STATISTICS** statement outputs four part of data page statistics information; data_page_fetches, data_page_dirties, data_page_ioreads, and data_page_iowrites. The result columns consist of variable column (name of statistics name) and value column (value of statistics value). Once the **SHOW EXEC STATISTICS** statement is executed, the statistics information which has been accumulated is initialized.
  - The **SHOW EXEC STATISTICS ALL** statement outputs all items of statistics information.

For details, see statdump.

```
SHOW EXEC STATISTICS [ALL];
```

The following shows the examples of this syntax.

```
-- set session variable @collect_exec_stats as 1 to start collecting
the statistical information.
SET @collect_exec_stats = 1;
SELECT * FROM db_class;

-- print the statistical information of the data pages.
SHOW EXEC STATISTICS;
```

```
variable                value
================================
'data_page_fetches'     332
'data_page_dirties'     85
'data_page_ioreads'     18
'data_page_iowrites'    28
```

```
SELECT * FROM db_index;

-- print all of the statistical information.
SHOW EXEC STATISTICS ALL;
```

```
variable                                    value
==========================================
```

```
'file_creates'                          0
'file_removes'                          0
'file_ioreads'                          6
'file_iowrites'                         0
'file_iosynches'                        0
'data_page_fetches'                     548
'data_page_dirties'                     34
'data_page_ioreads'                     6
'data_page_iowrites'                    0
'log_page_ioreads'                      0
'log_page_iowrites'                     0
'log_append_records'                    0
'log_archives'                          0
'log_start_checkpoints'                 0
'log_end_checkpoints'                   0
'log_wals'                              0
'page_locks_acquired'                   13
'object_locks_acquired'                 9
'page_locks_converted'                  0
'object_locks_converted'                0
'page_locks_re-requested'               0
'object_locks_re-requested'             8
'page_locks_waits'                      0
'object_locks_waits'                    0
'tran_commits'                          3
'tran_rollbacks'                        0
'tran_savepoints'                       0
'tran_start_topops'                     6
'tran_end_topops'                       6
'tran_interrupts'                       0
'btree_inserts'                         0
'btree_deletes'                         0
'btree_updates'                         0
'btree_covered'                         0
'btree_noncovered'                      2
'btree_resumes'                         0
'btree_multirange_optimization'         0
'query_selects'                         4
'query_inserts'                         0
'query_deletes'                         0
'query_updates'                         0
'query_sscans'                          2
'query_iscans'                          4
'query_lscans'                          0
'query_setscans'                        2
'query_methscans'                       0
'query_nljoins'                         2
'query_mjoins'                          0
'query_objfetches'                      0
'query_holdable_cursors'                0
'sort_io_pages'                         0
'sort_data_pages'                       0
```

```
'network_requests'                    88
'adaptive_flush_pages'                0
'adaptive_flush_log_pages'            0
'adaptive_flush_max_pages'            0
'prior_lsa_list_size'                 0
'prior_lsa_list_maxed'                0
'prior_lsa_list_removed'              0
'heap_stats_bestspace_entries'        0
'heap_stats_bestspace_maxed'          0
```

Diagnostics

The following diagnostic SHOW statements can only be used by users who are DBA or members of the DBA group.

SHOW VOLUME HEADER

It shows the volume header of the specified volume in one row.

```
SHOW VOLUME HEADER OF volume_id;
```

This query has the following columns:

Column name	Type	Description
Volume_id	INT	Volume identifier
Magic_symbol	VARCHAR(100)	Magic value for for a volume file
Io_page_size	INT	Size of DB volume
Purpose	VARCHAR(32)	Volume purposes, ‘Permanent data purpose’ or ‘Temporary data purpose’
Type	VARCHAR(32)	Volume type, ‘Permanent Volume’ or ‘Temporary Volume’

Column name	Type	Description
Sector_size_in_pages	INT	Size of sector in pages
Num_total_sectors	INT	Total number of sectors
Num_free_sectors	INT	Number of free sectors
Num_max_sectors	INT	Maximum number of sectors
Hint_alloc_sector	INT	Hint for next sector to be allocated
Sector_alloc_table_size_in_pages	INT	Size of sector allocation table in page
Sector_alloc_table_first_page	INT	First page of sector allocation table
Page_alloc_table_size_in_pages	INT	Size of page allocation table in page
Page_alloc_table_first_page	INT	First page of page allocation table
Last_system_page	INT	Last system page
Creation_time	DATETIME	Volume creation time
Db_charset	INT	Charset number of database
Checkpoint_lsa	VARCHAR(64)	Lowest log sequence address to start the

Column name	Type	Description
		recovery process of this volume
Boot_hfid	VARCHAR(64)	System Heap file for booting purposes and multi volumes
Full_name	VARCHAR(255)	The full path of volume
Next_volume_id	INT	Next volume identifier
Next_vol_full_name	VARCHAR(255)	The full path of next volume
Remarks	VARCHAR(64)	Volume remarks

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW VOLUME HEADER OF 0;
```

```
<00001> Volume_id                       : 0
        Magic_symbol                    : 'MAGIC SYMBOL = CUBRID/
Volume at disk location = 32'
        Io_page_size                    : 16384
        Purpose                         : 'Permanent data purpose'
        Type                            : 'Permanent Volume'
        Sector_size_in_pages            : 64
        Num_total_sectors               : 512
        Num_free_sectors                : 459
        Num_max_sectors                 : 512
        Hint_alloc_sector               : 0
        Sector_alloc_table_size_in_pages: 1
        Sector_alloc_table_first_page   : 1
        Last_system_page                : 1
        Creation_time                   : 09:46:41.000 PM 05/23/2017
        Db_charset                      : 3
        Checkpoint_lsa                  : '(0|12832)'
        Boot_hfid                       : '(0|41|50)'
        Full_name                       : '/home1/brightest/CUBRID/
```

```
databases/demodb/demodb'
        Next_volume_id                  : -1
        Next_vol_full_name              : ''
        Remarks                         : ''
```

SHOW LOG HEADER

It shows the header information of an active log file.

```
SHOW LOG HEADER [OF file_path];
```

If you omit **OF** *file_path*, it shows the header information of a memory; if you include **OF** *file_path*, it shows the header information of *file_path*. In file_path, you can input either the relative path or the absolute path of the active log file. When entering a relative path, the base path is the directory where the active log file is located.

This query has the following columns:

Column name	Type	Description
Volume_id	INT	Volume identifier
Magic_symbol	VARCHAR(32)	Magic value for log file
Magic_symbol_location	INT	Magic symbol location from log page
Creation_time	DATETIME	Volume creation time
Release	VARCHAR(32)	CUBRID Release version
Compatibility_disk_version	VARCHAR(32)	Compatibility of the database against the current release of CUBRID
Db_page_size	INT	Size of pages in the database

Column name	Type	Description
Log_page_size	INT	Size of log pages in the database
Shutdown	INT	Was the log shutdown
Next_trans_id	INT	Next transaction identifier
Num_avg_trans	INT	Number of average transactions
Num_avg_locks	INT	Average number of object locks
Num_active_log_pages	INT	Number of pages in the active log portion
Db_charset	INT	Charset number of database
First_active_log_page	BIGINT	Logical pageid at physical location 1 in active log
Current_append	VARCHAR(64)	Current append location
Checkpoint	VARCHAR(64)	Lowest log sequence address to start the recovery process
Next_archive_page_id	BIGINT	Next logical page to archive
Active_physical_page_id	INT	Physical location of logical page to archive

Column name	Type	Description
Next_archive_num	INT	Next log archive number
Last_archive_num_for_syscrashes	INT	Last log archive needed for system crashes
Last_deleted_archive_num	INT	Last deleted archive number
Backup_lsa_level0	VARCHAR(64)	LSA of backup level 0
Backup_lsa_level1	VARCHAR(64)	LSA of backup level 1
Backup_lsa_level2	VARCHAR(64)	LSA of backup level 2
Log_prefix	VARCHAR(256)	Log prefix name
Has_logging_been_skipped	INT	Whether or not logging skipped
Perm_status	VARCHAR(64)	Reserved for future expansion
Backup_info_level0	VARCHAR(128)	detail information of backup level 0. currently only backup start-time is used
Backup_info_level1	VARCHAR(128)	detail information of backup level 1. currently only backup start-time is used
Backup_info_level2	VARCHAR(128)	

Column name	Type	Description
		detail information of backup level 2. currently only backup start-time is used
Ha_server_state	VARCHAR(32)	current ha state, one of flowing value: na, idle, active, to-be-active, standby, to-be-standby, maintenance, dead
Ha_file	VARCHAR(32)	ha replication status, one of following value: clear, archived, sync
Eof_lsa	VARCHAR(64)	EOF LSA
Smallest_lsa_at_last_checkpoint	VARCHAR(64)	The smallest LSA of the last checkpoint, can be NULL LSA
Next_mvcc_id	BIGINT	The next MVCCID will be used for the next transaction
Mvcc_op_log_lsa	VARCHAR(32)	The LSA used to link log entries for MVCC operation
Last_block_oldest_mvcc_id	BIGINT	Used to find the oldest MVCCID in a block of log data, can be NULL
Last_block_newest_mvcc_id	BIGINT	Used to find the newest MVCCID in a block of log data, can be NULL

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW LOG HEADER;
```

```
<00001> Volume_id                       : -2
        Magic_symbol                    : 'CUBRID/LogActive'
        Magic_symbol_location           : 16
        Creation_time                   : 09:46:41.000 PM 05/23/2017
        Release                         : '10.0.0'
        Compatibility_disk_version      : '10'
        Db_page_size                    : 16384
        Log_page_size                   : 16384
        Shutdown                        : 0
        Next_trans_id                   : 17
        Num_avg_trans                   : 3
        Num_avg_locks                   : 30
        Num_active_log_pages            : 1279
        Db_charset                      : 3
        First_active_log_page           : 0
        Current_append                  : '(102|5776)'
        Checkpoint                      : '(101|7936)'
        Next_archive_page_id            : 0
        Active_physical_page_id         : 1
        Next_archive_num                : 0
        Last_archive_num_for_syscrashes: -1
        Last_deleted_archive_num        : -1
        Backup_lsa_level0               : '(-1|-1)'
        Backup_lsa_level1               : '(-1|-1)'
        Backup_lsa_level2               : '(-1|-1)'
        Log_prefix                      : 'mvccdb'
        Has_logging_been_skipped        : 0
        Perm_status                     : 'LOG_PSTAT_CLEAR'
        Backup_info_level0              : 'time: N/A'
        Backup_info_level1              : 'time: N/A'
        Backup_info_level2              : 'time: N/A'
        Ha_server_state                 : 'idle'
        Ha_file                         : 'UNKNOWN'
        Eof_lsa                         : '(102|5776)'
        Smallest_lsa_at_last_checkpoint: '(101|7936)'
        Next_mvcc_id                    : 6
        Mvcc_op_log_lsa                 : '(102|5488)'
        Last_block_oldest_mvcc_id       : 4
        Last_block_newest_mvcc_id       : 5
```

```
SHOW LOG HEADER OF 'demodb_lgat';
```

```
<00001> Volume_id                       : -2
        Magic_symbol                    : 'CUBRID/LogActive'
        Magic_symbol_location           : 16
        Creation_time                   : 09:46:41.000 PM 05/23/2017
        Release                         : '10.0.0'
        Compatibility_disk_version      : '10'
        Db_page_size                    : 16384
        Log_page_size                   : 16384
        Shutdown                        : 0
        Next_trans_id                   : 15
        Num_avg_trans                   : 3
        Num_avg_locks                   : 30
        Num_active_log_pages            : 1279
        Db_charset                      : 3
        First_active_log_page           : 0
        Current_append                  : '(101|8016)'
        Checkpoint                      : '(101|7936)'
        Next_archive_page_id            : 0
        Active_physical_page_id         : 1
        Next_archive_num                : 0
        Last_archive_num_for_syscrashes: -1
        Last_deleted_archive_num        : -1
        Backup_lsa_level0               : '(-1|-1)'
        Backup_lsa_level1               : '(-1|-1)'
        Backup_lsa_level2               : '(-1|-1)'
        Log_prefix                      : 'mvccdb'
        Has_logging_been_skipped        : 0
        Perm_status                     : 'LOG_PSTAT_CLEAR'
        Backup_info_level0              : 'time: N/A'
        Backup_info_level1              : 'time: N/A'
        Backup_info_level2              : 'time: N/A'
        Ha_server_state                 : 'idle'
        Ha_file                         : 'UNKNOWN'
        Eof_lsa                         : '(101|8016)'
        Smallest_lsa_at_last_checkpoint: '(101|7936)'
        Next_mvcc_id                    : 4
        Mvcc_op_log_lsa                 : '(-1|-1)'
        Last_block_oldest_mvcc_id       : NULL
        Last_block_newest_mvcc_id       : NULL
```

SHOW ARCHIVE LOG HEADER

It shows the header information of an archive log file.

```
SHOW ARCHIVE LOG HEADER OF file_name;
```

In file_path, you can input either the relative path or the absolute path of the archive log file. When entering a relative path, the base path is the directory where the archive log file is located.

This query has the following columns:

Column name	Type	Description
Volume_id	INT	Identifier of log volume
Magic_symbol	VARCHAR(32)	Magic value for file/magic Unix utility
Magic_symbol_location	INT	Magic symbol location from log page
Creation_time	DATETIME	Volume creation time
Next_trans_id	BIGINT	Next transaction identifier
Num_pages	INT	Number of pages in the archive log
First_page_id	BIGINT	Logical page id at physical location 1 in archive log
Archive_num	INT	The archive log number

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW ARCHIVE LOG HEADER OF 'demodb_lgar001';
```

```
<00001> Volume_id             : -20
        Magic_symbol          : 'CUBRID/LogArchive'
        Magic_symbol_location: 16
        Creation_time         : 04:42:28.000 PM 12/11/2013
        Next_trans_id         : 22695
        Num_pages             : 1278
```

```
        First_page_id         : 1278
        Archive_num           : 1
```

SHOW HEAP HEADER

It shows shows the header page of the table.

```
SHOW  [ALL] HEAP HEADER OF [schema_name.]table_name;
```

- ALL: If “ALL” is given in syntax in the partition table, the basic table and its partitioned tables are shown.

This query has the following columns:

Column name	Type	Description
Class_name	VARCHAR(256)	Table name
Class_oid	VARCHAR(64)	Format: (volid|pageid|slotid)
Volume_id	INT	Volume identifier where the file reside
File_id	INT	File identifier
Header_page_id	INT	First page identifier (the header page)
Overflow_vfid	VARCHAR(64)	Overflow file identifier (if any)
Next_vpid	VARCHAR(64)	Next page (i.e., the 2nd page of heap file)
Unfill_space	INT	

Column name	Type	Description
		Stop inserting when page has run below this. leave it for updates
Estimates_num_pages	BIGINT	Estimation of number of heap pages.
Estimates_num_recs	BIGINT	Estimation of number of objects in heap
Estimates_avg_rec_len	INT	Estimation total length of records
Estimates_num_high_best	INT	Number of pages in the best array that we believe have at least HEAP_DROP_FREE_SPACE. When this number goes to zero and there are at least other HEAP_NUM_BEST_SPACESTATS best pages, we look for them
Estimates_num_others_high_best	INT	Total of other believed known best pages, which are not included in the best array and we believe they have at least HEAP_DROP_FREE_SPACE
Estimates_head	INT	Head of best circular array
Estimates_best_list	VARCHAR(512)	Format: ‘((best[0].vpid.volid| best[0].vpid.pageid), best[0].freespace), ... ,

Column name	Type	Description
		((best[9].vpid.volid| best[9].vpid.pageid), best[9].freespace)’
Estimates_num_second_best	INT	Number of second best hints. The hints are in “second_best” array. They are used when finding new best pages.
Estimates_head_second_best	INT	Index of head of second best hints. A new second best hint will be stored on this index.
Estimates_num_substitutions	INT	Number of page substitutions. This will be used to insert a new second best page into second best hints.
Estimates_second_best_list	VARCHAR(512)	Format: ‘(second_best[0].vpid.volid| second_best[0].vpid.pageid), ... , (second_best[9].vpid.volid| second_best[9].vpid.pageid)’
Estimates_last_vpid	VARCHAR(64)	Format: ‘(volid|pageid)’
Estimates_full_search_vpid	VARCHAR(64)	Format: ‘(volid|pageid)’

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW HEAP HEADER OF athlete;
```

```
<00001> Class_name                    : 'athlete'
        Class_oid                     : '(0|463|8)'
        Volume_id                     : 0
        File_id                       : 147
        Header_page_id                : 590
        Overflow_vfid                 : '(-1|-1)'
        Next_vpid                     : '(0|591)'
        Unfill_space                  : 1635
        Estimates_num_pages           : 27
        Estimates_num_recs            : 6677
        Estimates_avg_rec_len         : 54
        Estimates_num_high_best       : 1
        Estimates_num_others_high_best: 0
        Estimates_head                : 0
        Estimates_best_list           : '((0|826), 14516), ((-1|-1),
0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0),
((-1|-1), 0), ((-1|-1), 0), ((-1|-1),0), ((-1|-1), 0)'
        Estimates_num_second_best     : 0
        Estimates_head_second_best    : 0
        Estimates_tail_second_best    : 0
        Estimates_num_substitutions   : 0
        Estimates_second_best_list    : '(-1|-1), (-1|-1), (-1|-1),
(-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1)'
        Estimates_last_vpid           : '(0|826)'
        Estimates_full_search_vpid    : '(0|590)'
```

```
CREATE TABLE participant2 (
    host_year INT,
    nation CHAR(3),
    gold INT,
    silver INT,
    bronze INT
)
PARTITION BY RANGE (host_year) (
    PARTITION before_2000 VALUES LESS THAN (2000),
    PARTITION before_2008 VALUES LESS THAN (2008)
);
```

```
SHOW ALL HEAP HEADER OF participant2;
```

```
<00001> Class_name                    : 'participant2'
        Class_oid                     : '(0|467|6)'
```

```
        Volume_id                     : 0
        File_id                       : 374
        Header_page_id                : 940
        Overflow_vfid                 : '(-1|-1)'
        Next_vpid                     : '(-1|-1)'
        Unfill_space                  : 1635
        Estimates_num_pages           : 1
        Estimates_num_recs            : 0
        Estimates_avg_rec_len         : 0
        Estimates_num_high_best       : 1
        Estimates_num_others_high_best: 0
        Estimates_head                : 1
        Estimates_best_list           : '((0|940), 16308), ((-1|-1),
0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0),
((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0)'
        Estimates_num_second_best     : 0
        Estimates_head_second_best    : 0
        Estimates_tail_second_best    : 0
        Estimates_num_substitutions   : 0
        Estimates_second_best_list    : '(-1|-1), (-1|-1), (-1|-1),
(-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1)'
        Estimates_last_vpid           : '(0|940)'
        Estimates_full_search_vpid    : '(0|940)'
<00002> Class_name                    :
'participant2__p__before_2000'
        Class_oid                     : '(0|467|7)'
        Volume_id                     : 0
        File_id                       : 376
        Header_page_id                : 950
        Overflow_vfid                 : '(-1|-1)'
        Next_vpid                     : '(-1|-1)'
        Unfill_space                  : 1635
        Estimates_num_pages           : 1
        Estimates_num_recs            : 0
        Estimates_avg_rec_len         : 0
        Estimates_num_high_best       : 1
        Estimates_num_others_high_best: 0
        Estimates_head                : 1
        Estimates_best_list           : '((0|950), 16308), ((-1|-1),
0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0),
((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0)'
        Estimates_num_second_best     : 0
        Estimates_head_second_best    : 0
        Estimates_tail_second_best    : 0
        Estimates_num_substitutions   : 0
        Estimates_second_best_list    : '(-1|-1), (-1|-1), (-1|-1),
(-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1)'
        Estimates_last_vpid           : '(0|950)'
        Estimates_full_search_vpid    : '(0|950)'
<00003> Class_name                    :
'participant2__p__before_2008'
        Class_oid                     : '(0|467|8)'
```

```
        Volume_id                    : 0
        File_id                      : 378
        Header_page_id               : 960
        Overflow_vfid                : '(-1|-1)'
        Next_vpid                    : '(-1|-1)'
        Unfill_space                 : 1635
        Estimates_num_pages          : 1
        Estimates_num_recs           : 0
        Estimates_avg_rec_len        : 0
        Estimates_num_high_best      : 1
        Estimates_num_others_high_best: 0
        Estimates_head               : 1
        Estimates_best_list          : '((0|960), 16308), ((-1|-1),
0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0),
((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0)'
        Estimates_num_second_best    : 0
        Estimates_head_second_best   : 0
        Estimates_tail_second_best   : 0
        Estimates_num_substitutions  : 0
        Estimates_second_best_list   : '(-1|-1), (-1|-1), (-1|-1),
(-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1)'
        Estimates_last_vpid          : '(0|960)'
        Estimates_full_search_vpid   : '(0|960)'
```

```
SHOW HEAP HEADER OF participant2__p__before_2008;
```

```
<00001> Class_name                   :
'participant2__p__before_2008'
        Class_oid                    : '(0|467|8)'
        Volume_id                    : 0
        File_id                      : 378
        Header_page_id               : 960
        Overflow_vfid                : '(-1|-1)'
        Next_vpid                    : '(-1|-1)'
        Unfill_space                 : 1635
        Estimates_num_pages          : 1
        Estimates_num_recs           : 0
        Estimates_avg_rec_len        : 0
        Estimates_num_high_best      : 1
        Estimates_num_others_high_best: 0
        Estimates_head               : 1
        Estimates_best_list          : '((0|960), 16308), ((-1|-1),
0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0),
((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0), ((-1|-1), 0)'
        Estimates_num_second_best    : 0
        Estimates_head_second_best   : 0
        Estimates_tail_second_best   : 0
        Estimates_num_substitutions  : 0
        Estimates_second_best_list   : '(-1|-1), (-1|-1), (-1|-1),
```

```
(-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1), (-1|-1)'
        Estimates_last_vpid           : '(0|960)'
        Estimates_full_search_vpid    : '(0|960)'
```

SHOW HEAP CAPACITY

It shows the capacity of the table.

```
SHOW [ALL] HEAP CAPACITY OF [schema_name.]table_name;
```

- ALL: If “all” is given in syntax, the basic table and its partition table(s) is shown.

This query has the following columns:

Column name	Type	Description
Table_name	VARCHAR(256)	Table name
Class_oid	VARCHAR(64)	Heap file descriptor
Volume_id	INT	Volume identifier where the file reside
File_id	INT	File identifier
Header_page_id	INT	First page identifier (the header page)
Num_recs	BIGINT	Total Number of objects
Num_relocated_recs	BIGINT	Number of relocated records
Num_overflowed_recs	BIGINT	Number of big records
Num_pages	BIGINT	Total number of heap pages

Column name	Type	Description
Avg_rec_len	INT	Average object length
Avg_free_space_per_page	INT	Average free space per page
Avg_free_space_per_page_without_last_page	INT	Average free space per page without taking in consideration last page
Avg_overhead_per_page	INT	Average overhead per page
Repr_id	INT	Currently cached catalog column info
Num_total_attrs	INT	total number of columns
Num_fixed_width_attrs	INT	Number of the fixed width columns
Num_variable_width_attrs	INT	Number of variable width columns
Num_shared_attrs	INT	Number of shared columns
Num_class_attrs	INT	Number of table columns
Total_size_fixed_width_attrs	INT	Total size of the fixed width columns

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW HEAP CAPACITY OF athlete;
```

```
<00001> Table_name                                  : 'athlete'
        Class_oid                                   : '(0|463|8)'
        Volume_id                                   : 0
        File_id                                     : 147
        Header_page_id                              : 590
        Num_recs                                    : 6677
        Num_relocated_recs                          : 0
        Num_overflowed_recs                         : 0
        Num_pages                                   : 27
        Avg_rec_len                                 : 53
        Avg_free_space_per_page                     : 2139
        Avg_free_space_per_page_except_last_page: 1663
        Avg_overhead_per_page                       : 993
        Repr_id                                     : 1
        Num_total_attrs                             : 5
        Num_fixed_width_attrs                       : 3
        Num_variable_width_attrs                    : 2
        Num_shared_attrs                            : 0
        Num_class_attrs                             : 0
        Total_size_fixed_width_attrs                : 8
```

```
SHOW ALL HEAP CAPACITY OF participant2;
```

```
<00001> Table_name                                  : 'participant2'
        Class_oid                                   : '(0|467|6)'
        Volume_id                                   : 0
        File_id                                     : 374
        Header_page_id                              : 940
        Num_recs                                    : 0
        Num_relocated_recs                          : 0
        Num_overflowed_recs                         : 0
        Num_pages                                   : 1
        Avg_rec_len                                 : 0
        Avg_free_space_per_page                     : 16016
        Avg_free_space_per_page_except_last_page: 0
        Avg_overhead_per_page                       : 4
        Repr_id                                     : 1
        Num_total_attrs                             : 5
        Num_fixed_width_attrs                       : 5
        Num_variable_width_attrs                    : 0
        Num_shared_attrs                            : 0
        Num_class_attrs                             : 0
        Total_size_fixed_width_attrs                : 20
<00002> Table_name                                  :
'participant2__p__before_2000'
        Class_oid                                   : '(0|467|7)'
        Volume_id                                   : 0
        File_id                                     : 376
```

```
        Header_page_id                          : 950
        Num_recs                                : 0
        Num_relocated_recs                      : 0
        Num_overflowed_recs                     : 0
        Num_pages                               : 1
        Avg_rec_len                             : 0
        Avg_free_space_per_page                 : 16016
        Avg_free_space_per_page_except_last_page: 0
        Avg_overhead_per_page                   : 4
        Repr_id                                 : 1
        Num_total_attrs                         : 5
        Num_fixed_width_attrs                   : 5
        Num_variable_width_attrs                : 0
        Num_shared_attrs                        : 0
        Num_class_attrs                         : 0
        Total_size_fixed_width_attrs            : 20
<00003> Table_name                              :
'participant2__p__before_2008'
        Class_oid                               : '(0|467|8)'
        Volume_id                               : 0
        File_id                                 : 378
        Header_page_id                          : 960
        Num_recs                                : 0
        Num_relocated_recs                      : 0
        Num_overflowed_recs                     : 0
        Num_pages                               : 1
        Avg_rec_len                             : 0
        Avg_free_space_per_page                 : 16016
        Avg_free_space_per_page_except_last_page: 0
        Avg_overhead_per_page                   : 4
        Repr_id                                 : 1
        Num_total_attrs                         : 5
        Num_fixed_width_attrs                   : 5
        Num_variable_width_attrs                : 0
        Num_shared_attrs                        : 0
        Num_class_attrs                         : 0
        Total_size_fixed_width_attrs            : 20
```

SHOW SLOTTED PAGE HEADER

It shows the header information of specified slotted page.

```
SHOW SLOTTED PAGE HEADER (WHERE | OF) VOLUME = volume_num AND PAGE =
page_num;
```

This query has the following columns:

Column name	Type	Description
Volume_id	INT	Volume id of the page
Page_id	INT	page id of the page
Num_slots	INT	Number of allocated slots for the page
Num_records	INT	Number of records on page
Anchor_type	VARCHAR(32)	One of flowing: ANCHORED, ANCHORED_DONT_REUSE_SLOTS, UNANCHORED_ANY_SEQUENCE, UNANCHORED_KEEP_SEQUENCE
Alignment	VARCHAR(8)	Alignment for records, one of flowing: CHAR, SHORT, INT, DOUBLE
Total_free_area	INT	Total free space on page
Contiguous_free_area	INT	Contiguous free space on page
Free_space_offset	INT	Byte offset from the beginning of the page to the first free byte area on the page
Need_update_best_hint	INT	True if saving is need for recovery (undo)

Column name	Type	Description
Is_saving	INT	True if we should update best pages hint for this page.
Flags	INT	Flag value of the page

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW SLOTTED PAGE HEADER OF VOLUME=0 AND PAGE=140;
```

```
<00001> Volume_id            : 0
        Page_id              : 140
        Num_slots            : 3
        Num_records          : 3
        Anchor_type          : 'ANCHORED_DONT_REUSE_SLOTS'
        Alignment            : 'INT'
        Total_free_area      : 15880
        Contiguous_free_area : 15880
        Free_space_offset    : 460
        Need_update_best_hint: 1
        Is_saving            : 0
        Flags                : 0
```

SHOW SLOTTED PAGE SLOTS

It shows the information of all slots in the specified slotted page.

```
SHOW SLOTTED PAGE SLOTS (WHERE | OF) VOLUME = volume_num AND PAGE =
page_num;
```

This query has the following columns:

Column name	Type	Description
Volume_id	INT	Volume id of the page

Column name	Type	Description
Page_id	INT	Page id of the page
Slot_id	INT	The slot id
Offset	INT	Byte offset from the beginning of the page to the beginning of the record
Type	VARCHAR(32)	Record type, one of flowing: REC_UNKNOWN, REC_ASSIGN_ADDRESS, REC_HOME, REC_NEWHOME, REC_RELOCATION, REC_BIGONE, REC_MARKDELETED, REC_DELETED_WILL_REUSE
Length	INT	Length of record
Waste	INT	Whether or not wasted

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW SLOTTED PAGE HEADER OF VOLUME=0 AND PAGE=140;
```

```
<00001> Volume_id: 0
        Page_id  : 140
        Slot_id  : 0
        Offset   : 40
        Type     : 'HOME'
        Length   : 292
        Waste    : 0
<00002> Volume_id: 0
        Page_id  : 140
        Slot_id  : 1
```

```
         Offset   : 332
         Type     : 'HOME'
         Length   : 64
         Waste    : 0
<00003>  Volume_id: 0
         Page_id  : 140
         Slot_id  : 2
         Offset   : 396
         Type     : 'HOME'
         Length   : 64
         Waste    : 0
```

SHOW INDEX HEADER

It shows the index header page of the index of the table.

```
SHOW INDEX HEADER OF [schema_name.]table_name.index_name;
```

If ALL keyword is used and an index name is omitted, it shows the entire headers of the indexes of the table.

```
SHOW ALL INDEXES HEADER OF [schema_name.]table_name;
```

This query has the following columns:

Column name	Type	Description
Table_name	VARCHAR(256)	Table name
Index_name	VARCHAR(256)	Index name
Btid	VARCHAR(64)	BTID (volid|fileid| root_pageid)
Node_level	INT	Node level (1 for LEAF, 2 or more for NON_LEAF)

Column name	Type	Description
Max_key_len	INT	Maximum key length for the subtree
Num_oids	INT	Number of OIDs stored in the Btree
Num_nulls	INT	Number of NULLs (they aren't stored)
Num_keys	INT	Number of unique keys in the Btree
Topclass_oid	VARCHAR(64)	Topclass oid or NULL OID (non unique index)(volid|pageid|slotid)
Unique	INT	Unique or non-unique
Overflow_vfid	VARCHAR(32)	VFID (volid|fileid)
Key_type	VARCHAR(256)	Type name
Columns	VARCHAR(256)	the list of columns which consists of the index

The following shows the examples of this syntax.

```
-- Prepare test environment
CREATE TABLE tbl1(a INT, b VARCHAR(5));
CREATE INDEX index_ab ON tbl1(a ASC, b DESC);
```

```
-- csql> ;line on
SHOW INDEX HEADER OF tbl1.index_ab;
```

```
<00001> Table_name   : 'tbl1'
        Index_name   : 'index_ab'
        Btid         : '(0|378|950)'
        Node_level   : 'LEAF'
        Max_key_len  : 0
        Num_oids     : -1
        Num_nulls    : -1
        Num_keys     : -1
        Topclass_oid : '(0|469|4)'
        Unique       : 0
        Overflow_vfid: '(-1|-1)'
        Key_type     : 'midxkey(integer,character varying(5))'
        Columns      : 'a,b DESC'
```

SHOW INDEX CAPACITY

It shows the index capacity of the index of the table.

```
SHOW INDEX CAPACITY OF [schema_name.]table_name.index_name;
```

If ALL keyword is used and an index name is omitted, it shows the entire capacity of the indexes of the table.

```
SHOW ALL INDEXES CAPACITY OF [schema_name.]table_name;
```

This query has the following columns:

Column name	Type	Description
Table_name	VARCHAR(256)	Table name
Index_name	VARCHAR(256)	Index name
Btid	VARCHAR(64)	BTID (volid|fileid| root_pageid)

Column name	Type	Description
Num_distinct_key	INT	Distinct key count (in leaf pages)
Total_value	INT	Total number of values stored in tree
Deduplicate_distinct_key	INT	Deduplicated distinct key count (in leaf pages)
Num_fence_key	INT	Number of fence keys
Avg_num_value_per_key	INT	Average number of values (OIDs) per key
Avg_num_value_per_deduplicate_key	INT	Average number of values (OIDs) per deduplicated key
Num_leaf_page	INT	Leaf page count
Num_non_leaf_page	INT	NonLeaf page count
Num_ovf_page	INT	Leaf's overflow page count
Num_total_page	INT	Total page count
Height	INT	Height of the tree
Avg_key_len	INT	Average key length
Avg_rec_len	INT	Average page record length

Column name	Type	Description
Total_space	VARCHAR(64)	Total space occupied by index
Total_used_space_non_ovf	VARCHAR(64)	Total space used in allocated pages (excluded leaf's overflow pages)
Total_free_space_non_ovf	VARCHAR(64)	Total space unused in allocated pages (excluded leaf's overflow pages)
Total_used_space_ovf	VARCHAR(64)	Total space used in pages allocated to leaf's overflow
Total_free_space_ovf	VARCHAR(64)	Total space unused in pages allocated to leaf's overflow
Avg_num_key_per_page_non_ovf	INT	Average page key count in leaf pages
Avg_free_space_per_page_non_ovf	VARCHAR(64)	Average page free space
Avg_num_ovf_page_per_key	INT	Average page key count in leaf pages
Avg_free_space_per_page_ovf	VARCHAR(64)	Average page free space in leaf's overflow pages
Max_num_ovf_page_a_key	INT	Maximum number of leaf's overflow pages for one key

Note

Fence key is a virtual key added to a leaf node to help operate the B-tree index.

The following shows the examples of this syntax.

```
-- Prepare test environment
CREATE TABLE tbl1(a INT, b VARCHAR(5));
CREATE INDEX index_a ON tbl1(a ASC);
CREATE INDEX index_b ON tbl1(b ASC);
```

```
-- csql> ;line on
SHOW INDEX CAPACITY OF tbl1.index_a;
```

```
<00001> Table_name                      : 'dba.tbl1'
        Index_name                      : 'index_a'
        Btid                            : '(0|4160|4161)'
        Num_distinct_key                : 0
        Total_value                     : 0
        Deduplicate_distinct_key        : 0
        Num_fence_key                   : 0
        Avg_num_value_per_key           : 0
        Avg_num_value_per_deduplicate_key: 0
        Num_leaf_page                   : 1
        Num_non_leaf_page               : 0
        Num_ovf_page                    : 0
        Num_total_page                  : 1
        Height                          : 1
        Avg_key_len                     : 0
        Avg_rec_len                     : 0
        Total_space                     : '16.0K'
        Total_used_space_non_ovf        : '120.0B'
        Total_free_space_non_ovf        : '15.8K'
        Total_used_space_ovf            : '0.0B'
        Total_free_space_ovf            : '0.0B'
        Avg_num_key_per_page_non_ovf    : 0
        Avg_free_space_per_page_non_ovf : '15.8K'
        Avg_num_ovf_page_per_key        : 0
        Avg_free_space_per_page_ovf     : '0.0B'
        Max_num_ovf_page_a_key          : 0
```

```
SHOW ALL INDEXES CAPACITY OF tbl1;
```

```
<00001> Table_name                          : 'dba.tbl1'
        Index_name                          : 'index_a'
        Btid                                : '(0|4160|4161)'
        Num_distinct_key                    : 0
        Total_value                         : 0
        Deduplicate_distinct_key            : 0
        Num_fence_key                       : 0
        Avg_num_value_per_key               : 0
        Avg_num_value_per_deduplicate_key: 0
        Num_leaf_page                       : 1
        Num_non_leaf_page                   : 0
        Num_ovf_page                        : 0
        Num_total_page                      : 1
        Height                              : 1
        Avg_key_len                         : 0
        Avg_rec_len                         : 0
        Total_space                         : '16.0K'
        Total_used_space_non_ovf            : '120.0B'
        Total_free_space_non_ovf            : '15.8K'
        Total_used_space_ovf                : '0.0B'
        Total_free_space_ovf                : '0.0B'
        Avg_num_key_per_page_non_ovf        : 0
        Avg_free_space_per_page_non_ovf     : '15.8K'
        Avg_num_ovf_page_per_key            : 0
        Avg_free_space_per_page_ovf         : '0.0B'
        Max_num_ovf_page_a_key              : 0
<00002> Table_name                          : 'dba.tbl1'
        Index_name                          : 'index_b'
        Btid                                : '(0|4224|4225)'
        Num_distinct_key                    : 0
        Total_value                         : 0
        Deduplicate_distinct_key            : 0
        Num_fence_key                       : 0
        Avg_num_value_per_key               : 0
        Avg_num_value_per_deduplicate_key: 0
        Num_leaf_page                       : 1
        Num_non_leaf_page                   : 0
        Num_ovf_page                        : 0
        Num_total_page                      : 1
        Height                              : 1
        Avg_key_len                         : 0
        Avg_rec_len                         : 0
        Total_space                         : '16.0K'
        Total_used_space_non_ovf            : '124.0B'
        Total_free_space_non_ovf            : '15.8K'
        Total_used_space_ovf                : '0.0B'
        Total_free_space_ovf                : '0.0B'
        Avg_num_key_per_page_non_ovf        : 0
        Avg_free_space_per_page_non_ovf     : '15.8K'
        Avg_num_ovf_page_per_key            : 0
```

```
        Avg_free_space_per_page_ovf       : '0.0B'
        Max_num_ovf_page_a_key            : 0
```

SHOW CRITICAL SECTIONS

Total critical section (hereafter CS) information of a database is shown.

```
SHOW CRITICAL SECTIONS;
```

This query has the following columns:

Column name	Type	Description
Index	INT	The index of CS
Name	VARCHAR(32)	The name of CS
Num_holders	VARCHAR(16)	The number of CS holders. This has one of these values: ‘N readers’, ‘1 writer’, ‘none’
Num_waiting_readers	INT	The number of waiting readers
Num_waiting_writers	INT	The number of waiting writers
Owner_thread_index	INT	The thread index of CS owner writer, NULL if no owner
Owner_tran_index	INT	Transaction index of CS owner writer, NULL if no owner

Column name	Type	Description
Total_enter_count	BIGINT	Total count of enterers
Total_waiter_count	BIGINT	Total count of waiters
Waiting_promoter_thread_index	INT	The thread index of waiting promoter, NULL if no waiting promoter
Max_waiting_msecs	NUMERIC(10,3)	Maximum waiting time (millisecond)
Total_waiting_msecs	NUMERIC(10,3)	Total waiting time (millisecond)

The following shows the examples of this syntax.

```
SHOW CRITICAL SECTIONS;
```

```
Index  Name                          Num_holders
Num_waiting_readers  Num_waiting_writers  Owner_thread_index
Owner_tran_index     Total_enter_count    Total_waiter_count
Waiting_promoter_thread_index  Max_waiting_msecs
Total_waiting_msecs
============================================================================
============================================================================
============================================================================
============================================
    0  'ER_LOG_FILE'
'none'                                    0
0                NULL              NULL
217                    0                          NULL
0.000                  0.000
    1  'ER_MSG_CACHE'
'none'                                    0
0                NULL              NULL
0                      0                        NULL
0.000                  0.000
    2  'WFG'
'none'                                    0
```

```
0                NULL               NULL
0                     0                          NULL
0.000                 0.000
    3  'LOG'
'none'                                  0
0                NULL               NULL
11                     0                          NULL
0.000                 0.000
    4  'LOCATOR_CLASSNAME_TABLE'
'none'                                  0
0                NULL               NULL
33                     0                          NULL
0.000                 0.000
    5  'QPROC_QUERY_TABLE'
'none'                                  0
0                NULL               NULL
3                     0                          NULL
0.000                 0.000
    6  'QPROC_LIST_CACHE'
'none'                                  0
0                NULL               NULL
1                     0                          NULL
0.000                 0.000
    7   'DISK_CHECK'
'none'                                  0
0                NULL               NULL
3                     0                          NULL
0.000                 0.000
    8  'CNV_FMT_LEXER'
'none'                                  0
0                NULL               NULL
0                     0                          NULL
0.000                 0.000
    9  'HEAP_CHNGUESS'
'none'                                  0
0                NULL               NULL
10                     0                          NULL
0.000                 0.000
    10  'TRAN_TABLE'
'none'                                  0
0                NULL               NULL
7                     0                          NULL
0.000                 0.000
    11  'CT_OID_TABLE'
'none'                                  0
0                NULL               NULL
0                     0                          NULL
0.000                 0.000
    12  'HA_SERVER_STATE'
'none'                                  0
0                NULL               NULL
2                     0                          NULL
```

```
0.000                   0.000
    13  'COMPACTDB_ONE_INSTANCE'
'none'                                  0
0                NULL               NULL
0                     0                          NULL
0.000                   0.000
    14  'ACL'
'none'                                  0
0                NULL               NULL
0                     0                          NULL
0.000                   0.000
    15  'PARTITION_CACHE'
'none'                                  0
0                NULL               NULL
1                     0                          NULL
0.000                   0.000
    16  'EVENT_LOG_FILE'
'none'                                  0
0                NULL               NULL
0                     0                          NULL
0.000                   0.000
    17  'LOG_ARCHIVE'
'none'                                  0
0                NULL               NULL
0                     0                          NULL
0.000                   0.000
    18  'ACCESS_STATUS'
'none'                                  0
0                NULL               NULL
1                     0                          NULL
0.000                   0.000
```

SHOW TRANSACTION TABLES

It shows internal information of transaction descriptors which is internal data structure to manage each transaction. It only shows valid transactions and the result may not be a consistent snapshot of a transaction descriptor.

```
SHOW { TRAN | TRANSACTION } TABLES [ WHERE expr ];
```

This query has the following columns:

Column name	Type	Description
Tran_index	INT	Index on the transaction table or NULL for

Column name	Type	Description
		unassigned transaction descriptor slot
Tran_id	INT	Transaction Identifier
Is_loose_end	INT	0 for Ordinary transactions, 1 for loose-end transactions
State	VARCHAR(64)	State of the transaction. Either one of the followings: 'TRAN_RECOVERY', 'TRAN_ACTIVE', 'TRAN_UNACTIVE_COMMITTED', 'TRAN_UNACTIVE_WILL_COMMIT', 'TRAN_UNACTIVE_COMMITTED_WITH_POSTPONE', 'TRAN_UNACTIVE_ABORTED', 'TRAN_UNACTIVE_UNILATERALLY_ABORTED', 'TRAN_UNACTIVE_2PC_PREPARE', 'TRAN_UNACTIVE_2PC_COLLECTING_PARTICIPANT_VOTES', 'TRAN_UNACTIVE_2PC_ABORT_DECISION', 'TRAN_UNACTIVE_2PC_COMMIT_DECISION', 'TRAN_UNACTIVE_COMMITTED_INFORMING_PARTICIPANTS', 'TRAN_UNACTIVE_ABORTED_INFORMING_PARTICIPANTS','TRAN_STATE_UNKNOWN'
Isolation	VARCHAR(64)	Isolation level of the transaction. Either one of

Column name	Type	Description
		the followings: ‘SERIALIZABLE’, ‘REPEATABLE READ’, ‘COMMITTED READ’, ‘TRAN_UNKNOWN_ISOLATION’
Wait_msecs	INT	Wait until this number of milliseconds for locks.
Head_lsa	VARCHAR(64)	First log address of transaction.
Tail_lsa	VARCHAR(64)	Last log record address of transaction.
Undo_next_lsa	VARCHAR(64)	Next log record address of transaction for UNDO purposes.
Postpone_next_lsa	VARCHAR(64)	Next address of a postpone record to be executed.
Savepoint_lsa	VARCHAR(64)	Address of last save-point.
Topop_lsa	VARCHAR(64)	Address of last top operation.
Tail_top_result_lsa	VARCHAR(64)	Address of last partial abort/ commit.
Client_id	INT	Unique identifier of client application bind to transaction.

Column name	Type	Description
Client_type	VARCHAR(40)	Type of the client. Either one of the followings: 'SYSTEM_INTERNAL', 'DEFAULT', 'CSQL', 'READ_ONLY_CSQL', 'BROKER', 'READ_ONLY_BROKER', 'SLAVE_ONLY_BROKER', 'ADMIN_UTILITY', 'ADMIN_CSQL', 'LOG_COPIER', 'LOG_APPLIER', 'RW_BROKER_REPLICA_ONLY', 'RO_BROKER_REPLICA_ONLY', 'SO_BROKER_REPLICA_ONLY', 'ADMIN_CSQL_WOS', 'UNKNOWN'
Client_info	VARCHAR(256)	General information of client application.
Client_db_user	VARCHAR(40)	Current login database account from client application.
Client_program	VARCHAR(256)	Program name of client application.
Client_login_user	VARCHAR(16)	Current login user of OS which running the client application.
Client_host	VARCHAR(64)	Hostname of client application.
Client_pid	INT	Process id of client application.

Column name	Type	Description
Topop_depth	INT	Depth of nested top operation.
Num_unique_btrees	INT	Number of unique btrees contained in unique_stat_info array.
Max_unique_btrees	INT	Size of unique_stat_info_array.
Interrupt	INT	The flag of whether or not interrupt current transaction. 0 for No, 1 for Yes.
Num_transient_classnames	INT	Number of transient classnames by this transaction.
Repl_max_records	INT	Capacity of replication record array.
Repl_records	VARCHAR(20)	Replication record buffer array, display address pointer as 0x12345678 or NULL for 0x00000000.
Repl_current_index	INT	Current position of replication record in the array.

Column name	Type	Description
Repl_append_index	INT	Current position of appended record in the array.
Repl_flush_marked_index	INT	Index of flush marked replication record at first.
Repl_insert_lsa	VARCHAR(64)	Insert Replication target LSA.
Repl_update_lsa	VARCHAR(64)	Update Replication target LSA.
First_save_entry	VARCHAR(20)	First save entry for the transaction, display address pointer as 0x12345678 or NULL for 0x00000000.
Tran_unique_stats	VARCHAR(20)	Local statistical info for multiple row. display address pointer as 0x12345678 or NULL for 0x00000000.
Modified_class_list	VARCHAR(20)	List of dirty classes, display address pointer as 0x12345678 or NULL for 0x00000000.
Num_temp_files	INT	Number of temporary files.

Column name	Type	Description
Waiting_for_res	VARCHAR(20)	Waiting resource. Just display address pointer as 0x12345678 or NULL for 0x00000000.
Has_deadlock_priority	INT	Whether or not have deadlock priority. 0 for No, 1 for Yes.
Suppress_replication	INT	Suppress writing replication logs when flag is set.
Query_timeout	DATETIME	A query should be executed before query_timeout time or NULL for waiting until query complete.
Query_start_time	DATETIME	Current query start time or NULL for query completed.
Tran_start_time	DATETIME	Current transaction start time or NULL for transaction completed.
Xasl_id	VARCHAR(64)	vpid:(volid|pageid),vfid: (volid|pageid) or NULL for query completed.
Disable_modifications	INT	Disable modification if greater than zero.
Abort_reason	VARCHAR(40)	Reason of transaction aborted. Either one of the followings: 'NORMAL',

Column name	Type	Description
		'ABORT_DUE_TO_DEADLOCK', 'ABORT_DUE_ROLLBACK_ON_ESCALATION'

The following shows the examples of the statement.

```
SHOW TRAN TABLES WHERE CLIENT_TYPE = 'CSQL';
```

```
=== <Result of SELECT Command in Line 1> ===

<00001> Tran_index              : 1
        Tran_id                 : 58
        Is_loose_end            : 0
        State                   : 'ACTIVE'
        Isolation               : 'COMMITTED READ'
        Wait_msecs              : -1
        Head_lsa                : '(-1|-1)'
        Tail_lsa                : '(-1|-1)'
        Undo_next_lsa           : '(-1|-1)'
        Postpone_next_lsa       : '(-1|-1)'
        Savepoint_lsa           : '(-1|-1)'
        Topop_lsa               : '(-1|-1)'
        Tail_top_result_lsa     : '(-1|-1)'
        Client_id               : 108
        Client_type             : 'CSQL'
        Client_info             : ''
        Client_db_user          : 'PUBLIC'
        Client_program          : 'csql'
        Client_login_user       : 'cubrid'
        Client_host             : 'cubrid001'
        Client_pid              : 13190
        Topop_depth             : 0
        Num_unique_btrees       : 0
        Max_unique_btrees       : 0
        Interrupt               : 0
        Num_transient_classnames: 0
        Repl_max_records        : 0
        Repl_records            : NULL
        Repl_current_index      : 0
        Repl_append_index       : -1
        Repl_flush_marked_index : -1
        Repl_insert_lsa         : '(-1|-1)'
        Repl_update_lsa         : '(-1|-1)'
        First_save_entry        : NULL
```

```
        Tran_unique_stats       : NULL
        Modified_class_list     : NULL
        Num_temp_files          : 0
        Waiting_for_res         : NULL
        Has_deadlock_priority   : 0
        Suppress_replication    : 0
        Query_timeout           : NULL
        Query_start_time        : 03:10:11.425 PM 02/04/2016
        Tran_start_time         : 03:10:11.425 PM 02/04/2016
        Xasl_id                 : 'vpid: (32766|50), vfid: (32766|
43)'
        Disable_modifications   : 0
        Abort_reason            : 'NORMAL'
```

SHOW THREADS

It shows internal information of each thread. The results are sorted by “Index” column with ascending order and may not be a consistent snapshot of thread entries. The statement under SA MODE shows an empty result.

```
SHOW THREADS [ WHERE EXPR ];
```

This query has the following columns:

Column name	Type	Description
Index	INT	Thread entry index.
Jobq_index	INT	Job queue index only for worker threads. NULL for non-worker threads.
Thread_id	BIGINT	Thread id.
Tran_index	INT	Transaction index to which this thread belongs. If no related tran index, NULL.
Type	VARCHAR(8)	Thread type. Either one of the followings: ‘MASTER’,

Column name	Type	Description
		'SERVER', 'WORKER', 'DAEMON', 'VACUUM_MASTER', 'VACUUM_WORKER', 'NONE', 'UNKNOWN'.
Status	VARCHAR(8)	Thread status. Either one of the followings: 'FREE', 'RUN', 'WAIT', 'CHECK'.
Resume_status	VARCHAR(32)	Resume status. Either one of the followings: 'RESUME_NONE', 'RESUME_DUE_TO_INTERRUPT', 'RESUME_DUE_TO_SHUTDOWN', 'PGBUF_SUSPENDED', 'PGBUF_RESUMED', 'JOB_QUEUE_SUSPENDED', 'JOB_QUEUE_RESUMED', 'CSECT_READER_SUSPENDED', 'CSECT_READER_RESUMED', 'CSECT_WRITER_SUSPENDED', 'CSECT_WRITER_RESUMED', 'CSECT_PROMOTER_SUSPENDED', 'CSECT_PROMOTER_RESUMED', 'CSS_QUEUE_SUSPENDED', 'CSS_QUEUE_RESUMED', 'QMGR_ACTIVE_QRY_SUSPENDED', 'QMGR_ACTIVE_QRY_RESUMED', 'QMGR_MEMBUF_PAGE_SUSPENDED', 'QMGR_MEMBUF_PAGE_RESUMED', 'HEAP_CLSREPR_SUSPENDED',

Column name	Type	Description
		'HEAP_CLSREPR_RESUMED', 'LOCK_SUSPENDED', 'LOCK_RESUMED', 'LOGWR_SUSPENDED', 'LOGWR_RESUMED'
Net_request	VARCHAR(64)	The net request name in net_requests array, such as: 'LC_ASSIGN_OID'. If not request name, shows NULL
Conn_client_id	INT	Client id whom this thread is responding, if no client id, shows NULL
Conn_request_id	INT	Request id which this thread is processing, if no request id, shows NULL
Conn_index	INT	Connection index, if not connection index, shows NULL
Last_error_code	INT	Last error code
Last_error_msg	VARCHAR(256)	Last error message, if message length is more than 256, it will be truncated, If no error message, shows NULL

Column name	Type	Description
Private_heap_id	VARCHAR(20)	The address of id of thread private memory allocator, such as: 0x12345678. If no related heap id, shows NULL.
Query_entry	VARCHAR(20)	The address of the QMGR_QUERY_ENTRY*, such as: 0x12345678, if no related QMGR_QUERY_ENTRY, shows NULL.
Interrupted	INT	0 or 1, is this request/transaction interrupted
Shutdown	INT	0 or 1, is server going down?
Check_interrupt	INT	0 or 1
Wait_for_latch_promote	INT	0 or 1, whether this thread is waiting for latch promotion.
Lockwait_blocked_mode	VARCHAR(24)	Lockwait blocked mode. Either one of the followings: 'NULL_LOCK', 'IS_LOCK', 'S_LOCK', 'IS_LOCK', 'IX_LOCK', 'SIX_LOCK', 'X_LOCK', 'SCH_M_LOCK', 'UNKNOWN'
Lockwait_start_time	DATETIME	Start blocked time, if not in blocked state, shows NULL
Lockwait_msecs	INT	

Column name	Type	Description
		Time in milliseconds, if not in blocked state, shows NULL
Lockwait_state	VARCHAR(24)	The lock wait state such as: 'SUSPENDED', 'RESUMED', 'RESUMED_ABORTED_FIRST', 'RESUMED_ABORTED_OTHER', 'RESUMED_DEADLOCK_TIMEOUT', 'RESUMED_TIMEOUT', 'RESUMED_INTERRUPT'. If not in blocked state, shows NULL
Next_wait_thread_index	INT	The next wait thread index, if not exist, shows NULL
Next_tran_wait_thread_index	INT	The next wait thread index in lock manager, if not exist, shows NULL
Next_worker_thread_index	INT	The next worker thread index in css_Job_queue.worker_thrd_list, if not exist, shows NULL

The following shows the examples of the statement.

```
SHOW THREADS WHERE RESUME_STATUS != 'RESUME_NONE' AND STATUS !=
'FREE';
```

```
=== <Result of SELECT Command in Line 1> ===
<00001> Index                    : 183
        Jobq_index               : 3
        Thread_id                : 140077788813056
        Tran_index               : 3
        Type                     : 'WORKER'
```

```
        Status                      : 'RUN'
        Resume_status               : 'JOB_QUEUE_RESUMED'
        Net_request                 : 'QM_QUERY_EXECUTE'
        Conn_client_id              : 108
        Conn_request_id             : 196635
        Conn_index                  : 3
        Last_error_code             : 0
        Last_error_msg              : NULL
        Private_heap_id             : '0x02b3de80'
        Query_entry                 : '0x7f6638004cb0'
        Interrupted                 : 0
        Shutdown                    : 0
        Check_interrupt             : 1
        Wait_for_latch_promote      : 0
        Lockwait_blocked_mode       : NULL
        Lockwait_start_time         : NULL
        Lockwait_msecs              : NULL
        Lockwait_state              : NULL
        Next_wait_thread_index      : NULL
        Next_tran_wait_thread_index: NULL
        Next_worker_thread_index    : NULL
<00002> Index                       : 192
        Jobq_index                  : 2
        Thread_id                   : 140077779339008
        Tran_index                  : 2
        Type                        : 'WORKER'
        Status                      : 'WAIT'
        Resume_status               : 'LOCK_SUSPENDED'
        Net_request                 : 'LC_FIND_LOCKHINT_CLASSOIDS'
        Conn_client_id              : 107
        Conn_request_id             : 131097
        Conn_index                  : 2
        Last_error_code             : 0
        Last_error_msg              : NULL
        Private_heap_id             : '0x02bcdf10'
        Query_entry                 : NULL
        Interrupted                 : 0
        Shutdown                    : 0
        Check_interrupt             : 1
        Wait_for_latch_promote      : 0
        Lockwait_blocked_mode       : 'SCH_S_LOCK'
        Lockwait_start_time         : 10:47:45.000 AM 02/03/2016
        Lockwait_msecs              : -1
        Lockwait_state              : 'SUSPENDED'
        Next_wait_thread_index      : NULL
        Next_tran_wait_thread_index: NULL
        Next_worker_thread_index    : NULL
```

SHOW JOB QUEUES

It shows the status of job queue. The statement under SA MODE shows an empty result.

```
SHOW JOB QUEUES;
```

This query has the following columns:

Column name	Type	Description
Jobq_index	INT	The index of job queue
Num_total_workers	INT	Total number of work threads of the queue
Num_busy_workers	INT	The number of busy worker threads of the queue
Num_connection_workers	INT	The number of connection worker threads of the queue

SHOW PAGE BUFFER STATUS

It shows the status of the data page buffer pool.

```
SHOW PAGE BUFFER STATUS;
```

This query has the following columns:

Column name	Type	Description
Hit_rate	NUMERIC(13,10)	The buffer pool hit rate (since the last printout)
Num_hit	BIGINT	The number of buffer hits (since the last printout)

Column name	Type	Description
Num_page_request	BIGINT	The number of page requests (since the last printout)
Pool_size	INT	Buffer pool size in pages
Page_size	INT	Data page size
Free_pages	INT	The number of free pages in the buffer pool
Victim_candidate_pages	INT	The number of victim candidate pages in the cold LRU area
Clean_pages	INT	The number of clean pages in the buffer pool
Dirty_pages	INT	The number of dirty pages in the buffer pool
Num_index_pages	INT	The number of index pages in the buffer pool
Num_data_pages	INT	The number of data pages in the buffer pool
Num_system_pages	INT	The number of system pages in the buffer pool
Num_temp_pages	INT	The number of temp pages in the buffer pool

Column name	Type	Description
Num_pages_created	BIGINT	The number of pages created in the buffer pool (since the last printout)
Num_pages_written	BIGINT	The number of pages written to disk in the buffer pool (since the last printout)
Pages_written_rate	NUMERIC(20,10)	The number of pages written per second (since the last printout)
Num_pages_read	BIGINT	The number of pages read from disk in the buffer pool (since the last printout)
Pages_read_rate	NUMERIC(20,10)	The number of pages read per second (since the last printout)
Num_flusher_waiting_threads	INT	The number of waiting threads for flusher

The following shows the examples of this syntax.

```
-- csql> ;line on
SHOW PAGE BUFFER STATUS;
```

```
<00001> Hit_rate                  : 0.0000000000
        Num_hit                   : 0
        Num_page_request          : 0
        Pool_size                 : 32768
        Page_size                 : 16392
        Free_pages                : 32739
        Victim_candidate_pages    : 0
        Clean_pages               : 32767
```

```
        Dirty_pages                : 1
        Num_index_pages            : 2
        Num_data_pages             : 15
        Num_system_pages           : 12
        Num_temp_pages             : 0
        Num_pages_created          : 0
        Num_pages_written          : 0
        Pages_written_rate         : 0.0000000000
        Num_pages_read             : 0
        Pages_read_rate            : 0.0000000000
        Num_flusher_waiting_threads: 0
```

User Schema

A schema is a logical collection of database objects. The object can be identified through the schema name, and the schema object is not stored in a physically separate storage space. An object can only exist in one schema and cannot exist in another schema at the same time. A schema cannot be created by a user, and when a database user is created, the user owns a single schema with the same name as the username. A schema contains tables, serials, and triggers. An object in a schema must have a unique name within the same schema, but it can have a name that is duplicated with an object in another schema. Table, serial, and trigger names are prefixed with the schema name, so the schema can be identified by the name.

To access the object, the user must use the path expression of “schema_name.table_name”. If the user’s schema is the same as the object’s schema (owner), the schema name can be omitted from the path expression. If the schema name is omitted, the user’s schema name is used. Even if the path expression of “schema_name.table_name” is used, the user must have permission to use the object.

Note

Since version 11.2, “Schema ≒ Database” has been changed to “Schema ≒ User”. Before version 11.2, there was only a single schema in the database. Since version 11.2, schema is available for each user. It is not possible to access a table owned by another user only by the table name. Tables can be accessed using the schema (owner) name as a prefix.

The following difference occurs in demodb, which is the default example database.

```
csql -u dba demodb
```

```
SELECT name FROM athlete LIMIT 1;
```

```
    ERROR: before ' ; '
    Unknown class "dba.athlete".

SELECT name FROM public.athlete LIMIT 1;

      name
    ======================
      'Fernandez Jesus'
```

A unique_name column has been added to the system tables (_db_class, db_serial, db_trigger) that manage tables, serials, and triggers. The unique_name column stores names prefixed with schema names. In the unique_name column of _db_class, the system table name is not prefixed with the schema name.

```
csql -u public demodb
```

```
CREATE TABLE table_1 (column_1 INTEGER);
CREATE TABLE table_2 (column_1 INTEGER);
CREATE TRIGGER trigger_1 AFTER INSERT ON table_1 EXECUTE INSERT INTO
table_2 VALUES (obj.column_1);
```

```
csql -u dba demodb
```

```
SELECT unique_name, class_name, owner.name FROM _db_class ORDER BY
unique_name;

      unique_name              class_name               owner.name
    ============================================================
      '_db_attribute'          '_db_attribute'          'DBA'
      '_db_auth'               '_db_auth'               'DBA'
      '_db_charset'            '_db_charset'            'DBA'
      '_db_class'              '_db_class'              'DBA'
      '_db_collation'          '_db_collation'          'DBA'
          ...
      'db_attr_setdomain_elm'  'db_attr_setdomain_elm'  'DBA'
      'db_attribute'           'db_attribute'           'DBA'
      'db_auth'                'db_auth'                'DBA'
      'db_authorization'       'db_authorization'       'DBA'
      'db_authorizations'      'db_authorizations'      'DBA'
          ...
      'public.athlete'         'athlete'                'PUBLIC'
      'public.code'            'code'                   'PUBLIC'
      'public.event'           'event'                  'PUBLIC'
      'public.game'            'game'                   'PUBLIC'
```

```
      'public.history'         'history'                'PUBLIC'

SELECT unique_name, name, owner.name FROM db_serial ORDER BY
unique_name;

      unique_name               name               owner.name
    ============================================================
      'public.athlete_ai_code'  'athlete_ai_code'  'PUBLIC'
      'public.event_no'         'event_no'         'PUBLIC'
      'public.stadium_no'       'stadium_no'       'PUBLIC'

SELECT unique_name, name, owner.name FROM db_trigger ORDER BY
unique_name;

      unique_name         name         owner.name
    ================================================
      'public.trigger_1'  'trigger_1'  'PUBLIC'
```

System Catalog

You can easily get various schema information from the SQL statement by using the system catalog virtual class. For example, you can get the following schema information by using the catalog virtual class.

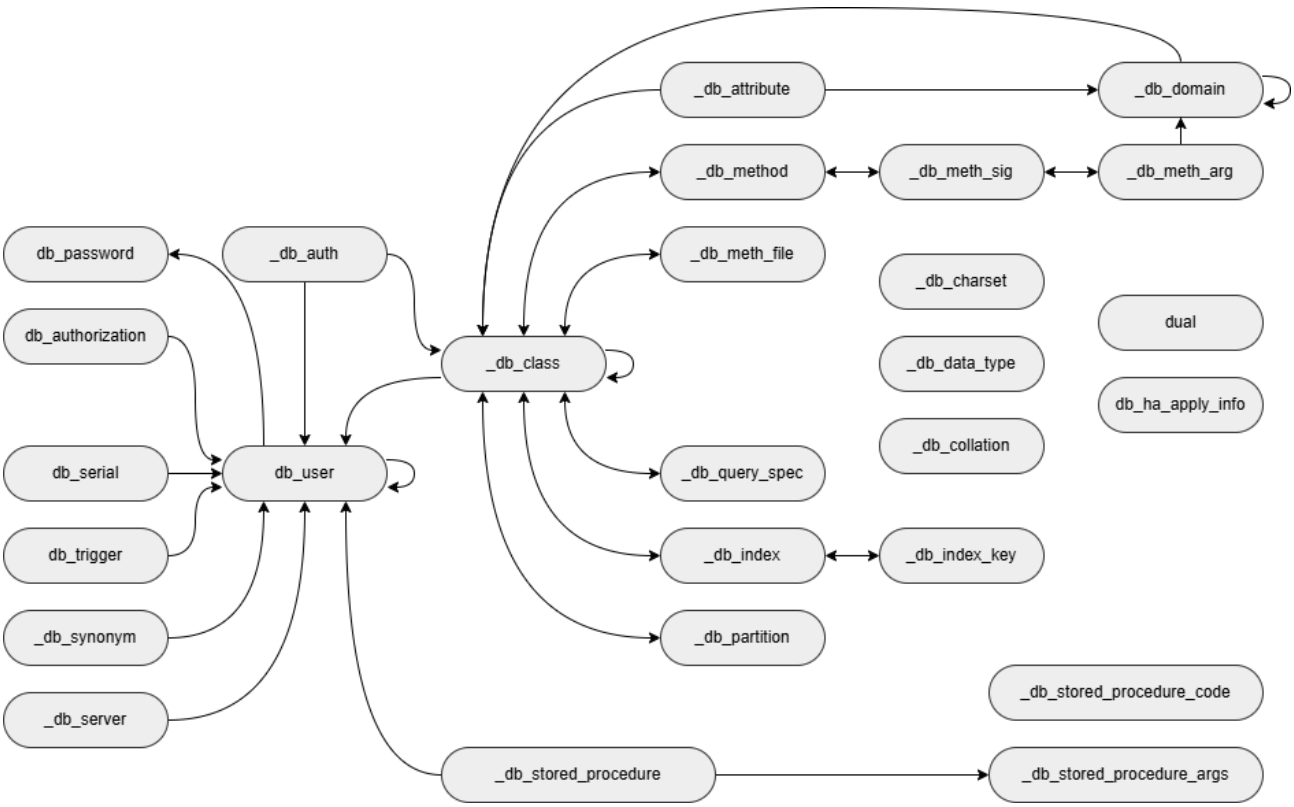
```
-- Classes that refer to the 'db_user' class
SELECT class_name
FROM db_attribute
WHERE domain_class_name = 'db_user';

-- The number of classes that the current user can access
SELECT COUNT(*)
FROM db_class;

-- Attribute of the 'db_user' class
SELECT attr_name, data_type
FROM db_attribute
WHERE class_name = 'db_user';
```

System Catalog Classes

To define a catalog virtual class, define a catalog class first. The figure below shows catalog classes to be added and their relationships. The arrows represent the reference relationship between classes, and the classes that start with an underline (_) are catalog classes.



Added catalog classes represent information about all classes, attributes and methods in the database. Catalog classes are made up of class composition hierarchy and designed to have OIDs of catalog class instances for cross reference.

Class Name	Description
_db_class	Class information
_db_attribute	Attribute information
_db_domain	Domain information
_db_charset	Charset information
_db_collation	Collation information
_db_method	Method information
_db_meth_sig	

Class Name	Description
	Configuration information of C functions on the method
_db_meth_arg	Method argument information
_db_meth_file	File information in which the method is defined
_db_query_spec	The SQL statement of a virtual class
_db_index	Index information
_db_index_key	Key information on an index
_db_auth	User authorization information of classes
_db_data_type	The data type supported by CUBRID
_db_partition	Partition information
_db_stored_procedure	Stored procedure information
_db_stored_procedure_args	Stored procedure argument information
_db_stored_procedure_code	Stored procedure code information
_db_server	Server information for DBLink
_db_synonym	Target object information of synonyms

Class Name	Description
db_user	User information
db_authorization	User authorization information of classes
db_serial	Serial information
db_trigger	Trigger information
db_ha_apply_info	The progress status the **applylogdb** utility applies replication logs
dual	Dummy table

_db_class

Represents class information. An index for unique_name and an index for class_name and owner are created.

Attribute Name	Data Type	Description
class_of	object	A class object. Represents a meta information object for the class stored in the system.
unique_name	VARCHAR(255)	Class name prefixed with schema name
class_name	VARCHAR(255)	Class name
class_type	INTEGER	

Attribute Name	Data Type	Description
		0 for a class, and 1 for a virtual class
is_system_class	INTEGER	0 for a user-defined class, and 1 for a system class
owner	db_user	Class owner
inst_attr_count	INTEGER	The number of instance attributes
class_attr_count	INTEGER	The number of class attributes
shared_attr_count	INTEGER	The number of shared attributes
inst_meth_count	INTEGER	The number of instance methods
class_meth_count	INTEGER	The number of class methods
collation_id	INTEGER	Collation id
tde_algorithm	INTEGER	TDE encryption algorithm 0: NONE, 1: AES, 2: ARIA
sub_classes	SEQUENCE OF _db_class	Class one level down
super_classes	SEQUENCE OF _db_class	Class one level up

Attribute Name	Data Type	Description
inst_attrs	SEQUENCE OF _db_attribute	Instance attribute
class_attrs	SEQUENCE OF _db_attribute	Class attribute
shared_attrs	SEQUENCE OF _db_attribute	Shared attribute
inst_meths	SEQUENCE OF _db_method	Instance method
class_meths	SEQUENCE OF _db_method	Class method
meth_files	SEQUENCE OF _db_meth_file	File path in which the function for the method is located
query_specs	SEQUENCE OF _db_query_spec	SQL definition statement for a virtual class
indexes	SEQUENCE OF _db_index	Index created in the class
comment	VARCHAR(2048)	Comment to describe the class
partition	SEQUENCE of _db_partition	Partition information

The following example shows how to retrieve all sub classes under the class owned by user '**PUBLIC**' (for the child class *female_event* in the result, see the example in ADD SUPERCLASS Clause).

```
SELECT class_name, SEQUENCE(SELECT class_name FROM _db_class s WHERE
s IN c.sub_classes)
FROM _db_class c
WHERE c.owner.name = 'PUBLIC' AND c.sub_classes IS NOT NULL;
```

```
  class_name            sequence((select class_name from _db_class s
where s in c.sub_classes))
============================================
  'event'               {'female_event'}
```

Note

All examples of system catalog classes have been written in the csql utility. In this example, **--no-auto-commit** (inactive mode of autocommit) and **-u** (specifying user DBA) options are used.

```
% csql --no-auto-commit -u dba demodb
```

_db_attribute

Represents attribute information. An index for class_of, attr_name and attr_type is created.

Attribute Name	Data Type	Description
class_of	_db_class	Class to which the attribute belongs
attr_name	VARCHAR(255)	Attribute name
attr_type	INTEGER	Type defined for the attribute. 0 for an instance attribute, 1 for a class attribute, and 2 for a shared attribute.
from_class_of	_db_class	If the attribute is inherited, the super class in which the attribute is defined is specified. Otherwise, **NULL** is specified.

Attribute Name	Data Type	Description
from_attr_name	VARCHAR(255)	Inherited attribute. If an attribute name has changed to resolve a name conflict, the original name define in the super class is specified. Otherwise, **NULL** is specified.
def_order	INTEGER	Order of attributes in the class. Begins with 0. If the attribute is inherited, the order is the one defined in the super class. For example, if class y inherits attribute a from class x and a was first defined in x, def_order becomes 0.
data_type	INTEGER	Data type of the attribute. One of the values specified in the “Data Types Supported by CUBRID” table below.
default_value	VARCHAR(255)	Default value. Stores as a character string regardless of data types. If there is no default value, NULL. If the default value is **NULL**, **NULL** is used. If the data type is an object, ‘volume id | page id | slot id’ is used. If the data type is a collection, ‘{element 1, element 2, ... is used.
domains	SEQUENCE OF _db_domain	

Attribute Name	Data Type	Description
		Domain information of the data type
is_nullable	INTEGER	0 if a not null constraint is configured, and 1 otherwise.
comment	VARCHAR(2048)	Comment to describe the attribute.

Data Types Supported by CUBRID

Value	Meaning	Value	Meaning
1	INTEGER	23	BIT
2	FLOAT	24	VARBIT
3	DOUBLE	25	CHAR
4	STRING	27	VARCHAR
5	OBJECT	31	BIGINT
6	SET	32	DATETIME
7	MULTISET	33	BLOB
8	SEQUENCE	34	CLOB
9	ELO	35	ENUM

Value	Meaning	Value	Meaning
10	TIME	36	TIMESTAMPTZ
11	TIMESTAMP	37	TIMESTAMPLTZ
12	DATE	38	DATETIMETZ
18	SHORT	39	DATETIMELTZ
22	NUMERIC	40	JSON

Character Sets Supported by CUBRID

Value	Meaning
0	US English - ASCII encoding
2	Binary
3	Latin 1 - ISO 8859 encoding
4	KSC 5601 1990 - EUC encoding
5	UTF8 - UTF8 encoding

The following example shows how to retrieve user classes (from_class_of.is_system_class = 0) among the ones owned by user '**PUBLIC**".

```
SELECT class_of.class_name, attr_name
FROM _db_attribute
WHERE class_of.owner.name = 'PUBLIC' AND
from_class_of.is_system_class = 0
ORDER BY 1, def_order;
```

```
class_of.class_name   attr_name
============================================
  'female_event'        'code'
  'female_event'        'sports'
  'female_event'        'name'
  'female_event'        'gender'
  'female_event'        'players'
```

_db_domain

Represents domain information. Indexes for object_of and data_type are created.

Attribute Name	Data Type	Description
object_of	object	Attribute that refers to the domain, which can be a method parameter or domain
data_type	INTEGER	Data type of the domain (a value in the “Value” column of the “Data Types Supported by CUBRID” table in _db_attribute)
prec	INTEGER	Precision of the data type. 0 is used if the precision is not specified.
scale	INTEGER	Scale of the data type. 0 is used if the scale is not specified.
class_of	_db_class	Domain class if the data type is an object, **NULL** otherwise.

Attribute Name	Data Type	Description
code_set	INTEGER	Character set (value of table “character sets supported by CUBRID” in _db_attribute) if it is character data type. 0 otherwise.
collation_id	INTEGER	Collation id
enumeration	SEQUENCE OF CHARACTER VARYING(1073741823)	String printed enumeration type definition
set_domains	SEQUENCE OF _db_domain	Domain information about the data type of collection element if it is collection data type. **NULL** otherwise.
json_schema	CHARACTER VARYING(1073741823)	Schema information defining the JSON data structure if it is a JSON data type. **NULL** otherwise.

_db_charset

Represents charset information.

Attribute Name	Data type	Description
charset_id	INTEGER	Charset ID
charset_name	CHARACTER VARYING(32)	Charset name
default_collation	INTEGER	Default collation ID

Attribute Name	Data type	Description
char_size	INTEGER	One character’s byte size

_db_collation

The information on collation.

Attribute Name	Data Type	Description
coll_id	INTEGER	Collation ID
coll_name	VARCHAR(32)	Collation name
charset_id	INTEGER	Charset ID
built_in	INTEGER	Built-in or not while installing the product (0: Not built-in, 1: Built-in)
expansions	INTEGER	Expansion support (0: Not supported, 1: Supported)
contractions	INTEGER	Contraction support (0: Not supported, 1: Supported)
uca_strength	INTEGER	Weight strength
checksum	VARCHAR(32)	Checksum of a collation file

_db_method

Represents method information. An index for class_of and meth_name is created.

Attribute Name	Data Type	Description
class_of	_db_class	Class to which the method belongs
meth_name	VARCHAR(255)	Method name
meth_type	INTEGER	Type of the method defined in the class. 0 for an instance method, and 1 for a class method.
from_class_of	_db_class	If the method is inherited, the super class in which it is defined is used otherwise **NULL**
from_meth_name	VARCHAR(255)	If the method is inherited and its name is changed to resolve a name conflict, the original name defined in the super class is used otherwise **NULL**
signatures	SEQUENCE OF _db_meth_sig	C function executed when the method is called

The following example shows how to retrieve class methods of the class with a class method (c.class_meth_count > 0), among classes owned by user 'DBA.'

```
SELECT class_name, SEQUENCE(SELECT meth_name
                            FROM _db_method m
                            WHERE m in c.class_meths)
FROM _db_class c
```

```
WHERE c.owner.name = 'DBA' AND c.class_meth_count > 0
ORDER BY 1;
```

```
  class_name            sequence((select meth_name from _db_method m
where m in c.class_meths))
============================================
  'db_serial'           {'change_serial_owner'}
  'db_authorizations'   {'add_user', 'drop_user', 'find_user',
'print_authorizations', 'info', 'change_owner',
                         'change_triggr_owner', 'get_owner'}
  'db_authorization'    {'check_authorization'}
  'db_user'             {'add_user', 'drop_user', 'find_user',
'login'}
  'db_root'             {'add_user', 'drop_user', 'find_user',
'print_authorizations', 'info', 'change_owner',
                         'change_triggr_owner', 'get_owner',
'change_sp_owner'}
```

_db_meth_sig

Represents configuration information of C functions on the method. An index for meth_of is created.

Attribute Name	Data Type	Description
meth_of	_db_method	Method for the function information
func_name	VARCHAR(255)	Function name
arg_count	INTEGER	The number of input arguments of the function
return_value	SEQUENCE OF _db_meth_arg	Return value of the function
arguments	SEQUENCE OF _db_meth_arg	Input arguments of the function

_db_meth_arg

Represents method argument information. An index for meth_sig_of is created.

Attribute Name	Data Type	Description
meth_sig_of	_db_meth_sig	Information of the function to which the argument belongs
data_type	INTEGER	Data type of the argument (a value in the "Value" column of the "Data Types Supported by CUBRID" in _db_attribute)
index_of	INTEGER	Order of the argument listed in the function definition. Begins with 0 if it is a return value, and 1 if it is an input argument.
domains	SEQUENCE OF _db_domain	Domain of the argument

_db_meth_file

Represents information of a file in which a function is defined. An index for class_of is created.

Attribute Name	Data Type	Description
class_of	_db_class	Class to which the method file information belongs

Attribute Name	Data Type	Description
from_class_of	_db_class	If the file information is inherited, the super class in which it is defined is used otherwise, **NULL**
path_name	VARCHAR(255)	File path in which the method is located

_db_query_spec

Represents the SQL statement of a virtual class. An index for class_of is created.

The data type of attribute 'spec' is VARCHAR (4096) for prior versions including 10.1 Patch 3.

Attribute Name	Data Type	Description	Classification (10.1 Only)
class_of	_db_class	Class information of the virtual class	
spec	VARCHAR(1073741823)	SQL definition statement of the virtual class	10.1 Patch 4 or later
	VARCHAR(4096)		10.1 Patch 3 or earlier

_db_index

Represents index information. An index for class_of is created.

Attribute Name	Data Type	Description
class_of	_db_class	Class to which to index belongs
index_name	varchar(255)	Index name
is_unique	INTEGER	1 if the index is unique, and 0 otherwise.
key_count	INTEGER	The number of attributes that comprise the key
key_attrs	SEQUENCE OF _db_index_key	Attributes that comprise the key
is_reverse	INTEGER	1 for a reverse index, and 0 otherwise.
is_primary_key	INTEGER	1 for a primary key, and 0 otherwise.
is_foreign_key	INTEGER	1 for a foreign key, and 0 otherwise.
filter_expression	VARCHAR(255)	The conditions of filtered indexes
have_function	INTEGER	1 for a function index, and 0 otherwise.
comment	VARCHAR (1024)	Comment to describe the index

Attribute Name	Data Type	Description
status	INTEGER	Index status

The following example shows how to retrieve names of indexes that belong to the class.

```
SELECT class_of.class_name, index_name
FROM _db_index
ORDER BY 1;
```

```
  class_of.class_name   index_name
=============================================
  '_db_attribute'
'i__db_attribute_class_of_attr_name_attr_type'
  '_db_auth'            'i__db_auth_grantee'
  '_db_auth'            'i__db_auth_object_of'
  '_db_class'           'i__db_class_unique_name'
  '_db_class'           'i__db_class_class_name_owner'
  '_db_domain'          'i__db_domain_object_of'
  '_db_domain'          'i__db_domain_data_type'
  '_db_index'           'i__db_index_class_of'
  '_db_index_key'       'i__db_index_key_index_of'
  '_db_meth_arg'        'i__db_meth_arg_meth_sig_of'
  '_db_meth_file'       'i__db_meth_file_class_of'
  '_db_meth_sig'        'i__db_meth_sig_meth_of'
  '_db_method'          'i__db_method_class_of_meth_name'
  '_db_partition'       'i__db_partition_class_of_pname'
  '_db_query_spec'      'i__db_query_spec_class_of'
  '_db_server'          'pk__db_server_link_name_owner'
  '_db_stored_procedure'  'pk_db_stored_procedure_unique_name'
  '_db_stored_procedure_args'  'i__db_stored_procedure_args_sp_of'
  '_db_stored_procedure_code'  'pk__db_stored_procedure_code_name'
  '_db_synonym'         'pk__db_synonym_unique_name'
  '_db_synonym'         'i__db_synonym_name_owner_is_public'
  'athlete'             'pk_athlete_code'
  'code'                'pk_code'
  'db_ha_apply_info'    'u_db_ha_apply_info_db_name_copied_log_path'
  'db_serial'           'pk_db_serial_unique_name'
  'db_serial'           'u_db_serial_name_owner'
  'db_user'             'u_db_user_name'
  'event'               'pk_event_code'
  'game'                'pk_game_host_year_event_code_athlete_code'
  'game'                'fk_game_event_code'
  'game'                'fk_game_athlete_code'
  'history'             'pk_history_event_code_athlete'
  'nation'              'pk_nation_code'
```

```
   'olympic'              'pk_olympic_host_year'
   'participant'          'pk_participant_host_year_nation_code'
   'participant'          'fk_participant_host_year'
   'participant'          'fk_participant_nation_code'
   'record'
'pk_record_host_year_event_code_athlete_code_medal'
   'stadium'              'pk_stadium_code'
```

_db_index_key

Represents key information on an index. An index for index_of is created.

Attribute Name	Data Type	Description
index_of	_db_index	Index to which the key attribute belongs
key_attr_name	VARCHAR(255)	Name of the attribute that comprises the key
key_order	INTEGER	Order of the attribute in the key. Begins with 0.
asc_desc	INTEGER	1 if the order of attribute values is descending, and 0 otherwise.
key_prefix_length	INTEGER	Length of prefix to be used as a key
func	VARCHAR(1023)	Functional expression of function based index

The following example shows how to retrieve the names of index that belongs to the class.

```
SELECT class_of.class_name, SEQUENCE(SELECT key_attr_name
                                     FROM _db_index_key k
                                     WHERE k in i.key_attrs)
```

```
FROM _db_index i
WHERE key_count >= 2;
```

```
  class_of.class_name   sequence((select key_attr_name from
_db_index_key k where k in
i.key_attrs))
=============================================
  '_db_server'          {'link_name', 'owner'}
  '_db_synonym'         {'name', 'owner', 'is_public'}
  'db_ha_apply_info'    {'db_name', 'copied_log_path'}
  'db_serial'           {'name', 'owner'}
  '_db_partition'       {'class_of', 'pname'}
  '_db_method'          {'class_of', 'meth_name'}
  '_db_attribute'       {'class_of', 'attr_name', 'attr_type'}
  '_db_class'           {'class_name', 'owner'}
  'participant'         {'host_year', 'nation_code'}
  'game'                {'host_year', 'event_code', 'athlete_code'}
  'record'              {'host_year', 'event_code', 'athlete_code',
'medal'}
  'history'             {'event_code', 'athlete'}
```

_db_auth

Represents user authorization information for objects. An index is created on grantee.

Attribute Name	Data Type	Description
grantor	db_user	Authorization grantor
grantee	db_user	Authorization grantee
object_type	INTEGER	Type of the object for which authorization is granted (0: class, 5: stored procedure)
object_of	object	Object for which authorization is granted

Attribute Name	Data Type	Description
auth_type	VARCHAR(7)	Type name of the authorization granted
is_grantable	INTEGER	1 if authorization for the object can be granted to other users, and 0 otherwise.

Authorization types supported by CUBRID are as follows:

- **SELECT**
- **INSERT**
- **UPDATE**
- **DELETE**
- **ALTER**
- **INDEX**
- **EXECUTE**

The following example shows how to retrieve authorization information defined in the class *db_trig*.

```
SELECT a.grantor.name, a.grantee.name, a.auth_type
FROM _db_auth a JOIN _db_class c ON a.object_of = c.class_of
WHERE c.class_name = 'db_trig';
```

```
  grantor.name          grantee.name          auth_type
=============================================================
  'DBA'                 'PUBLIC'              'SELECT'
```

_db_data_type

Represents the data type supported by CUBRID (see the “Data Types Supported by CUBRID” table in _db_attribute).

Attribute Name	Data Type	Description
type_id	INTEGER	Data type identifier. Corresponds to the “Value” column in the “Data Types Supported by CUBRID” table.
type_name	VARCHAR(16)	Data type name. Corresponds to the “Meaning” column in the “Data Types Supported by CUBRID” table.

The following example shows how to retrieve attributes and type names of the *event* class.

```
SELECT a.attr_name, t.type_name
FROM _db_attribute a join _db_data_type t ON a.data_type = t.type_id
WHERE class_of.class_name = 'event'
ORDER BY a.def_order;
```

```
  attr_name             type_name
============================================
  'code'                'INTEGER'
  'sports'              'STRING'
  'name'                'STRING'
  'gender'              'CHAR'
  'players'             'INTEGER'
```

_db_partition

Represents partition information. An index for class_of and pname is created.

Attribute Name	Data Type	Description
class_of	_db_class	OID of the parent class
pname	VARCHAR(255)	Parent - **NULL**

Attribute Name	Data Type	Description
ptype	INTEGER	0 - HASH 1 - RANGE 2 - LIST
pexpr	VARCHAR(2048)	Parent only
pvalues	SEQUENCE OF	Parent - Column name, Hash size RANGE - MIN/MAX value : - Infinite MIN/MAX is stored as **NULL** LIST - value list
comment	VARCHAR(1024)	Comment to describe the partition

_db_stored_procedure

Represents Stored procedure information. An index for unique_name is created.

Attribute Name	Data Type	Description
unique_name	VARCHAR(255)	Stored procedure name prefixed with the schema name
sp_name	VARCHAR(255)	Stored procedure name
sp_type	INTEGER	Stored procedure type (function or procedure)
return_type	INTEGER	Return value type
arg_count	INTEGER	The number of arguments

Attribute Name	Data Type	Description
args	SEQUENCE OF _db_stored_procedure_args	Argument list
lang	INTEGER	Implementation language
pkg_name	VARCHAR(255)	Package name containing the stored procedure
is_system_generated	INTEGER	Indicates whether the stored procedure is system-generated
directive	INTEGER	Execution behavior attributes of the stored procedure
target_class	VARCHAR(1024)	Class name of the stored procedure to execute
target_method	VARCHAR(4096)	Method name of the stored procedure to execute
owner	db_user	Owner
comment	VARCHAR(1024)	Comment to describe the stored procedure

Note

- **sp_type**: A value of **1** indicates a stored procedure, and **2** indicates a stored function.
- **return_type**: The return type is one of the following values:

```
NULL = 0, INTEGER = 1, FLOAT = 2, DOUBLE = 3,
STRING = 4,
SET = 6, MULTISET = 7, SEQUENCE = 8, TIME = 10,
TIMESTAMP = 11, DATE = 12, SHORT = 18, NUMERIC
= 22, CHAR = 25,
CURSOR = 28, BIGINT = 31, DATETIME = 32
```

    - Java stored procedures support all the data types listed above.
    - PL/CSQL supports the data types listed above except for SET, MULTISET, SEQUENCE, and CURSOR.
    - Data types not listed above are not supported.

- **lang**: A value of **0** indicates PL/CSQL, and **1** indicates Java.
- **pkg_name**: Package name. Currently, the package name is only used in the **DBMS_OUTPUT** system package.
- **is_system_generated**: A value of **1** indicates a system-generated stored procedure, while **0** indicates a user-created stored procedure.
- **directive**: The value uses bits from **Bit 0** to **Bit 31**, starting from the rightmost bit.
    - **Bit 0: Represents the execution rights (AUTHID) of the stored procedure.**
        - A value of **0** indicates owner's rights (also referred to as definer's rights), and **1** indicates caller's rights (also referred to as invoker's rights).
    - **Bit 1: Represents the deterministic property of the stored procedure.**
        - A value of **1** indicates deterministic, and **0** indicates non-deterministic.
    - The other bits (**Bit 2** ~ **Bit 31**) are not currently used.

_db_stored_procedure_args

Represents Stored procedure argument information. An index for sp_of is created.

Attribute Name	Data Type	Description
sp_of	_db_stored_procedure	Stored procedure object
index_of	INTEGER	Order of the arguments
is_system_generated	INTEGER	Indicates whether the stored procedure is system-generated
arg_name	CHARACTER VARYING(255)	Argument name
data_type	INTEGER	Argument data type
mode	INTEGER	Arguemnt mode (IN, OUT, INOUT)
default_value	CHARACTER VARYING(255)	Default value of the argument
is_optional	INTEGER	Whether the parameter is optional
comment	CHARACTER VARYING(1024)	Comment to describe the argument

Note

- **index_of**: The order of the arguments starts from 0.
- **is_system_generated**: A value of **1** indicates a system-generated stored procedure argument, while **0** indicates a user-created stored procedure argument.
- **data_type**: The data type of the argument is one of the following values

```
NULL = 0, INTEGER = 1, FLOAT = 2, DOUBLE = 3,
STRING = 4,
SET = 6, MULTISET = 7, SEQUENCE = 8, TIME = 10,
TIMESTAMP = 11, DATE = 12, SHORT = 18, NUMERIC
= 22, CHAR = 25,
CURSOR = 28, BIGINT = 31, DATETIME = 32
```

    - Java stored procedures support all the data types listed above.
    - PL/CSQL supports the data types listed above except for SET, MULTISET, SEQUENCE, and CURSOR.
    - Data types not listed above are not supported.

- **mode**: The argument mode is one of the following values

```
IN = 0, OUT = 1, INOUT = 2
```

- **default_value**: The default value of the argument. For more details, refer to Using Default Arguments.
- **is_optional**: A value of **1** indicates the argument is optional, while **0** indicates it is required.

_db_stored_procedure_code

Represents Stored procedure code information. An index for name is created.

Attribute Name	Data Type	Description
name	CHARACTER VARYING(1024)	Name of the stored procedure code
created_time	CHARACTER VARYING(16)	Creation time
owner	db_user	Owner

Attribute Name	Data Type	Description
is_static	INTEGER	Indicates whether the stored procedure code is loaded statically
is_system_generated	INTEGER	Indicates whether the stored procedure code is system-generated
stype	INTEGER	Type of the source code
scode	CHARACTER VARYING(1073741823)	Source code
otype	INTEGER	Type of the object code
ocode	CHARACTER VARYING(1073741823)	Object code

Note

· **is_static**

◦ A value of **1** indicates that the stored procedure code is loaded only once and changes to the procedure code are not applied until the server restarts.

◦ A value of **0** indicates that the stored procedure code is loaded every time, and changes are applied immediately using DDL or procedure load utilities.

◦ Used in Java Native Interface (JNI) Support.

· **is_system_generated**: A value of **1** indicates that the code is system-generated, while **0** indicates it is user-written code.

- **stype**: Represents the type of the source code. Currently, only PL/CSQL code is supported, and the value is **0**.
- **scode**: Stores the source code of the stored procedure entered by the user.
- **otype**: Represents the type of the object code compiled from the source code of the stored procedure.
    - A value of **0** indicates a Java class file.
    - A value of **1** indicates a Java archive file (Jar).
    - Other values are not currently supported.
- **ocode**: Stores the object code compiled from the source code of the stored procedure, which can be executed on the PL execution server.

_db_server

Attribute Name	Data Type	Description
link_name	VARCHAR(255)	Connection name
host	VARCHAR(255)	Hostname of a server
port	INTEGER	Connection port of a server
db_name	VARCHAR(255)	Database name of a server
user_name	VARCHAR(255)	Database username of a server
password	VARCHAR(1073741823)	Database user password of a server
properties	VARCHAR(2048)	

Attribute Name	Data Type	Description
		Property information used for connection
owner	db_user	The owner of this connection information
comment	VARCHAR(1024)	Comment to describe the server

_db_synonym

Represents target object information of synonyms. An index for unique_name and an index for name owner, and is_public are created.

Attribute Name	Data Type	Description
unique_name	VARCHAR(255)	Name prefixed with the schema name of the synonym
name	VARCHAR(255)	The name of the synonym
owner	db_user	The owner of the synonym
is_public	INTEGER	1 for a public synonym, and 0 for a private synonym.
target_unique_name	VARCHAR(255)	Name prefixed with the schema name of the target object
target_name	VARCHAR(255)	

Attribute Name	Data Type	Description
		The name of the target object
target_owner	db_user	The owner name of the target object
comment	VARCHAR(2048)	Comment to describe the synonym

> **Warning**
>
> It does not support public synonym yet.

db_user

Attribute Name	Data Type	Description
name	VARCHAR(1073741823)	Username
id	INTEGER	User identifier
password	db_password	User password. Not displayed to the user.
direct_groups	SET OF db_user	Groups to which the user belongs directly
groups	SET OF db_user	Groups to which the user belongs directly or indirectly

Attribute Name	Data Type	Description
authorization	db_authorization	Information of the authorization owned by the user
triggers	SEQUENCE OF object	Triggers that occur due to user actions
comment	VARCHAR (1024)	Comment to describe the user

Function Names

- **set_password** ()
- **set_password_encoded** ()
- **set_password_encoded_sha1** ()
- **add_member** ()
- **drop_member** ()
- **print_authorizations** ()
- **add_user** ()
- **drop_user** ()
- **find_user** ()
- **login** ()

db_authorization

Method Name

- **check_authorization** (varchar(255), integer)

db_serial

Attribute Name	Data Type	Description
unique_name	VARCHAR(1073741823)	Serial name prefixed with schema name.
name	VARCHAR(1073741823)	Serial name.
owner	db_user	Serial owner.
current_val	NUMERIC(38,0)	Current serial value. Default is 1.
increment_val	NUMERIC(38,0)	Interval of serial values. Default is 1.
max_val	NUMERIC(38,0)	The maximum value of the serial. Default is 99999999999999999999999999999999999.
min_val	NUMERIC(38,0)	The minimum value of the cereal. Default is 1.
cyclic	INTEGER	1 (CYCLE) if a value can be generated by cycling after reaching the maximum or minimum value of the serial; 0 (NOCYCLE) if not.
started	INTEGER	1 if the value has been created at least once after creation, otherwise 0.
class_name	VARCHAR(1073741823)	

Attribute Name	Data Type	Description
		AUTO_INCREMENT In case of serial, the table name is stored. or **NULL**.
attr_name	VARCHAR(1073741823)	AUTO_INCREMENT In case of serial, the column name is stored. or **NULL**.
cached_num	INTEGER	The number of serial values to pre-create in memory to improve performance. Default is 0.
comment	VARCHAR (1024)	Comment to describe the serial.

Method Name

· **change_serial_owner** ()

db_trigger

Attribute Name	Data Type	Description
unique_name	VARCHAR(1073741823)	Trigger name prefixed with schema name
owner	db_user	Trigger owner
name	VARCHAR(1073741823)	Trigger name
status	INTEGER	

Attribute Name	Data Type	Description
		1 for INACTIVE, and 2 for ACTIVE. The default value is 2.
priority	DOUBLE	Execution priority between triggers. The default value is 0.
event	INTEGER	0 is set for UPDATE, 1 for UPDATE STATEMENT, 2 for DELETE, 3 for DELETE STATEMENT, 4 for INSERT, 5 for INSERT STATEMENT, 8 for COMMIT, and 9 for ROLLBACK.
target_class	object	Class object for the trigger target class
target_attribute	VARCHAR(1073741823)	Trigger target attribute name. If the target attribute is not specified, *NULL** is used.
target_class_attribute	INTEGER	If the target attribute is an instance attribute, 0 is used. If it is a class attribute, 1 is used. The default value is 0.
condition_type	INTEGER	If a condition exist, 1; otherwise **NULL**.
condition	VARCHAR(1073741823)	Action condition specified in the IF statement

Attribute Name	Data Type	Description
condition_time	INTEGER	1 for BEFORE, 2 for AFTER, and 3 for DEFERRED if a condition exists; **NULL**, otherwise.
action_type	INTEGER	1 for one of INSERT, UPDATE, DELETE, and CALL, 2 for REJECT, 3 for INVALIDATE_TRANSACTION, and 4 for PRINT.
action_definition	VARCHAR(1073741823)	Execution statement to be triggered
action_time	INTEGER	1 for BEFORE, 2 for AFTER, and 3 for DEFERRED.
comment	VARCHAR (1024)	Comment to describe the trigger

db_ha_apply_info

A table that stores the progress status every time the **applylogdb** utility applies replication logs. This table is updated at every point the **applylogdb** utility commits, and the accumulative count of operations are stored in the *_counter column. The meaning of each column is as follows:

Column Name	Column Type	Description
db_name	VARCHAR(255)	Name of the database stored in the log
db_creation_time	DATETIME	

Column Name	Column Type	Description
		Creation time of the source database for the log to be applied
copied_log_path	VARCHAR(4096)	Path to the log file to be applied
committed_lsa_pageid	BIGINT	The page id of commit log lsa reflected last. Although applylogdb is restarted, the logs before last_committed_lsa are not reflected again.
committed_lsa_offset	INTEGER	The offset of commit log lsa reflected last. Although applylogdb is restarted, the logs before last_committed_lsa are not reflected again.
committed_rep_pageid	BIGINT	The page id of the replication log lsa reflected last. Check whether the reflection of replication has been delayed or not.
committed_rep_offset	INTEGER	The offset of the replication log lsa reflected last. Check whether the reflection of replication has been delayed or not.
append_lsa_page_id	BIGINT	The page id of the last replication log lsa at the last reflection. Saves

Column Name	Column Type	Description
		append_lsa of the replication log header that is being processed by applylogdb at the time of reflecting the replication. Checks whether the reflection of replication has been delayed or not at the time of reflecting the replication log.
append_lsa_offset	INTEGER	The offset of the last replication log lsa at the last reflection. Saves append_lsa of the replication log header that is being processed by applylogdb at the time of reflecting the replication. Checks whether the reflection of replication has been delayed or not at the time of reflecting the replication log.
eof_lsa_page_id	BIGINT	The page id of the replication log EOF lsa at the last reflection. Saves eof_lsa of the replication log header that is being processed by applylogdb at the time of reflecting the replication. Checks whether the reflection of replication has been delayed or not at the time of reflecting the replication log.

Column Name	Column Type	Description
eof_lsa_offset	INTEGER	The offset of the replication log EOF lsa at the last reflection. Saves eof_lsa of the replication log header that is being processed by applylogdb at the time of reflecting the replication. Checks whether the reflection of replication has been delayed or not at the time of reflecting the replication log.
final_lsa_pageid	BIGINT	The pageid of replication log lsa processed last by applylogdb. Checks whether the reflection of replication has been delayed or not.
final_lsa_offset	INTEGER	The offset of replication log lsa processed last by applylogdb. Checks whether the reflection of replication has been delayed or not.
required_page_id	BIGINT	The smallest page which should not be deleted by the log_max_archives parameter. The log page number from which the replication will be reflected.
required_page_offset	INTEGER	The offset of the log page from which the replication will be reflected.

Column Name	Column Type	Description
log_record_time	DATETIME	Timestamp included in replication log committed in the slave database, i.e. the creation time of the log
log_commit_time	DATETIME	The time of reflecting the last commit log
last_access_time	DATETIME	The final update time of the db_ha_apply_info catalog
status	INTEGER	Progress status (0: IDLE, 1: BUSY)
insert_counter	BIGINT	Number of times that applylogdb was inserted
update_counter	BIGINT	Number of times that applylogdb was updated
delete_counter	BIGINT	Number of times that applylogdb was deleted
schema_counter	BIGINT	Number of times that applylogdb changed the schema
commit_counter	BIGINT	Number of times that applylogdb was committed
fail_counter	BIGINT	Number of times that applylogdb failed to be inserted/updated/deleted/

Column Name	Column Type	Description
		committed and to change the schema
start_time	DATETIME	Time when the applylogdb process accessed the slave database

dual

The dual class is a one-row, one-column table that is used as a dummy table. It is used to select a constant, expression, or pseudo column such as SYS_DATE or USER. Pseudo columns can be provided as functions in CUBRID. More details and examples are in Operators and Functions. However, it is not mandatory to have FROM clause when selecting a constant, expression, or pseudo column because dual class will be referenced automatically. Like other system catalog classes, dual class is created to be owned by dba but dba can only execute SELECT operation. Unlike other system catalog classes, however, any user can execute SELECT operation on dual class.

Attribute Name	Data Type	Description
dummy	VARCHAR(1)	Value used for dummy purpose only

The following example shows the result which ran the query that select pseudo column after inputting ";plan detail" or "SET OPTIMIZATION LEVEL 513;" in CSQL (Viewing Query Plan). This shows the dual class is referenced automatically even if there is no FROM clause.

```
SET OPTIMIZATION LEVEL 513;
SELECT SYS_DATE;
```

```
Join graph segments (f indicates final):
seg[0]: [0]
Join graph nodes:
node[0]: dual dual(1/1) (loc -1)

Query plan:
```

```
  sscan
    class: dual node[0]
    cost:  1 card 1

Query stmt:

select  SYS_DATE  from dual dual

=== <Result of SELECT Command in Line 1> ===

        SYS_DATE
      ============
        11/26/2020
```

System Catalog Virtual Class

General users can only see information of classes for which they have authorization through system catalog virtual classes. This section explains which information each system catalog virtual class represents, and virtual class definition statements.

Virtual Class Name	Description
DB_CLASS	Class information
DB_DIRECT_SUPER_CLASS	Super class information
DB_VCLASS	The SQL statement of a virtual class
DB_ATTRIBUTE	Attribute information
DB_ATTR_SETDOMAIN_ELM	Data type for elements of collection type (SET, MULTISET, SEQUENCE) attributes
DB_CHARSET	Charset information
DB_COLLATION	Collation information

Virtual Class Name	Description
DB_METHOD	Method information
DB_METH_ARG	Method argument information
DB_METH_ARG_SETDOMAIN_ELM	Data type for elements of collection type (SET, MULTISET, SEQUENCE) argument
DB_METH_FILE	File information in which the method is defined
DB_INDEX	Index information
DB_INDEX_KEY	Key information on an index
DB_AUTH	User authorization information of classes
DB_TRIG	Trigger information
DB_PARTITION	Partition information
DB_STORED_PROCEDURE	Stored procedure information
DB_STORED_PROCEDURE_ARGS	Stored procedure argument information
DB_SERVER	Server information for DBLink
DB_SYNONYM	Target object information of synonyms

DB_CLASS

Represents information of classes for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
class_name	VARCHAR(255)	Class name
owner_name	VARCHAR(255)	Owner Name of class
class_type	VARCHAR(6)	'CLASS' for a class, and 'VCLASS' for a virtual class
is_system_class	VARCHAR(3)	'YES' for a system class, and 'NO' otherwise.
tde_algorithm	VARCHAR(32)	TDE encryption algorithm
partitioned	VARCHAR(3)	'YES' for a partitioned group class, and 'NO' otherwise.
is_reuse_oid_class	VARCHAR(3)	'YES' for a REUSE_OID class, and 'NO' otherwise.
collation	VARCHAR(32)	Collation name
comment	VARCHAR(2048)	Comment to describe the class

The following example shows how to retrieve classes owned by the current user.

```
/* CURRENT_USER: PUBLIC */
SELECT class_name, owner_name
FROM db_class
WHERE owner_name = CURRENT_USER;
```

```
  class_name            owner_name
=============================================
  'stadium'             'PUBLIC'
  'code'                'PUBLIC'
  'nation'              'PUBLIC'
  'event'               'PUBLIC'
  'athlete'             'PUBLIC'
  'participant'         'PUBLIC'
  'olympic'             'PUBLIC'
  'game'                'PUBLIC'
  'record'              'PUBLIC'
  'history'             'PUBLIC'
  'female_event'        'PUBLIC'
```

The following example shows how to retrieve virtual classes that can be accessed by the current user.

```
SELECT class_name
FROM db_class
WHERE class_type = 'VCLASS';
```

```
  class_name
======================
  'db_synonym'
  'db_server'
  'db_charset'
  'db_collation'
  'db_stored_procedure_args'
  'db_stored_procedure'
  'db_partition'
  'db_trig'
  'db_auth'
  'db_index_key'
  'db_index'
  'db_meth_file'
  'db_meth_arg_setdomain_elm'
  'db_meth_arg'
  'db_method'
  'db_attr_setdomain_elm'
  'db_attribute'
  'db_vclass'
  'db_direct_super_class'
  'db_class'
```

The following example shows how to retrieve system classes that can be accessed by the current user.

```
SELECT class_name
FROM db_class
WHERE is_system_class = 'YES' AND class_type = 'CLASS'
ORDER BY class_name;
```

```
  class_name
======================
  'db_authorization'
  'db_authorizations'
  'db_ha_apply_info'
  'db_root'
  'db_serial'
  'db_user'
  'dual'
```

DB_DIRECT_SUPER_CLASS

Represents the names of super classes (if any) of the class for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
class_name	VARCHAR(255)	Class name
owner_name	VARCHAR(255)	Owner Name of class
super_class_name	VARCHAR(255)	Super class name
super_owner_name	VARCHAR(255)	Owner Name of super class

The following example shows how to retrieve super classes of the *female_event* class. (see ADD SUPERCLASS Clause)

```
SELECT super_class_name
FROM db_direct_super_class
WHERE class_name = 'female_event';
```

```
  super_class_name
======================
  'event'
```

The following example shows how to retrieve super classes of the class owned by the current user.

```
/* CURRENT_USER: PUBLIC */
SELECT class_name, super_class_name
FROM  db_direct_super_class
WHERE owner_name = CURRENT_USER
ORDER BY class_name;
```

```
  class_name            super_class_name
============================================
  'female_event'        'event'
```

DB_VCLASS

Represents SQL definition statements of virtual classes for which the current user has access authorization to a database.

The data type of attribute 'vclass_def' is VARCHAR (4096) for prior versions including 10.1 Patch 3.

Attribute Name	Data Type	Description	Classification (10.1 Only)
vclass_name	VARCHAR(255)	Virtual class name	
owner_name	VARCHAR(255)	Owner name of virtual class	
vclass_def	VARCHAR(1073741823)	SQL definition statement of the virtual class	10.1 Patch 4 or later
	VARCHAR(4096)		10.1 Patch 3 or earlier

Attribute Name	Data Type	Description	Classification (10.1 Only)
comment	VARCHAR(2048)	Comment to describe the virtual class	

The following example shows how to retrieve SQL definition statements of the *db_class* virtual class.

```
SELECT vclass_def
FROM db_vclass
WHERE vclass_name = 'db_class';
```

```
  vclass_def
======================
  'SELECT [c].[class_name], CAST([c].[owner].[name] AS
VARCHAR(255)), CASE [c].[class_type] WHEN 0 THEN 'CLASS' WHEN 1 THEN
'VCLASS' ELSE 'UNKNOW' END, CASE WHEN MOD([c].[is_system_class], 2)
= 1 THEN 'YES' ELSE 'NO' END, CASE [c].[tde_algorithm] WHEN 0 THEN
'NONE' WHEN 1 THEN 'AES' WHEN 2 THEN 'ARIA' END, CASE WHEN [c].
[sub_classes] IS NULL THEN 'NO' ELSE NVL((SELECT 'YES' FROM
[_db_partition] [p] WHERE [p].[class_of] = [c] and [p].[pname] IS
NULL), 'NO') END, CASE WHEN MOD([c].[is_system_class] / 8, 2) = 1
THEN 'YES' ELSE 'NO' END, [coll].[coll_name], [c].[comment] FROM
[_db_class] [c], [_db_collation] [coll] WHERE [c].[collation_id] =
[coll].[coll_id] AND (CURRENT_USER = 'DBA' OR {[c].[owner].[name]}
SUBSETEQ (SELECT SET{CURRENT_USER} + COALESCE(SUM(SET{[t].[g].
[name]}), SET{}) FROM [db_user] [u], TABLE([groups]) AS [t]([g])
WHERE [u].[name] = CURRENT_USER) OR {[c]} SUBSETEQ ( SELECT
SUM(SET{[au].[class_of]}) FROM [_db_auth] [au] WHERE {[au].[grantee].
[name]} SUBSETEQ ( SELECT SET{CURRENT_USER} + COALESCE(SUM(SET{[t].
[g].[name]}), SET{}) FROM [db_user] [u], TABLE([groups]) AS [t]([g])
WHERE [u].[name] = CURRENT_USER) AND [au].[auth_type] = 'SELECT'))'
```

DB_ATTRIBUTE

Represents the attribute information of a class for which the current user has access authorization in the database.

Attribute Name	Data Type	Description
attr_name	VARCHAR(255)	Attribute name
class_name	VARCHAR(255)	Name of the class to which the attribute belongs
owner_name	VARCHAR(255)	Owner name of the class to which the attribute belongs
attr_type	VARCHAR(8)	'INSTANCE' for an instance attribute, 'CLASS' for a class attribute, and 'SHARED' for a shared attribute.
def_order	INTEGER	Order of attributes in the class. Begins with 0. If the attribute is inherited, the order is the one defined in the super class.
from_class_name	VARCHAR(255)	If the attribute is inherited, the super class in which it is defined is used. Otherwise, **NULL**.
from_owner_name	VARCHAR(255)	If the attribute is inherited, the owner name of the super class in which it is defined is used. Otherwise, **NULL**.
from_attr_name	VARCHAR(255)	If the attribute is inherited and its name is changed to resolve a name conflict, the original name defined in the

Attribute Name	Data Type	Description
		super class is used. Otherwise, **NULL**.
data_type	VARCHAR(9)	Data type of the attribute (one in the "Meaning" column of the "Data Types Supported by CUBRID" table in _db_attribute)
prec	INTEGER	Precision of the data type. 0 is used if the precision is not specified.
scale	INTEGER	Scale of the data type. 0 is used if the scale is not specified.
charset	VARCHAR (32)	charset name
collation	VARCHAR (32)	collation name
domain_class_name	VARCHAR(255)	Domain class name if the data type is an object. **NULL** otherwise.
domain_owner_name	VARCHAR(255)	Owner name of the domain class if the data type is an object. **NULL** otherwise.
default_value	VARCHAR(255)	Saved as a character string by default, regardless of data types. If no default value is specified, **NULL** is stored. If a default value is **NULL**, it is displayed as

Attribute Name	Data Type	Description
		'NULL'. An object data type is represented as 'volume id | page id | slot id' while a set data type is represented as '{element 1, element 2, ... }'.
is_nullable	VARCHAR(3)	'NO' if a not null constraint is set, and 'YES' otherwise.
comment	VARCHAR(1024)	Comment to describe the attribute.

The following example shows how to retrieve attributes and data types of the *event* class.

```
SELECT attr_name, data_type, domain_class_name
FROM db_attribute
WHERE class_name = 'event'
ORDER BY def_order;
```

```
  attr_name             data_type             domain_class_name
=======================================================================
  'code'                'INTEGER'             NULL
  'sports'              'STRING'              NULL
  'name'                'STRING'              NULL
  'gender'              'CHAR'                NULL
  'players'             'INTEGER'             NULL
```

The following example shows how to retrieve attributes of the *female_event* class and its super class.

```
SELECT attr_name, from_class_name
FROM db_attribute
WHERE class_name = 'female_event'
ORDER BY def_order;
```

```
  attr_name             from_class_name
=========================================
```

```
  'code'                'event'
  'sports'              'event'
  'name'                'event'
  'gender'              'event'
  'players'             'event'
```

The following example shows how to retrieve classes whose attribute names are similar to *name*, among the ones owned by the current user. (The user is **PUBLIC**.)

```
/* CURRENT_USER: PUBLIC */
SELECT class_name, attr_name
FROM db_attribute
WHERE owner_name = CURRENT_USER AND attr_name like '%name%'
ORDER BY class_name;
```

```
  class_name            attr_name
============================================
  'athlete'             'name'
  'code'                'f_name'
  'code'                's_name'
  'event'               'name'
  'female_event'        'name'
  'nation'              'name'
  'stadium'             'name'
```

DB_ATTR_SETDOMAIN_ELM

Among attributes of the class to which the current user has access authorization in the database, if an attribute's data type is a collection (SET, MULTISET, SEQUENCE), this macro represents the data type of the element of the collection.

Attribute Name	Data Type	Description
attr_name	VARCHAR(255)	Attribute name
class_name	VARCHAR(255)	Name of the class to which the attribute belongs
owner_name	VARCHAR(255)	Owner name of the class to which the attribute belongs

Attribute Name	Data Type	Description
attr_type	VARCHAR(8)	'INSTANCE' for an instance attribute, 'CLASS' for a class attribute, and 'SHARED' for a shared attribute.
data_type	VARCHAR(9)	Data type of the element
prec	INTEGER	Precision of the data type of the element
scale	INTEGER	Scale of the data type of the element
code_set	INTEGER	Character set if the data type of the element is a character
domain_class_name	VARCHAR(255)	Domain class name if the data type of the element is an object
domain_owner_name	VARCHAR(255)	Owner name of the domain class if the data type of the element is an object

If the set_attr attribute of class D is of a SET (A, B, C) type, the following three records exist.

Attr_name	Class_name	Attr_type	Data_type	Prec	Scale	Code_set	Domain_class_name
'set_attr'	'D'	'INSTANCE'	'SET'	0	0	0	'A'

Attr_name	Class_name	Attr_type	Data_type	Prec	Scale	Code_set	Domain_class_name
‘set_attr’	‘D’	‘INSTANCE’	‘SET’	0	0	0	‘B’
‘set_attr’	‘D’	‘INSTANCE’	‘SET’	0	0	0	‘C’

The following example shows how to retrieve collection type attributes and data types of the *city* class (the *city* table defined in Containment Operators is created).

```
SELECT attr_name, attr_type, data_type, domain_class_name
FROM db_attr_setdomain_elm
WHERE class_name = 'city';
```

```
  attr_name             attr_type             data_type
domain_class_name
=============================================================================
==========
  'sports'              'INSTANCE'            'STRING'
NULL
```

DB_CHARSET

Represents charset information.

Attribute name	Data type	Description
charset_id	INTEGER	Charset ID
charset_name	CHARACTER VARYING(32)	Charset name
default_collation	CHARACTER VARYING(32)	Default collation name

Attribute name	Data type	Description
char_size	INTEGER	One character’s byte size

DB_COLLATION

The information on collation.

Attribute Name	Data Type	Description
coll_id	INTEGER	Collation ID
coll_name	VARCHAR(32)	Collation name
charset_name	VARCHAR(32)	Charset name
is_builtin	VARCHAR(3)	Built-in or not while installing the product(Yes, No)
has_expansions	VARCHAR(3)	Having expansion or not(Yes, No)
contractions	INTEGER	Whether to include abbreviation
uca_strength	VARCHAR(255)	Weight strength (Not applicable, Primary, Secondary, Tertiary, Quaternary, Identity, Unknown)

DB_METHOD

Represents method information of a class for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
meth_name	VARCHAR(255)	Method name
class_name	VARCHAR(255)	Name of the class to which the method belongs
owner_name	VARCHAR(255)	Owner name of the class to which the method belongs
meth_type	VARCHAR(8)	'INSTANCE' for an instance method, and 'CLASS' for a class method.
from_class_name	VARCHAR(255)	If the method is inherited, the super class in which it is defined is used otherwise **NULL**
from_owner_name	VARCHAR(255)	If the method is inherited, the owner name of the superclass in which the method is defined is used otherwise **NULL**
from_meth_name	VARCHAR(255)	If the method is inherited and its name is changed to resolve a name conflict, the original name defined in the super class is used otherwise **NULL**
func_name	VARCHAR(255)	

Attribute Name	Data Type	Description
		Name of the C function for the method

The following example shows how to retrieve methods of the *db_user* class.

```
SELECT meth_name, meth_type, func_name
FROM db_method
WHERE class_name = 'db_user'
ORDER BY meth_type, meth_name;
```

```
  meth_name                    meth_type   func_name
=================================================================================
  'add_user'                   'CLASS'     'au_add_user_method'
  'drop_user'                  'CLASS'     'au_drop_user_method'
  'find_user'                  'CLASS'     'au_find_user_method'
  'login'                      'CLASS'     'au_login_method'
  'add_member'                 'INSTANCE'  'au_add_member_method'
  'drop_member'                'INSTANCE'  'au_drop_member_method'
  'print_authorizations'       'INSTANCE'  'au_describe_user_method'
  'set_password'               'INSTANCE'  'au_set_password_method'
  'set_password_encoded'       'INSTANCE'
'au_set_password_encoded_method'
  'set_password_encoded_sha1'  'INSTANCE'
'au_set_password_encoded_sha1_method'
```

DB_METH_ARG

Represents the input/output argument information of the method of the class for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
meth_name	VARCHAR(255)	Method name
class_name	VARCHAR(255)	Name of the class to which the method belongs

Attribute Name	Data Type	Description
owner_name	VARCHAR(255)	Owner name of the class to which the method belongs
meth_type	VARCHAR(8)	'INSTANCE' for an instance method, and 'CLASS' for a class method.
index_of	INTEGER	Order in which arguments are listed in the function definition. Begins with 0 if it is a return value, and 1 if it is an input argument.
data_type	VARCHAR(9)	Data type of the argument
prec	INTEGER	Precision of the argument
scale	INTEGER	Scale of the argument
code_set	INTEGER	Character set if the data type of the argument is a character.
domain_class_name	VARCHAR(255)	Domain class name if the data type of the argument is an object.
domain_owner_name	VARCHAR(255)	Owner name of the domain class if the data type of the argument is an object.

The following example shows how to retrieve input arguments of the method of the *db_user* class.

```
SELECT meth_name, data_type, prec
FROM db_meth_arg
WHERE class_name = 'db_user';
```

```
  meth_name             data_type                    prec
============================================================
  'append_data'         'STRING'                1073741823
```

DB_METH_ARG_SETDOMAIN_ELM

If the data type of the input/output argument of the method of the class is a set, for which the current user has access authorization in the database, this macro represents the data type of the element of the set.

Attribute Name	Data Type	Description
meth_name	VARCHAR(255)	Method name
class_name	VARCHAR(255)	Name of the class to which the method belongs
owner_name	VARCHAR(255)	Owner name of the class to which the method belongs
meth_type	VARCHAR(8)	‘INSTANCE’ for an instance method, and ‘CLASS’ for a class method.
index_of	INTEGER	Order of arguments listed in the function definition. Begins with 0 if it is a return value, and 1 if it is an input argument.
data_type	VARCHAR(9)	Data type of the element

Attribute Name	Data Type	Description
prec	INTEGER	Precision of the element
scale	INTEGER	Scale of the element
code_set	INTEGER	Character set if the data type of the element is a character
domain_class_name	VARCHAR(255)	Domain class name if the data type of the element is an object
domain_owner_name	VARCHAR(255)	Owner name of the domain class if the data type of the element is an object

DB_METH_FILE

Represents information of a file in which the method of the class for which the current user has access authorization in the database is defined.

Attribute Name	Data Type	Description
class_name	VARCHAR(255)	Name of the class to which the method file belongs
owner_name	VARCHAR(255)	Owner name of the class to which the method file belongs
path_name	VARCHAR(255)	File path in which the C function is defined

Attribute Name	Data Type	Description
from_class_name	VARCHAR(255)	Name of the super class in which the method file is defined if the method is inherited, and otherwise **NULL**
from_owner_name	VARCHAR(255)	Owner Name of the super class in which the method file is defined if the method is inherited, and otherwise **NULL**

DB_INDEX

Represents information of indexes created for the class for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
index_name	VARCHAR(255)	Index name
is_unique	VARCHAR(3)	'YES' for a unique index, and 'NO' otherwise.
is_reverse	VARCHAR(3)	'YES' for a reversed index, and 'NO' otherwise.
class_name	VARCHAR(255)	Name of the class to which the index belongs
owner_name	VARCHAR(255)	Owner name of the class to which the index belongs

Attribute Name	Data Type	Description
key_count	INTEGER	The number of attributes that comprise the key
is_primary_key	VARCHAR(3)	'YES' for a primary key, and 'NO' otherwise.
is_foreign_key	VARCHAR(3)	'YES' for a foreign key, and 'NO' otherwise.
filter_expression	VARCHAR(1073741823)	Conditions of filtered indexes
have_function	VARCHAR(3)	'YES' for function based and 'NO' otherwise.
comment	VARCHAR(1024)	Comment to describe the index

The following example shows how to retrieve index information of the class.

```
SELECT class_name, index_name, is_unique
FROM db_index
ORDER BY owner_name, class_name;
```

```
  class_name
index_name                                                      is_unique
================================================================================
  'db_ha_apply_info'
'u_db_ha_apply_info_db_name_copied_log_path'                    'YES'
  'db_serial'
'pk_db_serial_unique_name'                                      'YES'
  'db_serial'
'u_db_serial_name_owner'                                        'YES'
  'db_user'
'u_db_user_name'                                                'YES'
  'athlete'
```

```
'pk_athlete_code'                                    'YES'
  'event'
'pk_event_code'                                      'YES'
  'female_event'
'pk_event_code'                                      'YES'
  'game'
'pk_game_host_year_event_code_athlete_code'          'YES'
  'game'
'fk_game_event_code'                                 'NO'
  'game'
'fk_game_athlete_code'                               'NO'
  'history'
'pk_history_event_code_athlete'                      'YES'
  'nation'
'pk_nation_code'                                     'YES'
  'olympic'
'pk_olympic_host_year'                               'YES'
  'participant'
'pk_participant_host_year_nation_code'               'YES'
  'participant'
'fk_participant_host_year'                           'NO'
  'participant'
'fk_participant_nation_code'                         'NO'
  'record'
'pk_record_host_year_event_code_athlete_code_medal'  'YES'
  'stadium'
'pk_stadium_code'                                    'YES'
```

DB_INDEX_KEY

Represents the key information of indexes created for the class for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
index_name	VARCHAR(255)	Index name
class_name	VARCHAR(255)	Name of the class to which the index belongs
owner_name	VARCHAR(255)	Owner name of the class to which the index belongs

Attribute Name	Data Type	Description
key_attr_name	VARCHAR(255)	Name of attributes that comprise the key
key_order	INTEGER	Order of attributes in the key. Begins with 0.
asc_desc	VARCHAR(4)	'DESC' if the order of attribute values is descending, and 'ASC' otherwise.
key_prefix_length	INTEGER	The length of prefix to be used as a key
func	VARCHAR(1023)	Functional expression of function based index

The following example shows how to retrieve index key information of the class.

```
SELECT class_name, key_attr_name, index_name
FROM db_index_key
ORDER BY owner_name, class_name, key_order
LIMIT 20;
```

```
  class_name            key_attr_name         index_name
=======================================================================
  'db_ha_apply_info'    'db_name'
'u_db_ha_apply_info_db_name_copied_log_path'
  'db_ha_apply_info'    'copied_log_path'
'u_db_ha_apply_info_db_name_copied_log_path'
  'db_serial'           'name'
'u_db_serial_name_owner'
  'db_serial'           'unique_name'
'pk_db_serial_unique_name'
  'db_serial'           'owner'
'u_db_serial_name_owner'
  'db_user'             'name'                'u_db_user_name'
  'athlete'             'code'                'pk_athlete_code'
```

```
  'code'                's_name'              'pk_code'
  'event'               'code'                'pk_event_code'
  'female_event'        'code'                'pk_event_code'
  'game'                'host_year'
'pk_game_host_year_event_code_athlete_code'
  'game'                'athlete_code'        'fk_game_athlete_code'
  'game'                'event_code'          'fk_game_event_code'
  'game'                'event_code'
'pk_game_host_year_event_code_athlete_code'
  'game'                'athlete_code'
'pk_game_host_year_event_code_athlete_code'
  'history'             'event_code'
'pk_history_event_code_athlete'
  'history'             'athlete'
'pk_history_event_code_athlete'
  'nation'              'code'                'pk_nation_code'
  'olympic'             'host_year'           'pk_olympic_host_year'
  'participant'         'host_year'
'pk_participant_host_year_nation_code'
```

DB_AUTH

Represents authorization information for objects the current user is authorized to access.

Attribute Name	Data Type	Description
grantor_name	VARCHAR(255)	Name of the user who grants authorization
grantee_name	VARCHAR(255)	Name of the user who is granted authorization
object_type	VARCHAR(16)	Type of the object for which authorization is granted (CLASS, VCLASS, PROCEDURE, FUNCTION)
object_name	VARCHAR(255)	Name of the object for which authorization is granted

Attribute Name	Data Type	Description
owner_name	VARCHAR(255)	Owner name of the object for which authorization is granted
auth_type	VARCHAR(7)	Name of the authorization type granted
is_grantable	VARCHAR(3)	'YES' if authorization for the object can be granted to other users, and 'NO' otherwise.

The following example show how to retrieve authorization information for objects whose names begin with *db_a*.

```
SELECT object_name, auth_type, grantor_name
FROM db_auth
WHERE object_name like 'db_a%'
ORDER BY 1;
```

```
  object_name              auth_type             grantor_name
=============================================================
  'db_attr_setdomain_elm'  'SELECT'              'DBA'
  'db_attribute'           'SELECT'              'DBA'
  'db_auth'                'SELECT'              'DBA'
  'db_authorization'       'SELECT'              'DBA'
  'db_authorization'       'EXECUTE'             'DBA'
  'db_authorizations'      'SELECT'              'DBA'
  'db_authorizations'      'EXECUTE'             'DBA'
```

DB_TRIG

Represents information of a trigger that has the class for which the current user has access authorization to a database, or its attribute as the target.

Attribute Name	Data Type	Description
trigger_name	VARCHAR(255)	Trigger name
owner_name	VARCHAR(255)	Owner name of trigger
target_class_name	VARCHAR(255)	Target class name
target_owner_name	VARCHAR(255)	Target class owner name
target_attr_name	VARCHAR(255)	Target attribute. If not specified in the trigger, **NULL**
target_attr_type	VARCHAR(8)	Target attribute type. If specified, 'INSTANCE' is used for an instance attribute, and 'CLASS' is used for a class attribute.
action_type	INTEGER	1 for one of INSERT, UPDATE, DELETE, and CALL, 2 for REJECT, 3 for INVALIDATE_TRANSACTION, and 4 for PRINT.
action_time	INTEGER	1 for BEFORE, 2 for AFTER, and 3 for DEFERRED.
comment	VARCHAR(1024)	Comment to describe the trigger.

DB_PARTITION

Represents information of partitioned classes for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
class_name	VARCHAR(255)	Class name
owner_name	VARCHAR(255)	Owner name
partition_name	VARCHAR(255)	Partition name
partition_class_name	VARCHAR(255)	Partitioned class name
partition_type	VARCHAR(32)	Partition type (HASH, RANGE, LIST)
partition_expr	VARCHAR(255)	Partition expression
partition_values	SEQUENCE OF	MIN and MAX values if the partition type is RANGE, **NULL** if MIN and MAX are infinite If the partition type is LIST, List of values
comment	VARCHAR(1024)	Comment to describe the partition

The following example shows how to retrieve the partition information currently configured for the participant2 class.

```
SELECT *
FROM db_partition
WHERE class_name = 'participant2';
```

```
  class_name      owner_name  partition_name
partition_class_name           partition_type  partition_expr
partition_values  comment
========================================================================
========================================================================
```

```
  'participant2'  'PUBLIC'    'before_2000'
'participant2__p__before_2000'  'RANGE'         '[host_year]'
{NULL, 2000}      NULL
  'participant2'  'PUBLIC'    'before_2008'
'participant2__p__before_2008'  'RANGE'         '[host_year]'
{2000, 2008}      NULL
```

DB_STORED_PROCEDURE

Represents information of Stored procedure for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
sp_name	VARCHAR(255)	Stored procedure name
pkg_name	VARCHAR(255)	Package name containing the stored procedure
sp_type	VARCHAR(16)	Stored procedure type (function or procedure)
return_type	VARCHAR(16)	Return value type name
arg_count	INTEGER	The number of arguments
lang	VARCHAR(16)	Implementation language name
authid	VARCHAR(16)	Execution privileges of the stored procedure
is_deterministic	VARCHAR(3)	Indicates whether the function is deterministic
target	VARCHAR(4096)	

Attribute Name	Data Type	Description
		Name of the target stored procedure code to execute
owner	VARCHAR(256)	Owner
code	VARCHAR(1073741823)	Source code of the stored procedure
comment	VARCHAR(1024)	Comment to describe the stored procedure

Note

- **sp_type**
  - **PROCEDURE** or **FUNCTION**.
- **pkg_name: Package name. Currently, the package name is only used in the DBMS_OUTPUT system package.**
  - **dbms_output** or **NULL**.
- **return_type**: The return type can be one of the following values:

```
NULL, INTEGER, FLOAT, DOUBLE, STRING,
SET, MULTISET, SEQUENCE, TIME,
TIMESTAMP, DATE, SHORT, NUMERIC, CHAR,
CURSOR, BIGINT, DATETIME
```

  - Other values are not currently supported.
- **lang: The implementation language of the stored procedure, which can be one of the following values:**
  - **PLCSQL**: PL/CSQL
  - **JAVA**: Java stored procedure
- **authid: Indicates the execution rights (AUTHID) of the stored procedure.**
  - **DEFINER**: Owner rights (or definer rights)

    - **CURRENT_USER**: Invoker rights

- **is_deterministic: Indicates whether the function is deterministic.**
    - **YES**: Deterministic function
    - **NO**: Non-deterministic function

The following example shows how to retrieve Java stored procedures owned by the current user.

```
CREATE OR REPLACE FUNCTION hello RETURN VARCHAR AS BEGIN RETURN
'Hello'; END;

CREATE OR REPLACE FUNCTION sp_int(p_int INTEGER) RETURN INTEGER AS
BEGIN RETURN p_int; END;

-- csql
;line on

/* CURRENT_USER: PUBLIC */
SELECT * from db_stored_procedure
WHERE sp_type = 'FUNCTION' AND owner = CURRENT_USER;

    ::

           <00001> sp_name          : 'hello'
                   pkg_name         : NULL
                   sp_type          : 'FUNCTION'
                   return_type      : 'STRING'
                   arg_count        : 0
                   lang             : 'PLCSQL'
                   authid           : 'DEFINER'
                   is_deterministic: 'NO'
                   target           : 'Func_HELLO_9.HELLO() return
java.lang.String'
                   owner            : 'DBA'
                   code             : 'CREATE OR REPLACE FUNCTION
hello RETURN VARCHAR AS BEGIN RETURN 'Hello'; END'
                   comment          : NULL
           <00002> sp_name          : 'sp_int'
                   pkg_name         : NULL
                   sp_type          : 'FUNCTION'
                   return_type      : 'INTEGER'
                   arg_count        : 1
                   lang             : 'PLCSQL'
                   authid           : 'DEFINER'
                   is_deterministic: 'NO'
                   target           :
'Func_SP_INT_10.SP_INT(java.lang.Integer) return java.lang.Integer'
```

```
                    owner           : 'DBA'
                    code            : 'CREATE OR REPLACE FUNCTION
sp_int(p_int INTEGER) RETURN INTEGER AS BEGIN RETURN p_int; END'
                    comment         : NULL
```

DB_STORED_PROCEDURE_ARGS

Represents argument information of Stored procedure for which the current user has access authorization to a database.

Attribute Name	Data Type	Description
sp_name	VARCHAR(255)	Stored procedure name
owner_name	VARCHAR(255)	Owner
pkg_name	VARCHAR(255)	Package name containing the stored procedure
index_of	INTEGER	Order of the arguments
arg_name	VARCHAR(255)	Argument name
data_type	VARCHAR(16)	Data type of the argument
mode	VARCHAR(6)	Mode (IN, OUT, INOUT)
is_optional	VARCHAR(3)	Whether the parameter is optional
default_value	VARCHAR(255)	Default value of the argument

Attribute Name	Data Type	Description
comment	VARCHAR(1024)	Comment to describe the argument

Note

- **index_of**: The order of the arguments starts from 0.
- **data_type**: The data type of the argument is one of the following values

```
NULL, INTEGER, FLOAT, DOUBLE, STRING,
SET, MULTISET, SEQUENCE, TIME,
TIMESTAMP, DATE, SHORT, NUMERIC, CHAR,
CURSOR, BIGINT, DATETIME
```

  - Other values are not currently supported.

- **mode**: The argument mode is one of the following values

```
IN, OUT, INOUT
```

- **is_optional**: A value of **YES** indicates the argument is optional, while **NO** indicates it is required.
- **default_value**: The default value of the argument. For more details, refer to Using Default Arguments.

The following example shows how to retrieve arguments the 'process_order' Stored procedure in the order of the arguments.

```
CREATE OR REPLACE PROCEDURE process_order(
  p_order_id IN NUMBER,
  p_status IN VARCHAR2 DEFAULT 'NEW'
)
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Processing Order ID: ' || p_order_id);
    DBMS_OUTPUT.PUT_LINE('Order Status: ' || p_status);
END;
```

```
SELECT index_of, arg_name, data_type, mode, is_optional,
default_value
FROM db_stored_procedure_args
WHERE sp_name = 'process_order'
ORDER BY index_of;

    ::

           index_of  arg_name             data_type
mode       is_optional  default_value

================================================================================
==============================
                   0  'p_order_id'          'NUMBER'
'IN'       'NO'          NULL
                   1  'p_status'            'VARCHAR2'
'IN'       'YES'         'NEW'
```

DB_SERVER

Attribute Name	Data Type	Description
link_name	VARCHAR(255)	Connection name
host	VARCHAR(255)	Hostname of a server
port	INTEGER	Connection port of a server
db_name	VARCHAR(255)	Database name of a server
user_name	VARCHAR(255)	Database username of a server
properties	VARCHAR(2048)	Property information used for connection

Attribute Name	Data Type	Description
owner	VARCHAR(256)	The name of the owner of this connection information
comment	VARCHAR(1024)	Comment to describe the server

DB_SYNONYM

Represents target object information for the synonym to which the current user has access authorization in the database.

Attribute Name	Data Type	Description
synonym_name	VARCHAR(255)	The name of the synonym
synonym_owner_name	VARCHAR(255)	The owner of the synonym
is_public_synonym	VARCHAR(3)	“YES” for a public synonym, and “NO” for a private synonym
target_name	VARCHAR(255)	The name of the target object
target_owner_name	VARCHAR(255)	The owner name of the target object
comment	VARCHAR(2048)	Comment to describe the synonym

⚠ Warning

It does not support public synonym yet.

Catalog Class/Virtual Class Authorization

Catalog classes are created to be owned by **dba**. However, **dba** can only execute **SELECT** operations. If **dba** executes operations such as **UPDATE** / **DELETE**, an authorization failure error occurs. General users cannot execute queries on system catalog classes.

Although catalog virtual classes are created to be owned by **dba**, all users can perform the **SELECT** statement on catalog virtual classes. Of course, **UPDATE** / **DELETE** operations on catalog virtual classes are not allowed.

Updating catalog classes/virtual classes is automatically performed by the system when users execute a DDL statement that creates/modifies/deletes a class/attribute/index/user/ authorization.

Querying on Catalog

To query on catalog classes, you must convert identifiers such as class, virtual class, attribute, trigger, method and index names to lowercases, and create them. Therefore, you must use lowercases when querying on catalog classes. But, DB username is changed as uppercases and stored into db_user system catalog table.

```
CREATE TABLE Foo(name varchar(255));
SELECT class_name, partitioned FROM db_class WHERE class_name =
'Foo';
```

```
There are no results.
```

```
SELECT class_name, partitioned FROM db_class WHERE class_name =
'foo';
```

```
  class_name   partitioned
============================
  'foo'        'NO'
```

```
CREATE USER tester PASSWORD 'testpwd';
SELECT name, password FROM db_user;
```

```
  name                  password
============================================
  'DBA'                 NULL
  'PUBLIC'              NULL
  'TESTER'              db_password
```

CUBRID Procedural Language

This chapter introduces the procedural language extension features provided by CUBRID to develop stored procedures and functions.

CUBRID supports stored procedures and stored functions as procedural language extension features. Stored procedures and functions help handle complex business logic that is difficult to implement with SQL alone and simplify data manipulation tasks.

They offer the following major advantages:

- Productivity and Usability: Once created, stored procedures and stored functions can be reused repeatedly. Users can call them from SQL and also invoke them easily from applications using JDBC.
- Performance: Since stored procedures and functions run on the database server, they reduce network traffic and enhance the performance of the database server.
- Security: Stored procedures can be granted execution privileges to specific users, enabling fine-grained control over data access and modification.
- Interoperability and Portability: Stored procedures and functions are designed to work across various languages and execution environments, maximizing the usability of the database.

CUBRID supports the following two procedural languages for stored procedures/functions:

    - PL/CSQL
    - Java

The main contents described in this chapter are as follows:

- Create a Stored Procedure: Describes the creation of stored procedures and stored functions.
- Call a Stored Procedure: Describes how to call stored procedures and stored functions.

- Stored Procedure Authorization: Describes how to grant call permissions to other users and the difference between Owner's Rights and Caller's Rights.
- Transaction Commit and Rollback Support: Describes the use of transaction commit and rollback within stored procedures.
- Performance Tuning: Describes methods to achieve optimized performance when using stored procedures.
- System Package: Describes the system packages supported by CUBRID.
- PL/CSQL: Describes the overview and syntax of PL/CSQL, an extension of CUBRID's SQL procedural language.
- Java Stored Procedure: Describes how to develop stored procedures using the Java language.
- Method: Describes the methods, which are built-in functions of CUBRID.

Create a Stored Procedure

Stored procedures and stored functions can be created using CREATE PROCEDUREand CREATE FUNCTIONrespectively. In CUBRID, there is no significant functional difference between stored procedures and stored functions. The only difference is that stored functions define a return type, whereas stored procedures have a return type of void and always return NULL.

This chapter provides a detailed explanation of the considerations when creating stored procedures and stored functions.

Schema and Procedure Naming

```
CREATE [OR REPLACE] [PROCEDURE] [schema_name.]procedure_name

CREATE [OR REPLACE] [FUNCTION] [schema_name.]function_name
```

The names of stored procedures and stored functions are identifiers and must be created according to the principles outlined in Identifier. The name is limited to a maximum length of 222 bytes, and exceeding this length will result in an error.

When specifying the name of a stored procedure or stored function, the schema name (**schema_name**) does not need to be specified; it will be created and stored in the schema of the currently logged-in user. Users with DBA privileges can create stored procedures and stored functions in other users' schemas, but regular users can only create them in their own schemas.

If a regular user attempts to create stored procedures and stored functions in another user's schema, an error will occur.

```
-- Login as DBA
CREATE USER U1;
CREATE USER U2;
CALL login ('U1', '') ON CLASS db_user;
CREATE PROCEDURE u2.test_proc (arg1 INT, arg2 INT)
AS BEGIN
    DBMS_OUTPUT.put_line (arg1 + arg2);
END;
```

```
ERROR: before ' ; '
Only DBA, members of the DBA group, and the owner can perform CREATE
PROCEDURE/FUNCTION.
```

Specifying Parameters for Stored Procedures

Specifying parameters for stored procedures and stored functions allows for passing arguments when the procedure or function is called. When specifying parameters, you must specify the name and data type of the parameter, and you can specify up to **64** parameters.

Stored functions receive input parameters and return a result value, so you must specify a result data type. Stored procedures, on the other hand, are of the void type and always return NULL. The result data type for stored procedures and stored functions is a subset of the data types supported by CUBRID SQL (Data Types). For more information, refer to Supported Data Types of Arguments and Return.

```
CREATE PROCEDURE test_proc (arg1 INT, arg2 INT)
AS BEGIN
    DBMS_OUTPUT.put_line (arg1 + arg2);
END;

CREATE FUNCTION test_func (arg1 INT, arg2 INT) RETURN INT
AS BEGIN
    RETURN arg1 + arg2;
END;

SELECT test_proc(1, 2);
SELECT test_func(1, 2);
```

```
  test_proc(1, 2)
======================
  NULL
```

```
<DBMS_OUTPUT>
====
3

  test_func(1, 2)
==================
                 3
```

Note

Only the names of the data types of the parameters or return values of stored procedures and stored functions can be specified. That is, precision and scale are not specified for types such as VARCHAR or NUMERIC. Precision and scale are determined variably according to the actual data passed at runtime.

```
create or replace funtion t (i_min VARCHAR, i_max VARCHAR) RETURN
VARCHAR as
    var_min VARCHAR(5) := i_min;
    var_max VARCHAR(5) := i_max;
begin
    return var_min || ',' || var_max;
end;
```

Supported Data Types of Arguments and Return

Only a subset of the data types supported by CUBRID SQL can be specified for stored procedure/function arguments and return values. The following table shows the data types supported by language extensions:

		Support Status (O, X)	
Type	**Data Type**	**PL/CSQL**	**Java SP**
Numeric	SHORT, SMALLINT	O	O
	INTEGER, INT	O	O

		Support Status (O, X)	
Type	**Data Type**	**PL/CSQL**	**Java SP**
	BIGINT	O	O
	NUMERIC, DECIMAL	O	O
	FLOAT, REAL	O	O
	DOUBLE, DOUBLE PRECISION	O	O
Date/Time	DATE, TIME, TIMESTAMP, DATETIME	O	O
	TIMESTAMPLTZ, TIMESTAMPTZ DATETIMELTZ, DATETIMETZ	X	X
String	CHAR, VARCHAR, STRING, CHAR VARYING	O	O
Collection	SET, MULTISET, LIST, SEQUENCE	X	O
Other	BIT, BIT VARYING	X	X
	ENUM	X	X

		Support Status (O, X)	
Type	**Data Type**	**PL/CSQL**	**Java SP**
	BLOB/CLOB	X	X
	JSON	X	X
	CURSOR	X	O*

* In Java SP, only return types are supported for CURSOR, not argument types.

If an unsupported data type is used when creating a stored procedure, the following errors occur:

```
CREATE FUNCTION unsupported_json() RETURN JSON
AS BEGIN RETURN NULL; END;

CREATE PROCEDURE unsupported_args (arg TIMESTAMPLTZ)
AS BEGIN NULL; END;
```

```
ERROR: Unsupported return type 'json' of the stored procedure

ERROR: before ' )
AS BEGIN NULL; END; '
Unsupported argument type 'timestampltz' of the stored procedure
```

Specifying the Mode of Arguments

The modes of arguments are IN, OUT, and IN OUT. If no mode is specified when specifying arguments, they are assumed to be IN mode by default.

Stored procedures and stored functions can use the **CALL**statement and OUT or IN OUT arguments to return the values of arguments that have been changed in the stored procedure or stored function.

```
CREATE OR REPLACE PROCEDURE test_proc (arg1 OUT INT)
AS BEGIN
    arg1 := 5;
END;
```

```
CREATE OR REPLACE FUNCTION test_func (arg1 IN OUT INT) RETURN INT
AS BEGIN
    arg1 := arg1 + 1;
    RETURN arg1 + 5;
END;
```

```
SELECT 1 into :a;
select :a;
CALL test_proc(:a);
select :a;
```

```
-- select :a;

           :a
=============
            1

-- select :a;

           :a
=============
            5
```

If you specify the OUT or IN OUT mode for any argument in a stored procedure or stored function, it cannot be called within a query statement (**SELECT**, **UPDATE**, **DELETE**, etc.); it will return an error.

```
SELECT test_func(1);
```

```
In the command from line 1,
ERROR: Semantic: Stored procedure/function 'u1.test_func' has OUT or
IN OUT arguments
```

Using Default Arguments

You can specify default values for the arguments of stored procedures and stored functions.

- You can specify default values using the **:=** or **DEFAULT** keywords.
- When a default value is specified, you can omit the argument when calling the stored procedure or function. The omitted argument will be replaced with the default value.

- Default values can be specified as literal values and are stored as string values up to 255 bytes. An error will occur if this size is exceeded.
- The following functions are allowed as default values in addition to literal values:

Default Value	Data Type
SYS_TIMESTAMP	TIMESTAMP
UNIX_TIMESTAMP()	INTEGER
CURRENT_TIMESTAMP	TIMESTAMP
SYS_DATETIME	DATETIME
CURRENT_DATETIME	DATETIME
SYS_DATE	DATE
CURRENT_DATE	DATE
SYS_TIME	TIME
CURRENT_TIME	TIME
USER, USER()	STRING
TO_CHAR(date_time[, format])	STRING
TO_CHAR(number[, format])	STRING

Note

If a function that is not listed in the table above is used as a **DEFAULT** value, the result evaluated at the time of procedure creation is stored as a **DEFAULT** value. Therefore, the default value of the column will be the result value of the function at the time of **INSERT** execution, not the result value of the function at the time of procedure creation. Stored functions cannot be used as **DEFAULT** values, so an error will be returned.

```
CREATE FUNCTION fn_int (a int) return int as begin return a; end;

-- error
CREATE FUNCTION default_val_func_test (id INT DEFAULT fn_int (1))
return int as begin return id; end;
```

```
ERROR: before ' ); '
'fn_int(1)' function can not be used in DEFAULT clause.
```

Here is a simple example of specifying a literal value as a default:

```
CREATE FUNCTION default_args (
        a INT := 1,
        b INT DEFAULT 2
) RETURN INT
AS BEGIN RETURN a + b; END;

SELECT default_args();
SELECT default_args(3);
SELECT default_args(3, 4);
```

```
  default_args()
================
              3

  default_args(3)
=================
              5

  default_args(3, 4)
====================
              7
```

Here is an example of specifying a function as a default value:

```
CREATE FUNCTION get_age (
    birth DATE DEFAULT DATE'2000-01-01',
    today DATE DEFAULT SYS_DATE
) RETURN INT
AS
BEGIN
    RETURN YEAR(today) - YEAR(birth)
        - CASE WHEN TO_CHAR(today, 'MMDD') < TO_CHAR(birth, 'MMDD')
THEN 1 ELSE 0 END;
END;

SELECT get_age();
SELECT get_age(DATE'2000-05-10');
SELECT get_age(DATE'2000-05-10', DATE'2025-03-24');
```

```
get_age()
=============
           25

get_age(date '2000-05-10')
============================
                         24

get_age(date '2000-05-10', date '2025-03-24')
===============================================
                                           24
```

Function Overloading Not Supported

Function overloading is the process of defining multiple functions with the same name but different argument types or numbers. **CUBRID does not support function overloading**. This means that you cannot create multiple stored procedures or stored functions with the same name if they have different argument types or numbers. Therefore, when using the **CREATE OR REPLACE** statement, the new procedure will overwrite the existing procedure rather than create a new one; use it with caution to avoid accidental replacement.

```
CREATE FUNCTION test_func (arg1 INT) RETURN INT AS
BEGIN
    RETURN arg1;
END;

SELECT test_func (1);

CREATE OR REPLACE FUNCTION test_func (arg1 INT, arg2 INT) RETURN INT
AS
BEGIN
```

```
    RETURN arg1 + arg2;
END;

SELECT test_func (1); -- error
SELECT test_func (1, 2); -- 3
```

```
ERROR: Parameter count is invalid. expected: 2, actual: 1

  test_func(1, 2)
==================
                3
```

Object Dependencies

The dependencies of stored procedures and stored functions on the database objects they internally reference are only checked when registering in PL/CSQL. Therefore, if the table, view, sequence, or index objects referenced in the PL/CSQL procedure do not exist, an error will occur when registering the procedure.

```
DROP TABLE IF EXISTS test_tbl;

CREATE OR REPLACE FUNCTION test_func () RETURN INT
    k INT;
AS
BEGIN
    SELECT id INTO k FROM test_tbl LIMIT 1;
    RETURN k;
END;
```

```
ERROR: Stored procedure compile error: Syntax: before '  LIMIT 1'
Unknown class "dba.test_tbl". select id from [dba.test_tbl] limit 1
```

The following example will create the test_tbl table correctly.

```
DROP TABLE IF EXISTS test_tbl;
CREATE TABLE test_tbl (id INT);
INSERT INTO test_tbl VALUES (1);

CREATE OR REPLACE FUNCTION test_func () RETURN INT
AS
    k INT;
BEGIN
    SELECT id INTO k FROM test_tbl LIMIT 1;
    RETURN k;
```

```
END;

SELECT test_func();
```

```
  test_func()
==============
             1
```

In the case of PL/CSQL Dynamic SQL or Java SP, the stored string is executed at runtime, so it is not examined when registering the procedure, and is instead validated when the procedure is called.

```
DROP TABLE IF EXISTS test_tbl;

CREATE OR REPLACE FUNCTION test_func () RETURN INT
AS
    k INT;
BEGIN
    execute immediate 'SELECT id FROM test_tbl LIMIT 1' INTO k;
    RETURN k;
END;

SELECT test_func();
```

```
ERROR: Stored procedure execute error:
  (line 5, column 9) Syntax: before '  LIMIT 1'
Unknown class "dba.test_tbl". select id from [dba.test_tbl] limit 1
```

If an object referenced internally by a stored procedure or stored function is deleted, the system does not perform dependency checks, so even if the routine is registered correctly, an error may occur when the procedure is called. Therefore, you should be careful so that the object referenced by the stored procedure or stored function is not deleted.

```
DROP TABLE IF EXISTS test_tbl;
CREATE TABLE test_tbl (id INT);
INSERT INTO test_tbl VALUES (1);

CREATE OR REPLACE FUNCTION test_func () RETURN INT
AS
    k INT;
BEGIN
    SELECT id INTO k FROM test_tbl LIMIT 1;
    RETURN k;
END;
```

```
DROP TABLE IF EXISTS test_tbl;
SELECT test_func();
```

```
ERROR: Stored procedure execute error:
  (line 5, column 9) Syntax: Unknown class "dba.test_tbl". select
[dba.test_tbl].id from [dba.test_tbl] [dba.test_tbl] ...
```

Considerations for String Charset Encoding

All string data in CUBRID has a charset. In the CUBRID PL engine, string data within stored procedures and stored functions is handled using a single charset. No charset conversion occurs when string data is input or output. String data is processed according to the following rules:

- The charset of **string type argument values** is assumed to be the same as the database's charset.
- The charset of **string type result values** is assumed to be the same as the database's charset.
- The charset of **string literal values** is assumed to be the same as the database's charset.
- If the **result of an SQL query** is of string type, the value is internally converted from the column's charset to the database's charset.

Note

- The database's charset is specified using the **charset** option in the cubrid createdb utility and cannot be changed after creation.
- All operations that input or output strings in stored procedures/functions must use a charset that matches the database's charset to obtain correct results.

Example 1

The following example demonstrates a case where a string type argument is combined with the literal value '한글' to return a result. It is assumed that the charset of the string type argument, result, and literal value is the same as the database's charset. Therefore, to execute the following SQL statement correctly, it must be stored and executed with UTF-8 encoding.

```
CREATE OR REPLACE FUNCTION test_func (arg VARCHAR) RETURN VARCHAR
AS
BEGIN
    RETURN arg || ',' || '한글';
```

```
END;

SELECT test_func('큐브리드');
```

```
test_func('큐브리드')
=====================
  큐브리드,한글
```

Example 2

The charset used for the result of an SQL query generally follows the database's charset, but as explained in Charset and Collation of Column, each column in a table can have its own specified charset. In such cases, the following scenarios may occur:

- If the column charset and the database charset are the same: No conversion is needed.
- If the column charset and the database charset differ: CUBRID internally converts the charset to the database's charset. Be cautious as incorrect character conversion or data loss may occur during this process.

The following example demonstrates a case where a database set to UTF-8 has a table column specified with the EUC-KR charset. The string returned as a result of the query is internally converted from EUC-KR to UTF-8 before being combined with a string literal and returned.

```
-- This file should be saved in euc-kr, and '큐브리드' should be
written with euc-kr encoding.
CREATE TABLE tblcharcol (col STRING CHARSET euckr) COLLATE euckr_bin;
INSERT INTO tblcharcol VALUES ('큐브리드');
```

```
-- This file should be saved in utf-8, and '한글' should be written
with utf-8 encoding.
CREATE OR REPLACE FUNCTION test_func () RETURN VARCHAR
AS
    v VARCHAR;
BEGIN
    SELECT col INTO v FROM tblcharcol LIMIT 1;
    RETURN v || '한글';
END;

SELECT test_func();
```

```
test_func()
============
  큐브리드한글
```

Call a Stored Procedure

Registered stored procedures and stored functions can be called using the CALL statement or in SQL statements. In general, stored procedures are called using the CALL statement, and stored functions are used in SQL statements where a return value is needed.

This chapter provides a detailed explanation of the considerations when calling stored procedures and stored functions.

CALL Statement

You can call a stored procedure using the CALL statement as follows in the CSQL interpreter.

```
-- Execute in csql
CREATE PROCEDURE hello ()
AS
BEGIN
    DBMS_OUTPUT.put_line('Hello, CUBRID!');
END;

;server-output on

-- Call
CALL hello();
```

```
  Result
======================
  NULL

<DBMS_OUTPUT>
====
Hello, CUBRID!
```

Calling in SQL Statements

In the case of calling in SQL statements, stored functions are typically used to return values. The following example shows how to calculate the number of medals for a specific country using a stored function.

```
CREATE FUNCTION count_medals(nation STRING) RETURN INT
AS
```

```
cnt INT;
BEGIN
    SELECT COUNT(*) INTO cnt
    FROM game
    WHERE nation_code = nation;
    RETURN cnt;
END;

SELECT count_medals('USA');
SELECT count_medals('KOR');
```

```
  count_medals('USA')
======================
                 1118

  count_medals('KOR')
======================
                  316
```

Limitations on Nested Procedure Calls

The maximum allowed nested procedure call limit is **16**. Nested procedure call refers to calling another procedure from within a procedure. In the case of recursive calls, the limit is not applied.

The following is an example of the limitations on nested procedure calls.

```
CREATE OR REPLACE FUNCTION test_factorial_query(n BIGINT) RETURN
BIGINT
AS
k BIGINT;
BEGIN
    IF n = 0 THEN
        RETURN 1;
    ELSE
        SELECT test_factorial_query(n - 1) INTO k;
        RETURN n * k;
    END IF;
END;

SELECT test_factorial_query(15);
SELECT test_factorial_query(16);
```

```
  test_factorial_query(15)
===========================
             1307674368000
```

```
-- SELECT test_factorial_query(16);
ERROR: Stored procedure execute error:
  ...
  (line 8, column 13) Stored procedure execute error:
  (line 8, column 13) Stored procedure execute error: Too many
nested stored procedure call.
```

The following is an example of a recursive call that does not go through a query. In this case, the nested call limit is not applied. Recursively calling the following example may result in an overflow of the BIGINT value or excessive use of system resources, so be careful.

```
CREATE OR REPLACE FUNCTION test_factorial_constant(n BIGINT) RETURN
BIGINT
AS
k BIGINT;
BEGIN
    IF n = 0 THEN
        RETURN 1;
    ELSE
        RETURN n * test_factorial_constant(n - 1);
    END IF;
END;

SELECT test_factorial_constant(16);
SELECT test_factorial_constant(25);
```

```
  test_factorial_constant(16)
==============================
               20922789888000

-- SELECT test_factorial_constant(25);
ERROR: Stored procedure execute error:
  (line 8, column 20) data overflow in multiplication of BIGINT
values
```

Stored Procedure Authorization

This chapter explains the permissions required to call a stored procedure and how to grant call permissions to other users. It also details the difference between Owner's Rights and Caller's Rights, specifying under which user's permissions the call is made.

Granting Procedure Call Permission

When a stored procedure is created, it belongs to the schema of the user who created it, and only users with **owner** and **DBA** permissions can call it. If the owner of the stored procedure grants call permission to another user, the user granted permission can call the stored procedure of the other user.

The important thing to note is that if a routine that is not authorized is referenced in the implementation, it may result in a compilation or execution error. Therefore, it is necessary to check whether the stored procedure references authorized objects.

CUBRID allows you to grant call permission to a stored procedure using the GRANT statement.

```
GRANT EXECUTE ON PROCEDURE procedure_name TO user_name;
```

Conversely, you can revoke the execution privilege on a procedure by using the REVOKE statement.

```
REVOKE EXECUTE ON PROCEDURE procedure_name FROM user_name;
```

Note

Stored procedures and stored functions do not support the **EXECUTE ON PROCEDURE** permission with the **WITH GRANT OPTION** option. Therefore, the owner of the stored procedure cannot grant the permission to another user.

The Difference Between Owner's Rights and Caller's Rights

When a stored procedure or function has call permission, it can specify which user's rights to execute it. Stored procedures and functions can be executed with either **Owner's Rights** or **Caller's Rights**. The execution rights can be chosen during the creation of the stored procedure or function, and it will be executed with the specified rights.

The following explains each type of rights and their differences:

- **Owner's Rights (also known as Definer's Rights)**
    - Executed with the rights of the user who created the stored procedure.
    - If the creator grants rights to another user, the granted user can also execute the stored procedure.
    - All rights of the owner of the stored procedure take effect, allowing it to access database objects that the owner has permission to access without requiring

additional grants. If a stored procedure is created with the **DBA**'s owner's rights, it can access all database objects, so special attention should be paid to security issues such as SQL Injection when using dynamic SQL.

- Suitable for common tasks or data access needs.

In contrast, Caller's Rights are as follows:

- **Caller's Rights (also known as Invoker's Rights)**
  - Executed with the rights of the caller, not the creator of the stored procedure.
  - If a stored procedure is created with Caller's Rights and rights are granted to another user, it will be executed with the rights of the user calling the stored procedure.
  - When a stored procedure is created with Caller's Rights, it is executed at the caller's rights level, not the creator's, allowing access to database objects at the caller's rights level. Therefore, when creating a stored procedure with Caller's Rights, the caller's rights level should be considered.
  - Suitable when different rights need to be applied per caller.

Understanding the difference between Owner's Rights and Caller's Rights when creating stored procedures and utilizing them can lead to more efficient database management.

When creating a stored procedure or function, the **AUTHID** attribute can be specified to determine whether it operates with **Owner's Rights** or **Caller's Rights**. For detailed definitions, refer to STORED FUNCTION/PROCEDURE DEFINITION STATEMENTS.

The **AUTHID** attribute can be specified as follows, with **DEFINER** and **OWNER** being synonyms, as well as **CURRENT_USER** and **CALLER**.

- **Owner's Rights**: AUTHID DEFINER or AUTHID OWNER
- **Caller's Rights**: AUTHID CURRENT_USER or AUTHID CALLER

If the attribute is not specified, it defaults to **Owner's Rights**.

The following is an example of logging in as a DBA user, creating a stored function with DBA Owner's Rights that returns CURRENT_USER, USER, and calling it from user U1.

```
-- Login as DBA
CREATE USER U1;

CREATE OR REPLACE FUNCTION fn_current_user() RETURN STRING AUTHID
DEFINER AS
BEGIN
        RETURN CURRENT_USER;
END;
```

```
GRANT EXECUTE ON PROCEDURE fn_current_user TO U1;

CALL login ('U1', '') ON CLASS db_user;

SELECT dba.fn_current_user();
```

```
-- As shown, even when called by user U1, the result shows 'DBA@'
because the function executes with Owner's Rights

dba.fn_current_user()
======================
'DBA@<host>'
```

Warning

- Currently, CUBRID's PL/CSQL only supports **Owner's Rights**, and does not support **Caller's Rights**.
- To use **Caller's Rights**, stored procedures must be written using Java SP.

Note

- **Owner's Rights** are supported from CUBRID version 11.4.

Transaction Commit and Rollback Support

In stored procedures and stored functions, transactions can be completed or aborted by **COMMIT** or **ROLLBACK**. This is an important feature that maintains data integrity and provides flexibility in transaction processing. The logic within stored procedures and functions can control transactions based on conditions or exception handling, which greatly aids in managing complex data logic safely and efficiently.

Advantages of Condition and Exception Handling with Transaction Control

1. **Condition-Based Transaction Processing** Within stored procedures and functions, transactions can be committed or rolled back based on conditions. For example, if certain conditions are not met, a rollback can ensure data integrity.

2. **Safe Rollback on Exception Occurrence** If an error (exception) occurs within a stored procedure, the transaction can be rolled back, returning the operation to a safe state. This is useful for preventing data corruption in unexpected situations.

3. **Stability in Handling Complex Logic** Even while processing complex logic involving multiple operations, the success of a transaction can be clearly controlled based on the situation. This enhances the reliability of data processing.

Example: Condition and Exception Handling with Transaction Control

The following is an example of controlling transactions using conditions and exception handling in a stored procedure.

```
;server-output on

CREATE TABLE orders (
  id INT NOT NULL PRIMARY KEY,
  product VARCHAR(100) NOT NULL,
  quantity INT NOT NULL
);

CREATE TABLE inventory (
  product VARCHAR(100) NOT NULL PRIMARY KEY,
  stock INT NOT NULL
);

CREATE OR REPLACE PROCEDURE process_transaction()
AS
current_stock INT;
BEGIN
        INSERT INTO orders (id, product, quantity) VALUES (1,
'Laptop', 5);

        UPDATE inventory SET stock = stock - 5 WHERE product =
'Laptop';

        SELECT stock INTO current_stock FROM inventory WHERE product
= 'Laptop';

        IF current_stock < 0 THEN
          ROLLBACK;
```

```
            DBMS_OUTPUT.PUT_LINE('Transaction failed: Insufficient
stock.');
        ELSE
            COMMIT;
            DBMS_OUTPUT.PUT_LINE('Transaction successful.');
        END IF;
EXCEPTION
        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Transaction failed: An error
occurred.');
END;
```

```
process_transaction ();

=== <Result of CALL Command in Line 2> ===

Result
======================
NULL

<DBMS_OUTPUT>
====
Transaction successful.
```

This example covers the following scenarios:

- Rollback the transaction to maintain data integrity if there is insufficient stock.
- Safely rollback the transaction if an exception occurs during the operation.
- Commit the transaction to complete the operation if conditions are met.

Cautions

- In PL/CSQL, **COMMIT** and **ROLLBACK** statements are supported by default, allowing explicit control of transaction processing within stored procedures.
- However, in case of Java Stored Procedures, the **pl_transaction_control** setting is used to maintain backward compatibility, and COMMIT and ROLLBACK statements can only be used when the parameter is set to **yes**.
- The default value of **pl_transaction_control** is **no**, and COMMIT and ROLLBACK are ignored.

Performance Tuning

Since stored procedures and stored functions are written by users, performance tuning may be necessary. This chapter provides various guides to optimize the performance of stored procedures and stored functions.

Contents

Performance Tuning 1365

- SQL Query Optimization 1365
- Optimization of Stored Function Calls in Queries 1366
- Use Built-in Functions 1366
- Use Deterministic Functions 1367

SQL Query Optimization

The most common performance overhead in stored procedures and stored functions can occur from inefficient SQL statement execution. You should sufficiently tune the SQL statements used in internal routines to prevent the following inefficiencies:

- **Table Full Scan**: Performance can degrade if the entire table is scanned without using an index.
- **Communication Overhead**: To reduce communication overhead between the CUBRID database server and the CUBRID PL server, you should write queries to return the minimum number of records.
- **Inefficient Query Calls within Loops**: When using loops within stored procedures and stored functions, it is recommended to fetch data with a single query instead of calling multiple queries within the loop.

To tune inefficient queries used in stored procedures and stored functions, refer to the Join Methods document for query optimization.

Optimization of Stored Function Calls in Queries

Unnecessary repeated calls to stored functions executed in queries can degrade performance. Therefore, consider the following methods to optimize stored function calls:

- **Minimize Unnecessary Repeated Calls: Reducing the number of calls to stored functions is the best way to improve performance.**
    - Use indexes to reduce the number of records for which the stored function is called.
    - Group duplicate data to handle it with a single call for avoiding repeated calls for the same arguments.
    - If the function logic is deterministic, use Use Deterministic Functionsto improve performance by caching the results of correlated subqueries.
- **Minimize the Size of Function Arguments and Return Values**: Design the function to return only the necessary values, avoiding the return of unnecessarily large data.

Use Built-in Functions

Built-in functions provided by CUBRID (Operators and Functions) are optimized to work efficiently with CUBRID's query execution behavior. On the other hand, user-written stored functions are **black boxes** whose internal routines are unknown, and thus may perform worse than built-in functions. Therefore, it is recommended to use built-in functions rather than implementing stored functions that can be achieved with simple combinations of built-in functions.

Below is an example of a stored function that performs the same function as the built-in CONCAT function. Using the built-in function concat() has performance advantages over the user-defined stored function my_concat().

```
CREATE OR REPLACE FUNCTION my_concat (a STRING, b STRING) RETURN
STRING AS
BEGIN
  RETURN a || b;
END;

SELECT COUNT(*) FROM (SELECT /*+ NO_MERGE */ concat (name, event)
FROM athlete);

SELECT COUNT(*) FROM (SELECT /*+ NO_MERGE */ my_concat (name, event)
FROM athlete);
```

```
-- Using concat function
             count(*)
======================
                 6677
```

```
1 row selected. (0.019853 sec) Committed. (0.000000 sec)


-- Using my_concat function
              count(*)
======================
                  6677

1 row selected. (0.302333 sec) Committed. (0.000000 sec)
```

Use Deterministic Functions

A deterministic function is a function that always returns the same result for the same arguments during a transaction. If a stored function is deterministic, its results can be reused to improve performance.

- To make a stored function deterministic, you can specify the **DETERMINISTIC** option in the **CREATE FUNCTION** statement. For more details, refer to CREATE FUNCTION.
- When the **DETERMINISTIC** attribute is specified, the stored function can be used for correlated subquery result caching optimization and will always return the same result for the same arguments. For more details on correlated subquery cache behavior, refer to SUBQUERY CACHE (correlated).

Below is an example of a stored function created as a deterministic function with **DETERMINISTIC**. This example shows the process of optimizing performance by caching results when using correlated subqueries.

```
CREATE TABLE dummy_tbl (col1 INTEGER);
INSERT INTO dummy_tbl VALUES (1), (2), (1), (2);

CREATE OR REPLACE FUNCTION pl_csql_not_deterministic (n INTEGER)
RETURN INTEGER AS
BEGIN
  return n + 1;
END;

CREATE OR REPLACE FUNCTION pl_csql_deterministic (n INTEGER) RETURN
INTEGER DETERMINISTIC AS
BEGIN
  return n + 1;
END;

SELECT sp_name, owner, sp_type, is_deterministic from
db_stored_procedure;
```

```
   sp_name                          owner
sp_type               is_deterministic
================================================================================
==================
   'pl_csql_not_deterministic'   'DBA'
'FUNCTION'            'NO'
   'pl_csql_deterministic'       'DBA'
'FUNCTION'            'YES'
```

In the above example, the pl_csql_not_deterministic function does not use query caching in correlated subqueries because it is **NOT DETERMINISTIC**. On the other hand, the pl_csql_deterministic function uses query caching in correlated subqueries because it is specified with the **DETERMINISTIC** keyword, optimizing performance.

```
;trace on
SELECT (SELECT pl_csql_not_deterministic (t1.col1) FROM dual) AS
results FROM dummy_tbl t1;
```

```
     results
=============
           2
           3
           2
           3

=== Auto Trace ===
   ...
   Trace Statistics:
     SELECT (time: 4, fetch: 11, fetch_time: 0, ioread: 0)
       FUNC (time: 4, fetch: 2, ioread: 0, calls: 4)
       SCAN (table: dba.dummy_tbl), (heap time: 0, fetch: 5, ioread:
0, readrows: 4, rows: 4)
       SUBQUERY (correlated)
         SELECT (time: 4, fetch: 6, fetch_time: 0, ioread: 0)
           FUNC (time: 4, fetch: 2, ioread: 0, calls: 4)
           SCAN (table: dual), (heap time: 0, fetch: 4, ioread: 0,
readrows: 4, rows: 4)
```

The pl_csql_not_deterministic function does not cache the results of correlated subqueries because it is **NOT DETERMINISTIC**.

```
;trace on
SELECT (SELECT pl_csql_deterministic (t1.col1) FROM dual) AS results
FROM dummy_tbl t1;
```

```
     results
=============
           2
           3
           2
           3

=== Auto Trace ===
   ...
   Trace Statistics:
     SELECT (time: 2, fetch: 9, fetch_time: 0, ioread: 0)
       FUNC (time: 2, fetch: 2, ioread: 0, calls: 2)
       SCAN (table: dba.dummy_tbl), (heap time: 0, fetch: 5, ioread:
0, readrows: 4, rows: 4)
       SUBQUERY (correlated)
         SELECT (time: 2, fetch: 4, fetch_time: 0, ioread: 0)
           FUNC (time: 2, fetch: 2, ioread: 0, calls: 2)
           SCAN (table: dual), (heap time: 0, fetch: 2, ioread: 0,
readrows: 2, rows: 2)
           SUBQUERY_CACHE (hit: 2, miss: 2, size: 150808, status:
enabled)
```

In the trace results of the pl_csql_deterministic function, the **SUBQUERY_CACHE** item is displayed (hit: 2, miss: 2, size: 150808, status: enabled), and the number of records read (**readrows**) in the **SCAN (table: dual)** at the top is reduced compared to the **NOT DETERMINISTIC** example.

⚠ Warning

- The **DETERMINISTIC** attribute is not supported in stored procedures.
- If a function that returns non-deterministic results is used the **DETERMINISTIC** option, it may not return the expected results.

```
CREATE TABLE test_table (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100)
);

INSERT INTO test_table (name) VALUES
('Alice'),
('Bob'),
('Charlie'),
('Alice'),
('Bob');

CREATE SERIAL my_serial;
```

```
CREATE OR REPLACE FUNCTION cnt_name (name VARCHAR) RETURN VARCHAR
DETERMINISTIC AS BEGIN RETURN name || my_serial.NEXT_VALUE; END;

SELECT
id,
name,
  (SELECT cnt_name(name) FROM DUAL) AS result
FROM test_table;
```

```
        id  name                  result
============================================================
        1  'Alice'               'Alice1'
        2  'Bob'                 'Bob2'
        3  'Charlie'             'Charlie3'
        4  'Alice'               'Alice1'
        5  'Bob'                 'Bob2
```

In the above example, the my_serial.NEXT_VALUE inside the cnt_name function returns non-deterministic results, so it returns unexpected results due to the correlated subquery cache. It is recommended to specify the **DETERMINISTIC** option considering the implementation of the stored function.

System Package

CUBRID provides system packages for user convenience. More packages will be added in future versions to expand CUBRID's functionality, but currently, only the DBMS_OUTPUT package is available.

DBMS_OUTPUT

The DBMS_OUTPUT package provides functionality to store and read string messages in the DBMS_OUTPUT buffer. Developers of stored procedures/functions can use the PUT_LINE or PUT functions of this package to accumulate relevant messages in the DBMS_OUTPUT buffer. Client tools like CSQL or DBeaver use the ENABLE, DISABLE, GET_LINE, and GET_LINES functions of this package to activate/deactivate the message storage feature and retrieve messages accumulated in the buffer. Developers can effectively use these messages for monitoring program progress or debugging.

This section explains the usage and examples of the DBMS_OUTPUT package. The functions of the DBMS_OUTPUT package are as follows:

- DBMS_OUTPUT.ENABLE

- DBMS_OUTPUT.DISABLE
- DBMS_OUTPUT.PUT
- DBMS_OUTPUT.PUT_LINE
- DBMS_OUTPUT.NEW_LINE
- DBMS_OUTPUT.GET_LINE
- DBMS_OUTPUT.GET_LINES

DBMS_OUTPUT.ENABLE

DBMS_OUTPUT.ENABLE(*size*)

Enable the DBMS_OUTPUT message buffer and sets the size of the buffer to store messages.

Parameters:

size – Specifies the size of the buffer in bytes. The maximum size is 32767 bytes, and exceeding this value will result in an error.

Note

Calling **;server-output on** in the CSQL interpreter is internally equivalent to calling the default DBMS_OUTPUT.ENABLE(20000). For more details, refer to CSQL session command server-output.

DBMS_OUTPUT.DISABLE

DBMS_OUTPUT.DISABLE()

Clears the messages stored in the current buffer and deactivates it. Consequently, no output will appear even if other procedures in the DBMS_OUTPUT package are called.

Note

Calling **;server-output off** in the CSQL interpreter is internally equivalent to calling DBMS_OUTPUT.DISABLE(). For more details, refer to CSQL session command server-output.

DBMS_OUTPUT.PUT

DBMS_OUTPUT.PUT(*str VARCHAR*)

Stores the specified string in the buffer without a newline.

Parameters:

str – Specifies the string to store. If the string to store is NULL, no action is taken.

DBMS_OUTPUT.PUT_LINE

DBMS_OUTPUT.PUT_LINE(*line VARCHAR*)

Stores the specified string in the buffer and adds a newline.

Parameters:

line – Specifies the string to store. If the string to store is NULL, no action is taken.

DBMS_OUTPUT.NEW_LINE

DBMS_OUTPUT.NEW_LINE()

Adds a newline character to the buffer. After adding a string with the PUT function, call the NEW_LINE function to read it line by line with GET_LINE.

DBMS_OUTPUT.GET_LINE

DBMS_OUTPUT.GET_LINE(*line OUT VARCHAR*, *status OUT INTEGER*)

Reads the first line of the string message stored in the buffer. The read line is then deleted from the buffer.

Parameters:

- **line** – Stores the string read from the buffer.
- **status** – Stores 0 if the string is successfully read, otherwise stores 1.

DBMS_OUTPUT.GET_LINES

DBMS_OUTPUT.GET_LINES(*lines OUT VARCHAR*, *num_lines IN OUT INTEGER*)

Reads the specified number of lines of string messages stored in the buffer. The read lines are deleted from the buffer.

Parameters:

- **lines** – Stores the string read from the buffer.
- **numlines** – Specifies the number of lines to read.

Usage Example

The following is a simple example of using the DBMS_OUTPUT package with the CSQL interpreter. The PUT_LINE function is used by the stored function developer, and the ENABLE, DISABLE, GET_LINE functions are used internally by the CSQL interpreter for functionality implementation.

```
;server-output on    -- CSQL internally calls DBMS_OUTPUT.ENABLE

CREATE OR REPLACE FUNCTION test() RETURN VARCHAR
AS
BEGIN
        DBMS_OUTPUT.PUT_LINE('Hello, World!');
        DBMS_OUTPUT.PUT_LINE('Hello, CUBRID!');
        DBMS_OUTPUT.PUT_LINE('Hello, DBMS_OUTPUT!');
        RETURN 'Success';
END;

SELECT test();

;server-output off   -- CSQL internally calls DBMS_OUTPUT.DISABLE
```

```
test ()
=======
```

```
'Success'

<DBMS_OUTPUT>       <-- CSQL internally calls DBMS_OUTPUT.GET_LINE
multiple times to retrieve messages for output
====
Hello, World!
Hello, CUBRID!
Hello, DBMS_OUTPUT!
```

PL/CSQL

CUBRID supports the development of stored functions and procedures using PL/CSQL (Procedural Language extension of CUBRID SQL). PL/CSQL is an extension of CUBRID SQL to a procedural language, supporting procedural language features such as conditional statements, loops, variables, error handling, and modularization through internal procedures/functions, which are difficult to implement with declarative SQL alone. Of course, SQL statements can be included and used together within a PL/CSQL program.

Overview

Creating Stored Procedures/Functions

PL/CSQL is used to create stored procedures and stored functions. The behavior of the stored procedure or function to be created is described by writing PL/CSQL code after the AS (or IS) keyword in the CREATE PROCEDURE or CREATE FUNCTION statement.

```
<create_procedure> ::=
    CREATE [ OR REPLACE ] PROCEDURE [schema_name.]<identifier> [ (
<seq_of_parameters> ) ]
    [ <procedure_properties> ] { IS | AS } [ LANGUAGE PLCSQL ] [
<seq_of_declare_specs> ] <body> ;
<create_function> ::=
    CREATE [ OR REPLACE ] FUNCTION [schema_name.]<identifier> [ (
<seq_of_parameters> ) ] RETURN <type_spec>
    [ <function_properties> ] { IS | AS } [ LANGUAGE PLCSQL ] [
<seq_of_declare_specs> ] <body> ;
```

In the above syntax, the *body* of a stored procedure/function contains PL/CSQL statements. The declaration section, *seq_of_declare_specs*, declares variables, constants, exceptions, etc., that will be used within the execution statements. For more details on these syntax elements, refer to CREATE PROCEDURE, CREATE FUNCTION, Declarations and Statements.

Stored procedures/functions are always executed with autocommit disabled. This applies even if the autocommit feature is enabled in the calling session.

Stored procedures/functions cannot have the same name as any CUBRID built-in functions. Declaring a procedure or function with the same name as a built-in function will result in a compilation error during the execution of the **CREATE** statement.

The following are examples of stored procedures/functions written using PL/CSQL.

```
CREATE OR REPLACE PROCEDURE insert_athlete(
    p_name VARCHAR,
    p_gender VARCHAR,
    p_nation_code VARCHAR,
    p_event VARCHAR)
AS
BEGIN
    INSERT INTO athlete (name, gender, nation_code, event)
    VALUES (p_name, p_gender, p_nation_code, p_event);

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
END;
```

```
CREATE OR REPLACE PROCEDURE delete_athlete(c INTEGER)
AS
    n_deleted INTEGER;
BEGIN
    DELETE FROM athlete
    WHERE code = c;

    n_deleted := SQL%ROWCOUNT;    -- number of deleted rows
    DBMS_OUTPUT.put_line(n_deleted || ' rows deleted');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.put_line('exception occurred');
END;
```

```
CREATE OR REPLACE FUNCTION fibonacci(n INTEGER) RETURN INTEGER
IS
    invalid_input EXCEPTION;
BEGIN
    IF n <= 0 THEN
        RAISE invalid_input;
    END IF;

    IF n = 1 OR n = 2 THEN
```

```
        RETURN 1;
    ELSE
        RETURN fibonacci(n-1) + fibonacci(n-2);
    END IF;
EXCEPTION
    WHEN invalid_input THEN
        DBMS_OUTPUT.put_line('invalid input: ' || n);
        RETURN -1;
    WHEN OTHERS THEN
        DBMS_OUTPUT.put_line('unknown exception');
        RETURN -1;
END;
```

In the above examples, the *DBMS_OUTPUT.put_line()* statement stores the given string argument in the server's DBMS_OUTPUT buffer. If the argument is not of a string type, it is converted to a string before being stored. The string messages stored in the DBMS_OUTPUT buffer can be viewed in CSQL by executing the session command *;server-output on*. For more details, refer to CSQL session command server-output.

When executing a *CREATE PROCEDURE/FUNCTION* statement, various rules related to the syntax and execution semantics of the stored procedure/function are checked. If any errors are found during this process, an error message is displayed indicating the location and cause of the error. The following is an example of a stored procedure containing errors and compiling it emits an error message.

```
CREATE OR REPLACE PROCEDURE athlete_code(p_name VARCHAR) AS
BEGIN
     -- Error: Static SQL SELECT statement must have an INTO clause
    SELECT code
    FROM athlete a
    WHERE a.name = p_name;
END;

ERROR: In line 4, column 5
Stored procedure compile error: SELECT statement must have an INTO
clause

0 command(s) successfully processed.
```

Static SQL

Static SQL refers to SQL statements that are written directly within the code rather than being written in a string. It is a type of SQL where the access method is predefined, allowing syntax and semantics checks to be performed at compile time.

Although it has the drawback of being inflexible, it has the advantage of being optimized at compile time, making it faster and more efficient than Dynamic SQL, where the access method is determined at runtime.

The following SQL statements can be used directly as PL/CSQL statements:

- SELECT (including CTE, UNION, INTERSECT, MINUS)
- INSERT, UPDATE, DELETE, MERGE, REPLACE
- COMMIT, ROLLBACK
- TRUNCATE

For detailed syntax and meanings, refer to CUBRID SQL. SQL statements not included in the above list cannot be used directly, but they can be executed using the Dynamic SQL statements described below.

The SELECT statement can be used not only as an execution statement but also when declaring a cursor or in the OPEN-FOR statement. The INTO clause of a SELECT statement can be used to store query results in program variables or OUT parameters. In this case, the number of retrieved values must match the number of variables or OUT parameters in the INTO clause, and the values must have types that can be assigned to the corresponding variables or OUT parameters.

When using a SELECT statement as an execution statement, the INTO clause must be included. However, when using a SELECT statement in a cursor declaration or an OPEN-FOR statement, the INTO clause may not be included.

```
CREATE OR REPLACE PROCEDURE test_into_clause_1
AS
    h int;
    s varchar(10);
    CURSOR c IS SELECT host_year, score INTO h, s FROM history;
-- Error: INTO clause
BEGIN
    ...
END;

CREATE OR REPLACE PROCEDURE test_into_clause_2
AS
    h int;
    s varchar(10);
    r SYS_REFCURSOR;
BEGIN
    OPEN r FOR SELECT host_year, score INTO h, s FROM history;
-- Error: INTO clause
    ...
END;
```

```
CREATE OR REPLACE PROCEDURE test_into_clause_3
AS
BEGIN
    SELECT host_year, score FROM history WHERE event_code = 20023;
-- Error: no INTO clause
    ...
END;
```

The query result of a SELECT statement without an INTO clause must be a single record. If more than one record is returned, a *TOO_MANY_ROWS* exception occurs. If no records are found, a *NO_DATA_FOUND* exception occurs.

In places where values are required, such as in the WHERE clause or VALUES clause of a Static SQL statement, variables, constants, and procedure/function parameters declared in PL/CSQL can be used. However, these must not have the *BOOLEAN* or *SYS_REFCURSOR* type, as they are not included in SQL Data Types.

For Static SQL, the DBLink feature is supported for SELECT statements, but it is not supported for DML statements (INSERT, UPDATE, DELETE, MERGE, REPLACE). To use the DBLink feature inside DML statements, you have to use Dynamic SQL described below.

```
CREATE OR REPLACE PROCEDURE test_dblink_in_dml
AS
BEGIN
    INSERT INTO athlete@remote_svr(name, gender, nation_code, event)
    VALUES ('Park Taehwan', 'M', 'KOR', 'Swimming');
END;

In line 4, column 8,

ERROR: Stored procedure compile error: Semantic: before '
       VALUES ('Park Taehwan', 'M', 'KOR', 'Swimming')'
DBLink DML is not yet supported for PL/CSQL Static SQL.
```

The following is an example of using Static SQL.

```
CREATE OR REPLACE FUNCTION get_medal_count(p_name VARCHAR, p_medal
CHAR) RETURN INTEGER
AS
    n INTEGER;
BEGIN
    -- SELECT statement as a regular execution statement
    SELECT COUNT(medal)
    INTO n
    FROM athlete a, record r
    WHERE a.code = r.athlete_code   /* Join condition */
    AND a.name = p_name AND r.medal = p_medal;    /* Filter
condition */
```

```
    RETURN n;
END;
```

```
CREATE OR REPLACE PROCEDURE athlete_history(p_name VARCHAR)
AS
BEGIN
    -- SELECT statement inside a FOR loop
    FOR r IN (SELECT host_year, score FROM history WHERE athlete =
p_name) LOOP
        DBMS_OUTPUT.put_line('host_year: ' || r.host_year || '
score: ' || r.score);
    END LOOP;
END;
```

```
CREATE OR REPLACE PROCEDURE athlete_history(p_name VARCHAR)
AS
    -- SELECT statement in cursor definition
    CURSOR my_cursor IS
    SELECT host_year, score
    FROM history
    WHERE athlete = p_name;
BEGIN
    FOR r IN my_cursor LOOP
        DBMS_OUTPUT.put_line('host_year: ' || r.host_year || '
score: ' || r.score);
    END LOOP;
END;
```

If an error occurs during the execution of Static SQL, an *SQL_ERROR* exception is raised.

Dynamic SQL

Dynamic SQL refers to SQL statements that are stored in strings and executed by constructing the corresponding SQL string at runtime and executing it using the EXECUTE IMMEDIATE statement.

Dynamic SQL is mainly required in the following two cases:

- When it is difficult or impossible to determine the SQL statement at the time of program development
- When executing statements that are not supported by Static SQL, such as DDL statements

In the example below, the new table name includes a procedure parameter, which means it cannot be determined at the time of program development and is only decided at runtime.

Additionally, the *DROP TABLE* and *CREATE TABLE* statements are DDL statements, which are not supported by Static SQL.

```
CREATE OR REPLACE PROCEDURE collect_athlete_history(p_name VARCHAR)
AS
    new_table VARCHAR := p_name || '_history';
BEGIN
    EXECUTE IMMEDIATE 'drop table if exists ' || new_table;
    EXECUTE IMMEDIATE 'create table ' || new_table || ' like
history';
    EXECUTE IMMEDIATE 'insert into ' || new_table || ' select * from
history where athlete = ?'
    USING p_name;
END;
```

Writing Rules

When writing identifiers, reserved words, comments, and literals, follow the rules of SQL writing rules within Static/Dynamic SQL.

The rules for writing Non-static/dynamic SQL statements in PL/SQL mostly follow the same rules, but there are some exceptions:

- Unlike SQL, the # symbol cannot be used in identifiers. That is, identifiers must consist only of English letters (uppercase and lowercase), Korean letters, digits, and the underscore (_).
- Even if enclosed in double quotes, square brackets, or backticks, special characters cannot be used in identifiers. Only English letters (uppercase and lowercase), Korean letters, digits, and underscores (_) are allowed.
- Bit string literals cannot be used.

Examples of Allowed Identifiers

```
a
a_b
athleteName2
"select"        -- Reserved word enclosed in double quotes
```

Examples of Disallowed Identifiers

```
1a              -- Starts with a digit
a@b             -- Contains special character
athlete-name-2  -- Contains special character
```

```
[a@b]           -- Special characters not allowed even within [ ]
select          -- Reserved word
```

PL/CSQL reserved words are listed in the table below. In Non-static/dynamic SQL statements, the words in the table below cannot be used as identifiers representing variables, constants, exceptions, internal procedures/functions, etc. However, as in SQL statements, they can be used as identifiers if enclosed in double quotes (" "), square brackets ([ ]), or backticks (` `). Inside Static/Dynamic SQL, the list below does not apply; instead, the CUBRID reserved word list applies, which is used for general SQL statements.

ADDDATE	AND	AS	AUTHID
AUTONOMOUS_TRANSACTION			
BEGIN	BETWEEN	BIGINT	BOOLEAN
BOTH	BY		
CALLER	CASE	CAST	CHAR
CHARACTER	CHR	CLOSE	COMMENT
COMMIT	CONSTANT	CONTINUE	CREATE
CURRENT_USER	CURSOR		
DATE	DATETIME	DATETIMELTZ	DATETIMETZ
DATE_ADD	DATE_SUB	DAY	DAY_HOUR
DAY_MILLISECOND	DAY_MINUTE	DAY_SECOND	DBMS_OUTPUT
DEC	DECIMAL	DECLARE	DEFAULT

DEFINER	DELETE	DETERMINISTIC	DIV
DOUBLE			
ELSE	ELSIF	END	ESCAPE
EXCEPTION	EXECUTE	EXIT	EXTRACT
FALSE	FETCH	FLOAT	FOR
FROM	FUNCTION		
HOUR	HOUR_MILLISECOND	HOUR_MINUTE	HOUR_SECOND
IF	IMMEDIATE	IN	INOUT
INSERT	INT	INTEGER	INTERNAL
INTO	IS	ISO88591	LANGUAGE
LEADING	LIKE	LIST	LOOP
MERGE	MILLISECOND	MINUTE	MINUTE_MILLISECOND
MINUTE_SECOND	MOD	MONTH	MULTISET
NOT	NULL	NUMERIC	
OF	OPEN	OR	OUT

OWNER			
PLCSQL	POSITION	PRAGMA	PRECISION
PROCEDURE			
QUARTER			
RAISE	REAL	REPLACE	RETURN
REVERSE	ROLLBACK	RAISE_APPLICATION_ERROR	
SECOND	SECOND_MILLISECOND	SEQUENCE	SELECT
SECOND_MILLISECOND	SEQUENCE	SELECT	SET
SETEQ	SETNEQ	SHORT	SMALLINT
SQL	SQLCODE	SQLERRM	STRING
SUBDATE	SUBSET	SUBSETEQ	SUPERSET
SUPERSETEQ	SYS_REFCURSOR		
THEN	TIME	TIMESTAMP	TIMESTAMPLTZ
TIMESTAMPTZ	TRAILING	TRIM	TRUE
TRUNCATE			

UPDATE	USING	UTF8

The CREATE PROCEDURE/FUNCTION statement for creating PL/CSQL stored procedures/functions goes through a separate PL/CSQL syntax check, not the general SQL syntax check, and a different set of reserved words is applied. However, up to the AS/IS keywords, the general SQL syntax check also applies, so the CUBRID reserved word list is also in effect there.

In the following example, the parameter name *add* is not a PL/CSQL reserved word, but since it is a CUBRID reserved word, it causes a syntax error.

```
CREATE OR REPLACE PROCEDURE test_cubrid_reserved_word(add INT) AS
BEGIN
   NULL;
END;

ERROR: invalid create procedure
  ... ...
```

Among the list above, *AUTONOMOUS_TRANSACTION* is a reserved word that is pre-included for functionality to be added in the future.

Data Types

In Static/Dynamic SQL, all data types provided by SQL can be used.

On the other hand, in Non-static/dynamic SQL, only *BOOLEAN*, *SYS_REFCURSOR*, and a subset of SQL-provided data types can be used.

- *BOOLEAN*: Can have values TRUE, FALSE, or NULL. Since the *BOOLEAN* type is not supported in SQL, it cannot be used as a parameter or return type in the *CREATE PROCEDURE/FUNCTION* statement. However, it *can* be used as a parameter or return type when declaring local procedures/functions.
- *SYS_REFCURSOR*: Used when declaring cursor variables. For usage of cursor variables, see the OPEN-FOR statement. Similar to *BOOLEAN*, *SYS_REFCURSOR* cannot be used as a parameter or return type in the *CREATE PROCEDURE/FUNCTION* statement, but it *can* be used in local procedures/functions.

Among the data types provided by SQL, some are supported and others are not in PL/CSQL. (As noted above, all SQL-provided data types *can* be used in Static/Dynamic SQL.)

Category	Supported Types	Unsupported Types
Numeric	SHORT, SMALLINT,	
	INTEGER, INT,	
	BIGINT,	
	NUMERIC, DECIMAL,	
	FLOAT, REAL,	
	DOUBLE, DOUBLE PRECISION,	
Date/Time	DATE, TIME, TIMESTAMP, DATETIME,	TIMESTAMPLTZ, TIMESTAMPTZ, DATETIMELTZ, DATETIMETZ
String	CHAR, VARCHAR, STRING, CHAR VARYING	
Collection		SET, MULTISET, LIST, SEQUENCE
Others		BIT, BIT VARYING,
		ENUM,
		BLOB/CLOB,
		JSON

%TYPE

Appending *%TYPE* after a table column name allows you to reference the data type of that column. Below is an example using *%TYPE*.

```
CREATE OR REPLACE FUNCTION get_athlete_name(p_code
athlete.code%TYPE) RETURN athlete.name%TYPE
AS
    name athlete.name%TYPE;
BEGIN
    SELECT a.name
    INTO name
    FROM athlete a
    WHERE a.code = p_code;

    RETURN name;
END;
```

<table>.<column>%TYPE represents the data type of the specified table column at the time the *CREATE PROCEDURE/FUNCTION* statement is executed. However, if the column's data type is later changed, it will *not* be automatically reflected in the behavior of stored procedures/functions that use *<table>.<column>%TYPE*. Therefore, when the data type of a column using *%TYPE* is changed, you must recompile all stored procedures/functions that use that *%TYPE* by executing *ALTER PROCEDURE/FUNCTION <name> COMPILE*.

In addition to table columns, *%TYPE* can also be appended to procedure/function parameter or variable names to represent their data types.

```
...
a VARCHAR(10);
a_like a%TYPE;    -- Declares variable a_like with the same type as
variable a
...
```

%ROWTYPE

Appending *%ROWTYPE* after a table name represents a record type composed of fields that have the same names and types as the columns of that table. For example, for a table *tbl* declared as follows:

```
CREATE TABLE tbl(a INT, b CHAR, c VARCHAR);
```

Declaring a variable *r* with the type *tbl%ROWTYPE*:

```
r tbl%ROWTYPE;
```

The value of *r* becomes a record with fields *a*, *b*, and *c*, and *r.a*, *r.b*, and *r.c* will be of types *INT*, *CHAR*, and *VARCHAR*, respectively.

You can also append *%ROWTYPE* to a cursor name. In this case, it represents a record type corresponding to the result of the *SELECT* statement defined in the cursor.

```
CURSOR c IS SELECT a, b FROM tbl;
p c%ROWTYPE;      -- p.a and p.b are of types INT and CHAR,
respectively
```

If no initial value is provided in the declaration of a record variable, it is initialized as an "empty record" in which all fields are *NULL*.

```
r tbl%ROWTYPE;    -- r.a, r.b, r.c are all NULL, but r itself is not
NULL
```

Assigning *NULL* to a record variable initializes each field to *NULL*, but the record variable itself does not become *NULL*. In other words, a record variable never holds a *NULL* value after being declared.

Records of the same type can be compared using = and *!=* operators. Here, "same type" means not only records derived from the same table, but also records from different tables whose corresponding field names and types match exactly. The result of a = operation between two records is *TRUE* only if the <=> operation between all corresponding fields returns *TRUE*; otherwise, it returns *FALSE*. The result of the *!=* operation is the logical opposite of the = operation. Using = or *!=* between records of different types will result in a compile-time error.

```
create table tblA(a INT, b CHAR, c VARCHAR);
create table tblB(a INT, b CHAR, c VARCHAR);         -- tblA%ROWTYPE
and tblB%ROWTYPE are of the same type
create table tblC(aa INT, bb CHAR, cc VARCHAR);      -- tblA%ROWTYPE
and tblC%ROWTYPE are not of the same type

CREATE OR REPLACE PROCEDURE test_record_equality AS
    r1 tblA%ROWTYPE;
    r2 tblB%ROWTYPE;
    r3 tblC%ROWTYPE;
BEGIN
    ...
    if (r1 = r2) then        -- OK
    ...
    if (r1 = r3) then        -- Error
```

```
    ...
END;
```

Comparison operators other than = and *!=*, such as <=>, <, >, <=, and >=, cannot be used for record comparisons.

Assignment from one record variable *s* to another record variable *t* is allowed when the following conditions are met:

- The number of fields in *s* and *t* are the same.
- For each field position *i*, let the types of the *i*-th fields in *s* and *t* be S_i and T_i, respectively. It must be possible to assign a value from S_i to T_i.

The names of corresponding fields do *not* need to match in order for assignment between record variables to be valid.

```
CREATE TABLE tblAA(a NUMERIC, b DATETIME);
CREATE TABLE tblBB(m INT, n VARCHAR);
CREATE TABLE tblCC(x INT, y TIME);

CREATE OR REPLACE PROCEDURE test_record_assign AS
    r1 tblAA%ROWTYPE;
    r2 tblBB%ROWTYPE;
    r3 tblCC%ROWTYPE;
BEGIN
    ...
    r1 := r2;    -- OK
    r1 := r3;    -- Error: Cannot assign TIME to DATETIME
(incompatible types)
END;
```

%ROWTYPE can be used as parameter types and return types for local procedures/functions. However, since record types are not supported in SQL statements, *%ROWTYPE* cannot be used as parameter or return types in stored procedures/functions.

```
CREATE OR REPLACE PROCEDURE sp(a tbl%ROWTYPE) AS     -- Error

    PROCEDURE inner(b tbl%ROWTYPE) AS                -- OK
    BEGIN
        ...
    END;
BEGIN
    ...
END;
```

Record variables can be used in the *INTO* clause of Static/Dynamic SQL *SELECT* statements and *FETCH* statements. When using a record variable in an *INTO* clause, other variables cannot be used together. Also, the column names of the query result do not need to match the field names of the record variable, but the number of columns retrieved and the number of fields in the record variable must be the same. Additionally, the column types and record field types must be compatible.

```
CURSOR c IS SELECT a, b FROM tbl;
whole tbl%ROWTYPE;
part c%ROWTYPE;

-- Static SQL
SELECT * INTO whole FROM tbl;

-- Dynamic SQL
EXECUTE IMMEDIATE 'SELECT * FROM tbl' INTO whole;
EXECUTE IMMEDIATE 'SELECT a, b FROM tbl' INTO part;

-- Fetch
FETCH c INTO part;
```

In the *VALUES* clause of a Static SQL *INSERT*/*REPLACE* statement, a record variable can be used. When a record variable is used, it cannot be combined with other variables in the same *VALUES* clause. Also, the names of the target columns and the record fields do not need to match, but the number of columns and fields must be the same, and the column types must be compatible with the record field types.

```
INSERT INTO tbl VALUES whole;
INSERT INTO tbl(a, b) VALUES part;
```

The following forms are also allowed:

```
INSERT INTO tbl SET ROW = whole;
INSERT INTO tbl(a, b) SET ROW = part;
```

In a Static SQL *UPDATE* statement, a record variable can also be used with the *SET ROW = <record>* syntax as shown below. This is allowed only for single-table updates, and the field types of the record must be compatible with the corresponding column types in the same order.

```
UPDATE tbl SET ROW = whole WHERE a % 2 = 0;
```

Precision and Scale Specification Exceptions

Among the data types supported in PL/CSQL, *NUMERIC* allows specification of precision and scale, while *CHAR* and *VARCHAR* allow specification of length. However, specifying precision and scale is *not allowed* for parameter types and return types of stored procedures/functions. This restriction also applies to local procedures/functions.

```
CREATE OR REPLACE FUNCTION sf(a NUMERIC(5, 3)) RETURN VARCHAR(10)
AS ...     -- Error
CREATE OR REPLACE FUNCTION sf(a NUMERIC) RETURN VARCHAR
AS ...               -- OK
```

Generally, *NUMERIC* without precision and scale amounts to *NUMERIC(15, 0)*. However, in the *parameter type* position, it exceptionally means that any precision and scale are allowed with precision from 1 to 38 and scale from 0 to the precision, while in the *return type* position, it is treated as *NUMERIC(p, s)*, where *p* and *s* are determined by the system configuration parameter STORED_PROCEDURE_RETURN_NUMERIC_SIZE. The default values for *p* and *s* are 38 and 15, respectively.

```
CREATE OR REPLACE FUNCTION test_any_precision_scale(a NUMERIC)
RETURN NUMERIC
AS
BEGIN
    RETURN a;
END;

SELECT test_any_precision_scale(1.23);      -- Result:
1.230000000000000
SELECT test_any_precision_scale(1.234);     -- Result:
1.234000000000000
SELECT test_any_precision_scale(1.2345);    -- Result:
1.234500000000000
```

In addition, when *CHAR* and *VARCHAR* are used as parameter or return types, they do not behave as *CHAR(1)* or *VARCHAR(1073741823)* as in other contexts. Instead, they indicate that any string length is allowed with length ≤ 2048 for *CHAR* and length ≤ 1073741823 for *VARCHAR*.

```
CREATE OR REPLACE FUNCTION test_any_length(a CHAR) RETURN CHAR
AS
BEGIN
    RETURN a;
END;

SELECT test_any_length('ab');       -- Result: 'ab'
SELECT test_any_length('abc');      -- Result: 'abc'
SELECT test_any_length('abcd');     -- Result: 'abcd'
```

Even when parameter and return types are declared using %TYPE, the precision, scale, and length of the referenced original types are ignored, and the behavior follows the rules described above.

```
CREATE TABLE tbl(p NUMERIC(3,2), q CHAR(3));

CREATE OR REPLACE FUNCTION test_ptype_precision_scale(a tbl.p%TYPE)
RETURN tbl.p%TYPE
AS
BEGIN
    RETURN a;
END;

SELECT test_ptype_precision_scale(1.23);      -- Result:
1.230000000000000
SELECT test_ptype_precision_scale(1.234);     -- Result:
1.234000000000000
SELECT test_ptype_precision_scale(1.2345);    -- Result:
1.234500000000000

CREATE OR REPLACE FUNCTION test_ptype_length(a tbl.q%TYPE) RETURN
tbl.q%TYPE
AS
BEGIN
    RETURN a;
END;

SELECT test_ptype_length('ab');       -- Result: 'ab'
SELECT test_ptype_length('abc');      -- Result: 'abc'
SELECT test_ptype_length('abcd');     -- Result: 'abcd'
```

Operators and Functions

In Static/Dynamic SQL, all operators and functions provided by SQL can be used. On the other hand, in Non-static/dynamic SQL statements, most SQL-provided operators and functions can also be used in the same way, but there are a few exceptions:

- Operators and functions that take or return unsupported types (*BIT*, *ENUM*, *BLOB*/*CLOB*, *JSON*, etc.) cannot be used.
- The modulo operator *%* is not allowed. However, the *MOD* function with the same meaning can be used instead.
- Logical operators *&&*, *||*, and *!* are not allowed. Instead, use their equivalents: *AND*, *OR*, and *NOT*, respectively.

The following example shows that string functions *locate* and *substr*, as well as the string concatenation operator *||*, can be used in Non-static/dynamic SQL statements.

```
CREATE OR REPLACE PROCEDURE family_name_to_last
AS
    delim INTEGER;
    family_name VARCHAR;
    given_name VARCHAR;
BEGIN
    FOR r IN (SELECT a.name FROM athlete a LIMIT 5,5) LOOP
        delim := locate(' ', r.name);                    -- locate
function
        family_name := substr(r.name, 1, delim - 1);     -- substr
function
        given_name := substr(r.name, delim + 1);         -- substr
function
        DBMS_OUTPUT.put_line(given_name || ' ' || family_name);
-- string concatenation operator ||
    END LOOP;
END;
```

Exception

PL/CSQL, like many other programming languages, supports error handling through exception handlers (see: Block Statements). Users can define their own exceptions in the declarative section and use them in the execution section (see: Exception Declarations). Additionally, for major error situations, the following system-defined exceptions are available:

Name	SQLCODE	Description
CASE_NOT_FOUND	0	No matching WHEN clause and no ELSE clause in a CASE statement
CURSOR_ALREADY_OPEN	1	Attempt to open a cursor that is already open
INVALID_CURSOR	2	Invalid cursor operation (e.g., trying to close an unopened one)
NO_DATA_FOUND	3	*SELECT INTO* returned zero rows

Name	SQLCODE	Description
PROGRAM_ERROR	4	Internal system error
STORAGE_ERROR	5	Memory allocation failed due to insufficient memory
SQL_ERROR	6	Static/Dynamic SQL execution failed
TOO_MANY_ROWS	7	*SELECT INTO* returned more than one row
VALUE_ERROR	8	Error caused by an invalid value
ZERO_DIVIDE	9	Attempted division by zero

The *SQLCODE* of each exception above can be used inside an OTHERS exception handler block to identify the type of exception.

- *SQLCODE* values up to 999 are reserved for system exceptions.
- User-declared exceptions have a *SQLCODE* of 1000.
- The first argument to RAISE_APPLICATION_ERROR must be greater than 1000.

The following is a simple example that handles the system exceptions *NO_DATA_FOUND* and *TOO_MANY_ROWS* that can occur when executing a Static SQL *SELECT* statement.

```
CREATE OR REPLACE FUNCTION athlete_code(p_name VARCHAR) RETURN
integer
AS
    c INTEGER;
BEGIN
    -- The SELECT INTO statement must return exactly one row
    SELECT code
    INTO c
    FROM athlete a
    WHERE a.name = p_name;
```

```
    RETURN c;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.put_line('error: no rows found for athlete name
' || p_name);
        RETURN -1;
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.put_line('error: more than one rows found for
athlete name ' || p_name);
        RETURN -1;
END;
```

If an exception is not explicitly handled using a *WHEN ... THEN ...* clause, the location in the code where the exception occurred and the corresponding error message will be displayed.

For example, if the exception handler clauses are removed from the above *athlete_code()* function:

```
CREATE OR REPLACE FUNCTION athlete_code(p_name VARCHAR) RETURN
integer
AS
    c INTEGER;
BEGIN
    -- The SELECT INTO statement must return exactly one row
    SELECT code
    INTO c
    FROM athlete a
    WHERE a.name = p_name;

    RETURN c;
END;
```

Then, if a name that does not exist in the *athlete* table is passed as an argument in CSQL, a *NO_DATA_FOUND* exception will be raised as the result.

```
select athlete_code('x');

In line 1, column 22,

ERROR: Stored procedure execute error:
  (line 6, column 5) no data found


0 command(s) successfully processed.
```

In the example above, position *(1, 22)* indicates the location within the *SELECT* statement, and *(6, 5)* indicates the location within the *CREATE* statement that declares *athlete_code()*.

System Configuration Parameters Application

The behavior of Static/Dynamic SQL statements is uniformly affected by all system configuration parameters.

In contrast, in Non-static/dynamic SQL statements, only the following four system configuration parameters are effective:

- *compat_numeric_division_scale*
- *oracle_compat_number_behavior*
- *oracle_style_empty_string*
- *timezone*

```
CREATE OR REPLACE PROCEDURE test_system_config
AS
BEGIN
    -- When compat_numeric_division_scale is 'no': 0.125000000, when
'yes': 0.1
    dbms_output.put_line(1.0 / 8.0);

    -- When oracle_compat_number_behavior is 'no': 1, when 'yes': 2
    dbms_output.put_line(3 / 2);

    -- When oracle_style_empty_string is 'no': 'false', when 'yes':
'true'
    IF '' IS NULL THEN
        dbms_output.put_line('true');
    ELSE
        dbms_output.put_line('false');
    END IF;
END;
```

For details about these parameters, see system configuration parameters.

Other than the four listed above, system settings do not apply to Non-static/dynamic SQL statements. In particular:

- Regardless of the *no_backslash_escapes* setting, the backslash character is *not* treated as an escape character.
- Regardless of the *pipes_as_concat* setting, || is *not* used as a logical OR operator.

- Regardless of the *plus_as_concat* setting, + is treated as a string concatenation operator when applied to strings.

```
CREATE OR REPLACE PROCEDURE test_system_config_2
AS
BEGIN
    -- Regardless of no_backslash_escapes, prints: 'Hello\nworld'
    dbms_output.put_line('Hello\\nworld');

    -- Regardless of pipes_as_concat, prints: 'ab'
    dbms_output.put_line('a' || 'b');

    -- Regardless of plus_as_concat, prints: '12'
    dbms_output.put_line('1' + '2');
END;
```

Declarations

Stored procedure/function declarations and block statements contain a declaration section called *seq_of_declare_specs*. Within this section, you can declare variables, constants, exceptions, cursors, and local procedures/functions as defined by the grammar below. Each declared item can be referenced within the *body* that follows the declaration section.

```
<seq_of_declare_specs> ::= <declare_spec> { <declare_spec> }...
<declare_spec> ::=
      <variable_decl>
    | <constant_decl>
    | <exception_decl>
    | <cursor_decl>
    | <inner_procedure_decl>
    | <inner_function_decl>
```

See the sections below for descriptions of each declaration type.

Local procedure/function declarations and Block statements have their own declaration and execution blocks, forming nested scopes. When an inner scope declares a new item with the same name as one declared in an outer scope, the name in the inner scope refers to the newly defined item, hiding the outer declaration. However, outside the inner scope, the name still refers to the outer item.

```
CREATE OR REPLACE PROCEDURE hidden_variable
AS
    a INT := 3;
    b VARCHAR(10);
```

```
    -- Inner procedure
    PROCEDURE inner_proc
    AS
        a INT := 5;
        b FLOAT;
    BEGIN
        -- Here, a = 5, b is of type FLOAT
        ...
    END;

BEGIN
    -- Here, a = 3, b is of type VARCHAR(10)
    ...

    -- Block statement
    DECLARE
        a INT := 7;
        b DATETIME;
    BEGIN
        -- Here, a = 7, b is of type DATETIME
        ...
    END;

    -- Back to a = 3, b is VARCHAR(10)
    ...
END;
```

This kind of “name hiding” also applies to other types of declarations, such as constants, procedure/function parameters, exceptions, cursors, and local procedures/functions.

However, if the hidden name is referenced earlier in the same declaration block, such as in an initializer expression, a compile-time error will occur. Here’s a simple example: in procedure *poo*, the parameter *a* is referenced in the initializer of variable *i* inside the inner procedure. Then, a new variable with the same name *a* is declared below. This results in a compile error message stating that the name has already been used in the same declaration block.

```
CREATE OR REPLACE PROCEDURE poo(a INT) AS

    PROCEDURE inner AS
        i INT := a;
        a NUMERIC;
    BEGIN
        ...
    END;

BEGIN
    ...
END;
```

```
ERROR: In line 5, column 9
Stored procedure compile error: name A has already been used at line
4 and column 18 in the same declaration block
```

Variable Declarations

```
<variable_decl> ::=
    <identifier> <type_spec> [ [ NOT NULL ] <initial_value_part> ] ;

<type_spec> ::=
      <builtin_type>
    | <table>.<column>%TYPE
    | <variable>%TYPE
    | <table>%ROWTYPE
    | <cursor>%ROWTYPE
<initial_value_part> ::= { := | DEFAULT } <expression>
```

- *builtin_type*: system-defined types as described in Data Types

When declaring variables, you may optionally specify a *NOT NULL* constraint and an initial value. If *NOT NULL* is specified, a non-null initial value must also be provided. If no initializer is provided, the variable will implicitly be assigned *NULL*.

```
CREATE OR REPLACE PROCEDURE test_variable
AS
    a INT NOT NULL := 3;
    b VARCHAR(1) := 's';
    c FLOAT;        -- c = NULL
BEGIN
    ...
END;
```

Constant Declarations

```
<constant_decl> ::=
    <identifier> CONSTANT <type_spec> [ NOT_NULL ] <value_part> ;

<type_spec> ::=
      <builtin_type>
    | <table>.<column>%TYPE
    | <variable>%TYPE
    | <table>%ROWTYPE
```

```
    | <cursor>%ROWTYPE
<value_part> ::= { := | DEFAULT } <expression>
```

- *builtin_type*: system-defined types as described in Data Types

Constant declarations must always include a value. If *NOT NULL* is specified, the value must not be *NULL*.

```
CREATE OR REPLACE PROCEDURE test_constant
AS
    a CONSTANT INT NOT NULL := 3;
    b CONSTANT VARCHAR := 's';
    --c CONSTANT FLOAT;        -- Error
BEGIN
    ...
END;
```

Exception Declarations

```
<exception_decl> ::=
    <identifier> EXCEPTION ;
```

You can declare exceptions with any identifier name you choose. The declared exceptions can be used in the RAISE statement and in the WHEN clause of exception handling.

```
CREATE OR REPLACE FUNCTION text_exception(n INT) RETURN INT
AS
    negative_argument EXCEPTION;
    too_big_argument EXCEPTION;
BEGIN
    IF n < 0 THEN
        RAISE negative_argument;
    ELSIF n > 100 THEN
        RAISE too_big_argument;
    ELSIF n = 0 THEN
        RETURN 0;
    END IF;
    ...
EXCEPTION
    WHEN negative_argument THEN
        DBMS_OUTPUT.put_line('error: negative argument ' || n);
        RETURN -1;
    WHEN too_big_argument THEN
        DBMS_OUTPUT.put_line('error: too big argument ' || n);
```

```
        RETURN -2;
END;
```

Cursor Declarations

```
<cursor_decl> ::=
    CURSOR <identifier> [ ( <seq_of_cursor_parameters> ) ] IS
<select_statement> ;

<seq_of_cursor_parameters> ::= <cursor_parameter> { ,
<cursor_parameter> }...
<cursor_parameter> ::= <identifier> [ IN ] <type_spec>
<type_spec> ::=
      <builtin_type>
    | <table>.<column>%TYPE
    | <variable>%TYPE
    | <table>%ROWTYPE
    | <cursor>%ROWTYPE
```

- *builtin_type*: system-defined types described in Data Types

Cursors can have parameters just like procedures/functions, but only *IN* parameters are allowed. These parameters can be referenced within the *select_statement*. When calling OPEN, the provided arguments must match the declared number and types of the cursor parameters, and the *SELECT* statement will execute accordingly.

```
CREATE OR REPLACE PROCEDURE test_cursor(p_name VARCHAR, p_year
INTEGER)
AS
    CURSOR my_cursor(a VARCHAR, y INTEGER) IS
    SELECT host_year, score
    FROM history
    WHERE athlete = a AND host_year >= y;

    target_year INT;
    target_score VARCHAR(10);
BEGIN
    OPEN my_cursor(p_name, p_year);
    LOOP
        FETCH my_cursor INTO target_year, target_score;
        EXIT WHEN my_cursor%NOTFOUND;
        DBMS_OUTPUT.put_line('host_year: ' || target_year || '
score: ' || target_score);
    END LOOP;
```

```
    CLOSE my_cursor;
END;
```

Cursors can be used explicitly as shown above via *OPEN*, *FETCH*, and *CLOSE* statements. Alternatively, they can be used with a *FOR-LOOP*, in which the *OPEN*, *FETCH*, and *CLOSE* steps are performed implicitly:

```
CREATE OR REPLACE PROCEDURE test_cursor_loop(p_name VARCHAR, p_year
INTEGER)
AS
    CURSOR my_cursor(a VARCHAR, y INTEGER) IS
    SELECT host_year, score
    FROM history
    WHERE athlete = a AND host_year >= y;
BEGIN
    FOR r IN my_cursor(p_name, p_year) LOOP
        DBMS_OUTPUT.put_line('host_year: ' || r.host_year || '
score: ' || r.score);
    END LOOP;
END;
```

Note that *SELECT* statements used in cursor declarations must not include an *INTO* clause.

Local Procedure/Function Declarations

You can define local procedures/functions that are only used inside the currently defined stored procedure/function, following the syntax below. If a task is somewhat large or repeated, modularizing it as a local procedure/function improves code readability and reusability.

```
<inner_procedure_decl> ::=
    PROCEDURE <identifier> [ ( <seq_of_parameters> ) ] { IS | AS } [
<seq_of_declare_specs> ] <body> ;
<inner_function_decl> ::=
    FUNCTION <identifier> [ ( <seq_of_parameters> ) ] RETURN
<type_spec> { IS | AS } [ <seq_of_declare_specs> ] <body> ;

<seq_of_parameters> ::= [ <parameter> { , <parameter> }... ]
<parameter> ::= <identifier> [ { IN | IN OUT | INOUT | OUT } ]
<type_spec> [ COMMENT 'param_comment_string' ]
<type_spec> ::=
      <builtin_type>
    | <table>.<column>%TYPE
    | <variable>%TYPE
    | <table>%ROWTYPE
    | <cursor>%ROWTYPE
<body> ::= BEGIN <seq_of_statements> [ EXCEPTION <seq_of_handlers> ]
END [ <label_name> ]
```

```
<seq_of_declare_specs> ::= <declare_spec> { <declare_spec> }...
<seq_of_statements> ::= <statement> ; { <statement> ; }...
<seq_of_handlers> ::= <handler> { <handler> }...
<handler> ::= WHEN <exception_name> { OR <exception_name> }...  THEN
<seq_of_statements>
<exception_name> ::= identifier | OTHERS
```

- *parameter*: parameters can be declared as IN, IN OUT, INOUT, or OUT. IN OUT and INOUT are equivalent.
- *param_comment_string*: a comment string for the parameter.
- *builtin_type*: system-defined types described in Data Types
- *body*: consists of at least one statement and optionally one or more exception handlers.
- *label_name*: must match the name of the procedure/function.
- *declare_spec*: one of variable, constant, exception, cursor, or local procedure/function declarations.
- *statement*: see the Statements section.
- *handler*: specifies the statements to execute when the designated exception is raised.
- *exception_name*: the exception *identifier* must be a System Exception or a user-declared one via Exception Declaration. *OTHERS* matches any unhandled exceptions, and cannot be combined with other exception names using *OR*.

Unlike stored procedures/functions, a local procedure/function may have the same name as a CUBRID built-in function. In this case, the built-in function will be hidden within the scope of the local declaration.

In the case of a function, the *body* must return a value of the declared return type using a *RETURN* statement. If a control path reaches the end of the function body without a *RETURN*, a compile-time error will occur. For procedures, *RETURN* statements must not specify a return value.

Procedures/functions can reference themselves, enabling recursive calls.

```
CREATE OR REPLACE FUNCTION choose(m INT, n INT) RETURN INT
AS
    invalid_argument EXCEPTION;

    -- Local function declaration
    FUNCTION factorial(n INT) RETURN INT
    AS
    BEGIN
        IF n < 0 THEN
            RAISE invalid_argument;
```

```
        ELSIF n <= 1 THEN
            RETURN 1;
        ELSE
            RETURN n * factorial(n - 1);    -- Recursive call
        END IF;
    END;
BEGIN
    IF n > m OR n < 0 THEN
        RAISE invalid_argument;
    ELSE
        RETURN factorial(m) / factorial(n) / factorial(m - n);
    END IF;
END;
```

Mutual recursive calls are also possible between inner procedures/functions declared in the same declaration block.

```
CREATE OR REPLACE PROCEDURE ping_pong(cnt INT)
AS
    PROCEDURE ping(n INT)
    AS
    BEGIN
        IF n <= 0 THEN
            DBMS_OUTPUT.put_line('-- end --');
        ELSE
            DBMS_OUTPUT.put_line('ping ->');
            pong(n - 1);     -- mutual recursive call
        END IF;
    END;

    PROCEDURE pong(n INT)
    AS
    BEGIN
        IF n <= 0 THEN
            DBMS_OUTPUT.put_line('-- end --');
        ELSE
            DBMS_OUTPUT.put_line('      <- pong');
            ping(n - 1);     -- mutual recursive call
        END IF;
    END;
BEGIN
    ping(cnt);
END;
```

When using recursion, be sure to include an appropriate termination condition to avoid infinite loops.

Statements

Currently, PL/CSQL provides the following 15 types of statements:

```
<statement> ::=
      <block>
    | <sql_statement>
    | <cursor_manipulation>
    | <raise_application_error>
    | <execute_immediate>
    | <assignment_statement>
    | <continue_statement>
    | <exit_statement>
    | <null_statement>
    | <raise_statement>
    | <return_statement>
    | <procedure_call>
    | <if_statement>
    | <loop_statement>
    | <case_statement>
```

BLOCK

The BLOCK statement creates a nested scope within the list of executable statements, allowing new variables, constants, etc. to be declared and used inside it. Like procedures/functions, a BLOCK can have an exception handling structure.

```
<block> ::=
    [ DECLARE <seq_of_declare_specs> ] <body>

<body> ::= BEGIN <seq_of_statements> [ EXCEPTION <seq_of_handlers> ]
END [ <label_name> ]
<seq_of_declare_specs> ::= <declare_spec> { <declare_spec> }...
<seq_of_statements> ::= <statement> ; { <statement> ; }...
<seq_of_handlers> ::= <handler> { <handler> }...
<handler> ::= WHEN <exception_name> { OR <exeption_name> }... THEN
<seq_of_statements>
<exception_name> ::= identifier | OTHERS
```

- *body*: must consist of one or more statements, optionally followed by exception handlers.
- *declare_spec*: variable, constant, exception, cursor, or inner procedure/function declarations (see: Declarations)
- *handler*: statements to be executed when the specified exception occurs.

- *exception_name*: the *identifier* must be either a system exception or a user-declared exception. *OTHERS* matches all exceptions not otherwise matched, and cannot be combined with other exception names using *OR*.

Items declared inside a BLOCK cannot be accessed from outside the BLOCK. If an item declared in a BLOCK has the same name as one in an outer scope, the outer item is hidden within that BLOCK.

```
CREATE OR REPLACE PROCEDURE test_block
IS
    a INT := 3;
    b INT := 3;
    c INT := 3;
BEGIN
    DECLARE
        a INT := 5;
        b INT := 5;
    BEGIN
        DECLARE
            a INT := 7;
        BEGIN
            DBMS_OUTPUT.put_line(a || b || c);  -- '753'
        END;

        DBMS_OUTPUT.put_line(a || b || c);      -- '553'
    END;

    DBMS_OUTPUT.put_line(a || b || c);          -- '333'
END;
```

If any statements inside the body are unreachable during execution, a compile-time error will occur. The following is a simple example containing an unreachable statement:

```
CREATE OR REPLACE PROCEDURE test_unreachable_statement
AS
BEGIN
    RETURN;
    DBMS_OUTPUT.put_line('Hello world');
END;

ERROR: In line 5, column 5
Stored procedure compile error: unreachable statement
```

Static SQL

As explained in the Static SQL section, the following SQL statements (SELECT, INSERT, UPDATE, DELETE, MERGE, REPLACE, COMMIT, ROLLBACK, and TRUNCATE) can be used directly as executable statements within a program.

Cursor Manipulation

Cursor manipulation statements come in the following four forms:

```
<cursor_manipulation> ::=
      <open_statement>
    | <fetch_statement>
    | <close_statement>
    | <open_for_statement>

<open_statement> ::= OPEN <cursor> [ <function_argument> ]
<fetch_statement> ::= FETCH <cursor_expression> INTO <identifier>
{ , <identifier> }...
<close_statement> ::= CLOSE <cursor_expression>

<open_for_statement> ::= OPEN <identifier> FOR <select_statement>
```

- *cursor_expression*: an expression that evaluates to a cursor or SYS_REFCURSOR variable
- *open_statement*: opens a cursor. Note that this can only be used with explicitly declared cursors, not SYS_REFCURSOR variables. For cursors declared with parameters, the actual values provided must match the number and types of parameters. If you attempt to open an already open cursor, a *CURSOR_ALREADY_OPEN* exception will be raised.
- *fetch_statement*: fetches one row from the cursor into the specified variables or OUT parameters. The number of columns in the fetched row must match the number of target variables, and their types must be compatible. Attempting to fetch from a cursor that is not open will raise an *INVALID_CURSOR* exception.
- *close_statement*: closes a cursor. Attempting to close a cursor that is not open will raise an *INVALID_CURSOR* exception.
- *open_for_statement*: the *identifier* must be a variable declared with type SYS_REFCURSOR. Internally, this statement opens a cursor for the given *select_statement* and assigns it to the variable. If the *select_statement* includes an INTO clause, a compile-time error will occur.

The following example shows how to use the OPEN, FETCH, and CLOSE statements:

```
CREATE OR REPLACE PROCEDURE test_cursor(p_name VARCHAR, p_year
INTEGER)
AS
    CURSOR my_cursor(a VARCHAR, y INTEGER) IS
    SELECT host_year, score
    FROM history
    WHERE athlete = a AND host_year >= y;
```

```
    target_year INT;
    target_score VARCHAR(10);
BEGIN

    OPEN my_cursor(p_name, p_year);
    LOOP
        FETCH my_cursor INTO target_year, target_score;
        EXIT WHEN my_cursor%NOTFOUND;
        DBMS_OUTPUT.put_line('host_year: ' || target_year || '
score: ' || target_score);
    END LOOP;
    CLOSE my_cursor;
END;
```

The following example demonstrates how to use a local procedure with a SYS_REFCURSOR OUT parameter along with the OPEN-FOR statement to bind a specific SELECT statement to a SYS_REFCURSOR variable and fetch its result:

```
CREATE OR REPLACE PROCEDURE test_ref_cursor(p_name VARCHAR)
AS
    my_refcursor SYS_REFCURSOR;

    target_year INT;
    target_score VARCHAR(10);

    PROCEDURE open_refcursor(athlete_name VARCHAR, rc OUT
SYS_REFCURSOR)
    AS
        refcursor SYS_REFCURSOR;
    BEGIN
        OPEN refcursor FOR SELECT host_year, score FROM history
WHERE athlete = athlete_name;
        rc := refcursor;
    END;
BEGIN
    open_refcursor(p_name, my_refcursor);
    LOOP
        FETCH my_refcursor INTO target_year, target_score;
        EXIT WHEN my_refcursor%NOTFOUND;
        DBMS_OUTPUT.put_line('host_year: ' || target_year || '
score: ' || target_score);
    END LOOP;
    CLOSE my_refcursor;
END;
```

RAISE_APPLICATION_ERROR

RAISE_APPLICATION_ERROR is used to raise an Exception with a user-defined code and error message. Although it appears to be a built-in procedure call, it is internally handled as a PL/CSQL executable statement. The first argument must be an *INTEGER* value greater than 1000, as values ≤1000 are reserved for the system. The second argument can be any string message.

```
CREATE OR REPLACE PROCEDURE test_raise_app_err(i INT)
AS
BEGIN
    CASE i
    WHEN 1 THEN
        RAISE_APPLICATION_ERROR(1001, 'my error 1');
    WHEN 2 THEN
        RAISE_APPLICATION_ERROR(1002, 'my error 2');
    WHEN 3 THEN
        RAISE_APPLICATION_ERROR(1003, 'my error 3');
    END CASE;
EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line('code=' || SQLCODE || ', message=''' ||
SQLERRM || '''');
END;

CALL test_raise_app_err(1);     -- DBMS_OUTPUT output: code=1001,
message='my error 1'
CALL test_raise_app_err(2);     -- DBMS_OUTPUT output: code=1002,
message='my error 2'
CALL test_raise_app_err(3);     -- DBMS_OUTPUT output: code=1003,
message='my error 3'
```

EXECUTE IMMEDIATE

As described in the Dynamic SQL section, you can dynamically build SQL statements at runtime and execute them using the *EXECUTE IMMEDIATE* statement. The *USING* clause allows you to bind values from the program to host variables in the SQL. The *INTO* clause allows the program to retrieve result values from a *SELECT* query into variables or OUT parameters. The number of result columns must match the number of variables or OUT parameters, and the types must be compatible.

If an error occurs while executing the SQL, a *SQL_ERROR* exception will be raised. When using the *INTO* clause, the *SELECT* must return exactly one row. If there are no results, *NO_DATA_FOUND* is raised; if more than one row is returned, *TOO_MANY_ROWS* is raised.

```
<execute_immediate> ::=
    EXECUTE IMMEDIATE <dynamic_sql> { [ <into_clause> ] [
<using_clause> ] | <using_clause> <into_clause> }
```

```
    <using_clause> ::= USING <using_element> { , <using_element> }...
    <using_element> ::= [ IN ] <expression>
    <into_clause> ::= INTO <identifier> { , <identifier> }...
```

- *dynamic_sql*: an expression of string type. It must evaluate to a syntactically valid SQL string. You may use *?* placeholders for values. The number of *?* placeholders must match the number of expressions in the *using_clause*.
- *using_clause*: specifies the values to replace *?* placeholders in the SQL string. Expressions of type *BOOLEAN* or *SYS_REFCURSOR* are not allowed. Record-type values declared with %ROWTYPE or cursors are also not allowed.
- *into_clause*: specifies the variables or OUT parameters to receive result values from a *SELECT* statement. If *dynamic_sql* is a *SELECT* but no *INTO* clause is present, or vice versa, then a *SQL_ERROR* exception is raised.

Here is an example usage of *EXECUTE IMMEDIATE*:

```
CREATE OR REPLACE PROCEDURE collect_athlete_history(p_name VARCHAR)
AS
    new_table VARCHAR := p_name || '_history';
BEGIN
    EXECUTE IMMEDIATE 'drop table if exists ' || new_table;
    EXECUTE IMMEDIATE 'create table ' || new_table || ' like
history';
    EXECUTE IMMEDIATE 'insert into ' || new_table || ' select * from
history where athlete = ?'
    USING p_name;
END;
```

Assignment

```
<assignment_statement> ::=
    <identifier> := <expression>
```

- *identifier*: must be a variable or OUT parameter
- *expression*: an expression that evaluates to the value to be assigned (see the expressions section)

The type of *expression* must be the same as or implicitly convertible to the type of *identifier*. Otherwise, a compile-time error occurs.

CONTINUE, EXIT

```
<continue_statement> ::=
    CONTINUE [ <label_name> ] [ WHEN <expression> ]
```

```
<exit_statement> ::=
    EXIT [ <label_name> ] [ WHEN <expression> ]
```

The *CONTINUE* and *EXIT* statements can only be used within loop statements. The *CONTINUE* statement stops the current execution flow and jumps to the beginning of the loop to start the next iteration. The *EXIT* statement terminates the current loop and jumps to the statement immediately following the loop. If *label_name* is omitted, the statement applies to the innermost enclosing loop. If *label_name* is specified, it must reference a label declared in one of the enclosing loops; otherwise, a compile-time error occurs. When multiple loops are nested, *label_name* can be used to specify the target loop. The *WHEN* clause allows the branch to occur only when the BOOLEAN *expression* evaluates to TRUE.

```
CREATE OR REPLACE PROCEDURE test_continue_exit
AS
    i INT := 0;
BEGIN
    LOOP
        DBMS_OUTPUT.put_line(i);            -- 0, 1, 2, 3, 4, 5
        i := i + 1;
        CONTINUE WHEN i < 3;
        DBMS_OUTPUT.put_line(i);            -- 3, 4, 5
        EXIT WHEN i = 5;
    END LOOP;

    DBMS_OUTPUT.put_line(i);                -- 5
END;
```

NULL

```
<null_statement> ::=
    NULL
```

NULL can be used to explicitly indicate a no-operation statement, or as a placeholder for a statement to be implemented later. It is syntactically treated as a valid placeholder where a statement is required.

```
CREATE OR REPLACE PROCEDURE test_null(medal CHAR)
AS
BEGIN
    CASE medal
        WHEN 'G' THEN
            DBMS_OUTPUT.put_line('Gold');
        WHEN 'S' THEN
            DBMS_OUTPUT.put_line('Silver');
        WHEN 'B' THEN
            DBMS_OUTPUT.put_line('Bronze');
        ELSE
            NULL;
    END CASE;
END;
```

RAISE

```
<raise_statement> ::=
    RAISE [ <identifier> ]
```

Raises an exception. The exception name *identifier* must be a system exception or a user-declared exception. Within an exception handler's *THEN* clause, the exception name may be omitted, in which case the currently handled exception is raised again.

```
CREATE OR REPLACE FUNCTION fibonacci(n INTEGER) RETURN INTEGER
IS
    invalid_input EXCEPTION;
BEGIN
    IF n <= 0 THEN
        RAISE invalid_input;
    END IF;

    IF n = 1 OR n = 2 THEN
        RETURN 1;
    ELSE
        RETURN fibonacci(n-1) + fibonacci(n-2);
    END IF;
EXCEPTION
    WHEN invalid_input THEN
        DBMS_OUTPUT.put_line('invalid input: ' || n);
        RAISE;      -- Re-raises the currently handled
`invalid_input` exception
    WHEN OTHERS THEN
        DBMS_OUTPUT.put_line('unknown exception');
```

```
        RAISE;      -- Re-raises the currently handled exception
END;
```

RETURN

```
<return_statement> ::=
    RETURN [ <expression> ]
```

Branches execution to the point following the call to the current routine. If the current routine is a function, a return value *expression* must be provided that is convertible to the function's declared return type. If the current routine is a procedure, specifying a return value causes an error.

Procedure Call

```
<procedure_call> ::=
    <identifier> [ <function_argument> ]
<function_argument> ::= ( [ <expression> { , <expression> }... ] )
```

Calls the procedure identified by *identifier*, optionally passing *function_argument* values. The number and types of arguments must match those declared in the procedure. Arguments passed to OUT parameters must be variables or other OUT parameters, as their values will be modified by the called procedure.

```
CREATE OR REPLACE PROCEDURE callee(o OUT INT)
AS
BEGIN
    ...
END;

CREATE OR REPLACE PROCEDURE proc(i INT, o OUT INT)
AS
    v INT;
    c CONSTANT INT := 0;
BEGIN
    callee(i);   -- Error: IN parameter
    callee(o);   -- OK: OUT parameter
    callee(v);   -- OK: variable
    callee(c);   -- Error: constant
END;
```

The called procedure may be a stored procedure or a local procedure. If an error occurs during the execution of a stored procedure call, a *SQL_ERROR* exception is raised.

IF

```
<if_statement> ::=
    IF <expression> THEN <seq_of_statements>
    [ <elsif_part> { <elsif_part> }... ]
    [ <else_part> ]
    END IF
<elsif_part> ::= ELSIF <expression> THEN <seq_of_statements>
<else_part> ::= ELSE <seq_of_statements>
```

Provides an If-Then-Else construct similar to those in general-purpose programming languages. The *expression* following *IF* and *ELSIF* must be of type *BOOLEAN*.

LOOP

PL/CSQL provides five forms of loop statements as shown below. The first three are similar to loops found in general-purpose programming languages. The last two are used for iterating over the results of a SELECT statement.

```
<loop_statement> ::=
      <label_declaration>? LOOP <seq_of_statements> END
LOOP                          # basic-loop
    | <label_declaration>? WHILE <expression> LOOP
<seq_of_statements> END LOOP       # while-loop
    | <label_declaration>? FOR <iterator> LOOP <seq_of_statements>
END LOOP           # for-iter-loop
    | <label_declaration>? FOR <for_cursor> LOOP <seq_of_statements>
END LOOP         # for-cursor-loop
    | <label_declaration>? FOR <for_static_sql> LOOP
<seq_of_statements> END LOOP     # for-static-sql-loop

<label_declaration> ::= '<<' <identifier> '>>'

<iterator> ::= <identifier> IN [ REVERSE ] <lower_bound> ..
<upper_bound> [ BY <step> ]

<for_cursor>      ::= <record> IN <cursor> [ <function_argument> ]
<for_static_sql>  ::= <record> IN ( <select_statement> )
```

- *label_declaration*: a label can only be declared at the beginning of a loop statement. It is used by CONTINUE or EXIT statements in the loop body to indicate which loop to branch to.

- In the *while-loop*, the condition *expression* must be of type BOOLEAN.
- In the *for-iter-loop*, *lower_bound*, *upper_bound*, and *step* must all be of types convertible to INTEGER. At runtime, if *step* is less than or equal to 0, a VALUE_ERROR exception is raised. If REVERSE is not specified, the *identifier* is initialized with *lower_bound* and the loop body is executed if the value is less than or equal to *upper_bound*. After each iteration, it increases by *step* and continues while still satisfying the condition. If REVERSE is specified, *identifier* is initialized with *upper_bound* and the loop body is executed if the value is greater than or equal to *lower_bound*. After each iteration, it decreases by *step* and continues while the condition holds. The loop variable *identifier* is treated as an INTEGER-type variable within the loop body.
- *for-cursor-loop* and *for-static-sql-loop* forms are used to iterate over the result set of a cursor or SELECT statement. The SELECT statement must not contain an INTO clause or a compile-time error occurs. For each iteration, a row from the result set is assigned to *record*, and the loop body executes. Each column of the row can be referenced as *record.column* within the body.

A basic form of LOOP typically includes an internal condition to exit the loop, as shown below:

```
CREATE OR REPLACE PROCEDURE print_nation_athletes(nation CHAR)
AS
    code INT;
    name VARCHAR(40);
    CURSOR c IS SELECT code, name from athlete where nation_code =
nation;
BEGIN
    OPEN c;
    LOOP
        FETCH c INTO code, name;
        EXIT WHEN c%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('code: ' || code || ' name: ' || name);
    END LOOP;
    CLOSE c;
END;
```

The following is a simple example of using a WHILE loop:

```
CREATE OR REPLACE FUNCTION sum_upto(n INT) RETURN INT
AS
    sum INT := 0;
    i INT := 1;
BEGIN
    WHILE i <= n LOOP
        sum := sum + i;
        i := i + 1;
    END LOOP;
```

```
    RETURN sum;
END;
```

The example below demonstrates the usage of a FOR iterator loop:

```
CREATE OR REPLACE PROCEDURE mult_tables
AS
BEGIN
    FOR i IN 2 .. 9 LOOP
        DBMS_OUTPUT.put_line('table ' || i);

        FOR j IN 1 .. 9 LOOP
            DBMS_OUTPUT.put_line(i || ' x ' || j || ' = ' || i*j);
        END LOOP;

        DBMS_OUTPUT.put_line('');
    END LOOP;
END;
```

The following shows an example of querying the same SELECT statement using two different forms of FOR loop.

```
CREATE OR REPLACE PROCEDURE athlete_history(p_name VARCHAR)
AS
    CURSOR my_cursor IS
    SELECT host_year, score
    FROM history
    WHERE athlete = p_name;
BEGIN
    -- For-Cursor Loop
    FOR r IN my_cursor LOOP
        DBMS_OUTPUT.put_line('host_year: ' || r.host_year || '
score: ' || r.score);
    END LOOP;

    -- For-Select Loop
    FOR r IN (SELECT host_year, score FROM history WHERE athlete =
p_name) LOOP
        DBMS_OUTPUT.put_line('host_year: ' || r.host_year || '
score: ' || r.score);
    END LOOP;
END;
```

CASE Statement

The CASE statement sequentially evaluates multiple conditions and executes the statements associated with the first condition that evaluates to true.

```
<case_statement> ::=
      CASE <expression> { WHEN <expression> THEN
<seq_of_statements> }... [ ELSE <seq_of_statements> ] END CASE
    | CASE { WHEN <expression> THEN <seq_of_statements> }... [ ELSE
<seq_of_statements> ] END CASE
```

There are two forms of CASE statements:

- The first form includes an expression immediately after the CASE keyword. It first evaluates this expression once, then compares it sequentially with the expressions in the WHEN clauses to find a match. If a match is found, the corresponding THEN clause statements are executed.
- The second form does not include an expression after the CASE keyword. Each WHEN clause expression must be of BOOLEAN type and is evaluated one by one until the first TRUE expression is found. The associated THEN clause statements are then executed.

Both forms may optionally include an ELSE clause, which specifies statements to execute if no WHEN clause matches. If no WHEN clause matches and there is no ELSE clause, the system exception CASE_NOT_FOUND is raised.

The following is an example of the first form of the CASE statement:

```
CREATE OR REPLACE PROCEDURE print_even_odd(i INTEGER)
AS
BEGIN
    CASE i MOD 2
        WHEN 0 THEN
            DBMS_OUTPUT.put_line('Even');
        WHEN 1 THEN
            DBMS_OUTPUT.put_line('Odd');
        ELSE
            DBMS_OUTPUT.put_line('Null');
    END CASE;
END;
```

The following is an example of the second form of the CASE statement with similar behavior:

```
CREATE OR REPLACE PROCEDURE print_even_odd(i INTEGER)
AS
BEGIN
    CASE
        WHEN i MOD 2 = 0 THEN
```

```
            DBMS_OUTPUT.put_line('Even');
        WHEN i MOD 2 = 1 THEN
            DBMS_OUTPUT.put_line('Odd');
        ELSE
            DBMS_OUTPUT.put_line('Null');
    END CASE;
END;
```

Expressions

The types of expressions in PL/CSQL are summarized by the following syntax:

```
<expression> ::=
      <literal>                                  # literal
    | <identifier>                               # identifier
    | SQL%ROWCOUNT                               # result size of
Static SQL
    | <cursor_expression> <cursor_attribute>     # cursor attribute
    | <expression> <binary_op> <expression>      # binary operation
    | <unary_op> <expression>                    # unary operation
    | ( <expression> )                           # parentheses
    | <identifier>.<identifier>                  # record field
reference
    | <identifier> <function_argument>           # function call
    | <case_expression>                          # CASE expression
    | SQLCODE                                    # exception code
    | SQLERRM                                    # exception message
    | <expression> IS [ NOT ] NULL               # IS NULL expression
    | <expression> [ NOT ] BETWEEN <expression> AND
<expression>        # BETWEEN expression
    | <expression> [ NOT ] IN ( <expression> [ ,
<expression>, ... ] )  # IN expression
    | <expression> [ NOT ] LIKE <expression> [ ESCAPE
<expression> ]    # LIKE expression

<literal> ::=
      DATE <quoted_string>
    | TIME <quoted_string>
    | DATETIME <quoted_string>
    | TIMESTAMP <quoted_string>
    | <numeric>
    | <quoted_string>
    | NULL
    | TRUE
    | FALSE

<numeric> ::= UNSIGNED_INTEGER | FLOATING_POINT_NUM

<cursor_attribute> ::= { %ISOPEN | %FOUND | %NOTFOUND | %ROWCOUNT }
```

```
<binary_op> ::=
      AND | XOR | OR
    | = | <=> | != | <> | <= | >= | < | >
    | * | / | + | -
    | >> | << | & | ^ | '|'
    | ||

<unary_op> ::= + | - | NOT | ~

<case_expression> ::=
      CASE <expression> <case_expression_when_part>... [ ELSE
<expression> ] END
    | CASE <case_expression_when_part>... [ ELSE <expression> ] END

<case_expression_when_part> ::= WHEN <expression> THEN <expression>
```

Literals

Literals include date/time, numeric, string, NULL, TRUE, and FALSE values. Except that bit strings and collections are not supported, literal-related rules are the same as described in SQL Literals.

Identifiers

In PL/CSQL statements other than Static/Dynamic SQL, the following kinds of identifiers can be used:

- Variables, constants, cursors, exceptions, and local procedures/functions declared in the declaration section
- Parameters of procedures/functions
- Implicitly declared iterators of FOR loops of type integer or record

Using an identifier without explicit or implicit declaration results in a compile-time error.

Result Count for Static SQL

SQL%ROWCOUNT is an expression that represents the result count immediately after executing a Static SQL statement.

- For SELECT statements not associated with cursors, the INTO clause must be used, and the result must contain exactly one row. Therefore, when the SELECT executes successfully, *SQL%ROWCOUNT* will be 1. If the result has 0 or more than 1 row and causes a runtime error, the value of *SQL%ROWCOUNT* is undefined.

- For INSERT, UPDATE, DELETE, MERGE, REPLACE, and TRUNCATE statements, it returns the number of affected rows.
- For COMMIT and ROLLBACK, it returns 0.

Cursor Attributes

By appending %ISOPEN, %FOUND, %NOTFOUND, or %ROWCOUNT to a *cursor_expression* that evaluates to a cursor or SYS_REFCURSOR variable, you can check one of four cursor attributes.

- %ISOPEN: Whether the cursor is open (BOOLEAN)
- %FOUND: NULL before the first FETCH. Otherwise, whether the last FETCH returned 1 row (BOOLEAN). Raises INVALID_CURSOR Exception if the cursor is not open.
- %NOTFOUND: NULL before the first FETCH. Otherwise, whether the last FETCH returned 0 rows (BOOLEAN). Raises INVALID_CURSOR Exception if the cursor is not open.
- %ROWCOUNT: 0 before the first FETCH. Otherwise, the number of rows fetched so far (BIGINT). Raises INVALID_CURSOR Exception if the cursor is not open.

In the example below, the internal function *iterate_cursor()* iterates over records using cursor attributes and returns the total row count. If the passed cursor is not open (*%ISOPEN* is False), the function returns -1. Whether there are more records is determined using the *%NOTFOUND* attribute after FETCH. The *%ROWCOUNT* attribute increments with each FETCH and reflects the total number of rows fetched after the loop ends.

```
CREATE OR REPLACE PROCEDURE cursor_attributes AS
    ...

    FUNCTION iterate_cursor(rc SYS_REFCURSOR) RETURN INT
    AS
        v VARCHAR;
    BEGIN
        IF rc%ISOPEN THEN
            LOOP
                FETCH rc INTO v;
                EXIT WHEN rc%NOTFOUND;

                -- do something with v
                ...

            END LOOP;

            RETURN rc%ROWCOUNT;      -- number of records
        ELSE
            RETURN -1;               -- error
        END IF;
```

```
    END;
begin
    ...

end;
```

If the *cursor_expression* evaluates to NULL, an INVALID_CURSOR Exception is raised.

Binary Ops, Unary Ops, Parentheses

PL/CSQL uses the following precedence for operators:

Operator	Description
+, -, ~	Unary sign, bitwise NOT
*, /, DIV, MOD	Multiplication, division, integer division, modulus
+, -	Addition, subtraction
||	String concatenation
<<, >>	Bitwise shift
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
IS NULL	NULL test
LIKE	String pattern test

Operator	Description
BETWEEN	Range test
IN	Inclusion test
=, <=>, <, >, <=, >=, <>, !=	Comparison
NOT	Logical NOT
AND	Logical AND
XOR	Logical XOR
OR	Logical OR

- In Non-static/dynamic SQL statements, the % operator cannot be used for modulo, so use MOD instead.
- In Non-static/dynamic SQL statements, logical operators &&, ||, and ! cannot be used, so use AND, OR, and NOT instead.
- In Non-static/dynamic SQL statements, string comparisons follow UTF8 encoding regardless of DB settings, using lexicographical order of Unicode values. In Static/Dynamic SQL statements, encoding and comparison rules follow the database and table settings.

Parentheses can be used to explicitly specify evaluation precedence.

Record Field Reference

PL/CSQL supports record variables in the following two cases:

- Implicitly declared record variables used for iterating over SELECT results in a FOR loop
- Record variables declared using %ROWTYPE

To reference a field in a record variable, append a dot and the field name to the record variable.

```
CREATE PROCEDURE athlete_history(p_name VARCHAR)
AS
    CURSOR my_cursor IS
    SELECT host_year, score
    FROM history
    WHERE athlete = p_name;
BEGIN
    FOR r IN my_cursor LOOP     -- r: implicitly declared
        DBMS_OUTPUT.put_line('host_year: ' || r.host_year || '
score: ' || r.score);    -- r.<column-name>
    END LOOP;
END;
```

Function Call

In a function call expression, the number and types of arguments must match the function's declaration. Arguments passed to OUT parameters of the function must be assignable variables or other OUT parameters, as their values will be changed by the function call.

```
CREATE OR REPLACE FUNCTION callee(o OUT INT) RETURN INT
AS
BEGIN
    ...
END;

CREATE OR REPLACE PROCEDURE proc(i INT, o OUT INT)
AS
    v INT;
    c CONSTANT INT := 0;
BEGIN
    ... callee(i) ...   -- Error: IN argument
    ... callee(o) ...   -- OK: OUT argument
    ... callee(v) ...   -- OK: variable
    ... callee(c) ...   -- Error: constant
END;
```

Callable functions may be stored functions, local functions, or built-in functions. Among these, built-in functions refer to the CUBRID built-in functions listed in Operators and Functions. However, the built-in function IF cannot be used because it conflicts with PL/CSQL syntax.

If an error occurs while calling another stored function or built-in function, an SQL_ERROR Exception is raised.

CASE Expression

A CASE expression sequentially evaluates conditions and returns the value associated with the first condition that is satisfied.

Like the CASE statement, CASE expressions come in two forms:

- The first form includes an expression immediately after the CASE keyword. That expression is evaluated once, then compared against each WHEN clause's expression. When a match is found, the corresponding THEN expression is evaluated and becomes the result of the CASE expression.
- The second form does not have an expression after the CASE keyword. In this case, each WHEN clause must be a BOOLEAN expression. The first one that evaluates to TRUE determines which THEN expression is returned as the CASE result.

Both forms may include an optional ELSE clause, whose expression becomes the CASE value if no conditions match. If no conditions match and no ELSE is present, the CASE expression evaluates to NULL.

The following is an example of the first form:

```
CREATE OR REPLACE PROCEDURE print_even_odd(i INTEGER)
AS
    s VARCHAR(5);
BEGIN
    s := CASE i MOD 2
        WHEN 0 THEN 'Even'
        WHEN 1 THEN 'Odd'
        ELSE 'NULL'
    END;

    DBMS_OUTPUT.put_line(s);
END;
```

The following shows the second form with equivalent behavior:

```
CREATE OR REPLACE PROCEDURE print_even_odd(i INTEGER)
AS
    s VARCHAR(5);
BEGIN
    s := CASE
        WHEN i MOD 2 = 0 THEN 'Even'
        WHEN i MOD 2 = 1 THEN 'Odd'
        ELSE 'NULL'
    END;
```

```
    DBMS_OUTPUT.put_line(s);
END;
```

SQLCODE, SQLERRM

Inside an exception handling block, SQLCODE and SQLERRM represent the error code (INTEGER) and error message (STRING) of the current Exception. Outside an exception block, SQLCODE is 0 and SQLERRM is 'no error'.

```
CREATE OR REPLACE PROCEDURE test_sql_code_errm
AS
BEGIN
    ...
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.put_line('code=' || SQLCODE);
        DBMS_OUTPUT.put_line('error message' || SQLERRM);
END;
```

Java Stored Procedure

CUBRID supports the development of stored functions and procedures using Java. Java stored functions and procedures are executed on a Java Virtual Machine (JVM) hosted by CUBRID. Since Java stored functions and procedures use the JVM, they offer excellent interoperability and portability.

Introduction to Java Stored Procedure

CUBRID supports the development of stored functions and procedures in Java. Java stored functions/procedures are executed on the JVM (Java Virtual Machine) hosted by CUBRID.

You can call Java stored functions/procedures from SQL statements or from Java applications using JDBC.

The advantages of using Java stored functions/procedures are as follows:

- **Productivity and usability**: Java stored functions/procedures, once created, can be reused anytime. They can be called from SQL statements or from Java applications using JDBC.
- **Excellent interoperability and portability**: Java stored functions/procedures use the Java Virtual Machine. Therefore, they can be used on any system where the Java Virtual Machine is available.

Note

- The other languages except Java do not support stored function/procedure. In CUBRID, only Java can implement stored function/procedure.

Prerequisites for Writing and Using Java Stored Functions/Procedures

To use Java stored functions/procedures, the procedure language server must be ready. The procedure language server starts automatically when the database server is started. For more details, refer to CUBRID Procedural Langauge Server and Procedural Langauge Server Configuration.

How to Write and Load Java Stored Function/Procedure

To use a Java stored function/procedure, you can write and publish Java stored function/procedure as follows.

- **Step 1: Write the Java source code**
- **Step 2: Compile the Java source code**
- **Step 3: Load Java Class**
- **Step 4: Publish Stored function/procedure**

Write the Java source code

The following example shows how to write Java stored function/procedure. Here, the Java class method must be **public static**.

```
public class SpCubrid{
    public static String HelloCubrid() {
        return "Hello, Cubrid !!";
    }

    public static int SpInt(int i) {
        return i + 1;
    }

    public static void outTest(String[] o) {
        o[0] = "Hello, CUBRID";
    }
}
```

To access the database from a Java stored function/procedure, you must create a Connection object. See details on how to use the server-side JDBC driver, refer to the Using Server-side Internal JDBC Driver.

Compile the Java source code

Compile the SpCubrid.java file as follows:

```
javac SpCubrid.java
```

When using the server-side JDBC driver, you must compile as follows by specifying the path of JDBC using the **classpath(cp)** option. Note that you must use the latest JDBC driver of the database server to be loaded.

```
javac SpCubrid.java -cp $CUBRID/jdbc/cubrid_jdbc.jar
```

Load the compiled Java class into CUBRID

Load the compiled Java class into CUBRID. You can refer to the loadjava Utility.

```
% loadjava demodb SpCubrid.class
```

Registering Stored Functions/Procedures

CUBRID requires a process to register (publish) Java classes so that Java methods can be called from SQL statements or applications. Create a CUBRID stored function as follows to register the Java class.

```
...
AS LANGUAGE JAVA NAME '<java_call_specification>';
```

In the string following JAVA NAME, specify the class and method names to call the method of the Java class. This is called a **Java Call Specification**.

```
CREATE FUNCTION hello() RETURN STRING
AS LANGUAGE JAVA
NAME 'SpCubrid.HelloCubrid() return java.lang.String';
```

Java Call Specification

You must publish Java classes by using call specifications because it is not known how a function in a class will be called by SQL statements or applications when Java classes are loaded.

With call specifications, Java function names, parameter types, return values and their types can be accessed by SQL statements or applications. To write call specifications, use CREATE FUNCTION or CREATE PROCEDURE statement.

- Java stored function/procedure names are not case sensitive.
- The maximum number of characters a Java stored function/procedure can have is 254 bytes.
- The maximum number of parameters a Java stored function/procedure can have is 64.
- If the parameter of a Java stored function/procedure is set to **OUT**, it will be passed as a one-dimensional array whose length is 1. Therefore, a Java method must store its value to pass in the first space of the array.

```
CREATE PROCEDURE test_out(x OUT STRING)
AS LANGUAGE JAVA
NAME 'SpCubrid.outTest(java.lang.String[] o)';
```

Java Stored Function/Procedure Call

Java stored functions/procedures, once registered, can be called in the same way as PL/CSQL. For more details, refer to Call a Stored Procedure.

If an exception occurs during the execution of a Java stored function/procedure, the exception is logged and stored in the *dbname***_pl.log** file. To display the exception on the screen, change a handler value of the **$CUBRID/java/logging.properties** file to "java.lang.logging.ConsoleHandler" Then, the exception details are displayed on the screen.

Java Stored Procedure Examples

To call a Java stored function/procedure from a Java application, use a **CallableStatement** object.

Create a phone class in the CUBRID database.

```
CREATE TABLE phone(
     name VARCHAR(20),
     phoneno VARCHAR(20)
);
```

Compile the following **PhoneNumber.java** file, load the Java class file into CUBRID, and publish it.

```
import java.sql.*;
import java.io.*;

public class PhoneNumber{
    public static void Phone(String name, String phoneno) throws
Exception{
        String sql="INSERT INTO PHONE(NAME, PHONENO)"+ "VALUES
(?, ?)";
        try{
            Connection conn =
DriverManager.getConnection("jdbc:default:connection:");
            PreparedStatement pstmt = conn.prepareStatement(sql);

            pstmt.setString(1, name);
            pstmt.setString(2, phoneno);
            pstmt.executeUpdate();

            pstmt.close();
            conn.commit();
            conn.close();
        } catch (SQLException e) {
            System.err.println(e.getMessage());
        }
    }
}
```

```
create PROCEDURE phone_info(name varchar, phoneno varchar) as
language java
name 'PhoneNumber.Phone(java.lang.String, java.lang.String)';
```

Create and run the following Java application.

```
import java.sql.*;

public class StoredJDBC{
    public static void main(){
        Connection conn = null;
        Statement stmt= null;
        int result;
        int i;

        try{
            conn =
DriverManager.getConnection("jdbc:CUBRID:localhost:
33000:demodb:::","","");

            CallableStatement cs;
            cs = conn.prepareCall("CALL PHONE_INFO(?, ?)");
```

```
            cs.setString(1, "Jane");
            cs.setString(2, "010-1111-1111");
            cs.executeUpdate();

            conn.commit();
            cs.close();
            conn.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Retrieve the phone class after executing the program above; the following result would be displayed.

```
SELECT * FROM phone;
```

```
name                    phoneno
============================================
    'Jane'                  '010-111-1111'
```

Using Server-side Internal JDBC Driver

To access the database from a Java stored function/procedure, you must use the server-side JDBC driver. The following are possible with the server-side JDBC driver.

- **Executing SQL Statements**
- **Processing Query Result**

The following classes are supported by the server-side JDBC driver. For details on JDBC API support, refer to Appendix.

- **java.sql.Connection**
- **java.sql.Statement**
- **java.sql.PreparedStatement**
- **java.sql.CallableStatement**
- **java.sql.ResultSet**
- **java.sql.ResultSetMetaData**

Warning

java.sql.DatabaseMetaData is not supported yet.

Database operations using the server-side JDBC have the following characteristics.

- Database operations executed in the Java stored function/procedure belongs to the transaction that is called the Java stored function/procedure.
- Transaction-related APIs are ignored.
- There is no need to make the connection to the server-side JDBC driver again.

Creating Connection

To access the database from a Java stored function/procedure, you must use the server-side JDBC driver. To acquire a connection to the database using the server-side JDBC driver, you can use "**jdbc:default:connection:**" as the URL for JDBC connection.

```
Connection conn =
DriverManager.getConnection("jdbc:default:connection:");
```

Note

From the CUBRID 11.2, Acquiring a connection in the following way is no longer supported:

Connection conn = cubrid.jdbc.driver.CUBRIDDriver.getDefaultConnection();

Note

The server-side JDBC is already registered, and you do not need to call "Class.forName("cubrid.jdbc.driver.CUBRIDDriver")"

Executing SQL Statements

When implementing Java stored functions/procedures, queries can be executed using the following JDBC interface in the same way as developing Java applications.

- **java.sql.Statement**
- **java.sql.PreparedStatement**
- **java.sql.CallableStatement**

The following are the queries that can be executed using the above class.

- **DML (Data Manipulation Language)**: Data Manipulation Statements
- **DDL (Data Definition Language)**: Data Definition Statements

> **Note**
>
> The JDBC Statement objects must contain only one SQL statement. Therefore, an error occurs in the following cases:

```
stmt = new Statement ("select * from t1;select * from t2;");
```

The following statements are not supported.

- **TCL (Transaction Control Language)**: Database Transaction

> **Note**
>
> - *commit()*, *rollback()* JDBC API methods corresponding to **COMMIT** and **ROLLBACK** statements respectively are ignored.
> - JDBC API methods corresponding to **SAVEPOINT** statement are not supported.

The example of executing statements

Execute a query that returns a result set and process the query result set

The following example shows how to execute a **SELECT** statement that returns a result set. **SELECT** statement can be executed by creating a **java.sql.Statement** or **java.sql.PreparedStatement** object. The query result can be processed using the result set (**java.sql.ResultSet**).

Note

- java.sql.ResultSet is forward-only and read-only.
- In the case of the client-side JDBC driver, when a query result set is created, cursor holdability is performed by default. In the server-side JDBC driver, resources are managed by the server, so the query result set is internally closed at the end of the stored function/procedure without maintaining a cursor.

Also, result set metadata (**java.sql.ResultSetMetaData**) can be created from the query result set by using the **getMetaData()** function.

```
CREATE OR REPLACE FUNCTION sp_get_athlete_by_ncode (nc STRING)
RETURN STRING as language java name
'TestQuery.printAthelete(java.lang.String) return java.lang.String';
```

```
import java.sql.*;

public class TestQuery {
    public static String printAthelete(String nation_code_filter)
throws SQLException {
        String sql = "SELECT * FROM public.athlete WHERE nation_code
= ?";

        StringBuilder builder = new StringBuilder();
        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn =
DriverManager.getConnection("jdbc:default:connection:");
            pstmt = conn.prepareStatement(sql);

            pstmt.setString(1, nation_code_filter);

            ResultSet rs = pstmt.executeQuery();
            ResultSetMetaData rsmd = rs.getMetaData();

            builder.append("<Column Details>:\n");
            int colCount = rsmd.getColumnCount();
            for (int i = 1; i <= colCount; i++) {
                String colName = rsmd.getColumnName(i);
                String colType = rsmd.getColumnTypeName(i);
                builder.append(colName + "," + colType);
```

```
                if (i != colCount) builder.append("|");
            }

            builder.append("\n<Rows>:\n");
            while (rs.next()) {
                for (int i = 1; i <= rsmd.getColumnCount(); i++) {
                    Object object = rs.getObject(i);
                    if (object != null) {
                        readColumn(i, rsmd, rs, builder);
                    }

                    if (i != rsmd.getColumnCount()) builder.append
("|");
                }
                builder.append("\n");
            }

            rs.close();
        } catch (Exception e) {
            builder.append(e.getMessage());
        } finally {
            if (pstmt != null) pstmt.close();
            if (conn != null) conn.close();
        }

        return builder.toString();
    }

    private static void readColumn(int idx, ResultSetMetaData rsmd,
ResultSet rs, StringBuilder stringBuilder) throws SQLException {
        switch (rsmd.getColumnType(idx)) {
            case java.sql.Types.DOUBLE:
                stringBuilder.append(rs.getDouble(idx));
                break;
            case java.sql.Types.FLOAT:
                stringBuilder.append(rs.getFloat(idx));
                break;
            case java.sql.Types.VARCHAR:

stringBuilder.append("\"").append(rs.getString(idx)).append("\"");
                break;
            case java.sql.Types.INTEGER:
            case java.sql.Types.TINYINT:
            case java.sql.Types.SMALLINT:
            case java.sql.Types.BIGINT:
                stringBuilder.append(rs.getInt(idx));
                break;
            case java.sql.Types.DATE:

stringBuilder.append("\"").append(rs.getDate(idx)).append("\"");
                break;
            case java.sql.Types.TIMESTAMP:
```

```
stringBuilder.append("\"").append(rs.getTimestamp(idx)).append("\"");
                break;
            default:
                stringBuilder.append(rs.getObject(idx));
                break;
        }
    }
}
```

```
SELECT sp_get_athlete_by_ncode ('ESP');

sp_get_athlete_by_ncode('ESP')
======================
'<Column Details>:
code,INTEGER|name,VARCHAR|gender,CHAR|nation_code,CHAR|event,VARCHAR
<Rows>:
10999|"Fernandez Jesus"|M|ESP|"Handball"
10997|"Fernandez Isabel"|W|ESP|"Judo"
10994|"Fernandez Abelardo"|M|ESP|"Football"
10948|"Etxaburu Aitor"|M|ESP|"Handball"
10941|"Estiarte Manuel"|M|ESP|"Water Polo"
...
```

INSERT, UPDATE, DELETE

The following is an example of executing the **INSERT** statement. **INSERT**, **UPDATE**, **DELETE** statements can be executed through the **executeUpdate()** function.

```
import java.sql.*;

public class Athlete {
    public static void insertAthlete(String name, String gender,
String nation_code, String event) throws SQLException {
        String sql = "INSERT INTO ATHLETE(NAME, GENDER, NATION_CODE,
EVENT)" + "VALUES (?, ?, ?, ?)";

        Connection conn = null;
        PreparedStatement pstmt = null;

        try{
            conn =
DriverManager.getConnection("jdbc:default:connection:");
            pstmt = conn.prepareStatement(sql);

            pstmt.setString(1, name);
            pstmt.setString(2, gender);
            pstmt.setString(3, nation_code);
            pstmt.setString(4, event);;
```

```
            pstmt.executeUpdate();

            pstmt.close();
            conn.commit();
            conn.close();
        } catch (Exception e) {
            System.err.println(e.getMessage());
        } finally {
            if (pstmt != null) pstmt.close();
            if (conn != null) conn.close();
        }
    }
}
```

Note

conn.commit() is ignored at the Athlete class example above.

OUT Parameters of Primitive Types

When changing an argument value in Java in a Java stored function/procedure of CUBRID, the changed value must be passed when an argument is passed as an OUT argument as a one-dimensional array.

```
CREATE PROCEDURE sp_increment_me(x IN OUT INT) AS LANGUAGE JAVA NAME
'OutTest.incrementInt(int[])';
```

```
public class OutTest {
    public static void incrementInt(int[] arg) {
        arg[0] = arg[0] + 1;
    }
}
```

OUT Parameters of Set Types

Parameters of Java methods corresponding to an OUT (or IN OUT) parameter of an SQL set type must be declared as an two-dimensional array of an appropriate type.

```
CREATE PROCEDURE setoid(x in out set, z object) AS LANGUAGE JAVA
NAME 'SetOIDTest.SetOID(cubrid.sql.CUBRIDOID[][],
cubrid.sql.CUBRIDOID)';
```

```
import cubrid.sql.CUBRIDOID;

public class SetOIDTest {
    public static void SetOID(CUBRIDOID[][] set, CUBRIDOID aoid) {
        String ret="";
        Vector v = new Vector();

        CUBRIDOID[] set1 = set[0];

        try {
            if(set1 != null) {
                int len = set1.length;
                int i = 0;

                for (i = 0; i < len; i++)
                    v.add(set1[i]);
            }

            v.add(aoid);
            set[0] = (CUBRIDOID[]) v.toArray(new CUBRIDOID[]{});

        } catch(Exception e) {
            e.printStackTrace();
            System.err.println("SQLException:"+e.getMessage());
        }
    }
}
```

OUT Parameters of CUBRID OID type

In case of using an OUT (or IN OUT) parameter of CUBRID OID type, declare the corresponding parameter of the Java method as an array of CUBRIDOID class (cubrid.sql.CUBRIDOID).

```
CREATE PROCEDURE tOID(i inout object, q string) AS LANGUAGE JAVA
NAME 'OIDtest.tOID(cubrid.sql.CUBRIDOID[], java.lang.String)';
```

```
import java.sql.*;
import cubrid.sql.CUBRIDOID;

public class OIDtest {
    public static void tOID(CUBRIDOID[] oid, String query)
    {
        Connection conn = null;
        Statement stmt = null;
        String ret = "";

        try {
```

```
            conn =
DriverManager.getConnection("jdbc:default:connection:");

            conn.setAutoCommit(false);
            stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query);
            System.out.println("query:"+ query);

            while(rs.next()) {
                oid[0] = (CUBRIDOID) rs.getObject(1);
                System.out.println("oid:" + oid[0].getTableName());
            }

            stmt.close();
            conn.close();

        } catch (SQLException e1) {
            e1.printStackTrace();
            System.err.println("SQLException:" + e1.getMessage());
        } catch (Exception e2) {
            e2.printStackTrace();
            system.err.println("Exception:" + e2.getMessage());
        }
    }
}
```

Returning java.sql.ResultSet in Java Stored Procedure

In CUBRID, a query result set (**java.sql.ResultSet**) can be returned, and **CURSOR** is used as the returned data type when declared.

Note

- **java.sql.ResultSet** cannot be used as an input argument of a function, and an error occurs if it is passed as an IN argument.
- An error also occurs when calling a function that returns **ResultSet** in a non-Java environment.

```
CREATE FUNCTION rset() RETURN CURSOR AS LANGUAGE JAVA
NAME 'JavaSP2.TResultSet() return java.sql.ResultSet'
```

```
import java.sql.*;
```

```
public class JavaSP2 {
    public static ResultSet TResultSet(){
        try {
            Connection conn =
DriverManager.getConnection("jdbc:default:connection:");

            String sql = "select * from station";
            Statement stmt=conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql);

            return rs;
        } catch (Exception e) {
            e.printStackTrace();
        }

        return null;
    }
}
```

In the calling block, you must set the OUT argument with **Types.JAVA_OBJECT**, get the argument to the **getObject** () function, and then cast it to the **java.sql.ResultSet** type before you use it. In addition, the **java.sql.ResultSet** is only available to use in **CallableStatement** of JDBC.

```
import java.sql.*;

public class TestResultSet{
    public static void main(String[] args) {
        Connection conn = null;

        try {
            conn =
DriverManager.getConnection("jdbc:default:connection:");

            CallableStatement cstmt = conn.prepareCall("?=CALL
rset()");
            cstmt.registerOutParameter(1, Types.JAVA_OBJECT);
            cstmt.execute();
            ResultSet rs = (ResultSet) cstmt.getObject(1);

            while(rs.next()) {
                System.out.println(rs.getString(1));
            }

            rs.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Getting information about connection client

```
CREATE OR REPLACE FUNCTION sp_client_info () RETURN STRING as
language java name 'SpTestClientInfo.getClientInfo() return
java.lang.String';
```

```
import java.util.Properties;
import java.sql.*;

public class SpTestClientInfo {
    public static String getClientInfo() {
        Connection conn = null;
        String result = "";

        try {
            conn =
DriverManager.getConnection("jdbc:default:connection:");

            Properties props = conn.getClientInfo();

            // How to get from the Properties
            // String user = props.getProperty ("user");

            result = props.toString ();
        } catch (Exception e) {
            result = e.getMessage ();
        }
        return result;
    }
}
```

```
SELECT sp_client_info ();

sp_client_info()
======================
'{pid=200270, user=DBA, login=cubrid, program=csql, type=2,
host=cubrid, ip=192.168.2.201}'
```

Connecting to Other Databases

You can connect to another outside database instead of the currently connected one even when the server-side JDBC driver is being used. Acquiring a connection to an outside database is not different from a generic JDBC connection. For details, see JDBC API.

⚠ Warning

If you connect to other databases, the connection to the CUBRID database does not terminate automatically even when the execution of the Java method ends. Therefore, the connection must be explicitly closed so that the result of transaction operations such as **COMMIT** or **ROLLBACK** will be reflected in the database. That is, a separate transaction will be performed because the database that called the Java stored function/procedure is different from the one where the actual connection is made.

```
import java.sql.*;

public class SelectData {
    public static void SearchSubway(String[] args) throws Exception {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            conn =
DriverManager.getConnection("jdbc:CUBRID:localhost:
33000:demodb:::","","");

            String sql = "select line_id, line from line";
            stmt = conn.createStatement();
            rs = stmt.executeQuery(sql);

            while(rs.next()) {
                int host_year = rs.getString("host_year");
                String host_nation = rs.getString("host_nation");

                System.out.println("Host Year ==> " + host_year);
                System.out.println(" Host Nation==> " + host_nation);
                System.out.println("\n==========\n");
            }

            rs.close();
        } catch (SQLException e1) {
            System.err.println(e1.getMessage());
        } catch (Exception e2) {
            System.err.println(e2.getMessage());
        } finally {
            if (stmt != null) stmt.close();
            if (conn != null) conn.close();
        }
```

```
    }
}
```

When Java stored functions/procedures are executed, they should run only on a JVM located in the database server. You can check where they are running by calling System.getProperty ("cubrid.server.version") from the Java programs. The result is the database version if it is called from the database; otherwise, it is **NULL**.

Java Native Interface (JNI) Support

Using the Java Native Interface (JNI), you can invoke functions in native languages like C/C++ from the Java Virtual Machine (JVM). Java Stored Procedures (SP) in CUBRID provide support for JNI functionality, but you should be cautious when using it because issues in native code can have unexpected impacts on Procedural Languge Server (cub_pl) process and its operation. Java Classes used for loading native libraries should be registered using the **-j** option of **loadjava** to prevent them from being dynamically loaded. For more details, refer to :ref:jsp-load-java.

The following is an example of invoking a native function through JNI in a CUBRID Java stored function:

HelloJNI.h

```
#include <jni.h>

#ifndef _Included_HelloJNI
#define _Included_HelloJNI
#ifdef __cplusplus
extern "C" {
#endif

/*
* Class:     HelloJNI
* Method:    sayHello
* Signature: ()V
*/
JNIEXPORT jstring JNICALL Java_HelloJNI_sayHello(JNIEnv *,
jobject, jstring);

#ifdef __cplusplus
}
#endif
#endif
```

HelloJNI.c

```
#include <jni.h>
#include <stdio.h>
#include <string.h>
#include "HelloJNI.h"

// Implementation of native method sayHello() of HelloJNI class
JNIEXPORT jstring JNICALL Java_HelloJNI_sayHello(JNIEnv *env,
jobject thisObj, jstring javaString) {
    const char *nativeString = (*env)->GetStringUTFChars(env,
javaString, 0);
    // printf("Java_HelloJNI_sayHello : %sn", nativeString);
    const char *greeting = " Hello!";
    char cap[1024];
    strcpy(cap, nativeString);
    strcat(cap, greeting);
    (*env)->ReleaseStringUTFChars(env, javaString, nativeString);
    return (*env)->NewStringUTF(env, cap);
}
```

HelloJNI.java

```
import java.io.File;

public class HelloJNI {
    static {
        try {
            String cubridPath = System.getenv("CUBRID"); // get
$CUBRID
            System.load(
                cubridPath
                + File.separator
                + "jni"
                + File.separator
                + "libhello.so"); // $CUBRID/jni/libhello.so
        } catch(UnsatisfiedLinkError e) {
            e.printStackTrace();
        }
    }

    // Declare
    private native String sayHello(String string);

    // CUBRID
    public static String cubridSayHello(String string) {
        return new HelloJNI().sayHello(string); // invoke the
native method
    }
}
```

```
-- compile and copy HelloJNI.c
gcc -fPIC -I${JAVA_HOME}/include -I${JAVA_HOME}/include/linux -
shared -o libhello.so HelloJNI.c
mkdir -p $CUBRID/jni
cp libhello.so $CUBRID/jni

-- loadjava
javac HelloJNI.java
loadjava -j demodb HelloJNI.class
```

```
CREATE FUNCTION hello(str VARCHAR) RETURN VARCHAR AS LANGUAGE JAVA
NAME 'HelloJNI.cubridSayHello(java.lang.String) return
java.lang.String';

SELECT hello ('CUBRID');
```

```
hello('CUBRID')
=======================
'CUBRID Hello!'
```

⚠ Warning

When a java stored procedure/function included JNI code is loaded without **-j** or **–jni** option, the following error is returned during execution. 'Library load not allowed. Please load your class by using 'loadjava' with '-jni' option'

To address this issue, please consider the following:

- **If you are loading multiple Java class files that call System.load() for the same native library path:**
    - Modify the Java class files to load the native library from only one class file.
    - Register the class that loads the native library using the **-j** option of loadjava.
    - Restart the PL server. (see CUBRID Procedural Langauge Server)
- **If you are overwriting a previously loaded Java class file using loadjava:**
    - Re-register the class using the -j option with loadjava.
    - Restart the pl utility. (see CUBRID Procedural Langauge Server)

loadjava Utility

You can load a Java *.class or *.jar file using **loadjava** utility. The file is moved to a database internal path.

```
loadjava [option] database-name java-class-file
```

- *database-name*: The name of the database where the Java file to be loaded.
- *java-class-file*: The name of the Java class or jar file to be loaded.
- [*option*]
  - **-y** (or **–overwrite**): Automatically overwrites an existing file with the same name. By default, the tool prompts for confirmation if a file with the same name exists and this option is not provided.
  - **-j** (or **–jni**): Specifies that the Java *.class or *.jar file includes JNI (native libraries). This prevents native library loading errors and moves the file to the static load path.
  - **-p** (or **–package**): Specifies the package name when loading Java class files with a package declaration. If the class belongs to the default package, this option is not required.

```
* In the examples provided in the manual, the Java files do not
specify a package at the top of the code. In this case, the default
package is used, and there is no need to use the **-p** option.
* However, when loading a Java file that includes a package
declaration, you must use the **-p** option to specify the package
name.
```

Example: Loading a Java Class with a Java package name

If a Java class includes a package declaration, such as:

```
package org.cubrid;

public class NumberUtils {
    public static int addIntegers(int num1, int num2) {
        return num1 + num2;
    }
}
```

1. **Compile the Java File**

```
javac NumberUtils.java
```

2. **Load the Compiled File into the Database**

```
loadjava demodb -p org.cubrid NumberUtils.class
```

3. **Create a Stored Function in SQL**

```
CREATE OR REPLACE FUNCTION add_integer(a INT, b INT) RETURN INT
AS LANGUAGE JAVA
NAME 'org.cubrid.NumberUtils.addIntegers(int, int) return int';
```

4. **Use the Function**

```
SELECT add_integer(1, 2);
```

Output:

```
add_integer(1, 2)
====================
                   3
```

Caution

- java.sql.DatabaseMetaData is not supported.
- JDBC API related to BLOB/CLOB type is not supported.
- Functions not related to query execution and used only in client-side JDBC are not supported. For details, refer to Appendix.
- Multiple SQL statements are not supported when executing a query with one JDBC object.
- ResultSet created by query execution is non-updatable, non-scrollable, and non-sensitive.
- Java ignores precision, scale, and length parts of SQL types of IN/OUT parameters, matches only the type name parts, and delivers values as they are.
- A stored procedure can call another stored procedure or call itself recursively. The maximum nesting depth is 16.

Limitations on the precision of IN/OUT parameters and a return value

To limit the return value of Java stored function/procedure and precision type on IN/OUT, CUBRID processes as follows:

- Checks the SQL type of the Java stored function/procedure.
- Passes the value returned by Java to the database with only the type converted if necessary, ignoring the number of digits defined during creating the Java stored function/procedure.
- In principle, the user should directly manipulates the data which is passed to the database.

Take a look at the following **typestring** () Java stored function.

```
public class JavaSP1 {
    public static String typestring() {
        String temp = " ";
        for(int i = 0; i < 1; i++) {
            temp = temp + "1234567890";
        }
        return temp;
    }
}
```

```
CREATE FUNCTION typestring() RETURN CHAR(5) AS LANGUAGE JAVA
NAME 'JavaSP1.typestring() return java.lang.String';

CALL typestring();
```

```
  Result
======================
  ' 1234567890'
```

Appendix

Data Type Mapping

When a Java stored function/procedure is published, it is not checked whether the return definition of the Java stored function/procedure coincides with the one in the declaration of the Java method. Therefore, the Java stored function/procedure follows the return definition (SQL Type) provided at the time of registration. The return definition in the declaration is significant only as user-defined information.

In call specifications, the data types of SQL must correspond to the data types of Java parameter and return value. In addition, when implementing a Java stored function/procedure, the data types of Java must match the data types of query result (ResultSet). The following table shows SQL/Java data types allowed in CUBRID.

Data Type Mapping

Category	SQL Type	Java Type
Numeric Types	SHORT, SMALLINT	short, java.lang.Short
	INT, INTEGER	int, java.lang.Integer
	BIGINT	long, java.lang.Long
	NUMERIC, DECIMAL	java.math.BigDecimal
	FLOAT, REAL	float, java.lang.Float
	DOUBLE, DOUBLE PRECISION	double, java.lang.Double
Date/Time Types	DATE	java.sql.Date
	TIME	java.sql.Time
	TIMESTAMP	java.sql.Timestamp
	DATETIME	java.sql.Timestamp
	TIMESTAMPLTZ	X (not supported)
	TIMESTAMPTZ	X (not supported)

Category	SQL Type	Java Type
	DATETIMELTZ	X (not supported)
	DATETIMETZ	X (not supported)
Bit String Types	BIT	X (not supported)
	VARBIT	X (not supported)
Character String Types	CHAR	java.lang.String
	VARCHAR	java.lang.String
Enum Type	ENUM	X (not supported)
LOB Types	CLOB, BLOB	X (not supported)
Collection Types	SET, MULTISET, SEQUENCE	java.lang.Object[], java primitive type array, java wrapper class array
Special Types	JSON	X (not supported)
	OBJECT, OID	cubrid.sql.CUBRIDOID <interface>
	CURSOR	java.sql.ResultSet <interface>

Implicit Data Type Conversion

If the data type of SQL and the data type of Java do not correspond as shown in the table above, CUBRID implicitly attempts data type conversion according to the following table. Please note that implicit data conversion may result in data loss.

	Java Data Types									
SQL Data Types	**byte, java.lang.Byte**	**short, java.lang.Short**	**int, java.lang.Integer**	**long, java.lang.Long**	**float, java.lang.Float**	**double, java.lang.Double**	**java.math.BigDecimal**	**java.lang.String**	**java.sql.Time**	**java.sql.Timestamp**
SHORT, SMALLINT	O	O	O	O	O	O	O	O	X	X
INT, INTEGER	O	O	O	O	O	O	O	O	X	X
BIGINT	O	O	O	O	O	O	O	O	X	X
NUMERIC, DECIMAL	O	O	O	O	O	O	O	O	X	X
FLOAT, REAL	O	O	O	O	O	O	O	O	X	X
DOUBLE DOUBLE	O	O	O	O	O	O	O	O	X	X

	Java Data Types									
SQL Data Types	byte, java.lang.Byte	short, java.lang.Short	int, java.lang.Integer	long, java.lang.Long	float, java.lang.Float	double, java.lang.Double	java.math.BigDecimal	java.lang.String	java.sql.Time	java.sql.Timestamp
PRECISION										
DATE	X	X	X	X	X	X	X	O	O	O
TIME										
TIMESTAMP										
DATETIME										
CHAR	O	O	O	O	O	O	O	O	O	O
VARCHAR										
SET	X	X	X	X	X	X	X	X	X	X
MULTISET										

	Java Data Types									
SQL Data Types	byte, java.lang.Byte	short, java.lang.Short	int, java.lang.Integer	long, java.lang.Long	float, java.lang.Float	double, java.lang.Double	java.math.BigDecimal	java.lang.String	java.sql.Time	java.sql.Timestamp
SEQUENCE										

- X: Conversion not allowed
- O: Implicit conversion

Table of Supproting JDBC API

JDBC Interface	Support/Unsupport
java.sql.CallableStatement	Support
java.sql.Connection	Support
java.sql.Driver	Support (Creating Connection)
java.sql.PreparedStatement	Support
java.sql.ResultSet	Support
java.sql.ResultSetMetaData	Support
CUBRIDOID	Support

JDBC Interface	Support/Unsupport
java.sql.Statement	Support
java.sql.DriverManager	Support
Java.sql.SQLException	Support
java.sql.Array	Unsupport
java.sql.Blob	Unsupport
java.sql.Clob	Unsupport
java.sql.DatabaseMetaData	Unsupport
java.sql.ParameterMetaData	Unsupport
java.sql.Ref	Unsupport
java.sql.Savepoint	Unsupport
java.sql.SQLData	Unsupport
java.sql.SQLInput	Unsupport
java.sql.Struct	Unsupport

Note

JDBC APIs not specified in the table below are not supported and return SQLException.

java.sql.Connection

Method	Description
Properties getClientInfo()	Getting information about connection client
void rollback()	do nothing
Statement createStatement()	Executing SQL Statements
Statement createStatement(int resultSetType, int resultSetConcurrency)	Executing SQL Statements
Statement createStatement(int resultSetType, int resultSetConcurrency, int resultSetHoldability)	Executing SQL Statements
CallableStatement prepareCall(String sql)	Executing SQL Statements
CallableStatement prepareCall(String sql, int resultSetType, int resultSetConcurrency)	Executing SQL Statements
CallableStatement prepareCall(String sql, int resultSetType, int resultSetConcurrency, int resultSetHoldability)	Executing SQL Statements
PreparedStatement prepareStatement(String sql)	Executing SQL Statements
PreparedStatement prepareStatement(String sql, int autoGeneratedKeys)	Executing SQL Statements
	Executing SQL Statements

Method	Description
PreparedStatement prepareStatement(String sql, int[] columnIndexes)	
PreparedStatement prepareStatement(String sql, int resultSetType, int resultSetConcurrency)	Executing SQL Statements
PreparedStatement prepareStatement(String sql, int resultSetType, int resultSetConcurrency, int resultSetHoldability)	Executing SQL Statements
PreparedStatement prepareStatement(String sql, String[] columnNames)	Executing SQL Statements
void clearWarnings()	do nothing
void close()	close all statements
void commit()	do nothing
boolean getAutoCommit()	return false
String getCatalog()	return
int getHoldability()	return ResultSet.HOLD_CURSORS_OVER_COMMIT;
int getTransactionIsolation()	

Method	Description
SQLWarning getWarnings()	return null
boolean isClosed()	return false
boolean isReadOnly()	return false
boolean isValid(int timeout)	return true
void setAutoCommit(boolean autoCommit)	do nothing
void setCatalog(String catalog)	do nothing
void setHoldability(int holdability)	do nothing
void setReadOnly(boolean readOnly)	do nothing
void setTransactionIsolation(int level)	do nothing

java.sql.Statement

Method	Description
Connection getConnection()	
int getFetchDirection()	retruns ResultSet.FETCH_FORWARD
int getFetchSize()	
int getMaxFieldSize()	

Method	Description
int getMaxRows()	
int getQueryTimeout()	retruns 0
int getResultSetConcurrency()	retruns ResultSet.CONCUR_UPDATABLE
int getResultSetHoldability()	return ResultSet.HOLD_CURSORS_OVER_COMMIT or ResultSet.CLOSE_CURSORS_AT_COMMIT
int getResultSetType()	return ResultSet.TYPE_FORWARD_ONLY
int getUpdateCount()	return -1
boolean isClosed()	
void setFetchDirection(int direction)	
void setFetchSize(int rows)	
void setMaxFieldSize(int max)	
void setMaxRows(int max)	
void setQueryTimeout(int seconds)	
void close()	
boolean execute(String sql)	

Method	Description
boolean execute(String sql, int autoGeneratedKeys)	
boolean execute(String sql, int[] columnIndexes)	
boolean execute(String sql, String[] columnNames)	
executeBatch()	throws SQLException
ResultSet executeQuery(String sql)	
int executeUpdate(String sql)	
int executeUpdate(String sql, int autoGeneratedKeys)	
int executeUpdate(String sql, int[] columnIndexes)	
int executeUpdate(String sql, String[] columnNames)	
ResultSet getGeneratedKeys()	
boolean getMoreResults()	
ResultSet getResultSet()	

Method	Description
void cancel()	do nothing
void clearWarnings()	
SQLWarning getWarnings()	
void setCursorName(String name)	
void setEscapeProcessing(boolean enable)	

java.sql.PreparedStatement

Method	Description
boolean execute()	
ResultSet executeQuery()	
int executeUpdate()	
ResultSetMetaData getMetaData()	
void setBigDecimal(int parameterIndex, BigDecimal x)	
void setBoolean(int parameterIndex, boolean x)	

Method	Description
void setByte(int parameterIndex, byte x)	
void setBytes(int parameterIndex, byte[] x)	
void setDate(int parameterIndex, Date x)	
void setDate(int parameterIndex, Date x, Calendar cal)	
void setDouble(int parameterIndex, double x)	
void setFloat(int parameterIndex, float x)	
void setInt(int parameterIndex, int x)	
void setLong(int parameterIndex, long x)	
void setNull(int parameterIndex, int sqlType)	
void setNull(int parameterIndex, int sqlType, String typeName)	
void setObject(int parameterIndex, Object x)	
void setObject(int parameterIndex, Object x, int targetSqlType)	
void setObject(int parameterIndex, Object x, int targetSqlType, int scaleOrLength)	

Method	Description
void setShort(int parameterIndex, short x)	
void setString(int parameterIndex, String x)	
void setTime(int parameterIndex, Time x)	
void setTime(int parameterIndex, Time x, Calendar cal)	
void setTimestamp(int parameterIndex, Timestamp x)	
void setTimestamp(int parameterIndex, Timestamp x, Calendar cal)	

java.sql.CallableStatement

Method	Description
BigDecimal getBigDecimal(int parameterIndex)	
boolean getBoolean(int parameterIndex)	
byte getByte(int parameterIndex)	
byte[] getBytes(int parameterIndex)	

Method	Description
Date getDate(int parameterIndex)	
Date getDate(int parameterIndex, Calendar cal)	
double getDouble(int parameterIndex)	
getFloat(int parameterIndex)	
getInt(int parameterIndex)	
getLong(int parameterIndex)	
getObject(int parameterIndex)	
getShort(int parameterIndex)	
getString(int parameterIndex)	
getTime(int parameterIndex)	
getTime(int parameterIndex, Calendar cal)	
getTimestamp(int parameterIndex)	
getTimestamp(int parameterIndex, Calendar cal)	
registerOutParameter(int parameterIndex, int sqlType)	

Method	Description
registerOutParameter(int parameterIndex, int sqlType, int scale)	
registerOutParameter(int parameterIndex, int sqlType, String typeName)	
wasNull()	

java.sql.ResultSet

Method	Description
clearWarnings()	
close()	
deleteRow()	throws SQLException
findColumn(String columnLabel)	
first()	throws SQLException
getBoolean(int columnIndex)	
getBoolean(String columnLabel)	
getByte(int columnIndex)	
getByte(String columnLabel)	

Method	Description
getBytes(int columnIndex)	
getBytes(String columnLabel)	
getConcurrency()	return ResultSet.CONCUR_READ_ONLY;
getDate(int columnIndex)	
getDate(int columnIndex, Calendar cal)	
getDate(String columnLabel)	
getDate(String columnLabel, Calendar cal)	
getDouble(int columnIndex)	
getDouble(String columnLabel)	
getFetchDirection()	
getFetchSize()	
getFloat(int columnIndex)	
getFloat(String columnLabel)	
getHoldability()	
getInt(int columnIndex)	

Method	Description
getInt(String columnLabel)	
getLong(int columnIndex)	
getLong(String columnLabel)	
getMetaData()	
getObject(int columnIndex)	
getObject(String columnLabel)	
getRow()	
getShort(int columnIndex)	
getShort(String columnLabel)	
getStatement()	
getString(int columnIndex)	
getString(String columnLabel)	
getTime(int columnIndex)	
getTime(int columnIndex, Calendar cal)	
getTime(String columnLabel)	

Method	Description
getTime(String columnLabel, Calendar cal)	
getTimestamp(int columnIndex)	
getTimestamp(int columnIndex, Calendar cal)	
getTimestamp(String columnLabel)	
getTimestamp(String columnLabel, Calendar cal)	
getType()	retruns ResultSet.TYPE_FORWARD_ONLY
isAfterLast()	
isBeforeFirst()	
isClosed()	return false
isFirst()	
isLast()	
wasNull()	
getCursorName()	return
getWarnings()	return null

java.sql.ResultSetMetaData

Method	Description
getCatalogName (int column)	return
getColumnClassName(int column)	
getColumnCount()	
getColumnDisplaySize(int column)	
getColumnLabel(int column)	
getColumnName(int column)	
getColumnType(int column)	
getColumnTypeName(int column)	
getPrecision(int column)	
getScale(int column)	
getSchemaName(int column)	return
getTableName(int column)	
isAutoIncrement(int column)	
isCaseSensitive(int column)	

Method	Description
isCurrency(int column)	
isDefinitelyWritable(int column)	return false
isNullable(int column)	
isReadOnly(int column)	return false
isSearchable(int column)	return true
isSigned(int column)	
isWritable(int column)	return true

Method

A method is a program written in C with built-in functions of the CUBRID database system, and is called by the **CALL** statement. When a method is called, the program is loaded and linked with the running application by a dynamic loader. The return value generated by the method execution is delivered to the caller.

Limitations

- Currently, methods are only available in some systems of the CUBRID database, such as the User Authorization Management METHOD.
- User-defined methods are not recommended due to many constraints and are only used in system-built methods.

Method Type

The CSQL language supports the following two types of methods: class and instance methods.

- The **class method** is a method called by a class object. It is usually used to create a new class instance or to initialize it. It is also used to access or update class attributes.

- The **instance method** is a method called by a class instance. It is used more often than the class method because most operations are executed in the instance. For example, an instance method can be written to calculate or update the instance attribute. This method can be called from any instance of the class in which the method is defined or of the sub class that inherits the method.

The method inheritance rules are similar to those of the attribute inheritance. The sub class inherits classes and instance methods from the super class. The sub class can follow the class or instance method definition from the super class.

The rules for resolving method name conflicts are same as those for attribute name conflicts. For details about attribute/method inheritance conflicts, see Class Conflict Resolution.

CALL Statement

The **CALL** statement is used to call a method defined in the database. Both class and instance methods can be called by the **CALL** statement. If you want to see example of using the CALL statement, see User Authorization Management METHOD.

```
CALL <method_call> ;

<method_call> ::=
    method_name ([<arg_value> [{, <arg_value> } ...]]) ON
<call_target> [<to_variable>] |
    method_name (<call_target> [, <arg_value> [{,
<arg_value>} ...]] ) [<to_variable>]

    <arg_value> ::=
        any CSQL expression

    <call_target> ::=
        an object-valued expression

    <to_variable> ::=
        INTO variable |
        TO variable
```

- The *method_name* is either the method name defined in the table or the system-defined method name. A method requires one or more parameters. If there is no parameter for the method, a set of blank parentheses must be used.

- <*call_target*> can use an object-valued expression that contains a class name, a variable, another method call (which returns an object). To call a class method for a class object, you must place the **CLASS** keyword before the <*call_target*>. In this case, the table name must be the name of the class where the table method is defined. To call a record method, you must specify the expression representing the record object. You can optionally store the value

returned by the table or record method in the <*to_variable*>. This returned variable value can be used in the **CALL** statement just like the <*call_target*> or <*arg_value*> parameter.

- Calling nested methods is possible when other *method_call* is the <*call_target*> of the method or given as one of the <*arg_value*> parameters.

CUBRID Management

This chapter describes how the database administrators (**DBA**) operates the CUBRID system.

- It includes instructions on how to use the **cubrid** utility, which starts and stops various processes of the CUBRID server, the broker, procedure language server, and manager server. See Controlling CUBRID Processes.
- It includes instructions on the following: database management tasks (creating and deleting databases, adding volume, etc.), migration tasks (moving database to a different location or making changes so that it fits the system's version), and making backups and rollbacks of the database in case of failures. See cubrid Utilities.
- It includes instructions on the system configuration. See System Parameters.
- It includes how to use SystemTap, which can monitors and traces the operating processes dynamically. See SystemTap.
- It includes instructions on troubleshooting. See Troubleshooting.

The **cubrid** utilities provide features that can be used to comprehensively manage the CUBRID service. The CUBRID utilities are divided into the service management utility, which is used to manage the CUBRID service process, and the database management utility, which is used to manage the database.

The service management utilities are as follows:

- Service utility : Operates and manages the master process.
  - cubrid service
- Server utility : Operates and manages the server process.
  - cubrid server
- Broker utility : Operates and manages the broker process and application server (CAS) process.
  - cubrid broker

- Gateway utility : Operates and manages the gateway process and application server (CAS) process. * cubrid gateway
- Manager utility : Operates and manages the manager server process.
  - cubrid manager
- HA utility : Operates and manages the HA-related processes.
  - cubrid heartbeat
- PL utility : Operates and manages the procedure language (PL) server.
  - cubrid pl

See Controlling CUBRID Processes for details.

The database management utilities are as follows:

- Creating database, adding volume, and deleting database
  - cubrid createdb
  - cubrid addvoldb
  - cubrid deletedb
- Renaming database, altering host, copying/moving database, and registering database
  - cubrid renamedb
  - cubrid alterdbhost
  - cubrid copydb
  - cubrid installdb
- Backing up database
  - cubrid backupdb
- Restoring database
  - cubrid restoredb
- Unloading and Loading database
  - cubrid unloaddb
  - cubrid loaddb

- Checking and compacting database space
  - cubrid spacedb
  - cubrid compactdb
- Updating statistics and checking query plan
  - cubrid plandump
  - cubrid optimizedb
  - cubrid statdump
- Checking database lock, checking transaction and killing transaction
  - cubrid lockdb
  - cubrid tranlist
  - cubrid killtran
- Diagnosing database and dumping parameter
  - cubrid checkdb
  - cubrid diagdb
  - cubrid paramdump
- Changing HA mode, replicating/applying logs
  - cubrid changemode
  - cubrid applyinfo
- Compiling/Outputting locale
  - cubrid genlocale
  - cubrid dumplocale

See cubrid Utilities for details.

Note

If you want to control the service by using **cubrid** utility on Windows Vista or later, it is recommended that you run the command prompt with an administrator account. If you use

cubrid utility without an administrator account, the result message is not displayed even though you can run it through the User Account Control (UAC) dialog.

To run the command prompt on Windows Vista or later with an administrator account, right-click [Start] > [All Programs] > [Accessories] > [Command Prompt] and select [Run as Administrator]. In the dialog verifying authorization, click [Yes], and then the command prompt is run as an administrator account.

Controlling CUBRID Processes

CUBRID processes can be controlled by **cubrid** utility.

Controlling CUBRID Service

The following **cubrid** utility syntax shows how to control services registered in the configuration file. One of the following can be specified in <command>.

```
cubrid service <command>
<command>: {start|stop|restart|status}
```

- start: start services.
- stop: stop services.
- restart: restart services.
- status: check status.

No additional options or arguments are required.

Controlling Database Server

The following **cubrid** utility syntax shows how to control database server process.

```
cubrid server <command> [database_name]
<command>: {start|stop|restart|status}
```

One of the following can be specified in <command>:

- start: start a database server process.

- stop: stop a database server process.
- restart: restart a database server process.
- status: check status of a database server process.

Every command can specify a database name (**[database_name]**) as an argument.
If the database name is not specified, the **status** command displays the currently running database servers' information, and the commands except **status** refer to the database names in the **server** property of the **[service]** section of **cubrid.conf**.

```
# cubrid.conf

[service]

...

server=demodb,testdb

...
```

```
% cubrid server start

@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success

@ cubrid server start: testdb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
```

Controlling Broker

The following cubrid utility syntax shows how to control CUBRID broker process. One of the following can be specified in <command>: start is used to run services; stop is used to stop services; restart is used to restart services; status is used to check status; acl is used to limit broker access; on/off is used to enable/disable the specified broker; reset is used to reset the

connection to broker; info is used to display the broker configuration information; getid is used to get the SHARD ID(SHARD database ID) with SHARD key.

And more, SHARD feature can be used only after the broker is started and "SHARD", the broker parameter, whose value in cubrid_broker.conf is set to ON.

```
cubrid broker <command>
<command>: start
           |stop
           |restart
           |status [options] [broker_name_expr]
           |acl {status|reload} broker_name
           |on <broker_name> |off <broker_name>
           |reset broker_name
           |info
           |getid -b <broker_name> [-f] shard_key
```

Controlling Gateway

The following **cubrid** utility syntax shows how to control CUBRID gateway process.

```
cubrid gateway <command>
<command>: start
           |stop
           |restart
           |status [options] [gateway_name_expr]
           |acl {status|reload} gateway_name
           |on <gateway_name> |off <gateway_name>
           |reset gateway_name
           |info
```

- start: start gateway processes.
- stop: stop gateway processes.
- restart: restart gateway processes.
- status: check status of gateway processes.
- acl: limit gateway access.
- on/off: enable/disable the specified gateway.
- reset: reset the connection to gateway.
- info: display the gateway configuration information.

Controlling CUBRID Manager Server

To use the CUBRID Manager, the Manager server must be running where database server is running. The following **cubrid** utility syntax shows how to control the CUBRID Manager processes.

```
cubrid manager <command>
<command>: {start|stop|status}
```

- start: start manager server processes.
- stop: stop manager server processes.
- status: check the status of manager processes.

Controlling CUBRID HA

The following **cubrid heartbeat** utility syntax shows how to use CUBRID HA. One of the following can be specified in *command*.

```
cubrid heartbeat <command>
<command>: {start|stop|copylogdb|applylogdb|reload|status}
```

- start: start HA-related processes.
- stop: stop HA-related processes.
- copylogdb: start or stop copylogdb process.
- applylogdb: start or stop applylogdb process.
- reload: reload information on HA configuration.
- status: check HA status.

For details, see cubrid heartbeat Utility.

Controlling CUBRID Procedural Langauge (PL) Server

The following **cubrid** utility syntax is used to control the CUBRID Procedural Langauge (PL) server process.

```
cubrid pl <command> [database_name]
<command>: {restart|status}
```

- restart: restart the procedural langauge server process.

- status: check the status of the procedural langauge server process.

Note

- Every command can specify a database name (**[database_name]**) as an argument.
- The **restart** command forcibly stops the procedural langauge server, and the database server detects this and automatically restarts the procedural langauge server.
- If the database name is not specified when executing the **status** command, it displays the status information of the procedural langauge server for all running databases.

```
% cubrid pl restart demodb

@ cubrid pl start: demodb
++ cubrid pl start: success

% cubrid pl status demodb

@ cubrid pl status
Procedural Langauge Server (demobdb, pid 12345, UDS)
VM arguments:
------------------------------------------------
...
------------------------------------------------
```

Note

From CUBRID version 11.4, the CUBRID Procedural Langauge (PL) server has replaced the previously used CUBRID Java Stored Procedure (Java SP) server.

CUBRID Services

Registering Services

You can register database servers, CUBRID brokers, CUBRID gateways, CUBRID Manager(s) or CUBRID HA as CUBRID service in the configuration file (**cubrid.conf**). To register services, you can input for each **server**, **broker**, **gateway**, **manager** or **heartbeat** as a parameter value, and it is possible to input several values by concatenating them in comma(,).

If you do not register any service, only master process is registered by default. It is convenient for you to view status of all related processes at a glance or start and stop the processes at once with the **cubrid service** utility once it is registered as CUBRID service.

- For details on CUBRID HA configuration, see Registering HA to cubrid service.

The following example shows how to register database server and broker, gateway as service in the **cubrid.conf** file and enable databases (*demodb* and *testdb*) to start automatically at once when CUBRID server starts running.

```
# cubrid.conf
...

[service]

# The list of processes to be started automatically by 'cubrid
service start' command
# Any combinations are available with server, broker, gateway,
manager and heartbeat.
service=server,broker,gateway

# The list of database servers in all by 'cubrid service start'
command.
# This property is effective only when the above 'service' property
contains 'server' keyword.
server=demodb,testdb
```

Starting Services

In Linux environment, you can enter the code below to start CUBRID after installation. If no server is registered in the configuration file, only master process (cub_master) runs by default.

In the Windows environment, the code below is normally executed only if a user with system permission has logged in. An administrator or general user can start or stop the CUBRID server by clicking its icon on the taskbar tray.

```
% cubrid service start

@ cubrid master start
++ cubrid master start: success
```

The following message is returned if master process is already running.

```
% cubrid service start
```

```
@ cubrid master start
++ cubrid master is running.
```

The following message is returned if master process fails to run. The example shows that service fails to start due to conflicts of the **cubrid_port_id** parameter value specified in the cubrid.conf file. In such a case, you can resolve the problem by changing the port. If it fails to start even though no port is occupied by process, delete /tmp/CUBRID1523 file and then restart the process.

```
% cubrid service start

@ cubrid master start
cub_master: '/tmp/CUBRID1523' file for UNIX domain socket exist....
Operation not permitted
++ cubrid master start: fail
```

After registering service as explained in CUBRID Services, enter the code below to start the service. You can verify that database server process and broker, gateway as well as registered *demodb* and *testdb* are starting at once.

```
% cubrid service start

@ cubrid master start
++ cubrid master start: success
@ cubrid server start: demodb

This may take a long time depending on the amount of restore works
to do.
CUBRID 11.4

++ cubrid server start: success
@ cubrid server start: testdb

This may take a long time depending on the amount of recovery works
to do.
CUBRID 11.4

++ cubrid server start: success
@ cubrid broker start
++ cubrid broker start: success
@ cubrid gateway start
++ cubrid gateway start: success
```

Stopping Services

Enter code below to stop CUBRID service. If no services are registered by a user, only master process stops and then restarts.

```
% cubrid service stop
@ cubrid master stop
++ cubrid master stop: success
```

Enter code below to stop registered CUBRID service. You can verify that server process, broker process, gateway process and master process as well as *demodb* and *testdb* stop at once.

```
% cubrid service stop
@ cubrid server stop: demodb

Server demodb notified of shutdown.
This may take several minutes. Please wait.
++ cubrid server stop: success
@ cubrid server stop: testdb
Server testdb notified of shutdown.
This may take several minutes. Please wait.
++ cubrid server stop: success
@ cubrid broker stop
++ cubrid broker stop: success
@ cubrid gateway stop
++ cubrid gateway stop: success
@ cubrid master stop
++ cubrid master stop: success
@ cubrid master stop
++ cubrid master stop: success
```

Restarting Services

Enter code below to restart CUBRID service. If no services are registered by a user, only master process stops and then restarts.

```
% cubrid service restart

@ cubrid master stop
++ cubrid master stop: success
@ cubrid master start
++ cubrid master start: success
```

Enter code below to restart registered CUBRID service. You can verify that server process, broker process, gateway process and master process as well as *demodb* and *testdb* stop and then restart at once.

```
% cubrid service restart

@ cubrid server stop: demodb
```

```
Server demodb notified of shutdown.
This may take several minutes. Please wait.
++ cubrid server stop: success
@ cubrid server stop: testdb
Server testdb notified of shutdown.
This may take several minutes. Please wait.
++ cubrid server stop: success
@ cubrid broker stop
++ cubrid broker stop: success
@ cubrid gateway stop
++ cubrid gateway stop: success
@ cubrid master stop
++ cubrid master stop: success
@ cubrid master start
++ cubrid master start: success
@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
@ cubrid server start: testdb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
@ cubrid broker start
++ cubrid broker start: success
@ cubrid gateway start
++ cubrid gateway start: success
```

Managing Service Status

The following example shows how to check the status of master process and database server registered.

```
% cubrid service status

@ cubrid master status
++ cubrid master is running.
@ cubrid server status

Server testdb (rel 11.4, pid 31059)
Server demodb (rel 11.4, pid 30950)
```

```
@ cubrid broker status
NAME                     PID  PORT    AS   JQ
TPS                    QPS   SELECT   INSERT   UPDATE   DELETE
OTHERS     LONG-T     LONG-Q         ERR-Q  UNIQUE-ERR-Q  #CONNECT
#REJECT
================================================================================
================================================================================
============================================================
* query_editor          10877 30000     5    0
0                    0        0        0        0        0
0     0/60.0     0/60.0             0             0
0         0
* broker1               10889 33000     5    0
0                    0        0        0        0        0
0     0/60.0     0/60.0             0             0
0         0
@ cubrid gateway status
NAME                     PID  PORT    AS   JQ
TPS                    QPS   SELECT   INSERT   UPDATE   DELETE
OTHERS     LONG-T     LONG-Q         ERR-Q  UNIQUE-ERR-Q  #CONNECT
#REJECT
================================================================================
================================================================================
============================================================
* oracle_gateway        10903 53000     5    0
0                    0        0        0        0        0
0     0/60.0     0/60.0             0             0
0         0

@ cubrid manager server status
++ cubrid manager server is not running.
```

The following message is returned if master process has stopped.

```
% cubrid service status
@ cubrid master status
++ cubrid master is not running.
```

cubrid Utility Logging

CUBRID supports a logging feature about cubrid utility's running result.

Logging contents

The following contents are written to the **$CUBRID/log/cubrid_utility.log** file.

- All commands through cubrid utilities: only usage, version and parsing errors are not logged.

- Execution results by cubrid utilities: success/failure.
- An error message when failure.

Log file size

A size of **cubrid_utility.log** file is expanded by the size specified by **error_log_size** parameter in **cubrid.conf**; if this size is enlarged as the specified size, it is backed up as the **cubrid_utility.log.bak** file.

Log format

```
<time> (cubrid PID) <contents>
```

The following is an example of printing the log file.

```
13-11-19 15:27:19.426 (17724) cubrid manager stop
13-11-19 15:27:19.430 (17724) FAILURE: ++ cubrid manager server is
not running.
13-11-19 15:27:19.434 (17726) cubrid service start
13-11-19 15:27:19.439 (17726) FAILURE: ++ cubrid master is running.
13-11-19 15:27:22.931 (17726) SUCCESS
13-11-19 15:27:22.936 (17756) cubrid service restart
13-11-19 15:27:31.667 (17756) SUCCESS
13-11-19 15:27:31.671 (17868) cubrid service stop
13-11-19 15:27:34.909 (17868) SUCCESS
```

However, in Windows, some **cubrid** commands are executed through a service process; therefore, a duplicated information can be displayed again.

```
13-11-13 17:17:47.638 ( 3820) cubrid service stop
13-11-13 17:17:47.704 ( 7848) d:\CUBRID\bin\cubrid.exe service stop
--for-windows-service
13-11-13 17:17:56.027 ( 7848) SUCCESS
13-11-13 17:17:57.136 ( 3820) SUCCESS
```

And, in Windows, a process run through the service process cannot print out an error message; therefore, for error messages related to the service start, you should definitely check them in the **cubrid_utility.log** file.

Database Server

Starting Database Server

The following example shows how to run *demodb* server.

```
% cubrid server start demodb

@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
```

If you start *demodb* server while master process has stopped, master process automatically runs at first and then a specified database server runs.

```
% cubrid server start demodb

@ cubrid master start
++ cubrid master start: success
@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
```

The following message is returned while *demodb* server is running.

```
% cubrid server start demodb

@ cubrid server start: demodb
++ cubrid server 'demodb' is running.
```

cubrid server start runs cub_server process of a specific database regardless of HA mode configuration. To run database in HA environment, you should use **cubrid heartbeat start**.

Stopping Database Server

The following example shows how to stop *demodb* server.

```
% cubrid server stop demodb

@ cubrid server stop: demodb
Server demodb notified of shutdown.
This may take several minutes. Please wait.
++ cubrid server stop: success
```

The following message is returned while *demodb* server has stopped.

```
% cubrid server stop demodb

@ cubrid server stop: demodb
++ cubrid server 'demodb' is not running.
```

cubrid server stop stops cub_server process of a specific database regardless of HA mode configuration. Be careful not to restart the database server or occur failover. To stop database in HA environment, you should use **cubrid heartbeat stop** .

Restarting Database Server

The following example shows how to restart *demodb* server. *demodb* server that has already run stops and the server restarts.

```
% cubrid server restart demodb

@ cubrid server stop: demodb
Server demodb notified of shutdown.
This may take several minutes. Please wait.
++ cubrid server stop: success
@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
```

Checking Database Server Status

The following example shows how to check the status of a database server. Names of currently running database servers are displayed.

```
% cubrid server status

@ cubrid server status
Server testdb (rel 11.4, pid 24465)
Server demodb (rel 11.4, pid 24342)
```

The following example shows the message when master process has stopped.

```
% cubrid server status

@ cubrid server status
++ cubrid master is not running.
```

Limiting Database Server Access

To limit brokers and the CSQL Interpreter connecting to the database server, configure the parameter value of **access_ip_control** in the **cubrid.conf** file to yes and enter the path of a file in which the list of IP addresses allowed to access the **access_ip_control_file** parameter value is written. You should enter the absolute file path. If you enter the relative path, the system will search the file under the **$CUBRID/conf** directory on Linux and under the **%CUBRID%\conf** directory on Windows.

The following example shows how to configure the **cubrid.conf** file.

```
# cubrid.conf
access_ip_control=yes
access_ip_control_file="/home1/cubrid1/CUBRID/db.access"
```

The following example shows the format of the **access_ip_control_file** file.

```
[@<db_name>]
<ip_addr>
...
```

- <db_name>: The name of a database in which access is allowed

- <ip_addr>: The IP address allowed to access a database. Using an asterisk (*) at the last digit means that all IP addresses are allowed. Several lines of <ip_addr> can be added in the next line of the name of a database.

To configure several databases, it is possible to specify additional [@<db_name>] and <ip_addr>.

Accessing any IP address except localhost is blocked by server if **access_ip_control** is set to yes but **ip_control_file** is not configured. A server will not run if analyzing **access_ip_control_file** fails caused by incorrect format.

The following example shows **access_ip_control_file**.

```
[@dbname1]
192.10.20.30
192.156.*

[@dbname2]
*

[@dbname3]
192.168.1.15
```

The example above shows the *dbname1* database allows the access connections from the IP address 192.10.20.30 or from any IP address that starts with 192.156. *dbname2* database allows the access of every IP address; *dbname3* database allows the access of an IP address, 192.168.1.15, only.

For the database which has already been running, you can modify a configuration file or you can check the currently applied status by using the following commands.

To change the contents of **access_ip_control_file** and apply it to server, use the following command.

```
cubrid server acl reload <database_name>
```

To display the IP configuration of a server which is currently running, use the following command.

```
cubrid server acl status <database_name>
```

Database Server Log

Error Log

The following log is created in the file of a server error log if an IP address that is not allowed to access is used.

```
Time: 10/29/10 17:32:42.360 - ERROR *** ERROR CODE = -1022, Tran =
0, CLIENT = (unknown):(unknown)(-1), EID = 2
Address(10.24.18.66) is not authorized.
```

An error log of the database server is saved into **$CUBRID/log/server** directory, and the format of the filename is *<db_name>_<yyyymmdd>_<hhmi>.err*. The extension is ".err".

```
demodb_20130618_1655.err
```

Note

For details on how to limit an access to the broker server, see Limiting Broker Access.

Event Log

If an event which affects on the query performance occurs, this is saved into the event log.

The events which are saved on the event log are *SLOW_QUERY*, *MANY_IOREADS*, *LOCK_TIMEOUT*, *DEADLOCK* and *TEMP_VOLUME_EXPAND*.

This log file is saved into the **$CUBRID/log/server** directory, and the format of the filename is *<db_name>_<yyyymmdd>_<hhmi>.event*. The extension is ".event".

```
demodb_20130618_1655.event
```

SLOW_QUERY

If a slow query occurs, this event is written. If **sql_trace_slow** parameter value of cubrid.conf is set, this event will arise. The output example is as follows.

```
06/12/13 16:41:05.558 - SLOW_QUERY
  client: PUBLIC@testhost|csql(13173)
  sql: update [y] [y] set [y].[a]= ?:1  where [y].[a]= ?:0  using
```

```
index [y].[pk_y_a](+)
  bind: 5
  bind: 200
  time: 1015
  buffer: fetch=48, ioread=2, iowrite=0
  wait: cs=1, lock=1010, latch=0
```

- client: <DB user>@<application client hostname>|<program name>(<process ID>)
- sql: slow query
- bind: binding value. it is printed out as the number of <num> in the sql item, "?:<num>". The value of "?:0" is 5, and the value of "?:1" is 200.
- time: execution time(ms)
- buffer: execution statistics in the buffer
  - fetch: fetching pages count
  - ioread: I/O read pages count
  - iowrite: I/O write pages count
- wait: waiting time
  - cs: waiting time on the critical section(ms)
  - lock: waiting time to acquire the lock(ms)
  - latch: waiting time to acquire the latch(ms)

On the above example, the query execution time was 1015ms, and lock waiting time was 1010ms, so we can indicate that almost all execution time was from lock waiting.

MANY_IOREADS

Queries which brought many I/O reads are written on the event log. If I/O reads occurs more than **sql_trace_ioread_pages** parameter value of cubrid.conf, the event is written on the event log. The following is an output example.

```
06/12/13 17:07:29.457 - MANY_IOREADS
  client: PUBLIC@testhost|csql(12852)
  sql: update [x] [x] set [x].[a]= ?:1  where ([x].[a]> ?:0 ) using
index [x].[idx](+)
  bind: 8
  bind: 100
```

```
  time: 528
  ioreads: 15648
```

- client: <DB user>@<application client hostname>|<process name>(<process ID>)
- sql: an SQL which brought many I/O reads
- bind: binding value. it is printed out as the number of <num> in the sql item, "?:<num>". The value of "?:0" is 8, and the value of "?:1" is 100.
- time: execution time(ms)
- ioread: I/O read pages count

LOCK_TIMEOUT

When lock timeout occurs, queries of a waiter and a blocker are written on the event log. The following is an output example.

```
02/02/16 20:56:18.650 - LOCK_TIMEOUT
waiter:
  client: public@testhost|csql(21529)
  lock:    X_LOCK (oid=0|650|3, table=t)
  sql: update [t] [t] set [t].[a]= ?:0  where [t].[a]= ?:1
  bind: 2
  bind: 1

blocker:
  client: public@testhost|csql(21541)
  lock:    X_LOCK (oid=0|650|3, table=t)
  sql: update [t] [t] set [t].[a]= ?:0  where [t].[a]= ?:1
  bind: 3
  bind: 1
```

- waiter: a waiting client to acquire locks.
  - lock: lock type, table and index names
  - sql: a waiting SQL to acquire locks.
  - bind: binding value.
- blocker: a client to have locks.
  - lock: lock type, table and index names
  - sql: a SQL which is acquiring locks
  - bind: binding value

On the above, you can indicate the blocker which brought lock timeout and the waiter which is waiting locks.

DEADLOCK

When a deadlock occurs, lock information of that transaction is written into the event log. The following is an output example.

```
02/02/16 20:56:17.638 - DEADLOCK
hold:
    client: public@testhost|csql(21541) (Deadlock Victim)
    lock: X_LOCK (oid=0|650|5, table=t)
    sql: update [t] [t] set [t].[a]= ?:0  where [t].[a]= ?:1
    bind: 3
    bind: 1

wait:
    client: public@testhost|csql(21529)
    lock: X_LOCK (oid=0|650|5, table=t)
    sql: update [t] [t] set [t].[a]= ?:0  where [t].[a]= ?:1
    bind: 4
    bind: 1

hold:
    client: public@testhost|csql(21529)
    lock: X_LOCK (oid=0|650|6, table=t)
    sql: update [t] [t] set [t].[a]= ?:0  where [t].[a]= ?:1
    bind: 5
    bind: 2

wait:
    client: public@testhost|csql(21541) (Deadlock Victim)
    lock: X_LOCK (oid=0|650|6, table=t)
    sql: update [t] [t] set [t].[a]= ?:0  where [t].[a]= ?:1
    bind: 6
    bind: 2
```

- hold: an object which is acquiring a lock
  - client: <DB user>@<application client hostname>|<process name>(<process ID>)
  - lock: lock type, table name
  - sql: SQL which is acquiring locks
  - bind: binding value
- wait: an object which is waiting a lock
  - client: <DB user>@<application client hostname>|<process name>(<process ID>)

- lock: lock type, table name
- sql: SQL which is waiting a lock
- bind: binding value

On the above output, you can check the application clients and SQLs which brought the deadlock.

For more details on locks, see lockdb and Lock Protocol.

TEMP_VOLUME_EXPAND

When a temporary volumes are expanded, this time is written to the event log. By this log, you can check what transaction brought the expansion of a temporary volumes.

```
06/15/13 18:55:43.458 - TEMP_VOLUME_EXPAND
  client: public@testhost|csql(17540)
  sql: select [x].[a], [x].[b] from [x] [x] where (([x].[a]< ?:0 ))
group by [x].[b] order by 1
  bind: 1000
  time: 44
  pages: 24399
```

- client: <DB user>@<application client hostname>|<process name>(<process ID>)
- sql: SQL which requires a more space for temporary data. All INSERT statement except for INSERT ... SELECT syntax, and DDL statement are not delivered to the DB server, so it is shown as EMPTY SELECT, UPDATE and DELETE statements are shown on this item
- bind: binding value
- time: the required time to create a temporary volume(ms)
- pages: the number of available pages within new temporary volume.

Database Server Errors

Database server error processes use the server error code when an error has occurred. A server error can occur in any task that uses server processes. For example, server errors may occur while using the query handling program or the **cubrid** utility.

Checking the Database Server Error Codes

- Every data definition statement starting with **#define ER_** in the **$CUBRID/include/error_code.h** file indicate the server error codes.

- All message groups under "$set 5 MSGCAT_SET_ERROR" in the **$CUBRID/msg/en_US (in Korean, ko_KR.eucKR** or **ko_KR.utf8)/cubrid.msg** $ file indicates the server error messages.

When you write a C code with CCI driver, we recommend you to write a code with an error code name than with an error code number. For example, the error code number for violating the unique key is -670 or -886, but users can easily recognize the error when it is written as **ER_BTREE_UNIQUE_FAILED** or **ER_UNIQUE_VIOLATION_WITHKEY**.

However, when you write a JAVA code with JDBC driver, you have to use error code numbers because "dbi.h" file cannot be included into the JAVA code. For JDBC program, you can get an error number by using getErrorCode() method of SQLException class.

```
$ vi $CUBRID/include/error_code.h

#define NO_ERROR                                  0
#define ER_FAILED                                -1
#define ER_GENERIC_ERROR                         -1
#define ER_OUT_OF_VIRTUAL_MEMORY                 -2
#define ER_INVALID_ENV                           -3
#define ER_INTERRUPTED                           -4
...
#define ER_LK_OBJECT_TIMEOUT_SIMPLE_MSG         -73
#define ER_LK_OBJECT_TIMEOUT_CLASS_MSG           -74
#define ER_LK_OBJECT_TIMEOUT_CLASSOF_MSG         -75
#define ER_LK_PAGE_TIMEOUT                       -76
...
#define ER_PT_SYNTAX                            -493
...
#define ER_BTREE_UNIQUE_FAILED                  -670
...
#define ER_UNIQUE_VIOLATION_WITHKEY             -886
...
#define ER_LK_OBJECT_DL_TIMEOUT_SIMPLE_MSG      -966
#define ER_LK_OBJECT_DL_TIMEOUT_CLASS_MSG       -967
#define ER_LK_OBJECT_DL_TIMEOUT_CLASSOF_MSG     -968
...
```

The following are some of the server error code names, error code numbers, and error messages.

Error Code Name	Error Code Number	Error Message
ER_LK_OBJECT_TIMEOUT_SIMPLE_MSG	-73	Your transaction (index ?, ? @?|?) timed out waiting on ? lock on object ?|?|?.

Error Code Name	Error Code Number	Error Message
		You are waiting for user(s) ? to finish.
ER_LK_OBJECT_TIMEOUT_CLASS_MSG	-74	Your transaction (index ?, ? @?|?) timed out waiting on ? lock on class ?. You are waiting for user(s) ? to finish.
ER_LK_OBJECT_TIMEOUT_CLASSOF_MSG	-75	Your transaction (index ?, ? @?|?) timed out waiting on ? lock on instance ?|?|? of class ?. You are waiting for user(s) ? to finish.
ER_LK_PAGE_TIMEOUT	-76	Your transaction (index ?, ? @?|?) timed out waiting on ? on page ?|?. You are waiting for user(s) ? to release the page lock.
ER_PT_SYNTAX	-493	Syntax: ?
ER_BTREE_UNIQUE_FAILED	-670	Operation would have caused one or more unique constraint violations.
ER_UNIQUE_VIOLATION_WITHKEY	-886	“?” caused unique constraint violation.

Error Code Name	Error Code Number	Error Message
ER_LK_OBJECT_DL_TIMEOUT_SIMPLE_MSG	-966	Your transaction (index ?, ?@?|?) timed out waiting on ? lock on object ?|?|? because of deadlock. You are waiting for user(s) ? to finish.
ER_LK_OBJECT_DL_TIMEOUT_CLASS_MSG	-967	Your transaction (index ?, ?@?|?) timed out waiting on ? lock on class ? because of deadlock. You are waiting for user(s) ? to finish.
ER_LK_OBJECT_DL_TIMEOUT_CLASSOF_MSG	-968	Your transaction (index ?, ?@?|?) timed out waiting on ? lock on instance ?|?|? of class ? because of deadlock. You are waiting for user(s) ? to

Broker

Starting Broker

Enter the command below to start the broker. The SHARD function is activated when SHARD, which is the broker parameter of cubrid_broker.conf, is set to ON.

```
$ cubrid broker start
@ cubrid broker start
++ cubrid broker start: success
```

The following message is returned if the broker is already running.

```
$ cubrid broker start
@ cubrid broker start
++ cubrid broker is running.
```

⚠ Warning

The number of required file descriptor(fd) when starting SHARD in Linux system will be a little bit more than SHARD_MAX_CLIENTS in cubrid_broker.conf. Therefore, when you limit the number of fd by using "ulimit -n", it should be a little bit greater than the value of SHARD_MAX_CLIENTS. When the limited number of fd in Linux system is smaller than the fd number which is required in SHARD, starting SHARD fails and the required fd number is displayed in the error message.

Stopping Broker

Enter the command below to stop the broker. If SHARD, the broker parameter of the cubrid_broker.conf, is set to ON, the SHARD function is stopped.

```
$ cubrid broker stop
@ cubrid broker stop
++ cubrid broker stop: success
```

The following message is returned if the broker has stopped.

```
$ cubrid broker stop
@ cubrid broker stop
++ cubrid broker is not running.
```

Restarting Broker

Enter the command below to restart the whole brokers.

```
$ cubrid broker restart
```

Checking Broker Status

The **cubrid broker status** utility allows you to check the broker status such as number of completed jobs and the number of standby jobs by providing various options. The status of

clients accessed SHARD or the status of SHARD can be displayed by using **-c** and **-m** options when the **SHARD** broker parameter in **cubrid_broker.conf** is set to ON. Also, by using **-S** option or **-P** option, each shard DB or each proxy information can be displayed.

```
cubrid broker status [options] [expr]
```

- *expr*: A part of the broker name or "SERVICE=ON|OFF"

Specifying *expr* performs that the status of specific brokers which include *expr* in their names is monitored; specifying no argument means that status of all brokers which are registered in the broker environment configuration file (**cubrid_broker.conf**) is monitored.

If "SERVICE=ON" is specified on *expr*, only the status of working brokers is displayed; if "SERVICE=OFF" is specified, only the status of stopped brokers is displayed.

The following [options] are available with the **cubrid broker status** utility. -b, -q, -c, -m, -S, -P and -f are options to define the information to print; -s, -l and -t are options to control printing; -c, -m, -S and -P are options applied when using SHARD feature. All of these are possible to use as combining each other.

-b

Displays the status information of a broker but does not display information on broker application server.

-q

Displays standby jobs in the job queue.

-c

Displays the information of clients which access the proxy when **SHARD** in cubrid_broker.conf is set to ON.

-m

Displays the SHARD status and the statistical information when **SHARD** in cubrid_broker.conf is set to ON.

-S

For each shard DB, it displays **-b** option's items except NAME, PID, PORT and JQ and #CONNECT; in addition, it displays ID, SHARD-Q and #REQUEST.

-P

For each proxy, it displays **-b** option's items except NAME, PID, PORT and JQ; in addition, it displays ID, SHARD-Q and #RESTART.

-f

Displays information of DB and host accessed by broker.

If it is used with the **-b** option, additional information on CAS is displayed. But SELECT, INSERT, UPDATE, DELETE, OTHERS items which shown on **-b** option are excluded.

If it is used with the **-m** option, more detailed SHARD statistics are output.

If it is used with the **-c** option, each shard proxy additionally outputs CLIENT-ID, CLIENT-IP, CONN-TIME, LAST-REQ-TIME, LAST-RES-TIME, and LAST-REQ-CODE items.

If it is used with the **-P** option, STMT-POOL-RATIO is additionally printed. This item shows the ratio to use statements in the pool when you are using prepared statements.

-l SECOND

The **-l** option is only used with -f option together. It specifies accumulation period (unit: sec.) when displaying the number of application servers whose client status is Waiting or Busy. If it is omitted, the default value (1 second) is specified.

-t

Displays results in tty mode on the screen. The output can be redirected and used as a file.

-s SECOND

Regularly displays the status of broker based on specified period. It returns to a command prompt if q is entered.

If you do not specify options or arguments, the status of all brokers is displayed.

```
$ cubrid broker status
@ cubrid broker status
% query_editor
----------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
----------------------------------------
 1 28434     0     0 50144 IDLE
 2 28435     0     0 50144 BUSY
 3 28436     0     0 50144 CLIENT_WAIT
 4 28437     0     0 50140 CLIENT_WAIT
 5 28438     0     0 50144 CLOSE_WAIT

% broker1 OFF
```

- % query_editor: The broker name
- ID: Serial number of CAS within the broker
- PID: CAS process ID within the broker
- QPS: The number of queries processed per second
- LQS: The number of long-duration queries processed per second

- PSIZE: Size of CAS
- STATUS: The current status of CAS (BUSY, IDLE, CLIENT_WAIT, CLOSE_WAIT)
  - IDLE: The state in which the database is not connected and no operations are being performed.
  - BUSY: The state in which an SQL statement is being executed or a task requested by the application is being processed.
  - CLIENT_WAIT: The requested operation has been completed, but the transaction has not yet ended.
  - CLOSE_WAIT: The transaction has been completed, but the connection remains open.
- % broker1 OFF: broker1's SERVICE parameter is set to OFF. So, broker1 is not started.

Note

ID column which is displayed when checking the SHARD status shows the composition of " (serial number of proxy) - (serial number of shard DB) - (serial number of CAS accessing shard DB)"

```
$ cubrid broker status
@ cubrid broker status
% shard1
----------------------------------------------------------------
    ID      PID     QPS    LQS PSIZE STATUS
----------------------------------------------------------------
    1-1-1  2580     100      3 55968 IDLE
    1-2-1  2581     200      4 55968 IDLE
```

The following shows the detail status of broker for 5 seconds. The display will reset per 5 seconds as the new status information. To escape the display of the status, press <Q>.

```
$ cubrid broker status -b -s 5
@ cubrid broker status

 NAME                     PID  PORT   AS   JQ    TPS    QPS
SELECT   INSERT   UPDATE   DELETE   OTHERS     LONG-T     LONG-Q
ERR-Q  UNIQUE-ERR-Q  #CONNECT  #REJECT
================================================================
================================================================
============================
* query_editor         13200 30000    5    0      0      0
```

```
0         0         0         0         0     0/60.0     0/60.0
0              0          0         0
* broker1               13269 33000    5     0     70      60
10        20        10        10        10     0/60.0     0/60.0
30             10        213         1
```

- NAME: The broker name
- PID: Process ID of the broker
- PORT: Port number of the broker
- AS: The number of CAS
- JQ: The number of standby jobs in the job queue
- TPS: The number of transactions processed per second (calculated only when the option is configured to “-b -s <sec>”)
- QPS: The number of queries processed per second (calculated only when the option is configured to “-b -s <sec>”)
- SELECT: The number of SELECT queries after staring of the broker. When there is an option of “-b -s <sec>”, it is updated every time with the number of SELECTs which have been executed during the seconds specified by this option.
- INSERT: The number of INSERT queries after staring of the broker. When there is an option of “-b -s <sec>”, it is updated every time with the number of INSERTs which have been executed during the seconds specified by this option.
- UPDATE: The number of UPDATE queries after staring of the broker. When there is an option of “-b -s <sec>”, it is updated every time with the number of UPDATEs which have been executed during the seconds specified by this option.
- DELETE: The number of DELETE queries after staring of the broker. When there is an option of “-b -s <sec>”, it is updated every time with the number of DELETEs which have been executed during the seconds specified by this option.
- OTHERS: The number of queries like CREATE and DROP except for SELECT, INSERT, UPDATE, DELETE. When there is an option of “-b -s <sec>”, it is updated every time with the number of queries which have been executed during the seconds specified by this option.
- LONG-T: The number of transactions which exceed LONG_TRANSACTION_TIME. / the value of the LONG_TRANSACTION_TIME parameter. When there is an option of “-b -s <sec>”, it is updated every time with the number of transactions which have been executed during the seconds specified by this option.

- LONG-Q: The number of queries which exceed LONG_QUERY_TIME. / the value of the LONG_QUERY_TIME parameter. When there is an option of "-b -s <sec>", it is updated every time with the number of queries which have been executed during the seconds specified by this option.
- ERR-Q: The number of queries with errors found. When there is an option of "-b -s <sec>", it is updated every time with the number of errors which have occurred during the seconds specified by this option.
- UNIQUE-ERR-Q: The number of queries with unique key errors found. When there is an option of "-b -s <sec>", it is updated every time with the number of unique key errors which have occurred during the seconds specified by this option.
- #CONNECT: The number of connections that an application client accesses to CAS after starting the broker.
- #REJECT: The count that an application client excluded from ACL IP list is rejected to access a CAS. Regarding ACL setting, see Limiting Broker Access.

The following checks the status of broker whose name includes broker1 and job status of a specific broker in the job queue with the **-q** option. If you do not specify broker1 as an argument, list of jobs in the job queue for all brokers is displayed.

```
% cubrid broker status -q broker1
@ cubrid broker status
% broker1
----------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
----------------------------------------
 1 28444     0     0 50144 IDLE
 2 28445     0     0 50140 IDLE
 3 28446     0     0 50144 IDLE
 4 28447     0     0 50144 IDLE
 5 28448     0     0 50144 IDLE
```

The following monitors the status of a broker whose name includes broker1 with the **-s** option. If you do not specify broker1 as an argument, monitoring status for all brokers is performed regularly. It returns to a command prompt if q is not entered.

```
% cubrid broker status -s 5 broker1
% broker1
----------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
----------------------------------------
 1 28444     0     0 50144 IDLE
 2 28445     0     0 50140 IDLE
 3 28446     0     0 50144 IDLE
```

```
 4 28447     0     0 50144 IDLE
 5 28448     0     0 50144 IDLE
```

With the **-t** option, it display information of TPS and QPS to a file. To cancel displaying, press <Ctrl+C> to stop program.

```
% cubrid broker status -b -t -s 1 > log_file
```

The following views information of server/database accessed by broker, the last access times of applications, the IP addresses accessed to CAS and the versions of drivers etc. with the **-f** option.

```
$ cubrid broker status -f broker1
@ cubrid broker status
% broker1
--------------------------------------------------------------------------
--------------------------------------------------------------------------
------------------------------------------------
ID   PID   QPS   LQS PSIZE STATUS         LAST ACCESS TIME
DB       HOST   LAST CONNECT TIME       CLIENT IP   CLIENT
VERSION    SQL_LOG_MODE   TRANSACTION STIME  #CONNECT  #RESTART
--------------------------------------------------------------------------
--------------------------------------------------------------------------
------------------------------------------------
 1 26946     0     0 51168 IDLE         2011/11/16 16:23:42  demodb
localhost 2011/11/16 16:23:40      10.0.1.101
9.2.0.0062              NONE 2011/11/16 16:23:42         0         0
 2 26947     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                    -
0.0.0.0                                -                 -
0         0
 3 26948     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                    -
0.0.0.0                                -                 -
0         0
 4 26949     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                    -
0.0.0.0                                -                 -
0         0
 5 26950     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                    -
0.0.0.0                                -                 -
0         0
```

Meaning of each column in code above is as follows:

- LAST ACCESS TIME: Time when CAS runs or the latest time when an application client accesses CAS

- DB: Name of a database which CAS accesses most recently
- HOST: Name of a which CAS accesses most recently
- LAST CONNECT TIME: Most recent time when CAS accesses a database
- CLIENT IP: IP of an application clients currently being connected to an application server(CAS). If no application client is connected, 0.0.0.0 is displayed.
- CLIENT VERSION: A driver's version of an application client currently being connected to a CAS
- SQL_LOG_MODE: SQL logging mode of CAS. If the mode is same as the mode configured in the broker, "-" is displayed. For details, see SQL_LOG .
- TRANSACTION STIME: Transaction start time
- #CONNECT: The number of connections that an application client accesses to CAS after starting the broker
- #RESTART: The number of connection that CAS is re-running after starting the broker

Enter the command below with the **-b** and **-f** options to display AS(T W B Ns-W Ns-B) and CANCELED additionally.

```
// The -f option is added upon execution of broker status
information. Configuring Ns-W and Ns-B are displayed as long as N
seconds by using the -l.
% cubrid broker status -b -f -l 2
@ cubrid broker status
NAME          PID    PSIZE PORT  AS(T W B 2s-W 2s-B) JQ TPS QPS LONG-
T LONG-Q  ERR-Q UNIQUE-ERR-Q CANCELED ACCESS_MODE SQL_LOG  #CONNECT
#REJECT
================================================================================
================================================================================
======
query_editor 16784 56700 30000      5 0 0     0   0   0  16  29 0/
60.0 0/60.0      1            1        0          RW     ALL
4       1
```

Meaning of added columns in code above is as follows:

- AS(T): Total number of CAS being executed
- AS(W): The number of CAS in the status of Waiting
- AS(B): The number of CAS in the status of Busy
- AS(Ns-W): The number of CAS that the client belongs to has been waited for N seconds.

- AS(Ns-B): The number of CAS that the client belongs to has been Busy for N seconds.
- CANCELED: The number of queries have cancelled by user interruption since the broker starts (if it is used with the **-l** *N* option, it specifies the number of accumulations for *N* seconds).
- ACCESS_MODE: The mode in which the broker connects to the DB. Broker mode is classified into three types: Read Write, Read Only, and Standby Only. For details, see Broker Mode.
- SQL_LOG: SQL logging mode of CAS. SQL LOG is classified into five types: ALL, OFF, ERROR, NOTICE, and TIMEOUT. For details, see SQL_LOG .

Use the **-m** option to display SHARD status and statistics information. For details on the parameter of **cubrid_broker.conf**, see Broker Configuration.

```
$ cubrid broker status -m
@ cubrid broker status
% shard1
MODULAR : 256, LIBRARY_NAME : NOT DEFINED, FUNCTION_NAME : NOT
DEFINED
ACTIVE-PROXY : 1, NUM-NO-HINT-ERR-Q : 0

SHARD STATISTICS
ID  NUM-KEY-Q   NUM-ID-Q   NUM-NO-HINT-Q             SUM
------------------------------------------------------------
    0       1281          0               0            1281
    1       1281          0               0            1281
    2       1281          0               0            1281
    3       1281          0               0            1281

NUM_SHARD_Q
PROXY_ID      1
SHARD_ID
------------------
0                1
1                0
2                0
3                2
```

The description of each column is as follows.

- shard1: The broker name
- MODULAR: The **SHARD_KEY_MODULR** parameter value of **cubrid_broker.conf**
- LIBRARY_NAME: The **SHARD_KEY_LIBRARY_NAME** parameter value of **cubrid_broker.conf**
- FUNCTION_NAME: The **SHARD_KEY_FUNCTION_NAME** parameter value of **cubrid_broker.conf**
- ACTIVE-PROXY: The number of proxy processes which are running

- NUM-NO-HINT-ERR-Q: The number of errored queries because of no shard hints
- SHARD STATISTICS: The shard ID query information
  - ID: The shard DB serial number (shard ID)
  - NUM-KEY-Q: The number of query requests which include the shard key
  - NUM-ID-Q: The number of query requests which include the shard ID
  - NUM-NO-HINT-Q: The number of requests handled by load balancing without hint when **SHARD_IGNORE_HINT** is configured
  - SUM: NUM-KEY-Q + NUM-ID-Q
- NUM_SHARD_Q: The number of requests to run queries waiting on SHARD-Q
  - PROXY_ID: The proxy serial number
  - SHARD_ID: The shard DB serial number

SHARD-Q is an abbreviation of "Shard Waiting Queue". If proxy process requested to run the query but there was no CAS process to run this, then this request is waiting on SHARD-Q for a while. If the value of SHARD-Q is larger, it means that waiting cases are more. Therefore, you can consider to enlarge the value of MAX_NUM_APPL_SERVER.

Use the **-m -f** option to display more detailed SHARD statistics information. For details on the parameter of **cubrid_broker.conf**, see Broker Configuration.

```
$ cubrid broker status -m -f
@ cubrid broker status
% shard1
MODULAR : 256, LIBRARY_NAME : NOT DEFINED, FUNCTION_NAME : NOT
DEFINED
SHARD : 0 [HostA] [shard1], 1 [HostB] [shard1], 2 [HostC] [shard1],
3 [HostD] [shard1]
ACTIVE-PROXY : 1, NUM-NO-HINT-ERR-Q : 0

SHARD STATISTICS
       ID  NUM-KEY-Q   NUM-ID-Q   NUM-NO-HINT-Q             SUM
    ------------------------------------------------------------
        0       2309          0               0            2309
        1       2309          0               0            2309
        2       2309          0               0            2309
        3       2309          0               0            2309

NUM_SHARD_Q
   PROXY_ID      1
SHARD_ID
-----------------
```

```
0               1
1               0
2               0
3               2

RANGE STATISTICS : user_no
      MIN ~   MAX :      SHARD     NUM-Q
    ------------------------------------
        0 ~    31 :          0      1157
       32 ~    63 :          1      1157
       64 ~    95 :          2      1157
       96 ~   127 :          3      1157
      128 ~   159 :          0      1152
      160 ~   191 :          1      1152
      192 ~   223 :          2      1152
      224 ~   255 :          3      1152

DB Alias : shard1 [USER : shard, PASSWD : shard123]
```

A description of the added column is as follows.

- SHARD: About shard DB within proxy
  - 0: shard DB serial number (shard ID)
  - [HostA]: shard Connection Information
  - [shard1]: Physical DB Name
- RANGE STATISTICS: shard key query information
  - user_no: The shard key name
  - MIN: The minimum range of a shard key
  - MAX: The maximum range of a shard key
  - SHARD: The shard DB serial number (shard ID)
  - NUM-Q: The number of query requests which include the shard key

The below displays the information of clients which access the proxy by using the **-c** option.

```
$ cubrid broker status -c
@ cubrid broker status
% shard1(0), MAX-CLIENT : 50, CUR-CLIENT : 0
% shard1(1), MAX-CLIENT : 50, CUR-CLIENT : 0
```

- MAX-CLIENT: Maximum number of application clients that can access that proxy

- CUR-CLIENT: Number of application clients connected to that proxy

Adding the **-f** option to the **-c** option will output more detailed client information.

```
$ cubrid broker status -c -f
@ cubrid broker status
% shardqa(0), MAX-CLIENT : 50, CUR-CLIENT : 0
------------------------------------------------------------------------
----------------------------------------
 CLIENT-ID           CLIENT-IP             CONN-TIME         LAST-
REQ-TIME         LAST-RES-TIME  LAST-REQ-CODE
------------------------------------------------------------------------
----------------------------------------
         2           127.0.0.1   2014/01/21 18:07:29   2014/01/21
18:07:56   2014/01/21 18:07:56              2
% shardqa(1), MAX-CLIENT : 50, CUR-CLIENT : 0
------------------------------------------------------------------------
----------------------------------------
 CLIENT-ID           CLIENT-IP             CONN-TIME         LAST-
REQ-TIME         LAST-RES-TIME  LAST-REQ-CODE
------------------------------------------------------------------------
----------------------------------------
```

A description of the added column is as follows.

- CLIENT-ID: Client serial numbers assigned sequentially within the proxy
- CLIENT-IP: Client IP address
- CONN-TIME: Time of access to proxy
- LAST-REQ-TIME: Last time a query was requested from proxy
- LAST-RES-TIME: Last time a response was received from proxy
- LAST-REQ-CODE: Code of the last function performed. The functions corresponding to the main code are as follows.
  - 0: end_tran(Termination of transaction)
  - 1: prepare
  - 2: execute
  - 7: fetch

The following display the information for each shard DB with **-S** option.

```
$ cubrid broker status -S
@ cubrid broker status
% shard1
  SHARD_ID    AS SHARD-Q     TPS      QPS   SELECT   INSERT
UPDATE   DELETE   OTHERS     LONG-T     LONG-Q   ERR-Q  UNIQUE-ERR-
Q  #REQUEST
============================================================================
============================================================================
===
         0     2       0    3200     3772      956      960
928      928         0     0/60.0     0/60.0     700
0      6978
         1     2       0    3200     3776      960      960
928      928         0     0/60.0     0/60.0     704
0      6983
         2     2       0    3200     3762      960      960
928      914         0     0/60.0     0/60.0     690
0      6968
         3     2       0    3200     3776      960      960
928      928         0     0/60.0     0/60.0     704
0      6983
```

The following explains the additional columns.

- SHARD_ID: The index of a shard(starting from 0).
- SHARD-Q: The number of queries waiting on SHARD-Q(see SHARD-Q) for each shard.
- #REQUEST: The total number of requests which a CAS belonging to the shard get from application clients(requests includes not only a query-execution request, but also a connection request and etc.)

If **-f** option is added to **-S** option, AS items are divided into (T W B 1s-W 1s-B) and displayed in detail. Regarding AS items, see AS.

The below displays the information for each proxy with **-P** option.

```
$ cubrid broker status -P
% shard1
  PROXY_ID    AS SHARD-Q     TPS      QPS   SELECT   INSERT
UPDATE   DELETE   OTHERS     LONG-T     LONG-Q   ERR-Q  UNIQUE-ERR-
Q  #CONNECT  #REJECT  #RESTART
============================================================================
============================================================================
======================
         1     4       0   22174    26160    26160        0
0        0        0     0/60.0     0/60.0    5256
0       165        0          0
         2     4       0   35257    37903    23599     5152
```

```
4576      4576         0      0/60.0      0/60.0     4300
0        264         1          0
```

The additional displayed items with **-P** option compared to **-b** option are as below.

- PROXY_ID: The index of a proxy(starting from 1)
- SHARD-Q: The number of queries waiting on SHARD-Q(see SHARD-Q) for each proxy.
- #CONNECT: The count that application clients tried accessing a proxy
- #REJECT: The count that an application client excluded from ACL IP list is rejected to access a proxy. Regarding ACL setting, see Limiting Broker Access.
- #RESTART: The count that a proxy is restarted.

If **-f** option is added to **-S** option, AS items are divided into (T W B 1s-W 1s-B) and displayed in detail and STMT-POOL-RATIO item is added. Regarding AS items, see AS.

```
$ cubrid broker status -P -f
% shard1
  PROXY_ID  AS(T      W      B   1s-W  1s-B) SHARD-Q   TPS   QPS
LONG-T   LONG-Q  ERR-Q  UNIQUE-ERR-Q  #CONNECT  #REJECT  #RESTART
STMT-POOL-RATIO (%)
============================================================================
============================================================================
================
         1     4      0      0      0      0       0     0     0
0/60.0   0/60.0      0             0       165        0
0                    -
         2     4      0      0      0      0       0     0     0
0/60.0   0/60.0      0             0       264        1
0                    -
```

A description of the added column is as follows.

- STMT-POOL-RATIO: Percentage of use of statements in the pool when using prepare statements

When the **-b**, **-S** and **-P** options are used at the same time, the output is as follows.

```
$ cubrid broker status -b -S -P

@ cubrid broker status
  NAME          PID  PORT    AS   JQ       TPS      QPS   SELECT
INSERT   UPDATE   DELETE   OTHERS     LONG-T     LONG-Q   ERR-Q
UNIQUE-ERR-Q  #CONNECT #REJECT
============================================================================
============================================================================
=========================
```

```
* shard1      10204 56001     8    0     57431    64063    49759
5152     4576     4576        0     0/60.0     0/60.0
9556             0       429       0
* shard2      10221 56002     8    0     51913    58979
49844        0     4687     4448        0     0/60.0     0/60.0
9862             0       429       2
% broker1 OFF

<SHARD INFO>
% shard1
  SHARD_ID    AS SHARD-Q     TPS       QPS   SELECT   INSERT
UPDATE   DELETE   OTHERS     LONG-T     LONG-Q         ERR-Q  UNIQUE-
ERR-Q    #REQUEST
================================================================================
================================================================================
============
         0     2       0   14464     16165    12613     1312
1120     1120        0     0/60.0     0/60.0
2437             0       30645
         1     2       0   14464     15926    12310     1248
1184     1184        0     0/60.0     0/60.0
2198             0       30403
         2     2       0   14464     16347    12795     1312
1120     1120        0     0/60.0     0/60.0
2619             0       30824
         3     2       0   14039     15625    12041     1280
1152     1152        0     0/60.0     0/60.0
2302             0       29681
% shard2
  SHARD_ID    AS SHARD-Q     TPS       QPS   SELECT   INSERT
UPDATE   DELETE   OTHERS     LONG-T     LONG-Q         ERR-Q  UNIQUE-
ERR-Q    #REQUEST
================================================================================
================================================================================
============
         0     2       0   13085     14884    12580        0
1184     1120        0     0/60.0     0/60.0
2503             0       27985
         1     2       0   13056     14808    12507        0
1181     1120        0     0/60.0     0/60.0
2456             0       27878
         2     2       0   13056     14743    12453        0
1170     1120        0     0/60.0     0/60.0
2391             0       27812
         3     2       0   12716     14544    12304        0
1152     1088        0     0/60.0     0/60.0
2512             0       27273

% broker1 OFF

<PROXY INFO>
% shard1
```

```
  PROXY_ID    AS SHARD-Q      TPS     QPS   SELECT   INSERT
UPDATE   DELETE   OTHERS     LONG-T     LONG-Q  ERR-Q  UNIQUE-ERR-Q
#CONNECT  #REJECT  #RESTART
================================================================================
================================================================================
====================
         1     4       0    22174   26160    26160        0
0        0        0     0/60.0     0/60.0   5256             0
165        0         0
         2     4       0    35257   37903    23599     5152
4576     4576        0     0/60.0     0/60.0   4300
0       264        1         0
% shard2
  PROXY_ID    AS SHARD-Q      TPS     QPS   SELECT   INSERT
UPDATE   DELETE   OTHERS     LONG-T     LONG-Q  ERR-Q  UNIQUE-ERR-Q
#CONNECT  #REJECT  #RESTART
================================================================================
================================================================================
====================
         1     4       0    21590   25586    25586        0
0        0        0     0/60.0     0/60.0   5266             0
165        0         0
         2     4       0    30323   33393    24258        0
4687     4448        0     0/60.0     0/60.0   4596
0       264        1         0
% broker1 OFF
```

Limiting Broker Access

To limit the client applications accessing the broker, set to **ON** for the **ACCESS_ CONTROL** parameter in the **cubrid_broker.conf** file, and enter a name of the file in which the users and the list of databases and IP addresses allowed to access the **ACCESS_CONTROL_FILE** parameter value are written. If a broker name is not listed in the ACCESS_CONTROL_FILE, access to that broker is determined by the **ACCESS_CONTROL_DEFAULT_POLICY** parameter. If this parameter is set to DENY, access is restricted; if set to ALLOW, access is permitted. The default value of the **ACCESS_CONTROL** broker parameter is **OFF**, and the default value of **ACCESS_CONTROL_DEFAULT_POLICY** is **DENY**. The **ACCESS_CONTROL**, **ACCESS_CONTROL_FILE**, and **ACCESS_CONTROL_DEFAULT_POLICY** parameters must be specified under the [broker] section, where common parameters are defined.

The format of **ACCESS_CONTROL_FILE** is as follows:

```
[%<broker_name>]
<db_name>:<db_user>:<ip_list_file>
...
```

- <broker_name>: A broker name. It is the one of broker names specified in **cubrid_broker.conf** .
- <db_name>: A database name. If it is specified as *, all databases are allowed to access the broker server.
- <db_user>: A database user ID. If it is specified as *, all database user IDs are allowed to access the broker server.
- <ip_list_file>: Names of files in which the list of accessible IPs are stored. Several files such as ip_list_file1, ip_list_file2, ... can be specified by using a comma (,).

[%<*broker_name*>] and <*db_name*>:<*db_user*>:<*ip_list_file*> can be specified separately for each broker. A separated line can be specified for the same <*db_name*> and the same <*db_user*>. List of IPs can be written up to the maximum of 256 lines per <*db_name*>:<*db_user*> in a broker.

Note

When using an absolute path name in Windows, the DRIVE name must be specified (for example, C:\CUBRID\CONF\CUBRID_ACL.CONF).

The format of the ip_list_file is as follows:

```
<ip_addr>
...
```

- <ip_addr>: An IP address that is allowed to access the server. If the last digit of the address is specified as *, all IP addresses in that rage are allowed to access the broker server.

Even if the **ACCESS_CONTROL** value is set to ON, access from localhost is always allowed. (In other words, specifying the localhost IP (127.0.0.1) in the ip_list_file is ignored.)

Broker access restrictions not specified in **ACCESS_CONTROL_FILE**.

- Allow access only from localhost. (default)
- If **ACCESS_CONTROL_DEFAULT_POLICY** is set to **ALLOW**, all access is allowed.

If the analysis of **ACCESS_CONTROL_FILE** and ip_list_file fails when starting a broker, the broker will not be run.

```
# cubrid_broker.conf
[broker]
MASTER_SHM_ID           =30001
ADMIN_LOG_FILE          =log/broker/cubrid_broker.log
ACCESS_CONTROL    =ON
ACCESS_CONTROL_FILE     =/home1/cubrid/access_file.txt
[%QUERY_EDITOR]
SERVICE                 =ON
BROKER_PORT             =30000
......
```

The following example shows the content of **ACCESS_CONTROL_FILE**. The * symbol represents everything, and you can use it when you want to specify database names, database user IDs and IPs in the IP list file which are allowed to access the broker server.

```
[%QUERY_EDITOR]
dbname1:dbuser1:READIP.txt
dbname1:dbuser2:WRITEIP1.txt,WRITEIP2.txt
*:dba:READIP.txt
*:dba:WRITEIP1.txt
*:dba:WRITEIP2.txt

[%BROKER2]
dbname:dbuser:iplist2.txt

[%BROKER3]
dbname:dbuser:iplist2.txt

[%BROKER4]
dbname:dbuser:iplist2.txt
```

The brokers specified above are QUERY_EDITOR, BROKER2, BROKER3, and BROKER4.

The QUERY_EDITOR broker only allows the following application access requests.

- When a user logging into *dbname1* with a *dbuser1* account connects from IPs registered in READIP.txt
- When a user logging into *dbname1* with a *dbuser2* account connects from IPs registered in WRITEIP1.txt and WRITEIP2.txt
- When a user logging into every database with a **DBA** account connects from IPs registered in READIP.txt, WRITEIP1.txt, and WRITEIP2.txt

The following example shows how to specify the IPs allowed in ip_list_file.

```
192.168.1.25
192.168.*
192.*
*
```

The descriptions for the IPs specified in the example above are as follows:

- The first line setting allows access from 192.168.1.25.
- The second line setting allows access from all IPs starting with 192.168.
- The third line setting allows access from all IPs starting with 192.
- The fourth line setting allows access from all IPs.

For the broker which has already been running, you can modify the configuration file or check the currently applied status of configuration by using the following commands.

To configure databases, database user IDs and IPs allowed to access the broker and then apply the modified configuration to the server, use the following command.

```
cubrid broker acl reload [<BR_NAME>]
```

- <BR_NAME>: A broker name. If you specify this value, you can apply the changes only to specified brokers. If you omit it, you can apply the changes to all brokers.

Note

Existing connections are not affected by reconfiguration of the access control list using the acl reload command. It means, it is applied only to new connections. To take effect changes on the connected connections, the broker must be restarted.

To display the databases, database user IDs and IPs that are allowed to access the broker in running on the screen, use the following command.

```
cubrid broker acl status [<BR_NAME>]
```

- <BR_NAME>: A broker name. If you specify the value, you can display the specified broker configuration. If you omit it, you can display all broker configurations.

The below is an example of displaying results.

```
$ cubrid broker acl status
ACCESS_CONTROL=ON
ACCESS_CONTROL_FILE=access_file.txt
ACCESS_CONTROL_DEFAULT_POLICY=ALLOW

[%broker1]
demodb:dba:iplist1.txt
       CLIENT IP LAST ACCESS TIME
========================================
  192.20.30.40
  192.168.1.0 2023-11-07 15:19:14
  192.168.2.0
  192.168.3.0
  192.169.* 2023-11-07 15:20:50

testdb:dba:iplist2.txt
       CLIENT IP LAST ACCESS TIME
========================================
               * 2013-11-08 10:10:12

[%broker2]

++ cubrid broker acl: success
```

Broker Logs

If you try to access brokers through IP addresses that are not allowed, the following logs will be created.

- ACCESS_LOG

```
1 192.10.10.10 - - 1288340944.198 1288340944.198 2010/10/29
17:29:04 ~ 2010/10/29 17:29:04 14942 - -1 db1 dba : rejected
```

- SQL LOG

```
10/29 10:28:57.591 (0) CLIENT IP 192.10.10.10 10/29 10:28:
57.592 (0) connect db db1 user dba url jdbc:cubrid:
192.10.10.10:30000:db1::: - rejected
```

Note

For details on how to limit an access to the database server, see Limiting Database Server Access.

Packet Encryption

In an unencrypted communication environment, someone can monitor and interpret all the traffic between clients and a database server, and collected information could be used illegally. In order to access information in an unsafe communication environment while avoiding such an information leakage, data transmitted and received must be encrypted. CUBRID Broker can be configured in safe mode. In this case, all data transmitted and received between the database server and the client are encrypted.

CUBRID supports encrypted connections between clients and the server using TLS (Transport Layer Security) protocol. TLS provides data encryption mechanism as well as detecting data tampering, loss, hence ensures providing secure and trusted communication channel between clients and the server. CUBRID provides these TLS functions using OpenSSL.

CUBRID Broker can be configured for encrypted mode (**SSL = ON**) or non-encrypted mode (**SSL = OFF**) using **SSL** parameter in **cubrid_broker.conf**. A Broker must be restarted when the encryption parameter is changed. When a Broker is configured in encryption mode, clients such as **jdbc client** must connect in encryption mode, otherwise the connection to the broker will be rejected. The opposite is also true. That is, a connection request of clients using encryption mode to non-secure broker will be refused.

When SSL parameter is not specified in cubrid_broker.conf, that broker will be started in non-encrypted mode (**'SSL = OFF'** is the default). The following is an example of setting the Broker **'query_editor'** in **encrypted mode** (cubrid_broker.conf).

```
# cubrid_broker.conf
[query_editor]
SERVICE                 =ON
SSL                     =ON
BROKER_PORT             =30000
....
```

Certificate and Private Key

In order to exchange an encrypted **symmetric session key** which will be used in a secure communication session, a public key and a private key are required in the server.

The public key used by the server is included in the certificate **'cas_ssl_cert.crt'**, and the private key is included in **'cas_ssl_cert.key'**. The certificate and private key are located in the **$CUBRID/conf** directory.

This certificate, **'self-signed'** certificate, was created with the OpenSSL command tool utility, and can be replaced with another certificate issued by a public **CA** (Certificate Authorities, for example

IdenTrust or **DigiCert**) if desired. Or, existing certificate/private key can be replaced by generating new one using OpenSSL command utility as shown below.

```
$ openssl genrsa -out my_cert.key
2048                                              # create 2048 bit
size RSA private key
$ openssl req -new -key my_cert.key -out
my_cert.csr                                  # create CSR
(Certificate Signing Request)
$ openssl x509 -req -days 365 -in my_cert.csr -signkey my_cert.key -
out my_cert.crt  # create a certificate valid for 1 year.
```

And replace **my_cert.key** and **my_cert.crt** with $CUBRID/conf/cas_ssl_cert.key and $CUBRID/conf/cas_ssl_cert.crt respectively.

Managing a Specific Broker

Enter the code below to run *broker1* only. Note that *broker1* should have already been configured in the shared memory.

```
% cubrid broker on broker1
```

The following message is returned if *broker1* has not been configured in the shared memory.

```
% cubrid broker on broker1
Cannot open shared memory
```

Enter the code below to stop *broker1* only. Note that service pool of *broker1* can also be removed.

```
% cubrid broker off broker1
```

The broker reset feature enables broker application servers (CAS) to disconnect the existing connection and reconnect when the servers are connected to unwanted databases due to failover, etc. in HA. For example, once Read Only broker is connected to active servers, it is not automatically connected to standby servers although standby servers are available. Connecting to standby servers is allowed only with the **cubrid broker reset** command.

Enter the code below to reset broker1.

```
% cubrid broker reset broker1
```

Dynamically Changing Broker Parameters

You can configure the parameters related to running the broker in the configuration file (**cubrid_broker.conf**). You can also modify some broker parameters temporarily while the broker is running by using the **broker_changer** utility. For details, see Broker Configuration.

The syntax for the **broker_changer** utility, which is used to change broker parameters while the broker is running, is as follows. Enter the name of the currently running broker for the *broker_name* . The *parameters* can be used only for dynamically modifiable parameters. The *value* must be specified based on the parameter to be modified. You can specify the broker CAS identifier (*cas_id*) to apply the changes to the specific broker CAS.

Note

When CUBRID SHARD feature is activated(SHARD=ON in cubrid_broker.conf), you cannot apply the changes to the specific broker CAS by specifying the broker CAS identifier(cas_id).

cas_id is an ID to be output by **cubrid broker status** command.

```
broker_changer <broker_name> [<cas_id>] <conf-name> <conf-value>
```

Enter the following to configure the **SQL_LOG** parameter to **ON** so that SQL logs can be written to the currently running broker. Such dynamic parameter change is effective only while the broker is running.

```
% broker_changer query_editor sql_log on
OK
```

Enter the following to change the **ACCESS_MODE** to **Read Only** and automatically reset the broker in HA environment.

```
% broker_changer broker_m access_mode ro
OK
```

Note

If you want to control the service using cubrid utilities on Windows Vista or the later versions of Window, you are recommended to open the command prompt window as an administrator. For details, see the notes of CUBRID Utilities.

Broker configuration information

cubrid broker info dumps the currently "working" broker parameters' configuration information(cubrid_broker.conf). broker parameters' information can be dynamically changed by **broker_changer** command; with **cubrid broker info** command, you can see the configuration information of the working broker.

```
% cubrid broker info
```

As a reference, to see the configuration information of the currently "working" system(cubrid.conf), use **cubrid paramdump** *database_name* command. By **SET SYSTEM PARAMETERS** syntax, the configuration information of the system parameters can be changed dynamically; with **cubrid broker info** command, you can see the configuration information of the system parameters.

Check CUBRID SHARD ID

cubrid broker getid prints out SHARD ID to know in what DB a specific key is included.

```
cubrid broker getid -b <broker-name> [-f] shard-key
```

- -b <*broker-name*>: broker name
- -f: print detail information
- *shard-key*: shard key

The following is an example of outputting the SHARD ID for key 1 from the shard1 broker.

```
$ cubrid broker getid -b shard1 1
@ cubrid broker getid
% shard1
 SHARD_ID : 0, SHARD_KEY: 1
```

The following is an example of outputting detailed information using the -f option.

```
$ cubrid broker getid -b shard1 -f 1
@ cubrid broker getid
% shard1
 SHARD_ID : 0, SHARD_KEY : 1, KEY_COLUMN : student_no
 MODULAR : 256, LIBRARY_NAME : NOT DEFINED, FUNCTION_NAME : NOT
DEFINED
 RANGE STATISTICS : student_no
```

```
      MIN ~   MAX :      SHARD
    ---------------------------
        0 ~    31 :          0

 SHARD CONNECTION :
    SHARD_ID          DB NAME          CONNECTION_INFO
    ---------------------------------------------------
           0           shard1                192.168.10.1
```

Broker to DB Connection Test

cubrid broker testis a command to perform user-defined queries on a DB that connects to a specified broker. When the shard function is activated, query all SHARD DBs. After performing the query, the transaction is rolled back. This command allows you to check the success of the query for each SHARD DB by performing a query to all SHARD DBs accessing the specified broker, and if you set the SHARD HASH function, you can check on which SHARD DB the query you entered was performed.

```
cubrid broker test <broker_name> [-D <db_name>] [-u <db_user>] [-p
<db_password>] {-c <query> | -i <input_file>} [-o <output_file>] [-
s] [-v]
```

- db_name: DB name
- db_user: DB user account
- db_password: DB user password
- query: query
- input_file: File that stores the query you want to enter
- output_file: File to save the results

The following options are available with the **cubrid broker test** utility.

-D DB_NAME

Specifies the DB name of the test target. When this option is omitted, the value of SHARD_DB_NAME parameter is used if the value of a SHARD parameter in cubrid_broker.conf is ON. If the value of a SHARD parameter is OFF, an error occurs.

-u DB_USER

Specifies the DB account of the test target. When this option is omitted, the value of SHARD_DB_USER parameter is used if the value of a SHARD parameter in cubrid_broker.conf is ON. If the value of a SHARD parameter is OFF, “public” is input in CUBRID.

-p DB_PASSWORD

Specifies the DB password of the test target. When this option is omitted, the value of SHARD_DB_PASSWORD parameter is used if the value of a SHARD parameter in cubrid_broker.conf is ON. If the value of a SHARD parameter is OFF, an empty string("") is input in CUBRID.

-c QUERY

Specifies the query string. **-c** or **-i** option can be used to specify a query. If they are omitted, only the connection information between a broker and a DB is printed.

-i FILE_NAME

Specifies the file where you saved the queries to input. **-c** or **-i** option can be used to specify a query. If they are omitted, only the connection information between a broker and a DB is printed.

-o FILE_NAME

Specifies the filename to save the execution result to be displayed to the console. If this is omitted, the execution result is output only to the console.

-s

A query which includes a SHARD hint is performed only in the SHARD DB. If this is omitted, a query is performed on all SHARD DBs.

If the value of a SHARD parameter is OFF, this option does not affect to the query execution.

-v

The error message and the result set of the SELECT are output together with the following information.

- RESULT: Error returned after performing query. output [OK | FAIL]
- SHARD_ID: ID of the SHARD DB where the query was performed (if the SHARD parameter value is OFF, the item is not output)
- ROW COUNT: Number of rows affected by DML, or number of rows in case of SELECT query. Outputs -1 in case of query execution error
- EXECUTION TIME: Time the query was performed
- QUERY: Queries entered by the user

If this option is omitted, only "RESULT, SHARD_ID, ROW COUNT, EXECUTION TIME, QUERY" will be output.

The following is an example of using the above options.

- Query to DB

When the SHARD parameter value of the cubrid_broker.conf is OFF

Check if the DB connection is good.

```
$ cubrid broker test shard1 -D shard -u shard -p shard123 -c
"select 1 from db_root where charset = 3"

@ cubrid broker test
@ [OK] CONNECT broker1 DB [demodb] USER [shard]

@ SHARD OFF

RESULT ROW COUNT EXECUTION TIME QUERY
============================================================
OK 1 0.011341 sec select 1,'a' from db_root where charset = 3
@ [OK] QUERY TEST
```

When the SHARD parameter value of the cubrid_broker.conf is ON

Check if all SHARD DBs are accessible.

```
$ cubrid broker test shard1 -D shard -u shard -p shard123 -c
"select 1 from db_root where charset = 3"

@ cubrid broker test
@ [OK] CONNECT shard1 DB [shard] USER [shard]

@ SHARD ON

RESULT SHARD_ID ROW COUNT EXECUTION TIME QUERY
======================================================================
OK 0 1 0.003436 sec select 1 from db_root where charset = 3
OK 1 1 0.003010 sec select 1 from db_root where charset = 3
OK 2 1 0.003039 sec select 1 from db_root where charset = 3
OK 3 1 0.002916 sec select 1 from db_root where charset = 3
@ [OK] QUERY TEST
```

- Check User Permissions

User do not have INSERT privileges on one of the SHARD DBs

If user you do not have INSERT privileges, RESULT is displayed as FAIL.

```
$ cubrid broker test shard1 -c "insert into foo values (1,"a") " -v

@ cubrid broker test
@ [OK] CONNECT shard1 DB [shard] USER [shard]
```

```
@ SHARD ON

RESULT SHARD_ID ROW COUNT EXECUTION TIME QUERY
================================================================
OK 0 1 0.001322 sec insert into foo values(1,'a')
FAIL(-494) -1 -1 0.001608 sec insert into foo values(1,'a')
<Error>
ERROR CODE : -494
Semantic: INSERT is not authorized on foo. insert into foo foo
(foo.a, foo.b) values (1, cast('a' as v...[CAS INFO -
127.0.0.1:52002, 1, 18145].

OK 2 1 0.001334 sec insert into foo values(1,'a')
OK 3 1 0.001325 sec insert into foo values(1,'a')
@ [FAIL] QUERY TEST
```

When there is no UPDATE authority on one of DBs which access a broker

If DB do not have UPDATE privileges, RESULT is displayed as FAIL.

```
$ vi dml.txt

#query
select a from foo
insert into foo(b) values(3)
update foo set c = 2 where b = 3
delete foo where b = 3
```

```
$ cubrid broker test broker1 -D demodb -u shard -p shard123 -i
dml.txt -v

@ cubrid broker test
@ [OK] CONNECT broker1 DB [demodb] USER [shard]

@ SHARD OFF

RESULT ROW COUNT EXECUTION TIME QUERY
============================================================
OK 1 0.001612 sec select a from foo
<Result of SELECT Command>
  a
------------
  1

OK 1 0.001215 sec insert into foo(b) values(3)
FAIL(-494) -1 0.001291 sec update foo set c = 2 where b = 3
<Error>
ERROR CODE : -494
Semantic: UPDATE is not authorized on foo. update foo foo set
```

```
foo.c=2 where foo.b=3[CAS INFO - 127.0.0.1:52001, 1, 18139].

OK 0 0.001534 sec delete foo where b = 3
@ [FAIL] QUERY TEST
```

- Check SHARD HASH normal operation

  Check if the hashing is good for a specific key.

```
$ vi test_query.txt

#query
select number from demo_db where key = /*+ shard_key */ 14
select number from demo_db where key = /*+ shard_key */ 50
select number from demo_db where key = /*+ shard_key */ 80
select number from demo_db where key = /*+ shard_key */ 120
..
```

```
$ cubrid broker test shard1 -D shard -u shard -p shard123 -i
shard_key.txt -v -s

@ cubrid broker test
@ [OK] CONNECT shard1 DB [shard] USER [shard]

@ SHARD ON

RESULT SHARD_ID ROW COUNT EXECUTION TIME QUERY
================================================================
OK 0 1 0.002225 sec select * from foo where a = /*+ shard_key */ 10
<Result of SELECT Command>
  a b
---------------------------------
  10 'aaaa'

OK 1 1 0.001870 sec select * from foo where a = /*+ shard_key */ 40
<Result of SELECT Command>
  a b
---------------------------------
  40 'bbb'

OK 2 1 0.002004 sec select * from foo where a = /*+ shard_key */ 70
<Result of SELECT Command>
  a b
---------------------------------
  70 'cccc'

OK 3 1 0.002025 sec select * from foo where a = /*+ shard_key */ 100
<Result of SELECT Command>
  a b
```

```
---------------------------------
  100 'dddd'

@ [OK] QUERY TEST
```

- Whether to using the -v option

When using -v option

When the SELECT query is successful, the result set is output, and when the SELECT query is unsuccessful, an error message is output.

```
$ cubrid broker test broker1 -D demodb -u shard -p shard123 -i
dml.txt -v
@ cubrid broker test
@ [OK] CONNECT broker1 DB [demodb] USER [shard]

@ SHARD OFF

RESULT ROW COUNT EXECUTION TIME QUERY
OK 1 0.001311 sec select a from foo
<Result of SELECT Command>
  a
------------
  1

OK 1 0.001083 sec insert into foo(b) values(3)
FAIL(-494) -1 0.001166 sec update foo set c = 2 where b = 3
<Error>
ERROR CODE : -494
Semantic: UPDATE is not authorized on foo. update foo foo set
foo.c=2 where foo.b=3[CAS INFO - 127.0.0.1:52001, 1, 18139].

OK 0 0.001399 sec delete foo where b = 3
@ [FAIL] QUERY TEST
```

When not using the -v option

It only prints the success or failure of the query.

```
$ cubrid broker test broker1 -D demodb -u shard -p shard123 -i
dml.txt

@ cubrid broker test
@ [OK] CONNECT broker1 DB [demodb] USER [shard]

@ SHARD OFF

RESULT ROW COUNT EXECUTION TIME QUERY
```

```
OK 1 0.001485 sec select a from foo
OK 1 0.001123 sec insert into foo(b) values(3)
FAIL(-494) -1 0.001180 sec update foo set c = 2 where b = 3
OK 0 0.001393 sec delete foo where b = 3
@ [FAIL] QUERY TEST
```

- Check SHARD key value

When SHARD key hint is given with the -s option, query to its SHARD DB, and outputs the result. With SHARD_ID, you can confirm that the query is executed from any SHARD DB.

```
$ cubrid broker test shard1 -i shard_key.txt -s -v

@ cubrid broker test
@ [OK] CONNECT shard1 DB [shard1] USER [shard]

@ SHARD ON

RESULT SHARD_ID ROW COUNT EXECUTION TIME QUERY
OK 0 1 0.144730 sec select * from foo where a = /*+ shard_key */ 10
<Result of SELECT Command>
  a b
---------------------------------
  10 'aaaa'

OK 1 1 0.001870 sec select * from foo where a = /*+ shard_key */ 40
<Result of SELECT Command>
  a b
---------------------------------
  40 'bbb'

OK 2 1 0.002004 sec select * from foo where a = /*+ shard_key */ 70
<Result of SELECT Command>
  a b
---------------------------------
  70 'cccc'

OK 3 1 0.002025 sec select * from foo where a = /*+ shard_key */ 100
<Result of SELECT Command>
  a b
---------------------------------
  100 'dddd'

@ [OK] QUERY TEST
```

Broker Logs

There are three types of logs that relate to starting the broker: access, error and SQL logs. Each log can be found in the log directory under the installation directory. You can change the directory where these logs are to be stored through **LOG_DIR** and **ERROR_LOG_DIR** parameters of the broker configuration file (**cubrid_broker.conf**).

When **SHARD** = ON, the log directory of CUBRID proxy can be configured by using the **SHARD_PROXY_LOG_DIR** parameter.

Checking the Access Log

The access log file records information on the application client and is stored to **$CUBRID/log/broker/**<*broker_name*>**.access** file. If the **ACCESS_LOG** parameter is configured to **ON** in the broker configuration file, when the broker stops properly, the access log file is stored.

The maximum size of the ACCESS_LOG file can be specified through the ACCESS_LOG_MAX_SIZE parameter. When the ACCESS_LOG file is larger than the specified size, it is backed up in the name of broker_name.access.YYYYMMDDHHMISS, and the log is recorded in a new file (broker_name.access).

The record of denied access is recorded in broker_name.access.denied. It is backed up with the same rules as the ACCESS_LOG file.

The following example and description show an access log file created in the log directory:

```
1 192.168.56.4 2020/11/10 14:41:55 testdb dba NEW 6
```

- 1: ID assigned to the application server of the broker
- 192.168.56.4: IP address of the application client
- 2020/11/10 14:41:55: Time when the client's request processing started
- testdb: The name of the database that the client requested to connect to
- dba: The username of the database that the client requested to connect to
- NEW: Connection type
  - NEW: New connection
  - OLD: Change client or reconnection of existing connection due to CAS restart
  - REJ: Connction denied (Recorded only in access.denied file)
- 6: session-id (session-id assgined by server)

Checking the Error Log

The error log file records information on errors that occurred during the client's request processing and is stored to **$CUBRID/log/broker/error_log***<broker_name>_<app_server_num>***.err** file. For error codes and error messages, see CAS Error.

The following example and description show an error log:

```
Time: 02/04/09 13:45:17.687 - SYNTAX ERROR *** ERROR CODE = -493,
Tran = 1, EID = 38
Syntax: Unknown class "unknown_tbl". select * from unknown_tbl
```

- Time: 02/04/09 13:45:17.687: Time when the error occurred
- ◦ SYNTAX ERROR: Type of error (e.g. SYNTAX ERROR, ERROR, etc.)
- *** ERROR CODE = -493: Error code
- Tran = 1: Transaction ID. -1 indicates that no transaction ID is assigned.
- EID = 38: Error ID. This ID is used to find the SQL log related to the server or client logs when an error occurs during SQL statement processing.
- Syntax ...: Error message (An ellipsis (...) indicates omission.)

Managing the SQL Log

The SQL log file records SQL statements requested by the application client and is stored with the name of *<broker_name>_<app_server_num>*. sql.log. The SQL log is generated in the log/broker/sql_log directory when the SQL_LOG parameter is set to ON. Note that the size of the SQL log file to be generated cannot exceed the value set for the SQL_LOG_MAX_SIZE parameter. CUBRID offers the **broker_log_top** and **cubrid_replay** utilities to manage SQL logs. Each utility should be executed in a directory where the corresponding SQL log exists.

The following examples and descriptions show SQL log files:

```
13-06-11 15:07:39.282 (0) STATE idle
13-06-11 15:07:44.832 (0) CLIENT IP 192.168.10.100
13-06-11 15:07:44.835 (0) CLIENT VERSION 11.4.0.1089
13-06-11 15:07:44.835 (0) session id for connection 0
13-06-11 15:07:44.836 (0) connect db demodb user dba url jdbc:cubrid:
192.168.10.200:30000:demodb:dba:********: session id 12
13-06-11 15:07:44.836 (0) DEFAULT isolation_level 4, lock_timeout -1
13-06-11 15:07:44.840 (0) end_tran COMMIT
13-06-11 15:07:44.841 (0) end_tran 0 time 0.000
13-06-11 15:07:44.841 (0) *** elapsed time 0.004
```

```
13-06-11 15:07:44.844 (0) check_cas 0
13-06-11 15:07:44.848 (0) set_db_parameter lock_timeout 1000
13-06-11 15:09:36.299 (0) check_cas 0
13-06-11 15:09:36.303 (0) get_db_parameter isolation_level 4
13-06-11 15:09:36.375 (1) prepare 0 CREATE TABLE unique_tbl (a INT
PRIMARY key);
13-06-11 15:09:36.376 (1) prepare srv_h_id 1
13-06-11 15:09:36.419 (1) set query timeout to 0 (no limit)
13-06-11 15:09:36.419 (1) execute srv_h_id 1 CREATE TABLE unique_tbl
(a INT PRIMARY key);
13-06-11 15:09:38.247 (1) execute 0 tuple 0 time 1.827
13-06-11 15:09:38.247 (0) auto_commit
13-06-11 15:09:38.344 (0) auto_commit 0
13-06-11 15:09:38.344 (0) *** elapsed time 1.968

13-06-11 15:09:54.481 (0) get_db_parameter isolation_level 4
13-06-11 15:09:54.484 (0) close_req_handle srv_h_id 1
13-06-11 15:09:54.484 (2) prepare 0 INSERT INTO unique_tbl VALUES
(1);
13-06-11 15:09:54.485 (2) prepare srv_h_id 1
13-06-11 15:09:54.488 (2) set query timeout to 0 (no limit)
13-06-11 15:09:54.488 (2) execute srv_h_id 1 INSERT INTO unique_tbl
VALUES (1);
13-06-11 15:09:54.488 (2) execute 0 tuple 1 time 0.001
13-06-11 15:09:54.488 (0) auto_commit
13-06-11 15:09:54.505 (0) auto_commit 0
13-06-11 15:09:54.505 (0) *** elapsed time 0.021

...

13-06-11 15:19:04.593 (0) get_db_parameter isolation_level 4
13-06-11 15:19:04.597 (0) close_req_handle srv_h_id 2
13-06-11 15:19:04.597 (7) prepare 0 SELECT * FROM unique_tbl  WHERE
ROWNUM BETWEEN 1 AND 5000;
13-06-11 15:19:04.598 (7) prepare srv_h_id 2 (PC)
13-06-11 15:19:04.602 (7) set query timeout to 0 (no limit)
13-06-11 15:19:04.602 (7) execute srv_h_id 2 SELECT * FROM
unique_tbl  WHERE ROWNUM BETWEEN 1 AND 5000;
13-06-11 15:19:04.602 (7) execute 0 tuple 1 time 0.001
13-06-11 15:19:04.607 (0) end_tran COMMIT
13-06-11 15:19:04.607 (0) end_tran 0 time 0.000
13-06-11 15:19:04.607 (0) *** elapsed time 0.009
```

- 13-06-11 15:07:39.282: Time when the application sent the request
- (1): Sequence number of the SQL statement group. If prepared statement pooling is used, it is uniquely assigned to each SQL statement in the file.
- CLIENT IP: An IP of an application client
- CLIENT VERSION: A driver's version of an application client

- prepare 0: Whether or not it is a prepared statement
- prepare srv_h_id 1: Prepares the SQL statement as srv_h_id 1.
- (PC): It is displayed if the data in the plan cache is used.
- Execute 0 tuple 1 time 0.001: One row is executed. The time spent is 0.001 seconds.
- auto_commit/auto_rollback: Automatically committed or rolled back. The second auto_commit/auto_rollback is an error code. 0 indicates that the transaction has been completed without an error.

broker_log_top

The **broker_log_top** utility analyzes the SQL logs which are generated for a specific period. As a result, the information of SQL statements and time execution are displayed in files by order of the longest execution time; the results of SQL statements are stored in **log.top.q** and those of execution time are stored in **log.top.res**, respectively.

The **broker_log_top** utility is useful to analyze a long running query. The syntax is as follows:

```
broker_log_top [options] sql_log_file_list
```

- *sql_log_file_list*: names of log files to analyze.

The following is [options] used on **broker_log_top**.

-t

The result is displayed in transaction unit.

-F DATETIME

Specifies the execution start date and time of the SQL statements to be analyzed. The input format is YY-MM-DD[hh[:mm[:ss[.msec]]]], and the part enclosed by [] can be omitted. If you omit the value, it is regarded as that 0 is input for hh, mm, ss and msec.

-T DATETIME

Specifies the execution end date and time of the SQL statements to be analyzed. The input format is the same with the *DATE* in the **-F** options.

All logs are displayed by SQL statement if any option is not specified.

The following sets the search range to milliseconds

```
broker_log_top -F "13-01-19 15:00:25.000" -T "13-01-19 15:15:25.180"
log1.log
```

The part where the time format is omitted is set to 0 by default. This means that -F “13-01-19 00:00:00.000” -T “13-01-20 00:00:00.000” is input.

```
broker_log_top -F "13-01-19" -T "13-01-20" log1.log
```

The following logs are the results of executing the broker_log_top utility; logs are generated from Nov. 11th to Nov. 12th 2013, and it is displayed in the order of the longest execution of SQL statements. Each month and day are separated by a hyphen (-) when specifying period. Note that “*.sql.log” is not recognized so the SQL logs should be separated by a whitespace on Windows.

```
--Execution broker_log_top on Linux
% broker_log_top -F "13-11-11" -T "13-11-12" -t *.sql.log

query_editor_1.sql.log
query_editor_2.sql.log
query_editor_3.sql.log
query_editor_4.sql.log
query_editor_5.sql.log

--Executing broker_log_top on Windows
% broker_log_top -F "13-11-11" -T "13-11-12" -t
query_editor_1.sql.log query_editor_2.sql.log query_editor_3.sql.log
query_editor_4.sql.log query_editor_5.sql.log
```

The **log.top.q** and **log.top.res** files are generated in the same directory where the analyzed logs are stored when executing the example above; In the **log.top.q** file, you can see each SQL statement, and its line number. In the **log.top.res** file, you can see the minimum execution time, the maximum execution time, the average execution time, and the number of execution queries for each SQL statement.

```
--log.top.q file
[Q1]----------------------------------------
broker1_6.sql.log:137734
13-11-11 18:17:59.396 (27754) execute_all srv_h_id 34 select
a.int_col, b.var_col from dml_v_view_6 a, dml_v_view_6 b,
dml_v_view_6 c , dml_v_view_6 d, dml_v_view_6 e where
a.int_col=b.int_col and b.int_col=c.int_col and c.int_col=d.int_col
and d.int_col=e.int_col order by 1,2;
13-11-11 18:18:58.378 (27754) execute_all 0 tuple 497664 time 58.982
.
.
[Q4]----------------------------------------
broker1_100.sql.log:142068
13-11-11 18:12:38.387 (27268) execute_all srv_h_id 798 drop table
list_test;
13-11-11 18:13:08.856 (27268) execute_all 0 tuple 0 time 30.469
```

```
--log.top.res file

              max       min        avg   cnt(err)
-------------------------------------------------------
[Q1]        58.982    30.371    44.676     2 (0)
[Q2]        49.556    24.023    32.688     6 (0)
[Q3]        35.548    25.650    30.599     2 (0)
[Q4]        30.469     0.001     0.103  1050 (0)
```

cubrid_replay

cubrid_replay utility replays the SQL log in the broker and outputs the results sorted in order from the large difference(from the slower query than the existing one) by comparing the difference in the execution time of playback and the existing execution time.

This utility plays back the queries that are logged in the SQL log, but does not execute the queries to change the data. If any options are not given, only SELECT queries are run; if **-r** option is given, it changes the UPDATE and DELETE queries into SELECT queries and runs them.

This utility can be used to compare the performance between two different hosts; for example, there can be a performance difference for a same query between master and slave even if their h/w specs are the same.

```
cubrid_replay -I <broker_host> -P <broker_port> -d <db_name>
[options] <sql_log_file> <output_file>
```

- *broker_host*: IP address or hostname of the CUBRID broker
- *broker_port*: Port number of the CUBRID broker
- *db_name*: The name of database to run the query
- *sql_log_file*: SQL log file of the CUBRID broker($CUBRID/log/broker/sql_log/*.log, *.log.bak)
- *output_file*: Filename to save the execution result

The following is [options] used in **cubrid_replay**.

-u DB_USER

Specifies the DB account(default: public).

-p DB_PASSWORD

Specifies database password

-r

Changes UPDATE and DELETE queries into SELECT queries

-h SECOND

Specifies the term to wait between queries to run(default: 0.01 sec)

-D SECOND

The queries are output to *output_file* only when the specified term is bigger than (replayed execution time - previous execution time)(default: 0.01 sec).

-F DATETIME

Specifies the execution start date and time of the SQL statements to be replayed. The input format is YY[-MM[-DD[hh[:mm[:ss[.msec]]]]]], and the part enclosed by [] can be omitted. If you omit the value, it is regarded as that 01 is input for MM and DD, and 0 is input for hh, mm, ss and msec.

-T DATETIME

Specifies the execution end date and time of the SQL statements to be replayed. The input format is the same with the *DATE* in the **-F** options.

```
$ cubrid_replay -I testhost -P 33000 -d testdb -u dba -r
testdb_1_11_1.sql.log.bak output.txt
```

If you run the above command, the summary of execution result is displayed on the console.

```
------------------- Result Summary --------------------------
* Total queries : 153103
* Skipped queries (see skip.sql) : 5127
* Error queries (see replay.err) : 30
* Slow queries (time diff > 0.000 secs) : 89987
* Max execution time diff : 0.016
* Avg execution time diff : -0.001

cubrid_replay run time : 245.308417 sec
```

- Total queries: Number of total queries within the specified date and time. They include DDL and DML
- Skipped queries: Number of queries which cannot be changed from UPDATE/DELETE into SELECT when **-r** option is specified. These queries are saved into skip.sql
- Slow queries: Number of queries of which execution time difference is bigger than the specified value by **-D** option(the replayed execution time is slower than the previous execution time plus the specified value). If you omit the **-D** option, this option value is specified as 0.01 second
- Max execution time diff: The biggest value among the differences of the execution time(unit: sec)
- Avg execution time diff: Average value of the differences of the execution time(unit: sec)

- cubrid_replay run time: Execution time of this utility

"Skipped queries" are the cases which query-transform from UPDATE/DELETE to SELECT is impossible by the internal reason; the queries which are written to skip.sql are needed to check separately.

Also, you should consider that the execution time of the transformed queries does not include the data modification time.

In the *output.txt* file, SQLs that the replayed SQL execution time is slower than the SQL execution time in SQL log are written. That is, {(the replayed SQL execution time) - {(the execution time in SQL log) + (the specified time by **-D** option)} is sorted in descending order. Because **-r** option is used, UPDATE/DELETE is rewritten into SELECT and run.

```
EXEC TIME (REPLAY / SQL_LOG / DIFF): 0.003 / 0.001 / 0.002
SQL: UPDATE NDV_QUOTA_INFO SET last_mod_date = now() , used_quota =
( SELECT IFNULL(sum(file_size),0) FROM NDV_RECYCLED_FILE_INFO WHERE
user_id = ? ) + ( SELECT IFNULL(sum(file_size),0) FROM NDV_FILE_INFO
WHERE user_id = ? ) WHERE user_id = ? /+shard_val(6900403)/ /* SQL :
NDVMUpdResetUsedQuota */
REWRITE SQL: select NDV_QUOTA_INFO, class NDV_QUOTA_INFO,
cast( SYS_DATETIME as datetime), cast((select
ifnull(sum(NDV_RECYCLED_FILE_INFO.file_size), 0) from
NDV_RECYCLED_FILE_INFO NDV_RECYCLED_FILE_INFO where
(NDV_RECYCLED_FILE_INFO.user_id= ?:0 ))+(select
ifnull(sum(NDV_FILE_INFO.file_size), 0) from NDV_FILE_INFO
NDV_FILE_INFO where (NDV_FILE_INFO.user_id= ?:1 )) as bigint) from
NDV_QUOTA_INFO NDV_QUOTA_INFO where (NDV_QUOTA_INFO.user_id= ?:2 )
BIND 1: 'babaemo'
BIND 2: 'babaemo'
BIND 3: 'babaemo'
```

- EXEC TIME: (replay time / execution time in the SQL log / difference between the two execution times)
- SQL: The original SQL which exists in the SQL log of the broker
- REWRITE SQL: Transformed SELECT queries from UPDATE/DELETE queries by **-r** option.

Note

broker_log_runner is deprecated from 9.3. Therefore, instead of broker_log_runner, use cubrid_replay.

CAS Error

CAS error is an error which occurs in broker application server(CAS), and it can happen on all applications which access to CAS with drivers.

Below shows the CAS error code table. CCI and JDBC's error messages can be different each other on the same CAS error code. If there is only one message, they are the same, but if there are two messages, then the first one is CCI error message and the second one is JDBC error message.

Error Code Name(Error Number)	Error Message (CCI / JDBC)	Note
CAS_ER_INTERNAL(-10001)		
CAS_ER_NO_MORE_MEMORY(-10002)	Memory allocation error	
CAS_ER_COMMUNICATION(-10003)	Cannot receive data from client / Communication error	
CAS_ER_ARGS(-10004)	Invalid argument	
CAS_ER_TRAN_TYPE(-10005)	Invalid transaction type argument / Unknown transaction type	
CAS_ER_SRV_HANDLE(-10006)	Server handle not found / Internal server error	
CAS_ER_NUM_BIND(-10007)	Invalid parameter binding value argument / Parameter binding error	The number of data to be bound does not match with the number of data to be transferred.

Error Code Name(Error Number)	Error Message (CCI / JDBC)	Note
CAS_ER_UNKNOWN_U_TYPE(-10008)	Invalid T_CCI_U_TYPE value / Parameter binding error	
CAS_ER_DB_VALUE(-10009)	Cannot make DB_VALUE	
CAS_ER_TYPE_CONVERSION(-10010)	Type conversion error	
CAS_ER_PARAM_NAME(-10011)	Invalid T_CCI_DB_PARAM value / Invalid database parameter name	The name of the system parameter is not valid.
CAS_ER_NO_MORE_DATA(-10012)	Invalid cursor position / No more data	
CAS_ER_OBJECT(-10013)	Invalid oid / Object is not valid	
CAS_ER_OPEN_FILE(-10014)	Cannot open file / File open error	
CAS_ER_SCHEMA_TYPE(-10015)	Invalid T_CCI_SCH_TYPE value / Invalid schema type	
CAS_ER_VERSION(-10016)	Version mismatch	The DB server version does not compatible with the client (CAS) version.
CAS_ER_FREE_SERVER(-10017)	Cannot process the request. Try again later	The CAS which handles connection request of

Error Code Name(Error Number)	Error Message (CCI / JDBC)	Note
		applications cannot be assigned.
CAS_ER_NOT_AUTHORIZED_CLIENT(-10018)	Authorization error	Access is denied.
CAS_ER_QUERY_CANCEL(-10019)	Cannot cancel the query	
CAS_ER_NOT_COLLECTION(-10020)	The attribute domain must be the set type	
CAS_ER_COLLECTION_DOMAIN(-10021)	Heterogeneous set is not supported / The domain of a set must be the same data type	
CAS_ER_NO_MORE_RESULT_SET(-10022)	No More Result	
CAS_ER_INVALID_CALL_STMT(-10023)	Illegal CALL statement	
CAS_ER_STMT_POOLING(-10024)	Invalid plan	
CAS_ER_DBSERVER_DISCONNECTED(-10025)	Cannot communicate with DB Server	
CAS_ER_MAX_PREPARED_STMT_COUNT_EXCEEDED(-10026)	Cannot prepare more than MAX_PREPARED_STMT_COUNT statements	

Error Code Name(Error Number)	Error Message (CCI / JDBC)	Note
CAS_ER_HOLDABLE_NOT_ALLOWED(-10027)	Holdable results may not be updatable or sensitive	
CAS_ER_HOLDABLE_NOT_ALLOWED_KEEP_CON_OFF(-10028)	Holdable results are not allowed while KEEP_CONNECTION is off	
CAS_ER_NOT_IMPLEMENTED(-10100)	None / Attempt to use a not supported service	
CAS_ER_SSL_TYPE_NOT_ALLOWED(-10103)	None / The requested SSL mode is not permitted	
CAS_ER_IS(-10200)	None / Authentication failure	

Gateway

Starting Gateway

Enter the command below to start the gateway.

```
$ cubrid gateway start
@ cubrid gateway start
++ cubrid gateway start: success
```

The following message is returned if the gateway is already running.

```
$ cubrid gateway start
@ cubrid gateway start
++ cubrid gateway is running.
```

Stopping Gateway

Enter the command below to stop the gateway.

```
$ cubrid gateway stop
@ cubrid gateway stop
++ cubrid gateway stop: success
```

The following message is returned if the gateway has stopped.

```
$ cubrid gateway stop
@ cubrid gateway stop
++ cubrid gateway is not running.
```

Restarting Gateway

Enter the command below to restart the whole gateways.

```
$ cubrid gateway restart
```

Checking Gateway Status

The **cubrid gateway status** utility allows you to check the gateway status such as number of completed jobs and the number of standby jobs by providing various options.

```
cubrid gateway status [options] [expr]
```

- *expr*: A part of the gateway name or "SERVICE=ON|OFF"

Specifying *expr* performs that the status of specific gateways which include *expr* in their names is monitored; specifying no argument means that status of all gateways which are registered in the gateway environment configuration file (**cubrid_gateway.conf**) is monitored.

If "SERVICE=ON" is specified on *expr*, only the status of working gateways is displayed; if "SERVICE=OFF" is specified, only the status of stopped gateways is displayed.

The following [options] are available with the **cubrid gateway status** utility. -b, -q, -c, -m, -S, -P and -f are options to define the information to print; -s, -l and -t are options to control printing. All of these are possible to use as combining each other.

-b

Displays the status information of a gateway but does not display information on gateway application server.

-q

Displays standby jobs in the job queue.

-f

Displays information of DB and host accessed by gateway.

If it is used with the **-b** option, additional information on CAS is displayed. But SELECT, INSERT, UPDATE, DELETE, OTHERS items which shown on **-b** option are excluded.

If it is used with the **-P** option, STMT-POOL-RATIO is additionally printed. This item shows the ratio to use statements in the pool when you are using prepared statements.

-l SECOND

The **-l** option is only used with -f option together. It specifies accumulation period (unit: sec.) when displaying the number of application servers whose client status is Waiting or Busy. If it is omitted, the default value (1 second) is specified.

-t

Displays results in tty mode on the screen. The output can be redirected and used as a file.

-s SECOND

Regularly displays the status of gateway based on specified period. It returns to a command prompt if q is entered.

If you do not specify options or arguments, the status of all gateways is displayed.

```
$ cubrid gateway status
@ cubrid gateway status
% oracle_gateway
----------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
----------------------------------------
 1 28434     0     0 50144 IDLE
 2 28435     0     0 50144 IDLE
 3 28436     0     0 50144 IDLE
 4 28437     0     0 50140 IDLE
 5 28438     0     0 50144 IDLE
```

- % oracle_gateway: The gateway name
- ID: Serial number of CAS within the gateway
- PID: CAS process ID within the gateway
- QPS: The number of queries processed per second

- LQS: The number of long-duration queries processed per second
- PSIZE: Size of CAS
- STATUS: The current status of CAS (BUSY, IDLE, CLIENT_WAIT, CLOSE_WAIT)

The following shows the detail status of gateway for 5 seconds. The display will reset per 5 seconds as the new status information. To escape the display of the status, press <Q>.

```
$ cubrid gateway status -b -s 5
@ cubrid gateway status

 NAME                    PID  PORT   AS   JQ    TPS    QPS
SELECT   INSERT   UPDATE   DELETE   OTHERS     LONG-T     LONG-Q
ERR-Q  UNIQUE-ERR-Q  #CONNECT  #REJECT
============================================================================
============================================================================
============================
* oracle_gateway         13200 30000    5    0      0      0
0        0        0        0        0     0/60.0     0/60.0
0             0         0        0
```

- NAME: The gateway name
- PID: Process ID of the gateway
- PORT: Port number of the gateway
- AS: The number of CAS
- JQ: The number of standby jobs in the job queue
- TPS: The number of transactions processed per second (calculated only when the option is configured to “-b -s <sec>”)
- QPS: The number of queries processed per second (calculated only when the option is configured to “-b -s <sec>”)
- SELECT: The number of SELECT queries after staring of the gateway. When there is an option of “-b -s <sec>”, it is updated every time with the number of SELECTs which have been executed during the seconds specified by this option.
- INSERT: The number of INSERT queries after staring of the gateway. When there is an option of “-b -s <sec>”, it is updated every time with the number of INSERTs which have been executed during the seconds specified by this option.

- UPDATE: The number of UPDATE queries after staring of the gateway. When there is an option of "-b -s <sec>", it is updated every time with the number of UPDATEs which have been executed during the seconds specified by this option.
- DELETE: The number of DELETE queries after staring of the gateway. When there is an option of "-b -s <sec>", it is updated every time with the number of DELETEs which have been executed during the seconds specified by this option.
- OTHERS: The number of queries like CREATE and DROP except for SELECT, INSERT, UPDATE, DELETE. When there is an option of "-b -s <sec>", it is updated every time with the number of queries which have been executed during the seconds specified by this option.
- LONG-T: The number of transactions which exceed LONG_TRANSACTION_TIME. / the value of the LONG_TRANSACTION_TIME parameter. When there is an option of "-b -s <sec>", it is updated every time with the number of transactions which have been executed during the seconds specified by this option.
- LONG-Q: The number of queries which exceed LONG_QUERY_TIME. / the value of the LONG_QUERY_TIME parameter. When there is an option of "-b -s <sec>", it is updated every time with the number of queries which have been executed during the seconds specified by this option.
- ERR-Q: The number of queries with errors found. When there is an option of "-b -s <sec>", it is updated every time with the number of errors which have occurred during the seconds specified by this option.
- UNIQUE-ERR-Q: The number of queries with unique key errors found. When there is an option of "-b -s <sec>", it is updated every time with the number of unique key errors which have occurred during the seconds specified by this option.
- #CONNECT: The number of connections that an application client accesses to CAS after starting the gateway.
- #REJECT: The count that an application client excluded from ACL IP list is rejected to access a CAS. Regarding ACL setting, see Limiting Broker Access.

The following checks the status of gateway whose name includes oracle_gateway and job status of a specific gateway in the job queue with the **-q** option. If you do not specify oracle_gateway as an argument, list of jobs in the job queue for all gateways is displayed.

```
% cubrid gateway status -q oracle_gateway
@ cubrid gateway status
% oracle_gateway
----------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
----------------------------------------
 1 28444     0     0 50144 IDLE
```

```
 2 28445     0     0 50140 IDLE
 3 28446     0     0 50144 IDLE
 4 28447     0     0 50144 IDLE
 5 28448     0     0 50144 IDLE
```

The following monitors the status of a gateway whose name includes oracle_gateway with the **-s** option. If you do not specify oracle_gateway as an argument, monitoring status for all gateways is performed regularly. It returns to a command prompt if q is not entered.

```
% cubrid gateway status -s 5 oracle_gateway
% oracle_gateway
----------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
----------------------------------------
 1 28444     0     0 50144 IDLE
 2 28445     0     0 50140 IDLE
 3 28446     0     0 50144 IDLE
 4 28447     0     0 50144 IDLE
 5 28448     0     0 50144 IDLE
```

With the **-t** option, it display information of TPS and QPS to a file. To cancel displaying, press <Ctrl+C> to stop program.

```
% cubrid gateway status -b -t -s 1 > log_file
```

The following views information of server/database accessed by gateway, the last access times of applications, the IP addresses accessed to CAS and the versions of drivers etc. with the **-f** option.

```
$ cubrid gateway status -f oracle_gateway
@ cubrid gateway status
% oracle_gateway
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
------------------------------------------------
ID   PID   QPS   LQS PSIZE STATUS         LAST ACCESS TIME
DB       HOST   LAST CONNECT TIME       CLIENT IP   CLIENT
VERSION    SQL_LOG_MODE   TRANSACTION STIME  #CONNECT  #RESTART
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
------------------------------------------------
 1 26946     0     0 51168 IDLE         2011/11/16 16:23:42  demodb
localhost 2011/11/16 16:23:40      10.0.1.101
9.2.0.0062              NONE 2011/11/16 16:23:42         0         0
 2 26947     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                   -
0.0.0.0                                -                   -
0         0
```

```
 3 26948     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                   -
0.0.0.0                               -                   -
0         0
 4 26949     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                   -
0.0.0.0                               -                   -
0         0
 5 26950     0     0 51172 IDLE         2011/11/16 16:23:34
-          -                   -
0.0.0.0                               -                   -
0         0
```

Meaning of each column in code above is as follows:

- LAST ACCESS TIME: Time when CAS runs or the latest time when an application client accesses CAS
- DB: Name of a database which CAS accesses most recently
- HOST: Name of a which CAS accesses most recently
- LAST CONNECT TIME: Most recent time when CAS accesses a database
- CLIENT IP: IP of an application clients currently being connected to an application server(CAS). If no application client is connected, 0.0.0.0 is displayed.
- CLIENT VERSION: A driver's version of an application client currently being connected to a CAS
- SQL_LOG_MODE: SQL logging mode of CAS. If the mode is same as the mode configured in the gateway, "-" is displayed.
- TRANSACTION STIME: Transaction start time
- #CONNECT: The number of connections that an application client accesses to CAS after starting the gateway
- #RESTART: The number of connection that CAS is re-running after starting the gateway

Enter the command below with the **-b** and **-f** options to display AS(T W B Ns-W Ns-B) and CANCELED additionally.

```
// The -f option is added upon execution of gateway status
information. Configuring Ns-W and Ns-B are displayed as long as N
seconds by using the -l.
% cubrid gateway status -b -f -l 2
@ cubrid gateway status
NAME          PID    PSIZE PORT  AS(T W B 2s-W 2s-B) JQ TPS QPS LONG-
T LONG-Q  ERR-Q UNIQUE-ERR-Q CANCELED ACCESS_MODE SQL_LOG  #CONNECT
```

```
#REJECT
================================================================================
================================================================================
======
oracle_gateway 16784 56700 30000      5 0 0     0   0   0  16  29 0/
60.0 0/60.0      1            1        0          RW     ALL
4       1
```

Meaning of added columns in code above is as follows:

- AS(T): Total number of CAS being executed
- AS(W): The number of CAS in the status of Waiting
- AS(B): The number of CAS in the status of Busy
- AS(Ns-W): The number of CAS that the client belongs to has been waited for N seconds.
- AS(Ns-B): The number of CAS that the client belongs to has been Busy for N seconds.
- CANCELED: The number of queries have cancelled by user interruption since the gateway starts (if it is used with the **-l** *N* option, it specifies the number of accumulations for *N* seconds).

Note

Gateway is almost similar to broker, see below for more information on gateway

- Limiting Database Server Access
- Packet Encryption
- Managing a Specific Broker
- Broker configuration information
- Broker Logs

CUBRID Manager Server

Starting the CUBRID Manager Server

The following example shows how to start the CUBRID Manager server.

```
% cubrid manager start
```

The following message is returned if the CUBRID Manager server is already running.

```
% cubrid manager start
@ cubrid manager server start
++ cubrid manager server is running.
```

Stopping the CUBRID Manager Server

The following example shows how to stop the CUBRID Manager server.

```
% cubrid manager stop
@ cubrid manager server stop
++ cubrid manager server stop: success
```

CUBRID Manager Server Log

The logs of CUBRID Manager server are stored in the log/manager directory under the installation directory. There are four types of log files depending on server process of CUBRID Manager.

- auto_backupdb.log: Backup log about the backup-automated jobs which was reserved by the CUBRID Manager Client
- auto_execquery.log: Execution log about the query-automated jobs which was reserved by the CUBRID Manager Client
- cub_js.access.log: Access log regarding the successful logins and tasks in CUBRID Manager Server.
- cub_js.error.log: Access log regarding the failed logins and tasks in CUBRID Manager Server.

Configuring CUBRID Manager Server

The configuration filename for the CUBRID Manager server is **cm.conf** and located in the **$CUBRID/conf** directory. In the CUBRID Manager server configuration file, where parameter names and values are stored, comments are prefaced by "#." Parameter names and values are separated by spaces or an equal sign (=).

This page describes parameters that are specified in the **cm.conf** file.

cm_port

cm_port is a parameter used to configure a communication port for the connection between the CUBRID Manager server and the client. The default value is **8001** .

monitor_interval

monitor_interval is a parameter used to configure the monitoring interval of **cub_auto** in seconds. The default value is **5** .

allow_user_multi_connection

allow_user_multi_connection is a parameter used to have multiple client connections allowed to the CUBRID Manager server. The default value is **YES** . Therefore, more than one CUBRID Manager client can connect to the CUBRID Manager server, even with the same username.

server_long_query_time

server_long_query_time is a parameter used to configure delay reference time in seconds when configuring **slow_query** which is one of server diagnostics items. The default value is **10** . If the execution time of the query performed on the server exceeds this parameter value, the number of the **slow_query** parameters will increase.

auto_job_timeout

auto_job_timeout is a parameter used to configure timeout of auto job for cub_auto. The default value is 43200 (12 hour).

mon_cub_auto

mon_cub_auto is a parameter used to allow cub_js to restart cub_auto process when cub_auto is not running or not. The default value is NO.

token_active_time

token_active_time is a parameter used to configure timeout of token. The default value is 7200 (2 hour).

support_mon_statistic

support_mon_statistic is a parameter used to configure monitoring statistic of system or not. The default value is NO.

cm_process_monitor_interval

cm_process_monitor_interval is an interval time for collecting statistics. The default and the minimum value is 5 (5 minutes).

CUBRID Manager User Management Console

The account and password of CUBRID Manager user are used to access the CUBRID Manager server when starting the CUBRID Manager client, distinguishing this user from the database user. CUBRID Manager Administrator (cm_admin) is a CLI tool that manages user information and it executes commands in the console window to manage users. This utility only supports Linux OS.

The following shows how to use the CUBRID Manager (hereafter, CM) Administrator utilities. The utilities can be used through GUI on the CUBRID Manager client.

```
cm_admin <utility_name>
<utility_name>:
    adduser [<option>] <cmuser-name> <cmuser-password>   --- Adds a
CM user
    deluser <cmuser-name>   --- Deletes a CM user
    viewuser [<cmuser-name>]   --- Displays CM user information
    changeuserauth [<option>] <cmuser-name>  --- Changes the CM user
authority
    changeuserpwd [<option>] <cmuser-name>  --- Changes the CM user
password
    adddbinfo [<option>] <cmuser-name> <database-name>  --- Adds
database information of the CM user
    deldbinfo <cmuser-name> <database-name>  --- Deletes database
information of the CM user
    changedbinfo [<option>] <database-name> number-of-pages ---
Changes database information of the CM user
```

CM Users

Information about CM users consists of the following:

- CM user authority: Includes the following information.
  - The permission to configure broker
  - The permission to create a database. For now, this authority is only given to the **admin** user.
  - The permission to monitor status
- Database information: A database that a CM user can use
- CM user password

The default user authority of CUBRID Manager is **admin** and its password is admin. Users who has **admin** authority have full administrative controls.

Adding CM Users

The **cm_admin adduser** utility creates a CM user who has been granted a specific authority and has database information. The permissions to configure broker, create a database, and monitor status can be granted to the CM user.

```
cm_admin adduser [options] cmuser-name cmuser-password
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **adduser**: A command to create a new CM user
- *cmuser-name*: Specifies a unique name to a CM user. Usable characters are 0~9, A~Z, a~z and _. Minimum length is 4 and maximum length is 32. If the specified name in *cmuser-name* is identical to the existing one, **cm_admin** will stop creating a new CM user.
- *cmuser-password*: A password of a CM user. Usable characters are 0~9, A~Z, a~z and _. Minimum length is 4 and maximum length is 32.

The following is [options] of **cm_admin adduser**.

-b, --broker AUTHORITY

Specifies the broker authority which will be granted to a new CM user.

You can use **admin**, **none** (default), and **monitor** as *AUTHORITY*

The following example shows how to create a CM user whose name is *testcm* and password is *testcmpwd* and then configure broker authority to monitor.

```
cm_admin adduser -b monitor testcm testcmpwd
```

-c, --dbcreate AUTHORITY

Specifies the authority to create a database which will be granted to a new CM user.

You can use **none** (default) and **admin** as *AUTHORITY*.

The following example shows how to create a CM user whose name is *testcm* and password is *testcmpwd* and then configure database creation authority to admin.

```
cm_admin adduser -c admin testcm testcmpwd
```

-m, --monitor AUTHORITY

Specifies the authority to monitor status which will be granted to a new CM user. You can use **admin**, **none** (default), and **monitor** as *AUTHORITY*

The following example shows how to create a CM user whose name is *testcm* and password is *testcmpwd* and then configure monitoring authority to admin.

```
cm_admin adduser -m admin testcm testcmpwd
```

-d, --dbinfo INFO_STRING

Specifies database information of a new CM user. The format of *INFO_STRING* must be "<dbname>;<uid>;<broker_ip>,<broker_port>". The following example shows how to add database information "testdb;dba;localhost,30000" to a CM user named *testcm* .

```
cm_admin adduser -d "testdb;dba;localhost,30000" testcm
testcmpwd
```

Deleting CM Users

The **cm_admin deluser** utility deletes a CM user.

```
cm_admin deluser cmuser-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **deluser**: A command to delete an existing CM user
- *cmuser-name*: The name of a CM user to be deleted

The following example shows how to delete a CM user named *testcm*.

```
cm_admin deluser testcm
```

Displaying CM User information

The **cm_admin viewuser** utility displays information of a CM user.

```
cm_admin viewuser cmuser-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **viewuser**: A command to display the CM user information
- *cmuser-name*: A CM username. If this value is entered, information only for the specified user is displayed; if it is omitted, information for all CM users is displayed.

The following example shows how to display information of a CM user named *testcm* .

```
cm_admin viewuser testcm
```

The information will be displayed as follows:

```
CM USER: testcm
  Auth info:
    broker: none
    dbcreate: none
    statusmonitorauth: none
  DB info:

================================================================
================================
     DBNAME
UID              BROKER INFO

================================================================
================================
     testdb
dba              localhost,30000
```

Changing the Authority of CM Users

The **cm_admin changeuserauth** utility changes the authority of a CM user.

```
cm_admin changeuserauth options cmuser-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **changeuserauth**: A command to change the authority of a CM user
- *cmuser-name*: The name of a CM user whose authority to be changed

The following is [options] of **cm_admin changeuserauth**.

-b, --broker AUTHORITY

Specifies the broker authority that will be granted to a CM user. You can use **admin**, **none**, and **monitor** as *AUTHORITY*.

The following example shows how to change the broker authority of a CM user named *testcm* to monitor.

```
cm_admin changeuserauth -b monitor testcm
```

-c, --dbcreate

Specifies the authority to create a database which will be granted to a CM user. You can use **admin** and **none** as *AUTHORITY* .

The following example shows how to change the database creation authority of a CM user named *testcm* to admin.

```
cm_admin changeuserauth -c admin testcm
```

-m, --monitor

Specifies the authority to monitor status which will be granted to a CM user. You can use **admin**, **none**, and **monitor** as *AUTHORITY*.

The following example shows how to change the monitoring authority of a CM user named *testcm* to admin.

```
cm_admin changeuserauth -m admin testcm
```

Changing the CM User Password

The **cm_admin changeuserpwd** utility changes the password of a CM user.

```
cm_admin changeuserpwd [options] cmuser-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **changeuserpwd**: A command to change the password of a CM user
- *cmuser-name*: The name of a CM user whose password to be changed

The following is [options] of **cm_admin changeuserpwd**.

-o, --oldpass PASSWORD

Specifies the existing password of a CM user.

The following example shows how to change a password of a CM user named *testcm* .

```
cm_admin changeuserpwd -o old_password -n new_password
testcm
```

--adminpass PASSWORD

The password of an admin user can be specified instead of old CM user's password that you don't know.

The following example shows how to change a password of a CM user named *testcm* by using an admin password.

```
cm_admin changeuserauth --adminpass admin_password -n
new_password testcm
```

-n, --newpass PASSWORD

Specifies a new password of a CM user.

Adding Database Information to CM Users

The **cm_admin adddbinfo** utility adds database information (database name, UID, broker IP, and broker port) to a CM user.

```
cm_admin adddbinfo options cmuser-name database-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **adddbinfo**: A command to add database information to a CM user
- *cmuser-name*: CM username
- *database-name*: The name of a database to be added

The following example shows how to add a database without specifying any user-defined values to a CM user named *testcm* .

```
cm_admin adddbinfo testcm testdb
```

The following is [options] of **cm_admin adddbinfo**.

-u, --uid ID

Specifies the ID of a database user to be added. The default value is **dba**.

The following example shows how to add a database whose name is *testdb* and user ID is *cubriduser* to a CM user named *testcm*.

```
cm_admin adddbinfo -u cubriduser testcm testdb
```

-h, --host IP

Specifies the host IP of a broker used when clients access a database. The default value is **localhost**.

The following example shows how to add a database whose name is *testdb* and the host IP of is *127.0.0.1* to a CM user named *testcm*.

```
cm_admin adddbinfo -h 127.0.0.1 testcm testdb
```

-p, --port NUMBER

Specifies the port number of a broker used when clients access a database. The default value: **30000**.

Deleting database information from CM Users

The **cm_admin deldbinfo** utility deletes database information of a specified CM user.

```
cm_admin deldbinfo cmuser-name database-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **deldbinfo**: A command to delete database information of a CM user
- *cmuser-name*: CM username
- *database-name*: The name of a database to be deleted

The following example shows how to delete database information whose name is *testdb* from a CM user named *testcm*.

```
cm_admin deldbinfo testcm testdb
```

Changing Database Information of a CM user

The **cm_admin changedbinfo** utility changes database information of a specified CM user.

```
cm_admin changedbinfo [options] cmuser-name database-name
```

- **cm_admin**: An integrated utility to manage CUBRID Manager
- **changedbinfo**: A command to change database information of a CM user
- *cmuser-name*: CM username
- *database-name*: The name of a database to be changed

The following is [options] of **cm_admin changedbinfo**.

-u, --uid ID

Specifies the ID of a database user.

The following example shows how to update user ID information to *uid* in the *testdb* database which belongs to a CM user named *testcm* .

```
cm_admin changedbinfo -u uid testcm testdb
```

-h, --host IP

Specifies the host of a broker used when clients access a database.

The following example shows how to update host IP information to *10.34.63.132* in the *testdb* database which belongs to a CM user named *testcm* .

```
cm_admin changedbinfo -h 10.34.63.132 testcm testdb
```

-p, --port NUMBER

Specifies the port number of a broker used when clients access a database.

The following example shows how to update broker port information to *33000* in the *testdb* database which belongs to a CM user named *testcm* .

```
cm_admin changedbinfo -p 33000 testcm testdb
```

CUBRID Procedural Langauge Server

Starting CUBRID Procedural Langauge Server

The CUBRID Procedural Langauge Server automatically starts when the database server starts and stops when the database server stops.

If you do not want to start the Procedural Langauge Server with the database server, set the **stored_procedure** value to **no** for the corresponding database in the CUBRID configuration file (**cubrid.conf**).

```
% cubrid server start demodb

@ cubrid pl status
```

For more details on other types of errors that may occur when starting the CUBRID Procedural Langauge Server, see CUBRID Procedural Langauge Errors.

Restarting CUBRID Procedural Langauge Server

The following is a method to restart the Procedural Langauge Server for *demodb*. When the restart command for the Procedural Langauge Server is issued, the Procedural Langauge Server stops and is restarted by the database server. If the database server is not running, the restart command does not work.

```
% cubrid pl restart demodb

@ cubrid pl restart: demodb
++ cubrid pl restart: success
```

..note:

```
Since the database server automatically starts the Procedural
Langauge Server, the restart command is not frequently used.
It can be used to stabilize the server when the GC of the Procedural
Langauge Server's VM is slow after performing large I/O operations
or analysis tasks using procedures.
```

⚠ Warning

Restarting the Procedural Langauge Server will interrupt and roll back any currently running transactions.

Checking the Status of CUBRID Procedural Langauge Server

The following is an example of checking the status of the CUBRID Procedural Langauge Server for *demodb*. The name of the target database, *demodb*, on which the Procedural Langauge Server is currently running, is displayed. Additionally, the server's PID, port number, and applied JVM options are displayed.

```
% cubrid pl status demodb

@ cubrid pl status: demodb
Procedural Langauge Server (demodb, pid 9220, UDS)
VM arguments :
--------------------------------------------------
...
--------------------------------------------------
```

Procedural Langauge Server Configuration

Procedural Language Environment: Using Bundled JDK

To use the procedural language feature in CUBRID, a JVM environment is used. The CUBRID server includes the Eclipse Temurin JDK 8 version in the $CUBRID/vm directory during installation, so there is no need to configure a separate JDK environment in the user's environment. This bundled JDK has been verified for use with the procedural language feature in CUBRID and is generally recommended for good performance and stability. To use a JDK configured in the user's environment instead of the bundled JDK, set the **CUBRID_JAVA_HOME** environment variable.

CUBRID_JAVA_HOME

The **CUBRID_JAVA_HOME** environment variable specifies the JDK installation directory. If the user does not specify the **CUBRID_JAVA_HOME** environment variable, the bundled JDK included during CUBRID installation is used. If an incorrect **CUBRID_JAVA_HOME** is specified, the procedural language server may not start properly and may return an error. The path of **CUBRID_JAVA_HOME** must include the JDK environment along with the libjvm library.

The following is an example of setting the **CUBRID_JAVA_HOME** environment variable.

```
**Example of setting in Windows environment**

    set CUBRID_JAVA_HOME=C:\Program Files\Java\jdk1.8.0_421

**Example of setting in Linux environment**

    export CUBRID_JAVA_HOME=/usr/java/jdk1.8.0_421
```

Procedural Langauge Server System Parameters

The following table lists the server parameters related to the Procedural Langauge Server that can be set in the configuration file (**cubrid.conf**).

Parameter Name	Type	Default Value	Min	Max
stored_procedure	bool	yes		
stored_procedure_uds	bool	yes		

stored_procedure_port	int	0	0	65535
stored_procedure_vm_options	string			

For more details on these parameters, see cubrid.conf Configuration File and Default Parameters.

CUBRID Procedural Langauge Server Logs

The logs of the CUBRID Procedural Langauge Server are stored in the **log/** directory of the installation directory. The following log files are created for each database.

- Error log ($CUBRID/log/[db_name]_pl.err)
- Java log ($CUBRID/log/[db_name]_pl.log)

Error Log

The error log of the Procedural Langauge Server for each database is stored in the **$CUBRID/log** directory, and the filename is in the format **<db_name>_pl.err**. The extension is **.err**.

```
demodb_pl.err
```

If an error occurs while starting the Procedural Langauge Server, the error message is saved in the error log file.

```
Time: 11/11/20 18:17:15.438 - ERROR *** file ../../src/jsp/jsp_sr.c,
line 501 ERROR CODE = -900, Tran = -1, EID = 1
Cannot find Java Virtual Machine library:
    Failed to get 'JVM_PATH' environment variable.
    Failed to load libjvm from 'JAVA_HOME' environment variable:
        /jre/lib/amd64/server/libjvm.so: cannot open shared object
file: No such file or directory
        /lib/server/libjvm.so: cannot open shared object file: No
such file or directory.
```

Note

For more details on what errors can occur, see CUBRID Procedural Langauge Errors.

Exception Log

The JVM exception messages that occur during the execution of procedures are saved. The exception log of the Procedural Langauge Server for each database is stored in the **$CUBRID/log** directory, and the filename is in the format **<db_name>_pl.log**. The extension is **.log**.

```
demodb_pl.log
```

If an exception occurs while executing stored procedures/functions in the VM, the exception string is saved in the exception log.

```
SEVERE:
java.lang.NullPointerException
at Test.testFunction(Test.java:50)
...
at com.cubrid.jsp.StoredProcedure.invoke(StoredProcedure.java:263)
at com.cubrid.jsp.ExecuteThread.run(ExecuteThread.java:197)
```

CUBRID Procedural Langauge Errors

The following are error messages for errors that may occur when starting the CUBRID Procedural Langauge Server. The error messages are stored in **$CUBRID/log**/*<db_name>_pl*.**err**.

Error Code	Error Message	Description	Action
-900	VM library is not found: %1$s.	CUBRID cannot find the JVM library from the CUBRID_JAVA_HOME	Check if the CUBRID_JAVA_HOME is set correctly. Refer to Procedural Language Environment: Using Bundled JDK.
-901	PL server can not be started: %1$s.	An unexpected error occurred within the JVM library or in	Try reinstalling the JRE, and if the same error occurs, try installing a different

Error Code	Error Message	Description	Action
		$CUBRID/java/jspserver.jar.	version of the JRE. Also, replace the $CUBRID/java/jspserver.jar file with the one from the same version of CUBRID.

The following are error messages for errors that may occur when there are connection issues, including when the CUBRID Procedural Langauge Server is not started. The error messages are stored in **$CUBRID/log/broker/error_log**/*<broker_name>_<app_server_num>***.err**.

Error Code	Error Message	Description	Action
-902	PL server is not running.	The Procedural Langauge Server is not started	Start the Procedural Langauge Server with the **cubrid pl start <db_name>** command. For more details, refer to CUBRID Procedural Langauge Server.
-903	Can't connect PL server: %1$s	The Procedural Langauge Server cannot connect from CAS This error can occur for various reasons. For example, if the Procedural Langauge Server is unstable or if CAS cannot connect to the Procedural Langauge Server, or if the Procedural Langauge	Restart the Procedural Langauge Server. If the restart fails, forcibly terminate the **cub_pl <db_name>** process using the Linux **kill** command. Then restart the Procedural Langauge Server again.

Error Code	Error Message	Description	Action
		unexpectedly terminates (kill) this error message is displayed.	Check if the Procedural Langauge Server's port is accessible from CAS using the **cubrid pl status <db_name>** command. The port may be blocked by a firewall, so open the port in the firewall. If necessary, set the **stored_procedure_port** parameter and restart the Procedural Langauge Server. For more details, refer to Port Setting.
-905	Networking with PL server failed: %1$d	CAS received an incorrect packet from the Procedure Language Server	

Database Management

Database Users

A CUBRID database user can have members with the same authorization. If authorization **A** is granted to a user, the same authorization is also granted to all members belonging to the user. A database user and its members are called a "group."; a user who has no members is called a "user."

CUBRID provides **DBA** and **PUBLIC** users by default.

- **DBA** can access every object in the database, that is, it has authorization at the highest level. Only **DBA** has sufficient authorization to add, alter and delete the database users.

- All users including **DBA** are members of **PUBLIC**. Therefore, all database users have the authorization granted to **PUBLIC** . For example, if authorization **B** is added to **PUBLIC** group, all database members will automatically have the **B** authorization.

databases.txt File

CUBRID stores information on the locations of all existing databases in the **databases.txt** file. This file is called the "database location file". A database location file is used when CUBRID executes utilities for creating, renaming, deleting or replicating databases; it is also used when CUBRID runs each database. By default, this file is located in the **databases** directory under the installation directory. The directory is located through the environment variable **CUBRID_DATABASES**.

```
db_name db_directory server_host logfile_directory
```

The format of each line of a database location file is the same as defined by the above syntax; it contains information on the database name, database path, server host and the path to the log files. The following example shows how to check the contents of a database location file.

```
% more databases.txt

dist_testdb /home1/user/CUBRID/bin d85007 /home1/user/CUBRID/bin
dist_demodb /home1/user/CUBRID/bin d85007 /home1/user/CUBRID/bin
testdb /home1/user/CUBRID/databases/testdb d85007 /home1/user/CUBRID/
databases/testdb
demodb /home1/user/CUBRID/databases/demodb d85007 /home1/user/CUBRID/
databases/demodb
```

By default, the database location file is stored in the **databases** directory under the installation directory. You can change the default directory by modifying the value of the **CUBRID_DATABASES** environment variable. The path to the database location file must be valid so that the **cubrid** utility for database management can access the file properly. You must enter the directory path correctly and check if you have write permission on the file. The following example shows how to check the value configured in the **CUBRID_DATABASES** environment variable.

```
% set | grep CUBRID_DATABASES
CUBRID_DATABASES=/home1/user/CUBRID/databases
```

An error occurs if an invalid directory path is set in the **CUBRID_DATABASES** environment variable. If the directory path is valid but the database location file does not exist, a new location information file is created. If the **CUBRID_DATABASES** environment variable has not been configured at all, CUBRID retrieves the location information file in the current working directory.

Database Volume

The volumes of CUBRID database are classified as permanent volume, temporary volume and backup volume.

- In the permanent volumes,
  - there are data volumes that usually store permanent data, but can also store temporary data.
  - there is the active log volume that includes late changes of database.
- In the temporary volumes,
  - there are temporary temp volumes that store temporary data.
  - there are archive log volumes that store exhausted logs from the active log.
  - there is one background archive log volume that stores archive log in the background.

For more details on volumes, see Database Volume Structure.

The following is an example of files related to the database when *testdb* database operates.

Filename	Size	Purpose	Classification	Description
testdb	512MB	permanent data	Database volume	The firstly created volume when DB is created. This volume stores permanent data (system, heap and index files). This volume includes database meta information. **cubrid createdb** uses by default the size specified by **db_volume_size** in **cubrid.conf**.
testdb_perm	512MB	permanent data		Manually added volume using **cubrid addvoldb** utility This volume stores permanent data (system, heap and index files).
testdb_temp	512MB	temporary data		Manually added volume using **cubrid addvoldb** utility This volume stores temporary data (query results,

Filename	Size	Purpose	Classification	Description
				list files, sort files, join object hashes).
testdb_x003	512MB	permanent data		Automatically created when database requires more space. This volume stores permanent data (system, heap and index files).
testdb_x004	512MB	permanent data		Automatically created when database requires more space. This volume stores permanent data (system, heap and index files).
testdb_x005	512MB	permanent data		Automatically created when database requires more space. This volume stores permanent data (system, heap and index files).

Filename	Size	Purpose	Classification	Description
testdb_x006	64MB	permanent data		Automatically created when database requires more space. This volume stores permanent data (system, heap and index files). The size of volume is not maximized (yet).
testdb_t32766	512MB	temporary data	Temporary Volume	Automatically created when database requires more space. This volume stores temporary data (query results, list files, sort files, join object hashes).
testdb_lgar_t	512MB	background archiving	Log volume	A log file which is related to the background archiving feature. This is used when storing the archiving log.
		archive		

Filename	Size	Purpose	Classification	Description
testdb_lgar224	512MB			Archiving logs are continuously archived and the files ending with three digits are created. At this time, archiving logs from 001~223 seem to be removed normally by **cubrid backupdb** -r option or the setting of **log_max_archives** in **cubrid.conf**. When archiving logs are removed, you can see the removed archiving log numbers in the REMOVE section of lginf file. See Managing Archive Logs.
testdb_lgat	512MB	active		Active log file
testdb_dwb	1MB	temporary data	Double write buffer	Double write buffer storage file, where

Filename	Size	Purpose	Classification	Description
				flushed pages are written first.

- Database volume file
  - In the table above, *testdb*, *testdb_perm*, *testdb_temp*, *testdb_x003 ~ testdb_x006* are classified as the database volume files.
  - File size is determined by **db_volume_size** in **cubrid.conf** or the **--db-volume-size** option of **cubrid createdb** and **cubrid addvoldb**.
  - When database remains out of space, it automatically expands existing volumes and creates new volumes.
- Temporary volume
  - Temporary volumes are usually used to store temporary data. They are automatically created and destroyed by database.
  - File size is determined by **db_volume_size** in **cubrid.conf**.
- Log volume file
  - In the above, *testdb_lgar_t*, *testdb_lgar224* and *testdb_lgat* are classified as the log volume files.
  - File size is determined by **log_volume_size** in **cubrid.conf** or the **--log-volume-size** option of **cubrid createdb**.
- Double write buffer file * Double write buffer file is a storage area used to protect against I/O errors (partial writes). * Every data page write is first written into the buffer and then flushed to its location in the permanent data volumes. * During database reboot, partially written page is detected and replaced with the counterpart page in double write buffer. * The file size is determined by **double_write_buffer_size** in **cubrid.conf**. If set to zero, no file is created and double write buffer is disabled.

Note

Any data that has to be persistent over database restart and crash is stored in the database volumes created for permanent data purpose. The volumes store table rows (heap files), indexes (b-tree files) and several system files.

Intermediate and final results of query processing and sorting need only temporary storage. Based on the size of required temporary data, it will be first stored in memory (the space size is determined by the system parameter **temp_file_memory_size_in_pages** specified in **cubrid.conf**). Exceeding data has to be stored on disk.

Database will usually create and use temporary volumes to allocate disk space for temporary data. Administrator may however assign permanent database volumes with the purpose of storing temporary data using by running **cubrid addvoldb -p temp** command. If such volumes exist, they will have priority over temporary volumes when disk space is allocated for temporary data.

The examples of queries that can use temporary data are as follows:

- Queries creating the resultset like **SELECT**
- Queries including **GROUP BY** or **ORDER BY**
- Queries including a subquery
- Queries executing sort-merge join
- Queries including the **CREATE INDEX** statement

To have complete control on the disk space used for temporary data and to prevent it from consuming all system disk space, our recommendation is to:

- create permanent database volumes in advance to secure the required space for temporary data
- limit the size of the space used in the temporary volumes when a queries are executed by setting **temp_file_max_size_in_pages** parameter in **cubrid.conf** (there is no limit by default).

cubrid Utilities

The following shows how to use the cubrid management utilities.

```
cubrid utility_name
utility_name:
    createdb [option] <database_name> <locale_name>  --- Creating a
database
    deletedb [option] <database_name>   --- Deleting a database
```

```
    installdb [option] <database-name>   --- Installing a database
    renamedb [option] <source-database-name> <target-database-name>
--- Renaming a database
    copydb [option] <source-database-name> <target-database-name>
--- Copying a database
    backupdb [option] <database-name>  --- Backing up a database
    restoredb [option] <database-name>  --- Restoring a database
    addvoldb [option] <database-name>  --- Adding a database volume
file
    spacedb [option] <database-name>  --- Displaying details of
database space
    lockdb [option] <database-name>  --- Displaying details of
database lock
    tranlist [option] <database-name>  --- Checking transactions
    killtran [option] <database-name>  --- Removing transactions
    optimizedb [option] <database-name>  --- Updating database
statistics
    statdump [option] <database-name>  --- Dumping statistic
information of database server execution
    compactdb [option] <database-name>  --- Optimizing space by
freeing unused space
    diagdb [option] <database-name>  --- Displaying internal
information
    checkdb [option] <database-name>  --- Checking database
consistency
    alterdbhost [option] <database-name>  --- Altering database host
    plandump [option] <database-name>  --- Displaying details of the
query plan
    loaddb [option] <database-name>  --- Loading data and schema
    unloaddb [option] <database-name>  --- Unloading data and schema
    paramdump [option] <database-name>  --- Checking out the
parameter values configured in a database
    changemode [option] <database-name>  --- Displaying or changing
the server HA mode
    applyinfo [option] <database-name>   --- Displaying the status
of being applied transaction log to the other node in HA replication
environment
    synccolldb [option] <database-name>  --- Synchronizing the DB
collation with the system collation
    genlocale [option] <database-name>  --- Compiling the locale
information to use
    dumplocale [option] <database-name>   --- Printing human
readable text for the compiled binary locale information
    gen_tz [option] [<database-name>]  --- Generates C source file
containing timezone data ready to be compiled into a shared library
    dump_tz [option]  --- Displaying timezone related information
    tde <operation> [option] <database-name> --- Managing
Transparent Data Encryption (TDE)
    vacuumdb [option] <database-name>  --- Vacuuming deleted records
or unnecessary mvcc related information from records in the database
    flashback [option] <database-name> <owner_name.class_name> ---
Provides statements to rewind a specific transaction.
```

```
    memmon [option] <database-name> --- Provides information of
current heap memory usage on the server.
```

cubrid Utility Logging

CUBRID supports logging feature for the execution result of **cubrid** utilities; for details, see cubrid Utility Logging.

createdb

The **cubrid createdb** utility creates databases and initializes them with the built-in CUBRID system tables. It can also define initial users to be authorized in the database and specify the locations of the logs and databases. In general, the **cubrid createdb** utility is used only by DBA.

> **⚠ Warning**
>
> When you create database, a locale name and a charset name after a DB name must be specified(e.g. ko_KR.utf8). It affects the length of string type, string comparison operation, etc. The specified charset when creating database cannot be changed later, so you should be careful when specifying it.
>
> For charset, locale and collation setting, see An Overview of Globalization.

```
cubrid createdb [options] database_name locale_name.charset
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **createdb**: A command used to create a new database.
- *database_name*: Specifies a unique name for the database to be created, without including the path name to the directory where the database will be created. If the specified database name is the same as that of an existing database name, CUBRID halts creation of the database to protect existing files.
- *locale_name*: A locale name to use in the database should be input. For a locale name which can be used in CUBRID, refer to Step 1: Selecting a Locale.
- *charset*: A characterset to use in the database should be input. A character set which can be used in CUBRID is iso88591, euckr or utf8.
    - If *locale_name* is en_US and *charset* is omitted, a character set will be iso88591.

- If *locale_name* is ko_KR and *charset* is omitted, a character set will be utf8.
- All locale names except en_US and ko_KR cannot omit *charset*, and a *charset* can be specified only with utf8.

The maximum length of database name is 17 in English.

The following shows [options] available with the **cubrid createdb** utility.

```
    --db-volume-size=SIZE           size of first information volume
    --db-page-size=SIZE             database page SIZE in bytes
    --log-volume-size=SIZE          size of log archives
    --log-page-size=SIZE            log page SIZE in bytes
    --comment=COMMENT               COMMENT for a database
-F, --file-path=PATH                directory PATH for database
volumes
-L, --log-path=PATH                 directory PATH for log volumes
-B, --lob-base-path=PATH            base directory PATH for external
LOB data
    --server-name=HOST              server's HOST name
-r, --replace                       replace an existing database
    --more-volume-file=FILE         FILE contain specifications for
creation of additional volumes
    --user-definition-file=FILE     FILE contain user definitions
    --csql-initialization-file=FILE FILE contain SQLs for an
initialization of the database
-o, --output-file=FILE              redirect output messages to FILE
-v, --verbose                       enable verbose status messages
```

--db-volume-size=SIZE

This option specifies the size of the database volume that will be created first. The default value is the value of the system parameter **db_volume_size**. You can set units as K, M, G and T, which stand for kilobytes (KB), megabytes (MB), gigabytes (GB), and terabytes (TB) respectively; if you omit the unit, bytes will be applied. The size of the database is always rounded up to 64 disk sectors, which depends on the size of a page and can be 16M, 32M or 64M for page size 4k, 8k and 16k respectively.

The following example shows how to create a database named *testdb* and assign 512 MB to its first volume.

```
cubrid createdb --db-volume-size=512M testdb en_US
```

--db-page-size=SIZE

This option specifies the size of the database page; the minimum value is 4K and the maximum value is **16K** (default). K stands for kilobytes (KB). The value of page size is one of

the following: 4K, 8K, or 16K. If a value between 4K and 16K is specified, system rounds up the number. If a value greater than 16K or less than 4K, the specified number is used.

The following example shows how to create a database named *testdb* and configure its page size 16K.

```
cubrid createdb --db-page-size=16K testdb en_US
```

--log-volume-size=SIZE

This option specifies the size of the database log volume. The default value is the same as database volume size, and the minimum value is 20M. You can set units as K, M, G and T, which stand for kilobytes (KB), megabytes (MB), gigabytes (GB), and terabytes (TB) respectively. If you omit the unit, bytes will be applied.

The following example shows how to create a database named *testdb* and assign 256 MB to its log volume.

```
cubrid createdb --log-volume-size=256M testdb en_US
```

--log-page-size=SIZE

This option specifies the size of the log volume page. The default value is the same as data page size. The minimum value is 4K and the maximum value is 16K. K stands for kilobytes (KB). The value of page size is one of the following: 4K, 8K, or 16K. If a value between 4K and 16K is specified, system rounds up the number. If a value greater than 16K or less than 4K, the specified number is used.

The following example shows how to create a database named *testdb* and configure its log volume page size 8K.

```
cubrid createdb --log-page-size=8K testdb en_US
```

--comment=COMMENT

This option specifies a comment to be included in the database volume header. If the character string contains spaces, the comment must be enclosed in double quotes.

The following example shows how to create a database named *testdb* and add a comment to the database volume.

```
cubrid createdb --comment "a new database for study" testdb en_US
```

-F, --file-path=PATH

The **-F** option specifies an absolute path to a directory where the new database will be created. If the **-F** option is not specified, the new database is created in the current working directory.

The following example shows how to create a database named *testdb* in the directory /dbtemp/new_db.

```
cubrid createdb -F "/dbtemp/new_db/" testdb en_US
```

-L, --log-path=PATH

The **-L** option specifies an absolute path to the directory where database log files are created. If the **-L** option is not specified, log files are created in the directory specified by the **-F** option. If neither **-F** nor **-L** option is specified, database log files are created in the current working directory.

The following example shows how to create a database named *testdb* in the directory /dbtemp/newdb and log files in the directory /dbtemp/db_log.

```
cubrid createdb -F "/dbtemp/new_db/" -L "/dbtemp/db_log/" testdb
en_US
```

-B, --lob-base-path=PATH

This option specifies a directory where **LOB** data files are stored when **BLOB/CLOB** data is used. If the **--lob-base-path** option is not specified, LOB data files are store in <*location of database volumes created*>/**lob** directory.

The following example shows how to create a database named *testdb* in the working directory and specify /home/data1 of local file system as a location of LOB data files.

```
cubrid createdb --lob-base-path "file:/home/data1" testdb en_US
```

--server-name=HOST

This option enables the server of a specific database to run in the specified host when CUBRID client/server is used. The information of a host specified is stored in the **databases.txt** file. If this option is not specified, the current localhost is specified by default.

The following example shows how to create a database named *testdb* and register it on the host *aa_host*.

```
cubrid createdb --server-name aa_host testdb en_US
```

-r, --replace

This option creates a new database and overwrites an existing database if one with the same name exists.

The following example shows how to create a new database named *testdb* and overwrite the existing database with the same name.

```
cubrid createdb -r testdb en_US
```

--more-volume-file=FILE

This option creates an additional volume based on the specification contained in the file specified by the option. The volume is created in the same directory where the database is created. Instead of using this option, you can add a volume by using the **cubrid addvoldb** utility.

The following example shows how to create a database named *testdb* as well as an additional volume based on the specification stored in the **vol_info.txt** file.

```
cubrid createdb --more-volume-file vol_info.txt testdb en_US
```

The following is a specification of the additional volume contained in the **vol_info.txt** file. The specification of each volume must be written on a single line.

```
#xxxx
```

- *volcmnts*: Comment to be written in the volume header. It contains information on the additional volume to be created. The specification of the comment on a volume can also be omitted.

- *volpurp*: The purpose for which the volume will be used. It can be either permanent data (default option) or temporary.

> **Note**
>
> For backward compatibility, all old keywords, **data**, **index**, **temp**, or **generic** are accepted. **temp** stands for temporary data purpose, while the rest stand for permanent data purpose.

- *volnpgs*: The number of pages of the additional volume to be created. The specification of the number of pages of the volume cannot be omitted; it must be specified. The actual volume size is rounded up to the next multiple of **64 sectors**.

--user-definition-file=FILE

This option adds users who have access to the database to be created. It adds a user based on the specification contained in the user information file specified by the parameter. Instead of using the **--user-definition-file** option, you can add a user by using the **CREATE USER** statement (for details, see CREATE USER).

The following example shows how to create a database named *testdb* and add users to *testdb* based on the user information defined in the **user_info.txt** file.

```
cubrid createdb --user-definition-file=user_info.txt testdb en_US
```

The syntax of a user information file is as follows:

```
USER user_name [ <groups_clause> | <members_clause> ]

<groups_clause>:
    [ GROUPS <group_name> [ { <group_name> }... ] ]

<members_clause>:
    [ MEMBERS <member_name> [ { <member_name> }... ] ]
```

- The *user_name* is the name of the user who has access to the database. It must not include spaces.

- The **GROUPS** clause is optional. The *group_name* is the upper level group that contains the *user_name* . Here, the *group_name* can be multiply specified and must be defined as **USER** in advance.

- The **MEMBERS** clause is optional. The *member_name* is the name of the lower level member that belongs to the *user_name* . Here, the *member_name* can be multiply specified and must be defined as **USER** in advance.

Comments can be used in a user information file. A comment line must begin with a consecutive hyphen lines (--). Blank lines are ignored.

The following example shows a user information in which *grandeur* and *sonata* are included in *sedan* group, *tuscan* is included in *suv* group, and *i30* is included in *hatchback* group. The name of the user information file is **user_info.txt**.

```
--
-- Example 1 of a user information file
--
USER sedan
USER suv
USER hatchback
USER grandeur GROUPS sedan
USER sonata GROUPS sedan
USER tuscan GROUPS suv
USER i30 GROUPS hatchback
```

The following example shows a file that has the same user relationship information as the file above. The difference is that the **MEMBERS** statement is used in the file below.

```
--
-- Example 2 of a user information file
--
USER grandeur
USER sonata
USER tuscan
USER i30
USER sedan MEMBERS sonata grandeur
USER suv MEMBERS tuscan
USER hatchback MEMBERS i30
```

--csql-initialization-file=FILE

This option executes an SQL statement on the database to be created by using the CSQL Interpreter. A schema can be created based on the SQL statement contained in the file specified by the parameter.

The following example shows how to create a database named *testdb* and execute the SQL statement defined in table_schema.sql through the CSQL Interpreter.

```
cubrid createdb --csql-initialization-file table_schema.sql
testdb en_US
```

-o, --output-file=FILE

This option stores messages related to the database creation to the file given as a parameter. The file is created in the same directory where the database was created. If the **-o** option is not specified, messages are displayed on the console screen. The **-o** option allows you to use information on the creation of a certain database by storing messages, generated during the database creation, to a specified file.

The following example shows how to create a database named *testdb* and store the output of the utility to the **db_output** file instead of displaying it on the console screen.

```
cubrid createdb -o db_output testdb en_US
```

-v, --verbose

This option displays all information on the database creation operation onto the screen. Like the **-o** option, this option is useful in checking information related to the creation of a specific database. Therefore, if you specify the **-v** option together with the **-o** option, you can store the output messages in the file given as a parameter; the messages contain the operation information about the **cubrid createdb** utility and database creation process.

The following example shows how to create a database named *testdb* and display detailed information on the operation onto the screen.

```
cubrid createdb -v testdb en_US
```

Note

- **temp_file_max_size_in_pages** is a parameter used to configure the maximum size of temporary volumes - used for complicated queries or storing arrays - on the disk. With the default value **-1**, the temporary volumes size is only limited by the capacity of the disk specified by the **temp_volume_path** parameter. If the value is 0, no temporary volumes can be created. In this case, a permanent volume with temporary data purpose should be added by using the cubrid addvoldb utility. For an efficient storage management, it is recommended to use the latter approach.

- By using the cubrid spacedb utility, you can check the remaining space of each volume. By using the cubrid addvoldb utility, you can add more volumes as needed while managing the database. You are advised to add more volumes when there is less system load. When all preassigned volumes are completely in use, the database system automatically creates new volumes.

The following example shows how to create a database, with additional volumes, including one for temporary data purpose.

```
cubrid createdb --db-volume-size=512M --log-volume-size=256M
cubriddb en_US
cubrid addvoldb -S -n cubriddb_DATA01 --db-volume-size=512M cubriddb
cubrid addvoldb -S -p temp -n cubriddb_TEMP01 --db-volume-size=512M
cubriddb
```

Note

Creating a database using an existing key file

When the database is created, a key file is created together by default. If you want to use an existing key file when creating a database:

1. Copy the key file with the name **<database-name>_keys**.
2. Specify the directory path of the copied key file by the system parameter **tde_keys_file_path**.
3. Create a database by using the **createdb utility**.

For more information on the TDE key file, see File-based Master Key Management.

addvoldb

If you want to micromanage CUBRID storage volumes, addvoldb is the tool for you. You can finely tune each filename, path, purpose, and size. The database system can handle all storage by itself, but it uses default values to configure each new volume.

The command for manually adding a database volume is as follows.

```
cubrid addvoldb [options] database_name
```

- **cubrid**: An integrated utility for CUBRID service and database management.
- **addvoldb**: A command that adds a specified number of pages of the new volume to a specified database.
- *database_name*: Specifies the name of the database to which a volume is to be added without including the path name to the directory where the database is to be created.

The following example shows how to create a database, with additional multi-purpose volumes.

```
cubrid createdb --db-volume-size=512M --log-volume-size=256M
cubriddb en_US
cubrid addvoldb -S -n cubriddb_DATA01 --db-volume-size=512M cubriddb
cubrid addvoldb -S -p temp -n cubriddb_TEMP01 --db-volume-size=512M
cubriddb
```

The following shows [options] available with the **cubrid addvoldb** utility.

```
    --db-volume-size=SIZE        size of additional volume
-n, --volume-name=NAME           NAME of information volume
-F, --file-path=PATH             PATH for adding volume file
    --comment=COMMENT            COMMENT for adding volume file
-p, --purpose=PURPOSE            PURPOSE for adding volume file
-S, --SA-mode                    stand-alone mode execution
-C, --CS-mode                    client-server mode execution
    --max-writesize-in-sec=SIZE  the amount of volume written per
second
```

--db-volume-size=SIZE

--db-volume-size is an option that specifies the size of the volume to be added to a specified database. If the **--db-volume-size** option is omitted, the value of the system parameter **db_volume_size** is used by default. You can set units as K, M, G and T, which stand for kilobytes (KB), megabytes (MB), gigabytes (GB), and terabytes (TB) respectively. If you omit the unit, bytes will be applied. The size of the database is always rounded up to 64 disk sectors, which depends on the size of a page and can be 16M, 32M or 64M for page size 4k, 8k and 16k respectively.

The following example shows how to add a volume for which 256 MB are assigned to the *testdb* database.

```
cubrid addvoldb --db-volume-size=256M testdb
```

-n, --volume-name=NAME

This option specifies the name of the volume to be added to a specified database. The volume name must follow the filename protocol of the operating system and be a simple one without including the directory path or spaces. If the **-n** option is omitted, the name of the volume to be added is configured by the system automatically as "database name_volume identifier". For example, if the database name is *testdb*, the volume name *testdb_x001* is automatically configured.

The following example shows how to specify a different name, *testdb_v1*, to newly added volume.

```
cubrid addvoldb -n testdb_v1 testdb
```

-F, --file-path=PATH

This option specifies the directory path where the volume to be added will be stored. If the **-F** option is omitted, the value of the system parameter **volume_extension_path** is used by default.

The following example shows how to add a volume in the */dbtemp/addvol* directory. Since the **-n** option is not specified for the volume name, the volume name *testdb_x001* will be created.

```
cubrid addvoldb -F /dbtemp/addvol/ testdb
```

--comment COMMENT

This option facilitates to retrieve information on the added volume by adding such information in the form of comments. It is recommended that the contents of a comment include the name of **DBA** who adds the volume, or the purpose of adding the volume. The comment must be enclosed in double quotes.

The following example shows how to add a volume and inserts a comment with additional information.

```
cubrid addvoldb --comment "Data volume added by cheolsoo kim
because permanent data space was almost depleted." testdb
```

-p, --purpose=PURPOSE

This option specifies the purpose of the volume to be added. The purpose defines the type of files that will be stored in added volume:

- **PERMANENT DATA** to store table rows, indexes and system files.
- **TEMPORARY DATA** to store intermediate and final results of query processing and sorting.

If not specified, the purpose of the volume is by default considered **PERMANENT DATA**. The following example shows how to change it to temporary.

```
cubrid addvoldb -p temp testdb
```

Note

PERMANENT DATA volumes used to be classified as generic, data and index. The design of volumes has been changed, and since then the classification no longer exists. In order to avoid invalidating your old scripts, we chose to keep the keywords as valid options, but their effect will be the same. The only remaining option value with a real effect is temp.

For detailed information on each purpose, see Database Volume Structure.

-S, --SA-mode

This option accesses the database in standalone mode without running the server process. This option has no parameter. If the **-S** option is not specified, the system assumes to be in client/server mode.

```
cubrid addvoldb -S --db-volume-size=256M testdb
```

-C, --CS-mode

This option accesses the database in client/server mode by running the server and the client separately. There is no parameter. Even when the **-C** option is not specified, the system assumes to be in client/server mode by default.

```
cubrid addvoldb -C --db-volume-size=256M testdb
```

--max-writesize-in-sec=SIZE

The **--max-writesize-in-sec** is used to limit the impact of system operating when you add a volume to the database. This can limit the maximum writing size per second. The unit of this option is K(kilobytes) and M(megabytes). The minimum value is 160K. If you set this value as less than 160K, it is changed as 160K. It can be used only in client/server mode.

The below is an example to limit the writing size of the 2GB volume as 1MB. Consuming time will be about 35 minutes(= (2048MB/1MB) /60 sec.).

```
cubrid addvoldb -C --db-volume-size=2G --max-writesize-in-sec=1M
testdb
```

deletedb

The **cubrid deletedb** utility is used to delete a database. You must use the **cubrid deletedb** utility to delete a database, instead of using the file deletion commands of the operating system; a database consists of a few interdependent files.

The **cubrid deletedb** utility also deletes the information on the database from the database location file (**databases.txt**). The **cubrid deletedb** utility must be run offline, that is, in standalone mode when nobody is using the database.

```
cubrid deletedb  [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **deletedb**: A command to delete a database, its related data, logs and all backup files. It can be executed successfully only when the database is in a stopped state.
- *database_name*: Specifies the name of the database to be deleted without including the path name.

The following shows [options] available with the **cubrid deletedb** utility.

```
-o, --output-file=FILE       redirect output messages to FILE
-d, --delete-backup          delete backup volumes
```

-o, --output-file=FILE

This option specifies the filename for writing messages:

```
cubrid deletedb -o deleted_db.out testdb
```

The **cubrid deletedb** utility also deletes the database information contained in the database location file (**databases.txt**). The following message is returned if you enter a utility that tries to delete a non-existing database.

```
cubrid deletedb testdb
Database "testdb" is unknown, or the file "databases.txt" cannot
be accessed.
```

-d, --delete-backup

This option deletes database volumes, backup volumes and backup information files simultaneously. If the -**d** option is not specified, backup volume and backup information files are not deleted.

```
cubrid deletedb -d testdb
```

renamedb

The **cubrid renamedb** utility renames a database. The names of information volumes, log volumes and control files are also renamed to conform to the new database one.

In contrast, the **cubrid alterdbhost** utility configures or changes the hostname of the specified database. In other words, it changes the hostname configuration in the **databases.txt** file.

```
cubrid renamedb [options] src_database_name dest_database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **renamedb**: A command that changes the existing name of a database to a new one. It executes successfully only when the database is in a stopped state. The names of related information volumes, log volumes and control files are also changed to new ones accordingly.
- *src_database_name*: The name of the existing database to be renamed. The path name to the directory where the database is to be created must not be included.
- *dest_database_name*: The new name of the database. It must not be the same as that of an existing database. The path name to the directory where the database is to be created must not be included.

The following shows [options] available with the **cubrid renamedb** utility.

```
-E, --extended-volume-path=PATH directory PATH for extended
information volumes
-i, --control-file=FILE         renaming volumes over several
locations written in the control file
-d, --delete-backup             delete backup volumes
```

-E, --extended-volume-path=PATH

This option renames an extended volume created in a specific directory path (e.g. /dbtemp/addvol/), and then moves the volume to a new directory. This specifies a new directory path (e.g. /dbtemp/newaddvols/) where the renamed extended volume will be moved.

If it is not specified, the extended volume is only renamed in the existing path without being moved. If a directory path outside the disk partition of the existing database volume or an invalid one is specified, the rename operation is not executed. This option cannot be used together with the **-i** option.

```
cubrid renamedb -E /dbtemp/newaddvols/ testdb testdb_1
```

-i, --control-file=FILE

The option specifies an input file in which directory information is stored to change all database name of volumes or files and assign different directory at once. To perform this work, the **-i** option is used. The **-i** option cannot be used together with the **-E** option.

```
cubrid renamedb -i rename_path testdb testdb_1
```

The following are the syntax and example of a file that contains the name of each volume, the current directory path and the directory path where renamed volumes will be stored.

```
volid source_fullvolname dest_fullvolname
```

- *volid*: An integer that is used to identify each volume. It can be checked in the database volume control file (database_name_vinf).
- *source_fullvolname*: The current directory path to each volume.
- *dest_fullvolname*: The target directory path where renamed volumes will be moved. If the target directory path is invalid, the database rename operation is not executed.

```
-5  /home1/user/testdb_vinf       /home1/CUBRID/databases/
testdb_1_vinf
-4  /home1/user/testdb_lginf      /home1/CUBRID/databases/
testdb_1_lginf
-3  /home1/user/testdb_bkvinf     /home1/CUBRID/databases/
testdb_1_bkvinf
-2  /home1/user/testdb_lgat       /home1/CUBRID/databases/
testdb_1_lgat
 0  /home1/user/testdb            /home1/CUBRID/databases/
testdb_1
 1  /home1/user/backup/testdb_x001/home1/CUBRID/databases/backup/
testdb_1_x001
```

-d, --delete-backup

This option renames the *testdb* database and at once forcefully delete all backup volumes and backup information files that are in the same location as *testdb*. Note that you cannot

use the backup files with the old names once the database is renamed. If the **-d** option is not specified, backup volumes and backup information files are not deleted.

```
cubrid renamedb -d testdb testdb_1
```

alterdbhost

The **cubrid alterdbhost** utility sets or changes the hostname of the specified database. It changes the hostname set in the **databases.txt** file.

```
cubrid alterdbhost [option] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management
- **alterdbhost**: A command used to change the hostname of the current database

The following shows the option available with the **cubrid alterdbhost** utility.

-h, --host=HOST

The *-h* option specifies the hostname to be changed. When this option is omitted, specifies the hostname to localhost.

copydb

The **cubrid copydb** utility copy or move a database to another location. As arguments, source and target name of database must be given. A target database name must be different from a source database name. When the target name argument is specified, the location of target database name is registered in the **databases.txt** file.

The **cubrid copydb** utility can be executed only offline (that is, state of a source database stop).

```
cubrid copydb [options] src-database-name dest-database-name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **copydb**: A command that copy or move a database from one to another location.
- *src-database-name*: The names of source and target databases to be copied or moved.
- *dest-database-name*: A new (target) database name.

If options are omitted, a target database is copied into the same directory of a source database.

The following shows [options] available with the **cubrid copydb** utility.

```
    --server-name=HOST          server's HOST name
-F, --file-path=PATH            directory PATH for database volumes
-L, --log-path=PATH             directory PATH for log volumes
-E, --extended-volume-path=PATH directory PATH for extended
information volumes
-i, --control-file=FILE         copying volumes over several
locations written in the control file
-r, --replace                   replace an existing database
-d, --delete-source             delete a source database
    --copy-lob-path             copy external storage PATH for LOB
data from the source database. By default, the LOB path is not copied
-B, --lob-base-path=PATH        base directory PATH for external LOB
data
```

--server-name=HOST

The **--server-name** option specifies a hostname of new database. The hostname is registered in the **databases.txt** file. If this option is omitted, a local host is registered.

```
cubrid copydb --server-name=cub_server1 demodb new_demodb
```

-F, --file-path=PATH

The *-F* option specifies a specific directory path where a new database volume is stored with an **-F** option. It represents specifying an absolute path. If the specified directory does not exist, an error is displayed. If this option is omitted, a new database volume is created in the current working directory. And this information is specified in **vol-path** of the **databases.txt** file.

```
cubrid copydb -F /home/usr/CUBRID/databases demodb new_demodb
```

-L, --log-path=PATH

The *-L* option specifies a specific directory path where a new database volume is stored with an **-L** option. It represents specifying an absolute path. If the specified directory does not exist, an error is displayed. If this option is omitted, a new database volume is created in the current working directory. And this information is specified in **log-path** of the **databases.txt** file.

```
cubrid copydb -L /home/usr/CUBRID/databases/logs demodb
new_demodb
```

-E, --extended-volume-path=PATH

The *-E* option specifies a specific directory path where a new database extended volume is stored with an **-E**. If this option is omitted, a new database extended volume is created in the location of a new database volume or in the registered path of controlling file. The **-i** option cannot be used with this option.

```
cubrid copydb -E /home/usr/CUBRID/databases/extvols demodb
new_demodb
```

-i, --control-file=FILE

The **-i** option specifies an input file where a new directory path information and a source volume are stored to copy or move multiple volumes into a different directory, respectively. This option cannot be used with the **-E** option. An input file named copy_path is specified in the example below.

```
cubrid copydb -i copy_path demodb new_demodb
```

The following is an example of input file that contains each volume name, current directory path, and new directory and volume names.

```
# volid   source_fullvolname   dest_fullvolname
0 /usr/databases/demodb        /drive1/usr/databases/new_demodb
1 /usr/databases/demodb_data1  /drive1/usr/databases/
new_demodb_data1
2 /usr/databases/ext/demodb_ext1 /drive2//usr/databases/
new_demodb_ext1
3 /usr/databases/ext/demodb_ext2  /drive2/usr/databases/
new_demodb_ext2
```

- *volid*: An integer that is used to identify each volume. It can be checked in the database volume control file (**database_name_vinf**).
- *source_fullvolname*: The current directory path to each source database volume.
- *dest_fullvolname*: The target directory path where new volumes will be stored. You should specify a valid path.

-r, --replace

If the **-r** option is specified, a new database name overwrites the existing database name if it is identical, instead of outputting an error.

```
cubrid copydb -r -F /home/usr/CUBRID/databases demodb new_demodb
```

-d, --delete-source

If the **-d** option is specified, a source database is deleted after the database is copied. This execution brings the same the result as executing **cubrid deletedb** utility after copying a database. Note that if a source database contains LOB data, LOB file directory path of a source database is copied into a new database and it is registered in the **lob-base-path** of the **databases.txt** file.

```
cubrid copydb -d -F /home/usr/CUBRID/databases demodb new_demodb
```

--copy-lob-path

Sets the LOB directory location of the target database to be the same as the source database's LOB directory location. If this option is omitted, the default location will be <target_database_directory>/lob. Note that this option does not copy LOB files and cannot be used together with the -B option ::

cubrid copydb –copy-lob-path demodb new_demodb

-B, --lob-base-path=PATH

If the **-B** option is specified, a specified directory is specified as for LOB files of a new database and a source database is copied. This option cannot be used with the **--copy-lob-path** option.

```
cubrid copydb -B /home/usr/CUBRID/databases/new_lob demodb
new_demodb
```

installdb

The **cubrid installdb** utility is used to register the information of a newly installed database to **databases.txt**, which stores database location information. The execution of this utility does not affect the operation of the database to be registered.

```
cubrid installdb [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **installdb**: A command that registers the information of a moved or copied database to **databases.txt**.
- *database_name*: The name of database to be registered to **databases.txt**.

If no [options] are used, the command must be executed in the directory where the corresponding database exists.

The following shows [options] available with the **cubrid installdb** utility.

```
    --server-name=HOST       server HOST name
-F, --file-path=PATH         directory PATH for database volumes
-L, --log-path=PATH          directory PATH for log volumes
```

--server-name=HOST

This option registers the server host information of a database to **databases.txt** with a specific hostname. If this is not specified, the current host information is registered.

```
cubrid installdb --server-name=cub_server1 testdb
```

-L, --log-path=PATH

This option registers the absolute directory path of a database log volume to **databases.txt** by using the **-L** option. If this option is not specified, the directory path of a volume is registered.

```
cubrid installdb -L /home/cubrid/CUBRID/databases/logs/testdb
testdb
```

backupdb

A database backup is the procedure of storing CUBRID database volumes, control files and log files, and it is executed by using the **cubrid backupdb** utility or the CUBRID Manager. **DBA** must regularly back up the database so that the database can be properly restored in the case of storage media or file errors. The restore environment must have the same operating system and the same version of CUBRID as the backup environment. For such a reason, you must perform a backup in a new environment immediately after migrating a database to a new version.

To recover all database pages, control files and the database to the state at the time of backup, the **cubrid backupdb** utility copies all necessary log records.

```
cubrid backupdb [options] database_name[@hostname]
```

- @*hostname*: It is omitted when you do backup in standalone mode. If you do backup on the HA environment, specify "@*hostname*" after the database name. *hostname* is a name specified in $CUBRID_DATABASES/databases.txt. If you want to setup a local server, you can specify it as "@localhost".

The following shows options available with the **cubrid backupdb** utility (options are case sensitive).

```
-D, --destination-path=PATH  store backup volumes to directory PATH
-r, --remove-archive         delete unnecessary log-archives
-l, --level=LEVEL            backup level
-o, --output-file=FILE       print detailed backup messages to FILE
-S, --SA-mode                stand-alone mode execution
-C, --CS-mode                client-server mode execution
    --no-check               don't check a consistency of the
database
-t, --thread-count=COUNT     number of threads
    --no-compress            don't compress backup volumes
    --sleep-msecs=N          sleep N millisecond per 1M read
-k, --separate-keys          keys file (_keys) for TDE is not
included in the backup volume, but is separated into a file instead
```

-D, --destination-path=PATH

The following shows how to use the **-D** option to store backup files in the specified directory. The backup file directory must be specified before performing this job. If the **-D** option is not specified, backup files are stored in the log directory specified in the **databases.txt** file which stores database location information.

```
cubrid backupdb -D /home/cubrid/backup demodb
```

The following shows how to store backup files in the current directory by using the **-D** option. If you enter a period (.) following the **-D** option as an argument, the current directory is specified.

```
cubrid backupdb -D . demodb
```

-r, --remove-archive

Writes an active log to a new archive log file when the active log is full. If a backup is performed in such a situation and backup volumes are created, backup logs created before the backup will not be used in subsequent backups. The **-r** option is used to remove archive log files that will not be used anymore in subsequent backups after the current one is complete. The **-r** option only removes unnecessary archive log files that were created before backup, and does not have any impact on backup; however, if an administrator removes the archive log file after a backup, it may become impossible to restore everything. For this reason, archive logs should be removed only after careful consideration.

If you perform an incremental backup (backup level 1 or 2) with the **-r** option, there is the risk that normal recovery of the database will be impossible later on. Therefore, it is recommended that the **-r** option only be used when a full backup is performed.

```
cubrid backupdb -r demodb
```

-l, --level=LEVEL

The following shows how to execute an incremental backup of the level specified by using the **-l** option. If the **-l** option is not specified, a full backup is performed. For details on backup levels, see Incremental Backup .

```
cubrid backupdb -l 1 demodb
```

-o, --output-file=FILE

The following shows how to write the progress of the database backup to the info_backup file by using the **-o** option.

```
cubrid backupdb -o info_backup demodb
```

The following shows the contents of the info_backup file. You can check the information on the number of threads, compression method, backup start time, the number of permanent volumes, backup progress and backup end time.

```
[ Database(demodb) Full Backup start ]
- num-threads: 1
- compression method: NONE
- backup start time: Mon Jul 21 16:51:51 2008
- number of permanent volumes: 1
- backup progress status
------------------------------------------------------------------------
------------
 volume name                  | # of pages | backup progress
status    | done
------------------------------------------------------------------------
------------
 demodb_keys                  |          1 |
########################## | done
 demodb_vinf                  |          1 |
########################## | done
 demodb                       |      25000 |
########################## | done
 demodb_lginf                 |          1 |
########################## | done
 demodb_lgat                  |      25000 |
########################## | done
------------------------------------------------------------------------
------------
# backup end time: Mon Jul 21 16:51:53 2008
[Database(demodb) Full Backup end]
```

-S, --SA-mode

The following shows how to perform backup in standalone mode (that is, backup offline) by using the **-S** option. If the **-S** option is not specified, the backup is performed in client/server mode.

```
cubrid backupdb -S demodb
```

-C, --CS-mode

The following shows how to perform backup in client/server mode by using the **-C** option and the *demodb* database is backed up online. If the **-C** option is not specified, a backup is performed in client/server mode.

```
cubrid backupdb -C demodb
```

--no-check

The following shows how to execute backup without checking the consistency of the database by using the **--no-check** option.

```
cubrid backupdb --no-check demodb
```

-t, --thread-count=COUNT

The following shows how to execute parallel backup with the number of threads specified by the administrator by using the **-t** option. Even when the argument of the **-t** option is not specified, a parallel backup is performed by automatically assigning as many threads as CPUs in the system.

```
cubrid backupdb -t 4 demodb
```

--no-compress

By default, CUBRID compresses the target database in order to reduce the size of the backup volume and then saves it. If a user wants to eliminate the CPU usage by the compression, they can set the **--no-compress** option not to compress the backup volume.

```
cubrid backupdb --no-compress demodb
```

--sleep-msecs=NUMBER

This option allows you to specify the interval of idle time during the database backup. The default value is 0 in milliseconds. The system becomes idle for the specified amount of time whenever it reads 1 MB of data from a file. This option is used to reduce the performance degradation of an active server during a live backup. The idle time will prevent excessive disk I/O operations.

```
cubrid backupdb --sleep-msecs=5 demodb
```

-k, --separate-keys

This option is used not to include the key file in the backup volume. The key file that is not included is separated into a file named **<database_name>_bk<backup_level>_keys**. If this option is not given, the key file is included in the backup volume by default. For a detailed description of key file separation, see Backup Encryption.

```
cubrid backupdb -k demodb
```

Backup Strategy and Method

The following must be considered before performing a backup:

- **Selecting the data to be backed up**
  - Determine whether it is valid data worth being preserved.
  - Determine whether to back up the entire database or only part of it.
  - Check whether there are other files to be backed up along with the database.
- **Choosing a backup method**
  - Choose the backup method from one of incremental and online backups. Also, specify whether to use compression backup, parallel backup, and mode.
  - Prepare backup tools and devices available.
- **Determining backup time**
  - Identify the time when the least usage in the database occur.
  - Check the size of the archive logs.
  - Check the number of clients using the database to be backed up.

Online Backup

An online backup (or a hot backup) is a method of backing up a currently running database. It provides a snapshot of the database image at a certain point in time. Because the backup target is a currently running database, it is likely that uncommitted data will be stored and the backup may affect the operation of other databases.

To perform an online backup, use the **cubrid backupdb -C** command.

Offline Backup

An offline backup (or a cold backup) is a method of backing up a stopped database. It provides a snapshot of the database image at a certain point in time.

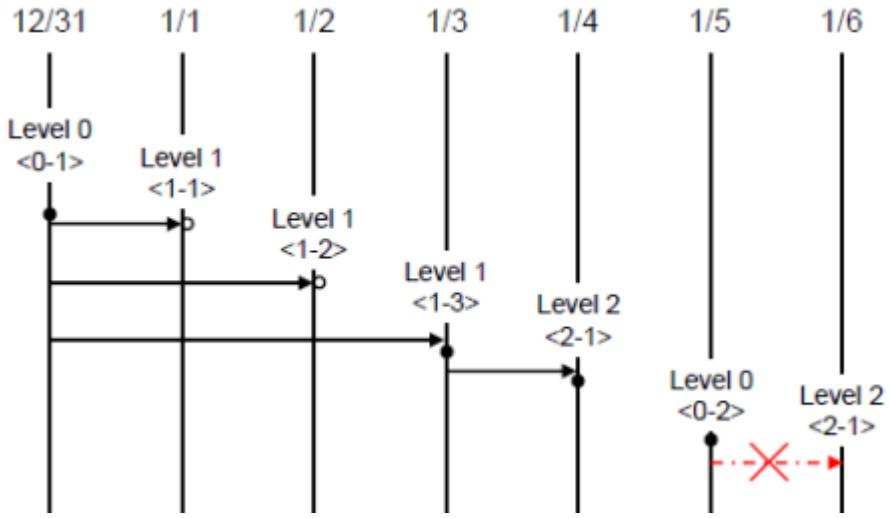
To perform an offline backup, use the **cubrid backupdb -S** command.

Incremental Backup

An incremental backup, which is dependent upon a full backup, is a method of only backing up data that have changed since the last backup. This type of backup has an advantage of requiring less volume and time than a full backup. CUBRID supports backup levels 0, 1 and 2. A higher level backup can be performed sequentially only after a lower lever backup is complete.

To perform an incremental backup, use the **cubrid backupdb -l** *LEVEL* command.

The following example shows incremental backup. Let's example backup levels in details.



- **Full backup (backup level 0)** : Backup level 0 is a full backup that includes all database pages.

The level of a backup which is attempted first on the database naturally becomes a 0 level. **DBA** must perform full backups regularly to prepare for restore situations. In the example, full backups were performed on December 31st and January 5th.

- **First incremental backup (backup level 1)** : Backup level 1 is an incremental backup that only stores changes since the level 0 full backup, and is called a "first incremental backup."

Note that the first incremental backups are attempted sequentially such as <1-1>, <1-2> and <1-3> in the example, but they are always performed based on the level 0 full backup.

Suppose that backup files are created in the same directory. If the first incremental backup <1-1> is performed on January 1st and then the first incremental backup <1-2> is attempted again on January 2nd, the incremental backup file created in <1-1> is overwritten. The final incremental

backup file is created on January 3rd because the first incremental backup is performed again on that day.

Since there can be a possibility that the database needs to be restored the state of January 1st or January 2nd, it is recommended for **DBA** to store the incremental backup files <1-1> and <1-2> separately in storage media before overwriting with the final incremental file.

- **Second incremental backup (backup level 2)** : Backup level 2 is an incremental backup that only stores data that have changed since the first incremental backup, and is called a "second incremental backup."

A second incremental backup can be performed only after the first incremental backup. Therefore, the second incremental backup attempted on January fourth succeeds; the one attempted on January sixth fails.

Backup files created for backup levels 0, 1 and 2 may all be required for database restore. To restore the database to its state on January fourth, for example, you need the second incremental backup generated at <2-1>, the first incremental backup file generated at <1-3>, and the full backup file generated at <0-1>. That is, for a full restore, backup files from the most recent incremental backup file to the earliest created full backup file are required.

Compress Backup

A compress backup is a method of backing up the database by compressing it. This type of backup reduces disk I/O costs and stores disk space because it requires less backup volume.

From version 11.2 of CUBRID, compress backup is used by default.

Parallel Backup Mode

A parallel or multi-thread backup is a method of performing as many backups as the number of threads specified. In this way, it reduces backup time significantly. Basically, threads are given as many as the number of CPUs in the system.

To perform a parallel backup, use the **cubrid backupdb -t** | **--thread-count** command.

Managing Backup Files

One or more backup files can be created in sequence based on the size of the database to be backed up. A unit number is given sequentially (000, 001-0xx) to the extension of each backup file based in the order of creation.

Managing Disk Capacity during the Backup

During the backup process, if there is not enough space on the disk to store the backup files, a message saying that the backup cannot continue appears on the screen. This message contains the name and path of the database to be backed up, the backup filename, the unit number of

backup files and the backup level. To continue the backup process, the administrator can choose one of the following options:

- Option 0: An administrator enters 0 to discontinue the backup.
- Option 1: An administrator inserts a new disk into the current device and enters 1 to continue the backup.
- Option 2: An administrator changes the device or the path to the directory where backup files are stored and enters 2 to continue the backup.

```
*************************************************************
Backup destination is full, a new destination is required to
continue:
Database Name: /local1/testing/demodb
     Volume Name: /dev/rst1
        Unit Num: 1
    Backup Level: 0 (FULL LEVEL)
Enter one of the following options:
Type
   -  0 to quit.
   -  1 to continue after the volume is mounted/loaded. (retry)
   -  2 to continue after changing the volume's directory or device.
*************************************************************
```

Managing Archive Logs

You must not delete archive logs by using the file deletion command such as rm or del by yourself; the archive logs should be deleted by system configuration or the **cubrid backupdb** utility. In the following three cases, archive logs can be deleted.

- In non-HA environment (ha_mode=off):

  If you configure the **force_remove_log_archives** value to yes, archive logs are kept only the number specified in **log_max_archives** value; and the left logs are automatically deleted. However, if there is an active tranaction in the oldest archive log file, this file is not deleted until the transaction is completed.

- In an HA environment (ha_mode=on):

  If you configure the **force_remove_log_archives** values to no and specify the number specified in **log_max_archives** value, archive logs are automatically deleted after replication is applied.

Note

If you set **force_remove_log_archives** as yes when "ha_mode=on", unapplied archive logs can be deleted; therefore, this setting is not recommended. However, if keeping disk space is prior to keeping replication, set **force_remove_log_archives** as yes and set **log_max_archives** as a proper value.

- Use **cubrid backupdb -r** and run this command then archive logs are deleted; note that **-r** option should not be used in an HA environment.

If you want to delete logs as much as possible while operating a database, configure the value of **log_max_archives** to a small value or 0 and configure the value of **force_remove_log_archives** to yes. Note that in an HA environment, if the value of **force_remove_log_archives** is yes, archive logs that have not replicated in a slave node are deleted, which can cause replication errors. Therefore, it is recommended that you configure it to no. Although the value of **force_remove_log_archives** is set to no, files that are complete for replication can be deleted by HA management process.

restoredb

A database restore is the procedure of restoring the database to its state at a certain point in time by using the backup files, active logs and archive logs which have been created in an environment of the same CUBRID version. To perform a database restore, use the **cubrid restoredb** utility or the CUBRID Manager.

The **cubrid restoredb** utility restores the database from the database backup by using the information written to all the active and archive logs since the execution of the last backup.

```
cubrid restoredb [options] database_name
```

If no option is specified, a database is restored to the point of the last commit by default. If no active/archive log files are required to restore to the point of the last commit, the database is restored only to the point of the last backup.

```
cubrid restoredb demodb
```

The following table shows options available with the **cubrid restoredb** utility (options are case sensitive).

```
-d, --up-to-date=DATE               restore the database up to its
condition at given DATE
    --list                          display a list of backup volumes
without restoring
```

```
-B, --backup-file-path=PATH       PATH is a directory of backup
volumes to be restored
-l, --level=LEVEL                 LEVEL of backup to be restored;
see backup usage
-p, --partial-recovery            perform partial recovery if any
log archive is absent
-o, --output-file=FILE            redirect output messages to FILE
-u, --use-database-location-path  restore the database and log
volumes to the path specified in the database location file
-k, --keys-file-path              Path of key file (_keys) for TDE
to be used during restoring
```

-d, --up-to-date=DATE

A database can be restored to the given point by the date-time specified by the **-d** option. The user can specify the restoration point manually in the dd-mm-yyyy:hh:mi:ss (e.g. 14-10-2008:14:10:00) format. If no active log/archive log files are required to restore to the point specified, the database is restored only to the point of the last backup.

```
cubrid restoredb -d 14-10-2008:14:10:00 demodb
```

If the user specifies the restoration point by using the **backuptime** keyword, it restores a database to the point of the last backup.

```
cubrid restoredb -d backuptime demodb
```

--list

This option displays information on backup files of a database; restoration procedure is not performed. This option is available even if the database is working, from CUBRID 9.3.

```
cubrid restoredb --list demodb
```

The following example shows how to display backup information by using the **--list** option. You can specify the path to which backup files of the database are originally stored as well as backup levels.

```
*** BACKUP HEADER INFORMATION ***
Database Name: /local1/testing/demodb
   DB Creation Time: Mon Oct 1 17:27:40 2008
   Pagesize: 4096
Backup Level: 1 (INCREMENTAL LEVEL 1)
   Start_lsa: 513|3688
   Last_lsa: 513|3688
Backup Time: Mon Oct 1 17:32:50 2008
   Backup Unit Num: 0
```

```
Release: 8.1.0
   Disk Version: 8
Backup Pagesize: 4096
Zip Method: 0 (NONE)
   Zip Level: 0 (NONE)
Previous Backup Level: 0
   Time: Mon Oct 1 17:31:40 2008
   (start_lsa was -1|-1)
Database Volume Name: /local1/testing/demodb_keys
   Volume Identifier: -6, Size: 65 bytes (1 pages)
Database Volume Name: /local1/testing/demodb_vinf
   Volume Identifier: -5, Size: 308 bytes (1 pages)
Database Volume Name: /local1/testing/demodb
   Volume Identifier: 0, Size: 2048000 bytes (500 pages)
Database Volume Name: /local1/testing/demodb_lginf
   Volume Identifier: -4, Size: 165 bytes (1 pages)
Database Volume Name: /local1/testing/demodb_bkvinf
   Volume Identifier: -3, Size: 132 bytes (1 pages)
```

With the backup information displayed by using the **--list** option, you can check that backup files have been created at the backup level 1 as well as the point where the full backup of backup level 0 has been performed. Therefore, to restore the database in the example, you must prepare backup files for backup levels 0 and 1.

-B, --backup-file-path=PATH

You can specify the directory where backup files are to be located by using the **-B** option. If this option is not specified, the system retrieves the backup information file (*dbname* **_bkvinf**) generated upon a database backup; the backup information file in located in the **log-path** directory specified in the database location information file (**databases.txt**). And then it searches the backup files in the directory path specified in the backup information file. However, if the backup information file has been damaged or the location information of the backup files has been deleted, the system will not be able to find the backup files. Therefore, the administrator must manually specify the directory where the backup files are located by using the **-B** option.

```
cubrid restoredb -B /home/cubrid/backup demodb
```

If the backup files of a database is in the current directory, the administrator can specify the directory where the backup files are located by using the **-B** option.

```
cubrid restoredb -B . demodb
```

-l, --level=LEVEL

You can perform restoration by specifying the backup level of the database to 0, 1, or 2. For details on backup levels, see Incremental Backup .

```
cubrid restoredb -l 1 demodb
```

-p, --partial-recovery

You can perform partial restoration without requesting for the user's response by using the **-p** option. If active or archive logs written after the backup point are not complete, by default the system displays a request message informing that log files are needed and prompting the user to enter an execution option. The partial restoration can be performed directly without such a request message by using the **-p** option. Therefore, if the **-p** option is used when performing restoration, data is always restored to the point of the last backup.

```
cubrid restoredb -p demodb
```

When the **-p** option is not specified, the message requesting the user to select the execution option is as follows:

```
************************************************************
Log Archive /home/cubrid/test/log/demodb_lgar002
 is needed to continue normal execution.
   Type
   -  0 to quit.
   -  1 to continue without present archive. (Partial recovery)
   -  2 to continue after the archive is mounted/loaded.
   -  3 to continue after changing location/name of archive.
************************************************************
```

- Option 0: Stops restoring
- Option 1: Performing partial restoration without log files.
- Option 2: Performing restoration after locating a log to the current device.
- Option 3: Resuming restoration after changing the location of a log

-o, --output-file=FILE

The following syntax shows how to write the restoration progress of a database to the info_restore file by using the **-o** option.

```
cubrid restoredb -o info_restore demodb
```

-u, --use-database-location-path

This option restores a database to the path specified in the database location file(**databases.txt**). The **-u** option is useful when you perform a backup on server A and store the backup file on server B.

```
cubrid restoredb -u demodb
```

-k, --keys-file-path=PATH

This option specifies the path of a key file required to restore the database. If the valid key file is not given, it could fail to restore. If this option is not given, the appropriate key file is searched automatically according to a priority. For more information on this, refer to Backup Encryption.

```
cubrid restoredb -k /home/cubrid/backup_keys/demodb_bk1_keys
demodb
```

NOTIFICATION messages, log recovery starting time and ending time are written into the server error log file or the restoredb error log file when database server is started or backup volume is restored; so you can check the elapsed time of these operations. In this message, the number of log records to be applied(redo) and the number of log pages are written together.

```
Time: 06/14/13 21:29:04.059 - NOTIFICATION *** file ../../src/
transaction/log_recovery.c, line 748 CODE = -1128 Tran = -1, EID = 1
Log recovery is started. The number of log records to be applied:
96916. Log page: 343 ~ 5104.
.....
Time: 06/14/13 21:29:05.170 - NOTIFICATION *** file ../../src/
transaction/log_recovery.c, line 843 CODE = -1129 Tran = -1, EID = 4
Log recovery is finished.
```

Restoring Strategy and Procedure

You must consider the following before restoring databases.

- **Preparing backup files**
  - Identify the directory where the backup and log files are to be stored.
  - If the database has been incrementally backed up, check whether an appropriate backup file for each backup level exists.
  - Check whether the backed-up CUBRID database and the CUBRID database to be backed up are the same version.
- **Choosing restore method**
  - Determine whether to perform a partial or full restore.
  - Determine whether or not to perform a restore using incremental backup files.

- Prepare restore tools and devices available.

· **Determining restore point**

- Identify the point in time when the database server was terminated.
- Identify the point in time when the last backup was performed before database failure.
- Identify the point in time when the last commit was made before database failure.

Database Restore Procedure

The following procedure shows how to perform backup and restoration described in the order of time.

1. Performs a full backup of *demodb* which stopped running at 2008/8/14 04:30.
2. Performs the first incremental backup of *demodb* running at 2008/8/14 10:00.
3. Performs the first incremental backup of *demodb* running at 2008/8/14 15:00. Overwrites the first incremental backup file in step 2.
4. A system failure occurs at 2008/8/14 15:30, and the system administrator prepares the restore of *demodb*. Sets the restore time as 15:25, which is the time when the last commit was made before database failure
5. The system administrator prepares the full backup file created in Step 1 and the first incremental backup file created in Step 3, restores the *demodb* database up to the point of 15:00, and then prepares the active and archive logs to restore the database up to the point of 15:25.

Time	Command	Description
2008/8/14 04:25	cubrid server stop demodb	Shuts down *demodb*.
2008/8/14 04:30	cubrid backupdb -S -D / home/backup -l 0 demodb	Performs a full backup of *demodb* in offline mode and creates backup files in the specified directory.
2008/8/14 05:00	cubrid server start demodb	Starts *demodb*.
2008/8/14 10:00		Performs the first incremental backup of

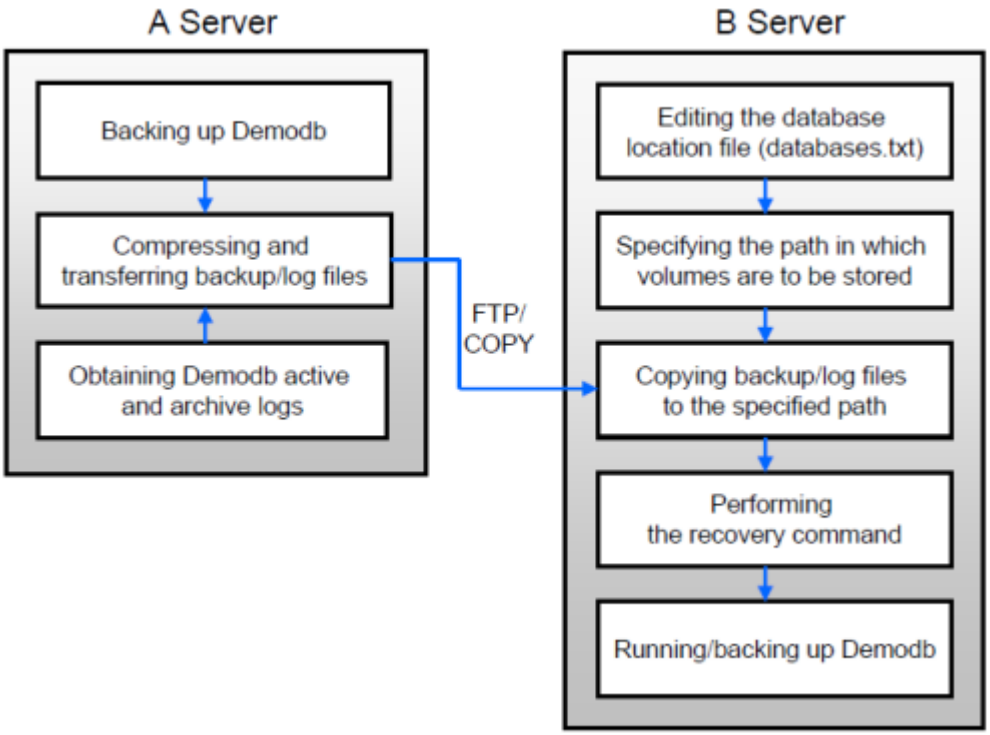
Time	Command	Description
	cubrid backupdb -C -D /home/backup -l 1 demodb	*demodb* online and creates backup files in the specified directory.
2008/8/14 15:00	cubrid backupdb -C -D /home/backup -l 1 demodb	Performs the first incremental backup of *demodb* online and creates backup files in the specified directory. Overwrites the first incremental backup file created at 10:00.
2008/8/14 15:30		A system failure occurs.
2008/8/14 15:40	cubrid restoredb -l 1 -d 08/14/2008:15:25:00 demodb	Restores *demodb* based on the full backup file, first incremental backup file, active logs and archive logs. The database is restored to the point of 15:25 by the full and first incremental backup files, the active and archive logs.

Restoring Database to Different Server

The following shows how to back up *demodb* on server *A* and restore it on server *B* with the backed up files.

Backup and Restore Environments

Suppose that *demodb* is backed up in the /home/cubrid/db/demodb directory on server *A* and restored into /home/cubrid/data/demodb on server *B*.



1. Backing up on server A

   Back up *demodb* on server *A*. If a backup has been performed earlier, you can perform an incremental backup for data only that have changed since the last backup. The directory where the backup files are created, if not specified in the **-D** option, is created by default in the location where the log volume is stored. The following is a backup command with recommended options. For details on the options, see backupdb .

   ```
   cubrid backupdb -z demodb
   ```

2. Editing the database location file on Server B

   Unlike a general scenario where a backup and restore are performed on the same server, in a scenario where backup files are restored using a different server, you need to add the location information on database restore in the database location file (**databases.txt**) on server *B*. In the diagram above, it is supposed that *demodb* is restored in the /home/cubrid/data/demodb directory on server *B* (hostname: pmlinux); edit the location information file accordingly and create the directory on server *B*.

   Put the database location information in one single line. Separate each item with a space. The line should be written in [database name]. [data volume path] [hostname] [log volume path] format; that is, write the location information of *demodb* as follows:

   ```
   demodb /home/cubrid/data/demodb pmlinux /home/cubrid/data/demodb
   ```

3. Transferring backup files to server B

   For a restore, you must prepare backup files. Therefore, transfer a backup file (e.g. demodb_bk0v000) from server *A* to server *B*. That is, a backup file must be located in a directory (e.g. /home/cubrid/temp) on server *B*.

   Note

   If you want to restore until the current time after the backup, logs after backup, that is, an active log (e.g. demodb_lgat) and archive logs (e.g. demodb_lgar000) are additionally required to copy.

   An active log and archive logs should be located to the log directory of the database to be restored; that is, the directory of log files specified in $CUBRID/databases/databases.txt. (e.g. $CUBRID/databases/demodb/log)

   Also, if you want to apply the added logs after backup, archive logs should be copied before they are removed. By the way, the default of log_max_archives, which is a system parameter related to delete archive logs, is 0; therefore, archive logs after backup can be deleted. To prevent this situation, the value of log_max_archives should be big enough. See log_max_archives.

4. Restoring the database on server B

   Perform database restore by calling the **cubrid restoredb** utility from the directory into which the backup files which were transferred to server *B* had been stored. With the **-u** option, *demodb* is restored in the directory path from the **databases.txt** file.

```
cubrid restoredb -u demodb
```

   To call the **cubrid restoredb** utility from a different path, specify the directory path to the backup file by using the **-B** option as follows:

```
cubrid restoredb -u -B /home/cubrid/temp demodb
```

unloaddb

The purposes of loading/unloading databases are as follows:

- To rebuild databases by volume reconfiguration

- To migrate database in different system environments
- To migrate database in different versions

```
cubrid unloaddb [options] database_name
```

cubrid unloaddb utility creates the following files:

- Schema file(*database-name*_**schema**): A file that contains information on the schema defined in the database.
- Object file(*database-name*_**objects**): A file that contains information on the records in the database.
- Index file(*database-name*_**indexes**): A file that contains information on the indexes defined in the database.
- Trigger file(*database-name*_**trigger**): A file that contains information on the triggers defined in the database. If you don't want triggers to be running while loading the data, load the trigger definitions after the data loading has completed.
- Split schema files (*database-name*_**schema_objecttype**): When using the **--split-schema-files** option, the schema information defined in the database is split into object units and These files contain only object information. For a list of split files, see the **--split-schema-files** option.
- Schema list file (*database-name*_**schema_info**): When using the **--split-schema-files** option, this file contains a list of split schema information files.

The schema, object, index, and trigger files are created in the same directory.

The following is [options] used in **cubrid unloaddb**.

```
-u, --user=ID               user ID for database access
-p, --password=PASS         user password
-i, --input-class-file=FILE input FILE of table names
    --include-reference     include referenced tables
    --input-class-only      include specified class only
    --estimated-size=NUMBER estimated NUMBER of instances
    --cached-pages=NUMBER   NUMBER of cached pages
-O, --output-path=PATH      PATH of output directory
-s, --schema-only           dump schema only
-d, --data-only             dump objects only
    --output-prefix=PREFIX  PREFIX for output files
    --hash-file=FILE        FILE for hash
    --latest-image          when dumping objects, it dumps the
latest image of objects in data volumes.
-t, --thread-count=COUNT    number of threads; default: 1 maximum:
127
```

```
    --enhanced-estimates    obtain the exact number of records in
the table to unload; default: disabled
-v, --verbose               enable verbose status messages
    --use-delimiter         use '"' where an identifier begins and
ends
-S, --SA-mode               stand-alone mode execution
-C, --CS-mode               client-server mode execution
    --datafile-per-class    create a object file for each class
    --split-schema-files    split schema information by object and
generate each file
    --skip-index-detail     Do not print the specified WITH clause
options when creating the index
    --as-dba                If the login user is a member of the DBA
group, extract the same schema file as the DBA.
```

-u, --user=ID

Specify a user account of a database to be unloaded. If this is not specified, the default is **DBA**.

```
cubrid unloaddb -u dba -i table_list.txt demodb
```

-p, --password=PASS

Specify a user's password of a database to be unloaded. If this is not specified, it is regarded as the empty string is entered.

```
cubrid unloaddb -u dba -p dba_pwd -i table_list.txt demodb
```

-i, --input-class-file FILE

Unload all schema and index of all tables; however, only the data of specified tables in this file are unloaded. If the to-be-unloaded schema (See the -u option) of the user and the schema of the table are different, the table name must be prefixed with the schema name.

```
cubrid unloaddb -i table_list.txt demodb
```

The following example shows an input file (table_list.txt).

```
table_1
table_2
..
table_n
```

If this option is used together with the **--input-class-only** option, it creates schema, index, and data files of tables only specified in the input file of **-i** option.

```
cubrid unloaddb --input-class-only -i table_list.txt demodb
```

If this option is used together with the **--include-reference** option, it unloads the referenced tables as well.

```
cubrid unloaddb --include-reference -i table_list.txt demodb
```

--include-reference

This option is used together with the **-i** option, and also unloads the referenced tables.

--input-class-only

This option is used together with the **-i** option, and creates only a schema file of tables specified in the input file of **-i** option.

--estimated-size=NUMBER

This option allows you to assign hash memory to store records of the database to be unloaded. If the **--estimated-size** option is not specified, the number of records of the database is determined based on recent statistics information. This option can be used if the recent statistics information has not been updated or if a large amount of hash memory needs to be assigned. Therefore, if the number given as the argument for the option is too small, the unload performance deteriorates due to hash conflicts.

```
cubrid unloaddb --estimated-size=1000 demodb
```

--cached-pages=NUMBER

The **--cached-pages** option specifies the number of pages of tables to be cached in the memory. Each page is 4,096 bytes. The administrator can configure the number of pages taking into account the memory size and speed. If this option is not specified, the default value is 100 pages.

```
cubrid unloaddb --cached-pages 500 demodb
```

-O, --output-path=PATH

This option specifies the directory in which to create schema and object files. If this is not specified, files are created in the current directory.

```
cubrid unloaddb -O ./CUBRID/Databases/demodb demodb
```

If the specified directory does not exist, the following error message will be displayed.

```
unloaddb: No such file or directory.
```

-s, --schema-only

This option specifies that only the schema file will be created from amongst all the output files which can be created by the unload operation.

```
cubrid unloaddb -s demodb
```

-d, --data-only

This option specifies that only the data file will be created from amongst all of the output files which can be created by the unload operation.

```
cubrid unloaddb -d demodb
```

--output-prefix=PREFIX

This option specifies the prefix for the names of schema and object files created by the unload operation. Once the example is executed, the schema filename becomes *abcd_schema* and the object filename becomes *abcd_objects*. If the **--output-prefix** option is not specified, the name of the database to be unloaded is used as the prefix.

```
cubrid unloaddb --output-prefix abcd demodb
```

--hash-file=FILE

This option specifies the name of the hash file.

--latest-image

This option makes it to unload the latest image of objects in data volumes when unloading instances (the MVCC version is ignored and it does not refer to the log volumes). It may include uncommitted data and deleted data that has not been vacuumed.

-t, --thread-count=COUNT

This option sets the number of threads to be used in the unload process. It is performed in parallel using as many threads as the specified COUNT value, which must be in the range of 0 to 127. If this setting is omitted, it is the same as specifying COUNT as 1. If specified as 0, unloaddb does not operate in thread mode. Even if thread mode is specified, if a table contains an Object type or a Set, MultiSet, or Sequence (List) type that contains an Object type, it will not operate in thread mode for that table.

```
cubrid unloaddb -t 4 demodb
```

--enhanced-estimates

The -v option must be specified together. When determining the number of records in the target table, this option obtains the actual number of records instead of relying on statistical information. Using this option may increase the execution time of unloaddb because doing so adds the time needed to calculate the number of records. Therefore, this option should be used with caution.

```
cubrid unloaddb --enhanced-estimates demodb
```

-v, --verbose

This option displays detailed information on the database tables and records being unloaded while the unload operation is under way.

```
cubrid unloaddb -v demodb
```

--use-delimiter

This option writes the double quote(”) on the beginning and end of an identifier. The default is not to write the double quote(“).

-S, --SA-mode

The **-S** option performs the unload operation by accessing the database in standalone mode.

```
cubrid unloaddb -S demodb
```

-C, --CS-mode

The **-C** option performs the unload operation by accessing the database in client/server mode.

```
cubrid unloaddb -C demodb
```

--datafile-per-class

This option specifies that the output file generated through unload operation creates a data file per each table. The filename is generated as *<Database Name>_<Table Name>_* **objects** for each table. However, all column values in object types are unloaded as NULL and %id class_name class_id part is not written in the unloaded file (see How to Write Files to Load Database).

```
cubrid unloaddb --datafile-per-class demodb
```

--split-schema-files

Schema information is splited and created in object units, and a file containing a list of splited files is also created. For details, see to loaddb 's –schema-file-list=FILE. If this option is not specified, a single schema file containing all objects is created.

```
cubrid unloaddb --split-schema-files demodb
```

List of files created by spliting schema information into object units

```
demodb_schema_user
demodb_schema_class
demodb_schema_vclass
demodb_schema_synonym
demodb_schema_serial
demodb_schema_procedure
demodb_schema_server
demodb_schema_pk
demodb_schema_fk
demodb_schema_uk
demodb_schema_grant
demodb_schema_vclass_query_spec
```

Filename containing a list of splited schema files

```
demodb_schema_info
```

--as-dba

If the user is a member of the DBA group, create a schema file with the same contents as for the DBA user.

```
cubrid unloaddb -u u1_dba_group --as-dba demodb
```

--skip-index-detail

This option doesn't output of the WITH clause options specified when creating the index. The default operation of unloaddb is to always print the WITH clause options regardless of the **print_index_detail** setting. If this option is specified, options of indexes in the WITH clause are omitted and printed. Affected output files are schema file (*database-name*_schema) and index file (*database-name*_indexes). If this option is omitted, even if *deduplicate level* is **0**, it will always be output explicitly.

```
cubrid unloaddb --skip-index-detail demodb
```

In the example below, you can see the difference whether this option is specified or not.

Schema file

--skip-index-detail not specified	ALTER CLASS [dba].[ts] ADD CONSTRAINT [fk_ts_fid] FOREIGN KEY([fid]) WITH DEDUPLICATE=0 REFERENCES [dba].[tm] ON DELETE RESTRICT ON UPDATE RESTRICT ;
--skip-index-detail specified	ALTER CLASS [dba].[ts] ADD CONSTRAINT [fk_ts_fid] FOREIGN KEY([fid]) REFERENCES [dba].[tm] ON DELETE RESTRICT ON UPDATE RESTRICT ;

Index file

--skip-index-detail not specified	CREATE INDEX [idx_val] ON [dba].[ts] ([val]) WITH DEDUPLICATE=0;
--skip-index-detail specified	CREATE INDEX [idx_val] ON [dba].[ts] ([val]);

Note

To shorten the execution time of the unloaddb, it is possible to run multiple **'cubrid unloaddb'** processes concurrently for groups of different tables. (or you can use **unloaddb.sh**, refer unloaddb.sh script).

loaddb

You can load a database by using the **cubrid loaddb** utility in the following situations:

- Migrating previous version of CUBRID database to new version of CUBRID database
- Migrating a third-party DBMS database to a CUBRID database
- Inserting massive amount of data faster than using the **INSERT** statement

In general, the **cubrid loaddb** utility uses files (schema definition, object input, and index definition files) created by the **cubrid unloaddb** utility.

```
cubrid loaddb [options] database_name
```

Input Files

- Schema file(*database-name*_**schema**): A file generated by the unload operation; it contains schema information defined in the database.
- Object file(*database-name*_**objects**): A file created by an unload operation. It contains information on the records in the database.
- Index file(*database-name*_**indexes**): A file created by an unload operation. It contains information on the indexes defined in the database.
- Trigger file(*database-name*_**trigger**): A file created by an unload operation. It contains information on the triggers defined in the database.
- User-defined object file(*user_defined_object_file*): A file in table format written by the user to enter mass data. (For details, see How to Write Files to Load Database .)
- Schema list file (*database-name*_**schema_info**): A file created by unloading with the **--split-schema-files** option. This is a file that has a list of files in which schema information defined in the database is splited into object units.(For details, see `unloaddb --split-schema-files`.)

Execution order of input files

The **cubrid loaddb** utility follows a specific order of parsing and executing input files, ensuring both correctness and optimized performance. The order is as follows:

1. Load schema file (including primary keys)
2. Load object file
3. Load index file (secondary keys and foreign keys)
4. Load trigger file

Loading modes

The **cubrid loaddb** utility provides two modes of loading the data into a database.

- **Stand-alone** mode loads the data offline, within a single thread environment. CUBRID 10.1 and older versions run only in this mode.

- **Client-server** mode connects to a server and loads the data online and in parallel. **Client-server** mode can be much faster, but object references and class/shared attributes are not supported in this mode and they can only be loaded in **stand-alone** mode

The following table shows [options] available with the **cubrid loaddb** utility.

```
-u, --user=ID                 user ID for database access
-p, --password=PASS           user password
-S, --SA-mode                 standalone mode execution
-C, --CS-mode                 client-server mode execution
    --data-file-check-only    check syntax for data file without
load
-l, --load-only               load data file without prior syntax
checking
    --estimated-size=NUMBER   estimated NUMBER of instances
-v, --verbose                 enable verbose status messages
-c, --periodic-commit=COUNT   insertion COUNT for periodic commit
    --no-oid                  don't use OID
    --no-statistics           do not update the statistics while
loading
-s, --schema-file=FILE[:LINE] create schema from LINE with FILE
-i, --index-file=FILE[:LINE]  create index from LINE with FILE
-d, --data-file=FILE          load data with FILE
-t, --table=TABLE             table name that is substituted for
missing class header in data file
    --error-control-file=FILE FILE to control error(s) during
loading
    --ignore-class-file=FILE  input file of class names that skip
load
    --trigger-file=FILE[:LINE] create trigger from LINE with FILE
    --no-user-specified-name  Find classes, serials, and triggers
by their object names without their owner names.
    --schema-file-list=FILE   The content of this file is a list of
schema file names to be used in loaddb.
```

-u, --user=ID

This option specifies the user account of a database where records are loaded. If the option is not specified, the default value is **PUBLIC**.

```
cubrid loaddb -u admin -d demodb_objects newdb
```

-p, --password=PASS

This option specifies the password of a database user who will load records. If the option is not specified, you will be prompted to enter the password.

```
cubrid loaddb -u dba -p pass -d demodb_objects newdb
```

-S, --SA-MODE

This option loads data into a database in **stand-alone** mode. This is the default loading mode of **loaddb** and it is fully compatible with CUBRID 10.1 and older database versions.

```
cubrid loaddb -S -u dba -d demodb_objects newdb
```

-C, --CS-MODE

This option connects to a running server process. Unlike the **stand-alone** mode, the **client-server** mode loads the data using multiple threads, leading to greater performance for large amounts of data. Multiple **loaddb** sessions may run at the same time. Object references and class/shared attributes are not supported in this mode.

```
cubrid loaddb -C -u dba -d demodb_objects newdb
```

Note

When in **client-server** mode (-C, --CS-MODE) on the **HA** (High Availability), only the -d option (data loading) is available.

--data-file-check-only

This option checks only the syntax of the data contained in demodb_objects, and does not load the data to the database.

```
cubrid loaddb --data-file-check-only -d demodb_objects newdb
```

-l, --load-only

Stand-alone mode does, by default, one prior syntax check on the whole data file before actually loading the data into the database. If a syntax error occurs, no data is loaded and the error is reported.

Using the **-l** option, the prior syntax check is skipped, thus increasing the loading speed. Syntax is still checked before loading each row into the database. Loading is stopped if a syntax error occurs, but data may be partially loaded.

Client-server mode doesn't do the prior syntax check, so this option has no effect.

```
cubrid loaddb -l -d demodb_objects newdb
```

--estimated-size=NUMBER

This option can be used to improve loading performance when the number of records to be unloaded exceeds the default value of 5,000. You can improve the load performance by assigning large hash memory for record storage with this option.

```
cubrid loaddb --estimated-size 8000 -d demodb_objects newdb
```

-v, --verbose

This option shows how to display detailed information on the tables and records of the database being loaded while the database loading operation is performed. You can check the detailed information such as the progress, the class being loaded and the number of records to be entered.

```
cubrid loaddb -v -d demodb_objects newdb
```

-c, --periodic-commit=COUNT

This option commits periodically every time COUNT records are entered into the database. If this option is not specified, all records included in demodb_objects are loaded to the database before the transaction is committed. If this option is used together with the **-s** or **-i** option, commit is performed periodically every time 100 DDL statements are loaded.

The recommended commit interval varies depending on the data to be loaded. It is recommended that the parameter of the **-c** option be configured to 50 for schema loading and 1 for index loading. For record loading, the recommended and default value is 10240.

The previous versions of **CUBRID**, like 10.1 or older, did not have a default value for **--periodic-commit** and would commit the transation only after loading the whole data. With the current approach, if you want to use the replicate the same behavior, you have to set the **--periodic-commit** argument to a very high value, so that the commit is done after loading all the data.

```
cubrid loaddb -c 100 -d demodb_objects newdb
```

--no-oid

The following is a command that loads records into newdb ignoring the OIDs in demodb_objects.

```
cubrid loaddb --no-oid -d demodb_objects newdb
```

--no-statistics

The following is a command that does not update the statistics information of newdb after loading demodb_objects. It is useful especially when small data is loaded to a relatively big database; you can improve the load performance by using this command.

```
cubrid loaddb --no-statistics -d demodb_objects newdb
```

-s, --schema-file=FILE[:LINE]

This option loads the schema information defined in the schema file, from the LINE-th. You can load the actual records after loading the schema information first by using the **-s** option.

On the following example, demodb_schema is a file created by the unload operation and contains the schema information of the unloaded database.

```
    cubrid loaddb -u dba -s demodb_schema newdb

    Start schema loading.
    Total       86 statements executed.
    Schema loading from demodb_schema finished.
    Statistics for Catalog classes have been updated.

The following loads the triggers defined in *demodb* into the
newly created newdb database. demodb_trigger is a file created
by the unload operation and contains the trigger information of
the unloaded database. It is recommended to load the schema
information after loading the records. ::

    cubrid loaddb -u dba -s demodb_trigger newdb
```

-i, --index-file=FILE[:LINE]

The following loads the index information defined in the index file, from the LINE-th. On the following example, demodb_indexes is a file created by the unload operation and contains the index information of the unloaded database. You can create indexes with the **-i** option, after loading records with the **-d** option.

```
cubrid loaddb -c 100 -d demodb_objects newdb
cubrid loaddb -u dba -i demodb_indexes newdb
```

-d, --data-file=FILE

This option loads the record information into newdb by specifying the data file or the user-defined object file. demodb_objects is either an object file created by the unload operation or a user-defined object file written by the user for mass data loading.

```
cubrid loaddb -u dba -d demodb_objects newdb
```

-t, --table=TABLE

This option specifies the table name if a table name header is omitted in the data file to be loaded. The table name must be prefixed with the schema name.

```
cubrid loaddb -u dba -d demodb_objects -t tbl_name newdb
```

--error-control-file=FILE

This option specifies the file that describes how to handle specific errors occurring during database loading.

```
cubrid loaddb --error-control-file=error_test -d demodb_objects
newdb
```

For the server error code name, see the **$CUBRID/include/dbi.h** file.

For error messages by error code (error number), see the number under $set 5 MSGCAT_SET_ERROR in the **$CUBRID/msg/***<character set name>***/cubrid.msg** file.

```
vi $CUBRID/msg/en_US/cubrid.msg

$set 5 MSGCAT_SET_ERROR
1 Missing message for error code %1$d.
2 Internal system failure: no more specific information is
available.
3 Out of virtual memory: unable to allocate %1$ld memory bytes.
4 Has been interrupted.
...
670 Operation would have caused one or more unique constraint
violations.
...
```

The format of a file that details specific errors is as follows:

- -<error code>: Configures to ignore the error that corresponds to the <error code> (**loaddb** is continuously executed even when an error occurs while it is being executed).
- +<error code>: Configures not to ignore the error that corresponds to the <error code> (**loaddb** is stopped when an error occurs while it is being executed).
- +DEFAULT: Configures not to ignore errors from 24 to 33.

If the file that details errors is not specified by using the **--error-control-file** option, the **loaddb** utility is configured to ignore errors from 24 to 33 by default. As a warning error, it indicates that there is no enough space in the database volume. If there is no space in the assigned database volume, a generic volume is automatically created.

The following example shows a file that details errors.

- The warning errors from 24 to 33 indicating DB volume space is insufficient are not ignored by configuring +DEFAULT.
- The error code 2 is not ignored because +2 has been specified later, even when -2 has been specified first.
- -670 has been specified to ignore the error code 670, which is a unique violation error.
- #-115 has been processed as a comment since # is added.

```
vi error_file

+DEFAULT
-2
-670
#-115 --> comment
+2
```

--ignore-class-file=FILE

You can specify a file that lists classes to be ignored during loading records. All records of classes except ones specified in the file will be loaded.

```
cubrid loaddb --ignore-class-file=skip_class_list -d
demodb_objects newdb
```

--trigger-file=FILE[:LINE]

This option loads the trigger information defined in the trigger file, from the LINE-th.

```
cubrid loaddb --trigger-file=demodb_triggers -d demodb_objects
newdb
```

--no-user-specified-name

Files unloaded in a version older than 11.2 can be loaded in version 11.2 only if the **--no-user-specified-name** option is used.

This option works if **DBA** is specified as the user account of the database with the -u option.

```
cubrid loaddb -u dba -s demodb_schema -d demodb_objects --no-
user-specified-name demodb
```

⚠ Warning

The **--no-user-specified-name** option must not be used to load a file unloaded in version 11.2 or after version 11.2.

--schema-file-list=FILE

This option loads all schema files included in the list at once. There must be a list of schema files, and they are loaded sequentially from the top schema file to the bottom of this list file. This option and the -s option cannot be used together.

This list file is automatically created when using the **--split-schema-files** option in unloaddb.

```
cubrid loaddb -u dba --schema-file-list=demodb_schema_info
```

⚠ Warning

You can change the order of schema files written in this file, but loading errors may occur if the order is wrong. For example, an error may occur if the PK file is loaded first while the file with CLASS information is not loaded. Therefore, in the list, the filename for loading CLASS information must be higher than the file for loading PK.

⚠ Warning

The **--no-logging** option enables to load data file quickly when **loaddb** is executed by not storing transaction logs; however, it has risk, which data cannot be recovered in case of errors occurred such as incorrect file format or system failure. In this case, you must rebuild database to solve the problem. Thus, in general, it is not recommended to use this option exception of building a new database which does not require data recovery. If you use this option, **loaddb** does not check the errors like unique violation. To use this option, you should consider these issues.

⚠ Warning

When running **loaddb**, foreign key constraints are not checked to ensure performance for loading large amounts of data and to minimize dependencies between loaded data. So, the

data inconsistencies with the master node may occur because foreign key constraints are checked during the replication process of the slave or replica node in the HA environment if the **-C** option is used.

How to Write Files to Load Database

You can add mass data to the database more rapidly by writing the object input file used in the **cubrid loaddb** utility. An object input file is a text file in simple table form that consists of comments and command/data lines.

Comment

In CUBRID, a comment is represented by two hyphens (--).

```
-- This is a comment!
```

Command Line

A command line begins with a percent character (%) and consists of **%class** and **%id** commands; the former defines classes, and the latter defines aliases and identifiers used for class identification.

- Assigning an Identifier to a Class

  You can assign an identifier to class reference relations by using the **%id** command.

  ```
  %id [schema_name.]class_name class_id
  schema_name:
      identifier
  class_name:
      identifier
  class_id:
      integer
  ```

  The *class_name* specified by the **%id** command is the class name defined in the database, and *class_id* is the numeric identifier which is assigned for object reference. The *class_name* uses the *schema_name* as a prefix and the *schema_name* is the name of the schema in which the class is defined. The *schema_name* can be omitted, and if the *schema_name* is omitted, the username of the database specified with the -u option is used as the schema name.

```
%id [employee] 21
%id [office] 22
%id [project] 23
%id [phone] 24
```

The *schema_name* and *class_name* must be enclosed in square brackets ([ ]). Each of *schema_name* and *class_name* or together can be enclosed in square brackets.

```
%id [public].[employee] 21
%id [public].[office] 22
%id [public].[project] 23
%id [public].[phone] 24
```

```
%id [public.employee] 21
%id [public.office] 22
%id [public.project] 23
%id [public.phone] 24
```

- Specifying the Class and Attribute

You can specify the classes (tables) and attributes (columns) upon loading data by using the **%class** command. The data line should be written based on the order of attributes specified. When a class name is provided by using the **-t** option while executing the **cubrid loaddb** utility, you don't have to specify the class and attribute in the data file. However, the order of writing data must comply with the order of the attribute defined when creating a class.

```
%class [schema_name.]class_name (attr_name [attr_name...])
schema_name:
    identifier
class_name:
    identifier
attr_name:
    identifier
```

The class must be pre-defined in the database to be loaded.

The *class_name* specified by the **%class** command is the class name defined in the database and the *schema_name* is the name of the schema in which the class is defined. The *attr_name* is the name of the attribute defined. The method of writing *schema_name* and *class_name* is the same as above.

The following example shows how to specify a class and three attributes by using the **%class** command to enter data into a class named *employee*. Three pieces of data should be entered on the data lines after the **%class** command. For this, see Configuring Reference Relation.

```
%class [public].[employee] (name age department)
```

Data Line

A data line comes after the **%class** command line. Data loaded must have the same type as the class attributes specified by the **%class** command. The data loading operation stops if these two types are different.

Data for each attribute must be separated by at least one space and be basically written as a single line. However, if the data to be loaded takes more than one line, you should specify the plus sign (+) at the end of the first data line to enter data continuously on the following line. Note that no space is allowed between the last character of the data and the plus sign.

- Loading an Instance

  As shown below, you can load an instance that has the same type as the specified class attribute. Each piece of data is separated by at least one space.

```
%class [public].[employee] (name)
'jordan'
'james'
'garnett'
'malone'
```

- Assigning an Instance Number

  You can assign a number to a given instance at the beginning of the data line. An instance number is a unique positive number in the specified class. Spaces are not allowed between the number and the colon (:). Assigning an instance number is used to configure the reference relation for later.

```
%class [public].[employee] (name)
1: 'jordan'
2: 'james'
3: 'garnett'
4: 'malone'
```

- Configuring Reference Relation

  You can configure the object reference relation by specifying the reference class after an “at sign (**@**)” and the instance number after the “vertical line (|).”

```
@class_ref | instance_no
class_ref:
```

```
    [schema_name.]class_name
    class_id
```

Specify a class name or a class id after the **@** sign, and an instance number after a vertical line (|). Spaces are not allowed before and after a vertical line (|). The method of wirting *class_ref* is the same as the method of wirting *schema_name* and *class_name* in the Command Line. However, enclosing each of *schema_name* and *class_name* in square brackets ([ ]) is not allowed.

The following example shows how to load class instances into the *paycheck* class. The *name* attribute references an instance of the *employee* class. As in the last line, data is loaded as **NULL** if you configure the reference relation by using an instance number not specified earlier.

```
%class [public].[paycheck] (name department salary)
@[public.employee]|1   'planning'   8000000
@[public.employee]|2   'planning'   6000000
@[public.employee]|3   'sales'   5000000
@[public.employee]|4   'development'   4000000
@[public.employee]|5   'development'   5000000
```

Since the id 21 was assigned to the *employee* class by using the **%id** command in the Assigning an Identifier to a Class section, the above example can be written as follows:

```
%class [public].[paycheck] (name department salary)
@21|1   'planning'   8000000
@21|2   'planning'   6000000
@21|3   'sales'   5000000
@21|4   'development'   4000000
@21|5   'development'   5000000
```

Migrating Database

To use a new version of CUBRID database, you may need to migrate an existing data to a new one. For this purpose, you can use the “Export to an ASCII text file” and “Import from an ASCII text file” features provided by CUBRID.

The following section explains migration steps using the **cubrid unloaddb** and **cubrid loaddb** utilities.

Recommended Scenario and Procedures

The following steps describes migration scenario that can be applied while the existing version of CUBRID is running. For database migration, you should use the **cubrid unloaddb** and **cubrid loaddb** utilities. For details, see unloaddb and loaddb.

1. Stop the existing CUBRID service

    Execute **cubrid service stop** to stop all service processes running on the existing CUBRID and then check whether all CUBRID-related processes have been successfully stopped.

    To verify whether all CUBRID-related processes have been successfully stopped, execute **ps -ef | grep cub_** in Linux. If there is no process starting with cub_, all CUBRID-related processes have been successfully stopped. In Windows, press the <Ctrl+Alt+Delete> key and select [Start Task Manager]. If there is no process starting with cub_ in the [Processes] tab, all CUBRID-related processes have been successfully stopped. In Linux, when the related processes remain even after the CUBRID service has been terminated, use **kill** command to forcibly terminate them, and use **ipcs -m** command to check and release the memory shard by CUBRID broker. To forcibly terminate related processes in Windows, go to the [Processes] tab of Task Manager, right-click the image name, and then select [End Process].

2. Back up the existing database

    Perform backup of the existing version of the database by using the **cubrid backupdb** utility. The purpose of this step is to safeguard against failures that might occur during the database unload/load operations. For details on the database backup, see backupdb.

3. Unload the existing database

    Unload the database created for the existing version of CUBRID by using the **cubrid unloaddb** utility. For details on unloading a database, see unloaddb .

4. Store the existing CUBRID configuration files

    Store the configurations files such as **cubrid.conf**, **cubrid_broker.conf** and **cm.conf ** in the **CUBRID/conf** directory. The purpose of this step is to conveniently apply parameter values for the existing CUBRID database environment to the new one.

5. Install a new version of CUBRID

    Once backing up and unloading of the data created by the existing version of CUBRID have been completed, delete the existing version of CUBRID and its databases and then install the new version of CUBRID. For details on installing CUBRID, see Getting Started.

6. Configure the new CUBRID environment

Configure the new version of CUBRID by referring to configuration files of the existing database stored in the step 3, " **Store the existing CUBRID configuration files** ." For details on configuring new environment, see Installing and Running CUBRID in "Getting Started."

7. Load the new database

Create a database by using the **cubrid createdb** utility and then load the data which had previously been unloaded into the new database by using the **cubrid loaddb** utility. Please see createdb for creating a database and loaddb for loading a database.

8. Back up the new database

Once the data has been successfully loaded into the new database, back up the database created for the new version of CUBRID by using the **cubrid backupdb** utility. The reason for this step is because you cannot restore the data backed up in the existing version of CUBRID when using the new version. For details on backing up the database, see backupdb .

⚠ Warning

Even if the version is identical, the 32-bit database volume and the 64-bit database volume are not compatible for backup and recovery. Therefore, it is not recommended to recover a 32-bit database backup on the 64-bit CUBRID or vice versa.

⚠ Warning

When using the TDE feature in CUBRID 11, it does not provide backward compatibility. So, the unloaded file in CUBRID 11 cannot be loaded into a lower version if more the one TDE table exists.

spacedb

The **cubrid spacedb** utility is used to check how much space of database volumes is being used. The tool can show brief aggregated information on database space usage, or detailed descriptions of all volumes and files in use, based on its options. Information returned by the **cubrid spacedb** utility includes the volume ID's, names, purpose and total/free space of each volume.

```
cubrid spacedb [options] database_name
```

- **cubrid** : An integrated utility for the CUBRID service and database management.
- **spacedb** : A command that checks the space in the database. It executes successfully only when the database is in a stopped state.
- *database_name* : The name of the database whose space is to be checked. The path-name to the directory where the database is to be created must not be included.

The following shows [options] available with the **cubrid spacedb** utility.

```
-o, --output-file=FILE       redirect output messages to FILE
-S, --SA-mode                stand-alone mode execution
-C, --CS-mode                client-server mode execution
    --size-unit={PAGE|M|G|T|H} set the display unit (page, mega,
giga, tera, human readable)
-s, --summarize              display only a summary for each volume
purpose
-p, --purpose                display the used size of each
purpose(data/index/temp) of each volume
```

-o FILE

This option stores the result of checking the space information of *testdb* to a file named *db_output*.

```
cubrid spacedb -o db_output testdb
```

-S, --SA-mode

This option is used to access a database in standalone, which means it works without processing server; it does not have an argument. If **-S** is not specified, the system recognizes that a database is running in client/server mode.

```
cubrid spacedb --SA-mode testdb
```

-C, --CS-mode

This option is used to access a database in client/server mode, which means it works in client/server process respectively; it does not have an argument. If **-C** is not specified, the system recognize that a database is running in client/server mode by default.

```
cubrid spacedb --CS-mode testdb
```

--size-unit={PAGE|M|G|T|H}

This option specifies the size unit of the space information of the database to be one of PAGE, M(MB), G(GB), T(TB), H(print-friendly). The default value is **H**. If you set the value to H, the unit is automatically determined as follows: M if 1 MB = DB size < 1024 MB, G if 1 GB = DB size < 1024 GB.

```
$ cubrid spacedb --size-unit=H testdb

Space description for database 'testdb' with pagesize 16.0K.
(log pagesize: 16.0K)

type                purpose            volume_count
used_size           free_size           total_size
PERMANENT           PERMANENT DATA                2
61.0 M             963.0 M                1.0 G
PERMANENT           TEMPORARY DATA                1
12.0 M             500.0 M              512.0 M
TEMPORARY           TEMPORARY DATA                1
40.0 M              88.0 M               128.0 M
-                   -                             4
113.0 M               1.5 G                1.6 G

Space description for all volumes:
volid               type                purpose
used_size           free_size           total_size
volume_name
    0               PERMANENT           PERMANENT DATA
60.0 M             452.0 M              512.0 M         /home1/
cubrid/testdb
    1               PERMANENT           PERMANENT DATA
1.0 M              511.0 M             512.0 M         /home1/
cubrid/testdb_x001
    2               PERMANENT           TEMPORARY DATA
12.0 M             500.0 M              512.0 M         /home1/
cubrid/testdb_x002
32766               TEMPORARY           TEMPORARY DATA
40.0 M              88.0 M               128.0 M         /home1/
cubrid/testdb_t32766

LOB space description file:/home1/cubrid/lob
```

-s, --summarize

This option aggregates volume count, used size, free size and total size by volume types and purposes. There are three classes of volumes: permanent volumes with permanent data, permanent volumes with temporary data and temporary volume with temporary data; no temporary volumes with permanent data. Last row shows the total values for all types of volumes.

```
$ cubrid spacedb -s testdb

Space description for database 'testdb' with pagesize 16.0K.
(log pagesize: 16.0K)

type                purpose            volume_count
used_size           free_size           total_size
PERMANENT           PERMANENT DATA                2
61.0 M             963.0 M                1.0 G
PERMANENT           TEMPORARY DATA                1
12.0 M             500.0 M              512.0 M
TEMPORARY           TEMPORARY DATA                1
40.0 M              88.0 M              128.0 M
-                   -                             4
113.0 M               1.5 G                1.6 G
```

-p, --purpose

This option shows detailed information on the purpose of stored data. The information includes number of files, used size, size of file tables, reserved sectors size and total size.

```
Space description for database 'testdb' with pagesize 16.0K.
(log pagesize: 16.0K)

Detailed space description for files:
data_type           file_count           used_size
file_table_size     reserved_size       total_size
INDEX                       17                0.3
M                    0.3 M            16.5 M           17.0 M
HEAP                        28                7.6
M                    0.4 M            26.0 M           34.0 M
SYSTEM                       8                0.4
M                    0.1 M             7.5 M            8.0 M
TEMP                        10                0.0
M                    0.2 M            49.8 M           50.0 M
-                           63                8.2
M                    1.0 M            99.8 M          109.0 M
```

compactdb

The **cubrid compactdb** utility is used to secure unused space of the database volume. In case the database server is not running (offline), you can perform the job in standalone mode. In case the database server is running, you can perform it in client-server mode.

Note

The **cubrid compactdb** utility secures the space being taken by OIDs of deleted objects and by class changes. When an object is deleted, the space taken by its OID is not immediately freed because there might be other objects that refer to the deleted one.

Therefore, when you make a table to reuse OIDs, it is recommended to use a REUSE_OID option as below.

```
CREATE TABLE tbl REUSE_OID
(
    id INT PRIMARY KEY,
    b VARCHAR
);
```

However, a table with a REUSE_OID option cannot be referred by the other table. That is, this table cannot be used as a type of the other table.

```
CREATE TABLE reuse_tbl (a INT PRIMARY KEY) REUSE_OID;
CREATE TABLE tbl_1 ( a reuse_tbl);
```

```
ERROR: The class 'reuse_tbl' is marked as REUSE_OID and is non-
referable. Non-referable classes can't be the domain of an
attribute and their instances' OIDs cannot be returned.
```

To see details of REUSE_OID, please refer to REUSE_OID.

Reference to the object deleted during compacting is displayed as **NULL**, which means this can be reused by OIDs.

```
cubrid compactdb [options] database_name [schema_name.class_name [{,
schema_name.class_name}]]
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **compactdb**: A command that compacts the space of the database so that OIDs assigned to deleted data can be reused.
- *database_name*: The name of the database whose space is to be compacted. The path name to the directory where the database is to be created must not be included.
- *class_name_list*: You can specify the list of tables names that you want to compact space after a database name; the **-i** option cannot be used together. The table name must be prefixed with

the schema name. If you use the lists on client/server mode, it skips securing space taken by objects such as catalog, delete files and tracker, etc.

-I, **-c**, **-d**, **-p** options are applied in client/server mode only.

The following shows [options] available with the **cubrid compactdb** utility.

```
-v, --verbose                    enable verbose status messages
-S, --SA-mode                    stand-alone mode execution
-C, --CS-mode                    client-server mode execution
-i, --input-class-file=FILE      input FILE of table names
-p, --pages-committed-once=NUMBER maximum number of pages to commit
once
-d, --delete-old-repr            delete old class representations
from catalog
-I, --Instance-lock-timeout      timeout for instance lock
-c, --class-lock-timeout         timeout for class lock
```

-v, --verbose

You can output messages that shows which class is currently being compacted and how many instances have been processed for the class by using the **-v** option.

```
cubrid compactdb -v testdb
```

-S, --SA-mode

This option specifies to compact used space in standalone mode while database server is not running; no argument is specified. If the **-S** option is not specified, system recognizes that the job is executed in client/server mode.

```
cubrid compactdb --SA-mode testdb
```

-C, --CS-mode

This option specifies to compact used space in client/server mode while database server is running; no argument is specified. Even though this option is omitted, system recognizes that the job is executed in client/server mode.

-i, --input-class-file=FILE

You can specify an input filename that contains the table name with this option. Write one table name in a single line; invalid table name is ignored. The table name must be prefixed with the schema name. Note that you cannot specify the list of the table names after a database name in case of you use this option. If you use this option on client/server mode, it skips securing space taken by objects such as catalog, delete files and tracker, etc.

The following options can be used in client/server mode only.

-p, --pages-committed-once=NUMBER

You can specify the number of maximum pages that can be committed once with this option. The default value is 10, the minimum value is 1, and the maximum value is 10. The less option value is specified, the more concurrency is enhanced because the value for class/instance lock is small; however, it causes slowdown on operation, and vice versa.

```
cubrid compactdb --CS-mode -p 10 testdb public.tbl1,
public.tbl2, public.tbl5
```

-d, --delete-old-repr

You can delete an existing table representation (schema structure) from catalog with this option. Generally you'd better keep the existing table representation because schema updating cost will be saved when you keep the status as referring to the past schema for the old records.

-I, --Instance-lock-timeout=NUMBER

You can specify a value of instance lock timeout with this option. The default value is 2 (seconds), the minimum value is 1, and the maximum value is 10. The less option value is specified, the more operation speeds up. However, the number of instances that can be processed becomes smaller, and vice versa.

-c, --class-lock-timeout=NUMBER

You can specify a value of instance lock timeout with this option. The default value is 10 (seconds), the minimum value is 1, and the maximum value is 10. The less option value is specified, the more operation speeds up. However, the number of tables that can be processed becomes smaller, and vice versa.

optimizedb

Updates statistical information such as the number of objects, the number of pages to access, and the distribution of attribute values.

```
cubrid optimizedb [<option>] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **optimizedb**: Updates the statistics information, which is used for cost-based query optimization of the database. If the option is specified, only the information of the specified class is updated.
- *database_name*: The name of the database whose cost-based query optimization statistics are to be updated.

The following shows <option> available with the **cubrid optimizedb** utility.

-n, --class-name

The following example shows how to update the query statistics information of the given class by using the **-n** option. The table name must be prefixed with the schema name.

```
cubrid optimizedb -n public.event_table testdb
```

The following example shows how to update the query statistics information of all classes in the database.

```
cubrid optimizedb testdb
```

plandump

The **cubrid plandump** utility is used to display information on the query plans stored (cached) on the server.

```
cubrid plandump [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **plandump**: A utility that displays the query plans stored in the current cache of a specific database.
- *database_name*: The name of the database where the query plans are to be checked or dropped from its server cache.

The following shows [options] available with the **cubrid plandump** utility.

```
-d, --drop                  drop all plans in the server's cache
-o, --output-file=FILE      redirect output messages to FILE
-s, --sha1=SHA1             drop a specific plan using its SHA1
hash from the server’s cache
```

-d, --drop

This option drops the query plans stored in the cache.

```
cubrid plandump -d testdb
```

-o, --output-file=FILE

This option stores the results of the query plans stored in the cache to a file.

```
cubrid plandump -o output.txt testdb
```

-s, --sha1=SHA1

This option drops specific query plans identified by their SHA1 hashes from the server's cache.

```
$ cubrid plandump testdb

Entries:

  XASL_ID = {
              sha1 = { da9cab5d 597f2fe4 48aa8ae8 66b87d81
49fdd7ca },
                   time_stored = 1718614753 sec, 257499 usec
            }
  sql info:
    SQL_ID = 4dfa6d901e048
    sql user text = select * from game where host_year > '2004'
    sql plan text = Sequential scan(public.game dba.game)

cubrid plandump -s 'da9cab5d 597f2fe4 48aa8ae8 66b87d81
49fdd7ca' demodb
```

By default, it examines the query plans stored in the plan cache if no options are specified.

```
cubrid plandump testdb
```

```
XASL cache
Stats:
Max size:                   1000        -- Maximum number of plan
cache entries allowed
Current entry count:        4           -- Number of currently
cached execution plans
Lookups:                    25          -- Number of plan cache
lookup attempts
Hits:                       15          -- Number of successful
lookups where the plan already existed
Miss:                       10          -- Number of failed lookups
Inserts:                    5
Found at insert:            0
Recompiles:                 0
Failed recompiles:          0
Deletes:                    1
Fix:                        15
```

```
Unfix:                      20
Cache cleanups:             0
Deletes at cleanup:     0

Entries:

XASL_ID = {
          sha1 = { 939db0ab 7ead8a87 2a1ed142 67e2f059 36a3e601 },
          time_stored = 1750657813 sec, 784232 usec
        }
fix_count = 0
cache flags = 00000000
reference count = 4
time second last used = 1750657814
clone count = 1
sql info:
    SQL_ID = 67d47c14f637a
    sql user text = select * from game where host_year > '2004'
    sql hash text = select [dba.game].[host_year], [dba.game].
[event_code], [dba.game].[athlete_code], [dba.game].[stadium_code],
[dba.game].[nation_code], [dba.game].[medal], [dba.game].[game_date]
from [dba.game] [dba.game] w          here ([dba.game].
[host_year]> ?:0 )?193="en_US";194="en_US";249="Asia/Seoul";user=0|
897|1;bind_var_cnt=1
            sql plan text =
Sequential scan(public.game dba.game)
OID_LIST (count = 1):
            OID = 0|208|10, LOCK =     IS_LOCK, TCARD =       31
```

CUBRID supports the reuse of SQL execution plans through the Plan Cache feature. The Plan Cache is managed on the server side and operates through cooperation between the Common Application Server (CAS) and the database server. The main processing steps are as follows:

1. Query Analysis (Parsing and Rewrite) : When an SQL query is sent from the client, it is first parsed and rewritten by the CAS. During this step, the query is analyzed and internally transformed for optimization and normalization purposes.

2. XASL_ID Generation : Based on the rewritten query, a XASL_ID is generated. This XASL_ID serves as a unique identifier for the query and is used to check whether the corresponding execution plan (XASL) is already stored in the server's Plan Cache.

3. Plan Cache Lookup : Using the generated XASL_ID, the CAS queries the server to determine if a matching XASL (Execution Plan) already exists in the Plan Cache.

    - If the XASL is found: The server uses the existing plan to execute the query.

- If the XASL is not found: The CAS generates a new Tree(Execution Plan) and sends it to the server, which may then decide whether or not to store it in the Plan Cache.

Note

The Plan Cache improves performance by reusing XASL(execution plan).

The following describes **the detailed fields** included in the output of plandump:

- **XASL_IDThe SQL hash text is converted into a SHA1 hash string, which is then used as the key for the plan cache.**
  - time_stored: The timestamp indicating when the plan was saved in the plan cache. It is represented as a Unix timestamp and can be converted to a human-readable format using the date command.
  - ex) $> date -d @1750657813.784232 (Mon Jun 23 14:50:13 KST 2025)
- fix_count : Number of concurrently used threads
- **cache flagsStatus information of the cache entry**
  - 8000000 : mark_deleted
  - 4000000 : to be recompiled
  - 2000000 : was recompiled
  - 0800000 : clean up
  - 0400000 : recomiled requested
- reference count: The number of times the execution plan has been referenced (i.e., reused) by queries.
- **time second last used: The UNIX timestamp indicating when this plan was last used. You can convert it to human-readable format using:date -d @<timestamp>**
  - ex) $> date -d @1750657814 (Mon Jun 23 14:50:14 KST 2025)
- clone count: The number of clone instances created from this plan.When an execution plan is stored in memory as a stream format, it must be converted to an XASL (eXtended Access Specification Language) structure before execution.To reduce this transformation cost, a clone cache is used.The clone cache holds reusable XASL structures.If a thread requests a clone and none are available, a new clone is created and added to the cache. Each time this happens, the clone count increases.

- **SQL_ID: A unique identifier generated from the original SQL statement using the MD5 hash function.From the resulting 32-character hexadecimal hash, the last 13 hexadecimal characters are used as the SQL_ID.**
    - If the MD5 hash value is e1faffb3e614e6c2fba74296962386b7, the corresponding SQL ID is the last 13 characters of the hash: 74296962386b7.
- sql user text : The original SQL query submitted by the user.
- sql hash text : The rewritten form of the SQL statement used for caching and plan reuse.
- sql plan text : The text representation of the execution plan generated for the SQL.
- OID_LIST : A list of Object ID for database objects (e.g., tables, serials) referenced by this execution plan.

Note

- You can convert a Unix timestamp to a human-readable date and time format using the date command.

statdump

cubrid statdump utility checks statistics information processed by the CUBRID database server. The statistics information mainly consists of the following: File I/O, Page buffer, Logs, Transactions, Concurrency/Lock, Index, and Network request.

You can also use CSQL's session commands to check the statistics information only about the CSQL's connection. For details, see Dumping CSQL execution statistics information.

```
cubrid statdump [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **statdump**: A command that dumps the statistics information on the database server execution.
- *database_name*: The name of database which has the statistics data to be dumped.

The following shows [options] available with the **cubrid statdump** utility.

```
-i, --interval=S            print stats every S secs
-o, --output-file=FILE      redirect output message to FILE
-c, --cumulative            print cumulative stats
```

```
-s, --substr=STRING          print stat item only that match the
substring
```

-i, --interval=SECOND

This option specifies the periodic number of Dumping statistics as seconds.

The following outputs the accumulated values per second.

```
cubrid statdump -i 1 -c demodb
```

The following outputs the accumulated values during 1 second, as starting with 0 value per every 1 second.

```
cubrid statdump -i 1 demodb
```

The following outputs the last values which were executed with **-i** option.

```
cubrid statdump demodb
```

The following outputs the same values with the above. **-c** option doesn't work if it is not used with **-i** option together.

cubrid statdump -c demodb

The following outputs the values per every 5 seconds.

```
$ cubrid statdump -i 5 -c testdb


Mon November 11 23:44:36 KST 2019

 *** SERVER EXECUTION STATISTICS ***
Num_file_creates              =          0
Num_file_removes              =          0
Num_file_ioreads              =          0
Num_file_iowrites             =          3
Num_file_iosynches            =          3
The timer values for file_iosync_all are:
Num_file_iosync_all           =          0
Total_time_file_iosync_all    =          0
Max_time_file_iosync_all      =          0
Avg_time_file_iosync_all      =          0
Num_file_page_allocs          =          0
Num_file_page_deallocs        =          0
Num_data_page_fetches         =          0
Num_data_page_dirties         =          0
```

```
Num_data_page_ioreads          =           0
Num_data_page_iowrites         =           0
Num_data_page_flushed          =           0
Num_data_page_private_quota    =       11327
Num_data_page_private_count    =           0
Num_data_page_fixed            =           1
Num_data_page_dirty            =           0
Num_data_page_lru1             =          18
Num_data_page_lru2             =          10
Num_data_page_lru3             =           0
Num_data_page_victim_candidate =           0
Num_log_page_fetches           =           0
Num_log_page_ioreads           =           0
Num_log_page_iowrites          =           6
Num_log_append_records         =           9
Num_log_archives               =           0
Num_log_start_checkpoints      =           0
Num_log_end_checkpoints        =           0
Num_log_wals                   =           0
Num_log_page_iowrites_for_replacement =           0
Num_log_page_replacements      =           0
Num_page_locks_acquired        =           0
Num_object_locks_acquired      =           0
Num_page_locks_converted       =           0
Num_object_locks_converted     =           0
Num_page_locks_re-requested    =           0
Num_object_locks_re-requested  =           0
Num_page_locks_waits           =           0
Num_object_locks_waits         =           0
Num_object_locks_time_waited_usec =           0
Num_tran_commits               =           0
Num_tran_rollbacks             =           0
Num_tran_savepoints            =           0
Num_tran_start_topops          =           0
Num_tran_end_topops            =           0
Num_tran_interrupts            =           0
Num_btree_inserts              =           0
Num_btree_deletes              =           0
Num_btree_updates              =           0
Num_btree_covered              =           0
Num_btree_noncovered           =           0
Num_btree_resumes              =           0
Num_btree_multirange_optimization =           0
Num_btree_splits               =           0
Num_btree_merges               =           0
Num_btree_get_stats            =           0
Num_btree_online_inserts       =           0
Num_btree_online_inserts_same_page_hold =           0
Num_btree_online_inserts_reject_no_more_keys =           0
Num_btree_online_inserts_reject_max_key_len =           0
Num_btree_online_inserts_reject_no_space =           0
Num_btree_online_release_latch =           0
```

```
Num_btree_online_inserts_reject_key_not_in_range1 =           0
Num_btree_online_inserts_reject_key_not_in_range2 =           0
Num_btree_online_inserts_reject_key_not_in_range3 =           0
Num_btree_online_inserts_reject_key_not_in_range4 =           0
Num_btree_online_inserts_reject_key_false_failed_range1
=          0
Num_btree_online_inserts_reject_key_false_failed_range2
=          0
The timer values for btree_online are:
Num_btree_online              =          0
Total_time_btree_online       =          0
Max_time_btree_online         =          0
Avg_time_btree_online         =          0
The timer values for btree_online_insert_task are:
Num_btree_online_insert_task  =          0
Total_time_btree_online_insert_task =          0
Max_time_btree_online_insert_task =          0
Avg_time_btree_online_insert_task =          0
The timer values for btree_online_prepare_task are:
Num_btree_online_prepare_task =          0
Total_time_btree_online_prepare_task =          0
Max_time_btree_online_prepare_task =          0
Avg_time_btree_online_prepare_task =          0
The timer values for btree_online_insert_same_leaf are:
Num_btree_online_insert_same_leaf =          0
Total_time_btree_online_insert_same_leaf =          0
Max_time_btree_online_insert_same_leaf =          0
Avg_time_btree_online_insert_same_leaf =          0
Num_query_selects             =          0
Num_query_inserts             =          0
Num_query_deletes             =          0
Num_query_updates             =          0
Num_query_sscans              =          0
Num_query_iscans              =          0
Num_query_lscans              =          0
Num_query_setscans            =          0
Num_query_methscans           =          0
Num_query_nljoins             =          0
Num_query_mjoins              =          0
Num_query_objfetches          =          0
Num_query_holdable_cursors    =          0
Num_sort_io_pages             =          0
Num_sort_data_pages           =          0
Num_network_requests          =          3
Num_adaptive_flush_pages      =          0
Num_adaptive_flush_log_pages  =          0
Num_adaptive_flush_max_pages  =      14464
Num_prior_lsa_list_size       =          0
Num_prior_lsa_list_maxed      =          0
Num_prior_lsa_list_removed    =          3
Time_ha_replication_delay     =          0
Num_plan_cache_add            =          0
```

```
Num_plan_cache_lookup         =          0
Num_plan_cache_hit            =          0
Num_plan_cache_miss           =          0
Num_plan_cache_full           =          0
Num_plan_cache_delete         =          0
Num_plan_cache_invalid_xasl_id =          0
Num_plan_cache_entries        =          0
Num_vacuum_log_pages_vacuumed =          0
Num_vacuum_log_pages_to_vacuum =          0
Num_vacuum_prefetch_requests_log_pages =          0
Num_vacuum_prefetch_hits_log_pages =          0
Num_heap_home_inserts         =          0
Num_heap_big_inserts          =          0
Num_heap_assign_inserts       =          0
Num_heap_home_deletes         =          0
Num_heap_home_mvcc_deletes    =          0
Num_heap_home_to_rel_deletes  =          0
Num_heap_home_to_big_deletes  =          0
Num_heap_rel_deletes          =          0
Num_heap_rel_mvcc_deletes     =          0
Num_heap_rel_to_home_deletes  =          0
Num_heap_rel_to_big_deletes   =          0
Num_heap_rel_to_rel_deletes   =          0
Num_heap_big_deletes          =          0
Num_heap_big_mvcc_deletes     =          0
Num_heap_home_updates         =          0
Num_heap_home_to_rel_updates  =          0
Num_heap_home_to_big_updates  =          0
Num_heap_rel_updates          =          0
Num_heap_rel_to_home_updates  =          0
Num_heap_rel_to_rel_updates   =          0
Num_heap_rel_to_big_updates   =          0
Num_heap_big_updates          =          0
Num_heap_home_vacuums         =          0
Num_heap_big_vacuums          =          0
Num_heap_rel_vacuums          =          0
Num_heap_insid_vacuums        =          0
Num_heap_remove_vacuums       =          0
The timer values for heap_insert_prepare are:
Num_heap_insert_prepare       =          0
Total_time_heap_insert_prepare =          0
Max_time_heap_insert_prepare  =          0
Avg_time_heap_insert_prepare  =          0
The timer values for heap_insert_execute are:
Num_heap_insert_execute       =          0
Total_time_heap_insert_execute =          0
Max_time_heap_insert_execute  =          0
Avg_time_heap_insert_execute  =          0
The timer values for heap_insert_log are:
Num_heap_insert_log           =          0
Total_time_heap_insert_log    =          0
Max_time_heap_insert_log      =          0
```

```
Avg_time_heap_insert_log      =          0
The timer values for heap_delete_prepare are:
Num_heap_delete_prepare       =          0
Total_time_heap_delete_prepare =          0
Max_time_heap_delete_prepare  =          0
Avg_time_heap_delete_prepare  =          0
The timer values for heap_delete_execute are:
Num_heap_delete_execute       =          0
Total_time_heap_delete_execute =          0
Max_time_heap_delete_execute  =          0
Avg_time_heap_delete_execute  =          0
The timer values for heap_delete_log are:
Num_heap_delete_log           =          0
Total_time_heap_delete_log    =          0
Max_time_heap_delete_log      =          0
Avg_time_heap_delete_log      =          0
The timer values for heap_update_prepare are:
Num_heap_update_prepare       =          0
Total_time_heap_update_prepare =          0
Max_time_heap_update_prepare  =          0
Avg_time_heap_update_prepare  =          0
The timer values for heap_update_execute are:
Num_heap_update_execute       =          0
Total_time_heap_update_execute =          0
Max_time_heap_update_execute  =          0
Avg_time_heap_update_execute  =          0
The timer values for heap_update_log are:
Num_heap_update_log           =          0
Total_time_heap_update_log    =          0
Max_time_heap_update_log      =          0
Avg_time_heap_update_log      =          0
The timer values for heap_vacuum_prepare are:
Num_heap_vacuum_prepare       =          0
Total_time_heap_vacuum_prepare =          0
Max_time_heap_vacuum_prepare  =          0
Avg_time_heap_vacuum_prepare  =          0
The timer values for heap_vacuum_execute are:
Num_heap_vacuum_execute       =          0
Total_time_heap_vacuum_execute =          0
Max_time_heap_vacuum_execute  =          0
Avg_time_heap_vacuum_execute  =          0
The timer values for heap_vacuum_log are:
Num_heap_vacuum_log           =          0
Total_time_heap_vacuum_log    =          0
Max_time_heap_vacuum_log      =          0
Avg_time_heap_vacuum_log      =          0
The timer values for heap_stats_sync_bestspace are:
Num_heap_stats_sync_bestspace =          0
Total_time_heap_stats_sync_bestspace =          0
Max_time_heap_stats_sync_bestspace =          0
Avg_time_heap_stats_sync_bestspace =          0
Num_heap_stats_bestspace_entries =          0
```

```
Num_heap_stats_bestspace_maxed =          0
The timer values for bestspace_add are:
Num_bestspace_add             =          0
Total_time_bestspace_add      =          0
Max_time_bestspace_add        =          0
Avg_time_bestspace_add        =          0
The timer values for bestspace_del are:
Num_bestspace_del             =          0
Total_time_bestspace_del      =          0
Max_time_bestspace_del        =          0
Avg_time_bestspace_del        =          0
The timer values for bestspace_find are:
Num_bestspace_find            =          0
Total_time_bestspace_find     =          0
Max_time_bestspace_find       =          0
Avg_time_bestspace_find       =          0
The timer values for heap_find_page_bestspace are:
Num_heap_find_page_bestspace  =          0
Total_time_heap_find_page_bestspace =          0
Max_time_heap_find_page_bestspace =          0
Avg_time_heap_find_page_bestspace =          0
The timer values for heap_find_best_page are:
Num_heap_find_best_page       =          0
Total_time_heap_find_best_page =          0
Max_time_heap_find_best_page  =          0
Avg_time_heap_find_best_page  =          0
The timer values for bt_fix_ovf_oids are:
Num_bt_fix_ovf_oids           =          0
Total_time_bt_fix_ovf_oids    =          0
Max_time_bt_fix_ovf_oids      =          0
Avg_time_bt_fix_ovf_oids      =          0
The timer values for bt_unique_rlocks are:
Num_bt_unique_rlocks          =          0
Total_time_bt_unique_rlocks   =          0
Max_time_bt_unique_rlocks     =          0
Avg_time_bt_unique_rlocks     =          0
The timer values for bt_unique_wlocks are:
Num_bt_unique_wlocks          =          0
Total_time_bt_unique_wlocks   =          0
Max_time_bt_unique_wlocks     =          0
Avg_time_bt_unique_wlocks     =          0
The timer values for bt_leaf are:
Num_bt_leaf                   =          0
Total_time_bt_leaf            =          0
Max_time_bt_leaf              =          0
Avg_time_bt_leaf              =          0
The timer values for bt_traverse are:
Num_bt_traverse               =          0
Total_time_bt_traverse        =          0
Max_time_bt_traverse          =          0
Avg_time_bt_traverse          =          0
The timer values for bt_find_unique are:
```

```
Num_bt_find_unique            =          0
Total_time_bt_find_unique     =          0
Max_time_bt_find_unique       =          0
Avg_time_bt_find_unique       =          0
The timer values for bt_find_unique_traverse are:
Num_bt_find_unique_traverse   =          0
Total_time_bt_find_unique_traverse =          0
Max_time_bt_find_unique_traverse =          0
Avg_time_bt_find_unique_traverse =          0
The timer values for bt_range_search are:
Num_bt_range_search           =          0
Total_time_bt_range_search    =          0
Max_time_bt_range_search      =          0
Avg_time_bt_range_search      =          0
The timer values for bt_range_search_traverse are:
Num_bt_range_search_traverse  =          0
Total_time_bt_range_search_traverse =          0
Max_time_bt_range_search_traverse =          0
Avg_time_bt_range_search_traverse =          0
The timer values for bt_insert are:
Num_bt_insert                 =          0
Total_time_bt_insert          =          0
Max_time_bt_insert            =          0
Avg_time_bt_insert            =          0
The timer values for bt_insert_traverse are:
Num_bt_insert_traverse        =          0
Total_time_bt_insert_traverse =          0
Max_time_bt_insert_traverse   =          0
Avg_time_bt_insert_traverse   =          0
The timer values for bt_delete_obj are:
Num_bt_delete_obj             =          0
Total_time_bt_delete_obj      =          0
Max_time_bt_delete_obj        =          0
Avg_time_bt_delete_obj        =          0
The timer values for bt_delete_obj_traverse are:
Num_bt_delete_obj_traverse    =          0
Total_time_bt_delete_obj_traverse =          0
Max_time_bt_delete_obj_traverse =          0
Avg_time_bt_delete_obj_traverse =          0
The timer values for bt_mvcc_delete are:
Num_bt_mvcc_delete            =          0
Total_time_bt_mvcc_delete     =          0
Max_time_bt_mvcc_delete       =          0
Avg_time_bt_mvcc_delete       =          0
The timer values for bt_mvcc_delete_traverse are:
Num_bt_mvcc_delete_traverse   =          0
Total_time_bt_mvcc_delete_traverse =          0
Max_time_bt_mvcc_delete_traverse =          0
Avg_time_bt_mvcc_delete_traverse =          0
The timer values for bt_mark_delete are:
Num_bt_mark_delete            =          0
Total_time_bt_mark_delete     =          0
```

```
Max_time_bt_mark_delete       =          0
Avg_time_bt_mark_delete       =          0
The timer values for bt_mark_delete_traverse are:
Num_bt_mark_delete_traverse   =          0
Total_time_bt_mark_delete_traverse =          0
Max_time_bt_mark_delete_traverse =          0
Avg_time_bt_mark_delete_traverse =          0
The timer values for bt_undo_insert are:
Num_bt_undo_insert            =          0
Total_time_bt_undo_insert     =          0
Max_time_bt_undo_insert       =          0
Avg_time_bt_undo_insert       =          0
The timer values for bt_undo_insert_traverse are:
Num_bt_undo_insert_traverse   =          0
Total_time_bt_undo_insert_traverse =          0
Max_time_bt_undo_insert_traverse =          0
Avg_time_bt_undo_insert_traverse =          0
The timer values for bt_undo_delete are:
Num_bt_undo_delete            =          0
Total_time_bt_undo_delete     =          0
Max_time_bt_undo_delete       =          0
Avg_time_bt_undo_delete       =          0
The timer values for bt_undo_delete_traverse are:
Num_bt_undo_delete_traverse   =          0
Total_time_bt_undo_delete_traverse =          0
Max_time_bt_undo_delete_traverse =          0
Avg_time_bt_undo_delete_traverse =          0
The timer values for bt_undo_mvcc_delete are:
Num_bt_undo_mvcc_delete       =          0
Total_time_bt_undo_mvcc_delete =          0
Max_time_bt_undo_mvcc_delete  =          0
Avg_time_bt_undo_mvcc_delete  =          0
The timer values for bt_undo_mvcc_delete_traverse are:
Num_bt_undo_mvcc_delete_traverse =          0
Total_time_bt_undo_mvcc_delete_traverse =          0
Max_time_bt_undo_mvcc_delete_traverse =          0
Avg_time_bt_undo_mvcc_delete_traverse =          0
The timer values for bt_vacuum are:
Num_bt_vacuum                 =          0
Total_time_bt_vacuum          =          0
Max_time_bt_vacuum            =          0
Avg_time_bt_vacuum            =          0
The timer values for bt_vacuum_traverse are:
Num_bt_vacuum_traverse        =          0
Total_time_bt_vacuum_traverse =          0
Max_time_bt_vacuum_traverse   =          0
Avg_time_bt_vacuum_traverse   =          0
The timer values for bt_vacuum_insid are:
Num_bt_vacuum_insid           =          0
Total_time_bt_vacuum_insid    =          0
Max_time_bt_vacuum_insid      =          0
Avg_time_bt_vacuum_insid      =          0
```

```
The timer values for bt_vacuum_insid_traverse are:
Num_bt_vacuum_insid_traverse  =          0
Total_time_bt_vacuum_insid_traverse =          0
Max_time_bt_vacuum_insid_traverse =          0
Avg_time_bt_vacuum_insid_traverse =          0
The timer values for vacuum_master are:
Num_vacuum_master             =        561
Total_time_vacuum_master      =       1259
Max_time_vacuum_master        =          6
Avg_time_vacuum_master        =          2
The timer values for vacuum_job are:
Num_vacuum_job                =          0
Total_time_vacuum_job         =          0
Max_time_vacuum_job           =          0
Avg_time_vacuum_job           =          0
The timer values for vacuum_worker_process_log are:
Num_vacuum_worker_process_log =          0
Total_time_vacuum_worker_process_log =          0
Max_time_vacuum_worker_process_log =          0
Avg_time_vacuum_worker_process_log =          0
The timer values for vacuum_worker_execute are:
Num_vacuum_worker_execute     =          0
Total_time_vacuum_worker_execute =          0
Max_time_vacuum_worker_execute =          0
Avg_time_vacuum_worker_execute =          0
Time_get_snapshot_acquire_time =          0
Count_get_snapshot_retry      =          0
Time_tran_complete_time       =          0
The timer values for compute_oldest_visible are:
Num_compute_oldest_visible    =        561
Total_time_compute_oldest_visible =        569
Max_time_compute_oldest_visible =          4
Avg_time_compute_oldest_visible =          1
Count_get_oldest_mvcc_retry   =          0
Data_page_buffer_hit_ratio    =       0.00
Log_page_buffer_hit_ratio     =       0.00
Vacuum_data_page_buffer_hit_ratio =       0.00
Vacuum_page_efficiency_ratio  =       0.00
Vacuum_page_fetch_ratio       =       0.00
Data_page_fix_lock_acquire_time_msec =       0.00
Data_page_fix_hold_acquire_time_msec =       0.00
Data_page_fix_acquire_time_msec =       0.00
Data_page_allocate_time_ratio =       0.00
Data_page_total_promote_success =       0.00
Data_page_total_promote_fail  =       0.00
Data_page_total_promote_time_msec =       0.00
Num_unfix_void_to_private_top =          0
Num_unfix_void_to_private_mid =          0
Num_unfix_void_to_shared_mid  =          0
Num_unfix_lru1_private_to_shared_mid =          0
Num_unfix_lru2_private_to_shared_mid =          0
Num_unfix_lru3_private_to_shared_mid =          0
```

```
Num_unfix_lru2_private_keep   =          0
Num_unfix_lru2_shared_keep    =          0
Num_unfix_lru2_private_to_top =          0
Num_unfix_lru2_shared_to_top  =          0
Num_unfix_lru3_private_to_top =          0
Num_unfix_lru3_shared_to_top  =          0
Num_unfix_lru1_private_keep   =          0
Num_unfix_lru1_shared_keep    =          0
Num_unfix_void_to_private_mid_vacuum =          0
Num_unfix_lru1_any_keep_vacuum =          0
Num_unfix_lru2_any_keep_vacuum =          0
Num_unfix_lru3_any_keep_vacuum =          0
Num_unfix_void_aout_found     =          0
Num_unfix_void_aout_not_found =          0
Num_unfix_void_aout_found_vacuum =          0
Num_unfix_void_aout_not_found_vacuum =          0
Num_data_page_hash_anchor_waits =          0
Time_data_page_hash_anchor_wait =          0
The timer values for flush_collect are:
Num_flush_collect             =          0
Total_time_flush_collect      =          0
Max_time_flush_collect        =          0
Avg_time_flush_collect        =          0
The timer values for flush_flush are:
Num_flush_flush               =          0
Total_time_flush_flush        =          0
Max_time_flush_flush          =          0
Avg_time_flush_flush          =          0
The timer values for flush_sleep are:
Num_flush_sleep               =          4
Total_time_flush_sleep        =    4000307
Max_time_flush_sleep          =    1000077
Avg_time_flush_sleep          =    1000076
The timer values for flush_collect_per_page are:
Num_flush_collect_per_page    =          0
Total_time_flush_collect_per_page =          0
Max_time_flush_collect_per_page =          0
Avg_time_flush_collect_per_page =          0
The timer values for flush_flush_per_page are:
Num_flush_flush_per_page      =          0
Total_time_flush_flush_per_page =          0
Max_time_flush_flush_per_page =          0
Avg_time_flush_flush_per_page =          0
Num_data_page_writes          =          0
Num_data_page_dirty_to_post_flush =          0
Num_data_page_skipped_flush   =          0
Num_data_page_skipped_flush_need_wal =          0
Num_data_page_skipped_flush_already_flushed =          0
Num_data_page_skipped_flush_fixed_or_hot =          0
The timer values for compensate_flush are:
Num_compensate_flush          =          0
Total_time_compensate_flush   =          0
```

```
Max_time_compensate_flush     =          0
Avg_time_compensate_flush     =          0
The timer values for assign_direct_bcb are:
Num_assign_direct_bcb         =          0
Total_time_assign_direct_bcb  =          0
Max_time_assign_direct_bcb    =          0
Avg_time_assign_direct_bcb    =          0
The timer values for wake_flush_waiter are:
Num_wake_flush_waiter         =          0
Total_time_wake_flush_waiter  =          0
Max_time_wake_flush_waiter    =          0
Avg_time_wake_flush_waiter    =          0
The timer values for alloc_bcb are:
Num_alloc_bcb                 =          0
Total_time_alloc_bcb          =          0
Max_time_alloc_bcb            =          0
Avg_time_alloc_bcb            =          0
The timer values for alloc_bcb_search_victim are:
Num_alloc_bcb_search_victim   =          0
Total_time_alloc_bcb_search_victim =          0
Max_time_alloc_bcb_search_victim =          0
Avg_time_alloc_bcb_search_victim =          0
The timer values for alloc_bcb_cond_wait_high_prio are:
Num_alloc_bcb_cond_wait_high_prio =          0
Total_time_alloc_bcb_cond_wait_high_prio =          0
Max_time_alloc_bcb_cond_wait_high_prio =          0
Avg_time_alloc_bcb_cond_wait_high_prio =          0
The timer values for alloc_bcb_cond_wait_low_prio are:
Num_alloc_bcb_cond_wait_low_prio =          0
Total_time_alloc_bcb_cond_wait_low_prio =          0
Max_time_alloc_bcb_cond_wait_low_prio =          0
Avg_time_alloc_bcb_cond_wait_low_prio =          0
Num_alloc_bcb_prioritize_vacuum =          0
Num_victim_use_invalid_bcb    =          0
The timer values for
alloc_bcb_get_victim_search_own_private_list are:
Num_alloc_bcb_get_victim_search_own_private_list =          0
Total_time_alloc_bcb_get_victim_search_own_private_list
=          0
Max_time_alloc_bcb_get_victim_search_own_private_list =
0
Avg_time_alloc_bcb_get_victim_search_own_private_list =
0
The timer values for
alloc_bcb_get_victim_search_others_private_list are:
Num_alloc_bcb_get_victim_search_others_private_list =          0
Total_time_alloc_bcb_get_victim_search_others_private_list
=          0
Max_time_alloc_bcb_get_victim_search_others_private_list
=          0
Avg_time_alloc_bcb_get_victim_search_others_private_list
=          0
```

```
The timer values for alloc_bcb_get_victim_search_shared_list are:
Num_alloc_bcb_get_victim_search_shared_list =          0
Total_time_alloc_bcb_get_victim_search_shared_list =          0
Max_time_alloc_bcb_get_victim_search_shared_list =          0
Avg_time_alloc_bcb_get_victim_search_shared_list =          0
Num_victim_assign_direct_vacuum_void =          0
Num_victim_assign_direct_vacuum_lru =          0
Num_victim_assign_direct_flush =          0
Num_victim_assign_direct_panic =          0
Num_victim_assign_direct_adjust_lru =          0
Num_victim_assign_direct_adjust_lru_to_vacuum =          0
Num_victim_assign_direct_search_for_flush =          0
Num_victim_shared_lru_success =          0
Num_victim_own_private_lru_success =          0
Num_victim_other_private_lru_success =          0
Num_victim_shared_lru_fail    =          0
Num_victim_own_private_lru_fail =          0
Num_victim_other_private_lru_fail =          0
Num_victim_all_lru_fail       =          0
Num_victim_get_from_lru       =          0
Num_victim_get_from_lru_was_empty =          0
Num_victim_get_from_lru_fail  =          0
Num_victim_get_from_lru_bad_hint =          0
Num_lfcq_prv_get_total_calls  =          0
Num_lfcq_prv_get_empty        =          0
Num_lfcq_prv_get_big          =          0
Num_lfcq_shr_get_total_calls  =          0
Num_lfcq_shr_get_empty        =          0
The timer values for Time_DWB_flush_block_time are:
Num_Time_DWB_flush_block_time =          0
Total_time_Time_DWB_flush_block_time =          0
Max_time_Time_DWB_flush_block_time =          0
Avg_time_Time_DWB_flush_block_time =          0
The timer values for Time_DWB_flush_block_helper_time are:
Num_Time_DWB_flush_block_helper_time =          0
Total_time_Time_DWB_flush_block_helper_time =          0
Max_time_Time_DWB_flush_block_helper_time =          0
Avg_time_Time_DWB_flush_block_helper_time =          0
The timer values for Time_DWB_flush_block_cond_wait_time are:
Num_Time_DWB_flush_block_cond_wait_time =       5352
Total_time_Time_DWB_flush_block_cond_wait_time =    5629578
Max_time_Time_DWB_flush_block_cond_wait_time =       1059
Avg_time_Time_DWB_flush_block_cond_wait_time =       1051
The timer values for Time_DWB_flush_block_sort_time are:
Num_Time_DWB_flush_block_sort_time =          0
Total_time_Time_DWB_flush_block_sort_time =          0
Max_time_Time_DWB_flush_block_sort_time =          0
Avg_time_Time_DWB_flush_block_sort_time =          0
The timer values for Time_DWB_flush_remove_hash_entries are:
Num_Time_DWB_flush_remove_hash_entries =          0
Total_time_Time_DWB_flush_remove_hash_entries =          0
Max_time_Time_DWB_flush_remove_hash_entries =          0
```

```
Avg_time_Time_DWB_flush_remove_hash_entries =          0
The timer values for Time_DWB_checksum_time are:
Num_Time_DWB_checksum_time    =          0
Total_time_Time_DWB_checksum_time =          0
Max_time_Time_DWB_checksum_time =          0
Avg_time_Time_DWB_checksum_time =          0
The timer values for Time_DWB_wait_flush_block_time are:
Num_Time_DWB_wait_flush_block_time =          0
Total_time_Time_DWB_wait_flush_block_time =          0
Max_time_Time_DWB_wait_flush_block_time =          0
Avg_time_Time_DWB_wait_flush_block_time =          0
The timer values for Time_DWB_wait_flush_block_helper_time are:
Num_Time_DWB_wait_flush_block_helper_time =          0
Total_time_Time_DWB_wait_flush_block_helper_time =          0
Max_time_Time_DWB_wait_flush_block_helper_time =          0
Avg_time_Time_DWB_wait_flush_block_helper_time =          0
The timer values for Time_DWB_flush_force_time are:
Num_Time_DWB_flush_force_time =          0
Total_time_Time_DWB_flush_force_time =          0
Max_time_Time_DWB_flush_force_time =          0
Avg_time_Time_DWB_flush_force_time =          0
Num_alloc_bcb_wait_threads_high_priority =          0
Num_alloc_bcb_wait_threads_low_priority =          0
Num_flushed_bcbs_wait_for_direct_victim =          0
Num_lfcq_big_private_lists    =          0
Num_lfcq_private_lists        =          0
Num_lfcq_shared_lists         =          0
Num_data_page_avoid_dealloc   =          0
Num_data_page_avoid_victim    =          0
Num_data_page_fix_ext:
Num_data_page_promote_ext:
Num_data_page_promote_time_ext:
Num_data_page_unfix_ext:
Time_data_page_lock_acquire_time:
Time_data_page_hold_acquire_time:
Time_data_page_fix_acquire_time:
Num_mvcc_snapshot_ext:
Time_obj_lock_acquire_time:
Thread_stats_counters_timers:
Thread_pgbuf_daemon_stats_counters_timers:
Num_dwb_flushed_block_volumes:
Thread_loaddb_stats_counters_timers:
```

The following are the explanation about the above statistical information. You can find the statistic category (database module), the name, the stat type and a brief description for each statistic.

There are several types of statistic, based on how they are collected:

- Accumulator: The stat values are incremented whenever the tracked action happens.

- Counter/timer: The stat tracks both the number and the duration of an action. Also biggest and average duration are tracked.
- Snapshot: The stat is peeked from database.
- Complex: The stat tracks multiple values for an action, separated by various attributes.

Most statistics are accumulators (they are incremented when an action happens). Other statistics can be counter/timers (they track both number of actions and their duration), some are peeked from database (snapshot) and some are computed based on other values. Lastly, there are several complex statistics which track detailed information on some operations.

File I/O

Stat name	Stat type	Description
Num_file_removes	Accumulator	The number of files removed
Num_file_creates	Accumulator	The number of files created
Num_file_ioreads	Accumulator	The number of files read
Num_file_iowrites	Accumulator	The number of files stored
Num_file_iosynches	Accumulator	The number of file synchronization
..file_iosync_all	Counter/timer	The number and duration of sync all files
Num_file_page_allocs	Accumulator	The number of page allocations
Num_file_page_deallocs	Accumulator	The number of page deallocations

Page Buffer

Stat name	Stat type	Description
Num_data_page_fetches	Accumulator	The number of fetched pages
Num_data_page_dirties	Accumulator	The number of dirty pages
Num_data_page_ioreads	Accumulator	The number of pages read from disk (more means less efficient, it correlates with lower hit ratio)
Num_data_page_iowrites	Accumulator	The number of pages write to disk (more means less efficient)
Num_data_page_private_quota	Snapshot	The target number of pages for private LRU lists
Num_data_page_private_count	Snapshot	The actual number of pages for private LRU lists
Num_data_page_fixed	Snapshot	The number of fixed pages in data buffer
Num_data_page_dirty	Snapshot	The number of dirty pages in data buffer
Num_data_page_lru1	Snapshot	The number of pages in LRU1 zone in data buffer
Num_data_page_lru2	Snapshot	The number of pages in LRU2 zone in data buffer

Num_data_page_lru3	Snapshot	The number of pages in LRU3 zone in data buffer
Num_data_page_victim_candidate	Snapshot	The number of victim candidate pages in data buffer

Logs

Stat name	Stat type	Description
Num_log_page_fetches	Accumulator	The number of fetched log pages
Num_log_page_ioreads	Accumulator	The number of log pages read
Num_log_page_iowrites	Accumulator	The number of log pages stored
Num_log_append_records	Accumulator	The number of log records appended
Num_log_archives	Accumulator	The number of logs archived
Num_log_start_checkpoints	Accumulator	The number of started checkpoints
Num_log_end_checkpoints	Accumulator	The number of ended checkpoints
Num_log_wals	Accumulator	The number of log flushes requested to write a data page.

Num_log_page_iowrites_for_replacement	Accumulator	The number of log data pages written to disk due to replacements (should be zero)
Num_log_page_replacements	Accumulator	The number of log data pages discarded due to replacements
Num_prior_lsa_list_size	Accumulator	Current size of the prior LSA(Log Sequence Address) list. CUBRID write the order of writing into the prior LSA list, before writing operation from the log buffer to the disk; this list is used to raise up the concurrency by reducing the waiting time of the transaction from writing to disk
Num_prior_lsa_list_maxed	Accumulator	The count of the prior LSA list being reached at the maximum size. The maximum size of the prior LSA list is log_buffer_size * 2. If this value is big, we can assume that log writing jobs happen a lot at the same time

Num_prior_lsa_list_removed	Accumulator	The count of LSA being moved from prior LSA list into log buffer. We can assume that the commits have happened at the similar count with this value
Log_page_buffer_hit_ratio	Computed	Hit ratio of log page buffers (Num_log_page_fetches - Num_log_page_ioreads)*100 / Num_log_page_fetches

Concurrency/lock

Stat name	Stat type	Description
Num_page_locks_acquired	Accumulator	The number of locked pages acquired
Num_object_locks_acquired	Accumulator	The number of locked objects acquired
Num_page_locks_converted	Accumulator	The number of locked pages converted
Num_object_locks_converted	Accumulator	The number of locked objects converted
Num_page_locks_re-requested	Accumulator	The number of locked pages requested
Num_object_locks_re-requested	Accumulator	The number of locked objects requested

Num_page_locks_waits	Accumulator	The number of locked pages waited
Num_object_locks_waits	Accumulator	The number of locked objects waited
Num_object_locks_time_waited_usec	Accumulator	The time in microseconds spent on waiting for all object locks
Time_obj_lock_acquire_time	Complex	Time consumer for locking objects classified by lock mode

Transactions

Stat name	Stat type	Description
Num_tran_commits	Accumulator	The number of commits
Num_tran_rollbacks	Accumulator	The number of rollbacks
Num_tran_savepoints	Accumulator	The number of savepoints
Num_tran_start_topops	Accumulator	The number of top operations started
Num_tran_end_topops	Accumulator	The number of top operations stopped
Num_tran_interrupts	Accumulator	The number of interruptions
	Accumulator	

Num_tran_postpone_cache_hits		The number of cache hits when executing transaction postpone ops
Num_tran_postpone_cache_miss	Accumulator	The number of cache misses when executing transaction postpone ops Cache misses may degrade performance of transaction commits and may impact all log operations
Num_tran_topop_postpone_cache_hits	Accumulator	The number of cache hits when executing top operations postpone ops
Num_tran_topop_postpone_cache_miss	Accumulator	The number of cache misses when executing top operations postpone Cache misses may degrade performance of top operations commits and may impact all log operations

Index

Stat name	Stat type	Description
Num_btree_inserts	Accumulator	The number of nodes inserted
Num_btree_deletes	Accumulator	The number of nodes deleted

Num_btree_updates	Accumulator	The number of nodes updated
Num_btree_covered	Accumulator	The number of cases in which an index includes all data upon query execution
Num_btree_noncovered	Accumulator	The number of cases in which an index includes some or no data upon query execution
Num_btree_resumes	Accumulator	The exceeding number of index scan specified due to too many results
Num_btree_multirange_optimization	Accumulator	The number of executions on multi-range optimization for the WHERE ... IN ... LIMIT condition query statement
Num_btree_splits	Accumulator	The number of B-tree split-operations
Num_btree_merges	Accumulator	The number of B-tree merge-operations
Num_btree_get_stats	Accumulator	The number of B-tree get stat calls
..bt_leaf	Counter/timer	The number and duration of all operations in index leaves
..bt_find_unique	Counter/timer	

		The number and duration of B-tree ‘find-unique’ operations
..btrange_search	Counter/timer	The number and duration of B-tree ‘range-search’ operations
..bt_insert_obj	Counter/timer	The number and duration of B-tree ‘insert object’ operations
..bt_delete_obj	Counter/timer	The number and duration of B-tree ‘physical delete object’ operations
..bt_mvcc_delete	Counter/timer	The number and duration of B-tree ‘mvcc delete’ operations
..bt_mark_delete	Counter/timer	The number and duration of B-tree mark delete operations
..bt_undo_insert	Counter/timer	The number and duration of B-tree ‘undo insert’ operations
..bt_undo_delete	Counter/timer	The number and duration of B-tree ‘undo physical delete’ operations
..bt_undo_mvcc_delete	Counter/timer	The number and duration of B-tree ‘undo mvcc delete’ operations

..bt_vacuum	Counter/timer	The number and duration of B-tree vacuum deleted object operations
..bt_vacuum_insid	Counter/timer	The number and duration of vacuum operations on B-tree ‘insert id’
..bt_fix_ovf_oids	Counter/timer	The number and duration of B-tree overflow page fixes
..bt_unique_rlocks	Counter/timer	The number and duration of blocked read locks on unique indexes
..bt_unique_wlocks	Counter/timer	The number and duration of blocked write locks on unique indexes
..bt_traverse	Counter/timer	The number and duration of B-tree traverse
..bt_find_unique_traverse	Counter/timer	The number and duration of B-tree traverse for ‘find unique’
..bt_range_search_traverse	Counter/timer	The number and duration of B-tree traverse for ‘range search’
..bt_insert_traverse	Counter/timer	The number and duration of B-tree traverse for ‘insert’
..bt_delete_traverse	Counter/timer	

		The number and duration of B-tree traverse for ‘physical delete’
..bt_mvcc_delete_traverse	Counter/timer	The number and duration of B-tree traverse for ‘mvcc delete’
..bt_mark_delete_traverse	Counter/timer	The number and duration of B-tree traverse for ‘mark delete’
..bt_undo_insert_traverse	Counter/timer	The number and duration of B-tree traverse for ‘undo physical insert’
..bt_undo_delete_traverse	Counter/timer	The number and duration of B-tree traverse for ‘undo physical delete’
..bt_undo_mvcc_delete_traverse	Counter/timer	The number and duration of B-tree traverse for ‘undo delete’
..bt_vacuum_traverse	Counter/timer	The number and duration of B-tree traverse for vacuum deleted object
..bt_vacuum_insid_traverse	Counter/timer	The number and duration of B-tree traverse for vacuum ‘insert id’

Load Index Online

Stat name	Stat type	Description

..btree_online_load	Counter/timer	The number and duration of B-tree online index loading statements
..btree_online_insert_task	Counter/timer	The number and duration of batch insert tasks by index loader
..btree_online_prepare_task	Counter/timer	The number and duration of preparing (sorting) index loader tasks
..btree_online_insert_leaf	Counter/timer	The number and duration of leaf pages processed by index loader (multiple keys may be inserted while a leaf page is processed)
Num_btree_online_inserts	Accumulator	The number of key inserts by index loader
Num_btree_online_inserts_same_page_hold	Accumulator	The number of successive inserts by index loader in same leaf page (loader optimization to avoid expensive index traversals)
Num_btree_online_inserts_retry	Accumulator	The number of restarted inserts because successive keys do not belong to same leaf or because there is not enough space
Num_btree_online_inserts_retry_nice	Accumulator	The number of restarted inserts to let others access leaf page

Query

Stat name	Stat type	Description
Num_query_selects	Accumulator	The number of SELECT query execution
Num_query_inserts	Accumulator	The number of INSERT query execution
Num_query_deletes	Accumulator	The number of DELETE query execution
Num_query_updates	Accumulator	The number of UPDATE query execution
Num_query_sscans	Accumulator	The number of sequential scans (full scan)
Num_query_iscans	Accumulator	The number of index scans
Num_query_lscans	Accumulator	The number of LIST scans
Num_query_setscans	Accumulator	The number of SET scans
Num_query_methscans	Accumulator	The number of METHOD scans
Num_query_nljoins	Accumulator	The number of nested loop joins
Num_query_mjoins	Accumulator	The number of parallel joins

Num_query_objfetches	Accumulator	The number of fetch objects
Num_query_holdable_cursors	Snapshot	The number of holdable cursors in the current server.

Sort

Stat name	Stat type	Description
Num_sort_io_pages	Accumulator	The number of pages fetched on the disk during sorting (more means less efficient)
Num_sort_data_pages	Accumulator	The number of pages found on the page buffer during sorting (more means less efficient)

Network

Stat name	Stat type	Description
Num_network_requests	Accumulator	The number of network requested

Heap

Stat name	Stat type	Description
Num_heap_stats_sync_bestspace	Accumulator	The updated number of the “best page” list. “Best pages” means that the data pages of which the free space is

		more than 30% in the environment of multiple INSERTs and DELETEs. Only some information of these pages are saved as the “best page” list. In the “best page” list, the information of a million pages is saved at once. This list is searched when INSERTing a record, and then this list is updated when there are no free space to store this record on the pages. If there are still no free space to store this record even this list is updated for several times, this recored is stored into a new page.
Num_heap_home_inserts	Accumulator	The number of inserts in heap HOME type records
Num_heap_big_inserts	Accumulator	The number of inserts in heap BIG type records
Num_heap_assign_inserts	Accumulator	The number of inserts in heap ASSIGN type records
Num_heap_home_deletes	Accumulator	The number of deletes from heap HOME type records in non-MVCC mode
Num_heap_home_mvcc_deletes	Accumulator	The number of deletes from heap HOME type records in MVCC mode

Num_heap_home_to_rel_deletes	Accumulator	The number of deletes from heap HOME to RELOCATION type records in MVCC mode
Num_heap_home_to_big_deletes	Accumulator	The number of deletes from heap HOME to BIG type records in MVCC mode
Num_heap_rel_deletes	Accumulator	The number of deletes from heap RELOCATION type records in non-MVCC mode
Num_heap_rel_mvcc_deletes	Accumulator	The number of deletes from heap RELOCATION type records in MVCC mode
Num_heap_rel_to_home_deletes	Accumulator	The number of deletes from heap RELOCATION to HOME type records in MVCC mode
Num_heap_rel_to_big_deletes	Accumulator	The number of deletes from heap RELOCATION to BIG type records in MVCC mode
Num_heap_rel_to_rel_deletes	Accumulator	The number of deletes from heap RELOCATION to RELOCATION type records in MVCC mode
Num_heap_big_deletes	Accumulator	The number of deletes from heap BIG type records in non-MVCC mode

Num_heap_big_mvcc_deletes	Accumulator	The number of deletes from heap BIG type records in MVCC mode
Num_heap_home_updates	Accumulator	The number of updates in place of heap HOME type records in non-MVCC mode(*)
Num_heap_home_to_rel_updates	Accumulator	The number of updates of heap HOME to RELOCATION type records in non-MVCC mode(*)
Num_heap_home_to_big_updates	Accumulator	The number of updates of heap HOME to BIG type records in non-MVCC mode(*)
Num_heap_rel_updates	Accumulator	The number of updates of heap RELOCATION type records in non-MVCC mode(*)
Num_heap_rel_to_home_updates	Accumulator	The number of updates of heap RELOCATION to HOME type records in non-MVCC mode(*)
Num_heap_rel_to_rel_updates	Accumulator	The number of updates of heap RELOCATION to RELOCATION type records in non-MVCC mode(*)
Num_heap_rel_to_big_updates	Accumulator	The number of updates of heap RELOCATION to BIG type records in non-MVCC mode(*)
Num_heap_big_updates	Accumulator	

		The number of updates of heap BIG type records in non-MVCC mode(*)
Num_heap_home_vacuums	Accumulator	The number of vacuumed heap HOME type records
Num_heap_big_vacuums	Accumulator	The number of vacuumed heap BIG type records
Num_heap_rel_vacuums	Accumulator	The number of vacuumed heap RELOCATION type records
Num_heap_insid_vacuums	Accumulator	The number of vacuumed heap newly inserted records
Num_heap_remove_vacuums	Accumulator	The number of vacuum operations that remove heap records
..heap_insert_prepare	Counter/timer	The number and duration of preparing heap insert operation
..heap_insert_execute	Counter/timer	The number and duration of executing heap insert operation
..heap_insert_log	Counter/timer	The number and duration of logging heap insert operation
..heap_delete_prepare	Counter/timer	

		The number and duration of preparing heap delete operation
..heap_delete_execute	Counter/timer	The number and duration of executing heap delete operation
..heap_delete_log	Counter/timer	The number and duration of logging heap delete operation
..heap_update_prepare	Counter/timer	The number and duration of preparing heap update operation
..heap_update_execute	Counter/timer	The number and duration of executing heap update operation
..heap_update_log	Counter/timer	The number and duration of logging heap update operation
..heap_vacuum_prepare	Counter/timer	The number and duration of preparing heap vacuum operation
..heap_vacuum_execute	Counter/timer	The number and duration of executing heap vacuum operation
..heap_vacuum_log	Counter/timer	The number and duration of logging heap vacuum operation

Query plan cache

Stat name	Stat type	Description
Num_plan_cache_add	Accumulator	The number of entries added to query cache
Num_plan_cache_lookup	Accumulator	The number of lookups in query cache
Num_plan_cache_hit	Accumulator	The number of hits in query cache
Num_plan_cache_miss	Accumulator	The number of misses in query cache
Num_plan_cache_full	Accumulator	The number of times query cache becomes full
Num_plan_cache_delete	Accumulator	The number of entries deleted from query cache
Num_plan_cache_invalid_xasl_id	Accumulator	The number of failed attempts of retrieving entries by XASL ID.
Num_plan_cache_entries	Snapshot	The current number of entires in query cache

High Availibility

Stat name	Stat type	Description
Time_ha_replication_delay	Accumulator	Replication latency time (sec.)

Vacuuming

Stat name	Stat type	Description
Num_vacuum_log_pages_vacuumed	Accumulator	The number of log data pages processed by vacuum workers.
Num_vacuum_log_pages_to_vacuum	Accumulator	The number of log data pages to be vacuumed by vacuum workers. (if value is much bigger than Num_vacuum_log_pages_vacuumed, it means vacuum system lags behind)
Num_vacuum_prefetch_requests_log_pages	Accumulator	The number of requests to prefetch buffer for log pages from vacuum
Num_vacuum_prefetch_hits_log_pages	Accumulator	The number of hits to prefetch buffer for log pages from vacuum
..vacuum_master	Counter/timer	The number and duration of vacuum master iterations.
..vacuum_job	Counter/timer	The number and duration of vacuum jobs
..vacuum_worker_process_log	Counter/timer	The number and duration of process log tasks
..vacuum_worker_execute	Counter/timer	The number and duration of execute vacuum tasks

Vacuum_data_page_buffer_hit_ratio	Computed	Hit ratio of vacuuming data page buffers
Vacuum_page_efficiency_ratio	Computed	Ratio between number of page unfix of vacuum with dirty flag and total number of page unfix of vacuum. Ideally, the vacuum process performsonly write operations since it cleans up all unused records. Even with an optimized vacuum process, 100% efficiency is not possible.
Vacuum_page_fetch_ratio	Computed	Ratio (percentage) of page unfix from vacuum module versus total.
Num_data_page_avoid_dealloc	Snapshot	The number of data pages that cannot be deallocated by vacuum

Heap Bestspace

Stat name	Stat type	Description
..heap_stats_sync_bestspace	Counter/timer	The number and duration of bestspace synchronization operations
Num_heap_stats_bestspace_entries	Accumulator	The number of best pages which are saved on the “best page” list

Num_heap_stats_bestspace_maxed	Accumulator	The maximum number of pages which can be saved on the “best page” list
..bestspace_add	Counter/timer	The number and duration of adding entries to bestspace cache
..bestspace_del	Counter/timer	The number and duration of deleting from bestpace cache
..bestspace_find	Counter/timer	The number and duration of finding a bestpace cache entry
..heap_find_page_bestspace	Counter/timer	The number and duration of searching heap pages in bestspace cache
..heap_find_best_page	Counter/timer	The number and duration of finding or creating (if not found) a page to insert in heap.

Page buffer fix

Stat name	Stat type	Description
Data_page_fix_lock_acquire_time_msec	Computed	Time waiting for other transaction to load page from disk
Data_page_fix_hold_acquire_time_msec	Computed	Time to obtain page latch

Data_page_fix_acquire_time_msec	Computed	Total time to fix page
Data_page_allocate_time_ratio	Computed	Ratio of time necessary for page loading from disk versus the total time of fixing a page
Data_page_total_promote_success	Computed	Number of successful page latch promotions from shared to exclusive
Data_page_total_promote_fail	Computed	Number of failed page latch promotions
Data_page_total_promote_time_msec	Computed	Time for promoting page latches
Num_data_page_hash_anchor_waits	Accumulator	Number of waits on page buffer hash bucket
Time_data_page_hash_anchor_wait	Accumulator	Total wait time on page buffer hash bucket
Num_data_page_fix_ext	Complex	Number of data page fixes classified by: - module (system, worker, vacuum) - page type - page fetch/found mode - page latch mode - page latch condition
Time_data_page_lock_acquire_time	Complex	Time consumed waiting for other thread to load data page from disk: - module (system, worker, vacuum) - page type

		- page fetch/found mode - page latch mode - page latch condition
Time_data_page_hold_acquire_time	Complex	Time consumed waiting for data page latch: - module (system, worker, vacuum) - page type - page fetch/found mode - page latch mode
Time_data_page_fix_acquire_time	Complex	Time consumed fixing data page: - module (system, worker, vacuum) - page type - page fetch/found mode - page latch mode - page latch condition
Num_data_page_promote_ext	Complex	Number of data page promotions classified by: - module (system, worker, vacuum) - page type - promote latch condition - holder latch mode - successful/failed promotion
Num_data_page_promote_time_ext	Complex	Time consumed for data page promotions classified by: - module (system, worker, vacuum) - page type - promote latch condition - holder latch mode - successful/failed promotion

Page buffer unfix

Stat name	Stat type	Description
Num_unfix_void_to_private_top	Accumulator	Unfix newly loaded data page and add to top of private LRU list
Num_unfix_void_to_private_mid	Accumulator	Unfix newly loaded data page and add to middle of private LRU list
Num_unfix_void_to_shared_mid	Accumulator	Unfix newly loaded data page and add to middle of shared LRU list
Num_unfix_lru1_private_to_shared_mid	Accumulator	Unfix data page and move from zone 1 of private list to shared middle
Num_unfix_lru2_private_to_shared_mid	Accumulator	Unfix data page and move from zone 2 of private list to shared middle
Num_unfix_lru3_private_to_shared_mid	Accumulator	Unfix data page and move from zone 3 of private list to shared middle
Num_unfix_lru2_private_keep	Accumulator	Unfix data page and keep it in zone 2 of private list
Num_unfix_lru2_shared_keep	Accumulator	Unfix data page and keep it in zone 2 of shared list
Num_unfix_lru2_private_to_top	Accumulator	Unfix data page and boost it from zone 2 of private list to its top

Num_unfix_lru2_shared_to_top	Accumulator	Unfix data page and boost it from zone 2 of shared list to its top
Num_unfix_lru3_private_to_top	Accumulator	Unfix data page and boost it from zone 3 of private list to its top
Num_unfix_lru3_shared_to_top	Accumulator	Unfix data page and boost it from zone 3 of shared list to its top
Num_unfix_lru1_private_keep	Accumulator	Unfix data page and keep it in zone 1 of private list
Num_unfix_lru2_shared_keep	Accumulator	Unfix data page and keep it in zone 2 of shared list
Num_unfix_void_to_private_mid_vacuum	Accumulator	Unfix newly loaded data page and add to middle of private LRU list (vacuum thread)
Num_unfix_lru1_any_keep_vacuum	Accumulator	Unfix data page and keep it in zone 1 of private/ shared list (vacuum thread)
Num_unfix_lru2_any_keep_vacuum	Accumulator	Unfix data page and keep it in zone 2 of private/ shared list (vacuum thread)
Num_unfix_lru3_any_keep_vacuum	Accumulator	Unfix data page and keep it in zone 3 of private/ shared list (vacuum thread)
Num_unfix_void_aout_found	Accumulator	

		Newly loaded data page was found in AOUT list
Num_unfix_void_aout_not_found	Accumulator	Newly loaded data page was not found in AOUT list
Num_unfix_void_aout_found_vacuum	Accumulator	Newly loaded data page was found in AOUT list (vacuum thread)
Num_unfix_void_aout_not_found_vacuum	Accumulator	Newly loaded data page was not found in AOUT list (vacuum thread)
Num_data_page_unfix_ext	Complex	Number of data page unfixes classified by: - module (system, worker, vacuum) - page type - dirty or not - dirtied by holder or not - holder latch mode

Page buffer I/O

Stat name	Stat type	Description
Data_page_buffer_hit_ratio	Computed	Hit ratio of data page buffers (Num_data_page_fetches - Num_data_page_ioreads)* 100 / Num_data_page_fetches
Num_adaptive_flush_pages	Accumulator	The number of data pages requested from adaptive flush controller.

Num_adaptive_flush_log_pages	Accumulator	The number of log data pages requested from adaptive flush controller
Num_adaptive_flush_max_pages	Accumulator	The total number of page tokens assigned by adaptive flush controller
..compensate_flush	Counter/timer	The number and duration of flush compensations force by adaptive flush controller
..assign_direct_bcb	Counter/timer	The number and duration of assigning bcb's directly to waiters
..wake_flush_waiter	Counter/timer	The number and duration of waking up a thread waiting for bcb
..flush_collect	Counter/timer	The number and duration of flush thread collecting BCB sets
..flush_flush	Counter/timer	The number and duration of flush thread flushing BCB sets
..flush_sleep	Counter/timer	The number and duration of flush thread pauses
..flush_collect_per_page	Counter/timer	The number and duration of flush thread collecting one BCB
..flush_flush_per_page	Counter/timer	

		The number and duration of flush thread flushing one BCB
Num_data_page_writes	Accumulator	The total number of data pages flushed to disk
Num_data_page_dirty_to_post_flush	Accumulator	Number of flushed pages sent to post-flush thread for processing
Num_data_page_skipped_flush	Accumulator	The total number of BCB's that flush thread skipped
Num_data_page_skipped_flush_need_wal	Accumulator	The number of BCB's that flush thread skipped because it required log data pages be flushed first
Num_data_page_skipped_flush_already_flushed	Accumulator	The number of BCB's that flush thread skipped because they have been flushed already
Num_data_page_skipped_flush_fixed_or_hot	Accumulator	The number of BCB's that flush thread skipped because they are fixed or have been fixed since collected.

Page buffer victimization

Stat name	Stat type	Description
..alloc_bcb	Counter/timer	The number and duration of BCB allocation to store new data page.

		When a database is just started, the page buffer has available BCB's ready to be picked. However, once page buffer becomes full all BCB's are in use, one must be victimized. The time tracked here includes BCB victimization and loading from disk.
..alloc_bcb_search_victim	Counter/timer	The number and duration of searches through all LRU lists for victims
..alloc_bcb_cond_wait_high_prio	Counter/timer	The number and duration of direct victim waits in high-priority queue
..alloc_bcb_cond_wait_low_prio	Counter/timer	The number and duration of direct victim waits in low-priority queue
Num_alloc_bcb_prioritize_vacuum	Accumulator	The number of vacuum direct victim waits in high-priority queue
Num_alloc_bcb_wait_threads_high_priority	Snapshot	The current number of direct victim waiters in high-priority queue
Num_alloc_bcb_wait_threads_low_priority	Snapshot	The current number of direct victim waiters in low-priority queue
Num_flushed_bcbs_wait_for_direct_victim	Snapshot	The current number of BCB's waiting for post-flush thread to process

		them and assign directly.
Num_victim_use_invalid_bcb	Accumulator	The number of BCB's allocated from invalid list
..alloc_bcb_get_victim_search_own_private_list	Counter/timer	The number and duration of getting a victim from own private list
..alloc_bcb_get_victim_search_others_private_list	Counter/timer	The number and duration of getting a victim from other private lists
..alloc_bcb_get_victim_searchshared_list	Counter/timer	The number and duration of getting a victim from a shared list
Num_data_page_avoid_victim	Accumulator	The number of BCB's that cannot be victimized because they are in process of being flushed to disk
Num_victim_assign_direct_vacuum_void	Accumulator	The number of direct victims assigned from void zone by vacuum worker
Num_victim_assign_direct_vacuum_lru	Accumulator	The number of direct victims assigned from LRU zone 3 by vacuum worker
Num_victim_assign_direct_flush	Accumulator	The number of direct victims assigned by flush thread
Num_victim_assign_direct_panic	Accumulator	

		The number of direct victims assigned by panicked LRU searches. If there are a lot of waiters for victims, threads that found other victims while searching LRU list, will also try to assign more directly. Page buffer maintenance thread assignments are also counted here
Num_victim_assign_direct_adjust_lru	Accumulator	The number of direct victims assigned when BCB falls to LRU zone 3
Num_victim_assign_direct_adjust_lru_to_vacuum	Accumulator	The number of BCB's falling to LRU zone 3 **not** assigned as direct victims because a vacuum thread is expected to access it
Num_victim_assign_direct_search_for_flush	Accumulator	The number of direct victims assigned by flush thread while collecting BCB sets for flush
Num_victim_shared_lru_success	Accumulator	The number of successful victim searches in shared LRU lists
Num_victim_own_private_lru_success	Accumulator	The number of successful victim searches in own private LRU lists
	Accumulator	

Num_victim_other_private_lru_success		The number of successful victim searches in other private LRU lists
Num_victim_shared_lru_fail	Accumulator	The number of failed victim searches in shared LRU lists
Num_victim_own_private_lru_fail	Accumulator	The number of failed victim searches in own private LRU lists
Num_victim_other_private_lru_fail	Accumulator	The number of failed victim searches in other private LRU lists
Num_victim_all_lru_fail	Accumulator	The number of unlucky streaks to find victims in the sequence: 1. Own private LRU list (if over quota) 2. Other private LRU list (if own private is over quota) 3. Shared LRU list (this is simplified explanation, see *pgbuf_get_victim* function)
Num_victim_get_from_lru	Accumulator	The total number of victim searches in any LRU list
Num_victim_get_from_lru_was_empty	Accumulator	The number of victim searches in any LRU list that stop immediately because candidate count is zero
	Accumulator	

Num_victim_get_from_lru_fail		The number of failed victim searches in any LRU list although the candidate count was not zero
Num_victim_get_from_lru_bad_hint	Accumulator	The number of failed victim searches in any LRU list because victim was wrong
Num_lfcq_prv_get_total_calls	Accumulator	The number of victim searches in non-zero candidate private LRUs queue
Num_lfcq_prv_get_empty	Accumulator	The number of times non-zero candidate private LRUs queue was empty
Num_lfcq_prv_get_big	Accumulator	The number of victim searches in only very big non-zero candidate private LRUs queue (in this context, very big means way over quota)
Num_lfcq_shr_get_total_calls	Accumulator	The number of victim searches in non-zero candidate shared LRUs queue
Num_lfcq_shr_get_empty	Accumulator	The number of times non-zero candidate shared LRUs queue was empty
Num_lfcq_big_private_lists	Snapshot	The current number of very big non-zero candidate private LRU lists

Num_lfcq_private_lists	Snapshot	The current number of non-zero candidate private LRU lists
Num_lfcq_shared_lists	Snapshot	The current number of non-zero candidate shared LRU lists

Double write buffer

Stat name	Stat type	Description
..DWB_flush_block	Counter/timer	The number of blocks flushed and total writing duration
..DWB_file_sync_helper	Counter/timer	The number and duration of files synchronized by DWB helper
..DWB_flush_block_cond_wait	Counter/timer	The number and duration of DWB thread waits
..DWB_flush_block_sort	Counter/timer	The number and duration of sorting pages for flush
..DWB_decache_pages_after_write	Counter/timer	The number and duration of removing pages from DWB cache after flush
..DWB_wait_flush_block	Counter/timer	The number and duration of waiting for block to be flush to add a page
..DWB_wait_file_sync_helper	Counter/timer	

		The number and duration of waiting for DWB helper to sync file
..DWB_flush_force	Counter/timer	The number and duration of forced full DWB flushes
Num_dwb_flushed_block_volumes	Complex	A histogram of number of files synchronized by each block flush (last value is for ten or more files)

MVCC Snapshot

Stat name	Stat type	Description
Time_get_snapshot_acquire_time:	Accumulator	Total time consumed by all transactions to get a snapshot
Count_get_snapshot_retry:	Accumulator	The number of retries to acquire MVCC snapshot
Time_tran_complete_time:	Accumulator	Time spent to invalidate snapshot and MVCCID on commit/rollback
Time_get_oldest_mvcc_acquire_time:	Accumulator	Time spend to acquire “global oldest MVCC ID”
Count_get_oldest_mvcc_retry:	Accumulator	The number of retries to acquire “global oldest MVCC ID”
Num_mvcc_snapshot_ext	Complex	Number of data page fixes classified by:

		- snapshot type - insert/delete MVCCID's status - visible/invisible

Thread workers

Statistics collected by thread worker pools:

Stat name	Stat type	Description
..start_thread	Counter/timer	The number and duration of starting new threads
..create_context	Counter/timer	The number and duration of creating thread context
..execute_task	Counter/timer	The number and duration of executed tasks
..retire_task	Counter/timer	The number and duration of retiring task objects
..found_task_in_queue	Counter/timer	The number of tasks found in queue and the duration of claiming from queue after finishing previous task
..wakeup_with_task	Counter/timer	The number of tasks assigned to threads on standby and the duration to wakeup thread
..recycle_context	Counter/timer	The number and duration of thread context recyclings between task executions

..retire_context	Counter/timer	The number and duration of retired thread contexts

Statistics are collected from two worker pools:

- **Thread_stats_counters_timers**: statistics for workers executing client requests.
- **Thread_loaddb_stats_counters_timers**: statistics for workers loading data into database (using **loaddb** command). *At least one load session must be active to see these statistics.*

Thread daemons

Statistics collected by background daemon threads:

Stat name	Stat type	Description
..daemon_loop_count	Accumulator	The number of loops executed by daemon thread
..daemon_execute_time	Accumulator	The total duration spent by daemon thread on execution
..daemon_pause_time	Accumulator	The total duration spent by daemon thread between executions
..looper_sleep_count	Accumulator	The number of times thread looper was put to sleep
..looper_sleep_time	Accumulator	The total duration thread looper spent on sleep
..looper_reset_count	Accumulator	The number of times an incremental looper is reset by wakeup

..waiter_wakeup_count	Accumulator	The number of wakeup requests on daemon thread
..waiter_lock_wakeup_count	Accumulator	The number of wakeup requests on daemon thread that required lock
..waiter_sleep_count	Accumulator	The number of times daemon thread went to sleep
..waiter_timeout_count	Accumulator	The number of daemon thread waits ended with timeout
..waiter_no_sleep_count	Accumulator	The number of times daemon thread spins with no wait
..waiter_awake_count	Accumulator	The number of times daemon thread is awaken by other thread request
..waiter_wakeup_delay_time	Accumulator	The total duration of awaking daemon thread

Statistics are collected for next daemon threads:

Daemon name	**Description**
Page_flush_daemon_thread	Background thread that flushes data pages to disk
Page_post_flush_daemon_thread	

	Background thread that may reclaim memory space occupied by flushes data pages
Page_flush_control_daemon_thread	Background thread that controls data flush rate
Page_maintenance_daemon_thread	Background thread that recalculates quotas for private LRU lists
Deadlock_detect_daemon_thread	Background thread that looks for deadlocks and chooses victims to unblock the system
Log_flush_daemon_thread	Background thread that flushes log data to disk

Note

(*) : These statistics measure the non-MVCC operations or MVCC operations which are performed in-place (decided internally)

-o, --output-file=FILE

-o options is used to store statistics information of server processing for the database to a specified file.

```
cubrid statdump -o statdump.log testdb
```

-c, --cumulative

You can display the accumulated operation statistics information of the target database server by using the **-c** option.

Num_data_page_fix_ext, Num_data_page_unfix_ext, Time_data_page_hold_acquire_time, Time_data_page_fix_acquire_time information can be output only when this option is specified; however, these informations will be omitted because they are for CUBRID Engine developers.

By combining this with the **-i** option, you can check the operation statistics information at a specified interval.

```
cubrid statdump -i 5 -c testdb
```

-s, --substr=STRING

You can display statistics about items, the names of which include the specified string by using **-s** option.

The following example shows how to display statistics about items, the names of which include “data”.

```
cubrid statdump -s data testdb

 *** SERVER EXECUTION STATISTICS ***
Num_data_page_fetches         =          0
Num_data_page_dirties         =          0
Num_data_page_ioreads         =          0
Num_data_page_iowrites        =          0
Num_data_page_flushed         =          0
Num_data_page_private_quota   =        327
Num_data_page_private_count   =        898
Num_data_page_fixed           =          1
Num_data_page_dirty           =          3
Num_data_page_lru1            =        857
Num_data_page_lru2            =        873
Num_data_page_lru3            =        898
Num_data_page_victim_candidate =        898
Num_sort_data_pages           =          0
Vacuum_data_page_buffer_hit_ratio =       0.00
Num_data_page_hash_anchor_waits =          0
Time_data_page_hash_anchor_wait =          0
Num_data_page_writes          =          0
Num_data_page_dirty_to_post_flush =          0
Num_data_page_skipped_flush   =          0
Num_data_page_skipped_flush_need_wal =          0
Num_data_page_skipped_flush_already_flushed =          0
Num_data_page_skipped_flush_fixed_or_hot =          0
Num_data_page_avoid_dealloc   =          0
Num_data_page_avoid_victim    =          0
Num_data_page_fix_ext:
Num_data_page_promote_ext:
Num_data_page_promote_time_ext:
Num_data_page_unfix_ext:
Time_data_page_lock_acquire_time:
Time_data_page_hold_acquire_time:
Time_data_page_fix_acquire_time:
```

Note

Each status information consists of 64-bit INTEGER data and the corresponding statistics information can be lost if the accumulated value exceeds the limit (highly unlikely though).

Note

Some sets of performance statistics are activated/deactivated by **extended_statistics_activation** system parameter. Each set is represented by a value power of two. To be activated, it needs to be present in the base-2 representation of the system parameter. This is the lists of sets that can be manipulated:

Value	Name	Active Default	Description
1	**Detailed b-tree pages**	Yes	Classifies b-tree pages into 3 categories: root, non-leaf and leaf Affected statistics: - Num_data_page_fix_ext - Time_data_page_lock_acquire_time - Time_data_page_hold_acquire_time - Time_data_page_fix_acquire_time

Value	Name	Active Default	Description
			- Num_data_page_promote_ext - Num_data_page_promote_time_ext - Num_data_page_unfix_ext
2	**MVCC Snapshot**	Yes	Activates statistics collection for MVCC snapshot: - Num_mvcc_snapshot_ext
4	**Time locks**	Yes	Activate statistics collection for timing lock waits: - Num_object_locks_time_waited_usec
8	**Hash anchor waits**	Yes	Activate statistics collection for hash anchor waits: - Num_data_page

Value	Name	Active Default	Description
			_hash_anchor_waits - Time_data_page_hash_anchor_wait
16	**Extended victimization**	No	Activate statistics collection for extended page buffer I/O and victimization module: - Num_data_page_writes - flush_collect_per_page - flush_flush_per_page - Num_data_page_skipped_flush_already_flushed - Num_data_page_skipped_flush_need_wal - Num_data_page_skipped_flush_fixed_or_hot - Num_data_page

Value	Name	Active Default	Description
			_dirty_to_post_flush - Num_alloc_bcb_prioritize_vacuum - Num_victim_assign_direct_vacuum_void - Num_victim_assign_direct_vacuum_lru - Num_victim_assign_direct_flush - Num_victim_assign_direct_panic - Num_victim_assign_direct_adjust_lru - Num_victim_assign_direct_adjust_lru_to_vacuum - Num_victim_assign_direct_search_for_flush - Num_victim_shared_lru_success - Num_victim_ow

Value	Name	Active Default	Description
			n_private_lru_success - Num_victim_other_private_lru_success - Num_victim_shared_lru_fail - Num_victim_own_private_lru_fail - Num_victim_other_private_lru_fail - Num_victim_get_from_lru - Num_victim_get_from_lru_was_empty - Num_victim_get_from_lru_fail - Num_victim_get_from_lru_bad_hint - Num_lfcq_prv_get_total_calls - Num_lfcq_prv_get_empty

Value	Name	Active Default	Description
			- Num_lfcq_prv_get_big - Num_lfcq_shr_get_total_calls - Num_lfcq_shr_get_empty
32	**Thread workers**	No	Activate statistics collection for thread worker pools - Thread_stats_counters_timers - Thread_loaddb_stats_counters_timers
64	**Thread daemons**	No	Activate statistics collection for daemon threads - Thread_pgbuf_daemon_stats_counters_timers
128	**Extended DWB**	No	Activate extented statistics collection for

Value	Name	Active Default	Description
			Double Write Buffer - Num_dwb_flushed_block_volumes
MAX	**All statistics**	No	Activate collection of all statistics

lockdb

The **cubrid lockdb** utility is used to check the information on the lock being used by the current transaction in the database.

```
cubrid lockdb [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **lockdb**: A command used to check the information on the lock being used by the current transaction in the database.
- *database_name*: The name of the database where lock information of the current transaction is to be checked.

The following example shows how to display lock information of the *testdb* database on a screen without any option.

```
cubrid lockdb testdb
```

The following shows [options] available with the **cubrid lockdb** utility.

-o, --output-file=FILE

The **-o** option displays the lock information of the *testdb* database as a output.txt.

```
cubrid lockdb -o output.txt testdb
```

-c, --contention

The **-c** option displays information on locks that are in contention.

```
cubrid lockdb -c testdb
```

Output Contents

The output contents of **cubrid lockdb** are divided into three logical sections.

- Server lock settings
- Clients that are accessing the database
- The contents of an object lock table

Server lock settings

The first section of the output of **cubrid lockdb** is the database lock settings.

```
*** Lock Table Dump ***
 Lock Escalation at = 100000, Run Deadlock interval = 0
```

The lock escalation level is 100,000 records, and the interval to detect deadlock is set to 0 seconds.

For a description of the related system parameters, **lock_escalation** and **deadlock_detection_interval**, see Concurrency/Lock-Related Parameters.

Clients that are accessing the database

The second section of the output of **cubrid lockdb** includes information on all clients that are connected to the database. This includes the transaction index, program name, user ID, hostname, process ID, isolation level and lock timeout settings of each client.

```
Transaction (index 1, csql, dba@cubriddb|12854)
Isolation COMMITTED READ
Timeout_period : Infinite wait
```

Here, the transaction index is 1, the program name is csql, the user ID is dba, the hostname is cubriddb, the client process identifier is 12854, the isolation level is COMMITTED READ and the lock timeout is unlimited.

A client for which transaction index is 0 is the internal system transaction. It can obtain the lock at a specific time, such as the processing of a checkpoint by a database. In most cases, however, this transaction will not obtain any locks.

Because **cubrid lockdb** utility accesses the database to obtain the lock information, the **cubrid lockdb** is an independent client and will be output as such.

Object lock table

The third section of the output of the **cubrid lockdb** includes the contents of the object lock table. It shows which client has the lock for which object in which mode, and which client is waiting for which object in which mode. The first part of the result of the object lock table shows how many objects are locked and how many lock objects are allocated to memory. CUBRID reuses lock objects for performance, but if the number of locks exceeds the value of **lock_escalation**, the lock objects are freed from memory at the end of the transaction. For more details, see Concurrency/Lock-Related Parameters.

```
Object lock Table:
    Current number of objects which are locked = 100
    Current number of objects which are allocated = 1000
    Current size of objects which are allocated = 242K
```

cubrid lockdb outputs the OID, object type and table name of each object that obtained lock. In addition, it outputs the number of transactions that hold lock for the object (*Num holders*), the number of transactions (*Num blocked-holders*) that hold lock but are blocked since it could not convert the lock to the upper lock (e.g., conversion from **SCH_S_LOCK** to **SCH_M_LOCK**), and the number of different transactions that are waiting for the lock of the object (*Num waiters*). It also outputs the list of client transactions that hold lock, blocked client transactions and waiting client transactions. For rows, but not class, MVCC information is also shown.

The example below shows an object in which the object type is a class, that will be blocked because the class OID(0| 62| 5) that has **IX_LOCK** for transaction 1 and **SCH_S_LOCK** for transaction 2 cannot be converted into **SCH_M_LOCK**. It also shows that transaction 3 is blocked because transaction 2 is waiting for **SCH_M_LOCK** even when transaction 3 is only waiting for **SCH_S_LOCK**.

```
OID = 0| 62| 5
Object type: Class = athlete.
Num holders = 1, Num blocked-holders= 1, Num waiters = 1
LOCK HOLDERS :
    Tran_index = 1, Granted_mode = IX_LOCK, Count = 1, Nsubgranules
= 1
BLOCKED LOCK HOLDERS :
    Tran_index = 2, Granted_mode = SCH_S_LOCK, Count = 1,
Nsubgranules = 0
    Blocked_mode = SCH_M_LOCK
```

```
                    Start_waiting_at = Wed Feb 3 14:44:31 2016
                    Wait_for_secs = -1
LOCK WAITERS :
    Tran_index = 3, Blocked_mode = SCH_S_LOCK
                    Start_waiting_at = Wed Feb 3 14:45:14 2016
                    Wait_for_secs = -1
```

The next example shows an instance of class, object OID(2| 50| 1), that was inserted by transaction 1 which holds **X_LOCK** on the object. The class has a unique index and the key of inserted instance is about to be modified by transaction 2, which is blocked until transaction 1 is completed. The **-c** option displays only the information on locks that are involved in contention.

```
OID = 2|50|1
Object type: instance of class ( 0| 62| 5) = athlete.
MVCC info: insert ID = 6, delete ID = missing.
Num holders = 1, Num blocked-holders= 1, Num waiters = 1
LOCK HOLDERS :
    Tran_index =   1, Granted_mode =   X_LOCK, Count =   1
LOCK WAITERS :
    Tran_index =   2, Blocked_mode = X_LOCK
                      Start_waiting_at = Wed Feb 3 14:45:14 2016
                      Wait_for_secs = -1
```

Granted_mode refers to the mode of the obtained lock, and *Blocked_mode* refers to the mode of the blocked lock. *Start_waiting_at refers* to the time at which the lock was requested, and *Wait_for_secs* refers to the waiting time of the lock. The value of *Wait_for_secs* is determined by **lock_timeout**, a system parameter.

When the object type is a class (table), *Nsubgranules* is displayed, which is the sum of the record locks and the key locks obtained by a specific transaction in the table.

```
OID = 0| 62| 5
Object type: Class = athlete.
Num holders = 2, Num blocked-holders= 0, Num waiters= 0
LOCK HOLDERS:
Tran_index = 3, Granted_mode = IX_LOCK, Count = 2, Nsubgranules = 0
Tran_index = 1, Granted_mode = IX_LOCK, Count = 3, Nsubgranules = 1
Tran_index = 2, Granted_mode = IX_LOCK, Count = 2, Nsubgranules = 1
```

tranlist

The **cubrid tranlist** is used to check the transaction information of the target database.

```
cubrid tranlist [options] database_name
```

If you omit the [options], it displays the total information about each transaction.

“cubrid tranlist demodb” outputs the similar result with “cubrid killtran -q demodb”, but tranlist outputs more items; “User name” and “Host name”. “cubrid tranlist -s demodb” outputs the same result with “cubrid killtran -d demodb”.

The following shows what information is displayed when you run “cubrid tranlist demodb”.

```
$ cubrid tranlist demodb

Tran index          User name      Host name      Process id
Program name              Query time    Tran time       Wait for
lock holder      SQL_ID       SQL Text
------------------------------------------------------------------------
------------------------------------------------------------------------
--------------------------------
   1(ACTIVE)           public     test-server           1681
broker1_cub_cas_1               0.00
0.00                        -1     *** empty ***
   2(ACTIVE)           public     test-server           1682
broker1_cub_cas_2               0.00
0.00                        -1     *** empty ***
   3(ACTIVE)           public     test-server           1683
broker1_cub_cas_3               0.00
0.00                        -1     *** empty ***
   4(ACTIVE)           public     test-server           1684
broker1_cub_cas_4               1.80         1.80
3, 2, 1     e5899a1b76253   update ta set a = 5 where a > 0
   5(ACTIVE)           public     test-server           1685
broker1_cub_cas_5               0.00
0.00                        -1     *** empty ***
------------------------------------------------------------------------
------------------------------------------------------------------------
--------------------------------

SQL_ID: e5899a1b76253
Tran index : 4
update ta set a = 5 where a > 0
```

In the above example, when each three transaction is running INSERT statement, UPDATE statement is tried to run in the other transaction. In the above, UPDATE statement with “Tran index” 4 waits for the transactions 3,2,1, which are found in “Wait for lock holder”, to be ended.

“SQL Text” is SQLs which are stored into the query plan cache; this is printed out as **empty** when this query’s execution is terminated.

Each column’s meaning is as following.

- Tran index : the index of transaction

- User name: database user's name
- Host name: hostname of CAS which running this transaction
- Process id : client's process id
- Program name : program name of a client
- Query time : total execution time for the running query (unit: second)
- Tran time : total run time for the current transaction (unit: second)
- Wait for lock holder : the list of transactions which own the lock when the current transaction is waiting for a lock
- SQL_ID: an ID for SQL Text
- SQL Text : running SQL text (maximum 30 characters)

Transaction status messages, which are shown on "Tran index", are as follows.

- ACTIVE : active state
- RECOVERY : recovering transaction
- COMMITTED : transaction which is already committed and will be ended soon.
- COMMITTING : transaction which is committing
- ABORTED : transaction which is rolled back and will be ended soon.
- KILLED : transaction which is forcefully killed by the server.

The following shows [options] available with the **cubrid tranlist** utility.

```
-s, --summary                print summarized information of
transaction
    --sort-key=COLUMN_NUMBER column number to be sorted
    --reverse                reverse the sort result
```

-s, --summary

This option outputs only summarized information(it omits query execution information or locking information).

```
$ cubrid tranlist -s demodb

Tran index          User name      Host name      Process
id      Program name
------------------------------------------------------------------
```

```
--------------
   1(ACTIVE)           public     test-server           1681
broker1_cub_cas_1
   2(ACTIVE)           public     test-server           1682
broker1_cub_cas_2
   3(ACTIVE)           public     test-server           1683
broker1_cub_cas_3
   4(ACTIVE)           public     test-server           1684
broker1_cub_cas_4
   5(ACTIVE)           public     test-server           1685
broker1_cub_cas_5
------------------------------------------------------------------
--------------
```

--sort-key=NUMBER

This option outputs the ascending values sorted by the NUMBERth column. If the type of the column is the number, it is sorted by the number; if not, it is sorted by the string. If this option is omitted, the output is sorted by "Tran index".

The following is an example which outputs the sorted information by specifying the "Process id", the 4th column.

```
$ cubrid tranlist --sort-key=4 demodb

Tran index          User name      Host name      Process id
Program name              Query time    Tran time       Wait for
lock holder      SQL_ID       SQL Text
------------------------------------------------------------------
------------------------------------------------------------------
----------------------------------------
   1(ACTIVE)           public     test-server           1681
broker1_cub_cas_1               0.00
0.00                    -1     *** empty ***
   2(ACTIVE)           public     test-server           1682
broker1_cub_cas_2               0.00
0.00                    -1     *** empty ***
   3(ACTIVE)           public     test-server           1683
broker1_cub_cas_3               0.00
0.00                    -1     *** empty ***
   4(ACTIVE)           public     test-server           1684
broker1_cub_cas_4               1.80
1.80                 3, 1, 2     e5899a1b76253   update ta set
a = 5 where a > 0
   5(ACTIVE)           public     test-server           1685
broker1_cub_cas_5               0.00
0.00                    -1     *** empty ***
------------------------------------------------------------------
------------------------------------------------------------------
----------------------------------------
```

```
SQL_ID: e5899a1b76253
Tran index : 4
update ta set a = 5 where a > 0
```

--reverse

This option outputs the reversely sorted values.

The following is an example which outputs the reversely sorted values by the "Tran index".

```
Tran index          User name      Host name      Process id
Program name              Query time    Tran time     Wait for
lock holder      SQL_ID       SQL Text
------------------------------------------------------------------------
------------------------------------------------------------------------
---------------------------------------
   5(ACTIVE)           public     test-server           1685
broker1_cub_cas_5               0.00
0.00                      -1     *** empty ***
   4(ACTIVE)           public     test-server           1684
broker1_cub_cas_4               1.80         1.80
3, 2, 1     e5899a1b76253   update ta set a = 5 where a > 0
   3(ACTIVE)           public     test-server           1683
broker1_cub_cas_3               0.00
0.00                      -1     *** empty ***
   2(ACTIVE)           public     test-server           1682
broker1_cub_cas_2               0.00
0.00                      -1     *** empty ***
   1(ACTIVE)           public     test-server           1681
broker1_cub_cas_1               0.00
0.00                      -1     *** empty ***
------------------------------------------------------------------------
------------------------------------------------------------------------
---------------------------------------

SQL_ID: e5899a1b76253
Tran index : 4
update ta set a = 5 where a > 0
```

killtran

The **cubrid killtran** is used to check transactions or abort specific transaction. Only a **DBA** can use options for killing a transaction.

```
cubrid killtran [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management

- **killtran**: A utility that manages transactions for a specified database
- *database_name*: The name of database whose transactions are to be killed

Some [options] refer to killing specified transactions; others refer to print active transactions. If no option is specified, **-d** is specified by default so all transactions are displayed on the screen.

```
cubrid killtran demodb

Tran index      User name   Host name      Process id      Program
name
------------------------------------------------------------------------
----------
   1(ACTIVE)          dba      myhost             664
cub_cas
   2(ACTIVE)          dba      myhost            6700
csql
   3(ACTIVE)          dba      myhost            2188
cub_cas
   4(ACTIVE)          dba      myhost             696
csql
   5(ACTIVE)       public      myhost            6944
csql
------------------------------------------------------------------------
----------
```

The following shows [options] available with the **cubrid killtran** utility.

```
-i, --kill-transaction-index=INDEX kill transaction with transaction
INDEX
    --kill-user-name=ID            kill all transactions with user ID
    --kill-host-name=HOST          kill all transactions with client
HOST
    --kill-program-name=NAME       kill all transactions with client
program NAME
    --kill-sql-id=SQL_ID           kill all transactions with SQL_ID
-p, --dba-password=PASS            password of the DBA user
-q, --query-exec-info              display information about query
execution information of active transactions
-d, --display-information          display information about active
transactions
-f, --force                        kill the transaction without a
prompt for verification
```

-i, --kill-transaction-index=ID1,ID2,ID3

This option kills transactions in a specified index. Several transaction indexes can be specified by separating with comma(,). If there is an invalid transaction ID among several IDs, it is ignored.

```
$ cubrid killtran -i 1 demodb
Ready to kill the following transactions:

Tran index          User name      Host name      Process
id      Program name
------------------------------------------------------------------------
--------------
   1(ACTIVE)              DBA         myhost
15771              csql
   2(ACTIVE)              DBA         myhost
2171              csql
------------------------------------------------------------------------
--------------
Do you wish to proceed ? (Y/N)y
Killing transaction associated with transaction index 1
Killing transaction associated with transaction index 2
```

--kill-user-name=ID

This option kills transactions for a specified OS user ID.

```
cubrid killtran --kill-user-name=os_user_id demodb
```

--kill-host-name=HOST

This option kills transactions of a specified client host.

```
cubrid killtran --kill-host-name=myhost demodb
```

--kill-program-name=NAME

This option kills transactions for a specified program.

```
cubrid killtran --kill-program-name=cub_cas demodb
```

--kill-sql-id=SQL_ID

This option kills transactions for a specified SQL ID.

```
cubrid killtran --kill-sql-id=5377225ebc75a demodb
```

-p, --dba-password=PASSWORD

This option can only be used, if using killing option such as **-i** and **--kill** options. A value followed by the -p option is a password of the **DBA**, and should be entered in the prompt.

-q, --query-exec-info

The difference with the output of "cubrid tranlist" command is that there are no "User name" column and "Host name" column. See tranlist.

-d, --display-information

This is the default option and it displays the summary of transactions. Its output is the same as the output of "cubrid tranlist" with **-s** option. See `tranlist -s`

-f, --force

This option omits a prompt to check transactions to be stopped.

```
cubrid killtran -f -i 1 demodb
```

checkdb

The **cubrid checkdb** utility is used to check the consistency of a database. You can use **cubrid checkdb** to identify data structures that are different from indexes by checking the internal physical consistency of the data and log volumes. If the **cubrid checkdb** utility reveals any inconsistencies, you must try automatic repair by using the **--repair** option.

```
cubrid checkdb [options] database_name [schema_name.table_name [{,
schema_name.table_name}]]
```

- **cubrid**: An integrated utility for CUBRID service and database management.
- **checkdb**: A utility that checks the data consistency of a specific database.
- *database_name*: The name of the database whose consistency status will be either checked or restored.
- *schema_name*: The schema name of the table.
- *table_name*: List the table names for consistency check or recovery

The following shows [options] available with the **cubrid checkdb** utility.

```
-S, --SA-mode                stand-alone mode execution
-C, --CS-mode                client-server mode execution
-r, --repair                 repair when inconsistency is found
    --check-prev-link        check only previous links of indices
    --repair-prev-link       repair previous links of indices
-i, --input-class-file=FILE  input FILE of class names
    --check-file-tracker     check all allocated pages of file
    --check-heap             check all pages of heap files
    --check-catalog          check catalog consistency
    --check-btree            check all btree indices
```

```
     --check-class-name       check class name consistency
     --check-btree-entries    check entries of btree consistency
 -I, --index-name=INDEX_NAME  index name that will be checked only
```

-S, --SA-mode

This option is used to access a database in standalone, which means it works without processing server; it does not have an argument. If **-S** is not specified, the system recognizes that a database is running in client/server mode.

```
cubrid checkdb -S demodb
```

-C, --CS-mode

This option is used to access a database in client/server mode, which means it works in client/server process respectively; it does not have an argument. If **-C** is not specified, the system recognize that a database is running in client/server mode by default.

```
cubrid checkdb -C demodb
```

-r, --repair

This option is used to restore an issue if a consistency error occurs in a database.

```
cubrid checkdb -r demodb
```

--check-prev-link

This option is used to check if there are errors on previous links of an index.

```
$ cubrid checkdb --check-prev-link demodb
```

--repair-prev-link

This option is used to restore if there are errors on previous links of an index.

```
$ cubrid checkdb --repair-prev-link demodb
```

-i, --input-class-file=FILE

You can specify tables to check the consistency or to restore, by specifying the **-i** *FILE* option or listing the table names after a database name. Both ways can be used together. If a target is not specified, entire database will be a target of consistency check or restoration.

```
cubrid checkdb demodb public.table_1 public.table_2
cubrid checkdb -r demodb public.table_1 public.table_2
```

```
cubrid checkdb -r -i table_list.txt demodb public.table_1
public.table_2
```

Empty string, tab, carriage return and comma are separators among table names in the table list file specified by **-i** option. The following example shows the table list file; from t1 to t10, it is recognized as a table for consistency check or restoration.

```
t1 t2 t3,t4 t5
t6, t7 t8   t9

     t10
```

--check-file-tracker

Check about all pages of all files in file-trackers.

--check-heap

Check about all heap-files.

--check-catalog

Check the consistency about catalog information.

--check-btree

Check the validity about all B-tree indexes.

--check-class-name

Check the identical between the hash table of a class name and the class information(oid) brought from a heap file.

--check-btree-entries

Check the consistency of all B-tree entries.

-I, --index-name=INDEX_NAME

Check if the index specified with this option about checking table. If you use this option, there is no heap validation check. Only one table and one index are permitted when you use this option; if you don't input a table name or input two tables, an error occurs.

diagdb

You can check various pieces of internal information on the database with the **cubrid diagdb** utility. Information provided by **cubrid diagdb** is helpful in diagnosing the current status of the database or figuring out a problem.

```
cubrid diagdb options database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **diagdb**: A command that is used to check the current storage state of the database by Dumping the information contained in the binary file managed by CUBRID in text format. It normally executes only when the database is in a stopped state. You can check the whole database or the file table, file size, heap size, class name or disk bitmap selectively by using the provided option.
- *database_name*: The name of the database whose internal information is to be diagnosed.

The following shows [options] available with the **cubrid diagdb** utility.

```
-d, --dump-type=TYPE          TYPE of dump
-o, --output-file=FILE        redirect output messages to FILE
    --emergency               emergency restart
```

-d, --dump-type=TYPE

This option specifies the output range when you display the information of all files in the *demodb* database. If any option is not specified, the default value of -1 is used.

```
cubrid diagdb -d 1 demodb
```

The utility has 9 types of **-d** options as follows:

Type	Description
-1	Displays all database information.
1	Displays file table information.
2	Displays file capacity information.
3	Displays heap capacity information.
4	Displays index capacity information.

5	Displays class name information.
6	Displays disk bitmap information.
7	Displays catalog information.
8	Displays log information.
9	Displays heap information.

-o, --output-file=FILE

The **-o** option is used to store information of the parameters used in the server/client process of the database into a specified file. The file is created in the current directory. If the **-o** option is not specified, the message is displayed on a console screen.

```
cubrid diagdb -d8 -o logdump_output demodb
```

--emergency

Use **--emergency** option to suppress recovery. **This option is meant ONLY for debugging, if there are recovery issues. It is recommended to backup your database before using this option.**

paramdump

The **cubrid paramdump** utility outputs parameter information used in the server/client process.

```
cubrid paramdump [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management
- **paramdump**: A utility that outputs parameter information used in the server/client process
- *database_name*: The name of the database in which parameter information is to be displayed.

The following shows [options] available with the **cubrid paramdump** utility.

```
-o, --output-file=FILE       redirect output message to FILE
-b, --both                   dump both client and server parameters
```

```
-S, --SA-mode                stand-alone mode execution
-C, --CS-mode                client-server mode execution
```

-o, --output-file=FILE

The **-o** option is used to store information of the parameters used in the server/client process of the database into a specified file. The file is created in the current directory. If the **-o** option is not specified, the message is displayed on a console screen.

```
cubrid paramdump -o db_output demodb
```

-b, --both

The **-b** option is used to display parameter information used in server/client process on a console screen. If the **-b** option is not specified, only server-side information is displayed.

```
cubrid paramdump -b demodb
```

-S, --SA-mode

This option displays parameter information of the server process in standalone mode.

```
cubrid paramdump -S demodb
```

-C, --CS-mode

This option displays parameter information of the server process in client/server mode.

```
cubrid paramdump -C demodb
```

tde

The **cubrid tde** utility is used to manage the TDE encryption of the database and can only be executed by the **DBA** user. With **cubrid tde** utility, you can change the key set on the database, add a new key, or remove a key in the key file stably. Also, you can inquire about the keys added to the key file and the key set on the database. For more information, see TDE (Transparent Data Encryption).

```
cubrid tde <operation> [option] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **tde**: A utility that manages TDE encryption applied to the database.

- *operation*: There are four types of operation: key addition, key deletion, key change, and showing keys' information. One operation must be set, and they are exclusive.
- *database_name*: The name of the database on which TDE administration operations to be performed.

The following table shows <operation> available with the cubrid tde utility.

```
-s, --show-keys               print key information set on the
database and the key file (_keys)
-n, --generate-new-key        generate a new key in the key file
(_keys)
-d, --delete-key=KEY_INDEX    delete a key in the key file (_keys)
-c, --change-key=KEY_INDEX    change the key set on the database for
another key in the key file (_keys)
```

-s, --show-keys

This option displays information about the keys set on the database and keys in the key file (_keys).

```
$ cubrid tde -s testdb
Key File: /home/usr/CUBRID/databases/testdb/testdb_keys

The current key set on testdb:
Key Index: 2
Created on Fri Nov 27 11:14:54 2020
Set     on Fri Nov 27 11:15:30 2020

Keys Information:
Key Index: 0 created on Fri Nov 27 11:11:27 2020
Key Index: 1 created on Fri Nov 27 11:14:47 2020
Key Index: 2 created on Fri Nov 27 11:14:54 2020
Key Index: 3 created on Fri Nov 27 11:14:55 2020

The number of keys: 4
```

The current key section shows the information on the key set on the current database, which displays the index for the key in the key file, the creation time, and the set time. The set key can be identified by the key index and creation time, and a key change plan can be established with the set time.

Keys Information section shows the keys added and being managed in the key file, through which the key indexes and creation time of them is checked.

-n, --generate-new-key

This option is used to add a new key to the key file (up to 128). If it is successful, the index of the added key is displayed, and this index is used to identify the key when changing or removing the key later. Information of added keys can be checked by **--show-keys**.

```
$ cubrid tde -n testdb
Key File: /home/usr/CUBRID/databases/testdb/testdb_keys

SUCCESS: A new key has been generated - key index: 1 created on
Tue Dec  1 11:30:47 2020
```

-d, --delete-key=KEY_INDEX

This option is used to remove a key specified by index from the key file. The key currently set on the database cannot be removed.

```
$ cubrid tde -d 1 testdb
Key File: /home/usr/CUBRID/databases/testdb/testdb_keys

SUCCESS: The key (index: 1) has been deleted
```

-c, --change-key=KEY_INDEX

This option is used to change the key set on the database to another key existing in the key file. When changing, both the previously set key and the new key to be set must exist.

```
$ cubrid tde -c 2 testdb
Key File: /home/usr/CUBRID/databases/testdb/testdb_keys

Trying to change the key from the key (index: 0) to the key
(index: 2)..
SUCCESS: The key has been changed from the key (index: 0) to the
key (index: 2)
```

To change the key set on the database, a user must first create a key to be set by the **--generate-new-key** option. The user can create a new key to change, or create multiple keys in advance for changing the key according to their own security plans.

The following table shows [options] available with the cubrid tde utility.

-p, --dba-password=PASSWORD

This option specifies the password of the DBA.

vacuumdb

The **cubrid vacuumdb** utility is used to vacuum deleted records and unnecessary mvcc related information from records in the database. And it is also used to get the status information about the vacuum.

```
cubrid vacuumdb [options] database_name
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **vacuumdb**: A utility that vacuum deleted records and unnecessary mvcc related information from records in the database.
- *database_name*: The name of the database to vacuum.

The following shows [options] available with the **cubrid vacuumdb** utility.

```
-o, --output-file=FILE       redirect output messages to FILE
    --dump                   dump current state of vacuum
-S, --SA-mode                stand-alone mode execution
-C, --CS-mode                client-server mode execution
```

-o, --output-file=FILE

This option is used to store the status information about the vacuum in a specified file. The file is created in the current directory. If the **-o** option is not specified, the message is displayed on a console screen.

```
cubrid vacuumdb -o db_output demodb
```

--dump

This option is used to display the status information about the vacuum. It currently displays the first log page ID referenced by the vacuum and the log volume name in which it resides. Users can check the progress of the vacuum operation through the change of the first log page ID and they can know the minimum log volume to keep for the vacuum operation. It can be used with both the standalone mode and the client/server mode. If specified together with the **-C** option, information on indexes that have read more leaf's overflow pages than the threshold is also displayed, when the index is vacuumed. The related system parameters are **vacuum_ovfp_check_duration** and **vacuum_ovfp_check_threshold**.

```
cubrid vacuumdb --dump demodb
```

```
$ cubrid vacuumdb -C --dump demodb

*** Vacuum Dump ***
First log page ID referenced = 85653 (in demodb_lgar013)

*** Exceeding read threshold (1000 pages) for OID overflow pages
(OVFP), Since 09/04/23 08:53:50 ***
  Class name            Index name            Count         Num
of OVFP recent read    Max num of OVFP read
=============================================================================
=============================================
  dba.tm                v_idx                       272451
1089 (09/04/23 10:49:52)    1478 (09/04/23 10:28:46)
  dba.tbl               idx                         322728
1023 (09/04/23 09:53:47)    1475 (09/04/23 09:29:29)

$
```

- Exceeding read threshold (**1000** pages) : Shows vacuum_ovfp_check_threshold parameter.
- Since **09/04/23 08:53:50** : Shows the time information when OVFP monitoring started
- Class name: Class name(table name), included the owner's name.
- Index name: Index name
- Count: The number of times an index's overflow pages were read by exceeding the threshold (vacuum_ovfp_check_threshold) during index vacuum operation.
- Num of OVFP recent read: The latest time an index's overflow page was read by exceeded the threshold (vacuum_ovfp_check_threshold) during index vacuum operation.
- Max num of OVFP read: The latest time an index's overflow page was maximally read by exceeding the threshold (vacuum_ovfp_check_threshold) during index vacuum operation.

-S, --SA-mode

This option performs vacuuming the database in standalone mode.

```
cubrid vacuumdb -S demodb
```

-C, --CS-mode

This option performs vacuuming the database in client/server mode. But it is not currently supported to vacuum in this mode.

```
cubrid vacuumdb -C demodb
```

flashback

The **cubrid flashback** utility is used to get SQL statements to rewind a specific committed transaction, and can only be executed by the **DBA** user. The system parameter **supplemental_log** must be turned on, and flashback is supported only for DML executed after the **supplemental_log** is set. This utility can only be performed in client/server mode and cannot be executed more than once simultaneously.

```
cubrid flashback [options] database_name owner_name.class_name1
[owner_name.class_name2, ...]
```

- **cubrid**: An integrated utility for the CUBRID service and database management.
- **flashback**: A utility that provides statements to rewind a specific transaction.
- *database_name*: The name of the database whose transaction is to be rewound.
- *owner_name.class_name*: The classes to rewind. When specifying the class_name, it must be specified along with the owner name.

The following shows what information is displayed when a user runs "cubrid flashback demodb dba.tbl".

```
$ csql -u dba demodb

csql> CREATE TABLE tbl (a INT);
csql> INSERT INTO tbl VALUES (10);

$ cubrid flashback demodb dba.tbl

Flashback Summary
Number of Transaction: 1
Start date - End date: 03-05-2022:10:52:56 - 03-05-2022:11:02:56
Transaction id  User name                          Start
time            End time              Num_insert  Num_update
Num_delete  Table
           135  DBA
03-05-2022:11:02:40   03-05-2022:11:02:41            1
0           0  dba.tbl
Enter transaction id (press -1 to quit): 135

delete from [dba.tbl] where [a] = 10 limit 1;
```

In the above example, when the flashback is executed, it displays information about transactions performed within the specified time period. If users do not specify a period, the transaction history up to 10 minutes before the current time is displayed. When the user selects a transaction ID, SQL statements are provided to execute the flashback on DML executed within the selected

transaction. The user must enter the transaction ID within 300 seconds which can be adjusted using the system parameter flashback_timeout .

Each column's meaning in the **Flashback Summary** is as following.

- Transaction id : The identifier of the transaction
- User name : Transaction user
- Start time : Transaction start time (approximate time)
- End time : Transaction end time
- Num_insert : Number of insert operations executed within the transaction
- Num_update : Number of update operations executed within the transaction
- Num_delete : Number of delete operations executed within the transaction
- Table : List of tables for which DMLs have been performed within the transaction

Note

If users attempt to rewind a table on which a trigger is created, unintended results may be obtained. Users are recommended to disable triggers on the table before executing the flashback. For more information, see ALTER TRIGGER.

Note

The flashback utility does not support complex data types such as SET, MULTISET, LIST, and JSON. When performing flashback on a class that includes columns with unsupported data types, those columns are displayed as having null values. In addition, for LOB type columns, only the location information of the file(LOB locator) is displayed, not the data itself.

The following shows [options] available with the **cubrid flashback** utility.

```
-o, --output-file=FILE          redirect output message to FILE
-u, --user=ID                   specify the user who executed the SQL
statement to flashback
-p, --dba-password              password of the DBA user
-s, --start-date=DATE           specify starting DATE for flashback
-e, --end-date=DATE             specify end DATE for the flashback
--detail                        output a detailed information for each
```

```
SQL statement.
--oldest                        flashback is performed in
chronological order for SQL statements executed from start time to
end time;
```

-o, --output-file=FILE

This option is used to store the SQL statements that can rewind a specific transaction in a specified file. If the **-o** option is not specified, the message is displayed on a console screen.

```
cubrid flashback -o db_output demodb dba.tbl
```

-u, --user=ID

This option is used to specify the user who executed the SQL statements. If the **-u** option is not specified, then it searches for statements executed by all users.

```
$ csql -u public demodb

csql> CREATE TABLE tbl (a int);
csql> INSERT INTO tbl VALUES (10);

csql> CALL login ('dba', '') ON CLASS db_user;

csql> INSERT INTO public.tbl VALUES (20);

$ cubrid flashback -u public demodb dba.tbl

Flashback Summary
Number of Transaction: 1
Start date - End date: 03-05-2022:10:52:56 - 03-05-2022:11:02:56
Transaction id  User name                          Start
time            End time              Num_insert  Num_update
Num_delete  Table
           135  PUBLIC
03-05-2022:11:02:40   03-05-2022:11:02:41            1
0           0  public.tbl
Enter transaction id (press -1 to quit): 135

delete from [public.tbl] where [a] = 10 limit 1;
```

-p, --dba-password=PASSWORD

This option specifies the password of the dba. If the **-p** option is not specified, then the password should be entered in the prompt.

-s, --start-date=DATE

This option specifies the start date in the dd-mm-yyyy:hh:mi:ss (e.g. 28-04-2022:14:10:00) format. It finds transactions that started after the specified time or are in progress at the specified time among committed transactions.

```
cubrid flashback -s 28-04-2022:14:10:00 demodb dba.tbl
```

-e, --end-date=DATE

This option specifies the end date in the dd-mm-yyyy:hh:mi:ss (e.g. 28-04-2022:14:10:00) format. It finds transactions that have committed before the specified time.

If no start date is specified, the start date is set to the value of 10 minutes before the end date. Also, if an end date is not specified, the end date is set to 10 minutes after the start date. If neither start date nor end date is set, transactions performed up to 10 minutes before the current time are searched for.

```
cubrid flashback -e 28-04-2022:14:10:00 demodb dba.tbl
```

--detail

This option is used to display the detailed information for each SQL statement. Transaction ID, transaction user, and original SQL statements are displayed with the SQL statements for flashback. The original SQL statement is not the exact syntax user executed. It simply represents the user-executed statement as INSERT/UPDATE/DELETE statements that are executed row-by-row. For example, An INSERT ... SELECT statement is expressed as multiple INSERT statements. If the **--detail** option is not specified, only the SQL statements for flashback is displayed.

```
cubrid flashback --detail demodb dba.tbl

Flashback Summary
Number of Transaction: 1
Start date - End date: 03-05-2022:10:52:56 - 03-05-2022:11:02:56
Transaction id  User name                          Start
time            End time              Num_insert  Num_update
Num_delete  Table
           135  DBA                                03-05-2022:11:
02:40   03-05-2022:11:02:41            1           0
0  dba.tbl
Enter transaction id (press -1 to quit): 135

[TRANSACTION ID] 135
[USER]           DBA
[ORIGINAL]       insert into [dba.tbl] values (10);
[FLASHBACK]      delete from [dba.tbl] where [a] = 10 limit 1;
```

--oldest

This option is used to display the SQL statements executed within the specified transaction in chronological order. If the **--oldest** is not specified, the SQL statements executed within the transaction are displayed in reverse chronological order.

```
cubrid flashback --oldest demodb dba.tbl
```

memmon

The **cubrid memmon** utility prints the heap memory usage currently allocated to the server process. When the system parameter **enable_memory_monitoring** is set to *yes*, the server memory monitoring module tracks and manages the total usage of heap memory and detailed memory allocation information based on the CUBRID source code and the line information where memory allocation occurs. This allows users to check the current state of the server's heap memory usage at the time the utility is executed.

```
cubrid memmon [option] database_name
```

- **cubrid**: Integrated utility for CUBRID service and database management.
- **memmon**: A utility that provides information of heap memory usage on the server process.
- *database_name*: The name of the database to check the heap memory usage status.

The following example shows the result of executing "cubrid memmon demodb".

```
====================cubrid memmon====================
Server Name: demodb
Total Memory Usage: 935240660 Bytes (for meta info: 470300 Bytes)
-------------------------------------------------------
        File
Name
|     Memory Usage(Ratio)
        storage/page_buffer.c:
4977
|        525315320 Bytes( 56%)
        transaction/log_page_buffer.c:
584
|        262147560 Bytes( 28%)
        ...
```

When **cubrid memmon** is executed, it displays the following information: the name of the database server, the total amount of heap memory in use, and the amount of heap memory used to store meta information for tracking and managing allocated heap memory. Detailed items include the memory usage and usage ratio for each file and line [*file:line*] at which heap

memory allocation occurred, sorted by usage. If the system parameter **enable_memory_monitoring** sets to no or the –disable-force option is used to forcibly stop the memory monitoring feature, an error message will be displayed.

The information output by the **cubrid memmon** utility has the following meanings.

- Server name : The name of database
- Total Memory Usage : The total amount of heap memory in use
  - for meta info : The memory usage of the metadata used to track and manage the total heap memory usage
- File Name : The name of the CUBRID source code file and the line number where the heap memory allocation occurred
- Memory Usage(Ratio) : Heap memory usage (ratio compared to the total heap memory usage)

Note

To record the history of heap memory allocations, the point at which heap memory is deallocated and the current memory usage becomes 0 is also included in the output of **cubrid memmon**.

The following are the [option] used by **cubrid memmon**.

-o, --output-file=FILE

Option to save the result of the utility execution to a specified file. If the **-o** option is not specified, messages will be displayed on the console screen.

```
cubrid memmon -o memory_usage_output demodb
```

--disable-force

Option to forcibly disable the memory monitoring feature. If the memory monitoring feature is forcibly disabled, the **cubrid memmon** utility will not display the server's heap memory usage. To resume tracking memory usage, the server must be restarted.

```
cubrid memmon --disable-force demodb
```

HA Commands

cubrid changemode utility prints or changes the HA mode.

cubrid applyinfo utility prints the information of applied transaction logs in the HA environment.

For more details, see Registering HA to cubrid service.

Locale Commands

cubrid genlocale utility compiles the locale information to use. This utility is executed in the **make_locale.sh** script (**.bat** for Windows).

cubrid dumplocale utility dumps the compiled binary locale (CUBRID locale library) file as a human-readable format on the console. It is better to save the output as a file by output redirection.

cubrid synccolldb utility checks if the collations between database and locale library are consistent or not, and synchronize them.

For more detailed usage, see Locale Setting.

Timezone Commands

cubrid gen_tz utility has two modes:

- **new** mode when it compiles the IANA timezone data stored in the tzdata folder into a C source code file. This file is then converted into a .so shared library for Linux or .dll library for Windows using the **make_tz.sh** (Linux) / **make_tz.bat** (Windows) scripts.
- **extend** mode is similar to new but is used when you want to update your timezone data to a different version and ensure backward compatibility with the old data. It is always used with a database name argument. In some situations, when it is impossible to ensure backward compatibility just by merging the two versions of timezone data, an update of the data in the tables of the database is done. It is executed using **make_tz.sh -g extend** for Linux and **make_tz.bat /extend** for Windows.

cubrid dump_tz utility dumps the compiled CUBRID timezone library file as a human-readable format on the console. It is better to save the output as a file by output redirection.

System Parameters

This chapter provides information about configuring system parameters that can affect the system performance. System parameters determine overall performance and operation of the

system. This chapter explains how to use configuration files for database server and broker as well as a description of each parameter.

This chapter covers the following topics:

- Configuring the database server
- Configuring the broker
- HA Configuration
- SHARD Configuration

Configuring the Database Server

Scope of Database Server Configuration

CUBRID consists of the database server, the broker and the CUBRID Manager. Each component has its configuration file. The system parameter configuration file for the database server is **cubrid.conf** located in the **$CUBRID/conf** directory. System parameters configured in **cubrid.conf** affect overall performance and operation of the database system. Therefore, it is very important to understand the database server configuration.

The CUBRID database server has a client/server architecture. To be more specific, it is divided into a database server process linked to the server library and the broker process linked to the client library. The server process manages the database storage structure and provides concurrency and transaction functionalities. The client process prepares for query execution and manages object/schema.

System parameters for the database server, which can be set in the **cubrid.conf** file, are classified into a client parameter, a server parameter and a client/server parameter according to the range to which they are applied. A client parameter is only applied to client processes such as the broker. A server parameter affects the behaviors of the server processes. A client/server parameter must be applied to both server and client.

Note

Location of cubrid.conf File and How It Works

The **cubrid.conf** file is located on the **$CUBRID/conf** directory. For setting by database, it divides into a section in the **cubrid.conf** file.

Changing Database Server Configuration

Editing the Configuration File

You can add/delete parameters or change parameter values by manually editing the system parameter configuration file (**cubrid.conf**) in the **$CUBRID/conf** directory.

The following parameter syntax rules are applied when configuring parameters in the configuration file:

- Parameter names are not case-sensitive.
- The name and value of a parameter must be entered in the same line.
- An equal sign (=) can be to configure the parameter value. Spaces are allowed before and after the equal sign.
- If the value of a parameter is a character string, enter the character string without quotes. However, use quotes if spaces are included in the character string.

Using SQL Statements

You can configure a parameter value by using SQL statements in the CSQL Interpreter or CUBRID Manager's Query Editor. Note that you cannot change every parameter. For updatable parameters, see cubrid.conf Configuration File and Default Parameters.

```
SET SYSTEM PARAMETERS 'parameter_name=value [{; name=value}...]'
```

parameter_name is the name of a client parameter whose value is editable. In this syntax, *value* is the value of the given parameter. You can change multiple parameter values by separating them with a semicolon(;). You should be careful when you change a parameter.

The following example shows how to retrieve the result of an index scan in OID order and configure the number of queries to be stored in the history of the CSQL Interpreter to 70.

```
SET SYSTEM PARAMETERS 'index_scan_in_oid_order=1;
csql_history_num=70';
```

DEFAULT for *value* will reset the parameter to its default value with an exception of **call_stack_dump_activation_list** parameter.

```
SET SYSTEM PARAMETERS 'lock_timeout=DEFAULT';
```

Using Session Commands of the CSQL Interpreter

You can configure system parameter values by using session commands (;SET) in the CSQL Interpreter. Note that you cannot change every parameter. For updatable parameters, see cubrid.conf Configuration File and Default Parameters.

The following example shows how to configure the block_ddl_statement parameter to yes so that execution of DDL statements is not allowed.

```
csql> ;set block_ddl_statement=yes
=== Set Param Input ===
block_ddl_statement=yes
```

cubrid.conf Configuration File and Default Parameters

CUBRID consists of the database server, the broker and the CUBRID Manager. The name of the configuration file for each component is as follows. These files are all located in the **$CUBRID/conf** directory.

- Database server configuration file: **cubrid.conf**
- Broker configuration file: **cubrid_broker.conf**
- CUBRID Manager server configuration file: **cm.conf**

cubrid.conf is a configuration file that sets system parameters for the CUBRID database server and determines overall performance and operation of the database system. In the **cubrid.conf** file, some important parameters needed for system installation are provided, having their default values.

Database Server System Parameters

The following are database server system parameters that can be used in the **cubrid.conf** configuration file. On the following table, “Applied” column’s “client parameter” means that they are applied to CAS, CSQL, **cubrid** utilities. Its “server parameter” means that they are applied to the DB server (cub_server process). For the scope of **client** and **server parameters**, see Scope of Database Server Configuration.

You can change the parameters that are capable of changing dynamically the setting value through the **SET SYSTEM PARAMETERS** statement or a session command of the CSQL Interpreter, For more semantics on **;set** see Viewing Query Plan . **;set** while running the DB. If you are a DBA, you can change parameters regardless of the applied classification. However, if you are not a DBA, you can only change “session” parameters. (on the below table, a parameter of which “session” item’s value is O.)

On the below table, if “Applied” is “server parameter”, that parameter affects to cub_server process; If “client parameter”, that parameter affects to CAS, CSQL or “cubrid” utilities which run on client/server mode (--CS-mode). “Client/server parameter” affects to all of cub_server, CAS, CSQL and “cubrid” utilities.

“Dynamic Change” and “Session or not” are marked on the below table. The affected range of the parameter which “Dynamic Change” is “available” depends on “Applied” and “Session” items.

- If “Dynamic Change” is “available” and “Applied” is “server parameter”, that parameter’s changed value is applied to DB server. Then applications use the changed value of the parameter until the DB server is restarted.
- If “Dynamic Change” is “available” and “Applied” is “client parameter”, this belongs to the “session” parameter and that parameter’s changed value is applied only to that DB session. In other words, the changed value is only applied to the application which requested to change that value. For example, if **block_ddl_statement** parameter’s value is changed into **yes**, then only the application requested to change that parameter cannot use DDL statements.
- If “Dynamic Change” is “available”, “Applied” is “client parameter” and;
  - this belongs to the “session” parameter, that parameter’s changed value is applied only to that DB session. In other words, the changed value is only applied to the application requested to change that value. For example, if **add_column_update_hard_default** parameter’s value is changed into **yes**, then only the application requested to change that parameter lets the newly added column with NOT NULL constraint have hard default value.
  - this does not belong to the “session” parameter, the values of “client” side and “server” side are changed. For example, **error_log_level** parameter is applied to each of “server” side and “client” side; if this value is changed from “ERROR” into “WARNING”, this is applied only to “server” (cub_server process) and “client” (CAS or CSQL) which requested to change this value. Other “clients” keeps the value of “ERROR”.

Note

If you want to change the value of a parameter permanently, restart all of DB server and broker after changing configuration values of cubrid.conf.

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
				int	1523	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
Connection-Related Parameters	cubrid_port_id	client parameter				
	check_peer_alive	client/server parameter	O	string	both	available
	db_hosts	client parameter	O	string	NULL	available
	max_clients	server parameter		int	100	
	tcp_keepalive	client/server parameter		bool	yes	
	use_user_hosts	client/server parameter		bool	off	
Memory-Related Parameters	data_buffer_size	server parameter		byte	32,768 * db_page_size	
	index_scan_oid_buffer_size	server parameter		byte	4 * db_page_size	
	max_agg_hash_size	server parameter		byte	2,097,152(2M)	
				byte		

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	max_hash_list_scan_size	server parameter			8,388,608(8M)	
	max_subquery_cache_size	server parameter		byte	2,097,152(2M)	DBA only
	sort_buffer_size	server parameter		byte	128 * db_page_size	
	temp_file_memory_size_in_pages	server parameter		int	4	
	thread_stacksize	server parameter		byte	1,048,576	
Disk-Related Parameters	db_volume_size	server parameter		byte	512M	
	dont_reuse_heap_file	server parameter		bool	no	
	log_volume_size	server parameter		byte	512M	
	temp_file_max_size_in_pages	server parameter		int	-1	
				string	NULL	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	temp_volume_path	server parameter				
	unfill_factor	server parameter		float	0.1	
	volume_extension_path	server parameter		string	NULL	
	double_write_buffer_size	server parameter		byte	2,097,152(2M)	
	data_file_os_advise	server parameter		int	0	
Error Message-Related Parameters	call_stack_dump_activation_list	client/ server parameter		string	DEFAULT	DBA only
	call_stack_dump_deactivation_list	client/ server parameter		string	NULL	DBA only
	call_stack_dump_on_error	client/ server parameter		bool	no	DBA only
	error_log	client/ server parameter		string	cub_client.err,	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
					cub_server.err	
	error_log_level	client/ server parameter		string	NOTIFICATION	DBA only
	error_log_warning	client/ server parameter		bool	no	DBA only
	error_log_size	client/ server parameter		int	512M	DBA only
Concurrency/Lock-Related Parameters	deadlock_detection_interval_in_secs	server parameter		float	1.0	DBA only
	isolation_level	client parameter	O	int	4	available
	lock_escalation	server parameter		int	100,000	
	lock_timeout	client parameter	O	msec	-1	available
	rollback_on_lock_escalation	server parameter		bool	no	DBA only

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
Logging-Related Parameters	adaptive_flush_control	server parameter		bool	yes	DBA only
	background_archiving	server parameter		bool	yes	DBA only
	checkpoint_every_size	server parameter		byte	100,000 * log_page_size	
	checkpoint_interval	server parameter		msec	6min	DBA only
	checkpoint_sleep_msecs	server parameter		msec	1	DBA only
	force_remove_log_archives	server parameter		bool	yes	DBA only
	log_buffer_size	server parameter		byte	16k * log_page_size	
	log_max_archives	server parameter		int	INT_MAX	DBA only
	log_trace_flush_time	server parameter		msec	0	DBA only

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	max_flush_size_per_second	server parameter		byte	10,000 * db_page_size	DBA only
	remove_log_archive_interval_in_secs	server parameter		sec	0	DBA only
	sync_on_flush_size	server parameter		byte	200 * db_page_size	DBA only
	ddl_audit_log	client parameter		bool	no	
	ddl_audit_log_size	client parameter		byte	10M	
Transaction Processing-Related Parameters	async_commit	server parameter		bool	no	
	group_commit_interval_in_msecs	server parameter		msec	0	DBA only
	dblink_auto_commit	server parameter		bool	yes	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
Statement/ Type-Related Parameters	add_column_update_hard_default	client/ server parameter	O	bool	no	available
	alter_table_change_type_strict	client/ server parameter	O	bool	yes	available
	allow_truncated_string	client/ server parameter	O	bool	no	available
	ansi_quotes	client parameter		bool	yes	
	block_ddl_statement	client parameter	O	bool	no	available
	block_nowhere_statement	client parameter	O	bool	no	available
	compat_numeric_division_scale	client/ server parameter	O	bool	no	available
	create_table_reuseoid	client parameter	O	bool	yes	available

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	cte_max_recursions	client/ server parameter	O	int	2000	available
	default_week_format	client/ server parameter	O	int	0	available
	group_concat_max_len	server parameter	O	byte	1,024	DBA only
	intl_check_input_string	client parameter	O	bool	no	available
	intl_collation	client parameter	O	string		available
	intl_date_lang	client parameter	O	string		available
	intl_number_lang	client parameter	O	string		available
	json_max_array_idx	server parameter	O	int	65,536	available
	max_query_per_tran	server parameter		int	100	
	no_backslash_escapes	client parameter		bool	yes	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	only_full_group_by	client parameter	O	bool	no	available
	oracle_compat_number_behavior	server parameter		bool	no	
	oracle_style_empty_string	client parameter		bool	no	
	pipes_as_concat	client parameter		bool	yes	
	plus_as_concat	client parameter		bool	yes	
	require_like_escape_character	client parameter		bool	no	
	return_null_on_function_errors	client/ server parameter	O	bool	no	available
	stored_procedure_return_numeric_size	client/ server parameter		list	38,15	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	string_max_size_bytes	client/server parameter	O	byte	1,048,576	available
	unicode_input_normalization	client parameter	O	bool	no	available
	unicode_output_normalization	client parameter	O	bool	no	available
	update_use_attribute_references	client parameter	O	bool	no	available
Thread-Related Parameters	thread_connection_pooling	server parameter		bool	yes	
	thread_connection_timeout_seconds	server parameter		int	300	
	thread_worker_pooling	server parameter		bool	yes	
	thread_core_count	server parameter		int	# of system cores	
				int	300	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	thread_worker_timeout_seconds	server parameter				
	loaddb_worker_count	server parameter		int	8	
Timezone Parameter	server_timezone	server parameter		string	OS timezone	available
	timezone	client/ server parameter	O	string	the value of server_timezone	available
	tz_leap_second_support	server parameter		bool	no	available
Query Plan Cache-Related Parameters	max_plan_cache_entries	client/ server parameter		int	1,000	
	max_plan_cache_clones	server parameter		int	1,000	
	xasl_cache_time_threshold_in_minutes	client/ server parameter		int	360	
				int	1,000	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	max_filter_pred_cache_entries	client/ server parameter				
Query Cache-Related Parameters	max_query_cache_entries	server parameter		int	0	available
	query_cache_size_in_pages	server parameter		int	0	available
Utility-Related Parameters	backup_volume_max_size_bytes	server parameter		byte	0	
	communication_histogram	client parameter	O	bool	no	available
	compactdb_page_reclaim_only	server parameter		int	0	
	csql_history_num	client parameter	O	int	50	available
HA-Related Parameters	ha_mode	server parameter		string	off	
Other Parameters	access_ip_control	server parameter		bool	no	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	access_ip_control_file	server parameter		string		
	agg_hash_respect_order	client parameter	O	bool	yes	available
	auto_restart_server	server parameter	O	bool	yes	DBA only
	enable_string_compression	client/ server parameter		bool	yes	
	index_scan_in_oid_order	client parameter	O	bool	no	available
	index_unfill_factor	server parameter		float	0.05	
	stored_procedure	server parameter		bool	yes	
	stored_procedure_uds	server parameter		bool	yes	
	stored_procedure_port	server parameter		int	0	
		server parameter		string		

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	stored_procedure_vm_options					
	multi_range_optimization_limit	server parameter	O	int	100	DBA only
	optimizer_enable_merge_join	client parameter	O	bool	no	available
	use_stat_estimation	server parameter		bool	no	
	server	server parameter		string		
	service	server parameter		string		
	session_state_timeout	server parameter		sec	21,600	
	sort_limit_max_count	client parameter	O	int	1,000	available
	sql_trace_slow	server parameter	O	msec	-1	DBA only

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	sql_trace_execution_plan	server parameter	O	bool	no	DBA only
	use_orderby_sort_limit	server parameter	O	bool	yes	DBA only
	vacuum_prefetch_log_mode	server parameter		int	1	DBA only
	vacuum_prefetch_log_buffer_size	server parameter		int	3200 * log_page_size	DBA only
	data_buffer_neighbor_flush_pages	server parameter		int	8	DBA only
	data_buffer_neighbor_flush_nondirty	server parameter		bool	no	DBA only
	tde_keys_file_path	server parameter		string	NULL	
	tde_default_algorithm	server parameter		string	AES	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	recovery_progress_logging_interval	server parameter		int	0	
	supplemental_log	client/ server parameter		int	0	
	regexp_engine	client/ server parameter		string	re2	available
	vacuum_ovfp_check_threshold	server parameter		int	1000	
	vacuum_ovfp_check_duration	server parameter		int	45000	
	deduplicate_key_level	client/ server parameter		int	-1	
	print_index_detail	client/ server parameter		bool	no	
	enable_memory_monitoring	client/ server parameter		bool	no	
				int	300	

Category	Parameter Name	Applied	Session	Type	Default Value	Dynamic Change
	flashback_timeout	client parameter				

- **log_page_size**: A log volume page size specified by **--log-page-size** option when you are creating database. Default: 16KB. log page related parameter's value is rounded off by page unit. For example, the value of checkpoint_every_size is divided by 16KB and its decimal point is dropped, then it is multiplied by 16KB.
- **db_page_size**: A DB volume page size specified by **--db-page-size** option when you are creating database. Default: 16KB. DB page related parameter's value is rounded off by page unit. For example, the value of data_buffer_size is divided by 16KB and its decimal point is dropped, then it is multiplied by 16KB.

Section by Parameter

Parameters specified in **cubrid.conf** have the following four sections:

- Used when the CUBRID service starts: [service] section
- Applied commonly to all databases: [common] section
- Applied individually to each database: [@<*database*>] section
- Used only when the cubrid utilities are run with stand-alone mode(--SA-mode): [standalone] section

Where <*database*> is the name of the database to which each parameter applies. If a parameter configured in [common] is the same as the one configured in [@<*database*>], the one configured in [@<*database*>] is applied.

```
.....
[common]
.....
sort_buffer_size=2M
.....
[standalone]

sort_buffer_size=256M
.....
```

Configuration defined in [standalone] is used only when cubrid utilities started with “cubrid” are run with stand-alone mode. For example, on the above configuration, if DB is started with --CS-mode(default)(cubrid databases start db_name), “sort_buffer_size=2M” is applied. However, if DB is stopped and “cubrid loaddb --SA-mode” is executed, “sort_buffer_size=256M” is applied. If you run “cubrid loaddb --SA-mode”, bigger size of sort buffer will be required during index creation; therefore, increasing sort buffer size will be better for the performance of “loaddb” execution.

Default Parameters

cubrid.conf, a default database configuration file created during the CUBRID installation, includes some default database server parameters that must be changed. You can change the value of a parameter that is not included as a default parameter by manually adding or editing one.

The following is the content of the **cubrid.conf** file.

```
# Copyright (C) 2008 Search Solution Corporation. All rights
reserved by Search Solution.
#
# $Id$
#
# cubrid.conf#

# For complete information on parameters, see the CUBRID
# Database Administration Guide chapter on System Parameters

# Service section - a section for 'cubrid service' command
[service]

# The list of processes to be started automatically by 'cubrid
service start' command
# Any combinations are available with server, broker and manager.
service=server,broker,manager

# The list of database servers in all by 'cubrid service start'
command.
# This property is effective only when the above 'service' property
contains 'server' keyword.
#server=demodb,testdb

# Common section - properties for all databases
# This section will be applied before other database specific
sections.
[common]

# Read the manual for detailed description of system parameters
# Manual > System Configuration > Database Server Configuration >
Default Parameters

# Size of data buffer are using K, M, G, T unit
```

```
data_buffer_size=512M

# Size of log buffer are using K, M, G, T unit
log_buffer_size=256M

# Size of sort buffer are using K, M, G, T unit
# The sort buffer should be allocated per thread.
# So, the max size of the sort buffer is sort_buffer_size *
max_clients.
sort_buffer_size=2M

# The maximum number of concurrent client connections the server
will accept.
# This value also means the total # of concurrent transactions.
max_clients=100

# TCP port id for the CUBRID programs (used by all clients).
cubrid_port_id=1523
```

If you want to set **data_buffer_size** as 128M and **max_clients** as 10 only on *testdb*, set as follows.

```
[service]

service=server,broker,manager

[common]

data_buffer_size=512M
log_buffer_size=256M
sort_buffer_size=2M
max_clients=100

# TCP port id for the CUBRID programs (used by all clients).
cubrid_port_id=1523

[@testdb]
data_buffer_size=128M
max_clients=10
```

Connection-Related Parameters

The following are parameters related to the database server. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
cubrid_port_id	int	1523	1	
check_peer_alive	string	both		
db_hosts	string	NULL		
max_clients	int	100	10	4000
tcp_keepalive	bool	yes		
use_user_hosts	bool	off		

cubrid_port_id

cubrid_port_id is a parameter to configure the port to be used by the master process. The default value is **1523**. If port 1523 is already being used on the server where CUBRID is installed or is blocked by a firewall, an error message is displayed indicating that the master server is not connected because the master process cannot run properly. If such a port conflict occurs, the administrator must change the value of **cubrid_port_id** considering the server environment.

check_peer_alive

check_peer_alive is a parameter to decide whether you execute the function checking that the client/server processes work well. The default is **both**.

The client processes connecting with a server process are the broker application server(cub_cas) process, the process copying replication logs(copylogdb), the process applying replication logs. (applylogdb), CSQL interpreter(csql), etc. The server process and the client process which connected with it wait each other's response. But if one of them cannot get the data for a long time(example: exceeding 5 sec), it will check or not if the other works well based on the configuration of check_peer_alive parameter. During this processes, if it is judged that the process doesn't work properly, it disconnect forcibly.

The values and the working methods are as follows.

- **both**: As the server process accesses to the client process by ECHO(7) port, it checks if the client process works well. The client process also does the same thing to the server process(The default value).
- **server_only**: Only the server process checks whether the client process works well.
- **client_only**: Only the client process checks whether the server process works well.
- **none**: None of the server and client processes check whether the other process works well.

Specially, if ECHO(7) port is blocked by the firewall configuration, each process can mistake that the other process was exited. Therefore, you should avoid this problem by setting this parameter's value as none.

db_hosts

db_hosts is a parameter to configure a list of the database server hosts to which clients can connect, and the connection order. The server host list consists of multiple server hostnames, and hostnames are separated by spaces or colons (:). Duplicate or non-existent names are ignored.

The following example shows the values of the **db_hosts** parameter. In this example, connections are attempted in the order of **host1** > **host2** > **host3**.

```
db_hosts="hosts1:hosts2:hosts3"
```

To connect to the server, the client first tries to connect to the specified server host referring to the database location file (**databases.txt**). If the connection fails, the client then tries to connect to the first one of the secondarily specified server hosts by referring to the value of the **db_hosts** parameter in the database configuration file (**cubrid.conf**).

max_clients

max_clients is a parameter to configure the maximum number of clients (usually broker application processes (CAS)) which allow concurrent connections to the database server. The **max_clients** parameter refers to the number of concurrent transactions per database server process. The default value is **100**.

To guarantee performance while increasing the number of concurrent users in CUBRID environment, you need to make the appropriate value of the **max_clients** (**cubrid.conf**) parameter and the MAX_NUM_APPL_SERVER (**cubrid_broker.conf**) parameter. That is, you are required to configure the number of concurrent

connections allowed by databases with the **max_clients** parameter. You should also configure the number of concurrent connections allowed by brokers with the **MAX_NUM_APPL_SERVER** parameter.

For example, in the **cubrid_broker.conf** file, two node of a broker where the **MAX_NUM_APPL_SERVER** value of [%query_editor] is 50 and the **MAX_NUM_APPL_SERVER** value of [%BROKER1] is 50 is trying to connect one database server, the concurrent connections (**max_clients** value) allowed by the database server can be configured as follows:

- (the maximum number of 50 by each node of a broker) * (two node of a broker) + (10 spare for database server connections of internal CUBRID process such as database server connection of CSQL Interpreter or HA log replication process) = 110

Especially, in HA environment, the value must be greater than the sum specified in **MAX_NUM_APPL_SERVER** of every broker node which connects to the same database.

Note that the memory usage is affected by the value specified in **max_clients**. That is, if the number of value is high, the memory usage will increase regardless of whether or not the clients actually access the database.

Note

In Linux system, max_clients parameter is related to “ulimit -n” command, which specifies the maximum number of file descriptors which a process can use. File descriptor includes not only a file, but also a network socket. Therefore, the number of “ulimit -n” should be greater than the number of max_clients.

tcp_keepalive

tcp_keepalive is a parameter which specifies if you apply SO_KEEPALIVE option to TCP network protocol or not. The default is **yes**. If this value is **no**, DB server-side connection can be disconnected when transaction logs are not copied for a long time in the firewall environment between master and slave.

use_user_hosts

use_user_hosts is a system parameter that is used to select the host look up between **hostname** and **IP address**, required by the CUBRID service from the services below. The default value is **OFF**.

- The host/IP address lookup library that **OS** provides. (**glibc**, Linux)
- **The host/IP address lookup library** that **CUBRID** provides.

use_user_hosts=off (default)

- Looks up between IP address and hostname using the system library.
- In general, provides /etc/hosts host lookup, DNS Query commonly

use_user_hosts=on

- Looks up between IP address and hostname using CUBRID host lookup library.
- Uses **$CUBRID/conf/cubrid_hosts.conf** file to look up between IP address and hostname.
- executes regardless of the read permission of /etc/hosts, /etc/nsswitch.conf file.

⚠ Warning

If **use_user_hosts** parameter is changed during service operation, CUBRID service cannot terminate normally. So, the user must change the parameter after CUBRID service terminates.

- The format of **$CUBRID/conf/cubrid_hosts.conf** is same as **/etc/hosts** but there are some restrictions as follows.
  - Allow **IPv4** format address only. (Not allow **IPv6** format address)
  - Not allow **alias**.

```
172.31.0.1 host1 alias1 alias2
```

  - If there are multiple IP addresses, the top of information ,{ip, hostname}, is used except others.

```
172.31.0.1 host1
172.31.0.1 alias1
172.31.0.1 alias2
```

  - If you use the same hostname for more than one IP, only the topmost IP will be applied.

```
172.31.0.1 host1
178.31.0.2 host1
```

- The following is an example of $CUBRID/conf/cubrid_hosts.conf.

```
#
# hosts file for CUBRID user specific host service
#
127.0.0.1       localhost
172.31.0.1      node1
192.168.0.31    node4.kr          # Seoul
192.168.2.31    node5.gov.or.kr   # Daejeon
```

⚠ Warning

You must change $CUBRID/conf/cubrid_hosts.conf after terminating all CUBRID processes, and **the changes will be applied after restarting.** In addition, you must write including **localhost** and **'hostname'** (The output of hostname command by among Linux commands) in the cubrid_hosts.conf.

⚠ Warning

The hostname adheres the following format in the $CUBRID/conf/cubrid_hosts.conf (The Linux hostname naming rule).

- Only English letters, numbers (0 to 9), hyphen ('-'), and dot ("") characters can be used for the hostname.
- The first character of the hostname must be an English letter.
- The last character of the hostname must be an English letter and a number.
- FQDN (Fully Qualified Domain Name) format hostname can be used (Example: www.cubrid.com).

Allow the following hostname.

```
cubrid-dev1
CUB2.dev
```

Memory-Related Parameters

The following are parameters related to the memory used by the database server or client. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
data_buffer_size	byte	32,768 * db_page_size	1,024 * db_page_size	2G(32bit), INT_MAX * db_page_size (64bit)
index_scan_oid_buffer_size	byte	4 * db_page_size	0.05 * db_page_size	16 * db_page_size
max_agg_hash_size	byte	2,097,152(2M)	32,768(32K)	134,217,728(128MB)
max_hash_list_scan_size	byte	8,388,608(8M)	0	128MB
max_subquery_cache_size	byte	2,097,152(2M)	0	16,777,216(16M)
sort_buffer_size	byte	128 * db_page_size	1 * db_page_size	2G(32bit), INT_MAX * db_page_size (64bit)
temp_file_memory_size_in_pages	int	4	0	20
thread_stacksize	byte	1,048,576	65,536	

data_buffer_size

data_buffer_size is a parameter to configure the size of data buffer to be cached in the memory by the database server. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If you omit the unit, bytes will be applied. The default value is 32,768 * db_page_size (**512M** when db_page_size is 16K), and the minimum value is 1,024 * db_page_size (**16M** when db_page_size is 16K). The maximum value in 64-bit CUBRID is INT_MAX * db_page_size. Note that the maximum value in 32-bit CUBRID is **2G**.

The greater the value of the **data_buffer_size** parameter, the more data pages to be cached in the buffer, thus providing the advantage of decreased disk I/O cost. However, if this parameter is too large, the buffer pool can be swapped out by the operating system because the system memory is excessively occupied. It is recommended to configure the **data_buffer_size** parameter in a way the required memory size is less than two-thirds of the system memory size.

- Required memory size = data buffer size (**data_buffer_size**)

index_scan_oid_buffer_size

index_scan_oid_buffer_size is a parameter to configure the size of buffer where the OID list is to be temporarily stored during the index scan. You can set unit K, which stands for KB (kilobytes). If you omit the unit, bytes will be applied. The default value is 4 * db_page_size (**64K** when db_page_size is 16K), and the minimum value is 0.05 * db_page_size (about **1K** when db_page_size is 16K).

The size of the OID buffer tends to vary in proportion to the value of the **index_scan_oid_buffer_size** parameter and the page size set when the database was created. In addition, the bigger the size of such OID buffer, the more the index scan cost. You can set the value of the **index_scan_oid_buffer_size** by considering these factors.

max_agg_hash_size

max_agg_hash_size is a parameter to configure the maximum memory per transaction allocated for hashing the tuple groups in a query containing aggregation. The default is **2,097,152**(2M), the minimum size is 32,768(32K), and the maximum size is 134,217,728(128MB).

If NO_HASH_AGGREGATE hint is specified, hash aggregate evaluation will not be used. As a reference, see agg_hash_respect_order.

max_hash_list_scan_size

max_hash_list_scan_size is a parameter to configure the maximum memory per transaction allocated for building hash table in a query containing subquerys. The default is 8MB, the minimum size is 0, and the maximum size is 128MB.

If this parameter is set to 0 or If NO_HASH_LIST_SCAN hint is specified, hash list scan will not be used.

max_subquery_cache_size

max_subquery_cache_size is a parameter used to set the size of the subquery cache (correlated). You can set a unit as B, K or M, which stand for bytes, kilobytes (KB), and megabytes (MB), respectively. If you omit the unit, bytes will be applied. The default value is **2,097,152** (2M) bytes, the minimum value is **0**, and the maximum value is **16,777,216** (16M) bytes. The subquery cache is allocated for the number of subqueries in a query and is deallocated when the main query is completed.

If max_subquery_cache_size is set to 0, the NO_SUBQUERY_CACHE hint is specified, or there is insufficient memory space, the subquery cache optimization is not enabled.

sort_buffer_size

sort_buffer_size is a parameter to configure the size of buffer to be used when a query is processing sorting. The server assigns one sort buffer for each client's sorting-request, and releases the assigned buffer memory when sorting is complete. A sorting query includes not only SELECT sorting query, but also index-creating query.

You can set a unit as B, K, M, G or T, which stand for bytes, kilobytes (KB), megabytes (MB), gigabytes (GB), and terabytes (TB) respectively. If you omit the unit, bytes will be applied. The default value is 128 * db_page_size (**2M** when db_page_size is 16K), and the minimum value is 1 * db_page_size (**16K** when db_page_size is 16K).

temp_file_memory_size_in_pages

temp_file_memory_size_in_pages is a parameter to configure the number of buffer pages to cache temporary result of a query. The default value is **4** and the maximum value is 20.

- Required memory size = the number of temporary memory buffer pages (**temp_file_memory_size_in_pages** * **page size**)
- The number of temporary memory buffer pages = the value of the **temp_file_memory_size_in_pages** parameter
- Page size = the value of the page size specified by the **-s** option of the **cubrid createdb** utility during the database creation

The spaces to store the temporary result are as follows.

- Cache buffer to store the temporary result (acquired by **temp_file_memory_size_in_pages** parameter)
- Permanent volumes with the purpose of storing temporary data.

・Temporary volumes

If the previous space is exhausted, then the next space is used as the following order: Cache buffer for storing temporary result -> Permanent volumes -> Temporary volumes.

thread_stacksize

thread_stacksize is a parameter to configure the stack size of a thread. The default value is **1048576** bytes. The value of the **thread_stacksize** parameter must not exceed the stack size allowed by the operating system.

Disk-Related Parameters

The following are disk-related parameters for defining database volumes and storing files. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
db_volume_size	byte	512M	0	20G
dont_reuse_heap_file	bool	no		
log_volume_size	byte	512M	20M	4G
temp_file_max_size_in_pages	int	-1		
temp_volume_path	string	NULL		
unfill_factor	float	0.1	0.0	0.3
volume_extension_path	string	NULL		

Parameter Name	Type	Default	Min	Max
double_write_buffer_size	byte	2,097,152 (2M)	0	33,554,432 (32M)
data_file_os_advise	int	0	0	6

db_volume_size

db_volume_size is a parameter to configure the following values. You can set a unit as B, K, M, G or T, which stand for bytes, kilobytes (KB), megabytes (MB), gigabytes (GB), and terabytes (TB) respectively. If you omit the unit, bytes will be applied. The default value is **512M**.

- The default database volume size when **cubrid createdb** and **cubrid addvoldb** utility is used without **--db-volume-size** option.
- The default size of volume that is maximum size of an automatically added volume when the database is full. (The auto-added volume is incrementally increased by the needed amount, and another volume is created when the volume exceeds db_volume_size.)

Note

The actual volume size will always be rounded up to a multiple of the size of 64 sectors. Sector size depends on page size, therefore 64 sectors size is 16M, 32M or 64M for page size 4k, 8k or 16k respectively.

dont_reuse_heap_file

dont_reuse_heap_file is a parameter to configure whether or not heap files, which are deleted when deleting the table (**DROP TABLE**), are to be reused when creating a new table (**CREATE TABLE**). If this parameter is set to no, the deleted heap files can be reused; if it is set to yes, the deleted heap files are not used when creating a new table. The default value is **no**.

log_volume_size

log_volume_size is a parameter to configure the default size of log volume file when the **cubrid createdb** utility is used without **--log-volume-size** option. You can set a unit as B, K, M, G or T, which stand for bytes, kilobytes (KB), megabytes (MB), gigabytes (GB) and terabytes (TB) respectively. If you omit the unit, bytes will be applied. The default value is **512M**.

temp_file_max_size_in_pages

temp_file_max_size_in_pages is a parameter to configure the maximum number of pages to which temporary volumes can be extended. By default, this value is **-1**, which means that temporary volumes can occupy an unlimited disk space. A positive value will set a limit to these values (exceeding it may show an error and cancel some big queries).

If the parameter is configured to **0**, temporary volumes are not created automatically; the administrator must create permanent volumes with the purpose of storing temporary data by using the **cubrid addvoldb** utility.

For more details see Temporary Volume

temp_volume_path

temp_volume_path is a parameter to configure the directory in which to create temporary volumes used for the execution of complex queries or sorting. The default value is the volume location configured during the database creation.

unfill_factor

unfill_factor is a parameter to configure the rate of disk space to be allocated in a heap page for data updates. The default value is **0.1**. That is, the rate of free space is configured to 10%. In principle, data in the table is inserted in physical order. However, if the size of the data increases due to updates and there is not enough space for storage in the given page, performance may degrade because updated data must be relocated to another page. To prevent such a problem, you can configure the rate of space for a heap page by using the **unfill_factor** parameter. The allowable maximum value is 0.3 (30%). In a database where data updates rarely occur, you can configure this parameter to 0.0 so that space will not be allocated in a heap page for data updates. If the value of the **unfill_factor** parameter is negative or greater than the maximum value, the default value (**0.1**) is used.

volume_extension_path

volume_extension_path is a parameter to configure the directory where automatically extended volumes are to be created. The default value is the volume location configured during the database creation.

double_write_buffer_size

double_write_buffer_size is a parameter to configure the memory and disk size of double writer buffer. Double write buffer protection against partial I/O writes can be disabled by setting this size to zero. By default, it is enabled and its size is 2M. The value operates in bytes, and must be set within the range of 2,097,152 bytes (minimum) to 33,554,432 bytes (maximum).

data_file_os_advise

data_file_os_advise is a UNIX-only parameter that may be used to boost I/O performance. The parameter value is converted into a *posix_fadvise()* flag (for details about the flags see here).

Parameter Value	posix_fadvise flag
0	0
1	POSIX_FADV_NORMAL
2	POSIX_FADV_SEQUENTIAL
3	POSIX_FADV_RANDOM
4	POSIX_FADV_NOREUSE
5	POSIX_FADV_WILLNEED
6	POSIX_FADV_DONTNEED

Warning

Make sure posix_fadvise flags and how data is accessed are perfectly understood. The parameter can help improve performance but it can also degrade it if misused. In most scenarios it is best to use the default value.

Error Message-Related Parameters

The following are parameters related to processing error messages recorded by CUBRID. The type and value range for each parameter are as follows:

Parameter Name	Type	Default
call_stack_dump_activation_list	string	DEFAULT
call_stack_dump_deactivation_list	string	NULL
call_stack_dump_on_error	bool	no
error_log	string	cub_client.err, cub_server.err
error_log_level	string	NOTIFICATION
error_log_warning	bool	no
error_log_size	int	512M

call_stack_dump_activation_list

call_stack_dump_activation_list is a parameter to configure a certain error number for which a call stack is to be dumped to a server error log (located in $CUBRID/log/server directory) as an exception even when you configure that a call stack will not be dumped for any errors. Therefore, the **call_stack_dump_activation_list** parameter is effective only when **call_stack_dump_on_error=no**.

If this value is not configured, the default value is “DEFAULT” keyword. This keyword includes below errors. “DEFAULT” keyword can be used together with other error numbers.

Error Number	Error Message
-2	Internal system failure: no more specific information is available.
-7	Trying to format disk volume xxx with an incorrect value xxx for number of pages.
-13	An I/O error occurred while reading page xxx of volume xxx.
-14	An I/O error occurred while writing page xxx of volume xxx.
-17	Internal error: fetching deallocated pageid xxx of volume xxx.
-19	Internal error: pageptr = xxx of page xxx of volume xxx is not fixed.
-21	Internal error: unknown sector xxx of volume xxx.
-22	Internal error: unknown page xxx of volume xxx.
-45	Slot xxx on page xxx of volume xxx is allocated to an anchored record. A new record cannot be inserted here.
-46	Internal error: slot xxx on page xxx of volume xxx is not allocated.

Error Number	Error Message
-48	Accessing deleted object xxx|xxx|xxx.
-50	Internal error: relocation record of object xxx|xxx|xxx may be corrupted.
-51	Internal error: object xxx|xxx|xxx may be corrupted.
-52	Internal error: object overflow address xxx|xxx|xxx may be corrupted.
-76	Your transaction (index xxx, xxx@xxx|xxx) timed out waiting on xxx on page xxx|xxx. You are waiting for user(s) xxx to release the page lock.
-78	Internal error: an I/O error occurred while reading logical log page xxx (physical page xxx) of xxx.
-79	Internal error: an I/O error occurred while writing logical log page xxx (physical page xxx) of xxx.
-81	Internal error: logical log page xxx may be corrupted.
-90	Redo logging is always a page level logging operation. A data page pointer must be given as part of the address.
-96	Media recovery may be needed on volume xxx.

Error Number	Error Message
-97	Internal error: unable to find log page xxx in log archives.
-313	Object buffer underflow while reading.
-314	Object buffer overflow while writing.
-407	Unknown key xxx referenced in B+tree index {vfid: (xxx, xxx), rt_pgid: xxx, key_type: xxx}.
-414	Unknown class identifier: xxx|xxx|xxx.
-415	Invalid class identifier: xxx|xxx|xxx.
-416	Unknown representation identifier: xxx.
-417	Invalid representation identifier: xxx.
-583	Trying to allocate an invalid number (xxx) of pages.
-603	Internal Error: Sector/page table of file VFID xxx|xxx seems corrupted.
-836	LATCH ON PAGE(xxx|xxx) TIMEDOUT
-859	LATCH ON PAGE(xxx|xxx) ABORTED
-890	Partition failed.

Error Number	Error Message
-891	Appropriate partition does not exist.
-976	Internal error: Table size overflow (allocated size: xxx, accessed size: xxx) at file table page xxx|xxx(volume xxx)
-1040	HA generic: xxx.
-1075	Descending index scan aborted because of lower priority on B+tree with index identifier: (vfid = (xxx, xxx), rt_pgid: xxx).

The following example shows how to make error numbers only -115 and -116, perform call-stack dump.

```
call_stack_dump_on_error= no
call_stack_dump_activation_list=-115,-116
```

The following example shows how to make error numbers -115, -116 and “DEFAULT” error numbers, perform call-stack dump.

```
call_stack_dump_on_error= no
call_stack_dump_activation_list=-115,-116, DEFAULT
```

call_stack_dump_deactivation_list

call_stack_dump_deactivation_list is a parameter to configure a certain error number for which a call stack is not to be dumped when you configure that a call stack will be dumped for any errors. Therefore, the **call_stack_dump_deactivation_list** parameter is effective only when **call_stack_dump_on_error** is set to **yes**.

The following example shows how to configure the parameter so that call stacks will be dumped for any errors, except the ones whose numbers are -115 and -116.

```
call_stack_dump_on_error= yes
call_stack_dump_deactivation_list=-115,-116
```

call_stack_dump_on_error

call_stack_dump_on_error is a parameter to configure whether or not to dump a call stack when an error occurs in the database server. If this parameter is set to **no**, a call stack for any errors is not dumped. If it is set to **yes**, a call stack for all errors is dumped. The default value is **no**.

error_log

error_log is a server/client parameter to configure the name of the error log file when an error occurs in the database server. The name of the error log file must be in the form of *<database_name>_<date>_<time>.err*. However, the naming rule of the error log file does not apply to errors for which the system cannot find the database server information. Therefore, error logs are recorded in the **cubrid.err** file. The error log file **cubrid.err** is stored in the **$CUBRID/log/server** directory.

error_log_level

error_log_level is a server parameter to configure an error message to be stored based on severity. There are five different levels which range from **WARNING** (lowest level), to **FATAL** (highest level). The inclusion relation in messages is **FATAL** < **ERROR** < **SYNTAX** < **NOTIFICATION** < **WARNING**. The default is **NOTIFICATION**. If severity of error is **NOTIFICATION**, error messages with **NOTIFICATION**, **SYNTAX**, **ERROR** and **FATAL** levels are written to the log file.

error_log_warning

error_log_warning is a parameter to configure whether or not error messages with a severity level of **WARNING** are to be displayed. Its default value is **no**. For this reason, you must set **error_log_warning** to **yes** to store **WARNING** messages to an error log file.

error_log_size

error_log_size is a parameter to configure the maximum size per an error log file. The default value is **512M**. If it reaches up to the specified size, the *<database_name>_<date>_<time>.err.bak* file is created.

Concurrency/Lock-Related Parameters

The following are parameters related to concurrency control and locks of the database server. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
deadlock_detection_interval_in_secs	float	1.0	0.1	
isolation_level	int	4	4	6
lock_escalation	int	100,000	5	
lock_timeout	msec	-1(inf)	0(no wait)	INT_MAX
rollback_on_lock_escalation	bool	no		

deadlock_detection_interval_in_secs

deadlock_detection_interval_in_secs is a parameter to configure the interval (in seconds) in which deadlocks are detected for stopped transactions. If a deadlock occurs, CUBRID resolves the problem by rolling back one of the transactions. The default value is 1 second and the minimum value is 0.1 second. This value is rounded up by 0.1 sec. unit. For example, if an input value is 0.12 seconds, the value is rounded up to 0.2 seconds. Note that deadlocks cannot be detected if the detection interval is too long.

isolation_level

isolation_level is a parameter to configure the isolation level of a transaction. The higher the isolation level, the less concurrency and the less interruption by other concurrent transactions. The **isolation_level** parameter can be configured to an integer value from 4 to 6, which represent isolation levels, or character strings. The default value is **READ COMMITTED**. For details about each isolation level and parameter values, see Transaction Isolation Level and the following table.

Isolation Level	isolation_level Parameter Value
SERIALIZABLE	“TRAN_SERIALIZABLE” or 6
REPEATABLE READ	“TRAN_REP_CLASS_REP_INSTANCE” or “TRAN_REP_READ” or 5
READ COMMITTED	“TRAN_REP_CLASS_COMMIT_INSTANCE” or “TRAN_READ_COMMITTED” or “TRAN_CURSOR_STABILITY” or 4

- **TRAN_SERIALIZABLE** : This isolation level ensures the highest level of consistency. For details, see SERIALIZABLE Isolation Level.
- **TRAN_REP_READ** : This isolation level prevents dirty reads, non-repeatable reads, and phantom reads due to snapshot isolation. For details, see REPEATABLE READ Isolation Level.
- **TRAN_READ_COMMITTED** : This isolation level can incur unrepeatable read. For details, see READ COMMITTED Isolation Level.

Note

9.3 or less version supports the below levels additionally. From 10.0, concurrency can be guaranteed more because MVCC method is applied when multiple concurrent transactions are processed; therefore, the below isolation levels are not used anymore.

- **TRAN_REP_CLASS_UNCOMMIT_INSTANCE** : This isolation level can incur dirty read.
- **TRAN_COMMIT_CLASS_COMMIT_INSTANCE** : This isolation level can incur unrepeatable read. It allows modification of table schema by current transactions while data is being retrieved.
- **TRAN_COMMIT_CLASS_UNCOMMIT_INSTANCE** : This isolation level can incur dirty read. It allows modification of table schema by current transactions while data is being retrieved.

lock_escalation

lock_escalation is a parameter to configure the maximum number of locks permitted before row level locking is extended to table level locking. The default value is **100,000**. If the value of the **lock_escalation** parameter is small, the overhead by memory lock management is small as well; however, the concurrency decreases. On the other hand, if the configured value is large, the overhead is large as well; however, the concurrency increases.

lock_timeout

lock_timeout is a client parameter to configure the lock waiting time. If the lock is not permitted within the specified time period, the given transaction is canceled, and an error message is returned. If the parameter is configured to **-1**, which is the default value, the waiting time is infinite until the lock is permitted. If it is configured to **0**, there is no waiting for locks.

You can set a unit as s, min or h, which stands for seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied, and it is rounded up to seconds. For example, 1ms will be 1s, and 1001ms will be 2s.

rollback_on_lock_escalation

It specifies rolling back the transaction or not when the lock escalation occurs. The default is **no**.

If this parameter is specified with **yes**, the error log is written without lock escalation on the lock-escalating time and this lock escalation request is failed with rolling back the transaction. If it is specified with **no**, the lock escalation is performed and the transaction is continued.

When the lock escalation occurs, record locks are transformed into a table lock and lock-releasing time can take long, so other transaction's access to the table can be impossible. However, if you specify the value of **lock_escalation** parameter(it specifies the number of record locks occurring the lock escalation) bigger, the system can overuse the memory resource.

Logging-Related Parameters

The following are parameters related to logs used for database backup and restore. The types and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
adaptive_flush_control	bool	yes		
background_archiving	bool	yes		
checkpoint_every_size	byte	10,000 * log_page_size	10 * log_page_size	log_page_size
checkpoint_interval	msec	6min	1min	35,791,394min
checkpoint_sleep_msecs	msec	1	0	
force_remove_log_archives	bool	yes		
log_buffer_size	byte	16k * log_page_size	128 * log_page_size	INT_MAX * log_page_size
log_max_archives	int	INT_MAX	0	INT_MAX
log_trace_flush_time	int	0	0	INT_MAX
max_flush_size_per_second	byte	10,000 * db_page_size	1 * db_page_size	INT_MAX * db_page_size
	sec	0	0	

Parameter Name	Type	Default	Min	Max
remove_log_archive_interval_in_secs				
sync_on_flush_size	byte	200 * db_page_size	1 * db_page_size	INT_MAX * db_page_size
ddl_audit_log	bool	no		
ddl_audit_log_size	byte	10M	10M	2G

adaptive_flush_control

adaptive_flush_control is a parameter used automatically to adjust the flush capacity at every 50 ms depending on the current status of the flushing operation. The default value is **yes**. That is, this capacity is increased if a large number of **INSERT** or **UPDATE** operations are concentrated at a certain point of time and the number of flushed pages reaches the **max_flush_size_per_second** parameter value; and is decreased otherwise. In the same way, you can distribute the I/O load by adjusting the flush capacity on a regular basis depending on the workload.

background_archiving

background_archiving is a parameter used to create temporary archive logs periodically at a specific time. It is useful when balancing disk I/O load which has been caused by archiving logs. The default is **yes**.

checkpoint_every_size

checkpoint_every_size is a parameter to configure checkpoint interval by log page. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If you omit the unit, bytes will be applied. The default value is **10,000** * log_page_size (**156.25M** when log_page_size is 16K).

You can distribute disk I/O overload at the checkpoint by specifying lower size in the **checkpoint_every_size** parameter, especially in the environment where **INSERT** / **UPDATE** are heavily loaded at a specific time.

Checkpoint is a job to record every modified page except for the temp page in data buffers to database volumes (disk) at a specific point. Checkpoint can shrink a restore time after the database failure because this makes the transaction logs which have been generated previously than the checkpoint time needless when the restore is processed. However, an efficient checkpoint interval should be properly considered because this job can occur a lot of disk I/O.

Note

There are three ways to run checkpoint in CUBRID. The following two ways are provided by the setting of cubrid.conf.

- **checkpoint_interval**: After the previous checkpoint is done, checkpoint is periodically processed at every time after the term of this parameter value.
- **checkpoint_every_size**: Checkpoint is periodically processed at every time after the transaction log size of this parameter value.

If one condition on the above parameters is satisfied, checkpoint is processed.

The following two ways are provided by a user's command.

- If you run ";checkpoint" command in the CSQL interpreter, which is run with a "DBA" user, checkpoint is processed.

As a reference, if you run backup command during checkpoint, backup command is blocked until checkpoint is ended.

checkpoint_interval

checkpoint_interval is a parameter to configure execution period of checkpoint. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied, and it is rounded up to seconds. For example, 1ms will be 1s, and 1001ms will be 2s. The default value is **6min** and the minimum value is 1min.

checkpoint_sleep_msecs

checkpoint_sleep_msecs is a parameter to let the job which flushes a buffer's data into a disk process slowly. The default is **1** (millisecond).

force_remove_log_archives

force_remove_log_archives is a parameter to configure whether to allow the deletion of the files other than the recent log archive files whose number is specified by **log_max_archives**. The default value is **yes**.

If the value is set to **yes**, the files will be deleted other than the recent log archive files for which the number is specified by **log_max_archives**.

If it is set to **no**, the log archive files will not be deleted. Exceptionally, if **ha_mode** is set to **on**, the files other than the log archive files required for the HA-related processes and the recent log archive files of which the number is specified by **log_max_archives** will be deleted.

For setting up the CUBRID HA environment, see Environment Configuration.

log_buffer_size

log_buffer_size is a parameter to configure the size of log buffer to be cached in the memory. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If you omit the unit, bytes will be applied. The default value is 16k * log_page_size (**256M** when log_page_size is 16K).

If the value of the **log_buffer_size** parameter is large, performance can be improved (due to the decrease in disk I/O) in an environment where transactions are long and numerous. Moreover, CUBRID Multiversion Concurrency Control system relies on log to access previous row versions and to vacuum invisible versions from database. It is recommended to configure an appropriate value considering the memory size and operations of the system where CUBRID is installed.

- Required memory size = the size of log buffer (**log_buffer_size**)

log_max_archives

log_max_archives is a parameter to configure the maximum number of archive log files. The minimum value is 0 and default value is **INT_MAX** (2,147,483,647). Its operations can differ depending on the configuration of **force_remove_log_archives**. For example, when **log_max_archives** is 3 and **force_remove_log_archives** is **yes** in the cubrid.conf file, the most recent three archive log files are recorded and when a fourth archiving log file is generated, the oldest archive log file is automatically deleted; the information about the deleted archive logs are recorded in the ***_lginf** file. When a user sets ha_mode=off and force_remove_log_archives=n, the system automatically assigns the value of INT_MAX (2,147,483,647) to log_max_archives instead of the value entered by the user.

However, if an active transaction still refers to an existing archive log file, the archive log file will not be deleted. That is, if a transaction starts at the point that the first archive log file is generated, and it is still active until the fifth archive log is generated, the first archive log file cannot be deleted.

Also, if the information of archive logs is not applied to database volumes, these are not deleted. (Archive logs after which a checkpoint has occurred keep the information of modified pages of a data buffer; therefore, they are required to restore a database.)

If you change the value of **log_max_archives** dynamically during database operation, changed value will be applied when a new log archive file is created. For example, if you change this value from 10 to 5, old 5 files will be deleted when a new log archive file is created.

For setting up the CUBRID HA environment, see Environment Configuration.

Note

In 2008 R4.3 or lower and in 9.1, **log_max_archives** was also used for specifying the maximum number of keeping replication log files in HA environment. From 2008 R4.4 and 9.2, ha_copy_log_max_archives of cubrid_ha.conf is in charge of this role.

log_trace_flush_time

When the log flushing time takes more time than the time you set in this parameter, this event is recorded in the log of the database server.

The example of written information is as below.

```
03/18/14 10:20:45.889 - LOG_FLUSH_THREAD_WAIT
  total flush count: 1 page(s)
  total flush time: 310 ms
  time waiting for log writer: 308 ms
  last log writer info
    client: DBA@cdbs037.cub|copylogdb(15312)
    time spent by log writer: 308 ms
```

- LOG_FLUSH_THREAD_WAIT: Event name
- total flush count: The number of flushed pages when the event occurs
- total flush time: Total spent time when the log is flushed

- time waiting for log writer: The time LFT(Log Flushing Thread) has been waiting for LWT(Log Writer Thread)
- last log writer info
  - DBA@cdbs037.cub|copylogdb(15312): copylogdb information related to LWT which let LFT wait <user@host_name|client_name(pid)>
  - time spent by log writer: Time spend by LWT measured in LWT; in general, this value is the same as the value of "time waiting for log writer")

max_flush_size_per_second

max_flush_size_per_second is a parameter to configure the maximum flush capacity when the flushing operation is performed from a buffer to a disk. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If you omit the unit, bytes will be applied. The default value is 10,000 * db_page_size (**156.25M** when db_page_size is 16K). That is, you can prevent concentration of I/O load at a certain point of time by configuring this parameter to control the maximum flush capacity per second.

If a large number of **INSERT** or **UPDATE** operations are concentrated at a certain point of time, and the flush capacity reaches the maximum capacity set by this parameter, only log pages are flushed to the disk, and data pages are no longer flushed. Therefore, you must set an appropriate value for this parameter considering the workload of the service environment.

remove_log_archive_interval_in_secs

Archive logs which exceed the specified number in the **log_max_archives** are removed when checkpoint occurs. By the way, many archive logs can be removed at once frequently after being piled up when jobs such as data migration or big data batch processing are performed. If files are removed at once like these, I/O overhead of database server is rapidly risen; therefore, we need to decrease this burden.

remove_log_archive_interval_in_secs parameter lets archive logs delete slowly to shrink this burden. The default is **0** (second). In the situations which jobs like big data batch processing occur frequently, it is recommended to set the deletion interval as a 60 seconds if the disk space is enough.

sync_on_flush_size

sync_on_flush_size is a parameter to configure the interval in pages between after data and log pages are flushed from buffer and before they are synchronized with FILE I/O of operating system. The default value is 200 * db_page_size (**3.125M** when db_page_size is 16K). That is, the CUBRID Server performs synchronization with the

FILE I/O of the operating system whenever 200 pages have been flushed. This is also a parameter related to I/O load.

ddl_audit_log

ddl_audit_log is a parameter to turn on/off DDL logging facility. The default value is no. If this value is set to yes, all DDL executed will be logged into the logfile. The path of log files is $CUBRID/log/ddl_audit, and refer to :doc:/admin/ddl_audit for each DDL AUDIT log filename and description of log files in detail.

ddl_audit_log_size

ddl_audit_log_size specifies the maximum size of the DDL AUDIT log file. If the ddl audit log file is larger than the specified size, that ddl audit log file is backed up with the name of .bak appended to the ddl audit log file, and new recording will be started with the file from the beginning of the file. You can set the size with a size unit as B, K, M, or G, which stand for bytes, kilobytes (KB), megabytes (MB), and gigabytes (GB) respectively. If you omit the size unit, bytes will be applied. The default is 10M, and it can be set up to 2G.

Transaction Processing-Related Parameters

The following are parameters for improving transaction commit performance. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
async_commit	bool	no		
group_commit_interval_in_msecs	msec	0	0	

async_commit

async_commit is a parameter used to activate the asynchronous commit functionality. If the parameter is set to **no**, which is the default value, the asynchronous commit is not performed; if it is set to **yes**, the asynchronous commit is executed. The asynchronous commit is a functionality that improves commit performance by completing the commit for the client before commit logs are flushed on the disk and having the log flush thread (LFT) perform log flushing in the background. Note that already committed transactions cannot be restored if a failure occurs on the database server before log flushing is performed.

group_commit_interval_in_msecs

group_commit_interval_in_msecs is a parameter to configure the interval (in milliseconds), at which the group commit is to be performed. If the parameter is configured to **0**, which is the default value, the group commit is not performed. The group commit is a functionality that improves commit performance by combining multiple commits that occurred in the specified time period into a group so that commit logs are flushed on the disk at once.

dblink_auto_commit

dblink_auto_commit is a setting that affects the commit behavior of queries executed through dblink during distributed transactions. The default value is **yes**.

- When this parameter is set to **yes**, queries executed via dblink are automatically committed immediately upon execution.
- Conversely, if it is set to **no**, the queries on the remote server connected through dblink are committed only when the entire transaction, including the dblink, is committed.

In distributed transactions, the system attempts to commit the dblink queries first. If any of these commits fail, the entire transaction is rolled back.

- If some dblink queries have already been committed and the overall transaction commit later fails, those prior commits cannot be rolled back.
- Savepoints are not supported in dblink transactions.

Statement/Type-Related Parameters

The following are parameters related to SQL statements and data types supported by CUBRID. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
add_column_update_hard_default	bool	no		
alter_table_change_type_strict	bool	yes		

Parameter Name	Type	Default	Min	Max
allow_truncated_string	bool	no		
ansi_quotes	bool	yes		
block_ddl_statement	bool	no		
block_nowhere_statement	bool	no		
compat_numeric_division_scale	bool	no		
create_table_reuseoid	bool	yes		
cte_max_recursions	int	2,000	2	1,000,000
default_week_format	int	0		
group_concat_max_len	byte	1,024	4	INT_MAX
intl_check_input_string	bool	no		
intl_collation	string			

Parameter Name	Type	Default	Min	Max
intl_date_lang	string			
intl_number_lang	string			
json_max_array_idx	int	65,536	1,024	1,048,576
max_query_per_tran	int	100	1	32,767
no_backslash_escapes	bool	yes		
only_full_group_by	bool	no		
oracle_compat_number_behavior	bool	no		
oracle_style_empty_string	bool	no		
pipes_as_concat	bool	yes		
plus_as_concat	bool	yes		
require_like_escape_character	bool	no		

Parameter Name	Type	Default	Min	Max
return_null_on_function_errors	bool	no		
stored_procedure_return_numeric_size	list	38,15	1,0	38,38
string_max_size_bytes	byte	1,048,576	64	33,554,432
unicode_input_normalization	bool	no		
unicode_output_normalization	bool	no		
update_use_attribute_references	bool	no		

add_column_update_hard_default

add_column_update_hard_default is a parameter to configure whether or not to provide the hard default value as the input value for a column when you add a new column to the **ALTER TABLE ... ADD COLUMN** clause.

When there is **NOT NULL** constraint and no **DEFAULT** constraint, if a value of this parameter is set to **yes**, the value of newly added column will be inserted as hard default value; if it is set to **no**, CUBRID returns an error. For the hard default for each type, see the CHANGE/MODIFY Clauses of the **ALTER TABLE** statement.

```
SET SYSTEM PARAMETERS 'add_column_update_hard_default=yes';

CREATE TABLE tbl (i int);
INSERT INTO tbl VALUES (1),(2);
ALTER TABLE tbl ADD COLUMN j INT NOT NULL;
```

```
SELECT * FROM tbl;
```

```
            i           j
=======================
            1           0
            2           0
```

```
SET SYSTEM PARAMETERS 'add_column_update_hard_default=no';

CREATE TABLE tbl (i INT);
INSERT INTO tbl VALUES (1),(2);
ALTER TABLE tbl ADD COLUMN j INT NOT NULL;
```

```
ERROR: Cannot add NOT NULL constraint for attribute "j":
there are existing NULL values for this attribute.
```

alter_table_change_type_strict

alter_table_change_type_strict is a parameter to configure whether to allow the conversion of column values according to the type change, and the default value is **yes**. If a value for this parameter is set to **no**, the value may be changed when you change the column types or when you add **NOT NULL** constraints; if it is set to **yes**, the value does not change. For details, see CHANGE Clause in the CHANGE/MODIFY Clauses.

allow_truncated_string

allow_truncated_string is a parameter to configure whether to allow the truncation of string values according to the string manipulation operations used in insert or update query, and the default value is **no**. If the value for this parameter is set to **no**, the string value is not allowed to be truncated when you do operation for any string related to insert or update query; however the string related to select query may be truncated regardless of this configuration. If it is set to **yes**, the string value may be truncated regardless of the type of (INSERT/UPDATE/SELECT) query.

ansi_quotes

ansi_quotes is a parameter used to enclose symbols and character string to handle identifiers. The default value is **yes**. If this parameter value is set to **yes**, double quotations are handled as identifier symbols and single quotations are handled as character string symbols. If it is set to **no**, both double and single quotations are handled as character string symbols.

block_ddl_statement

block_ddl_statement is a parameter used to limit the execution of DDL (Data Definition Language) statements by the client. If the parameter is set to **no**, the given client is allowed to execute DDL statements. If it is set to **yes**, the client is not permitted to execute DDL statements. If set to Yes, update statistics may also be restricted. The default value is **no**.

block_nowhere_statement

block_nowhere_statement is a parameter used to limit the execution of **UPDATE** / **DELETE** statements without a condition clause (**WHERE**) by the client. If the parameter is set to **no**, the given client is allowed to execute **UPDATE** / **DELETE** statements without a condition clause. If it is set to **yes**, the client is not permitted to execute **UPDATE** / **DELETE** statements without a condition clause. The default value is **no**.

compat_numeric_division_scale

compat_numeric_division_scale is a parameter to configure the scale to be displayed in the result (quotient) of a division operation. If the parameter is set to **no**, the scale of the quotient is 9, if it is set to **yes**, the scale is determined by that of the operand. The default value is **no**.

create_table_reuseoid

create_table_reuseoid is a parameter to specify whether to use the **REUSE_OID** or **DONT_REUSE_OID** option when creating a table without table option. If it is set to **yes**, the table is created with **REUSE_OID** option. The default value is **yes**.

For detail, see REUSE_OID and DONT_REUSE_OID .

cte_max_recursions

cte_max_recursions is a parameter to limit the maximum number of iterations when executing the recursive part of the CTE (Common Table Expressions) statement. This avoids infinite loop and potential issues produced by the size of temporary lists.

default_week_format

default_week_format is a parameter to configure default value for the *mode* attribute of the `WEEK()` function. The default value is **0**. For details, see `WEEK()` .

intl_check_input_string

intl_check_input_string is a parameter to determine whether or not to check that string entered is correctly corresponded to character set used. The default value is **no**. If this value is **no** and character set is UTF-8 and incorrect data is enter which violate

UTF-8 byte sequence, it can show abnormal behavior or database server and applications can be terminated abnormally. However, if it is guaranteed this problem does not happen, it has advantage in performance not to do it.

UTF-8 and EUC-KR can be checked; ISO-8859-1 is one-byte encoding so it does not have to be checked because every byte is valid.

group_concat_max_len

group_concat_max_len is a parameter used to limit the return value size of the `GROUP_CONCAT()` function. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If you omit the unit, bytes will be applied. The default value is **1,024**. The minimum value is 4 and the maximum value is INT_MAX (about 2G) bytes.

This function is affected by **string_max_size_bytes** parameter; if the value of **group_concat_max_len** is greater than the value **string_max_size_bytes** and the result size of **GROUP_CONCAT** exceeds the value of **string_max_size_bytes**, an error occurs.

intl_check_input_string

intl_check_input_string is a parameter to determine whether or not to check that string entered is correctly corresponded to character set used. The default value is **no**. If this value is no and character set is UTF-8 and incorrect data is enter which violate UTF-8 byte sequence, it can show abnormal behavior or database server and applications can be terminated abnormally. However, if it is guaranteed this problem does not happen, it has advantage in performance not to do it.

UTF-8 and EUC-KR can be checked; ISO-8859-1 is one-byte encoding so it does not have to be checked because every byte is valid.

intl_collation

intl_collation is a parameter which specifies a collation name about a specific application client. Specifying this parameter is the same as changing a collation of application client by using “SET NAMES” statement. Specifying a collation includes a charset.

The following two statements behave the same.

```
SET NAMES utf8;
SET SYSTEM PARAMETERS 'intl_collation=utf8_bin';
```

For an available value of **intl_collation**, see Collation.

intl_date_lang

intl_date_lang is a parameter used to input/output the values of **TIME**, **DATE**, **DATETIME**, and **TIMESTAMP**. If language name is omitted, it specifies a locale format of string of localized calendar (month, weekday, and AM/PM).

The values allowed are as follows: Note that to use all values, locale library should be configured except built-in locale. For configuring locale, see Locale Setting.

Language	Locale Name of Language
English	en_US
German	de_DE
Spanish	es_ES
French	fr_FR
Italian	it_IT
Japanese	ja_JP
Cambodian	km_KH
Korean	ko_KR
Turkish	tr_TR
Vietnamese	vi_VN
Chinese	zh_CN
Romanian	ro_RO

The function recognizing input string based on calendar format of specified language is as follows:

- `TO_DATE()`
- `TO_TIME()`
- `TO_DATETIME()`
- `TO_TIMESTAMP()`
- `STR_TO_DATE()`

The function outputting string based on calendar format of specified language is as follows:

- `TO_CHAR()`
- `DATE_FORMAT()`
- `TIME_FORMAT()`

intl_number_lang

intl_number_lang is a parameter used to specify locale applied when numeric format is assigned to input/output string in the function where a string is converted to number or number is converted to string. A delimiter and decimal symbol are used for numeric localization. In general, a comma and period are used; however, it can be changeable based on locale. For example, while number 1000.12 is written as 1,000.12 in most locale, it is written as 1.000,12 in tr_TR locale.

The function recognizing input string based on calendar format of specified language is as follows:

- `TO_NUMBER()`

The function outputting string based on calendar format of specified language is as follows:

- `FORMAT()`
- `TO_CHAR()`

json_max_array_idx

json_max_array_idx is a parameter used by JSON functions to set a limit on the size of arrays after changes.

If a very large index is fed to functions like **JSON_ARRAY_INSERT** by accident, the JSON document will occupy a lot of memory and disk space. Use this parameter to limit the risks.

The limit is not applied to arrays generated by parsing strings.

max_query_per_tran

max_query_per_tran is a parameter used to set the maximum number of queries (unclosed queries) that can be held per transaction.

The default setting is 100, the minimum value is 1, and the maximum value is 32,767. Be careful not to set this value too high, as it may increase memory usage and affect overall system performance.

no_backslash_escapes

no_backslash_escapes is a parameter to configure whether or not to use backslash (\) as an escape character, and the default value is **yes**. If a value for this parameter is set to **no**, backslash (\) will be used as an escape character; if it is set to **yes**, backslash (\) will be used as a normal character. For example, if this value is set to **no**, "\n" means a newline character. For details, see Escape Special Characters.

only_full_group_by

only_full_group_by is a parameter to configure whether or not to use extended syntax about using **GROUP BY** statement.

If this parameter value is set to **no**, an extended syntax is applied thus, a column that is not specified in the **GROUP BY** statement can be specified in the **SELECT** column list. If it is set to yes, a column that is only specified in the **GROUP BY** statement can be the **SELECT** column list.

The default value is **no**. Therefore, specify the **only_full_group_by** parameter value to **yes** to execute queries by SQL standards. Because the extended syntax is not applied in this case, an error below is displayed.

```
ERROR: Attributes exposed in aggregate queries must also
appear in the group by clause.
```

oracle_compat_number_behavior

oracle_compat_number_behavior is a parameter used to improve compatibility with other DBMS (Database Management Systems); for NUMERIC type, DOUBLE and FLOAT types, it does not display decimal point trailing 0, and in case of DOUBLE and FLOAT, it does not display the exponent. For example, if this parameter setting value is **no**,

the result of a query that searches the a_double table composed of DOUBLE types is displayed in the form of an exponent as shown below, but if the parameter setting value is **yes**, only decimal point is displayed.

```
csql> ;get oracle_compat_number_behavior

=== Get Param Input ===

oracle_compat_number_behavior=n

SELECT a FROM a_double;
```

```
                         a
============================
     1.234567890123457e+19
     1.234567890123457e-08
     1.230000000000000e-08
     1.000000000000000e+00
     1.200000000000000e+00
    -4.939030300000000e-03
    -1.293934894993939e+18
    -1.938943893939394e+16
```

```
csql> ;get oracle_compat_number_behavior

=== Get Param Input ===

oracle_compat_number_behavior=y

SELECT a FROM a_double;
```

```
                         a
============================
      12345678901234570000
  0.00000001234567890123457
             0.0000000123
                        1
                      1.2
            -0.0049390303
     -1293934894993939000
       -19389438939393940
```

Also, when **oracle_compat_number_behavior** is set to **yes**, the type of division operation for integer numbers becomes a real number type (that is, a NUMERIC type)

not an integer type. In the example below, if this parameter is **yes**, affected integer types are INT, SHORT and BIGINT.

```
csql> ;get oracle_compat_number_behavior

=== Get Param Input ===

oracle_compat_number_behavior=y

SELECT 1/2;
```

```
       1/2
==========
       0.5
```

Note

The following JDBC/CCI functions are affected by the oracle_compat_number_behavior setting:

JDBC

getString(int columnIndex), getString(String columnLabel)

- Data Types Affected: NUMERIC, DOUBLE, FLOAT

getObject(int columnIndex), getObject(String columnLabel)

- Data Types Affected: NUMERIC

CCI

cci_get_data(CCI_A_TYPE_STR as type), Example) cci_get_data(req, i, CCI_A_TYPE_STR, &data, &ind)

- Data Types Affected: NUMERIC, DOUBLE, FLOAT

oracle_style_empty_string

oracle_style_empty_string is a parameter used to improve compatibility with other DBMS (Database Management Systems); it specifies to process empty string and **NULL** as the same value. The default is **no**. If the **oracle_style_empty_string** parameter is set to **no**, the character string is processed as a valid string; if it is set to **yes**, according to each function, the empty string is processed as **NULL** or **NULL** is processed as the empty string.

Note

Other functions except below functions are not affected by **oracle_style_empty_string** parameter.

- Functions processing an empty string and NULL into NULL when **oracle_style_empty_string=yes**.
  - `ASCII()`
  - `CONCAT_WS()`
  - `ELT()`
  - `FIELD()`
  - `FIND_IN_SET()`
  - `FROM_BASE64()`
  - `INSERT()`
  - `INSTR()`
  - `LOWER()`
  - `LEFT()`
  - `LOCATE()`
  - `LPAD()`
  - `LTRIM()`
  - `MID()`
  - `POSITION()`
  - `REPEAT()`
  - `REVERSE()`
  - `RIGHT()`
  - `RPAD()`
  - `RTRIM()`

    - `SPACE()`
    - `STRCMP()`
    - `SUBSTR()`
    - `SUBSTRING()`
    - `SUBSTRING_INDEX()`
    - `TO_BASE64()`
    - `TRANSLATE()`
    - `TRIM()`
    - `UPPER()`
- Functions processing an empty string and NULL into an empty string when **oracle_style_empty_string=yes**.
    - `CONCAT()`
    - `REPLACE()`

> **Note**
>
> `REPLACE()` function has the different behavior in the previous versions of 10.0 when **oracle_style_empty_string=yes**.
>
> ```
> SELECT REPLACE ('abc', 'a', '');
> ```
>
> In the above query, the version of 10.0 or more return 'bc' because it processes the input of an empty string as an empty string; the previous version of 10.0 returns NULL because it processes the input of an empty string as NULL.

pipes_as_concat

pipes_as_concat is a parameter to configure how to handle a double pipe symbol. The default value is **yes**. If this parameter value is set to **yes**, a double pipe symbol is handled as a concatenation operator if **no**, it is handled as the **OR** operator.

plus_as_concat

plus_as_concat is a parameter to configure the plus (+) operator, and the default value is **yes**. If a value for this parameter is set to **yes**, the plus (+) operator will be interpreted as a concatenation operator; if it is set to **no**, the operator will be interpreted as a numeric operator.

```
-- plus_as_concat = yes
SELECT '1'+'1';
```

```
        '1'+'1'
======================
        '11'
```

```
SELECT '1'+'a';
```

```
        '1'+'a'
======================
        '1a'
```

```
-- plus_as_concat = no
SELECT '1'+'1';
```

```
               '1'+'1'
============================
 2.000000000000000e+000
```

```
SELECT '1'+'a';
```

```
ERROR: Cannot coerce 'a' to type double.
```

require_like_escape_character

require_like_escape_character is parameter to configure whether or not to use an ESCAPE character in the **LIKE** clause, and the default value is **no**. If a value for this parameter is set to **yes** and a value for **no_backslash_escapes** is set to **no**, backslash (\) will be used as an ESCAPE character in the strings of the LIKE clause, otherwise you should specify an ESCAPE character by using the **LIKE ... ESCAPE** clause. For details, see LIKE.

return_null_on_function_errors

return_null_on_function_errors is a parameter used to define actions when errors occur in some SQL functions, and the default value is **no**. If a value for this parameter is set to **yes**, **NULL** is returned; if it is set to **no**, an error is returned when the error occurs in functions, and the related message is displayed.

The following SQL functions are affected by this system parameter.

Date/Time functions

- `ADDDATE()`
- `ADDTIME()`
- `DATEDIFF()`
- `DAY()`
- `DAYOFMONTH()`
- `DAYOFWEEK()`
- `DAYOFYEAR()`
- `FROM_DAYS()`
- `FROM_UNIXTIME()`
- `HOUR()`
- `LAST_DAY()`
- `MAKEDATE()`
- `MAKETIME()`
- `MINUTE()`
- `MONTH()`
- `QUARTER()`
- `SEC_TO_TIME()`
- `SECOND()`
- `TIME()`
- `TIME_TO_SEC()`

- `TIMEDIFF()`
- `TO_DAYS()`
- `WEEK()`
- `WEEKDAY()`
- `YEAR()`

String functions

- `ASCII()`
- `BIN()`
- `BIT_LENGTH()`
- `CHR()`

Numeric functions

- `ABS()`
- `ACOS()`
- `ASIN()`
- `ATAN()`
- `ATAN2()`
- `CEIL()`
- `CONV()`
- `COS()`
- `COT()`
- `DEGREES()`
- `EXP()`
- `FLOOR()`
- `LN()`
- `LOG2()`
- `LOG10()`

- `MOD()`
- `POW()`
- `RADIANS()`
- `SIGN()`
- `SIN()`
- `SQRT()`
- `TAN()`
- `TRUNC()`
- `WIDTH_BUCKET()`

```
SET SYSTEM PARAMETERS 'return_null_on_function_errors=no';
SELECT YEAR('12:34:56');
```

```
ERROR: Conversion error in time format.
```

```
SET SYSTEM PARAMETERS 'return_null_on_function_errors=yes';
SELECT YEAR('12:34:56');
```

```
   year('12:34:56')
======================
   NULL
```

stored_procedure_return_numeric_size

stored_procedure_return_numeric_size is a parameter that sets the precision and scale of values returned as the NUMERIC type in a stored procedure. The setting is specified as a list of positive integers, with the precision and scale designated respectively. For example, to set the precision to 15 and the scale to 10, specify it as 15,10. The default value is 38,15, and the valid range is from 1 to 38 for precision and from 0 to 38 for scale. Additionally, the scale value cannot exceed the precision value.

string_max_size_bytes

string_max_size_bytes is a parameter to define the maximum byte allowable in string functions or operators. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If

you omit the unit, bytes will be applied. The default value is **1,048,576**(1M). The minimum value is 64 and the maximum value is 33,554,432(32M). If the maximum value is exceeded, REPEAT and SPACE functions return NULL, while other functions return an error.

The functions and operators affected by this parameter are as follows:

- `SPACE()`
- `CONCAT()`
- `CONCAT_WS()`
- '**+**': Operand of string
- `REPEAT()`
- `GROUP_CONCAT()`: This function is affected not only by **string_max_size_bytes** parameter, but also by **group_concat_max_len**.
- `INSERT()` function

unicode_input_normalization

unicode_input_normalization is a parameter to determine whether or not to input unicode stored in system level or not. The default value is **no**.

In general, unicode text can be stored in "fully composed" or "fully decomposed". When character 'Ä' has 00C4 (if it is encoded in UTF-8, it becomes 2 bytes of C3 84) which is only one code point. In "fully decomposed" mode, it has two code points/characters. It is 0041 (character "A") and 0308(COMBINING DIAERESIS). In case of UTF-8 encoding, it becomes 3 bytes of 41 CC 88.

CUBRID can work with fully composed unicode. For clients which have fully decomposed texts, configure the value of **unicode_input_normalization** to yes so that it can be converted to fully composed mode; and then it can be reverted to fully decomposed mode. For normalization of unicode encapsulation of CUBRID, compatibility equivalence is not applied. In general, normalization of unicode is not possible to revert after composition, CUBRID supports revert for characters as many as possible, and it applies normalization of unicode encapsulation. The characteristics of CUBRID normalization are as follows:

- In case of language specific, normalization does not depend on locale.

  If one or more locale can be used, this means every CAS/CSQL process, not CUBRID server. The **unicode_input_normalization** system parameter determines whether composition of input codes by normalization in system level. The

unicode_output_normalization system parameter determines whether composition of output codes by normalization in system level.

- Collation and normalization does not have direct relationship.

  Even though the value of **unicode_input_normalization** is no, the string of extensible collation (utf8_de_exp, utf8_jap_exp, utf8_km_exp) is properly sorted fully decomposed mode, it is not intended; it is side-effect of UCA(Unicode Collation Algorithm). The extensible collation is implemented only with fully composed texts.

- In CUBRID, composition and decomposition for normalization does not work separately.

  It is generally used when **unicode_input_normalization** and **unicode_output_normalization** are yes. In this case, codes entered from clients are stored in composed mode and output in decomposed mode.

If the application client sends the decomposed text data into CUBRID, let CUBRID deal with the composed code, by setting **unicode_input_normalization** as **yes**.

If the application client can deal with the decomposed text data only, let CUBRID always send the decomposed code, by setting **unicode_output_normalization** as **yes**.

If the application client knows both of input and output, leave the setting **unicode_input_normalization** and **unicode_output_normalization** as **no**.

For more details, see An Overview of Globalization.

unicode_output_normalization

unicode_output_normalization is a parameter to determine whether or not to output unicode stored in system level. The default value is **no**. For details, see the above **unicode_input_normalization** description.

update_use_attribute_references

update_use_attribute_references is a parameter whether a value X of a column to be updated in an **UPDATE** statement affects to another column's update which uses X. The default is **no**.

The below result of an **UPDATE** statement is dependent on the value of **update_use_attribute_references** parameter.

```
CREATE TABLE tbl(a INT, b INT);
INSERT INTO tbl values (10, NULL);
```

```
UPDATE tbl SET a=1, b=a;
```

If this parameter's value is **yes**, the updated value of the column "b" will be "1" which is affected by "a=1".

```
SELECT * FROM tbl;
```

```
1, 1
```

If this parameter's value is **no**, the updated value of the column "b" will be "NULL", which is affected by the value of "a" stored in the record, not by "a=1".

```
SELECT * FROM tbl;
```

```
1, NULL
```

Thread-Related Parameters

Thread management can be configured by threads parameters. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
thread_connection_pooling	bool	true		
thread_connection_timeout_seconds	int	300	-1	3600
thread_worker_pooling	bool	true		
	int		1	1024

Parameter Name	Type	Default	Min	Max
thread_core_count		# of system core		
thread_worker_timeout_seconds	int	300	-1	3600
loaddb_worker_count	bool	8	2	64

thread_connection_pooling

If **thread_connection_pooling** parameter is true, all threads used for client connection management are pooled on server boot.

thread_connection_timeout_seconds

thread_connection_timeout_seconds is a parameter that configures wait time before stopping for threads handling connection management. After closing a connection, the thread will wait the value of the parameter expressed in seconds to be assigned a new connection. If no connection is assigned and the wait time expires, the thread stops. Another thread may be started the next time a connection comes. If parameter value is **-1**, threads never stop. They sleep until they are given a new assignment.

thread_worker_pooling

If **thread_worker_pooling** parameter is true, all threads used for client requests execution are pooled on server boot.

thread_core_count

The number of groups of pooled threads is configured according to the **thread_core_count** parameter. The default value is set to the number of system cores. If the number of threads in a group does not reach 3 or more according to the parameter value, the system adjusts this value so that at least 3 threads belong to each group.

thread_worker_timeout_seconds

thread_worker_timeout_seconds is a parameter that configures wait time before stopping for threads handling client requests. After executing a request, the thread

will wait the value of the parameter expressed in seconds to be assigned a request. If no client request is assigned and the wait time expires, the thread stops. Another thread may be started the next time a client request comes. If parameter value is **-1**, threads never stop. They sleep until they are given a new assignment.

loaddb_worker_count

loaddb_worker_count is a parameter that configures the maximum number of threads that can be dedicated for **loaddb** sessions. If a single **loaddb** session runs, it may use all threads. If multiple **loaddb** sessions run concurrently, the total number of threads of all sessions cannot exceed the parameter's value.

Timezone Parameter

The following are the parameters related to timezone. The type and the value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
server_timezone	string	OS timezone		
timezone	string	server_timezone		
tz_leap_second_support	bool	no		

- **timezone**

  Specifies a timezone for a session. The default is a value of **server_timezone**. This value can be specified by a timezone offset (e.g. +01:00, +02) or a timezone region name (e.g. Asia/Seoul). This value can be changed during operating database.

- **server_timezone**

Specifies a timezone for a server. The default is a OS timezone. To apply the changed value, database should be restarted. The timezone of operating system is read depending on the operating system and information found in operating system configuration files:

- on Windows, the API tzset() function and tzname[0] variable are used to retrieve an Windows style timezone name. This name is translated into IANA/CUBRID style name using the CUBRID mapping data (the mapping file is %CUBRID% \timezones\tzdata\windowsZones.xml).
- on Linux, CUBRID attempts to read and parse the file "/etc/sysconfig/clock". If this file is not available, then the value of link "/etc/localtime" is read and used.
- on AIX, the value of "TZ" operating system environment variable is used.

On all operating systems, if the server_timezone is not specified, and the value for timezone from operating system cannot be read, then "Asia/Seoul" zone is used as server timezone.

- **tz_leap_second_support**

Specifies to support a leap second or not as yes or no. The default is **no**.

A leap second is a one-second adjustment that is occasionally applied to Coordinated Universal Time (UTC) in order to keep its time of day close to the mean solar time.

To apply the changed value, database should be restarted.

Query Plan Cache-Related Parameters

The following are parameters related to the query plan cache functionality. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
max_plan_cache_entries	int	1,000		
max_plan_cache_clones	int	1,000		
	int	360		

Parameter Name	Type	Default	Min	Max
xasl_cache_time_threshold_in_minutes				
max_filter_pred_cache_entries	int	1,000		

max_plan_cache_entries

max_plan_cache_entries is a parameter to configure the maximum number of query plans to be cached in the memory. If the **max_plan_cache_entries** parameter is configured to -1 or 0, generated query plans are not stored in the memory cache; if it is configured to an integer value equal to 0 or greater than 1, a specified number of query plans are cached in the memory.

The following example shows how to cache up to 1,000 queries.

```
max_plan_cache_entries=1000
```

max_plan_cache_clones

The plan cache stores the XASL for a query in a serializable form. When executing the query, the XASL is converted back into its deserialized form, populated with values, and then executed. The clone cache stores deserialized XASL to reuse it instead of discarding. The **max_plan_cache_clones** is a parameter that sets the maximum number of cloned cache entries a single plan can hold, and its default value is set to 1000. When a plan is deleted, its clones are also deleted.

xasl_cache_time_threshold_in_minutes

It is a parameter that determines the time threshold for deciding whether to recompile (clean-up) a cached plan. It is also used when searching for a candidate plan to remove when there is no space in the plan cache. This parameter can be set in minutes, with a default value of 360 minutes.

max_filter_pred_cache_entries

max_filter_pred_cache_entries is a parameter used to specify the maximum number of filtered index expressions. The filtered index expressions are stored with them complied and can be immediately used in server. If it is not stored in cache, the

process is required which filtered index expressions are fetched from database schema and interpreted.

Query Cache-Related Parameters

The following are the parameters related to the query cache functionality. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
max_query_cache_entries	int	0	0	INT_MAX
query_cache_size_in_pages	int	0	0	INT_MAX

If one of the parameters is set to 0 or negative value, the query cache is disabled regardless using the query hint **QUERY_CACHE**.

max_query_cache_entries

max_query_cache_entries is a parameter to configure the maximum number of query to be cached. If it is configured to an integer value equal to 0 or greater than 1, a specified number of queries are cached with the result.

The following example shows how to cache up to 500 queries.

```
max_query_cache_entries=500
```

query_cache_size_in_pages

query_cache_size_in_pages is a parameter to configure the maximum page of result to be cached. If it is configured to an integer value equal to 0 or greater than 1, specified pages in results are cached as temp files.

The following example shows how to cache up to 4,000 pages.

```
query_cache_size_in_pages=4000
```

Utility-Related Parameters

The following are parameters related to utilities used in CUBRID. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
backup_volume_max_size_bytes	byte	0	32K	
communication_histogram	bool	no		
compactdb_page_reclaim_only	int	0		
csql_history_num	int	50	1	200

backup_volume_max_size_bytes

backup_volume_max_size_bytes is a parameter to configure the size of the backup volume file created by the **cubrid backupdb** utility in byte unit. You can set a unit as B, K, M, G or T, which stands for bytes, kilobytes(KB), megabytes(MB), gigabytes(GB) or terabytes(TB) respectively. If you omit the unit, bytes will be applied. The default value is **0**, and the minimum value is 32K.

If the parameter is configured to **0**, which is the default value, the created backup volume is not partitioned; if it is configured to a value greater than 0, the backup volume is partitioned as much as it is specified size.

communication_histogram

communication_histogram is a parameter associated with Session Commands "**;.h**" of the CSQL Interpreter and the default value is **no**. For details, see Dumping CSQL execution statistics information.

compactdb_page_reclaim_only

compactdb_page_reclaim_only is a parameter to configure the **compactdb** utility, which compacts the storage of already deleted objects to reuse OIDs of the already assigned storage. Storage optimization with the **compactdb** utility can be divided into

three steps. The optimization steps can be selected through the **compactdb_page_reclaim_only** parameter. If the parameter is configured to **0**, which is the default value, step 1, 2 and 3 are all performed, so the storage is optimized in data, table and file units. If it is configured to 1, step 1 is skipped to have the storage optimized in table and file units. If it is configured to 2, steps 1 and 2 are skipped to have the storage optimized only in file units.

- Step 1: Optimizes the storage only in data unit.
- Step 2: Optimizes the storage in table unit.
- Step 3: Optimizes the storage in file (heap file) unit.

csql_history_num

csql_history_num is a parameter to configure the CSQL Interpreter and the number of SQL statements to be stored in the history of the CSQL Interpreter. The default value is **50**.

HA-Related Parameters

The following are HA-related parameters. The type and value range for each parameter are as follows:

Parameter Name	Type	Default Value
ha_mode	string	off

ha_mode

The **ha_mode** parameter is used to set CUBRID HA, and the default value is **off**.

- off : CUBRID HA is not used.
- on : CUBRID HA is used using the configured node as a node for failover.
- replica : CUBRID HA is used without using the configured node as a node for failover.

To use the CUBRID HA feature, you should set HA-related parameters in the **cubrid_ha.conf** file in addition to the **ha_mode** parameter. For details, see CUBRID HA.

Other Parameters

The following are other parameters. The type and value range for each parameter are as follows:

Parameter Name	Type	Default	Min	Max
access_ip_control	bool	no		
access_ip_control_file	string			
agg_hash_respect_order	bool	yes		
auto_restart_server	bool	yes		
enable_string_compression	bool	yes		
index_scan_in_oid_order	bool	no		
index_unfill_factor	float	0.05	0	0.5
stored_procedure	bool	yes		
stored_procedure_uds	bool	yes		
stored_procedure_port	int	0	0	65535
	string			

Parameter Name	Type	Default	Min	Max
stored_procedure_vm_options				
multi_range_optimization_limit	int	100	0	10,000
optimizer_enable_merge_join	bool	no		
use_stat_estimation	bool	no		
server	string			
service	string			
session_state_timeout	sec	21,600 (6 hours)	60(1 minute)	31,536,000 (1 year)
sort_limit_max_count	int	1000	0	INT_MAX
sql_trace_slow	msec	-1(inf)	0	86,400,000 (24 hours)
sql_trace_execution_plan	bool	no		
use_orderby_sort_limit	bool	yes		

Parameter Name	Type	Default	Min	Max
vacuum_prefetch_log_mode	int	1	0	1
vacuum_prefetch_log_buffer_size	int	50M	25M	INT_MAX
data_buffer_neighbor_flush_pages	int	8	0	32
data_buffer_neighbor_flush_nondirty	bool	no		
tde_keys_file_path	string	NULL		
tde_default_algorithm	string	AES		
recovery_progress_logging_interval	int	0 (off)	0	3600
supplemental_log	int	0 (off)	0	2
regexp_engine	string	re2		
vacuum_ovfp_check_threshold	int	1000	2	INT_MAX

Parameter Name	Type	Default	Min	Max
vacuum_ovfp_check_duration	int	45000	1	600000
deduplicate_key_level	int	-1	-1	14
print_index_detail	bool	no		
enable_memory_monitoring	bool	no		
flashback_timeout	int	300	0	3600

access_ip_control

access_ip_control is a parameter to configure whether to use feature limiting the IP addresses that allow server access. The default value is **no**. For details, see Limiting Database Server Access.

access_ip_control_file

access_ip_control_file is a parameter to configure the filename in which the list of IP addresses allowed by servers is stored. If **access_ip_control** value is set to **yes**, database server allows the list of IP addresses only stored in the file specified by this parameter. For details, see Limiting Database Server Access.

agg_hash_respect_order

agg_hash_respect_order is a parameter to configure whether the groups in an aggregate function will be returned ordered or not. The default is **yes**. As a reference, see max_agg_hash_size.

If all the groups (keys and accumulators) can fit into hash memory, then “agg_hash_respect_order=no” will skip sorting them before writing to output, so it is fair to assume that the order cannot be guaranteed in this case. However, when

overflows occur, then a sort step must be performed and you will get the results in-order even with “agg_hash_respect_order=no”.

auto_restart_server

auto_restart_server is a parameter used to configure whether to restart the process when it terminates abnormally due to fatal errors occurring in the database server process (cub_server). The default value is **yes**.

If this parameter is set to **yes**, the master process (cub_master) detects the abnormal termination of the server process and automatically restarts it. However, if the server process, which was restarted by this functionality, abnormally terminates again within 120 seconds, the system will not attempt another restart to prevent potential issues caused by repeated abnormal terminations. Additionally, server processes that have been stopped through a normal process (**cubrid server stop** command) will not be automatically restarted.

This parameter only operates in Linux environments and affects server processes that are not configured for HA. It will not function properly if the master process is not running.

enable_string_compression

enable_string_compression is a parameter to configure whether string compression should be used when storing variable string type value into heap, index or list. If **enable_string_compression** value is set to **yes**, and the string is at least 255 bytes in size and the compressed string requires less size than original string, then the string is stored in compressed form.

index_scan_in_oid_order

index_scan_in_oid_order is a parameter to configure the result data to be retrieved in OID order after the index scan. If the parameter is set to **no**, which is the default value, results are retrieved in data order; if it is set to **yes**, they are retrieved in OID order.

index_unfill_factor

If there is no free space because index pages are full when the **INSERT** or **UPDATE** operation is executed after the first index is created, the split of index page nodes occurs. This substantially affects the performance by increasing the operation time. **index_unfill_factor** is a parameter to configure the percent of free space defined for each index page node when an index is created. The **index_unfill_factor** value is applied only when an index is created for the first time. The percent of free space defined for the page is not maintained dynamically. Its value ranges between 0 and 0.5. The default value is **0.05**.

If an index is created without any free space for the index page node (**index_unfill_factor** is set to 0), the split of index page nodes occurs every time an additional insertion is made. This may degrade the performance.

If the value of **index_unfill_factor** is large, a large amount of free space is available when an index is created. Therefore, better performance can be obtained because the split of index nodes does not occur for a relatively long period of time until the free space for the nodes is filled after the first index is created.

If this value is small, the amount of free space for the nodes is small when an index is created. Therefore, it is likely that the index nodes are spilt by **INSERT** or **UPDATE** because free space for the index nodes is filled in a short period of time.

stored_procedure

stored_procedure is a parameter to enable the use of stored procedures by running the cub_pl process. The default value is **yes**. If set to **no**, the cub_pl process will not run, and stored procedures cannot be used.

stored_procedure_uds

stored_procedure_uds is a parameter to use a Unix domain socket connection between the cub_pl process and the cub_server process when calling stored procedures. The default value is **yes**. If set to **no** or on Windows, the connection operates using a TCP socket regardless of the parameter value.

Note

For information on the **CUBRID_TMP** environment variable that specifies the Unix domain socket file path for the cub_pl process, see Configuring Environment Variables.

stored_procedure_port

stored_procedure_port is a parameter to set the TCP port number for calling stored procedures on the database server. This value must be less than 65,536. The default value is **0**, which means the port number is automatically assigned from an ephemeral port range. This parameter is only applicable when the **stored_procedure** parameter is set to **yes**. Note that configuring this parameter in the [common] section of cubrid.conf will result in an error.

```
.....
[common]
.....
```

```
# an error occurs. remove the following line.
stored_procedure_port=4333
.....
[@testdb]
.....
# the parameter is configured successfully for testdb
stored_procedure_port=4334
.....
```

stored_procedure_vm_options

stored_procedure_vm_options is a parameter to configure the virtual machine settings of Procedural Language Server where stored procedures are executed. Each option string should be separated by spaces. The default is an empty string. If the parameter is set in the [@<database>] section, the options set in the [common] section do not apply to that database.

Note

The cub_pl process internally uses the Java Virtual Machine (JVM), so JVM options can be specified. There are standard, non-standard, and advanced JVM options. Non-standard and advanced options are not guaranteed to be supported by all JVM implementations.

```
.....
[common]
.....
stored_procedure_vm_options="-Xms1024m -Xmx1024m -
XX:PermSize=512m -XX:MaxPermSize=512m"
.....
[@testdb]
.....

# Note that -XX:PermSize=512m and -XX:MaxPermSize=512m will
not be applied for testdb, even though they are specified
in the [common] section.
stored_procedure_vm_options="-Xms2048m -Xmx2048m"
.....
```

multi_range_optimization_limit

If the number of rows specified by the **LIMIT** clause in the query, which has multiple ranges (col IN (?, ?, ... ,?)) and is available to use an index, is within the number

specified in the **multi_range_optimization_limit** parameter, the optimization for the way of index sorting will be performed. The default value is **100**.

For example, if a value for this parameter is set to 50, LIMIT 10 means that it is within the value specified by this parameter, so that the values that meet the conditions will be sorted to produce the result. If LIMIT is 60, it means that it exceeds the parameter configuration value, so that it gets and sorts out all values that meet the conditions.

Depending on the setting value, the differences are made between collecting the result with on-the-fly sorting of the intermediate values and sorting the result values after collecting them, and the bigger value could make more unfavorable performance.

optimizer_enable_merge_join

optimizer_enable_merge_join is a parameter to specify whether to include sort merge join plan as a candidate of query plans or not. The default is **no**. Regarding sort merge join, see Using SQL Hint.

use_stat_estimation

use_stat_estimation is a parameter to specify whether to use the estimated information in calculating statistics or not. The default is no. The estimated information generated by the heap manager while processing DML is associated with the number of added objects. it is relatively accurate for the number of total objects, NOT for the number of distinct values.

server

server is a parameter used to register the name of database server process which will run automatically when CUBRID server starts.

service

service is a parameter to configure process that starts automatically when the CUBRID service starts. There are four types of processes: **server**, **broker**, **manager**, and **heartbeat**. Three processes are usually registered as in **service=server,broker,manager**.

- If the parameter is set to **server**, the database process specified by the **@server** parameter gets started.
- If the parameter is set to **broker**, the broker process gets started.
- If the parameter is set to **manager**, the manager process gets started.
- If the parameter is set to **heartbeat**, the HA-related processes get started.

session_state_timeout

session_state_timeout is a parameter used to define how long the CUBRID session data will be kept. The session data will be deleted when the driver terminates the connection or the session time expires. The session time will expire after the specified time if a client terminates abnormally.

Custom variables defined by **SET** and **PREPARE** statements can be deleted by **DROP** / **DEALLOCATE** statements before session timeout.

The default value is **21,600** seconds(6 hours), and this parameter's unit is second.

sort_limit_max_count

Regarding SORT-LIMIT optimization which can be applied when top-N rows are sorted by "ORDER BY ... LIMIT *N*" statement, this parameter specifies the LIMIT count to limit applying this optimization. When the value of *N* is smaller than a value of **sort_limit_max_count**, SORT-LIMIT optimization is applied. The default is 1000, the minimum is 0(which means that this optimization is disabled), and the maximum is INT_MAX.

For more details, see SORT-LIMIT optimization.

sql_trace_slow

sql_trace_slow is a parameter to configure the execution time of a query which will be judged as a long time execution. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied. The default value is -1 and the maximum value is 86,400,000 msec (24 hour). -1 means that the infinite time, so any queries will not be judged as a long duration query. For details, see the below **sql_trace_execution_plan**.

> **Note**
>
> The system parameter **sql_trace_slow** judges the query execution time based on the server, but the broker parameter **MAX_QUERY_TIMEOUT** judges the query execution time based on the broker.

sql_trace_execution_plan

sql_trace_execution_plan is a parameter to configure if the query plan of the long running query is written to the log or not. The default value is **no**.

If it is set to yes, a long running SQL, a query plan and the output of cubrid statdump command are written to the server error log file(located on $CUBRID/log/server directory) and CAS log file(located on $CUBRID/log/broker/sql_log directory) .

If it is set to no, only a long running SQL is written to the server error log file and CAS log file.

For example, if you want to write the execution plan of the slow query to the log file, and specify the query which executes more than 5 seconds as the slow query, then configure the value of the **sql_trace_slow** parameter as 5000(ms) and configure the value of the **sql_trace_execution_plan** parameter as yes.

But, on the server error log file, the related information is written only when the value of error_log_level is NOTIFICATION.

use_orderby_sort_limit

use_orderby_sort_limit is a parameter to configure whether to keep the intermediate result of sorting and merging process in the statement including the **ORDER BY ... LIMIT** *row_count* clause as many as *row_count*. If it is set to **yes**, you can decrease unnecessary comparing and merging processes because as many as intermediate results will be kept as the value of *row_count*. The default value is **yes**.

vacuum_prefetch_log_mode

vacuum_prefetch_log_mode is a parameter to configure the prefetch mode of log pages on behalf of vacuum.

In mode 0, the vacuum master thread prefetch the required log pages in a shared buffer. In mode 1 (default), each vacuum worker prefetches the required log pages in its own buffer. Mode 0 also requires that **vacuum_prefetch_log_buffer_size** system parameter is configured, in mode 0 this parameter is ignored and each vacuum worker prefetches an entire vacuum log block (default 32 log pages).

vacuum_prefetch_log_buffer_size

vacuum_prefetch_log_buffer_size is a parameter to configure the log prefetch buffer size of vacuum (it is used only if **vacuum_prefetch_log_mode** is set to 0).

data_buffer_neighbor_flush_pages

data_buffer_neighbor_flush_pages is a parameter to control the number of neighbor pages to be flushed with background flush (victim candidates flushing). When is less or equal to 1, the neighbor flush feature is considered deactivated.

data_buffer_neighbor_flush_nondirty

data_buffer_neighbor_flush_nondirty is a parameter to control the flushing of non-dirty neighbor pages. When victim candidates pages are flushed, and neighbor flush is activated (**data_buffer_neighbor_flush_pages** is greater than 1), than single non-dirty pages which completes a chain of neighbor (dirty) pages are also flushed.

tde_keys_file_path

tde_keys_file_path is a parameter to configure the path of the key file for TDE. The key file's name is fixed as <database_name>_keys, and the directory where the key file exists is designated. If this system parameter is not set, the key file is searched in the same location as the database volume. For a detailed description of the key file, see File-based Master Key Management.

tde_default_algorithm

tde_default_algorithm is a parameter that configures the default algorithm used when creating the TDE encryption table. Log and temporary data are always encrypted using the algorithm set with this parameter when they have to be encrypted. **AES** or **ARIA** can be set. For more information on encryption algorithms, refer to Encryption Algorithm.

recovery_progress_logging_interval

recovery_progress_logging_interval is a parameter to decide whether the details of recovery are printed and configure its period in seconds. If it is set bigger than 0, the total works and remained works to do of the three phases of recovery: Analysis, Redo and Undo are printed. When this is set smaller than 5, it is set to 5.

supplemental_log

supplemental_log is a parameter to determine whether information needed to support the CDC (Change Data Capture) or flashback is written to the log volume. CDC and flashback must be able to see how transactions logically changed the database through the physical logs. Any additional information required to interpret the physical logs is saved as **supplemental_log** . Setting this parameter bigger than 0 affects performance and log space because more logs are created and stored in addition to the existing transaction logs. If this parameter is set to 1, the information necessary to interpret DML and DDL executed by the user is logged. If it is set to 2, only information necessary to interpret the DML is logged.

regexp_engine

regexp_engine is a parameter to choose a library in which regular expression operators and functions will perform. **cppstd** or **re2** can be set and the default value is **re2**. For more information on regular expression functionalities, refer to Regular Expressions Functions and Operators.

vacuum_ovfp_check_threshold

vacuum_ovfp_check_threshold collects index information when the number of leaf's overflow pages to be read is greater than the value, when index vacuum is performed. The default is 1000 pages.

vacuum_ovfp_check_duration

vacuum_ovfp_check_duration specifies the duration for which data related to the count of index overflow pages, gathered by vacuum threads, is retained. Data that remains unchanged within the specified duration will be automatically removed. The unit of it's value is minutes.

deduplicate_key_level

deduplicate_key_level determines the automatic inclusion and value setting of the WITH DEDUPLICATE statement within the index creation statement. For details on DEDUPLICATE, see DEDUPLICATE. The default is -1(which means that the WITH DEDUPLICATE is not included implicitly).

Note

- If **deduplicate_key_level** is set to **-1**, even if the *deduplicate level* is explicitly specified in the CREATE INDEX statement, it is ignored.

print_index_detail

It specifies whether option information in the **WITH** clause is displayed when index syntax information is displayed, such as in the SHOW CREATE TABLE statement. Default is NO. However, the unloaddb tool is not affected by this setting.

enable_memory_monitoring

enable_memory_monitoring specifies whether to monitor the server's heap memory usage. The default value is NO. To apply any modifications to this value, a server restart is required. Setting the value to YES activates the server's memory monitoring feature, which continuously tracks and manages the server's heap memory usage. Heap memory usage is tracked based on the file and line where dynamic memory allocation occurs in the CUBRID source code. If multiple memory allocations occur at the same location, the memory usage is accumulated. When the tracked memory is deallocated, the amount of deallocated memory is subtracted from the accumulated total, continuously tracking the real-time heap memory usage. The monitored heap memory usage can be checked using the memmon utility.

Note

- This feature is only supported on Linux.
- The memory monitoring feature incurs additional overhead as it tracks usage every time memory is allocated on the server. In an HA environment, this can lead to performance issues, particularly for slave nodes, which experience a high proportion of memory allocations while processing logs from the master node. Therefore, it is not recommended to use this feature on slave nodes.
- The memory usage tracked by CUBRID may differ from the memory usage displayed by commands like pmap -d and htop. This is because the memory monitoring feature in CUBRID does not track memory allocations occurring within header files and glibc internals, to prevent conflicts that arise from automating the feature at the code level. Additionally, there are a few notable differences:
    - The heap memory usage of a process in pmap -d can be identified through the writeable/private section. However, this metric reflects the memory occupancy of the process based on the memory management policies of the OS. As a result, there may be differences between the memory usage shown by CUBRID's memory monitoring feature and what is displayed in pmap -d.
    - htop provides the memory usage of a process through the RES(Resident Size) section. However, since this shows the physical memory usage of a process, it cannot be used to check only the heap memory usage of a process.

flashback_timeout

It specifies the timeout for entering a transaction identifier in the flashback utility. If a transaction to be rewind is not entered within the specified timeout, an error message will be displayed, and the flashback utility will terminate. If there is no timeout, it may cause an issue where the archive log volumes required by the flashback utility are not deleted. Therefore, a method to remove the timeout is not provided. The default value is 300 seconds.

Broker Configuration

cubrid_broker.conf Configuration File and Default Parameters

Broker System Parameters

The following table shows the broker parameters available in the broker configuration file (**cubrid_broker.conf**). For details, see Common Parameters and Parameter by Broker. You can temporarily change the parameter of which configuration values can be dynamically changed by using the **broker_changer** utility. To apply configuration values even after restarting all brokers with **cubrid broker restart**, you should change the values in the **cubrid_broker.conf** file.

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
Common Parameters	Access	ACCESS_CONTROL	bool	no	
		ACCESS_CONTROL_FILE	string		
		ACCESS_CONTROL_DEFAULT_POLICY	bool	DENY	
	Logging	ADMIN_LOG_FILE	string	log/broker/cubrid_broker.log	
	Broker server (cub_broker)	MASTER_SHM_ID	int	30,001	
Parameter by Broker	Access	ACCESS_LIST	string		
		ACCESS_MODE	string	RW	available

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
		BROKER_PORT	int	30,000(max : 65,535)	
		CONNECT_ORDER	string	SEQ	available
		ENABLE_MONITOR_HANG	string	OFF	
		KEEP_CONNECTION	string	AUTO	available
		MAX_NUM_DELAYED_HOSTS_LOOKUP	int	-1	
		PREFERRED_HOSTS	string		available
		RECONNECT_TIME	sec	600	available
		REPLICA_ONLY	string	OFF	
	Broker App. Server(CAS)	APPL_SERVER_MAX_SIZE	MB	Windows 32bit: 40, Windows 64bit: 80, Linux: 0(max: 2,097,151)	available

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
		APPL_SERVER_MAX_SIZE_HARD_LIMIT	MB	1,024	available
		APPL_SERVER_PORT	int	BROKER_PORT+1	
		APPL_SERVER_SHM_ID	int	30,000	
		AUTO_ADD_APPL_SERVER	string	ON	
		MAX_NUM_APPL_SERVER	int	40	
		MIN_NUM_APPL_SERVER	int	5	
		TIME_TO_KILL	sec	120	available
	Transaction & Query	CCI_DEFAULT_AUTOCOMMIT	string	ON	
		LONG_QUERY_TIME	sec	60(max: 86,400)	available
		LONG_TRANSACTION_TIME	sec	60(max: 86,400)	available

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
		MAX_PREPARED_STMT_COUNT	int	2,000(min: 1)	available
		MAX_QUERY_TIMEOUT	sec	0(max: 86,400)	available
		SESSION_TIMEOUT	sec	300	available
		STATEMENT_POOLING	string	ON	available
		JDBC_CACHE	string	OFF	available
		JDBC_CACHE_HINT_ONLY	string	OFF	available
		JDBC_CACHE_LIFE_TIME	sec	1000	available
		TRIGGER_ACTION	string	ON	available
	Logging	ACCESS_LOG	string	OFF	available
		ACCESS_LOG_DIR	string	log/broker	
		ACCESS_LOG_MAX_SIZE	KB	10M(max: 2G)	available

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
		ERROR_LOG_DIR	string	log/broker/error_log	available
		LOG_DIR	string	log/broker/sql_log	available
		SLOW_LOG	string	ON	available
		SLOW_LOG_DIR	string	log/broker/sql_log	available
		SQL_LOG	string	ON	available
		SQL_LOG_MAX_SIZE	KB	10,000	available
	Shard	SHARD	string	OFF	
		SHARD_CONNECTION_FILE	string	shard_connection.txt	
		SHARD_DB_NAME	string		
		SHARD_DB_PASSWORD	string		
		SHARD_DB_USER	string		
			string	OFF	

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
		SHARD_IGNORE_HINT			
		SHARD_KEY_FILE	string	shard_key.txt	
		SHARD_KEY_FUNCTION_NAME	string		
		SHARD_KEY_LIBRARY_NAME	string		
		SHARD_KEY_MODULAR	int	256	
		SHARD_MAX_CLIENTS	int	256	
		SHARD_MAX_PREPARED_STMT_COUNT	int	10,000	
		SHARD_NUM_PROXY	int	1	
		SHARD_PROXY_CONN_WAIT_TIMEOUT	sec	8h	
		SHARD_PROXY_LOG	string	ERROR	available

Category	Use	Parameter Name	Type	Default Value	Dynamic Changes
		SHARD_PROXY_LOG_DIR	string	log/broker/proxy_log	
		SHARD_PROXY_LOG_MAX_SIZE	KB	100,000	available
		SHARD_PROXY_SHM_ID	int		
		SHARD_PROXY_TIMEOUT	sec	30(second)	
	Etc	MAX_STRING_LENGTH	int	-1	
		SERVICE	string	ON	
		SSL	string	OFF	
		SOURCE_ENV	string	cubrid.env	
		NET_BUF_SIZE	KB	16K	

Default Parameters

The **cubrid_broker.conf** file, the default broker configuration file created when installing CUBRID, includes some parameters that must be modified by default. If you want to modify the values of parameters that are not included in the configuration file by default, you can add or modify one yourself.

The following is the content of the **cubrid_broker.conf** file provided by default.

```
[broker]
MASTER_SHM_ID           =30001
ADMIN_LOG_FILE          =log/broker/cubrid_broker.log

[%query_editor]
SERVICE                 =ON
BROKER_PORT             =30000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =30000
LOG_DIR                 =log/broker/sql_log
ERROR_LOG_DIR           =log/broker/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO

[%BROKER1]
SERVICE                 =ON
BROKER_PORT             =33000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =33000
LOG_DIR                 =log/broker/sql_log
ERROR_LOG_DIR           =log/broker/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO
```

Broker Configuration File Related Environment Variables

You can specify the location of broker configuration file (**cubrid_broker.conf**) file by using the **CUBRID_BROKER_CONF_FILE** variable. The variable is used when executing several brokers with different configuration.

Common Parameters

The following are parameters commonly applied to entire brokers; it is written under [broker] section.

Access

ACCESS_CONTROL

ACCESS_CONTROL is a parameter used to limit applications which are trying to connect a broker. The default value is **OFF**. For details, see Limiting Broker Access.

ACCESS_CONTROL_FILE

ACCESS_CONTROL_FILE is a parameter to configure the name of a file in which a database name, database user ID, and the list of IPs are stored. List of IPs can be written up to the maximum of 256 lines per <*db_name*>:<*db_user*> in a broker. For details, see Limiting Broker Access.

ACCESS_CONTROL_DEFAULT_POLICY

If no broker is specified in the **ACCESS_CONTROL_FILE** and the value of **ACCESS_CONTROL_DEFAULT_POLICY** is **ALLOW**, access to the broker is permitted for everyone. The default is **DENY**. For more information, see Limiting Broker Access.

Note

The settings value ALLOW or DENY for **ACCESS_CONTROL_DEFAULT_POLICY** is valid only when **ACCESS_CONTROL** is set to **ON**. If it is set to **OFF**, the setting value is not applicable.

Logging

ADMIN_LOG_FILE

ADMIN_LOG_FILE is a parameter to configure the file in which time of running CUBRID broker is stored. The default value is a **log/broker/cubrid_broker.log** file.

Broker Server(cub_broker)

MASTER_SHM_ID

MASTER_SHM_ID is a parameter used to specify the identifier of shared memory which is used to manage the CUBRID broker. Its value must be unique in the system. The default value is **30001**.

Parameter by Broker

The following describes parameters to configure the environment variables of brokers; each parameter is located under *[%broker_name]*. The maximum length of *broker_name* is 63 characters in English.

Access

ACCESS_LIST

ACCESS_LIST is a parameter to configure the name of a file where the list of IP addresses of an application which allows access to the CUBRID broker is stored. To allow access by IP addresses access 192.168.10.* and 192.168.20.* store them to a file (ip_lists.txt) and then assign the filename with the value of this parameter.

ACCESS_MODE

ACCESS_MODE is a parameter to configure default mode of the broker. The default value is **RW**. For details, see cubrid_broker.conf.

BROKER_PORT

BROKER_PORT is a parameter to configure the port number of the broker; the value must be unique and smaller than 65,535. The default port value of **query_editor**' broker is **30,000** and the port value of the **broker1** is **33,000**.

CONNECT_ORDER

CONNECT_ORDER is a parameter to specify whether a CAS tries to connect to one of hosts in the order or randomly in **$CUBRID_DATABASES/databases.txt**, when a CAS decides the order of connecting to a host. The default is **SEQ**; a CAS tries to connect to a host in the order. if this value is **RANDOM**, a CAS tries to connect to a host randomly. If **PREFERRED_HOSTS** parameter is specified, firstly a CAS tries to connect to one of hosts specified in **PREFERRED_HOSTS**, then uses db-host values in **$CUBRID_DATABASES/databases.txt** only when the connection is failed.

ENABLE_MONITOR_HANG

ENABLE_MONITOR_HANG is a parameter to configure whether to block the access from the application to the broker or not, when more than a certain ratio of CASes on that broker are hung. If the **ENABLE_MONITOR_HANG** parameter value is **ON**, blocking feature is processed. The default value is **OFF**. If it is **OFF**, don't do the behavior.

The broker process judges the CAS as hung if the hanging status of the CAS keeps more than one minute, then block the access from applications to that broker; it brings the behavior which the applications try to access to the alternative hosts(altHosts) configured by the connection URL.

KEEP_CONNECTION

KEEP_CONNECTION is a parameter to configure the way of connection between CAS and application clients; it is set to one of the following: **ON** or **AUTO**. If this value is **ON**, it is connected in connection unit. If it is **AUTO** and the number of servers is

more than that of clients, transaction unit is used; in the reverse case, connection unit is used. The default value is **AUTO**.

MAX_NUM_DELAYED_HOSTS_LOOKUP

When almost all DB servers have the delay of replication in the HA environment where multiple DB servers on db-host of databases.txt are specified, check if the connection is established or not until the number of delayed replication servers; the number is specified in **MAX_NUM_DELAYED_HOSTS_LOOKUP** (whether the delay of replication in the DB server is judged only with the standby hosts; it is determined by the setting of ref:*ha_delay_limit* <*ha_delay_limit*>). See MAX_NUM_DELAYED_HOSTS_LOOKUP for further information.

PREFERRED_HOSTS

PREFERRED_HOSTS is a parameter to specify the order of a host to which a CAS tries to connect in a first priority. If the connection is failed after trying connection in the order specified in **PREFERRED_HOSTS**, a CAS tries to connect to the one of hosts specified in **$CUBRID_DATABASES/databases.txt**. The default value is **NULL**. For details, see cubrid_broker.conf.

RECONNECT_TIME

If the time specified by **RECONNECT_TIME** is elapsed in a certain status, CAS will try to reconnect to the other DB server. The default of this parameter is **600s(10min)**. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, s will be applied.

a certain status which CAS tries to reconnect is as follows.

- when CAS is connected to not a DB server in **PREFERRED_HOSTS**, but the DB server of db-host in databases.txt.
- when CAS with “ACCESS_MODE=RO”(Read Only) is connected to not the standby DB server, but the active DB server.
- when CAS is connected to the DB server of which replication is delayed.

When **RECONNECT_TIME** is 0, CAS does not try to reconnect.

REPLICA_ONLY

If a value of **REPLICA_ONLY** is **ON**, CAS is connected only to replicas. The default is **OFF**. Even though the value of **REPLICA_ONLY** is **ON**, when a value of **ACCESS_MODE** is **RW**, it is possible to write directly to the replica DB. However, the data to be written directly to the replica DB are not replicated.

Note

Please note that replication mismatch occurs when you write the data directly to the replica DB.

Broker App. Server(CAS)

APPL_SERVER_MAX_SIZE

APPL_SERVER_MAX_SIZE is a parameter to configure the maximum size of the process memory usage handled by CAS. You can set a unit as B, K, M or G, which stands for bytes, kilobytes(KB), megabytes(MB) or gigabytes(GB) respectively. If you omit the unit, M will be applied.

Specifying this parameter makes transactions terminate (commit or rollback) only when it is executed by a user. In contrast to this, specifying **APPL_SERVER_MAX_SIZE_HARD_LIMIT** makes transactions forcibly terminate (rollback) and restart CAS.

Note that the default values of Windows and Linux are different from each other.

For 32-bit Windows, the default value is **40** MB; for 64-bit Windows, it is **80** MB. The maximum value is the same on Windows and Linux as 2,097,151 MB. When current process size exceeds the value of **APPL_SERVER_MAX_SIZE**, broker restarts the corresponding CAS.

For Linux, the default value of **APPL_SERVER_MAX_SIZE** is **0**; CAS restarts in the following conditions.

- **APPL_SERVER_MAX_SIZE** is zero or negative: At the point when current process size becomes twice as large as initial memory
- **APPL_SERVER_MAX_SIZE** is positive: At the point when it exceeds the value specified in **APPL_SERVER_MAX_SIZE**

Note

Be careful not to make the value too small because application servers may restart frequently and unexpectedly. In general, the value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is greater than that of

APPL_SERVER_MAX_SIZE. For details, see description of **APPL_SERVER_MAX_SIZE_HARD_LIMIT**.

APPL_SERVER_MAX_SIZE_HARD_LIMIT

APPL_SERVER_MAX_SIZE_HARD_LIMIT is a parameter to configure the maximum size of process memory usage handled by CAS. You can set a unit as B, K, M or G, which stands for bytes, kilobytes(KB), megabytes(MB) or gigabytes(GB) respectively. If you omit the unit, M will be applied. The default value is **1,024** (MB), and the maximum value is 2,097,151 (MB).

Specifying this parameter makes transactions being processed forcibly terminate (rollback) and restart CAS. In contrast to this, specifying **APPL_SERVER_MAX_SIZE** makes transactions terminate only when it is executed by a user.

Note

Be careful not to make the value too small because application servers may restart frequently and unexpectedly. When restarting CAS, **APPL_SERVER_MAX_SIZE** is specified to wait for normal termination of transactions although memory usage increases; **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is specified to forcibly terminate transactions if memory usage exceeds the maximum value allowed. Therefore, in general, the value of **APPL_SERVER_MAX_SIZE_HARD_LIMIT** is greater than that of **APPL_SERVER_MAX_SIZE**.

APPL_SERVER_PORT

APPL_SERVER_PORT is a parameter to designate the TCP ports of CAS that communicates with application clients (**Windows only**).

On Linux, an established TCP connection between the BROKER and the client will be passed to the CAS. Therefore, the client can communicate with the CAS without additional TCP connection.

On the other hand, in Windows, all available CASes are waiting for a connection on an independent TCP port, for example, 33001/tcp, 33002/tcp. In these circumstances, when a client connects to a BROKER, the BROKER delivers the TCP port number, for example, 33001/tcp, for connecting to an available CAS to the client. In sequence, the client terminates the current network connection with the BROKER and establishes a new connection with the CAS using the port number received from the BROKER.

If the **APPL_SERVER_PORT** parameter is not additionally specified, this value is the value obtained by adding 1 to the value of **BROKER_PORT**. For example, if the value of **BROKER_PORT** is 30,000 and the **APPL_SERVER_PORT** parameter has not been specified, and if the **MIN_NUM_APPL_SERVER** value is 5, then five CASes use the TCP ports between 30,001 and 30,005, respectively. On the other hand, if the value of **APPL_SERVER_PORT** is 35,000 under the same conditions, 5 CASes use TCP ports from 35,000 to 35,004. The maximum number of CAS specified in the **MAX_NUM_APPL_SERVER** parameter in **cubrid_broker_conf**; therefore, the maximum number of connection ports is also determined by the value of **MAX_NUM_APPL_SERVER** parameter.

On the Windows system, if a firewall system exists between an application and a CUBRID broker, all TCP ports specified in **BROKER_PORT** and **APPL_SERVER_PORT** must be opened.

Note

For the **CUBRID_TMP** environment variable that specifies the UNIX domain socket file path of **cub_master** and **cub_broker** processes, see Configuring Environment Variables.

APPL_SERVER_SHM_ID

APPL_SERVER_SHM_ID is a parameter to configure the ID of shared memory used by CAS; the value must be unique within system. The default value is the same as the port value of the broker.

AUTO_ADD_APPL_SERVER

AUTO_ADD_APPL_SERVER is a parameter to configure whether CAS increase automatically to the value specified in **MAX_NUM_APPL_SERVER** in case of needed; the value will be either **ON** or **OFF** (default: **ON**).

MAX_NUM_APPL_SERVER

MAX_NUM_APPL_SERVER is a parameter to configure the maximum number of simultaneous connections allowed. The default value is **40**.

MIN_NUM_APPL_SERVER

MIN_NUM_APPL_SERVER is a parameter to configure the minimum number of CAS even if any request to connect the broker has not been made. The default value is **5**.

TIME_TO_KILL

TIME_TO_KILL is a parameter to configure the time to remove CAS in idle state among CAS added dynamically. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, s will be applied. The default value is **120** (seconds).

An idle state is one in which the server is not involved in any jobs. If this state continues exceeding the value specified in **TIME_TO_KILL**, CAS is removed.

The value configured in this parameter affects only CAS added dynamically, so it applies only when the **AUTO_ADD_APPL_SERVER** parameter is configured to **ON**. Note that times to add or remove CAS will be increased more if the **TIME_TO_KILL** value is so small.

Transaction & Query

CCI_DEFAULT_AUTOCOMMIT

CCI_DEFAULT_AUTOCOMMIT is a parameter to configure whether to make application implemented in CCI interface or CCI-based interface such as PHP, OLE DB, Perl, Python, and Ruby commit automatically. The default value is **ON**. This parameter does not affect applications implemented in JDBC.

If the **CCI_DEFAULT_AUTOCOMMIT** parameter value is **OFF**, the broker application server (CAS) process is occupied until the transaction is terminated. Therefore, it is recommended to execute commit after completing fetch when executing the **SELECT** statement.

Note

The **CCI_DEFAULT_AUTOCOMMIT** parameter has been supported from 2008 R4.0, and the default value is **OFF** for the version. Therefore, if you use CUBRID 2008 R4.1 or later versions and want to keep the configuration **OFF**, you should manually change it to **OFF** to avoid autocommit of unexpected transaction.

Warning

In ODBC driver, the setting of **CCI_DEFAULT_AUTOCOMMIT** is ignored and worked as **ON**; therefore, you should set the autocommit on or off in the program directly.

LONG_QUERY_TIME

LONG_QUERY_TIME is a parameter to configure execution time of query which is evaluated as long-duration query. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied. The default value is **60** (seconds) and the maximum value is 86,400(1 day). When you run a query and this query's running time takes more than the specified time, a value of LONG-Q, which is printed out from "cubrid broker status" command, is increased 1; this SQL is written to SLOW log file ($CUBRID/log/broker/sql_log/*.slow.log) of CAS. See SLOW_LOG.

This value can be valued in milliseconds with a decimal separator. For example, the value can be configured into 0.5 to configure 500 msec.

Note that if a parameter value is configured to **0**, a long-duration query is not evaluated.

LONG_TRANSACTION_TIME

LONG_TRANSACTION_TIME is a parameter to configure execution time of query which is evaluated as long-duration transaction. The default value is **60** (seconds) and the maximum value is 86,400(1 day).

This value can be valued in milliseconds with a decimal separator. For example, the value can be configured into 0.5 to configure 500 msec.

Note that if a parameter value is configured to **0**, a long-duration transaction is not evaluated.

MAX_PREPARED_STMT_COUNT

MAX_PREPARED_STMT_COUNT is a parameter used to limit the number of prepared statements by user (application) access. The default value is **2,000** and the minimum value is 1. The problem in which prepared statement exceeding allowed memory is mistakenly generated by system can be prohibited by making users specify the parameter value.

Note

When you want to change the value of **MAX_PREPARED_STMT_COUNT** dynamically by **broker_changer** command, this can be changed only when this is bigger than the existing value; this cannot be changed when this is smaller than the existing value.

MAX_QUERY_TIMEOUT

MAX_ QUERY_TIMEOUT is a parameter to configure timeout value of query execution. When time exceeds a value specified in this parameter after starting query execution, the query being executed stops and rolls back. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied. The default value is **0** (seconds) and it means infinite waiting. The value range is available from 0 to 86,400 seconds(one day).

The smallest value (except 0) between the **MAX_QUERY_TIMEOUT** value and query timeout value of an application is applied if query timeout is configured in an application.

Note

See the `cci_connect_with_url()` and `cci_set_query_timeout()` functions to configure query timeout of CCI applications. For configuring query timeout of JDBC applications, see the **setQueryTimeout** method.

SESSION_TIMEOUT

SESSION_TIMEOUT is a parameter to configure timeout value for the session of the broker. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied. The default value is **300** (seconds).

If there is no response to the job request for the specified time period, session will be terminated. If a value exceeds the value specified in this parameter without any action taken after starting transaction, the connections are terminated.

STATEMENT_POOLING

STATEMENT_POOLING is a parameter to configure whether to use statement pool feature. The default value is **ON**.

CUBRID closes all handles of prepared statement in the corresponding client sessions when transaction commit or rollback is made. If the value of **STATEMENT_POOLING** is set to **ON**, the handles are reusable because they are maintained in the pool. Therefore, in an environment where libraries, such as general applications reusing prepared statement or DBCP where statement pooling is implemented is applied, the default configuration (**ON**) should be maintained.

If the prepared statement is executed after transaction commit or termination while **STATEMENT_POOLING** is set to **OFF**, the following message will be displayed.

```
Caused by: cubrid.jdbc.driver.CUBRIDException: Attempt to
access a closed Statement.
```

JDBC_CACHE

JDBC_CACHE is a parameter to configure whether to use result-cache fetaure. The default value is **OFF**.

If the parameter is **ON**, all of SELECT queries from JDBC is cached at client for life time which is configured by **JDBC_CACHE_LIFE_TIME**

JDBC_CACHE_HINT_ONLY

JDBC_CACHE_HINT_ONLY is a parameter to configure whether to use result-cache feature only by query hint /*+ JDBC_CACHE */.

It works as if the parameter is **ON** when the query hint is given.

JDBC_CACHE_LIFE_TIME

JDBC_CACHE_HINT_ONLY is a parameter to configure JDBC client's result-cache life time. The default value is 1000 (sec).

For only cache life time, the result-cache is available. After the cache lifetime expired, the prior cached results are no more available and a new result is cached.

The cache life time works only when the paramter **JDBC_CACHE** or **JDBC_CACHE_HINT_ONLY** is configured to "ON".

WARNING

JDBC_CACHE, **JDBC_CACHE_HINT_ONLY**, and **JDBC_CACHE_LIFE_TIME** parameters are meaningless

when the query cache-related system parameter both of **max_query_cache_entries** and **query_cache_size_in_pages** are not set to positive value.

For result cache working, the SELECT query must include query hint /*+ QUERY_CACHE */ together with these JDBC related paramter setting.

TRIGGER_ACTION

Turn on or off of the trigger's action about the broker which specified this parameter. Specify **ON** or **OFF** as a value; The default is **ON**.

Logging

ACCESS_LOG

ACCESS_LOG is a parameter to configure whether to store the access log of the broker. The default value is **OFF**. The name of the access log file for the broker is *broker_name***.access** and the file is stored under **$CUBRID/log/broker** directory.

ACCESS_LOG_DIR

ACCESS_LOG_DIR specifies the directory for broker access logging files(**ACCESS_LOG**) to be created. The default is **log/broker**.

ACCESS_LOG_MAX_SIZE

ACCESS_LOG_MAX_SIZE specifies the maximum size of broker access logging files(**ACCESS_LOG**); if a broker access logging file is bigger than a specified size, this file is backed up into the name of *broker_name***.access.***YYYYMMDDHHMISS*, then logging messages are written to the new file(*broker_name*.**access**). The default is 10M and the maximum is 2G. It can be dynamically changed during operating a broker.

ERROR_LOG_DIR

ERROR_LOG_DIR is a parameter to configure default directory in which error logs about broker is stored. The default value is **log/broker/error_log**. The log filename for the broker error is *broker_ name_id.err*.

LOG_DIR

LOG_DIR is a parameter to configure the directory where SQL logs are stored. The default value is **log/broker/sql_log**. The filename of the SQL logs is *broker_name_id.sql.log*.

SLOW_LOG

SLOW_LOG is a parameter to configure whether to log. The default value is **ON**. If the value is **ON**, long transaction query which exceeds the time specified in **LONG_QUERY_TIME** or query where an error occurred is stored in the **SLOW SQL** log file. The name of file created is *broker_name_id.slow.log* and it is located under **SLOW_LOG_DIR**.

SLOW_LOG_DIR

SLOW_LOG_DIR is a parameter to configure the location of directory where the log file is generated. The default value is **log/broker/sql_log**.

SQL_LOG

SQL_LOG is a parameter to configure whether to leave logs for SQL statements processed by CAS when CAS handles requests from a client. The default value is **ON**. When this parameter is configured to **ON**, all logs are stored. The log filename becomes *broker_name_id.sql.log*. The file is created in the **log/broker/sql_log** directory under the installation directory. The parameter values are as follows:

- **OFF** : Does not leave any logs.
- **ERROR** : Stores logs for queries which occur an error. only queries where an error occurs.
- **NOTICE** : Stores logs for the long-duration execution queries which exceeds the configured time/transaction, or leaves logs for queries which occur an error.
- **TIMEOUT** : Stores logs for the long-duration execution queries which exceeds the configured time/transaction.
- **ON** / **ALL** : Stores all logs.

SQL_LOG_MAX_SIZE

SQL_LOG_MAX_SIZE is a parameter to configure the maximum size of the SQL log file. You can set a unit as B, K, M or G, which stands for bytes, kilobytes(KB) or megabytes(MB) or gigabytes(GB) respectively. If you omit the unit, M will be applied. The default value is **10,000*** (KB).

- If the size of the SQL log file, which is created when the **SQL_LOG** parameter is configured to **ON**, reaches to the size configured by the parameter, *broker_name_id.sql.log.bak* is created.
- If the size of the SLOW SQL log file, which is created when the **SLOW_LOG** parameter is configured to **ON**, reaches to the size configured by the parameter, *broker_name_id.slow.log.bak* is created.

SHARD

To use SHARD feature, configure the below parameters in **cubrid_broker.conf** as referring to **cubrid_broker.conf.shard**.

SHARD

It specifies to activate/deactivate SHARD feature. You can set this value as **ON** or **OFF**. The default is **OFF**.

SHARD_CONNECTION_FILE

The path of the shard connection file. The shard connection file should be located in **$CUBRID/conf**. For more information, see the shard connection file.

SHARD_DB_NAME

The name of the shard DB, used to verify the request for connection from an application.

SHARD_DB_PASSWORD

The user password of the backend shard DB, used to connect to the backend DBMS for the CAS process as well as to verify the request for connection from an application. Passwords of all shard DBs should be identical.

The environment variable can be used when you don't want to expose **SHARD_DB_PASSWORD** to cubrid_broker.conf. The format of this environment variable name is <*broker_name*>**_SHARD_DB_PASSWORD**, and <*broker_name*> always should be changed as upper cases. For example, if the name of broker is *shard1*, the name of an environment variable which configures the shard DB password will be **SHARD1_SHARD_DB_PASSWORD**. But, if the SHARD feature is restarted by "cubrid broker restart" command, the environment variable of **SHARD_DB_PASSWORD** or the value of **SHARD_DB_PASSWORD** in cubrid_broker.conf must be configured.

```
export SHARD1_SHARD_DB_PASSWORD=shard123
```

Note

SHARD_DB_USER/SHARD_DB_PASSWORD parameters is deprecated. Therefore, it is recommended to deliver the connection information in an application.

SHARD_DB_USER

The name of the backend shard DB user, used to connect to the backend DBMS for the CAS process as well as to verify the request for connection from an application. Usernames on all shard DBs should be identical.

Note

SHARD_DB_USER/SHARD_DB_PASSWORD parameters is deprecated. Therefore, it is recommended to deliver the connection information in an application.

SHARD_IGNORE_HINT

When this value is **ON**, the hint provided to connect to a specific shard is ignored and the database to connect is selected based on the defined rule. The default value is **OFF**. It can be used to balance the read load while all databases are copied with the same data. For example, to give the load of an application to only one node among several replication nodes, the proxy automatically determines the node (database) with one connection to a specific shard.

SHARD_KEY_FILE

The path of the shard key configuration file. The shard key configuration file should be located in **$CUBRID/conf**. For more information, see the shard key configuration file.

SHARD_KEY_FUNCTION_NAME

The parameter to specify the name of the user hash function for shard key. For more information, see Setting User-Defined Hash Function.

SHARD_KEY_LIBRARY_NAME

Specify the library path loadable at runtime to specify the user hash function for the shard key. If the **SHARD_KEY_LIBRARY_NAME** parameter is set, the **SHARD_KEY_FUNCTION_NAME** parameter should also be set. For more information, see Setting User-Defined Hash Function.

SHARD_KEY_MODULAR

The parameter to specify the range of results of the default shard key hash function. The result of the function is shard_key(integer) % SHARD_KEY_MODULAR. The minimum value is 1, and the maximum value is 256. For related issues, see shard key configuration file and Setting User-Defined Hash Function.

SHARD_MAX_CLIENTS

The number of applications that can be concurrently connected by using the proxy. The default value is 256 and the maximum value is 10,000 per proxy.

SHARD_MAX_PREPARED_STMT_COUNT

The maximum size of statement pool managed by proxy. The default is 10,000.

SHARD_NUM_PROXY

The number of proxy processes.

SHARD_PROXY_CONN_WAIT_TIMEOUT

If there is no request anymore during the time specified in this parameter, CAS disconnect with DB. The default is **8h**. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, second(s) will be applied. CAS which has a previous password should be exit because it cannot be used anymore; this feature protects that CAS is still kept unnecessarily.

SHARD_PROXY_LOG

The proxy log level. It can be set to one of the following values:

Proxy log level policy: Setting the higher level leaves all the lower logs.

- Example) When SCHEDULE is set, ERROR | TIMEOUT | NOTICE | SHARD | SCHEDULE log is left.
- **ALL** : All logs
- **ON** : All logs
- **SHARD** : Logs for selecting and processing shard DBs.
- **SCHEDULE** : Logs for scheduling tasks.
- **NOTICE** : Logs for key notices.
- **TIMEOUT** : Logs for timeouts.
- **ERROR** : Logs for errors.
- **NONE** : No logs recorded.
- **OFF** : No logs recorded.

SHARD_PROXY_LOG_DIR

The directory path where the proxy logs will be saved.

SHARD_PROXY_LOG_MAX_SIZE

The maximum size of the proxy log file. You can set a unit as B, K, M or G, which stands for bytes, kilobytes(KB) or megabytes(MB) or gigabytes(GB) respectively. If you omit the unit, K will be applied. The maximum value is 1,000,000(KB).

SHARD_PROXY_SHM_ID

A parameter to configure the ID of shared memory used by proxy

SHARD_PROXY_TIMEOUT

The maximum waiting time by which the statement is prepared or CAS is available to use. The default value is 30(seconds). If this value is 0, the waiting time is decided by the value of the query_timeout system parameter; if the value of query_timeout is also 0, the waiting time is infinite. IF the value SHARD_PROXY_TIMEOUT is larger than 0, the maximum value between query_timeout and SHARD_PROXY_TIMEOUT decides the waiting time. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, s will be applied.

Note

Required parameters for configuring proxy

To configure CUBRID proxy, you should specify SHARD_MAX_CLIENTS, MAX_NUM_APPL_SERVER and SHARD_NUM_PROXY.

- In Linux, the number of file descriptors(fd) per proxy process is limited as follows.
  - "((SHARD_MAX_CLIENTS + MAX_NUM_APPL_SERVER) / SHARD_NUM_PROXY) + 256" <= 10,000

The following are detail descriptions on above formulas.

- SHARD_MAX_CLIENTS is the maximum number of applications which access the SHARD system.
- MAX_NUM_APPL_SERVER is the maximum number of all CASes which can access proxy system.
- SHARD_NUM_PROXY is the maximum number of proxy processes which can use on the SHARD system.
- "SHARD_MAX_CLIENTS / SHARD_NUM_PROXY" is the maximum number of applications which can access per proxy process.
- "MAX_NUM_APPL_SERVER / SHARD_NUM_PROXY" is the maximum number of CASes which can access per proxy process.
- 256 is the number of file descriptors which are used internally per process on Linux.

As an example of configuring SHARD parameters in Linux system, if you specify the maximum concurrent access number of applications (SHARD_MAX_CLIENTS) as 5,000, the maximum number of CASes(MAX_NUM_APPL_SERVER) as 200 and the maximum number of proxy process(SHARD_NUM_PROXY) as 1, then file descriptors per proxy process becomes (5,000 + 200)/1 + 256 = 5,456, and it is less than 10,000; it is possible configuration.

Regarding above, the following is the connection-relationship between each process. “proxy” intermediates a connection between “app. client” and “CAS”.

In the below, [] indicates a process, and -> indicates the requesting direction.

```
[app. client]   --(initial access request)-----------> [broker]
(select proxy)
                <--(announce proxy to access)---
                ------------------------------------> [proxy] --
(select CAS)--> [CAS] ---> [DB server]
                <-------------------------------------
<----------------       <---
                <--(now use the same proxy)---------->
<---------------->      <-->
```

broker is used only once when the application client requires the initial access, and then CUBRID keep connections among “[app. client] - [proxy] - [CAS] - [DB Server]”.

Also, CAS keeps connection with DB server after DB connection is completed.

Etc

MAX_STRING_LENGTH

MAX_STRING_LENGTH is a parameter to configure the maximum string length for BIT, VARBIT, CHAR and VARCHAR data types. If the value is **-1**, which is the default value, the length defined in the database is used. If the value is **100**, the value acts like 100 being applied even when a certain attribute is defined as VARCHAR(1000).

SERVICE

SERVICE is a parameter to configure whether to run the broker. It can be either **ON** or **OFF**. The default value is **ON**. The broker can run only when this value is configured to **ON**.

SSL

SSL is a parameter to configure whether to apply packet encryption (**SSL**) to the broker. It can be either **ON** or **OFF**. The default value is **OFF**. When this value is configured to **ON**, packet encryption using **TLS** will be applied to the broker/CAS.

⚠ Warning

When the broker is configured to do **TLS** (**SSL=ON**), clients such as **jdbc** client must connect in encryption mode, otherwise the connection request to the broker will be rejected. The opposite is also true. The connection request of SSL clients to the non-SSL broker will be rejected.

SOURCE_ENV

SOURCE_ENV is a parameter used to determine the file where the operating system variable for each broker is configured. The extension of the file must be **env**. All parameters specified in **cubrid.conf** can also be configured by environment variables. For example, the **lock_timeout** parameter in **cubrid.conf** can also be configured by the **CUBRID_LOCK_TIMEOUT** environment variable. As another example, to block execution of DDL statements on broker1, you can configure **CUBRID_BLOCK_DDL_STATEMENT** to 1 in the file specified by **SOURCE_ENV**.

An environment variable, if exists, has priority over **cubrid.conf**. The default value is **cubrid.env**.

NET_BUF_SIZE

NET_BUF_SIZE is a parameter used to determine the network buffer size used by **CAS** to transmit results to the client. The result sets will be buffered into the **CAS** network buffer, and are sent to the client when the buffer is full. **NET_BUF_SIZE** is the size of the packet that CAS transmits to the client, and it may affect transmission efficiency.

HA Configuration

Regarding HA configuration, see Environment Configuration.

SHARD Configuration

Regarding SHARD configuration, see Configuration.

SystemTap

Overview

SystemTap is a tool that can be used to dynamically monitor and track the process of running. CUBRID supports SystemTap; therefore, it is possible to find the cause of a performance bottleneck.

The basic idea of SystemTap script is that you can specify the name of the event and grant a handler there. Handler is script statements that specify the action to be performed each time an event occurs.

To monitor the performance of CUBRID using SystemTap, you need to install SystemTap. After installing SystemTap, you can write and run a SystemTap script which is like a C language. With this script, you can monitor the performance of the System.

SystemTap supports only on Linux.

See https://sourceware.org/systemtap/index.html for further information and installation.

Installing SystemTap

Checking Installation

1. Check if you have group accounts, stapusr and stapdev in /etc/group file. If they don't exist, SystemTap may not be installed.

2. Add CUBRID user account when you install CUBRID into stapusr and stapdev group accounts. Here let's assume the CUBRID user account as "cubrid".

```
$ vi /etc/group

stapusr:x:156:cubrid
stapdev:x:158:cubrid
```

3. To check if SystemTap is runnable, simply run the below command.

```
$ stap -ve 'probe begin { log("hello world") exit() }'
```

Version

To execute SystemTap scripts in CUBRID, you should use SystemTap 2.2 or higher.

The below is an example to install SystemTap in CentOS 6.3. Checking version and installing SystemTap can be different among Linux distributors.

1. Check the current version of the installed SystemTap.

```
$ sudo yum list|grep  systemtap
systemtap.x86_64
1.7-5.el6_3.1                  @update
```

```
systemtap-client.x86_64
1.7-5.el6_3.1                  @update
systemtap-devel.x86_64
1.7-5.el6_3.1                  @update
systemtap-runtime.x86_64
1.7-5.el6_3.1                  @update
systemtap-grapher.x86_64
1.7-5.el6_3.1                  update
systemtap-initscript.x86_64
1.7-5.el6_3.1                  update
systemtap-sdt-devel.i686
1.7-5.el6_3.1                  update
systemtap-sdt-devel.x86_64
1.7-5.el6_3.1                  update
systemtap-server.x86_64
1.7-5.el6_3.1                  update
systemtap-testsuite.x86_64
1.7-5.el6_3.1                  update
```

2. If the lower version of SystemTap than 2.2 version is installed, remove it.

```
$ sudo yum remove systemtap-runtime
$ sudo yum remove systemtap-devel
$ sudo yum remove systemtap
```

3. Install the RPM distributed package of SystemTap 2.2 or higher. You can find the RPM distributed package in http://rpmfind.net/linux/rpm2html/.

```
$ sudo rpm -ivh systemtap-devel-2.3-3.el6.x86_64.rpm
$ sudo rpm -ivh systemtap-runtime-2.3-3.el6.x86_64.rpm
$ sudo rpm -ivh systemtap-client-2.3-3.el6.x86_64.rpm
$ sudo rpm -ivh systemtap-2.3-3.el6.x86_64.rpm
```

Related Terms

Marker

A marker placed in code provides a hook to call a function (probe) that you can provide at runtime. The function you provide is called each time the marker is executed, in the execution context of the caller. When the function provided ends its execution, it returns to the caller.

The function a user provides, the probe can be used for tracing and performance accounting.

Probe

Probe is a kind of function to define the behavior when a certain event occurs; it is separated into an event and a handler.

SystemTap script defines a specific event, that is, the behavior when a marker occurs.

SystemTap script is able to have several probes; a handler of the probe is called as probe body.

SystemTap script accepts the insertion of the probing code without recompiling codes and gives more flexibility for handlers. Events are executed as triggers as handlers can be run. Handler can write data and be specified as the action to print out the data.

For CUBRID's markers, see CUBRID markers.

Asynchronous Events

Asynchronous events are defined internally; it is not dependent on special jobs or locations on the code. These kinds of probing events are mainly counter and timer, etc.

Examples of asynchronous events are as follows.

- begin

  The start of SystemTap session.

  e.g. The moment the SystemTap starts.

- end

  The end of SystemTap.

- timer events

  The event specifying the handler works periodically

  e.g. The below prints "hello world" per 5 seconds.

```
probe timer.s(5)
{
  printf("hello world\n")
}
```

Using SystemTap in CUBRID

Building CUBRID source

SystemTap can be used only on Linux.

To use SystemTap by building CUBRID source, **ENABLE_SYSTEMTAP** is **ON** which is set by default.

This option is already included in the release build, a user not building the CUBRID source but installing CUBRID with the installation package can also use SystemTap script.

The below is an example of building the CUBRID source.

```
build.sh -m release
```

Running SystemTap script

Examples of SystemTap scripts in CUBRID are located in $CUBRID/share/systemtap directory.

The below is an example of running buffer_access.stp file.

```
cd $CUBRID/share/systemtap/tapset/scripts
stap -k buffer_access.stp -o result.txt
```

Printing Results

When you run a certain script, it displays the requested result to the console by the script code. With "-o *filename*" option, it writes the requested result to the *filename*.

The below is the result of the above example.

```
Page buffer hit count: 172
Page buffer miss count: 172
Miss ratio: 50
```

CUBRID markers

A very useful feature of SystemTap is the possibility of placing markers in the user source code (CUBRID code) and writing probes that triggers when these markers are reached. Below is the list of CUBRID markers and their meaning.

Connection markers

We might be interested in gathering information helpful for an analysis related to connection activity (number of connections, duration of individual connections, average duration of a connection, maximum number of connections achieved etc.) during a period of time. In order for such monitoring scripts to be written, we must provide at least two helpful markers: connection-start and connection-end.

conn_start(*connection_id*, *user*)

This marker is triggered when the query execution process on the server has begun.

Parameters:

- **connection_id** – an integer containing the connection ID.
- **user** – The username used by this connection.

conn_end(*connection_id*, *user*)

This marker is triggered when the query execution process on the server has ended.

Parameters:

- **connection_id** – an integer containing the connection ID.
- **user** – The username used by this connection.

Query markers

Markers for query execution related events can prove very useful in monitor tasks, although they do not contain global information related to the entire system. At least two markers are essential: those corresponding to the start of the execution of a query and the end of the execution.

query_exec_start(*query_string*, *query_id*, *connection_id*, *user*)

This marker is triggered after the query execution has begun on the server.

Parameters:

- **query_string** – string representing the query to be executed
- **query_id** – Query identifier
- **connection_id** – Connection ID
- **user** – The username used by this connection

query_exec_end(*query_string*, *query_id*, *connection_id*, *user*, *status*)

This marker is triggered when the query execution process on the server has ended.

Parameters:

- **query_string** – string representing the query to be executed
- **query_id** – Query identifier
- **connection_id** – Connection ID
- **user** – The username used by this connection
- **status** – The status returned by the query execution (Success, Error)

Object operation markers

Operations involving the storage engine are critical and probing updates in a table at object level can greatly help monitor database activity. Markers will be triggered for each object inserted/updated/deleted, which may bring performance drawbacks on both the monitoring scripts and the server.

obj_insert_start(*table*)

This marker is triggered before an object is inserted.

Parameters:

table – Target table of the operation

obj_insert_end(*table*, *status*)

This marker is triggered after an object has been inserted.

Parameters:

- **table** – Target table of the operation
- **status** – Value showing whether the operation ended with success or not

obj_update_start(*table*)

This marker is triggered before an object is updated.

Parameters:

table – Target table of the operation

obj_update_end(*table*, *status*)

This marker is triggered after an object has been updated

Parameters:

- **table** – Target table of the operation
- **status** – Value showing whether the operation ended with success or not

obj_deleted_start(*table*)

This marker is triggered before an object is deleted.

Parameters:

table – Target table of the operation

obj_delete_end(*table*, *status*)

This marker is triggered after an object has been deleted.

Parameters:

- **table** – Target table of the operation
- **status** – Value showing whether the operation ended with success or not

Index operation markers

The object operation markers presented above are table-related, but below are index-related markers.

Indexes and their misuse can be the cause of many problems in a system and the possibility of monitoring them can be very helpful. The proposed markers are similar to those used for tables, since indexes support the same operations.

idx_insert_start(*classname*, *index_name*)

This marker is triggered before an insertion in the B-Tree.

Parameters:

- **classname** – Name of the class having the target index
- **index_name** – Target index of the operation

idx_insert_end(*classname*, *index_name*, *status*)

This marker is triggered after an insertion in the B-Tree.

Parameters:

- **classname** – Name of the class having the target index
- **index_name** – Target index_name of the operation
- **status** – Value showing whether the operation ended with success or not

idx_update_start(*classname*, *index_name*)

This marker is triggered before an update in the B-Tree.

Parameters:

- **classname** – Name of the class having the target index
- **index_name** – Target index_name of the operation

idx_update_end(*classname*, *index_name*, *status*)

This marker is triggered after an update in the B-Tree.

Parameters:

- **classname** – Name of the class having the target index
- **index_name** – Target index_name of the operation
- **status** – Value showing whether the operation ended with success or not

idx_delete_start(*classname*, *index_name*)

This marker is triggered before a deletion in the B-Tree.

Parameters:

- **classname** – Name of the class having the target index
- **index_name** – Target index_name of the operation

idx_delete_end(*classname*, *index_name*, *status*)

This marker is triggered after a deletion in the B-Tree.

Parameters:

- **classname** – Name of the class having the target index
- **index_name** – Target index_name of the operation
- **status** – Value showing whether the operation ended with success or not

Locking markers

Markers that involve locking events are perhaps the most important for global monitoring tasks. The locking system has a deep impact on the server performance and a comprehensive analysis on lock waiting times and count, number of deadlocks and aborted transactions is very useful in finding problems.

lock_acquire_start(*OID*, *table*, *type*)

This marker is triggered before a lock is requested.

Parameters:

- **OID** – Target object of the lock request.
- **table** – Table holding the object
- **type** – Lock type (X_LOCK, S_LOCK etc.)

lock_acquire_end(*OID*, *table*, *type*, *status*)

This marker is triggered after a lock request has been completed.

Parameters:

- **OID** – Target object of the lock request.
- **table** – Table holding the object

- **type** – Lock type (X_LOCK, S_LOCK etc.)
- **status** – Value showing whether the request has been granted or not.

lock_release_start(*OID*, *table*, *type*)

This marker is triggered before a lock is released.

Parameters:

- **OID** – Target object of the lock request.
- **table** – Table holding the object
- **type** – Lock type (X_LOCK, S_LOCK etc.)

lock_release_end(*OID*, *table*, *type*)

This marker is triggered after a lock release operation has been completed.

Parameters:

- **OID** – Target object of the lock request
- **table** – Table holding the object
- **type** – Lock type(X_LOCK, S_LOCK etc)

Transaction markers

Another interesting measure in server monitoring is transaction activity. A simple example: the number of transactions aborted is closely related to the number of deadlocks occurred, a very important performance indicator. Another straightforward use of such markers is the availability of a simple method of gathering system performance statistics such as TPS by using a simple SystemTap script.

tran_commit(*tran_id*)

This marker is triggered after a transaction completes successfully.

Parameters:

tran_id – Transaction identifier.

tran_abort(*tran_id*, *status*)

This marker is triggered after a transaction has been aborted.

Parameters:

- **tran_id** – Transaction identifier.
- **status** – Exit status.

tran_start(*tran_id*)

This marker is triggered after a transaction is started.

Parameters:

tran_id – Transaction identifier.

tran_deadlock()

This marker is triggered when a deadlock has been detected.

I/O markers

I/O access is the main bottleneck of a RDBMS and we should provide markers that allow the monitoring of I/O performance. The markers should be placed in a manner that will make it possible for user scripts to measure I/O page access time and aggregate various and complex statistics based on this measure.

pgbuf_hit()

This marker is triggered when a requested page was found in the page buffer and there is no need to retrieve it from disk.

pgbuf_miss()

This marker is triggered when a requested page was not found in the page buffer and it must be retrieved from disk.

io_write_start(*query_id*)

This marker is triggered when a the process of writing a page onto disk has begun.

Parameters:

query_id – Query identifier

io_write_end(*query_id*, *size*, *status*)

This marker is triggered when a the process of writing a page onto disk has ended.

Parameters:

- **query_id** – Query identifier
- **size** – number of bytes written
- **status** – Value showing whether the operation ended successfully or not

io_read_start(*query_id*)

This marker is triggered when a the process of reading a page from disk has begun.

Parameters:

query_id – Query identifier

io_read_end(*query_id*, *size*, *status*)

This marker is triggered when a the process of reading a page from disk has ended.

Parameters:

- **query_id** – Query identifier
- **size** – number of bytes read
- **status** – Value showing whether the operation ended successfully or not

Other markers

sort_start()

This marker is triggered when a sort operation is started.

sort_end(*nr_rows*, *status*)

This marker is triggered when a sort operation has been completed.

Parameters:

- **nr_rows** – number of rows sorted
- **status** – Value showing whether the operation ended successfully or not

cubrid scripts

unloaddb.sh script

To shorten the execution time of **unloaddb** of a large size database, it is possible to run **cubrid unloaddb** processes concurrently over groups of different tables. **unloaddb.sh** is a shell script that automates this process.

To evenly distribute unloaddb load over multiple child processes, the **unloadb.sh** gets the size information of the tables to be unloaded from the system catalog (this requires dba privileges).

Tables to be unloaded are allocated to each child process based on the table size, and each child process simultaneously unloads the allocated tables.

⚠ Warning

restrictions:

- Require Linux Environments
- Only users with **DBA privileges** can run this script

The following is [options] used in **unloaddb.sh**.

-u user

Specify a user account of a database to be unloaded. If this is not specified, the default is **DBA** (The **user** must have **DBA privileges**).

```
sh unloaddb.sh -u dba demodb
```

-t no-process

Number of child processes to run concurrently (default: 8, maximum 16)

```
sh unloaddb.sh -t 4 demodb
```

-i input-class-file

Unload schemas, indexes and data of specified tables in the file. (default: all tables in the database)

```
sh unloaddb.sh -i table_list.txt demodb
```

The following example shows an input file (table_list.txt).

```
public.table_1
public.table_2
..
public.table_n
```

-D output-directory

Specifies the directory in which to create schema and object files. If it is not specified, files are created in the current directory.

```
sh unloaddb.sh -D /tmp demodb
```

-s

Specifies only the schema file will be created (**schema only**).

```
sh unloaddb.sh -s demodb
```

-d

Specifies only the data file will be created (**data only**).

```
sh unloaddb.sh -d demodb
```

-v

Verbose mode. Show additional information during process (for example size of tables) (**verbose mode**).

```
sh unloaddb.sh -v demodb
```

Files/Directory created by unloaddb.sh

- single schema file for all tables unloaded: **{database name}_schema**
- **object files (as many as the number of child processes, suffix starts from 0)**
  - {database name}_0_objects, {database name}_1_objects, ..., {database name}_n-1_objects
- **LOG files / Directory:**
  - **{database name}_unloaddb.log** will be created.

- Various log-files will be created here.

unloaddb.sh example

```
sh unloaddb.sh -t 4 demodb
```

Files/Directory created by the result of this command:

```
* demodb_schema: schema file for all tables unloaded
* demodb_0_objects: object files unloaded by the 1st child process
* demodb_1_objects: object files unloaded by the 2nd child process
* demodb_2_objects: object files unloaded by the 3rd child process
* demodb_3_objects: object files unloaded by the 4th child process
* demodb_unloaddb.log/demodb_0.files: table names unloaded by 1st
child process
* demodb_unloaddb.log/demodb_0.pid: process id of 1st child process
* demodb_unloaddb.log/demodb_0_unloaddb.log: 'cubrid unloaddb' log
of the 1st child process
* demodb_unloaddb.log/demodb_0.status: execution result of the 1st
child process (success/fail)
```

Note

- If the interrupt key (**CTRL-C**) is pressed before termination of **unloaddb.sh**, all in-progress unloaddb **object files** will be **deleted** (object files that unload completed will not be deleted).

restore_to_newdb.sh script

When data recovery is required due to user errors or system failures, the recovery process typically involves restoring a backup to the production server. This often requires additional steps such as creating new user accounts or installing database engines, making the process cumbersome.

The restore_to_newdb.sh script allows you to restore a backup and create a new database instance under a new database name without any additional setup, using the existing CUBRID environment.

Warning

Restrictions:

- Only available on Linux environments.
- The backup files for the target database must exist.
- The new database name must differ from the original database name.
- The new database name must not already be registered in $CUBRID/databases.txt.

Basic Usage

```
sh restore_to_new.sh [options] backuped-database-name new-database-
name
```

- **restore_to_new.sh** : A script that restores a backup and creates a new DBMS instance.
- **backup-database-name** : Name of the backed-up database to be restored.
- **new-database-name** : Name of the newly created database after restoration.

[Options] for restore_to_new.sh

-F path

Specifies the absolute path for the new database. If not specified, the current path is used by default.

```
sh restore_to_newdb.sh -F /home/cubrid/newdbpath backuped-
database-name new-database-name
```

-B path

Specifies the absolute path where the backup files are located. If not specified, the current path is used by default.

```
sh restore_to_newdb.sh -B /home/cubrid/backupdbpath backuped-
database-name new-database-name
```

-d date

Restores the database up to a specific point in time. The format must be *dd-mm-yyyy:hh:mi:ss* (e.g., *14-10-2008:14:10:00*).

This option works the same as the *-d* option in *cubrid restoredb*.

```
sh restore_to_newdb.sh backuped-database-name new-database-name -
F /home/cubrid -B /home/cubrid/DB/backupdbpath -d 30-10-2025:12:
20:00
```

-l level

Specifies the backup level (0, 1, or 2) to restore from. This option function the same as the *-l* option in *cubrid restoredb*.

```
sh restore_to_newdb.sh -B /home/cubrid/DB/backupdbpath -l 1
backuped-database-name new-database-name
```

-p

Performs a partial restore when log archives are unavailable. Refer to the *-p* option in *cubrid restoredb* for more details.

```
sh restore_to_newdb.sh -B /home/cubrid/DB/backupdbpath -p
backuped-database-name new-database-name
```

-k path

Specifies the path of the key file required for restoration. Refer to the *-k* option in *cubrid restoredb* for more details.

```
sh restore_to_newdb.sh -B /home/cubrid/DB/backupdbpath -k /home/
cubdev/DB/backupdbpath/backupdb_bk1_keys backuped-database-name
new-database-name
```

Example of Running restore_to_newdb.sh

```
sh restore_to_newdb.sh -B /home/cubrid/DB/backupdbpath backuped-
database-name new-database-name
```

```
Validation passed: newdb_path and vol-path are different.
Validation passed: 'new-database-name' does not exist in /home/
cubrid/CUBRID-11.4-Linux.x86_64/databases/databases.txt.

Confirmed: Files found for pattern (/home/cubrid/DB/backupdbpath/
backuped-database-name_bk0v*).
file: /home/cubrid/DB/backupdbpath/backuped-database-name_bk0v*
Warning: /home/cubrid/CUBRID-11.4-Linux.x86_64/databases/
databases.txt already exists.
Existing databases.txt backed up as databases.txt.bak.
Restoring... /
```

```
CUBRID 11.4

Restoring... |Updated CUBRID-11.4-Linux.x86_64/databases/
databases.txt
successfully.
database restoration completed successfully.
```

Once the database is successfully restored, start the new DBMS as follows:

```
[cubrid ~]$ cubrid server start new-database-name
@ cubrid server start: new-database-name

This may take a long time depending on the amount of recovery works
to do.

CUBRID 11.4

++ cubrid server start: success
Calling java stored procedure is allowed
```

Note

- Since the recovery process using this script modifies $CUBRID_DATABASES, it is recommended to verify that the newly created database has been properly registered after the script finishes
- It is recommended to delete recovered DBMS instances that are no longer in use using the deletedb command.

Checking the Log File After Running restore_to_newdb.sh

After the restoration, a log file named restoredb_YYYYMMDDhhmiss.log will be created in the same directory where the script was executed. If any issues occur, you can check this log file to review the restoration history.

```
[cubrid ~]$ cat restoredb_20250411185128.log

[ Database(db1) Restore (level = 0) start ]

- restore start time: Fri Api 11 18:51:28 2025

- restore steps: 1
 step 1) restore using (level = 0) backup data
```

```
- restore progress status (using level = 0 backup data)

------------------------------------------------------------------------
--------
 volume name                  | # of pages |  restore progress
status  | done

------------------------------------------------------------------------
--------
 dbms_keys                    |          1 |
######################### | done
 dbms_vinf                    |          1 |
######################### | done
 dbms                         |      32768 |
######################### | done
 dbms_lgar001                 |      32768 |
######################### | done
 dbms_lgar002                 |      32768 |
######################### | done
 dbms_lginf                   |          1 |
######################### | done
 dbms_lgat                    |      32768 |
######################### | done

------------------------------------------------------------------------
--------

- restore end time: Fri Api 11 18:51:30 2025

[ Database(db1) Restore (level = 0) end ]
```

Troubleshooting

Checking SQL Log

SQL log of CAS

When a specific error occurs, generally you can check SQL logs of broker application server(CAS)

One SQL log file is generated per each CAS; it is hard to find an SQL log in which an error occurred because SQL log files are many when CAS processes are many. However, SQL log filename includes CAS ID in the end part, you can find easily if you know the CAS ID in which an error occurred.

Note

SQL log filename in a CAS is <broker_name>_<app_server_num>.sql.log(see Broker Logs); <app_server_num> is CAS ID.

Function getting CAS information

`cci_get_cas_info()` function or cubrid.jdbc.driver.CUBRIDConnection.toString() method in JDBC) prints out the information including the broker host and CAS ID in which the query is executed when the query is run; with this information, you can find an SQL log file of that CAS easily.

```
<host>:<port>,<cas id>,<cas process id>
e.g. 127.0.0.1:33000,1,12916
```

Application log

If you specify the connection URL for printing out the log in the application, you can check the CAS ID which brought an error when an error occurs in the specific query. The following are examples that an application log is written when an error occurs.

JDBC application log

```
Syntax: syntax error, unexpected IdName [CAS INFO - localhost:
33000,1,30560],[SESSION-16],[URL-jdbc:cubrid:localhost:
33000:demodb::********:?
logFile=driver_1.log&logSlowQueries=true&slowQueryThresholdMillis=5].
```

CCI application log

```
Syntax: syntax error, unexpected IdName [CAS INFO - 127.0.0.1:33000,
1, 30560].
```

Slow query

When a slow query occurs, you should find the reason of a slow query by an application log and an SQL log of CAS.

To find where the cause exists when slow query occurs(in the application-broker section? or in the broker-DB section?), you should check application log or SQL log of CAS because CUBRID is composed of 3-tiers; application-broker-DB server.

There is a slow query in the application log but that is not printed as slow query in the SQL log of CAS; then there will be a cause to make the speed low in the application-broker section.

Some examples are as below.

- Check if there is a low speed of the network between application and broker.
- Check if there is a case where CAS was restarted by checking the broker log (located in the **$CUBRID/log/broker** directory). If it is revealed that the number of CASes is insufficient (please tune for any slow queries first), you should increase the number of CASes. To do so, the value of MAX_NUM_APPL_SERVER should be increased appropriately. Also, increase the value of max_clients if needed.

If application log and CAS SQL log show the slow query log together and there is almost no gab between the slow query times of application log and the CAS SQL log, the cause which the query was slow will exist between the broker and DB server. For example, the query execution in the DB server was slow.

There are examples of each application log when a slow query occurs.

JDBC application log

```
2013-05-09 16:25:08.831|INFO|SLOW QUERY
[CAS INFO]
localhost:33000, 1, 12916
[TIME]
START: 2013-05-09 16:25:08.775, ELAPSED: 52
[SQL]
SELECT * from db_class a, db_class b
```

CCI application log

```
2013-05-10 18:11:23.023 [TID:14346] [DEBUG][CONHANDLE - 0002][CAS
INFO - 127.0.0.1:33000, 1, 12916] [SLOW QUERY - ELAPSED : 45] [SQL -
select * from db_class a, db_class b]
```

Slow query information in an application and in a broker is stored in each file when the setting is as following.

- The slow query information in application is stored in application log file when the value of **logSlowQueries** property in the connection URL is set to **yes** and the value of **slowQueryThresholdMillis** is set; it is stored to the application logfile specified with the **logFile** property (see `cci_connect_with_url()` and Configuration Connection).
- The slow query information in broker is stored in the $CUBRID/log/broker/sql_log directory when **SLOW_LOG** of Broker Configuration is set to ON and **LONG_QUERY_TIME** is set.

Server Error Log

You can get various information from the server error log by setting **error_log_level** parameter in cubrid.conf. The default of **error_log_level** is **NOTIFICATION**. For how to set this parameter, see Error Message-Related Parameters.

Detecting Overflow Keys or Overflow Pages

When overflow keys or overflow pages occur, **NOTIFICATION** messages are written to the server error log. Through this message, users can detect DB performance became slow because of overflow keys or overflow pages. If possible, overflow keys or overflow pages should not appear. That is, it is better not to use the index on the big size column, and not to define the record size largely.

```
Time: 06/14/13 19:23:40.485 - NOTIFICATION *** file ../../src/
storage/btree.c, line 10617 CODE = -1125 Tran = 1, CLIENT =
testhost:csql(24670), EID = 6
Created the overflow key file. INDEX idx(B+tree: 0|131|540) ON CLASS
hoo(CLASS_OID: 0|522|2). key: 'z ..... '(OID: 0|530|1).
...........

Time: 06/14/13 19:23:41.614 - NOTIFICATION *** file ../../src/
storage/btree.c, line 8785 CODE = -1126 Tran = 1, CLIENT =
testhost:csql(24670), EID = 9
Created a new overflow page. INDEX i_foo(B+tree: 0|149|580) ON CLASS
foo(CLASS_OID: 0|522|3). key: 1(OID: 0|572|578).
...........

Time: 06/14/13 19:23:48.636 - NOTIFICATION *** file ../../src/
storage/btree.c, line 5562 CODE = -1127 Tran = 1, CLIENT =
testhost:csql(24670), EID = 42
Deleted an empty overflow page. INDEX i_foo(B+tree: 0|149|580) ON
CLASS foo(CLASS_OID: 0|522|3). key: 1(OID: 0|572|192).
```

Detecting log recovery time

When DB sever is started or backup volume is restored, you can check the duration of the log recovery by printing out the **NOTIFICATION** messages, the starting time and the ending time of the log recovery, to the server error log or an error log file of restoredb. In these messages, the number of logs and the number of log pages to redo are written together.

```
Time: 06/14/13 21:29:04.059 - NOTIFICATION *** file ../../src/
transaction/log_recovery.c, line 748 CODE = -1128 Tran = -1, EID = 1
Log recovery is started. The number of log records to be applied:
96916. Log page: 343 ~ 5104.
.....
```

```
Time: 06/14/13 21:29:05.170 - NOTIFICATION *** file ../../src/
transaction/log_recovery.c, line 843 CODE = -1129 Tran = -1, EID = 4
Log recovery is finished.
```

Detecting a Deadlock

Locks related information is written to the server error log. The server error log file is saveed in the **$CUBRID/log/server** directory as named in *<databases>_<date>_<time>.err*.

```
demodb_20160202_1811.err

      ...

Your transaction (index 1, public@testhost|csql(21541)) timed out
waiting on    X_LOCK lock on instance 0|650|3 of class t because of
deadlock. You are waiting for user(s) public@testhost|csql(21529) to
finish.

      ...
```

Detecting the change of HA status

Detecting the change of HA status can be checked in the cub_master process log. This log file is stored in the **$CUBRID/log** directory as named in *<host_name>.cub_master.err*.

Detecting HA split-brain

When there is an abnormal status that two or more nodes are in charge of master role in HA environment, we call it “split-brain”.

To resolve the split-brain status, one of two is dead for itself; cub_master log file of this node includes the following information.

```
Time: 01/10/25 13:46:34.782 - ERROR *** file ../../src/executables/
master_heartbeat.c, line 873 ERROR CODE = -988, Tran = -1, EID = 35
Node event: [Failback] [Diagnosis] Multiple master nodes
(testhost02, testhost01) are detected.

Time: 01/10/25 13:46:34.782 - ERROR *** file ../../src/executables/
master_heartbeat.c, line 876 ERROR CODE = -988, Tran = -1, EID = 36
Node event: HA Node Info
============================================================================
===========
* group_id : hagrp   host_name : testhost02   state : master
----------------------------------------------------------------------------
-----------
```

```
name                priority   state          score      missed
heartbeat
------------------------------------------------------------------------
-----------
testhost02              2          master          -32766     0
testhost01              1          master          -32767     0
========================================================================
===========

Time: 01/10/25 13:46:34.782 - ERROR *** file ../../src/executables/
master_heartbeat.c, line 889 ERROR CODE = -988, Tran = -1, EID = 37
Node event: HA Ping Host Info
========================================================================
===========
* PING check is enabled
------------------------------------------------------------------------
-----------
hostname            PING check result
------------------------------------------------------------------------
-----------
8.8.8.8             SUCCESS
========================================================================
===========

Time: 01/10/25 13:46:34.782 - ERROR *** file ../../src/executables/
master_heartbeat.c, line 1371 ERROR CODE = -988, Tran = -1, EID = 38
Node event: [Failback] [Success] Current node has been successfully
demoted to slave.
```

Above example is the information to print out into the cub_master log when testhost02 server detects split-brain status and it is dead for itself.

Detecting Failover, Failback

If failover or failback occurs, a node changes its role.

The following is the log file of the cub_master that changes to a slave node after failback or to a master node after failover; log messages for failover and failback are prefixed with [Failover] or [Failback].

The log messages for failover and failback include both diagnosis and result messages.

A diagnosis message is logged when a failover or failback is triggered for a specific reason. These messages indicate the reason why the failover or failback was initiated and are prefixed with [Diagnosis].

A result message is logged when a failover or failback is either canceled or successfully completed. If the failover or failback is canceled, the result message logs the reason for the cancellation and is prefixed with [Cancelled]. If the failover or failback is completed successfully,

the result message logs the outcome and is prefixed with [Success]. For details, see Determine the Cause of Failover.

```
Time: 01/09/25 16:14:13.005 - ERROR *** file ../../src/executables/
master_heartbeat.c, line 924 ERROR CODE = -988, Tran = -1, EID = 6
Node event: [Failover] [Diagnosis] The master node (testhost01) has
lost its role due to server process problem, such as disk failure.

Time: 01/09/25 16:14:13.614 - ERROR *** file /../../src/executables/
master_heartbeat.c, line 1179 ERROR CODE = -988, Tran = -1, EID = 7
Node event: [Failover] [Success] Current node has been successfully
promoted to master.

Time: 01/09/25 16:14:13.614 - ERROR *** file ../../src/executables/
master_heartbeat.c, line 1196 ERROR CODE = -988, Tran = -1, EID = 8
Node event: HA Node Info
========================================================================
===========
* group_id : hagrp   host_name : testhost02   state : master
------------------------------------------------------------------------
-----------
name                priority   state           score      missed
heartbeat
------------------------------------------------------------------------
-----------
testhost02          2          to-be-master    -4094      0
testhost01          1          unknown         32767      1
========================================================================
===========
```

The above example shows information which is printed in the cub_master log; it describes the process where the 'testhost02' host changes its role from slave to master during a failover because the master node 'testhost01' has lost its role due to server process issue.

Failure on HA Start

The following is examples that replicated DB volumes' restoration is impossible without user intervention.

- When logs to copy in copylogdb process are deleted from a source node.
- When archive logs to apply from active server are already deleted.
- When a restoration of server is failed.

When replicated DB volumes' restoration is impossible like above cases, **"cubrid heartbeat start"** command is failed; for each case, you should fix it properly.

Typical Unrestorable Failure

If server process is the cause of the cases that automatic restoration of DB volumes without user intervention is impossible, that cases will be very various, so descriptions for those are omitted. The following describes the error messages when **copylogdb** or **applylogdb** process is the cause.

- When **copylogdb** process is the cause

Cause	Error message
A log not copied yet is already deleted from the target node.	log writer: failed to get log page(s) starting from page id 80.
Detected as the other DB's log.	Log "/home1/cubrid/DB/tdb01_cdbs037.cub/tdb01_lgat" does not belong to the given database.

- When **applylogdb** process is the cause

Cause	Error message
Archive logs including logs to apply in replication are already deleted.	Internal error: unable to find log page 81 in log archives. Internal error: logical log page 81 may be corrupted.
Different between db_ha_apply_info catalog time and DB creation time in the current replication logs. That is, it's not the previous log to be being applied.	HA generic: Failed to initialize db_ha_apply_info.
Different database locale.	Locale initialization: Active log file(/home1/cubrid/DB/tdb01_cdbs037.cub/tdb01_lgat) charset is not valid (iso88591), expecting utf8.

How to fix when a Failure on HA start

Status	How to fix
When the source node, the cause of failure, is in master status.	Rebuild replication.
When the source node, the cause of failure, is in slave status.	Initialize replicated logs and db_ha_apply_info catalog then restart.

DDL Audit Log

Overview

CUBRID has the capability of recording DDL (Data Definition Language) that changes the database system configuration such as create/delete/modify tables as well as changing the access privilege of a table. DDLs issued through CAS, csql, and loaddb could be recorded in log files in addition to the copy of the files executed if required.

The DDL Audit log will be created in the $CUBRID/log/ddl_audit directory when ddl_audit_log of the system parameter is turned on. Note that the size of each log file cannot exceed the value specified in the ddl_audit_log_size parameter. For parameters related to DDL audit, refer to system parameters of CUBRID Managment System Parameters .

Note

If you use a method from the System Catalog , the DDL Audit Log will not be created.

DDL Audit Log filename convention

- cas: {broker_name}_{app_server_num}_ddl.log
- csql interactive: csql_{db_name}_ddl.log
- loaddb: loaddb_{db_name}_ddl.log

Additional filename convention:

- csql from file: csql/{csql_file}_{YYYYMMDD}_{HHMMSS}_{pid}

- loaddb: loaddb/{loaddb_file}_{YYYYMMDD}_{HHMMSS}_{pid}

DDL Audit Logfile format of CAS

- [Time] [ip_addr]|[db_name]|[user_name]|[result]|[elapsed time]|[auto commit/rollback]|[sql_text]

  Description:

  - [Time]: Time starting execution of the DDL (e.g. 20-12-18 12:08:32.327)
  - [db_name]: Database name where DDL was executed
  - [ip_addr]: An IP address of an application client (e.g. 172.31.0.70)
  - [user_name]: the database username who issued DDL
  - [result]: statement execution result. OK if successful, otherwise error code (e.g. ERROR:-494)
  - [elapsed time]: Elapsed time of statement execution
  - [auto commit/rollback]: Automatically committed or rolled back, with the error code of it
  - [sql_text]: executed DDL text

DDL Audit Logfile format of CSQL

- [Time] [pid]|[user_name]|[result]|[elapsed time]|[auto commit/rollback]|[sql_text]

  Description:

  - [Time]: Time starting execution of the DDL (e.g. 20-11-20 13:26:51.765)
  - [pid]: csql process id
  - [user_name]: the database username who issued DDL
  - [result]: statement execution result. OK if successful, otherwise error code (e.g. ERROR:-272)
  - [elapsed time]: Elapsed time of statement execution
  - [auto commit/rollback]: Automatically committed or rolled back, with the error code
  - [sql_text]: executed DDL text or executed csql filename

DDL Audit Log format of LOADDB

- [Time] [pid]|[user_name]|[result]|[log contents]|[filename]

  Description:

  - [Time]: Time starting execution of the DDL (e.g. 20-12-18 12:08:32.327)
  - [pid]: loaddb process id
  - [user_name]: the database username who issued DDL
  - [result]: Result of loaddb execution, OK if successful, otherwise error code (e.g. ERROR:-494)
  - [log contents]: The number of total statements, or the number of commit in case of error and error line
  - [filename]: Copy the file loaded from loaddb. At this time, it is the name of the copied file

CUBRID HA

High Availability (HA) refers to a feature to provide uninterrupted service in the event of hardware, software, or network failure. This ability is a critical element in the network computing area where services should be provided 24/7. An HA system consists of more than two server systems, each of which provides uninterrupted services, even when a failure occurs in one of them.

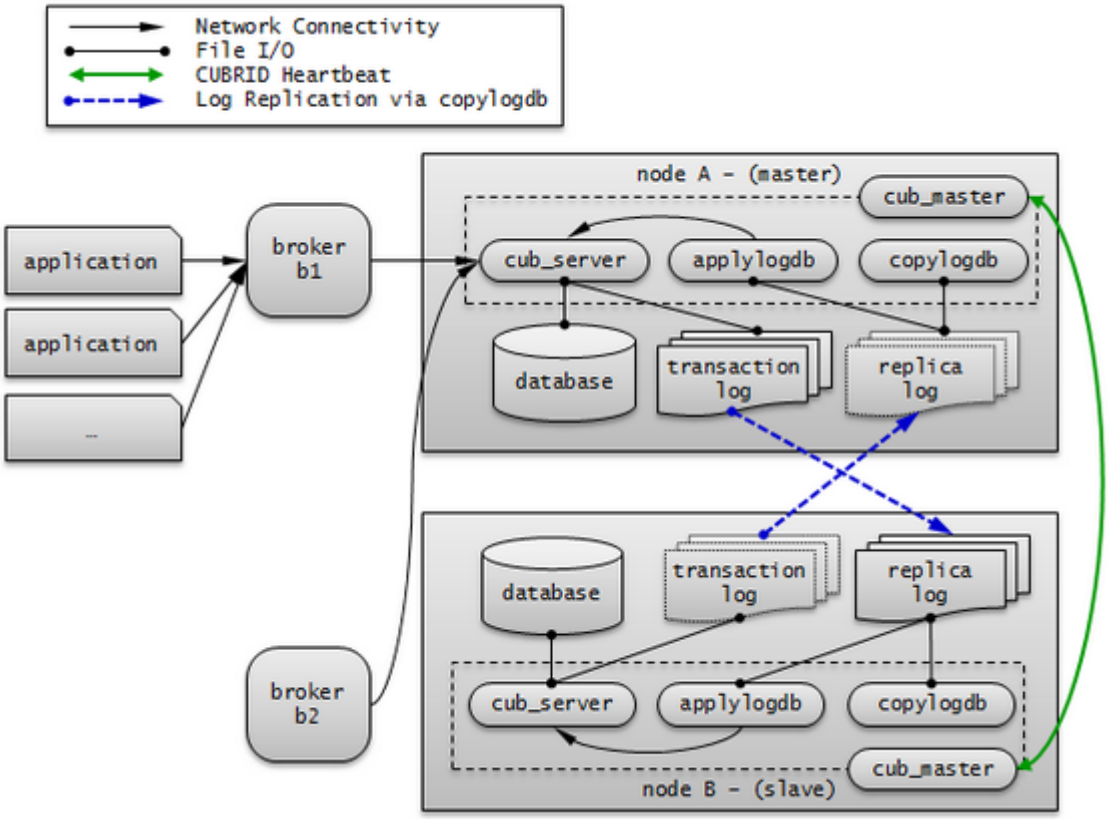
CUBRID HA is an implementation of High Availability. CUBRID HA ensures database synchronization among multiple servers when providing service. When an unexpected failure occurs in the system which is operating services, this feature minimizes the service down time by allowing the other system to carry out the service automatically.

CUBRID HA is in a shared-nothing structure. To synchronize data from an active server to a standby server, CUBRID HA executes the following two steps.

1. Transaction log multiplexing: Replicates the transaction logs created by an active server to another node in real time.
2. Transaction log reflection: Analyzes replicated transaction logs in real time and reflects the data to a standby server.

CUBRID HA executes the steps described above in order to always maintain data synchronization between an active server and a standby server. For this reason, if an active server is not working

properly because of a failure occurring in the master node that had been providing service, the standby server of the slave node provides service instead of the failed server. CUBRID HA monitors the status of the system and CUBRID in real time. It uses heartbeat messages to execute an automatic failover when a failure occurs.



CUBRID HA Concept

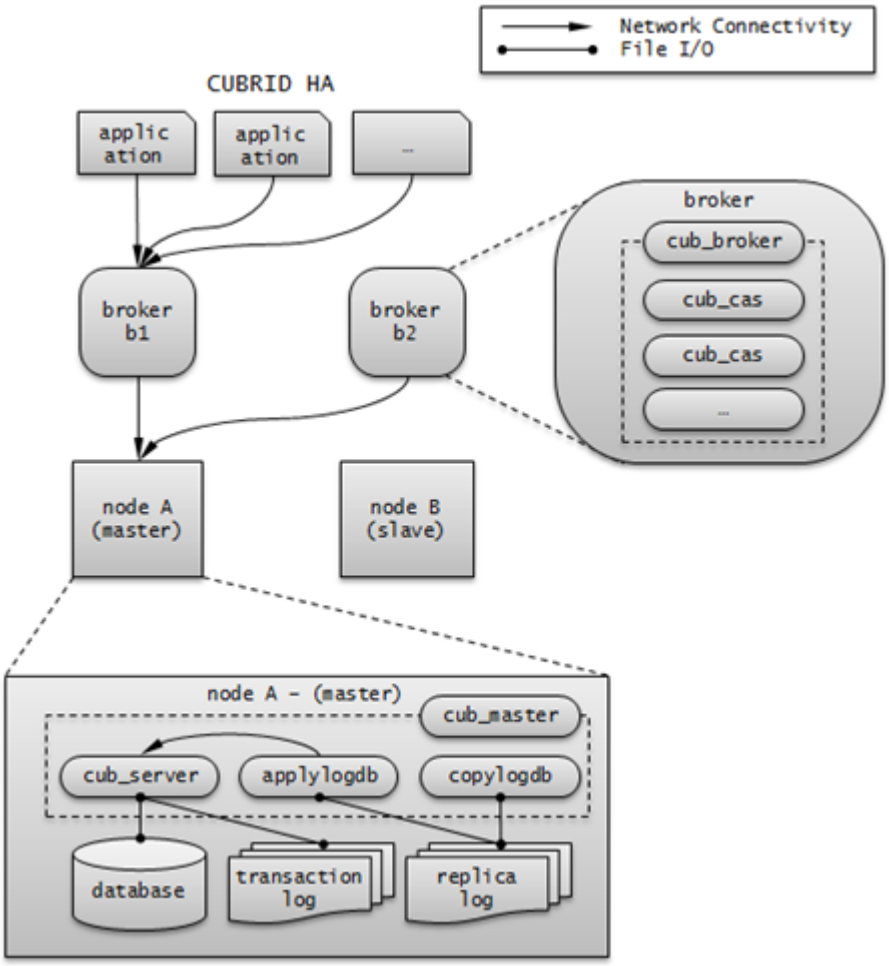
Nodes and Groups

A node is a logical unit that makes up CUBRID HA. It can become one of the following nodes according to its status: master node, slave node, or replica node.

- **Master node** : A node to be replicated. It provides all services which are read, write, etc. using an active server.
- **Slave node** : A node that has the same information as a master node. Changes made in the master node are automatically reflected to the slave node. It provides the read service using a standby server, and a failover will occur when the master node fails.
- **Replica node** : A node that has the same information as a master node. Changes made in the master node are automatically reflected to the replica node. It provides the read service using a standby server, and no failover will occur when the master node fails.

The CUBRID HA group consists of the nodes described above. You can configure the members of this group by using the **ha_node_list** and **ha_replica_list** in the **cubrid_ha.conf** file. Nodes in a group have the same information. They exchange status checking messages periodically and a failover will occurs when the master node fails.

A node includes the master process (cub_master), the database server process (cub_server), the replication log copy process (copylogdb), the replication log reflection process (applylogdb), etc.



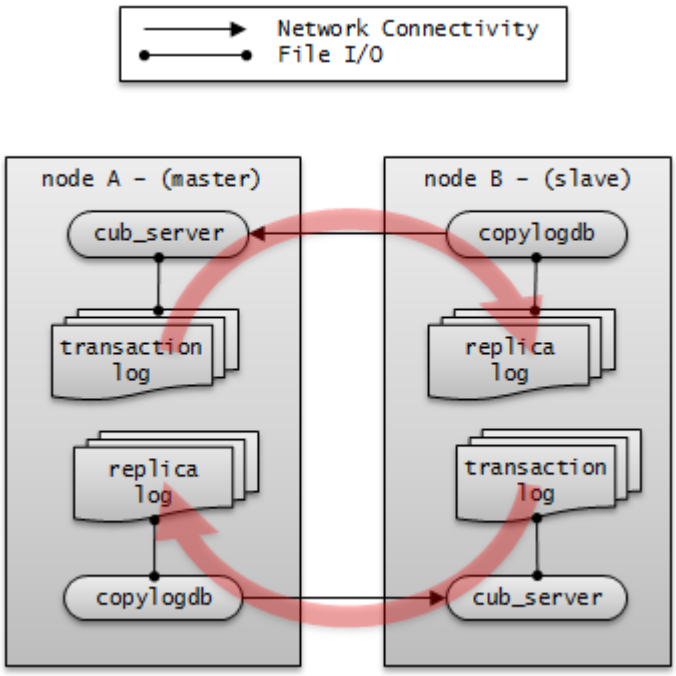
Processes

A CUBRID HA node consists of one master process (cub_master), one or more database server processes (cub_server), one or more replication log copy processes (copylogdb), and one or more replication log reflection processes (applylogdb). When a database is configured, database server processes, replication log copy processes, and replication log reflection processes will start. Because copy and reflection of a replication log are executed by different processes, the delay in replicating reflections does not affect the transaction that is being executed.

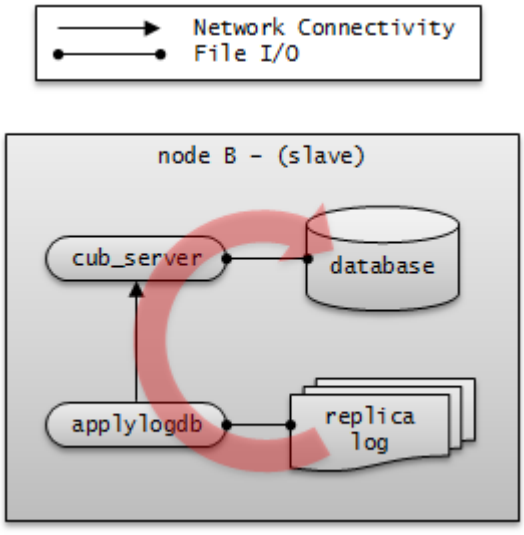
- **Master process (cub_master)** : Exchanges heartbeat messages to control the internal management processes of CUBRID HA.

- **Database server process (cub_server)** : Provides services such as read or write to the user. For details, see Servers.

- **Replication log copy process (copylogdb)** : Copies all transaction logs in a group. When the replication log copy process requests a transaction log from the database server process of the target node, the database server process returns the corresponding log. The location of copied transaction logs can be configured in the **ha_copy_log_base** of **cubrid_ha.conf**. Use applyinfo utility to verify the information of copied replication logs. The replication log copy process has following two modes: SYNC and ASYNC. You can configure it with the **ha_copy_sync_mode** of **cubrid_ha.conf**. For details on these modes, see Log Multiplexing.



- **Replication log reflection process (applylogdb)** : Reflects the log that has been copied by the replication log copy process to a node. The information of reflected replications is stored in the internal catalog (db_ha_apply_info). You can use the applyinfo utility to verify this information.

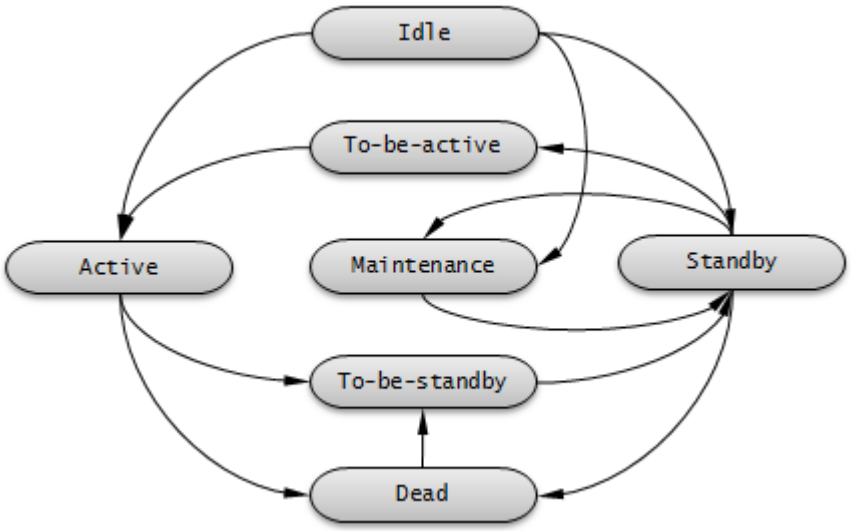


Servers

Here, the word “server” is a logical representation of database server processes. Depending on its status, a server can be either an active server or a standby server.

- **Active server** : A server that belongs to a master node; the status is active. An active server provides all services, including read, write, etc. to the user.
- **Standby server** : A standby server that belongs to a non-master node; the status is standby. A standby server provides only the read service to the user.

The server status changes based on the status of the node. You can use the cubrid changemode utility to verify server status. The maintenance mode exists for operational convenience and you can change it by using the **cubrid changemode** utility.



- **active** : The status of servers that run on a master node is usually active. In this status, all services including read, write, etc. are provided.
- **standby** : The status of servers that run on a slave node or a replica node is standby. In this status, only the read service is provided.
- **maintenance** : The status of servers can be manually changed for operational convenience is maintenance. In this status, only a csql can access and no service is provided to the user.
- **to-be-active** : The status in which a standby server will become active for reasons such as failover, etc. is to-be-active. In this status, servers prepare to become active by reflecting transaction logs from the existing master node to its own server. The node in this status can accept only SELECT query.
- Other : This status is internally used.

When the node status is changed, on cub_master process log and cub_server process log, following error messages are saved. But, they are saved only when the value of **error_log_level** in **cubrid.conf** is **error** or less.

- The following log information of cub_master process is saved on **$CUBRID/log/***<hostname>***_master.err** file.

```
HA generic: Send changemode request to the server. (state:
1[active], args:[cub_server demodb ], pid:25728).
HA generic: Receive changemode response from the server. (state:
1[active], args:[cub_server demodb ], pid:25728).
```

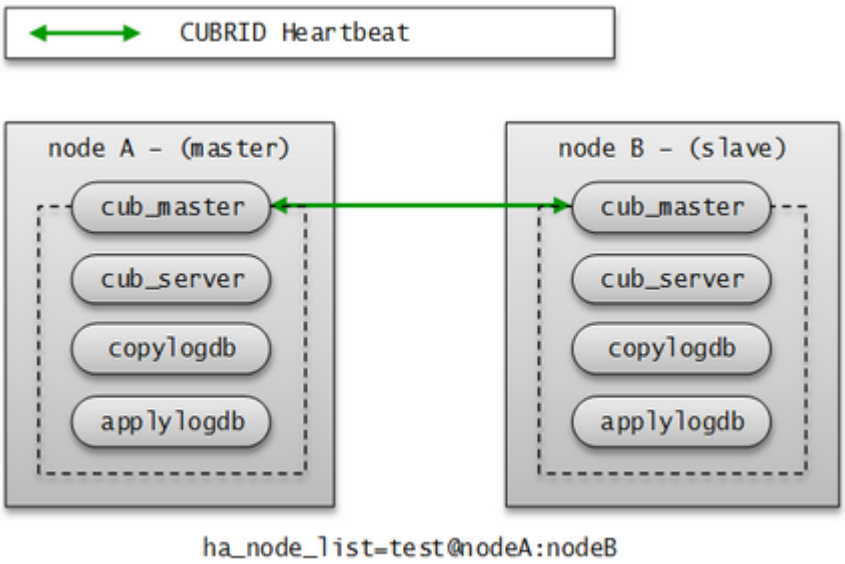
- The following log information of cub_server is saved on **$CUBRID/log/server/***<db_name>_<date>_<time>***.err** file.

```
Server HA mode is changed from 'to-be-active' to 'active'.
```

heartbeat Message

As a core element to provide HA, it is a message exchanged among master, slave, and replica nodes to monitor the status of other nodes. A master process periodically exchanges heartbeat messages with all other master processes in the group. A heartbeat message is exchanged through the UDP port configured in the **ha_port_id** parameter of **cubrid_ha.conf**. The exchange interval of heartbeat messages is determined by an internally configured value.

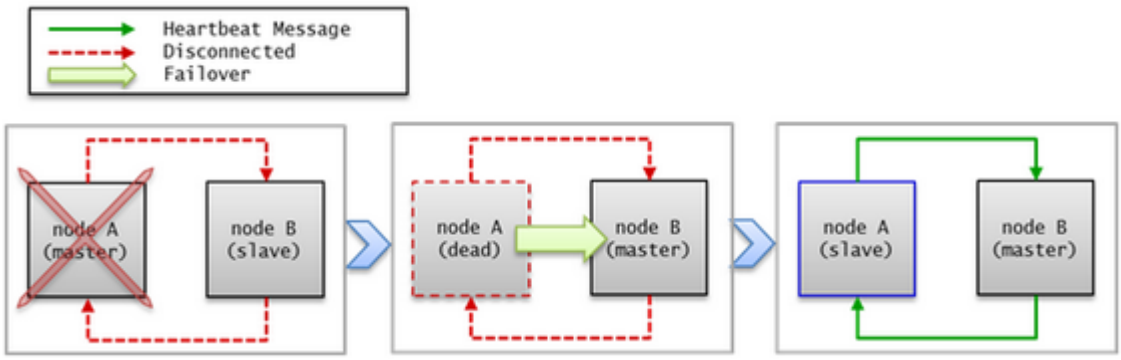
When the master node fails, a failover occurs to a slave node.



failover and failback

A failover means that the highest priority slave node automatically becomes a new master node when the original master node fails to provide services due to a failure. A master process calculates scores for all nodes in the CUBRID HA group based on the collected information, promotes slave nodes to master modes when it is necessary, and then notifies the management process of the changes it has made.

A failback means that the previously failed master node automatically becomes a master node back after the failure node is restored. The CUBRID HA does not currently support this functionality.



If a heartbeat message fails to deliver, a failover will occur. For this reason, servers with unstable connection may experience failover even though no actual failures occur. To prevent a failover from occurring in the situation described above, configure **ha_ping_hosts**. Configuring **ha_ping_hosts** will send a ping message to the hosts specified in **ha_ping_hosts** in order to verify whether the network is stable or not when a heartbeat message fails to deliver. For details on configuring **ha_ping_hosts**, see cubrid_ha.conf.

Broker Mode

A broker can access a server with one of the following modes: **Read Write**, **Read Only** or **Standby Only**. This configuration value is determined by a user.

A broker finds and connects to a suitable DB server by trying to establish a connection in the order of DB server connections; this is, if it fails to establish a connection, it tries another connection to the next DB server defined until it reaches the last DB server. If no connection is made even after trying all servers, the broker fails to connect to a DB server.

For details on how to configure broker mode, see cubrid_broker.conf.

DB connection is affected by **PREFERRED_HOSTS**, **CONNECT_ORDER** and **MAX_NUM_DELAYED_HOSTS_LOOKUP** parameters in **cubrid_broker.conf**. See Connecting a Broker to DB for further information.

The below is the description if the above parameters are not specified.

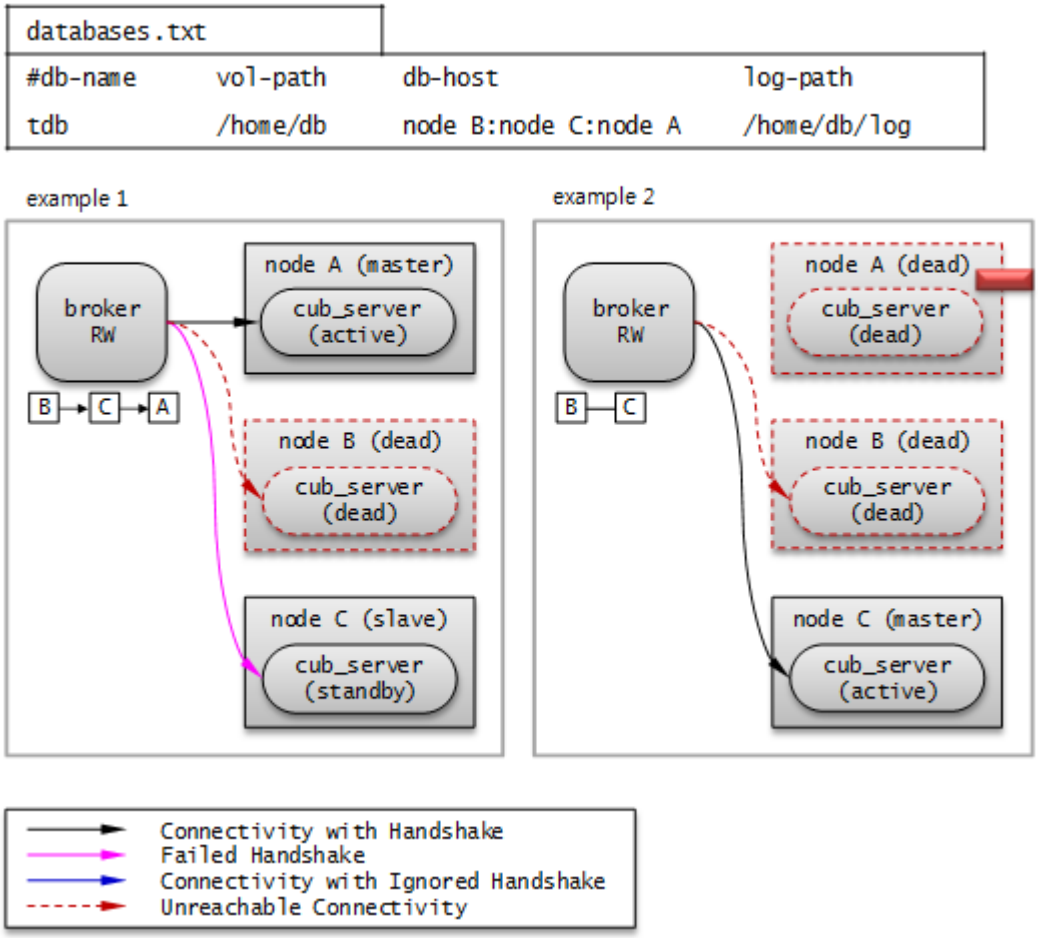
Read Write

"ACCESS_MODE=RW"

A broker that provides read and write services. This broker is usually connected to an active server. If there is no active server, this broker will be connected to a standby server temporarily. Therefore, a Read Write broker can be temporarily connected to a standby server.

When the broker temporarily establishes a connection to a standby server, it will disconnect itself from the standby server at the end of every transaction so that it can attempt to find an active server at the beginning of the next transaction. When it is connected to the standby server, only read service is available. Any write requests will result in a server error.

The following picture shows how a broker connects to the host through the **db-host** configuration.



The broker tries to connect as the order of *B*, *C*, *A* because db-host in databases.txt is "node B:node C:node A". At this time, "node B:node C:node A" specified in db-host is the real hostnames defined in the /etc/hosts file.

- Example 1. *node B* is crashed, *node C* is in standby status, and *node A* is in active status. Therefore, at last, the broker connects to *node A*.
- Example 2. *node B* is crashed, and *node C* is in active status. Therefore, at last, the broker connects to *node C*.

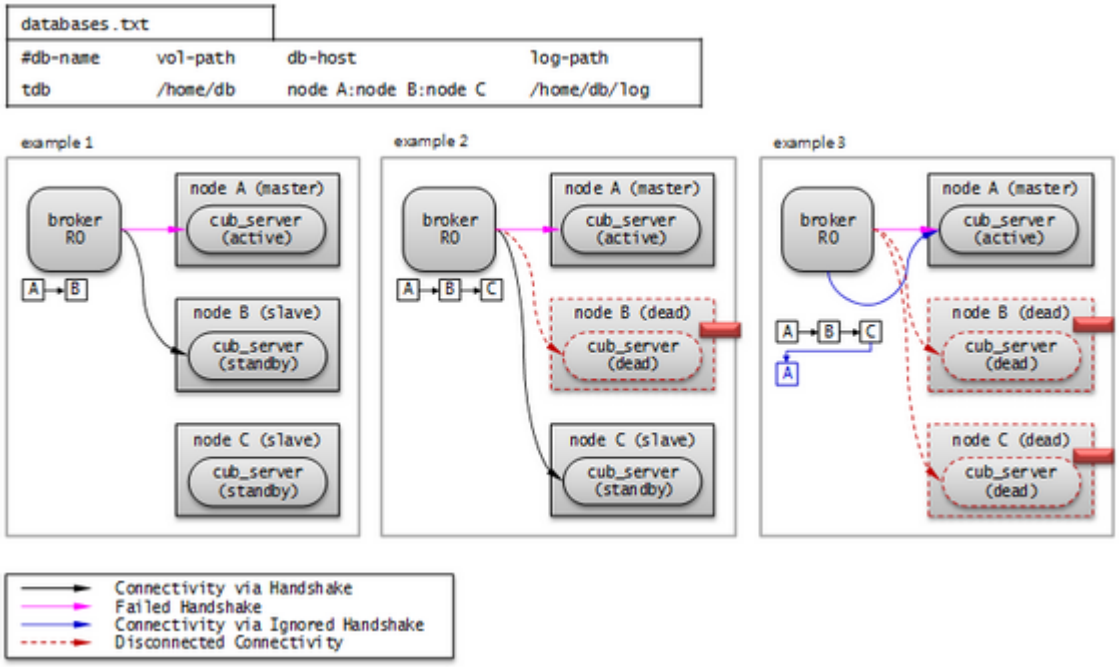
Read Only

"ACCESS_MODE=RO"

A broker that provides the read service. This broker is connected to a standby server if possible. Therefore, the Read Only broker can be connected to an active server temporarily.

Once it establishes a connection with an active server, it will maintain that connection until the time specified by RECONNECT_TIME. After RECONNECT_TIME, the broker tries to reconnect as disconnecting the old connection. Or you can reconnect to the standby server by running **cubrid broker reset**. If a write request is delivered to the Read Only broker, an error occurs in the broker; therefore, only the read service will be available even if it is connected to an active server.

The following picture shows how a broker connects to the host through the **db-host** configuration.



The broker tries to connect as the order of *A*, *B*, *C* because db-host in databases.txt is "node A:node B:node C". At this time, "node A:node B:node C" specified in db-host is the real hostnames defined in the /etc/hosts file.

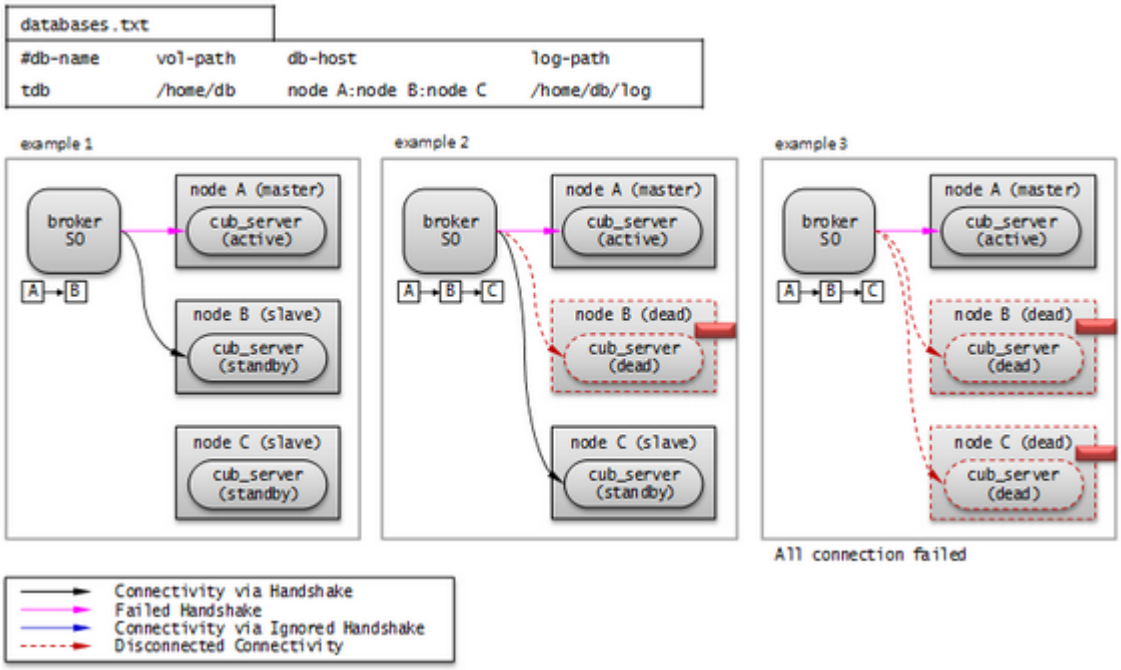
- Example 1. *node A* is in active status, *node B* is in standby status. Therefore, at last, the broker connects to *node B*.
- Example 2. *node A* is in active status, *node B* is crashed, and *node C* is in standby status. Therefore, at last, the broker connects to *node C*.
- Example 3. *node A* is in active status, *node B* and *node C* are crashed. Therefore, at last, the broker connects to *node A*.

Standby Only

"ACCESS_MODE=SO"

A broker that provides the read service. This broker can only be connected to a standby server. If no standby server exists, no service will be provided.

The following picture shows how a broker connects to the host through the **db-host** configuration.



The broker tries to connect as the order of *A*, *B*, *C* because db-host in databases.txt is “node A:node B:node C”. At this time, “node A:node B:node C” specified in db-host is the real hostnames defined in the /etc/hosts file.

- Example 1. *node A* is in active status, *node B* is in standby status. Therefore, at last, the broker connects to *node B*.
- Example 2. *node A* is in active status, *node B* is crashed, and *node C* is in standby status. Therefore, at last, the broker connects to *node C*.
- Example 3. *node A* is in active status, *node B* and *node C* are crashed. Therefore, at last, the broker does not connect to any node. This is the difference with Read Only broker.

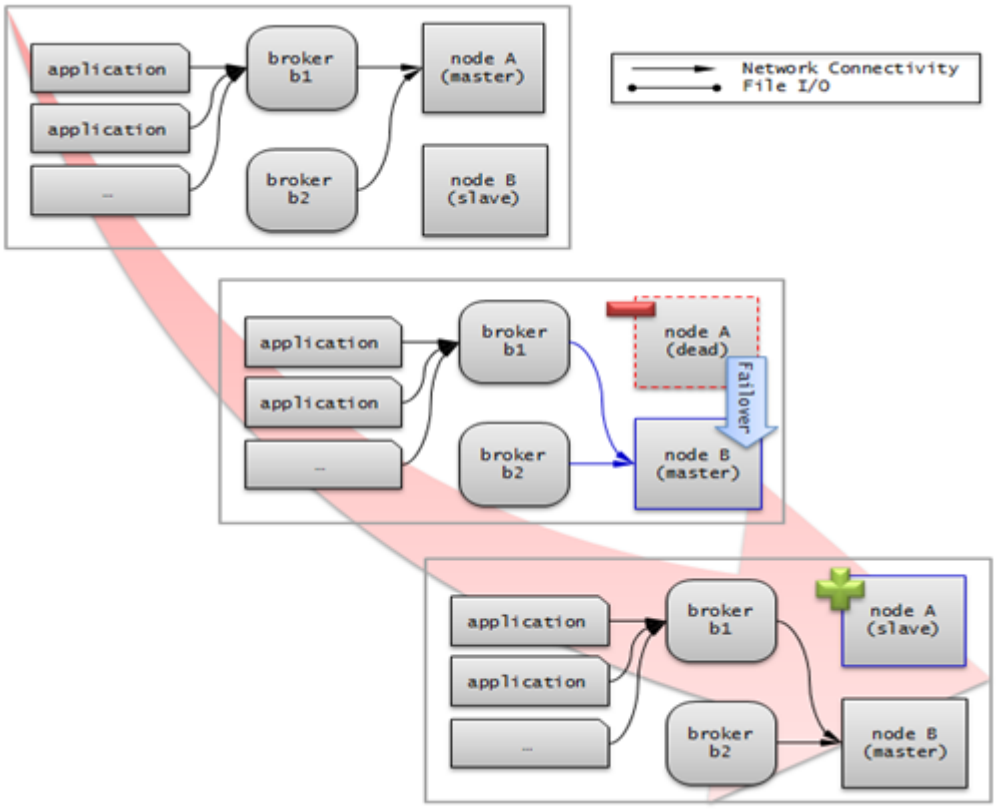
CUBRID HA Features

Duplexing Servers

Duplexing servers is building a system by configuring duplicate hardware equipment to provide CUBRID HA. This method will prevent any interruptions in a server in case of occurring a hardware failure.

Server failover

A broker defines server connection order and connects to a server according to the defined order. If the connected server fails, the broker connects to the server with the next highest priority. This requires no processing in the application side. The actions taken when the broker connects to another server may differ according to the current mode of the broker. For details on the server connection order and configuring broker mode, see cubrid_broker.conf.



Server failback

CUBRID HA does not automatically support server failback. Therefore, to manually apply failback, restore the master node that has been abnormally terminated and run it as a slave node, terminate the node that has become the master from the slave due to failover, and finally, change the role of each node again.

For example, when *nodeA* is the master and *nodeB* is the slave, *nodeB* becomes the master and *nodeA* becomes the slave after a failover. After terminating *nodeB* (**cubrid heartbeat stop**) check (**cubrid heartbeat status**) whether the status of *nodeA* has become active. Start (**cubrid heartbeat start**) *nodeB* and it will become the slave.

Duplexing Brokers

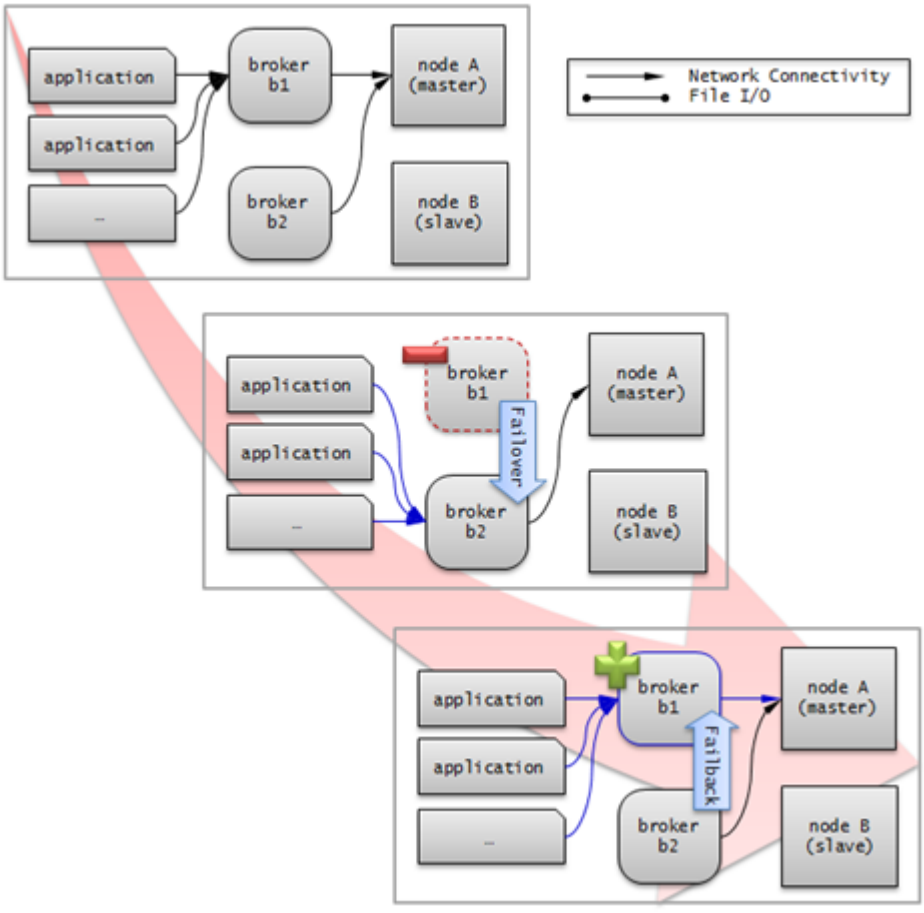
As a 3-tier DBMS, CUBRID has middleware called the broker which relays applications and database servers. To provide HA, the broker also requires duplicate hardware equipment. This method will prevent any interruptions in a broker in case of occurring a hardware failure.

The configuration of broker redundancy is not determined by the configuration of server redundancy; it can be user-defined. In addition, it can be separated by piece of individual equipment.

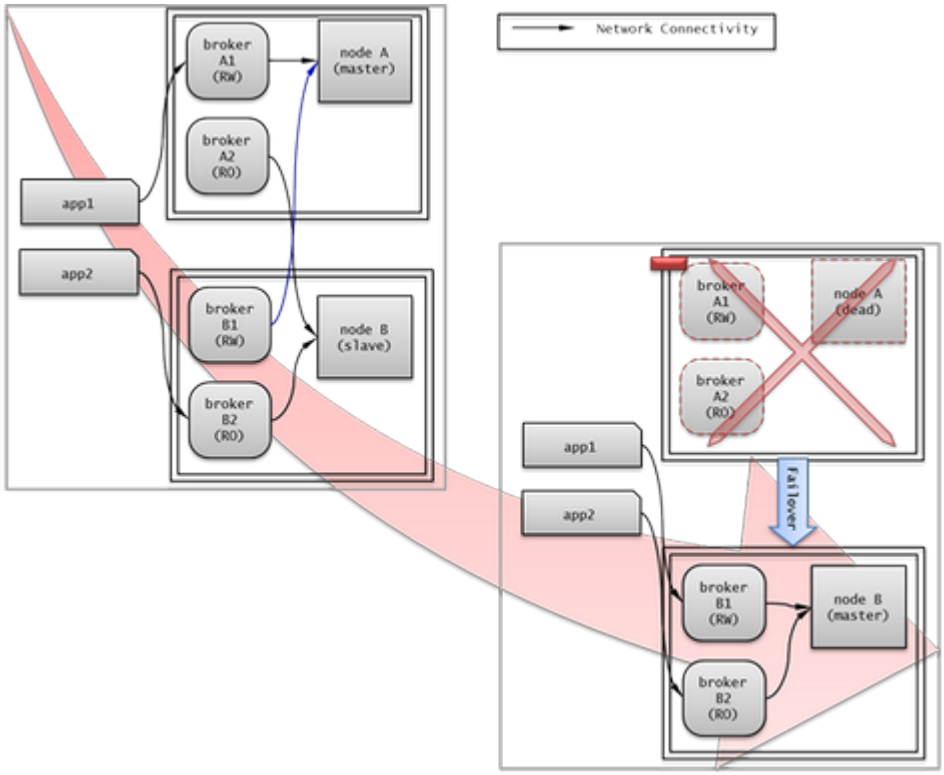
To use the failover and failback functionalities of a broker, the **altHosts** attribute must be added to the connection URL of the JDBC, CCI, or PHP. For a description of this, see JDBC Configuration, CCI Configuration and PHP Configuration.

To set a broker, configure the **cubrid_broker.conf** file. To set the order of failovers of a database server, configure the **databases.txt** file. For more information, see Configuring and Starting Broker, and Verifying the Broker Status.

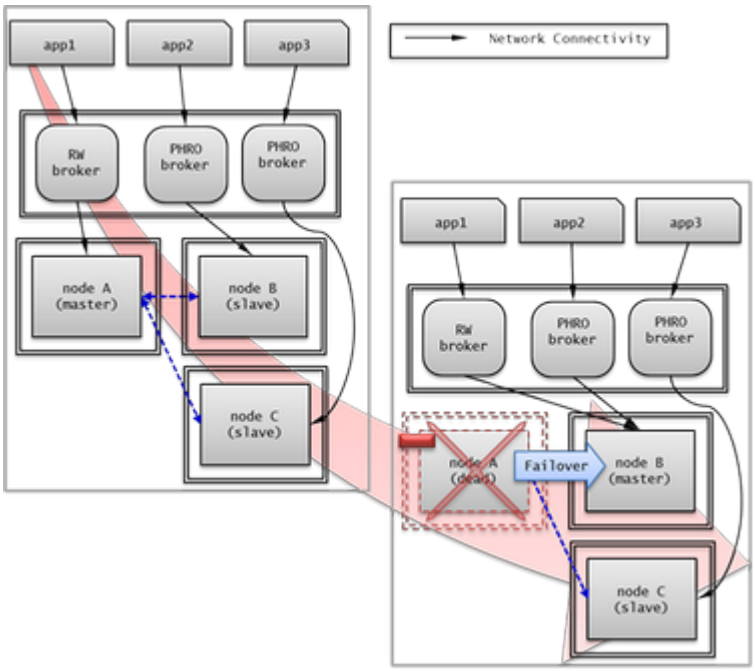
The following is an example in which two Read Write (RW) brokers are configured. When the first connection broker of the application URL is set to *broker B1* and the second connection broker to *broker B2*, the application connects to *broker B2* when it cannot connect to *broker B1*. When *broker B1* becomes available again, the application reconnects to *broker B1*.



The following is an example in which the Read Write (RW) broker and the Read Only (RO) broker are configured on each piece of equipment for the master node and the slave node. First, the *app1* and *app2* URLs connect to *broker A1* (RW) and *broker B2* (RO), respectively. The second connection (altHosts) is made to *broker A2* (RO) and *broker B1* (RW). When the equipment that includes *nodeA* fails, *app1* and *app2* connect to the broker that includes *nodeB*.



The following is an example of a configuration in which broker equipment includes one Read Write broker (master node) and two Preferred Host Read Only brokers (slave nodes). The Preferred Host Read Only brokers are connected to nodeB and nodeC to distribute the reading load.



Broker failover

The broker failover is not automatically failed over by the settings of system parameters. It is available in the JDBC, CCI, and PHP applications only when broker hosts are configured in the **altHosts** of the connection URL. Applications connect to the broker with the highest priority. When the connected broker fails, the application connects to the broker with the next highest

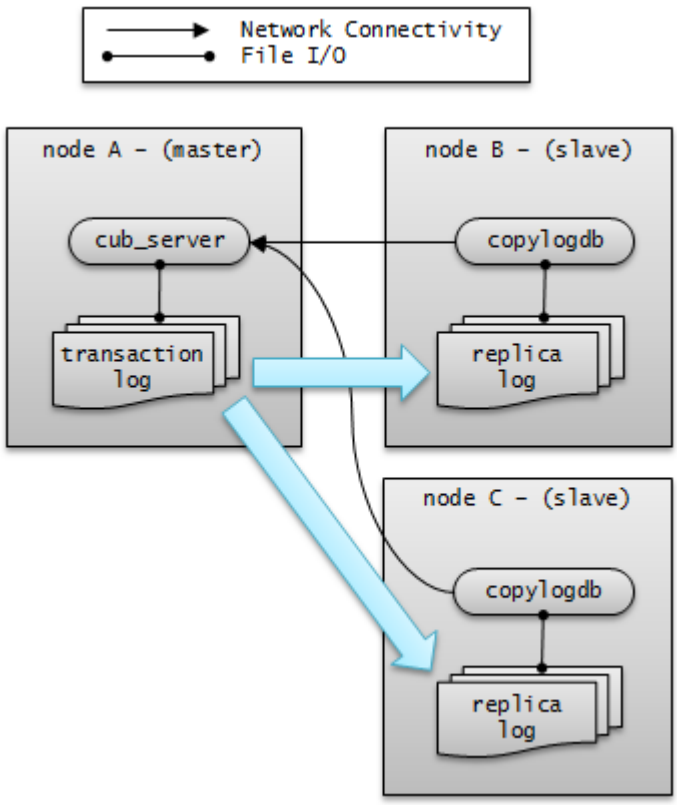
priority. Configuring the **altHosts** of the connection URL is the only necessary action, and it is processed in the JDBC, CCI, and PHP drivers.

Broker failback

If the failed broker is recovered after a failover, the connection to the existing broker is terminated and a new connection is established with the recovered broker which has the highest priority. This requires no processing in the application side as it is processed within the JDBC, CCI, and PHP drivers. Execution time of failback depends on the value configured in JDBC connection URL. For details, see JDBC Configuration.

Log Multiplexing

CUBRID HA keeps every node in the CUBRID HA group with the identical structure by copying and reflecting transaction logs to all nodes included in the CUBRID HA group. As the log copy structure of CUBRID HA is a mutual copy between the master and the slave nodes, it has a disadvantage of increasing the size of a log volume. However, it has an advantage of flexibility in terms of configuration and failure handling, comparing to the chain-type copy structure.



The transaction log copy modes include **SYNC** and **ASYNC**. This value can be configured by the user in cubrid_ha.conf file.

SYNC Mode

When transactions are committed, the created transaction logs are copied to the slave node and stored as a file. The transaction commit is complete after receiving a notice on its success. Although the time to execute commit in this mode may take longer than that in **ASYNC** mode, this is the safest method because the copied transaction logs are always guaranteed to be reflected to the standby server even if a failover occurs.

ASYNC Mode

When transactions are committed, commit is complete without verifying the transfer of transaction logs to a slave node. Therefore, it is not guaranteed that committed transactions are reflected to a slave node in a master node side.

Although **ASYNC** mode provides a better performance as it has almost no delay when executing commit, there may be data inconsistency in its nodes.

Note

SEMISYNC mode is deprecated, and this operates in the same way as **SYNC** mode.

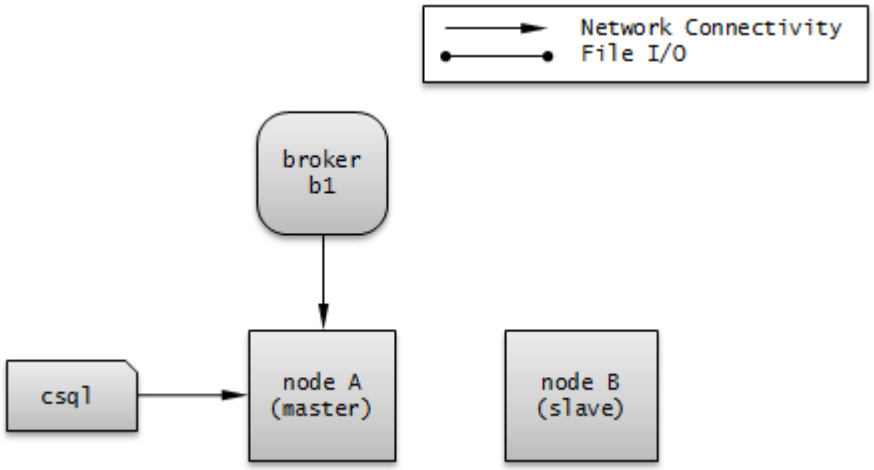
Quick Start

This chapter simply explains how to build a master node and a slave node as 1:1 from DB creation. For details of various replication building methods, see Building Replication.

Preparation

Structure Diagram

The diagram below aims to help users who are new to CUBRID HA, by explaining a simple procedure of the CUBRID HA configuration.



Specifications

Linux and CUBRID version 2008 R2.2 or later must be installed on the equipment to be used as the master and the slave nodes. CUBRID HA does not support Windows operating system.

Specifications of Configuring the CUBRID HA Equipment

	CUBRID Version	OS
For master nodes	CUBRID 2008 R2.2 or later	Linux
For slave nodes	CUBRID 2008 R2.2 or later	Linux

Note

This document describes the HA configuration in CUBRID 9.2 or later versions. Note that the previous versions have different settings. For example, **cubrid_ha.conf** is only available in CUBRID 2008 R4.0 or later. **ha_make_slavedb.sh** is introduced from CUBRID 2008 R4.1 Patch 2 or later.

Creating Databases and Configuring Servers

Creating Databases

Create databases to be included in CUBRID HA at each node of the CUBRID HA in the same manner. Modify the options for database creation as needed.

```
[nodeA]$ cd $CUBRID_DATABASES
[nodeA]$ mkdir testdb
[nodeA]$ cd testdb
[nodeA]$ mkdir log
[nodeA]$ cubrid createdb -L ./log testdb en_US
Creating database with 512.0M size. The total amount of disk space
needed is 1.5G.

CUBRID 10.0

[nodeA]$
```

cubrid.conf

Ensure **ha_mode** of **$CUBRID/conf/cubrid.conf** in every CUBRID HA node has the same value. Especially, take caution when configuring the **log_max_archives** and **force_remove_log_archives** parameters (logging parameters) and the **ha_mode** parameter (HA parameter).

```
# Service parameters
[service]
service=server,broker,manager

# Common section
[common]
service=server,broker,manager

# Server parameters
server=testdb
data_buffer_size=512M
log_buffer_size=4M
sort_buffer_size=2M
max_clients=100
cubrid_port_id=1523
db_volume_size=512M
log_volume_size=512M

# Adds when configuring HA (Logging parameters)
log_max_archives=100
force_remove_log_archives=no

# Adds when configuring HA (HA mode)
ha_mode=on
```

cubrid_ha.conf

Ensure **ha_port_id**, **ha_node_list**, **ha_db_list** of **$CUBRID/conf/cubrid_ha.conf** in every CUBRID HA node has the same value. In the example below, we assume that the hostname of a master node is *nodeA* and that of a slave node is *nodeB*.

```
[common]
ha_port_id=59901
ha_node_list=cubrid@nodeA:nodeB
ha_db_list=testdb
ha_copy_sync_mode=sync:sync
ha_apply_max_mem_size=500
```

databases.txt

Ensure that you must configure the hostnames (*nodeA:nodeB*) of master and slave nodes in db-host of **$CUBRID_DATABASES/databases.txt**; if **$CUBRID_DATABASES** is not configured, do it in **$CUBRID/databases/databases.txt**).

```
#db-name vol-path db-host log-path lob-base-path
testdb /home/cubrid/DB/testdb nodeA:nodeB /home/cubrid/DB/testdb/log
file:/home/cubrid/DB/testdb/lob
```

Starting and Verifying CUBRID HA

Starting CUBRID HA

Execute the **cubrid heartbeat start** at each node in the CUBRID HA group. Note that the node executing **cubrid heartbeat start** first will become a master node. In the example below, we assume that the hostname of a master node is *nodeA* and that of a slave node is *nodeB*.

- Master node

```
[nodeA]$ cubrid heartbeat start
```

- Slave node

```
[nodeB]$ cubrid heartbeat start
```

Verifying CUBRID HA Status

Execute **cubrid heartbeat status** at each node in the CUBRID HA group to verify its configuration status.

```
[cubrid@nodeA]$ cubrid heartbeat status
@ cubrid heartbeat list
 HA-Node Info (current nodeA, state master)
   Node nodeB-node-name (priority 2, state slave)
   Node nodeA-node-name (priority 1, state master)
 HA-Process Info (nodeA 9289, state nodeA)
   Applylogdb testdb@localhost:/home1/cubrid1/DB/testdb_nodeB (pid
9423, state registered)
   Copylogdb testdb@nodeB-node-name:/home1/cubrid1/DB/testdb_nodeB
(pid 9418, state registered)
   Server testdb (pid 9306, state registered_and_active)

[nodeA]$
```

Use the **cubrid changemode** utility at each node in the CUBRID HA group to verify the status of the server.

- Master node

```
[nodeA]$ cubrid changemode testdb@localhost
The server 'testdb@localhost''s current HA running mode is active.
```

- Slave node

```
[nodeB]$ cubrid changemode testdb@localhost
The server 'testdb@localhost''s current HA running mode is standby.
```

Verifying the CUBRID HA Operation

Verify that action is properly applied to standby server of the slave node after performing write in an active server of the master node. To make a success connection via the CSQL Interpreter in HA environment, you must specify the hostname to be connected after the database name like "@<*host_name*>"). If you specify a hostname as localhost, it is connected to local node.

⚠ Warning

Ensure that primary key must exist when creating a table to have replication successfully processed.

- Master node

```
[nodeA]$ csql -u dba testdb@localhost -c "create table abc(a int, b
int, c int, primary key(a));"
[nodeA]$ csql -u dba testdb@localhost -c "insert into abc values
(1,1,1);"
[nodeA]$
```

- Slave node

```
[nodeB]$ csql -u dba testdb@localhost -l -c "select * from abc;"
=== <Result of SELECT Command in Line 1> ===
<00001> a: 1
        b: 1
        c: 1
[nodeB]$
```

Configuring and Starting Broker, and Verifying the Broker Status

Configuring the Broker

To provide normal service during a database failover, it is necessary to configure an available database node in the **db-host** of **databases.txt**. And **ACCESS_MODE** in the **cubrid_broker.conf** file must be specified; if it is omitted, the default value is configured to Read Write mode. If you want to divide into a separate device, you must configure **cubrid_broker.conf** and **databases.txt** in the broker device.

- databases.txt

```
#db-name        vol-path                db-host        log-
path        lob-base-path
testdb          /home1/cubrid1/CUBRID/testdb  nodeA:nodeB        /
home1/cubrid1/CUBRID/testdb/log file:/home1/cubrid1/CUBRID/testdb/
lob
```

- cubrid_broker.conf

```
[%testdb_RWbroker]
SERVICE                 =ON
BROKER_PORT             =33000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =33000
LOG_DIR                 =log/broker/sql_log
ERROR_LOG_DIR           =log/broker/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO
CCI_DEFAULT_AUTOCOMMIT  =ON

# broker mode parameter
ACCESS_MODE             =RW
```

Starting Broker and Verifying its Status

A broker is used to access applications such as JDBC, CCI or PHP. Therefore, to simply test server redundancy, execute the CSQL interpreter that is directly connected to the server processes, without having to start a broker. To start a broker, execute **cubrid broker start**. To stop it, execute **cubrid broker stop**.

The following example shows how to execute a broker from the master node.

```
[nodeA]$ cubrid broker start
@ cubrid broker start
++ cubrid broker start: success
[nodeA]$ cubrid broker status
@ cubrid broker status
```

```
% testdb_RWbroker
------------------------------------------------------------
ID   PID   QPS   LQS PSIZE STATUS
------------------------------------------------------------
 1  9532     0     0  48120  IDLE
```

Configuring Applications

Specifies the hostname (*nodeA_broker*, *nodeB_broker*) and port for an application to connect in the connection URL. The **altHosts** attribute defines the broker where the next connection will be made when the connection to a broker fails. The following is an example of a JDBC program. For more information on CCI and PHP, see CCI Configuration and PHP Configuration.

```
Connection connection =
DriverManager.getConnection("jdbc:CUBRID:nodeA_broker:
33000:testdb:::?charSet=utf-8&altHosts=nodeB_broker:33000", "dba",
"");
```

Environment Configuration

The below is the description for setting the HA environment. See Connecting a Broker to DB for further information regarding the process connecting between a broker and a DB server.

cubrid.conf

The **cubrid.conf** file that has general information on configuring CUBRID is located in the **$CUBRID/conf** directory. This page provides information about **cubrid.conf** parameters used by CUBRID HA.

HA or Not

ha_mode

ha_mode is a parameter used to configure whether to use CUBRID HA. The default value is **off**. CUBRID HA does not support Windows; it supports Linux only.

- **off** : CUBRID HA is not used.
- **on** : CUBRID HA is used. Failover is supported for its node.
- **replica** : CUBRID HA is used. Failover is not supported for its node.

If **ha_mode** is **on**, the CUBRID HA values are configured by reading **cubrid_ha.conf**.

This parameter cannot be modified dynamically. To modify the value of this parameter, you must restart it.

Logging

log_max_archives

log_max_archives is a parameter used to configure the minimum number of archive log files to be archived. The minimum value is 0 and the default is **INT_MAX** (2147483647). When CUBRID has installed for the first time, this value is set to 0 in the **cubrid.conf** file. The behavior of the parameter is affected by **force_remove_log_archives**.

If the value of **force_remove_log_archives** is set to **no**, the existing archive log files to which the activated transaction refers or the archive log files of the master node not reflected to the slave node in HA environment will not be deleted. For details, see the following **force_remove_log_archives**.

For details about **log_max_archives**, see Logging-Related Parameters.

force_remove_log_archives

It is recommended to configure **force_remove_log_archives** to **no** so that archive logs to be used by HA-related processes always can be maintained to set up HA environment by configuring **ha_mode** to **on**.

If you configure the value for **force_remove_log_archives** to yes, the archive log files which will be used in the HA-related process can be deleted, and this may lead to an inconsistency between replicated databases. If you want to maintain free disk space even though doing this could lead to risk, you can configure the value to yes.

For details about **force_remove_log_archives**, see Logging-Related Parameters.

> **Note**
>
> From 2008 R4.3 in replica mode, it will be always deleted except for archive logs as many as specified in the **log_max_archives** parameter, regardless the **force_remove_log_archives** value specified.

Access

max_clients

max_clients is a parameter used to configure the maximum number of clients to be connected to a database server simultaneously. The default is **100**.

Because the replication log copy and the replication log reflection processes start by default if CUBRID HA is used, you must configure the value to twice the number of all nodes in the CUBRID HA group, except the corresponding node. Furthermore, you must consider the case in which a client that is connected to another node at the time of failover attempts to connect to that node.

For details about **max_clients**, see Connection-Related Parameters.

The Parameters That Must Have the Same Value for All Nodes

- **log_buffer_size** : The size of a log buffer. This must be same for all nodes, as it affects the protocol between **copylogdb** that duplicate the server and logs.
- **log_volume_size** : The size of a log volume. In CUBRID HA, the format and contents of a transaction log are the same as that of the replica log. Therefore, the parameter must be same for all nodes. If each node creates its own DB, the **cubrid createdb** options (**--db-volume-size**, **--db-page-size**, **--log-volume-size**, **--log-page-size**, etc.) must be the same.
- **cubrid_port_id** : The TCP port number for creating a server connection. It must be same for all nodes in order to connect **copylogdb** that duplicate the server and logs.
- **HA-related parameters** : HA parameters included in **cubrid_ha.conf** must be identical by default. However, the following parameters can be set differently according to the node.

The Parameters That Can be Different Among Nodes

- The **ha_mode** parameter in replica node
- The **ha_copy_sync_mode** parameter
- The **ha_ping_hosts** parameter
- The **ha_tcp_ping_hosts** parameter

Example

The following example shows how to configure **cubrid.conf**. Please take caution when configuring **log_max_archives** and **force_remove_log_archives** (logging-related parameters), and **ha_mode** (an HA-related parameter).

```
# Service Parameters
[service]
service=server,broker,manager

# Server Parameters
server=testdb
data_buffer_size=512M
log_buffer_size=4M
sort_buffer_size=2M
max_clients=200
```

```
cubrid_port_id=1523
db_volume_size=512M
log_volume_size=512M

# Adds when configuring HA (Logging parameters)
log_max_archives=100
force_remove_log_archives=no

# Adds when configuring HA (HA mode)
ha_mode=on
log_max_archives=100
```

cubrid_ha.conf

The **cubrid_ha.conf** file that has generation information on CUBRID HA is located in the **$CUBRID/conf** directory. CUBRID HA does not support Windows; it supports Linux only.

See Connecting a Broker to DB for further information regarding the process connecting between a broker and a DB server.

Node

ha_node_list

ha_node_list is a parameter used to configure the group name to be used in the CUBRID HA group and the hostname of member nodes in which failover is supported. The group name is separated by @. The name before @ is for the group, and the names after @ are for hostnames of member nodes. A comma(,) or colon(:) is used to separate individual hostnames. The default is **localhost@localhost**.

Note

The hostname of the member nodes specified in this parameter cannot be replaced with the IP. You should use the hostnames which are registered in **/etc/hosts**.

If the hostname is not specified properly, the below message is written into the server.err error log file.

```
Time: 04/10/12 17:49:45.030 - ERROR *** file ../../src/connection/
tcp.c, line 121 ERROR CODE = -353 Tran = 0, CLIENT = (unknown):
(unknown)(-1), EID = 1 Cannot make connection to master server on
host "Wrong_HOST_NAME".... Connection timed out
```

A node in which the **ha_mode** value is set to **on** must be specified in **ha_node_list**. The value of the **ha_node_list** of all nodes in the CUBRID HA group must be identical. When a failover occurs, a node becomes a master node in the order specified in the parameter.

This parameter can be modified dynamically. If you modify the value of this parameter, you must execute cubrid heartbeat reload to apply the changes.

ha_replica_list

ha_replica_list is a parameter used to configure the group name, which is used in the CUBRID HA group, and the replica nodes, which are hostnames of member nodes in which failover is not supported. There is no need to specify this if you do not construct replica nodes. The group name is separated by @. The name before @ is for the group, and the names after @ are for hostnames of member nodes. A comma(,) or colon(:) is used to separate individual hostnames. The default is **NULL**.

The group name must be identical to the name specified in **ha_replica_list**. The hostnames of member nodes and the hostnames of nodes specified in this parameter must be registered in **/etc/hosts**. A node in which the **ha_mode** value is set to **replica** must be specified in **ha_replica_list**. The **ha_replica_list** values of all nodes in the CUBRID HA group must be identical.

This parameter can be modified dynamically. If you modify the value of this parameter, you must execute cubrid heartbeat reload to apply the changes.

Note

The hostname of the member nodes specified in this parameter cannot be replaced with the IP. You should use the hostnames which are registered in **/etc/hosts**.

ha_db_list

ha_db_list is a parameter used to configure the name of the database that will run in CUBRID HA mode. The default is **NULL**. You can specify multiple databases by using a comma (,).

Access

ha_port_id

ha_port_id is a parameter used to configure the UDP port number; the UDP port is used to detect failure when exchanging heartbeat messages. The default is **59,901**.

If a firewall exists in the service environment, the firewall must be configured to allow the configured port to pass through it.

ha_ping_hosts

ha_ping_hosts is a parameter used to configure the host which verifies whether or not a failover occurs due to unstable network when a failover has started in a slave node. The default is **NULL**. A comma(,) or colon(:) is used to separate individual hostnames.

The hostname of the member nodes specified in this parameter can be replaced with the IP. When a hostname is used, the name must be registered in **/etc/hosts**.

CUBRID checks hosts specified in **ha_ping_hosts** every hour; if there is a problem on a host, "ping check" is paused temporarily and checks every 5 minutes if the host is normalized or not.

Configuring this parameter can prevent split-brain, a phenomenon in which two master nodes simultaneously exist as a result of the slave node erroneously detecting an abnormal termination of the master node due to unstable network status and then promoting itself as the new master.

However, the "ping check" does not work if the ICMP protocol is disabled. CUBRID provides **ha_tcp_ping_hosts** as an alternative to address this.

ha_tcp_ping_hosts

ha_tcp_ping_hosts is a parameter that can be used as an alternative to **ha_ping_hosts** when the ICMP protocol is disabled. **ha_tcp_ping_hosts** works like **ha_ping_hosts** except that the TCP layer is used instead of the IP layer for the "ping check". The default is **NULL**. A comma(,) is used to separate individual hostnames, and a colon(:) is used to separate a hostname and a port number. So, the format of this parameter is like "ha_tcp_ping_hosts=host1:port1,host2:port2". In order to use the TCP ping properly, a TCP socket that can receive the requests must be opened in advance with the port number on the host specified in **ha_tcp_ping_hosts** and the firewall must not block the requests. **ha_tcp_ping_hosts** is ignored if the **ha_ping_hosts** is also set.

Replication

ha_copy_sync_mode

ha_copy_sync_mode is a parameter used to configure the mode of storing the replication log, which is a copy of transaction log. The default is **SYNC**.

The value can be one of the following: **SYNC** and **ASYNC**. The number of values must be the same as the number of nodes specified in **ha_node_list**. They must be ordered by the specified value. You can specify multiple modes by using a comma(,) or colon(:). The replica node is always working in **ASNYC** mode regardless of this value.

For details, see Log Multiplexing.

ha_copy_log_base

Specifies the parent path for saving replication logs. The default value is the directory path set in the $CUBRID_DATABASES environment variable. Replication logs are stored in a subdirectory of <db_name>_<host_name>, depending on the server and database name.

The replication log path can be set to either a relative or absolute path. The following are examples of each setting.

ex1) ha_copy_log_base=copylog: Considered a relative path and stores replication logs in $CUBRID_DATABASES/copylog. ex2) ha_copy_log_base=/log/copy_log: Saves replication logs in /log/copy_log as an absolute path.

ha_copy_log_max_archives

ha_copy_log_max_archives is a parameter used to configure the maximum number of keeping replication log files. The default is 1. However, even if the number of replication log files exceeds the specified number of replication log files, they are not deleted if they are not applied to the database.

To prevent wasting needless disk space, it is recommended to keep this value as 1, the default.

ha_apply_max_mem_size

ha_apply_max_mem_size is a parameter used to configure the value of maximum memory that the replication log reflection process of CUBRID HA can use. The default value is **500** (unit: MB) and the maximum value is **INT_MAX** (2147483647). When the value is larger than the size allowed by the system, memory allocation fails and the HA replication reflection process may malfunction. For this reason, you must check whether or not the memory resource can handle the specified value before setting it.

ha_applylogdb_ignore_error_list

ha_applylogdb_ignore_error_list is a parameter used to configure for proceeding replication in CUBRID HA process by ignoring an error occurrence. The error codes to be ignored are separated by a comma (,). This value has a high priority. Therefore, when this value is the same as the value of the **ha_applylogdb_retry_error_list** parameter or the error code of "List of Retry Errors," the values of the **ha_applylogdb_retry_error_list** parameter or the error code of "List of Retry Errors" are ignored and the tasks that cause the error are not retried. For "List of Retry Errors," see the description of **ha_applylogdb_retry_error_list** below.

ha_applylogdb_retry_error_list

ha_applylogdb_retry_error_list is a parameter used to configure for retrying tasks that caused an error in the replication log reflection process of CUBRID HA until the task succeeds. When specifying errors to be retried, separate each error with a comma (,). The following table shows the default "List of Retry Errors." If these values exist in **ha_applylogdb_ignore_error_list**, the error will be overridden.

List of Retry Errors

Error Code Name	Error Code
ER_LK_UNILATERALLY_ABORTED	-72
ER_LK_OBJECT_TIMEOUT_SIMPLE_MSG	-73
ER_LK_OBJECT_TIMEOUT_CLASS_MSG	-74
ER_LK_OBJECT_TIMEOUT_CLASSOF_MSG	-75
ER_LK_PAGE_TIMEOUT	-76
ER_PAGE_LATCH_TIMEDOUT	-836
ER_PAGE_LATCH_ABORTED	-859
ER_LK_OBJECT_DL_TIMEOUT_SIMPLE_MSG	-966
ER_LK_OBJECT_DL_TIMEOUT_CLASS_MSG	-967
ER_LK_OBJECT_DL_TIMEOUT_CLASSOF_MSG	-968
ER_LK_DEADLOCK_CYCLE_DETECTED	-1021

ha_replica_delay

This parameter specifies the term of applying the replicated data between a master node and a replica node. CUBRID intentionally delays replicating by the specified time. You can set a unit as ms, s, min or h, which stands for milliseconds, seconds, minutes or hours respectively. If you omit the unit, milliseconds(ms) will be applied. The default value is 0.

ha_replica_time_bound

In a master node, only the transactions which have been run on the specified time with this parameter are applied to the replica node. The format of this value is "YYYY-MM-DD hh:mi:ss". There is no default value.

Note

The following example shows how to configure **cubrid_ha.conf**.

```
[common]
ha_node_list=cubrid@nodeA:nodeB
ha_db_list=testdb
ha_copy_sync_mode=sync:sync
ha_apply_max_mem_size=500
```

Note

The following example shows how to configure the value of /etc/hosts (a hostname of a member node: nodeA, IP: 192.168.0.1).

```
127.0.0.1 localhost.localdomain localhost
192.168.0.1 nodeA
```

ha_delay_limit

ha_delay_limit is a standard time for CUBRID itself to measure replication delay status, and **ha_delay_limit_delta** is a value to subtract a time which replication delay is released from a replication delay time. Once a server is measured as a replication delay, it keeps this status until the replication delay time is equal or lower than (**ha_delay_limit** - **ha_delay_limit_delta**). A slave node or a replica node corresponds to a standby DB server, that is a target server to judge whether replication is delayed or not.

For example, if you want set replication delay time as 10 minutes and replication-delay-releasing time as 8 minutes, the value of **ha_delay_limit** will be 600s(or 10min) and the value of **ha_delay_limit_delta** will be 120s(or 2min).

If it is measured as replication delay, CAS judges that there is a problem for standby DB to process jobs, and attempts to reconnect the other standby DBs.

CAS, which is connected to the DB which has the lower priority because of the replication delay, expects that the replication delay is released when the time by RECONNECT_TIME in **cubrid_broker.conf** is elapsed, then attempts to reconnect to the standby DB which has higher priority.

ha_delay_limit_delta

See the above description of **ha_delay_limit**.

ha_copy_log_timeout

This is the maximum value of the time in which a node's database server process (cub_server) waits a response from another node's replication-log-copy process (copylogdb). The default is 5(seconds). If this value is -1, this means to be infinite wait. It only works with **SYNC** log copy mode(**ha_copy_sync_mode**) parameter.

ha_check_disk_failure_interval

CUBRID judges the disk failure for each time which is set to the value of this parameter. The default is 15, and the unit is second.

- If the value of **ha_copy_log_timeout** parameter is -1, the value of **ha_check_disk_failure_interval** parameter is ignored and the disk failure is not judged.
- If the value of **ha_check_disk_failure_interval** parameter is smaller than the value of **ha_copy_log_timeout** parameter, the disk failure is judged for each **ha_copy_log_timeout** + 20 seconds.

ha_unacceptable_proc_restart_timediff

When the abnormal status of a server process is kept, the server can be restarted infinitely; it is better to remove this kind of node from the HA components. Because the server is restarted within a short time when the abnormal status is continued, specify this term with this parameter to detect this situation. If the server is restarted within the specified term, CUBRID assumes that this server is abnormal and remove(demote) this node from the HA components. The default is 2min. If the unit is omitted, it is specified as milliseconds(msec).

SQL Logging

ha_enable_sql_logging

If the value of this parameter is **yes**, CUBRID generates the log file of SQL which **applylogdb** process applies to the DB volume. The log file is located under the sql_log of the replication log directory(**ha_copy_log_base**). The default is **no**.

The format of this log filename is *<db name>_<master hostname>***.sql.log.***<id>*, and *<id>* starts from 0. If this size is over **ha_sql_log_max_size_in_mbytes**, a new file with "*<id>* + 1" is created.

For example, if “ha_sql_log_max_size_in_mbytes=100”, demodb_nodeA.sql.log.1 is newly created as the size of demodb_nodeA.sql.log.0 file becomes 100MB. *<id>* will wrap around to 0 once it exceeds the maximum value (4,294,967,295).

By default, only two latest SQL log files are maintained, and the maximum number of them can be adjusted through **ha_sql_log_max_count**.

The SQL log format is as follows.

- INSERT/DELETE/UPDATE

```
-- date | SQL id | SELECT query's length for sampling | the length
of a transformed query
-- SELECT query for sampling
transformed query
```

```
-- 2013-01-25 15:16:41 | 40083 | 33 | 114
-- SELECT * FROM [t1] WHERE "c1"=79186;
INSERT INTO [t1]("c1", "c2", "c3") VALUES
(79186,'b3beb3decd2a6be974',0);
```

- DDL

```
-- date | SQL id | 0 | the length of a transformed query
DDL query
(GRANT query will follow when CREATE TABLE, to grant the authority
to the table created with DBA authority.)
```

```
-- 2013-01-25 14:22:59 | 1 | 0 | 50
create class t1 ( id integer, primary key (id)  );
-- 2013-01-25 14:22:59 | 2 | 0 | 38
GRANT ALL PRIVILEGES ON [t1] TO public;
```

⚠ Warning

When you apply this SQL log from a specific point as creating other DB, triggers should be turned off because the jobs performed by triggers from master node are written to the SQL log file.

- See TRIGGER_ACTION for turning off the triggers with a broker configuration.

- See `csql --no-trigger-action` for tunning off the triggers when you run CSQL.

ha_sql_log_max_size_in_mbytes

The value of this parameter is the maximum size of the file which is created when SQL applied to DB by **applylogdb** process is logged. The new file is created when the size of a log file is over this value.

ha_sql_log_max_count

The value of this parameter indicates the maximum number of SQL log files to be maintained. The oldest ones are removed when the number of SQL log files exceeds this. This parameter can be set within the range of 2 to 5, and the default is 2. **ha_sql_log_max_count** and **ha_sql_log_max_size_in_mbytes** need to be set properly in order to adjust the amount of SQL log to maintain.

ha_sql_log_path

The value of this parameter indicates the path where the SQL log files will be created. The default value is **NULL**. This parameter can be set to an absolute or relative path. If a relative path is set, it is determined based on the replication log directory described in **ha_enable_sql_logging**. The database server process (cub_server) should have appropriate permissions on the path to create SQL log files. If this parameter is not set, the SQL log files will be created on the path described in **ha_enable_sql_logging**

cubrid_broker.conf

The **cubrid_broker.conf** file that has general information on configuring CUBRID broker is located in the **$CUBRID/conf** directory. This section explains the parameters of **cubrid_broker.conf** that are used by CUBRID HA.

See Connecting a Broker to DB for further information regarding the process connecting between a broker and a DB server.

Access Target

ACCESS_MODE

ACCESS_MODE is a parameter used to configure the mode of a broker. The default is **RW**.

Its value can be one of the following: **RW** (Read Write), **RO** (Read Only), **SO** (Standby Only), or **PHRO** (Preferred Host Read Only). For details, see Broker Mode.

REPLICA_ONLY

CAS is only accessed to the replica DB if the value of **REPLICA_ONLY** is **ON**. The default is **OFF**. If the value of **REPLICA_ONLY** is **ON** and the value of **ACCESS_MODE** is **RW**, writing job is possible to even replica DB.

Access Order

CONNECT_ORDER

This parameter specifies whether the host-connecting order from a CAS is sequential or random. The host is configured from **db-host** of **$CUBRID_DATABASES/databases.txt**.

The default is **SEQ**; CAS tries to connect in the order. When it is **RANDOM**, CAS tries to randomly connect. If **PREFERRED_HOSTS** parameter is given, CAS tries to connect to the hosts configured in **PREFERRED_HOSTS** with the order, then uses the value of **db-host** only when the connection by **PREFERRED_HOSTS** fails; and **CONNECT_ORDER** does not affects on the order of **PREFERRED_HOSTS**.

If you concern that the connections are centralized into one DB, set this value as **RANDOM**.

PREFERRED_HOSTS

Specify the order to connect by listing hostnames. The default value is **NULL**.

You can specify multiple nodes by using a colon (:). First, it tries to connect to host in the following order: host specified in the **PREFERRED_HOSTS** parameter first and then host specified in **$CUBRID_DATABASES/databases.txt**.

The following example shows how to configure **cubrid_broker.conf**. To access localhost in a first priority, set **PREFERRED_HOSTS** as *localhost*.

```
[%PHRO_broker]
SERVICE                 =ON
BROKER_PORT             =33000
MIN_NUM_APPL_SERVER     =5
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =33000
LOG_DIR                 =log/broker/sql_log
ERROR_LOG_DIR           =log/broker/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =AUTO
CCI_DEFAULT_AUTOCOMMIT  =ON

# Broker mode setting parameter
ACCESS_MODE             =RO
PREFERRED_HOSTS         =localhost
```

Access Limitation

MAX_NUM_DELAYED_HOSTS_LOOKUP

When replication is delayed on all DB servers in the HA environment which specified multiple DB servers to **db-host** of **databases.txt**, CUBRID checks the replication-delayed servers until only the specified numbers in the **MAX_NUM_DELAYED_HOSTS_LOOKUP** parameter and decides the connection(checking the delay of replication is judged for the standby hosts; the delayed time is decided by the ha_delay_limit parameter). Also, **MAX_NUM_DELAYED_HOSTS_LOOKUP** is not applied to **PREFERRED_HOSTS**.

For example, when **db-host** is specified as "host1:host2:host3:host4:host5" and "MAX_NUM_DELAYED_HOSTS_LOOKUP=2", if the status of them as follows:

- *host1*: active status
- *host2*: standby status, replication is delayed
- *host3*: unable to access
- *host4*: standby status, replication is delayed
- *host5*: standby status, replication is not delayed

then the broker tries to access the two hosts, *host2* and *host4* which their replications are delayed, then decides to access *host4*.

The reason to behave like the above is that CUBRID assumes that the replication will be delayed to the other hosts if the replication of the number(specified by **MAX_NUM_DELAYED_HOSTS_LOOKUP**) of hosts are delayed; therefore, CUBRID decides to connect to the last hosts which CUBRID have tried to access, as CUBRID does not try to access for the left hosts. However, if **PREFERRED_HOSTS** is specified together, CUBRID tries to access to them first and then tries to access to the hosts of db-host list from the first.

The step which the broker accesses CUBRID is divided into the primary connection and the secondary connection.

- The primary connection: the step which the broker tries to access DB at first.

  It checks the DB status(active/standby) and whether the replication is delayed or not. At this time, the broker checks DB's status if it's active or standby based on the **ACCESS_MODE** then decides the connection.

- The secondary connection: After the failure of the primary connection, the broker tries to connect from the failed position. At this time, the broker ignores the DB status(active/standby) and the delay of replication. However, **SO** broker always accepts a connection only to a standby DB.

At this time, the connection is decided if the DB is accessible, by ignoring the delay of replication and DB status(active/standby). However, the error can occur during the query execution. For example, If **ACCESS_MODE** of the broker is **RW** but the broker accesses standby DB, it occurs an error during INSERT operation. Regardless of the error, after it is connected to a standby DB and the transaction is executed, the broker retries the primary connection. However, **SO** broker can never connect to the active DB.

Depending on the value of **MAX_NUM_DELAYED_HOSTS_LOOKUP**, how the number of hosts, attempting to connect is limited as follows:

- MAX_NUM_DELAYED_HOSTS_LOOKUP=-1

  The same as you do not specify this parameter, which is the default value. In this case, at the primary step, the delay of replication and the DB status are checked to the end, then the connection is decided. At the second step, even if there is a replication, or even if that is not the expected DB status(active/standby), the broker connects to the last host which was accessible.

- MAX_NUM_DELAYED_HOSTS_LOOKUP=0

  The secondary connection is processed after the connection is tried only to **PREFERRED_HOSTS** at the primary step; and at the secondary step, the broker tries to connect to a host even it is delayed in replication or it is not an expected DB status(active/standby). That is, because it is the secondary connection, **RW** broker can connect to a standby host and **RO** broker can connect to an active host. However, **SO** broker can never connect to the active DB.

- MAX_NUM_DELAYED_HOSTS_LOOKUP=n(>0)

  The broker tries to connect until the specified number of replication-delayed hosts. At the primary connection, the broker inspects until the specified number of replication-delayed hosts; at the secondary connection, the broker connects to a host that there is a delay of replication.

Reconnection

RECONNECT_TIME

When a broker tries to connect a DB server which are not in **PREFERRED_HOSTS**, **RO** broker tries to connect to active DB server, or a broker tries to connect to the replication-delayed DB server, if connecting time is over **RECONNECT_TIME**(default: 10min), the broker tries to reconnect.

See RECONNECT_TIME for further information.

databases.txt

The **databases.txt** file has information on the order of servers for the CAS of a broker to connect. It is located in the **$CUBRID_DATABASES** (if not specified, $CUBRID/databases) directory; the information can be configured by using **db_hosts**. You can specify multiple nodes by using a colon (:). If "**CONNECT_ORDER=RANDOM**", the connection order is decided as randomly. But if **PREFERRED_HOSTS** is specified, the specified hosts have the first priority of the connection order.

The following example shows how to configure **databases.txt**.

```
#db-name    vol-path        db-host     log-path     lob-base-path
testdb       /home/cubrid/DB/testdb nodeA:nodeB   /home/cubrid/DB/
testdb/log  file:/home/cubrid/DB/testdb/lob
```

JDBC Configuration

To use CUBRID HA in JDBC, you must specify the connection information of another broker (*nodeB_broker*) to be connected when a failure occurs in broker (*nodeA_broker*). The attribute configured for CUBRID HA is **altHosts** which represents information of one or more broker nodes to be connected. For details, see Configuration Connection.

The following example shows how to configure JDBC:

```
Connection connection =
DriverManager.getConnection("jdbc:CUBRID:nodeA_broker:
33000:testdb:::?charSet=utf-8&altHosts=nodeB_broker:33000", "dba",
"");
```

CCI Configuration

To use CUBRID HA in CCI, you must use the `cci_connect_with_url()` function which additionally allows specifying connection information in connection URL; the connection information is used when a failure occurs in broker. The attribute configured for CUBRID HA is **altHosts** which represents information of one or more broker nodes to be connected.

The following example shows how to configure CCI.

```
con = cci_connect_with_url ("cci:CUBRID:nodeA_broker:33000:testdb:::?
altHosts=nodeB_broker:33000", "dba", NULL);
if (con < 0)
{
      printf ("cannot connect to database\n");
```

```
        return 1;
}
```

PHP Configuration

To use the functions of CUBRID HA in PHP, connect to the broker by using cubrid_connect_with_url, which is used to specify the connection information of the failover broker in the connection URL. The attribute specified for CUBRID HA is **altHosts**, the information on one or more broker nodes to be connected when a failover occurs.

The following example shows how to configure PHP.

```
<?php
$con = cubrid_connect_with_url ("cci:CUBRID:nodeA_broker:
33000:testdb:::?altHosts=nodeB_broker:33000", "dba", NULL);
if ($con < 0)
{
        printf ("cannot connect to database\n");
        return 1;
}
?>
```

Note

If you want to run smoothly the broker's failover in the environment which the broker's failover is enabled by setting **altHosts**, you should set the value of **disconnectOnQueryTimeout** in URL as **true**.

If this value is **true**, an application program releases the existing connection from a broker and reconnects to the other broker which is specified on **altHosts**.

Connecting a Broker to DB

A broker in HA environment should decide the one DB server to connect among multiple DB servers. At this time, it is different depending on the setting of the DB server and broker; how to connect to the DB server and what DB server should be chosen. In this chapter, we will look over how a broker choose DB server by the setting of HA environment. See Environment Configuration for the description about each parameters used in the environment setting.

Here are the main parameters used in the DB connection with the broker.

Location	Configuration file	Parameter name	Description
DB server	cubrid.conf	ha_mode	HA mode(on/off/replica) of DB server. Default: off
	cubrid_ha.conf	ha_delay_limit	A period to determine whether the replication-delay
		ha_delay_limit_delta	Time subtracting the resolution time of replication-delay from the time of replication-delay
Broker	cubrid_broker.conf	ACCESS_MODE	Broker mode(RW/RO/SO). Default: RW
		REPLICA_ONLY	Connectible to REPLICA server or not(ON/OFF). Default: OFF
		PREFERRED_HOSTS	Connecting to the host that is specified here in priority to the host that you set in the db-host of databases.txt
		MAX_NUM_DELAYED_HOSTS_LOOKUP	The number of hosts to determine the delay of replication in databases.txt. If up

Location	Configuration file	Parameter name	Description
			to the specified number of hosts was determined as the delay of replication, the broker is connected to the host checked at last. • -1: check all hosts specified in databases.txt whether replication is delayed or not • 0: do not check whether replication-delay or not, and process the secondary connection immediately • n(>0): check up to n hosts whether replication is delayed or not
		RECONNECT_TIME	Time to try reconnecting after the broker is connected to the improper DB server. Default: 600s. If this value is 0, no try for reconnection.

Location	Configuration file	Parameter name	Description
		CONNECT_ORDER	A parameter specifying the connecting order whether to connect as or the random order(SEQ/ RANDOM). Default: SEQ

Connection Process

When a broker accesses DB server, it tries the primary connection; if it fails, it tries the secondary connection.

- The primary connection: Check the DB status(active/standby) and the delay of replication.
  1. A broker tries to connect as the order specified by **PREFERRED_HOSTS**. The broker rejects connecting to the improper DB of which the status does not match with **ACCESS_MODE** or in which the replication is delayed.
  2. By the **CONNECT_ORDER**, a broker tries to connect to the host in the order specified in **databases.txt** or the random order. The broker checks the DB status followed by the **ACCESS_MODE** and checks the replication-delayed host up to the number specified in **MAX_NUM_DELAYED_HOSTS_LOOKUP**.
- The secondary connection: Ignore the DB status(active/standby) and the delay of replication. However, **SO** broker always accepts to connect only to standby DB.
  1. A broker tries to connect as the order specified by **PREFERRED_HOSTS**. The broker accepts connecting to the improper DB of which status does not match with **ACCESS_MODE** or in which the replication is delayed. However, **SO** broker can never connect to active DB.
  2. By the **CONNECT_ORDER**, a broker tries to connect to the host in the order specified in **databases.txt** or the random order. The broker ignores the DB status(active/standby) and the delay of replication; it is connected if possible.

Examples on Behaviours by Configuration

The following shows the example of configuration.

Host DB status

- *host1*: active
- *host2*: standby, replication is delayed.
- *host3*: standby, replica, unable to access.
- *host4*: standby, replica, replication is delayed.
- *host5*: standby, replica, replication is delayed.

When the status of host DBs are as the above, the below shows samples of behaviours by the configuration.

Behaviours by configuration

- 2-1, 2-2, 2-3: From 2, (+) is addition and (#) is modification.
- 3-1, 3-2, 3-3: From 3, (+) is addition and (#) is modification.

No.	Configuration	Behavior
1	• **ACCESS_MODE=RW** • PREFERRED_HOSTS=host2:host3 • db-host=host1:host2:host3:host4:host5 • MAX_NUM_DELAYED_HOSTS_LOOKUP=-1 • CONNECT_ORDER=SEQ	At the primary connection try, a broker checks if DB status is active. • *host2* of PREFERRED_HOSTS is replication-delay and *host3* is unable to access; therefore, it tries to connect to db-host • connection is established because *host1* is on active status. The broker tries to reconnect after the RECONNECT_TIME because it did not connect to PREFERRED_HOSTS.
2	• **ACCESS_MODE=RO**	

No.	Configuration	Behavior
	• db-host=host1:host2:host3:host4:host5 • MAX_NUM_DELAYED_HOSTS_LOOKUP=-1 • CONNECT_ORDER=SEQ	At the primary connection try, a broker checks if DB status is standby. • All hosts of which DB status is standby are all replication-delayed or unable to access; therefore, it will be failed at the primary connection. At the secondary connection try, a broker ignores DB status and replication-delay. • The last accessed host, *host5* is successful to connect to a broker. Because the broker accessed the replication-delayed server, it tries to reconnect after RECONNECT_TIME.
2-1	• (+)PREFERRED_HOSTS=host1:host3	At the primary connection try, a broker checks if DB status is standby. • In PREFERRED_HOSTS, *host1* is active and *host3* is unable to access; therefore, the broker tries to connect to db-host. • Because hosts of which DB status is standby are all replication-delayed or unable to access;

No.	Configuration	Behavior
		therefore, it will be failed at the primary connection. At the secondary connection try, a broker ignores DB status and replication-delay. • *host1* in PREFERRED_HOSTS is active, but it is possible to access; therefore, it is successful to connect to a broker. Because the broker accessed the active server, it tries to reconnect after RECONNECT_TIME.
2-2	• (+)PREFERRED_HOSTS=host1:host3 • (#)MAX_NUM_DELAYED_HOSTS_LOOKUP=0	At the primary connection try, a broker checks if DB status is standby. • In PREFERRED_HOSTS, *host1* is active and *host3* is unable to access. • no tries to connect to db-host. At the secondary connection try, a broker ignores DB status and replication-delay. • In PREFERRED_HOSTS, *host1* is active, but it is possible to access; therefore, it is successful to connect to a broker.

No.	Configuration	Behavior
		Because the broker accessed the active server, it tries to reconnect after RECONNECT_TIME.
2-3	• (#)MAX_NUM_DELAYED_HOSTS_LOOKUP=2	At the primary connection try, a broker checks if DB status is standby. • It fails at the primary connection after checking that *host2*, *host4* are replication-delayed where DB status is standby. At the secondary connection try, a broker ignores DB status and replication-delay. • The lastly accessed *host4* is successful to connect to a broker. Because the broker accessed the replication-delayed server, it tries to reconnect after RECONNECT_TIME.
3	• **ACCESS_MODE=SO** • db-host=host1:host2:host3:host4:host5 • MAX_NUM_DELAYED_HOSTS_LOOKUP=-1	At the primary connection try, a broker checks if DB status is standby. • It fails at the primary connection because hosts of which status is standby are all replication-

No.	Configuration	Behavior
	• CONNECT_ORDER=SEQ	delayed or unable to access. At the secondary connection try, a broker checks if DB status is standby but ignores replication-delay. • The lastly accessed *host4* is successful to connect to a broker. Because the broker accessed the replication-delayed server, it tries to reconnect after RECONNECT_TIME.
3-1	• (+)PREFERRED_HOSTS=host1:host3	At the primary connection try, a broker checks if DB status is standby. • In PREFERRED_HOSTS, *host1* is active and *host3* is unable to access; therefore, it tries to connect to db-host. • It fails at the primary connection because hosts of which status is standby are all replication-delayed or unable to access. At the secondary connection try, a broker checks if DB status is

No.	Configuration	Behavior
		standby but ignores replication-delay. • In PREFERRED_HOSTS, *host1* is active and *host3* is unable to access; therefore, it tries to connect to db-host. • The first host of which DB status is standby is *host2*; therefore, it connects to *host2*. Because the broker accessed the replication-delayed server, it tries to reconnect after RECONNECT_TIME.
3-2	• PREFERRED_HOSTS=host1:host3 • (#)MAX_NUM_DELAYED_HOSTS_LOOKUP=0	At the primary connection try, a broker checks if DB status is standby. • In PREFERRED_HOSTS, *host1* is active and *host3* is unable to access. • It does not try to connect to db-host. At the secondary connection try, a broker checks if DB status is standby but ignores replication-delay. • In PREFERRED_HOSTS, *host1* is active and *host3* is unable to access; therefore, it tries to connect to db-host.

No.	Configuration	Behavior
		• The first host of which DB status is standby is *host2*; therefore, it connects to *host2*. Because the broker accessed the replication-delayed server, it tries to reconnect after RECONNECT_TIME.
3-3	• (#)MAX_NUM_DELAYED_HOSTS_LOOKUP=2	At the primary connection try, a broker checks if DB status is standby. • It fails at the primary connection after checking that *host2*, *host4* are replication-delayed where DB status is standby. At the secondary connection try, a broker checks if DB status is standby but ignores replication-delay. • The lastly accessed *host4* is successful to connect to a broker. Because the broker accessed the replication-delayed server, it tries to reconnect after RECONNECT_TIME.

Running and Monitoring

cubrid heartbeat Utility

cubrid heartbeat command can be run as **cubrid hb**, the abbreviated command.

start

This utility is used to activate CUBRID HA feature and start all processes of CUBRID HA in the node(database server process, replication log copy process, and replication log reflection process). Note that a master node or a slave node is determined based on the execution order of **cubrid heartbeat start**.

How to execute the command is as shown below.

```
$ cubrid heartbeat start
```

The database server process configured in HA mode cannot be started with the **cubrid server start** command.

Specify the database name at the end of the command to run only the HA configuration processes (database server process, replication log copy process, and replication log reflection process) of a specific database in the node. For example, use the following command to run the database *testdb* only:

```
$ cubrid heartbeat start testdb
```

stop

This utility is used to disable and stop all components of CUBRID. The node that executes this command stops and a failover occurs to the next slave node according to the CUBRID HA configuration.

How to use this utility is as shown below.

```
$ cubrid heartbeat stop
```

The database server process cannot be stopped with the **cubrid server stop** command.

Specify the database name at the end of the command to stop only the HA configuration processes (database server process, replication log copy process, and replication log reflection

process) of a specific database in the node. For example, use the following command to run the database *testdb* only:

```
$ cubrid heartbeat stop testdb
```

If you want to deactivate CUBRID HA feature immediately, add -i option into the “cubrid heartbeat stop” command. This option is used when the speedy quitting is required because the DB server process is working improperly.

```
$ cubrid heartbeat stop -i
or
$cubrid heartbeat stop --immediately
```

copylogdb

This utility is used to start or stop the **copylogdb** process that copies the transaction logs for the *db_name* of a specific peer_node in the CUBRID HA configuration. You can pause log copy for rebuilding replications in the middle of operation and then rerun it whenever you want.

Even though only the **cubrid heartbeat copylogdb start** command has succeeded, the functions of detecting and recovering the failure between the nodes are executed. Since the node is the target of failover, the slave node can be changed to the master node.

How to use this utility is as shown below.

```
$ cubrid heartbeat copylogdb <start|stop> [ -h <host-name> ] db_name
peer_node
```

```
<host-name>: the name of the remote host where copylogdb command
will be executed
```

When *nodeB* is a node to run a command and *nodeA* is *peer_node*, you can run the command as follows.

```
[nodeB]$ cubrid heartbeat copylogdb stop testdb nodeA
[nodeB]$ cubrid heartbeat copylogdb start testdb nodeA
```

When the **copylogdb** process is started/stopped, the configuration information of the **cubrid_ha.conf** is used. We recommend that you do not change the configuration as possible after you have set the configuration once. If you need to change it, it is recommended to restart the whole nodes.

applylogdb

This utility is used to start or stop the **copylogdb** process that reflects the transaction logs for the *db_name* of a specific peer_node in the CUBRID HA configuration. You can pause log copy for rebuilding replications in the middle of operation and then rerun it whenever you want.

Even though only the **cubrid heartbeat copylogdb start** command has succeeded, the functions of detecting and recovering the failure between the nodes are executed. Since the node is the target of failover, the slave node can be changed to the master node.

How to use this utility is as shown below.

```
$ cubrid heartbeat applylogdb <start|stop> [ -h <host-name> ]
db_name peer_node
```

```
<host-name>: the name of the remote host where appplylogdb command
will be executed
```

When *nodeB* is a node to run a command and *nodeA* is *peer_node*, you can run the command as follows.

```
[nodeB]$ cubrid heartbeat applylogdb stop testdb nodeA
[nodeB]$ cubrid heartbeat applylogdb start testdb nodeA
```

When the **applylogdb** process is started/stopped, the configuration information of the **cubrid_ha.conf** is used. We recommend that you do not change the configuration as possible after you have set the configuration once. If you need to change it, it is recommended to restart the whole nodes.

reload

This utility is used to retrieve the CUBRID HA information again. To start/stop HA replication processes of the added/removed nodes at once, you can use "**cubrid heartbeat replication start/stop**" command.

How to use this utility is as shown below.

```
$ cubrid heartbeat reload
```

Reconfigurable parameters are **ha_node_list** and **ha_replica_list**. Even if an error occurs on a special node during running this command, the left jobs are continued. After **reload** command is finished, check if the reconfiguration of nodes is applied well or not. If it fails, find the reason and resolve it.

replication(or repl) start

This utility is used to run in batch HA processes(copylogdb/applylogdb) related to a specific node; in general, it is used to run in batch HA replication processes of added nodes after running **cubrid heartbeat reload**.

replication command can be abbreviated by **repl**.

```
cubrid heartbeat repl start <node_name>
```

- *node_name*: one of nodes specified in **ha_node_list** of cubrid_ha.conf.

replication(or repl) stop

This utility is used to stop in batch HA processes(copylogdb/applylogdb) related to a specific node; in general, it is used to stop in batch HA replication processes of removed nodes after running **cubrid heartbeat reload**.

replication command can be abbreviated by **repl**.

```
cubrid heartbeat repl stop <node_name>
```

- *node_name*: one of nodes specified in **ha_node_list** of cubrid_ha.conf.

status

```
$ cubrid heartbeat status [-v] [ -h <host-name> ]
```

```
<host-name>: the name of the remote host where status command will
be executed
```

This utility is used to output the information of CUBRID HA group and CUBRID HA components. How to use this utility is as shown below.

```
$ cubrid heartbeat status
@ cubrid heartbeat status

 HA-Node Info (current nodeB, state slave)
   Node nodeB (priority 2, state slave)
   Node nodeA (priority 1, state master)
```

```
 HA-Process Info (master 2143, state slave)
   Applylogdb testdb@localhost:/home/cubrid/DB/testdb_nodeA (pid
2510, state registered)
   Copylogdb testdb@nodeA:/home/cubrid/DB/testdb_nodeA (pid 2505,
state registered)
   Server testdb (pid 2393, state registered_and_standby)
```

The -v option displays detailed information about the node.

- score : Indicates the priority of the node, with a lower score indicating a higher priority.
- missed heartbeat : Indicates the loss rate of heartbeat signals sent between nodes configured in an HA environment. If this value is unusually high, the configuration, network, or firewall settings should be inspected.

The event occurrence times for the Applylogdb, Copylogdb, and Server processes are also displayed. If no event has occurred, it is displayed as “00:00:00.000.”

- registered-time : The time when a process startup request was made via a user command.
- deregistered-time : The time when a remote process stop request was made via a user command (applicable only to copylogdb and applylogdb).
- shutdown-time : The time that HA-manager(cub_master) stopped the process
- start-time : The time that HA-manager(cub_master) restarted the process

Example

```
$ cubrid heartbeat status -v
@ cubrid heartbeat status

HA-Node Info (current cubrid1, state master)
  Node cubrid2 (priority 2, state slave)
    - score 2
    - missed heartbeat 0
  Node cubrid1 (priority 1, state master)
    - score -32767
    - missed heartbeat 0

HA-Process Info (master 7392, state master)
Copylogdb testdb@cubrid2:/home/cubha/CUBRID-11.4-latest-Linux.x86_64/
databases/testdb_cubrid2 (pid 7841, state registered)
 - exec-path [/home/cubha/CUBRID-11.4-latest-Linux.x86_64/bin/
cub_admin]
 - argv      [cub_admin copylogdb -L /home/cubha/CUBRID-11.4-latest-
Linux.x86_64/databases/testdb_cubrid2 -m sync testdb@bagus2 ]
 - registered-time   08/26/24 14:28:37.019
 - deregistered-time 00/00/00 00:00:00.000
 - shutdown-time     08/26/24 14:28:35.010
```

```
 - start-time        08/26/24 14:28:36.012
Applylogdb testdb@localhost:/home/cubha/CUBRID-11.4-latest-
Linux.x86_64/databases/testdb_cubrid2 (pid 7746, state registered)
 - exec-path [/home/cubha/CUBRID-11.4-latest-Linux.x86_64/bin/
cub_admin]
 - argv      [cub_admin applylogdb -L /home/cubha/CUBRID-11.4-latest-
Linux.x86_64/databases/testdb_cubrid2 --max-mem-size=300
testdb@localhost ]
  - registered-time   08/26/24 14:27:14.566
  - deregistered-time 00/00/00 00:00:00.000
  - shutdown-time     08/26/24 14:27:12.552
  - start-time        08/26/24 14:27:13.558
 Server testdb (pid 7904, state registered_and_active)
  - exec-path [/home/cubha/CUBRID-11.4-latest-Linux.x86_64/bin/
cub_server]
  - argv      [cub_server testdb ]
  - registered-time   08/26/24 14:29:28.955
  - deregistered-time 00/00/00 00:00:00.000
  - shutdown-time     08/26/24 14:29:27.593
  - start-time        08/26/24 14:29:28.594
```

Note

act, **deact**, and **deregister** commands which were used in versions lower than CUBRID 9.0 are no longer used.

Registering HA to cubrid service

If you register heartbeat to CUBRID service, you can use the utilities of **cubrid service** to start, stop or check all the related processes at once. The processes specified by **service** parameter in [**service**] section in **cubrid.conf** file are registered to CUBRID service. If this parameter includes **heartbeat**, you can start/stop all the service processes and the HA-related processes by using **cubrid service start** / **stop** command.

How to configure **cubrid.conf** file is shown below.

```
# cubrid.conf

...

[service]

...

service=broker,heartbeat
```

```
...
[common]
...
ha_mode=on
```

applyinfo

This utility is used to check the copied and applied status of replication logs by CUBRID HA.

```
cubrid applyinfo [option] <database-name>
```

- *database-name* : Specifies the name of a server to monitor. A node name is not included.

The following shows the [options] used on **cubrid applyinfo**.

-r, --remote-host-name=HOSTNAME

Configures the name of a target node in which transaction logs are copied. Using this option will output the information of active logs (Active Info.) of a target node.

-a, --applied-info

Outputs the information of replication reflection of a node executing cubrid applyinfo. The **-L** option is required to use this option.

-L, --copied-log-path=PATH

Configures the location of transaction logs copied from the other node. Using this option will output the information of transaction logs copied (Copied Active Info.) from the other node.

-p, --pageid=ID

Outputs the information of a specific page in the copied logs. This is available only when the **-L** option is enabled. The default is 0, it means the active page.

-v

Outputs detailed information.

-i, --interval=SECOND

Outputs the copied status and applied status of transaction logs per specified seconds. To see the delayed status of the replicated log, this option is mandatory.

Example

The following example shows how to check log information (Active Info.) of the master node, the status information of log copy (Copied Active Info.) of the slave node, and the applylogdb info (Applied Info.) of the slave node by executing **applyinfo** in the slave node.

- Applied Info.: Shows the status information after the slave node applies the replication log.
- Copied Active Info.: Shows the status information after the slave node copies the replication log.
- Copied Archive Info. : Shows the status information after the slave node copies the archive log.
- Active Info.: Shows the status information after the master node records the transaction log.
- Delay in Copying Active Log: Shows the status information which the transaction logs' copy is delayed.
- Delay in Applying Copied Log: Shows the status information which the transaction logs' application is delayed.

```
[nodeB] $ cubrid applyinfo -L /home/cubrid/DB/testdb_nodeA -r nodeA -
a -p 0 -i 3 testdb

 *** Applied Info. ***
Insert count                    : 289492
Update count                    : 71192
Delete count                    : 280312
Schema count                    : 20
Commit count                    : 124917
Fail count                      : 0

 *** Copied Active Info. ***
DB name                         : testdb
DB creation time                : 04:29:00.000 PM 11/04/2012
(1352014140)
Vol creation time               : 04:29:10.000 PM 11/04/2012
(1352014150)
EOF LSA                         : 27722 | 10088
Append LSA                      : 27722 | 10088
HA server state                 : active

 *** Copied Archive Info. ***
DB name                         : testdb
DB creation time                : 04:29:00.000 PM 11/04/2012
(1352014140)
Vol creation time               : 04:29:20.000 PM 11/04/2012
(1352014160)
Archive number                  : 0
Log page 0 (phy: 1 pageid: 0, offset 0)
offset:0000 (tid:1 bck p:-1,o:-1 frw p:0,o:96 type:3)
offset:0096 (tid:1 bck p:0,o:0 frw p:0,o:320 type:5)
offset:0320 (tid:1 bck p:0,o:96 frw p:0,o:552 type:4)
```

```
offset:0552 (tid:1 bck p:0,o:320 frw p:0,o:608 type:4)
...

 *** Active Info. ***
DB name                         : testdb
DB creation time                : 04:29:00.000 PM 11/04/2012
(1352014140)
Vol creation time               : 04:29:10.000 PM 11/04/2012
(1352014150)
EOF LSA                         : 27726 | 2512
Append LSA                      : 27726 | 2512
HA server state                 : active

 *** Delay in Copying Active Log ***
Delayed log page count          : 4
Estimated Delay                 : 0 second(s)

 *** Delay in Applying Copied Log ***
Delayed log page count          : 1459
Estimated Delay                 : 22 second(s)
```

The items shown by each status are as follows:

- Applied Info.
  - Committed page: The information of committed pageid and offset of a transaction reflected last through replication log reflection process. The difference between this value and the EOF LSA of “Copied Active Info. represents the amount of replication delay.
  - Insert Count: The number of Insert queries reflected through replication log reflection process.
  - Update Count: The number of Update queries reflected through replication log reflection process.
  - Delete Count: The number of Delete queries reflected through replication log reflection process.
  - Schema Count: The number of DDL statements reflected through replication log reflection process.
  - Commit Count: The number of transactions reflected through replication log reflection process.
  - Fail Count: The number of DML and DDL statements in which log reflection through replication log reflection process fails.

- Copied Active Info.
  - DB name: Name of a target database in which the replication log copy process copies logs
  - DB creation time: The creation time of a database copied through replication log copy process
  - Vol creation time: The creation time of the volume copied through replication log copy process
  - EOF LSA: Information of pageid and offset copied at the last time on the target node by the replication log copy process. There will be a delay in copying logs as much as difference with the EOF LSA value of “Active Info.” and with the Append LSA value of “Copied Active Info.”
  - Append LSA: Information of pageid and offset written at the last time on the disk by the replication log copy process. This value can be less than or equal to EOF LSA. There will be a delay in copying logs as much as difference between the EOF LSA value of “Copied Active Info.” and this value.
  - HA server state: Status of a database server process which replication log copy process receives logs from. For details on status, see Servers.
- Copied Archive Info.
  - DB name: Name of a target database in which the replication log copy process copies logs
  - DB creation time: The creation time of a database copied through replication log copy process
  - Vol creation time: The creation time of the volume copied through replication log copy process
  - Archive number: The number of archive logs copied through replication log copy process
  - Log page: The information of pageid and offset of the archive log copied through replication log copy process.
  - offset: The information of pageid and offset of the archive log copied through replication log copy process.
- Active Info.
  - DB name: Name of a database whose node was configured in the **-r** option.
  - DB creation time: Database creation time of a node that is configured in the **-r** option.
  - Vol creation time: Volume creation time of a node that is configured in the **-r** option.

    - EOF LSA: The last information of pageid and offset of a database transaction log of a node that is configured in the **-r** option. There will be a delay in copying logs as much as difference between the EOF LSA value of "Copied Active Info." and this value.
    - Append LSA: Information of pageid and offset written at the last time on the disk by the database whose node was configured in the **-r** option.
    - HA server state: The server status of a database server whose node was configured in the **-r** option.

- Delay in Copying Active Log
    - Delayed log page count: the count of transaction log pages which the copy is delayed.
    - Estimated Delay: the expected time which the logs copying is completed.
- Delay in Applying Copied Log
    - Delayed log page count: the count of transaction log pages which the application is delayed.
    - Estimated Delay: the expected time which the logs applying is completed.

When you run this command in replica node, if "ha_replica_delay=30s" is specified in cubrid.conf, the following information is printed out additionally.

```
 *** Replica-specific Info. ***
Deliberate lag                 : 30 second(s)
Last applied log record time   : 2013-06-20 11:20:10
```

Each item of the status information is as below.

- Replica-specific Info.
    - Deliberate lag: delayed time a user defined by ha_replica_delay parameter
    - Last applied log record time: the time where the replication log of being applied in the replica node recently was actually applied in the master node.

When you run this command in replica node, if "ha_replica_delay=30s" and "ha_replica_time_bound=2013-06-20 11:31:00" are specified in cubrid.conf, "ha_replica_delay=30s" is ignored and the following information is printed out additionally.

```
 *** Replica-specific Info. ***
Last applied log record time   : 2013-06-20 11:25:17
Will apply log records up to   : 2013-06-20 11:31:00
```

Each item of the status information is as below.

- Replica-specific Info.
  - Last applied log record time: the time where the replication log of being applied in the replica node recently was actually applied in the master node.
  - Will apply log records up to: the replica node will apply the master node's logs replicated up to this time.

When applylogdb stops the replication after the time of being specified by **ha_replica_time_bound**, the error message which is printed out in the file, **$CUBRID/log/***db-name***@***local-node-name***_applylogdb_***db-name*_*remote-node-name***.err** is as below.

```
Time: 06/20/13 11:51:05.549 - ERROR *** file ../../src/transaction/
log_applier.c, line 7913 ERROR CODE = -1040 Tran = 1, EID = 3
HA generic: applylogdb paused since it reached a log record
committed on master at 2013-06-20 11:31:00 or later.
Adjust or remove ha_replica_time_bound and restart applylogdb to
resume.
```

cubrid changemode

This utility is used to check and change the server status of CUBRID HA.

```
cubrid changemode [options] <database-name@node-name>
```

- *database-name@node-name*: Specifies the name of a server to be checked or changed and separates each node name by using @. If [options] is omitted, server status is displayed.

The following shows [options] used in **cubrid changemode**.

-m, --mode=MODE

Changes the server status. You can enter one of the following:

standby, **maintenance** or **active**.

- You can change a mode as **maintenance** if a server's status is **standby**.
- You can change a mode as **standby** if a server's status is **maintenance**.
- You can change a mode as **active** if a server's status is **to-be-active**. However, this should be used together with **--force** option. See the below descrioption regarding **--force** option.

-f, --force

Configures whether or not to forcibly change the server status. This option must be configured if you want to change the server status from to-be-active to active. |

If it is not configured, the status will not be changed to active. Forcibly change may cause data inconsistency among replication nodes; so it is not recommended. |

-t, --timeout=SECOND

The default is 5(seconds). Configures the waiting time for the normal completion of the transaction that is being processed when the node status switches from **standby** to **maintenance**.

If the transaction is still in progress beyond the configured time, it will be forced to terminate and switch to **maintenance** status; if all transactions have completed normally within the configured time, it will switch to **maintenance** status immediately.

Status Changeable

This table shows changeable modes depending on current status.

		Changeable		
		active	standby	maintenance
Current Status	standby	X	O	O
	to-be-standby	X	X	X
	active	O	X	X
	to-be-active	O*	X	X
	maintenance	X	O	O

* When the server status is to-be-active, forcibly change may cause data inconsistency among replication nodes. It is not recommended if you are not skilled enough.

Example

The following example shows how to switch the *testdb* server status in the localhost node to maintenance. The waiting time for all transactions in progress to complete normally is 5 seconds,

which is the default value for the **-t** option. If all transactions are complete within this time limit, the status will be switched immediately. However, if there are transactions still being processed after this time limit, they will be rolled back before changing the status.

```
$ cubrid changemode -m maintenance testdb@localhost
The server 'testdb@localhost''s current HA running mode is
maintenance.
```

The following example shows how to retrieve status of the *testdb* server in the localhost node.

```
$ cubrid changemode testdb@localhost
The server 'testdb@localhost''s current HA running mode is active.
```

Monitoring CUBRID Manager HA

CUBRID Manager is a dedicated CUBRID database management tool that provides the CUBRID database management and query features in a GUI environment. CUBRID Manager provides the HA dashboard, which shows the relationship diagram for the CUBRID HA group and server status. For details, see CUBRID Manager manual.

Structures of HA

There are four possible structures for CUBRID HA: The default structure, multiple-slave node structure, load balancing structure, and multiple-standby server structure. In the table below, M stands for a master node, S for a slave node, and R for a replica node.

Structure	Node structure (M:S:R)	Characteristic
Default Structure	1:1:0	The most basic structure of CUBRID HA consists of one master node and one slave node and provides availability which is a unique feature of CUBRID HA.
Multiple-Slave Node Structure	1:N:0	This is a structure in which availability is increased by several slave nodes. However, note that there

Structure	Node structure (M:S:R)	Characteristic
		may be a situation in which data is inconsistent in the CUBRID HA group when multiple failures occur.
Load Balancing Structure	1:1:N	Several replica nodes are added in the basic structure. Read service load can be distributed, and the HA load is reduced, comparing to a multiple-slave node structure. Note that replica nodes do not failover.
Multiple-Standby Server Structure	1:1:0	Basically, this structure is the same as the basic structure. However, several slave nodes are installed on a single physical server.

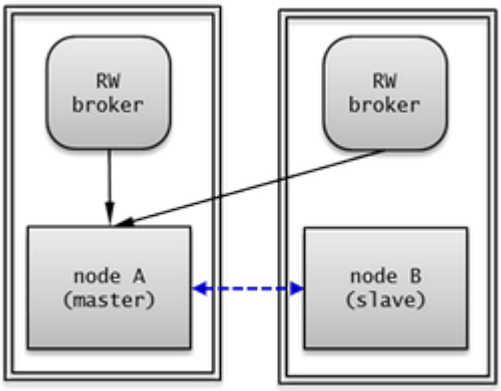
In the following description, it is assumed that there is no data in the testdb database on each node. To build HA feature as replicating database, see Building Replication or Rebuilding Replication Script.

Basic Structure of HA

The most basic structure of CUBRID HA consists of one master node and one slave node.

The default configuration is one master node and one slave node. To distribute the write load, a multi-slave node or load-distributed configuration is recommended. In addition, to access a specific node such as a slave node or replica node in read-only mode, configure the Read Only broker or the Preferred Host Read Only broker. For details about broker configuration, see Duplexing Brokers.

An Example of Node Configuration



You can configure each node in the basic structure of HA as shown below:

- **node A** (master node)
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

  - The following example shows how to configure **cubrid_ha.conf**:

```
ha_port_id=59901
ha_node_list=cubrid@nodeA:nodeB
ha_db_list=testdb
```

- **node B** (slave node): Configure this node in the same manner as *node A*.

For the **databases.txt** file of a broker node, it is necessary to configure the list of hosts configured as HA in **db-host** according to their priority. The following example shows the **databases.txt** file.

```
#db-name    vol-path                db-host       log-
path                  lob-base-path
testdb     /home/cubrid/DB/testdb     nodeA:nodeB   /home/cubrid/DB/
testdb/log file:/home/cubrid/DB/testdb/lob
```

The **cubrid_broker.conf** file can be set in a variety of ways according to configuration of the broker. It can also be configured as separate equipment with the **databases.txt** file.

The example below shows that the RW broker is set in each node, and *node A* and *node B* have the same value.

```
[%RW_broker]
...
```

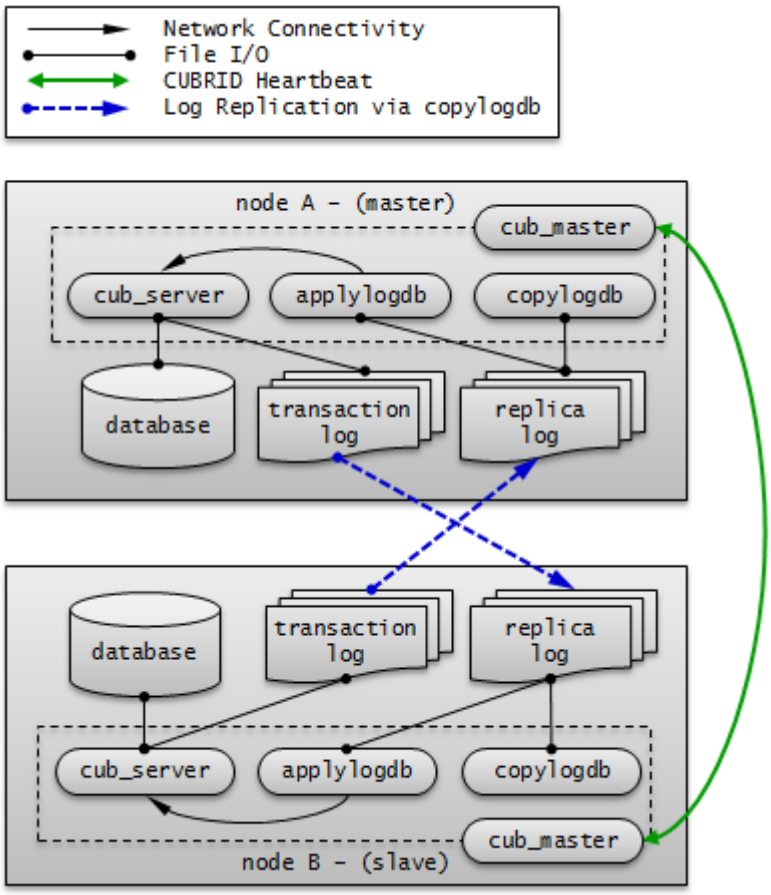
```
# Broker mode setting parameter
    ACCESS_MODE             =RW
```

Connection Configuration of Applications

See JDBC Configuration, CCI Configuration, and PHP Configuration in Environment Configuration.

Remark

The moving path of a transaction log in these configurations is as follows:



Multiple-Slave Node Structure

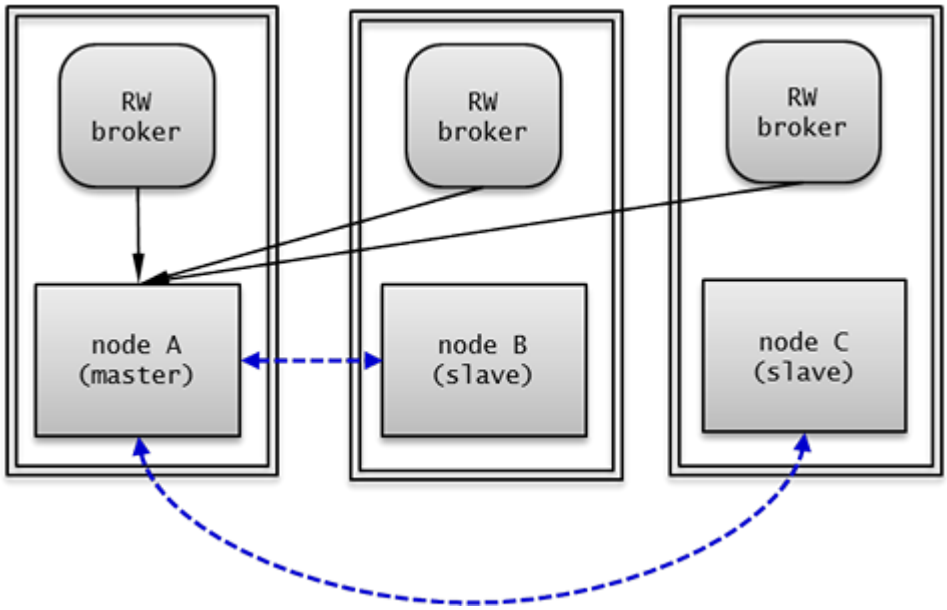
In multiple-slave node structure, there is one master node and several slave nodes to improve the service availability of CUBRID.

Because replication log copy process and replication log reflection process are running at all nodes in the CUBRID HA group, a load of copying replication log occurs. Therefore, all nodes in the CUBRID HA group have high network and disk usage.

Because there are many nodes with HA enabled, read and write services never fail as long as a single node is alive.

In the multiple-slave node structure, the node becoming a master node when failover occurs is determined by the order specified in **ha_node_list**. If the value of **ha_node_list** is node1:node2:node3 and the master node is *node A*, *node B* will become a new master node when the master node fails.

An Example of Node Configuration



You can configure each node in the basic structure of HA as shown below:

- **node A** (master node)
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

  - The following example shows how to configure **cubrid_ha.conf**:

```
ha_port_id=59901
ha_node_list=cubrid@nodeA:nodeB:nodeC
ha_db_list=testdb
ha_copy_sync_mode=sync:sync:sync
```

- **node B**, **node C** (slave node): Configure this node in the same manner as *node A*.

You must enter the list of hosts configured in HA in order of priority in the **databases.txt** file of a broker node. The following is an example of the **databases.txt** file.

```
#db-name   vol-path                  db-host           log-
path                  lob-base-path
```

```
testdb      /home/cubrid/DB/testdb      nodeA:nodeB:nodeC    /home/
cubrid/DB/testdb/log file:/home/cubrid/DB/testdb/lob
```

The **cubrid_broker.conf** file can be set in a variety of ways according to configuration of the broker. It can also be configured as separate equipment with the **databases.txt** file. In this example, the RW broker is configured in *node A*, *node B*, and *node C*.

The following is an example of the **databases.txt** file in *node A*, *node B*, and *node C*.

```
[%RW_broker]
...

# Broker mode setting parameter
ACCESS_MODE              =RW
```

Connection Configuration of Applications

Connect the application to access to the broker of *node A*, *node B*, or *node C*.

```
Connection connection = DriverManager.getConnection(
    "jdbc:CUBRID:nodeA:33000:testdb:::?charSet=utf-8&altHosts=nodeB:
33000,nodeC:33000", "dba", "");
```

For details, see JDBC Configuration, CCI Configuration, and PHP Configuration in Environment Configuration.

Note

The data in the CUBRID HA group may lose integrity when there are multiple failures in this structure and the example is shown below.

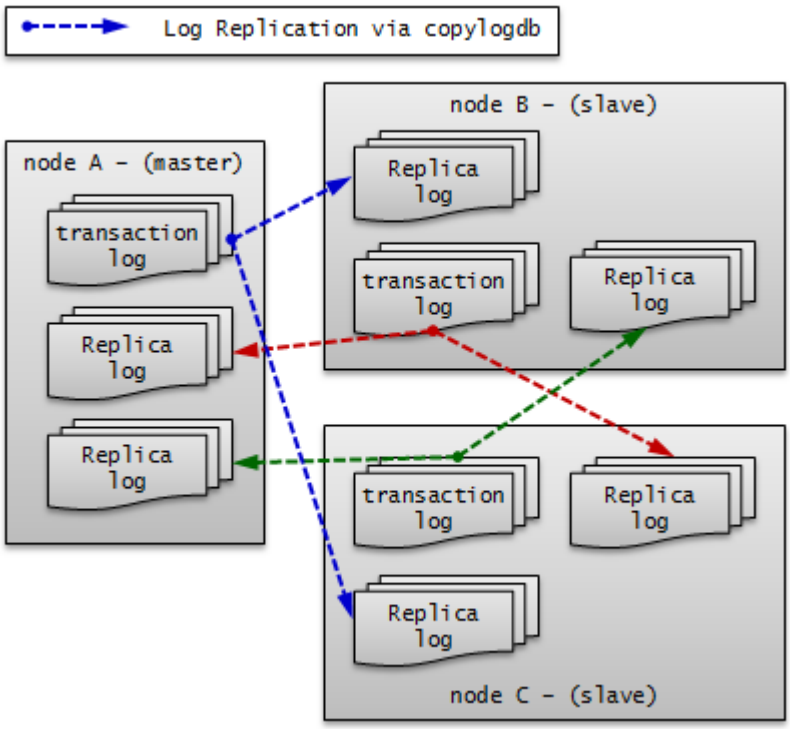
- n a situation where a failover occurs in the first slave node while replication in the second slave node is being delayed due to restart
- In a situation where a failover re-occurs before replication reflection of a new master node is not complete due to frequent failover

In addition, if the mode of replication log copy process is ASYNC, the data in the CUBRID HA group may lose integrity.

If the data in the CUBRID HA group loses integrity for any of the reasons above, all data in the CUBRID HA group should be fixed as the same by using Building Replication or Rebuilding Replication Script.

Remark

The moving path of a transaction log in these configurations is as follows:



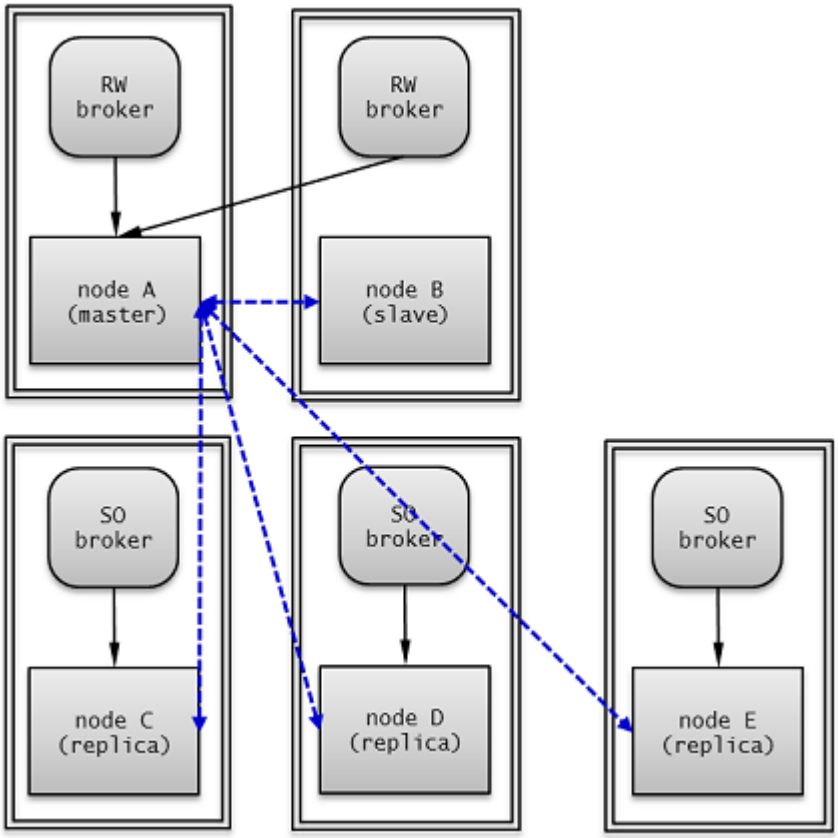
Load Balancing Structure

The load balancing structure increases the availability of the CUBRID service by placing several nodes in the HA configuration (one master node and one slave node) and distributes read-load.

Because the replica nodes receive replication logs from the master node in the HA configuration and maintain the same data, and because the master node in the HA configuration does not receive replication logs from the replica nodes, its network and disk usage rate is lower than that of the multiple-slave structure.

Because replica nodes are not included in the HA structure, they provide read service without failover, even when all other nodes in the HA structure fail.

An Example of Node Configuration



You can configure each node in load balancing structure as shown below:

- **node A** (master node)
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

  - The following example shows how to configure **cubrid_ha.conf**:

```
ha_port_id=59901
ha_node_list=cubrid@nodeA:nodeB
ha_replica_list=cubrid@nodeC:nodeD:nodeE
ha_db_list=testdb
ha_copy_sync_mode=sync:sync
```

- **node B** (slave node): Configure this node in the same manner as *node A*.
- **node C**, **node D**, **node E** (replica node)
  - Configure the **ha_mode** of the **cubrid.conf** file to **replica**.

```
ha_mode=replica
```

- You can configure the **cubrid_ha.conf** file in the same manner as *node A*.

The **cubrid_broker.conf** can be set in a variety of ways according to configuration of the broker. It can also be configured as separate equipment with the **databases.txt** file.

In this example, broker and DB server exist on the same machine; the RW broker is configured in *node A* and *node B*; the SO broker with "CONNECT_ORDER=RANDOM" and "PREFERRED_HOSTS=localhost" is configured in *node C*, *node D* and *node E*. each of *node C*, *node D* or *node E* tries to connect to local DB server first because they are set as "PREFERRED_HOSTS=localhost". When it is failed to connect to localhost, it tries to connect to one of **db-hosts** in **databases.txt** randomly because they are set as "CONNECT_ORDER=RANDOM".

The following is an example of **cubrid_broker.conf** in *node A* and *node B*.

```
[%RW_broker]
...

# Broker mode setting parameter
ACCESS_MODE             =RW
```

The following is an example **cubrid_broker.conf** in *node C*, *node D* and *node E*.

```
[%PHRO_broker]
...

# Broker mode setting parameter
ACCESS_MODE             =SO
PREFERRED_HOSTS         =localhost
```

You must enter the list of DB server hosts in the order so that each broker can be connected appropriate HA or load balancing server in the **databases.txt** file of a broker node.

The following is an example of the **databases.txt** file in *node A* and *node B*.

```
#db-name    vol-path                db-host       log-
path            lob-base-path
testdb     /home/cubrid/DB/testdb   nodeA:nodeB   /home/cubrid/DB/
testdb/log file:/home/cubrid/CUBRID/testdb/lob
```

The following is an example of the **databases.txt** file in *node C*, *node D* and *node E*.

```
#db-name    vol-path                db-host            log-
path                    lob-base-path
testdb     /home/cubrid/DB/testdb   nodeC:nodeD:nodeE  /home/cubrid/
DB/testdb/log  file:/home/cubrid/CUBRID/testdb/lob
```

Connection Configuration of Applications

Connect the application to access in read/write mode to the broker of *node A* or *node B*. The following is an example of a JDBC application.

```
Connection connection = DriverManager.getConnection(
    "jdbc:CUBRID:nodeA:33000:testdb:::?charSet=utf-8&altHosts=nodeB:
33000", "dba", "");
```

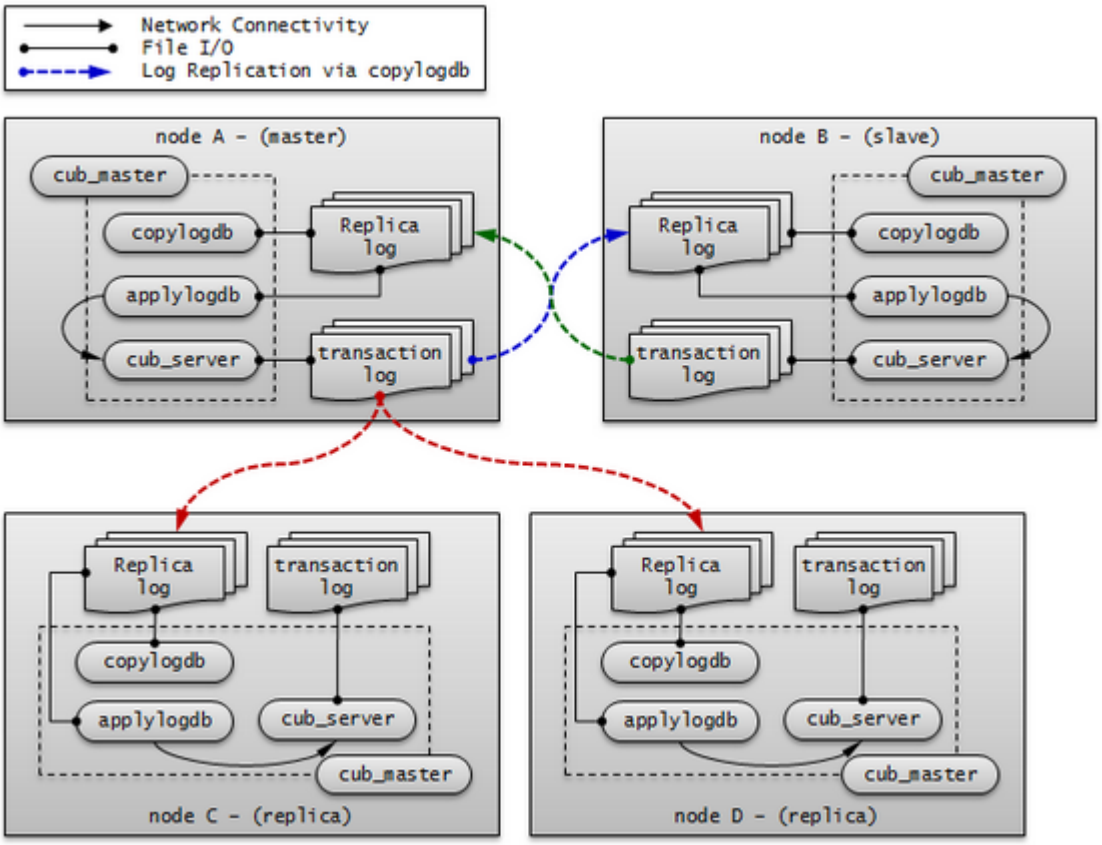
Connect the application to access in read-only mode to the broker of *node C*, *node D* or *node E*. The following is an example of a JDBC application. Configure "**loadBalance**=sh" on the URL to connect randomly to the main host and hosts which are specified by **altHosts**.

```
Connection connection = DriverManager.getConnection(
    "jdbc:CUBRID:nodeC:33000:testdb:::?
charSet=utf-8&loadBalance=sh&altHosts=nodeD:33000,nodeE:33000",
"dba", "");
```

For details, see JDBC Configuration, CCI Configuration, and PHP Configuration in Environment Configuration.

Remark

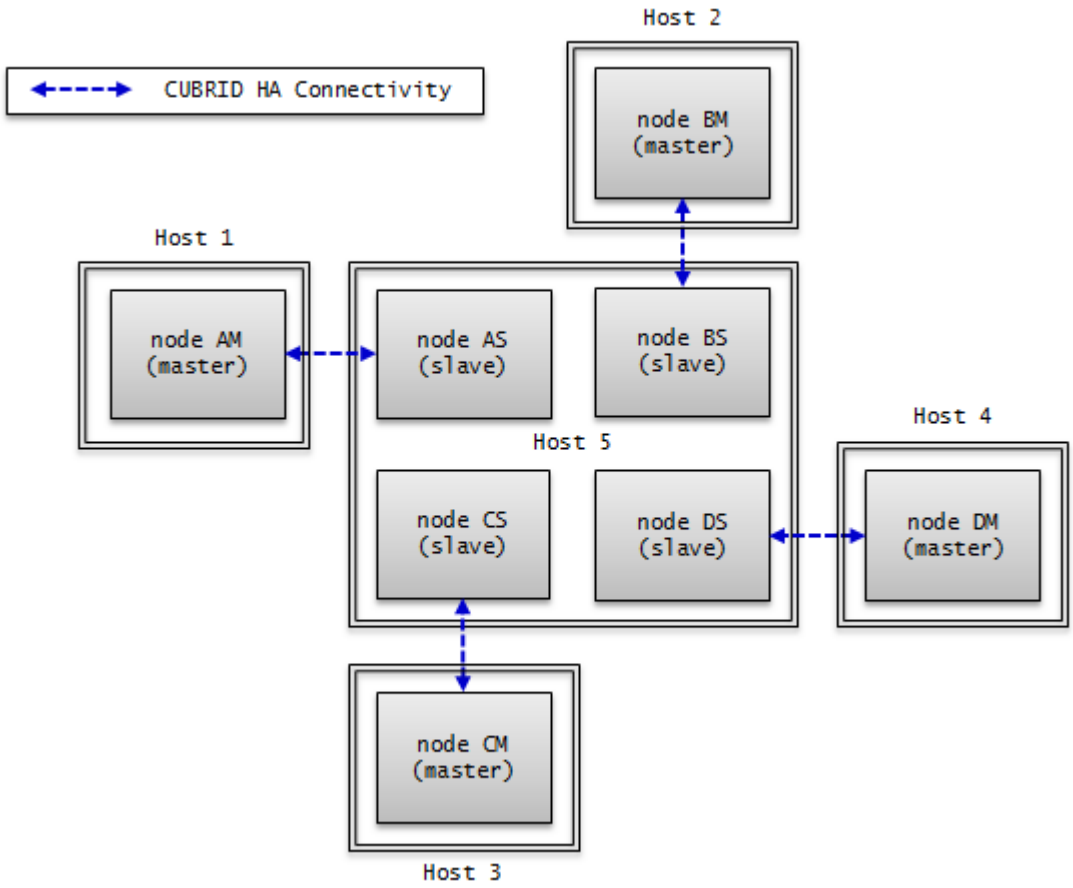
The path of a transaction log in these configurations is as follows:



Multiple-Standby Server Structure

Although its node structure has a single master node and a single slave node, many slave nodes from different services are physically configured in a single server.

This structure is for very small services in which the reading load of slave nodes is light. It is strictly for the availability of the CUBRID service. For this reason, when a master node that failed after a failover has been restored, the load must be moved back to the original master node to minimize the load of the server with multiple-slave nodes.



An Example of Node Configuration

You can configure each node in the basic structure of HA as shown below:

- **node AM**, **node AS** : Configure them in the same manner.
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

  - The following example shows how to configure **cubrid_ha.conf**.

```
ha_port_id=10000
ha_node_list=cubridA@Host1:Host5
ha_db_list=testdbA1,testdbA2
ha_copy_sync_mode=sync:sync
```

- **node BM**, **node BS** : Configure them in the same manner.
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

- The following example shows how to configure **cubrid_ha.conf**.

```
ha_port_id=10001
ha_node_list=cubridB@Host2:Host5
ha_db_list=testdbB1,testdbB2
ha_copy_sync_mode=sync:sync
```

- **node CM**, **node CS** : Configure them in the same manner.
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

  - The following example shows how to configure **cubrid_ha.conf**.

```
ha_port_id=10002
ha_node_list=cubridC@Host3:Host5
ha_db_list=testdbC1,testdbC2
ha_copy_sync_mode=sync:sync
```

- **node DM**, **node DS** : Configure them in the same manner.
  - Configure the **ha_mode** of the **cubrid.conf** file to **on**.

```
ha_mode=on
```

  - The following is an example of the **cubrid_ha.conf** configuration.

```
ha_port_id=10003
ha_node_list=cubridD@Host4:Host5
ha_db_list=testdbD1,testdbD2
ha_copy_sync_mode=sync:sync
```

HA Constraints

Supported Platforms

Currently, CUBRID HA is supported by Linux only. All nodes within CUBRID HA groups must be configured on the same platforms.

Table Primary Key

CUBRID HA synchronizes data among nodes with the following method (as known as transaction log shipping): It replicates the primary key-based replication logs generated from the server of a master node to a slave node and then reflects the replication logs to the slave node.

If data of the specific table within CUBRID HA groups is not synchronized, you should check whether the appropriate primary key has specified for the table.

On the partitioned table, the table which has promoted some partitions by the **PROMOTE** statement replicates all data to the slave. However, since the table does not have the primary key, the data changes on the table made by the master are not applied to the slave.

Note

The data replication of tables without a primary key is supported by **USE_SBR** hint. For more information, see Using SQL Hint.

Stored Procedure

In CUBRID HA, **PL/CSQL stored procedures** are replicated, but **Java stored procedures** only replicate the declaration, not the execution program. Therefore, the execution program must be manually copied to all nodes in the HA.

To execute stored procedures, the CUBRID procedural langauge server must be running, which is automatically started when the database server starts.

However, the **cubrid.conf** configuration for running the CUBRID language server is not replicated, so you must ensure that **stored_procedure** is set to **yes** (default) on both the master and slave nodes.

Method

CUBRID HA synchronizes data among nodes within CUBRID HA groups based on replication logs, So using method that does not generate replication logs may cause data inconsistency among nodes within CUBRID HA groups.

Therefore, in CUBRID HA environment, it is not recommended to use method. (e.g. CALL login('dba', '') ON CLASS dbuser;)

Standalone Mode

The replication logs are not generated as for tasks performed in standalone mode. For this reason, data inconsistency among nodes within CUBRID HA groups may occur when performing tasks in standalone mode.

Serial Cache

To enhance performance, a serial cache does not access Heap and does not generate replication logs when retrieving or updating serial information. Therefore, if you use a serial cache, the current values of serial caches will be inconsistent among the nodes within CUBRID HA groups.

cubrid backupdb -r

This command is used to back up a specified database. If the **-r** option is used, logs that are not required for recovery will be deleted. This deletion may result in data inconsistency among nodes within CUBRID HA groups. Therefore, you must not use the **-r** option.

Online backup

If you want to perform online backup in HA environment, add @*hostname*after the database name. *hostname*is a name defined in $CUBRID_DATABASES/databases.txt. Specify "@localhost" because you generally perform online backup on the local database.

```
cubrid backupdb -C -D ./ -l 0 -z testdb@localhost
```

Backup during running database may occur disk I/O load. Therefore, it is recommended to run backup on slave DB than master DB.

INCR/DECR Functions

If you use **INCR** / **DECR** (click counter functions) in a slave node of HA configuration, an error is returned.

LOB (BLOB/CLOB) Type

In a CUBRID HA environment, the meta data (Locator) of a **LOB** column is replicated and **LOB** data is not replicated. Therefore, if storage of a **LOB** type is located on the local machine, no tasks corresponding to columns are allowed in slave nodes or master nodes after failover.

Note

On previous version of CUBRID 9.1, using triggers in CUBRID HA can cause duplicate executions. This may cause data inconsistency among nodes within CUBRID HA groups. Therefore, you should not use triggers on the previous version of 9.1.

Note

UPDATE STATISTICS

From 10.0, UPDATE STATISTICS statement is replicated.

In the previous version of 10.0, UPDATE STATISTICS statement is not replicated; therefore, as a separated operation, you should run this statement in the slave/replica node. When you want to apply UPDATE STATISTICS on the slave/replica node in the previous version of 10.0, you should run this in the CSQL with **--sysadm** and **--write_on_slave** options.

Operational Scenarios

Operation Scenario during Read/Write Service

The operation scenario written in this page is not affected by read/write services. Therefore, its impact on the services caused by CUBRID operation is very limited. There can be two types of operation scenarios in which failover occurs or it does not occur.

When Failover Does Not Occur

You can perform the following operations without stopping and restarting nodes in CUBRID HA groups.

General Operation	Scenario	Consideration
Online Backup	Operation task is performed at each master node and slave node each during operation.	Note that there may be a delay in the transaction of master node due to the operation task.
Schema change (excluding basic key change), index change, authorization change	When an operation task occurs at a master node, it is automatically replication reflected to a slave node.	Because replication log is copied and reflected to a slave node after an operation task is completed in a master node, operation task time is doubled. Changing schema must be processed without any failover. Index change and authority change other than the schema change can be performed by stopping each node and executing standalone mode (ex: the **-S** option of the **csql** utility)

General Operation	Scenario	Consideration
		when the operation time is important.
Add volume	Operation task is performed at each DB regardless of HA structure.	Note that there may be a delay in the transaction of master node due to the operation task. If operation task time is an issue, operation task can be performed by stopping each node and executing standalone mode (ex: the **-S** of the **cubrid addvoldb** utility).
Failure node server replacement	It can be replaced without restarting the CUBRID HA group when a failure occurs.	The failure node must be registered in the ha_node_list of CUBRID HA group, and the node name must not be changed during replacement.
Failure broker server replacement	It can be replaced without restarting the broker when a failure occurs.	The connection to a broker replaced at a client can be made by rcTime which is configured in URL string.
DB server expansion	You can execute **cubrid heartbeat reload** in each node after configuration change (ha_node_list, ha_replica_list) without restarting the previously configured CUBRID HA group.	Starts or stops the **copylogdb/applylogdb** processes which were added or deleted by loading changed configuration information.
Broker server expansion		

General Operation	Scenario	Consideration
	Run additional brokers without restarting existing brokers.	Modify the URL string to connect to a broker where a client is added.

When Failover Occurs

You must stop nodes in CUBRID HA group and complete operation before performing the following operations.

General Operation	Scenario	Consideration
DB server configuration change	A node whose configuration is changed is restarted when the configuration in **cubrid.conf** is changed.	
Change broker configuration, add broker , and delete broker	A broker whose configuration is changed is restarted when the configuration in **cubrid_broker.conf** is changed.	
DBMS version patch	Restart nodes and brokers in HA group after version patch.	Version patch means there is no change in the internal protocol, volume, and log of CUBRID.

Operation Scenario during Read Service

The operation scenario written in this page is only applied to read service. It is required to allow read service only or dynamically change mode configuration of broker to Read Only. There can be two types of operation scenarios in which failover occurs or it does not occur.

When Failover Does Not Occur

You can perform the following operations without stopping and restarting nodes in CUBRID HA groups.

General Operation	Scenario	Consideration
Schema change (primary key change)	When an operation task is performed at the master node, it is automatically reflected to the slave node.	In order to change the primary key, the existing key must be deleted and a new one added. For this reason, replication reflection may not occur due to the HA internal structure which reflects primary key-based replication logs. Therefore, operation tasks must be performed during the read service.
Schema change (excluding basic key change), index change, authorization change	When an operation task is performed at the master node, it is automatically reflected to the slave node.	Because replication log is copied and reflected to a slave node after an operation task is completed in a master node, operation task time is doubled. Changing schema must be processed without any failover. Index change and authority change other than the schema change can be performed by stopping each node and executing standalone mode(ex: the span class=”nkeyword”>-S option of **csql**) when the operation time is important.

When Failover Occurs

You must stop nodes in CUBRID HA group and complete operation before performing the following operations.

General Operation	Scenario	Consideration
DBMS version upgrade	Restart each node and broker in the CUBRID HA group after they are upgraded.	A version upgrade means that there have been changed in the internal protocol, volume, or log of CUBRID. Because there are two different versions of the protocols, volumes, and logs of a broker and server during an upgrade, an operation task must be performed to make sure that each client and broker (before/after upgrade) are connected to the corresponding counterpart in the same version.
Massive data processing (INSERT/UPDATE/DELETE)	Stop the node that must be changed, perform an operation task, and then execute the node.	This processes massive data that cannot be segmented.

Operation Scenario after Stopping a Service

You must stop all nodes in CUBRID HA group before performing the following operation.

General Operation	Scenario	Consideration
Changing the hostname and IP of a DB server	Stop all nodes in the CUBRID HA group, and restart them after the operation task.	When a hostname has been changed, change the **databases.txt** file of each broker and reset the broker connection with **cubrid broker reset**.

Setting Replica Replication Delay

This scenario delays the replication of the master node data in replica node, and stops the replication of the master node data at the specific time, to detect the case which someone deleted the data by mistake and stop the replication at the specified time.

General Operation	Scenario	Consideration
Setting the delay of replication in replica node	Specify the term of replicated delay to replica node, and let the replication stop on the specified time	specify **ha_replica_delay** and ha_replica_time_bound in cubrid.conf

Building Replication

restoreslave

cubrid restoreslave is the same as **cubrid restoredb** which restores a database from a backup but it includes several convenient features when rebuilding a slave or a replica. With **cubrid restoreslave**, user does not need to manually collect replication-related information from a backup output to create a replication catalog which is stored in **db_ha_apply_info**. It automatically reads any necessary information from a backup image and an active log and then it adds the relevant replication catalog into **db_ha_apply_info**. All you need to do is to provide two mandatory options: the state of the node where the backup image was created, and the hostname of the current master node. Please also refer restoredb for more details.

```
cubrid restoreslave [OPTION] database-name
```

-s, --source-state=STATE

You need to specify the state of the node where the backup image was created. STATE may be 'master', 'slave', or 'replica'

-m, --master-host-name=NAME

You need to specify the hostname of the current master node.

--list

This option displays information on backup files of a database; restoration procedure is not performed. For further information, see **--list** of restoredb

-B, --backup-file-path=PATH

You can specify the directory where backup files are to be located by using the -B option. For further information, see -B of restoredb

-o, --output-file=FILE

You can specify a filename to store output messages. For further information, see -o of restoredb

-u, --use-database-location-path

This option restores a database to the path specified in the database location file(databases.txt). For further information, see -u of restoredb

-k, --keys-file-path=PATH

This option specifies the path of the key file required to restore. For more information, see restoredb.

Example Scenarios of Building Replication

In this section, we will see various scenarios regarding adding a new node or removing a node during HA service.

Note

- Please note that only tables which have a primary key can be replicated.
- Please note that volume paths between a master node and a slave (or replica) node must be the same.
- A node includes replication processing information for all other nodes. For example, let's consider HA architecture with a master *nodeA*, and two slaves *nodeB* and *nodeC*. *nodeA* has the replication process information for *nodeB* and *nodeC*; *nodeB* has information for *nodeB* and *nodeC*; *nodeC* has information for *nodeA* and *nodeC*.
- Under the assumption of no failover, add a new node. If failover occurs during building replication, it is recommended to rebuild replication from the beginning.

- Add a Slave after Stopping a Service

  Add a new slave node after the service stops when operating a database with only a machine.

- Build Another Slave during HA Service

  Add a new slave using the existing slave node in the HA environment which one master and one slave are built.

- Remove a Slave during Service

  Remove a slave when one master and two slaves are built.

- Add a Replica during Service

  Add a new replica node using the existing slave in the environment which a master and a slave are built.

- Rebuild a Slave during Service

  Rebuild the slave using the existing master in the environment which a master, a slave and a replica are built. A new information should be updated by a user in the db_ha_apply_info catalog table.

Now let's see in detail about the above scenarios.

When you want to rebuild only the abnormal node in the HA environment, see Rebuilding Replication Script.

Note

- During building a replication, please note that added or rebuilt node's replication information and replication logs should be updated or removed when you add, rebuild or remove a certain node. Replication information is saved on the db_ha_apply_info table, and it has other nodes' replication process information.

  However, because role change does not occur in a replica node, the other node is not required to have replication information and a replication logs about the replica node. On the other hand, the replica node has all information about a master and a slave.

- When you run a scenario of Rebuild a Slave during Service, you should update db_ha_apply_info's information directly.

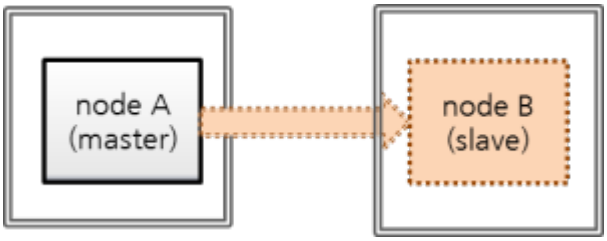
- When you run a scenario of Add a Slave after Stopping a Service, a slave is newly built; therefore, you can use the automatically added information.

- When you run a a scenario of Build Another Slave during HA Service or Add a Replica during Service, backed-up database from the existing slave is used; you can use as it exists because master's db_ha_apply_info information is included in the existing slave.

- When you run a scenario of Remove a Slave during Service, you must remove replication information about a removed slave. However, this information does not make a problem even if this is left.

Add a Slave after Stopping a Service

Now let's see the case of building a new slave node with a single master node - making a 1:1 master-slave composition.



- nodeA: a master node's hostname
- nodeB: a slave node's hostname to be added newly

This scenario assumes that the database has been created by below commands. When you run createdb, a locale name and a charset should be the same between a master node and a slave node.

```
export CUBRID_DATABASES=/home/cubrid/DB
mkdir $CUBRID_DATABASES/testdb
mkdir $CUBRID_DATABASES/testdb/log
cd $CUBRID_DATABASES/testdb
cubrid createdb testdb -L $CUBRID_DATABASES/testdb/log en_US.utf8
```

The backup file is saved in the $CUBRID_DATABASES/testdb directory by default if the location is not specified.

URL's change in an application is not required if a broker node is composed of a separated machine and if adding a new broker is not considered.

Note

If a broker's machine is not separated from a database's, to prepare a fault of a master node's machine, broker's access on a slave machine should be possible, too. To do this, altHosts should be added on an application's access URL and databases.txt's configuration should be changed for broker duplexing.

For broker's configuration on HA environment, see cubrid_broker.conf.

In this scenario, we assume that broker's node is not installed on a separated machine.

With the above in mind, let's work as follows.

1. Configure HA of master node and slave node.

   - Set the **$CUBRID/conf/cubrid.conf** identically on master and slave nodes.

```
[service]
service=server,broker,manager

# Add the database name to run when starting the service
server=testdb

[common]
...

# Add when configuring the HA (Logging parameters)
force_remove_log_archives=no

# Add when configuring the HA (HA mode)
ha_mode=on
```

   - Set the **$CUBRID/conf/cubrid_ha.conf** identically on master and slave nodes.

```
[common]
ha_port_id=59901

# cubrid is a group name of HA system, nodeA and nodeB are host
names.
ha_node_list=cubrid@nodeA:nodeB

ha_db_list=testdb
ha_copy_sync_mode=sync:sync
ha_apply_max_mem_size=500
```

   - Set the **$CUBRID_DATABASES/databases.txt** identically on master and slave nodes.

```
#db-name    vol-path                db-host         log-
path                    lob-base-path
testdb      /home/cubrid/DB/testdb  nodeA:nodeB     /home/cubrid/
DB/testdb/log  file:/home/cubrid/DB/testdb/lob
```

- Set brokers and applications in HA environment.

  For broker setting and adding altHosts into URL, see cubrid_broker.conf.

- Set locale library identically on master and slave nodes.

```
[nodeA]$ scp $CUBRID/conf/cubrid_locales.txt cubrid_usr@nodeB:
$CUBRID/conf/.
[nodeB]$ make_locale.sh -t64
```

- Create a directory to a slave node.

```
[nodeB]$ cd $CUBRID_DATABASES
[nodeB]$ mkdir testdb
```

- Create a log directory to a slave node. (create this on the same path with a master node.)

```
[nodeB]$ cd $CUBRID_DATABASES/testdb
[nodeB]$ mkdir log
```

2. Stop a master node, backup a master database.

```
[nodeA]$ cubrid service stop
```

Back up the database of the master node and copy the backup file to the slave node.

If the location where the backup file will be saved in the master node is not specified, the location is set as the log directory of *testdb* by default. Copy the backup file to the same location in the slave node. *testdb*_bk0v000 is the backup volume file and *testdb*_bkvinf is the backup volume information file.

```
[nodeA]$ cubrid backupdb -z -S testdb
Backup Volume Label: Level: 0, Unit: 0, Database testdb, Backup
Time: Thu Apr 19 16:05:18 2012
[nodeA]$ cd $CUBRID_DATABASES/testdb/log
[nodeA]$ scp testdb_bk* cubrid_usr@nodeB:$CUBRID_DATABASES/testdb/
log
cubrid_usr@nodeB's password:
testdb_bk0v000                                100% 6157KB   6.0MB/s
00:00
testdb_bkvinf                                 100%   66     0.1KB/s
00:00
```

3. Restore a database from a slave node.

Restore the database in the slave node. At this time, the volume path of the master node must be identical to that of the slave node.

```
[nodeB]$ cubrid restoredb -B $CUBRID_DATABASES/testdb/log demodb
```

4. Start a master node, start a slave node.

```
[nodeA]$ cubrid heartbeat start
```

After confirming that the master node has been started, start the slave node.

If *nodeA*'s state is changed from registered_and_to_be_active to registered_and_active, it means that the master node has been successfully started.

```
[nodeA]$ cubrid heartbeat status
@ cubrid heartbeat status

 HA-Node Info (current nodeA, state master)
   Node nodeB (priority 2, state unknown)
   Node nodeA (priority 1, state master)

 HA-Process Info (master 123, state master)

   Applylogdb testdb@localhost:/home1/cubrid/DB/tdb01_nodeB (pid
234, state registered)
   Copylogdb testdb@nodeB:/home1/cubrid/DB/tdb01_nodeB (pid 345,
state registered)
   Server tdb01 (pid 456, state registered_and_to_be_active)

[nodeB]$ cubrid heartbeat start
```

Confirm that the HA configurations of the master node and the slave node are successfully running

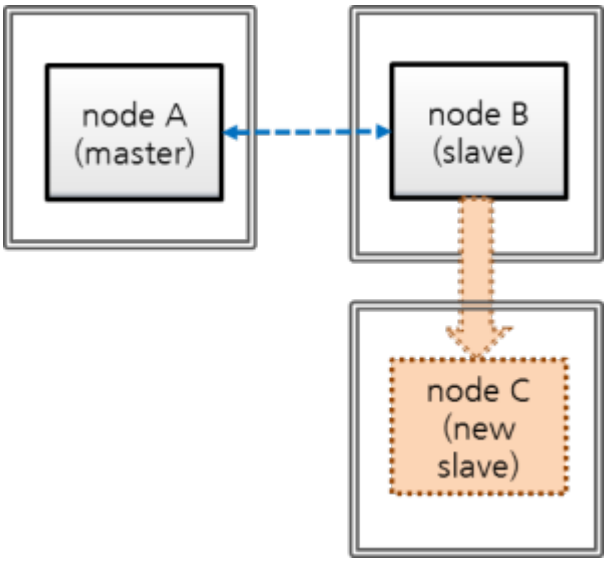
```
[nodeA]$ csql -u dba testdb@localhost -c"create table tbl(i int
primary key);insert into tbl values (1),(2),(3)"

[nodeB]$ csql -u dba testdb@localhost -c"select * from tbl"

            i
=============
            1
            2
            3
```

Build Another Slave during HA Service

The following is a scenario to add a new slave from an existing slave during a HA service. “Master:Slave” will be from 1:1 to 1:2.



- nodeA: a master node’s hostname
- nodeB: a slave node’s hostname
- nodeC: a slave node’s hostname to be added

You can use an existing master or slave if you want to add a new slave during HA service. In this scenario, let’s add a new slave by using an existing slave because we assume that slave’s disk I/O is less than master’s.

Note

Why possible to use a slave instead of a master when adding a node

The reason to use a slave instead of a master is that master’s transaction log (active log + archive log) is replicated to a slave as it is, and master node’s transaction logs and slave node’s replication logs (replicated transaction logs) are the same in format and content.

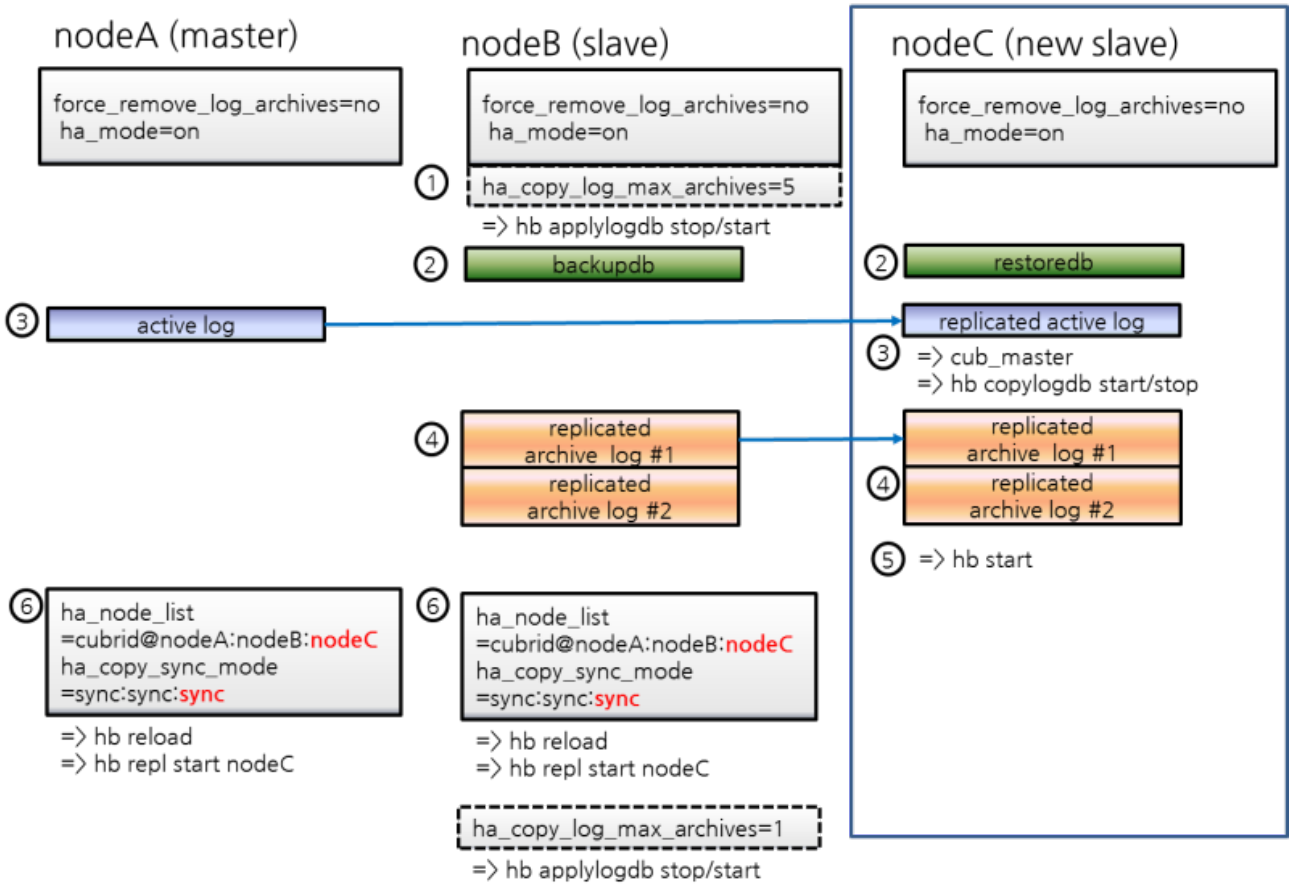
Note

Build an initial database

It assumes that the database is already created with the below command. a locale name and a charset should be the same between a master and a slave when you run “createdb”.

```
export CUBRID_DATABASES=/home/cubrid/DB
mkdir $CUBRID_DATABASES/testdb
mkdir $CUBRID_DATABASES/testdb/log
cd $CUBRID_DATABASES/testdb
cubrid createdb testdb -L $CUBRID_DATABASES/testdb/log en_US.utf8
```

At this time, if you don't specify backup files' saving path as an option, backup files are saved on the log directory specified in databases.txt.



1. HA configuration of *nodeC*

   ◦ Set **$CUBRID/conf/cubrid.conf** of *nodeC* identically with *nodeB*'s.

   "force_remove_log_archives=no" should be set not to let no-replicated logs be deleted during HA service.

```
[service]
service=server,broker,manager

[common]
...
```

```
# Add when configuring the HA (Logging parameters)
force_remove_log_archives=no

# Add when configuring the HA (HA mode)
ha_mode=on
```

- Set **$CUBRID/conf/cubrid_ha.conf** of *nodeC* identically with *nodeB*'s.

  However, add *nodeC* into ha_node_list, and add one more "sync" into ha_copy_sync_mode.

```
[common]
ha_port_id=59901

# cubrid is a group name of HA system, nodeA, nodeB, and nodeC
are host names.
ha_node_list=cubrid@nodeA:nodeB:nodeC

ha_db_list=testdb
ha_copy_sync_mode=sync:sync:sync
ha_apply_max_mem_size=500
```

- Set a locale library of *nodeC* identically with that of *nodeA* or *nodeB*.

```
[nodeB]$ scp $CUBRID/conf/cubrid_locales.txt cubrid_usr@nodeC:
$CUBRID/conf/.
[nodeC]$ make_locale.sh -t64
```

- Add *nodeA*, *nodeB*, and *nodeC* into db-host of **$CUBRID_DATABASES/databases.txt** in *nodeC*.

```
#db-name     vol-path                  db-host             log-
path                        lob-base-path
testdb       /home/cubrid/DB/testdb     nodeA:nodeB:nodeC   /
home/cubrid/DB/testdb/log      file:/home/cubrid/DB/testdb/lob
```

  db-host of **databases.txt** is related to the access order to the database server by a broker; therefore, you can change the value as you want. For example, you can write only nodeA:nodeB or localhost into **db-host**. However, by considering the local machine access, (e.g. csql -u dba testdb@localhost) you should add at least localhost or the name of localhost into **db-host**.

2. Apply configuration for preventing from deleting replication logs of *nodeB* during restoring after backup

   Replication logs of *nodeB* can be added if service continues after restoring a database from the *nodeB* backup file to *nodeC*. By the way, slave's replication logs can be deleted before being

copied to *nodeC* by configuration. To prevent this case, you can configure system parameters as follows.

- Change the value of **ha_copy_log_max_archives** bigger without restarting *nodeB*'s database.

  From backup of *nodeB*, transactions which have been executed after backup of *nodeB* should be kept; then transactions after backup can be applied to *nodeC*.

  Therefore, added replication logs of *nodeB* after backup should be kept.

  Here, let's assume the number of replication logs as 5, which can sufficiently save executed transactions during building *nodeC*.

  Edit **$CUBRID/conf/cubrid_ha.conf**

```
ha_copy_log_max_archives=5
```

  Apply the change of the parameter **ha_copy_log_max_archives** to the applylogdb process of *nodeB*.

```
[nodeB]$ cubrid heartbeat applylogdb stop testdb nodeA
[nodeB]$ cubrid heartbeat applylogdb start testdb nodeA
```

  Note

  If you build *nodeC* with only *nodeA*, a master node, to change the value of **ha_copy_log_max_archives** from *nodeB* is needless because copying replication logs from *nodeB* is needless.

  Instead, you should apply the changed **log_max_archives** to *nodeA*.

  The value of **log_max_archives** parameter of cubrid.conf can be changed during service by using "SET SYSTEM PARAMETERS" syntax.

```
[nodeA]$ csql -u dba -c "SET SYSTEM PARAMETERS
'log_max_archives=5'" testdb@localhost
```

3. Backup from *nodeB*, restore into *nodeC*

   - Backup a database from *nodeB*.

```
[nodeB]$ cd $CUBRID_DATABASES/testdb/log
[nodeB]$ cubrid backupdb --sleep-msecs=10 -C -o output.txt
testdb@localhost
```

Note

--sleep-msecs is an option to set the rest time in each cycle when 1MB is written in a backup file.The unit is millisecond. Consider setting this option if the disk I/O overhead of a machine to back up is severe; however, it is recommended to set this value as small as possible and do backup when backup overhead is small enough because if this value is bigger, backup time takes longer.

If there is no I/O overhead on *nodeB*, you can omit this option.

On the file specified in -o option, the backup result information is written.

- Copy backup files to *nodeC*.

```
[nodeB]$ cd $CUBRID_DATABASES/testdb/log
[nodeB]$ scp -l 131072 testdb_bk* cubrid_usr@nodeC:
$CUBRID_DATABASES/testdb/log
```

Note

scp command's -l option is an option to control the amount of copying file; when you copy files between nodes, use this value properly as considering I/O overhead. The unit is Kbits. 131072 is 16MB.

- Restore the database into *nodeC*.

  At this time, the volume path of the slave must be the same with the master.

```
[nodeC]$ cubrid restoredb -B $CUBRID_DATABASES/testdb/log testdb
```

4. Copy the active log of *nodeA* into *nodeC*

   In *nodeA*'s active log, there exists a number information of the archive log which is recently created. This information is needed for archive logs, which has been created after the time in which the active log had been copied, to be automatically copied by copylogdb process after

the start of HA service of *nodeC*; therefore, the user just needs to manually copy archive logs which has been created before copying an active log.

- Start cub_master which manages HA connection in *nodeC*.

```
[nodeC]$ cub_master
```

- Start copylogdb process for *nodeA*.

```
[nodeC]$ cubrid heartbeat copylogdb start testdb nodeA
```

- Check if *nodeA*'s active log was copied.

  To copy the current active log means to obtain the number information of the last archive log. Archive logs after this time are automatically copied after the process [5. Start HA server in *nodeC*].

```
[nodeC]$ cd $CUBRID_DATABASES/testdb_nodeA
[nodeC]$ ls
testdb_lgar_t  testdb_lgat  testdb_lgat__lock
```

- Stop copylogdb process for *nodeA* on *nodeC*.

```
[nodeC]$ cubrid heartbeat copylogdb stop testdb nodeA
```

- Stop cub_master of *nodeC*.

```
[nodeC]$ cubrid service stop
```

5. Copy required log files after restoring a database into *nodeC*

- Copy all replicated archive logs of *nodeB* into *nodeC*.

```
[nodeC]$ cd $CUBRID_DATABASES/testdb_nodeA
[nodeC]$ scp -l 131072 cubrid@nodeB:$CUBRID_DATABASES/
testdb_nodeA/testdb_lgar0* .
```

  In this process, please note that all replicated archive logs which are required for replication should exist.

Note

scp command's -l option is an option to control the amount of copying file; when you copy files between nodes, use this value properly as considering I/O overhead. The unit is Kbits. 131072 is 16MB.

Note

Copy files of which the names end with a number. Here, we copied with "testdb_lgar0*" because numbers of names of all replication log files are started with 0.

Note

Preparation plan when the required replication logs are already deleted

Replication logs which are created between backup time and active log creation time can be deleted if the value of **ha_copy_log_max_archives** is not large enough; in this case, an error can occur when running the below process, [5. Start HA service on *nodeC*].

When an error occurs, start HA service of *nodeC* by adding archive logs in master if the required logs exist in master node. For example, you can copy an archive log #1 from master if there are archive logs #1, #2, and #3 in master; and there are only replicated archive logs #2, and #3 in slave and #1 is required additionally. However, the case in which master's archive log files are left more can happen only when the value of **log_max_archives** of master is big enough.

If there is no wanted number's archive log in master's archive logs, change the value of **ha_copy_log_max_archives** big enough and do backup-and-restore procedure from the beginning.

Note

Check if required replication logs are deleted

You can check if requried replication logs are deleted or not by watching the testdb_lginf file.

1. If this is the case of restoring a database from a slave backup file, you can check what pages are required, from the page of master by watching the value of required_lsa_pageid in the db_ha_apply_info catalog table.

2. Watch the contents of the testdb_lginf file located in $CUBRID_DATABASES/testdb/log directory from bottom to top.

```
Time: 03/16/15 17:44:23.767 - COMMENT: CUBRID/LogInfo for
database /home/cubrid/DB/databases/testdb/testdb
Time: 03/16/15 17:44:23.767 - ACTIVE: /home/cubrid/DB/
databases/testdb/log/testdb_lgat 1280 pages
Time: 03/16/15 17:54:40.892 - ARCHIVE: 0 /home/cubrid/DB/
databases/testdb/log/testdb_lgar000 0 1277
Time: 03/16/15 17:57:29.451 - COMMENT: Log archive /home/
cubrid/DB/databases/testdb/log/testdb_lgar000 is not needed
any longer unless a database media crash occurs.
Time: 03/16/15 18:03:08.166 - ARCHIVE: 1 /home/cubrid/DB/
databases/testdb/log/testdb_lgar001 1278 2555
Time: 03/16/15 18:03:08.167 - COMMENT: Log archive /home/
cubrid/DB/databases/testdb/log/testdb_lgar000, which contains
log pages before 2556, is not needed any longer by any HA
utilities.
Time: 03/16/15 18:03:29.378 - COMMENT: Log archive /home/
cubrid/DB/databases/testdb/log/testdb_lgar001 is not needed
any longer unless a database media crash occurs.
```

When the contents of testdb_lginf is written as the above, the last two numbers in “Time: 03/16/15 17:54:40.892 - ARCHIVE: 0 /home/cubrid/DB/databases/testdb/log/testdb_lgar000 0 1277” are the start and end IDs of pages which this file store.

For example, if the page ID at backup is 2300, we can see this ID is between no. 1278 and no. 2555 by watching “testdb_lgar001 1278 2555”; therefore, we can know that the archive logs of master (or replicated archive logs of slave) are required from no. 1 when restoring a database.

6. Start HA service on *nodeC*

```
[nodeC]$ cubrid heartbeat start
```

7. Change HA configuration of *nodeA* and *nodeB*

   - Turn back the value of ha_copy_log_max_archive of *nodeB*.

   ```
   [nodeB]$ vi cubrid_ha.conf

   ha_copy_log_max_archives=1
   ```

   - Apply the changed value of ha_copy_log_max_archives to the applylogdb process of *nodeB*.

   ```
   [nodeB]$ cubrid heartbeat applylogdb stop testdb nodeA
   [nodeB]$ cubrid heartbeat applylogdb start testdb nodeA
   ```

   - In cubrid_ha.conf, add *nodeC* into ha_node_list, and add one "sync" more to *nodeA* and *nodeB*.

   ```
   $ cd $CUBRID/conf
   $ vi cubrid_ha.conf

   ha_node_list=cubrid@nodeA:nodeB:nodeC
   ha_copy_sync_mode=sync:sync:sync
   ```

   - Apply changed ha_node_list.

   ```
   [nodeA]$ cubrid heartbeat reload
   [nodeB]$ cubrid heartbeat reload
   ```

   - Start coplylogdb and applylogdb for *nodeC*.

   ```
   [nodeA]$ cubrid heartbeat repl start nodeC
   [nodeB]$ cubrid heartbeat repl start nodeC
   ```

8. Add a broker and add this broker's hostname into altHosts of application URL

   Add a broker if needed, and add this broker's hostname into altHosts, the property of application URL to access this broker.

   If you are not considering to add a broker, you do not need to do this job.

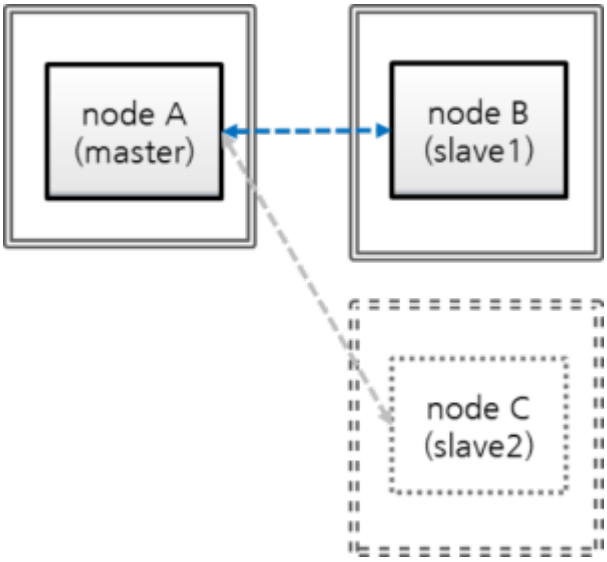
   - From all of *nodeA* and *nodeB*, add *nodeC* into db-host of **$CUBRID_DATABASES/databases.txt**.

```
#db-name      vol-path                    db-host             log-
path                          lob-base-path
testdb        /home/cubrid/DB/testdb      nodeA:nodeB:nodeC   /home/
cubrid/DB/testdb/log      file:/home/cubrid/DB/testdb/lob
```

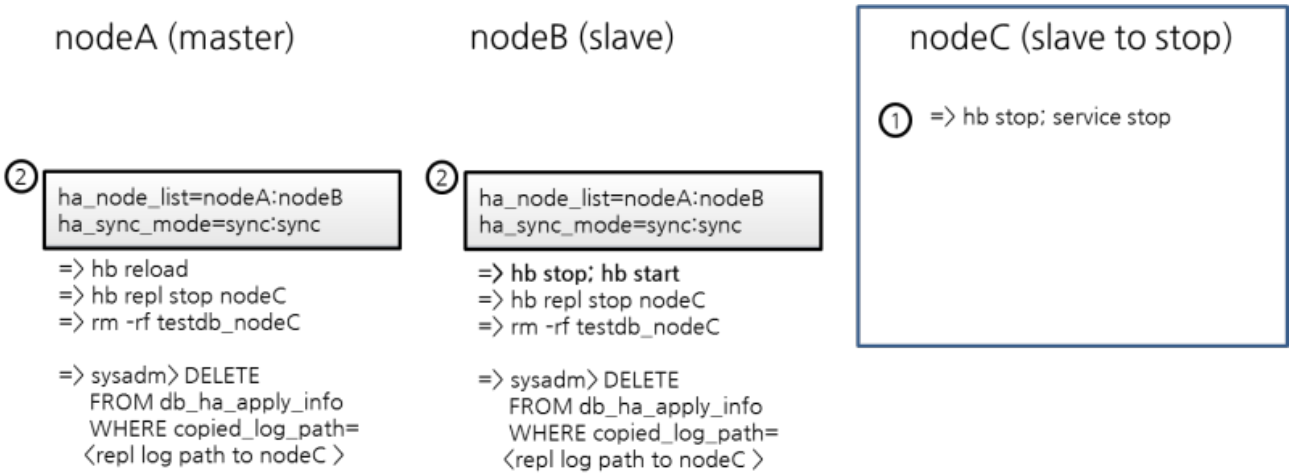
db-host of **databases.txt** is a configuration parameter related to the order of access from a broker to database servers; therefore, you can change as you want.For example, you can write only “nodeA:nodeB” or “localhost” into **db-host**. However, if you access database from a local machine (e.g. csql -u dba testdb@localhost), you should include “localhost” or the localhost name into **db-host**.

Remove a Slave during Service

Let’s remove a slave when the HA environment is composed of “master:slave = 1:2”.



- nodeA: a master node’s hostname
- nodeB: a slave node’s hostname
- nodeC: a slave node’s hostname to be removed



1. Stop the service of *nodeC*

```
$ cubrid heartbeat stop     # Processes related to heartbeat
(applylogdb, copylogdb, cub_server) are stopped.
$ cubrid service stop       # Processes such as cub_master,
cub_broker, and cub_manager are stopped.
```

2. Remove *nodeC* from the configuration of *nodeA*, and *nodeB*

   - Remove *nodeC* in ha_node_list from cubrid_ha.conf of *nodeA* and *nodeB*; remove one “sync” in ha_copy_sync_mode.

     ```
     $ vi cubrid_ha.conf

     ha_node_list=cubrid@nodeA:nodeB
     ha_copy_sync_mode=sync:sync
     ```

   - Apply the changed ha_node_list.

     Run “reload” command at the master, *nodeA*.

     ```
     [nodeA]$ cubrid heartbeat reload
     ```

     Run “stop” and “start” commands at the slave, *nodeB*.

     ```
     [nodeB]$ cubrid heartbeat stop
     [nodeB]$ cubrid heartbeat start
     ```

   - Stop coplylogdb and applylogdb for *nodeC*.

```
[nodeA]$ cubrid heartbeat repl stop nodeC
[nodeB]$ cubrid heartbeat repl stop nodeC
```

- Remove replication logs about *nodeC* from *nodeA* and *nodeB*.

```
$ cd $CUBRID_DATABASES
$ rm -rf testdb_nodeC
```

- Remove replication information about *nodeC* from *nodeA* and *nodeB*.

  You can run DELELTE operation from a slave node only if you add **--sysadm** and **--write-on-standby** options on running csql.

```
$ csql -u dba --sysadm --write-on-standby testdb@localhost

sysadm> DELETE FROM db_ha_apply_info WHERE
copied_log_path='/home/cubrid/DB/databases/testdb_nodeC';
```

  copied_log_path is the path in which replication log files are stored.

1. Remove *nodeC* from broker's configuration

   - Remove *nodeC* if this exists in db-host of databases.txt from *nodeA* and *nodeB*.

```
$ cd $CUBRID_DATABASES
$ vi databases.txt

#db-name     vol-path                    db-host             log-
path                        lob-base-path
testdb       /home/cubrid/DB/testdb      nodeA:nodeB         /
home/cubrid/DB/testdb/log      file:/home/cubrid/DB/testdb/lob
```

   - Restart brokers of *nodeA* and *nodeB*.

```
$ cubrid broker restart
```

Note

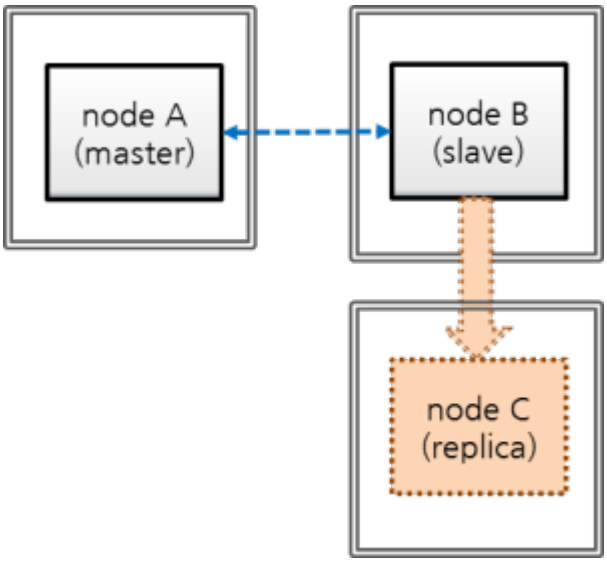
You don't need to hurry to restart these brokers because *nodeC* is already stopped and an application will not access the *nodeC* database. If you restart these brokers, you should consider that the applications connected to these brokers can be disconnected.

Moreover, if multiple brokers with the same configuration exist, one broker can substitute others; therefore, restart a broker one by one.

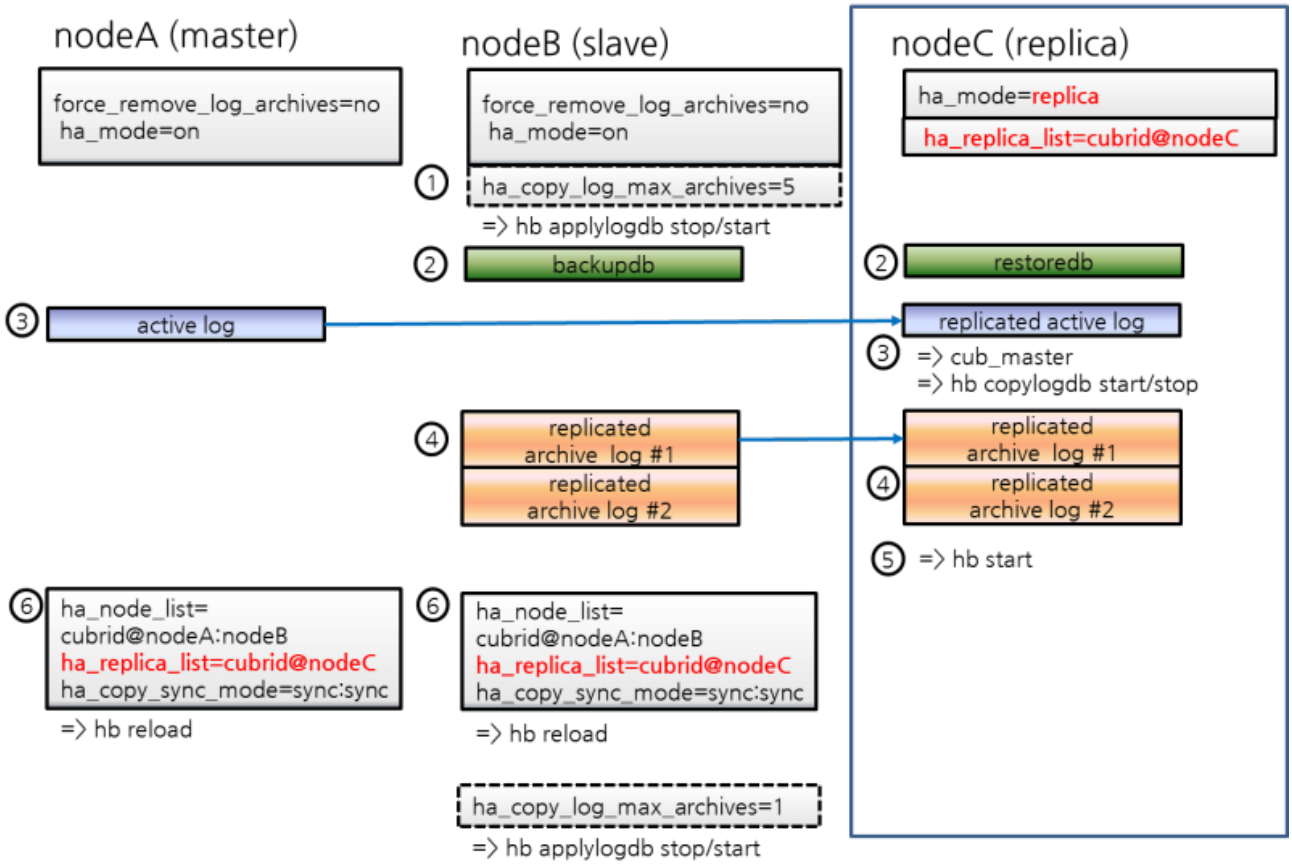
For example, on the above, *nodeA* and *nodeB* are the same configuration; therefore, you can restart *nodeB* first then restart *nodeA*.

Add a Replica during Service

Now let's add a replica when HA environment is set as "master:slave=1:1". The beginning database setting is the same as the setting in Build an initial database of Build Another Slave during HA Service.



- nodeA: a master node's hostname
- nodeB: a slave node's hostname
- nodeC: a replica node's hostname to be added newly



1. Configure HA parameter of *nodeC*

   - Set *nodeC*'s **$CUBRID/conf/cubrid.conf** as the same with *nodeB* except for ha_mode.

     Set "ha_mode=replica".

```
[service]
service=server,broker,manager

[common]

# Add this when doing HA configuration (HA mode)
ha_mode=replica
```

> **Note**
>
> Replica node's transaction logs are not used to replication; therefore, whatever the value of force_remove_log_archives is set to, it works as yes.

   - Configure *nodeC*'s **$CUBRID/conf/cubrid_ha.conf**

     Add *nodeC* into ha_replica_list.

```
[common]
ha_port_id=59901

# cubrid is a group name of HA system, nodeA, nodeB, and nodeC
are host names.
ha_node_list=cubrid@nodeA:nodeB

ha_replica_list=cubrid@nodeC

ha_db_list=testdb
ha_copy_sync_mode=sync:sync
ha_apply_max_mem_size=500
```

- Set a locale library of *nodeC* identically with that of *nodeA* or *nodeB*.

```
[nodeB]$ scp $CUBRID/conf/cubrid_locales.txt cubrid_usr@nodeC:
$CUBRID/conf/.
[nodeC]$ make_locale.sh -t64
```

- Add *nodeC* into db-host of **$CUBRID_DATABASES/databases.txt** in *nodeC*.

```
#db-name     vol-path                    db-host             log-
path                        lob-base-path
testdb       /home/cubrid/DB/testdb      nodeC    /home/cubrid/DB/
testdb/log      file:/home/cubrid/DB/testdb/lob
```

db-host of **databases.txt** is related to the access order to the database server by a broker; therefore, you can change the value as you want. For example, you can write only nodeA:nodeB or localhost into **db-host**. However, by considering the local machine access, (e.g. csql -u dba testdb@localhost) you should add at least localhost or the name of localhost into **db-host**.

2. Apply configuration for preventing from deleting replication logs of *nodeB* during restoring after backup

   nodeB's replication logs can be added after restoring if a service keeps going during restoring on *nodeC* after backup of *nodeB*; at this time, some replication logs which are needed for *nodeC* can be deleted by the configuration. To protect this, configure nodes as following.

   - Change the value of **ha_copy_log_max_archives** larger without restarting a database on *nodeB*.

     To apply transactions after backup time to *nodeC*, transactions should be kept until the *nodeC*'s data which restore *nodeB*'s backup data become the same with master(*nodeA*)'s.

     Therefore, *nodeB*'s replication logs which are added after backup should be kept.

Here, let's assume the number of replication log files which can sufficiently store transactions executed during *nodeC*'s building as 5.

Edit **$CUBRID/conf/cubrid_ha.conf**

```
ha_copy_log_max_archives=5
```

Apply the change of ha_copy_log_max_archives to *nodeB*'s applylogdb process.

```
[nodeB]$ cubrid heartbeat applylogdb stop testdb nodeA
[nodeB]$ cubrid heartbeat applylogdb start testdb nodeA
```

Note

If you build *nodeC* with *nodeA*, a master node only, copying replication logs from *nodeB* is not needed; therefore, changing the value of **ha_copy_log_max_archives** from *nodeB* is also needless.

Instead, it requires to apply a changed value of **log_max_archives** to *nodeA*.

The value of **log_max_archives** parameter in cubrid.conf can be changed during service with the "SET SYSTEM PARAMETERS" syntax.

```
[nodeA]$ csql -u dba -c "SET SYSTEM PARAMETERS
'log_max_archives=5'" testdb@localhost
```

3. Backup *nocdB* and restore on *nodeC*

◦ Backup a database from *nodeB*.

```
[nodeB]$ cd $CUBRID_DATABASES/testdb/log
[nodeB]$ cubrid backupdb --sleep-msecs=10 -C -o output.txt
testdb@localhost
```

Note

Please refer to the explanation of –sleep-msecs within Build Another Slave during HA Service.

- Copy backup files to *nodeC*.

```
[nodeB]$ cd $CUBRID_DATABASES/testdb/log
[nodeB]$ scp -l 131072 testdb_bk* cubrid_usr@nodeC:
$CUBRID_DATABASES/testdb/log
```

> **Note**
>
> scp command's -l option is an option to control the amount of copying file; when you copy files between nodes, use this value properly as considering I/O overhead. The unit is Kbits. 131072 is 16MB.

- Restore the database into *nodeC*.

  At this time, volume paths between a master and a slave should be the same.

```
[nodeC]$ cubrid restoredb -B $CUBRID_DATABASES/testdb/log testdb
```

4. Copy *nodeA*'s active log to *nodeC*

   In *nodeA*'s active log, there exists a number information of the archive log which is recently created. This information is needed for archive logs, which has been created after the time in which the active log had been copied, to be automatically copied by copylogdb process after the start of HA service of *nodeC*; therefore, the user just needs to manually copy archive logs which has been created before copying an active log.

   - Start cub_master, which manages HA connections, on *nodeC*.

```
[nodeC]$ cub_master
```

   - Start copylogdb process for *nodeA*.

```
[nodeC]$ cubrid heartbeat copylogdb start testdb nodeA
```

   - Check if *nodeA*'s active log was copied.

     To copy the current active log means to obtain the number information of the last archive log. Archive logs after this time are automatically copied after the process [5. Start HA server in *nodeC*].

```
[nodeC]$ cd $CUBRID_DATABASES/testdb_nodeA
[nodeC]$ ls
testdb_lgar_t  testdb_lgat  testdb_lgat__lock
```

- Stop copylogdb process for *nodeA* on *nodeC*.

```
[nodeC]$ cubrid heartbeat copylogdb stop testdb nodeA
```

- Stop cub_master of *nodeC*.

```
[nodeC]$ cubrid service stop
```

5. Copy required log files after restoring a database into *nodeC*

- Copy all replicated archive logs of *nodeB* into *nodeC*.

```
[nodeC]$ cd $CUBRID_DATABASES/testdb_nodeA
[nodeC]$ scp -l 131072 cubrid@nodeB:$CUBRID_DATABASES/
testdb_nodeA/testdb_lgar0* .
```

In this process, please note that all replicated archive logs which are required for replication should exist.

Note

scp command's -l option is an option to control the amount of copying file; when you copy files between nodes, use this properly as considering I/O overhead. The unit is Kbits. 131072 is 16MB.

Note

Copy files of which the names end with a number. Here, we copied with "testdb_lgar0*" because numbers of the names of all replication log files are started with 0.

Note

Replication logs which are created between backup time and active log creation time can be deleted if the value of **ha_copy_log_max_archives** is not large enough; in this case, an error can occur when running the below process, [5. Start HA service on *nodeC*].

When an error occurs, take actions by referring to Preparation plan when the required replication logs are already deleted described on the above.

6. Start HA service on *nodeC*

```
[nodeC]$ cubrid heartbeat start
```

7. Change HA configuration of *nodeA* and *nodeB*

◦ Turn back ha_copy_log_max_archive of *nodeB*.

```
[nodeB]$ vi cubrid_ha.conf

ha_copy_log_max_archives=1
```

◦ Apply the changed value of ha_copy_log_max_archives to the applylogdb process of *nodeB*.

```
[nodeB]$ cubrid heartbeat applylogdb stop testdb nodeA
[nodeB]$ cubrid heartbeat applylogdb start testdb nodeA
```

◦ In cubrid_ha.conf, add *nodeC* into ha_node_list, and add one “sync” more to *nodeA* and *nodeB*.

```
$ cd $CUBRID/conf
$ vi cubrid_ha.conf

ha_replica_list=cubrid@nodeC
```

◦ Apply changed ha_node_list.

```
[nodeA]$ cubrid heartbeat reload
[nodeB]$ cubrid heartbeat reload
```

At this time, coplylogdb and applylogdb processes for *nodeC* on *nodeA* and *nodeB* are needless to start because *nodeC* is a replica node.

8. Add a broker and add this broker’s hostname into altHosts of application URL

Add a broker if needed, and add this broker's hostname into altHosts, the property of application URL to access this broker.

If you are not considering to add a broker, you do not need to do this job.

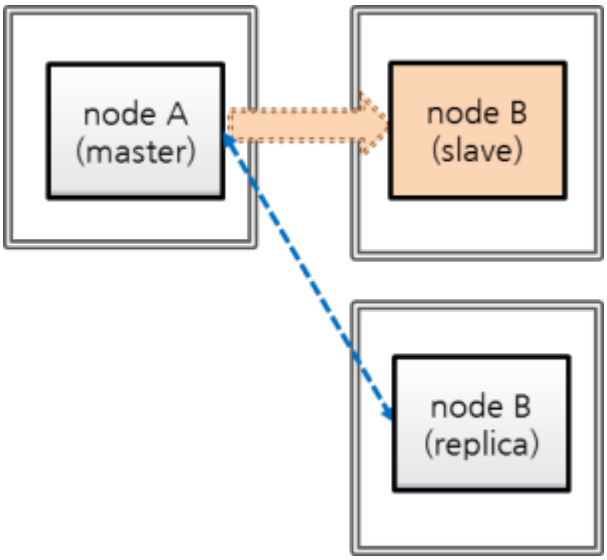
- From all of *nodeA* and *nodeB*, add *nodeC* into db-host of **$CUBRID_DATABASES/databases.txt**.

```
#db-name      vol-path                    db-host             log-
path                          lob-base-path
testdb        /home/cubrid/DB/testdb      nodeA:nodeB:nodeC   /home/
cubrid/DB/testdb/log      file:/home/cubrid/DB/testdb/lob
```

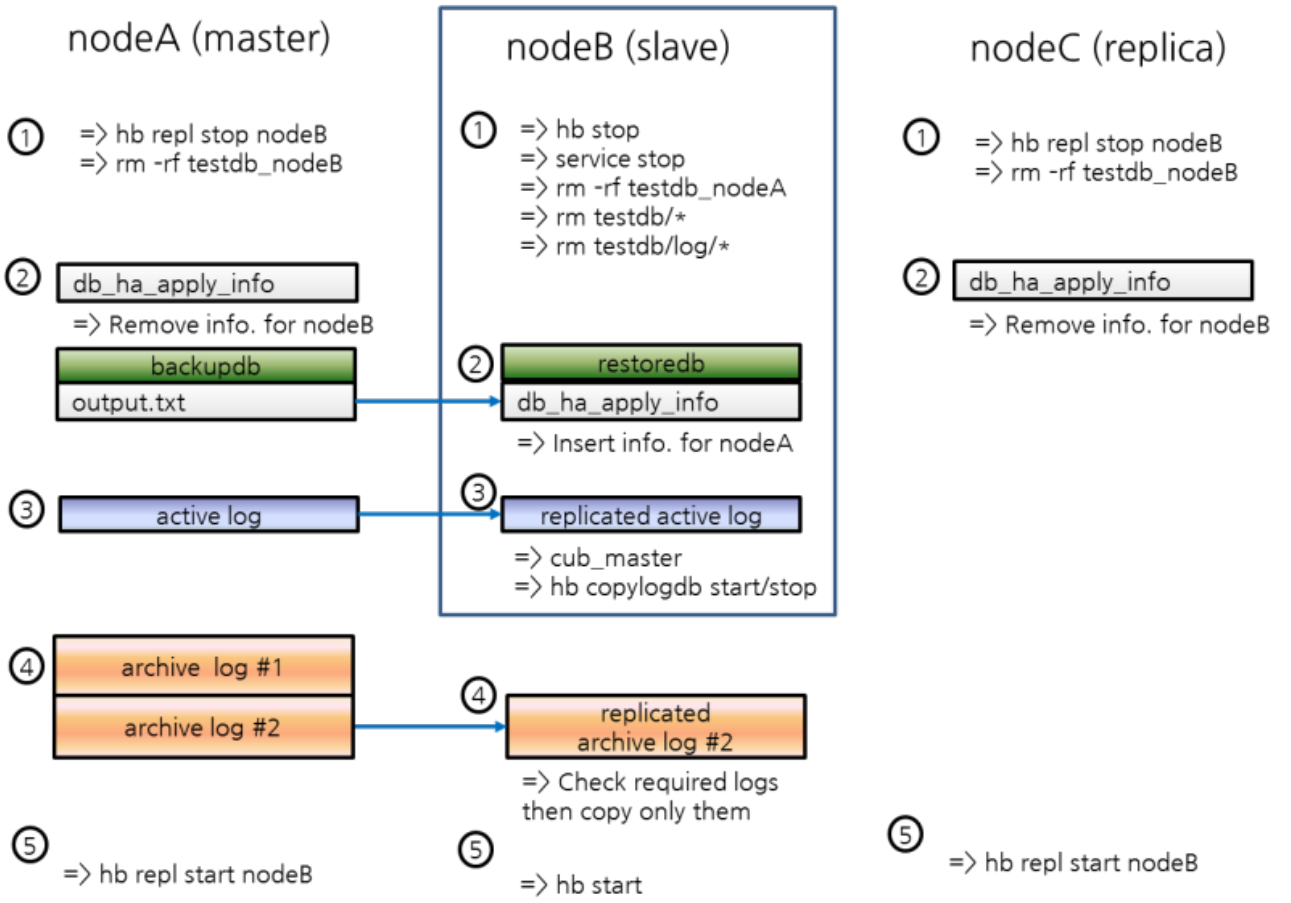
db-host of **databases.txt** is a configuration parameter related to the order of access from a broker to database servers; therefore, you can change as you want. For example, you can write only "nodeA:nodeB" or "localhost" into **db-host**. However, if you access database from a local machine (e.g. csql -u dba testdb@localhost), you should include "localhost" or the localhost name into **db-host**.

Rebuild a Slave during Service

Now let's see the case of rebuilding a existing slave node during a service in a composition of "master:slave:relica = 1:1:1". The beginning database setting is the same as the setting in Build an initial database of Build Another Slave during HA Service.



- nodeA: a hostname of a master node
- nodeB: a hostname of a slave node to be rebuilt
- nodeC: a hostname of a replica node



1. Stop *nodeB*'s service and remove this database

   After stopping *nodeB*'s service, remove *nodeB*'s database volumes and *nodeB*'s replication logs located in *nodeA* and *nodeC*.

   - Stop *nodeB*'s service.

     ```
     [nodeB]$ cubrid heartbeat stop
     [nodeB]$ cubrid service stop
     ```

   - Remove *nodeB*'s database volumes and replication logs.

     ```
     [nodeB]$ cd $CUBRID_DATABASES
     [nodeB]$ rm testdb/*
     [nodeB]$ rm testdb/log/*

     [nodeB]$ rm -rf testdb_nodeA
     [nodeB]$ rm $CUBRID/var/APPLYLOGDB/testdb
     ```

   - Stop log replication processes of *nodeB* on *nodeA* and *nodeC*.

```
[nodeA]$ cubrid heartbeat repl stop nodeB
[nodeC]$ cubrid heartbeat repl stop nodeB
```

- Remove replication logs for *nodeB* from *nodeA* and *nodeC*.

```
[nodeA]$ rm -rf $CUBRID_DATABASES/testdb_nodeB
[nodeC]$ rm -rf $CUBRID_DATABASES/testdb_nodeB
```

2. Restore *nodeB*'s database from *nodeA*'s backup, and add data to HA catalog table.

- Delete db_ha_apply_info data for *nodeB* from *nodeA* and *nodeC*.

```
[nodeA]$ csql --sysadm -u dba testdb@localhost
csql> DELETE FROM db_ha_apply_info WHERE copied_log_path='/
home/cubrid/DB/databases/testdb_nodeB';

[nodeC]$ csql --sysadm --write-on-standby -u dba
testdb@localhost
csql> DELETE FROM db_ha_apply_info WHERE copied_log_path='/
home/cubrid/DB/databases/testdb_nodeB';
```

- Backup a database from *nodeA*.

```
[nodeA]$ cd $CUBRID_DATABASES/testdb/log
[nodeA]$ cubrid backupdb --sleep-msecs=10 -C -o output.txt
testdb@localhost
```

Note

Please refer to –sleep-msecs of Build Another Slave during HA Service.

- Copy backup files to *nodeB*.

```
[nodeA]$
[nodeA]$ scp testdb_bk* cubrid_usr@nodeB:$CUBRID_DATABASES/
testdb/log
```

Note

scp command's -l option is an option to control the amount of copying file; when you copy files between nodes, use this properly as considering I/O overhead. The unit is Kbits. 131072 is 16MB.

◦ Restore the database into *nodeB*.

At this time, the volume path of the master node must be identical to that of the slave node.

```
[nodeB]$ cubrid restoredb -B $CUBRID_DATABASES/testdb/log testdb
```

◦ Add a replication information record for *nodeA* to *nodeB*'s db_ha_apply_info.

Get an update information of **db_ha_apply_info** from output.txt file, which has backup result messages in *nodeA*. output.txt is saved at the directory on which a user ran "cubrid backupdb" command.

```
[nodeA]$ vi $CUBRID_DATABASES/testdb/log/output.txt

[ Database(testdb) Full Backup start ]

- num-threads: 2

- compression method: NONE

- sleep 10 millisecond per 1M read.

- backup start time: Fri Mar 20 18:18:53 2015

- number of permanent volumes: 2

- HA apply info: testdb 1426495463 12922 16192

- backup progress status

...
```

Make the below script and run. At "HA apply info" on the above output, put the first number, 1426495463 into $db_creation, the second number, 12922 into $pageid, and the third number, 16192 into $offset; put testdb into db_name, the database name and *nodeA* into master_host, the master node's hostname.

```
[nodeB]$ vi update.sh

#!/bin/sh
```

```
db_creation=1426495463
page_id=12922
offset=16192
db_name=testdb
master_host=nodeA

repl_log_home_abs=$CUBRID_DATABASES

repl_log_path=$repl_log_home_abs/${db_name}_${master_host}

local_db_creation=`awk 'BEGIN { print strftime("%m/%d/%Y %H:%M:
%S", '$db_creation') }'`
    csql_cmd="\
    INSERT INTO \
            db_ha_apply_info \
    VALUES \
    ( \
            '$db_name', \
            datetime '$local_db_creation', \
            '$repl_log_path', \
            $page_id, $offset, \
            $page_id, $offset, \
            $page_id, $offset, \
            $page_id, $offset, \
            $page_id, $offset, \
            $page_id, $offset, \
            NULL, \
            NULL, \
            NULL, \
            0, \
            0, \
            0, \
            0, \
            0, \
            0, \
            0, \
            NULL \
    )"

# Insert nodeA's HA info.
csql --sysadm -u dba -c "$csql_cmd" -S testdb
```

```
[nodeB]$ sh update.sh
```

Check if the data is successfully inserted.

```
[nodeB]$ csql -u dba -S testdb
```

```
csql> ;line on
csql> SELECT * FROM db_ha_apply_info;
```

3. Copy *nodeA*'s active log into *nodeB*

   In *nodeA*'s active log, there exists a number information of the archive log which is recently created. This information is needed for archive logs, which has been created after the time in which the active log had been copied, to be automatically copied by copylogdb process after the start of HA service of *nodeC*; therefore, the user just needs to manually copy archive logs which has been created before copying an active log.

   - Start cub_master which manages HA connection in *nodeB*.

     ```
     [nodeB]$ cub_master
     ```

   - Start copylogdb process for *nodeA* on *nodeB*.

     ```
     [nodeB]$ cubrid heartbeat copylogdb start testdb nodeA
     ```

   - Check if an active log of *nodeA* is copied.

     To copy the current active log means to obtain the number information of the last archive log. Archive logs after this time are automatically copied after the process [5. Start HA server in *nodeB*].

     ```
     [nodeB]$ cd $CUBRID_DATABASES/testdb_nodeA
     [nodeB]$ ls
     testdb_lgar_t  testdb_lgat  testdb_lgat__lock
     ```

   - Stop copylogdb process for *nodeA* on *nodeB*.

     ```
     [nodeB]$ cubrid heartbeat copylogdb stop testdb nodeA
     ```

   - Stopo *nodeB*'s cub_master.

     ```
     [nodeB]$ cubrid service stop
     ```

4. Copy required log files after restoring a database into *nodeB*

   - Copy all archive logs of *nodeA* into *nodeB*.

```
[nodeB]$ cd $CUBRID_DATABASES/testdb_nodeA
[nodeB]$ scp -l 131072 cubrid@nodeA:$CUBRID_DATABASES/testdb/log/
testdb_lgar0* .
```

In this process, please note that all replicated archive logs which are required for replication should exist.

Note

scp command's -l option is an option to control the amount of copying file; when you copy files between nodes, use this properly as considering I/O overhead. The unit is Kbits. 131072 is 16MB.

Note

Copy files of which the names end with a number. Here, we copied with "testdb_lgar0*" because numbers of names of all replication log files are started with 0.

Note

When required logs are already deleted

Archive logs which are created between backup time and active log creation time can be deleted if the value of **log_max_archives** is not large enough; in this case, an error can occur when running the below process, [5. Start HA service on *nodeB* ...].

When an error occurs, enlarge the value of **log_max_archives** enough and proceed again backup-and-restore from the beginning.

5. Start HA service on *nodeB*; restart copylogdb and applylogdb processes on *nodeA* and *nodeC*

```
[nodeB]$ cubrid heartbeat start
```

```
[nodeA]$ cubrid heartbeat applylogdb start testdb nodeB
[nodeA]$ cubrid heartbeat copylogdb start testdb nodeB
```

```
[nodeC]$ cubrid heartbeat applylogdb start testdb nodeB
[nodeC]$ cubrid heartbeat copylogdb start testdb nodeB
```

Detection of Replication Mismatch

How to Detect Replication Mismatch

Replication mismatch between replication nodes, indicating that data of the master node and the slave node (or the replica node) is not identical, can be detected to some degree by the following process. You can also use checksumdb utility to detect a replication inconsistency. However, please note that there is no more accurate way to detect a replication mismatch than by directly comparing the data of the master node to the data of the slave node (or the replica node). If it is determined that there has been a replication mismatch, you should rebuild the database of the master node to the slave node (or the replica node) (see Rebuilding Replication Script.)

- Execute **cubrid statdump** command and check **Time_ha_replication_delay**. When this value is bigger, replication latency can be larger; the bigger latency time shows the possibility of the larger replication mismatch.

- On the slave node (or the replica node), execute **cubrid applyinfo** to check the "Fail count" value. If the "Fail count" is 0, it can be determined that no transaction has failed in replication (see applyinfo.)

```
[nodeB]$ cubrid applyinfo -L /home/cubrid/DB/testdb_nodeA -r nodeA -
a testdb

 *** Applied Info. ***
Committed page                  : 1913 | 2904
Insert count                    : 645
Update count                    : 0
Delete count                    : 0
Schema count                    : 60
Commit count                    : 15
Fail count                      : 0
...
```

- To check whether copying replication logs has been delayed or not on the slave node (or the replica node), execute **cubrid applyinfo** and compare the "Append LSA" value of "Copied Active Info." to the "Append LSA" value of "Active Info.". If there is a big difference between the two values, it means that delay has occurred while copying the replication logs to the slave node (or the replica node) (see applyinfo.)

```
[nodeB]$ cubrid applyinfo -L /home/cubrid/DB/testdb_nodeA -r nodeA -
a testdb
```

```
...

 *** Copied Active Info. ***
DB name                         : testdb
DB creation time                : 11:28:00.000 AM 12/17/2010
(1292552880)
Vol creation time               : 11:28:10.000 AM 12/17/2010
(1292552890)
EOF LSA                         : 1913 | 2976
Append LSA                      : 1913 | 2976
HA server state                 : active

 ***  Active Info. ***
DB name                         : testdb
DB creation time                : 11:28:00.000 AM 12/17/2010
(1292552880)
Vol creation time               : 11:28:10.000 AM 12/17/2010
(1292552890)
EOF LSA                         : 1913 | 2976
Append LSA                      : 1913 | 2976
HA server state                 : active
```

- If a delay seems to occur when copying the replication logs, check whether the network line speed is slow, whether there is sufficient free disk space, disk I/O is normal, etc.

- To check the delay in applying the replication log in the slave node (or the replica node), execute **cubrid applyinfo** and compare the "Committed page" value of "Applied Info." to the "EOF LSA" value of "Copied Active Info.". If there is a big difference between the two values, it means that a delay has occurred while applying the replication logs to the slave database (or the replica database) (see applyinfo.)

```
[nodeB]$ cubrid applyinfo -L /home/cubrid/DB/testdb_nodeA -r nodeA -
a testdb

 *** Applied Info. ***
Committed page                  : 1913 | 2904
Insert count                    : 645
Update count                    : 0
Delete count                    : 0
Schema count                    : 60
Commit count                    : 15
Fail count                      : 0

 *** Copied Active Info. ***
DB name                         : testdb
DB creation time                : 11:28:00.000 AM 12/17/2010
(1292552880)
Vol creation time               : 11:28:10.000 AM 12/17/2010
```

```
(1292552890)
EOF LSA                         : 1913 | 2976
Append LSA                      : 1913 | 2976
HA server state                 : active
...
```

- If the delay in applying the replication logs is too long, it may be due to a transaction with a long execution time. If the transaction is performed normally, a delay in applying the replication logs may normally occur. To determine whether it is normal or abnormal, continuously execute **cubrid applyinfo** and check whether applylogdb continuously applies replication logs to the slave node (or the replica node) or not.
- Check the error log message created by the copylogdb process and the applylogdb process (see the error message).
- Compare the number of records on the master database table to that on the slave (or the replica) database table.

checksumdb

checksumdb provides a simple way to check replication integrity. Basically, it divides each table from a master node into fixed-size chunks and then calculates CRC32 values. The calculation itself, not the calculated value, is then replicated through CUBRID HA. Consequently, by comparing CRC32 values calculated on master and slave nodes (or replica nodes), **checksumdb** can report the replication integrity. Note that **checksumdb** might affect master's performance even though it is designed to minimize the performance degradation.

```
cubrid checksumdb [options] <database-name>@<hostname>
```

- *<hostname>* : When you initiates checksum calculation, you need to specify the hostname of a master node. When you need to get the result after the calculation is completed, specify the hostname of a node you want to check.

-c, --chunk-size=NUMBER

You can specify the number of rows to select for each CRC32 calculation. (default: 500 rows, minimum: 100 rows)

-s, --sleep=NUMBER

checksumdb sleeps the specified amount of time after calculating each chunk (default: 100 ms)

-i, --include-class-file=FILE

You can specify tables to check the replication mismatch by specifying the -i FILE option. If it is not specified, entire tables will be checked. Empty string, tab, carriage return and comma are separators among table names in the file.

-e, --exclude-class-file=FILE

You can specify tables to exclude from checking the replication mismatch by specifying the -e FILE option. Note that either -i or -e can be used, not both.

-t, --timeout=NUMBER

You can specify a calculation timeout with this option. (default: 1000 ms) If the timeout is reached, the calculation will be cancelled and will be resumed after a short period of time.

-n, --table-name=STRING

You can specify a table name to save checksum results. The table name must include the owner's name which can only be dba. (default: dba.db_ha_checksum)

-r, --report-only

After checksum calculation is completed, you can get a report with this option.

--resume

When checksum calculation is aborted, you can resume the calculation using this option.

--schema-only

When this option is given, checksumdb does not calculate CRC32 but only check schema of each table

--cont-on-error

Without this option, checksumdb halts on errors.

The following example shows how to start checksumdb

```
cubrid checksumdb -c 100 -s 10 testdb@master
```

When no replication mismatch found,

```
$ cubrid checksumdb -r testdb@slave
========================================================================
 target DB: testdb@slave (state: standby)
 report time: 2016-01-14 16:33:30
 checksum table name: db_ha_checksum, db_ha_checksum_schema
========================================================================

------------------------
 different table schema
------------------------
NONE
```

```
------------------------------------------------------------------
table name  diff chunk id   chunk lower bound
------------------------------------------------------------------
NONE

------------------------------------------------------------------------------
-----------------
table name  total # of chunks       # of diff chunks        total/
avg/min/max time
------------------------------------------------------------------------------
-----------------
t1              7                       0                       88 /
12 / 5 / 14 (ms)
t2              7                       0                       96 /
13 / 11 / 15 (ms)
```

When there is a replication mismatch in table *t1*,

```
$ cubrid checksumdb -r testdb@slave
==================================================================
 target DB: testdb@slave (state: standby)
 report time: 2016-01-14 16:35:57
 checksum table name: db_ha_checksum, db_ha_checksum_schema
==================================================================

------------------------
 different table schema
------------------------
NONE

------------------------------------------------------------------
table name  diff chunk id   chunk lower bound
------------------------------------------------------------------
t1              0               (id>=1)
t1              1               (id>=100)
t1              4               (id>=397)

------------------------------------------------------------------------------
-----------------
table name  total # of chunks       # of diff chunks        total/
avg/min/max time
------------------------------------------------------------------------------
-----------------
t1              7                       3                       86 /
12 / 5 / 14 (ms)
t2              7                       0                       93 /
13 / 11 / 15 (ms)
```

When there is a schema mismatch in table *t1*,

```
$ cubrid checksumdb -r testdb@slave
========================================================================
 target DB: testdb@slave (state: standby)
 report time: 2016-01-14 16:40:56
 checksum table name: db_ha_checksum, db_ha_checksum_schema
========================================================================

------------------------
 different table schema
------------------------
<table name>
t1
<current schema - collected at 04:40:53.947 PM 01/14/2016>
CREATE TABLE [t1] ([id] INTEGER NOT NULL, [col1] CHARACTER
VARYING(20), [col2] INTEGER, [col3] DATETIME, [col4] INTEGER,
CONSTRAINT [pk_t1_id] PRIMARY KEY  ([id])) COLLATE iso88591_bin
<schema from master>
CREATE TABLE [t1] ([id] INTEGER NOT NULL, [col1] CHARACTER
VARYING(20), [col2] INTEGER, [col3] DATETIME,  CONSTRAINT [pk_t1_id]
PRIMARY KEY  ([id])) COLLATE iso88591_bin

* Due to schema inconsistency, the checksum difference of the above
table(s) may not be reported.
------------------------------------------------------------------
table name  diff chunk id   chunk lower bound
------------------------------------------------------------------
NONE

------------------------------------------------------------------------
-----------------
table name  total # of chunks       # of diff chunks        total/
avg/min/max time
------------------------------------------------------------------------
-----------------
t1              7                       0                       95 /
13 / 11 / 16 (ms)
t2              7                       0                       94 /
13 / 11 / 15 (ms)
```

HA Error Messages

The following are error messages about the errors which can be reasons for the replication mismatch.

CAS process(cub_cas)

Error messages in CAS process are written to **$CUBRID/log/broker/error_log/***<broker_name>*_*<app_server_num>***.err**. The following table shows connection error messages in the HA environment.

Error messages for handshaking between a broker and a DB server

Error Code	Error message	Severity	Description	Solution
-1139	Handshake error (peer host ?): incompatible read/write mode. (client: ?, server: ?)	error	ACCESS_MODE of broker and the status of server (active/standby) is not the same (see Broker Mode).	
-1140	Handshake error (peer host ?): HA replication delayed.	error	Replication is delayed in the server which set **ha_delay_limit**.	
-1141	Handshake error (peer host ?): replica-only client to non-replica server.	error	A broker(CAS) which can access only a replica attempts to connect to the server which is not a replica(see Broker Mode).	
-1142	Handshake error (peer host ?): remote access to	error	It attempts to connect to a server with HA maintenance	

Error Code	Error message	Severity	Description	Solution
	server not allowed.		mode(see Servers).	
-1143	Handshake error (peer host ?): unidentified server version.	error	Server version is unidentified.	

Connection Error Messages

Error Code	Error message	Severity	Description	Solution
-353	Cannot make connection to master server on host ?.	error	cub_master process went down.	
-1144	Timed out attempting to connect to ?. (timeout: ? sec(s))	error	The machine went down.	

Replication Log Copy Process(copylogdb)

The error messages from the replication log copy process are stored in **$CUBRID/log/***<db-name>***@***<remote-node-name>***_copylogdb.err**. The severity levels of error messages found in the replication log copy process are as follows: fatal, error, and notification. The default level is error. Therefore, to record notification error messages, it is necessary to change the value of **error_log_level** in the **cubrid.conf** file. For details, see Error Message-Related Parameters.

Initialization Error Messages

The error messages that can be found in initialization stage of replication log copy process are as follows:

Error Code	Error Message	Severity	Description	Solution
-10	Cannot mount the disk volume ?.	error	Fails to open a replication log file.	Check if replication logs exist. For the location of replication logs, see Default Environment Configuration.
-78	Internal error: an I/O error occurred while reading logical log page ? (physical page ?) of ?	fatal	Fails to read a replication log.	Check the replication log by using the cubrid applyinfo utility.
-81	Internal error: logical log page ? may be corrupted.	fatal	A replication log page error, in which the replication log copy process has been copied from the connected database server process.	Check the error log of the database server process to which the replication log copy process is connected. This error log can be found in $CUBRID/log/server.
-1039	log writer: log writer has been started. mode: ?	error	The replication log copy process has been successfully	No action is required because this error message is recorded to display the start

Error Code	Error Message	Severity	Description	Solution
			initialized and started.	information of the replication log copy process. Ignore any error messages which are displayed between the start of replication log copy process and output of this error message since there is a possibility that an error message is shown up even in normal situation.

Replication Log Request and Reception Error Messages

The replication log copy process requests a replication log from the connected database server and receives the corresponding replication log. Error messages that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-89	Log ? is not included in the given database.	error	The previously replicated log and the log to be replication do not match.	Check information of the database server/host to which the replication log copy process is connected. If

Error Code	Error Message	Severity	Description	Solution
				you need to change the database server/host information, reinitialize it by deleting the existing replication log and then restarting.
-186	Data receiver error from the server	error	Incorrect information has been received from the database server to which the replication log copy process is connected.	It will be internally recovered.
-199	The server is not responding.	error	The connection to the database server has been terminated.	It will be internally recovered.

Replication Log Write Error Messages

The replication log copy process copies the replication log that was received from the connected database server process to the location (**ha_copy_log_base**) specified in the **cubrid_ha.conf** file. Error messages that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-10	Cannot mount the disk volume ?.	error	Fails to open a replication log file.	Check if replication logs exist.
-79	Internal error: an I/O error occurred while writing logical log page ? (physical page ?) of ?.	fatal	Fails to write a replication log.	It will be internally recovered.
-80	An error occurred due to insufficient space in operating system device while writing logical log page ?(physical page ?) of ?. Up to ? bytes in size are allowed.	fatal	Fails to write a replication log due to insufficient file system space.	Check if there is sufficient space left in the disk partition.

Replication Log Archive Error Messages

The replication log copy process periodically archives the replication logs that have been received from the connected database server process. Error messages that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-78	Internal error: an I/O error occurred while	fatal	Fails to read a replication log	Check the replication log by using the

Error Code	Error Message	Severity	Description	Solution
	reading logical log page ? (physical page ?) of ?.		during archiving.	cubrid applyinfo utility.
-79	Internal error: an I/O error occurred while writing logical log page ? (physical page ?) of ?.	fatal	Fails to write an archive log.	It will be internally recovered.
-81	Internal error: logical log page ? may be corrupted.	fatal	Found an error on the replication log during archiving.	Check the replication log by using the cubrid applyinfo utility.
-98	Cannot create an archive log ? to archive pages from ? to ?.	fatal	Fails to create an archive log file.	Check if there is sufficient space left in the disk partition.
-974	An archive log ? to archive pages from ? to ? has been created.	notification	Information on an archive log file	No action is required because this error message is recorded to keep information on newly created archive.

Stop and Restart Error Messages

Error messages that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-1037	log writer: log writer is terminated by signal.	error	The copylogdb process has been terminated by a specified signal.	It will be internally recovered.

Replication Log Reflection Process(applylogdb)

The error messages from the replication log reflection process are stored in **$CUBRID/log/***db-name***@***local-node-name***_applylogdb_***db-name*_*remote-node-name***.err**. The severity levels of error message found in the replication log reflection process are as follow: fatal, error, and notification. The default level is error. Therefore, to record notification error messages, it is necessary to change the value of **error_log_level** in the **cubrid.conf** file. For details, see Error Message-Related Parameters.

Initialization Error Messages

The error messages that can be found in initialization stage of replication log reflection process are as follows:

Error Code	Error Message	Severity	Description	Solution
-10	Cannot mount the disk volume ?.	error	An applylogdb that is trying to reflect the same replication log is already running.	Check if there is an applylogdb process that is trying to reflect the same replication log.
-1038	log applier: log applier has been started. required LSA: ?|?. last committed	error	It will be started normally after initialization of	No action is required because this error is recorded to display the

Error Code	Error Message	Severity	Description	Solution
	LSA: ?|?. last committed repl LSA: ?|?		applylogdb succeeds.	start information of the replication log reflection process.

Log Analysis Error Messages

The replication log reflection process reads, analyzes, and reflects the replications logs that have been copied by the replication log copy process. The error message that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-13	An I/O error occurred while reading page ? in volume ?.	error	Fails to read a log page to be reflected.	Check the replication log by using the cubrid applyinfo utility.
-17	Internal error: Trying to read page ? of the volume ? which has been already released.	fatal	Trying to read a log page that does not exist in the replication log.	Check the replication log by using the cubrid applyinfo utility.
-81	Internal error: logical log page ? may be corrupted.	fatal	There is an inconsistency between an old log under replication reflection and the current log, or there is a	Check the replication log by using the cubrid applyinfo utility.

Error Code	Error Message	Severity	Description	Solution
			replication log record error.	
-82	Cannot mount the disk volume/file ?.	error	No replication log file exists.	Check if replication logs exist. Check the replication log by using the cubrid applyinfo utility.
-97	Internal error: unable to find log page ? in log archives.	error	No log page exists in the replication log.	Check the replication log by using the cubrid applyinfo utility.
-897	Decompression failure	error	Fails to decompress the log record.	Check the replication log by using the cubrid applyinfo utility.
-1028	log applier: Unexpected EOF log record exists in the Archive log. LSA: ?|?.	error	Incorrect log record exists in the archive log.	Check the replication log by using the cubrid applyinfo utility.
-1029	log applier: Incorrect log page/offset. page HDR: ?|?,	error	Incorrect log record exists.	Check the replication log by using the cubrid

Error Code	Error Message	Severity	Description	Solution
	final: ?|?, append LSA: ?|?, EOF LSA: ?|?, ha file status: ?, is end-of-log: ?.			applyinfo utility.
-1030	log applier: Incorrect log record. LSA: ?|?, forw LSA: ?|?, backw LSA: ?|?, Trid: ?, prev tran LSA: ?|?, type: ?.	error	Log record header error	Check the replication log by using the cubrid applyinfo utility.

Replication Log Reflection Error Messages

The replication log reflection process reads, analyzes, and reflects the replication logs that have been copied by the replication log copy process. Error messages that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-72	The transaction (index ?, ?@?|?) has been cancelled by system.	error	Fails to reflect replication due to deadlock, etc.	It will be recovered internally.
-111	Your transaction has been cancelled due to server failure or a mode change.	error	Fails to reflect replication because the database server process in which replication is	It will be recovered internally.

Error Code	Error Message	Severity	Description	Solution
			supposed to be reflected has been terminated or its mode has been changed.	
-191	Cannot connect to server ? on ?.	error	The connection to the database server process in which replication is supposed to be reflected has been terminated.	It will be recovered internally.
-195	Server communication error: ?.	error	The connection to the database server process in which replication is supposed to be reflected has been terminated.	It will be recovered internally.
-224	The database has not been resumed.	error	The connection to the database server process in which replication is supposed to be reflected has	It will be recovered internally.

Error Code	Error Message	Severity	Description	Solution
			been terminated.	
-1027	log applier: Failed to change the reflection status from ? to ?.	error	Fails to change of replication reflection.	It will be recovered internally.
-1031	log applier: Failed to reflect the Schema replication log. class: ?, schema: ?, internal error: ?.	error	Fails to reflect SCHEMA replication.	Check the consistency of the replication. If it is inconsistent, reconfigure the HA replication.
-1032	log applier: Failed to reflect the Insert replication log. class: ?, key: ?, internal error: ?.	error	Fails to reflect INSERT replication.	Check the consistency of the replication. If it is inconsistent, reconfigure the HA replication.
-1033	log applier: Failed to reflect the Update replication log. class: ?, key: ?, internal error: ?.	error	Fails to reflect UPDATE replication.	Check the consistency of the replication. If it is inconsistent, reconfigure the HA replication.
-1034	log applier: Failed to	error		Check the consistency of

Error Code	Error Message	Severity	Description	Solution
	reflect the Delete replication log. class: ?, key: ?, internal error: ?.		Fails to reflect DELETE replication.	the replication. If it is inconsistent, reconfigure the HA replication.
-1040	HA generic: ?.	notification	Changes the last record of the archive log or replication reflection status.	No action is required because this error message is recorded to provide general information.

Stop and Restart Error Messages

The error messages that can be found in this stage are as follows:

Error Code	Error Message	Severity	Description	Solution
-1035	log applier: The memory size (? MB) of the log applier is larger than the maximum memory size (? MB), or is doubled the starting memory size (? MB) or more. required LSA: ?|?. last committed LSA: ?|?.	error	The replication log reflection process has been restarted due to reaching the maximum memory size limit.	It will be recovered internally.

Error Code	Error Message	Severity	Description	Solution
-1036	log applier: log applier is terminated by signal.	error	The replication log reflection process has been terminated by a specified signal.	It will be recovered internally.

Determine the Cause of Failover

Failover, Failback messages

In an HA environment, messages for failover and failback are prefixed with [Failover], [Failback], respectively. These messages can be categorized into diagnosis messages, which are logged when failover or failback is determined, and result messages, which are logged when failover or failback is either canceled or successfully completed.

Diagnosis messages indicate the reason why the failover or failback was initiated and are prefixed with [Diagnosis]. Result messages include the prefix [Cancelled] along with the reason if failover or failback is canceled, and the prefix [Success] along with the result if failover or failback is successfully completed.

Failover and failback messages make it possible to determine the cause of failover. Below is a list of situations in which failover occurs and the corresponding failover and failback messages that may be logged. Failover messages are logged on the slave node, while failback messages are logged on the master node.

Isolation of master node

Isolation of master node

When the master node determines that it is isolated from the network due to failing to receive heartbeat messages from all slave nodes, the failback of the master node and the failover of the slave nodes are decided. The **ha_ping_hosts** are used to verify whether the network of the master node is truly partitioned. Failback is successfully completed only if the master node's network partition is confirmed through this verification.

Similarly, for slave nodes, failover is successfully completed only when a ping check using **ha_ping_hosts** confirms that there is no current network partition. Below is a list of failover and failback messages that could be logged when the master node is isolated.

Diagnosis Messages	Description
[Failback] [Diagnosis] The master node has failed to receive heartbeat messages from all other slave nodes, resulting in a network partition.	Failback has been initiated since master node failed to receive heartbeat messages from all slave nodes
[Failover] [Diagnosis] The current node has failed to receive heartbeat messages from the master node (master_node_name).	Failover has been initiated since slave node failed to receive heartbeat messages from the master node

Result Messages	Description
[Failback] [Cancelled] Ping check succeeded for the hosts registered in ha_ping_hosts, determining that it is not a network partition.	Failback has been cancelled since master node succeeded ping check to ha_ping_hosts
[Failback] [Cancelled] No hosts are registered in ha_ping_hosts, or all registered hosts are invalid, making it impossible to determine the network partition.	Failback has been cancelled due to the inability to determine the network partition of the master node, as the hosts registered in ha_ping_hosts are either empty or invalid.
[Failback] [Success] Current node has been successfully demoted to slave.	Failback is successfully completed, and the master node has been demoted to a slave node
[Failover] [Cancelled] Ping check has been failed to all hosts registered in ha_ping_hosts, indicating a network partition.	Failover has been cancelled because the slave node determined a network partition due to a failed ping check and is unable to assume the master role.
[Failover] [Success] Current node has been successfully promoted to master.	Failover is successfully completed, and the slave node has been promoted to a master node

Loss of the master node's role

Loss of the master node's role

When the master node loses its role due to reasons such as the failure to restart cub_server or disk errors, a failback of the master node and a failover of the slave node are initiated. If a disk failure has been detected, an error message like "Disk failure has occurred: prev_eof_lsa(4096, 128), eof_lsa(4096, 128)" is logged in the server error log.

For the slave node, failover is successfully completed only when a ping check using **ha_ping_hosts** confirms that there is no current network partition. Below is a list of failover and failback messages that could be logged when the master node loses its role.

Diagnosis Messages	Description
[Failback] [Diagnosis] The master node failed to restart the server process.	Failback has been initiated since the master node failed to restart a cub_server process that has been terminated abnormally
[Failback] [Diagnosis] The master node has lost its role due to server process problem, such as disk failure.	Failback has been initiated since the master node lost its role due to a disk failure or similar reasons.
[Failover] [Diagnosis] The master node (master_node_name) has lost its role due to server process problem, such as disk failure.	Failover has been initiated as the slave node determined that the master node has lost its role due to a disk failure or similar reasons.

Result Messages	Description
[Failback] [Success] Current node has been successfully demoted to slave.	Failback is successfully completed, and the master node has been demoted to a slave node
[Failover] [Cancelled] Ping check has been failed to all hosts registered in ha_ping_hosts, indicating a network partition.	Failover has been cancelled because the slave node determined a network partition due to a failed ping check and is unable to assume the master role.

Result Messages	Description
[Failover] [Success] Current node has been successfully promoted to master.	Failover is successfully completed, and the slave node has been promoted to a master node

Multiple master node (split-brain)

Multiple master node (split-brain)

When there is an abnormal situation where two or more servers assume the master role in an HA environment, it is referred to as a “split-brain” scenario. In such cases, all master nodes, except for the one designated to perform the master role, will fail back to slave nodes to resolve the split-brain situation. Below is a list of failback messages that may be logged during a split-brain scenario.

Diagnosis Messages	Description
[Failback] [Diagnosis] Multiple master nodes (master_node_name1, master_node_name2) are detected.	Failback of the master nodes has been initiated to resolve the split-brain situation caused by the detection of multiple master nodes.

Result Messages	Description
[Failback] [Success] Current node has been successfully demoted to slave.	Failback is successfully completed, and the master node has been demoted to a slave node

Rebuilding Replication Script

Replication rebuilding is required in CUBRID HA when data in the CUBRID HA group is inconsistent because of multiple failures in multiple-slave node structure, or because of a generic error. Rebuilding replications in CUBRID HA is perform done through a **ha_make_slavedb.sh** script. With the **cubrid applyinfo** utility, you can check the replication progress; however replication

inconsistency is not detected. If you want to determine whether replication is inconsistent correctly, you must examine data of the master and slave nodes yourself.

When you rebuild only a slave because the slave is abnormal in the environment of master-slave composition, the process is as follows.

1. Backup the master database.

2. Restore the database from slave.

3. Save backup time into the slave's HA meta table (db_ha_apply_info).

4. Start HA service from slave. (cubrid hb start)

For rebuilding replications, the following environment must be the same in master, slave and replica nodes.

- CUBRID version
- Environmental variable (**$CUBRID**, **$CUBRID_DATABASES**, **$LD_LIBRARY_PATH**, **$PATH**, **$CUBRID_TMP**, **$TMPDIR**)
- The paths of database volume, log, and replication
- Username and password of the Linux server
- HA-related parameters except for **ha_mode**, **ha_copy_sync_mode**, **ha_ping_hosts** and **ha_tcp_ping_hosts**

You can rebuild replication by running **ha_make_slavedb.sh** script only in these cases.

- *from-master-to-slave*
- *from-slave-to-replica*
- *from-replica-to-replica*
- *from-replica-to-slave*

For the other cases, you should build manually. For the manual building scenarios, see Building Replication.

If the case is not for rebuilding but for newly building, configure cubrid.conf, cubrid_ha.conf, and databases.txt files as the same with master's.

In the below description, first, we will look over the cases to use **ha_make_slavedb.sh** script, which is used to rebuilding replication.

As a reference, you cannot use **ha_make_slavedb.sh** script when you want to build multiple slave nodes.

ha_make_slavedb.sh Script

To rebuild replications, use the **ha_make_slavedb.sh** script. This script is located in **$CUBRID/share/scripts/ha**. Before rebuilding replications, the following items must be configured for the environment of the user. This script is supported since the version 2008 R2.2 Patch 9 and its configuration is different from 2008 R4.1 Patch 2 or earlier. This document describes it in CUBRID 2008 R4.1 Patch 2 or later.

- **target_host** : The hostname of the source node (master node in general) for rebuilding replication. It should be registered in **/etc/hosts**. A slave node can rebuild replication by using the master node or the replica node as the source. A replica node can rebuild replication by using the slave node or another replica node as the source.

- **repl_log_home** : Specifies the home directory of the replication log of the master node. It is usually the same as **$CUBRID_DATABASES**. You must enter an absolute path and should not use a symbolic link. You also cannot use a slash (/) after the path.

The following are optional items:

- **db_name** : Specifies the name of the database to be replicated. If not specified, the first name that appears in **ha_db_list** in **$CUBRID/conf/cubrid_ha.conf** is used.

- **backup_dest_path** : Specifies the path in which the backup volume is created when executing **backupdb** in source node for rebuilding replication.

- **backup_option** : Specifies necessary options when executing **backupdb** in the source node in which replication will be rebuilt.

- **restore_option** : Specifies necessary options when executing **restoredb** in the target node in which replication will be rebuilt.

- **scp_option** : Specifies the **scp** option which enables backup of source node in which replication is rebuilt to copy into the target node. The default option is **-l 131072**, which does not impose an overload on network (limits the transfer rate to 16 MB).

- **ssh_port** : Specifies the **port** number of the ssh and scp used in the script. This option also applies to **expect** run from the script. The default port number is **22**.

Once the script has been configured, execute the **ha_make_slavedb.sh** script in the target node in which replication will be rebuilt. When the script is executed, rebuilding replication happens in a number of phases. To move to the next stage, the user must enter an appropriate value. The following are the descriptions of available values.

- **yes** : Keeps going.

- **no** : Does not move forward with any stages from now on.

- **skip**: Skips to the next stage. This input value is used to ignore a stage that has not necessarily been executed when retrying the script after it has failed.

Warning

- ha_make_slavedb.sh script executes the connection command into the remote node by **expect** and **ssh**; therefore, **expect** command should be installed('yum install expect' in root account) and **ssh** should be possible in remote connection.

Note

- Backup volumes are required for rebuilding replication

  To replicate, you must copy the physical image of the database volume in the target node to the database of the node to be replicated. However, **cubrid unloaddb** backs up only logical images so replication using **cubrid unloaddb** and **cubrid loaddb** is unavailable. Because **cubrid backupdb** backs up physical images, replication is possible by using this utility. The **ha_make_slavedb.sh** script performs replication by using **cubrid backupdb**.

- Online-backup of a source node and restoration of a rebuilding node during rebuilding replication.

  ha_make_slavedb.sh script executes online backup about a source node during DB operation, and restore to the being-rebuilt node. To apply added transactions after backup to the being-rebuilt node, **ha_make_slavedb.sh** script use the archive logs of "master" node by copying(in all cases, archive logs of "master" are used; all cases include how to build slave from master, how to build replica from slave and how to build replica from replica).

  Therefore, so as not to remove archive logs which have been added to the source node during the execution of the online backup, there is a need to properly set **force_remove_log_max_archives** and **log_max_archives** in cubrid.conf from the master node. For details, see the below building examples.

- **Error while executing the rebuilding replication script**

  The rebuilding replication script is not automatically rolled back to its previous stage even when an error occurs during the execution. This is because the slave node cannot provide normal service before rebuilding replication script is executed. To return to the phase before rebuilding replication script is executed, you must back up the existing replication

logs and **db_ha_apply_info** information which is internal catalog of the master and slave nodes before building replication is executed.

CUBRID Security

This chapter describes the CUBRID security features. CUBRID provides Packet Encryption, ACL(Access Control List), authorization, and TDE(Transparent Data Encryption) features to protect the user database.

Packet Encryption

Requirements of secure communication

Without an encrypted connection between the client and the server, a hacker with access to the network can monitor all traffic and steal and exploit data exchanged between the client and the server. In this way, a third party (middle-man) steals and makes bad use of information, it is called a man in the middle (MITM) attack. This MITM attack can be prevented by adding a secure authentication procedure to the connection between the client and the server.

Packet encryption method

CUBRID uses SSL/TLS (Secure Socket Layer/Transport Layer Security) protocol to encrypt data transmitted between the client and the server. The CUBRID server uses OpenSSL for encryption, and the client can make an encrypted connection using JDBC or CCI Driver. The encryption protocols supported between the client and server are SSLv3, TLSv1, TLSv1.1, and TLSv1.2.

Note

SSL/TLS

SSL was first developed by Netscape as an encryption protocol that provides authentication and data encryption between a client and server working over a network. Netscape released version 3.0, which improved security flaws in 1996; Since then, the SSL 3.0 version became the basis for TLS 1.0; and it was defined by the IETF in January 1999 as the RFC2246 standard. The last update is RFC5246 . TLS is defined based on SSL 3.0, and it is almost similar to SSL 3.0.

SSL/TLS provides Server Authentication, Client Authentication, and Data Encryption functions. Authentication refers to a procedure to verify that the other party is correct, and encryption refers to preventing access to contents even if data is stolen.

Server setup for packet encryption

Setting encryption mode and non-encryption mode

CUBRID can set either an encryption mode or a non-encryption mode, and the default is non-encryption mode. To change to the encryption mode, you can set the encryption mode by changing the SSL parameter value in cubrid_broker.conf. If you change the SSL parameter value of cubrid_broker.conf, you must restart the broker. For detailed configuration method, refer to Broker Configuration.

Certificate and Private Key

In order to exchange an encrypted symmetric session key which will be used in a secure communication session, a public key and a private key are required in the server.

The public key used by the server is included in the certificate 'cas_ssl_cert.crt', and the private key is included in 'cas_ssl_cert.key'. The certificate and private key are located in the $CUBRID/conf directory.

This certificate, 'self-signed' certificate, was created with the OpenSSL command tool utility and can be replaced with another certificate issued by a public CA (Certificate Authorities, for example, IdenTrust or DigiCert) if desired. Or, the existing certificate/private key can be replaced by generating a new one using the OpenSSL command utility as shown below.

```
$ openssl genrsa -out my_cert.key
2048                                          # create 2048 bit
size RSA private key
$ openssl req -new -key my_cert.key -out
my_cert.csr                                  # create CSR
(Certificate Signing Request)
$ openssl x509 -req -days 365 -in my_cert.csr -signkey my_cert.key -
out my_cert.crt  # create a certificate valid for 1 year.
```

And replace my_cert.key and my_cert.crt with $CUBRID/conf/cas_ssl_cert.key and $CUBRID/conf/cas_ssl_cert.crt respectively.

Supported driver

CUBRID provides various drivers. Currently, the drivers that support packet encryption feature are JDBC and CCI.

Encrypted connection method

The client can make an encrypted connection with the server by using the useSSL property of db-url during driver connection setup. For more information on how to use it, refer to Configuration Connection of the JDBC driver or cci_connect_with_url of the CCI driver.

ACL (Access Control List)

CUBRID supports the ACL function that can be accessed only for users or applications authorized in two layers: the broker layer and the database server layer. The ACL of the broker layer is mainly used to control the access of application programs such as WEB and WAS, and the ACL of the database server layer is mainly used to control the access of the brokers and csqls.

For more details, see Limiting Broker Access and Limiting Database Server Access.

Authorization

CUBRID can create users(or groups) and provide a function to control the access of the other users(or groups) to tables created by a user.

If you want to allow other users(or groups) to access your tables, you could provide access privileges to the users(or groups) by GRANT. Also, to revoke access privileges of other users, you can use REVOKE. Access to the (virtual) table created by a PUBLIC user is allowed to all users.

For more details, see User Management.

TDE (Transparent Data Encryption)

CUBRID TDE Concept

CUBRID supports **Transparent Data Encryption (henceforth, TDE)**. TDE means transparently encrypting data from the user's point of view. This allows users to encrypt data stored on disk with little to no application change.

CUBRID TDE provides encryption and decryption at the engine level to minimize performance degradation due to encryption. When a user creates an encrypted table, all relevant user data stored on disk (data at rest) is automatically encrypted. By providing TDE, CUBRID helps users to comply with security regulations and guidelines required in various sites.

Table Encryption

In CUBRID, a **table** is the unit for TDE-encryption. To use the TDE feature, create a table using the **ENCRYPT** option as follows. For more information, see Table Encryption (TDE).

```
CREATE TABLE tde_tbl (att1 INT, att2 VARCHAR(20)) ENCRYPT=AES;
```

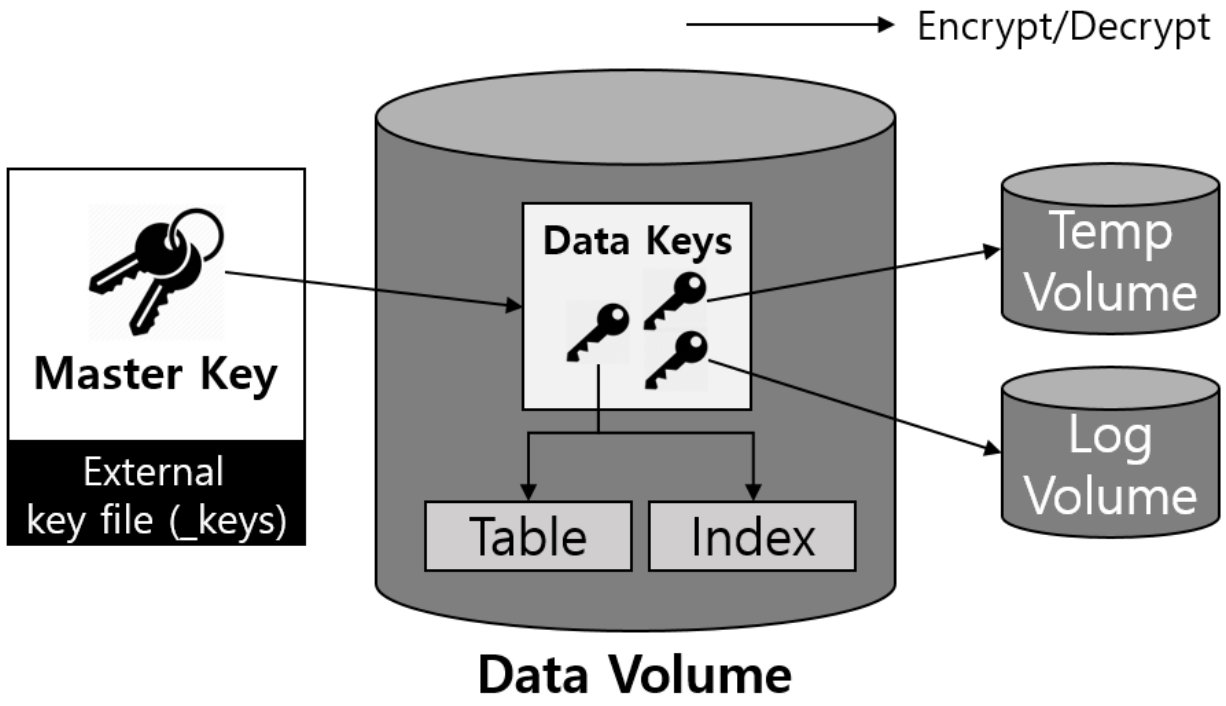
When an encrypted table is created, all data related to the table is automatically encrypted when written to disk; and decrypted when read into memory. Related data includes not only tables but also indexes created on the table, temporary data created while executing queries related to the table, logs created when data is changed, DWB, and backups. For more details, see Encryption Target and TDE Restriction.

Key Management

CUBRID uses symmetric key algorithms to encrypt the data. Keys used for encryption are managed in two levels consisting of master keys and data keys for efficiency. Master keys managed by the user are stored in a separate file, and CUBRID provides a utility to manage it.

2-Level Key Management

CUBRID TDE manages keys in two levels as follows:



- **Master key**: A key used when encrypting and decrypting data keys, and it is managed by DBA user.
- **Data Key**: A key used when encrypting user data such as table and log, and it is managed by CUBRID Engine.

Data keys are stored within the data volume and are always securely encrypted using a master key when written to disk. The master key is stored in a separate file, and it must be managed safely according to the security policy users comply with.

Managing keys in two levels makes it possible to perform the key change operation efficiently. If there is only a key that encrypts the user data, it takes a long time to work when you change the key. All the data that has been encrypted has to be read, decrypted, and re-encrypted. Also, the overall performance of the database may be degraded during this process.

⚠ Warning

Loss of Master Key

If the master key is lost, data encrypted by TDE cannot be read or changed.

File-based Master Key Management

Master keys are separately stored and managed as a separate key file so that the user can manage master keys in various ways according to individual security requirements. This key file contains all the information of master keys, so if it is leaked, there may be a security problem, and if it is lost, the encrypted data cannot be read (When TDE is unavailable). So, be careful to manage this key file.

By default, the key file is created with the name of **<database-name>_keys** at the location where the data volume is created when creating a database using **cubrid createdb** utility. Without additional configuration for the key file, this key file is automatically used. The location of the key file te be used can be changed by a system parameter. For more information, see Disk-Related Parameters.

The key file can contain several master keys (up to 128). A master key among those keys is set on the database to encrypt the database, data keys technically. One master key is created and set by default when the key file is created, and DBA can add, delete, change, and search keys using the TDE utility (TDE utility). When deleting a key, the key to delete must exist in the key file, and the key set on the database currently cannot be removed. When changing a key to set to encrypt a database, both the previously key set on the database and the key to be set must exist in the key file. Through key inquiry, you can check the number of keys and creation time of them, and you can check the current key set on the database and setting time.

```
$ cubrid tde --show-keys testdb
Key File: /home/usr/CUBRID/databases/testdb/testdb_keys

The current key set on testdb:
Key Index: 2
Created on Fri Nov 27 11:14:54 2020
Set     on Fri Nov 27 11:15:30 2020

Keys Information:
Key Index: 0 created on Fri Nov 27 11:11:27 2020
Key Index: 1 created on Fri Nov 27 11:14:47 2020
Key Index: 2 created on Fri Nov 27 11:14:54 2020
Key Index: 3 created on Fri Nov 27 11:14:55 2020

The number of keys: 4
```

Note

Creating a database using an existing key file

If you want to create a new database using a key file that was previously managed for reasons such as security policy, copy or move the key file to the directory where the database will be created before creating it. The name of the key file must be changed to

<database_name>_keys. If you're using the **tde_keys_file_path** system parameter, you have to copy the key file to the path.

Encryption Target

Permanent Data Encryption

The encrypted table data and all index data created on the table are encrypted. For more information on the encrypted table, see Table Encryption (TDE).

Temporary Data Encryption

In addition to persistent data such as tables, temporary data created during queries related to encrypted tables are also encrypted. For example, all temporary data created in executing a query such as *SELECT * FROM tde_tbl ORDER BY att1* or creating an index on *tde_tbl* are encrypted when it is written to disk. For more information on temporary data, see Temporary Volume.

Log Data Encryption

Since the data which has to be encrypted may be included in the REDO and UNDO log records generated when the encrypted table is manipulated, all log data related to the encrypted table is encrypted. Encryption is applied to both the active log and the archive log. For more information on log volumes, see Database Volume.

DWB Encryption

Persistent data is temporarily written to the Double Write Buffer (DWB) before being written to the data volume. It may be encrypted even at this time because the data for the encrypted table can be included. For more information on DWB, see Database Volume.

Backup Encryption

If there are encrypted data in data volumes and log volumes, they are also stored as encrypted in backup volumes. For more information on backup, see backupdb.

Backup Key File

The backup volume contains the key file by default. If the backup volume, including the key file, is leaked, meaning the master key is also leaked. There may be a security problem even though the data in the volume is encrypted. To prevent this, you can backup the key file separately by using the **--separate-keys** option. However, in the case of separating the key file, it must be managed carefully to prevent losing the key file for database restore. The separated backup key file is created in the same directory path as the backup volume and has the name **<database_name>_bk<backup_level>_keys**.

```
$ cubrid backupdb -S --separate-keys testdb
Backup Volume Label: Level: 0, Unit: 0, Database testdb, Backup
Time: Mon Nov 30 14:34:49 2020
$ ls
lob  testdb  testdb_bk0_keys  testdb_bk0v000  testdb_bkvinf
testdb_keys
testdb_lgar_t  testdb_lgat  testdb_lginf  testdb_vinf
```

The key file used to restore

The key file separated during backup can be given as the key file for restoration by using the **--keys-file-path** option (restoredb). If the valid key file does not exist in the specified path, restore fails.

If the **--keys-file-path** option is not given, the key file to be used is searched according to the following priority. If the valid key file cannot be found, restore fails.

Key file classification

- Server key file: A key file that is generally used when running the server. It can be set with the tde_keys_file_path system parameter or in the default path same as the data volume.
- Backup key file: A key file created during backup included in the backup volume or separated by **--separate-keys** option.

The priority of the key file to use for restore

1. The backup key file that the backup volume contains.
2. The backup key file created with the **--separate-keys** option during backup (e.g. testdb_bk0_keys). This key file must exist in the same path as the backup volume.
3. The server key file in the path specified by the **tde-keys-file-path** system parameter.
4. The server key file in the same path as the data volume (e.g., testdb_keys).

Note

In the case of (1), If the backup volume contains a backup key file, the backup key file is copied with the same name as the one created by **--separate-keys** during restore.

Even if the valid key file is not found, restore could be successful if there is no encrypted data in the backup volume. However, since the key file does not exist, you cannot use TDE functions later.

Note

Incremental Backup

When performing restoration using multiple level backup volumes by incremental backup, the backup key file of the level specified by the **--level** option is used. If the **--level** option is not specified, the highest level backup key file is used. If only the key file to be used exists, restore can succeed.

Note

Loss of the backup key file

If the backup key file is lost, the restore would fail. However, if the key is not changed, the backup key file of the previous volume can be used by using the **--keys-file-path** option. Also, if the key at the backup time exists in the server key, it can be used for backup recovery. Generally, restore can succeed if any key file that has the key intactly at the backup time is given.

Note

The case in which the key is changed automatically after restore

Suppose the key set on the database does not exist in the server key file at the end of the restoration process. In that case, the backup key file is copied to the server key file, and the first key in the key file is arbitrarily set on the database for encrypting the database. This is because the key set on the database may not exist in any key file after the restore is complete.

Encryption Algorithm

CUBRID supports the following encryption algorithms for TDE.

TDE Encryption Algorithm

Algorithm	Key Size	Option Name
Advanced Encryption Standard	256 bits	AES
ARIA	256 bits	ARIA

Advanced Encryption Algorithm (AES) is a specification established by the National Institute of Standards and Technology (NIST) and is widely used worldwide. It has high stability and many optimizations are supported by many platforms such as hardware acceleration, so there is little performance degradation during encryption/decryption. ARIA is one of Korea's national standard encryption algorithms and is optimized for lightweight environments and hardware implementation.

Note

Default Encryption Algorithm for TDE

If the algorithm is not specified when creating the TDE encryption table, AES is used by default. If you want to change the default encryption algorithm, you can specify it by the system parameter **tde_default_algorithm**. This default encryption algorithm is used to encrypt logs or temporary data in addition to tables. For details on specifying the encryption algorithm when creating a table, see Table Encryption (TDE).

Table Encryption Checking

You can check whether the table is encrypted by following three ways.

SHOW CREATE TABLE

```
csql> show create table tde_tbl1;

=== <Result of SELECT Command in Line 1> ===
```

```
  TABLE                 CREATE TABLE
============================================
  'tde_tbl1'          'CREATE TABLE [tde_tbl1] ([a] INTEGER)
REUSE_OID, COLLATE iso88591_bin ENCRYPT=AES'

1 row selected. (0.144627 sec) Committed.

1 command(s) successfully processed.
```

Inquiry to db_class

Encryption of each table and encryption algorithm can be checked by the **tde_algorithm** column of the system catalog **db_class** or **_db_class**. For more information on the system catalog, see System Catalog.

```
csql> select class_name, tde_algorithm from db_class where
class_name like '%tde%';

=== <Result of SELECT Command in Line 1> ===

  class_name            tde_algorithm
============================================
  'tde_tbl1'          'AES'
  'tde_tbl2'          'ARIA'
  'not_tde_tbl'       'NONE'

3 rows selected. (0.057243 sec) Committed.

1 command(s) successfully processed.
```

Using cubrid diagdb utility

You can check by referring to **tde_algorithm** among the file header information of encrypted tables and index files from the result by **cubrid diagdb** utility with the -d1 (dump file tables) option. For more details, see diagdb.

```
$ cubrid diagdb -d1 testdb
...
Dumping file 0|3520
        file header:
                vfid = 0|3520
                permanent
                regular
                tde_algorithm: AES
                page: total = 64, user = 1, table = 1, free = 62
```

```
                sector: total = 1, partial = 1, full = 0, empty = 0
...
```

TDE on HA

In a HA environment, TDE is applied independently to each node. This means that for each node, the key file and TDE-related system parameters can be managed independently.

However, the TDE information of the replicated table is shared and the same. So, if the TDE module of the slave node is not loaded, the replication will stop when attempting to manipulate an encrypted table from the master node. In this case, not only the changes to a TDE-enctyped table, but also any subsequent changes cannot be replicated. Afterward, if the slave node's TDE configuration is correct and restarted, replication resumes from the stopped point.

When TDE is unavailable

In the following cases, the TDE feature cannot be used, and an error occurs because the TDE module cannot be loaded correctly.

- When the valid key file cannot be found
- When the key set on the database cannot be found in the key file

Even if the TDE module is not loaded, the server can start normally, and users can access unencrypted tables. This means that all DML and DDL such as SELECT and INSERT only for TDE-encrypted tables cannot be executed.

However, the case log data has been encrypted is different. If the log data is encrypted when the TDE module is not loaded and the log is accessed by recovery, HA, VACUUM, etc., the system cannot be properly executed, and the entire server has no option but to stop running the server.

TDE Restriction

In addition to the restrictions described above, there are the following.

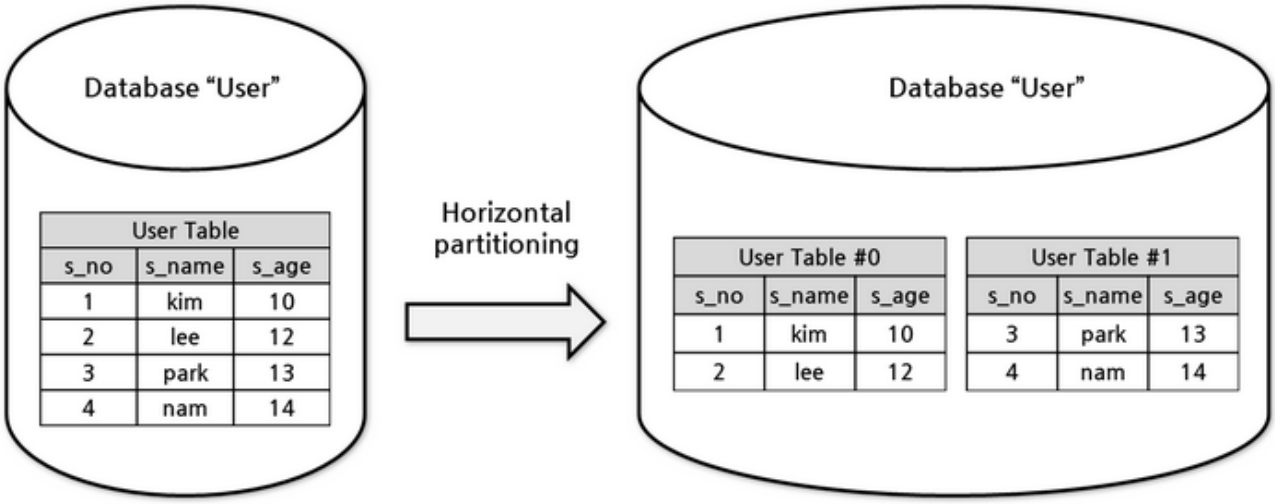
1. The replication log is not encrypted in HA.
2. CUBRID does not support the **ALTER TABLE** statement to change the TDE table option, which means you cannot set TDE to existing tables. If you want to do that, you need to move the data to the new table created with the TDE table option.
3. SQL log is not encrypted. For more information on the SQL log, see Managing the SQL Log.

CUBRID SHARD

Database Sharding

Horizontal partitioning

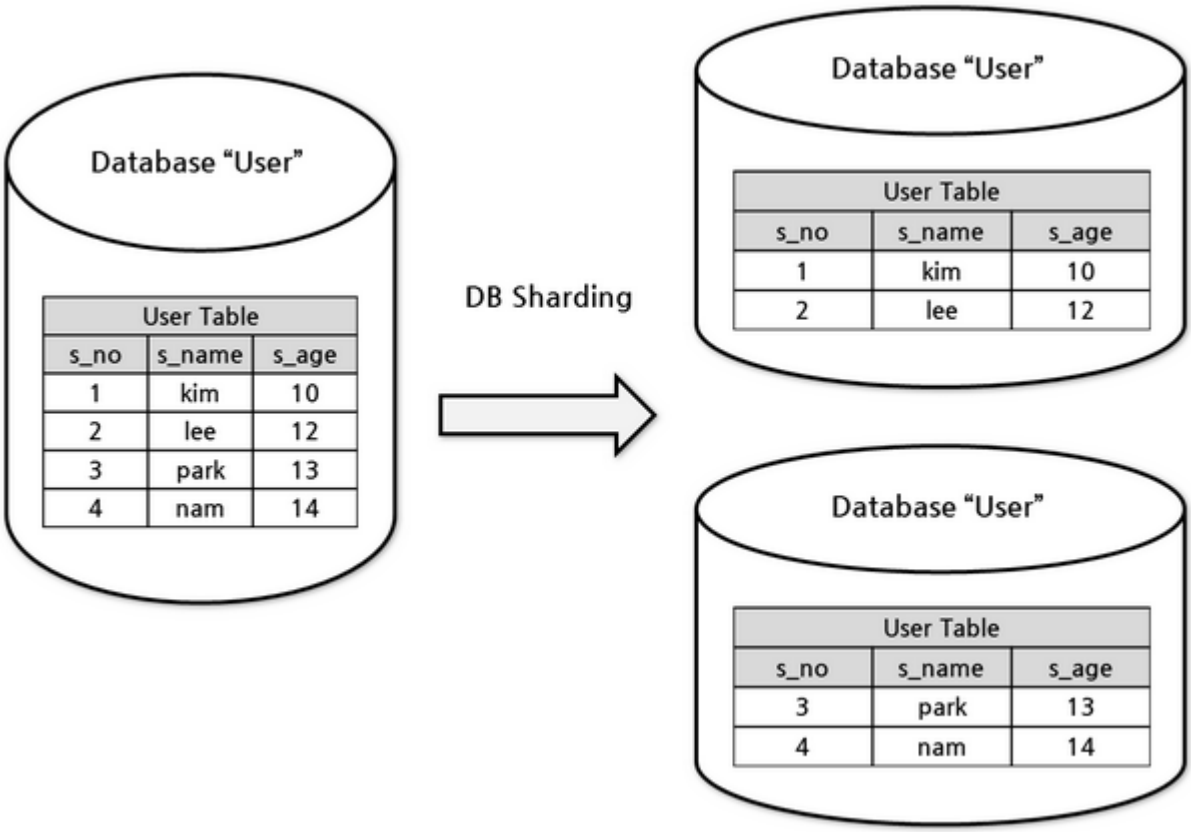
Horizontal partition is a design to partition the data for which schema is identical, based on the row, into multiple tables and store them. For example, 'User Table' with identical schema can be divided into 'User Table #0' in which users less than 13 years are stored and 'User Table #1' in which users 13 or greater than 13 years old.



Database sharding

Database sharding is to store data into physically separate databases through horizontal partitioning and to retrieve them. For example, it is a method to store users less than 13 years old in database 0 and store users 13 or greater than 13 years old in database 1 when 'User Table' is located through multiple database. In addition to performance, database sharding is used to distribute and save large data which cannot be saved into one database instance.

Each partitioned database is called a shard or database shard.



CUBRID SHARD

The CUBRID SHARD is middleware for database sharding and its characteristics are as follows:

- Middleware that is used to minimize changes in existing applications, allowing access to transparently sharded data by using Java Database Connectivity (JDBC), a popular choice, or CUBRID C Interface (CCI), which is CUBRID C API.
- In this function, a hint may be added to an existing query to indicate a shard in which the query would be executed.
- It guarantees the unique characteristics of certain transactions.

CUBRID SHARD Terminologies

The terminologies used to describe CUBRID SHARD are as follows:

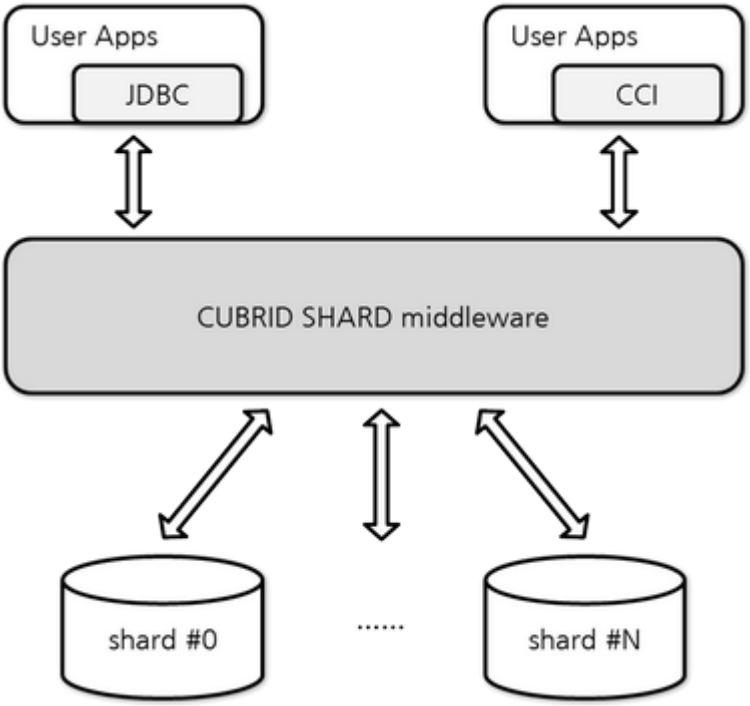
- **shard DB** : A database that includes a split table and data and processes user requests
- **shard metadata** : Configuration information for the operation of a CUBRID SHARD. It analyzes the requested query and includes information to select a shard DB which will execute a query and to create a database session with the shard DB.
- **shard key (column)** : A column used as an identifier to select a shard in the sharding table

- **shard key data** : A shard key value that corresponds to the hint to identify the shard from the query
- **shard ID** : An identifier to identify a shard DB
- **proxy** : A CUBRID middleware process that analyzes hints included in user queries and sends requests to a shard DB which will process the query, based on the analyzed hint and the shard metadata

CUBRID SHARD Main Features

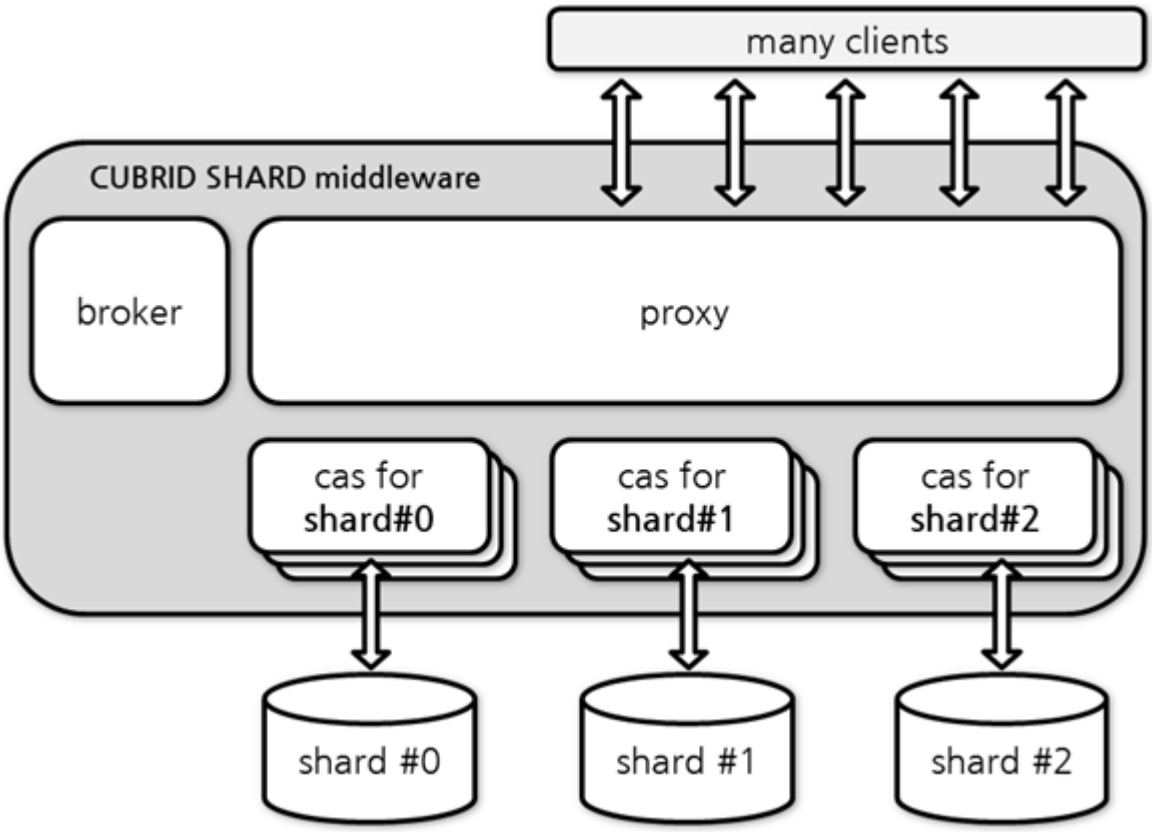
Middleware Structure

The CUBRID SHARD is middleware, positioned between an application and logically, or physically, split shards. It keeps connection with multiple applications, sends requests from applications to the appropriate shard, and returns the results to the applications.



Generally, it uses Java Database Connectivity (JDBC) or CUBRID C Interface (CCI), an interface used to connect to the CUBRID SHARD, and processes the requests from applications. It does not require additional driver or framework, minimizing the changes in applications.

The CUBRID SHARD middleware consists of three processes (broker/proxy/CAS) and the brief features of each process are as follows:



· **broker**

◦ Receives the initial connection request from drivers such as JDBC/CCI and then sends the received request to the proxy based on the load balancing policy.

◦ Monitors and restores the status of the proxy process and the CAS process.

· **proxy**

◦ Sends the user request from the driver and then returns the processing result to the application.

◦ Manages the connection between the driver and the CAS and processes transactions.

· **CAS**

◦ Creates a connection with the split shard DB and processes the user request received from the proxy by using the connection.

◦ Processes transactions.

Selecting a Shard DB through the Shard SQL Hint

Shard SQL Hint

With the hints and configuration data included in a SQL hint statement, the CUBRID SHARD selects a shard DB that will process the requests from applications. The types of available SQL hints are as follows:

SQL hint	Description
/*+ shard_key */	A hint to specify the position of the bind variable or literal value that corresponds to the shard key column.
/*+ shard_val(*value* **) */**	A hint to explicitly specify the shard key within the hint when there is no column that corresponds to the shard key in the query.
/*+ shard_id(*shard_id* **) */**	A hint used when a user specifies a shard DB to process queries.

The terms are summarized as shown below: For more information on shard terms, see CUBRID SHARD Terminologies.

- **shard key** : A column to distinguish shard DBs. In general, this column exists in all or most tables in a shard DB and has a unique value.
- **shard id** : An identifier that can be used to logically distinguish shards. For example, when one DB is split into four shard DBs, there are four shard IDs.

For more information on the query process using hints and configuration information, see General Procedure of Executing Queries by Using Shard SQL Hint.

Note

- When more than one shard hint exist on a query, it works normally if shard hints indicate the same shards, but it fails if each of them indicates the different shard.

```
SELECT * FROM student WHERE shard_key = /*+ shard_key */ 250 OR
shard_key = /*+ shard_key */ 22;
```

On the above case, it works normally if the shard keys 250 and 22 indicate the same shard, but it fails if they indicate the different shards.

- On some driver functions which batches the queries with an array by binding the several values(ex. PreparedStatement.executeBatch in JDBC, cci_execute_array in CCI), if at least the one which accesses to the other shard exists, all executions of the queries fail.
- Functions to run several statements at one time on shard environment(ex. Statement.executeBatch in JDBC, cci_execute_batch in CCI) will be supported later.

shard_key Hint

The **shard_key** hint is to specify the position of a bind or literal variable. This hint should be positioned in front of either of them.

Ex) Specifies the position of a bind variable. Executes the query in the shard DB corresponding to the student_no value that would be bound when executed.

```
SELECT name FROM student WHERE student_no = /*+ shard_key */ ?;
```

Ex) Specifies the position of a literal value. Executes the query in the shard DB corresponding to the student_no value (the literal value) that is 123 when executed.

```
SELECT name FROM student WHERE student_no = /*+ shard_key */ 123;
```

shard_val Hint

The **shard_val** hint is used when there is no shard column that can be used to identify the shard DB in the query. It sets the shard key column as the value of the **shard_val** hint. The **shard_val** hint can be positioned anywhere in an SQL statement.

Ex) When the shard key is not included in the student_no or in the query, the query is performed in the shard DB in which the shard key (student_no) is 123.

```
SELECT age FROM student WHERE name =? /*+ shard_val(123) */;
```

shard_id Hint

Regardless of the shard key column value, the **shard_id** hint can be used when the user specifies a shard for query execution. The **shard_id** hint can be positioned anywhere in an SQL statement.

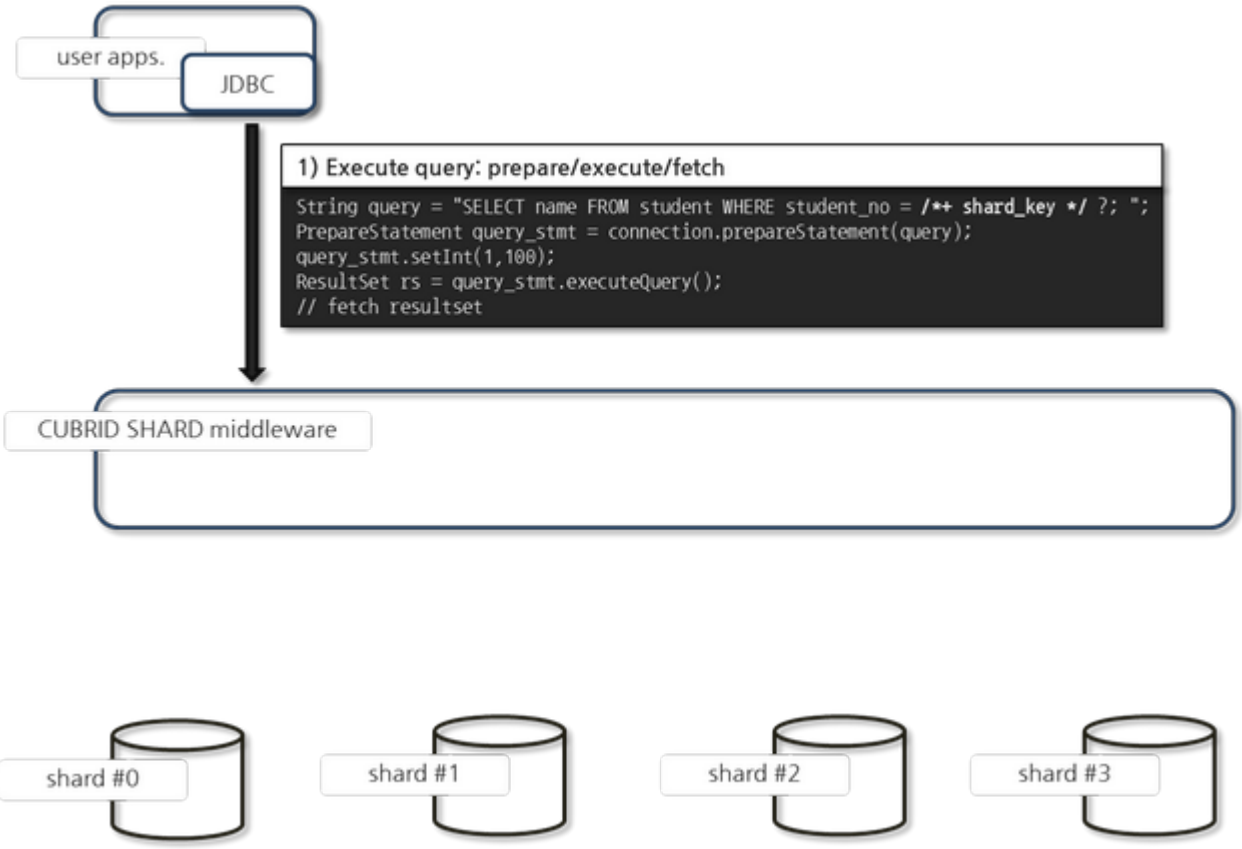
Ex) When the query is performed in shard DB #3, queries students whose value of age is greater than 17 in the shard DB #3.

```
SELECT * FROM student WHERE age > 17 /*+ shard_id(3) */;
```

General Procedure of Executing Queries by Using Shard SQL Hint

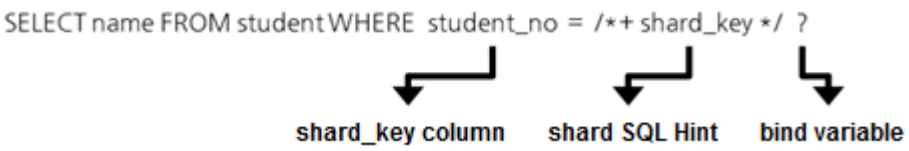
1. Executing Queries

The following shows how a user-requested query is executed.

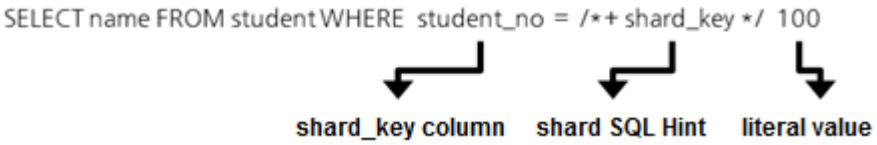


- An application makes a request for a query to the CUBRID SHARD through the JDBC interface. It adds the **shard_key** hint to the SQL statement to specify the shard DB from where the query will be executed.
- The SQL hint, like the example above, in the SQL statement, should be positioned in front of the bind variable or literal value of the column specified by the shard key.

The shard SQL hint configured by the bind variable is as follows:

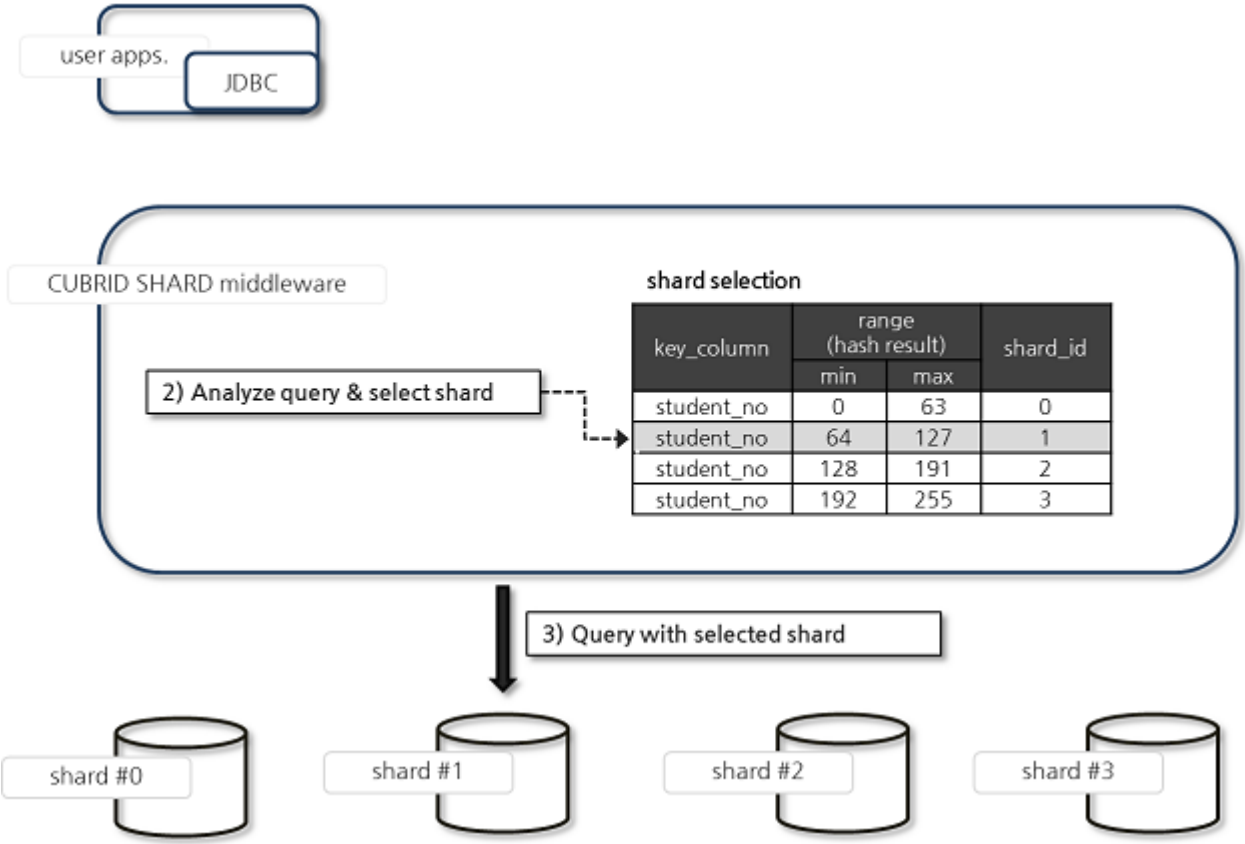


The shard SQL hint specified in the literal value is as follows:



2. Select a Shard DB to Analyze and Perform a Query

Select a shard DB to analyze and perform the query by following the steps below:



◦ SQL queries received from users are rewritten in the format that is appropriate for internal processing.

◦ Select the shard DB that executed the query by using the SQL statement and hint requested by the user.

▪ When the SQL hint is set in the bind variable, select the shard DB which will execute the query by using the result of hashing the value of the shard_key bind variable and the configuration information.

▪ The hash function can be specified by the user. If not specified, the shard_key value is hashed by using the default hash function. Default hash functions are as follows:

▪ When the shard_key is an integer

```
Default hash function (shard_key) = shard_key mod
SHARD_KEY_MODULAR parameter (default value 256)
```

- When the shard_key is a string

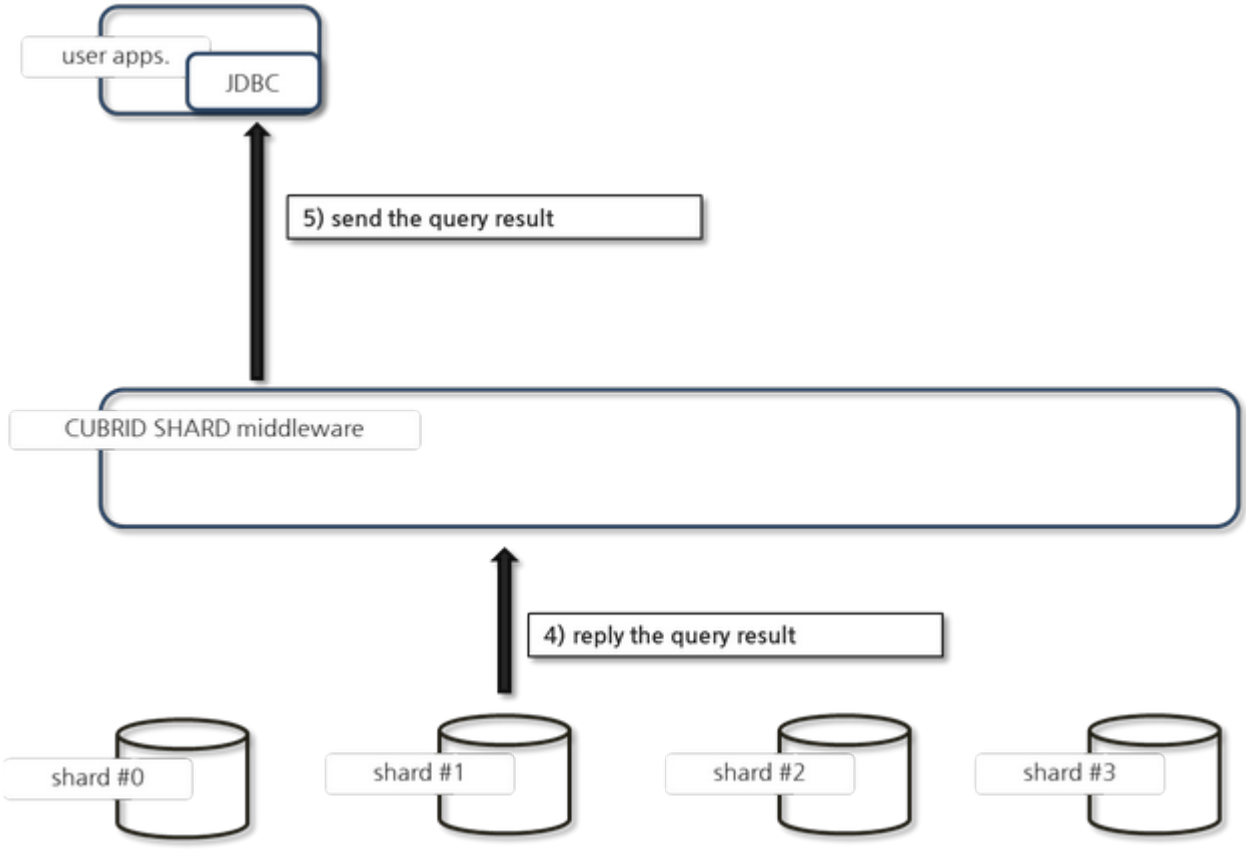
```
Default hash function (shard_key) = shard_key[0] mod
SHARD_KEY_MODULAR parameter (default value 256)
```

Note

When the shard_key bind variable value is 100, "Default hash function (shard_key) = 100 % 256 = 100." Therefore, the shard DB #1 (the hash result is 100) will be selected and then the user request will be sent to the selected shard DB #1.

3. Return the Query Execution Result

Return the query execution result as follows:



- Receives the query execution result from the shard DB #1 and then returns it to the requested application.

Note

On the driver functions which do a batch query processing with the array which binds several values(ex. executeBatch in JDBC, cci_execute_array and cci_execute_batch in CCI), they fail to run if there is a value which accesses to a different shard.

Transaction Support

Transaction Processing

The CUBRID SHARD executes an internal processing procedure to guarantee atomicity among ACID. For example, when an exception such as abnormal termination of an application occurs, the CUBRID SHARD sends a request to rollback to the shard DB which has been processing the request from the application in order to invalidate all changes in the transaction.

The ACID, the characteristic of general transactions, is guaranteed, based on the characteristics and settings of the backend DBMS.

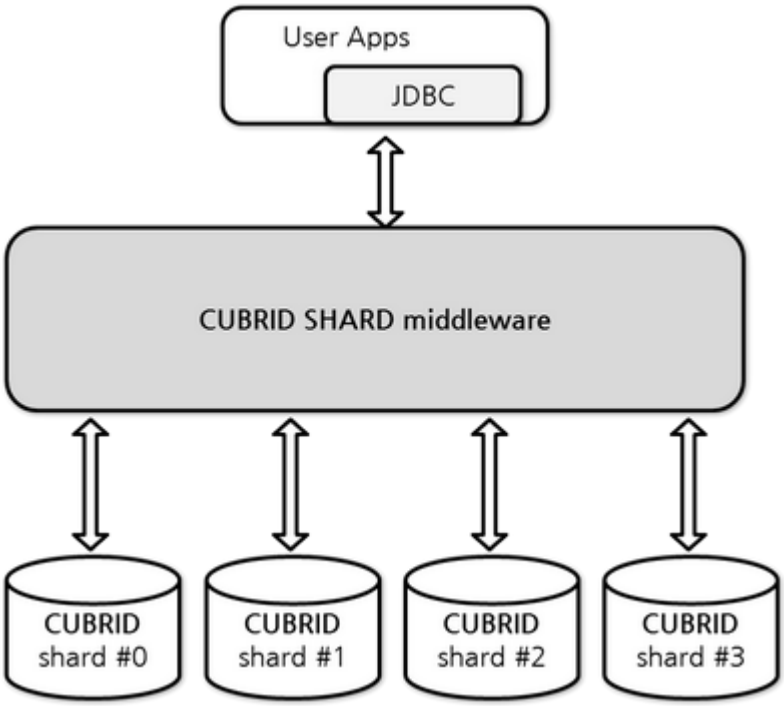
Constraints

2 Phase Commit (2PC) is unavailable; therefore, an error occurs when a query is executed by using several shard DBs in a single transaction.

Quick Start

Configuration Example

The CUBRID SHARD to be explained consists of four CUBRID SHARD DBs as shown below. The application uses the JDBC interface to process user requests.



Start after creating the shard DB and user account

As shown in the example above, after each shard DB node creates a shard DB and a user account, it starts the instance of the database.

- shard DB name: *shard1*
- shard DB user account: *shard*
- shard DB user password: *shard123*

```
sh> # Creating CUBRID SHARD DB
sh> cubrid createdb shard1 en_US

sh> # Creating CUBRID SHARD user account
sh> csql -S -u dba shard1 -c "create user shard password 'shard123'"

sh> # Starting CUBRID SHARD DB
sh> cubrid server start shard1
```

Changing the shard Configurations

cubrid_broker.conf

Change **cubrid_broker.conf** as shown below by referring to **cubrid_broker.conf.shard**:

⚠ Warning

The port number and the shared memory identifier should be appropriately changed to the value which has not been assigned by the system.

```
[broker]
MASTER_SHM_ID           =30001
ADMIN_LOG_FILE          =log/broker/cubrid_broker.log

[%shard1]
SERVICE                 =ON
BROKER_PORT             =36000
MIN_NUM_APPL_SERVER     =20
MAX_NUM_APPL_SERVER     =40
APPL_SERVER_SHM_ID      =36000
LOG_DIR                 =log/broker/sql_log
ERROR_LOG_DIR           =log/broker/error_log
SQL_LOG                 =ON
TIME_TO_KILL            =120
SESSION_TIMEOUT         =300
KEEP_CONNECTION         =ON
MAX_PREPARED_STMT_COUNT =1024
SHARD                   =ON
SHARD_DB_NAME           =shard1
SHARD_DB_USER           =shard
SHARD_DB_PASSWORD       =shard123
SHARD_NUM_PROXY         =1
SHARD_PROXY_LOG_DIR     =log/broker/proxy_log
SHARD_PROXY_LOG         =ERROR
SHARD_MAX_CLIENTS       =256
SHARD_PROXY_SHM_ID      =36090
SHARD_CONNECTION_FILE   =shard_connection.txt
SHARD_KEY_FILE          =shard_key.txt
```

For CUBRID, the server port number is not separately configured in the **shard_connection.txt** but the **cubrid_port_id** parameter of the **cubrid.conf** configuration file is used. Therefore, set the **cubrid_port_id** parameter of the **cubrid.conf** identical to the server.

```
# TCP port id for the CUBRID programs (used by all clients).
cubrid_port_id=41523
```

shard_key.txt

Set **shard_key.txt**, the shard DB mapping configuration file, for the shard key hash value as follows:

- [%shard_key]: Sets the shard key section

- Executing the query at shard #0 when the shard key hash result created by default hash function is between 0 and 63
- Executing the query at shard #1 when the shard key hash result created by default hash function is between 64 and 127
- Executing the query at shard #2 when the shard key hash result created by default hash function is between 128 and 191
- Executing the query at the shard #3 when the shard key hash result created by default hash function is between 192 and 255

```
[%shard_key]
#min    max     shard_id
0       63      0
64      127     1
128     191     2
192     255     3
```

shard_connection.txt

Configure the **shard_connection.txt** file which is shard database configuration file, as follows:

- Real database name and connection information of shard #0
- Real database name and connection information of shard #1
- Real database name and connection information of shard #2
- Real database name and connection information of shard #3

```
# shard-id  real-db-name  connection-info
#                         * cubrid : hostname, hostname, ...
0           shard1        HostA
1           shard1        HostB
2           shard1        HostC
3           shard1        HostD
```

Starting Service and Monitoring

Starting CUBRID SHARD

To use CUBRID SHARD feature, start the CUBRID SHARD as shown below:

```
sh> cubrid broker start
@ cubrid broker start
++ cubrid broker start: success
```

Retrieving the CUBRID SHARD Status

Retrieve the CUBRID SHARD status as follows to check the parameter and the status of the process.

```
sh> cubrid broker status
@ cubrid broker status
% shard1
----------------------------------------------------------------
ID      PID   QPS   LQS PSIZE STATUS
----------------------------------------------------------------
1-0-1 21272     0     0 53292 IDLE
1-1-1 21273     0     0 53292 IDLE
1-2-1 21274     0     0 53292 IDLE
1-3-1 21275     0     0 53292 IDLE

sh> cubrid broker status -f
@ cubrid broker status
% shard1
----------------------------------------------------------------------------
----------------------------------------------------------------------------
----------------
ID      PID   QPS   LQS PSIZE STATUS          LAST ACCESS
TIME               DB             HOST   LAST CONNECT TIME
SQL_LOG_MODE
----------------------------------------------------------------------------
----------------------------------------------------------------------------
----------------
1-0-1 21272     0     0 53292 IDLE         2013/01/31 15:00:24
shard1@HostA           HostA 2013/01/31 15:00:25               -
1-1-1 21273     0     0 53292 IDLE         2013/01/31 15:00:24
shard1@HostB           HostB 2013/01/31 15:00:25               -
1-2-1 21274     0     0 53292 IDLE         2013/01/31 15:00:24
shard1@HostC           HostC 2013/01/31 15:00:25               -
1-3-1 21275     0     0 53292 IDLE         2013/01/31 15:00:24
shard1@HostD           HostD 2013/01/31 15:00:25               -
```

Writing a Sample

Check that the CUBRID SHARD operates normally by using a simple Java program.

Writing a Sample Table

Write a temporary table for the example in all shard DBs.

```
sh> csql -C -u shard -p 'shard123' shard1@localhost -c "create table
student (s_no int, s_name varchar, s_age int, primary key(s_no))"
```

Writing Code

The following example program is to enter student information from 0 to 1023 to the shard DB. Check the **cubrid_broker.conf** modified in the previous procedure and then set the address/port information and the user information into the connection url.

```
import java.sql.DriverManager;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.PreparedStatement;
import java.sql.Date;
import java.sql.*;
import cubrid.jdbc.driver.*;

public class TestInsert {

        static  {
                try {

Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
                } catch (ClassNotFoundException e) {
                        throw new RuntimeException(e);
                }
        }

        public static void DoTest(int thread_id) throws SQLException
{
                Connection connection = null;

                try {
                        connection =
DriverManager.getConnection("jdbc:cubrid:localhost:36000:shard1:::?
charSet=utf8", "shard", "shard123");
                        connection.setAutoCommit(false);

                        for (int i=0; i < 1024; i++) {
                                String query = "INSERT INTO student
VALUES (/*+ shard_key */ ?, ?, ?)";
                                PreparedStatement query_stmt =
connection.prepareStatement(query);

                                String name="name_" + i;
                                query_stmt.setInt(1, i);
```

```
                                query_stmt.setString(2, name);
                                query_stmt.setInt(3, (i%64)+10);

                                query_stmt.executeUpdate();
                                System.out.print(".");

                                query_stmt.close();
                                connection.commit();
                        }

                        connection.close();
                } catch(SQLException e) {
                        System.out.print("exception occurs : " +
e.getErrorCode() + " - " + e.getMessage());
                        System.out.println();
                        connection.close();
                }
        }


        /**
         * @param args
         */
        public static void main(String[] args) {
                // TODO Auto-generated method stub

                try {
                        DoTest(1);
                } catch(Exception e){
                        e.printStackTrace();
                }
        }
}
```

Executing a Sample

Execute the sample program as follows:

```
sh> javac -cp ".:$CUBRID/jdbc/cubrid_jdbc.jar" *.java
sh> java -cp ".:$CUBRID/jdbc/cubrid_jdbc.jar" TestInsert
```

Checking the Result

Execute the query in each shard DB and check whether or not the partitioned information has been correctly entered.

- shard #0

```
sh> csql -C -u shard -p 'shard123' shard1@localhost -c "select *
from student order by s_no"

         s_no  s_name                        s_age
=============================================
            0  'name_0'                        10
            1  'name_1'                        11
            2  'name_2'                        12
            3  'name_3'                        13
            ...
```

- shard #1

```
sh> $ csql -C -u shard -p 'shard123' shard1@localhost -c "select *
from student order by s_no"

         s_no  s_name                        s_age
=============================================
           64  'name_64'                       10
           65  'name_65'                       11
           66  'name_66'                       12
           67  'name_67'                       13
           ...
```

- shard #2

```
sh> $ csql -C -u shard -p 'shard123' shard1@localhost -c "select *
from student order by s_no"

          s_no  s_name                         s_age
=============================================
          128  'name_128'                      10
          129  'name_129'                      11
          130  'name_130'                      12
          131  'name_131'                      13
          ...
```

- shard #3

```
sh> $ csql -C -u shard -p 'shard123' shard1@localhost -c "select *
from student order by s_no"

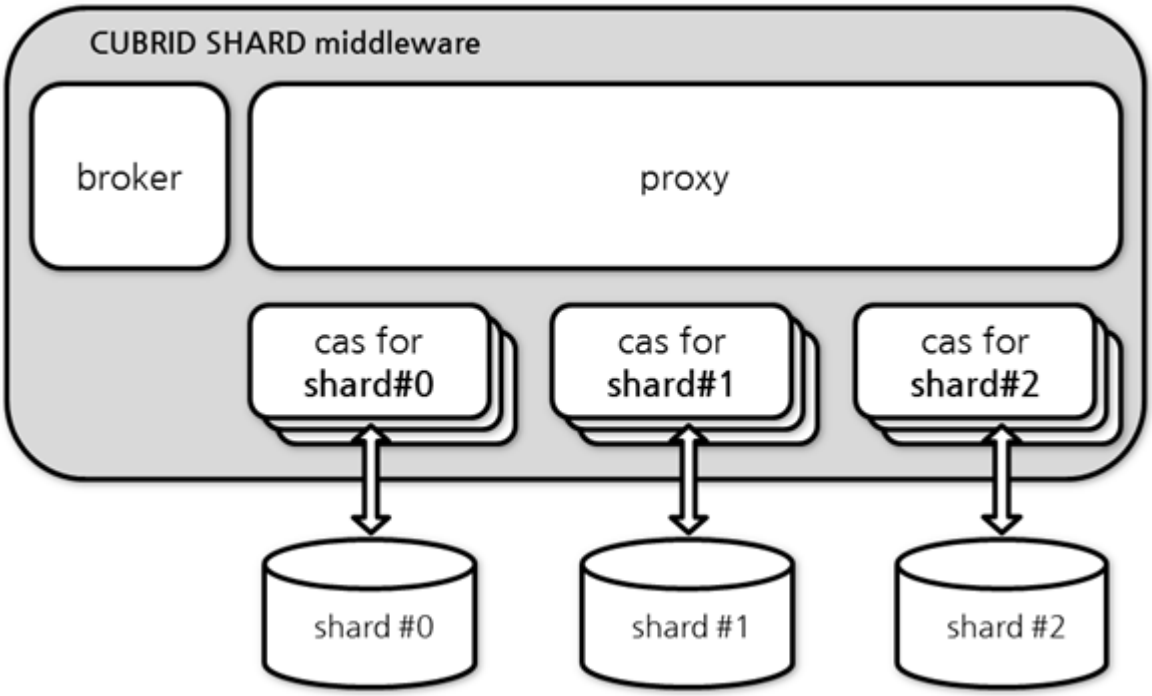
         s_no  s_name                        s_age
=============================================
          192  'name_192'                      10
          193  'name_193'                      11
          194  'name_194'                      12
```

```
          195  'name_195'                    13
          ...
```

Architecture and Configuration

Architecture

The CUBRID SHARD is middleware, consisting of a broker, proxy, and CAS process as shown below.



Configuration

To use the CUBRID SHARD feature, the parameters needed to run SHARD related processes, shard connection file(SHARD_CONNECTION_FILE) and shard key file(SHARD_KEY_FILE) should be configured.

cubrid_broker.conf

The **cubrid_broker.conf** file is used for setting the CUBRID SHARD feature. Refer to **cubrid_broker.conf.shard** when configuring **cubrid_broker.conf**. For details of **cubrid_broker.conf**, see Broker Configuration.

Shard Connection File(SHARD_CONNECTION_FILE)

To connect to the backend shard DB, the CUBRID SHARD loads the shard connection configuration file specified in the **SHARD_CONNECTION_FILE** parameter of **cubrid_broker.conf**.

The shard DB can be set until 256 shards.

If **SHARD_CONNECTION_FILE** is not specified in **cubrid_broker.conf**, it loads the **shard_connection.txt** file by default.

Format

The basic example and format of a shard connection configuration file are as follows:

```
#
# shard-id      real-db-name    connection-info
#                               * cubrid : hostname, hostname, ...

# CUBRID
0               shard1          HostA
1               shard1          HostB
2               shard1          HostC
3               shard1          HostD
```

Note

As shown in the general CUBRID settings, the content after # is converted to comment.

CUBRID

When the backend shard DB is CUBRID, the format of the connection configuration file is as follows:

```
# CUBRID
# shard-id      real-db-name            connection-info
# shard identifier( >0 )        The real name of backend shard DB
host name

0           shard_db_1          host1
1           shard_db_2          host2
2           shard_db_3          host3
3           shard_db_4          host4
```

For CUBRID, a separate backend shard DB port number is not specified in the above configuration file, but the **CUBRID_PORT_ID** parameter in the **cubrid.conf** file is used. The **cubrid.conf** file is by default located in the **$CUBRID/conf**.

```
$ vi cubrid.conf

...

# TCP port id for the CUBRID programs (used by all clients).
cubrid_port_id=41523
```

Shard key file(SHARD_KEY_FILE)

The CUBRID SHARD loads the shard key configuration file specified in the **SHARD_KEY_FILE** parameter of **cubrid_broker.conf** to determine which backend shard DB should process the user requests.

If **SHARD_KEY_FILE** is not specified in **cubrid_broker.conf**, it loads the **shard_key.txt** file by default.

Format

The example and format of a shard key configuration file are as follows:

```
[%student_no]
#min    max     shard_id
0       31      0
32      63      1
64      95      2
96      127     3
128     159     0
160     191     1
192     223     2
224     255     3
```

- [%shard_key_name]: Specifies the name of the shard key.
- min: The minimum value range of the shard key hash results.
- max: The maximum range of the shard key hash results.
- shard_id: The shard identifier

Note

As shown in the general CUBRID settings, the content after # is converted to comment.

⚠ Warning

- min of the shard key should always start from 0.
- max should be up to 255.
- No empty value between min and max is allowed.
- The default hash function should not exceed the value of the **SHARD_KEY_MODULAR** parameter(min. 1, max 256).
- The result of shard key hashing should be within a range from 0 to (**SHARD_KEY_MODULAR** -1).

Setting User-Defined Hash Function

To select a shard that will perform queries, the CUBRID SHARD uses the results of hashing the shard key and the metadata configuration information. For this, users can use the default hash function or define a hash function.

Default Hash Function

When the **SHARD_KEY_LIBRARY_NAME** and **SHARD_KEY_FUNCTION_NAME** parameters of **cubrid_broker.conf** are not set, the shard key is hashed by using the default hash function. The default hash function is as follows:

- When the shard_key is an integer

```
Default hash function (shard_key) = shard_key mod SHARD_KEY_MODULAR
parameter (default value: 256)
```

- When the shard_key is a string

```
Default hash function (shard_key) = shard_key[0] mod
SHARD_KEY_MODULAR parameter (default value: 256)
```

Setting User-Defined Hash Function

The CUBRID SHARD can hash the shard key by using the user-defined hash function, in addition to the default hash function.

Implementing and Creating a Library

The user-defined hash function must be implemented as a **.so** library loadable at runtime. Its prototype is as shown below:

```
/*
   return value :
        success - shard key id(>0)
        fail    - invalid argument(ERROR_ON_ARGUMENT),
shard key id make fail(ERROR_ON_MAKE_SHARD_KEY)
   type         : shard key value type
   val          : shard key value
*/
typedef int (*FN_GET_SHARD_KEY) (const char *shard_key,
T_SHARD_U_TYPE type,
                                 const void *val, int
val_size);
```

- The return value of the hash function should be within the range of the hash results of the **shard_key.txt** configuration file.
- To build a library, the **$CUBRID/include/shard_key.h** file of the CUBRID source must be included. The file lets you know the details such as error code that can be returned.

Changing the cubrid_broker.conf Configuration File

To apply a user-defined hash function, the **SHARD_KEY_LIBRARY_NAME** and **SHARD_KEY_FUNCTION_NAME** parameters of **cubrid_broker.conf** should be set according to the implementation.

- **SHARD_KEY_LIBRARY_NAME** : The (absolute) path of the user-defined hash library.
- **SHARD_KEY_FUNCTION_NAME** : The name of the user-defined hash function.

Example

The following example shows how to use a user-defined hash. First, check the **shard_key.txt** configuration file.

```
[%student_no]
#min    max     shard_id
0       31      0
32      63      1
64      95      2
```

```
96      127     3
128     159     0
160     191     1
192     223     2
224     255     3
```

To set the user-defined hash function, implement a **.so** shared library that is loadable at runtime. The result of the hash function should be within the range of hash function results defined in the **shard_key.txt** configuration file. The following example shows a simple implementation.

- When the shard_key is an integer
  - Select shard #0 when the shard_key is an odd number
  - Select shard #1 when the shard_key is an even number
- When the shard_key is a string
  - Select shard #0 when the shard_key string starts with 'a' or 'A'.
  - Select shard #1 when the shard_key string starts with 'b' or 'B'.
  - Select shard #2 when the shard_key string starts with 'c' or 'C'.
  - Select shard #3 when the shard_key string starts with 'd' or 'D'.

```
// <shard_key_udf.c>

#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include "shard_key.h"

int
fn_shard_key_udf (const char *shard_key, T_SHARD_U_TYPE
type,
                  const void *value, int value_len)
{
  unsigned int ival;
  unsigned char c;

  if (value == NULL)
    {
      return ERROR_ON_ARGUMENT;
    }

  switch (type)
    {
    case SHARD_U_TYPE_INT:
```

```
      ival = (unsigned int) (*(unsigned int *) value);
      if (ival % 2)
        {
          return 32;            // shard #1
        }
      else
        {
          return 0;             // shard #0
        }
      break;

    case SHARD_U_TYPE_STRING:
      c = (unsigned char) (((unsigned char *) value)[0]);
      switch (c)
        {
        case 'a':
        case 'A':
          return 0;             // shard #0
        case 'b':
        case 'B':
          return 32;            // shard #1
        case 'c':
        case 'C':
          return 64;            // shard #2
        case 'd':
        case 'D':
          return 96;            // shard #3
        default:
          return ERROR_ON_ARGUMENT;
        }

      break;

    default:
      return ERROR_ON_ARGUMENT;
    }
  return ERROR_ON_MAKE_SHARD_KEY;
}
```

Build the user-defined function as a shared library. The following example is Makefile for building a hash function.

```
# Makefile

CC = gcc
LIBS = $(LIB_FLAG)
CFLAGS = $(CFLAGS_COMMON) -fPIC -I$(CUBRID)/include -I$
(CUBRID_SRC)/src/broker

SHARD_CC = gcc -g -shared -Wl,-soname,shard_key_udf.so
```

```
SHARD_KEY_UDF_OBJS = shard_key_udf.o

all:$(SHARD_KEY_UDF_OBJS)
        $(SHARD_CC) $(CFLAGS) -o shard_key_udf.so $
(SHARD_KEY_UDF_OBJS) $(LIBS)

clean:
        -rm -f *.o core shard_key_udf.so
```

To include the user-defined hash function, modify the **SHARD_KEY_LIBRARY_NAME** and **SHARD_KEY_FUNCTION_NAME** parameters as shown in the above implementation.

```
[%student_no]
SHARD_KEY_LIBRARY_NAME =$CUBRID/conf/shard_key_udf.so
SHARD_KEY_FUNCTION_NAME =fn_shard_key_udf
```

Note

- When you define a user's hash function in the application, 16bit(short), 32bit(int) and 64bit(INT64) integer can be used as the value of shard key.
- The user should define a hash function when you have to use VARCHAR type.

Running and Monitoring

By using the CUBRID SHARD utility, CUBRID SHARD function can be started or stopped and various status information can be retrieved. For more details, see Broker.

Configuration Test

With **cubrid broker test** command, you can test if the configuration works normally. For details, see Broker to DB Connection Test.

CUBRID SHARD Log

There are four types of logs that relate to starting the shard: access, proxy, error and SQL logs. Changing the directory of each log is available through **LOG_DIR**, **ERROR_LOG_DIR**, and **SHARD_PROXY_LOG_DIR** parameters of the shard configuration file (**cubrid_broker.conf**).

SHARD PROXY Log

Access Log

- Parameter: **ACCESS_LOG**
- Description: Log the client access (the existing broker logs at the CAS).
- Default directory: $CUBRID/log/broker/
- Filename: <broker_name>_<proxy_index>.access
- Log type: All strings, except the access log and the cas_index at the CAS, are identical

```
10.24.18.67 - - 1340243427.828 1340243427.828 2012/06/21 10:50:27 ~
2012/06/21 10:50:27 23377 - -1 shard1     shard1
10.24.18.67 - - 1340243427.858 1340243427.858 2012/06/21 10:50:27 ~
2012/06/21 10:50:27 23377 - -1 shard1     shard1
10.24.18.67 - - 1340243446.791 1340243446.791 2012/06/21 10:50:46 ~
2012/06/21 10:50:46 23377 - -1 shard1     shard1
10.24.18.67 - - 1340243446.821 1340243446.821 2012/06/21 10:50:46 ~
2012/06/21 10:50:46 23377 - -1 shard1     shard1
```

Proxy Log

- Parameter: **SHARD_PROXY_LOG_DIR**
- Description: Log the behavior of the inner proxy.
- Default directory: $CUBRID/log/broker/proxy_log
- Filename: <broker_name>_<proxy_index>log

```
06/21 10:50:46.822 [SRD] ../../src/broker/shard_proxy_io.c(1045):
New socket io created. (fd:50).
06/21 10:50:46.822 [SRD] ../../src/broker/shard_proxy_io.c(2517):
New client connected. client(client_id:3, is_busy:Y, fd:50, ctx_cid:
3, ctx_uid:4).
06/21 10:50:46.825 [DBG] ../../src/broker/shard_proxy_io.c(3298):
Shard status. (num_cas_in_tran=1, shard_id=2).
```

```
06/21 10:50:46.827 [DBG] ../../src/broker/shard_proxy_io.c(3385):
Shard status. (num_cas_in_tran=0, shard_id=2).
```

Proxy Log Level

- Parameter: **SHARD_PROXY_LOG**
- Proxy log level policy: When the upper level is set, all logs of the lower level will be left.
  - Ex) Set SCHEDULE and then all ERROR | TIMEOUT | NOTICE | SHARD | SCHEDULE logs will be left.
- Proxy Log Level Item
  - NONE or OFF: No log is left.
  - ERROR (default): An internal error occurs and logging is not successfully processed
  - TIMEOUT: Timeout such as session timeout or query timeout
  - NOTICE: When the error is not a query without hint or other errors
  - SHARD: Scheduling that shows which shard and which CAS the client request have sent to and whether the request has responded to the client or not
  - SCHEDULE: Shard processing such as getting the shard key ID through parsing the hit or hashing
  - ALL: All logs

SHARD CAS Log

SQL Log

- Parameter: **SQL_LOG**
- Description: Log queries such as prepare/execute/fetch and other cas information.
- Default directory: $CUBRID/log/broker/sql_log
- Filename: %broker_name%_%proxy_index%_%shard_index%_%as_index%.sql.log

```
13-06-21 10:13:00.005 (0) STATE idle
13-06-21 10:13:01.035 (0) CAS TERMINATED pid 31595
13-06-21 10:14:20.198 (0) CAS STARTED pid 23378
13-06-21 10:14:21.227 (0) connect db shard1@HostA user dba url
shard1 session id 3
13-06-21 10:14:21.227 (0) DEFAULT isolation_level 3, lock_timeout -1
13-06-21 10:50:28.259 (1) prepare srv_h_id 1
```

```
13-06-21 10:50:28.259 (0) auto_rollback
13-06-21 10:50:28.259 (0) auto_rollback 0
```

Error log

- Parameter: **ERROR_LOG_DIR**
- Description: For CUBRID, the cs library logs EID and error strings to the corresponding file.
- Default directory: $CUBRID/log/broker/error_log
- Filename: %broker_name%_%proxy_index%_%shard_index%_%cas_index%.err

```
Time: 06/21/12 10:50:27.776 - DEBUG *** file ../../src/transaction/
boot_cl.c, line 1409
trying to connect 'shard1@localhost'
Time: 06/21/12 10:50:27.776 - DEBUG *** file ../../src/transaction/
boot_cl.c, line 1418
ping server with handshake
Time: 06/21/12 10:50:27.777 - DEBUG *** file ../../src/transaction/
boot_cl.c, line 966
boot_restart_client: register client { type 4 db shard1 user dba
password (null) program cubrid_cub_cas_1 login cubrid_user host
HostA pid 23270 }
```

Constraints

Linux only support

CUBRID SHARD feature can be used only in Linux.

One transaction can be run on only one shard DB

One transaction should be performed within only one shard DB, so the following constraints exist.

- It is unavailable to change data in several shard DBs through changing the shard key (**UPDATE**). If necessary, use **DELETE** / **INSERT**.
- A query about more than one shard DB, such as join, subquery, or, union, group by, between, like, in, exist, or any/some/all, is not supported.

Session information is valid only in each shard DB

Session information is valid within each shard DB only. Therefore, the results from session-related functions such as `LAST_INSERT_ID()` may be different from the intended result.

No support SET NAMES statement

In SHARD environment, SET NAMES statement is not recommended to use because it can work abnormally.

auto increment is valid only in each shard DB

The auto increment attribute or SERIAL is valid within each shard DB only. So a result different from the intended result may be returned.

DDL syntax with SHARD hint syntax not supported

Since DDL phrases such as schema creation and change in the SHARD configuration environment do not support SHARD hints, each SHARD DB must be accessed to process schema creation and change. The shard_id (0) is normally processed, and an error is generated from the shard_id (1).

Error example)

```
CREATE TABLE foo (col1 INT NOT NULL) /*+ SHARD_ID(1) /
```

```
DROP TABLE IF EXISTS foo /+ SHARD_ID(1) */
```

API Reference

This chapter covers the following API:

JDBC Driver

JDBC Overview

CUBRID JDBC driver (**cubrid_jdbc.jar**) implements an interface to enable access from applications in Java to CUBRID database server. CUBRID JDBC driver is installed in the <*directory where CUBRID is installed*>/**jdbc** directory. The driver has been developed based on the JDBC 2.0 specification and the default driver provided is complied with JDK 1.8.

Verifying CUBRID JDBC Driver Version

You can verify the version of JDBC driver as follows:

```
% jar -tf cubrid_jdbc.jar
META-INF/
```

```
META-INF/MANIFEST.MF
cubrid/
cubrid/jdbc/
cubrid/jdbc/driver/
cubrid/jdbc/jci/
cubrid/sql/
cubrid/jdbc/driver/CUBRIDBlob.class
...
CUBRID-JDBC-11.3.0.0047
```

Registering CUBRID JDBC Driver

Use the **Class.forName** (*driver-class-name*) method to register CUBRID JDBC driver. The following example shows how to load the **cubrid.jdbc.driver.CUBRIDDriver** class to register CUBRID JDBC driver.

```
import java.sql.*;
import cubrid.jdbc.driver.*;

public class LoadDriver {
   public static void main(String[] Args) {
       try {
           Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
       } catch (Exception e) {
           System.err.println("Unable to load driver.");
           e.printStackTrace();
       }
       ...
```

Installing and Configuring JDBC

Requirements

- JDK 1.8 or later
- CUBRID 2008 R2.0 (8.2.0) or later
- CUBRID JDBC driver 2008 R1.0 or later

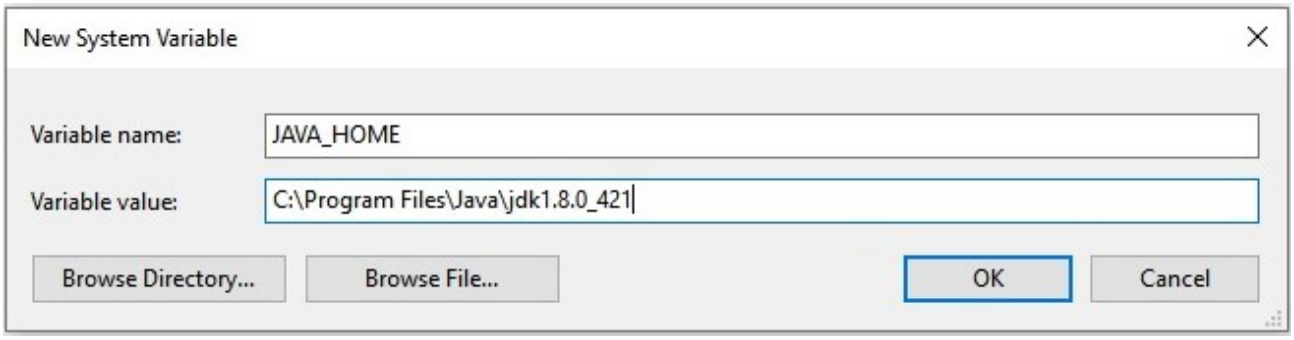
Installing Java and Configuring Environment

You must already have Java installed and the **JAVA_HOME** environment variable configured in your system. You can download Java from the Developer Resources for Java Technology website (https://www.oracle.com/java/technologies/downloads/#java8).

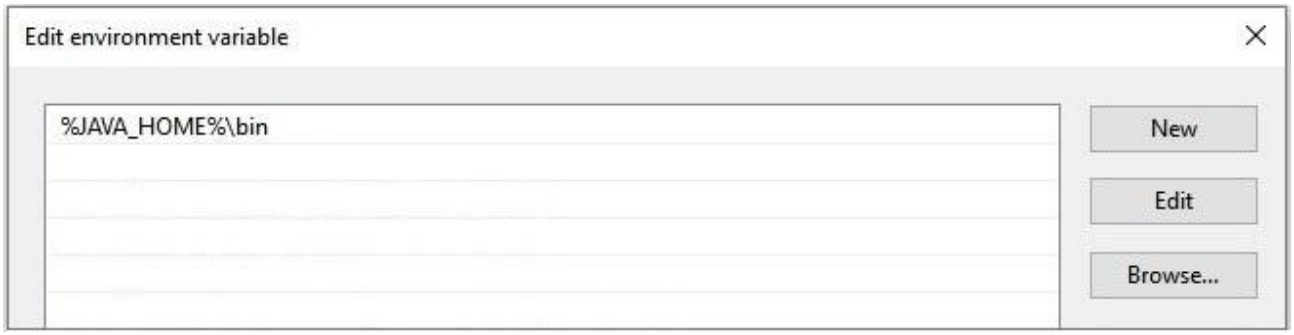
Configuring the environment variables for Windows

After installing Java, right-click on [This PC] and select [Properties]. Then, click on [Advanced system settings] under the Related settings to open the System Properties dialog box. Click on [Environment Variables] in the [Advanced] tab to open the Environment Variables dialog box.

In the [System variables], click [New]. Enter **JAVA_HOME** as the variable name and input the Java installation path (e.g., C:Program FilesJavajdk1.8.0_421) as the variable value, the click [OK].



Of system variables, select Path and then click [Edit]. Add **%JAVA_HOME%\bin** in the [Variable value] and then click [OK].



You can also configure **JAVA_HOME** and **PATH** values in the shell instead of using the way described above.

```
set JAVA_HOME= C:\Program Files\Java\jdk1.8.0_421
set PATH=%PATH%;%JAVA_HOME%\bin
```

Configuring the environment variables for Linux

Specify the directory path where Java is installed (example: /usr/java/jdk1.8.0_4211) as a **JAVA_HOME** environment variable and add **$JAVA_HOME/bin** to the **PATH** environment variable.

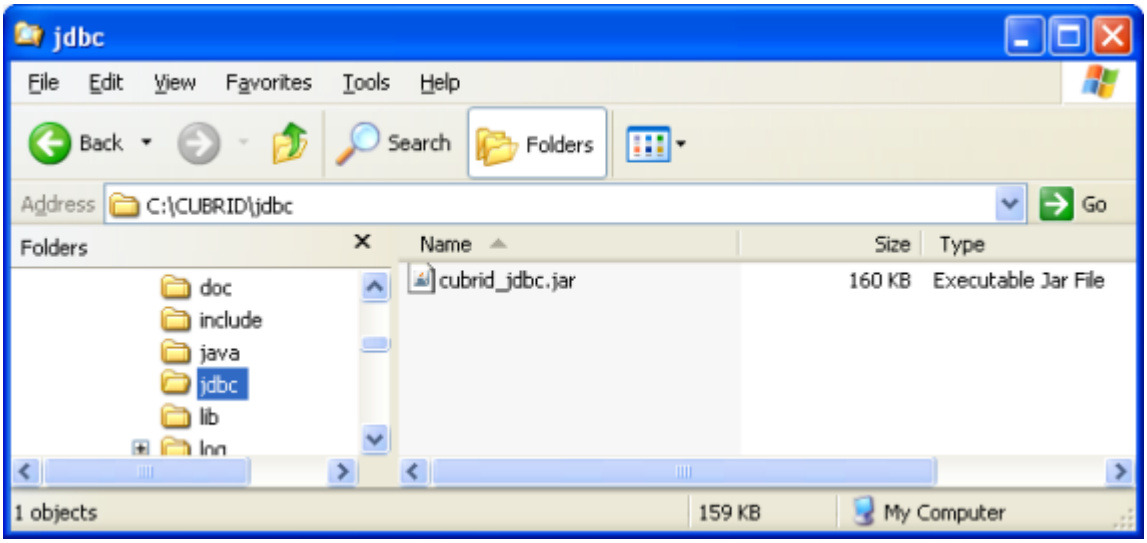
```
export JAVA_HOME=/usr/java/jdk1.8.0_421       #bash
export PATH=$JAVA_HOME/bin:$PATH            #bash

setenv JAVA_HOME /usr/java/jdk1.8.0_421       #csh
set path = ($JAVA_HOME/bin $path)           #csh
```

Configuring JDBC Driver

To use JDBC, you should specify the path where the CUBRID JDBC driver is located in the **CLASSPATH** environment variable.

The CUBRID JDBC driver (**cubrid_jdbc.jar**) is located in the jdbc directory under the directory of CUBRID installation.



Configuring the CLASSPATH environment variable for Windows

```
set CLASSPATH=C:\CUBRID\jdbc\cubrid_jdbc.jar:.
```

Configuring the CLASSPATH environment variable for Linux

```
export CLASSPATH=$HOME/CUBRID/jdbc/cubrid_jdbc.jar:.
```

Warning

If a general CUBRID JDBC driver has been installed in the same library directory (**$JAVA_HOME/jre/lib/ext**) where the JRE is installed, it may be loaded ahead of the server-side JDBC driver used by the Java stored procedure, which causing it to malfunction. In a Java stored procedure environment, make sure not to install a general CUBRID JDBC driver in the directory where the JRE is installed (**$JAVA_HOME/jre/lib/ext**).

JDBC Programming

Configuration Connection

The **DriverManager** is an interface for managing the JDBC driver. It is used to select a driver and create new database connection. If CUBRID JDBC driver is registered, you can connect a database by calling the **DriverManager.getConnection** (*db-url*, *user-id*, *password*) method.

The **getConnection** method returns the **Connection** object and it is used to execute queries and commands, and commit and roll back transactions. The syntax below shows the *db-url* argument for configuring connection.

```
jdbc:cubrid:<host>:<port>:<db-name>:[user-id]:[password]:[?
<property> [& <property>]]

<host> ::=
hostname | ip_address

<property> ::= altHosts=<alternative_hosts>
             | rcTime=<second>
             | loadBalance=<balance_mode>
             | connectTimeout=<second>
             | queryTimeout=<second>
             | charSet=<character_set>
             | zeroDateTimeBehavior=<behavior_type>
             | logFile=<file_name>
             | logOnException=<bool_type>
             |
logSlowQueries=<bool_type>&slowQueryThresholdMillis=<millisecond>
             | useLazyConnection=<bool_type>
             | useSSL=<bool_type>
             | clientCacheSize=<unit_size>
             | usePreparedStmtCache=<bool_type>
             | preparedStmtCacheSize=<unit_size>
             | preparedStmtCacheSqlLimit=<unit_size>
             | hold_cursor=<bool_type>

    <alternative_hosts> ::=
    <standby_broker1_host>:<port> [,<standby_broker2_host>:<port>]
    <behavior_type> ::= exception | round | convertToNull
    <bool_type> ::= true | false
    <unit_size> ::= multiple of mega byte
    <balance_mode> ::= true | false | rr | sh
```

- *host*: IP address or hostname where the CUBRID broker is running
- *port*: The CUBRID broker port number (default value: 33,000)
- *db-name*: The name of the database to connect

- *user-id*: The user ID which is connected to a database. There are two types of users in a database by default: **dba** and **public**. If this is NULL, it becomes <*db_user*> in *db-url*. If this is an empty string (“”), it becomes a **public** user.

- *password*: The password of a user who is to be connected to a database. If this is NULL, <*db_password*> in *db-url* is used. If this is an empty string (“”), DB password becomes an empty string. You cannot include ‘:’ in the password of the *db-url* string.

- <*property*>

  - **altHosts**: The host IP addresses and connection ports of one or more stand by brokers which will perform failover in the HA environment.

    Note

    Even if there are **RW** and **RO** together in *ACCESS_MODE** setting of brokers of main host and **altHosts**, application decides the target host to access without the relation for the setting of **ACCESS_MODE**. Therefore, you should define the main host and **altHosts** as considering **ACCESS_MODE** of target brokers.

  - **rcTime**: Interval time (in seconds) to try to connect active brokers during failover in the HA environment. See the below URL example.

  - **loadBalance**: When there are multiple hosts that a client can connect to, this is the property that selects the algorithm for determining the target host to connect to. Through load balancing, the client’s connection requests can be distributed so that they are not concentrated on a specific server. (default value: false)

    The behavior for each setting is as follows:

    - **false**: This property is not applied. * **true or rr**: Connects to the specified hosts using the **Round-Robin** method.

    - **sh**: Connects to the specified hosts using the **Random(Shuffle)** method.

  - **connectTimeout**: Timeout value (in seconds) for database connection. The default value is 30 seconds. If this value is 0, it means infinite waiting. This value is also applied when internal reconnection occurs after the initial connection. The **DriverManger.setLoginTimeout** () method can be used to configure it; however, the value configured in this method will be ignored if a value is configured in the connection URL.

  - **queryTimeout**: Timeout value (in seconds) for query execution (default value: 0, infinite). The maximum value is 2,000,000. This value can be changed by the **DriverManger.setQueryTimeout** () method.

Note

When you run executeBatch() method, the query timeout is applied in one method call, not in one query.

- **charSet**: The character set of a database to be connected
- **zeroDateTimeBehavior**: The property used to determine the way to handle an output value; because JDBC does not allow a value having zero for both date and time regardless of date and time in the object with the **java.sql.Date** type. For information about the value having zero for both date and date, see Date/Time Types.

  The default operation is **exception**. The operation for each configuration is as follows:

  - **exception**: Default operation. It is handled as an SQLException exception.
  - **round**: Converts to the minimum value allowed for a type to be returned. Exceptionally, when the value's type is TIMESTAMP, this value is rounded as '1970-01-01 00:00:00'(GST). (yyyy-mm-dd hh24:mi:ss)
  - **convertToNull**: Converts to **NULL**.
- **logFile**: The name of a log file for debugging (default value: cubrid_jdbc.log). If a path is not configured, it is stored the location where applications are running.
- **logOnException**: Whether to log exception handling for debugging (default value: false)
- **logSlowQueries**: Whether to log slow queries for debugging (default value: false)
  - **slowQueryThresholdMillis**: Timeout value (in milliseconds) of slow queries (default value: 60,000).
- **useLazyConnection**: If this is true, it returns success without connecting to the broker when user requests the connection, and it connects to the broker after calling prepare or execute function(default: false). If this value is true, it can prevent from access delay or failure as many application clients restart simultaniously and create connection pools.
- **useSSL**: Packet Encryption mode (Default: false)
  - Packet encryption: useSSL = true
  - Plain text: useSSL = false
- **clientCacheSize**: cache size for result-cache * the unit is multiple of mega-byte * the range is 1 ~ 1024 (1 mega-byte to 1 giga-byte) * the default value is 1

- **usePreparedStmtCache**: Prepared Statement Cache mode (Default: false)
- **preparedStmtCacheSize**: If usePreparedStmtCache is TRUE, the number of SQLs that can be cached (Default:25, Min:1, Max:2147483647)
- **preparedStmtCacheSqlLimit**: If usePreparedStmtCache is TRUE, length of SQL that can be cached (Default:256, Min:1, Max:2147483647)
- **hold_cursor**: Cursor holdability setting (default: true). If this value is false, CLOSE_CURSORS_AT_COMMIT is set, and if true, HOLD_CURSORS_OVER_COMMIT is set. For details, see Cursor Holdability.

Example 1

```
--connection URL string when user name and password omitted

URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::

--connection URL string when zeroDateTimeBehavior property specified
URL=jdbc:CUBRID:127.0.0.1:33000:demodb:public::?
zeroDateTimeBehavior=convertToNull

--connection URL string when charSet property specified

URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?charSet=utf-8

--connection URL string when queryTimeout and charSet property
specified

URL=jdbc:CUBRID:127.0.0.1:33000:demodb:public::?
queryTimeout=1&charSet=utf-8

--connection URL string when a property(altHosts) specified for HA
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?
altHosts=192.168.0.2:33000,192.168.0.3:33000

--connection URL string when properties(altHosts,rcTime,
connectTimeout) specified for HA
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?
altHosts=192.168.0.2:33000,192.168.0.3:33000&rcTime=600&connectTimeou
t=5

--connection URL string when properties(altHosts,rcTime, charSet)
specified for HA
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?
altHosts=192.168.0.2:33000,192.168.0.3:33000&rcTime=600&charSet=utf-8

--connection URL string when useSSL property specified for encrypted
connection
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?useSSL=true
```

```
--connection URL string when clientCacheSize property specified for
result-cache
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?clientCacheSize=1

--connection URL string when usePreparedStmtCache property specified
for prepared stament cache
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?
usePreparedStmtCache=true&preparedStmtCacheSize=100&preparedStmtCache
SqlLimit=1024"

--connection URL string when hold_cursor property specified for
cursor holdability
URL=jdbc:CUBRID:192.168.0.1:33000:demodb:public::?hold_cursor=true
```

Example 2

```
String url = "jdbc:cubrid:192.168.0.1:33000:demodb:public::";
String userid = "";
String password = "";

try {
   Connection conn =
           DriverManager.getConnection(url,userid,password);
   // Do something with the Connection

   ...

   } catch (SQLException e) {
       System.out.println("SQLException:" + e.getMessage());
       System.out.println("SQLState: " + e.getSQLState());
   }
   ...
```

Note

- Because a colon (:) and a question mark are used as a separator in the URL string, it is not allowed to use them as parts of a password. To use them in a password, you must specify a username (*user-id*) and a password (*password*) as a separate argument in the **getConnection** method.
- The database connection in thread-based programming must be used independently each other.
- The rollback method requesting transaction rollback will be ended after a server completes the rollback job.

- In autocommit mode, the transaction is not committed if all results are not fetched after running the SELECT statement. Therefore, although in autocommit mode, you should end the transaction by executing COMMIT or ROLLBACK if some error occurs during fetching for the resultset.

⚠ Warning

- The **useSSL** flag must match with mode of the broker trying to connect. If the encryption mode is different from the server that trying to connect, that connection request will be rejected. Please refer the following cases that are not allowed.
    - useSSL=true, connection request will be rejected when the broker is in 'normal mode' (**cubrid_broker.conf**: SSL = OFF)
    - useSSL=false, connection request will be rejected when the broker is in 'encryption mode' (**cubrid_broker.conf**: SSL = ON)
- The **clientCacheSize** works only when **JDBC_CACHE** or **JDBC_CACHE_ONLY_HINT** of broker configure is **ON**.

Connecting with DataSource

DataSource is the concept to be introduced in JDBC 2.0 API extention; it supports connection pooling and distributed transaction. CUBRID supports only connection pooling and do not supports distributed transaction and JNDI.

CUBRIDDataSource is DataSource implemented in CUBRID.

Creating a DataSource Object

To create a DataSource object, call as follows.

```
CUBRIDDataSource ds = null;
ds = new CUBRIDDataSource();
```

Setting Connection Properties

Connection properties are used to configure connections between datasource and CUBRID DBMS. General properties are a DB name, a hostname, a port number, a username and a password.

To set or get the values of properties, use below methods which is implemented in cubrid.jdbc.driver.CUBRIDDataSource.

```
public PrintWriter getLogWriter();
public void setLogWriter(PrintWriter out);
public void setLoginTimeout(int seconds);
public int getLoginTimeout();
public String getDatabaseName();
public String getDatabaseName();
public String getDataSourceName();
public String getDescription();
public String getNetworkProtocol();
public String getPassword();
public int getPortNumber();
public int getPort();
public String getRoleName();
public String getServerName();
public String getUser();
public String getURL();
public String getUrl();
public void setDatabaseName(String dbName);
public void setDescription(String desc);
public void setNetworkProtocol(String netProtocol);
public void setPassword(String psswd);
public void setPortNumber(int p);
public void setPort(int p);
public void setRoleName(String rName);
public void setServerName(String svName);
public void setUser(String uName);
public void setUrl(String urlString);
public void setURL(String urlString);
```

Especially, use a setURL() method to set the property through a URL string. Regarding URL string, see Configuration Connection.

```
import cubrid.jdbc.driver.CUBRIDDataSource;
...
CUBRIDDataSource ds = null;
ds = new CUBRIDDataSource();
ds.setUrl("jdbc:cubrid:192.168.0.1:55300:demodb:::?
charset=utf8&logSlowQueries=true&slowQueryThresholdMillis=1000&logTra
ceApi=true&logTraceNetwork=true");
```

Call getConnection method to get a connection object from DataSource.

```
Connection connection = null;
connection = ds.getConnection("dba", "");
```

CUBRIDConnectionPoolDataSource is an object which connection-pool datasource is implemented in CUBRID; it includes the methods of the same names with the methods of CUBRIDDataSource.

For detail examples, see **Connecting to a DataSource Object** in JDBC Sample Program.

Checking SQL LOG

The connection URL information can be printed out by calling cubrid.jdbc.driver.CUBRIDConnection.toString() method.

```
e.g.) cubrid.jdbc.driver.CUBRIDConnection(CAS ID : 1, PROCESS ID :
22922)
```

You can find SQL log of that CAS easily by CAS ID which is printed out.

For more details, see Checking SQL Log.

Checking Foreign Key Information

You can check foreign key information by using **getImportedKeys**, **getExportedKeys**, and **getCrossReference** methods of the **DatabaseMetaData** interface. The usage and example of each method are as follows:

```
getImportedKeys(String catalog, String schema, String table)

getExportedKeys(String catalog, String schema, String table)

getCrossReference(String parentCatalog, String parentSchema, String
parentTable, String foreignCatalog, String foreignSchema, String
foreignTable)
```

- **getImportedKeys** method: Retrieves information of primary key columns which are referred by foreign key columns in a given table. The results are sorted by **PKTABLE_NAME** and **KEY_SEQ**.
- **getExportedKeys** method: Retrieves information of all foreign key columns which refer to primary key columns in a given table. The results are sorted by **FKTABLE_NAME** and **KEY_SEQ**.
- **getCrossReference** method: Retrieves information of primary key columns which are referred by foreign key columns in a given table. The results are sorted by **PKTABLE_NAME** and **KEY_SEQ**.

Return Value

When the methods above are called, the ResultSet consisting of 14 columns listed in the table below is returned.

Name	Type	Note
PKTABLE_CAT	String	null without exception
PKTABLE_SCHEM	String	null without exception
PKTABLE_NAME	String	The name of a primary key table
PKCOLUMN_NAME	String	The name of a primary key column
FKTABLE_CAT	String	null without exception
FKTABLE_SCHEM	String	null without exception
FKTABLE_NAME	String	The name of a foreign key table
FKCOLUMN_NAME	String	The name of a foreign key column
KEY_SEQ	short	Sequence of columns of foreign keys or primary keys (starting from 1)
UPDATE_RULE	short	The corresponding values to referring actions defined as to foreign keys when primary keys are updated. Cascade=0, Restrict=2, No action=3, Set null=4
DELETE_RULE	short	The corresponding value to referring actions defined as

Name	Type	Note
		to foreign keys when primary keys are deleted. Cascade=0, Restrict=2, No action=3, Set null=4
FK_NAME	String	Foreign key name
PK_NAME	String	Primary key name
DEFERRABILITY	short	6 without exception (DatabaseMetaData.importedKeyInitiallyImmediate)

Example

```
import java.sql.*;
import java.lang.*;
import cubrid.jdbc.driver.*;


public class Metadata_Sample
{
        public static String getRuleName(String rule) {
                int r = Integer.parseInt(rule);
                switch(r) {
                        case DatabaseMetaData.importedKeyCascade:
                                return "cascade (" + rule + ")";
                        case DatabaseMetaData.importedKeyRestrict:
                                return "restrict (" + rule + ")";
                        case DatabaseMetaData.importedKeySetNull:
                                return "set null (" + rule + ")";
                        case DatabaseMetaData.importedKeyNoAction:
                                return "no action (" + rule + ")";
                        case DatabaseMetaData.importedKeySetDefault:
                                return "set default (" + rule + ")";
                        default:
                                return "unknown (" + rule + ")";
                }
        }

        public static String getDeferrabilityName(String
deferrability) {
```

```
                int d = Integer.parseInt(deferrability);
                switch(d) {
                        case
DatabaseMetaData.importedKeyInitiallyDeferred:
                                return "initially deferred (" +
deferrability + ")";
                        case
DatabaseMetaData.importedKeyInitiallyImmediate:
                                return "initially immediate (" +
deferrability + ")";
                        case
DatabaseMetaData.importedKeyNotDeferrable:
                                return "not deferrable (" +
deferrability + ")";
                        default:
                                return "unknown (" + deferrability +
")";
                }
        }

        public static void printFkInfo(ResultSet rs) throws
Exception {
                while(rs.next()) {
                        System.out.println("\n<" +
rs.getString("FK_NAME") + ">");
                        System.out.println("01. PKTABLE_CAT   : " +
rs.getString("PKTABLE_CAT"));
                        System.out.println("02. PKTABLE_SCHEM : " +
rs.getString("PKTABLE_SCHEM"));
                        System.out.println("03. PKTABLE_NAME  : " +
rs.getString("PKTABLE_NAME"));
                        System.out.println("04. PKCOLUMN_NAME : " +
rs.getString("PKCOLUMN_NAME"));
                        System.out.println("05. FKTABLE_CAT   : " +
rs.getString("FKTABLE_CAT"));
                        System.out.println("06. FKTABLE_SCHEM : " +
rs.getString("FKTABLE_SCHEM"));
                        System.out.println("07. FKTABLE_NAME  : " +
rs.getString("FKTABLE_NAME"));
                        System.out.println("08. FKCOLUMN_NAME : " +
rs.getString("FKCOLUMN_NAME"));
                        System.out.println("09. KEY_SEQ       : " +
rs.getString("KEY_SEQ"));
                        System.out.println("10. UPDATE_RULE   : " +
getRuleName(rs.getString("UPDATE_RULE")));
                        System.out.println("11. DELETE_RULE   : " +
getRuleName(rs.getString("DELETE_RULE")));
                        System.out.println("12. FK_NAME       : " +
rs.getString("FK_NAME"));
                        System.out.println("13. PK_NAME       : " +
rs.getString("PK_NAME"));
                        System.out.println("14. DEFERRABILITY : " +
```

```
getDeferrabilityName(rs.getString("DEFERRABILITY")));
                }
        }
        public static void main (String args []) {
                Connection conn = null;
                Statement stmt;
                DatabaseMetaData dbmd;
                ResultSet rs;

                try {
                        conn =
(CUBRIDConnection)DriverManager.getConnection("jdbc:cubrid:localhost:
33000:demodb:::", "dba", "");

                        stmt = conn.createStatement();

                        dbmd = conn.getMetaData();

                        stmt.executeUpdate("drop table if exists
refer_1");
                        stmt.executeUpdate("drop table if exists
refer_2");
                        stmt.executeUpdate("drop table if exists
referred_1");
                        stmt.executeUpdate("drop table if exists
referred_2");

                        stmt.executeUpdate("create table referred_1
("
                                        + "pk1 int,"
                                        + "pk2 int,"
                                        + "primary key (pk1, pk2))");

                        stmt.executeUpdate("create table referred_2
("
                                        + "pk int primary key)");

                        stmt.executeUpdate("create table refer_1 ("
                                        + "pk int primary key,"
                                        + "fk1 int,"
                                        + "fk2 int,"
                                        + "fk3 int,"
                                        + "constraint
refer_1_fk1_2_fk foreign key (fk1, fk2) references referred_1(pk1,
pk2),"
                                        + "constraint refer_1_fk3_fk
foreign key (fk3) references referred_2(pk))");

                        stmt.executeUpdate("create table refer_2 ("
                                        + "pk int primary key,"
                                        + "fk1 int,"
                                        + "fk2 int,"
```

```
                                        + "fk3 int,"
                                        + "constraint
refer_2_fk1_2_fk foreign key (fk1, fk2) references referred_1(pk1,
pk2) "
                                        + "on update set null on
delete no action,"
                                        + "constraint refer_2_fk3_fk
foreign key (fk3) references referred_2(pk) "
                                        + "on update restrict on
delete cascade)");

                        System.out.println("\n==============
getImportedKeys() ==================");

                        rs = dbmd.getImportedKeys(null, null,
"refer_1");
                        printFkInfo(rs);
                        rs.close();

                        rs = dbmd.getImportedKeys(null, null,
"refer_2");
                        printFkInfo(rs);
                        rs.close();

                        System.out.println("\n===============
getExportedKeys() ==================");
                        rs = dbmd.getExportedKeys(null, null,
"referred_1");
                        printFkInfo(rs);
                        rs.close();

                        rs = dbmd.getExportedKeys(null, null,
"referred_2");
                        printFkInfo(rs);
                        rs.close();

                        System.out.println("\n===============
getCrossReference() ==================");
                        rs = dbmd.getCrossReference(null, null,
"referred_1", null, null, "refer_1");
                        printFkInfo(rs);
                        rs.close();

                        rs = dbmd.getCrossReference(null, null,
"referred_1", null, null, "refer_2");
                        printFkInfo(rs);
                        rs.close();

                        rs = dbmd.getCrossReference(null, null,
"referred_2", null, null, "refer_1");
                        printFkInfo(rs);
                        rs.close();
```

```
                        rs = dbmd.getCrossReference(null, null,
"referred_2", null, null, "refer_2");
                        printFkInfo(rs);
                        rs.close();

                        rs = dbmd.getCrossReference(null, null,
"refer_1", null, null, "refer_2");
                        printFkInfo(rs);
                        rs.close();
                } catch (Exception e) {
                    e.printStackTrace();
                }
        }
}
```

Using Object Identifiers (OIDs) and Collections

In addition to the methods defined in the JDBC specification, CUBRID JDBC driver provides methods that handle OIDs and collections (set, multiset, and sequence).

To use these methods, you must import **cubrid.sql.*;** as well as the CUBRID JDBC driver classes which are imported by default. Furthermore, you should convert to not the **ResultSet** class, which is provided by the standard JDBC API) but the **CUBRIDResultSet** class to get result.

```
import cubrid.jdbc.driver.* ;
import cubrid.sql.* ;
...

CUBRIDResultSet urs = (CUBRIDResultSet) stmt.executeQuery(
    "SELECT city FROM location");
```

⚠ Warning

If extended API is used, transactions won't be automatically committed even though **AUTOCOMMIT** is set to TRUE. Therefore, you must explicitly commit transactions for open connections. The extended API of CUBRID is method that handle OIDs, collections, etc.

Using OIDs

You must follow the rules below when using OIDs.

- To use **CUBRIDOID**, you must **import cubrid.sql.** * . (a)

- You can get OIDs by specifying the class name in the **SELECT** statement. It can also be used together with other properties. (b)
- **ResultSet** for queries must be received by **CUBRIDResultSet**. (c)
- The method to get OIDs from **CUBRIDResultSet** is **getOID** (). (d)
- You can get the value from OIDs by using the **getValues** () method and the result will be **ResultSet**. (e)
- You can substitute OID with a value by using the **setValues** () method. (f)
- When you use extended API, you must always execute **commit** () for connection. (g)

Example

```
import java.sql.*;
import cubrid.sql.*; //a
import cubrid.jdbc.driver.*;

/*
CREATE TABLE oid_test(
   id INTEGER,
   name VARCHAR(10),
   age INTEGER
);

INSERT INTO oid_test VALUES(1, 'Laura', 32);
INSERT INTO oid_test VALUES(2, 'Daniel', 39);
INSERT INTO oid_test VALUES(3, 'Stephen', 38);
*/

class OID_Sample
{
   public static void main (String args [])
   {
      // Making a connection
      String url= "jdbc:cubrid:localhost:33000:demodb:public::";
      String user = "dba";
      String passwd = "";

      // SQL statement to get OID values
      String sql = "SELECT oid_test from oid_test"; //b
      // columns of the table
      String[] attr = { "id", "name", "age" } ;


      // Declaring variables for Connection and Statement
      Connection con = null;
      Statement stmt = null;
      CUBRIDResultSet rs = null;
```

```
        ResultSetMetaData rsmd = null;

        try {
            Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
        } catch (ClassNotFoundException e) {
            throw new IllegalStateException("Unable to load Cubrid
driver", e);
        }

        try {
            con = DriverManager.getConnection(url, user, passwd);
            stmt = con.createStatement();
            rs = (CUBRIDResultSet)stmt.executeQuery(sql); //c
            rsmd = rs.getMetaData();

            // Printing columns
            int numOfColumn = rsmd.getColumnCount();
            for (int i = 1; i <= numOfColumn; i++ ) {
                String ColumnName = rsmd.getColumnName(i);
                String JdbcType = rsmd.getColumnTypeName(i);
                System.out.print(ColumnName );
                System.out.print("("+ JdbcType + ")");
                System.out.print(" | ");
            }
            System.out.print("\n");

            // Printing rows
            CUBRIDResultSet rsoid = null;
            int k = 1;

            while (rs.next()) {
                CUBRIDOID oid = rs.getOID(1); //d
                System.out.print("OID");
                System.out.print(" | ");
                rsoid = (CUBRIDResultSet)oid.getValues(attr); //e

                while (rsoid.next()) {
                    for( int j=1; j <= attr.length; j++ ) {
                        System.out.print(rsoid.getObject(j));
                        System.out.print(" | ");
                    }
                }
                System.out.print("\n");

                // New values of the first row
                Object[] value = { 4, "Yu-ri", 19 };
                if (k == 1) oid.setValues(attr, value); //f

                k = 0;
            }
            con.commit(); //g
```

```
        } catch(CUBRIDException e) {
            e.printStackTrace();

        } catch(SQLException ex) {
            ex.printStackTrace();

        } finally {
            if(rs != null) try { rs.close(); } catch(SQLException e) {}
            if(stmt != null) try { stmt.close(); } catch(SQLException
e) {}
            if(con != null) try { con.close(); } catch(SQLException e)
{}
        }
    }
}
```

Using Collections

The line "a" in the example 1 is where data of collection types (**SET**, **MULTISET**, and **LIST**) is fetched from **CUBRIDResultSet**. The results are returned as array format. Note that this can be used only when data types of all elements defined in the collection types are same.

Example 1

```
import java.sql.*;
import java.lang.*;
import cubrid.sql.*;
import cubrid.jdbc.driver.*;

// create class collection_test(
// settest set(integer),
// multisettest multiset(integer),
// listtest list(Integer)
// );
//

// insert into collection_test values({1,2,3},{1,2,3},{1,2,3});
// insert into collection_test values({2,3,4},{2,3,4},{2,3,4});
// insert into collection_test values({3,4,5},{3,4,5},{3,4,5});

class Collection_Sample
{
    public static void main (String args [])
    {
        String url= "jdbc:cubrid:127.0.0.1:33000:demodb:public::";
        String user = "";
        String passwd = "";
        String sql = "select settest,multisettest,listtest from
collection_test";
```

```
        try {
            Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
        } catch(Exception e){
            e.printStackTrace();
        }
        try {
            Connection con =
DriverManager.getConnection(url,user,passwd);
            Statement stmt = con.createStatement();
            CUBRIDResultSet rs = (CUBRIDResultSet)
stmt.executeQuery(sql);
            CUBRIDResultSetMetaData rsmd = (CUBRIDResultSetMetaData)
rs.getMetaData();
            int numbOfColumn = rsmd.getColumnCount();
            while (rs.next ()) {
                for (int j=1; j<=numbOfColumn; j++ ) {
                    Object[] reset = (Object[])
rs.getCollection(j); //a
                    for (int m=0 ; m < reset.length ; m++)
                        System.out.print(reset[m] +",");
                    System.out.print(" | ");
                }
                System.out.print("\n");
            }
            rs.close();
            stmt.close();
            con.close();
        } catch(SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Example 2

```
import java.sql.*;
import java.io.*;
import java.lang.*;
import cubrid.sql.*;
import cubrid.jdbc.driver.*;

// create class collection_test(
// settest set(integer),
// multisettest multiset(integer),
// listtest list(Integer)
// );
//
// insert into collection_test values({1,2,3},{1,2,3},{1,2,3});
// insert into collection_test values({2,3,4},{2,3,4},{2,3,4});
// insert into collection_test values({3,4,5},{3,4,5},{3,4,5});
```

```
class SetOP_Sample {
        public static void main(String args[]) {
                String url = "jdbc:cubrid:
127.0.0.1:33000:demodb:public::";
                String user = "";
                String passwd = "";
                String sql = "select collection_test from
collection_test";
                try {

Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
                } catch (Exception e) {
                        e.printStackTrace();
                }
                try {
                        CUBRIDConnection con = (CUBRIDConnection)
DriverManager.getConnection(url, user, passwd);
                        Statement stmt = con.createStatement();
                        CUBRIDResultSet rs = (CUBRIDResultSet)
stmt.executeQuery(sql);
                        while (rs.next()) {
                                CUBRIDOID oid = rs.getOID(1);
                                oid.addToSet("settest",
Integer.valueOf(10));
                                oid.addToSet("multisettest",
Integer.valueOf(20));
                                oid.addToSequence("listtest", 1,
Integer.valueOf(30));
                                oid.addToSequence("listtest", 100,
Integer.valueOf(100));
                                oid.putIntoSequence("listtest", 99,
Integer.valueOf(99));
                                oid.removeFromSet("settest",
Integer.valueOf(1));
                                oid.removeFromSet("multisettest",
Integer.valueOf(2));
                                oid.removeFromSequence("listtest",
99);
                                oid.removeFromSequence("listtest",
1);
                        }
                        con.commit();
                        rs.close();
                        stmt.close();
                        con.close();
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

Getting Auto Increment Column Value

Auto increment (**AUTO_INCREMENT**) is a column-related feature that increments the numeric value of each row. For more information, see Column Definition. It can be defined only for numeric domains (**SMALLINT**, **INTEGER**, **DECIMAL** (*p*, 0), and **NUMERIC** (*p*, 0)).

Auto increment is recognized as automatically created keys in the JDBC programs. To retrieve the key, you need to specify the time to insert a row from which the automatically created key value is to be retrieved. To perform it, you must set the flag by calling **Connection.prepareStatement** and **Statement.execute** methods. In this case, the command executed should be the **INSERT** statement or **INSERT** within **SELECT** statement. For other commands, the JDBC driver ignores the flag-setting parameter.

Steps

- Use one of the following to indicate whether or not to return keys created automatically. The following method forms are used for tables of the database server that supports the auto increment columns. Each method form can be applied only to a single-row **INSERT** statement.
  - Write the **PreparedStatement** object as shown below.

```
Connection.prepareStatement(sql statement,
Statement.RETURN_GENERATED_KEYS);
```

  - To insert a row by using the **Statement.execute** method, use the **Statement.execute** method as shown below.

```
Statement.execute(sql statement,
Statement.RETURN_GENERATED_KEYS);
```

- Get the **ResultSet** object containing automatically created key values by calling the **PreparedStatement.getGeneratedKeys** or **Statement.getGeneratedKeys** method. Note that the data type of the automatically created keys in **ResultSet** is **DECIMAL** regardless of the data type of the given domain.

Example

The following example shows how to create a table with auto increment, enter data into the table so that automatically created key values are entered into auto increment columns, and check whether the key values are successfully retrieved by using the **Statement.getGeneratedKeys** () method. Each step is explained in the comments for commands that correspond to the steps above.

```
import java.sql.*;
import java.math.*;
```

```
import cubrid.jdbc.driver.*;

Connection con;
Statement stmt;
ResultSet rs;
java.math.BigDecimal iDColVar;
...
stmt = con.createStatement();     // Create a Statement object

// Create table with identity column
stmt.executeUpdate(
    "CREATE TABLE EMP_PHONE (EMPNO CHAR(6), PHONENO CHAR(4), " +
    "IDENTCOL INTEGER AUTO_INCREMENT)");

stmt.execute(
    "INSERT INTO EMP_PHONE (EMPNO, PHONENO) " +
    "VALUES ('000010', '5555')",               // Insert a row
<Step 1>
    Statement.RETURN_GENERATED_KEYS);        // Indicate you want
automatically


rs = stmt.getGeneratedKeys();    // generated keys

// Retrieve the automatically  <Step 2>
// generated key value in a ResultSet.
// Only one row is returned.
// Create ResultSet for query
while (rs.next()) {
    java.math.BigDecimal idColVar = rs.getBigDecimal(1);
    // Get automatically generated key value
    System.out.println("automatically generated key value = " +
idColVar);
}

rs.close();                          // Close ResultSet
stmt.close();                        // Close Statement
```

Using BLOB/CLOB

The interface that handles **LOB** data in JDBC is implemented based on JDBC 4.0 specification. The constraints of the interface are as follows:

- Only sequential write is supported when creating **BLOB** or **CLOB** object. Writing to arbitrary locations is not supported.
- You cannot change **BLOB** or **CLOB** data by calling methods of **BLOB** or **CLOB** object fetched from **ResultSet**.
- **Blob.truncate**, **Clob.truncate**, **Blob.position**, and **Clob.position** methods are supported.

- You cannot bind the LOB data by calling **PreparedStatement.setAsciiStream**, **PreparedStatement.setBinaryStream**, and **PreparedStatement.setCharacterStream** methods for **BLOB/CLOB** type columns.

- To use **BLOB** / **CLOB** type in the environment where JDBC 4.0 is not supported such as JDK versions 1.5 or earlier, you must do explicit type conversion for the conn object to **CUBRIDConnection**. See example below.

```
//JDK 1.6 or higher

import java.sql.*;

Connection conn = DriverManager.getConnection(url, id, passwd);
Blob blob = conn.createBlob();


//JDK 1.5 or lower

import java.sql.*;
import cubrid.jdbc.driver.*;

Connection conn = DriverManager.getConnection(url, id, passwd);
Blob blob = ((CUBRIDConnection)conn).createBlob();
```

Storing LOB Data

You can bind the **LOB** type data in the following ways.

- Create **java.sql.Blob** or **java.sql.Clob** object, store file content in the object, use **setBlob** () or **setClob** () of **PreparedStatement** (example 1).

- Execute a query, get **java.sql.Blob** or **java.sql.Clob** object from the **ResultSet** object, and bind the object in **PreparedStatement** (example 2).

Example 1

```
Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
Connection conn = DriverManager.getConnection
("jdbc:cubrid:localhost:33000:image_db:user1:password1:", "", "");

PreparedStatement pstmt1 = conn.prepareStatement("INSERT INTO
doc(image_id, doc_id, image) VALUES (?,?,?)");
pstmt1.setString(1, "image-21");
pstmt1.setString(2, "doc-21");

//Creating an empty file in the file system
Blob bImage = conn.createBlob();
byte[] bArray = new byte[256];
```

```
...

//Inserting data into the external file. Position is start with 1.
bImage.setBytes(1, bArray);
//Appending data into the external file
bImage.setBytes(257, bArray);
...

pstmt1.setBlob(3, bImage);
pstmt1.executeUpdate();
...
```

Example 2

```
Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
Connection conn = DriverManager.getConnection
("jdbc:cubrid:localhost:33000:image_db:user1:password1:", "", "");
conn.setAutoCommit(false);

PreparedStatement pstmt1 = conn.prepareStatement("SELECT image FROM
doc WHERE image_id = ? ");
pstmt1.setString(1, "image-21");
ResultSet rs = pstmt1.executeQuery();

while (rs.next())
{
    Blob bImage = rs.getBlob(1);
    PreparedStatement pstmt2 = conn.prepareStatement("INSERT INTO
doc(image_id, doc_id, image) VALUES (?,?,?)");
    pstmt2.setString(1, "image-22")
    pstmt2.setString(2, "doc-22")
    pstmt2.setBlob(3, bImage);
    pstmt2.executeUpdate();
    pstmt2.close();
}

pstmt1.close();
conn.commit();
conn.setAutoCommit(true);
conn.close();
...
```

Getting LOB Data

You can get the **LOB** type data in the following ways.

- Get data directly from **ResultSet** by using **getBytes** () or **getString** () method (example 1).

- Get the java.sql.Blob or java.sql.Clob object from **ResultSet** by calling **getBlob** () or **getClob** () method and get data for this object by using the **getBytes** () or **getSubString** () method (example 2).

Example 1

```
Connection conn = DriverManager.getConnection
("jdbc:cubrid:localhost:33000:image_db:user1:password1:", "", "");

// Getting data directly from ResetSet
PrepareStatement pstmt1 = conn.prepareStatement("SELECT content FROM
doc_t WHERE doc_id = ? ");
pstmt1.setString(1, "doc-10");
ResultSet rs = pstmt1.executeQuery();

while (rs.next())
{
    String sContent = rs.getString(1);
    System.out.println("doc.content= "+sContent.);
}
```

Example 2

```
Connection conn = DriverManager.getConnection
("jdbc:cubrid:localhost:33000:image_db:user1:password1:", "", "");

// Getting BLOB data from ResultSet and getting data from the BLOB
object
PrepareStatement pstmt2 = conn.prepareStatement("SELECT image FROM
image_t WHERE image_id = ?");
pstmt2.setString(1,"image-20");
ResultSet rs = pstmt2.executeQuery();

while (rs.next())
{
    Blob bImage = rs.getBlob(1);
    Bytes[] bArray = bImange.getBytes(1, (int)bImage.length());
}
```

Note

If a string longer than defined max length is inserted (**INSERT**) or updated (**UPDATE**), the string will be truncated.

setBoolean

prepareStatement.setBoolean(1, true) will set

- 1 for numeric types
- '1' for string types

prepareStatement.setBooelan(1, false) will set

- 0 for numeric types
- '0' for string types

Note

Behavior of legacy versions

prepareStatement.setBoolean(1, true) set

- as 2008 R4.1, 9.0, 1 of BIT(1) type
- as 2008 R4.3, 2008 R4.4, 9.1, 9.2, 9.3, -128 of SHORT type

JDBC Error Codes and Error Messages

JDBC error codes which occur in SQLException are as follows.

- All error codes are negative.
- After SQLException, error number can be shown by SQLException.getErrorCode() and error message can be shown by SQLException.getMessage().
- If the value of error code is between -21001 and -21999, it is caused by CUBRID JDBC methods.
- If the value of error code is between -10000 and -10999, it is caused by CAS and transferred by JDBC methods. For CAS errors, see CAS Error.
- If the value of error code is between 0 and -9999, it is caused by database server. For database server errors, see Database Server Errors.

Error Number	Error Message
-21001	Index's Column is Not Object

Error Number	Error Message
-21002	Server error
-21003	Cannot communicate with the broker
-21004	Invalid cursor position
-21005	Type conversion error
-21006	Missing or invalid position of the bind variable provided
-21007	Attempt to execute the query when not all the parameters are binded
-21008	Internal Error: NULL value
-21009	Column index is out of range
-21010	Data is truncated because receive buffer is too small
-21011	Internal error: Illegal schema type
-21012	File access failed
-21013	Cannot connect to a broker
-21014	Unknown transaction isolation level
-21015	

Error Number	Error Message
	Internal error: The requested information is not available
-21016	The argument is invalid
-21017	Connection or Statement might be closed
-21018	Internal error: Invalid argument
-21019	Cannot communicate with the broker or received invalid packet
-21020	No More Result
-21021	This ResultSet do not include the OID
-21022	Command is not insert
-21023	Error
-21024	Request timed out
-21101	Attempt to operate on a closed Connection.
-21102	Attempt to access a closed Statement.
-21103	Attempt to access a closed PreparedStatement.
-21104	Attempt to access a closed ResultSet.

Error Number	Error Message
-21105	Not supported method
-21106	Unknown transaction isolation level.
-21107	invalid URL -
-21108	The database name should be given.
-21109	The query is not applicable to the executeQuery(). Use the executeUpdate() instead.
-21110	The query is not applicable to the executeUpdate(). Use the executeQuery() instead.
-21111	The length of the stream cannot be negative.
-21112	An IOException was caught during reading the inputstream.
-21113	Not supported method, because it is deprecated.
-21114	The object does not seem to be a number.
-21115	Missing or invalid position of the bind variable provided.
-21116	The column name is invalid.

Error Number	Error Message
-21117	Invalid cursor position.
-21118	Type conversion error.
-21119	Internal error: The number of attributes is different from the expected.
-21120	The argument is invalid.
-21121	The type of the column should be a collection type.
-21122	Attempt to operate on a closed DatabaseMetaData.
-21123	Attempt to call a method related to scrollability of non-scrollable ResultSet.
-21124	Attempt to call a method related to sensitivity of non-sensitive ResultSet.
-21125	Attempt to call a method related to updatability of non-updatable ResultSet.
-21126	Attempt to update a column which cannot be updated.
-21127	The query is not applicable to the executeInsert().
-21128	The argument row can not be zero.

Error Number	Error Message
-21129	Given InputStream object has no data.
-21130	Given Reader object has no data.
-21131	Insertion query failed.
-21132	Attempt to call a method related to scrollability of TYPE_FORWARD_ONLY Statement.
-21133	Authentication failure
-21134	Attempt to operate on a closed PooledConnection.
-21135	Attempt to operate on a closed XAConnection.
-21136	Illegal operation in a distributed transaction
-21137	Attempt to access a CUBRIDOID associated with a Connection which has been closed.
-21138	The table name is invalid.
-21139	Lob position to write is invalid.
-21140	Lob is not writable.
-21141	Request timed out.

JDBC Sample Program

The following sample shows how to connect to CUBRID by using the JDBC driver, and retrieve and insert data. To run the sample program, make sure that the database you are trying to connect to and the CUBRID broker are running. In the sample, you will use the *demodb* database that is automatically created during the installation.

Loading JDBC Driver

To connect to CUBRID, load the JDBC driver by using the **forName** () method of the **Class**. For more information, see JDBC Overview of the JDBC driver.

```
Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
```

Connecting to Database

After loading the JDBC driver, use the **getConnection** () method of the **DriverManager** to connect to the database. To create a **Connection** object, you must specify information such as the URL which indicates the location of a database, username, password, etc. For more information, see Configuration Connection.

```
String url = "jdbc:cubrid:localhost:33000:demodb:::";
String userid = "dba";
String password = "";

Connection conn = DriverManager.getConnection(url,userid,password);
```

To connect to a database, it is possible to use a DataSource object, too. If you want to include connection properties to a connection URL string, setURL method implemented in CUBRIDDataSource can be used.

```
import cubrid.jdbc.driver.CUBRIDDataSource;
...

ds = new CUBRIDDataSource();
ds.setURL("jdbc:cubrid:127.0.0.1:33000:demodb:::?
charset=utf8&logSlowQueries=true&slowQueryThresholdMillis=1000&logTra
ceApi=true&logTraceNetwork=true");
```

For details about CUBRIDDataSource, see Connecting with DataSource.

Connecting to a DataSource Object

The following is an example to execute SELECT statements in multiple threads; they connect to DB with the setURL of CUBRIDDataSource, which is a DataSource implemented in CUBRID. Codes are separated with DataSourceMT.java and DataSourceExample.java.

- DataSourceMT.java includes a main() function. After a CUBRIDDataSource object is created and a setURL method is called to connect to DB, multiple threads run DataSourceExample.test method.
- In DataSourceExample.java, DataSourceExample.test is implemented; it is run on the threads in DataSourceMT.java.

DataSourceMT.java

```
import cubrid.jdbc.driver.*;

public class DataSourceMT {
    static int num_thread = 20;

    public static void main(String[] args) {
        CUBRIDDataSource ds = null;
        thrCPDSMT thread[];

        ds = new CUBRIDDataSource();
        ds.setURL("jdbc:cubrid:127.0.0.1:33000:demodb:::?
charset=utf8&logSlowQueries=true&slowQueryThresholdMillis=1000&logTra
ceApi=true&logTraceNetwork=true");

        try {
            thread = new thrCPDSMT[num_thread];

            for (int i = 0; i < num_thread; i++) {
                Thread.sleep(1);
                thread[i] = new thrCPDSMT(i, ds);
                try {
                    Thread.sleep(1);
                    thread[i].start();
                } catch (Exception e) {
                }
            }

            for (int i = 0; i < num_thread; i++) {
                thread[i].join();
                System.err.println("join thread : " + i);
            }

        } catch (Exception e) {
            e.printStackTrace();
            System.exit(-1);
        }
    }
```

```
}

class thrCPDSMT extends Thread {
    CUBRIDDataSource thread_ds;
    int thread_id;

    thrCPDSMT(int tid, CUBRIDDataSource ds) {
        thread_id = tid;
        thread_ds = ds;
    }

    public void run() {
        try {
            DataSourceExample.test(thread_ds);
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(-1);
        }

    }
}
```

DataSourceExample.java

```
import java.sql.*;
import javax.sql.*;
import cubrid.jdbc.driver.*;

public class DataSourceExample {

    public static void printdata(ResultSet rs) throws SQLException {
        try {
            ResultSetMetaData rsmd = null;

            rsmd = rs.getMetaData();
            int numberofColumn = rsmd.getColumnCount();

            while (rs.next()) {
                for (int j = 1; j <= numberofColumn; j++)
                    System.out.print(rs.getString(j) + "   ");
                System.out.println("");
            }
        } catch (SQLException e) {
            System.out.println("SQLException : " + e.getMessage());
            throw e;
        }
    }

    public static void test(CUBRIDDataSource ds) throws Exception {
        Connection connection = null;
```

```
        Statement statement = null;
        ResultSet resultSet = null;

        for (int i = 1; i <= 20; i++) {
            try {
                connection = ds.getConnection("dba", "");
                statement = connection.createStatement();
                String SQL = "SELECT * FROM code";
                resultSet = statement.executeQuery(SQL);

                while (resultSet.next()) {
                    printdata(resultSet);
                }

                if (i % 5 == 0) {
                    System.gc();
                }
            } catch (Exception e) {
                e.printStackTrace();
            } finally {
                closeAll(resultSet, statement, connection);
            }
        }
    }

    public static void closeAll(ResultSet resultSet, Statement
statement,
            Connection connection) {
        if (resultSet != null) {
            try {
                resultSet.close();
            } catch (SQLException e) {
            }
        }
        if (statement != null) {
            try {
                statement.close();
            } catch (SQLException e) {
            }
        }
        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {
            }
        }
    }
}
```

Manipulating Database (Executing Queries and Processing ResultSet)

To send a query statement to the connected database and execute it, create **Statement**, **PrepardStatement**, and **CallableStatemen** objects. After the **Statement** object is created, execute the query statement by using **executeQuery** () or **executeUpdate** () method of the **Statement** object. You can use the **next** () method to process the next row from the **ResultSet** that has been returned as a result of executing the **executeQuery** () method.

Note

In the version of 2008 R4.x or before, if you execute commit after query execution, **ResultSet** will be automatically closed. Therefore, you must not use **ResultSet** after commit. Generally CUBRID is executed in autocommit mode; if you do not want for CUBRID being executed in autocommit mode, you should specify **conn.setAutocommit(false);** in the code.

From 9.1, Cursor holdability is supported; therefore, you can use **ResultSet** after commit.

Disconnecting from Database

You can disconnect from a database by executing the **close** () method for each object.

CREATE, INSERT

The following is an example which connects to *demodb*, creates a table, executes a query with prepared statement and rolls back the query.

```
import java.util.*;
import java.sql.*;

public class Basic {
   public static Connection connect() {
      Connection conn = null;
      try {
           Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
           conn = DriverManager.getConnection("jdbc:cubrid:localhost:
33000:demodb:::","dba","");
           conn.setAutoCommit (false) ;
      } catch ( Exception e ) {
           System.err.println("SQLException : " + e.getMessage());
      }
      return conn;
   }

   public static void printdata(ResultSet rs) {
      try {
          ResultSetMetaData rsmd = null;

          rsmd = rs.getMetaData();
```

```
            int numberofColumn = rsmd.getColumnCount();

            while (rs.next ()) {
                for(int j=1; j<=numberofColumn; j++ )
                    System.out.print(rs.getString(j) + "   " );
                System.out.println("");
            }
        } catch ( Exception e ) {
            System.err.println("SQLException : " + e.getMessage());
        }
    }

    public static void main(String[] args) throws Exception {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;
        PreparedStatement preStmt = null;

        try {
            conn = connect();

            stmt = conn.createStatement();
            stmt.executeUpdate("CREATE TABLE xoo ( a INT, b INT, c
CHAR(10))");

            preStmt = conn.prepareStatement("INSERT INTO xoo
VALUES(?,?,'100')");
            preStmt.setInt (1, 1) ;
            preStmt.setInt (2, 1*10) ;
            int rst = preStmt.executeUpdate () ;

            rs = stmt.executeQuery("select a,b,c from xoo" );

            printdata(rs);

            conn.rollback();
            stmt.close();
            conn.close();
        } catch ( Exception e ) {
            conn.rollback();
            System.err.println("SQLException : " + e.getMessage());
        } finally {
            if ( conn != null ) conn.close();
        }
    }
}
```

SELECT

The following example shows how to execute the **SELECT** statement by connecting to *demodb* which is automatically created when installing CUBRID.

```
import java.sql.*;

public class SelectData {
    public static void main(String[] args) throws Exception {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
            conn =
DriverManager.getConnection("jdbc:cubrid:localhost:
33000:demodb:::","dba","");

            String sql = "SELECT name, players FROM event";
            stmt = conn.createStatement();
            rs = stmt.executeQuery(sql);

            while(rs.next()) {
               String name = rs.getString("name");
               String players = rs.getString("players");
               System.out.println("name ==> " + name);
               System.out.println("Number of players==> " + players);

System.out.println("\n=========================================\n");
            }

            rs.close();
            stmt.close();
            conn.close();
        } catch ( SQLException e ) {
            System.err.println(e.getMessage());
        } catch ( Exception e ) {
            System.err.println(e.getMessage());
        } finally {
            if ( conn != null ) conn.close();
        }
    }
}
```

INSERT

The following example shows how to execute the **INSERT** statement by connecting to *demodb* which is automatically created when installing CUBRID. You can delete or update data the same way as you insert data so you can reuse the code below by simply modifying the query statement in the code.

```
import java.sql.*;
```

```
public class insertData {
   public static void main(String[] args) throws Exception {
       Connection conn = null;
       Statement stmt = null;

       try {
           Class.forName("cubrid.jdbc.driver.CUBRIDDriver");
           conn = DriverManager.getConnection("jdbc:cubrid:localhost:
33000:demodb:::","dba","");
           String sql = "insert into olympic(host_year, host_nation,
host_city, opening_date, closing_date) values (2008, 'China',
'Beijing', to_date('08-08-2008','mm-dd-yyyy'),
to_date('08-24-2008','mm-dd-yyyy'))";
           stmt = conn.createStatement();
           stmt.executeUpdate(sql);
           System.out.println("A row is inserted.");
           stmt.close();
       } catch ( SQLException e ) {
           System.err.println(e.getMessage());
       } catch ( Exception e ) {
           System.err.println(e.getMessage());
       } finally {
           if ( conn != null ) conn.close();
       }
   }
}
```

JDBC API

For details about JDBC API, see Java API Specification (https://docs.oracle.com/javase/7/docs/api/) and for details about Java, see Java SE Documentation (https://www.oracle.com/java/technologies/javase-documentation.html).

If cursor holdability is not configured, a cursor is maintained by default.

The following table shows the JDBC standard and extended interface supported by CUBRID. Note that some methods are not supported even though they are included in the JDBC 2.0 specification.

Supported JDBC Interface by CUBRID

JDBC Standard Interface	JDBC Extended Interface	Supported
java.sql.Blob		Supported
java.sql.CallableStatement		Supported

JDBC Standard Interface	JDBC Extended Interface	Supported
java.sql.Clob		Supported
java.sql.Connection		Supported
java.sql.DatabaseMetaData		Supported
java.sql.Driver		Supported
java.sql.PreparedStatement	java.sql.CUBRIDPreparedStatement	Supported
java.sql.ResultSet	java.sql.CUBRIDResultSet	Supported
java.sql.ResultSetMetaData	java.sql.CUBRIDResultSetMetaData	Supported
N/A	CUBRIDOID	Supported
java.sql.Statement	java.sql.CUBRIDStatement	The getGeneratedKeys() method of JDBC 3.0 is supported.
java.sql.DriverManager		Supported
Java.sql.SQLException	Java.sql.CUBRIDException	Supported
java.sql.Array		Not Supported
java.sql.ParameterMetaData		Not Supported

JDBC Standard Interface	JDBC Extended Interface	Supported
java.sql.Ref		Not Supported
java.sql.Savepoint		Not Supported
java.sql.SQLData		Not Supported
java.sql.SQLInput		Not Supported
java.sql.Struct		Not Supported

Note

- If cursor holdability is not specified, cursor is hold in default.
- From CUBRID 2008 R4.3 version, the behavior of batching the queries on the autocommit mode was changed. The methods that batch the queries are PreparedStatement.executeBatch and Statement.executeBatch. Until 2008 R4.1 version, these methods had committed the transaction after executing all queries on the array. From 2008 R4.3, they commit each query on the array.
- In autocommit mode off, if the general error occurs during executing one of the queries in the array on the method which does a batch processing of the queries, the query with an error is ignored and the next query is executed continuously. But if the deadlock occurs, the error occurs as rolling back the transaction.

CCI Driver

CCI Overview

CUBRID CCI (C Client Interface) driver implements an interface to enable access from C-based application to CUBRID database server through broker. It is also used as back-end infrastructure

for creating tools (PHP, ODBC, etc.) which use the CAS application servers. In this environment, the CUBRID broker sends queries received from applications to a database and transfers the result to applications.

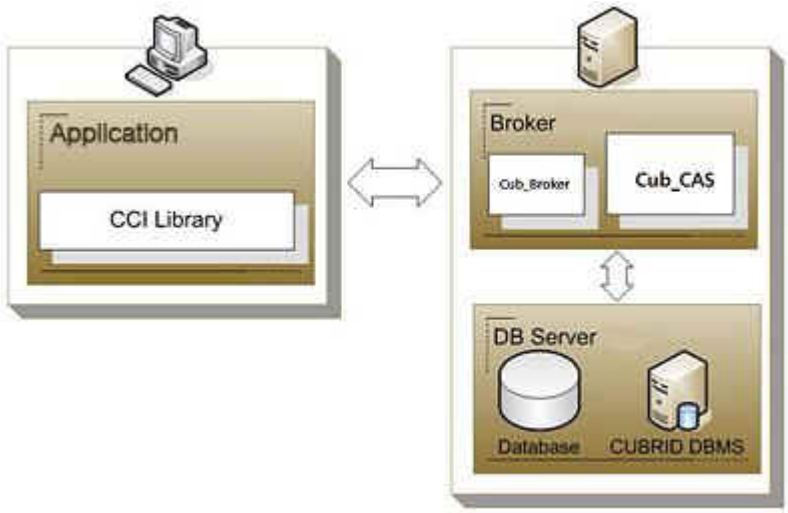
It is automatically installed upon CUBRID installation and can be found in the **$CUBRID/lib** directory. A header file as well as library files is required to use the driver.

	Windows	UNIX/Linux
C header file	cci/include/cas_cci.h	cci/include/cas_cci.h
Static library	cci/lib/cascci.lib	cci/lib/libcascci.a
Dynamic library	cci/bin/cascci.dll	cci/lib/libcascci.so

Note

- For Windows, Microsoft Visual C++ 2015 Redistributable (x86 or x64) must be installed to use the CCI Driver.
- From 11.2 Version, the path to the CCI driver has been changed. (cci/include, cci/lib, cci/bin)

Because CUBRID CCI driver is connected through the CUBRID broker, you can manage it the same way as other interfaces such as JDBC, PHP, ODBC, etc. In fact, CCI provides back-end infrastructure to implement PHP, ODBC, Python, and Ruby interfaces.



CCI Programming

Writing CCI Applications

The applications using CCI interact with CUBRID in the process of connecting to CAS, preparing queries, executing queries, handling response, and disconnecting. In each process, CCI communicates with applications through connection handle, query handle, and response handle.

The default value of autocommit mode can be configured by using CCI_DEFAULT_AUTOCOMMIT which is a broker parameter. If it is omitted, the default value is set to **ON**. To change autocommit mode within applications, you should use the `cci_set_autocommit()` function. If autocommit mode is **OFF**, you should explicitly commit or roll back transactions by using the `cci_end_tran()` function.

General process for writing applications is as follows. For using the prepared statement, additional step binding data to a variable is required; the examples 1 and 2 show the way to implement this.

- Opening a database connection handle (related functions: `cci_connect()`, `cci_connect_with_url()`)
- Getting the request handle for the prepared statement (related function: `cci_prepare()`)
- Binding data to the prepared statement (related function: `cci_bind_param()`)
- Executing the prepared statement (related function: `cci_execute()`)
- Processing the execution result (related functions: `cci_cursor()`, `cci_fetch()`, `cci_get_data()`, `cci_get_result_info()`)
- Closing the request handle (related function: `cci_close_req_handle()`)
- Closing the database connection handle (related function: `cci_disconnect()`)
- Using database connection pool (related functions: `cci_property_create()`, `cci_property_destroy()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_borrow()`, `cci_datasource_release()`, `cci_datasource_change_property()`)

Note

- If you want to compile the CCI application on Windows, “WINDOWS” should be defined. Therefore, “-DWINDOWS” option should be defined on the compiler.
- The database connection in thread-based programming must be used independently each other.
- In autocommit mode, the transaction is not committed if all results are not fetched after running the SELECT statement. Therefore, although in autocommit mode, you should end

the transaction by calling `cci_end_tran()` if some error occurs during fetching for the resultset.

Example 1

```
// Example to execute a simple query
// In Linux: gcc -o simple simple.c -m64 -I${CUBRID}/cci/include -
lnsl ${CUBRID}/cci/lib/libcascci.so -lpthread

#include <stdio.h>
#include "cas_cci.h"
#define BUFSIZE  (1024)

int
main (void)
{
    int con = 0, req = 0, col_count = 0, i, ind;
    int error;
    char *data;
    T_CCI_ERROR cci_error;
    T_CCI_COL_INFO *col_info;
    T_CCI_CUBRID_STMT stmt_type;
    char *query = "select * from code";

    //getting a connection handle for a connection with a server
    con = cci_connect ("localhost", 33000, "demodb", "dba", "");
    if (con < 0)
    {
        printf ("cannot connect to database\n");
        return 1;
    }

    //preparing the SQL statement
    req = cci_prepare (con, query, 0, &cci_error);
    if (req < 0)
    {
        printf ("prepare error: %d, %s\n", cci_error.err_code,
                cci_error.err_msg);
        goto handle_error;
    }

    //getting column information when the prepared statement is the
SELECT query
    col_info = cci_get_result_info (req, &stmt_type, &col_count);
    if (col_info == NULL)
    {
        printf ("get_result_info error: %d, %s\n",
cci_error.err_code,
                cci_error.err_msg);
```

```
        goto handle_error;
    }

    //Executing the prepared SQL statement
    error = cci_execute (req, 0, 0, &cci_error);
    if (error < 0)
    {
        printf ("execute error: %d, %s\n", cci_error.err_code,
                cci_error.err_msg);
        goto handle_error;
    }
    while (1)
    {

        //Moving the cursor to access a specific tuple of results
        error = cci_cursor (req, 1, CCI_CURSOR_CURRENT, &cci_error);
        if (error == CCI_ER_NO_MORE_DATA)
        {
            break;
        }
        if (error < 0)
        {
            printf ("cursor error: %d, %s\n", cci_error.err_code,
                    cci_error.err_msg);
            goto handle_error;
        }

        //Fetching the query result into a client buffer
        error = cci_fetch (req, &cci_error);
        if (error < 0)
        {
            printf ("fetch error: %d, %s\n", cci_error.err_code,
                    cci_error.err_msg);
            goto handle_error;
        }
        for (i = 1; i <= col_count; i++)
        {

            //Getting data from the fetched result
            error = cci_get_data (req, i, CCI_A_TYPE_STR, &data,
&ind);
            if (error < 0)
            {
                printf ("get_data error: %d, %d\n", error, i);
                goto handle_error;
            }
            printf ("%s\t|", data);
        }
        printf ("\n");
    }

    //Closing the request handle
```

```
    error = cci_close_req_handle (req);
    if (error < 0)
    {
        printf ("close_req_handle error: %d, %s\n",
cci_error.err_code,
                cci_error.err_msg);
        goto handle_error;
    }

    //Disconnecting with the server
    error = cci_disconnect (con, &cci_error);
    if (error < 0)
    {
        printf ("error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
        goto handle_error;
    }

    return 0;

handle_error:
    if (req > 0)
        cci_close_req_handle (req);
    if (con > 0)
    cci_disconnect (con, &cci_error);

    return 1;
}
```

Example 2

```
// Example to execute a query with a bind variable
// In Linux: gcc -o cci_bind cci_bind.c -m64 -I${CUBRID}/cci/include
-lnsl ${CUBRID}/cci/lib/libcascci.so -lpthread

#include <stdio.h>
#include <string.h>
#include "cas_cci.h"
#define BUFSIZE  (1024)

int
main (void)
{
    int con = 0, req = 0, col_count = 0, i, ind;
    int error;
    char *data;
    T_CCI_ERROR cci_error;
    T_CCI_COL_INFO *col_info;
    T_CCI_CUBRID_STMT stmt_type;
    char *query = "select * from nation where name = ?";
```

```
    char namebuf[128];

    //getting a connection handle for a connection with a server
    con = cci_connect ("localhost", 33000, "demodb", "dba", "");
    if (con < 0)
    {
        printf ("cannot connect to database\n");
        return 1;
    }

    //preparing the SQL statement
    req = cci_prepare (con, query, 0, &cci_error);
    if (req < 0)
    {
        printf ("prepare error: %d, %s\n", cci_error.err_code,
              cci_error.err_msg);
        goto handle_error;
    }

    //Binding date into a value
    strcpy (namebuf, "Korea");
    error =
    cci_bind_param (req, 1, CCI_A_TYPE_STR, namebuf,
CCI_U_TYPE_STRING,
                    CCI_BIND_PTR);
    if (error < 0)
    {
        printf ("bind_param error: %d ", error);
        goto handle_error;
    }

    //getting column information when the prepared statement is the
SELECT query
    col_info = cci_get_result_info (req, &stmt_type, &col_count);
    if (col_info == NULL)
    {
        printf ("get_result_info error: %d, %s\n",
cci_error.err_code,
              cci_error.err_msg);
        goto handle_error;
    }

    //Executing the prepared SQL statement
    error = cci_execute (req, 0, 0, &cci_error);
    if (error < 0)
    {
        printf ("execute error: %d, %s\n", cci_error.err_code,
              cci_error.err_msg);
        goto handle_error;
    }

    //Executing the prepared SQL statement
```

```
    error = cci_execute (req, 0, 0, &cci_error);
    if (error < 0)
    {
        printf ("execute error: %d, %s\n", cci_error.err_code,
                cci_error.err_msg);
        goto handle_error;
    }

    while (1)
    {

        //Moving the cursor to access a specific tuple of results
        error = cci_cursor (req, 1, CCI_CURSOR_CURRENT, &cci_error);
        if (error == CCI_ER_NO_MORE_DATA)
        {
            break;
        }
        if (error < 0)
        {
            printf ("cursor error: %d, %s\n", cci_error.err_code,
                    cci_error.err_msg);
            goto handle_error;
        }

        //Fetching the query result into a client buffer
        error = cci_fetch (req, &cci_error);
        if (error < 0)
        {
            printf ("fetch error: %d, %s\n", cci_error.err_code,
                    cci_error.err_msg);
            goto handle_error;
        }
        for (i = 1; i <= col_count; i++)
        {

            //Getting data from the fetched result
            error = cci_get_data (req, i, CCI_A_TYPE_STR, &data,
&ind);
            if (error < 0)
            {
                printf ("get_data error: %d, %d\n", error, i);
                goto handle_error;
            }
            if (ind == -1)
            {
                printf ("NULL\t");
            }
            else
            {
                printf ("%s\t|", data);
            }
        }
```

```
            printf ("\n");
    }

    //Closing the request handle
    error = cci_close_req_handle (req);
    if (error < 0)
    {
        printf ("close_req_handle error: %d, %s\n",
cci_error.err_code,
                cci_error.err_msg);
        goto handle_error;
    }

    //Disconnecting with the server
    error = cci_disconnect (con, &cci_error);
    if (error < 0)
    {
        printf ("error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
        goto handle_error;
    }

    return 0;

handle_error:
    if (req > 0)
        cci_close_req_handle (req);
    if (con > 0)
        cci_disconnect (con, &cci_error);
    return 1;
}
```

Example 3

```
// Example to use connection/statement pool in CCI
// In Linux: gcc -o cci_pool cci_pool.c -m64 -I${CUBRID}/cci/include
-lnsl ${CUBRID}/cci/lib/libcascci.so -lpthread

#include <stdio.h>
#include "cas_cci.h"

int main ()
{
    T_CCI_PROPERTIES *ps = NULL;
    T_CCI_DATASOURCE *ds = NULL;
    T_CCI_ERROR err;
    T_CCI_CONN cons;
    int rc = 1, i;

    ps = cci_property_create ();
```

```
    if (ps == NULL)
    {
        fprintf (stderr, "Could not create T_CCI_PROPERTIES.\n");
        rc = 0;
        goto cci_pool_end;
    }

    cci_property_set (ps, "user", "dba");
    cci_property_set (ps, "url", "cci:cubrid:localhost:
33000:demodb:::");
    cci_property_set (ps, "pool_size", "10");
    cci_property_set (ps, "max_wait", "1200");
    cci_property_set (ps, "pool_prepared_statement", "true");
    cci_property_set (ps, "login_timeout", "300000");
    cci_property_set (ps, "query_timeout", "3000");

    ds = cci_datasource_create (ps, &err);
    if (ds == NULL)
    {
        fprintf (stderr, "Could not create T_CCI_DATASOURCE.\n");
        fprintf (stderr, "E[%d,%s]\n", err.err_code, err.err_msg);
        rc = 0;
        goto cci_pool_end;
    }

    for (i = 0; i < 3; i++)
    {
        cons = cci_datasource_borrow (ds, &err);
        if (cons < 0)
        {
            fprintf (stderr,
                    "Could not borrow a connection from the data
source.\n");
            fprintf (stderr, "E[%d,%s]\n", err.err_code,
err.err_msg);
            continue;
        }
        // put working code here.
        cci_work (cons);
        cci_datasource_release (ds, cons, &err);

    }

cci_pool_end:
  cci_property_destroy (ps);
  cci_datasource_destroy (ds);

  return 0;
}

// working code
int cci_work (T_CCI_CONN con)
```

```
{
    T_CCI_ERROR err;
    char sql[4096];
    int req, res, error, ind;
    int data;

    cci_set_autocommit (con, CCI_AUTOCOMMIT_TRUE);
    cci_set_lock_timeout (con, 100, &err);
    cci_set_isolation_level (con, TRAN_REP_CLASS_COMMIT_INSTANCE,
&err);

    error = 0;
    snprintf (sql, 4096, "SELECT host_year FROM record WHERE
athlete_code=11744");
    req = cci_prepare (con, sql, 0, &err);
    if (req < 0)
    {
        printf ("prepare error: %d, %s\n", err.err_code,
err.err_msg);
        return error;
    }

    res = cci_execute (req, 0, 0, &err);
    if (res < 0)
    {
        printf ("execute error: %d, %s\n", err.err_code,
err.err_msg);
        goto cci_work_end;
    }

    while (1)
    {
    error = cci_cursor (req, 1, CCI_CURSOR_CURRENT, &err);
    if (error == CCI_ER_NO_MORE_DATA)
    {
        break;
    }
    if (error < 0)
    {
        printf ("cursor error: %d, %s\n", err.err_code, err.err_msg);
        goto cci_work_end;
    }

    error = cci_fetch (req, &err);
    if (error < 0)
    {
        printf ("fetch error: %d, %s\n", err.err_code, err.err_msg);
        goto cci_work_end;
    }

    error = cci_get_data (req, 1, CCI_A_TYPE_INT, &data, &ind);
    if (error < 0)
```

```
    {
        printf ("get data error: %d\n", error);
        goto cci_work_end;
    }
    printf ("%d\n", data);
    }

    error = 1;
cci_work_end:
    cci_close_req_handle (req);
    return error;
}
```

Configuring Library

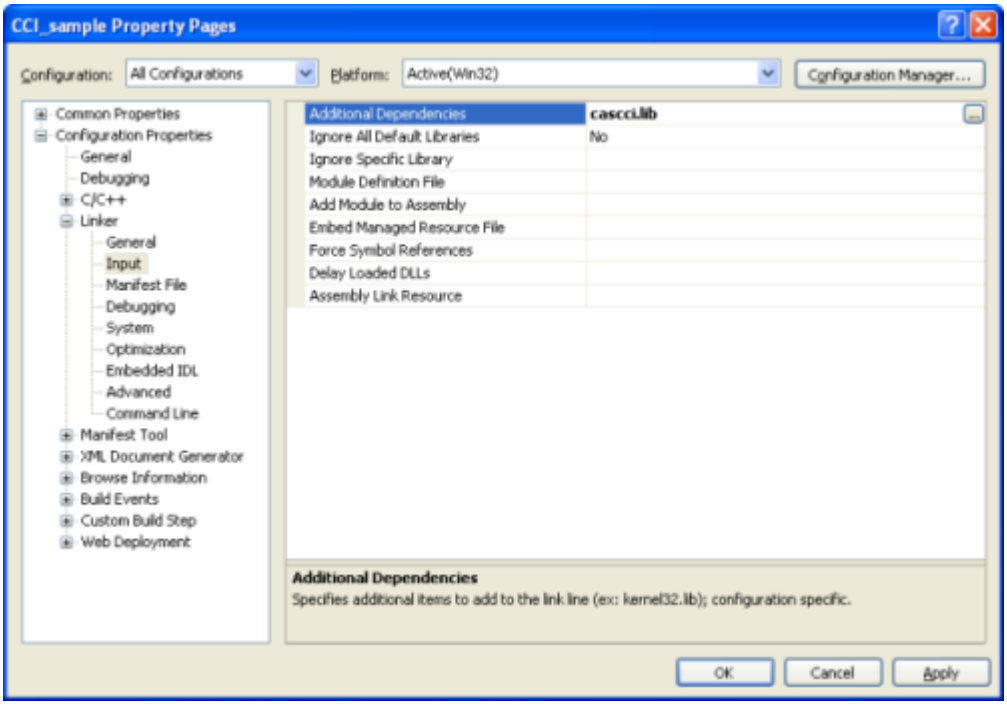
Once you have written applications using CCI, you should decide, according to its features, whether to execute CCI as static or dynamic link before you build it. See the table in CCI Overview to decide which library will be used.

The following is an example of Makefile, which makes a link by using the dynamic library on UNIX/Linux.

```
CC=gcc
CFLAGS = -g -Wall -I. -I$(CUBRID)/cci/include
LDFLAGS = -L$(CUBRID)/cci/lib -lcascci -lnsl

TEST_OBJS = test.o
EXES = test
all: $(EXES)
test: $(TEST_OBJS)
    $(CC) -o $@ $(TEST_OBJS) $(LDFLAGS)
```

The following image shows configuration to use static library on Windows.



Using BLOB/CLOB

Storing LOB Data

You can create **LOB** data file and bind the data by using the functions below in CCI applications.

- Creating **LOB** data files (related functions: `cci_blob_new()`, `cci_blob_write()`)
- Binding **LOB** data (related function: `cci_bind_param()`)
- Freeing memory for **LOB** struct (related function: `cci_blob_free()`)

Example

```
int con = 0; /* connection handle */
int req = 0; /* request handle */
int res;
int n_executed;
int i;
T_CCI_ERROR error;
T_CCI_BLOB blob = NULL;
char data[1024] = "bulabula";

con = cci_connect ("localhost", 33000, "tdb", "PUBLIC", "");
if (con < 0) {
    goto handle_error;
}
req = cci_prepare (con, "insert into doc (doc_id, content) values
(?,?)", 0, &error);
if (req< 0)
```

```
{
    goto handle_error;
}

res = cci_bind_param (req, 1 /* binding index*/, CCI_A_TYPE_STR,
"doc-10", CCI_U_TYPE_STRING, CCI_BIND_PTR);

/* Creating an empty LOB data file */
res = cci_blob_new (con, &blob, &error);
res = cci_blob_write (con, blob, 0 /* start position */, 1024 /*
length */, data, &error);

/* Binding BLOB data */
res = cci_bind_param (req, 2 /* binding index*/, CCI_A_TYPE_BLOB,
(void *)blob, CCI_U_TYPE_BLOB, CCI_BIND_PTR);

n_executed = cci_execute (req, 0, 0, &error);
if (n_executed < 0)
{
    goto handle_error;
}

/* Commit */
if (cci_end_tran(con, CCI_TRAN_COMMIT, &error) < 0)
{
    goto handle_error;
}

/* Memory free */
cci_blob_free(blob);
return 0;

handle_error:
if (blob != NULL)
{
    cci_blob_free(blob);
}
if (req > 0)
{
    cci_close_req_handle (req);
}
if (con > 0)
{
    cci_disconnect(con, &error);
}
return -1;
```

Retrieving LOB Data

You can retrieve **LOB** data by using the following functions in CCI applications. Note that if you enter data in the **LOB** type column, the actual **LOB** data is stored in the file located in external

storage and Locator value is stored in the **LOB** type column. Thus, to retrieve the **LOB** data stored in the file, you should call the `cci_blob_read()` function but the `cci_get_data()` function.

- Retrieving meta data (Locator) in the the **LOB** type column (related function: `cci_get_data()`)
- Retrieving the **LOB** data (related function: `cci_blob_read()`)
- Freeing memory for the **LOB** struct: (related function: `cci_blob_free()`)

Example

```
int con = 0; /* connection handle */
int req = 0; /* request handle */
int ind; /* NULL indicator, 0 if not NULL, -1 if NULL*/
int res;
int i;
T_CCI_ERROR error;
T_CCI_BLOB blob;
char buffer[1024];

con = cci_connect ("localhost", 33000, "image_db", "PUBLIC", "");
if (con < 0)
{
    goto handle_error;
}
req = cci_prepare (con, "select content from doc_t", 0 /*flag*/,
&error);
if (req< 0)
{
    goto handle_error;
}

res = cci_execute (req, 0/*flag*/, 0/*max_col_size*/, &error);

while (1) {
    res = cci_cursor (req, 1/* offset */, CCI_CURSOR_CURRENT/*
cursor position */, &error);
    if (res == CCI_ER_NO_MORE_DATA)
    {
        break;
    }
    res = cci_fetch (req, &error);

    /* Fetching CLOB Locator */
    res = cci_get_data (req, 1 /* colume index */, CCI_A_TYPE_BLOB,
    (void *)&blob /* BLOB handle */, &ind /* NULL indicator */);
    /* Fetching CLOB data */
    res = cci_blob_read (con, blob, 0 /* start position */, 1024 /*
length */, buffer, &error);
    printf ("content = %s\n", buffer);
}
```

```
/* Memory free */
cci_blob_free(blob);
res=cci_close_req_handle(req);
res = cci_disconnect (con, &error);
return 0;

handle_error:
if (req > 0)
{
    cci_close_req_handle (req);
}
if (con > 0)
{
    cci_disconnect(con, &error);
}
return -1;
```

CCI Error Codes and Error Messages

CCI API functions return a negative number as CCI or CAS (broker application server) error codes when an error occurs. The CCI error codes occur in CCI API functions and CAS error codes occur in CAS.

- All error codes are negative.
- All error codes and error messages of functions which have “T_CCI_ERROR err_buf” as a parameter can be found on err_buf.err_code and err_buf.err_msg.
- All error messages of functions which have no “T_CCI_ERROR err_buf” as a parameter can output by using `cci_get_err_msg()`.
- If the value of error code is between -20002 and -20999, it is caused by CCI API functions.
- If the value of error code is between -10000 and -10999, it is caused by CAS and transferred by CCI API functions. For CAS errors, see CAS Error.
- If the value of error code is **CCI_ER_DBMS** (-20001), it is caused by database server. You can check server error codes in err_buf.err_code of the database error buffer (err_buf). For database server errors, see Database Server Errors.

⚠ Warning

If an error occurs in server, the value of **CCI_ER_DBMS**, which is error code returned by a function may be different from the value of the err_buf.err_code. Except server errors, every error code stored in err_buf is identical to that returned by a function.

Note

CCI and CAS error codes have different values between the earlier version of CUBRID 9.0 and the version of CUBRID 9.0 or later. Therefore, the users who developed the applications by using the error code names must recompile them and the users who developed them by directly assigning error code numbers must recompile them after changing the number values.

The database error buffer (err_buf) is a struct variable of T_CCI_ERROR defined in the **cas_cci.h** header file. For how to use it, see the example below.

CCI error codes which starting with **CCI_ER** are defined in enum called **T_CCI_ERROR_CODE** under the **$CUBRID/include/cas_cci.h** file. Therefore, to use this error code name in program code, you should include a header file in the upper side of code by entering **#include "cas_cci.h"**.

The following example shows how to display error messages. In the example, the error code value (req) returned by `cci_prepare()` is **CCI_ER_DBMS**. -493 (server error code) is stored in **cci_error.err_code** and the error message, 'Syntax: Unknown class "notable". select * from notable' is stored in **cci_error.err_msg** of the database error buffer.

```
// gcc -o err err.c -m64 -I${CUBRID}/cci/include -lnsl ${CUBRID}/cci/
lib/libcascci.so -lpthread
#include <stdio.h>
#include "cas_cci.h"

#define BUFSIZE  (1024)

int
main (void)
{
    int con = 0, req = 0, col_count = 0, i, ind;
    int error;
    char *data;
    T_CCI_ERROR err_buf;
    char *query = "select * from notable";

    //getting a connection handle for a connection with a server
    con = cci_connect ("localhost", 33000, "demodb", "dba", "");
    if (con < 0)
    {
```

```
        printf ("cannot connect to database\n");
        return 1;
    }

    //preparing the SQL statement
    req = cci_prepare (con, query, 0, &err_buf);
    if (req < 0)
    {
        if (req == CCI_ER_DBMS)
        {
            printf ("error from server: %d, %s\n", err_buf.err_code,
err_buf.err_msg);
        }
        else
        {
            printf ("error from cci or cas: %d, %s\n",
err_buf.err_code, err_buf.err_msg);
        }
        goto handle_error;
    }
    // ...
}
```

The following list shows CCI error codes. For CAS errors, see CAS Error.

Error Code (Error Number)	Error Message	Note
CCI_ER_DBMS (-20001)	CUBRID DBMS Error	Error codes returned by functions when an error occurs in server. The causes of the error can be checked with err_code and err_msg stored in the T_CCI_ERROR struct.
CCI_ER_CON_HANDLE (-20002)	Invalid connection handle	
CCI_ER_NO_MORE_MEMORY (-20003)	Memory allocation error	Insufficient memory
CCI_ER_COMMUNICATION (-20004)	Cannot communicate with server	

Error Code (Error Number)	Error Message	Note
CCI_ER_NO_MORE_DATA (-20005)	Invalid cursor position	
CCI_ER_TRAN_TYPE (-20006)	Unknown transaction type	
CCI_ER_STRING_PARAM (-20007)	Invalid string argument	An error occurred when sql_stmt is NULL in `cci_prepare()`, and `cci_prepare_and_execute()`
CCI_ER_TYPE_CONVERSION (-20008)	Type conversion error	Cannot convert the given value into an actual data type.
CCI_ER_BIND_INDEX (-20009)	Parameter index is out of range	Index that binds data is not valid.
CCI_ER_ATYPE (-20010)	Invalid T_CCI_A_TYPE value	
CCI_ER_NOT_BIND (-20011)		Not available
CCI_ER_PARAM_NAME (-20012)	Invalid T_CCI_DB_PARAM value	
CCI_ER_COLUMN_INDEX (-20013)	Column index is out of range	
CCI_ER_SCHEMA_TYPE (-20014)		Not available
CCI_ER_FILE (-20015)	Cannot open file	Fails to open/read/write a file.

Error Code (Error Number)	Error Message	Note
CCI_ER_CONNECT (-20016)	Cannot connect to CUBRID CAS	Cannot connect to CUBRID CAS
CCI_ER_ALLOC_CON_HANDLE (-20017)	Cannot allocate connection handle %	
CCI_ER_REQ_HANDLE (-20018)	Cannot allocate request handle %	
CCI_ER_INVALID_CURSOR_POS (-20019)	Invalid cursor position	
CCI_ER_OBJECT (-20020)	Invalid oid string	
CCI_ER_CAS (-20021)		Not available
CCI_ER_HOSTNAME (-20022)	Unknown host name	
CCI_ER_OID_CMD (-20023)	Invalid T_CCI_OID_CMD value	
CCI_ER_BIND_ARRAY_SIZE (-20024)	Array binding size is not specified	
CCI_ER_ISOLATION_LEVEL (-20025)	Unknown transaction isolation level	
CCI_ER_SET_INDEX (-20026)	Invalid set index	Invalid index is specified when a set element in the T_CCI_SET struct is retrieved.

Error Code (Error Number)	Error Message	Note
CCI_ER_DELETED_TUPLE (-20027)	Current row was deleted %	
CCI_ER_SAVEPOINT_CMD (-20028)	Invalid T_CCI_SAVEPOINT_CMD value	Invalid T_CCI_SAVEPOINT_CMD value is used as an argument of the cci_savepoint() function.
CCI_ER_THREAD_RUNNING(-20029)	Invalid T_CCI_SAVEPOINT_CMD value	Invalid T_CCI_SAVEPOINT_CMD value is used as an argument of the cci_savepoint() function.
CCI_ER_INVALID_URL (-20030)	Invalid url string	
CCI_ER_INVALID_LOB_READ_POS (-20031)	Invalid lob read position	
CCI_ER_INVALID_LOB_HANDLE (-20032)	Invalid lob handle	
CCI_ER_NO_PROPERTY (-20033)	Could not find a property	
CCI_ER_PROPERTY_TYPE (-20034)	Invalid property type	
CCI_ER_INVALID_DATASOURCE (-20035)	Invalid CCI datasource	
	All connections are used	

Error Code (Error Number)	Error Message	Note
CCI_ER_DATASOURCE_TIMEOUT (-20036)		
CCI_ER_DATASOURCE_TIMEDWAIT (-20037)	pthread_cond_timedwait error	
CCI_ER_LOGIN_TIMEOUT (-20038)	Connection timed out	
CCI_ER_QUERY_TIMEOUT (-20039)	Request timed out	
CCI_ER_RESULT_SET_CLOSED (-20040)		
CCI_ER_INVALID_HOLDABILITY (-20041)	Invalid holdability mode. The only accepted values are 0 or 1	
CCI_ER_NOT_UPDATABLE (-20042)	Request handle is not updatable	
CCI_ER_INVALID_ARGS (-20043)	Invalid argument	
CCI_ER_USED_CONNECTION (-20044)	This connection is used already.	

C Type Definition

The following shows the structs used in CCI API functions.

Name	Type	Member	Description
T_CCI_ERROR	struct	char err_msg[1024]	Representation of database error info
		int err_code	
T_CCI_BIT	struct	int size	Representation of bit type
		char *buf	
T_CCI_DATE	struct	short yr	Representation of datetime, timestamp, date, and time type
		short mon	
		short day	
		short hh	
		short mm	
		short ss	
		short ms	
T_CCI_DATE_TZ	struct	short yr	Representation of date/time types with timezone
		short mon	
		short day	
		short hh	

Name	Type	Member	Description
		short mm	
		short ss	
		short ms	
		char tz[64]	
T_CCI_SET	void*		Representation of set type
T_CCI_COL_INFO	struct	**T_CCI_U_EXT_TYPE** type	Representation of column information for the **SELECT** statement
		char is_non_null	
		short scale	
		int precision	
		char *col_name	
		char *real_attr	
		char *class_name	
T_CCI_QUERY_RESULT	struct	int result_count	Results of batch execution
		int stmt_type	

Name	Type	Member	Description
		char *err_msg	
		char oid[32]	
T_CCI_PARAM_INFO	struct	**T_CCI_PARAM_MODE** mode	Representation of input parameter info
		T_CCI_U_EXT_TYPE type	
		short scale	
		int precision	
T_CCI_U_EXT_TYPE	unsigned char		Database type info
T_CCI_U_TYPE	enum	**CCI_U_TYPE_UNKNOWN**	Database type info
		CCI_U_TYPE_NULL	
		CCI_U_TYPE_CHAR	
		CCI_U_TYPE_STRING	
		CCI_U_TYPE_BIT	
		CCI_U_TYPE_VARBIT	
		CCI_U_TYPE_NUMERIC	

Name	Type	Member	Description
		CCI_U_TYPE_INT	
		CCI_U_TYPE_SHORT	
		CCI_U_TYPE_FLOAT	
		CCI_U_TYPE_DOUBLE	
		CCI_U_TYPE_DATE	
		CCI_U_TYPE_TIME	
		CCI_U_TYPE_TIMESTAMP	
		CCI_U_TYPE_SET	
		CCI_U_TYPE_MULTISET	
		CCI_U_TYPE_SEQUENCE	
		CCI_U_TYPE_OBJECT	
		CCI_U_TYPE_BIGINT	
		CCI_U_TYPE_DATETIME	
		CCI_U_TYPE_BLOB	

Name	Type	Member	Description
		CCI_U_TYPE_CLOB	
		CCI_U_TYPE_ENUM	
		CCI_U_TYPE_UINT	
		CCI_U_TYPE_USHORT	
		CCI_U_TYPE_UBIGINT	
		CCI_U_TYPE_TIMESTAMPTZ	
		CCI_U_TYPE_TIMESTAMPLTZ	
		CCI_U_TYPE_DATETIMETZ	
		CCI_U_TYPE_DATETIMELTZ	
T_CCI_A_TYPE	enum	**CCI_A_TYPE_STR**	Representation of type info used in API
		CCI_A_TYPE_INT	
		CCI_A_TYPE_FLOAT	
		CCI_A_TYPE_DOUBLE	

Name	Type	Member	Description
		CCI_A_TYPE_BIT	
		CCI_A_TYPE_DATE	
		CCI_A_TYPE_SET	
		CCI_A_TYPE_BIGINT	
		CCI_A_TYPE_BLOB	
		CCI_A_TYPE_CLOB	
		CCI_A_TYPE_CLOB	
		CCI_A_TYPE_REQ_HANDLE	
		CCI_A_TYPE_UINT	
		CCI_A_TYPE_UBIGINT	
		CCI_A_TYPE_DATE_TZ	
		CCI_A_TYPE_UINT	
T_CCI_DB_PARAM	enum	**CCI_PARAM_ISOLATION_LEVEL**	System parameter names
		CCI_PARAM_LOCK_TIMEOUT	

Name	Type	Member	Description
		CCI_PARAM_MAX_STRING_LENGTH	
		CCI_PARAM_AUTO_COMMIT	
T_CCI_SCH_TYPE	enum	CCI_SCH_CLASS	
		CCI_SCH_VCLASS	
		CCI_SCH_QUERY_SPEC	
		CCI_SCH_ATTRIBUTE	
		CCI_SCH_CLASS_ATTRIBUTE	
		CCI_SCH_METHOD	
		CCI_SCH_CLASS_METHOD	
		CCI_SCH_METHOD_FILE	
		CCI_SCH_SUPERCLASS	
		CCI_SCH_SUBCLASS	
		CCI_SCH_CONSTRAIT	

Name	Type	Member	Description
		CCI_SCH_TRIGGER	
		CCI_SCH_CLASS_PRIVILEGE	
		CCI_SCH_ATTR_PRIVILEGE	
		CCI_SCH_DIRECT_SUPER_CLASS	
		CCI_SCH_PRIMARY_KEY	
		CCI_SCH_IMPORTED_KEYS	
		CCI_SCH_EXPORTED_KEYS	
		CCI_SCH_CROSS_REFERENCE	
T_CCI_CUBRID_STMT	enum	CUBRID_STMT_ALTER_CLASS	
		CUBRID_STMT_ALTER_SERIAL	
		CUBRID_STMT_COMMIT_WORK	

Name	Type	Member	Description
		CUBRID_STMT_REGISTER_DATABASE	
		CUBRID_STMT_CREATE_CLASS	
		CUBRID_STMT_CREATE_INDEX	
		CUBRID_STMT_CREATE_TRIGGER	
		CUBRID_STMT_CREATE_SERIAL	
		CUBRID_STMT_DROP_DATABASE	
		CUBRID_STMT_DROP_CLASS	
		CUBRID_STMT_DROP_INDEX	
		CUBRID_STMT_DROP_LABEL	
		CUBRID_STMT_DROP_TRIGGER	
		CUBRID_STMT_DROP_SERIAL	

Name	Type	Member	Description
		CUBRID_STMT_EVALUATE	
		CUBRID_STMT_RENAME_CLASS	
		CUBRID_STMT_ROLLBACK_WORK	
		CUBRID_STMT_GRANT	
		CUBRID_STMT_REVOKE	
		CUBRID_STMT_STATISTICS	
		CUBRID_STMT_INSERT	
		CUBRID_STMT_SELECT	
		CUBRID_STMT_UPDATE	
		CUBRID_STMT_DELETE	
		CUBRID_STMT_CALL	
		CUBRID_STMT_GET_ISO_LVL	

Name	Type	Member	Description
		CUBRID_STMT_GET_TIMEOUT	
		CUBRID_STMT_GET_OPT_LVL	
		CUBRID_STMT_SET_OPT_LVL	
		CUBRID_STMT_SCOPE	
		CUBRID_STMT_GET_TRIGGER	
		CUBRID_STMT_SET_TRIGGER	
		CUBRID_STMT_SAVEPOINT	
		CUBRID_STMT_PREPARE	
		CUBRID_STMT_ATTACH	
		CUBRID_STMT_USE	
		CUBRID_STMT_REMOVE_TRIGGER	

Name	Type	Member	Description
		CUBRID_STMT_RENAME_TRIGGER	
		CUBRID_STMT_ON_LDB	
		CUBRID_STMT_GET_LDB	
		CUBRID_STMT_SET_LDB	
		CUBRID_STMT_GET_STATS	
		CUBRID_STMT_CREATE_USER	
		CUBRID_STMT_DROP_USER	
		CUBRID_STMT_ALTER_USER	
		CUBRID_STMT_SET_SYS_PARAMS	
		CUBRID_STMT_ALTER_INDEX	
		CUBRID_STMT_CREATE_STORED_PROCEDURE	

Name	Type	Member	Description
		CUBRID_STMT_DROP_STORED_PROCEDURE	
		CUBRID_STMT_PREPARE_STATEMENT	
		CUBRID_STMT_EXECUTE_PREPARE	
		CUBRID_STMT_DEALLOCATE_PREPARE	
		CUBRID_STMT_TRUNCATE	
		CUBRID_STMT_DO	
		CUBRID_STMT_SELECT_UPDATE	
		CUBRID_STMT_SET_SESSION_VARIABLES	
		CUBRID_STMT_DROP_SESSION_VARIABLES	
		CUBRID_STMT_MERGE	
		CUBRID_STMT_SET_NAMES	

Name	Type	Member	Description
		CUBRID_STMT_ALTER_STORED_PROCEDURE	
		CUBRID_STMT_KILL	
T_CCI_CURSOR_POS	enum	CCI_CURSOR_FIRST	
		CCI_CURSOR_CURRENT	
		CCI_CURSOR_LAST	
T_CCI_TRAN_ISOLATION	enum	TRAN_READ_COMMITTED	
		TRAN_REPEATABLE_READ	
		TRAN_SERIALIZABLE	
T_CCI_PARAM_MODE	enum	CCI_PARAM_MODE_UNKNOWN	
		CCI_PARAM_MODE_IN	
		CCI_PARAM_MODE_OUT	
		CCI_PARAM_MODE_INOUT	

Note

If a string longer than defined max length is inserted (**INSERT**) or updated (**UPDATE**), the string will be truncated.

CCI Sample Program

The sample program shows how to write a CCI application by using the *demodb* database which is included with the CUBRID installation package. You can practice the ways to connect to CAS, prepare queries, execute queries, handle response, disconnect from CAS, etc. by following sample program below. In the sample program, the dynamic link on Linux environment is used.

The code below shows information about *olympic* table schema in the *demodb* database which is used for sample program.

```
csql> ;sc olympic

=== <Help: Schema of a Class> ===


 <Class Name>

     olympic

 <Attributes>

     host_year             INTEGER NOT NULL
     host_nation           CHARACTER VARYING(40) NOT NULL
     host_city             CHARACTER VARYING(20) NOT NULL
     opening_date          DATE NOT NULL
     closing_date          DATE NOT NULL
     mascot                CHARACTER VARYING(20)
     slogan                CHARACTER VARYING(40)
     introduction          CHARACTER VARYING(1500)

 <Constraints>

     PRIMARY KEY pk_olympic_host_year ON olympic (host_year)
```

Preparing

Make sure that the *demodb* database and the broker are running before you execute the sample program. You can start the *demodb* database and the broker by executing the **cubrid** utility. The code below shows how to run a database server and broker by executing the **cubrid** utility.

```
[tester@testdb ~]$ cubrid server start demodb
@ cubrid master start
++ cubrid master start: success
@ cubrid server start: demodb

This may take a long time depending on the amount of recovery works
to do.

CUBRID 9.2

++ cubrid server start: success
[tester@testdb ~]$ cubrid broker start
@ cubrid broker start
++ cubrid broker start: success
```

Building

With the program source and the Makefile prepared, executing **make** will create an executable file named *test*. If you use a static library, there is no need to deploy additional files and the execution will be faster. However, it increases the program size and memory usage. If you use a dynamic library, there will be some performance overhead but the program size and memory usage can be optimized.

The code below a command line that makes a test program build by using a dynamic library instead of using **make** on Linux.

```
cc -o test test.c -I$CUBRID/cci/include -L$CUBRID/cci/lib -lnsl -
lcascci
```

Sample Code

```
#include <stdio.h>
#include <cas_cci.h>
char *cci_client_name = "test";
int main (int argc, char *argv[])
{
    int con = 0, req = 0, col_count = 0, res, ind, i;
    T_CCI_ERROR error;
    T_CCI_COL_INFO *res_col_info;
    T_CCI_CUBRID_STMT stmt_type;
    char *buffer, db_ver[16];
    printf("Program started!\n");
    if ((con=cci_connect("localhost", 30000, "demodb", "PUBLIC",
""))<0) {
        printf( "%s(%d): cci_connect fail\n", __FILE__, __LINE__);
        return -1;
    }
```

```
    if ((res=cci_get_db_version(con, db_ver, sizeof(db_ver)))<0) {
        printf( "%s(%d): cci_get_db_version fail\n", __FILE__,
__LINE__);
        goto handle_error;
    }
    printf("DB Version is %s\n",db_ver);
    if ((req=cci_prepare(con, "select * from event", 0,&error))<0) {
        if (req < 0) {
            printf( "%s(%d): cci_prepare fail(%d)\n", __FILE__,
__LINE__,error.err_code);
        }
        goto handle_error;
    }
    printf("Prepare ok!(%d)\n",req);
    res_col_info = cci_get_result_info(req, &stmt_type, &col_count);
    if (!res_col_info) {
        printf( "%s(%d): cci_get_result_info fail\n", __FILE__,
__LINE__);
        goto handle_error;
    }

    printf("Result column information\n"
           "=========================================\n");
    for (i=1; i<=col_count; i++) {
        printf("name:%s  type:%d(precision:%d scale:%d)\n",
            CCI_GET_RESULT_INFO_NAME(res_col_info, i),
            CCI_GET_RESULT_INFO_TYPE(res_col_info, i),
            CCI_GET_RESULT_INFO_PRECISION(res_col_info, i),
            CCI_GET_RESULT_INFO_SCALE(res_col_info, i));
    }
    printf("=========================================\n");
    if ((res=cci_execute(req, 0, 0, &error))<0) {
        if (req < 0) {
            printf( "%s(%d): cci_execute fail(%d)\n", __FILE__,
__LINE__,error.err_code);
        }
        goto handle_error;
    }

    while (1) {
        res = cci_cursor(req, 1, CCI_CURSOR_CURRENT, &error);
        if (res == CCI_ER_NO_MORE_DATA) {
            printf("Query END!\n");
            break;
        }
        if (res<0) {
            if (req < 0) {
                printf( "%s(%d): cci_cursor fail(%d)\n", __FILE__,
__LINE__,error.err_code);
            }
            goto handle_error;
```

```
        }

        if ((res=cci_fetch(req, &error))<0) {
            if (res < 0) {
                printf( "%s(%d): cci_fetch fail(%d)\n", __FILE__,
__LINE__,error.err_code);
            }
            goto handle_error;
        }

        for (i=1; i<=col_count; i++) {
            if ((res=cci_get_data(req, i, CCI_A_TYPE_STR, &buffer,
&ind))<0) {
                printf( "%s(%d): cci_get_data fail\n", __FILE__,
__LINE__);
                goto handle_error;
            }
            printf("%s \t|", buffer);
        }
        printf("\n");
    }
    if ((res=cci_close_req_handle(req))<0) {
        printf( "%s(%d): cci_close_req_handle fail", __FILE__,
__LINE__);
       goto handle_error;
    }
    if ((res=cci_disconnect(con, &error))<0) {
        if (res < 0) {
            printf( "%s(%d): cci_disconnect fail(%d)", __FILE__,
__LINE__,error.err_code);
        }
        goto handle_error;
    }
    printf("Program ended!\n");
    return 0;

    handle_error:
    if (req > 0)
        cci_close_req_handle(req);
    if (con > 0)
        cci_disconnect(con, &error);
    printf("Program failed!\n");
    return -1;
}
```

CCI API Reference

Contents

CCI API Reference 2135

- cci_bind_param 2140
- cci_bind_param_array 2144
- cci_bind_param_array_size 2145
- cci_bind_param_ex 2146
- cci_blob_free 2147
- cci_blob_new 2147
- cci_blob_read 2148
- cci_blob_size 2149
- cci_blob_write 2149
- cci_cancel 2150
- cci_clob_free 2154
- cci_clob_new 2155
- cci_clob_read 2155
- cci_clob_size 2156
- cci_clob_write 2157
- cci_close_query_result 2158
- cci_close_req_handle 2158
- cci_col_get 2159

- cci_col_seq_drop 2160
- cci_col_seq_insert 2161
- cci_col_seq_put 2162
- cci_col_set_add 2162
- cci_col_set_drop 2163
- cci_col_size 2164
- cci_connect 2165
- cci_connect_ex 2165
- cci_connect_with_url 2166
- cci_connect_with_url_ex 2171
- cci_cursor_update 2172
- cci_datasource_borrow 2174
- cci_datasource_change_property 2175
- cci_datasource_create 2177
- cci_datasource_destroy 2177
- cci_datasource_release 2178
- cci_disconnect 2179
- cci_end_tran 2179
- cci_escape_string 2180
- cci_execute 2181
- cci_execute_array 2183

- cci_execute_batch 2186
- cci_execute_result 2189
- cci_fetch 2190
- cci_fetch_buffer_clear 2191
- cci_fetch_sensitive 2191
- cci_fetch_size 2192
- cci_get_autocommit 2192
- cci_get_bind_num 2192
- cci_get_cas_info 2193
- cci_get_class_num_objs 2194
- CCI_GET_COLLECTION_DOMAIN 2194
- cci_get_cur_oid 2195
- cci_get_data 2195
- cci_get_db_parameter 2198
- cci_get_db_version 2199
- cci_get_err_msg 2200
- cci_get_error_msg 2201
- cci_get_holdability 2202
- cci_get_last_insert_id 2202
- cci_get_login_timeout 2204
- cci_get_query_plan 2204

- cci_query_info_free 2205
- cci_get_query_timeout 2206
- cci_get_result_info 2206
- CCI_GET_RESULT_INFO_ATTR_NAME 2208
- CCI_GET_RESULT_INFO_CLASS_NAME 2209
- CCI_GET_RESULT_INFO_IS_NON_NULL 2209
- CCI_GET_RESULT_INFO_NAME 2210
- CCI_GET_RESULT_INFO_PRECISION 2210
- CCI_GET_RESULT_INFO_SCALE 2210
- CCI_GET_RESULT_INFO_TYPE 2211
- CCI_IS_SET_TYPE 2211
- CCI_IS_MULTISET_TYPE 2212
- CCI_IS_SEQUENCE_TYPE 2212
- CCI_IS_COLLECTION_TYPE 2212
- cci_get_version 2213
- cci_init 2213
- cci_is_holdable 2214
- cci_is_updatable 2214
- cci_next_result 2215
- cci_oid 2216
- cci_oid_get 2217

- cci_oid_get_class_name 2218
- cci_oid_put 2218
- cci_oid_put2 2219
- cci_prepare 2221
- cci_prepare_and_execute 2222
- cci_property_create 2223
- cci_property_destroy 2224
- cci_property_get 2224
- cci_property_set 2225
- cci_query_result_free 2230
- CCI_QUERY_RESULT_ERR_NO 2230
- CCI_QUERY_RESULT_ERR_MSG 2231
- CCI_QUERY_RESULT_RESULT 2231
- CCI_QUERY_RESULT_STMT_TYPE 2232
- cci_register_out_param 2232
- cci_row_count 2236
- cci_savepoint 2237
- cci_schema_info 2238
- cci_set_allocators 2251
- cci_set_autocommit 2255
- cci_set_db_parameter 2256

- cci_set_element_type 2257
- cci_set_free 2257
- cci_set_get 2257
- cci_set_holdability 2259
- cci_set_isolation_level 2259
- cci_set_lock_timeout 2260
- cci_set_login_timeout 2261
- cci_set_make 2261
- cci_set_max_row 2262
- cci_set_query_timeout 2262
- cci_set_size 2263

cci_bind_param

int cci_bind_param(int req_handle, int index, T_CCI_A_TYPE a_type, void *value, T_CCI_U_TYPE u_type, char flag)

The **cci_bind_param** function binds data in the *bind* variable of prepared statement. This function converts *value* of the given *a_type* to an actual binding type and stores it. Subsequently, whenever `cci_execute()` is called, the stored data is sent to the server. If **cci_bind_param** () is called multiple times for the same *index*, the latest configured value is valid.

Parameters:

- **req_handle** – (IN) Request handle of a prepared statement
- **index** – (IN) Location of binding; it starts with 1.
- **a_type** – (IN) Data type of *value*
- **value** – (IN) Data value to bind

- **u_type** – (IN) Data type to be applied to the database
- **flag** – (IN) bind_flag(`CCI_BIND_PTR`).

Returns:

Error code (0: success)

- **CCI_ER_BIND_INDEX**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_REQ_HANDLE**
- **CCI_ER_TYPE_CONVERSION**
- **CCI_ER_USED_CONNECTION**

To bind **NULL** to the database, choose one of below settings.

- Set the value of *value* to a **NULL** pointer
- Set the value of *u_type* to `CCI_U_TYPE_NULL`

The following shows a part of code to bind NULL.

```
res = cci_bind_param (req, 2 /* binding index*/, CCI_A_TYPE_STR,
NULL, CCI_U_TYPE_STRING, CCI_BIND_PTR);
```

or

```
res = cci_bind_param (req, 2 /* binding index*/, CCI_A_TYPE_STR,
data, CCI_U_TYPE_NULL, CCI_BIND_PTR);
```

can be used.

If **CCI_BIND_PTR** is configured for *flag*, the pointer of *value* variable is copied (shallow copy), but no value is copied. If it is not configured for *flag*, the value of *value* variable is copied (deep copy) by allocating memory. If multiple columns are bound by using the same memory buffer, **CCI_BIND_PTR** must not be configured for the *flag*.

T_CCI_A_TYPE is a C language type that is used in CCI applications for data binding, and consists of primitive types such as int and float, and user-defined types defined by CCI such

as **T_CCI_BIT** and **T_CCI_DATE**. The identifier for each type is defined as shown in the table below.

a_type	value type
CCI_A_TYPE_STR	char *
CCI_A_TYPE_INT	int *
CCI_A_TYPE_FLOAT	float *
CCI_A_TYPE_DOUBLE	double *
CCI_A_TYPE_BIT	**T_CCI_BIT** *
CCI_A_TYPE_SET	**T_CCI_SET**
CCI_A_TYPE_DATE	**T_CCI_DATE** *
CCI_A_TYPE_BIGINT	int64_t * (For Windows: __int64 *)
CCI_A_TYPE_BLOB	**T_CCI_BLOB**
CCI_A_TYPE_CLOB	**T_CCI_CLOB**

T_CCI_U_TYPE is a column type of database and data bound though the *value* argument is converted into this type. The **cci_bind_param** () function uses two kinds of types to send information which is used to convert U-type data from A-type data; the U-type data can be interpreted by database language and the A-type data can be interpreted by C language.

There are various A-type data that are allowed by U-type data. For example, **CCI_U_TYPE_INT** can receive **CCI_A_TYPE_STR** as A-type data including **CCI_A_TYPE_INT**. For information on type conversion, see Implicit Type Conversion.

Both **T_CCI_A_TYPE** and **T_CCI_U_TYPE** enum(s) are defined in the **cas_cci.h** file. The definition of each identifier is described in the table below.

u_type	Corresponding a_type (default)
CCI_U_TYPE_CHAR	CCI_A_TYPE_STR
CCI_U_TYPE_STRING	CCI_A_TYPE_STR
CCI_U_TYPE_BIT	CCI_A_TYPE_BIT
CCI_U_TYPE_VARBIT	CCI_A_TYPE_BIT
CCI_U_TYPE_NUMERIC	CCI_A_TYPE_STR
CCI_U_TYPE_INT	CCI_A_TYPE_INT
CCI_U_TYPE_SHORT	CCI_A_TYPE_INT
CCI_U_TYPE_FLOAT	CCI_A_TYPE_FLOAT
CCI_U_TYPE_DOUBLE	CCI_A_TYPE_DOUBLE
CCI_U_TYPE_DATE	CCI_A_TYPE_DATE
CCI_U_TYPE_TIME	CCI_A_TYPE_DATE
CCI_U_TYPE_TIMESTAMP	CCI_A_TYPE_DATE
CCI_U_TYPE_OBJECT	CCI_A_TYPE_STR
CCI_U_TYPE_BIGINT	CCI_A_TYPE_BIGINT
CCI_U_TYPE_DATETIME	CCI_A_TYPE_DATE

u_type	Corresponding a_type (default)
CCI_U_TYPE_BLOB	**CCI_A_TYPE_BLOB**
CCI_U_TYPE_CLOB	**CCI_A_TYPE_CLOB**
CCI_U_TYPE_ENUM	**CCI_A_TYPE_STR**

When the string including the date is used as an input parameter of **DATE**, **DATETIME**, or **TIMESTAMP**, "YYYY/MM/DD" or "YYYY-MM-DD" is allowed for the date string type. Therefore, "2012/01/31" or "2012-01-31" is valid, but "01/31/2012" is invalid. The following is an example of having the string that includes the date as an input parameter of the date type.

```
// "CREATE TABLE tbl(aa date, bb datetime)";

char *values[][3] =
{
    {"1994/11/30", "1994/11/30 20:08:08"},
    {"2008-10-31", "2008-10-31 20:08:08"}
};

req = cci_prepare(conn, "insert into tbl (aa, bb) values
( ?, ?)", CCI_PREPARE_INCLUDE_OID, &error);

for(i=0; i< 2; i++)
{
    res = cci_bind_param(req, 1, CCI_A_TYPE_STR, values[i][0],
CCI_U_TYPE_DATE, (char)NULL);
    res = cci_bind_param(req, 2, CCI_A_TYPE_STR, values[i][1],
CCI_U_TYPE_DATETIME, (char)NULL);
    cci_execute(req, CCI_EXEC_QUERY_ALL, 0, err_buf);
}
```

cci_bind_param_array

int cci_bind_param_array(int req_handle, int index, T_CCI_A_TYPE a_type, void *value, int *null_ind, T_CCI_U_TYPE u_type)

The **cci_bind_param_array** function binds a parameter array for a prepared `cci_execute_array()` is called, data is sent to the server by the stored *value* pointer. If **cci_bind_param_array** () is called multiple times for the same *index*, the last configured value is valid. If **NULL** is bound to the data, a non-zero value is configured in *null_ind*. If

value is a **NULL** pointer, or *u_type* is **CCI_U_TYPE_NULL**, all data are bound to **NULL** and the data buffer used by *value* cannot be reused. For the data type of *value* for *a_type*, see the `cci_bind_param()` function description.

Parameters:

- **req_handle** – (IN) Request handle of the prepared statement
- **index** – (IN) Binding location
- **a_type** – (IN) Data type of *value*
- **value** – (IN) Data value to be bound
- **null_ind** – (IN) **NULL** indicator array (0 : not **NULL**, 1 : **NULL**)
- **u_type** – (IN) Data type to be applied to the database.

Returns:

Error code (0: success)

- **CCI_ER_BIND_INDEX**
- **CCI_ER_BIND_ARRAY_SIZE**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_REQ_HANDLE**
- **CCI_ER_TYPE_CONVERSION**
- **CCI_ER_USED_CONNECTION**

cci_bind_param_array_size

int cci_bind_param_array_size(int req_handle, int array_size)

The **cci_bind_param_array_size** function determines the size of the array to be used in `cci_bind_param_array()`. **cci_bind_param_array_size** () must be called first before `cci_bind_param_array()` is used.

Parameters:

- **req_handle** – Request handle of a prepared statement
- **array_size** – (IN) binding array size

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_REQ_HANDLE**
- **CCI_ER_USED_CONNECTION**

cci_bind_param_ex

int cci_bind_param_ex(int req_handle, int index, T_CCI_A_TYPE a_type, void *value, int length, T_CCI_U_TYPE u_type, char flag)

The **cci_bind_param_ex** function works as the same with `cci_bind_param()`. However, it has an additional argument, *length*, which specifies the byte length of a string if bound data is a string.

Parameters:

- **req_handle** – (IN) Request handle of the prepared statement
- **index** – (IN) Binding location, starting from 1
- **a_type** – (IN) Data type of *value*
- **value** – (IN) Data value to be bound
- **length** – (IN) Byte length of a string to be bound
- **u_type** – (IN) Data type to be applied to the database.
- **flag** – (IN) bind_flag(`CCI_BIND_PTR`).

Returns:

Error code(0: success)

The *length* argument can be used for binding a string which includes '\0' as below.

```
cci_bind_param_ex(statement, 1, CCI_A_TYPE_STR, "aaa\0bbb", 7,
CCI_U_TYPE_STRING, 0);
```

cci_blob_free

int cci_blob_free(T_CCI_BLOB blob)

The **cci_blob_free** function frees memory of *blob* struct.

Returns:

Error code (0: success)

- **CCI_ER_INVALID_LOB_HANDLE**

cci_blob_new

int cci_blob_new(int conn_handle, T_CCI_BLOB *blob, T_CCI_ERROR *error_buf)

The **cci_blob_new** function creates an empty file where **LOB** data is stored and returns locator referring to the data to *blob* struct.

Parameters:

- **conn_handle** – (IN) Connection handle
- **blob** – (OUT) **LOB** locator
- **error_buf** – (OUT) Error buffer

Returns:

Error code (0: success)

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**

- **CCI_ER_DBMS**
- **CCI_ER_INVALID_LOB_HANDLE**
- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_USED_CONNECTION**

cci_blob_read

int cci_blob_read(int conn_handle, T_CCI_BLOB blob, long start_pos, int length, char *buf, T_CCI_ERROR *error_buf)

The **cci_blob_read** function reads as much as data from *start_pos* to *length* of the **LOB** data file specified in *blob*; then it stores it in *buf* and returns it.

Parameters:

- **conn_handle** – (IN) Connection handle
- **blob** – (OUT) **LOB** locator
- **start_pos** – (IN) Index location of **LOB** data file
- **length** – (IN) **LOB** data length from buffer
- **buf** – (IN) Data buffer to read
- **error_buf** – (OUT) Error buffer

Returns:

Size of read value (>= 0: success), Error code (< 0: error)

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_DBMS**
- **CCI_ER_INVALID_LOB_HANDLE**
- **CCI_ER_INVALID_LOB_READ_POS**

- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_USED_CONNECTION**

cci_blob_size

long long cci_blob_size(T_CCI_BLOB *blob)

The **cci_blob_size** function returns data file size that is specified in *blob*.

Parameters:

- **blob** – (OUT) **LOB** locator

Returns:

Size of **BLOB** data file (>= 0: success), Error code (< 0: error)

- **CCI_ER_INVALID_LOB_HANDLE**

cci_blob_write

int cci_blob_write(int conn_handle, T_CCI_BLOB blob, long start_pos, int length, const char *buf, T_CCI_ERROR *error_buf)

The **cci_blob_write** function reads as much as data from *buf* to *length* and stores it from *start_pos* of the **LOB** data file specified in *blob*.

Parameters:

- **conn_handle** – (IN) Connection handle
- **blob** – (OUT) **LOB** locator
- **start_pos** – (IN) Index location of **LOB** data file
- **length** – (IN) Data length from buffer
- **buf** – (OUT) Data buffer to write
- **error_buf** – (OUT) Error buffer

Returns:

Size of written value (>= 0: success), Error code (< 0: error)

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_DBMS**
- **CCI_ER_INVALID_LOB_HANDLE**
- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_USED_CONNECTION**

cci_cancel

int cci_cancel(int conn_handle)

Cancel the running query on the other thread. This function executes the same behavior as Statement.cancel() method in JAVA.

Parameters:

- **conn_handle** – (IN) Connection handle

Returns:

Error code

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**

The following shows how to cancel the query execution of a thread.

```
/* gcc -o pthr pthr.c -m64 -I${CUBRID}/include -lnsl ${CUBRID}/
lib/libcascci.so -lpthread
*/


#include <stdio.h>
#include <cas_cci.h>
#include <unistd.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

#define QUERY "select * from db_class A, db_class B, db_class C,
db_class D, db_class E"

static void *thread_main (void *arg);
static void *execute_statement (int con, char *sql_stmt);

int
main (int argc, char *argv[])
{
    int thr_id = 0, conn_handle = 0, res = 0;
    void *jret;
    pthread_t th;
    char url[1024];
    T_CCI_ERROR error;
    snprintf (url, 1024, "cci:CUBRID:localhost:
33000:demodb:PUBLIC::");

    conn_handle = cci_connect_with_url_ex (url, NULL, NULL,
&error);

    if (conn_handle < 0)
    {
        printf ("ERROR: %s\n", error.err_msg);
        return -1;
    }
```

```
    res = pthread_create (&th, NULL, &thread_main, (void *)
&conn_handle);

    if (res < 0)
    {
        printf ("thread fork failed.\n");
        return -1;
    }
    else
    {
        printf ("thread started\n");
    }
    sleep (5);
    // If thread_main is still running, below cancels the query
of thread_main.
    res = cci_cancel (conn_handle);
    if (res < 0)
    {
        printf ("cci_cancel failed\n");
        return -1;
    }
    else
    {
        printf ("The query was canceled by cci_cancel.\n");
    }
    res = pthread_join (th, &jret);
    if (res < 0)
    {
        printf ("thread join failed.\n");
        return -1;
    }

    printf ("thread_main was cancelled with\n\t%s\n", (char *)
jret);
    free (jret);

    res = cci_disconnect (conn_handle, &error);
    if (res < 0)
    {
        printf ("ERROR: %s\n", error.err_msg);
      return res;
    }

    return 0;
}

void *
thread_main (void *arg)
{
    int con = *((int *) arg);
    int ret_val;
```

```
    void *ret_ptr;
    T_CCI_ERROR error;

    cci_set_autocommit (con, CCI_AUTOCOMMIT_TRUE);
    ret_ptr = execute_statement (con, QUERY);
    return ret_ptr;
}

static void *
execute_statement (int con, char *sql_stmt)
{
    int col_count = 1, ind, i, req;
    T_CCI_ERROR error;
    char *buffer;
    char *error_msg;
    int res = 0;

    error_msg = (char *) malloc (128);
    if ((req = cci_prepare (con, sql_stmt, 0, &error)) < 0)
    {
        snprintf (error_msg, 128, "cci_prepare ERROR: %s\n",
error.err_msg);
        goto conn_err;
    }

    if ((res = cci_execute (req, 0, 0, &error)) < 0)
    {
        snprintf (error_msg, 128, "cci_execute ERROR: %s\n",
error.err_msg);
        goto execute_error;
    }

    if (res >= 0)
    {
        while (1)
        {
            res = cci_cursor (req, 1, CCI_CURSOR_CURRENT,
&error);
            if (res == CCI_ER_NO_MORE_DATA)
            {
                break;
            }
            if (res < 0)
            {
                snprintf (error_msg, 128, "cci_cursor ERROR:
%s\n",
                    error.err_msg);
                return error_msg;
            }

            if ((res = cci_fetch (req, &error)) < 0)
            {
```

```
                snprintf (error_msg, 128, "cci_fetch ERROR:
%s\n",
                    error.err_msg);
                return error_msg;
            }

            for (i = 1; i <= col_count; i++)
            {
                if ((res = cci_get_data (req, i, CCI_A_TYPE_STR,
&buffer, &ind)) < 0)
                {
                    snprintf (error_msg, 128, "cci_get_data
ERROR\n");
                    return error_msg;
                }
            }
        }
    }

    if ((res = cci_close_query_result (req, &error)) < 0)
    {
        snprintf (error_msg, 128, "cci_close_query_result ERROR:
%s\n", error.err_msg);
        return error_msg;
    }
execute_error:
    if ((res = cci_close_req_handle (req)) < 0)
    {
        snprintf (error_msg, 128, "cci_close_req_handle
ERROR\n");
    }
conn_err:
    return error_msg;
}
```

cci_clob_free

int cci_clob_free(T_CCI_CLOB clob)

The **cci_clob_free** function frees memory of **CLOB** struct.

Parameters:

- **clob** – (IN) **LOB** locator

Returns:

Error code (0: success)

- **CCI_ER_INVALID_LOB_HANDLE**

cci_clob_new

int cci_clob_new(int conn_handle, T_CCI_CLOB *clob, T_CCI_ERROR *error_buf)

The **cci_clob_new** function creates an empty file where **LOB** data is stored and returns locator referring to the data to *clob* struct.

Parameters:

- **conn_handle** – ((IN) Connection handle
- **clob** – (OUT) **LOB** locator
- **error_buf** – (OUT) Error buffer

Returns:

Error code (0: success)

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_DBMS**
- **CCI_ER_INVALID_LOB_HANDLE**
- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_USED_CONNECTION**

cci_clob_read

int cci_clob_read(int conn_handle, T_CCI_CLOB clob, long start_pos, int length, char *buf, T_CCI_ERROR *error_buf)

The **cci_clob_read** function reads as much as data from *start_pos* to *length* in the **LOB** data file specified in *clob*; then it stores it in *buf* and returns it.

Parameters:

- **conn_handle** – (IN) Connection handle
- **clob** – (IN) **LOB** locator
- **start_pos** – (IN) Index location of **LOB** data file
- **length** – (IN) **LOB** data length from buffer
- **buf** – (IN) Data buffer to read
- **error_buf** – (OUT) Error buffer

Returns:

Size of read value (>= 0: success), Error code (< 0: Error)

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_DBMS**
- **CCI_ER_INVALID_LOB_HANDLE**
- **CCI_ER_INVALID_LOB_READ_POS**
- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_USED_CONNECTION**

cci_clob_size

long long cci_clob_size(T_CCI_CLOB *clob)

The **cci_clob_size** function returns data file size that is specified in *clob*.

Parameters:

- **clob** – (IN) **LOB** locator

Returns:

Size of **CLOB** data file (>= 0: success), Error code (< 0: error)

- **CCI_ER_INVALID_LOB_HANDLE**

cci_clob_write

int cci_clob_write(int conn_handle, T_CCI_CLOB clob, long start_pos, int length, const char *buf, T_CCI_ERROR *error_buf)

The **cci_clob_write** function reads as much as data from *buf* to *length* and then stores the value from *start_pos* in **LOB** data file specified in *clob*.

Parameters:

- **conn_handle** – (IN) Connection handle
- **clob** – (IN) **LOB** locator
- **start_pos** – (IN) Index location of **LOB** data file
- **length** – (IN) Data length from buffer
- **buf** – (OUT) Data buffer to write
- **error_buf** – (OUT) Error buffer

Returns:

Size of written value (>= 0: success), Error code (< 0: Error)

- **CCI_ER_COMMUNICATION**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_DBMS**
- **CCI_ER_INVALID_LOB_HANDLE**

- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_USED_CONNECTION**

cci_close_query_result

int cci_close_query_result(int req_handle, T_CCI_ERROR *err_buf)

The **cci_close_query_result** funciton closes the resultset returned by `cci_execute()`, `cci_execute_array()` or `cci_execute_batch()`. If you run `cci_prepare()` repeatedly without closing the request handle(req_handle), it is recommended to call this function before calling `cci_close_req_handle()`.

Parameters:

- **req_handle** – (IN) Request handle
- **err_buf** – (OUT) Error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_DBMS**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_REQ_HANDLE**
- **CCI_ER_RESULT_SET_CLOSED**
- **CCI_ER_USED_CONNECTION**

cci_close_req_handle

int cci_close_req_handle(int req_handle)

The **cci_close_req_handle** function closes the request handle obtained by `cci_prepare()`.

Parameters:

- **req_handle** – (IN) Request handle

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_REQ_HANDLE**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_DBMS**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_USED_CONNECTION**

cci_col_get

int cci_col_get(int conn_handle, char *oid_str, char *col_attr, int *col_size, int *col_type, T_CCI_ERROR *err_buf)

The **cci_col_get** function gets an attribute value of collection type. If the name of the class is C, and the domain of *set_attr* is set (multiset, sequence), the query looks like as follows:

```
SELECT a FROM C, TABLE(set_attr) AS t(a) WHERE C = oid;
```

That is, the number of members becomes the number of records.

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **col_attr** – (IN) Collection attribute name
- **col_size** – (OUT) Collection size (-1 : null)

- **col_type** – (OUT) Collection type (set, multiset, sequence: u_type)
- **err_buf** – (OUT) Database error buffer

Returns:

Request handle

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**
- **CCI_ER_DBMS**

cci_col_seq_drop

int cci_col_seq_drop(int conn_handle, char *oid_str, char *col_attr, int index, T_CCI_ERROR *err_buf)

The **cci_col_seq_drop** function drops the index-th (base: 1) member of the sequence attribute values. The following example shows how to drop the first member of the sequence attribute values.

```
cci_col_seq_drop(conn_handle, oid_str, seq_attr, 1, err_buf);
```

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **col_attr** – (IN) Collection attribute name
- **index** – (IN) Index
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**
- **CCI_ER_DBMS**

cci_col_seq_insert

int cci_col_seq_insert(int conn_handle, char *oid_str, char *col_attr, int index, char *value, T_CCI_ERROR *err_buf)

The **cci_col_seq_insert** function inserts one member at the index-th (base: 1) position of the sequence attribute values. The following example shows how to insert “a” at the first position of the sequence attribute values.

```
cci_col_seq_insert(conn_handle, oid_str, seq_attr, 1, "a",
err_buf);
```

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **col_attr** – (IN) Collection attribute name
- **index** – (IN) Index
- **value** – (IN) Sequential element (string)
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**

- **CCI_ER_DBMS**

cci_col_seq_put

int cci_col_seq_put(int conn_handle, char *oid_str, char *col_attr, int index, char *value, T_CCI_ERROR *err_buf)

The **cci_col_seq_put** function replaces the index-th (base: 1) member of the sequence attribute values with a new value. The following example shows how to replace the first member of the sequence attributes values with “a”.

```
cci_col_seq_put(conn_handle, oid_str, seq_attr, 1, "a", err_buf);
```

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **col_attr** – (IN) Collection attribute name
- **index** – (IN) Index
- **value** – (IN) Sequential value
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**
- **CCI_ER_DBMS**

cci_col_set_add

int cci_col_set_add(int conn_handle, char *oid_str, char *col_attr, char *value, T_CCI_ERRROR *err_buf)

The **cci_col_set_add** function adds one member to the set attribute values. The following example shows how to add “a” to the set attribute values.

```
cci_col_set_add(conn_handle, oid_str, set_attr, "a", err_buf);
```

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **col_attr** – (IN) collection attribute name
- **value** – (IN) set element
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**
- **CCI_ER_DBMS**

cci_col_set_drop

int cci_col_set_drop(int conn_handle, char *oid_str, char *col_attr, char *value, T_CCI_ERROR *err_buf)

The **cci_col_set_drop** function drops one member from the set attribute values. The following example shows how to drop “a” from the set attribute values.

```
cci_col_set_drop(conn_handle, oid_str, set_attr, "a", err_buf);
```

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid

- **col_attr** – (IN) collection attribute name
- **value** – (IN) set element (string)
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_LOGIN_TIMEOUT**
- **CCI_ER_COMMUNICATION**

cci_col_size

int cci_col_size(int conn_handle, char *oid_str, char *col_attr, int *col_size, T_CCI_ERROR *err_buf)

The **cci_col_size** function gets the size of the set (seq) attribute.

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **col_attr** – (IN) Collection attribute name
- **col_size** – (OUT) Collection size (-1: NULL)
- **err_buf** – Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**

- **CCI_ER_DBMS**

cci_connect

int cci_connect(char *ip, int port, char *db_name, char *db_user, char *db_password)

A connection handle to the database server is assigned and it tries to connect to the server. If it has succeeded, the connection handle ID is returned; if fails, an error code is returned.

Parameters:

- **ip** – (IN) A string that represents the IP address of the server (hostname)
- **port** – (IN) Broker port (The port configured in the **$CUBRID/conf/cubrid_broker.conf** file)
- **db_name** – (IN) Database name
- **db_user** – (IN) Database username
- **db_passwd** – (IN) Database user password

Returns:

Success: Connection handle ID (int), Failure: Error code

- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_HOSTNAME**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

cci_connect_ex

int cci_connect_ex(char *ip, int port, char *db_name, char *db_user, char *db_password, T_CCI_ERROR *err_buf)

The **cci_connect_ex** function returns **CCI_ER_DBMS** error and checks the error details in the database error buffer (*err_buf*) at the same time. In that point, it is different from `cci_connect()` and the others are the same as the `cci_connect()` function.

Parameters:

- **ip** – (IN) A string that represents the IP address of the server (hostname)
- **port** – (IN) Broker port (The port configured in the **$CUBRID/conf/cubrid_broker.conf** file)
- **db_name** – (IN) Database name
- **db_user** – (IN) Database username
- **db_passwd** – (IN) Database user password
- **err_buf** – Database error buffer

Returns:

Success: Connection handle ID (int), Failure: Error code

- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_HOSTNAME**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

cci_connect_with_url

int cci_connect_with_url(char *url, char *db_user, char *db_password)

The **cci_connect_with_url** function connects a database by using connection information passed with a *url* argument. If broker's HA feature is used in CCI, you must specify the connection information of the standby broker server with altHosts property, which is used for the failover, in the *url* argument of this function. It returns the ID of a connection handle

on success; it returns an error code on failure. For details about HA features of broker, see Duplexing Brokers.

Parameters:

- **url** – (IN) A string that contains server connection information.
- **db_user** – (IN) Database username. If this is NULL, it becomes <*db_user*> in *url*. If this is an empty string ("") or <*db_user*> in *url* is not specified, DB username becomes **PUBLIC**.
- **db_passwd** – (IN) Database user password. If this is NULL, <*db_password*> in *url* is used. If <*db_password*> in *url* is not specified, DB password becomes an empty string ("").

Returns:

Success: Connection handle ID (int), Failure: Error code

- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_HOSTNAME**
- **CCI_ER_INVALID_URL**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_LOGIN_TIMEOUT**

```
<url> ::=
cci:CUBRID:<host>:<port>:<db_name>:<db_user>:<db_password>:[?
<properties>]

<properties> ::= <property> [&<property>]
<property> ::= altHosts=<alternative_hosts> [ &rcTime=<time>]
[ &loadBalance=true|false]
             |{login_timeout|loginTimeout}=<milli_sec>
             |{query_timeout|queryTimeout}=<milli_sec>
```

```
              |{disconnect_on_query_timeout|
disconnectOnQueryTimeout}=true|false
              | logFile=<file_name>
              | logBaseDir=<dir_name>
              | logSlowQueries=true|
false[&slowQueryThresholdMillis=<milli_sec>]
              | logTraceApi=true|false
              | logTraceNetwork=true|false
              | useSSL=true|false

<alternative_hosts> ::= <host>:<port> [,<host>:<port>]

<host> := HOSTNAME | IP_ADDR
<time> := SECOND
<milli_sec> := MILLI SECOND
```

altHosts is the property related to connection target and **loginTimeout**, **queryTimeout**, and **disconnectOnQueryTimeout** are the properties related to timeout; **logSlowQueries**, **logTraceApi**, and **logTraceNetwork** are the properties related to log information configuration for debugging. Note that a property name which is a value to be entered in the *url* argument is not case sensitive.

- *host*: A hostname or IP address of the master database
- *port*: A port number
- *db_name*: A name of the database
- *db_user*: A name of the database user
- *db_password*: A database user password. You cannot include ':' in the password of the *url* string.
- **altHosts** = *standby_broker1_host*, *standby_broker2_host*, ...: Specifies the broker information of the standby server, which is used for failover when it is impossible to connect to the active server. You can specify multiple brokers for failover, and the connection to the brokers is attempted in the order listed in **alhosts**.

> **Note**
>
> Even if there are **RW** and **RO** together in *ACCESS_MODE** setting of brokers of main host and **altHosts**, application decides the target host to access without the relation for the setting of **ACCESS_MODE**. Therefore, you should define the main host and **altHosts** as considering **ACCESS_MODE** of target brokers.

- **rcTime**: After the failure occurred on the first connected broker, the application connects to the broker specified by **altHosts**(broker failover). Then it attempts to reconnect to the first connected broker at every **rcTime**(default value: 600 seconds).

- **loadBalance**: When this value is true, the applications try to connect to the main host and alternative hosts specified with the **altHosts** property as a random order. (default value: false).

- **login_timeout** | **loginTimeout**: Timeout value (unit: msec.) for database login. Upon timeout, a **CCI_ER_LOGIN_TIMEOUT** (-38) error is returned. The default value is **30,000**(30 sec.). If this value is 0, it means infinite waiting. This value is also applied when internal reconnection occurs after the initial connection.

- **query_timeout** | **queryTimeout**: If time specified in these properties has expired when calling `cci_prepare()`, `cci_execute()`, etc. a cancellation message for query request which was sent to a server will be delivered and called function returns a **CCI_ER_QUERY_TIMEOUT** (-39) error. The default value is 0. If this value is 0, it means infinite waiting. The value returned upon timeout may vary depending on a value specified in **disconnect_on_query_timeout**. For details, see **disconnect_on_query_timeout**.

> **Note**
>
> If you use `cci_execute_batch()` or `cci_execute_array()` function, or set **CCI_EXEC_QUERY_ALL** in `cci_execute()` function to run multiple queries at once, query timeout is applied to one function, not to one query. In other words, if query timeout occurs after the start of a function, a function running is quit.

- **disconnect_on_query_timeout** | **disconnectOnQueryTimeout**: Whether to disconnect socket immediately after time for query request has expired. It determines whether to terminate a socket connection immediately or wait for server response after sending cancellation message for query request to a server when calling `cci_prepare()`, `cci_execute()`, etc. The default value is **false**, meaning that it will wait for server response. It this value is true, a socket will be closed immediately after sending a cancellation message to a server upon timeout and returns the **CCI_ER_QUERY_TIMEOUT** (-39) error. (If an error occurs on database server side, not on broker side, it returns -1. If you want to view error details, see error codes in "database error buffer." You can get information how to check error codes in CCI Error Codes and Error Messages.) Please note that there is a possibility that a database server does not get a cancellation message and execute a query even after an error is returned.

- **logFile**: A log filename for debugging (default value: **cci_** <*handle_id*> **.log**). <*handle_id*> indicates the ID of a connection handle returned by this function.

- **logBaseDir**: A directory where a debug log file is created. The filename including the path will be logBaseDir/logFile, and the relative path is possible.
- **logSlowQueries**: Whether to log slow query for debugging (default value: **false**)
- **slowQueryThresholdMillis**: Timeout for slow query logging if slow query logging is enabled (default value: **60000**, unit: milliseconds)
- **logTraceApi**: Whether to log the start and end of CCI functions
- **logTraceNetwork**: Whether to log network data content transferred of CCI functions
- **useSSL**: Packet Encryption mode (Default: false)
    - Packet encryption: useSSL = true
    - Plain text: useSSL = false

Example

```
--connection URL string when a property(altHosts) is specified
for HA
URL=cci:CUBRID:192.168.0.1:33000:demodb:::?
altHosts=192.168.0.2:33000,192.168.0.3:33000

--connection URL string when properties(altHosts,rcTime) is
specified for HA
URL=cci:CUBRID:192.168.0.1:33000:demodb:::?
altHosts=192.168.0.2:33000,192.168.0.3:33000&rcTime=600

--connection URL string when
properties(logSlowQueries,slowQueryThresholdMills, logTraceApi,
logTraceNetwork) are specified for interface debugging
URL = "cci:cubrid:192.168.0.1:33000:demodb:::?
logSlowQueries=true&slowQueryThresholdMillis=1000&logTraceApi=tru
e&logTraceNetwork=true"

--connection URL string when useSSL property specified for
encrypted connection
URL = "cci:cubrid:192.168.0.1:33000:demodb:::?useSSL=true
```

⚠ Warning

- The **useSSL** flag must match with the mode of the broker trying to connect. If encryption mode is different from the server that trying to connect, that connection request will be rejected. Please refer to the following cases that are not allowed.
    - useSSL=true, connection request will be rejected when the broker is in 'normal mode' (**cubrid_broker.conf**: SSL = OFF)
    - useSSL=false, connection request will be rejected when the broker is in 'encryption mode' (**cubrid_broker.conf**: SSL = ON)

cci_connect_with_url_ex

int cci_connect_with_url_ex(char *url, char *db_user, char *db_password, T_CCI_ERROR *err_buf)

The **cci_connect_with_url_ex** function returns **CCI_ER_DBMS** error and checks the error details in the database error buffer (*err_buf*) at the same time. In that point, it is different from `cci_connect_with_url()` and the others are the same as the `cci_connect_with_url()` function.

Parameters:

- **err_buf** – Database error buffer

int cci_cursor(int req_handle, int offset, T_CCI_CURSOR_POS origin, T_CCI_ERROR *err_buf)

The **cci_cursor** function moves the cursor specified in the request handle to access the specific record in the query result executed by `cci_execute()`. The position of cursor is moved by the values specified in the *origin* and *offset* values. If the position to be moved is not valid, **CCI_ER_NO_MORE_DATA** is returned.

Parameters:

- **req_handle** – (IN) Request handle
- **offset** – (IN) Offset to be moved
- **origin** – (IN) Variable to represent a position. The type is **T_CCI_CURSOR_POS**. **T_CCI_CURSOR_POS** enum consists of **CCI_CURSOR_FIRST**, **CCI_CURSOR_CURRENT** and **CCI_CURSOR_LAST**.
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_NO_MORE_DATA**
- **CCI_ER_COMMUNICATION**

Example

```
//the cursor moves to the first record
cci_cursor(req, 1, CCI_CURSOR_FIRST, &err_buf);

//the cursor moves to the next record
cci_cursor(req, 1, CCI_CURSOR_CURRENT, &err_buf);

//the cursor moves to the last record
cci_cursor(req, 1, CCI_CURSOR_LAST, &err_buf);

//the cursor moves to the previous record
cci_cursor(req, -1, CCI_CURSOR_CURRENT, &err_buf);
```

cci_cursor_update

int cci_cursor_update(int req_handle, int cursor_pos, int index, T_CCI_A_TYPE a_type, void *value, T_CCI_ERROR *err_buf)

The **cci_cursor_update** function updates *cursor_pos* from the value of the *index* -th column to *value*. If the database is updated to **NULL**, *value* becomes **NULL**. For update conditions, see `cci_prepare()`. The data types of *value* for *a_type* are shown in the table below.

Parameters:

- **req_handle** – (IN) Request handle
- **cursor_pos** – (IN) Cursor position
- **index** – (IN) Column index
- **a_type** – (IN) *value* Type
- **value** – (IN) A new value
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_TYPE_CONVERSION**
- **CCI_ER_ATYPE**

Data types of *value* for *a_type* are as below.

a_type	value type
CCI_A_TYPE_STR	char *
CCI_A_TYPE_INT	int *
CCI_A_TYPE_FLOAT	float *
CCI_A_TYPE_DOUBLE	double *
CCI_A_TYPE_BIT	**T_CCI_BIT** *
CCI_A_TYPE_SET	**T_CCI_SET**

a_type	value type
CCI_A_TYPE_DATE	**T_CCI_DATE ***
CCI_A_TYPE_BIGINT	int64_t * (For Windows: __int64 *)
CCI_A_TYPE_BLOB	**T_CCI_BLOB**
CCI_A_TYPE_CLOB	**T_CCI_CLOB**

cci_datasource_borrow

T_CCI_CONN cci_datasource_borrow(T_CCI_DATASOURCE *datasource, T_CCI_ERROR *err_buf)

The **cci_datasource_borrow** function obtains CCI connection to be used in **T_CCI_DATASOURCE** struct.

Parameters:

- **datasource** – (IN) **T_CCI_DATASOURCE** struct pointer in which CCI connection exists
- **err_buf** – (OUT) Error code and message returned upon error occurrence

Returns:

Success: CCI connection handler identifier, Failure: -1

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_release()`, `cci_datasource_change_property()`

cci_datasource_change_property

int cci_datasource_change_property(T_CCI_DATASOURCE *datasource, const char *key, const char *val)

A property name of a DATASOURCE is specified in *key*, a value in *val*. The changed property value by this function is applied to all connections in the *datasource*.

Parameters:

- **datasource** – (IN) T_CCI_DATASOURCE struct pointer to obtain CCI connections.
- **key** – (IN) A pointer to the string of a property name
- **val** – (IN) A pointer to the string of a property value

Returns:

Error code(0: success)

- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_NO_PROPERTY**
- **CCI_ER_PROPERTY_TYPE**

The following shows names and values of changeable properties.

Property name	Type	Value	Description
default_autocommit	bool	true/false	Whether autocommit or not. The default is CCI_DEFAULT_AUTO COMMIT in cubrid_broker.conf; the default of this is ON(true)
default_lock_timeou t	msec	number	lock timeout

Property name	Type	Value	Description
default_isolation	string	See the table of `cci_property_set()`	isolation level. The default is isolation_level in cubrid.conf; the default of this is "READ COMMITTED".
login_timeout	msec	number	login timeout. The default is 0(infinite wait). It can also be used when you call prepare or execute functions; at this time reconnection can happen.

Example

```
...
ps = cci_property_create ();
...
ds = cci_datasource_create (ps, &err);
...
cci_datasource_change_property(ds, "login_timeout", "5000");
cci_datasource_change_property(ds, "default_lock_timeout",
"2000");
cci_datasource_change_property(ds, "default_isolation",
"TRAN_REP_CLASS_COMMIT_INSTANCE");
cci_datasource_change_property(ds, "default_autocommit", "true");
...
```

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_borrow()`, `cci_datasource_destroy()`, `cci_datasource_release()`

cci_datasource_create

T_CCI_DATASOURCE *cci_datasource_create(T_CCI_PROPERTIES *properties, T_CCI_ERROR *err_buf)

The **cci_datasource_create** function creates DATASOURCE of CCI.

Parameters:

- **properties** – (IN) **T_CCI_PROPERTIES** struct pointer in which configuration of struct pointer is stored. Values of properties will be set with `cci_property_set()`.
- **err_buf** – (OUT) Error buffer. Error code and message returned upon error occurrence

Returns:

Success: **T_CCI_DATASOURCE** struct pointer created, Failure: **NULL**

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_borrow()`, `cci_datasource_destroy()`, `cci_datasource_release()`, `cci_datasource_change_property()`

cci_datasource_destroy

void cci_datasource_destroy(T_CCI_DATASOURCE *datasource)

The **cci_datasource_destroy** function destroys DATASOURCE of CCI.

Parameters:

- **datasource** – (IN) **T_CCI_DATASOURCE** struct pointer to be deleted

Returns:

void

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_borrow()`, `cci_datasource_release()`, `cci_datasource_change_property()`

cci_datasource_release

int cci_datasource_release(T_CCI_DATASOURCE *datasource, T_CCI_CONN conn, T_CCI_ERROR *err_buf)

The **cci_datasource_release** function returns CCI connection released in **T_CCI_DATASOURCE** struct. If you want to reuse the connection after calling this function, recall `cci_datasource_borrow()`.

Parameters:

- **datasource** – (IN) **T_CCI_DATASOURCE** struct pointer which returns CCI connection
- **conn** – (IN) CCI connection handler identifier released
- **err_buf** – (OUT) Error buffer(returns error code and error message when an error occurs)

Returns:

Success: 1, Failure: 0

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_borrow()`, `cci_datasource_change_property()`

cci_disconnect

int cci_disconnect(int conn_handle, T_CCI_ERROR *err_buf)

The **cci_disconnect** function disconnects all request handles created for *conn_handle*. If a transaction is being performed, the handles are disconnected after `cci_end_tran()` is executed.

Parameters:

- **conn_handle** – (IN) Connection handle
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**

cci_end_tran

int cci_end_tran(int conn_handle, char type, T_CCI_ERROR *err_buf)

The **cci_end_tran** function performs commit or rollback on the current transaction. At this point, all open request handles are terminated and the connection to the database server is disabled. However, even after the connection to the server is disabled, the connection handle remains valid.

Parameters:

- **conn_handle** – (IN) Connection handle
- **type** – (IN) **CCI_TRAN_COMMIT** or **CCI_TRAN_ROLLBACK**

- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_TRAN_TYPE**

You can configure the default value of autocommit mode by using CCI_DEFAULT_AUTOCOMMIT broker parameter upon startup of an application. If configuration on broker parameter is omitted, the default value is **ON**; use the `cci_set_autocommit()` function to change autocommit mode within an application. If autocommit mode is **OFF**, you must explicitly commit or roll back transaction by using the `cci_end_tran()` function.

cci_escape_string

long cci_escape_string(int conn_handle, char *to, const char *from, unsigned long length, T_CCI_ERROR *err_buf)

Converts the input string to a string that can be used in the CUBRID query. The following parameters are specified in this function: connection handle or **no_backslash_escapes** setting value, output string pointer, input string pointer, the length of the input string, and the address of the **T_CCI_ERROR** struct variable.

Parameters:

- **conn_handle** – (IN) connection handle or **no_backslash_escapes** setting value. When a connection handle is given, the **no_backslash_escapes** parameter value is read to determine how to convert. Instead of the connection handle, **CCI_NO_BACKSLASH_ESCAPES_TRUE** or **CCI_NO_BACKSLASH_ESCAPES_FALSE** value can be sent to determine how to convert.
- **to** – (OUT) Result string
- **from** – (IN) Input string

- **length** – (IN) Maximum byte length of the input string
- **err_buf** – (OUT) Database error buffer

Returns:

Success: Byte length of the changed string, Failure: Error Code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_COMMUNICATION**

When the system parameter **no_backslash_escapes** is yes (default) or when the **CCI_NO_BACKSLASH_ESCAPES_TRUE** value is sent to the connection handle location, the string is converted to the following characters.

- ‘ (single quote) => ‘ + ‘ (escaped single quote)

When the system parameter **no_backslash_escapes** is no or when the **CCI_NO_BACKSLASH_ESCAPES_FALSE** value is sent to the connection handle location, the string is converted to the following characters:

- \n (new line character, ASCII 10) => \ + n (backslash + Alphabet n)
- \r (carriage return, ASCII 13) => \ + r (backslash + Alphabet r)
- \0 (ASCII 0) => \ + 0 (backslash + 0(ASCII 48)
- \ (backslash) => \ + \

You can assign the space where the result string will be saved by using the *length* parameter. It will take as much as the byte length of the maximum input string * 2 + 1.

cci_execute

int cci_execute(int req_handle, char flag, int max_col_size, T_CCI_ERROR *err_buf)

The **cci_execute** function executes the SQL statement (prepared statement) that has executed `cci_prepare()`. A request handle, *flag*, the maximum length of a column to be fetched, and the address of a **T_CCI_ERROR** construct variable in which error information being stored are specified as arguments.

Parameters:

- **req_handle** – (IN) Request handle of the prepared statement
- **flag** – (IN) exec flag (**CCI_EXEC_QUERY_ALL**)
- **max_col_size** – (IN) The maximum length of a column to be fetched when it is a string data type in bytes. If this value is 0, full length is fetched.
- **err_buf** – (OUT) Database error buffer

Returns:

- **SELECT** : Returns the number of results
- **INSERT**, **UPDATE** : Returns the number of rows reflected
- Others queries : 0
- Failure : Error code
  - **CCI_ER_REQ_HANDLE**
  - **CCI_ER_BIND**
  - **CCI_ER_DBMS**
  - **CCI_ER_COMMUNICATION**
  - **CCI_ER_QUERY_TIMEOUT**
  - **CCI_ER_LOGIN_TIMEOUT**

Through a *flag*, the way of query execution can be set as all queries or the first one.

Note

In 2008 R4.4 and from 9.2, CUBRID does not support setting the *flag* as **CCI_EXEC_ASYNC**, which brings the results as an asynchronous method.

If the *flag* is set to **CCI_EXEC_QUERY_ALL**, all prepared queries(separated by semicolon) are executed. If not, only the first query is executed.

If the *flag* is set to **CCI_EXEC_QUERY_ALL**, the following rules are applied.

- The return value is the result of the first query.
- If an error occurs in any query, the execution is processed as a failure.
- For a query composed of in a query composed of q1; q2; q3, even if an error occurs in q2 after q1 succeeds the execution, the result of q1 remains valid. That is, the previous successful query executions are not rolled back when an error occurs.
- If a query is executed successfully, the result of the second query can be obtained using `cci_next_result()`.

max_col_size is a value that is used to determine the maximum length of a column to be sent to a client when the columns of the prepared statement are **CHAR**, **VARCHAR**, **BIT** or **VARBIT**. If this value is 0, full length is fetched.

cci_execute_array

int cci_execute_array(int req_handle, T_CCI_QUERY_RESULT **query_result, T_CCI_ERROR *err_buf)

If more than one value is bound to the prepared statement, this gets the values of the variables to be bound and executes the query by binding each value to the variable.

Parameters:

- **req_handle** – (IN) Request handle of the prepared statement
- **query_result** – (OUT) Query results
- **err_buf** – (OUT) Database error buffer

Returns:

- Success: The number of executed queries (it has nothing to do with query execution success/failure.)
- Failure: Negative number
    - **CCI_ER_REQ_HANDLE**
    - **CCI_ER_BIND**
    - **CCI_ER_DBMS**

- **CCI_ER_COMMUNICATION**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_LOGIN_TIMEOUT**

To bind the data, call the `cci_bind_param_array_size()` function to specify the size of the array, bind each value to the variable by using the `cci_bind_param_array()` function, and execute the query by calling the `cci_execute_array()` function. The query result will be stored on the array of **T_CCI_QUERY_RESULT** structure.

`cci_execute_array()` function returns the results of queries to the *query_result* variable. You can use below macros to get the result of each query. In the macro, note that the validation check for each parameter entered is not performed.

After using the query_result variable, you must delete the query_result by using the `cci_query_result_free()` function.

Macro	Return Type	Description
`CCI_QUERY_RESULT_RESULT`	int	the number of affected rows or error identifier (-1: CAS error, -2: DBMS error)
`CCI_QUERY_RESULT_ERR_NO`	int	error number about a query
`CCI_QUERY_RESULT_ERR_MSG`	char *	error message about a query
`CCI_QUERY_RESULT_STMT_TYPE`	int(**T_CCI_CUBRID_STMT** enum)	type of a query statement

If autocommit mode is on, each query in the array is committed after executing.

Note

- In the previous version of 2008 R4.3, if the autocommit mode is on, all queries in the array were committed after all of them are executed. From 2008 R4.3 version, the transaction is committed every time when a query is executed.

- In autocommit mode off, if the general error occurs during executing one of the queries in the array on the cci_execute_array function which does a batch processing of the queries, the query with an error is ignored and the next query is executed continuously. But if the deadlock occurs, the error occurs as rolling back the transaction.

```
char *query =
    "update participant set gold = ? where host_year = ? and
nation_code = 'KOR'";
int gold[2];
char *host_year[2];
int null_ind[2];
T_CCI_QUERY_RESULT *result;
int n_executed;
...

req = cci_prepare (con, query, 0, &cci_error);
if (req < 0)
{
    printf ("prepare error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
    goto handle_error;
}

gold[0] = 20;
host_year[0] = "2004";

gold[1] = 15;
host_year[1] = "2008";

null_ind[0] = null_ind[1] = 0;
error = cci_bind_param_array_size (req, 2);
if (error < 0)
{
    printf ("bind_param_array_size error: %d\n", error);
    goto handle_error;
}

error =
    cci_bind_param_array (req, 1, CCI_A_TYPE_INT, gold,
null_ind, CCI_U_TYPE_INT);
if (error < 0)
{
    printf ("bind_param_array error: %d\n", error);
    goto handle_error;
}
error =
    cci_bind_param_array (req, 2, CCI_A_TYPE_STR, host_year,
```

```
null_ind, CCI_U_TYPE_INT);
if (error < 0)
{
    printf ("bind_param_array error: %d\n", error);
    goto handle_error;
}

n_executed = cci_execute_array (req, &result, &cci_error);
if (n_executed < 0)
{
    printf ("execute error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
    goto handle_error;
}
for (i = 1; i <= n_executed; i++)
{
    printf ("query %d\n", i);
    printf ("result count = %d\n", CCI_QUERY_RESULT_RESULT
(result, i));
    printf ("error message = %s\n", CCI_QUERY_RESULT_ERR_MSG
(result, i));
    printf ("statement type = %d\n",
            CCI_QUERY_RESULT_STMT_TYPE (result, i));
}
error = cci_query_result_free (result, n_executed);
if (error < 0)
{
    printf ("query_result_free: %d\n", error);
    goto handle_error;
}
error = cci_end_tran(con, CCI_TRAN_COMMIT, &cci_error);
if (error < 0)
{
    printf ("end_tran: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
    goto handle_error;
}
```

cci_execute_batch

int cci_execute_batch(int conn_handle, int num_sql_stmt, char **sql_stmt, T_CCI_QUERY_RESULT **query_result, T_CCI_ERROR *err_buf)

In CCI, multiple jobs can be processed simultaneously when using DML queries such as **INSERT** / **UPDATE** / **DELETE**. `CCI_QUERY_RESULT_RESULT` and `cci_execute_batch()` functions can be used to execute such batch jobs. Note that prepared statements cannot be used in the `cci_execute_batch()` function. The query result will be stored on the array of **T_CCI_QUERY_RESULT** structure.

Parameters:

- **conn_handle** – (IN) Connection handle
- **num_sql_stmt** – (IN) The number of *sql_stmt*
- **sql_stmt** – (IN) SQL statement array
- **query_result** – (OUT) The results of *sql_stmt*
- **err_buf** – (OUT) Database error buffer

Returns:

- Success: The number of executed queries (it has nothing to do with query execution success/failure.)
- Failure: Negative number
    - **CCI_ER_CON_HANDLE**
    - **CCI_ER_DBMS**
    - **CCI_ER_COMMUNICATION**
    - **CCI_ER_NO_MORE_MEMORY**
    - **CCI_ER_CONNECT**
    - **CCI_ER_QUERY_TIMEOUT**
    - **CCI_ER_LOGIN_TIMEOUT**

Executes *sql_stmt* as many times as *num_sql_stmt* specified as a parameter and returns the number of queries executed with the query_result variable. You can use `CCI_QUERY_RESULT_RESULT`, c:macro:*CCI_QUERY_RESULT_ERR_NO*, `CCI_QUERY_RESULT_ERR_MSG` and `CCI_QUERY_RESULT_STMT_TYPE` macros to get the result of each query. Regarding the summary of these macros, see the `cci_execute_array()` function.

Note that the validity check is not performed for each parameter entered in the macro.

After using the *query_result* variable, you must delete the query result by using the `cci_query_result_free()` function.

If autocommit mode is on, each query in the array is committed after executing.

Note

- In the previous version of 2008 R4.3, if the autocommit is on, all queries in the array were committed after all of them are executed. From 2008 R4.3 version, each query in the array is committed right after each running.
- If autocommit mode is off, after the general error occurs during executing one of the queries in the array on the cci_execute_batch function which does a batch processing of the queries, the query with an error is ignored and the next query is executed. But if the deadlock occurs, the error occurs as rolling back the transaction.

```
...
char **queries;
T_CCI_QUERY_RESULT *result;
int n_queries, n_executed;
...
count = 3;
queries = (char **) malloc (count * sizeof (char *));
queries[0] =
    "insert into athlete(name, gender, nation_code, event)
values('Ji-sung Park', 'M', 'KOR', 'Soccer')";
queries[1] =
    "insert into athlete(name, gender, nation_code, event)
values('Joo-young Park', 'M', 'KOR', 'Soccer')";
queries[2] =
    "select * from athlete order by code desc limit 2";

//calling cci_execute_batch()
n_executed = cci_execute_batch (con, count, queries, &result,
&cci_error);
if (n_executed < 0)
{
    printf ("execute_batch: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
    goto handle_error;
}
printf ("%d statements were executed.\n", n_executed);

for (i = 1; i <= n_executed; i++)
{
    printf ("query %d\n", i);
    printf ("result count = %d\n", CCI_QUERY_RESULT_RESULT
(result, i));
    printf ("error message = %s\n", CCI_QUERY_RESULT_ERR_MSG
(result, i));
    printf ("statement type = %d\n",
```

```
            CCI_QUERY_RESULT_STMT_TYPE (result, i));
}

error = cci_query_result_free (result, n_executed);
if (error < 0)
{
    printf ("query_result_free: %d\n", error);
    goto handle_error;
}
...
```

cci_execute_result

int cci_execute_result(int req_handle, T_CCI_QUERY_RESULT **query_result, T_CCI_ERROR *err_buf)

The **cci_execute_result** function stores the execution results (e.g. statement type, result count) performed by `cci_execute()` to the array of **T_CCI_QUERY_RESULT** structure. You can use `CCI_QUERY_RESULT_RESULT`, `CCI_QUERY_RESULT_ERR_NO`, `CCI_QUERY_RESULT_ERR_MSG`, `CCI_QUERY_RESULT_STMT_TYPE` macros to get the results of each query. Regarding the summary of these macros, see the `cci_execute_array()` function.

Note that the validity check is not performed for each parameter entered in the macro.

The memory of used query results must be released by the `cci_query_result_free()` function.

Parameters:

- **req_handle** – (IN) Request handle of the prepared statement
- **query_result** – (OUT) Query results
- **err_buf** – (OUT) Database error buffer

Returns:

Success: The number of queries, Failure: Negative number

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_COMMUNICATION**

```
...
T_CCI_QUERY_RESULT *qr;
```

```
...

cci_execute( ... );
res = cci_execute_result(req_h, &qr, &err_buf);
if (res < 0)
{
    /* error */
}
else
{
    for (i=1 ; i <= res ; i++)
    {
        result_count = CCI_QUERY_RESULT_RESULT(qr, i);
        stmt_type = CCI_QUERY_RESULT_STMT_TYPE(qr, i);
    }
    cci_query_result_free(qr, res);
}
...
```

cci_fetch

int cci_fetch(int req_handle, T_CCI_ERROR *err_buf)

The **cci_fetch** function fetches the query result executed by `cci_execute()` from the server-side CAS and stores it to the client buffer. The `cci_get_data()` function can be used to identify the data of a specific column from the fetched query result.

Parameters:

- **req_handle** – (IN) Request handle
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**
- **CAS_ER_HOLDABLE_NOT_ALLOWED**
- **CCI_ER_NO_MORE_DATA**
- **CCI_ER_RESULT_SET_CLOSED**
- **CCI_ER_DELETED_TUPLE**
- **CCI_ER_COMMUNICATION**

- **CCI_ER_NO_MORE_MEMORY**

cci_fetch_buffer_clear

int cci_fetch_buffer_clear(int req_handle)

The **cci_fetch_buffer_clear** function clears the records temporarily stored in the client buffer.

Parameters:

- **req_handle** – Request handle

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**

cci_fetch_sensitive

int cci_fetch_sensitive(int req_handle, T_CCI_ERROR *err_buf)

The **cci_fetch_sensitive** function sends changed values for sensitive column when the **SELECT** query result is delivered. If the results by *req_handle* are not sensitive, they are same as the ones by `cci_fetch()`. The return value of **CCI_ER_DELETED_TUPLE** means that the given row has been deleted.

Parameters:

- **req_handle** – (IN) Request handle
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_NO_MORE_DATA**
- **CCI_ER_COMMUNICATION**

- **CCI_ER_DBMS**
- **CCI_ER_DELETED_TUPLE**

sensitive column means the item that can provide updated value in the **SELECT** list when you re-request the results. For example, if a column is directly used as an item of the **SELECT** list without aggregation operation, this column can be called a sensitive column.

When you fetch the result again, the sensitive result receive the data from the server, not from the client buffer.

cci_fetch_size

int cci_fetch_size(int req_handle, int fetch_size)

This function is deprecated. Even if it's called, there will be ignored.

cci_get_autocommit

CCI_AUTOCOMMIT_MODE cci_get_autocommit(int conn_handle)

The **cci_get_autocommit** function returns the autocommit mode currently configured.

Parameters:

- **conn_handle** – (IN) Connection handle

Returns:

- **CCI_AUTOCOMMIT_TRUE**: Autocommit mode is ON
- **CCI_AUTOCOMMIT_FALSE**: Autocommit mode is OFF
- **CCI_ER_CON_HANDLE**
- **CCI_ER_USED_CONNECTION**

cci_get_bind_num

int cci_get_bind_num(int req_handle)

The **cci_get_bind_num** function gets the number of input bindings. If the SQL statement used during preparation is composed of multiple queries, it represents the number of input bindings used in all queries.

Parameters:

- **req_handle** – (IN) Request handle for the prepared statement

Returns:

The number of input bindings

- **CCI_ER_REQ_HANDLE**

cci_get_cas_info

int cci_get_cas_info(int conn_handle, char *info_buf, int buf_length, T_CCI_ERROR *err_buf)

Retrieve CAS information which is connected to conn_handle. The string of the below format is returned to the info_buf.

```
<host>:<port>,<cas id>,<cas process id>
```

The below is an example.

```
127.0.0.1:33000,1,12916
```

Through CAS ID, you can check the SQL log file of this CAS easily.

For details, see Checking SQL Log.

Parameters:

- **conn_handle** – (IN) connection handle
- **info_buf** – (OUT) connection information buffer
- **buf_length** – (IN) buffer length of the connection information
- **err_buf** – (OUT) Error buffer

Returns:

error code

- **CCI_ER_INVALID_ARGS**
- **CCI_ER_CON_HANDLE**

cci_get_class_num_objs

int cci_get_class_num_objs(int conn_handle, char *class_name, int flag, int *num_objs, int *num_pages, T_CCI_ERROR *err_buf)

The **cci_get_class_num_objs** function gets the number of objects of the *class_name* class and the number of pages being used. If the flag is configured to 1, an approximate value is fetched; if it is configured to 0, an exact value is fetched.

Parameters:

- **conn_handle** – (IN) Connection handle
- **class_name** – (IN) Class name
- **flag** – (IN) 0 or 1
- **num_objs** – (OUT) The number of objects
- **num_pages** – (OUT) The number of pages
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

CCI_GET_COLLECTION_DOMAIN

CCI_GET_COLLECTION_DOMAIN(u_type)

If *u_type* is set, multiset, or sequence type, this macro gets the domain of the set, multiset or sequence. If *u_type* is not a set type, the return value is the same as *u_type*.

Parameters:

- **u_type** (Type) – (IN)

Returns:

Type (CCI_U_TYPE)

cci_get_cur_oid

int cci_get_cur_oid(int req_handle, char *oid_str_buf)

The **cci_get_cur_oid** function gets OID of the currently fetched records if **CCI_INCLUDE_OID** is configured in execution. The OID is represented in string for a page, slot, or volume.

Parameters:

- **conn_handle** – (IN) Request handle
- **oid_str_buf** – (OUT) OID string

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**

cci_get_data

int cci_get_data(int req_handle, int col_no, int type, void *value, int *indicator)

The **cci_get_data** function gets the *col_no* -th value from the currently fetched result.

Parameters:

- **req_handle** – (IN) Request handle
- **col_no** – (IN) One-based column index. It starts with 1.
- **type** – (IN) Data type (defined in the **T_CCI_A_TYPE**) of *value* variable

- **value** – (OUT) Variable address for data to be stored. If *type* is one of (CCI_A_TYPE_STR, CCI_A_TYPE_SET, CCI_A_TYPE_BLOB or CCI_A_TYPE_CLOB) and the value of a column is NULL, the *value* will be NULL, too.
- **indicator** –

  (OUT) **NULL** indicator. (-1 : **NULL**)

  - If *type* is **CCI_A_TYPE_STR** : -1 is returned in case of **NULL**; the length of string stored in *value* is returned, otherwise.
  - If *type* is not **CCI_A_TYPE_STR** : -1 is returned in case of **NULL**, 0 is returned, otherwise.

Returns:

Error code (0: success)

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_TYPE_CONVERSION**
- **CCI_ER_COLUMN_INDEX**
- **CCI_ER_ATYPE**

The *type* of the *value* variable is determined based on the given *type* argument, and the value or the pointer is copied to the *value* variable accordingly. For a value to be copied, the memory for the address to be transferred to the *value* variable must have been previously assigned. Note that if a pointer is copied, a pointer in the application client library is returned, so the value becomes invalid next time the `cci_get_data()` function is called.

In addition, the pointer returned by the pointer copy must not be freed. However, if the type is **CCI_A_TYPE_SET**, the memory must be freed by using the `cci_set_free()` function after using the set because the set is returned after the **T_CCI_SET** type memory is allocated. The following table shows the summary of *type* arguments and data types of their corresponding *value* values.

type	value Type	Meaning
CCI_A_TYPE_STR	char **	pointer copy
CCI_A_TYPE_INT	int *	value copy
CCI_A_TYPE_FLOAT	float *	value copy
CCI_A_TYPE_DOUBLE	double *	value copy
CCI_A_TYPE_BIT	**T_CCI_BIT** *	value copy (pointer copy for each member)
CCI_A_TYPE_SET	**T_CCI_SET** *	memory allocation and value copy
CCI_A_TYPE_DATE	**T_CCI_DATE** *	value copy
CCI_A_TYPE_BIGINT	int64_t * (For Windows: __int64 *)	value copy
CCI_A_TYPE_BLOB	**T_CCI_BLOB** *	memory allocation and value copy
CCI_A_TYPE_CLOB	**T_CCI_CLOB** *	memory allocation and value copy

Remark

- For **LOB** type, if the `cci_get_data()` function is called, meta data with the **LOB** type column (locator) is displayed. To call data of the **LOB** type column, the `cci_blob_read()` function should be called.

The below example shows a part of a code to print out the fetched result with `cci_get_data()`.

```
...

if ((res=cci_get_data(req, i, CCI_A_TYPE_INT, &buffer, &ind))<0)
{
    printf( "%s(%d): cci_get_data fail\n", __FILE__, __LINE__);
    goto handle_error;
}
if (ind != -1)
        printf("%d \t|", buffer);
else
        printf("NULL \t|");
...
```

cci_get_db_parameter

int cci_get_db_parameter(int conn_handle, T_CCI_DB_PARAM param_name, void *value, T_CCI_ERROR *err_buf)

The **cci_get_db_parameter** function gets a parameter value specified in the database.

Parameters:

- **conn_handle** – (IN) Connection handle
- **param_name** – (IN) System parameter name
- **value** – (OUT) Parameter value
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_PARAM_NAME**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

The data type of *value* for *param_name* is shown in the table below.

param_name	value Type	note
CCI_PARAM_ISOLATION_LEVEL	int *	get/set
CCI_PARAM_LOCK_TIMEOUT	int *	get/set
CCI_PARAM_MAX_STRING_LENGTH	int *	get only
CCI_PARAM_AUTO_COMMIT	int *	get only

In `cci_get_db_parameter()` and `cci_set_db_parameter()`, the input/output unit of **CCI_PARAM_LOCK_TIMEOUT** is milliseconds.

> **Warning**
>
> In the earlier version of CUBRID 9.0, you should be careful that the output unit of **CCI_PARAM_LOCK_TIMEOUT** is second.

CCI_PARAM_MAX_STRING_LENGTH is measured in bytes and it gets a value defined in the **MAX_STRING_LENGTH** broker parameter.

cci_get_db_version

int cci_get_db_version(int conn_handle, char *out_buf, int out_buf_size)

The **cci_get_db_version** function gets the Database Management System (DBMS) version.

Parameters:

- **conn_handle** – (IN) Connection handle
- **out_buf** – (OUT) Result buffer
- **out_buf_size** – (IN) *out_buf* size

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

cci_get_err_msg

int cci_get_err_msg(int err_code, char *msg_buf, int msg_buf_size)

The **cci_get_err_msg** function stores error messages corresponding to the error code in the error message buffer. For details on error codes and error messages, see CCI Error Codes and Error Messages.

Parameters:

- **err_code** – (IN) Error code
- **msg_buf** – (OUT) Error message buffer
- **msg_buf_size** – (IN) *msg_buf* size

Returns:

0: Success, -1: Failure

note:: From CUBRID 9.1, CUBRID ensures that all functions which have err_buf parameter store an error value into err_buf parameter, so you don't need to use cci_get_err_msg function if a function has err_buf parameter.

```
req = cci_prepare (con, query, 0, &err_buf);
if (req < 0)
{
    printf ("error: %d, %s\n", err_buf.err_code,
err_buf.err_msg);
    goto handle_error;
}
```

On the previous version of 9.1, an err_buf value was stored only when CCI_ER_DBMS error occurred. So printing an error message should have been separated by CCI_ER_DBMS error.

```
req = cci_prepare (con, query, 0, &err_buf);
if (req < 0)
{
    if (req == CCI_ER_DBMS)
    {
        printf ("error: %s\n", err_buf.err_msg);
    }
    else
    {
        char msg_buf[1024];
        cci_get_err_msg(req, msg_buf, 1024);
        printf ("error: %s\n", msg_buf);
    }
    goto handle_error;
}
```

From 9.1, you can simplify the branch of the above code by using cci_get_error_msg function.

```
req = cci_prepare (con, query, 0, &err_buf);
if (req < 0)
{
    char msg_buf[1024];
    cci_get_error_msg(req, err_buf, msg_buf, 1024);
    printf ("error: %s\n", msg_buf);
    goto handle_error;
}
```

cci_get_error_msg

int cci_get_error_msg(int err_code, T_CCI_ERROR *err_buf, char *msg_buf, int msg_buf_size)

Saves the error messages corresponding to the CCI error codes in the message buffer. If the value of CCI error code is **CCI_ER_DBMS**, the database error buffer (*err_buf*) receives the error message sent from the data server and saves it in the message buffer(*msg_buf*). For details on error codes and messages, see CCI Error Codes and Error Messages.

Parameters:

- **err_code** – (IN) Error code
- **err_buf** – (IN) Database error buffer
- **msg_buf** – (OUT) Error message buffer
- **msg_buf_size** – (IN) *msg_buf* size

Returns:

0: Success, -1: Failure

cci_get_holdability

int cci_get_holdability(int conn_handle)

Returns the cursor holdability setting value about the result set from the connection handle. When it is 1, the connection is disconnected or the cursor is holdable until the result set is intentionally closed regardless of commit. When it is 0, the result set is closed when committed and the cursor is not holdable. For more details on cursor holdability, see Cursor Holdability.

Parameters:

- **conn_handle** – (IN) Connection handle

Returns:

0 (not holdable), 1 (holdable)

- **CCI_ER_CON_HANDLE**

cci_get_last_insert_id

int cci_get_last_insert_id(int conn_handle, void *value, T_CCI_ERROR *err_buf)

Gets the primary key of the INSERT statement which executed at the last time.

Parameters:

- **conn_handle** – (IN) Request handle
- **value** – (OUT) The pointer of the result buffer pointer(char **). It stores the last primary key of INSERT statement executed on the last time. The memory which this pointer indicates doesn't need to be released, because it is the fixed buffer inside the connection handle.
- **err_buf** – (OUT) Error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_USED_CONNECTION**
- **CCI_ER_INVALID_ARGS**

```
#include <stdio.h>
#include "cas_cci.h"

int main ()
{
    int con = 0;
    int req = 0;

    int error;
    T_CCI_ERROR cci_error;


    char *query = "insert into t1 values(NULL);";
    char *value = NULL;

    con = cci_connect ("localhost", 33000, "demodb", "dba", "");

    if (con < 0)
    {
        printf ("con error\n");
        return;
    }

    req = cci_prepare (con, query, 0, &cci_error);
    if (req < 0)
    {
        printf ("cci_prepare error: %d\n", req);
        printf ("cci_error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
        return;
    }
    error = cci_execute (req, 0, 0, &cci_error);
    if (error < 0)
    {
        printf ("cci_execute error: %d\n", error);
        return;
    }

    error = cci_get_last_insert_id (con, &value, &cci_error);
```

```
    if (error < 0)
    {
        printf ("cci_get_last_insert_id error: %d\n", error);
        return;
    }

    printf ("## last insert id: %s\n", value);

    error = cci_close_req_handle (req);
    error = cci_disconnect (con, &cci_error);
    return 0;
}
```

cci_get_login_timeout

int cci_get_login_timeout(int conn_handle, int *timeout, T_CCI_ERROR *err_buf)

Return login timeout value to *timeout*.

Parameters:

- **conn_handle** – (IN) Connection handle
- **timeout** – (OUT) A pointer to login timeout value(unit: millisecond)
- **err_buf** – (OUT) Error buffer

Returns:

Error code (0: success)

- **CCI_ER_INVALID_ARGS**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_USED_CONNECTION**

cci_get_query_plan

int cci_get_query_plan(int req_handle, char **out_buf_p)

Saves the query plan to the result buffer; the query plan about the request handle which the cci_prepare function returned. You can call this function whether to call the cci_execute function or not.

After calling the cci_get_query_plan function, if the use of a result buffer ends, you should call the `cci_query_info_free()` function to release the result buffer created by cci_get_query_plan function.

```
char *out_buf;
...
req = cci_prepare (con, query, 0, &cci_error);
...
ret = cci_get_query_plan(req, &out_buf);
...
printf("plan = %s", out_buf);
cci_query_info_free(out_buf);
```

Parameters:

- **req_handle** – (IN) Request handle
- **out_buf_p** – (OUT) The pointer of a result buffer pointer

Returns:

Error code

- **CCI_ER_REQ_HANDLE**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_USED_CONNECTION**

See also

`cci_query_info_free()`

cci_query_info_free

int cci_query_info_free(char *out_buf)

Releases the result buffer memory allocated from the `cci_get_query_plan()` function.

Parameters:

- **req_handle** – (IN) Request handle
- **out_buf** – (OUT) Result buffer pointer

Returns:

Error code

- **CCI_ER_NO_MORE_MEMORY**

See also

`cci_get_query_plan()`

cci_get_query_timeout

int cci_get_query_timeout(int req_handle)

The **cci_get_query_timeout** function returns timeout configured for query execution.

Parameters:

- **req_handle** – (IN) Request handle

Returns:

Success: Timeout value configured in current request handle (unit: msec.), Failure: Error code

- CCI_ER_REQ_HANDLE

cci_get_result_info

T_CCI_COL_INFO *cci_get_result_info(int req_handle, T_CCI_CUBRID_STMT *stmt_type, int *num)

If the prepared statement is **SELECT**, the **T_CCI_COL_INFO** struct that stores the column information about the execution result can be obtained by using this function. If it is not **SELECT**, **NULL** is returned and the *num* value becomes 0.

Parameters:

- **req_handle** – (IN) Request handle for the prepared statement
- **stmt_type** – (OUT) Command type
- **num** – (OUT) he number of columns in the **SELECT** statement (if *stmt_type* is **CUBRID_STMT_SELECT**)

Returns:

Success: Result info pointer, Failure: **NULL**

You can access the **T_CCI_COL_INFO** struct directly to get the column information from the struct, but you can also use a macro to get the information, which is defined as follows. The address of the **T_CCI_COL_INFO** struct and the column index are specified as parameters for each macro. The macro can be called only for the **SELECT** query. Note that the validity check is not performed for each parameter entered in each macro. If the return type of the macro is char*, do not free the memory pointer.

Macro	Return Type	Meaning
`CCI_GET_RESULT_INFO_TYPE`	**T_CCI_U_TYPE**	column type
`CCI_GET_RESULT_INFO_SCALE`	short	column scale
`CCI_GET_RESULT_INFO_PRECISION`	int	column precision
`CCI_GET_RESULT_INFO_NAME`	char *	column name
`CCI_GET_RESULT_INFO_ATTR_NAME`	char *	column attribute name
`CCI_GET_RESULT_INFO_CLASS_NAME`	char *	column class name
`CCI_GET_RESULT_INFO_IS_NON_NULL`	char (0 or 1)	whether a column is NULL

```
col_info = cci_get_result_info (req, &stmt_type, &col_count);
if (col_info == NULL)
{
  printf ("get_result_info error\n");
  goto handle_error;
}

for (i = 1; i <= col_count; i++)
{
  printf ("%-12s = %d\n", "type", CCI_GET_RESULT_INFO_TYPE
(col_info, i));
  printf ("%-12s = %d\n", "scale",
          CCI_GET_RESULT_INFO_SCALE (col_info, i));
  printf ("%-12s = %d\n", "precision",
          CCI_GET_RESULT_INFO_PRECISION (col_info, i));
  printf ("%-12s = %s\n", "name", CCI_GET_RESULT_INFO_NAME
(col_info, i));
  printf ("%-12s = %s\n", "attr_name",
          CCI_GET_RESULT_INFO_ATTR_NAME (col_info, i));
  printf ("%-12s = %s\n", "class_name",
          CCI_GET_RESULT_INFO_CLASS_NAME (col_info, i));
  printf ("%-12s = %s\n", "is_non_null",
          CCI_GET_RESULT_INFO_IS_NON_NULL (col_info,i) ?
"true" : "false");
}
```

CCI_GET_RESULT_INFO_ATTR_NAME

CCI_GET_RESULT_INFO_ATTR_NAME(res_info, index)

The **CCI_GET_RESULT_INFO_ATTR_NAME** macro gets the actual attribute name of the *index*-th column of a prepared **SELECT** list. If there is no name for the attribute (constant, function, etc), " " (empty string) is returned. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid. You cannot delete the returned memory pointer with **free**().

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) A pointer to the column information fetched by `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

Attribute name (char *)

CCI_GET_RESULT_INFO_CLASS_NAME

CCI_GET_RESULT_INFO_CLASS_NAME(res_info, index)

The **CCI_GET_RESULT_INFO_CLASS_NAME** macro gets the *index*-th column's class name of a prepared **SELECT** list. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid. You cannot delete the returned memory pointer with **free** (). The return value can be **NULL**.

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) Column info pointer by `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

Class name (char *)

CCI_GET_RESULT_INFO_IS_NON_NULL

CCI_GET_RESULT_INFO_IS_NON_NULL(res_info, index)

The **CCI_GET_RESULT_INFO_IS_NON_NULL** macro gets a value indicating whether the *index*-th column of a prepared **SELECT** list is nullable. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid.

When a column of a **SELECT** list is not a column but an expression, CUBRID cannot judge it's NON_NULL or not; therefore, CCI_GET_RESULT_INFO_IS_NON_NULL macro always returns 0.

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) Column info pointer by `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

0: nullable, 1: non **NULL**

CCI_GET_RESULT_INFO_NAME

CCI_GET_RESULT_INFO_NAME(res_info, index)

The **CCI_GET_RESULT_INFO_NAME** macro gets the *index*-th column's name of a prepared **SELECT** list. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid. You cannot delete the returned memory pointer with **free** ().

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) Column info pointer to `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

Column name (char *)

CCI_GET_RESULT_INFO_PRECISION

CCI_GET_RESULT_INFO_PRECISION(res_info, index)

The **CCI_GET_RESULT_INFO_PRECISION** macro gets the *index*-th column's precision of a prepared **SELECT** list. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid.

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) Column info pointer by `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

precision (int)

CCI_GET_RESULT_INFO_SCALE

CCI_GET_RESULT_INFO_SCALE(res_info, index)

The **CCI_GET_RESULT_INFO_SCALE** macro gets the *index*-th column's scale of a prepared **SELECT** list. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid.

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) Column info pointer by `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

scale (int)

CCI_GET_RESULT_INFO_TYPE

CCI_GET_RESULT_INFO_TYPE(res_info, index)

The **CCI_GET_RESULT_INFO_TYPE** macro gets the *index*-th column's type of a prepared **SELECT** list. It does not check whether the specified argument, *res_info*, is **NULL** and whether *index* is valid.

If you want to check which column is a SET type or not, use `CCI_IS_SET_TYPE`.

Parameters:

- **res_info** (T_CCI_COL_INFO*) – (IN) pointer to the column information fetched by `cci_get_result_info()`
- **index** (int) – (IN) Column index

Returns:

Column type (**T_CCI_U_TYPE**)

CCI_IS_SET_TYPE

CCI_IS_SET_TYPE(u_type)

The CCI_IS_SET_TYPE macro check whether *u_type* is set type.

Parameters:

- **u_type** (Type) – (IN)

Returns:

1 : set, 0 : not set

CCI_IS_MULTISET_TYPE

CCI_IS_MULTISET_TYPE(u_type)

The CCI_IS_SET_TYPE macro check whether *u_type* is multiset type.

Parameters:

- **u_type** (Type) – (IN)

Returns:

1 : multiset, 0 : not multiset

CCI_IS_SEQUENCE_TYPE

CCI_IS_SEQUENCE_TYPE(u_type)

The CCI_IS_SET_TYPE macro check whether *u_type* is sequence type.

Parameters:

- **u_type** (Type) – (IN)

Returns:

1 : sequence, 0 : not sequence

CCI_IS_COLLECTION_TYPE

CCI_IS_COLLECTION_TYPE(u_type)

The CCI_IS_SET_TYPE macro check whether *u_type* is collection (set, multiset, sequence) type.

Parameters:

- **u_type** (Type) – (IN)

Returns:

1 : collection (set, multiset, sequence), 0 : not collection

cci_get_version

int cci_get_version(int *major, int *minor, int *patch)

The cci_get_version function gets the version of CCI library. In case of version "9.2.0.0001", 9 is the major version, 2 is the minor version, and 0 is the patch version.

Parameters:

- **major** – (OUT) major version
- **minor** – (OUT) minor version
- **patch** – (OUT) patch version

Returns:

Zero without exception (success)

Note

In CUBRID for Linux, you can check the file version of CCI library by using the **strings** command.

```
$ strings /home/usr1/CUBRID/lib/libcascci.so | grep VERSION
VERSION=9.2.0.0001
```

cci_init

void cci_init()

If you compile the CCI application program for Windows with static linking library(.lib), this function should be called always. In the other cases, this does not need to be called.

cci_is_holdable

int cci_is_holdable(int req_handle)

The **cci_is_holdable** function returns whether the request handle(req_handle) is holdable or not.

Parameters:

- **req_handle** – (IN) Request handle for the prepared statement

Returns:

- 1: holdable
- 0: not holdable
- **CCI_ER_REQ_HANDLE**

See also

`cci_prepare()`

cci_is_updatable

int cci_is_updatable(int req_handle)

The **cci_is_updatable** function checks the SQL statement executing `cci_prepare()` can make updatable result set (which means CCI_PREPARE_UPDATABLE is configured in *flag* when executing `cci_prepare()`).

Parameters:

- **req_handle** – (IN) Request handle for the prepared statement

Returns:

- 1 : updatable
- 0 : not updatable

- **CCI_ER_REQ_HANDLE**

cci_next_result

int cci_next_result(int req_handle, T_CCI_ERROR *err_buf)

The **cci_next_result** function gets results of next query if **CCI_EXEC_QUERY_ALL** *flag* is set upon `cci_execute()`. The information about the query fetched by next_result can be obtained with `cci_get_result_info()`. If next_result is executed successfully, the database is updated with the information of the current query.

You can execute multiple queries in the written order when **CCI_EXEC_QUERY_ALL** *flag* is set on `cci_execute()`. At this time, CUBRID brings the first query's result after calling `cci_execute()`; CUBRID brings the second and the other queries' results after calling **cci_next_result** function. At this time, the column information about the query result can be brought by calling `cci_get_result_info()` function whenever `cci_execute()` function or **cci_next_result** function is called.

In other words, when you run Q1, Q2, and Q3 at once with calling `cci_prepare()` function, the result of Q1 is brought by calling `cci_execute()`; the result of Q2 or Q3 is brought by calling **cci_next_result**. The result of Q1, Q2 or Q3 is brought by calling `cci_get_result_info()` function for each time.

```
sql = "SELECT * FROM athlete; SELECT * FROM nation; SELECT *
FROM game";
req = cci_prepare (con, sql, 0, &error);
...
ret = cci_execute (req, CCI_EXEC_QUERY_ALL, 0, &error);
res_col_info = cci_get_result_info (req, &cmd_type, &col_count);
...
ret = cci_next_result (req, &error);
res_col_info = cci_get_result_info (req, &cmd_type, &col_count);
...
ret = cci_next_result (req, &error);
res_col_info = cci_get_result_info (req, &cmd_type, &col_count);
```

Parameters:

- **req_handle** – (IN) Request handle of a prepared statement
- **err_buf** – (OUT) Database error buffer

Returns:

- **SELECT** : The number of results
- **INSERT**, **UPDATE** : The number of records reflected
- Others : 0
- Failure : Error code
  - **CCI_ER_REQ_HANDLE**
  - **CCI_ER_DBMS**
  - **CCI_ER_COMMUNICATION**

The error code **CAS_ER_NO_MORE_RESULT_SET** means that no more result set exists.

cci_oid

int cci_oid(int conn_handle, T_CCI_OID_CMD cmd, char *oid_str, T_CCI_ERROR *err_buf)

By the value of *cmd* argument, it executes the following behavior.

- CCI_OID_DROP: Deletes the given oid.
- CCI_OID_IS_INSTANCE: Checks whether the given oid is an instance oid.
- CCI_OID_LOCK_READ: Sets read lock on the given oid.
- CCI_OID_LOCK_WRITE: Sets write lock on the given oid.

Parameters:

- **conn_handle** – (IN) Connection handle
- **cmd** – (IN) CCI_OID_DROP, CCI_OID_IS_INSTANCE, CCI_OID_LOCK_READ, CCI_OID_LOCK_WRITE
- **oid_str** – (IN) oid
- **err_buf** – (OUT) Database error buffer

Returns:

- when *cmd* is CCI_OID_IS_INSTANCE
  - 0 : Non-instance

- 1 : Instance
- < 0 : error

- when *cmd* is CCI_OID_DROP, CCI_OID_LOCK_READ or CCI_OID_LOCK_WRITE

  Error code (0: success)

  - **CCI_ER_CON_HANDLE**
  - **CCI_ER_CONNECT**
  - **CCI_ER_OID_CMD**
  - **CCI_ER_OBJECT**
  - **CCI_ER_DBMS**

cci_oid_get

int cci_oid_get(int conn_handle, char *oid_str, char **attr_name, T_CCI_ERROR *err_buf)

The **cci_oid_get** function gets the attribute values of the given oid. *attr_name* is an array of the attributes, and it must end with **NULL**. If *attr_name* is NULL, the information of all attributes is fetched. The request handle has the same form as when the SQL statement "SELECT attr_name FROM oid_class WHERE oid_class = oid" is executed.

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **attr_name** – (IN) A list of attributes
- **err_buf** – (OUT) Database error buffer

Returns:

Success: Request handle, Failure: Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_CONNECT**

cci_oid_get_class_name

int cci_oid_get_class_name(int conn_handle, char *oid_str, char *out_buf, int out_buf_len, T_CCI_ERROR *err_buf)

The **cci_oid_get_class_name** function gets the class name of the given oid.

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **out_buf** – (OUT) Out buffer
- **out_buf_len** – (IN) *out_buf* length
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_OBJECT**
- **CCI_ER_DBMS**

cci_oid_put

int cci_oid_put(int conn_handle, char *oid_str, char **attr_name, char **new_val_str, T_CCI_ERROR *err_buf)

The **cci_oid_put** function configures the *attr_name* attribute values of the given oid to *new_val_str*. The last value of *attr_name* must be **NULL**. Any value of any type must be represented as a string. The value represented as a string is applied to the database after being converted depending on the attribute type on the server. To insert a **NULL** value, configure the value of *new_val_str* [i] to **NULL**.

Parameters:

- **conn_handle** – (IN) Connection handle
- **oid_str** – (IN) oid
- **attr_name** – (IN) The list of attribute names
- **new_val_str** – (IN) The list of new values
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**

cci_oid_put2

int cci_oid_put2(int conn_handle, char *oidstr, char **attr_name, void **new_val, int *a_type, T_CCI_ERROR *err_buf)

The **cci_oid_put2** function sets the *attr_name* attribute values of the given oid to *new_val*. The last value of *attr_name* must be **NULL**. To insert a **NULL** value, set the value of *new_val* [i] to **NULL**.

Parameters:

- **conn_handle** – (IN) Connection handle
- **oidstr** – (IN) oid
- **attr_name** – (IN) A list of attribute names
- **new_val** – (IN) A new value array
- **a_type** – (IN) *new_val* type array

· **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

· **CCI_ER_CON_HANDLE**

· **CCI_ER_CONNECT**

The type of *new_val* [i] for *a_type* is shown in the table below.

Type of new_val[i] for a_type

Type	value type
CCI_A_TYPE_STR	char **
CCI_A_TYPE_INT	int *
CCI_A_TYPE_FLOAT	float *
CCI_A_TYPE_DOUBLE	double *
CCI_A_TYPE_BIT	**T_CCI_BIT** *
CCI_A_TYPE_SET	**T_CCI_SET** *
CCI_A_TYPE_DATE	**T_CCI_DATE** *
CCI_A_TYPE_BIGINT	int64_t * (For Windows: __int64 *)

```
char *attr_name[array_size]
void *attr_val[array_size]
int a_type[array_size]
int int_val
```

```
...

attr_name[0] = "attr_name0"
attr_val[0] = &int_val
a_type[0] = CCI_A_TYPE_INT
attr_name[1] = "attr_name1"
attr_val[1] = "attr_val1"
a_type[1] = CCI_A_TYPE_STR

...
attr_name[num_attr] = NULL

res = cci_put2(con_h, oid_str, attr_name, attr_val, a_type,
&error)
```

cci_prepare

int cci_prepare(int conn_handle, char *sql_stmt, char flag, T_CCI_ERROR *err_buf)

The `cci_prepare()` function prepares SQL execution by acquiring request handle for SQL statements. If a SQL statement consists of multiple queries, the preparation is performed only for the first query. With the parameter of this function, an address to **T_CCI_ERROR** where connection handle, SQL statement, *flag*, and error information are stored.

Parameters:

- **conn_handle** – (IN) Connection handle
- **sql_stmt** – (IN) SQL statement
- **flag** – (IN) prepare flag (CCI_PREPARE_UPDATABLE, CCI_PREPARE_INCLUDE_OID, CCI_PREPARE_CALL or CCI_PREPARE_HOLDABLE)
- **err_buf** – (OUT) Database error buffer

Returns:

Success: Request handle ID (int), Failure: Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_STR_PARAM**

- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_CONNECT**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_LOGIN_TIMEOUT**

CCI_PREPARE_UPDATABLE, **CCI_PREPARE_INCLUDE_OID**, **CCI_PREPARE_CALL** or **CCI_PREPARE_HOLDABLE** can be configured in *flag*. If **CCI_PREPARE_UPDATABLE** is configured, updatable resultset is created and **CCI_PREPARE_INCLUDE_OID** is automatically configured. **CCI_PREPARE_UPDATABLE** and **CCI_PREPARE_HOLDABLE** cannot be used simultaneously in *flag*. If you want to call the Java Stored Procedure, specify **CCI_PREPARE_CALL** flag into the **cci_prepare** function. You can see an related example on `cci_register_out_param()`.

The default value of whether to keep result set after commit is cursor holdability. Thus, if you want to configure **CCI_PREPARE_UPDATABLE** in *flag* of `cci_prepare()`, you should call `cci_set_holdability()` first before calling `cci_prepare()` so that cursor cannot be maintained.

However, not all updatable resultsets are created even though **CCI_PREPARE_UPDATABLE** is configured. So you need to check if the results are updatable by using `cci_is_updatable()` after preparation. You can use `cci_oid_put()` or `cci_oid_put2()` to update result sets.

The conditions of updatable queries are as follows:

- Must be **SELECT**.
- OID can be included in the query result.
- The column to be updated must be the one that belongs to the table specified in the **FROM** clause.

If **CCI_PREPARE_HOLDABLE** is set, a cursor is held as long as result set is closed or connection is disconnected after the statement is committed(see Cursor Holdability).

cci_prepare_and_execute

int cci_prepare_and_execute(int conn_handle, char *sql_stmt, int max_col_size, int *exec_retval, T_CCI_ERROR *err_buf)

The **cci_prepare_and_execute** function executes the SQL statement immediately and returns a request handle for the SQL statement. A request handle, SQL statement, the maximum length of a column to be fetched, error code, and the address of a **T_CCI_ERROR** construct variable in which error information being stored are specified as arguments. *max_col_size* is a value to configure the maximum length of a column to be sent to a client

when the column of a SQL statement is **CHAR**, **VARCHAR**, **BIT**, or **VARBIT**. If this value is 0, full length is fetched.

Parameters:

- **conn_handle** – (IN) Connection handle
- **sql_stmt** – (IN) SQL statement
- **max_col_size** – (IN) The maximum length of a column to be fetched when it is a string data type in bytes. If this value is 0, full length is fetched.
- **exec_retval** – (OUT) Success: Affected rows, Failure: Error code
- **err_buf** – (OUT) Database error buffer

Returns:

Success: Request handle ID (int), Failure: Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_STR_PARAM**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_CONNECT**
- **CCI_ER_QUERY_TIMEOUT**

cci_property_create

T_CCI_PROPERTIES *cci_property_create()

The **cci_property_create** function creates **T_CCI_PROPERTIES** struct to configure DATASOURCE of CCI.

Returns:

Success: **T_CCI_PROPERTIES** struct pointer in which memory is allocated, Failure: **NULL**

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_release()`

cci_property_destroy

void cci_property_destroy(T_CCI_PROPERTIES *properties)

The **cci_property_destroy** function destroys **T_CCI_PROPERTIES** struct.

Parameters:

- **properties** – **T_CCI_PROPERTIES** struct pointer to be destroyed

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_release()`

cci_property_get

char *cci_property_get(T_CCI_PROPERTIES *properties, char *key)

The **cci_property_get** function retrieves the property value configured in the **T_CCI_PROPERTIES** struct.

Parameters:

- **properties** – **T_CCI_PROPERTIES** struct pointer which gets value corresponding to *key*
- **key** – Name of property to be retrieved (For name and description available properties, see the `cci_property_set()`)

Returns:

Success: String pointer of value corresponding to *key*,
Failure: NULL

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_release()`

cci_property_set

int cci_property_set(T_CCI_PROPERTIES *properties, char *key, char *value)

It configures a property value in **T_CCI_PROPERTIES** struct.

Parameters:

- **properties** – **T_CCI_PROPERTIES** struct pointer in which *key* and *value* are stored
- **key** – String pointer of property name
- **value** – String pointer of property value

Returns:

Success: 1, Failure: 0

The property names and its meanings that can be configured in the struct are as follows:

Property name	Type	Default	Description
user	string		DB username.
password	string		DB user password.
url	string		Connection URL. For specifying connection URL

Property name	Type	Default	Description
			string, see `cci_connect_with_url()`.
pool_size	int	10	Maximum number of connections which a connection pool can have.
max_pool_size	int	pool_size	Total number of connections to create when creating an initial datasource. maximum value is INT_MAX.
max_wait	msec	1000	Maximum waiting time to get connection.
pool_prepared_statement	bool	false	Whether to enable statement pooling. true or false.
max_open_prepared_statement	int	1000	The maximum number of prepared statements to maintain in the statement pool.
login_timeout	msec	0(inf.)	Login timeout applied when you create a

Property name	Type	Default	Description
			datasource with `cci_datasource_create()` function or internal timeout occurs in prepare/execute function.
query_timeout	msec	0(inf.)	Query timeout.
disconnect_on_query_timeout	bool	no	Whether to terminate connection when execution is discarded due to query execution timeout. yes or no.
default_autocommit	bool	CCI_DEFAULT_AUTOCOMMIT in cubrid_broker.conf	Autocommit mode specified when a datasource is created by `cci_datasource_create()` function. true or false.
default_isolation	string	isolation_level in cubrid.conf	Transaction isolation level specified when a datasource is created by `cci_datasource_create()` function. See the below table.
default_lock_timeout	msec	lock_timeout in cubrid.conf	lock timeout specified when a datasource is

Property name	Type	Default	Description
			created by `cci_datasource_create()` function.

If the number of prepared statements exceeds **max_open_prepared_statement** value, the oldest prepared statement is released from the statement pool. If you reuse it later, it is added to the statement pool again.

A number of connections which a connection pool can contain is changable by using `cci_datasource_change_property()` function when it's required, but it cannot exceed **max_pool_size**. You can change this number when you want to handle the limitation of the number of connections. For example, usually set the number less than **max_pool_size**; raise the number when more connections are required than expected; shrink the number again when many connections are not required.

The number of connections which a connection pool can contain is limited until **pool_size**; it's maximum value is **max_pool_size**.

When you set **login_timeout**, **default_autocommit**, **default_isolation** or **default_lock_timeout**, these specified values are used in a connection which is returend when calling `cci_datasource_borrow()` function.

login_timeout, **default_autocommit**, **default_isolation** or **default_lock_timeout** can be changed even after calling `cci_datasource_borrow()` function. Regarding this, see `cci_set_login_timeout()`, `cci_set_autocommit()`, `cci_set_isolation_level()`, `cci_set_lock_timeout()`. Changed values by functions which the name starts with **cci_set_** are only applied to the changed connection objects, and after the connection is released, they are restored as the specified values by `cci_property_set()`. It is restored as the default value when there is no value specified by `cci_property_set()` function.

Note

- When specifying values together in `cci_property_set()` function and URL string, values specified in `cci_property_set()` function have the first priority.
- **login_timeout** is applied when you create a DATASOURCE object and internal reconnection occurs in `cci_prepare()` or `cci_execute()` function. Internal reconnection occurs when you get the connection object from DATASOURCE and from `cci_connect()` / `cci_connect_with_url()` function.

- Creating time of a DATASOURCE object can take more than an internal reconnection time; therefore, you can consider to apply **login_timeout** differently on these two cases. For example, if you want to set 5000(5 sec.) in the former and 2000(2 sec.) in the later, for the later, use `cci_set_login_timeout()` after creating a DATASOURCE.

default_isolation has one of the following configuration values. For details on isolation level, see SET TRANSACTION ISOLATION LEVEL.

isolation_level	Configuration Value
SERIALIZABLE	"TRAN_SERIALIZABLE"
REPEATABLE READ	"TRAN_REP_READ"
READ COMMITTED	"TRAN_READ_COMMITTED"

DB user's name and password can be specified by the setting of **user** and **password** directly, or by the setting of **user** and **password** in **url**.

The following shows how to work as the first priority if both are specified.

- If both are specified, the value of direct setting will be first.
- If one of them is NULL, the value which is not NULL is used.
- If both are NULL, they are used as NULL value.
- If the direct setting value of DB user is NULL then "public" is used, else if the direct setting value of the password is NULL then NULL is used.
- If the direct setting value of the password is NULL, it follows the setting of URL.

The following shows that the DB user's name becomes "dba" and the password becomes "cubridpwd".

```
cci_property_set(ps, "user", "dba");
cci_property_set(ps, "password", "cubridpwd");
  ...
cci_property_set(ps, "url", "cci:cubrid:
192.168.0.1:33000:demodb:public:mypwd:?
```

```
logSlowQueries=true&slowQueryThresholdMillis=1000&logTraceApi=tru
e&logTraceNetwork=true");
```

See also

`cci_property_create()`, `cci_property_destroy()`, `cci_property_get()`, `cci_property_set()`, `cci_datasource_create()`, `cci_datasource_destroy()`, `cci_datasource_release()`

cci_query_result_free

int cci_query_result_free(T_CCI_QUERY_RESULT *query_result, int num_query)

The **cci_query_result_free** function releases query results created by `cci_execute_batch()`, `cci_execute_array()` or `cci_execute_result()` function from memory.

Parameters:

- **query_result** – (IN) Query results to release from memory
- **num_query** – (IN) The number of arrays in *query_result*

Returns:

0: success

```
T_CCI_QUERY_RESULT *qr;
char **sql_stmt;

...

res = cci_execute_array(conn, &qr, &err_buf);

...

cci_query_result_free(qr, res);
```

CCI_QUERY_RESULT_ERR_NO

CCI_QUERY_RESULT_ERR_NO(query_result, index)

Since query results performed by `cci_execute_batch()`, `cci_execute_array()`, or `cci_execute_result()` function are stored as an array of **T_CCI_QUERY_RESULT** structure, you need to check the query result for each item of the array.

CCI_QUERY_RESULT_ERR_NO fetches the error number for the array item specified as *index*, if it is not an error, it returns 0.

Parameters:

- **query_result** (T_CCI_QUERY_RESULT*) – (IN) Query result to retrieve
- **index** (int) – (IN) Index of the result array(base :1). It represents a specific location of the result array.

Returns:

Error number

CCI_QUERY_RESULT_ERR_MSG

CCI_QUERY_RESULT_ERR_MSG(query_result, index)

The **CCI_QUERY_RESULT_ERR_MSG** macro gets error messages about query results executed by `cci_execute_batch()`, `cci_execute_array()` or `cci_execute_result()` function. If there is no error message, this macro returns “”(empty string). It does not check whether the specified argument, *query_result*, is **NULL**, and whether *index* is valid.

Parameters:

- **query_result** (T_CCI_QUERY_RESULT*) – (IN) Query results of to be executed
- **index** (int) – (IN) Column index (base: 1)

Returns:

Error message

CCI_QUERY_RESULT_RESULT

CCI_QUERY_RESULT_RESULT(query_result, index)

The **CCI_QUERY_RESULT_RESULT** macro gets the result count executed by `cci_execute_batch()`, `cci_execute_array()` or `cci_execute_result()` function. It does not check whether the specified argument, *query_result*, is **NULL** and whether *index* is valid.

Parameters:

- **query_result** (T_CCI_QUERY_RESULT*) – (IN) Query results to be retrieved
- **index** (int) – (IN) Column index (base: 1)

Returns:

result count

CCI_QUERY_RESULT_STMT_TYPE

CCI_QUERY_RESULT_STMT_TYPE(query_result, index)

Since query results performed by `cci_execute_batch()`, `cci_execute_array()` or `cci_execute_result()` fuction are stored as an array of T_CCI_QUERY_RESULT type, you need to check the query result for each item of the array.

The **CCI_QUERY_RESULT_STMT_TYPE** macro gets the statement type for the array items specified as *index*.

It does not check whether the specified argument, *query_result*, is **NULL** and whether *index* is valid.

Parameters:

- **query_result** (T_CCI_QUERY_RESULT*) – (IN) Query results to be retrieved
- **index** (int) – (IN) Column index (base: 1)

Returns:

statement type (**T_CCI_CUBRID_STMT**)

cci_register_out_param

int cci_register_out_param(int req_handle, int index)

The **cci_register_out_param** function is used to bind parameters as outbind in a stored procedure. The index value starts from 1. To call this function, **CCI_PREPARE_CALL** flag of **cci_prepare** function should be specified.

Parameters:

- **req_handle** – (IN) Request handle

Returns:

Error code

- **CCI_ER_BIND_INDEX**
- **CCI_ER_REQ_HANDLE**
- **CCI_ER_CON_HANDLE**
- **CCI_ER_USED_CONNECTION**

The following shows how to print the string “Hello, CUBRID” string using a stored procedure.

```
$ vi cubrid.conf

$ cubrid service start
$ cubrid server start demodb
```

Implement and compile the class to be used as a stored procedure.

```
public class SpCubrid{
    public static void outTest(String[] o) {
        o[0] = "Hello, CUBRID";
    }
}

%javac SpCubrid.java
```

Load the compiled Java class into CUBRID.

```
$ loadjava demodb SpCubrid.class
```

Register the loaded Java class.

```
    -- csql>

    CREATE PROCEDURE test_out(x OUT STRING)
    AS LANGUAGE JAVA
    NAME 'SpCubrid.outTest(java.lang.String[] o)';
```

On the CCI application program, specify **CCI_PREPARE_CALL** flag into the **cci_prepare** function, and bring the fetching result after setting the position of the outbind parameter by calling **cci_register_out_param**.

```
// On Linux, compile with "gcc -g -o jsp jsp.c -I$CUBRID/
include/ -L$CUBRID/lib/ -lcascci -lpthread"

#include <stdio.h>
#include <unistd.h>
#include <cas_cci.h>
char *cci_client_name = "test";

int
main (int argc, char *argv[])
{
    int con = 0, req = 0, res, ind, i, col_count;
    T_CCI_ERROR error;
    T_CCI_COL_INFO *res_col_info;
    T_CCI_CUBRID_STMT cmd_type;
    char *buffer, db_ver[16];

    if ((con = cci_connect ("localhost", 33000, "demodb", "dba",
"")) < 0)
    {
        printf ("%s(%d): cci_connect fail\n", __FILE__,
__LINE__);
        return -1;
    }

    if ((req = cci_prepare (con, "call test_out(?)",
CCI_PREPARE_CALL, &error)) < 0)
    {
        printf ("%s(%d): cci_prepare fail(%d)\n", __FILE__,
__LINE__,
            error.err_code);
        goto handle_error;
    }
    if ((res = cci_register_out_param (req, 1)) < 0)
    {
        printf ("%s(%d): cci_register_out_param fail(%d)\n",
__FILE__, __LINE__,
            error.err_code);
        goto handle_error;
```

```
    }
    if ((res = cci_execute (req, 0, 0, &error)) < 0)
    {
      printf ("%s(%d): cci_execute fail(%d)\n", __FILE__,
__LINE__,
              error.err_code);
      goto handle_error;
    }
    res = cci_cursor (req, 1, CCI_CURSOR_CURRENT, &error);
    if (res == CCI_ER_NO_MORE_DATA)
    {
        printf ("%s(%d): cci_cursor fail(%d)\n", __FILE__,
__LINE__,
              error.err_code);
        goto handle_error;
    }

    if ((res = cci_fetch (req, &error) < 0))
    {
        printf ("%s(%d): cci_fetch(%d, %s)\n", __FILE__,
__LINE__,
              error.err_code, error.err_msg);
      goto handle_error;
    }
    if ((res = cci_get_data (req, 1, CCI_A_TYPE_STR, &buffer,
&ind)) < 0)
    {
        printf ("%s(%d): cci_get_data fail\n", __FILE__,
__LINE__);
        goto handle_error;
    }
    // ind: string length, buffer: a string which came from the
out binding parameter of test_out(?) Java SP.
    if (ind != -1)
        printf ("%d, (%s)\n", ind, buffer);
    else // if ind== -1, then data is NULL
        printf ("NULL\n");

    if ((res = cci_close_query_result (req, &error)) < 0)
    {
        printf ("%s(%d): cci_close_query_result fail(%d)\n",
__FILE__, __LINE__, error.err_code);
        goto handle_error;
    }

    if ((res = cci_close_req_handle (req)) < 0)
    {
        printf ("%s(%d): cci_close_req_handle fail\n", __FILE__,
__LINE__);
        goto handle_error;
    }
```

```
    if ((res = cci_disconnect (con, &error)) < 0)
    {
        printf ("%s(%d): cci_disconnect fail(%d)\n", __FILE__,
__LINE__,
            error.err_code);
        goto handle_error;
    }
    printf ("Program ended!\n");
    return 0;

handle_error:
    if (req > 0)
        cci_close_req_handle (req);
    if (con > 0)
        cci_disconnect (con, &error);
    printf ("Program failed!\n");
    return -1;
}
```

See also

`cci_prepare()`

cci_row_count

int cci_row_count(int conn_handle, int *row_count, T_CCI_ERROR *err_buf)

The **cci_row_count** function gets the number of rows affected by the last executed query.

Parameters:

- **conn_handle** – (IN) Connection handle
- **row_count** – (OUT) The number of rows affected by the last executed query
- **err_buf** – (OUT) Error buffer

Returns:

Error code

- **CCI_ER_COMMUNICATION**
- **CCI_ER_LOGIN_TIMEOUT**

- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_DBMS**

cci_savepoint

int cci_savepoint(int conn_handle, T_CCI_SAVEPOINT_CMD cmd, char *savepoint_name, T_CCI_ERROR *err_buf)

The **cci_savepoint** function configures savepoint or performs transaction rollback to a specified savepoint. If *cmd* is set to **CCI_SP_SET**, it configures savepoint and if it is set to **CCI_SP_ROLLBACK**, it rolls back transaction to specified savepoint.

Parameters:

- **conn_handle** – (IN) Connection handle
- **cmd** – (IN) CCI_SP_SET or CCI_SP_ROLLBACK
- **savepoint_name** – (IN) Savepoint name
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_QUERY_TIMEOUT**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_DBMS**

```
con = cci_connect( ... );
... /* query execute */

/* Sets the savepoint named "savepoint1" */
cci_savepoint(con, CCI_SP_SET, "savepoint1", err_buf);

... /* query execute */
```

```
/* Rolls back to specified savepoint,"savepoint1" */
cci_savepoint(con, CCI_SP_ROLLBACK, "savepoint1", err_buf);
```

cci_schema_info

int cci_schema_info(int conn_handle, T_CCI_SCHEMA_TYPE type, char *class_name, char *attr_name, char flag, T_CCI_ERROR *err_buf)

The **cci_schema_info** function gets schema information. If it is performed successfully, the results are managed by the request handle and can be fetched by fetch and getdata. If you want to retrieve a *class_name* and *attr_name* by using pattern matching of the **LIKE** statement, you should configure *flag*.

Parameters:

- **conn_handle** – (IN) Connection handle
- **type** – (IN) Schema type
- **class_name** – (IN) Class name or NULL
- **attr_name** – (IN) Attribute name or NULL
- **flag** – (IN) Pattern matching flag (**CCI_CLASS_NAME_PATTERN_MATCH** or **CCI_ATTR_NAME_PATTERN_MATCH**)
- **err_buf** – (OUT) Database error buffer

Returns:

Success: Request handle, Failure: Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_CONNECT**

Two types of *flag* s, **CCI_CLASS_NAME_PATTERN_MATCH**, and **CCI_ATTR_NAME_PATTERN_MATCH**, are used for pattern matching; you can configure these two *flag* s by using the OR operator (|). To use pattern matching, search by using the **LIKE**

statement. For example, to search the information on a column whose *class_name* is "athlete" and *attr_name* is "code," you can enter as follows (in the example, "%code" is entered in the value of *attr_name*).

```
cci_schema_info(conn, CCI_SCH_ATTRIBUTE, "athlete", "%code",
CCI_ATTR_NAME_PATTERN_MATCH, &error);
```

The table below shows record type for each *type*.

Record for Each Type

Type	Column Order	Column Name	Column Type
CCI_SCH_CLASS	1	NAME	char *
	2	TYPE	short • 0: system class • 1: vclass • 2: class • 3: proxy
	3	REMARKS	char *
CCI_SCH_VCLASS	1	NAME	char *
	2	TYPE	short • 1: vclass • 3: proxy
	3	REMARKS	char *
CCI_SCH_QUERY_SPEC	1	QUERY_SPEC	char *

Type	Column Order	Column Name	Column Type
CCI_SCH_ATTRIBUTE	1	NAME	char *
	2	DOMAIN	short
	3	SCALE	short
	4	PRECISION	int
	5	INDEXED	short 1: indexed
	6	NON_NULL	short 1: non null
	7	SHARED	short 1: shared
	8	UNIQUE	short 1: unique
	9	DEFAULT	char *
	10	ATTR_ORDER	int base = 1
	11	CLASS_NAME	char *
	12	SOURCE_CLASS	char *

Type	Column Order	Column Name	Column Type
	13	IS_KEY	short 1: key
	14	REMARKS	char *
CCI_SCH_CLASS_ATTRIBUTE When the attribute of the CCI_SCH_CLASS_ATTRIBUTE column is INSTANCE or SHARED, the order and the name values are identical to those of the column of CCI_SCH_ATTRIBUTE.			
CCI_SCH_CLASS_METHOD	1	NAME	char *
	2	RET_DOMAIN	short
	3	ARG_DOMAIN	char *
CCI_SCH_METHOD_FILE	1	METHOD_FILE	char *
CCI_SCH_SUPERCLASS	1	CLASS_NAME	char *
	2	TYPE	short

Type	Column Order	Column Name	Column Type
CCI_SCH_SUBCLASS	1	CLASS_NAME	char *
	2	TYPE	short
CCI_SCH_CONSTRAINT	1	TYPE	short • 0: unique • 1: index • 2: reverse unique • 3: reverse index
	2	NAME	char *
	3	ATTR_NAME	char *
	4	NUM_PAGES	int
	5	NUM_KEYS	int
	6	PRIMARY_KEY	short • 1: primary key
	7	KEY_ORDER	short base = 1
	8	ASC_DESC	char *
CCI_SCH_TRIGGER	1	NAME	char *

Type	Column Order	Column Name	Column Type
	2	STATUS	char *
	3	EVENT	char *
	4	TARGET_CLASS	char *
	5	TARGET_ATTR	char *
	6	ACTION_TIME	char *
	7	ACTION	char *
	8	PRIORITY	float
	9	CONDITION_TIME	char *
	10	CONDITION	char *
	11	REMARKS	char *
CCI_SCH_CLASS_PRIVILEGE	1	CLASS_NAME	char *
	2	PRIVILEGE	char *
	3	GRANTABLE	char *
CCI_SCH_ATTR_PRIVILEGE	1	ATTR_NAME	char *
	2	PRIVILEGE	char *

Type	Column Order	Column Name	Column Type
	3	GRANTABLE	char *
CCI_SCH_DIRECT_SUPER_CLASS	1	CLASS_NAME	char *
	2	SUPER_CLASS_NAME	char *
CCI_SCH_PRIMARY_KEY	1	CLASS_NAME	char *
	2	ATTR_NAME	char *
	3	KEY_SEQ	int base = 1
	4	KEY_NAME	char *
CCI_SCH_IMPORTED_KEYS Used to retrieve primary key columns that are referred by a foreign key column in a given table. The results are sorted by PKTABLE_NAME and KEY_SEQ. If this type is specified as a parameter, a foreign key table is specified for class_name , and	1	PKTABLE_NAME	char *

Type	Column Order	Column Name	Column Type
NULL is specified for attr_name.	2	PKCOLUMN_NAME	char *
	3	FKTABLE_NAME	char *
	4	FKCOLUMN_NAME	char *
	5	KEY_SEQ	short
	6	UPDATE_ACTION	short • 0: cascade • 1: restrict • 2: no action • 3: set null
	7	DELETE_ACTION	short • 0: cascade • 1: restrict • 2: no action • 3: set null
	8	FK_NAME	char *
	9	PK_NAME	char *
CCI_SCH_EXPORTED_KEYS Used to retrieve primary key	1	PKTABLE_NAME	char *

Type	Column Order	Column Name	Column Type
columns that are referred by all foreign key columns. The results are sorted by FKTABLE_NAME and KEY_SEQ. If this type is specified as a parameter, a primary key table is specified for class_name , and NULL is specified for attr_name.	2	PKCOLUMN_NAME	char *
	3	FKTABLE_NAME	char *
	4	FKCOLUMN_NAME	char *
	5	KEY_SEQ	short
	6	UPDATE_ACTION	short • 0: cascade • 1: restrict • 2: no action • 3: set null
	7	DELETE_ACTION	short • 0: cascade • 1: restrict • 2: no action • 3: set null
	8	FK_NAME	char *
	9	PK_NAME	char *
CCI_SCH_CROSS_REFERENCE Used to retrieve foreign key	1	PKTABLE_NAME	char *

Type	Column Order	Column Name	Column Type
information when primary keys and foreign keys in a given table are cross referenced. The results are sorted by FKTABLE_NAME and KEY_SEQ. If this type is specified as a parameter, a primary key is specified for class_name , and a foreign key table is specified for attr_name.	2	PKCOLUMN_NAME	char *
	3	FKTABLE_NAME	char *
	4	FKCOLUMN_NAME	char *
	5	KEY_SEQ	short
	6	UPDATE_ACTION	short • 0: cascade • 1: restrict • 2: no action • 3: set null
	7	DELETE_ACTION	short • 0: cascade • 1: restrict • 2: no action • 3: set null
	8	FK_NAME	char *
	9	PK_NAME	char *

In the **cci_schema_info** function, the *type* argument supports the pattern matching of the **LIKE** statement for the *class_name* and *attr_name*.

type, class_name(table name), and attr_name(column name) That Supports Pattern Matching

type	class_name	attr_name
CCI_SCH_CLASS (VCLASS)	string or NULL	always NULL
CCI_SCH_ATTRIBUTE (CLASS ATTRIBUTE)	string or NULL	string or NULL
CCI_SCH_CLASS_PRIVILEGE	string or NULL	always NULL
CCI_SCH_ATTR_PRIVILEGE	always NULL	string or NULL
CCI_SCH_PRIMARY_KEY	string or NULL	always NULL
CCI_SCH_TRIGGER	string or NULL	always NULL

- The type with "NULL" in class_name doesn't support the pattern matching about the table name.
- The type with "NULL" in attr_name doesn't support the pattern matching about the column name.
- If the pattern *flag* is not configured, exact matching will be used for the given table and column names; in this case, no result will be returned if the value is **NULL**.
- If *flag* is configured and the value is **NULL**, the result will be the same as when "%" is given in the **LIKE** statement. In other words, the result about all tables or all columns will be returned.

Note

TYPE column of **CCI_SCH_CLASS** and **CCI_SCH_VCLASS**: The proxy type is added. When used in OLEDB, ODBC or PHP, vclass is represented without distinguishing between proxy and vclass.

```
// gcc -o schema_info schema_info.c -m64 -I${CUBRID}/include -
lnsl ${CUBRID}/lib/libcascci.so -lpthread
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "cas_cci.h"

int
main ()
{
  int conn = 0, req = 0, col_count = 0, res = 0, i, ind;
  char *data, query_result_buffer[1024];
  T_CCI_ERROR cci_error;
  T_CCI_COL_INFO *col_info;
  T_CCI_CUBRID_STMT cmd_type;


  conn = cci_connect ("localhost", 33000, "demodb", "dba", "");
  // get all columns' information of table "athlete"
  // req = cci_schema_info(conn, CCI_SCH_ATTRIBUTE, "athlete",
"code", 0, &cci_error);
  req =
    cci_schema_info (conn, CCI_SCH_ATTRIBUTE, "athlete", NULL,
                     CCI_ATTR_NAME_PATTERN_MATCH, &cci_error);

  if (req < 0)
  {
      fprintf (stdout, "(%s, %d) ERROR : %s [%d] \n\n",
__FILE__, __LINE__,
               cci_error.err_msg, cci_error.err_code);
      goto _END;
  }
  col_info = cci_get_result_info (req, &cmd_type, &col_count);
  if (!col_info && col_count == 0)
  {
      fprintf (stdout, "(%s, %d) ERROR :
cci_get_result_info\n\n", __FILE__,
               __LINE__);
      goto _END;
  }
  res = cci_cursor (req, 1, CCI_CURSOR_FIRST, &cci_error);
  if (res == CCI_ER_NO_MORE_DATA)
  {
      goto _END;
  }
  if (res < 0)
  {
      fprintf (stdout, "(%s, %d) ERROR : %s [%d] \n\n",
__FILE__, __LINE__,
               cci_error.err_msg, cci_error.err_code);
      goto _END;
  }
```

```
  while (1)
  {
      res = cci_fetch (req, &cci_error);
      if (res < 0)
      {
          fprintf (stdout, "(%s, %d) ERROR : %s [%d] \n\n",
__FILE__,
                   __LINE__, cci_error.err_msg,
cci_error.err_code);
          goto _END;
      }

      for (i = 1; i <= col_count; i++)
      {
          if ((res = cci_get_data (req, i, CCI_A_TYPE_STR,
&data, &ind)) < 0)
          {
              goto _END;
          }
          if (ind != -1)
          {
              strcat (query_result_buffer, data);
              strcat (query_result_buffer, "|");
          }
          else
          {
              strcat (query_result_buffer, "NULL|");
          }
      }
      strcat (query_result_buffer, "\n");

      res = cci_cursor (req, 1, CCI_CURSOR_CURRENT, &cci_error);
      if (res == CCI_ER_NO_MORE_DATA)
      {
          goto _END;
      }
      if (res < 0)
        {
          fprintf (stdout, "(%s, %d) ERROR : %s [%d] \n\n",
__FILE__,
                   __LINE__, cci_error.err_msg,
cci_error.err_code);
          goto _END;
        }
  }

_END:
  if (req > 0)
    cci_close_req_handle (req);
  if (conn > 0)
    res = cci_disconnect (conn, &cci_error);
```

```
  if (res < 0)
    fprintf (stdout, "(%s, %d) ERROR : %s [%d] \n\n", __FILE__,
__LINE__,
             cci_error.err_msg, cci_error.err_code);

  fprintf (stdout, "Result : %s\n", query_result_buffer);

  return 0;
}
```

cci_set_allocators

int cci_set_allocators(CCI_MALLOC_FUNCTION malloc_func, CCI_FREE_FUNCTION free_func, CCI_REALLOC_FUNCTION realloc_func, CCI_CALLOC_FUNCTION calloc_func)

The **cci_set_allocators** function registers the memory allocation/release functions used by users. By executing this function, you can use user-defined functions for every memory allocation/release jobs being processed in CCI API. If you do not use this function, system functions (malloc, free, realloc, and calloc) are used.

Note

This function can be used only on Linux, so cannot be used on Windows.

Parameters:

- **malloc_func** – (IN) Pointer of externally defined function corresponding to malloc
- **free_func** – (IN) Pointer of externally defined function corresponding to free
- **realloc_func** – (IN) Pointer of externally defined function corresponding to realloc
- **calloc_func** – Pointer of externally defined function corresponding to calloc

Returns:

Error code (0: success)

- **CCI_ER_NOT_IMPLEMENTED**

```
/*
       How to build: gcc -Wall -g -o test_cci test_cci.c -I$
{CUBRID}/include -L${CUBRID}/lib -lcascci
*/

#include <stdio.h>
#include <stdlib.h>
#include "cas_cci.h"

void *my_malloc(size_t size)
{
    printf ("my malloc: size: %ld\n", size);
    return malloc (size);
}

void *my_calloc(size_t nm, size_t size)
{
    printf ("my calloc: nm: %ld, size: %ld\n", nm, size);
    return calloc (nm, size);
}

void *my_realloc(void *ptr, size_t size)
{
    printf ("my realloc: ptr: %p, size: %ld\n", ptr, size);
    return realloc (ptr, size);
}
```

```
void my_free(void *ptr)
{
    printf ("my free: ptr: %p\n", ptr);
    return free (ptr);
}


int test_simple (int con)
{
    int req = 0, col_count = 0, i, ind;
    int error;
    char *data;
    T_CCI_ERROR cci_error;
    T_CCI_COL_INFO *col_info;
    T_CCI_CUBRID_STMT stmt_type;
    char *query = "select * from db_class";

    //preparing the SQL statement
    req = cci_prepare (con, query, 0, &cci_error);
    if (req < 0)
    {
        printf ("prepare error: %d, %s\n", cci_error.err_code,
            cci_error.err_msg);
        goto handle_error;
    }

    //getting column information when the prepared statement is
the SELECT query
    col_info = cci_get_result_info (req, &stmt_type, &col_count);
    if (col_info == NULL)
    {
        printf ("get_result_info error\n");
        goto handle_error;
    }

    //Executing the prepared SQL statement
    error = cci_execute (req, 0, 0, &cci_error);
    if (error < 0)
    {
        printf ("execute error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
        goto handle_error;
    }
    while (1)
    {
        //Moving the cursor to access a specific tuple of results
        error = cci_cursor (req, 1, CCI_CURSOR_CURRENT,
&cci_error);
        if (error == CCI_ER_NO_MORE_DATA)
        {
            break;
        }
```

```
        if (error < 0)
        {
            printf ("cursor error: %d, %s\n",
cci_error.err_code, cci_error.err_msg);
            goto handle_error;
        }

        //Fetching the query result into a client buffer
        error = cci_fetch (req, &cci_error);
        if (error < 0)
        {
            printf ("fetch error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
            goto handle_error;
        }
        for (i = 1; i <= col_count; i++)
        {

            //Getting data from the fetched result
            error = cci_get_data (req, i, CCI_A_TYPE_STR, &data,
&ind);
            if (error < 0)
            {
                printf ("get_data error: %d, %d\n", error, i);
                goto handle_error;
            }
            printf ("%s\t|", data);
        }
         printf ("\n");
    }

    //Closing the query result
    error = cci_close_query_result(req, &cci_error);
     if (error < 0)
    {
        printf ("cci_close_query_result error: %d, %s\n",
cci_error.err_code, cci_error.err_msg);
        goto handle_error;
    }

    //Closing the request handle
    error = cci_close_req_handle(req);
    if (error < 0)
    {
        printf ("cci_close_req_handle error\n");
        goto handle_error;
    }

    //Disconnecting with the server
    error = cci_disconnect (con, &cci_error);
    if (error < 0)
    {
```

```
        printf ("cci_disconnect error: %d, %s\n",
cci_error.err_code, cci_error.err_msg);
        goto handle_error;
    }
    return 0;

handle_error:
    if (req > 0)
        cci_close_req_handle (req);
    if (con > 0)
        cci_disconnect (con, &cci_error);

    return 1;
}

int main()
{
    int con = 0;

    if (cci_set_allocators (my_malloc, my_free, my_realloc,
my_calloc) != 0)
    {
        printf ("cannot register allocators\n");
        return 1;
    };

    //getting a connection handle for a connection with a server
    con = cci_connect ("localhost", 33000, "demodb", "dba", "");
    if (con < 0)
    {
        printf ("cannot connect to database\n");
        return 1;
    }

    test_simple (con);
    return 0;
}
```

cci_set_autocommit

int cci_set_autocommit(int conn_handle, CCI_AUTOCOMMIT_MODE autocommit_mode)

The **cci_set_autocommit** function configures the autocommit mode of current database connection. It is only used to turn ON/OFF of autocommit mode. When this function is called, every transaction being processed is committed regardless of configured mode.

Note

CCI_DEFAULT_AUTOCOMMIT in **cubrid_broker.conf** determines the default autocommit mode upon program startup.

Parameters:

- **conn_handle** – (IN) Connection handle
- **autocommit_mode** – (IN) Configures the autocommit mode. It has one of the following value: CCI_AUTOCOMMIT_FALSE or CCI_AUTOCOMMIT_TRUE

Returns:

Error code (0: success)

Note

CCI_DEFAULT_AUTOCOMMIT, a broker parameter configured in the **cubrid_broker.conf** file, determines whether it is in autocommit mode upon program startup.

cci_set_db_parameter

int cci_set_db_parameter(int conn_handle, T_CCI_DB_PARAM param_name, void *value, T_CCI_ERROR *err_buf)

The **cci_set_db_parameter** function configures a system parameter. For the type of *value* for *param_name*, see `cci_get_db_parameter()`.

Parameters:

- **conn_handle** – (IN) Connection handle
- **param_name** – (IN) System parameter name
- **value** – (IN) Parameter value
- **err_buf** – (OUT) Database error buffer

Returns:

Error code (0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_PARAM_NAME**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

cci_set_element_type

int cci_set_element_type(T_CCI_SET set)

The **cci_set_element_type** function gets the element type of the **T_CCI_SET** type value.

Parameters:

- **set** – (IN) cci set pointer

Returns:

Type

cci_set_free

void cci_set_free(T_CCI_SET set)

The **cci_set_free** function releases the memory allocated to the type value of **T_CCI_SET** fetched by **CCI_A_TYPE_SET** with `cci_get_data()`. The **T_CCI_SET** type value can be created through fetching `cci_get_data()` or `cci_set_make()` function.

Parameters:

- **set** – (IN) cci set pointer

cci_set_get

int cci_set_get(T_CCI_SET set, int index, T_CCI_A_TYPE a_type, void *value, int *indicator)

The **cci_set_get** function gets the *index* -th data for the type value of **T_CCI_SET**.

Parameters:

- **set** – (IN) cci set pointer
- **index** – (IN) set index (base: 1)
- **a_type** – (IN) Type
- **value** – (OUT) Result buffer
- **indicator** – (OUT) null indicator

Returns:

Error code

- **CCI_ER_SET_INDEX**
- **CCI_ER_TYPE_CONVERSION**
- **CCI_ER_NO_MORE_MEMORY**
- **CCI_ER_COMMUNICATION**

The data type of *value* for *a_type* is shown in the table below.

a_type	value type
CCI_A_TYPE_STR	char **
CCI_A_TYPE_INT	int *
CCI_A_TYPE_FLOAT	float *
CCI_A_TYPE_DOUBLE	double *
CCI_A_TYPE_BIT	**T_CCI_BIT** *
CCI_A_TYPE_DATE	**T_CCI_DATE** *

a_type	value type
CCI_A_TYPE_BIGINT	int64_t * (For Windows: __int64 *)

cci_set_holdability

int cci_set_holdability(int conn_handle, int holdable)

Sets whether to enable or disable cursor holdability of the result set from the connection level. When it is 1, the connection is disconnected or the cursor is holdable until the result set is intentionally closed regardless of commit. When it is 0, the result set is closed when committed and the cursor is not holdable. For more details on cursor holdability, see Cursor Holdability.

Parameters:

- **conn_handle** – (IN) Connection handle
- **holdable** – (IN) Cursor holdability setting value (0: not holdable, 1: holdable)

Returns:

Error Code

- **CCI_ER_INVALID_HOLDABILITY**

cci_set_isolation_level

int cci_set_isolation_level(int conn_handle, T_CCI_TRAN_ISOLATION new_isolation_level, T_CCI_ERROR *err_buf)

The **cci_set_isolation_level** function sets the transaction isolation level of connections. All further transactions for the given connections work as *new_isolation_level*.

Parameters:

- **conn_handle** – (IN) Connection handle
- **new_isolation_level** – (IN) Transaction isolation level
- **err_buf** – (OUT) Database error buffer

Returns:

Error code

- **CCI_ER_CON_HANDLE**
- **CCI_ER_CONNECT**
- **CCI_ER_ISOLATION_LEVEL**
- **CCI_ER_DBMS**

Note If the transaction isolation level is set by cci_set_db_parameter(), only the current transaction is affected. When the transaction is complete, the transaction isolation level returns to the one set by CAS. You must use **cci_set_isolation_level** () to set the isolation level for the entire connection.

cci_set_lock_timeout

int cci_set_lock_timeout(int conn_handle, int locktimeout, T_CCI_ERROR *err_buf)

The **cci_set_lock_timeout** function specifies the connection lock timeout as milliseconds. This is the same with calling **cci_set_db_parameter** (conn_id, CCI_PARAM_LOCK_TIMEOUT, &val, err_buf). See `cci_set_db_parameter()`.

Parameters:

- **conn_handle** – (IN) Connection handle
- **locktimeout** – (IN) lock timeout value(Unit: milliseconds).

Returns:

Error code(0: success)

- **CCI_ER_CON_HANDLE**

- **CCI_ER_PARAM_NAME**
- **CCI_ER_DBMS**
- **CCI_ER_COMMUNICATION**
- **CCI_ER_CONNECT**

cci_set_login_timeout

int cci_set_login_timeout(int conn_handle, int timeout, T_CCI_ERROR *err_buf)

The **cci_set_login_timeout** function specifies the login timeout as milliseconds. The login timeout is applied when an internal reconnection occurs as `cci_prepare()` or `cci_execute()` function is called. This change is only applied to the current connection.

Parameters:

- **conn_handle** – (IN) Connection handle
- **timeout** – (IN) Login timeout(unit: milliseconds)
- **err_buf** – (OUT) Error buffer

Returns:

Error code(0: success)

- **CCI_ER_CON_HANDLE**
- **CCI_ER_USED_CONNECTION**

cci_set_make

int cci_set_make(T_CCI_SET *set, T_CCI_U_TYPE u_type, int size, void *value, int *indicator)

The **cci_set_make** function makes a set of a new **CCI_A_TYPE_SET** type. The created set is sent to the server as **CCI_A_TYPE_SET** by `cci_bind_param()`. The memory for the set created by **cci_set_make()** must be freed by `cci_set_free()`. The type of *value* for *u_type* is shown in the table below.

Parameters:

- **set** – (OUT) cci set pointer

- **u_type** – (IN) Element type
- **size** – (IN) set size
- **value** – (IN) set element
- **indicator** – (IN) null indicator array

Returns:

Error code

cci_set_max_row

int cci_set_max_row(int req_handle, int max)

The **cci_set_max_row** function configures the maximum number of records for the results of the **SELECT** statement executed by `cci_execute()`. If the *max* value is 0, it is the same as not setting the value.

Parameters:

- **req_handle** – (IN) Connection handle
- **max** – (IN) The maximum number of rows

Returns:

Error code

```
req = cci_prepare( ... );
cci_set_max_row(req, 1);
cci_execute( ... );
```

cci_set_query_timeout

int cci_set_query_timeout(int req_handle, int milli_sec)

The **cci_set_query_timeout** function configures timeout value for query execution.

Parameters:

- **req_handle** – (IN) Request handle

- **milli_sec** – Timeout (unit: msec.)

Returns:

Success: Request handle ID (int), Failure: Error code

- CCI_ER_REQ_HANDLE

The timeout value configured by **cci_set_query_timeout** affects `cci_prepare()`, `cci_execute()`, `cci_execute_array()`, `cci_execute_batch()` functions. When timeout occurs in the function and if the **disconnect_on_query_timeout** value configured in `cci_connect_with_url()` connection URL is yes, it returns the **CCI_ER_QUERY_TIMEOUT** error.

These functions can return the **CCI_ER_LOGIN_TIMEOUT** error if **login_timeout** is configured in the connection URL, which is an argument of `cci_connect_with_url()` function; this means that login timeout happens between application client and CAS during re-connection.

It is going through the process of re-connection between application client and CAS when an application restarts or it is re-scheduled. Re-scheduling is a process that CAS chooses an application client, and starts and stops connection in the unit of transaction. If **KEEP_CONNECTION**, broker parameter, is OFF, it always happens; if AUTO, it can happen depending on its situation. For details, see the description of **KEEP_CONNECTION** in the Parameter by Broker

cci_set_size

int cci_set_size(T_CCI_SET set)

The **cci_set_size** function gets the number of elements for the type value of **T_CCI_SET**.

Parameters:

- **set** – (IN) cci set pointer

Returns:

Size

PHP Driver

CUBRID PHP driver implements an interface to enable access from application in PHP to CUBRID database. Every function offered by CUBRID PHP driver has a prefix **cubrid_** such as cubrid_connect() and cubrid_connect_with_url().

The official one is available as a PECL package. PECL is a repository for PHP extensions, providing a directory of all known extensions and holding facilities for downloading and development of PHP extensions. For more information about PECL, visit https://pecl.php.net/ .

CUBRID PHP driver is written based on CCI API so affected by CCI configurations such as **CCI_DEFAULT_AUTOCOMMIT**.

Installing and Configuring PHP

The easiest and fastest way to get all applications installed on your system is to install CUBRID, Apache web server, PHP, and CUBRID PHP driver at the same time.

For Linux

Configuring the Environment

- Operating system: 32-bit or 64-bit Linux
- Web server: Apache
- PHP: version 5.x, or version 7.x (https://www.php.net/downloads.php)
- We refer to the latest PHP versions 5.6.x ,7.1.x and 7.4.x as a convenience.

Installing CUBRID PHP Driver using PECL

If **PECL** package has been installed on your system, the installation of CUBRID PHP driver is straightforward. **PECL** will download and compile the driver for you.

1. Enter the following command to install the latest version of CUBRID PHP driver.

```
sudo pecl install cubrid
```

If you need earlier versions of the driver, you can install exact versions as follows:

```
sudo pecl install cubrid-10.1.0.0002
```

During the installation, you will be prompted to enter **CUBRID base install dir autodetect :**. Just to make sure your installation goes smoothly, enter the full path to the directory where you have installed CUBRID. For example, if CUBRID has been installed at **/home/cubridtest/CUBRID**, then enter **/home/cubridtest/CUBRID**.

2. Edit the configuration file.

   If you are using CentOS 6.0 and later or Fedora 15 and later, create a file named **cubrid.ini**, enter a command line **extension=cubrid.so**, and store the fine in the **/etc/php.d** directory.

   If you are using earlier versions of CentOS 6.0 or Fedora 15, edit the **php.ini** file (default location: **/etc/php5/apache2/** or **/etc/**) and add the following two command lines at the end of the file.

```
[CUBRID]
extension=cubrid.so
```

3. Restart the web server to apply changes.

Installing using apt-get on Ubuntu

1. If you do not have PHP itself installed, install it using the following command; if you have PHP installed on your system, skip this step.

```
sudo apt-get install php5

or for PHP version 7
sudo add-apt-repository ppa:ondrej/php
sudo apt-get update
sudo apt-get install php7.1
```

2. To install CUBRID PHP driver using **apt-get**, we need to add CUBRID's repository so that Ubuntu knows where to download the packages from and tell the operating system to update its indexes.

```
sudo add-apt-repository ppa:cubrid/cubrid
sudo apt-get update
```

3. Now install the driver.

```
sudo apt-get install php5-cubrid
```

   To install earlier versions, indicate the version as:

```
sudo apt-get install php5-cubrid-9.3.1
```

This will copy the **cubrid.so** driver to **/usr/local/lib/php/*** and add the following configuration lines to **/etc/php.ini**.

```
[PHP_CUBRID]
extension=cubrid.so
```

4. Restart the web server so that PHP can read the module.

```
service apache2 restart
```

For Windows

Requirements

- CUBRID: 9.3.x or later
- Operating system: 32-bit or 64 bit Windows
- Web server: Apache or IIS
- PHP: 5.6.x, 7.1.x or 7.4.x (https://windows.php.net/download/)
- For PHP 7.1.x or 7.4.x, you need to install Microsoft Visual C++ 2015 Redistributable Package for 32bit or 64bit.

Using CUBRID PHP Driver Installer

The CUBRID PHP API Installer is a Windows installer which automatically detects the CUBRID and PHP version and installs the proper driver for you by copying it to the default PHP extensions directory and adding the extension load directives to the **php.ini** file. In this section, we will explain how to use the CUBRID PHP API Installer to install the CUBRID PHP extension on Windows.

In case you want to remove the CUBRID PHP driver, you just have to run the CUBRID PHP API Installer again in uninstall mode (like any other un-installer on Windows) and it will reset all the changes made during installation.

Before you install CUBRID PHP driver, make sure that paths of PHP and CUBRID are added in the system variable, **Path**.

1. Download the CUBRID PHP API installer for Windows from the link below. The current installer includes the drivers for all CUBRID versions.

   https://www.cubrid.org/downloads#php

2. To install the PHP extension, run the installer. Once the installer starts, click the [Next] button.

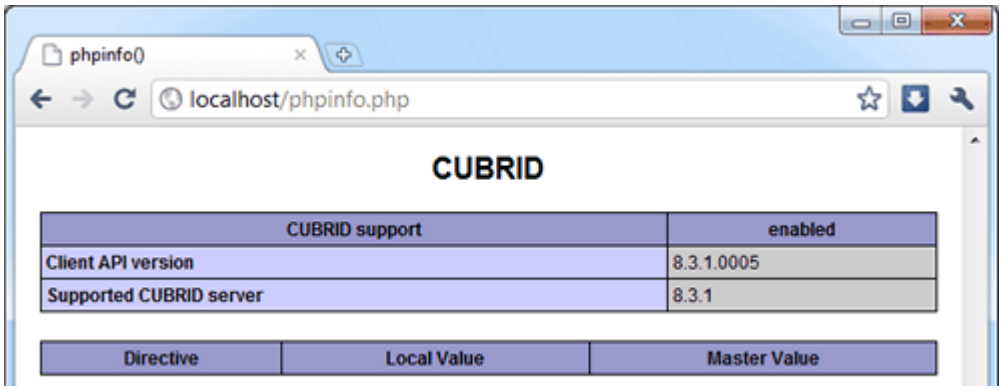
3. Agree with the BSD license terms and click the [Next] button.

4. Choose where you would like to install this CUBRID PHP API Installer and click the [Next] button. You should choose a new folder for this installer like **C:\Program Files\CUBRID PHP API**.

5. Give a folder name and click the [Install] button. If you fail installation, you should probably receive an error message. In this case, see "Configuring the environment" below.

6. If no error message is displayed, this should install the CUBRID PHP extension and update your **php.ini** file. Click [Finish] to close the installer.

7. For changes to take place, restart your web server and execute the phpinfo() to confirm CUBRID has successfully been installed.



Configuring the environment

If you have received an error messages, follow the steps below; if you can see CUBRID in phpinfo(), you do not need to look further. By default, when you install CUBRID, it automatically adds its installation directory to the **Path** system environment variable. To verify the variable have been correctly configured, launch the command prompt ([Start] > [Programs] > [Accessories] > [Command Prompt]) and enter the following commands one by one.

1. Enter command below in the command prompt as follows.

```
php --version
```

You can see the PHP version like below if it is properly configured.

```
        PHP 5.6.30 (cli) (built: Jun 13 2017 16:16:30)
        or for version 7.1.x
        PHP 7.1.7 (cli) (built: Aug  3 2017 10:59:35) ( NTS )

C:\Users\Administrator>php --version
PHP 5.6.30 (cli) (built: Jan 18 2017 19:47:28)
```

2. Enter command as follows.

```
cubrid --version
```

You can see the CUBRID version like below if it is properly configured.

```
C:\Users\Administrator>cubrid --version
cubrid.exe (CUBRID utilities)
        CUBRID 9.3 (9.3.8.0003) (64bit release build for
Windows_NT) (Apr 11 2017 11:54:08)
```

If you cannot get the result like above, it is highly likely that your PHP and CUBRID installations went wrong. Try to reinstall them and recheck again. If the path is not automatically specified even after you complete reinstallation, you can do it manually.

1. Right-click [My Computer] and select [Properties]. The [System Properties] dialog box will appear.

2. Go to [Advanced] tab and click on [Environment Variables].

3. Select the variable called **Path** in the [System variables] box and click [Edit] button. You will notice that the value of that variable contains system paths separated by semi-colon.

4. Add the paths for CUBRID and PHP in that variable. For example, if PHP is installed in **C:\Program Files\PHP** and also CUBRID in **C:\CUBRID\bin**, you will have to append (do not overwrite, just append) these values to the path like **C:\CUBRID\bin;C:\Program Files\PHP**.

5. Click [OK] to save and close the dialog box.

6. To confirm you have done everything correct, check the variable presence in the command prompt.

Downloading and Installing Compiled CUBRID PHP Driver

First, download CUBRID PHP/PDO driver of which versions match the versions of your operating system and PHP installed from https://www.cubrid.org/downloads#php .

After you download the driver, you will see the **php_cubrid.dll** file for CUBRID PHP driver or the **php_pdo_cubrid.dll** file for CUBRID PDO driver. Follow the steps below to install it.

1. Copy this driver to the default PHP extensions directory (usually located at **C:\Program Files\PHP\ext**).

2. Set your system environment. Check if the environment variable **PHPRC** is **C:\Program Files\PHP** and system variable path is added with **%PHPRC%** and **%PHPRC\ext**.

3. Edit **php.ini** (**C:\Program Files\PHP\php.ini**) and add the following two command lines at the end of the **php.ini** file.

```
[PHP_CUBRID]
extension=php_cubrid.dll
```

For CUBRID PDO driver, add command lines below.

```
[PHP_PDO_CUBRID]
extension = php_pdo_cubrid.dll
```

4. Restart your web server to apply changes.

Building CUBRID PHP Driver from Source Code

For Linux

In this section, we will introduce the way of building CUBRID PHP driver for Linux.

Configuring the environment

- CUBRID: Install CUBRID. Make sure the environment variable **%CUBRID%** is defined in your system.
- PHP 5.6.x, 7.1.x or 7.4.x source code: You can download PHP source code from https://www.php.net/downloads.php .
- Apache 2: It can be used to test PHP.
- CUBRID PHP driver source code: You can download the source code from https://www.cubrid.org/downloads#php . Make sure that the version you download is the same as the version of CUBRID which has been installed on your system.
- IF building CCI driver In Linux Or Mac, GNU Developer Toolset 8 or higher is required.

Compiling CUBRID PHP driver

1. Download the CUBRID PHP driver, extract it, and enter the directory.

```
$> tar zxvf php-<version>.tar.gz (or tar jxvf php-<version>.tar.bz2)
$> cd php-<version>/ext
```

2. Run **phpize**. For more information about getting **phpize**, see Remark.

```
cubrid-php> /usr/bin/phpize
```

3. Configure the project. It is recommended to execute **./configure -h** so that you can check the configuration options (we assume that Apache 2 has been installed in **/usr/local**).

```
cubrid-php>./configure --with-cubrid --with-php-config=/usr/local/
bin/php-config
```

   - --with-cubrid=shared: Includes CUBRID support.
   - --with-php-config=PATH: Enters an absolute path of php-config including the filename.

4. Build the project. If it is successfully compiled, the **cubrid.so** file will be created in the **/modules** directory.

5. Copy the **cubrid.so** to the **/usr/local/php/lib/php/extensions** directory; the **/usr/local/php** is a PHP root directory.

```
cubrid-php> mkdir /usr/local/php/lib/php/extensions
cubrid-php> cp modules/cubrid.so /usr/local/php/lib/php/extensions
```

6. In the **php.ini** file, set the **extension_dir** variable and add the CUBRID PHP driver to the **extension** variable as shown below.

```
extension_dir = "/usr/local/php/lib/php/extension/no-debug-zts-xxx"
extension = cubrid.so
```

Testing CUBRID PHP driver installation

1. Create a **test.php** file as follows:

```
<?php phpinfo(); ?>
```

2. Use web browser to visit http://localhost/test.php. If you can see the following result, it means that installation is successfully completed.

CUBRID	Value
Version	10.1.0.XXXX

Remark

phpize is a shell script to prepare the PHP extension for compiling. You can get it when you install PHP because it is automatically installed with PHP installation, in general. If it you do not have **phpize** installed on your system, you can get it by following the steps below.

1. Download the PHP source code. Make sure that the PHP version works with the PHP extension that you want to use. Extract PHP source code and enter its root directory.

```
$> tar zxvf php-<version>.tar.gz (or tar jxvf php-<version>.tar.bz2)
$> cd php-<version>
```

2. Configure the project, build, and install it. You can specify the directory you want install PHP by using the option, **--prefix**.

```
php-root> ./configure --prefix=prefix_dir; make; make install
```

3. You can find **phpize** in the **prefix_dir/bin** directory.

For Windows

In this section, we will introduce three ways of building CUBRID PHP driver for Windows. If you have no idea which version you choose, read the following contents first.

If you are using PHP as module with Apache builds from apache.org (not recommended) you need to use the older VC6 versions of PHP compiled with the legacy Visual Studio 6 compiler. Do NOT use VC11+ versions of PHP with the apache.org binaries.

With Apache you have to use the Thread Safe (TS) versions of PHP.

- If you are using PHP version 5.5.x or later, you should use the VC11 versions (Visual Studio 2012)
- If you are using PHP version 7.1.x or later, you should use the VC14 versions (Visual Studio 2015)

VC11 and VC14 versions are compiled with the Visual Studio 2012 and 2015 compiler respectively. The VC11 or VC14 versions have more improvements in performance and stability.

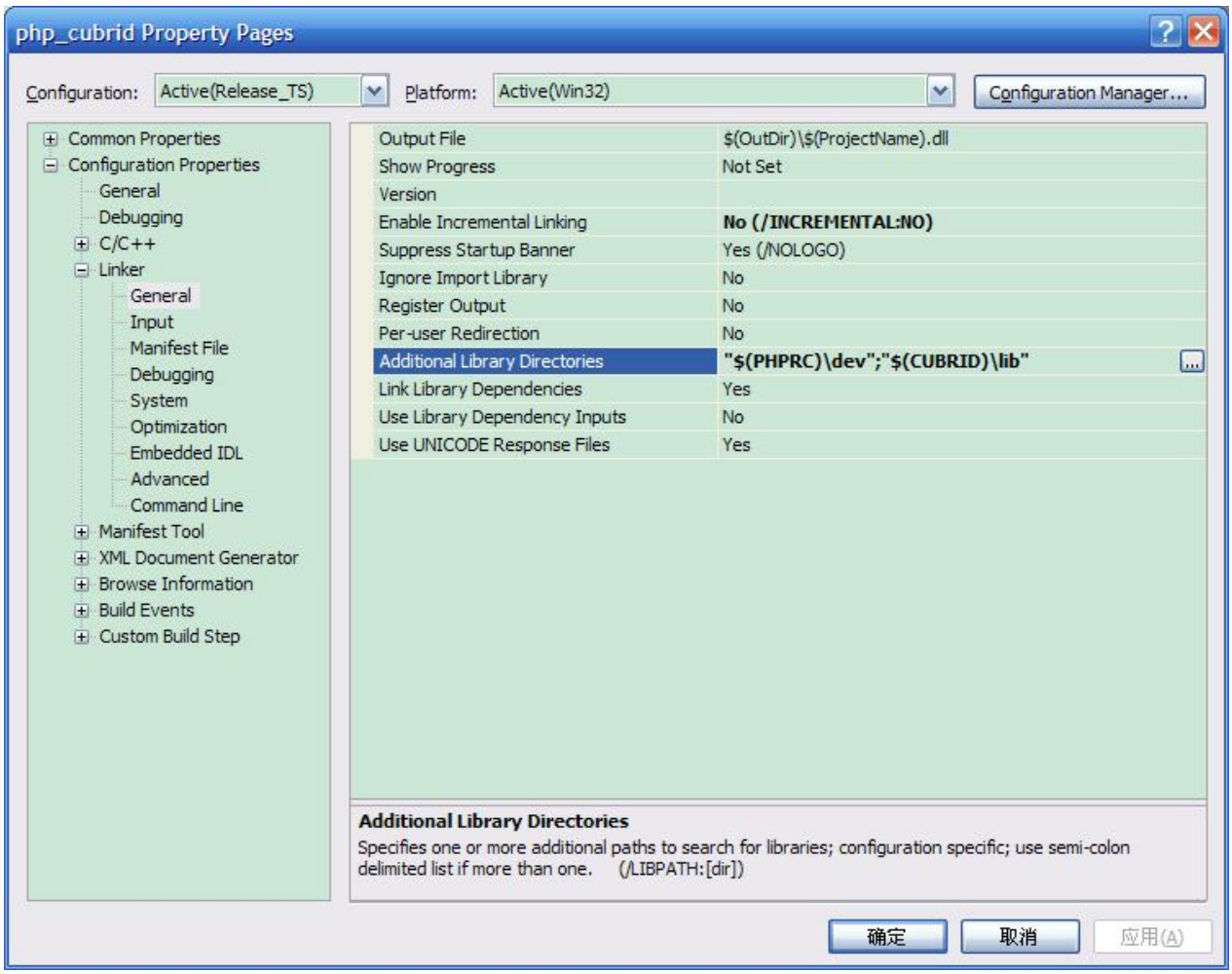
More recent versions of PHP are built with VC11, VC14 (Visual Studio 2012 or 2015 compiler respectively) and include improvements in performance and stability.

- The VC11 builds require to have the Visual C++ Redistributable for Visual Studio 2012 x86 or x64 installed
- The VC14 builds require to have the Visual C++ Redistributable for Visual Studio 2015 x86 or x64 installed

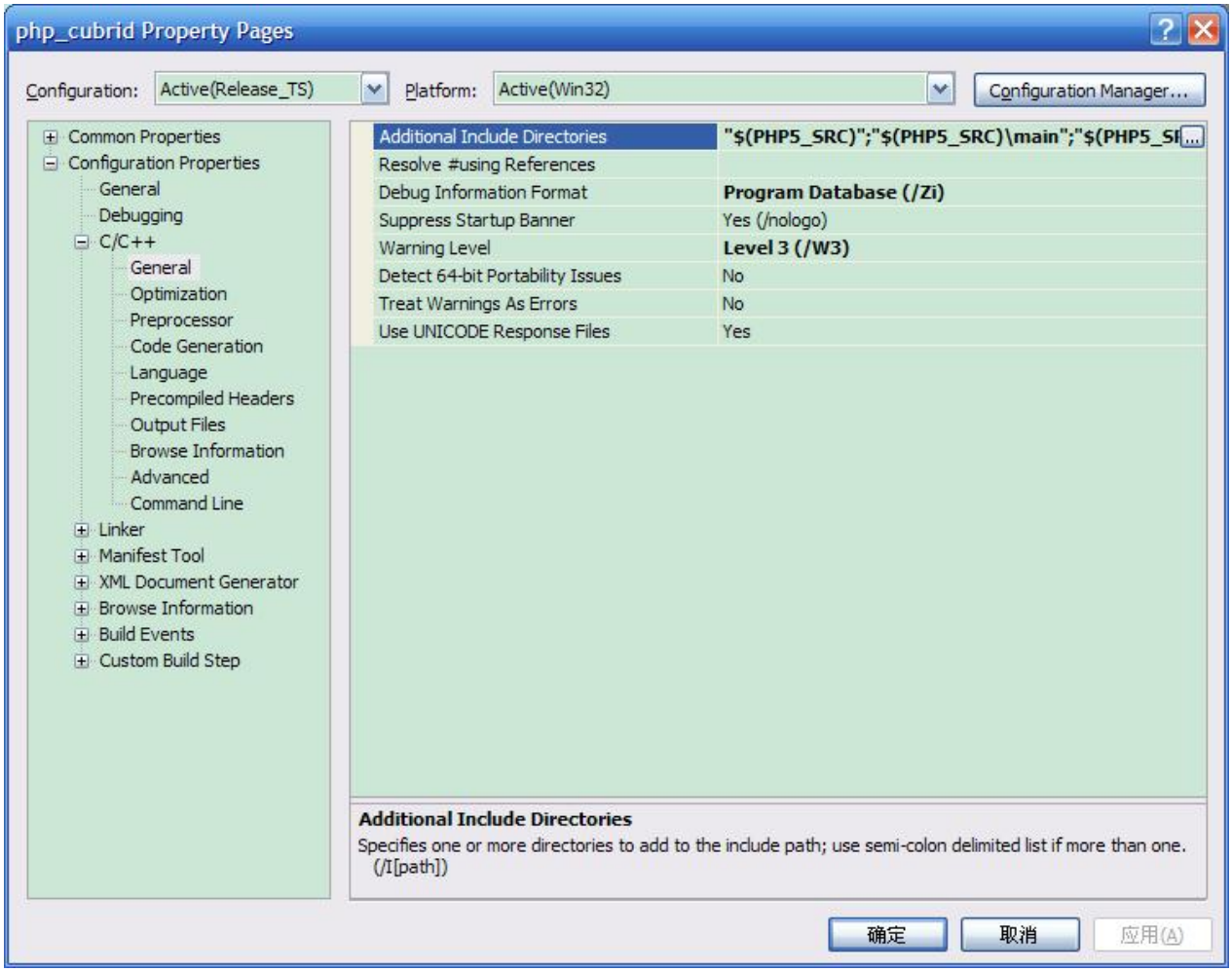
Building CUBRID PHP Driver with VC11 for PHP 5.6.x

Configuring the environment

- CUBRID: Install CUBRID. Make sure the environment variable **%CUBRID%** is defined in your system.
- Visual Studio 2012: You can alternately use the free Visual C++ Express Edition or the Visual C++ 11 compiler included in the Windows SDK if you are familiar with a makefile. Make sure that you have the Microsoft Visual C++ Redistributable Package installed on your system to use CUBRID PHP VC11 driver.
- PHP 5.6.x binaries: You can install VC11 x86 Non Thread Safe or VC11 x86 Thread Safe. Make sure that the **%PHPRC%** system environment variable is correctly set. In the [Property Pages] dialog box, select [General] under the [Linker] tree node. You can see **$(PHPRC)** in [Additional Library Directories].



- PHP 5.6.x source code: Remember to get the source code that matches your binary version. After you extract the PHP 5.6.x source code, add the **%PHP5_SRC%** system environment variable and set its value to the path of PHP 5.6.x source code. In the [Property Pages] dialog box, select [General] under the [C/C++] tree node. You can see **$(PHP5_SRC)** in [Additional Include Directories].



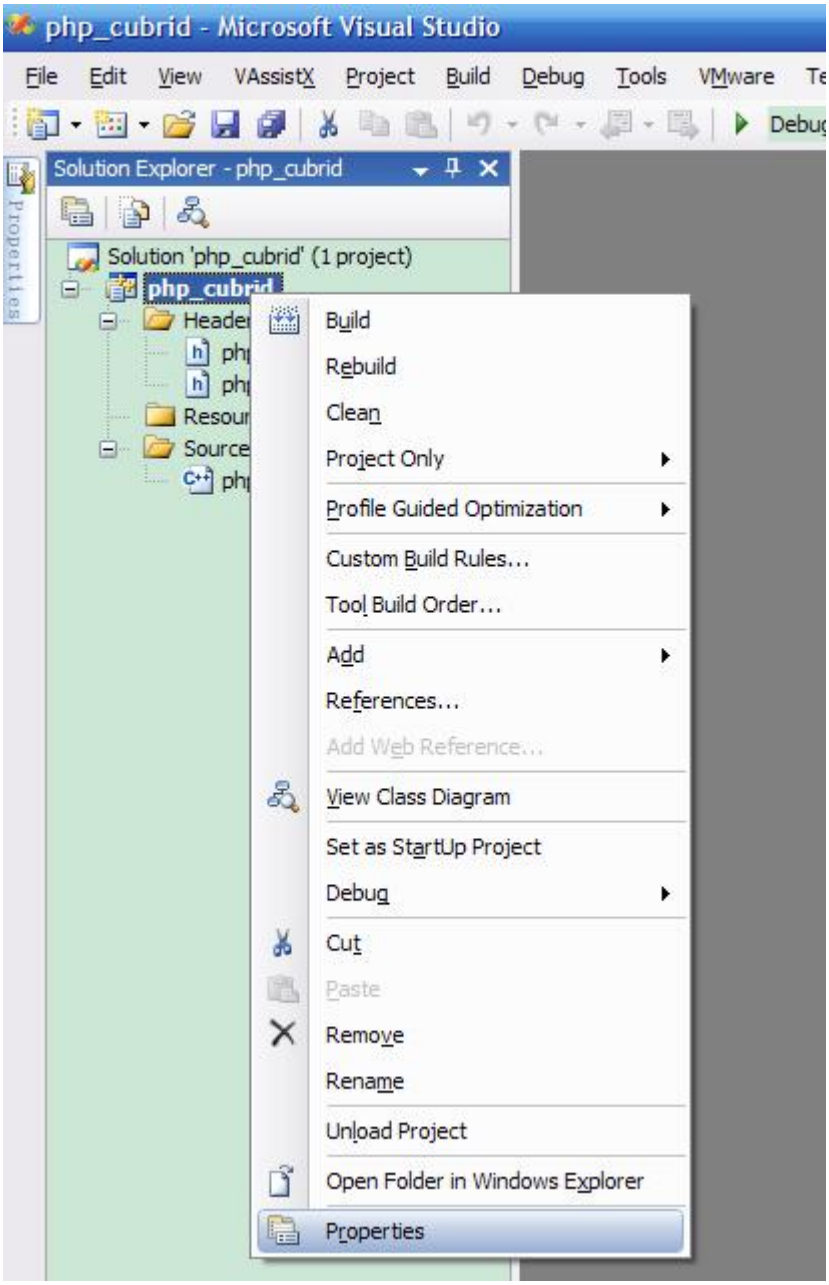
- CUBRID PHP driver source code: You can download CUBRID PHP driver source code of which the version is the same as the version of CUBRID that have been installed on your system. You can get it from https://www.cubrid.org/downloads#php .

Note

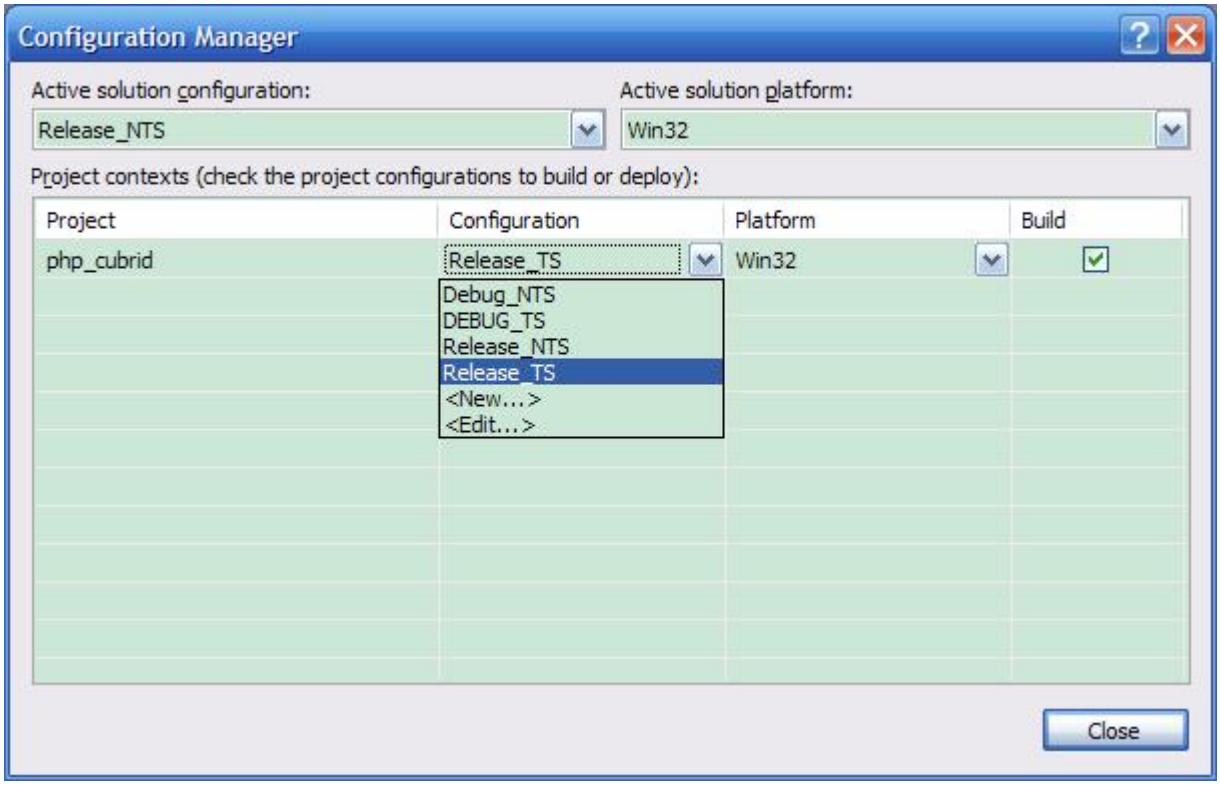
You do not need to build PHP 5.6.x from source code but configuring a project is required. If you do not make configuration settings, you will get the message that a header file (**config.w32.h**) cannot be found. Read https://wiki.php.net/internals/windows/stepbystepbuild to get more detailed information.

Building CUBRID PHP driver

1. Open the **php_cubrid.vcproj** file under the **\win** directory. In the [Solution Explorer] pane, right-click on the **php_cubrid** (project name) and select [Properties].



2. In the [Property Page] dialog box, click the [Configuration Manager] button. Select one of four values among Release_TS, Release_NTS, Debug_TS, and Debug_NTS in [Configuration] of [Project contexts] and click the [Close] button.



3. After you complete the properties modification, click the [OK] button and press the <F7> key to compile the driver. Then, we have the **php_cubrid.dll** file built.

4. You need to make PHP recognize the **php_cubrid.dll** file as an extension. To do this:

    - Create a new folder named **cubrid** where PHP has been installed and copy the **php_cubrid.dll** file to the **cubrid** folder. You can also put the **php_cubrid.dll** file in **%PHPRC%\ext** if this directory exists.

    - In the php.ini file, enter the path of the **php_cubrid.dll** file as an extension_dir variable value and enter **php_cubrid.dll** as an extension value.

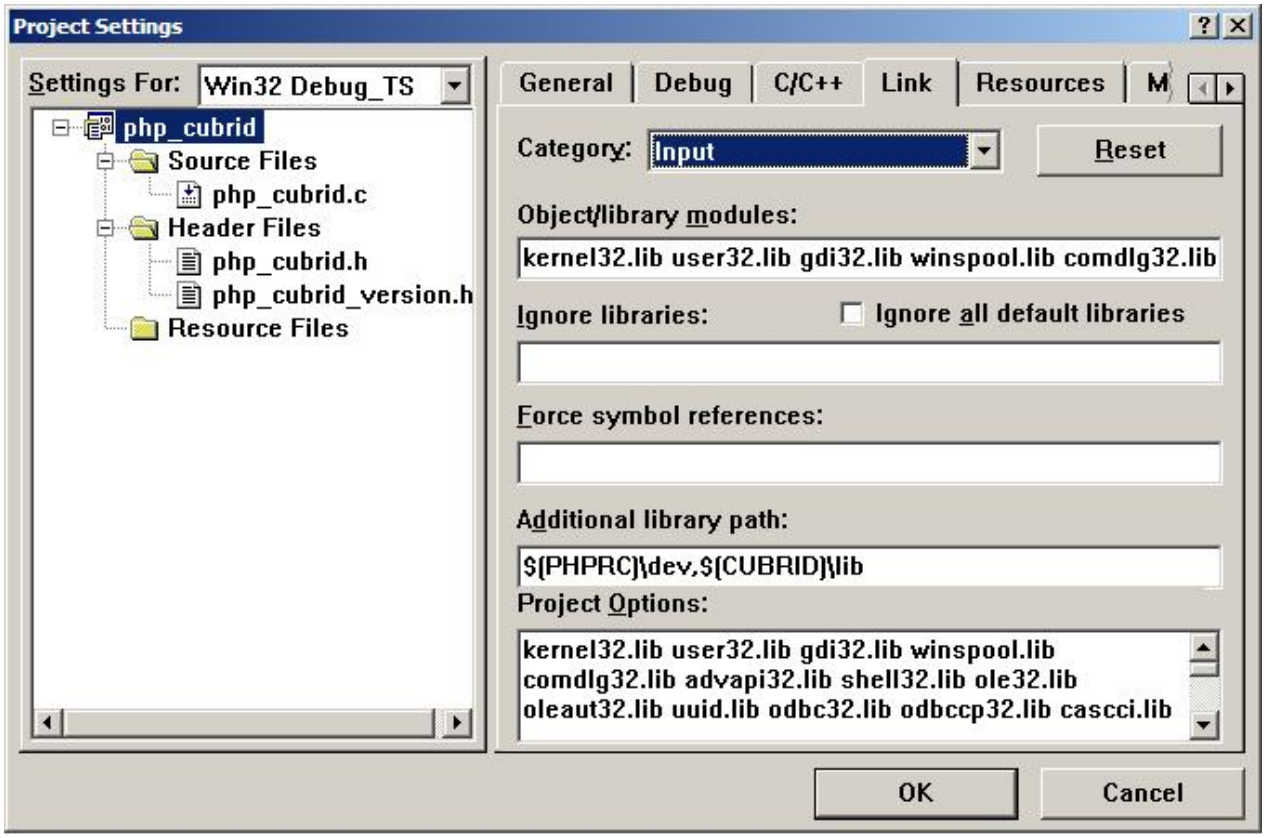
Building CUBRID PHP Driver with VC14 for PHP 7.1.x

Configuring the environment

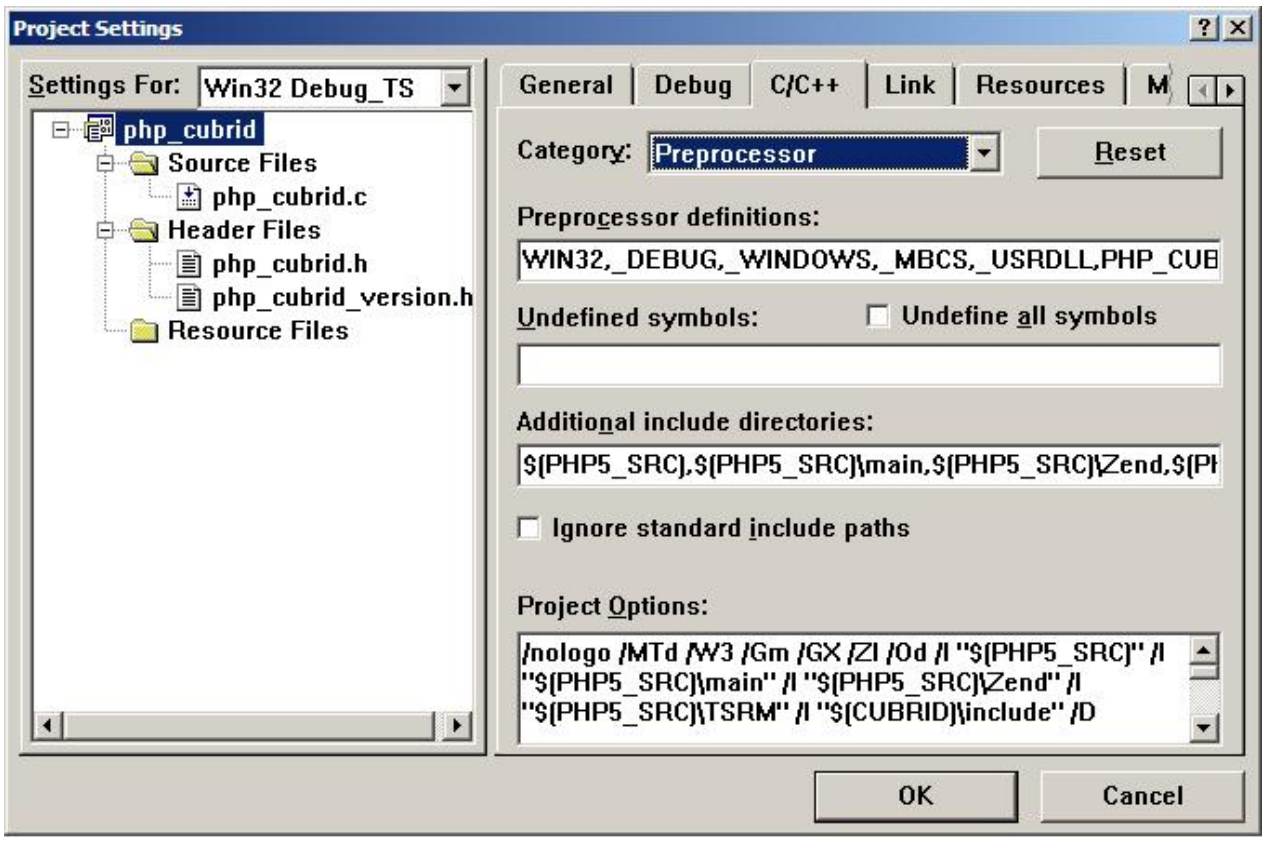
- CUBRID: Install CUBRID. Make sure that the environment variable **%CUBRID%** is defined in your system.

- Visual Studio 2015: You can alternately use the free Visual C++ Express Edition or the Visual C++ 14 compiler included in the Windows SDK if you are familiar with a makefile. Make sure that you have the Microsoft Visual C++ Redistributable Package installed on your system to use CUBRID PHP VC14 driver.

- PHP 7.1.x binaries: You can install VC14 x86 Non Thread Safe or VC14 x86 Thread Safe. Make sure that the value of the **%PHPRC%** system environment variable is correctly set. In the [Project Settings] dialog box, you can find **$(PHPRC)** in [Additional library path] of the [Link] tab.



- PHP 7.1.x source code: Remember to get the source that matches your binary version. After you extract the PHP 7.1.x source code, add the **%PHP7_SRC%** system environment variable and set its value to the path of PHP 7.1.x source code. In the [Project Settings] dialog box of VC11 project, you can find **$(PHP7_SRC)** in [Additional include directories] of the [C/C++] tab.



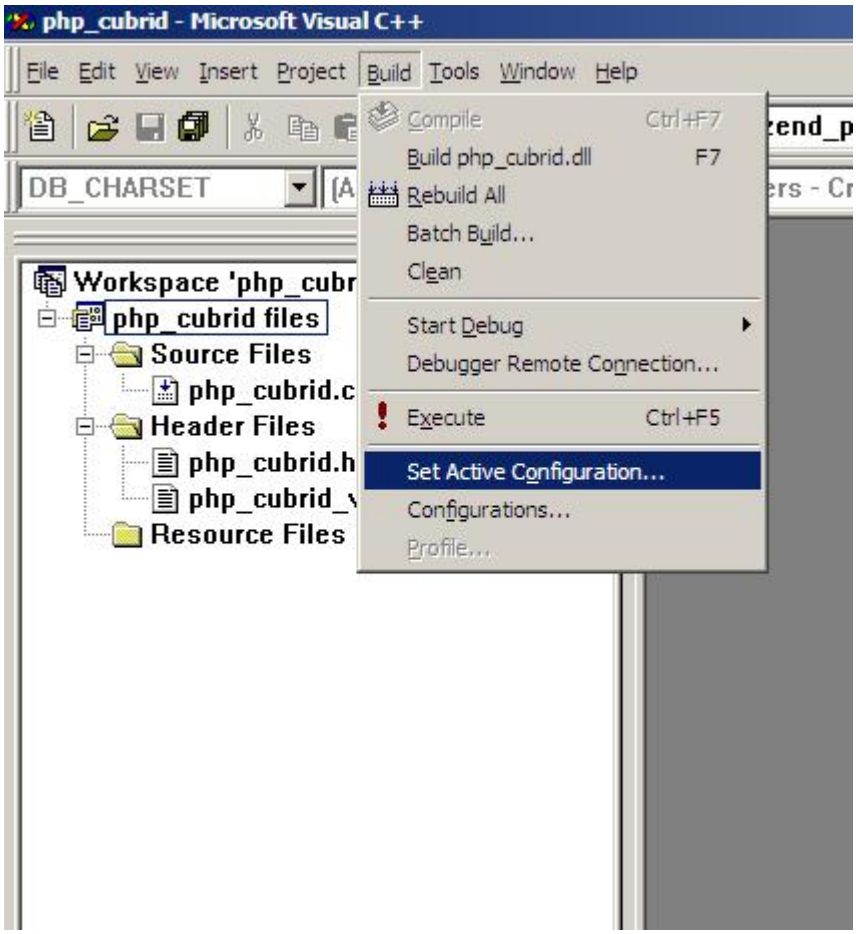
- CUBRID PHP driver source code: You can download CUBRID PHP driver source code of which the version is the same as the version of CUBRID that has been installed on your system. You can get it from https://www.cubrid.org/downloads#php .

Note

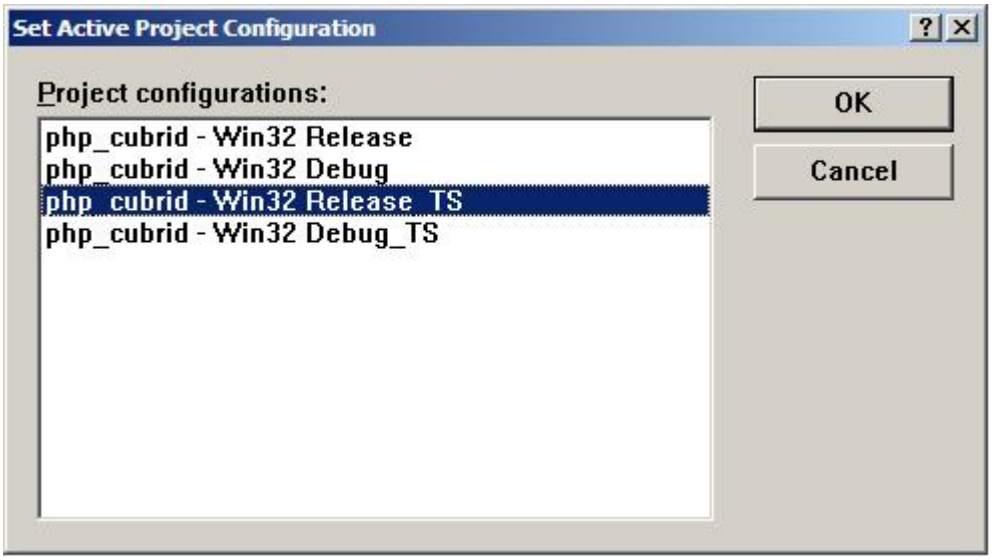
If you build CUBRID PHP driver with PHP 7.1.x source code, you need to make some configuration settings for PHP 7.1.x on Windows. If you do not make these settings, you will get the message that a header file (**config.w32.h**) cannot be found. Read https://wiki.php.net/internals/windows/stepbystepbuild to get more detailed information.

Building CUBRID PHP driver

1. Open the project in the [Build] menu and then select [Set Active Configuration].



2. There are four types of configuration settings (Win32 Release_TS, Win32 Release, Win32 Debug_TS, and Win32 Debug). Select one of them depending on your system and then click the [OK] button.



3. After you complete the properties modification, click the [OK] button and press the <F7> key to compile the driver. Then you have the **php_cubrid.dll** file built.

4. You need to make PHP recognize the **php_cubrid.dll** file as an extension. To do this:

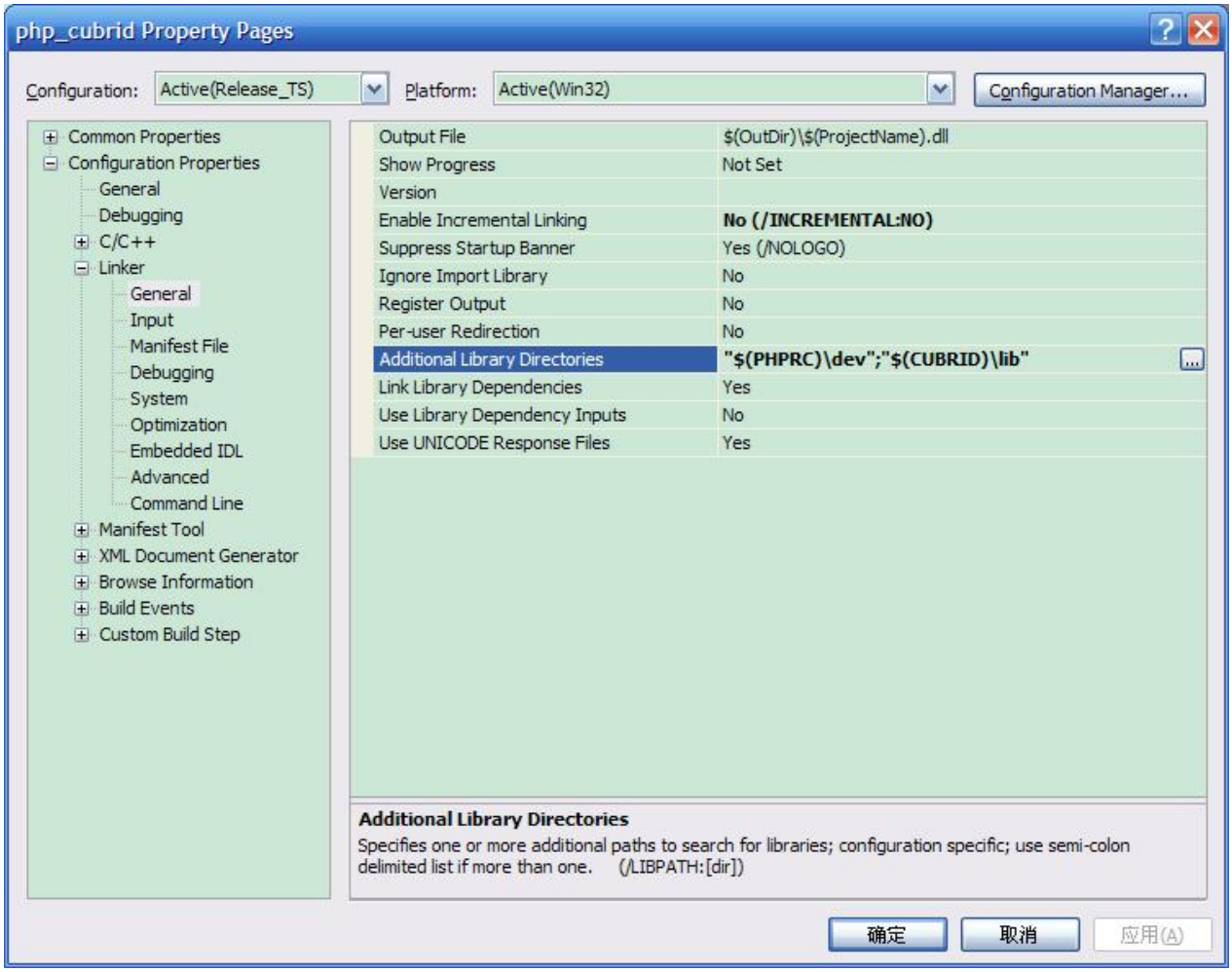
    - Create a new folder named **cubrid** where PHP is installed and copy **php_cubrid.dll** to the **cubrid** folder. You can also put **php_cubrid.dll** in **%PHPRC%\ext** if this directory exists.

    - Set the **extension_dir** variable and add CUBRID PHP driver to **extension** variable in the **php.ini** file.

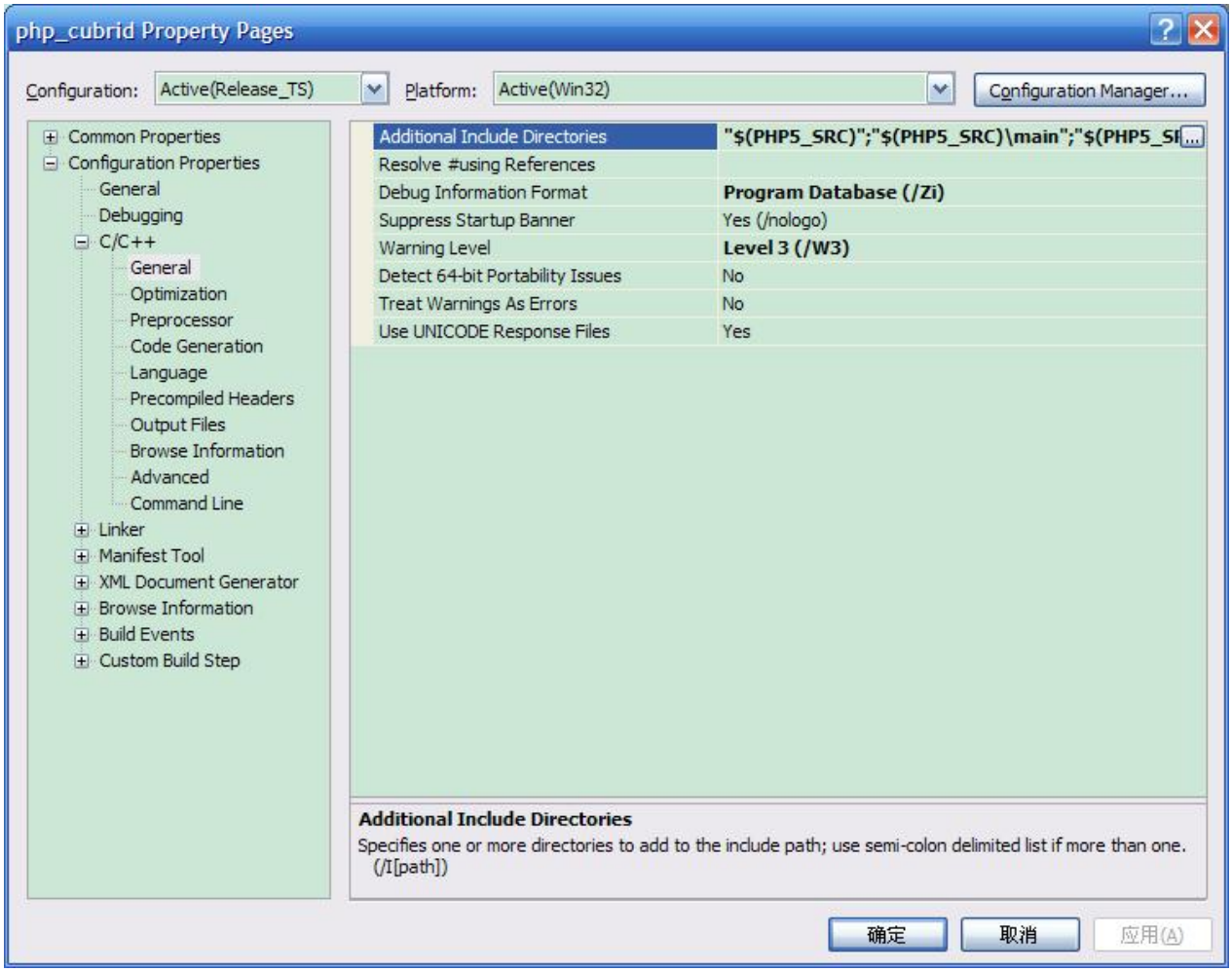
Building CUBRID PHP Driver for 64-bit Windows

PHP for 64-bit Windows

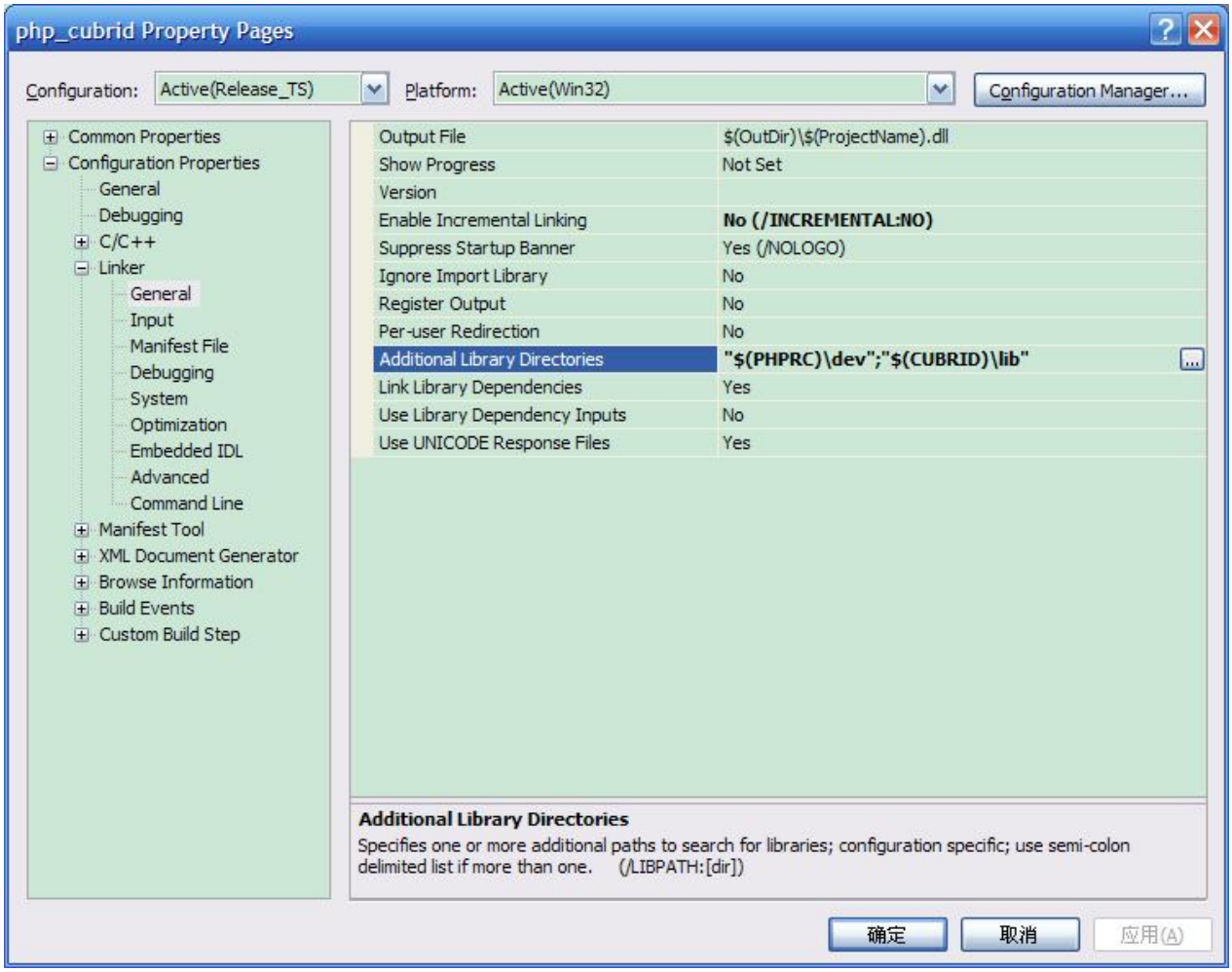
- PHP 5.6.x binaries: You can install VC11 x64 Non Thread Safe or VC11 x64 Thread Safe. Make sure that the **%PHPRC%** system environment variable is correctly set. In the [Property Pages] dialog box, select [General] under the [Linker] tree node. You can see **$(PHPRC)** in [Additional Library Directories].



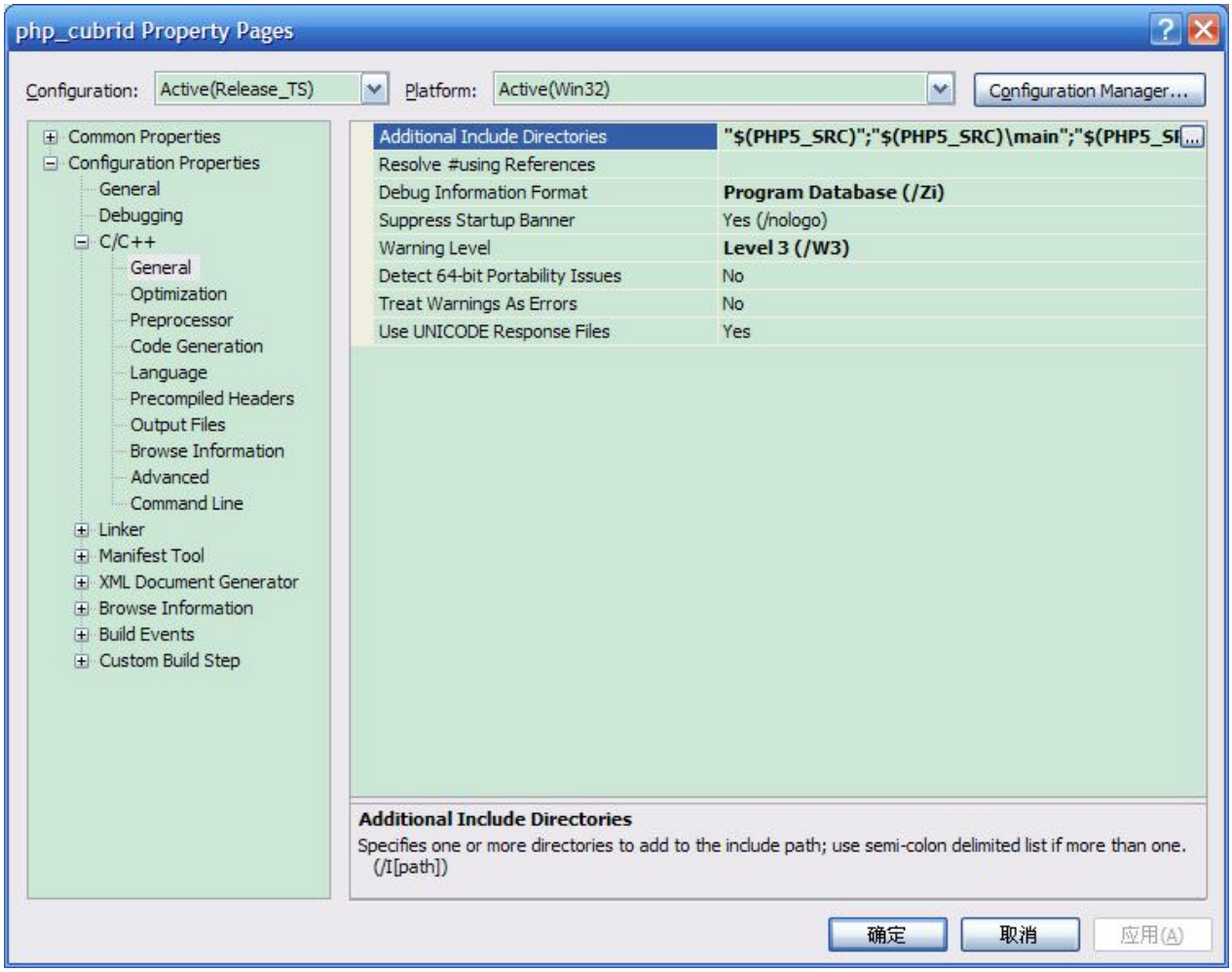
- PHP 5.6.x source code: Remember to get the source code that matches your binary version. After you extract the PHP 5.6.x source code, add the **%PHP5_SRC%** system environment variable and set its value to the path of PHP 5.6.x source code. In the [Property Pages] dialog box, select [General] under the [C/C++] tree node. You can see **$(PHP5_SRC)** in [Additional Include Directories].



- PHP 7.1.x binaries: You can install VC14 x64 Non Thread Safe or VC14 x64 Thread Safe. Make sure that the **%PHPRC%** system environment variable is correctly set. In the [Property Pages] dialog box, select [General] under the [Linker] tree node. You can see **$(PHPRC)** in [Additional Library Directories].



- PHP 7.1.x source code: Remember to get the source code that matches your binary version. After you extract the PHP 7.1.x source code, add the **%PHP7_SRC%** system environment variable and set its value to the path of PHP 7.1.x source code. In the [Property Pages] dialog box, select [General] under the [C/C++] tree node. You can see **$(PHP7_SRC)** in [Additional Include Directories].



- You can find the supported compilers to build PHP on Windows at https://wiki.php.net/internals/windows/compiler . You can see that both Visual C++ 11 (2012) and Visual C++ 14 (2015) can be used to build 64-bit PHP.

Apache for 64-bit Windows

- Apache Lounge has provided up-to-date Windows binaries including 64bit version. You can download the latest apache 2.2.34 64bit version on the following link.

  https://www.apachelounge.com/download/win64/binaries/httpd-2.2.34-win64.zip

Configuring the environment

- CUBRID for 64-bit Windows: You can install the latest version of CUBRID for 64-bit Windows. Make sure the environment variable **%CUBRID%** is defined in your system.

- Visual Studio 2012 or 2015: You can alternately use the free Visual C++ Express Edition or the Visual C++ compiler in the Windows SDK if you are familiar with a makefile.

- PHP 5.6.x or 7.1.x binaries for 64-bit Windows: You can build your own VC11 or VC14 x64 PHP. Both x64 Non Thread Safe and x64 Thread Safe are available. After you have installed it, check if the value of system environment variable **%PHPRC%** is correctly set.

- PHP 5.6.x source: Remember to get the src package that matches your binary version. After you extract the PHP 5.6.x src, add system environment variable **%PHP5_SRC%** and set its value to the path of PHP 5.6.x source code. In the VC11 [Property Pages] dialog box, select [General] under the [C/C++] tree node. You can see **$(PHP5_SRC)** in [Additional Include Directories].

- PHP 7.1.x source: Remember to get the src package that matches your binary version. After you extract the PHP 7.1.x src, add system environment variable **%PHP7_SRC%** and set its value to the path of PHP 7.1.s source code. In the VC14 [Property Pages] dialog box, select [General] under the [C/C++] tree node. You can see **$(PHP7_SRC)** in [Additional Include Directories].

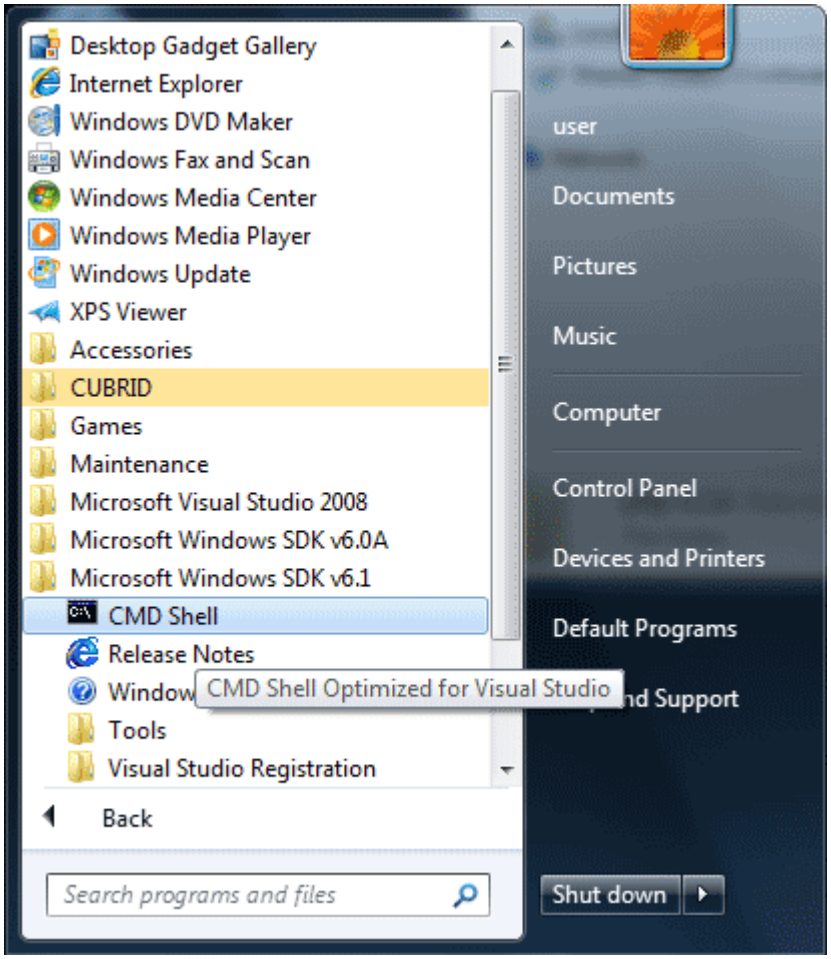
- CUBRID PHP driver source code: You can download CUBRID PHP driver source code of which the version is the same as the version of CUBRID that is installed on your system. You can get it from https://www.cubrid.org/downloads#php .

Note

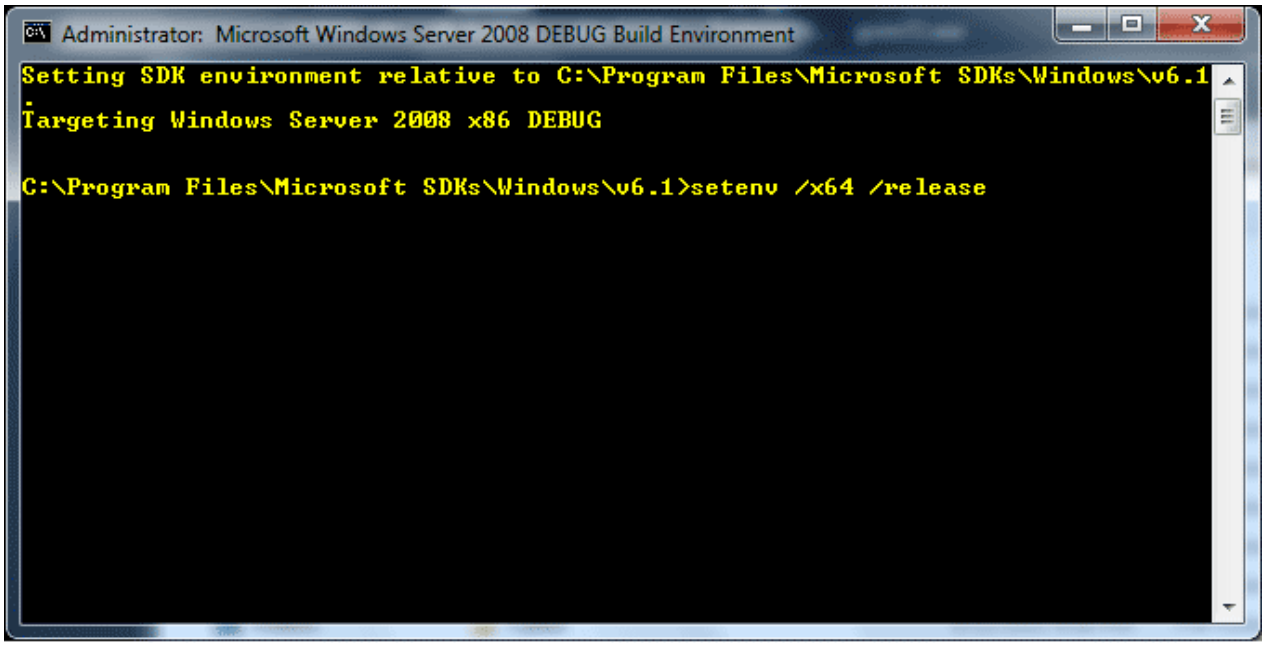
You do not need to build PHP 5.6.x or 7.1.x from source code; however, configuring a project is required. If you do not make configuration settings, you will get the message that a header file (**config.w32.h**) cannot be found. Read https://wiki.php.net/internals/windows/stepbystepbuild to get more detailed information.

Configuring PHP 5.6.x or 7.1.x

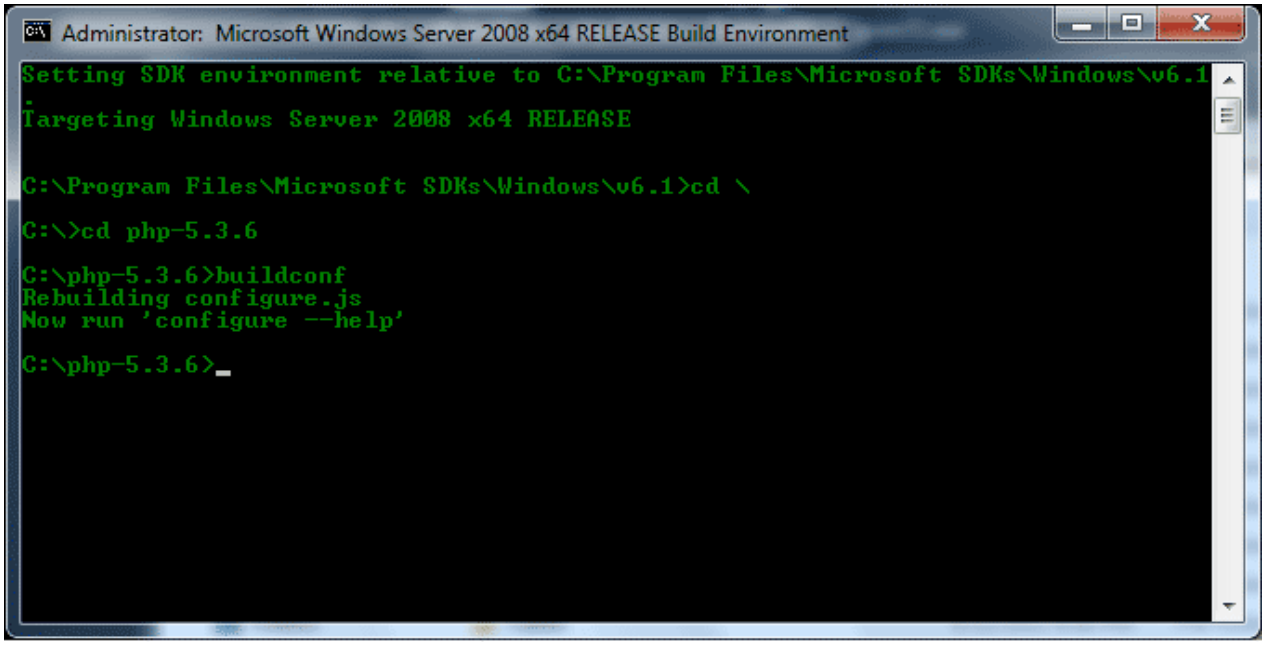
1. After you have installed SDK 6.1 or 8.1 later, click the [CMD Shell] shortcut under the [Microsoft Windows SDK v.x] folder (Windows Start menu).



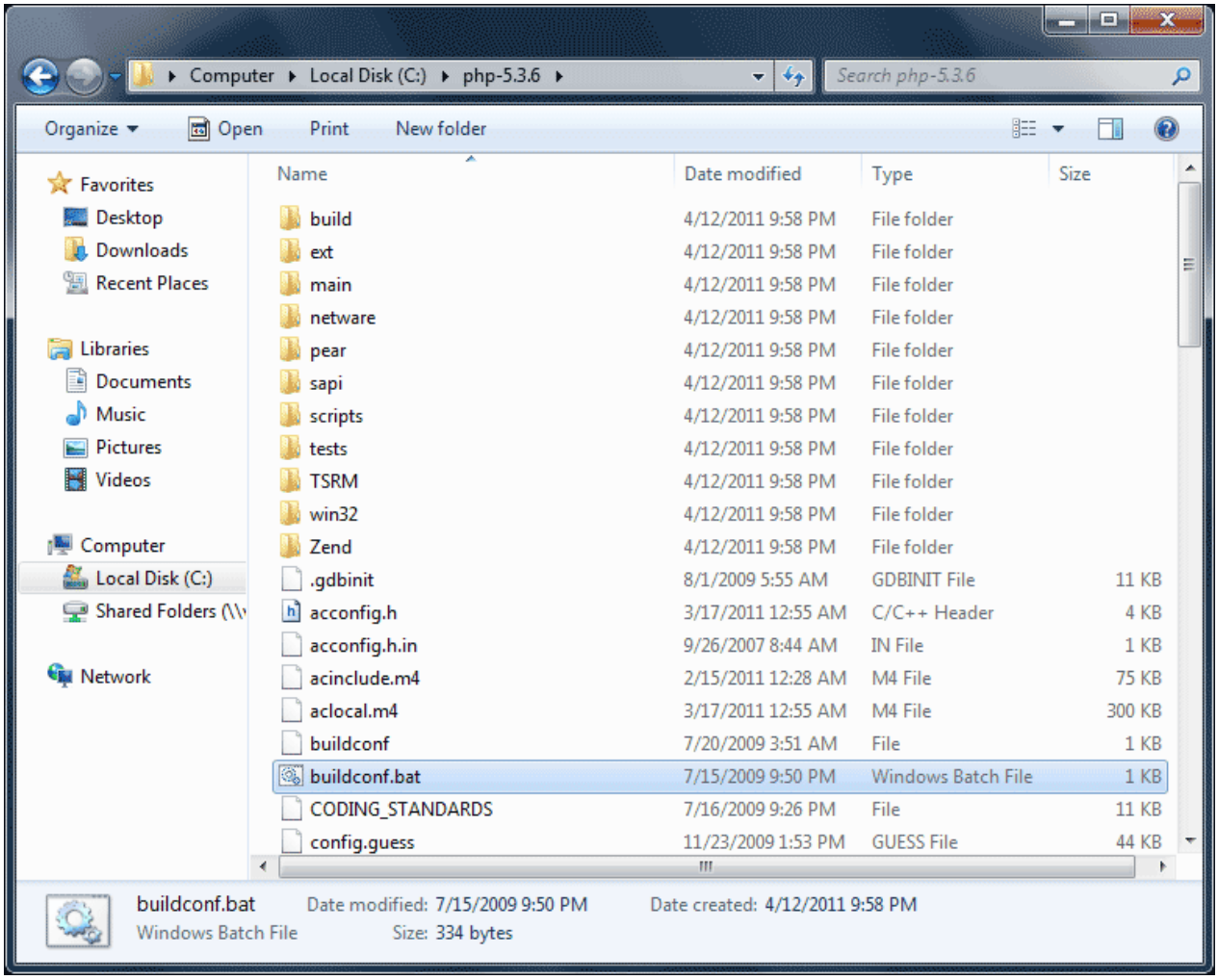
2. Run **setenv /x64 /release**.



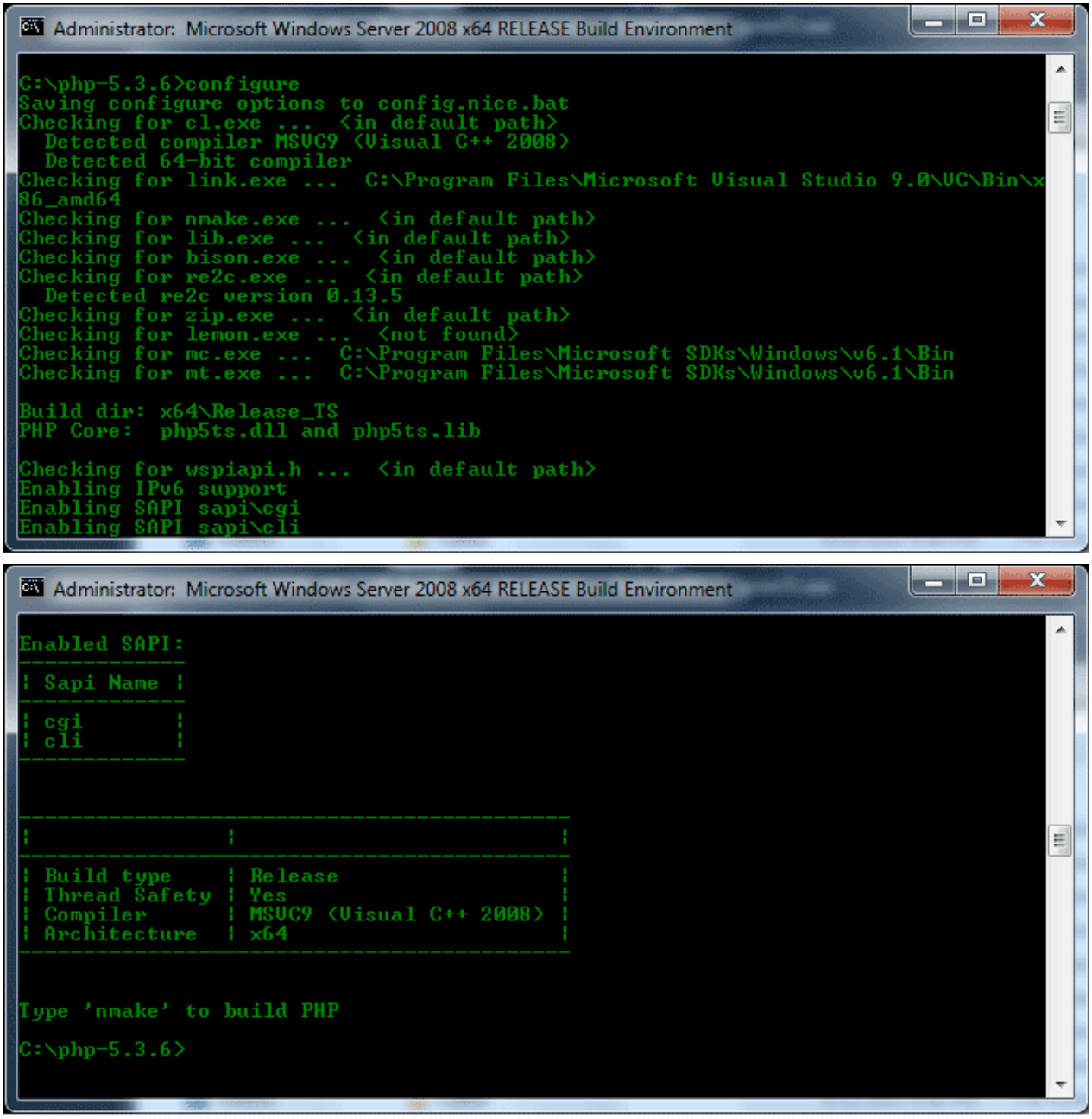
3. Enter PHP 5.6.x or 7.1.x source code directory in the command prompt and run **buildconf** to generate the **configure.js** file.



Or you can also double-click the **buildconf.bat** file.



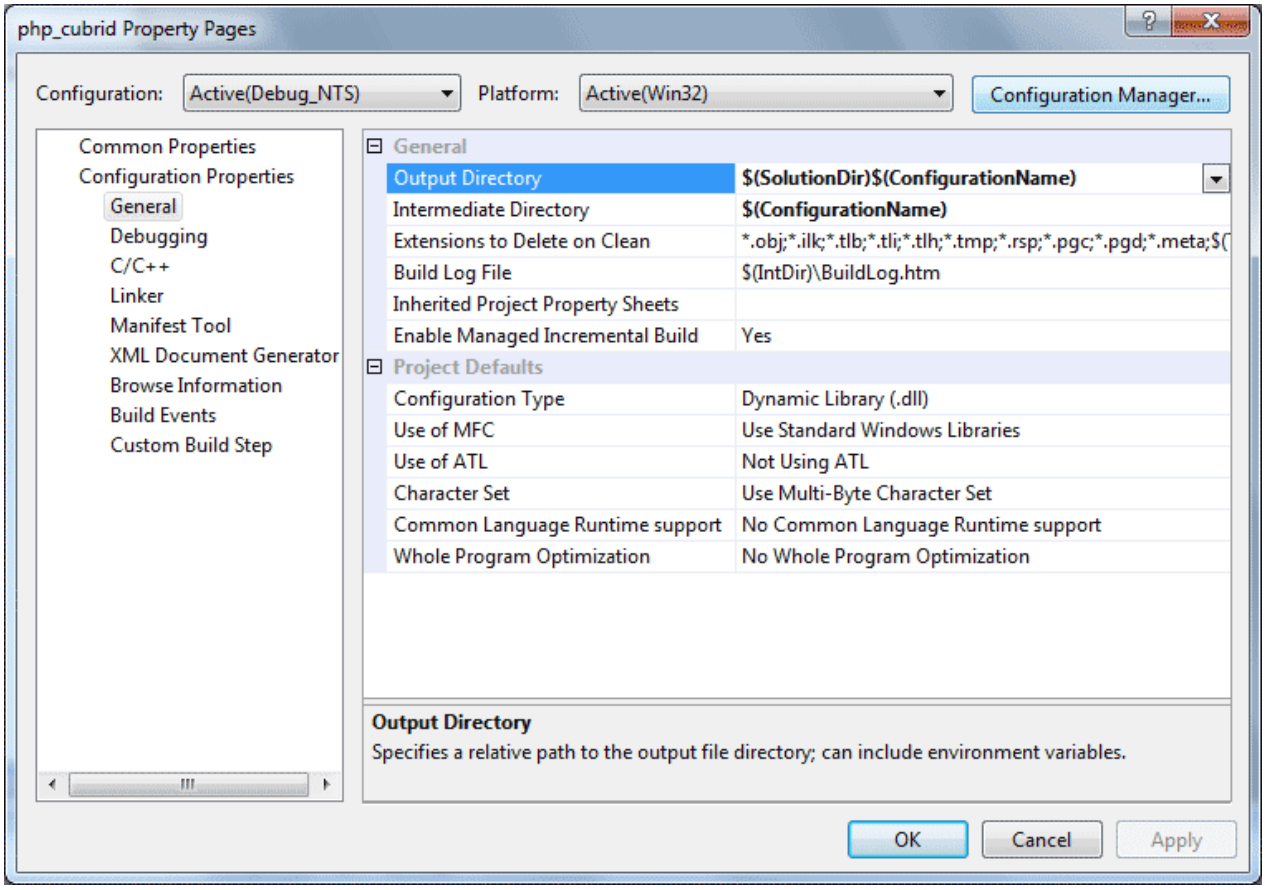
4. Run the **configure** command to configure the PHP project.



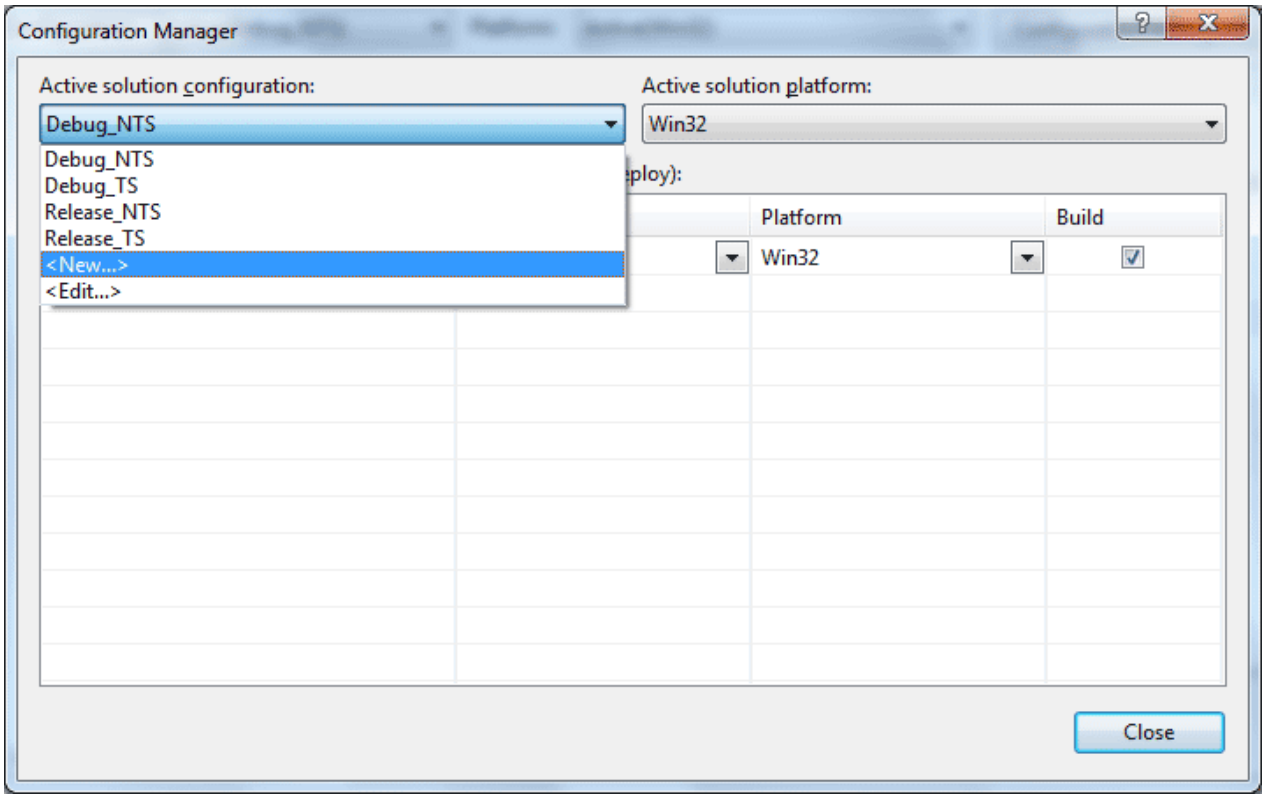
Building CUBRID PHP dirver

1. Open the **php_cubrid.vcproj** file under the **\win** directory. In the [Solution Explorer] on the left, right-click on the **php_cubrid** project name and select [Properties].

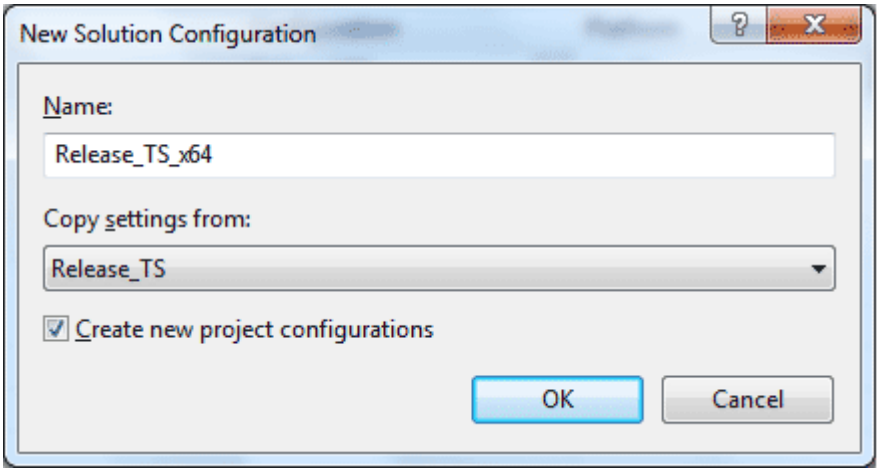
2. On the top right corner of the [Property Pages] dialog box, click [Configuration Manager].



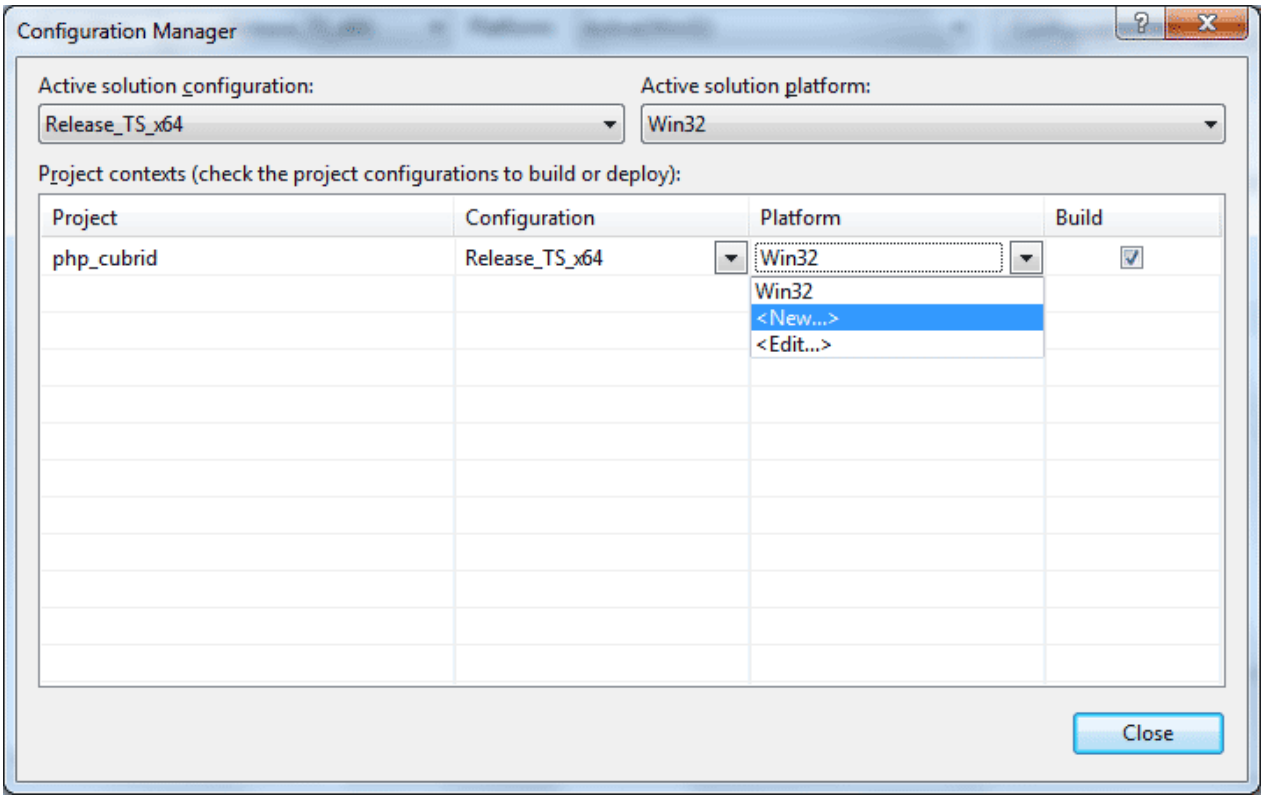
3. In the [Configuration Manager] dialog box, you can see four types of configurations (Release_TS, Release_NTS, Debug_TS, and Debug_NTS) in the [Active solution configuration] dropdown list. Select **New** in the dropdown list so that you can create a new one for your x64 build.



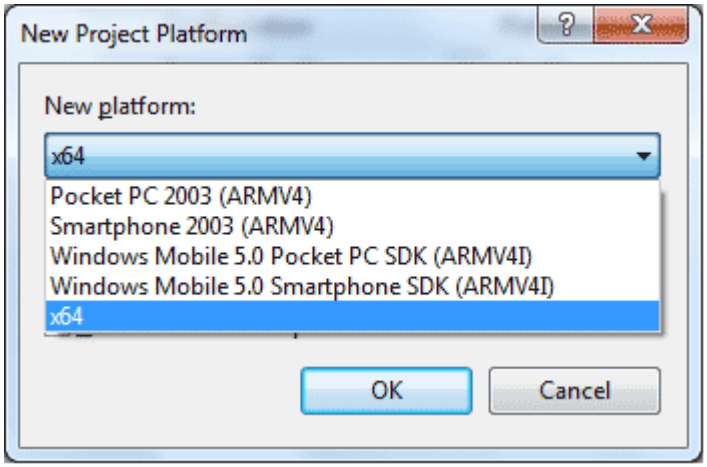
4. In the [New Solution Configuration] dialog box, enter a value in the **Name** box (e.g., **Release_TS_x64**). In the [Copy settings from] dropdown list, select the corresponding x86 configuration and click [OK].



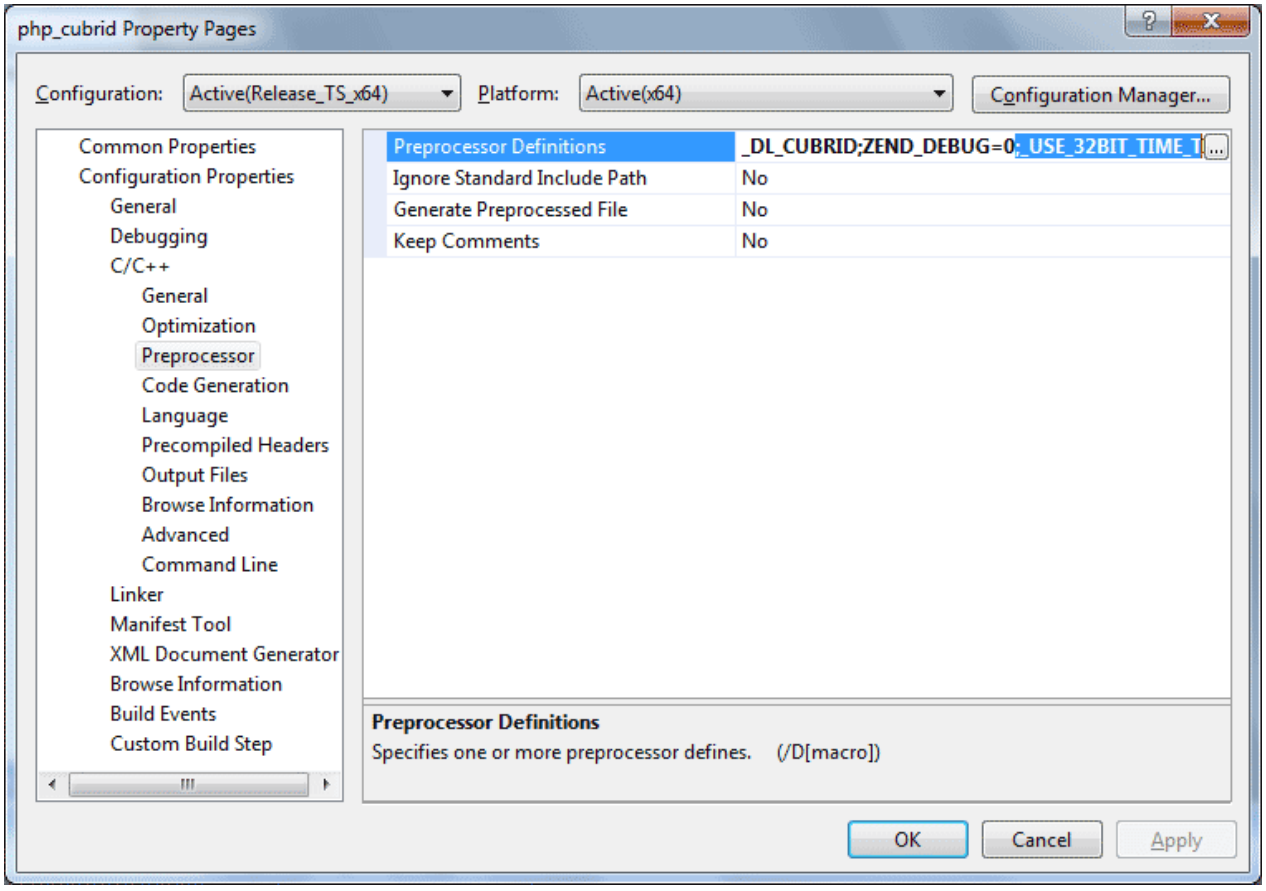
5. In the [Configuration Manager] dialog box, select the value **x64** in the [Platform] dropdown list. If it does not exist, select **New**.



   - In the [New Project Platform] dialog box, select **x64** option in the [New platform] dropdown list.



6. In the [Property Pages] dialog box, select [Preprocessor] under the [C/C++] tree node. In [Preprocessor Definitions], delete **_USE_32BIT_TIME_T** and click [OK] to close the dialog box.



7. Press the <F7> key to compile. Now you will get the CUBRID PHP driver for 64-bit Windows.

PHP Programming

Connecting to a Database

The first step of database applications is to use cubrid_connect () or cubrid_connect_with_url () function which provides database connection. Once cubrid_connect () or cubrid_connect_with_url

() function is executed successfully, you can use any functions available in the database. It is very important to call the cubrid_disconnect () function before applications are terminated. The cubrid_disconnect () function terminates the current transaction as well as the connection handle and all request handles created by the cubrid_connect () function.

Note

- The database connection in thread-based programming must be used independently each other.
- In autocommit mode, the transaction is not committed if all results are not fetched after running the SELECT statement. Therefore, although in autocommit mode, you should end the transaction by executing COMMIT or ROLLBACK if some error occurs during fetching for the resultset.

Transactions and Autocommit

CUBRID PHP supports transaction and autocommit mode. Autocommit mode means that every query that you run has its own implicit transaction. You can use the cubrid_get_autocommit () function to get the status of current connection autocommit mode and use the cubrid_set_autocommit () function to enable/disable autocommit mode of current connection. In autocommit mode, any transactions being executed are committed regardless of whether it is set to **ON** or **OFF**.

The default value of autocommit mode upon application startup is configured by the **CCI_DEFAULT_AUTOCOMMIT** (broker parameter). If the broker parameter value is not configured, the default value is set to **ON**.

If you set autocommit mode to **OFF** in the cubrid_set_autocommit () function, you can handle transactions by specifying a proper function; to commit transactions, use the cubrid_commit () function and to roll back transactions, use the cubrid_rollback () function. If you use the cubrid_disconnect () function, transactions will be disconnected and jobs which have not been committed will be rolled back.

Processing Queries

Executing queries

The following are the basic steps to execute queries.

- Creating a connection handle
- Creating a request handle for an SQL query request

- Fetching result
- Disconnecting the request handle

```
$con = cubrid_connect("192.168.0.10", 33000, "demodb");
if($con) {
    $req = cubrid_execute($con, "select * from code");
    if($req) {
        while ($row = cubrid_fetch($req)) {
            echo $row["s_name"];
            echo $row["f_name"];
        }
        cubrid_close_request($req);
    }
    cubrid_disconnect($con);
}
```

Column types and names of the query result

The cubrid_column_types () function is used to get arrays containing column types and the cubrid_column_types () functions is used to get arrays containing colunm names.

```
$req = cubrid_execute($con, "select host_year, host_city from
olympic");
if($req) {
    $col_types = cubrid_column_types($req);
    $col_names = cubrid_column_names($req);

    while (list($key, $col_type) = each($col_types)) {
        echo $col_type;
    }
    while (list($key, $col_name) = each($col_names))
        echo $col_name;
    }
    cubrid_close_request($req);
}
```

Controlling a cursor

The cubrid_move_cursor () function is used to move a cursor to a specified position from one of three points: beginning of the query result, current cursor position, or end of the query result).

```
$req = cubrid_execute($con, "select host_year, host_city from
olympic order by host_year");
if($req) {
    cubrid_move_cursor($req, 20, CUBRID_CURSOR_CURRENT)
    while ($row = cubrid_fetch($req, CUBRID_ASSOC)) {
        echo $row["host_year"]." ";
```

```
        echo $row["host_city"]."\n";
    }
}
```

Result array types

One of the following three types of arrays is used in the result of the cubrid_fetch () function. The array types can be determined when the cubrid_fetch () function is called. Of array types, the associative array uses string indexes and the numeric array uses number indexes. The last array includes both associative and numeric arrays.

- Numeric array

```
while (list($id, $name) = cubrid_fetch($req, CUBRID_NUM)) {
    echo $id;
    echo $name;
}
```

- Associative array

```
while ($row = cubrid_fetch($req, CUBRID_ASSOC)) {
    echo $row["id"];
    echo $row["name"];
}
```

Catalog Operations

The cubrid_schema () function is used to get database schema information such as classes, virtual classes, attributes, methods, triggers, and constraints. The return value of the cubrid_schema () function is a two-dimensional array.

```
$pk = cubrid_schema($con, CUBRID_SCH_PRIMARY_KEY, "game");
if ($pk) {
    print_r($pk);
}

$fk = cubrid_schema($con, CUBRID_SCH_IMPORTED_KEYS, "game");
if ($fk) {
    print_r($fk);
}
```

Error Handling

When an error occurs, most of PHP interfaces display error messages and return false or -1. The cubrid_error_msg (), cubrid_error_code () and cubrid_error_code_facility () functions are used to check error messages, error codes, and error facility codes.

The return value of the cubrid_error_code_facility () function is one of the following (**CUBRID_FACILITY_DBMS** (DBMS error), **CUBRID_FACILITY_CAS** (CAS server error), **CUBRID_FACILITY_CCI** (CCI error), or **CUBRID_FACILITY_CLIENT** (PHP module error).

Using OIDs

The OID value in the currently updated f record by using the cubrid_current_oid function if it is used together with query that can update the **CUBRID_INCLUDE_OID** option in the cubrid_execute () function.

```
$req = cubrid_execute($con, "select * from person where id = 1",
CUBRID_INCLUDE_OID);
if ($req) {
    while ($row = cubrid_fetch($req)) {
        echo cubrid_current_oid($req);
        echo $row["id"];
        echo $row["name"];
    }
    cubrid_close_request($req);
}
```

Values in every attribute, specified attributes, or a single attribute of an instance can be obtained by using OIDs.

If any attributes are not specified in the cubrid_get () function, values in every attribute are returned (a). If attributes is specified in the array data type, the array containing the specified attribute value is returned in the associative array (b). If a single attribute it is specified in the string type, a value of the attributed is returned (c).

```
$attrarray = cubrid_get ($con, $oid); // (a)
$attrarray = cubrid_get ($con, $oid, array("id", "name")); // (b)
$attrarray = cubrid_get ($con, $oid, "id"); // (c)
```

The attribute values of an instance can be updated by using OIDs. To update a single attribute value, specify attribute name and value in the string type (a). To update multiple attribute values, specify attribute names and values in the associative array (b).

```
$cubrid_put ($con, $oid, "id", 1); // (a)
$cubrid_put ($con, $oid, array("id"=>1, "name"=>"Tomas")); // (b)
```

Using Collections

You can use the collection data types through PHP array data types or functions that support array data types. The following example shows how to fetch query result by using the cubrid_fetch () function.

```
$row = cubrid_fetch ($req);
$col = $row["customer"];
while (list ($key, $cust) = each ($col)) {
   echo $cust;
}
```

You can get values of collection attributes. The example shows how to get values of collection attributes by using the cubrid_col_get () function.

```
$tels = cubrid_col_get ($con, $oid, "tels");
while (list ($key, $tel) = each ($tels)) {
   echo $tel."\n";
}
```

You can directly update values of collection types by using cubrid_set_add() or cubrid_set_drop() function.

```
$tels = cubrid_col_get ($con, $oid, "tels");
while (list ($key, $tel) = each ($tels)) {
   $res = cubrid_set_drop ($con, $oid, "tel", $tel);
}

cubrid_commit ($con);
```

Note

If a string longer than defined max length is inserted (**INSERT**) or updated (**UPDATE**), the string will be truncated.

PHP API

See http://ftp.cubrid.org/CUBRID_Docs/Drivers/.

PDO Driver

The official CUBRID PHP Data Objects (PDO) driver is available as a PECL package and it implements the PDO interface to enable access from PDO to CUBRID.PDO is available with PHP 5.1. For PHP 5.0, you can use it as a PECL extension. PDO cannot run with earlier versions of PHP 5.0 because it requires the new OO features in the core of PHP 5.0.

PDO provides a data-access abstraction layer, which means that, regardless of which database you are using, you use the same functions to issue queries and fetch data; PDO does not provide a database abstraction. Using PDO as a database interface layer can have important advantages over "direct" PHP database drivers as follows:

- Portable PHP code between different databases and database abstraction.
- Supports SQL parameters and bind.
- Safer SQLs (syntax verification, escaping, it helps protect against SQL injections etc.)
- Cleaner programming model

In particular, having a CUBRID PDO driver means that any application that uses PDO as a database interface should work with CUBRID.

CUBRID PDO driver is based on CCI API so affected by CCI configurations such as **CCI_DEFAULT_AUTOCOMMIT**.

Installing and Configuring PDO

Linux

Requirements

- Operating system: 32-bit or 64-bit Linux
- Web server: Apache
- PHP: 5.6.x, 7.1.x, or 7.4.x (https://www.php.net/downloads.php)

Installing CUBRID PHP Driver using PECL

If **PECL** package has been installed on your system, the installation of CUBRID PDO driver is straightforward. **PECL** will download and compile the driver for you.

1. Enter the following command to install the latest version of CUBRID PDO driver.

```
sudo pecl install pdo_cubrid
```

If you need earlier versions of the driver, you can install exact versions as follows:

```
sudo pecl install pdo_cubrid-8.3.1.0003
```

During the installation, you will be prompted to enter **CUBRID base install dir autodetect :**. Just to make sure your installation goes smoothly, enter the full path to the directory where you have CUBRID installed. For example, if CUBRID has been installed at **/home/cubridtest/CUBRID**, then enter **/home/cubridtest/CUBRID**.

2. Edit the configuration file.
    - If you are using CentOS 6.0 and later or Fedora 15 and later, create a file named **pdo_cubrid.ini**, enter a command line **extension=pdo_cubrid.so**, and store the file in the **/etc/php.d** directory.
    - If you are using earlier versions of CentOS or Fedora 15, edit the **php.ini** file (default location: **/etc/php5/apache2** or **/etc/**) and add the following two command lines at the end of the file.

```
[CUBRID]
extension=pdo_cubrid.so
```

3. Restart the web server to apply changes.

Windows

Requirements

- Operating system: 32-bit or 64-bit Windows
- Web server: Apache or IIS
- PHP: 5.6.x, 7.1.x, or 7.4.x (https://windows.php.net/download/)
- For PHP 7.1.x or 7.4.x, you need to install Microsoft Visual C++ 2015 Redistributable Package for 32bit or 64bit.

Downloading and Installing Compiled CUBRID PDO Driver

First, download CUBRID PHP/PDO driver of which versions match the versions of your operating system and PHP installed at https://www.cubrid.org/downloads#pdo.

After you download the driver, you will see the **php_cubrid.dll** file for CUBRID PHP driver or the **php_pdo_cubrid.dll** file for CUBRID PDO driver. Follow the steps below to install it.

1. Copy this driver to the default PHP extensions directory (usually located at **C:\Program Files\PHP\ext**).
2. Set your system environment. Check if the environment variable **PHPRC** is **C:\Program Files\PHP** and system variable path is added with **%PHPRC%** and **%PHPRC%\ext**.
3. Edit **php.ini** (**C:\Program Files\PHP\php.ini**) and add the following two lines at the end of the **php.ini** file.

   ```
   [PHP_PDO_CUBRID]
   extension=php_pdo_cubrid.dll
   ```

   For CUBRID PHP driver, add command lines below.

   ```
   [PHP_PDO_CUBRID]
   extension = php_pdo_cubrid.dll
   ```

4. Restart your web server to apply changes.

Building CUBRID PHP Driver from Source Code

Building CUBRID PDO driver and Building CUBRID PHP driver from source code are the same.

PDO Programming

Data Source Name (DSN)

The PDO_CUBRID data source name (DSN) consists of the following elements:

Element	Description
DSN prefix	The DSN prefix is **cubrid**.
host	The hostname on which the database server resides

Element	Description
port	The port number where the database server is listening
dbname	The name of the database

Example

```
"cubrid:host=127.0.0.1;port=33000;dbname=demodb"
```

Predefined Constants

The constants defined by CUBRID PDO driver are available only when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using PDO driver. Using driver-specific attributes with another driver may result in unexpected behavior.

The PDO::getAttribute() function may be used to obtain the **PDO_ATTR_DRIVER_NAME** attribute value to check the driver if your code can run.

The constants below can be used with the PDO::cubrid_schema function to get schema information.

Constant	Type	Description
PDO::CUBRID_SCH_TABLE	integer	Gets name and type of table in CUBRID.
PDO::CUBRID_SCH_VIEW	integer	Gets name and type of view in CUBRID.
PDO::CUBRID_SCH_QUERY_SPEC	integer	Get the query definition of view.
PDO::CUBRID_SCH_ATTRIBUTE	integer	

Constant	Type	Description
		Gets the attributes of table column.
PDO::CUBRID_SCH_TABLE_ATTRIBUTE	integer	Gets the attributes of table.
PDO::CUBRID_SCH_TABLE_METHOD	integer	Gets the instance method. The instance method is a method called by a class instance. It is used more often than the class method because most operations are executed in the instance.
PDO::CUBRID_SCH_METHOD_FILE	integer	Gets the information of the file where the method of the table is defined.
PDO::CUBRID_SCH_SUPER_TABLE	integer	Gets the name and type of table which table inherits attributes from.
PDO::CUBRID_SCH_SUB_TABLE	integer	Gets the name and type of table which inherits attributes from this table.
PDO::CUBRID_SCH_CONSTRAINT	integer	Gets the table constraints.
PDO::CUBRID_SCH_TRIGGER	integer	Gets the table triggers.
PDO::CUBRID_SCH_TABLE_PRIVILEGE	integer	Gets the privilege information of table.

Constant	Type	Description
PDO::CUBRID_SCH_COL_PRIVILEGE	integer	Gets the privilege information of column.
PDO::CUBRID_SCH_DIRECT_SUPER_TABLE	integer	Gets the direct super table of table.
PDO::CUBRID_SCH_DIRECT_PRIMARY_KEY	integer	Gets the table primary key.
PDO::CUBRID_SCH_IMPORTED_KEYS	integer	Gets imported keys of table.
PDO::CUBRID_SCH_EXPORTED_KEYS	integer	Gets exported keys of table.
PDO::CUBRID_SCH_CROSS_REFERENCE	integer	Gets reference relationship of two tables.

PDO Sample Program

Verifying CUBRID PDO Driver Version

If you want to verify that the CUBRID PDO driver is accessible, you can use the PDO::getAvailableDrivers () function.

```
<?php
echo'PDO Drivers available:
';
foreach(PDO::getAvailableDrivers()as $driver)
{
if($driver =="cubrid"){
echo" - Driver: <b>".$driver.'</b>
';
}else{
echo" - Driver: ".$driver.'
';
```

```
}
}
?>
```

This script will output all the currently installed PDO drivers:

```
PDO Drivers available:
- Driver: mysql
- Driver: pgsql
- Driver: sqlite
- Driver: sqlite2
- Driver: cubrid
```

Connecting to CUBRID

Use the data source name (DSN) to connect to the database server. For details about DSN, see Data Source Name (DSN).

Below is a simple PHP example script which performs a PDO connection to the CUBRID *demodb* database. You can notice that errors are handling in PDO by using a try-catch mechanism and the connection is closed by assigning **NULL** to the connection object.

```
<?php
$database ="demodb";
$host ="localhost";
$port ="30000";//use default value
$username ="dba";
$password ="";

try{
//cubrid:host=localhost;port=33000;dbname=demodb
$conn_str ="cubrid:dbname=".$database.";host=".$host.";port=".$port;
echo"PDO connect string: ".$conn_str."
";
$db =new PDO($conn_str, $username, $password );
echo"PDO connection created ok!"."
";
$db = null;//disconnect
}catch(PDOException $e){
echo"Error: ".$e->getMessage()."
";
}
?>
```

If connection succeeds, the output of this script is as follows:

```
PDO connect string: cubrid:dbname=demodb;host=localhost;port=30000
PDO connection created ok!
```

Executing a SELECT Statement

In PDO, there is more than one way to execute SQL queries.

- Using the query () function
- Using prepared statements (see prepare ()/ execute ()) functions)
- Using the exec () function

The example script below shows the simplest one - using the query () function. You can retrieve the return values from the resultset (a PDOStatement object) by using the column names, like $rs["*column_name*"].

Note that when you use the query () function, you must ensure that the query code is properly escaped. For information about escaping, see PDO::quote () function.

```
<?php
include("_db_config.php");
include("_db_connect.php");

$sql ="SELECT * FROM code";
echo"Executing SQL: <b>".$sql.'</b>
';
echo'
';

try{
foreach($db->query($sql)as $row){
echo $row['s_name'].' - '. $row['f_name'].'
';
}
}catch(PDOException $e){
echo $e->getMessage();
}

$db = null;//disconnect
?>
```

The output of the script is as follows:

```
Executing SQL: SELECT * FROM code

X - Mixed
```

```
W - Woman
M - Man
B - Bronze
S - Silver
G - Gold
```

Executing an UPDATE Statement

The following example shows how to execute an UPDATE statement by using a prepared statement and parameters. You can use the exec () function as an alternative.

```
<?php
include("_db_config.php");
include("_db_connect.php");

$s_name ='X';
$f_name ='test';
$sql ="UPDATE code SET f_name=:f_name WHERE s_name=:s_name";

echo"Executing SQL: <b>".$sql.'</b>
';
echo'
';

echo":f_name: <b>".$f_name.'</b>
';
echo'
';
echo":s_name: <b>".$s_name.'</b>
';
echo'
';

$qe = $db->prepare($sql);
$qe->execute(array(':s_name'=>$s_name,':f_name'=>$f_name));

$sql ="SELECT * FROM code";
echo"Executing SQL: <b>".$sql.'</b>
';
echo'
';

try{
foreach($db->query($sql)as $row){
echo $row['s_name'].' - '. $row['f_name'].'
';
}
}catch(PDOException $e){
echo $e->getMessage();
```

```
}

$db = null;//disconnect
?>
```

The output of the script is as follows:

```
Executing SQL: UPDATE code SET f_name=:f_name WHERE s_name=:s_name

:f_name: test

:s_name: X

Executing SQL: SELECT * FROM code

X - test
W - Woman
M - Man
B - Bronze
S - Silver
G - Gold
```

Using prepare and bind

Prepared statements are one of the major features offered by PDO and you can take following benefits by using them.

- SQL prepared statements need to be parsed only once even if they are executed multiple times with different parameter values. Therefore, using a prepared statement minimizes the resources and ,in general, the prepared statements run faster.
- It helps to prevent SQL injection attacks by eliminating the need to manually quote the parameters; however, if other parts of the SQL query are being built up with unescaped input, SQL injection is still possible.

The example script below shows how to retrieve data by using a prepared statement.

```
<?php
include("_db_config.php");
include("_db_connect.php");

$sql ="SELECT * FROM code WHERE s_name NOT LIKE :s_name";
echo"Executing SQL: <b>".$sql.'</b>
';

$s_name ='xyz';
echo":s_name: <b>".$s_name.'</b>
```

```
';

echo'
';

try{
$stmt = $db->prepare($sql);

$stmt->bindParam(':s_name', $s_name, PDO::PARAM_STR);
$stmt->execute();

$result = $stmt->fetchAll();
foreach($result as $row)
{
echo $row['s_name'].' - '. $row['f_name'].'
';
}
}catch(PDOException $e){
echo $e->getMessage();
}
echo'
';

$sql ="SELECT * FROM code WHERE s_name NOT LIKE :s_name";
echo"Executing SQL: <b>".$sql.'</b>
';

$s_name ='X';
echo":s_name: <b>".$s_name.'</b>
';

echo'
';

try{
$stmt = $db->prepare($sql);

$stmt->bindParam(':s_name', $s_name, PDO::PARAM_STR);
$stmt->execute();

$result = $stmt->fetchAll();
foreach($result as $row)
{
echo $row['s_name'].' - '. $row['f_name'].'
';
}
$stmt->closeCursor();
}catch(PDOException $e){
echo $e->getMessage();
}
echo'
';
```

```
$db = null;//disconnect
?>
```

The output of the script is as follows:

```
Executing SQL: SELECT * FROM code WHERE s_name NOT LIKE :s_name
:s_name: xyz

X - Mixed
W - Woman
M - Man
B - Bronze
S - Silver
G - Gold

Executing SQL: SELECT * FROM code WHERE s_name NOT LIKE :s_name
:s_name: X

W - Woman
M - Man
B - Bronze
S - Silver
G - Gold
```

Using the PDO::getAttribute() Function

The PDO::getAttribute () function is very useful to retrieve the database connection attributes. For example,

- Driver name
- Database version
- Autocommit state
- Error mode

Note that if you want to set attributes values (assuming that they are writable), you should use the PDO::setAttribute function.

The following example script shows how to retrieve the current versions of client and server by using the PDO::getAttribute () function.

```
<?php
include("_db_config.php");
include("_db_connect.php");
```

```
echo"Driver name: <b>".$db->getAttribute(PDO::ATTR_DRIVER_NAME)."</
b>";
echo"
";
echo"Client version: <b>".$db-
>getAttribute(PDO::ATTR_CLIENT_VERSION)."</b>";
echo"
";
echo"Server version: <b>".$db-
>getAttribute(PDO::ATTR_SERVER_VERSION)."</b>";
echo"
";

$db = null;//disconnect
?>
```

The output of the script is as follows:

```
Driver name: cubrid
Client version: 8.3.0
Server version: 8.3.0.0337
```

CUBRID PDO Extensions

In CUBRID, the PDO::cubrid_schema() function is offered as an extension; the function is used to retrieve the database schema and metadata information. Below is an example script that returns information about primary key for the *nation* table by using this function.

```
<?php
include("_db_config.php");
include("_db_connect.php");
try{
echo"Get PRIMARY KEY for table: <b>nation</b>:

";
$pk_list = $db->cubrid_schema(PDO::CUBRID_SCH_PRIMARY_KEY,"nation");
print_r($pk_list);
}catch(PDOException $e){
echo $e->getMessage();
}

$db = null;//disconnect
?>
```

The output of the script is as follows:

```
Get PRIMARY KEY for table: nation:
Array ( [0] => Array ( [CLASS_NAME] => nation [ATTR_NAME] => code
[KEY_SEQ] => 1 [KEY_NAME] => pk_nation_code ) )
```

PDO API

For more information about PHP Data Objects (PDO) API, see https://www.php.net/manual/en/book.pdo.php. The API provided by CUBRID PDO driver is as follows:

For more information about CUBRID PDO API provides, see http://ftp.cubrid.org/CUBRID_Docs/Drivers/PDO/.

ODBC Driver

CUBRID ODBC driver supports ODBC version 3.52. It also ODBC core and some parts of Level 1 and Level 2 API. Because CUBRID ODBC driver has been developed based on the ODBC Spec 3.x, backward compatibility is not completely ensured for programs written based on the ODBC Spec 2.x.

CUBRID ODBC driver is written based on CCI API, but it's not affected by CCI_DEFAULT_AUTOCOMMIT exceptionally.

Note

ODBC is not affected by CCI_DEFAULT_AUTOCOMMIT is from 9.3 version. In the previous versions, you should set CCI_DEFAULT_AUTOCOMMIT as OFF.

Data Type Mapping Between CUBRID and ODBC

The following table shows the data type mapping relationship between CUBRID and ODBC.

CUBRID Data Type	ODBC Data Type
CHAR	SQL_CHAR
VARCHAR	SQL_VARCHAR

CUBRID Data Type	ODBC Data Type
STRING	SQL_LONGVARCHAR
BIT	SQL_BINARY
VARYING BIT	SQL_VARBINARY
NUMERIC	SQL_NUMERIC
INT	SQL_INTEGER
SHORT	SQL_SMALLINT
FLOAT	SQL_FLOAT
DOUBLE	SQL_DOUBLE
BIGINT	SQL_BIGINT
DATE	SQL_TYPE_DATE
TIME	SQL_TYPE_TIME
TIMESTAMP	SQL_TYPE_TIMESTAMP
DATETIME	SQL_TYPE_TIMESTAMP
OID	SQL_CHAR(32)
SET, MULTISET, SEQUENCE	SQL_VARCHAR(MAX_STRING_LENGTH)

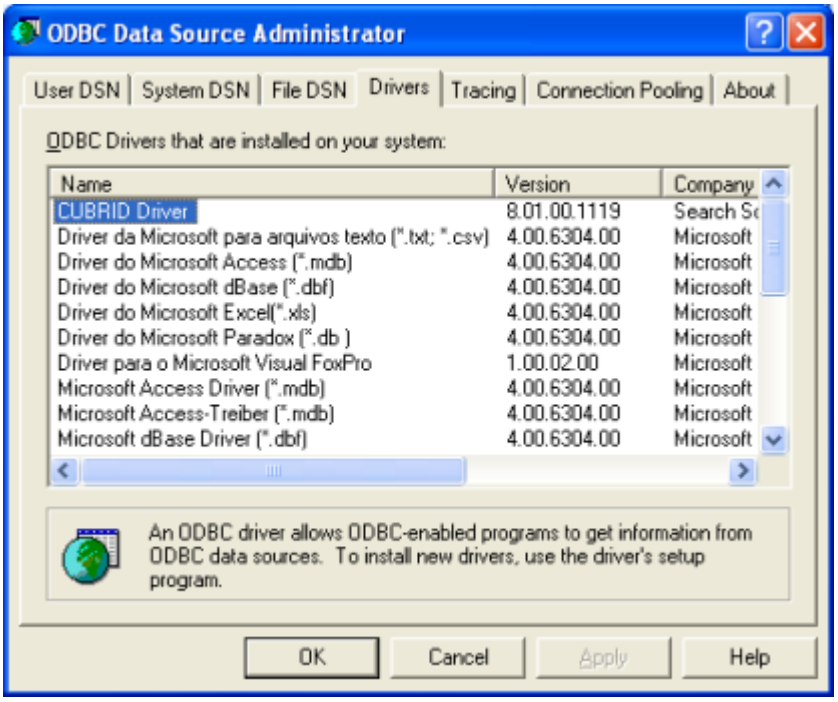
Configuring and Environment ODBC

Requirements

- CUBRID 2008 R4.4 (8.4.4) or later (32-bit or 64-bit)

Configuring CUBRID ODBC Driver

CUBRID ODBC driver is automatically installed upon CUBRID installation. You can check whether it is properly installed in the [Control Panel] > [Administrative Tools] > [Data Source (ODBC)] > [Drivers] tab.



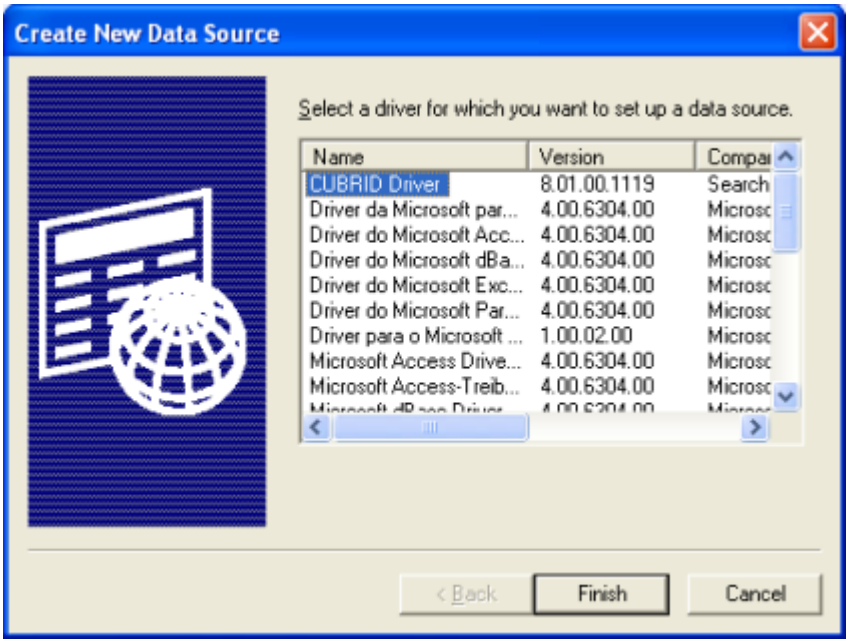
Choosing 32-bit ODBC driver on 64-bit Windows

To run 32-bit application, 32-bit ODBC driver is required. If you have to choose 32-bit ODBC driver on 64-bit Windows, run C:WINDOWSSysWOW64odbcad32.exe .

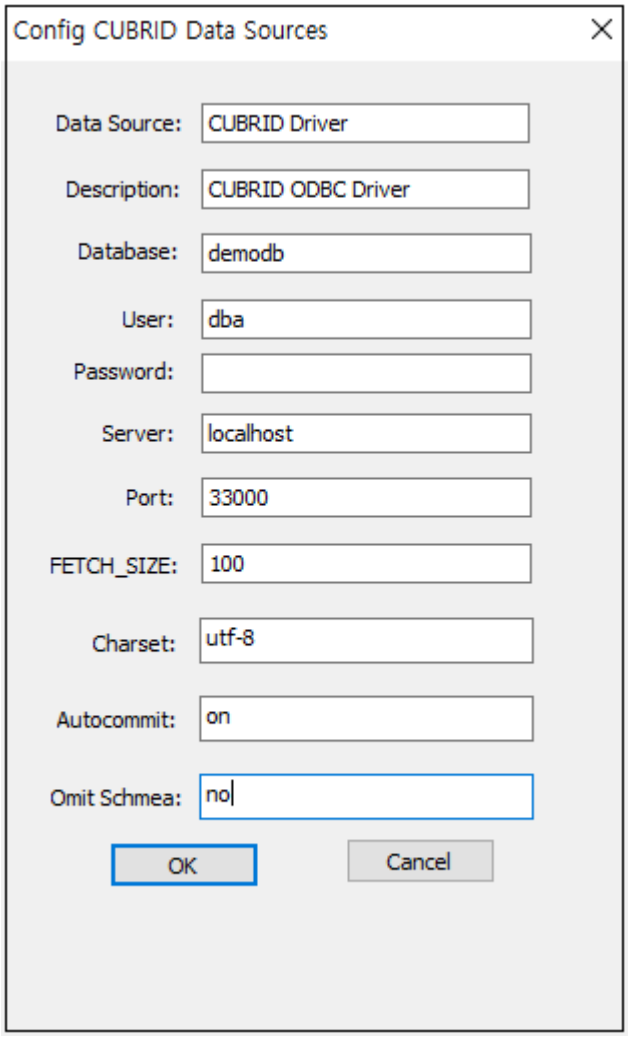
Microsoft Windows 64-bit platform support the environment to run 32-bit application on 64-bit environment, which is called WOW64 (Windows-32-on-Windows-64). This environment maintains its own copy of the registry that is only for 32-bit applications.

Configuring DSN

After you check the CUBRID ODBC driver installed, configure DSN as a database where the applications are trying to connect. To configure, click the [Add] button in the ODBC Data Source Administrator dialog box. Then, the following dialog box will appear. Select “CUBRID Driver” and then click the [Finish] button.

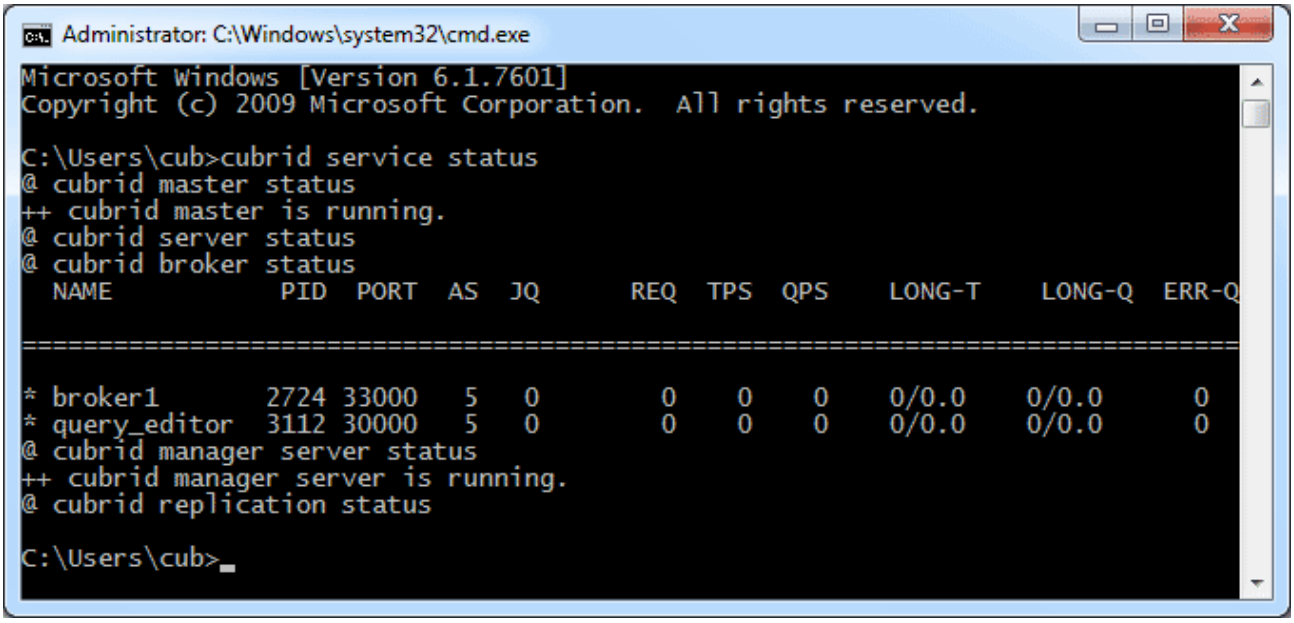


In the [Config CUBRID Data Sources] dialog box, enter information as follows:



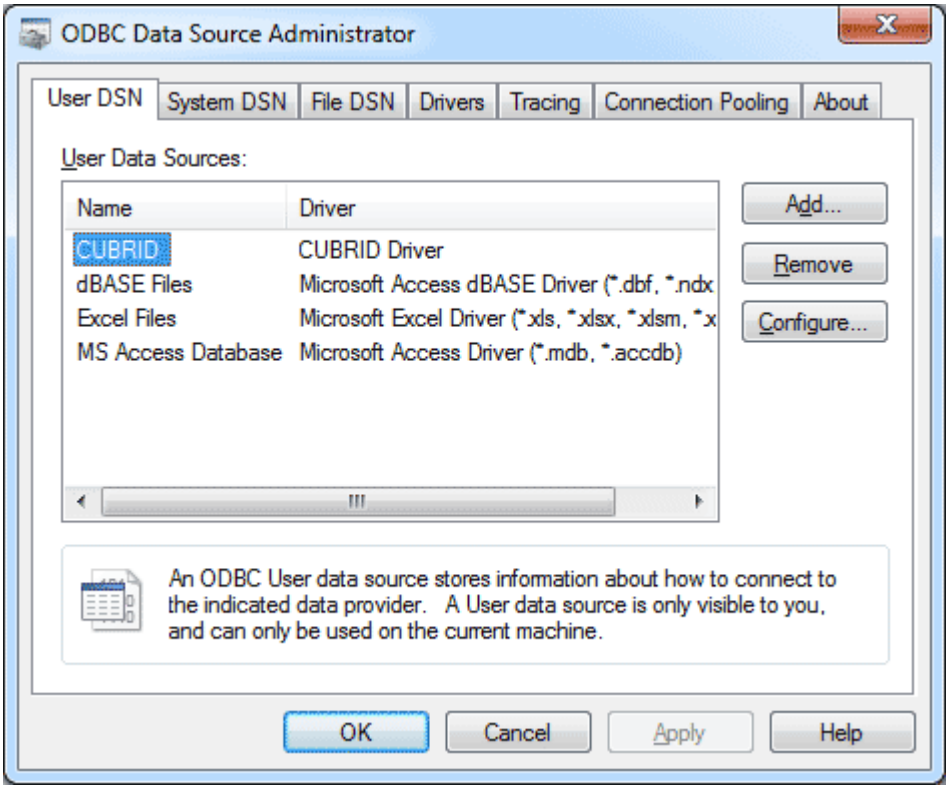
- **DSN** : The name of a source data
- **DB Name** : The name of a database to be connected

- **DB User** : The name of a database user
- **Password** : The password of a database user
- **Server Address** : The host address of a database. The value should be either localhost or the IP address of other server.
- **Server Port** : The number of a broker port. You can check the CUBRID broker port number in the **cubrid_broker.conf** file. The default value is 33,000. To verify the port number, check the BROKER_PORT value in the **cubrid_broker.conf** file or enter the **cubrid service status** in the command prompt. The result will be displayed as follows:



- **Charset** : The name of the database charset.
- **Autocommit** : Autocommit mode. Default is OFF.
- **Omit Schema** : This feature provides only the schema information created by the logged-in user when a user requests catalog information in a database server of version 11.2 or higher that supports the schema concept. This feature is required when **ERWin**™ uses CUBRID 11.2 or higher (omit_schema=yes). If **YES**, only information on the Tables/Views/Primary Keys/Foreign Keys owned by the logged-in user are provided, and the owner's name is removed on these objects. Default is **NO**.
- **FETCH_SIZE** : A value configures the number of records fetched from server whenever the **cci_fetch** () function of CCI library (which CUBRID ODBC driver internally uses) is called.

After you filled out every field, click the [OK] button. You will notice that data source is added in the [User Data Sources] as shown below.



Connecting to a Database Directly without DSN

It is also possible to connect to a CUBRID database directly in the application source code by using the connecting string. Below shows the example of connection string.

```
conn = "driver={CUBRID
Driver};server=localhost;port=33000;uid=dba;pwd=;db_name=demodb;"
```

Note

Make sure that your database is running before you try to connect to a CUBRID database. Otherwise, you will receive an error indicating that ODBC call has failed. To start the database called demodb, enter **cubrid server start demodb** in the command prompt.

ODBC Programming

Configuring Connection String

When you are programming CUBRID ODBC, write the connection strings as follows:

Category	Example	Description
Driver	CUBRID Driver Unicode	Driver name
UID	PUBLIC	User ID
PWD	xxx	Password
FETCH_SIZE	100	Fetch size
PORT	33000	The broker port number
SERVER	127.0.0.1	The IP address or the hostname of a CUBRID broker server
DB_NAME	demodb	Database name
DESCRIPTION	cubrid_test	Description
CHARSET	utf-8	Character set
AUTOCOMMIT	ON	Autocommit mode
OMIT_SCHEMA	NO	Single Schema feature (applicable only 11.2 or higher)

The following shows the result of using connection strings above.

```
"DRIVER={CUBRID Driver
Unicode};UID=PUBLIC;PWD=xxx;FETCH_SIZE=100;PORT=33000;SERVER=127.0.0.
1;DB_NAME=demodb;DESCRIPTION=cubrid_test;CHARSET=utf-8;OMIT_SCHEMA=YE
S"
```

If you use UTF-8 unicode, install a driver for unicode and input the driver name in the connection string as “Driver={CUBRID Driver Unicode}”. Unicode is only supported in 9.3.0.0002 or higher version of CUBRID ODBC driver.

Note

- Because a semi-colon (;) is used as a separator in URL string, it is not allowed to use a semi-colon as parts of a password (PWD) when specifying the password in connection strings.
- The database connection in thread-based programming must be used independently each other.
- In autocommit mode, the transaction is not committed if all results are not fetched after running the SELECT statement. Therefore, although in autocommit mode, you should end the transaction by executing COMMIT or ROLLBACK if some error occurs during fetching for the resultset.

ASP Sample Program

In the virtual directory where the ASP sample program runs, right-click “Default Web Site” and click [Properties].



In the picture above, if you select **(All Unassigned)** from the [IP Address] dropdown list under [Web Site Identification], it is recognized as localhost. If you want to see the sample program through a specific IP address, make an IP address recognize a directory as a virtual directory and register the IP address in the registration information.

Create the below code as cubrid.asp and store it in a virtual directory.

```
<HTML>
    <HEAD>
     <meta http-equiv="Content-Type" content="text/html; charset=EUC-
KR">
       <title>CUBRID Query Test Page</title>
  </HEAD>

 <BODY topmargin="0" leftmargin="0">

 <table border="0" width="748" cellspacing="0" cellpadding="0">
   <tr>
     <td width="200"></td>
     <td width="287">
       <p align="center"><font size="3" face="Times New
Roman"><b><font color="#FF0000">CUBRID</font>Query Test</b></font></
td>
     <td width="200"></td>
   </tr>
 </table>

 <form action="cubrid.asp" method="post" >
 <table border="1" width="700" cellspacing="0" cellpadding="0"
height="45">
   <tr>
     <td width="113" valign="bottom" height="16" bgcolor="#DBD7BD"
bordercolorlight="#FFFFCC"><font size="2">SERVER IP</font></td>
     <td width="78"  valign="bottom" height="16" bgcolor="#DBD7BD"
bordercolorlight="#FFFFCC"><font size="2">Broker PORT</font></td>
     <td width="148" valign="bottom" height="16" bgcolor="#DBD7BD"
bordercolorlight="#FFFFCC"><font size="2">DB NAME</font></td>
     <td width="113" valign="bottom" height="16" bgcolor="#DBD7BD"
bordercolorlight="#FFFFCC"><font size="2">DB USER</font></td>
     <td width="113" valign="bottom" height="16" bgcolor="#DBD7BD"
bordercolorlight="#FFFFCC"><font size="2">DB PASS</font></td>
     <td width="80" height="37" rowspan="4"
bordercolorlight="#FFFFCC" bgcolor="#F5F5ED">
       <p><input type="submit" value="Run" name="B1" tabindex="7"></
p></td>
   </tr>
   <tr>
     <td width="113" height="1" bordercolorlight="#FFFFCC"
bgcolor="#F5F5ED"><font size="2"><input type="text" name="server_ip"
size="20" tabindex="1" maxlength="15" value="<%=Request("server_ip")
```

```
%>"></font></td>
     <td width="78"  height="1" bordercolorlight="#FFFFCC"
bgcolor="#F5F5ED"><font size="2"><input type="text" name="cas_port"
size="15" tabindex="2" maxlength="6" value="<%=Request("cas_port")
%>"></font></td>
     <td width="148" height="1" bordercolorlight="#FFFFCC"
bgcolor="#F5F5ED"><font size="2"><input type="text" name="db_name"
size="20" tabindex="3" maxlength="20" value="<%=Request("db_name")
%>"></font></td>
     <td width="113" height="1" bordercolorlight="#FFFFCC"
bgcolor="#F5F5ED"><font size="2"><input type="text" name="db_user"
size="15" tabindex="4" value="<%=Request("db_user")%>"></font></td>
     <td width="113" height="1" bordercolorlight="#FFFFCC"
bgcolor="#F5F5ED"><font size="2"><input type="password"
name="db_pass" size="15" tabindex="5" value="<%=Request("db_pass")
%>"></font></td>
   </tr>
   <tr>
     <td width="573" colspan="5" valign="bottom" height="18"
bordercolorlight="#FFFFCC" bgcolor="#DBD7BD"><font size="2">QUERY</
font></td>
   </tr>
   <tr>
     <td width="573" colspan="5" height="25"
bordercolorlight="#FFFFCC" bgcolor="#F5F5ED"><textarea rows="3"
name="query" cols="92" tabindex="6"><%=Request("query")%></
textarea></td>
   </tr>
 </table>
 </form>
 <hr>

</BODY>
</HTML>

<%
    ' get DSN and SQL statement.
    strIP = Request( "server_ip" )
    strPort = Request( "cas_port" )
    strUser = Request( "db_user" )
    strPass = Request( "db_pass" )
    strName = Request( "db_name" )
    strQuery = Request( "query" )

if strIP = "" then
   Response.Write "Input SERVER_IP."
        Response.End ' exit if no SERVER_IP's input.
    end if
    if strPort = "" then
       Response.Write "Input port number."
        Response.End ' exit if no Port's input.
    end if
```

```
    if strUser = "" then
       Response.Write "Input DB_USER."
        Response.End ' exit if no DB_User's input.
    end if
    if strName = "" then
       Response.Write "Input DB_NAME"
        Response.End ' exit if no DB_NAME's input.
    end if
    if strQuery = "" then
       Response.Write "Input the query you want"
        Response.End ' exit if no query's input.
    end if
 ' create connection object.
  strDsn = "driver={CUBRID Driver};server=" & strIP & ";port=" &
strPort & ";uid=" & strUser & ";pwd=" & strPass & ";db_name=" &
strName & ";"
' DB connection.
Set DBConn = Server.CreateObject("ADODB.Connection")
       DBConn.Open strDsn
    ' run SQL.
    Set rs = DBConn.Execute( strQuery )
    ' show the message by SQL.
    if InStr(Ucase(strQuery),"INSERT")>0 then
        Response.Write "A record is added."
        Response.End
    end if

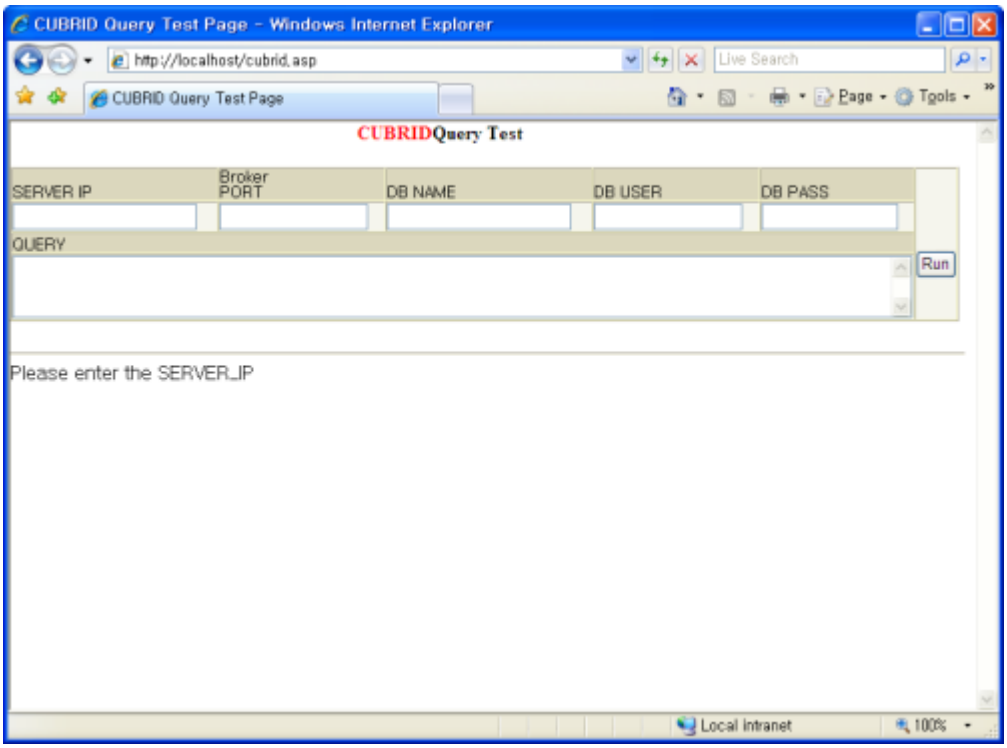
    if InStr(Ucase(strQuery),"DELETE")>0  then
        Response.Write "A record is deleted."
        Response.End
    end if

    if InStr(Ucase(strQuery),"UPDATE")>0  then
        Response.Write "A record is updated."
        Response.End
    end if
%>
<table>
<%
    ' show the field name.
    Response.Write "<tr bgColor=#f3f3f3>"
    For index =0 to ( rs.fields.count-1 )
        Response.Write "<td><b>" & rs.fields(index).name & "</b></
td>"
    Next
    Response.Write "</tr>"
    ' show the field value
    Do While Not rs.EOF
        Response.Write "<tr bgColor=#f3f3f3>"
        For index =0 to ( rs.fields.count-1 )
            Response.Write "<td>" & rs(index) & "</td>"
        Next
```

```
        Response.Write "</tr>"

        rs.MoveNext
    Loop
%>
<%
    set  rs = nothing
%>
</table>
```

You can check the result of the sample program by connecting to http://localhost/cubrid.asp. When you execute the ASP sample code above, you will get the following output. Enter an appropriate value in each field, enter the query statement in the Query field, and click [Run]. The query result will be displayed at the lower part of the page.



ODBC API

For ODBC API, see ODBC API Reference (https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/odbc-api-reference?view=sql-server-ver15) on the MSDN page. See the table below to get information about the list of functions, ODBC Spec version, and compatibility that CUBRID supports.

API	Version Introduced	Standards Compliance	Support
SQLAllocHandle	3.0	ISO 92	YES
SQLBindCol	1.0	ISO 92	YES
SQLBindParameter	2.0	ODBC	YES
SQLBrowseConnect	1.0	ODBC	NO
SQLBulkOperations	3.0	ODBC	YES
SQLCancel	1.0	ISO 92	YES
SQLCloseCursor	3.0	ISO 92	YES
SQLColAttribute	3.0	ISO 92	YES
SQLColumnPrivileges	1.0	ODBC	NO
SQLColumns	1.0	X/Open	YES
SQLConnect	1.0	ISO 92	YES
SQLCopyDesc	3.0	ISO 92	YES
SQLDescribeCol	1.0	ISO 92	YES
SQLDescribeParam	1.0	ODBC	NO

API	Version Introduced	Standards Compliance	Support
SQLDisconnect	1.0	ISO 92	YES
SQLDriverConnect	1.0	ODBC	YES
SQLEndTran	3.0	ISO 92	YES
SQLExecDirect	1.0	ISO 92	YES
SQLExecute	1.0	ISO 92	YES
SQLFetch	1.0	ISO 92	YES
SQLFetchScroll	3.0	ISO 92	YES
SQLForeignKeys	1.0	ODBC	YES (2008 R3.1 or later)
SQLFreeHandle	3.0	ISO 92	YES
SQLFreeStmt	1.0	ISO 92	YES
SQLGetConnectAttr	3.0	ISO 92	YES
SQLGetCursorName	1.0	ISO 92	YES
SQLGetData	1.0	ISO 92	YES
SQLGetDescField	3.0	ISO 92	YES

API	Version Introduced	Standards Compliance	Support
SQLGetDescRec	3.0	ISO 92	YES
SQLGetDiagField	3.0	ISO 92	YES
SQLGetDiagRec	3.0	ISO 92	YES
SQLGetEnvAttr	3.0	ISO 92	YES
SQLGetFunctions	1.0	ISO 92	YES
SQLGetInfo	1.0	ISO 92	YES
SQLGetStmtAttr	3.0	ISO 92	YES
SQLGetTypeInfo	1.0	ISO 92	YES
SQLMoreResults	1.0	ODBC	YES
SQLNativeSql	1.0	ODBC	YES
SQLNumParams	1.0	ISO 92	YES
SQLNumResultCols	1.0	ISO 92	YES
SQLParamData	1.0	ISO 92	YES
SQLPrepare	1.0	ISO 92	YES
SQLPrimaryKeys	1.0	ODBC	

API	Version Introduced	Standards Compliance	Support
			YES (2008 R3.1 or later)
SQLProcedureColumns	1.0	ODBC	YES (2008 R3.1 or later)
SQLProcedures	1.0	ODBC	YES (2008 R3.1 or later)
SQLPutData	1.0	ISO 92	YES
SQLRowCount	1.0	ISO 92	YES
SQLSetConnectAttr	3.0	ISO 92	YES
SQLSetCursorName	1.0	ISO 92	YES
SQLSetDescField	3.0	ISO 92	YES
SQLSetDescRec	3.0	ISO 92	YES
SQLSetEnvAttr	3.0	ISO 92	NO
SQLSetPos	1.0	ODBC	YES
SQLSetStmtAttr	3.0	ISO 92	YES
SQLSpecialColumns	1.0	X/Open	YES

API	Version Introduced	Standards Compliance	Support
SQLStatistics	1.0	ISO 92	YES
SQLTablePrivileges	1.0	ODBC	YES (2008 R3.1 or later)
SQLTables	1.0	X/Open	YES

Backward compatibility is not supported for some CUBRID functions. Refer to information in the mapping table below to change unsupported functions into appropriate ones.

ODBC 2.x Functions	ODBC 3.x Functions
SQLAllocConnect	SQLAllocHandle
SQLAllocEnv	SQLAllocHandle
SQLAllocStmt	SQLAllocHandle
SQLBindParam	SQLBindParameter
SQLColAttributes	SQLColAttribute
SQLError	SQLGetDiagRec
SQLFreeConnect	SQLFreeHandle
SQLFreeEnv	SQLFreeHandle
SQLFreeStmt with SQL_DROP	SQLFreeHandle

ODBC 2.x Functions	ODBC 3.x Functions
SQLGetConnectOption	SQLGetConnectAttr
SQLGetStmtOption	SQLGetStmtAttr
SQLParamOptions	SQLSetStmtAttr
SQLSetConnectOption	SQLSetConnectAttr
SQLSetParam	SQLBindParameter
SQLSetScrollOption	SQLSetStmtAttr
SQLSetStmtOption	SQLSetStmtAttr
SQLTransact	SQLEndTran

ADO.NET Driver

ADO.NET is a set of classes that expose data access services to the .NET programmer. ADO.NET provides a rich set of components for creating distributed, data-sharing applications. It is an integral part of the .NET Framework, providing access to relational, XML, and application data. ADO.NET supports a variety of development needs, including the creation of front-end database clients and middle-tier business objects used by applications, tools, languages, or Internet browsers.

Installing and Configuring ADO.NET

Requirements

- Windows (Windows Vista or Windows 7 recommended)
- .NET Framework 2.0 or later (4.0 or later versions recommended)

- Microsoft Visual Studio Express edition (https://visualstudio.microsoft.com/)

Installing and Configuring CUBRID ADO.NET Driver

Before you start developing .NET applications with CUBRID, you will need the CUBRID ADO.NET Data Provider library (**Cubrid.Data.dll**). You have the options to:

- Download the complied library along with other files from:

  https://www.cubrid.org/downloads#adonet

- Compile it yourself from source code. You can download the source code from GitHub.

  https://github.com/CUBRID/cubrid-adonet

The CUBRID .NET Data Provider is 100% full-managed .NET code and it does not rely on any CUBRID library files. This means that the usage of the driver does not require any kind of CUBRID installation or files on the local machine.

The easiest way to install CUBRID ADO.NET Data Provider is to use the official installer. If you choose to install using the default option (x86), the driver will be installed in the **Program Files\CUBRID\CUBRID ADO.NET Data Provider 8.4.1** directory.

You can also choose to install the driver in GAC (https://en.wikipedia.org/wiki/Global_Assembly_Cache). The best way to install the driver is to use the tlbimp (https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-2.0/tt0cf3sx(v=vs.80)) tool. See the below to import the required namespaces.

```
33  using System.Data.Common;
34  using System.Diagnostics;
35  using System.IO;
36  using System.Reflection;
37  using System.Text.RegularExpressions;
38  using CUBRID.Data.CUBRIDClient;
39
40  namespace CUBRID.ADO.NET.DataProvider.TestCases
41  {
42      /// <summary>
43      /// Implementation of test cases for CUBRID ADO.NET Da
```

ADO.NET Programming

A Simple Query/Retrieve Code

Let's take a look at a simple code which retrieves value from a CUBRID database table. Assume that the connection is already established.

```
String sql = "select * from nation order by `code` asc";

using (CUBRIDCommand cmd = new CUBRIDCommand(sql, conn))
{
    using (DbDataReader reader = cmd.ExecuteReader())
    {
        reader.Read();
        //(read the values using: reader.Get...() methods)
    }
}
```

Once you have created the DbDataReader object, all you have to do is to use the Get...() method to retrieve any column data. CUBRID ADO.NET driver implements all methods required to read any CUBRID data types.

```
reader.GetString(3)
reader.GetDecimal(1)
```

The Get...() method will use as an input parameter the 0-based index position of the retrieved column.

To retrieve specific CUBRID data types, you need to use CUBRIDDataReader, instead of the DbDataReader interface.

```
using (CUBRIDCommand cmd = new CUBRIDCommand("select * from t",
conn))
{
    CUBRIDDataReader reader = (CUBRIDDataReader)cmd.ExecuteReader();

    reader.Read();
    Debug.Assert(reader.GetDateTime(0) == new DateTime(2008, 10, 31,
10, 20, 30, 40));
    Debug.Assert(reader.GetDate(0) == "2008-10-31");
    Debug.Assert(reader.GetDate(0, "yy/MM/dd") == "08-10-31");
    Debug.Assert(reader.GetTime(0) == "10:20:30");
    Debug.Assert(reader.GetTime(0, "HH") == "10");
    Debug.Assert(reader.GetTimestamp(0) == "2008-10-31
10:20:30.040");
    Debug.Assert(reader.GetTimestamp(0, "yyyy HH") == "2008 10");
}
```

batch Commands

When using CUBRID ADO.NET Data Provider library, you can execute more than one query against the data service in a single batch. For more information, see https://learn.microsoft.com/en-us/previous-versions/dd744839(v=vs.90) .

For example, in CUBRID, you can write the code like:

```
string[] sql_arr = newstring3;
sql_arr0 = "insert into t values(1)";
sql_arr1 = "insert into t values(2)";
sql_arr2 = "insert into t values(3)";
conn.BatchExecute(sql_arr);
```

or you can write as follows:

```
string[] sqls = newstring3;
sqls0 = "create table t(id int)";
sqls1 = "insert into t values(1)";
sqls2 = "insert into t values(2)";

conn.BatchExecuteNoQuery(sqls);
```

Connection String

In order to establish a connection from .NET application to CUBRID, you must build the database connection string as the following format:

```
ConnectionString = "server=<server address>;database=<database
name>;port=<port number to use for connection to broker>;user=<user
name>;password=<user password>;"
```

All parameters are mandatory except for **port**. If you do not specify the broker port number, the default value is **30,000**.

The examples of connection string with different options are as follows:

- Connect to a local server, using the default *demodb* database.

```
ConnectionString =
"server=127.0.0.1;database=demodb;port=30000;user=public;password="
```

- Connect to a remote server, using the default *demodb* database, as user **dba**.

```
ConnectionString =
"server=10.50.88.1;database=demodb;user=dba;password="
```

- Connect to a remote server, using the default *demodb* database, as user **dba**, using password *secret*.

```
ConnectionString =
"server=10.50.99.1;database=demodb;port=30000;user=dba;password=secr
et"
```

As an alternative, you can use the CUBRIDConnectionStringBuilder class to build easily a connection string in the correct format.

```
CUBRIDConnectionStringBuilder sb = new
CUBRIDConnectionStringBuilder(localhost,"33000","demodb","public","");
using (CUBRIDConnection conn = new
CUBRIDConnection(sb.GetConnectionString()))
{
    conn.Open();
}
```

or you can write as follows:

```
sb = new CUBRIDConnectionStringBuilder();
sb.User = "public" ;
sb.Database = "demodb";
sb.Port = "33000";
sb.Server = "localhost";
using (CUBRIDConnection conn = new
CUBRIDConnection(sb.GetConnectionString()))
{
    conn.Open();
}
```

Note

The database connection in thread-based programming must be used independently each other.

CUBRID Collections

Collections are specific CUBRID data type. If you are not familiar with them, you can read information in Collection Types. Because collections are not common to any database, the support for them is implemented in some specific CUBRID collection method.

```
public void AddElementToSet(CUBRIDOid oid, String attributeName,
Object value)
public void DropElementInSet(CUBRIDOid oid, String attributeName,
Object value)
public void UpdateElementInSequence(CUBRIDOid oid, String
attributeName, int index, Object value)
public void InsertElementInSequence(CUBRIDOid oid, String
attributeName, int index, Object value)
public void DropElementInSequence(CUBRIDOid oid, String
attributeName, int index)
public int GetCollectionSize(CUBRIDOid oid, String attributeName)
```

Here below are two examples of using these CUBRID extensions.

Reading values from a Collection data type:

```
using (CUBRIDCommand cmd = new CUBRIDCommand("SELECT * FROM t",
conn))
{
    using (DbDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            object[] o = (object[])reader[0];
            for (int i = 0; i <SeqSize; i++)
            {
                //...
            }
        }
    }
}
```

Updating a Collection data type:

```
conn.InsertElementInSequence(oid, attributeName, 5, value);
SeqSize = conn.GetCollectionSize(oid, attributeName);
using (CUBRIDCommand cmd = new CUBRIDCommand("SELECT * FROM t",
conn))
{
    using (DbDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
```

```
        {
            int[] expected = { 7, 1, 2, 3, 7, 4, 5, 6 };
            object[] o = (object[])reader0;
        }
    }
}
conn.DropElementInSequence(oid, attributeName, 5);
SeqSize = conn.GetCollectionSize(oid, attributeName);
```

CUBRID BLOB/CLOB

Starting from CUBRID 2008 R4.0 (8.4.0), CUBRID deprecated the GLO data type and added support for LOB (BLOB, CLOB) data types. These data types are specific CUBRID data types so you need to use methods offered by CUBRID ADO.NET Data Provider.

Here are some basic source code examples.

Reading BLOB data:

```
CUBRIDCommand cmd = new CUBRIDCommand(sql, conn);
DbDataReader reader = cmd.ExecuteReader();

while (reader.Read())
{
    CUBRIDBlob bImage = (CUBRIDBlob)reader0;
    byte[] bytes = new byte[(int)bImage.BlobLength];
    bytes = bImage.getBytes(1, (int)bImage.BlobLength);
    //...
}
```

Updating CLOB data:

```
string sql = "UPDATE t SET c = ?";
CUBRIDCommand cmd = new CUBRIDCommand(sql, conn);

CUBRIDClob clob = new CUBRIDClob(conn);
str = conn.ConnectionString; //Use the ConnectionString for testing

clob.SetString(1, str);

CUBRIDParameter param = new CUBRIDParameter();

param.ParameterName = "?";
param.CUBRIDDataType = CUBRIDDataType.CCI_U_TYPE_CLOB;
param.Value = clob;
```

```
cmd.Parameters.Add(param);
cmd.ExecuteNonQuery();
```

CUBRID Metadata Support

CUBRID ADO.NET Data Provider supports for database metadata. Most of these methods are implemented in the CUBRIDSchemaProvider class. .. code-block:: c#

> public DataTable GetDatabases(string[] filters) public DataTable GetTables(string[] filters) public DataTable GetViews(string[] filters) public DataTable GetColumns(string[] filters) public DataTable GetIndexes(string[] filters) public DataTable GetIndexColumns(string[] filters) public DataTable GetExportedKeys(string[] filters) public DataTable GetCrossReferenceKeys(string[] filters) public DataTable GetForeignKeys(string[] filters) public DataTable GetUsers(string[] filters) public DataTable GetProcedures(string[] filters) public static DataTable GetDataTypes() public static DataTable GetReservedWords() public static String[] GetNumericFunctions() public static String[] GetStringFunctions() public DataTable GetSchema(string collection, string[] filters)

The example below shows how to get the list of tables in the current CUBRID database.

```
CUBRIDSchemaProvider schema = new CUBRIDSchemaProvider(conn);
DataTable dt = schema.GetTables(newstring[] { "%" });

Debug.Assert(dt.Columns.Count == 3);
Debug.Assert(dt.Rows.Count == 10);

Debug.Assert(dt.Rows[0].ToString() == "demodb");
Debug.Assert(dt.Rows[1].ToString() == "demodb");
Debug.Assert(dt.Rows[2].ToString() == "stadium");

Get the list of Foreign Keys in a table:

CUBRIDSchemaProvider schema = new CUBRIDSchemaProvider(conn);
DataTable dt = schema.GetForeignKeys(newstring[] { "game" });

Debug.Assert(dt.Columns.Count == 9);
Debug.Assert(dt.Rows.Count == 2);

Debug.Assert(dt.Rows[0].ToString() == "athlete");
Debug.Assert(dt.Rows[1].ToString() == "code");
Debug.Assert(dt.Rows[2].ToString() == "game");
Debug.Assert(dt.Rows[3].ToString() == "athlete_code");
Debug.Assert(dt.Rows[4].ToString() == "1");
Debug.Assert(dt.Rows[5].ToString() == "1");
Debug.Assert(dt.Rows[6].ToString() == "1");
```

```
Debug.Assert(dt.Rows[7].ToString() == "fk_game_athlete_code");
Debug.Assert(dt.Rows[8].ToString() == "pk_athlete_code");
```

The example below shows how to get the list of indexes in a table.

```
CUBRIDSchemaProvider schema = new CUBRIDSchemaProvider(conn);
DataTable dt = schema.GetIndexes(newstring[] { "game" });

Debug.Assert(dt.Columns.Count == 9);
Debug.Assert(dt.Rows.Count == 5);

Debug.Assert(dt.Rows32.ToString() ==
"pk_game_host_year_event_code_athlete_code"); //Index name
Debug.Assert(dt.Rows34.ToString() == "True"); //Is it a PK?
```

DataTable Support

The DataTable is a central object in the ADO.NET library and CUBRID ADO.NET Data Provider support the following features.

- DataTable populate
- Built-in commands: **INSERT**, **UPDATE**, and **DELETE**
- Column metadata/attributes
- DataSet, DataView inter-connection

The following example shows how to get columns attributes.

```
String sql = "select * from nation";
CUBRIDDataAdapter da = new CUBRIDDataAdapter();
da.SelectCommand = new CUBRIDCommand(sql, conn);
DataTable dt = new DataTable("nation");
da.FillSchema(dt, SchemaType.Source);//To retrieve all the column
properties you have to use the FillSchema() method

Debug.Assert(dt.Columns[0].ColumnName == "code");
Debug.Assert(dt.Columns[0].AllowDBNull == false);
Debug.Assert(dt.Columns[0].DefaultValue.ToString() == "");
Debug.Assert(dt.Columns[0].Unique == true);
Debug.Assert(dt.Columns[0].DataType == typeof(System.String));
Debug.Assert(dt.Columns[0].Ordinal == 0);
Debug.Assert(dt.Columns[0].Table == dt);
```

The following example shows how to insert values into a table by using the **INSERT** statement.

```
String sql = " select * from nation order by `code` asc";
using (CUBRIDDataAdapter da = new CUBRIDDataAdapter(sql, conn))
{
    using (CUBRIDDataAdapter daCmd = new CUBRIDDataAdapter(sql,
conn))
    {
        CUBRIDCommandBuildercmdBuilder = new
CUBRIDCommandBuilder(daCmd);
        da.InsertCommand = cmdBuilder.GetInsertCommand();
    }

    DataTable dt = new DataTable("nation");
    da.Fill(dt);

    DataRow newRow = dt.NewRow();

    newRow["code"] = "ZZZ";
    newRow["name"] = "ABCDEF";
    newRow["capital"] = "MyXYZ";
    newRow["continent"] = "QWERTY";

    dt.Rows.Add(newRow);
    da.Update(dt);
}
```

Transactions

CUBRID ADO.NET Data Provider implements support for transactions in a similar way with direct-SQL transactions support. Here is a code example showing how to use transactions.

```
conn.BeginTransaction();

string sql = "create table t(idx integer)";
using (CUBRIDCommand command = new CUBRIDCommand(sql, conn))
{
    command.ExecuteNonQuery();
}

conn.Rollback();

conn.BeginTransaction();

sql = "create table t(idx integer)";
using (CUBRIDCommand command = new CUBRIDCommand(sql, conn))
{
    command.ExecuteNonQuery();
}
```

```
conn.Commit();
```

Working with Parameters

In CUBRID, there is no support for named parameters, but only for position-based parameters. Therefore, CUBRID ADO.NET Data Provider provides support for using position-based parameters. You can use any name you want as long as parameters are prefixed with the character a question mark (?). Remember that you must declare and initialize them in the correct order.

The example below shows how to execute SQL statements by using the parameters. The most important thing is the order in which the **Add** () methods are called.

```
using (CUBRIDCommand cmd = new CUBRIDCommand("insert into t
values(?, ?)", conn))
{
    CUBRIDParameter p1 = new CUBRIDParameter("?p1",
CUBRIDDataType.CCI_U_TYPE_INT);
    p1.Value = 1;
    cmd.Parameters.Add(p1);

    CUBRIDParameter p2 = new CUBRIDParameter("?p2",
CUBRIDDataType.CCI_U_TYPE_STRING);
    p2.Value = "abc";
    cmd.Parameters.Add(p2);

    cmd.ExecuteNonQuery();
}
```

Error Codes and Messages

The following list displays the error code and messages shown up when using CUBRID ADO.NET Data Provider.

Code Number	Error Code	Error Message
0	ER_NO_ERROR	“No Error”
1	ER_NOT_OBJECT	“Index’s Column is Not Object”

Code Number	Error Code	Error Message
2	ER_DBMS	“Server error”
3	ER_COMMUNICATION	“Cannot communicate with the broker”
4	ER_NO_MORE_DATA	“Invalid dataReader position”
5	ER_TYPE_CONVERSION	“DataType conversion error”
6	ER_BIND_INDEX	“Missing or invalid position of the bind variable provided”
7	ER_NOT_BIND	“Attempt to execute the query when not all the parameters are binded”
8	ER_WAS_NULL	“Internal Error: NULL value”
9	ER_COLUMN_INDEX	“Column index is out of range”
10	ER_TRUNCATE	“Data is truncated because receive buffer is too small”
11	ER_SCHEMA_TYPE	“Internal error: Illegal schema paramCUBRIDDataType”
12	ER_FILE	“File access failed”

Code Number	Error Code	Error Message
13	ER_CONNECTION	“Cannot connect to a broker”
14	ER_ISO_TYPE	“Unknown transaction isolation level”
15	ER_ILLEGAL_REQUEST	“Internal error: The requested information is not available”
16	ER_INVALID_ARGUMENT	“The argument is invalid”
17	ER_IS_CLOSED	“Connection or Statement might be closed”
18	ER_ILLEGAL_FLAG	“Internal error: Invalid argument”
19	ER_ILLEGAL_DATA_SIZE	“Cannot communicate with the broker or received invalid packet”
20	ER_NO_MORE_RESULT	“No More Result”
21	ER_OID_IS_NOT_INCLUDED	“This ResultSet do not include the OID”
22	ER_CMD_IS_NOT_INSERT	“Command is not insert”
23	ER_UNKNOWN	“Error”

ADO.NET API

See http://ftp.cubrid.org/CUBRID_Docs/Drivers/ADO.NET/.

Perl Driver

DBD::cubrid is a CUBRID Perl driver that implements Perl5 Database Interface (DBI) to enable access to CUBRID database server. It provides full API support.

CUBRID Perl driver is written based on CCI API so affected by CCI configurations such as **CCI_DEFAULT_AUTOCOMMIT**.

Note

- The database connection in thread-based programming must be used independently each other.
- In autocommit mode, the transaction is not committed if all results are not fetched after running the SELECT statement. Therefore, although in autocommit mode, you should end the transaction by executing COMMIT or ROLLBACK if some error occurs during fetching for the resultset.

Installing and Configuring Perl

Requirements

- Perl: It is recommended to use an appropriate version of Perl based on your system environment. For example, all Linux and FreeBSD distributions come with Perl. For Windows, ActivePerl is recommended. For details, see https://www.activestate.com/platform/supported-languages/perl/ .
- CUBRID: To build the CUBRID Perl driver, the CCI driver is needed. You can obtain it by installing the CUBRID Engine, which can be downloaded from https://www.cubrid.org/downloads. After downloading and installing a proper version of the CUBRID Engine, you can find the required CCI driver in the installed CUBRID Engine directory, $CUBRID/cci.
- DBI: https://metacpan.org/dist/DBI/
- C compiler: In most cases, there are binary distributions of **DBD::cubrid** (https://www.cubrid.org/downloads#perl) available. However, if you want to build the driver from source code, a C compiler is required. Make sure to use the same C compiler that was used for

compiling Perl and CUBRID. Otherwise, you will encounter problems because of differences in the underlying C runtime libraries.

- For building CCI Driver In Linux, GNU Developer Toolset 8 or higher is required.

Comprehensive Perl Archive Network (CPAN) Installation

You can automatically install the driver from source code by using the CPAN module.

```
cpan
install DBD::cubrid
```

If you are using the CPAN module for the first time, it is recommended to accept default settings.

If you are using an older version, you might enter the command line below, instead of command line above.

```
perl -MCPAN -e shell
install DBD::cubrid
```

Manual Installation

If you cannot get the CPAN module, you should download the **DBD::cubrid** source code. The latest version is always available below:

https://www.cubrid.org/downloads#perl

The filename is typically something like this: **DBD-cubrid-X.X.X.tar.gz**. After extracting the archive, enter the command line below under the **DBD-cubrid-X.X.X** directory. (On Windows, you may need to replace **make** with **nmake** or **dmake**.)

```
Perl Makefile.PL
make
make test
```

If test seems to look fine, execute the command to build a driver.

```
make install
```

Perl API

Currently, CUBRID Perl driver provides only basic features and does not support LOB type and column information verification.

If you want to get details about CUBRID Perl driver API, see http://ftp.cubrid.org/CUBRID_Docs/Drivers/Perl/.

Python Driver

CUBRIDdb is a Python extension package that implements Python Database API 2.0 compliant support for CUBRID. In additional to the minimal feature set of the standard Python DB API, CUBRID Python API also exposes nearly the entire native client API of the database engine in **_cubrid**.

CUBRID Python driver is written based on CCI API so affected by CCI configurations such as **CCI_DEFAULT_AUTOCOMMIT**.

Installing and Configuring Python

Linux/UNIX

There are three ways to install CUBRID Python driver on Linux, UNIX, and UNIX-like operating systems. You can find instructions for each of them below.

Requirements

- Operating system: 32-bit or 64-bit Linux, UNIX, or UNIX-like operating systems
- Python: 2.4 or later (https://www.python.org/downloads/)

Building CUBRID Python Driver from Source Code (Linux)

To install CUBRID Python driver by compiling source code, you should have Python Development Package installed on your system.

1. For building CCI Driver, GNU Developer Toolset 8 or higher is required.

2. Download the source code from https://www.cubrid.org/downloads#python.

3. Extract the archive to the desired location.

```
tar xvfz cubrid-python-11.2-latest.tar.gz
```

4. Navigate to the directory where you have extracted the source code.

```
cd RB-11.0.0
```

5. Build the driver. At this and next step, make sure you are still under the root user.

```
python setup.py build
```

6. Install the driver. Here you also need root privileges.

```
python setup.py install
```

Using a Package Manager (EasyInstall) of CUBRID Python Driver (Linux)

EasyInstall is a Python module (**easy_install**) bundled with **setuptools** that lets you automatically download, build, install, and manage Python packages. It gives you a quick way to install packages remotely by connecting to other websites via HTTP as well as connecting to the Package Index. It is somewhat analogous to the CPAN and PEAR tools for Perl and PHP, respectively. For more information about EasyInstall, see https://setuptools.pypa.io/en/latest/deprecated/easy_install.html.

Enter the command below to install CUBRID Python driver by using EasyInstall.

```
easy_install CUBRID-Python
```

Windows

To install CUBRID Python driver on Windows, first download CUBRID Python driver as follows:

- Visit the website below to download the driver. You will be given to select your operating system and Python version installed on your system.

  https://www.cubrid.org/downloads#python

- Extract the archive you downloaded. You should see a folder and two files in the folder. Copy these files to the **Lib** folder where your Python has been installed; by default, it is **C:\Program Files\Python\Lib**.

Python Programming

The CUBRIDdb package is supposed to have the following constants according to Python Database API 2.0.

Name	Value
threadsafety	2
apilevel	2.0
paramstyle	qmark

Python Sample Program

This sample program will show steps that you need to perform in order to connect to the CUBRID database and run SQL statements from Python programming language. Enter the command line below to create a new table in your database.

```
csql -u dba -c "CREATE TABLE posts( id integer, title varchar(255),
body string, last_updated timestamp );" demodb
csql -u dba -c "grant ALL PRIVILEGES on posts to public;" demodb
```

Connecting to demodb from Python

1. Open a new Python console and enter the command line below to import CUBRID Python driver.

```
import CUBRIDdb
```

2. Establish a connection to the *demodb* database located on localhost.

```
conn = CUBRIDdb.connect('CUBRID:localhost:33000:demodb:::', 'dba',
'')
```

For the *demodb* database, it is not required to enter any password. In a real-world scenario, you will have to provide the password to successfully connect. The syntax to use the connect () function is as follows:

```
connect (url[,user[password]])
```

If the database has not started and you try to connect to it, you will receive an error such as this:

```
Traceback (most recent call last):
  File "tutorial.py", line 3, in <module>
    conn = CUBRIDdb.connect('CUBRID:localhost:30000:demodb:dba::')
  File "/usr/local/lib/python3.5/site-packages/CUBRIDdb/
__init__.py", line 61, in Connect
    return Connection(*args, **kwargs)
  File "/usr/local/lib/python3.5/site-packages/CUBRIDdb/
connections.py", line 22, in __init__
    self.connection = _cubrid.connect(*args, **kwargs2)
_cubrid.OperationalError: (-677, "ERROR: DBMS, -677, Failed to
connect to database server, 'demodb', on the following host(s):
localhost:localhost[CAS INFO-127.0.0.1:30000,0,0].")
```

If you provide wrong credentials, you will receive an error such as this:

```
Traceback (most recent call last):
  File "tutorial.py", line 3, in <module>
    con = CUBRIDdb.connect('CUBRID:localhost:
33000:demodb:::','a','b')
  File "/usr/local/lib/python3.5/site-packages/CUBRIDdb/
__init__.py", line 61, in Connect
    return Connection(*args, **kwargs)
  File "/usr/local/lib/python3.5/site-packages/CUBRIDdb/
connections.py", line 22, in __init__
    self.connection = _cubrid.connect(*args, **kwargs2)
_cubrid.DatabaseError: (-165, 'ERROR: DBMS, -165, User "a" is
invalid.[CAS INFO-127.0.0.1:33000,0,0].')
```

Executing an INSERT Statement

Now that the table is empty, insert data for the test. First, you have to obtain a cursor and then execute the **INSERT** statement.

```
cur = conn.cursor()
cur.execute("INSERT INTO posts (id, title, body, last_updated)
VALUES (1, 'Title 1', 'Test body #1', CURRENT_TIMESTAMP)")
conn.commit()
```

The autocommit in CUBRID Python driver is disabled by default. Therefore, you have to manually perform commit by using the commit () function after executing any SQL statement. This is equivalent to executing **cur.execute("COMMIT")** . The opposite to executing commit() is executing rollback (), which aborts the current transaction.

Another way to insert data is to use prepared statements. You can safely insert data into the database by defining a row that contains the parameters and passing it to the execute () function.

```
args = (2, 'Title 2', 'Test body #2')
cur.execute("INSERT INTO posts (id, title, body, last_updated)
VALUES (?, ?, ?, CURRENT_TIMESTAMP)", args)
```

The entire script up to now looks like this:

```
import CUBRIDdb
conn = CUBRIDdb.connect('CUBRID:localhost:33000:demodb:::', 'dba',
'')
cur = conn.cursor()

# Plain insert statement
cur.execute("INSERT INTO posts (id, title, body, last_updated)
VALUES (1, 'Title 1', 'Test body #1', CURRENT_TIMESTAMP)")

# Parameterized insert statement
args = (2, 'Title 2', 'Test body #2')
cur.execute("INSERT INTO posts (id, title, body, last_updated)
VALUES (?, ?, ?, CURRENT_TIMESTAMP)", args)

conn.commit()
```

Fetching all records at a time

You can fetch entire records at a time by using the fetchall () function.

```
cur.execute("SELECT * FROM posts ORDER BY last_updated")
rows = cur.fetchall()
for row in rows:
    print (row)
```

This will return the two rows inserted earlier in the following form:

```
[1, 'Title 1', 'Test body #1', '2011-4-7 14:34:46']
[2, 'Title 2', 'Test body #2', '2010-4-7 14:34:46']
```

Fetching a single record at a time

In a scenario where a lot of data must be returned into the cursor, you can fetch only one row at a time by using the fetchone () function.

```
cur.execute("SELECT * FROM posts")
row = cur.fetchone()
while row:
```

```
    print (row)
    row = cur.fetchone()
```

Fetching as many as records desired at a time

You can fetch a specified number of records at a time by using the fetchmany () function.

```
cur.execute("SELECT * FROM posts")
rows = cur.fetchmany(3)
for row in rows:
    print (row)
```

Accessing Metadata on the Returned Data

If it is necessary to get information about column attributes of the obtained records, you should call the description method.

```
for description in cur.description:
    print (description)
```

The output of the script is as follows:

```
('id', 8, 0, 0, 0, 0, 0)
('title', 2, 0, 0, 255, 0, 0)
('body', 2, 0, 0, 1073741823, 0, 0)
('last_updated', 15, 0, 0, 0, 0, 0)
```

Each of row has the following information.

```
(column_name, data_type, display_size, internal_size, precision,
scale, nullable)
```

For more information about numbers representing data types, see https://pythonhosted.org/CUBRID-Python/toc-CUBRIDdb.FIELD_TYPE-module.html .

Releasing Resource

After you have done using any cursor or connection to the database, you must release the resource by calling both object's close () function.

```
cur.close()
conn.close()
```

Python API

Python Database API is composed of connect() module class, Connection object, Cursor object, and many other auxiliary functions. For more information, see Python DB API 2.0 Official Documentation at https://peps.python.org/pep-0249/.

You can find the information about CUBRID Python API at http://ftp.cubrid.org/CUBRID_Docs/Drivers/Python/.

Ruby Driver

CUBRID Ruby driver implements the interface to enable access from applications in Ruby to CUBRID database server and official one is available as a RubyGem package.

CUBRID Ruby driver is written based on CCI API so affected by CCI configurations such as **CCI_DEFAULT_AUTOCOMMIT**.

Installing and Configuring Ruby

Requirements

- Ruby 1.8.7 or later
- CUBRID gem
- ActiveRecord gem

Linux

You can install the CUBRID Connector through **gem**. Make sure that you add the **-E** option so that the environment path where CUBRID has been installed cannot be reset by the **sudo** command.

```
sudo -E gem install cubrid
```

Windows

Enter the command line below to install the latest version of CUBRID Ruby driver.

```
gem install cubrid
```

Ruby Sample Program

This section will explain how to use Ruby ActiveRecord adapter to work with CUBRID database. Create tables by executing the following SQL script.

```
CREATE TABLE countries(
    id integer AUTO_INCREMENT,
    code character varying(3) NOT NULL UNIQUE,
    name character varying(40) NOT NULL UNIQUE,
    record_date datetime DEFAULT sysdatetime NOT NULL,
    CONSTRAINT pk_countries_id PRIMARY KEY(id)
);

CREATE TABLE cities(
    id integer AUTO_INCREMENT NOT NULL UNIQUE,
    name character varying(40) NOT NULL,
    country_id integer NOT NULL,
    record_date datetime DEFAULT sysdatetime NOT NULL,
    FOREIGN KEY (country_id) REFERENCES countries(id) ON DELETE
RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT pk_cities_id PRIMARY KEY(id)
);
```

Loading Library

Create a new file named *tutorial.rb* and add basic configuration.

```
require 'rubygems'
require 'active_record'
require 'pp'
```

Establishing Database Connection

Define the connection parameters as follows:

```
ActiveRecord::Base.establish_connection(
 :adapter => "cubrid",
 :host => "localhost",
 :database => "demodb" ,
 :user => "dba"
)
```

Inserting Objects into a Database

Before starting to operate on tables, you must declare the two tables' mapping in the database as ActiveRecord classes.

```
class Country < ActiveRecord::Base
end

class City < ActiveRecord::Base
end

Country.create(:code => 'ROU', :name => 'Romania')
Country.create(:code => 'HUN', :name => 'Hungary')
Country.create(:code => 'DEU', :name => 'Germany')
Country.create(:code => 'FRA', :name => 'France')
Country.create(:code => 'ITA', :name => 'Italy', :record_date =>
Time.now)
Country.create(:code => 'SPN', :name => 'Spain')
```

Selecting Records from a Database

Select records from a database as follows:

```
romania = Country.find(1)
pp(romania)

romania = Country.where(:code => 'ROU')
pp(romania)

Country.find_each do |country|
 pp(country)
end
```

Updating Database Records

Change the *Spain* code from *'SPN'* to *'ESP'*.

```
Country.transaction do
 spain = Country.where(:code => 'SPN')[0]
 spain.code = 'ESP'
 spain.save
end
```

Deleting Database Records

Delete records from a database as follows:

```
Country.transaction do
 spain = Country.where(:code => 'ESP')[0]
 spain.destroy
end
```

Working with Associations

One method to add cities to a country would be to select the *Country* and assign the country code to a new *City* object.

```
romania = Country.where(:code => 'ROU')[0]
City.create(:country_id => romania.id, :name => 'Bucharest');
```

A more elegant solution would be to let ActiveRecord know about this relationship and declare it in the *Country* class.

```
class Country < ActiveRecord::Base
 has_many :cities, :dependent => :destroy
end

class City < ActiveRecord::Base
end
```

In the code above, it is declared that one country can have many cities. Now it will be very easy to add new city to a country.

```
italy = Country.where(:code => 'ITA')[0]
italy.cities.create(:name => 'Milano');
italy.cities.create(:name => 'Napoli');

pp (romania.cities)
pp (italy.cities)
```

This would be very helpful because when we access cities we get all the cities recorded for the referenced country. Another use is that when you delete the country, all its cities are removed. All is done in one statement.

```
romania.destroy
```

ActiveRecord also supports other relationship including one-to-one, many-to-many, etc.

Working with Metadata

ActiveRecord enables the code to work with on different database backends without modifying the code.

Defining a database structure

A new table can be defined using **ActiveRecord::Schema.define**. Let's create two tables: books and authors with a one-to-many relationship between *authors* and *books* (one-to-many).

```
ActiveRecord::Schema.define do
 create_table :books do |table|
  table.column :title, :string, :null => false
  table.column :price, :float, :null => false
  table.column :author_id, :integer, :null => false
 end

 create_table :authors do |table|
  table.column :name, :string, :null => false
  table.column :address, :string
  table.column :phone, :string
 end

 add_index :books, :author_id
end
```

CUBRID-supported column types are **:string**, **:text**, **:integer**, **:float**, **:decimal**, **:datetime**, **:timestamp**, **:time**, **:boolean**, **:bit**, **:smallint**, **:bigint**, and **:char**. Currently, **:binary** is not supported.

Managing table columns

You can add, update, delete columns by using features from **ActiveRecord::Migration**.

```
ActiveRecord::Schema.define do
 create_table :todos do |table|
  table.column :title, :string
  table.column :description, :string
 end

 change_column :todos, :description, :string, :null => false
 add_column :todos, :created, :datetime, :default => Time.now
 rename_column :todos, :created, :record_date
 remove_column :todos, :record_date

end
```

Dumping database schema

You can use **ActiveRecord::SchemaDumper.dump** to dump information for currently used schema. This is done into a platform independent format that is understood by Ruby ActiveRecord.

Note that if you are using custom column types database specific (**:bigint**, **:bit**), this may not work.

Obtaining Server Capabilities

You can get database information extracted from the current connections as in the example below:

```
puts "Maximum column length        : " +
ActiveRecord::Base.connection.column_name_length.to_s
puts "SQL statement maximum length : " +
ActiveRecord::Base.connection.sql_query_length.to_s
puts "Quoting : '''test'''         : " +
ActiveRecord::Base.connection.quote("'''test'''")
```

Creating a schema

Due to the way CUBRID is functioning, you cannot programmatically create a schema as in the following example:

```
ActiveRecord::Schema.define do
 create_database('not_supported')
end
```

Ruby API

See http://ftp.cubrid.org/CUBRID_Docs/Drivers/Ruby/.

Node.js Driver

CUBRID Node.js driver is developed in 100% JavaScript and does not require specific platform compilation

Node.js is a platform built on Chrome's JavaScript runtime.

Node.js has the following specifics.

- Event-driven, server-side JavaScript.
- Good at handling lots of different kinds of I/O at the same time.
- Non-blocking I/O model that makes it lightweight and efficient.

For more details, see https://nodejs.org/en/.

If you want to download CUBRRID Node.js driver or find the recent information, see the following sites:

- source code main repository: https://github.com/CUBRID/node-cubrid

Installing Node.js

Requirements

- CUBRID 8.4.1 Patch 2 or higher
- Node.js

Installation

You can install CUBRID Node.js driver with "npm(Node Packaged Modules) install" command, but firstly you need to install node.js on https://nodejs.org/download/.

```
npm install node-cubrid
```

If you uninstall CUBRID Node.js driver, do the following command.

```
npm uninstall node-cubrid
```

CUBRID Node.js API

See http://ftp.cubrid.org/CUBRID_Docs/Drivers/Node.js/.

Release Notes

11.4 Release Notes

Contents

11.4 Release Notes 2352

- Release Notes Information 2364
- Overview 633
  - Driver Compatibility 2366

- 11.4 Changes 2366
  - New feature 2366
    - PL/CSQL 1374
    - SQL 2366
      - HASH JOIN Support 2366
      - Support for Changing Owner of SERIAL SQL Syntax 2366
      - Support for SQL Statements to Add/Remove Members from a Created Group 2367
      - Extended Query Cache Support 2367
      - Added **LEADING** Hint to Specify Join Order 2368
    - Utility 2369
      - Added Option to Delete Specific Plans in `cubrid plandump` 2369
      - Added Support for Separate loadjava Option for JNI-Based JavaSP 2369
      - Added Memory Monitoring Featured utility 2369
      - Added support for the `diagdb -d 9` option to dump data from a specific class (table) 2369
    - Broker,CAS,CMS 2370
      - Added Parameter to Set the Size of Data Sent to Clients in the Broker 2370
      - Added Parameter to Set ACL(Access Control) for Each Broker 2370
    - Others 2371
      - Support for Automatic Restart of Non-HA DB Server After Abnormal Termination 2371
      - Support for Script to Restore Backup as a New Database 2371

- Specification Changes 2371
  - SQL 61
    - Limit Maximum Number of Characters for CHAR Type to 2,048 (Previous Limit: 268,435,456) 2371
    - LOB Locator Path Changed to Relative Path 2372
    - Restriction on Use of Analytic Functions in UPDATE JOIN Queries 2372
    - Restrict Use of **for update** clause on System Tables and Views 2372
    - Data Type Consistency Check Disabled During View Creation 2372
    - Allow Use of NULL in SELECT Clause of View Query Specification 2372
    - Disallow Use of Both **AUTO_INCREMENT** and **DEFAULT** in TABLE Modification and Creation. 2373
    - Raise Error When Adding New Columns in **ALTER INDEX ... REBUILD** 2373
    - Raise Error When ROWNUM Value Exceeds NUMERIC Type Range 2373
  - Utility 61
    - Recognize Session Commands While Writing SQL or PL/CSQL Statements(like create, body and etc) in *csql* Interactive Mode 2373
    - Suppress Plan Information Output Before Deletion in `cubrid plandump -d` 2373
  - HA 2373
    - Raise Error When `ha_mode` is **on** and **myhost** is Included in **ha_node_list** 2373
  - Others 62
    - Column Name Changed in System Table db_serial (att_name → attr_name) 2373

- Improvements (Including Performance Enhancements) 2374
  - SQL 63
    - Performance Improvement with Caching for Correlated Subqueries 2374
    - Enhancement of Sort-Limit Optimization When Bind Variables or Expressions Are Used in the LIMIT Clause 2374
  - PL(incl JavaSP) 2375
    - Improved Automatic Restart of JavaSP Server When JNI Program Encounters Segmentation Fault 2375
    - Enhanced String Functionality to Support Multiple Character Encodings (euckr, utf8) 2375
  - Index 2375
    - Removed Length Limitation on WHERE Clause When Creating Filter Indexes 2375
    - Improved Index Scan Performance by Optimizing midxkey.buf Size Calculation for Multi-Column Indexes 2375
    - Optimized Index Scan Performance by Reducing Unnecessary Calculations and Comparisons 2375
  - Optimizer 2376
    - Performance Improvement Through Overall Optimizer Enhancements 2376
    - Improvement in Stored Procedure Execution Plans 2377
    - Improvement in Concurrency by Preventing Unnecessary X-LOCKs on Unneeded Rows 2377
    - Improvement in Index Scanning for LIKE Queries Using JavaSP Functions 2377
    - Improvement in Range Index Scanning for Queries with ⩽ and ⩾ Conditions on Function Indexes 2378

- Utility 64
  - Improvement to `unloaddb` in Databases Without Tables and Views to Extract Other Schemas(user, serial, sp, server, synonym, grant) 2378
  - Performance Improvement in unloaddb 2378
  - Improvement in Clarity of `csql` Environment Variable Names 2378
  - Improved the output of the `cubrid spacedb` command 2378
- Broker,CAS,CMS 65
  - Improvement in cubrid spacedb Output 2379
  - Error Occurrence for Incorrect ACL Settings for Improved User Awareness 2380
  - Improvement to Allow TLS v1.2 Clients to Connect to CMS(CUBRID Manager Server) 2380
  - Improvement to Prevent Zombie Processes When Broker/CAS Processes are Terminated Through CMS 2380
  - Improvement to CMS getlogfileinfo() API to Return SQL Log File Information Only Once 2380
  - Improvement to cms ha_status() API to Display Replica Node Status in HA Environments with Master, Slave, and Replica Configuration 2380
- HA 372
  - Clear Error Message Display for Incorrect Settings in **ha_node_list** and **ha_replica_list** for Improved User Awareness 2380
  - Clear Error Message Display for Failover and Failback Occurrences to Improve User Awareness 2380
  - Improvement to Prevent Incorrect Data Mismatch Error Message from Appearing When **restoreslave** is Executed on Slave Node 2380
- Others 372
  - Improvements to User Hosts (cubrid_host.conf) for User Convenience 2380

- Improvements to DDL Audit Logs 2381
- Performance Improvement in REDO Recovery with Parallel Threads 2381
- Simplified Lock Information Logged in **dbname_latest.event** File During **Deadlock** 2381
- Performance Improvement in time_format() and date_format() 2381
- Query Performance Improvement with TRACE Enabled 2381
- Performance Improvement by Removing Unnecessary Length Checks on String Types 2382
- Memory Usage Improvement by Limiting Lock Resource Retention to **lock_escalation** Setting 2382
- Improved User Awareness with Error Output for Invalid **optimization_level** Setting 2382
- Improvement in Checking DB Volume and Log Creation Times 2382
- Improvement in User Awareness by Adding Fence Key Information to **SHOW INDEX CAPACITY** and `diagdb` 2382

- Bug Fixes 2382
  - SQL 372
    - Fixed an issue where referencing a remote server object without specifying the user schema could fail to resolve to the correct object if an object with the same name exists in another user. 2384
    - Fixed Hidden Column Handling in Scalar Subqueries with ORDER BY 2384
    - Fixed an issue where View Merging of Inline Views and Scalar Subqueries Containing ORDER BY Clauses, Where ROWNUM Was Incorrectly Rewritten as ORDER_BY_NUM() 2384
    - Corrected an Issue When **oracle_style_empty_string=yes** is Set and The Result of The Expression 'null || string' Was Incorrectly Returned as NULL 2384
    - Fixed NULL Output in REPLACE Function When **oracle_style_empty_string=yes** 2384

- Fixed Incorrect Results in Covered Index Scans Using Function Indexes 2384
- Fixed Segment Fault Errors in CTE Queries with Always False (or NULL) Conditions 2384
- Fixed Segment Fault Errors in UNION ALL Queries Using Multiple Tables and Serial Next Values 2384
- Fixed Segment Fault Errors in Queries Using Constants Over 255 Characters Without Aliases 2384
- Fixed Errors During View Merging with OUTER JOIN 2384
- Fixed Displaying Both User and Public Views With the Same Name in **SHOW CREATE VIEW** 2385
- Fixed Missing Error Message and NULL Return in repeat() Function Exceeding string_max_size_bytes Setting 2385
- Fixed Incorrect Table Positioning During INNER JOIN Removal in Query Optimization 2385
- Fixed Errors with Host Variables in PREPARE Statements Using Certain Built-in Functions 2385
- Fixed Missing User Information Deletion in System Catalog Tables When Dropping Users by The `DROP USER` Clause 2385
- Fixed Oracle-Style LEFT OUTER JOIN Not Rewriting to INNER JOIN When Using Host Variables in SP Call or WHERE Clause 2385
- Fixed Incorrect Results in Subquery with **ORDERBY_NUM()** During View Merging 2385
- Fixed for Tables Without Join Relations Being Recognized as Join Tables During Query Optimization 2385
- Fixed ROWNUM Values Being Printed as 0 When Using Equal Condition (=) in WHERE Clause and the Same Column in ORDER BY 2385
- Fixed Errors That Occur When INNER JOIN and Oracle-Style OUTER JOIN Are Used Together and JOIN Condition Placement in WHERE Clause 2385
- Corrected Incorrect Results in **INSERT INTO tbl ... SELECT ... FROM View ... ON DUPLICATE KEY UPDATEI** Query 2385

- Corrected Incorrect Column Order Numbers in Analytical Functions Inside Views 2386
- Resolved a Case Where Range Conditions Were Not Recognized as Range Items When Using The Pipe Operator (||) 2386
- Fixed An Error That Occurred When Handling NULL in Host Variables of DBLINK Statements 2386
- Fixed Issue Where Korean Characters Were Corrupted When Inserting via Host Variables Using DBLINK on EUC-KR Oracle DB 2386

- PL(JavaSP) 2386
  - Resolved an Issue With Using DATETIMELTZ as a DATETIME Parameter in JavaSP 2386
  - Fixed An Error That Occured Due to Space Between Declared Java Method Name and Opening Parenthesis in JavaSP 2386
  - Fixed an Error that Occurred When Using Columns of Tables with Unsupported Types as Arguments in JavaSP 2386
  - Fixed The **Cannot allocate query entry any more** Error in JavaSP 2386

- Utility 372
  - Fixed The Exit Code Not Being Set to 3 When an Error Occurs During `loaddb` Execution 2387
  - Fixed Incorrect **count(*)** Value Returned After Executing `loaddb` with **–no-logging** 2387
  - Fixed Error That Occurred When Processing Serials Without Schema Names During `loaddb` Execution 2387
  - Fixed an Issue Where loaddb Would Continue Execution Even After an Error Occurred 2387
  - Resolved an Error That Occurred When Processing a Synonym for System Tables During `loaddb` Execution 2387
  - Corrected an Issue Where the Schema Name Was Incorrectly Stored or Missing for Trigger System-Table Columns (condition, action_definition) During `loaddb` Execution Using unloaddb Files From Version 11.4 or Earlier 2387

- Addressed a Case Where Reverse Unique-Index Comments Were Missing During `unloaddb` Execution 2387
- Corrected an Issue Where the **auto_increment** Value Was Reset to 1 for PK Columns With **auto_increment** Set During `unloaddb` Execution 2387
- Resolved an Issue Where the **dbname_schema_uk** File Was Created Even When No Unique Index Existed During `unloaddb` **–split-schema-files** Execution 2387
- Addressed a Case Where the **ALTER SERIAL** Statement Was Missing When Executing `unloaddb` With the **-i (–input-class-only)** Option 2387
- Resolved an Error That Occurred When current_val and max_val of a Serial Were Identical During `unloaddb` Execution 2388
- Corrected an Issue Where an Infinite Loop Occurred When Processing JSON Data Over 1MB During `unloaddb` Execution 2388
- Addressed a Case Where, When Running `unloaddb` as a Regular User, Schema Names Identical to the User Were Not Removed From the query_spec of Views 2388
- Resolved a Case Where Schema Names for Serials and Triggers Were Missing When Executing `unloaddb` as **DBA** User 2388
- Corrected an Issue That Occured When Running `unloaddb` as a DBA User Without the –as-dba Option, Where Procedure Privileges Granted by Other Users (e.g., GRANT ... ON PROCEDURE) Were Incorrectly Included in the Output 2388
- Corrected an Issue That Occured When Running `unloaddb` as a Regular User, Where Schema Names Identical to the User Were Not Removed From the Condition and action_definition of Triggers 2388
- Fixed for Unnecessary Archive Log Volumes Being Created During `backupdb` Execution 2388
- Fixed an Issue Where `flashback` Did Not Terminate Properly on Forced Exit (Ctrl+C) 2388
- Resolved a Case Where `backupdb` Could Hang in Certain Situations 2388

- Broker,CAS, CMS 373
  - Fixed a Potential Memory Leak In the Processing of *addBatch()* and *executeBatch()* in Broker/CAS/CMS 2389

- Corrected an Incorrect Output From the *getTransactionInfo()* Function Call 2389
- HA 373
  - Addressed a Case Where Replication Logs Were Not Deleted on a Replica Node When *ha_replica_delay* Was Set to 60 Seconds or More in an HA Environment With Master, Slave, and Replica Nodes 2389
  - Resolved an Issue Where the SQL Log File Was Not Automatically Deleted Under Certain Conditions During `applylogdb` Execution 2389
- Cautions 1364
  - New Cautions 2389
    - The database volume of CUBRID 11.4 is not compatible with that of CUBRID 11.3 and earlier versions. 2389
  - Existing Cautions 2389
    - 11.3 373
      - The query optimizer's statistical information is not automatically updated when a DDL statement is executed; the user must manually execute the UPDATE STATISTICS statement to update the statistical information. (see Join Methods) 8342390
      - When changing the type of a column that has the AUTO_INCREMENT property or a default value in an ALTER TABLE statement, an error occurs if the new type cannot use AUTO_INCREMENT or if the existing default value is a type whose existing default value cannot be changed. (see :ref:change-column) 2390
      - You cannot obtain a Connection object through cubrid.jdbc.driver.CUBRIDDriver.getDefaultConnection() from server-side JDBC. Instead, use DriverManager.getConnection("jdbc:default:connection:"). (see Creating Connection) 2390
      - If an invalid index name or table name is used in a USE INDEX (USING INDEX) statement, it is ignored instead of raising an error. (To record it in the log file, set the error_log warning parameter to yes.) 2390
      - When writing a CREATE INDEX statement, the position of COMMENT has been moved to the end of the statement, after the WITH or INVISIBLE clause. (see INDEX DEFINITION STATEMENTS) 2682390

- 11.2 373
  - The database volume of CUBRID 11.2 is not compatible with that of CUBRID 11.1 and earlier versions. 2390
  - By introducing the concept of user schema, the same object name can be used for each user, and the behavior is changed as follows 2390
  - The following functions and behavior changed when using “jdbc:default:connection:” or calling getDefaultConnection() in JavaSP. (see JavaSP Caution) 2391
  - The behavior of the TRUNCATE TABLE changed if there is set null or cascade of FK (see TRUNCATE) 8192391
  - Column properties not written during alter change/modify are changed to be maintained, and auto_increment and on update properties cannot be removed with the alter statement (see CHANGE/MODIFY Clauses) 2391
  - Changed to handle an error if only the column name exists in the where clause 2391
  - Multiple SQL must be separated by semicolons 2392
  - The CCI Driver directory in the CUBRID package is changed from $CUBRID/lib and $CUBRID/include to $CUBRID/cci/lib and $CUBRID/cci/include, respectively (see CCI Overview) 2392
  - Changed Compression (-z, --compress) option to default on backup (see backupdb) 2392
- 11.0 373
  - The database volume of CUBRID 11.0 is not compatible with that of CUBRID 10.2 and earlier versions. 2393
  - When creating a table without an option, it is created as a reuse_oid table. 2393
  - The maximum length of the CHAR data type has been changed to 256M character string. 2393
  - Modified to occur error when the input string length is longer than the set length of the string data type. 2393

- Modified to recognize the space character at the end of the string, it is recognized as a different character string according to the space character at the end of the string. 2393
- Due to the change in the statistics collection method, it is necessary to perform periodic statistics collection. 2393

- 10.2 373
  - The database volume of CUBRID 10.2 is not compatible with that of CUBRID 10.1 and earlier versions. 2393

- 10.1 2393
  - The database volume of CUBRID 10.1 is not compatible with that of CUBRID 10.0 and earlier versions. 2394

- Version 10.0 and earlier 2394
  - The database volume of CUBRID 10.0 is not compatible with that of CUBRID 9.3 and earlier versions. 2394
  - Locale(language and charset) is specified when creating DB 2394
  - CUBRID_CHARSET environment variable is removed 2394
  - [JDBC] Change zero date of TIMESTAMP into '1970-01-01 00:00:00'(GST) from '0001-01-01 00:00:00' when the value of zeroDateTimeBehavior in the connection URL is "round"(CUBRIDSUS-11612) 2394
  - Recommendation for installing CUBRID SH package in AIX(CUBRIDSUS-12251) 2394
  - CUBRID_LANG is removed, CUBRID_MSG_LANG is added 2395
  - Modify how to process an error for the array of the result of executing several queries at once in the CCI application(CUBRIDSUS-9364) 2395
  - In java.sql.XAConnection interface, HOLD_CURSORS_OVER_COMMIT is not supported(CUBRIDSUS-10800) 2397
  - From 9.0, STRCMP behaves case-sensitively 2397

- Since the 2008 R4.1 version, the Default value of CCI_DEFAULT_AUTOCOMMIT has been ON(CUBRIDSUS-5879) 2397
- From the 2008 R4.0 version, the options and parameters that use the unit of pages were changed to use the unit of volume size(CUBRIDSUS-5136) 2398
- Be cautious when setting db volume size if you are a user of a version before 2008 R4.0 Beta(CUBRIDSUS-4222) 2398
- The change of the default value of some system parameters of the versions before 2008 R4.0(CUBRIDSUS-4095) 2398
- Changed so that database services, utilities, and applications cannot be executed when the system parameter is incorrectly configured(CUBRIDSUS-5375) 2399
- Database fails to start if the data_buffer_size is configured with a value that exceeds 2G in CUBRID 32-bit version(CUBRIDSUS-5349) 2399
- Recommendations for controlling services with the CUBRID Utility in Windows Vista and higher(CUBRIDSUS-4186) 2400
- A manager server process-related error occurs in the execution of the CUBRID source after its build(CUBRIDSUS-3553) 2400
- GLO class which is used in 2008 r3.0 or before is not supported any longer(CUBRIDSUS-3826) 2400
- Port configuration is required if the protocol between the master and server processes is changed, or if two versions are running at the same time(CUBRIDSUS-3564) 2402
- Specifying a question mark when entering connection information as a URL string in JDBC(CUBRIDSUS-3217) 2402
- Not allowed to include @ in a database name(CUBRIDSUS-2828) 2403

Release Notes Information

This document contains information about CUBRID 11.4 (Build Number 11.4.0.1778).

CUBRID 11.4 includes bug fixes and feature improvements found in CUBRID 11.3, as well as all bug fixes and feature enhancements applied in previous versions.

Information about CUBRID 11.3 can be found at https://www.cubrid.org/manual/en/11.3/release_note/index.html.

Information about CUBRID 11.2 can be found at https://www.cubrid.org/manual/en/11.2/release_note/index.html.

Information about CUBRID 11.0 can be found at https://www.cubrid.org/manual/en/11.0/release_note/index.html.

Information about CUBRID 10.2 can be found at https://www.cubrid.org/manual/en/10.2/release_note/index.html.

Information about CUBRID 10.1 can be found at https://www.cubrid.org/manual/en/10.1/release_note/index.html.

Information about CUBRID 10.0 can be found at https://www.cubrid.org/manual/en/10.0/release_note/index.html.

Information about CUBRID 9.3 can be found at https://www.cubrid.org/manual/en/9.3.0/release_note/index.html.

Overview

CUBRID 11.4 is the latest stable version that includes new features, significant changes, and improvements.

CUBRID 11.4 introduces:

1. **PL/CSQL support for Oracle compatibility**
2. **Addition of HASH JOIN for large-scale processing**
3. **Performance improvements through optimizer and index processing enhancements**
4. **Performance boost in data recovery through parallel processing**
5. **Performance enhancement by expanding result caching**
6. **Improved data dump performance**
7. **Addition of memory monitoring features**
8. **Enhanced access control functionality**
9. **Improved user convenience for backup and recovery operations**

These updates bring not only new features but also various performance improvements across multiple areas.

Driver Compatibility

- CUBRID 11.4 ensures compatibility with earlier driver versions. To maximize the utilization of the advanced features introduced in this version, it is strongly recommended to adopt the latest driver.

11.4 Changes

New feature

PL/CSQL

- CUBRID offers the PL/CSQL feature to ensure compatibility with ORACLE's PL/SQL.
- For syntax and usage details, please refer to the manual. (CUBRID Procedural Language)

SQL

HASH JOIN Support

- HASH JOIN is now supported—a type of join processing method that creates a hash table based on the join condition column to efficiently execute the join.
- For the optimizer to apply a HASH JOIN, a join hint is required.
  - When `/*+ USE_HASH */` hint is added, HASH JOIN is considered.
  - When `/*+ NO_USE_HASH */` hint is added, HASH JOIN is not used.

Support for Changing Owner of SERIAL SQL Syntax

- Extended the **ALTER SERIAL** statement to support changing the owner of a SERIAL, which was previously only possible by calling the `call change_serial_owner()` method.

```
ALTER SERIAL serial_name OWNER TO user_name
```

- Only the DBA or members of the DBA group are allowed to change the owner of a SERIAL.

- An error is returned if the user does not have sufficient privileges or if the specified SERIAL does not exist.

Support for SQL Statements to Add/Remove Members from a Created Group

- Extended the **ALTER USER** statement to support adding or removing users to/from a created group, which was previously only possible by calling `call add_member()` and `call drop_member()` methods.
- In HA environments, this enhancement simplifies user management by eliminating the need for separate operations on the master and slave nodes when modifying group membership.

```
ALTER USER user_name
[PASSWORD password] |
[ADD MEMBERS user_name {, user_name}...] |
[DROP MEMBERS user_name {, user_name}...]
[COMMENT 'comment_string']
```

- DBA/DBA Group Members
  - Add or remove members from the DBA group
  - Add or remove members from user-defined groups, including PUBLIC
- General Users
  - Can add or remove members only from their own groups

Extended Query Cache Support

The query cache feature, which was previously available only at the full query level, has been extended to support subquery-level caching.

- Support query chache in CTE(Common Table Expressions).

```
with cte0 as (select /*+ query_cache */ count(*) from public.game
where host_year > 2000),
cte1 as (select /*+ query_cache */ count(*) from public.game where
host_year = 2004)
select * from cte1;
```

```
Trace Statistics:
SELECT (time: 0, fetch: 4, fetch_time: 0, ioread: 0)
  SCAN (temp time: 0, fetch: 4, ioread: 0, readrows: 1, rows: 1)
  SUBQUERY (uncorrelated)
```

```
    CTE (non_recursive_part)
      SELECT (time: 0, fetch: 0, fetch_time: 0, ioread: 0)
        RESULT CACHE (reference count: 1)
    CTE (non_recursive_part)
      SELECT (time: 0, fetch: 0, fetch_time: 0, ioread: 0)
        RESULT CACHE (reference count: 1)
```

- Support query cache in non-correlated subqueries.

```
select count(host_year)
from (select /*+ query_cache */ code from public.nation where code
like 'K%') n, public.game g
where n.code = g.nation_code;
```

```
Trace Statistics:
SELECT (time: 218, fetch: 872, fetch_time: 2, ioread: 0)
  SCAN (temp time: 0, fetch: 4, ioread: 0, readrows: 7, rows: 7)
    SCAN (table: public.game), (heap time: 215, fetch: 868, ioread:
0, readrows: 60571, rows: 386)
  SUBQUERY (uncorrelated)
    SELECT (time: 0, fetch: 0, fetch_time: 0, ioread: 0)
      RESULT CACHE (reference count: 3)
```

- Support displaying the reference count in trace information when executing queries that utilize the result cache by **RESULT CHACHE (reference count: number)**

Added **LEADING** Hint to Specify Join Order

- The **LEADING** hint allows specifying a particular table or set of tables to be used as the prefix in the execution plan, enabling more flexible control over the table join order than the **ORDERED** hint.

```
SELECT /*+ LEADING(e j) */ *
FROM  employees e, departments d, job_history j
WHERE e.department_id = d.department_id
AND e.hire_date = j.start_date;
```

- Conditions for Ignoring the LEADING Hint
  - Ignored when the ORDERED hint is present.
  - If multiple LEADING hints are used, only the first one is applied.
  - The hint is ignored if join graph dependencies prevent early joining of specified tables.

Utility

Added Option to Delete Specific Plans in `cubrid plandump`

- Option: *-s*

Added Support for Separate loadjava Option for JNI-Based JavaSP

- Option: *-j* or *–jni*
- Not using the new `loadjava -j` option will now result in an error when a JNI-based JavaSP is executed.

Added Memory Monitoring Featured utility

- The memory monitoring utility `cubrid memmon` retrieves and displays the following heap memory information from the server process (*cub_server*) memory management module.
  - Total heap memory usage allocated and currently in use by the server process.
  - Detailed memory allocation information based on the source code and line number where the memory allocation request was made.
- To use the server process memory management module, the system parameter enable_memory_monitoring must be set to “yes”, and the server needs to be restarted.
- For detailed usage and instructions, please refer to the manual. (memmon)

Added support for the `diagdb -d 9` option to dump data from a specific class (table)

- Option: -n class-name
- In the case of partitioned tables, all partitions are dumped. For sub-partitioned tables, only the corresponding partition table is dumped.

Broker,CAS,CMS

Added Parameter to Set the Size of Data Sent to Clients in the Broker

- A new parameter, **NET_BUF_SIZE**, has been introduced to the Broker to control the size of the data transmitted to clients.
- The values that can be set for **NET_BUF_SIZE**
  - 16K(default)
  - 32K
  - 48K
  - 64K

Added Parameter to Set ACL(Access Control) for Each Broker

- When using the broker's ACL (ACCESS_CONTROL=ON), the ACCESS_CONTROL_DEFAULT_POLICY parameter allows you to set the default action policy for brokers that are not specified in the ACCESS_CONTROL_FILE.
- Set the value of ACCESS_CONTROL_DEFAULT_POLICY to **DENY** or **ALLOW** in the [broker] section of the `cubrid_broker.conf` file (default value is **DENY**).
  - If a broker is not specified in the ACCESS_CONTROL_FILE, and the default policy is ALLOW, all clients are allowed to connect; if the policy is DENY, connections are denied.

```
$ cubrid broker acl status
ACCESS_CONTROL=ON
ACCESS_CONTROL_DEFAULT_POLICY=DENY
ACCESS_CONTROL_FILE=cubrid_acl.conf

[%query_editor]
testdb:dba:acl_ip_list.conf

CLIENT IP LAST ACCESS TIME
========================================
172.29.80.1
192.168.0.31
172.31.0.175

[%broker1]
```

```
++ cubrid broker acl: success
```

- **[%broker1]** does not have any client access information specified, all client connections are denied according to the default policy **DENY**.

Others

Support for Automatic Restart of Non-HA DB Server After Abnormal Termination

- When operating a non-HA DB server, a new feature enables automatic restart if the DB server is abnormally terminated, such as by the OOM killer.
- Notes
  - The server does not automatically restart after a normal shutdown.
  - This feature can be enabled or disabled using the auto_restart_server parameter.
  - Automatic restart is disabled if the server repeatedly terminates abnormally within a short period or fails to start successfully after a certain number of attempts.

Support for Script to Restore Backup as a New Database

- When data recovery from a backup is needed on a production server, the `restore_to_newdb.sh` script allows restoring the backup as a new database without the need for additional steps such as creating new accounts or installing extra engine instances. For detailed usage and instructions, please refer to the manual (restore_to_newdb.sh script)

Specification Changes

SQL

Limit Maximum Number of Characters for CHAR Type to 2,048 (Previous Limit: 268,435,456)

- To prevent potential issues such as memory allocation errors, the maximum number of characters allowed for the CHAR type has been reduced from 268,435,456 to 2,048.

- When migrating databases from versions prior to 11.4, any CHAR type columns exceeding 2,048 characters must be converted to VARCHAR.

LOB Locator Path Changed to Relative Path

- To improve manageability and ease of moving LOB data files, the locator path for LOBs has been changed from an absolute path to a relative path.
- If the LOB directory location for a DB listed in databases.txt is changed, all existing LOB files in the previous directory must be moved or copied to the new LOB directory.

Restriction on Use of Analytic Functions in UPDATE JOIN Queries

- To prevent unexpected behavior, the use of analytic functions in UPDATE JOIN queries involving two or more tables is now restricted.
- Queries using this pattern that were written in versions prior to 11.4 must be revised, as they will cause errors in version 11.4.

Restrict Use of **for update** clause on System Tables and Views

- To prevent lock waits caused by X-LOCKs on system tables, the use of the **for update** clause on system tables and views now results in an error.

Data Type Consistency Check Disabled During View Creation

- To improve Oracle compatibility, data type checks during view creation are no longer performed. Type consistency is now checked at runtime.
- If type conversion is not possible, an error will occur when the view is executed.

Allow Use of NULL in SELECT Clause of View Query Specification

- To enhance Oracle compatibility, the use of NULL values in the SELECT clause of a view query specification is now allowed.

```
CREATE VIEW a_view( col1 ) AS select NULL as col1 from a_tbl;
```

Disallow Use of Both **AUTO_INCREMENT** and **DEFAULT** in TABLE Modification and Creation.

- When executing **CREATE TABLE** or **ALTER COLUMN** statements, an error will now be raised if both **AUTO_INCREMENT** and **DEFAULT** attributes are specified together.

Raise Error When Adding New Columns in **ALTER INDEX ... REBUILD**

- An error will now be raised if new columns are added during an ALTER INDEX ... REBUILD operation, as adding columns is not supported in this context.
- Queries written in versions prior to 11.4 using this pattern will cause errors in version 11.4 and must be revised.

Raise Error When ROWNUM Value Exceeds NUMERIC Type Range

Utility

Recognize Session Commands While Writing SQL or PL/CSQL Statements(like create, body and etc) in *csql* Interactive Mode

Suppress Plan Information Output Before Deletion in `cubrid plandump -d`

HA

Raise Error When `ha_mode` is **on** and **myhost** is Included in **ha_node_list**

- In previous versions, this configuration would automatically switch the server to replica mode.

Others

Column Name Changed in System Table db_serial (att_name → attr_name)

- Queries that explicitly reference att_name in versions prior to 11.4 must be updated, as they will result in errors in version 11.4.

Improvements (Including Performance Enhancements)

SQL

Performance Improvement with Caching for Correlated Subqueries

- Correlated subqueries are executed individually for each row of the main query. When there are many repeated condition values, the same subquery can be executed multiple times, leading to performance degradation. By implementing caching, repeated executions of the same subquery for identical condition values are prevented, improving performance.

Enhancement of Sort-Limit Optimization When Bind Variables or Expressions Are Used in the LIMIT Clause

```
drop table if exists tbl1;
create table tbl1 (col1 int, col2 int);
insert into tbl1 select rownum, random() % 100000 +1 from db_class
a, db_class b, db_class c, db_class d limit 100000;
create index idx on tbl1(col1);

prepare stmt from
'SELECT a.col1, a.col2
FROM tbl1 a
   LEFT JOIN tbl1 b ON a.col1 = b.col1
   LEFT JOIN tbl1 c ON a.col1 = c.col1
ORDER BY a.col2,a.col1
LIMIT ?*10,?';

execute stmt using 10,10;
```

PL(incl JavaSP)

Improved Automatic Restart of JavaSP Server When JNI Program Encounters Segmentation Fault

Enhanced String Functionality to Support Multiple Character Encodings (euckr, utf8)

- The string handling functionality has been improved to support various character encodings, such as euckr and utf8, by processing the encoding scheme of stored procedure strings as byte arrays.

Index

Removed Length Limitation on WHERE Clause When Creating Filter Indexes

Improved Index Scan Performance by Optimizing midxkey.buf Size Calculation for Multi-Column Indexes

- In multi-column indexes, the previous method of repeatedly calculating the key length has been improved. Now, each column's **OFFSET** is directly referenced without needing calculations, which improves performance during **binary search, key filtering, and DML operations**.

Optimized Index Scan Performance by Reducing Unnecessary Calculations and Comparisons

- The process of reading and processing keys during Range-Scan has been optimized to reduce unnecessary operations.
- Unnecessary comparisons with *upper_key* during Range-Scan have been reduced, enhancing performance.
- Repetitive comparisons of index column IDs have been minimized, optimizing operations.
- Common prefix information has been added to leaf node headers, optimizing the processing of compressed leaf nodes by reducing repetitive comparisons and calculations.

Optimizer

Performance Improvement Through Overall Optimizer Enhancements

- The number of sampling pages has been increased to 5000 to improve the accuracy of collected statistics.
- Enhanced the method of collecting NDV(Number of Distinct Values), leading to more accurate statistics.
- Minimized Rule-Based Optimization (RBO) for Better Accuracy
  - Reduced the use of empirical weights (Heuristic Factor).
  - Removed RBO for cost differences within 1.x, transitioning to Cost-Based Optimization (CBO) for improved accuracy.
- Improved Cost Formula Calculation
  - **Reflected Index Filter Scan Selectivity**: Adjusted I/O cost calculation during index scan to account for selectivity.
  - Added selectivity for the *NOT LIKE* operator.
  - Added selectivity for **function-based indexes**.
  - Adjusted weight for **NDV**(Distinct Value Count) when duplicates are high: If the duplication rate in sampling data exceeds 1%, statistics weight is adjusted.
  - Introduced **SSCAN_DEFAULT_CARD**: Prevents inefficient plans in **NL JOIN**(Nested Loop Join) when cardinality estimation is too low.
  - Introduced cost and cardinality settings for the LIMIT clause to improve performance for small data retrievals, with **optimized LIMIT** and adjusted default cardinality values.
- Elimination of Redundant Conditions
  - Removed logically redundant predictable join conditions that were already evaluated.
  - Improved the removal of unnecessary INNER JOIN.
- Additional Improvements
    - Enhanced trace information by adding **fetch_time** for better execution time tracking.
    - Improved **index skip scan** to be selected even without the `index_ss` hint.

- Enhanced the optimizer to choose a better index over the Primary Key.

Improvement in Stored Procedure Execution Plans

- The execution plan processing for stored procedures has been modified to align with the execution plan of built-in functions.
- Improvements
  - Index scans are now supported.
  - Unnecessary joins have been eliminated.
  - **Result caching** is now available when using stored procedures in **correlated subqueries**.

Improvement in Concurrency by Preventing Unnecessary X-LOCKs on Unneeded Rows

- In certain queries, locks were being created too early, causing unnecessary locks. The system has been improved to release unnecessary locks after all conditions have been evaluated.

Improvement in Index Scanning for LIKE Queries Using JavaSP Functions

```
CREATE OR REPLACE FUNCTION stringTest(x String) RETURN String AS
LANGUAGE JAVA NAME 'SpTest.typeteststring(java.lang.String) return
java.lang.String';

CREATE TABLE tbl (ord INT, col_int INT, col_char char(1));
CREATE INDEX i_tbl ON tbl (ord);
CREATE INDEX i_tbl_char ON tbl (col_char);

INSERT INTO tbl VALUES (1,10,'a');
INSERT INTO tbl VALUES (2,10,'b');
INSERT INTO tbl VALUES (3,10,'c');
INSERT INTO tbl VALUES (4,10,'d');
INSERT INTO tbl VALUES (5,10,'e');

SELECT count(*) AS "like" FROM tbl WHERE col_char LIKE (SELECT
stringTest('a') FROM dual);
```

Improvement in Range Index Scanning for Queries with `⩽` and `⩾` Conditions on Function Indexes

Utility

Improvement to `unloaddb` in Databases Without Tables and Views to Extract Other Schemas(user, serial, sp, server, synonym, grant)

Performance Improvement in unloaddb

- Multi-threading has been implemented to significantly reduce data extraction time and improve overall performance.
- unloaddb options
  - **–thread-count**: Specifies the number of concurrent threads to execute (0–127).
  - **enhanced-estimates**: Provides more accurate record count estimation (for verbose mode only).

Improvement in Clarity of `csql` Environment Variable Names

- To improve the clarity of CSQL environment variable names, the prefix CUBRID_CSQL has been added to indicate that these environment variables are related to CUBRID.
- **Changes**
  - EDITOR → CUBRID_CSQL_EDITOR
  - SHELL → CUBRID_CSQL_SHELL
  - FORMATTER → CUBRID_CSQL_FORMATTER
- For backward compatibility, the original environment variable names can still be used.

Improved the output of the `cubrid spacedb` command

- Improved the classification of overflow pages so that pages used by users are categorized as either INDEX or HEAP based on their purpose, instead of being incorrectly classified as SYSTEM pages.

Broker,CAS,CMS

Improvement in cubrid spacedb Output

- Directory names used in configuration settings are now displayed as absolute paths.
- **ADMIN_LOG_FILE** display has been added.

Error Occurrence for Incorrect ACL Settings for Improved User Awareness

Improvement to Allow TLS v1.2 Clients to Connect to CMS(CUBRID Manager Server)

Improvement to Prevent Zombie Processes When Broker/CAS Processes are Terminated Through CMS

Improvement to CMS getlogfileinfo() API to Return SQL Log File Information Only Once

Improvement to cms ha_status() API to Display Replica Node Status in HA Environments with Master, Slave, and Replica Configuration

HA

Clear Error Message Display for Incorrect Settings in **ha_node_list** and **ha_replica_list** for Improved User Awareness

Clear Error Message Display for Failover and Failback Occurrences to Improve User Awareness

Improvement to Prevent Incorrect Data Mismatch Error Message from Appearing When **restoreslave** is Executed on Slave Node

Others

Improvements to User Hosts (cubrid_host.conf) for User Convenience

- Added “0.0.0.0 your_hostname” entry.
- Improved functionality to allow hostnames to be used without case sensitivity.

- Validation has been added for IP addresses and hostnames; errors will occur if invalid entries are detected.

Improvements to DDL Audit Logs

- Added the 'DB Name' to the DDL audit log file generated by CAS, enabling identification of which database the operation was performed on in environments with multiple databases.
- Modified the DDL audit log to record an ABORT log even if the transaction ends without a commit or rollback.
- Improved handling of multiple DDL statements executed in a single transaction with SetAutocommit(false) by ensuring a commit or rollback statement is recorded between commands.

Performance Improvement in REDO Recovery with Parallel Threads

- Applied parallel threads for page-by-page recovery during the REDO recovery process, which does not require synchronization for data consistency, improving recovery performance.
- The improvement in performance is most noticeable when the REDO operation takes up a significant portion of the recovery process and the parallel index is high.

Simplified Lock Information Logged in **dbname_latest.event** File During **Deadlock**

- When a deadlock occurs, the server log file (dbname_latest.event) now **only logs the lock information directly contributing to the deadlock**, simplifying the output.

Performance Improvement in time_format() and date_format()

- Removed unnecessary string operation function calls to improve performance.

Query Performance Improvement with TRACE Enabled

- When executing a query with `set trace on`, the performance was significantly worse compared to executing the same query with `set trace off`. This has been improved by reducing high-cost operations caused by TRACE's statistics gathering overhead, minimizing the performance degradation.

Performance Improvement by Removing Unnecessary Length Checks on String Types

Memory Usage Improvement by Limiting Lock Resource Retention to **lock_escalation** Setting

Improved User Awareness with Error Output for Invalid **optimization_level** Setting

Improvement in Checking DB Volume and Log Creation Times

- The `SHOW VOLUME HEADER`, `SHOW LOG HEADER`, and `SHOW ARCHIVE LOG HEADER` commands have been enhanced to display the **Creation_time**, allowing users to easily check the volume creation time.
- The `diagdb` utility has been improved to allow users to view the creation times of each volume.
- Modification to applyinfo Utility to Display added Volume Creation Time in Log Volume Header

Improvement in User Awareness by Adding Fence Key Information to **SHOW INDEX CAPACITY** and `diagdb`

- Information regarding the fence key used during leaf node key compression is now displayed separately from general key information, improving user awareness.

Bug Fixes

SQL

Fixed an issue where referencing a remote server object without specifying the user schema could fail to resolve to the correct object if an object with the same name exists in another user.

Fixed Hidden Column Handling in Scalar Subqueries with ORDER BY

Fixed an issue where View Merging of Inline Views and Scalar Subqueries Containing ORDER BY Clauses, Where ROWNUM Was Incorrectly Rewritten as ORDER_BY_NUM()

Corrected an Issue When **oracle_style_empty_string=yes** is Set and The Result of The Expression 'null || string' Was Incorrectly Returned as NULL

Fixed NULL Output in REPLACE Function When **oracle_style_empty_string=yes**

Fixed Incorrect Results in Covered Index Scans Using Function Indexes

Fixed Segment Fault Errors in CTE Queries with Always False (or NULL) Conditions

Fixed Segment Fault Errors in UNION ALL Queries Using Multiple Tables and Serial Next Values

Fixed Segment Fault Errors in Queries Using Constants Over 255 Characters Without Aliases

Fixed Errors During View Merging with OUTER JOIN

Fixed Displaying Both User and Public Views With the Same Name in **SHOW CREATE VIEW**

Fixed Missing Error Message and NULL Return in repeat() Function Exceeding string_max_size_bytes Setting

Fixed Incorrect Table Positioning During INNER JOIN Removal in Query Optimization

Fixed Errors with Host Variables in PREPARE Statements Using Certain Built-in Functions

Fixed Missing User Information Deletion in System Catalog Tables When Dropping Users by The `DROP USER` Clause

Fixed Oracle-Style LEFT OUTER JOIN Not Rewriting to INNER JOIN When Using Host Variables in SP Call or WHERE Clause

Fixed Incorrect Results in Subquery with **ORDERBY_NUM()** During View Merging

Fixed for Tables Without Join Relations Being Recognized as Join Tables During Query Optimization

Fixed ROWNUM Values Being Printed as 0 When Using Equal Condition (=) in WHERE Clause and the Same Column in ORDER BY

Fixed Errors That Occur When INNER JOIN and Oracle-Style OUTER JOIN Are Used Together and JOIN Condition Placement in WHERE Clause

Corrected Incorrect Results in **INSERT INTO tbl ... SELECT ... FROM View ... ON DUPLICATE KEY UPDATEI** Query

Corrected Incorrect Column Order Numbers in Analytical Functions Inside Views

Resolved a Case Where Range Conditions Were Not Recognized as Range Items When Using The Pipe Operator (||)

Fixed An Error That Occurred When Handling NULL in Host Variables of DBLINK Statements

Fixed Issue Where Korean Characters Were Corrupted When Inserting via Host Variables Using DBLINK on EUC-KR Oracle DB

PL(JavaSP)

Resolved an Issue With Using DATETIMELTZ as a DATETIME Parameter in JavaSP

Fixed An Error That Occured Due to Space Between Declared Java Method Name and Opening Parenthesis in JavaSP

Fixed an Error that Occurred When Using Columns of Tables with Unsupported Types as Arguments in JavaSP

Fixed The **Cannot allocate query entry any more** Error in JavaSP

Utility

Fixed The Exit Code Not Being Set to 3 When an Error Occurs During `loaddb` Execution

Fixed Incorrect **count(*)** Value Returned After Executing `loaddb` with **–no-logging**

Fixed Error That Occurred When Processing Serials Without Schema Names During `loaddb` Execution

Fixed an Issue Where loaddb Would Continue Execution Even After an Error Occurred

Resolved an Error That Occurred When Processing a Synonym for System Tables During `loaddb` Execution

Corrected an Issue Where the Schema Name Was Incorrectly Stored or Missing for Trigger System-Table Columns (condition, action_definition) During `loaddb` Execution Using unloaddb Files From Version 11.4 or Earlier

Addressed a Case Where Reverse Unique-Index Comments Were Missing During `unloaddb` Execution

Corrected an Issue Where the **auto_increment** Value Was Reset to 1 for PK Columns With **auto_increment** Set During `unloaddb` Execution

Resolved an Issue Where the **dbname_schema_uk** File Was Created Even When No Unique Index Existed During `unloaddb` **–split-schema-files** Execution

Addressed a Case Where the **ALTER SERIAL** Statement Was Missing When Executing `unloaddb` With the **-i (–input-class-only)** Option

Resolved an Error That Occurred When current_val and max_val of a Serial Were Identical During `unloaddb` Execution

Corrected an Issue Where an Infinite Loop Occurred When Processing JSON Data Over 1MB During `unloaddb` Execution

Addressed a Case Where, When Running `unloaddb` as a Regular User, Schema Names Identical to the User Were Not Removed From the query_spec of Views

Resolved a Case Where Schema Names for Serials and Triggers Were Missing When Executing `unloaddb` as **DBA** User

Corrected an Issue That Occured When Running `unloaddb` as a DBA User Without the –as-dba Option, Where Procedure Privileges Granted by Other Users (e.g., GRANT ... ON PROCEDURE) Were Incorrectly Included in the Output

Corrected an Issue That Occured When Running `unloaddb` as a Regular User, Where Schema Names Identical to the User Were Not Removed From the Condition and action_definition of Triggers

Fixed for Unnecessary Archive Log Volumes Being Created During `backupdb` Execution

Fixed an Issue Where `flashback` Did Not Terminate Properly on Forced Exit (Ctrl+C)

Resolved a Case Where `backupdb` Could Hang in Certain Situations

Broker,CAS, CMS

Fixed a Potential Memory Leak In the Processing of *addBatch()* and *executeBatch()* in Broker/CAS/CMS

Corrected an Incorrect Output From the *getTransactionInfo()* Function Call

HA

Addressed a Case Where Replication Logs Were Not Deleted on a Replica Node When *ha_replica_delay* Was Set to 60 Seconds or More in an HA Environment With Master, Slave, and Replica Nodes

Resolved an Issue Where the SQL Log File Was Not Automatically Deleted Under Certain Conditions During `applylogdb` Execution

Cautions

New Cautions

The database volume of CUBRID 11.4 is not compatible with that of CUBRID 11.3 and earlier versions.

- Due to the addition of new features, system catalog information has been modified or added (System Catalog). For other new precautions, please refer to Specification Changes.

Existing Cautions

11.3

The query optimizer's statistical information is not automatically updated when a DDL statement is executed; the user must manually execute the UPDATE STATISTICS statement to update the statistical information. (see Join Methods)

When changing the type of a column that has the AUTO_INCREMENT property or a default value in an ALTER TABLE statement, an error occurs if the new type cannot use AUTO_INCREMENT or if the existing default value is a type whose existing default value cannot be changed. (see :ref:change-column)

You cannot obtain a Connection object through cubrid.jdbc.driver.CUBRIDDriver.getDefaultConnection() from server-side JDBC. Instead, use DriverManager.getConnection("jdbc:default:connection:"). (see Creating Connection)

If an invalid index name or table name is used in a USE INDEX (USING INDEX) statement, it is ignored instead of raising an error. (To record it in the log file, set the error_log warning parameter to yes.)

When writing a CREATE INDEX statement, the position of COMMENT has been moved to the end of the statement, after the WITH or INVISIBLE clause. (see INDEX DEFINITION STATEMENTS)

11.2

The database volume of CUBRID 11.2 is not compatible with that of CUBRID 11.1 and earlier versions.

By introducing the concept of user schema, the same object name can be used for each user, and the behavior is changed as follows

- "." (dot) is not allowed in the object name.

- When using a query or utility command, it must be used as "[user name].object name". (However, the user name can be omitted when querying the object of the logged-in user) (see User Schema)
- Changed to include user name in info schema and show full tables results. (see SHOW)
- The loaddb file prior to 11.2 must be modified to "user name.table name" so that it can be executed in 11.2, or loaddb can be executed by setting the --no-user-specified-name option. (see loaddb)

The following functions and behavior changed when using "jdbc:default:connection:" or calling getDefaultConnection() in JavaSP. (see JavaSP Caution)

- All functions of java.sql.DatabaseMetaData are not supported.
- createClob() and createBlob() of java.sql.Connection are not supported.
- addBatch(), clearBatch(), executeBatch(), setMaxRows() and cancel() of java.sql.Statement are not supported.
- Multiple SQL is not supported for one prepare (or execute).
- The cursor is changed to non-holdable.
- The ResultSet is changed to non-scrollable, non-sensitive and non-updatable.

The behavior of the TRUNCATE TABLE changed if there is set null or cascade of FK (see TRUNCATE)

Column properties not written during alter change/modify are changed to be maintained, and auto_increment and on update properties cannot be removed with the alter statement (see CHANGE/MODIFY Clauses)

Changed to handle an error if only the column name exists in the where clause

- If used in the form of UPDATE t1 SET c1 = 9 WHERE c1; , an error occurs.

Multiple SQL must be separated by semicolons

The CCI Driver directory in the CUBRID package is changed from $CUBRID/lib and $CUBRID/include to $CUBRID/cci/lib and $CUBRID/cci/include, respectively (see CCI Overview)

- When using CCI, $CUBRID/cci/lib must be added to LD_LIBRARY_PATH in the environment variable.

Changed Compression (-z, --compress) option to default on backup (see backupdb)

- System catalog information changed or added due to the addition of new features (see System Catalog)

11.0

The database volume of CUBRID 11.0 is not compatible with that of CUBRID 10.2 and earlier versions.

When creating a table without an option, it is created as a reuse_oid table.

The maximum length of the CHAR data type has been changed to 256M character string.

Modified to occur error when the input string length is longer than the set length of the string data type.

Modified to recognize the space character at the end of the string, it is recognized as a different character string according to the space character at the end of the string.

Due to the change in the statistics collection method, it is necessary to perform periodic statistics collection.

10.2

The database volume of CUBRID 10.2 is not compatible with that of CUBRID 10.1 and earlier versions.

10.1

The database volume of CUBRID 10.1 is not compatible with that of CUBRID 10.0 and earlier versions.

Version 10.0 and earlier

The database volume of CUBRID 10.0 is not compatible with that of CUBRID 9.3 and earlier versions.

Locale(language and charset) is specified when creating DB

- It is changed as locale is specified when creating DB.

CUBRID_CHARSET environment variable is removed

- As locale(language and charset) is specified when creating DB from 9.2 version, CUBRID_CHARSET is not used anymore.

[JDBC] Change zero date of TIMESTAMP into '1970-01-01 00:00:00'(GST) from '0001-01-01 00:00:00' when the value of zeroDateTimeBehavior in the connection URL is "round"(CUBRIDSUS-11612)

- From 2008 R4.4, when the value of the property "zeroDateTimeBehavior" in the connection URL is "round", the zero date value of TIMESTAMP is changed into '1970-01-01 00:00:00'(GST) from '0001-01-01 00:00:00'. You should be cautious when using zero date in your application.

Recommendation for installing CUBRID SH package in AIX(CUBRIDSUS-12251)

- If you install CUBRID SH package by using ksh in AIX OS, it fails with the following error.

```
0403-065 An incomplete or invalid multibyte character encountered.
```

- Therefore, it is recommended to use ksh93 or bash instead of ksh.

```
$ ksh93 ./CUBRID-9.2.0.0146-AIX-ppc64.sh
$ bash ./CUBRID-9.2.0.0146-AIX-ppc64.sh
```

CUBRID_LANG is removed, CUBRID_MSG_LANG is added

- From version 9.1, CUBRID_LANG environment variable is no longer used.
- To output the utility message and the error message, the CUBRID_MSG_LANG environment variable is used.

Modify how to process an error for the array of the result of executing several queries at once in the CCI application(CUBRIDSUS-9364)

- When executing several queries at once in the CCI application, if an error has occurs from at least one query among the results of executing queries by using the cci_execute_array function, the cci_execute_batch function, the error code of the corresponding query was returned from 2008 R3.0 to 2008 R4.1. This problem has been fixed to return the number of the entire queries and check the error of each query by using the CCI_QUERY_RESULT_* macros from 2008 R4.3 and 9.1.
- In earlier versions of this modification, there is no way to know whether each query in the array is success or failure when an error occurs; therefore, it it requires certain conditions.

```
...
char *query = "INSERT INTO test_data (id, ndata, cdata, sdata,
ldata) VALUES (?, ?, 'A', 'ABCD', 1234)";
...
req = cci_prepare (con, query, 0, &cci_error);
...
error = cci_bind_param_array_size (req, 3);
...
error = cci_bind_param_array (req, 1, CCI_A_TYPE_INT, co_ex,
null_ind, CCI_U_TYPE_INT);
...
n_executed = cci_execute_array (req, &result, &cci_error);

if (n_executed < 0)
  {
    printf ("execute error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);

    for (i = 1; i <= 3; i++)
      {
        printf ("query %d\n", i);
```

```
        printf ("result count = %d\n", CCI_QUERY_RESULT_RESULT
(result, i));
        printf ("error message = %s\n", CCI_QUERY_RESULT_ERR_MSG
(result, i));
        printf ("statement type = %d\n", CCI_QUERY_RESULT_STMT_TYPE
(result, i));
      }
  }
...
```

- From the modified version, entire queries are regarded as failure if an error occurs. In case that no error occurred, it is determined whether each query in the array succeeds or not.

```
...
char *query = "INSERT INTO test_data (id, ndata, cdata, sdata,
ldata) VALUES (?, ?, 'A', 'ABCD', 1234)";
...
req = cci_prepare (con, query, 0, &cci_error);
...
error = cci_bind_param_array_size (req, 3);
...
error = cci_bind_param_array (req, 1, CCI_A_TYPE_INT, co_ex,
null_ind, CCI_U_TYPE_INT);
...
n_executed = cci_execute_array (req, &result, &cci_error);
if (n_executed < 0)
  {
    printf ("execute error: %d, %s\n", cci_error.err_code,
cci_error.err_msg);
  }
else
  {
    for (i = 1; i <= 3; i++)
      {
        printf ("query %d\n", i);
        printf ("result count = %d\n", CCI_QUERY_RESULT_RESULT
(result, i));
        printf ("error message = %s\n", CCI_QUERY_RESULT_ERR_MSG
(result, i));
        printf ("statement type = %d\n", CCI_QUERY_RESULT_STMT_TYPE
(result, i));
      }
  }
...
```

In java.sql.XAConnection interface, HOLD_CURSORS_OVER_COMMIT is not supported(CUBRIDSUS-10800)

- Current CUBRID does not support ResultSet.HOLD_CURSORS_OVER_COMMIT in java.sql.XAConnection interface.

From 9.0, STRCMP behaves case-sensitively

- Until the previous version of 9.0, STRCMP did not distinguish an uppercase and a lowercase. From 9.0, it compares the strings case-sensitively.
- To make STRCMP case-insensitive, you should use case-insensitive collation(e.g.: utf8_en_ci).

```
-- In previous version of 9.0 STRCMP works case-insensitively
SELECT STRCMP ('ABC','abc');
0

-- From 9.0 version, STRCMP distinguish the uppercase and the
lowercase when the collation is case-sensitive.
export CUBRID_CHARSET=en_US.iso88591

SELECT STRCMP ('ABC','abc');
-1

-- If the collation is case-insensitive, it distinguish the
uppercase and the lowercase.
export CUBRID_CHARSET=en_US.iso88591

SELECT STRCMP ('ABC' COLLATE utf8_en_ci ,'abc' COLLATE utf8_en_ci);
0
```

Since the 2008 R4.1 version, the Default value of CCI_DEFAULT_AUTOCOMMIT has been ON(CUBRIDSUS-5879)

- The default value for the CCI_DEFAULT_AUTOCOMMIT broker parameter, which affects the autocommit mode for applications developed with CCI interface, has been changed to ON since CUBRID 2008 R4.1. As a result of this change, CCI and CCI-based interface (PHP, ODBC, OLE DB etc.) users should check whether or not the application's autocommit mode is suitable for this.

From the 2008 R4.0 version, the options and parameters that use the unit of pages were changed to use the unit of volume size(CUBRIDSUS-5136)

- The options (-p, -l, -s), which use page units to specify the database volume size and log volume size of the cubrid createdb utility, will be removed. Instead, the new options, added after 2008 R4.0 Beta (–db-volume-size, –log-volume-size, –db-page-size, –log-page-size), are used.
- To specify the database volume size of the cubrid addvoldb utility, use the newly-added option (–db-volume-size) after 2008 R4.0 Beta instead of using the page unit.
- It is recommended to use the new system parameters in bytes because the page-unit system parameters will be removed. For details on the related system parameters, see the below.

Be cautious when setting db volume size if you are a user of a version before 2008 R4.0 Beta(CUBRIDSUS-4222)

- From the 2008 R4.0 Beta version, the default value of data page size and log page size in creating the database was changed from 4 KB to 16 KB. If you specify the database volume to the page count, the byte size of the volume may differ from your expectations. If you did not set any options, 100MB-database volume with 4KB-page size was created in the previous version. However, starting from the 2008 R4.0, 512MB-database volume with 16KB-page size is created.
- In addition, the minimum size of the available database volume is limited to 20 MB. Therefore, a database volume less than this size cannot be created.

The change of the default value of some system parameters of the versions before 2008 R4.0(CUBRIDSUS-4095)

- Starting from 2008 R4.0, the default values of some system parameters have been changed.
- Now, the default value of max_clients, which specifies the number of concurrent connections allowed by a DB server, and the default value of index_unfill_factor that specifies the ratio of reserved space for future updates while creating an index page, have been changed. Furthermore, the default values of the system parameters in bytes now use more memory when they exceed the default values of the previous system parameters per page.

Previous System Parameter	Added System Parameter	Previous Default Value	Changed Default Value (unit: byte)
max_clients	None	50	100
index_unfill_factor	None	0.2	0.05
data_buffer_pages	data_buffer_size	100M(page size=4K)	512M
log_buffer_pages	log_buffer_size	200K(page size=4K)	4M
sort_buffer_pages	sort_buffer_size	64K(page size=4K)	2M
index_scan_oid_buffer_pages	index_scan_oid_buffer_size	16K(page size=4K)	64K

- In addition, when a database is created using cubrid createdb, the minimum value of the data page size and the log page size has been changed from 1K to 4K.

Changed so that database services, utilities, and applications cannot be executed when the system parameter is incorrectly configured(CUBRIDSUS-5375)

- It has been changed so that now the related database services, utilities, and applications are not executed when configuring system parameters that are not defined in cubrid.conf or cubrid_ha.conf, when the value of system parameters exceed the threshold, or when the system parameters per page and the system parameters in bytes are used simultaneously.

Database fails to start if the data_buffer_size is configured with a value that exceeds 2G in CUBRID 32-bit version(CUBRIDSUS-5349)

- In the CUBRID 32-bit version, if the value of data_buffer_size exceeds 2G, the running database fails. Note that the configuration value cannot exceed 2G in the 32-bit version because of the OS limit.

Recommendations for controlling services with the CUBRID Utility in Windows Vista and higher(CUBRIDSUS-4186)

- To control services using cubrid utility from Windows Vista and higher, it is recommended to start the command prompt window with administrative privileges.
- If you don't start the command prompt window with administrative privileges and use the cubrid utility, you can still execute it with administrative privileges through the User Account Control (UAC) dialog box, but you will not be able to verify the resulting messages.

The procedures for starting the command prompt window as an administrator in Windows Vista and higher are as follows:

- Right-click [Start > All Programs > Accessories > Command Prompt].
- When [Execute as an administrator (A)] is selected, a dialog box to verify the privilege escalation is activated. Click "YES" to start with administrative privileges.

A manager server process-related error occurs in the execution of the CUBRID source after its build(CUBRIDSUS-3553)

- If users want to build the CUBRID source and install it themselves, they must build and install CUBRID and the CUBRID Manager respectively. If you check out only CUBRID source and run cubrid service start or cubrid manager start after build, the error "cubrid manager server is not installed" will occur.

GLO class which is used in 2008 r3.0 or before is not supported any longer(CUBRIDSUS-3826)

- CUBRID 2008 R3.0 and earlier versions processed Large Objects with the Generalized Large Object glo class, but the glo class has been removed from CUBRID 2008 R3.1 and later versions. Instead, they support BLOB and CLOB (LOB from this point forward) data types. (See BLOB/CLOB Data Types for more information about LOB data types).
- glo class users are recommended to carry out tasks as follows:
- After saving GLO data as a file, modify to not use GLO in any application and DB schema.
- Implement DB migration by using the unloaddb and loaddb utilities.
- Perform tasks to load files into LOB data according to the modified application.
- Verify the application that you modified operates normally.

- For reference, if the cubrid loaddb utility loads a table that inherits the GLO class or has the GLO class type, it stops the data from loading by displaying an error message, “Error occurred during schema loading.”
- With the discontinued support of GLO class, the deleted functions for each interface are as follows:

Interface	Deleted Functions
CCI	cci_glo_append_data cci_glo_compress_data cci_glo_data_size cci_glo_delete_data cci_glo_destroy_data cci_glo_insert_data cci_glo_load cci_glo_new cci_glo_read_data cci_glo_save cci_glo_truncate_data cci_glo_write_data
JDBC	CUBRIDConnection.getNewGLO CUBRIDOID.loadGLO CUBRIDOID.saveGLO

Interface	Deleted Functions
PHP	cubrid_new_glo
	cubrid_save_to_glo
	cubrid_load_from_glo
	cubrid_send_glo

Port configuration is required if the protocol between the master and server processes is changed, or if two versions are running at the same time(CUBRIDSUS-3564)

- Because the communication protocol between a master process (cub_master) and a server process (cub_server) has been changed, the master process of CUBRID 2008 R3.0 or later cannot communicate with the server process of a lower version, and the master process of a lower version cannot communicate with a server process of 2008 R3.0 version or later. Therefore, if you run two versions of CUBRID at the same time by adding a new version in an environment where a lower version has already been installed, you should modify the cubrid_port_id system parameter of cubrid.conf so that different ports are used by the different versions.

Specifying a question mark when entering connection information as a URL string in JDBC(CUBRIDSUS-3217)

- When entering connection information as a URL string in JDBC, property information was applied even if you did not enter a question mark (?) in the earlier version. However, you must specify a question mark depending on syntax in this CUBRID 2008 R3.0 version. If not, an error is displayed. In addition, you must specify colon (:) even if there is no username or password in the connection information.

```
URL=jdbc:CUBRID:
127.0.0.1:31000:db1:::altHosts=127.0.0.2:31000,127.0.0.3:31000 --
Error
URL=jdbc:CUBRID:127.0.0.1:31000:db1:::?
altHosts=127.0.0.2:31000,127.0.0.3:31000 -- Normal
```

Not allowed to include @ in a database name(CUBRIDSUS-2828)

- If @ is included in a database name, it can be interpreted that a hostname has been specified. To prevent this, a revision has been made so that @ cannot be included in a database name when running cubrid createdb, cubrid renamedb and cubrid copydb utilities.

General Information

Revision history

Revision Date	Description
Mar 2025	CUBRID 11.4 Release (11.4.0)

Bug Reports and User Feedback

CUBRID welcomes your active participation in bug reporting and looks forward to your feedback. You can register your bug reports and feedback on the following Websites:

Document	Description
Bug Report	CUBRID Issue Tracker: http://jira.cubrid.org/browse
User Feedback	CUBRID Open Source Project: https://github.com/CUBRID/cubrid CUBRID Website: https://www.cubrid.org

License

The Apache license 2.0 applies to the CUBRID server engine, and the BSD license applies to CUBRID MANAGER and interfaces (APIs). For more information, see the License Policy on https://www.cubrid.org/license.

Additional Information

Regarding CUBRID upgrade and migration, see Upgrade.

Regarding the recent CUBRID sources, visit https://github.com/CUBRID/cubrid and https://github.com/CUBRID.

Note on Drivers

Currently, CUBRID supports the following drivers: JDBC, CCI(CUBRID C API), Node.js, PHP, PDO, Python, Perl, Ruby, ADO.NET, ODBC, OLE DB. These drivers except JDBC, Node.js and ADO.NET are developed on top of CCI. Therefore, CCI changes can affect CCI-based drivers.

Index

- Index

© Copyright 2016, CUBRID Corparation.